Ironic Documentation

Release 35.1.0.dev10

OpenStack Foundation

Apr 01, 2026

CONTENTS

1 Introduction **1**

2 Installation Guide **3**
2.1 Installing Ironic Bare Metal Service 3
2.2 Bare Metal Service Upgrade Guide 118

3 User Guide **125**
3.1 Bare Metal Service User Guide 125
3.2 REST API Versioning for Ironic 138

4 Administrator Guide **155**
4.1 Drivers, Hardware Types, and Hardware Interfaces for Ironic 155
4.2 Bare Metal Service Features 253
4.3 Configuration and Operation 384
4.4 Administrator Command References 456
4.5 Configuration Reference for Ironic 460
4.6 Architecture and Implementation Details 720
4.7 Administrators Guide 725

5 Contributor Guide **751**
5.1 Developers Guide 751
5.2 Contributing to Ironic Docs 1557

6 Support Resources **1561**
6.1 Support Resources 1561

7 References **1563**
7.1 References 1563

Python Module Index **1569**

Index **1575**

CHAPTER

ONE

INTRODUCTION

Ironic is an OpenStack project which provisions bare metal (as opposed to virtual) machines. It may be used independently or as part of an OpenStack Cloud, and integrates with the OpenStack Identity (keystone), Compute (nova), Network (neutron), Image (glance), and Object (swift) services.

The Bare Metal service manages hardware through both common (eg. PXE and IPMI) and vendor-specific remote management protocols. It provides the cloud operator with a unified interface to a heterogeneous fleet of servers while also providing the Compute service with an interface that allows physical servers to be managed as though they were virtual machines.

This documentation is continually updated and may not represent the state of the project at any specific prior release. To access documentation for a previous release of ironic, append the OpenStack release name to the URL; for example, the `2025.1` release is available at https://docs.openstack.org/ironic/2025.1/.

Found a bug in one of our projects? Please see *Bug Reporting and Triaging Guide*.

Would like to engage with the community? See *Bare Metal Community*.

CHAPTER

TWO

INSTALLATION GUIDE

2.1 Installing Ironic Bare Metal Service

The Bare Metal service is a collection of components that provides support to manage and provision physical machines.

This chapter assumes a working setup of OpenStack following the OpenStack Installation Guides. It contains the following sections:

2.1.1 Overview of Ironic

The Bare Metal service, codenamed `ironic`, is a collection of components that provides support to manage and provision physical machines.

Why Provision Bare Metal

Here are a few use-cases for bare metal (physical server) provisioning in cloud; there are doubtless many more interesting ones:

- High-performance computing clusters
- Computing tasks that require access to hardware devices which cant be virtualized
- Database hosting (some databases run poorly in a hypervisor)
- **Single tenant, dedicated hardware for performance, security, dependability**
  and other regulatory requirements
- Or, rapidly deploying a cloud infrastructure

Bare Metal service components

The Bare Metal service includes the following components:

ironic-api
: A RESTful API that processes application requests by sending them to the ironic-conductor over remote procedure call (RPC). Can be run through WSGI or as a separate process.

ironic-conductor
: Adds/edits/deletes nodes; powers on/off nodes with IPMI or other vendor-specific protocol; provisions/deploys/cleans bare metal nodes.

  ironic-conductor uses *drivers* to execute operations on hardware.

ironic-python-agent
: A python service which is run in a temporary ramdisk to provide ironic-conductor with remote access, in-band hardware control, and hardware introspection.

ironic-novncproxy
: A python service which proxies graphical consoles from hosts using the NoVNC web browser interface.

ironic-networking
: A python service which interacts with network switches to configure switch ports when managing nodes. Nodes must be configured to use the `ironic-networking` network interface and must have appropriate attributes set on the Ironic port. See *standalone-networking*

Additionally, the Bare Metal service has certain external dependencies, which are very similar to other OpenStack services:

- A database to store hardware information and state. You can set the database back-end type and location. A simple approach is to use the same database back end as the Compute service. Another approach is to use a separate database back-end to further isolate bare metal resources (and associated metadata) from users.
- An oslo.messaging compatible queue, such as RabbitMQ. It may use the same implementation as that of the Compute service, but that is not a requirement. Used to implement RPC between ironic-api and ironic-conductor.

Interaction with OpenStack components

The Bare Metal service may, depending upon configuration, interact with several other OpenStack services. This includes:

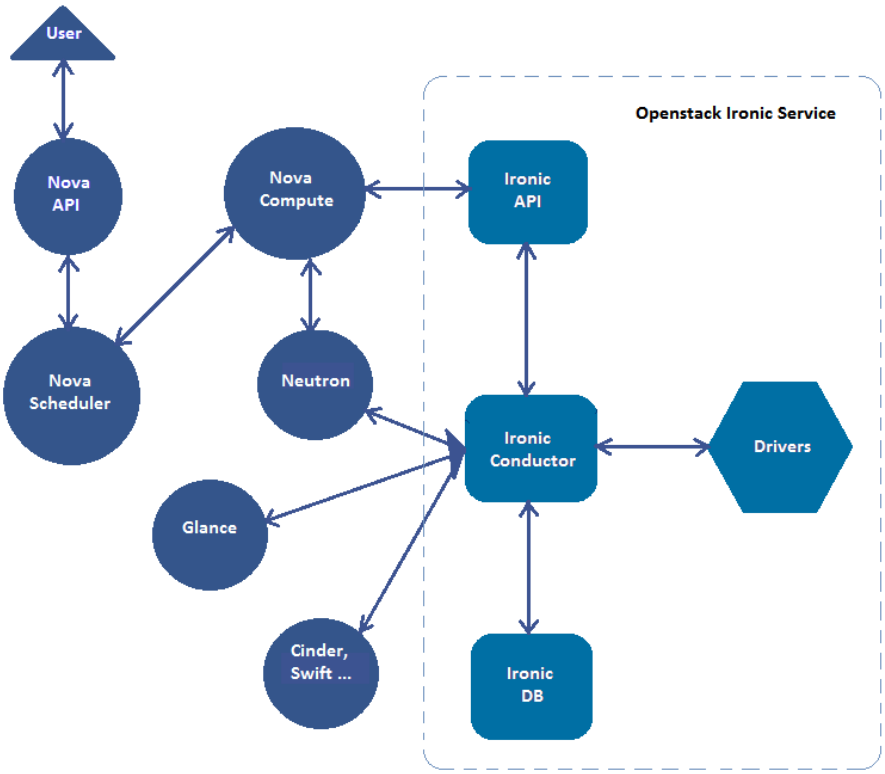
- the OpenStack Telemetry module (`ceilometer`) for consuming the IPMI metrics
- the OpenStack Identity service (`keystone`) for request authentication and to locate other OpenStack services
- the OpenStack Image service (`glance`) from which to retrieve images and image meta-data
- the OpenStack Networking service (`neutron`) for DHCP and network configuration
- the OpenStack Compute service (`nova`) works with the Bare Metal service and acts as a user-facing API for instance management, while the Bare Metal service provides the admin/operator API for hardware management. The OpenStack Compute service also provides scheduling facilities (matching flavors <-> images <-> hardware), tenant quotas, IP assignment, and other services which the Bare Metal service does not, in and of itself, provide.
- the OpenStack Object Storage (`swift`) provides temporary storage for the configdrive, user images, deployment logs and inspection data.

Logical architecture

The diagram below shows the logical architecture. It shows the basic components that form the Bare Metal service, the relation of the Bare Metal service with other OpenStack services and the logical flow of a boot instance request resulting in the provisioning of a physical server.

A users request to boot an instance is passed to the Compute service via the Compute API and the Compute Scheduler. The Compute service uses the *ironic virt driver* to hand over this request to the Bare Metal service, where the request passes from the Bare Metal API, to the Conductor, to a Driver to successfully provision a physical server for the user.

Figure 1.2. Logical Architecture



Just as the Compute service talks to various OpenStack services like Image, Network, Object Store etc to provision a virtual machine instance, here the Bare Metal service talks to the same OpenStack services for image, network and other resource needs to provision a bare metal instance.

See *Understanding Bare Metal Deployment* for a more detailed breakdown of a typical deployment process.

Associated projects

Optionally, one may wish to utilize the following associated projects for additional functionality:

python-ironicclient
: A command-line interface (CLI) and python bindings for interacting with the Bare Metal service.

ironic-ui
: Horizon dashboard, providing graphical interface (GUI) for the Bare Metal API.

diskimage-builder
: A related project to help facilitate the creation of ramdisks and machine images, such as those running the ironic-python-agent.

bifrost
: A set of Ansible playbooks that automates the task of deploying a base image onto a set of known hardware using ironic in a standalone mode.

Key Technologies for Bare Metal Hosting

Preboot Execution Environment (PXE)

PXE is part of the Wired for Management (WfM) specification developed by Intel and Microsoft. The PXE enables systems BIOS and network interface card (NIC) to bootstrap a computer from the network in place of a disk. Bootstrapping is the process by which a system loads the OS into local memory so that it can be executed by the processor. This capability of allowing a system to boot over a network simplifies server deployment and server management for administrators.

Dynamic Host Configuration Protocol (DHCP)

DHCP is a standardized networking protocol used on Internet Protocol (IP) networks for dynamically distributing network configuration parameters, such as IP addresses for interfaces and services. Using PXE, the BIOS uses DHCP to obtain an IP address for the network interface and to locate the server that stores the network bootstrap program (NBP).

Network Bootstrap Program (NBP)

NBP is equivalent to GRUB (GRand Unified Bootloader) or LILO (LInux LOader) - loaders which are traditionally used in local booting. Like the boot program in a hard drive environment, the NBP is responsible for loading the OS kernel into memory so that the OS can be bootstrapped over a network.

Trivial File Transfer Protocol (TFTP)

TFTP is a simple file transfer protocol that is generally used for automated transfer of configuration or boot files between machines in a local environment. In a PXE environment, TFTP is used to download NBP over the network using information from the DHCP server.

Intelligent Platform Management Interface (IPMI)

IPMI is a standardized computer system interface used by system administrators for out-of-band management of computer systems and monitoring of their operation. It is a method to manage systems that may be unresponsive or powered off by using only a network connection to the hardware rather than to an operating system.

Understanding Bare Metal Deployment

What happens when a boot instance request comes in? The below diagram walks through the steps involved during the provisioning of a bare metal instance.

These pre-requisites must be met before the deployment process:

- Dependent packages to be configured on the Bare Metal service node(s) where ironic-conductor is running like tftp-server, ipmi, grub/ipxe, etc for bare metal provisioning.
- Nova must be configured to make use of the bare metal service endpoint and compute driver should be configured to use ironic driver on the Nova compute node(s).
- Flavors to be created for the available hardware. Nova must know the flavor to boot from.
- Images to be made available in Glance. Listed below are some image types required for successful bare metal deployment:
  - bm-deploy-kernel
  - bm-deploy-ramdisk
  - user-image
  - user-image-vmlinuz
  - user-image-initrd
- Hardware to be enrolled via the bare metal API service.

Deploy Process

This describes a typical bare metal node deployment within OpenStack using PXE to boot the ramdisk. Depending on the ironic driver interfaces used, some of the steps might be marginally different, however the majority of them will remain the same.

1. A boot instance request comes in via the Nova API, through the message queue to the Nova scheduler.
2. Nova scheduler applies filters and finds the eligible hypervisor. The nova scheduler also uses the flavors `extra_specs`, such as `cpu_arch`, to match the target physical node.
3. Nova compute manager claims the resources of the selected hypervisor.
4. Nova compute manager creates (unbound) tenant virtual interfaces (VIFs) in the Networking service according to the network interfaces requested in the nova boot request. A caveat here is, the MACs of the ports are going to be randomly generated, and will be updated when the VIF is attached to some node to correspond to the node network interface cards (or bonds) MAC.
5. A spawn task is created by the nova compute which contains all the information such as which image to boot from etc. It invokes the `driver.spawn` from the virt layer of Nova compute. During the spawn process, the virt driver does the following:

1. Updates the target ironic node with the information about deploy image, instance UUID, requested capabilities and various flavor properties.
2. Validates nodes power and deploy interfaces, by calling the ironic API.
3. Attaches the previously created VIFs to the node. Each neutron port can be attached to any ironic port or port group, with port groups having higher priority than ports. On ironic side, this work is done by the network interface. Attachment here means saving the VIF identifier into ironic port or port group and updating VIF MAC to match the ports or port groups MAC, as described in bullet point 4.
4. Generates config drive, if requested.

6. Novas ironic virt driver issues a deploy request via the Ironic API to the Ironic conductor servicing the bare metal node.
7. Virtual interfaces are plugged in and Neutron API updates DHCP port to set PXE/TFTP options. In case of using `neutron` network interface, ironic creates separate provisioning ports in the Networking service, while in case of `flat` network interface, the ports created by nova are used both for provisioning and for deployed instance networking.
8. The ironic nodes boot interface prepares (i)PXE configuration and caches deploy kernel and ramdisk.
9. The ironic nodes management interface issues commands to enable network boot of a node.
10. The ironic nodes deploy interface caches the instance image (normal deployment), kernel and ramdisk (`ramdisk` deploy) or ISO (`ramdisk` deploy with virtual media).
11. The ironic nodes power interface instructs the node to power on.
12. The node boots the deploy ramdisk.
13. Depending on the exact driver used, the deploy ramdisk downloads the image from a URL (*Direct deploy*) or the conductor uses SSH to execute commands (*Ansible deploy*). The URL can be generated by Swift API-compatible object stores, for example Swift itself or RadosGW, or provided by a user.

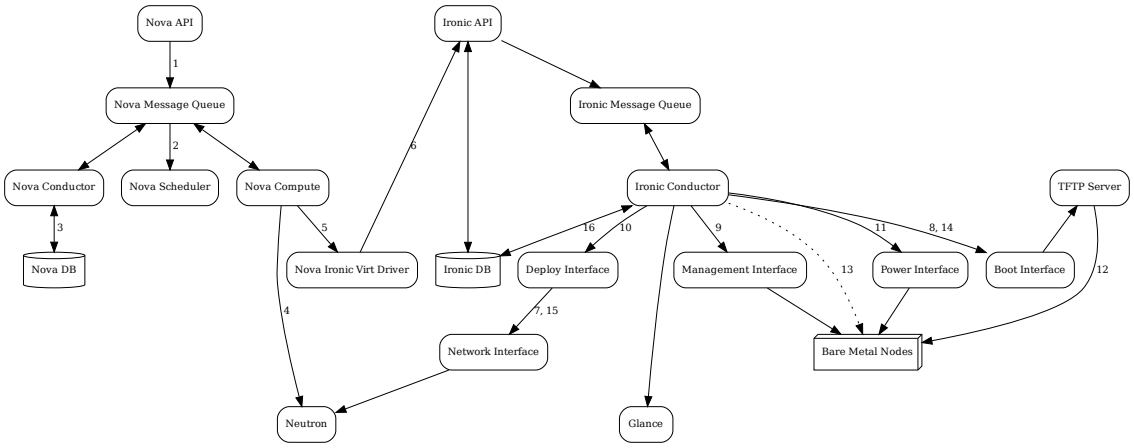
    The image deployment is done.
14. The nodes boot interface switches pxe config to refer to instance images (or, in case of local boot, sets boot device to disk), and asks the ramdisk agent to soft power off the node. If the soft power off by the ramdisk agent fails, the bare metal node is powered off via IPMI/BMC call.
15. The deploy interface triggers the network interface to remove provisioning ports if they were created, and binds the tenant ports to the node if not already bound. Then the node is powered on.

    Note

    There are 2 power cycles during bare metal deployment; the first time the node is powered-on when ramdisk is booted, the second time after the image is deployed.

16. The bare metal nodes provisioning state is updated to `active`.

Below is the diagram that describes the above process.



The following two examples describe what ironic is doing in more detail, leaving out the actions performed by nova and some of the more advanced options.

Example: PXE Boot and Direct Deploy Process

This process is how *Direct deploy* works.

(From a talk and slides)

Deployment architecture

The Bare Metal RESTful API service is used to enroll hardware that the Bare Metal service will manage. A cloud administrator usually registers it, specifying their attributes such as MAC addresses and IPMI credentials. There can be multiple instances of the API service.

The *ironic-conductor* process does the bulk of the work. For security reasons, it is advisable to place it on an isolated host, since it is the only service that requires access to both the data plane and IPMI control plane.

There can be multiple instances of the conductor service to support various class of drivers and also to manage fail over. Instances of the conductor service should be on separate nodes. Each conductor can itself run many drivers to operate heterogeneous hardware. This is depicted in the following figure.

The API exposes a list of supported drivers and the names of conductor hosts servicing them.

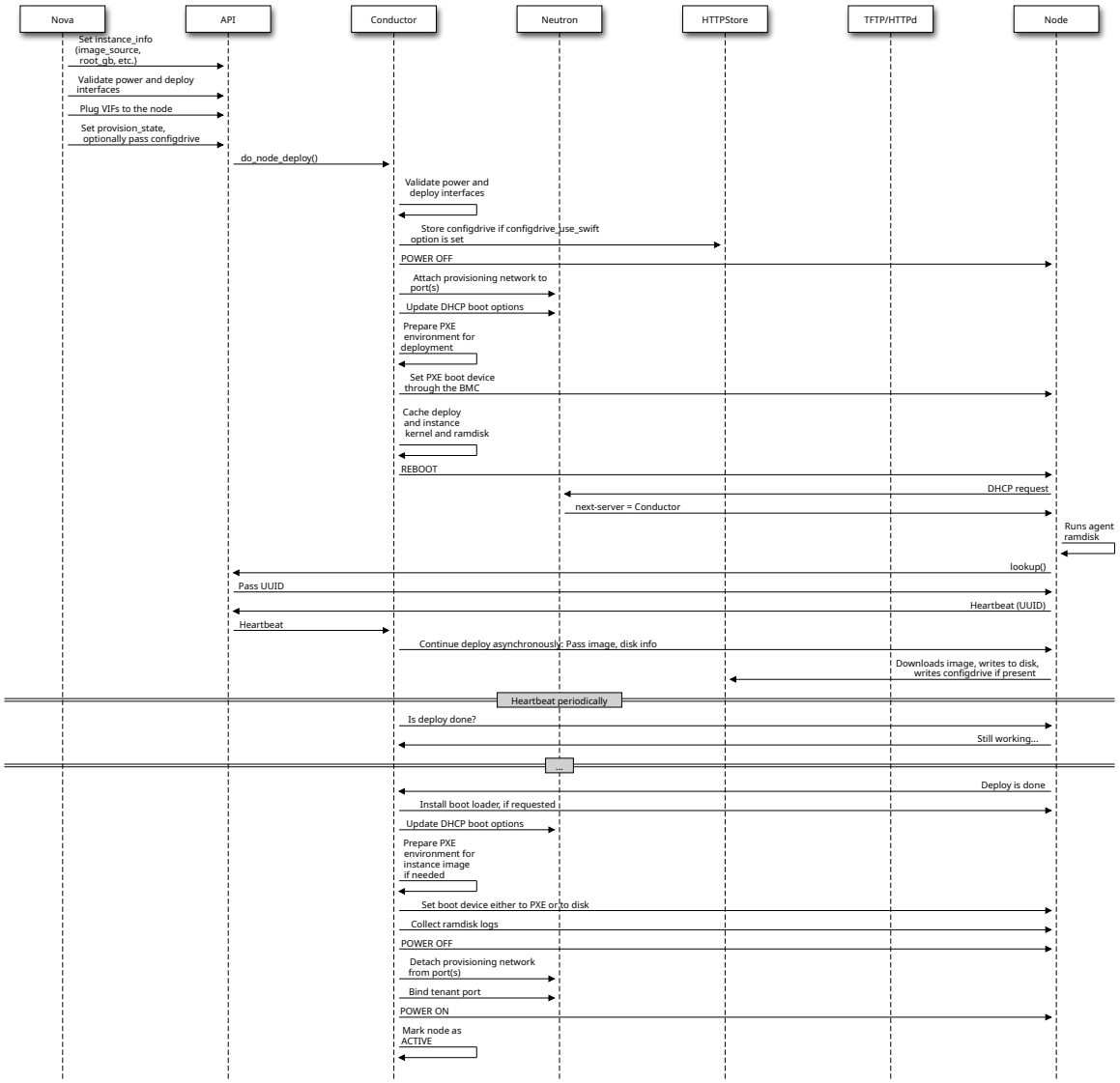
2.1.2 Reference Deploy Architectures

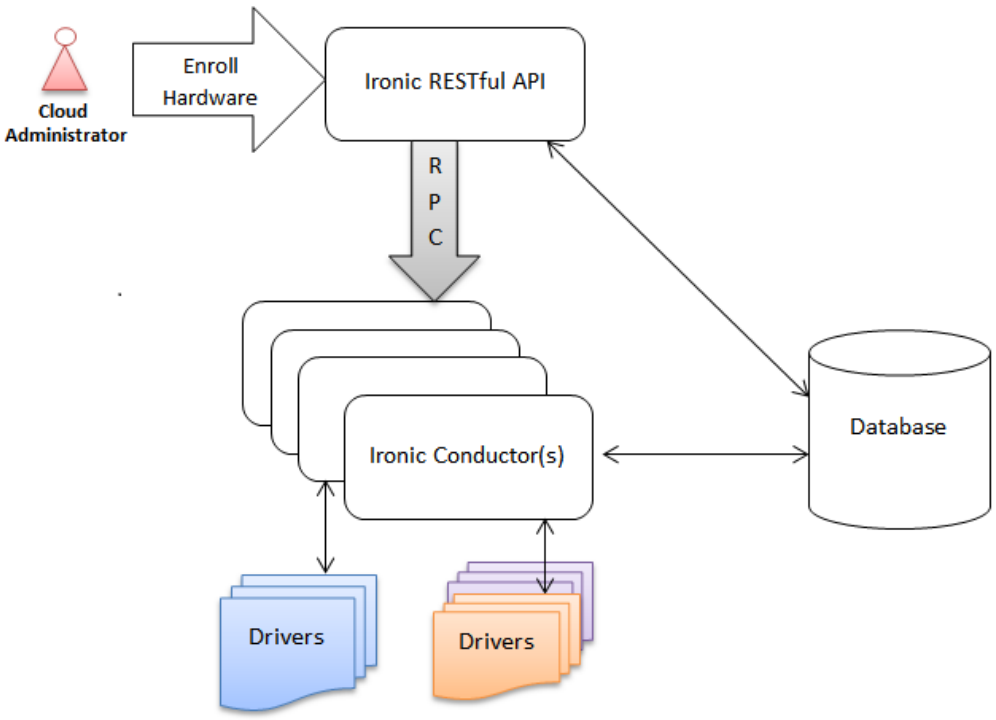
This section covers the way we recommend the Bare Metal service to be deployed and managed. It is assumed that a reader has already gone through *Bare Metal Service User Guide*. It may be also useful to try *Deploying Ironic with DevStack* first to get better familiar with the concepts used in this guide.

Common Considerations

This section covers considerations that are equally important to all described architectures.

- *Components*
- *Hardware and drivers*


Nova
API
Conductor
Neutron
HTTPStore
TFTP/HTTPd
Node
Set instance_info (image_source, root_gb, etc.)
Validate power and deploy interfaces
Plug VIFs to the node
Set provision_state, optionally pass configdrive
do_node_deploy()
Validate power and deploy interfaces
Store configdrive if configdrive_use_swift option is set
POWER OFF
Attach provisioning network to port(s)
Update DHCP boot options
Prepare PXE environment for deployment
Set PXE boot device through the BMC
Cache deploy and instance kernel and ramdisk
REBOOT
DHCP request
next-server = Conductor
Runs agent ramdisk
lookup()
Pass UUID
Heartbeat (UUID)
Heartbeat
Continue deploy asynchronously: Pass image, disk info
Downloads image, writes to disk, writes configdrive if present
Heartbeat periodically
Is deploy done?
Still working...
...
Deploy is done
Install boot loader, if requested
Update DHCP boot options
Prepare PXE environment for instance image if needed
Set boot device either to PXE or to disk
Collect ramdisk logs
POWER OFF
Detach provisioning network from port(s)
Bind tenant port
POWER ON
Mark node as ACTIVE



- *Power and management interfaces*
- *Boot interface*
- *Hardware specifications*

- *Image types*
- *Networking*
- *HA and Scalability*
  - *ironic-api*
  - *ironic-conductor*
    - *High availability*
    - *Performance*
    - *Disk space*
  - *ironic-novncproxy*
  - *Other services*

Components

As explained in *Overview of Ironic*, the Bare Metal service has four components.

- The Bare Metal API service (`ironic-api`) should be deployed in a similar way as the control plane API services. The exact location will depend on the architecture used.

- The Bare Metal conductor service (`ironic-conductor`) is where most of the provisioning logic lives. The following considerations are the most important when deciding on the way to deploy it:

  Tip

  For large deployments, consider *Availability Zones and Resource Isolation* to understand conductor groups and shards for resource partitioning.

  - The conductor manages a certain proportion of nodes, distributed to it via a hash ring. This includes constantly polling these nodes for their current power state and hardware sensor data (if enabled and supported by hardware, see *Collecting sensor data* for an example).
  - The conductor needs access to the management controller of each node it manages.
  - The conductor co-exists with TFTP (for PXE) and/or HTTP (for HTTPBoot and iPXE) services that provide the kernel and ramdisk to boot the nodes. The conductor manages them by writing files to their root directories.
  - If serial console is used, the conductor launches console processes locally. If the `nova-serialproxy` service (part of the Compute service) is used, it has to be able to reach the conductors. Otherwise, they have to be directly accessible by the users.
  - There must be mutual connectivity between the conductor and the nodes being deployed or cleaned. See *Networking* for details.
- The NoVNC graphical console proxy service (`ironic-novncproxy`) can be optionally run to enable connecting to graphical consoles for hosts which have a supported console driver. Like (`ironic-api`) it needs to be deployed like other control plane services, allowing users to access the NoVNC interface via a web browser. Additionally it also like (`ironic-conductor`) in requiring access to the management network to connect to the graphical console managed by the host BMC.
- The provisioning ramdisk which runs the `ironic-python-agent` service on start up.

  Warning

  The `ironic-python-agent` service is not intended to be used or executed anywhere other than a provisioning/cleaning/rescue ramdisk.

Hardware and drivers

The Bare Metal service strives to provide the best support possible for a variety of hardware. However, not all hardware is supported equally well. It depends on both the capabilities of hardware itself and the available drivers. This section covers various considerations related to the hardware interfaces. See *Enabling drivers and hardware types* for a detailed introduction into hardware types and interfaces before proceeding.

Power and management interfaces

The minimum set of capabilities that the hardware has to provide and the driver has to support is as follows:

1. getting and setting the power state of the machine
2. getting and setting the current boot device

3. booting an image provided by the Bare Metal service (in the simplest case, support booting using PXE and/or iPXE)

Note

Strictly speaking, it is possible to make the Bare Metal service provision nodes without some of these capabilities via some manual steps. It is not the recommended way of deployment, and thus it is not covered in this guide.

Once you make sure that the hardware supports these capabilities, you need to find a suitable driver. Most of enterprise-grade hardware has support for IPMI and thus can utilize *IPMI driver*. Some newer hardware also supports *Redfish driver*. Several vendors provide more specific drivers that usually provide additional capabilities. Check *Drivers, Hardware Types, and Hardware Interfaces for Ironic* to find the most suitable one.

Boot interface

The boot interface of a node manages booting of both the deploy ramdisk and the user instances on the bare metal node. The deploy interface orchestrates the deployment and defines how the image gets transferred to the target disk.

The main alternatives are to use PXE/iPXE or virtual media - see *Boot interfaces* for a detailed explanation. If a virtual media implementation is available for the hardware, it is recommended using it for better scalability and security. Otherwise, it is recommended to use iPXE, when it is supported by target hardware.

Hardware specifications

The Bare Metal services does not impose too many restrictions on the characteristics of hardware itself. However, keep in mind that

- By default, the Bare Metal service will pick the smallest hard drive that is larger than 4 GiB for deployment. Another hard drive can be used, but it requires setting *root device hints*.

  Note

  This device does not have to match the boot device set in BIOS (or similar firmware).

- The machines should have enough RAM to fit the deployment/cleaning ramdisk to run. The minimum varies greatly depending on the way the ramdisk was built. For example, *tinyipa*, the TinyCoreLinux-based ramdisk used in the CI, only needs 400 MiB of RAM, while ramdisks built by *diskimage-builder* may require 3 GiB or more.

Image types

The Bare Metal service can deploy two types of images:

- *Whole-disk* images that contain a complete partitioning table with all necessary partitions and a bootloader. Such images are the most universal, but may be harder to build.
- *Partition images* that contain only the root partition. The Bare Metal service will create the necessary partitions and install a boot loader, if needed.

Warning

Partition images are only supported with GNU/Linux operating systems.

For the Bare Metal service to set up the bootloader during deploy, your partition images must container either GRUB2 bootloader or ready-to-use EFI artifacts.

Networking

There are several recommended network topologies to be used with the Bare Metal service. They are explained in depth in specific architecture documentation. However, several considerations are common for all of them:

- There has to be a *provisioning* network, which is used by nodes during the deployment process. If allowed by the architecture, this network should not be accessible by end users, and should not have access to the internet.
- There has to be a *cleaning* network, which is used by nodes during the cleaning process.
- There should be a *rescuing* network, which is used by nodes during the rescue process. It can be skipped if the rescue process is not supported.

Note

In the majority of cases, the same network should be used for cleaning, provisioning and rescue for simplicity.

Unless noted otherwise, everything in these sections apply to all three networks.

- The baremetal nodes must have access to the Bare Metal API while connected to the provisioning/cleaning/rescuing network.

  Note

  Only two endpoints need to be exposed there:

  ```
  GET /v1/lookup
  POST /v1/heartbeat/[a-z0-9\-]+
  ```

  You may want to limit access from this network to only these endpoints, and make these endpoint not accessible from other networks.

- If the `pxe` boot interface (or any boot interface based on it) is used, then the baremetal nodes should have untagged (access mode) connectivity to the provisioning/cleaning/rescuing networks. It allows PXE firmware, which does not support VLANs, to communicate with the services required for provisioning.

  Note

  It depends on the *network interface* whether the Bare Metal service will handle it automatically. Check the networking documentation for the specific architecture.

Sometimes it may be necessary to disable the spanning tree protocol delay on the switch - see *DHCP during PXE or iPXE is inconsistent or unreliable*.

- The Baremetal nodes need to have access to any services required for provisioning/cleaning/rescue, while connected to the provisioning/cleaning/rescuing network. This may include:
  - a TFTP server for PXE boot and also an HTTP server when iPXE is enabled
  - either an HTTP server or the Object Storage service in case of the `direct` deploy interface and some virtual media boot interfaces
- The Baremetal Conductors need to have access to the booted baremetal nodes during provisioning/cleaning/rescue. A conductor communicates with an internal API, provided by **ironic-python-agent**, to conduct actions on nodes.

HA and Scalability

ironic-api

The Bare Metal API service is stateless, and thus can be easily scaled horizontally. It is recommended to deploy it as a WSGI application behind e.g. Apache or another WSGI container.

Note

This service accesses the ironic database for reading entities (e.g. in response to `GET /v1/nodes` request) and in rare cases for writing.

ironic-conductor

High availability

The Bare Metal conductor service utilizes the active/active HA model. Every conductor manages a certain subset of nodes. The nodes are organized in a hash ring that tries to keep the load spread more or less uniformly across the conductors. When a conductor is considered offline, its nodes are taken over by other conductors. As a result of this, you need at least 2 conductor hosts for an HA deployment.

Performance

Conductors can be resource intensive, so it is recommended (but not required) to keep all conductors separate from other services in the cloud. The minimum required number of conductors in a deployment depends on several factors:

- the performance of the hardware where the conductors will be running,
- the speed and reliability of the management controller of the bare metal nodes (for example, handling slower controllers may require having less nodes per conductor),
- the frequency, at which the management controllers are polled by the Bare Metal service (see the `sync_power_state_interval` option),
- the bare metal driver used for nodes (see *Hardware and drivers* above),
- the network performance,
- the maximum number of bare metal nodes that are provisioned simultaneously (see the `max_concurrent_builds` option for the Compute service).

We recommend a target of **100** bare metal nodes per conductor for maximum reliability and performance. There is some tolerance for a larger number per conductor. However, it was reported[1][2] that reliability degrades when handling approximately 300 bare metal nodes per conductor.

Note

For very large deployments, consider using *Availability Zones and Resource Isolation* strategies such as conductor groups to distribute load across multiple conductors, or even multiple Ironic deployments for complete isolation.

Disk space

Each conductor needs enough free disk space to cache images it uses. Depending on the combination of the deploy interface and the boot option, the space requirements are different:

- The deployment kernel and ramdisk are always cached during the deployment.
- When `agent.image_download_source` is set to `http` and Glance is used, the conductor will download instances images locally to serve them from its HTTP server. Use `swift` to publish images using temporary URLs and convert them on the nodes side.

  When `agent.image_download_source` is set to `local`, it will happen even for HTTP(s) URLs. For standalone case use `http` to avoid unnecessary caching of images.

  In both cases a cached image is converted to raw if `force_raw_images` is `True` (the default).

  See *Deploy with custom HTTP servers* and *Streaming raw images* for more details.
- When network boot is used, the instance image kernel and ramdisk are cached locally while the instance is active.

Note

All images may be stored for some time after they are no longer needed. This is done to speed up simultaneous deployments of many similar images. The caching can be configured via the `image_cache_size` and `image_cache_ttl` configuration options in the `pxe` group.

ironic-novncproxy

The NoVNC proxy service is stateless, and thus can be scaled horizontally. Any load balancing or reverse proxy architecture also needs to support websockets, as this is how the VNC traffic communicates with the service.

Other services

When integrating with other OpenStack services, more considerations may need to be applied. This is covered in other parts of this guide.

[1] http://lists.openstack.org/pipermail/openstack-dev/2017-June/118033.html
[2] http://lists.openstack.org/pipermail/openstack-dev/2017-June/118327.html

Scenarios

Small cloud with trusted tenants

Story

As an operator I would like to build a small cloud with both virtual and bare metal instances or add bare metal provisioning to my existing small or medium scale single-site OpenStack cloud. The expected number of bare metal machines is less than 100, and the rate of provisioning and unprovisioning is expected to be low. All users of my cloud are trusted by me to not conduct malicious actions towards each other or the cloud infrastructure itself.

As a user I would like to occasionally provision bare metal instances through the Compute API by selecting an appropriate Compute flavor. I would like to be able to boot them from images provided by the Image service or from volumes provided by the Volume service.

Components

This architecture assumes an OpenStack installation with the following components participating in the bare metal provisioning:

- The Compute service manages bare metal instances.
- The Networking service provides DHCP for bare metal instances.
- The Image service provides images for bare metal instances.

The following service can be optionally used by the Bare Metal service:

- The Volume service provides volumes to boot bare metal instances from.

Node roles

An OpenStack installation in this guide has at least these three types of nodes:

- A *controller* node hosts the control plane services.
- A *compute* node runs the virtual machines and hosts a subset of Compute and Networking components.
- A *block storage* node provides persistent storage space for both virtual and bare metal nodes.

The *compute* and *block storage* nodes are configured as described in the installation guides of the Compute service and the Volume service respectively. The *controller* nodes host the Bare Metal service components.

Networking

The networking architecture will highly depend on the exact operating requirements. This guide expects the following existing networks: *control plane*, *storage* and *public*. Additionally, two more networks will be needed specifically for bare metal provisioning: *bare metal* and *management*.

Control plane network

The *control plane network* is the network where OpenStack control plane services provide their public API.

The Bare Metal API will be served to the operators and to the Compute service through this network.

The `ironic-novncproxy` NoVNC proxy service will serve the graphical console user interface via this network.

Public network

The *public network* is used in a typical OpenStack deployment to create floating IPs for outside access to instances. Its role is the same for a bare metal deployment.

Note

Since, as explained below, bare metal nodes will be put on a flat provider network, it is also possible to organize direct access to them, without using floating IPs and bypassing the Networking service completely.

Bare metal network

The *Bare metal network* is a dedicated network for bare metal nodes managed by the Bare Metal service.

This architecture uses *flat bare metal networking*, in which both tenant traffic and technical traffic related to the Bare Metal service operation flow through this one network. Specifically, this network will serve as the *provisioning*, *cleaning* and *rescuing* network. It will also be used for introspection via the Bare Metal Introspection service. See *common networking considerations* for an in-depth explanation of the networks used by the Bare Metal service.

DHCP and boot parameters will be provided on this network by the Networking services DHCP agents.

For booting from volumes this network has to have a route to the *storage network*.

Management network

Management network is an independent network on which BMCs of the bare metal nodes are located.

The `ironic-conductor` process needs access to this network. The tenants of the bare metal nodes must not have access to it.

Note

The *direct deploy interface* and certain *Drivers, Hardware Types, and Hardware Interfaces for Ironic* require the *management network* to have access to the Object storage service backend.

The `ironic-novncproxy` NoVNC proxy service needs access to this network to connect to the host BMC graphical console.

Controllers

A *controller* hosts the OpenStack control plane services as described in the control plane design guide. While this architecture allows using *controllers* in a non-HA configuration, it is recommended to have at least three of them for HA. See *HA and Scalability* for more details.

Bare Metal services

The following components of the Bare Metal service are installed on a *controller* (see *components of the Bare Metal service*):

- The Bare Metal API service either as a WSGI application or the `ironic-api` process. Typically, a load balancer, such as HAProxy, spreads the load between the API instances on the *controllers*.

  The API has to be served on the *control plane network*. Additionally, it has to be exposed to the *bare metal network* for the ramdisk callback API.

- The `ironic-conductor` process. These processes work in active/active HA mode as explained in *HA and Scalability*, thus they can be installed on all *controllers*. Each will handle a subset of bare metal nodes.

  The `ironic-conductor` processes have to have access to the following networks:

  - *control plane* for interacting with other services
  - *management* for contacting nodes BMCs
  - *bare metal* for contacting deployment, cleaning or rescue ramdisks

- The `ironic-novncproxy` NoVNC proxy is run directly as a web server process and only serves consoles for bare metal nodes for the adjacent `ironic-controller`. This means one `ironic-novncproxy` needs to be run for each `ironic-conductor` process

  The NoVNC proxy has to be served on the *control plane network*. Additionally, it needs to make VNC connections to the console containers initiated by `ironic-conductor`.

- TFTP and HTTP service for booting the nodes. Each `ironic-conductor` process has to have a matching TFTP and HTTP service. They should be exposed only to the *bare metal network* and must not be behind a load balancer.

- The `nova-compute` process (from the Compute service). These processes work in active/active HA mode when dealing with bare metal nodes, thus they can be installed on all *controllers*. Each will handle a subset of bare metal nodes.

  Note

  There is no 1-1 mapping between `ironic-conductor` and `nova-compute` processes, as they communicate only through the Bare Metal API service.

- The networking-baremetal ML2 plugin should be loaded into the Networking service to assist with binding bare metal ports.

  The ironic-neutron-agent service should be started as well.

Shared services

A *controller* also hosts two services required for the normal operation of OpenStack:

- Database service (MySQL/MariaDB is typically used, but other enterprise-grade database solutions can be used as well).

  All Bare Metal service components need access to the database service.

- Message queue service (RabbitMQ is typically used, but other enterprise-grade message queue brokers can be used as well).

  Both Bare Metal API (WSGI application or `ironic-api` process) and the `ironic-conductor` processes need access to the message queue service. The Bare Metal Introspection service does not need it.

Note

These services are required for all OpenStack services. If youre adding the Bare Metal service to your cloud, you may reuse the existing database and messaging queue services.

Bare metal nodes

Each bare metal node must be capable of booting from network, virtual media or other boot technology supported by the Bare Metal service as explained in *Boot interface*. Each node must have one NIC on the *bare metal network*, and this NIC (and **only** it) must be configured to be able to boot from network. This is usually done in the *BIOS setup* or a similar firmware configuration utility. There is no need to alter the boot order, as it is managed by the Bare Metal service. Other NICs, if present, will not be managed by OpenStack.

The NIC on the *bare metal network* should have untagged connectivity to it, since PXE firmware usually does not support VLANs - see *Networking* for details.

Storage

If your hardware **and** its bare metal *driver* support booting from remote volumes, please check the driver documentation for information on how to enable it. It may include routing *management* and/or *bare metal* networks to the *storage network*.

In case of the standard *PXE boot*, booting from remote volumes is done via iPXE. In that case, the Volume storage backend must support iSCSI protocol, and the *bare metal network* has to have a route to the *storage network*. See *Boot From Volume* for more details.

2.1.3 Install and configure the Bare Metal service

This section describes how to install and configure the Bare Metal service, code-named ironic, manually from packages on one of the three popular families of Linux distributions.

Alternatively, you can use one of the numerous projects that install ironic. One of them is provided by the bare metal team:

- Bifrost installs ironic in the standalone mode (without the rest of OpenStack).

More installation projects are developed by other OpenStack teams:

- Kolla can install ironic in containers as part of OpenStack.
- OpenStack-Ansible has a role to install ironic.

Install and configure for Red Hat Enterprise Linux and CentOS

The contents has been migrated to the common location - see *Install and configure the Bare Metal service*.

Install and configure for Ubuntu

The contents has been migrated to the common location - see *Install and configure the Bare Metal service*.

Install and configure prerequisites

The Bare Metal service is a collection of components that provides support to manage and provision physical machines. You can configure these components to run on separate nodes or the same node. In this guide, the components run on one node, typically the Compute Services compute node.

It assumes that the Identity, Image, Compute, and Networking services have already been set up.

Set up the database for Bare Metal

The Bare Metal service stores information in a database. This guide uses the MySQL database that is used by other OpenStack services.

In MySQL, create an `ironic` database that is accessible by the `ironic` user. Replace `IRONIC_DBPASSWORD` with a suitable password:

```
# mysql -u root -p
mysql> CREATE DATABASE ironic CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_
↪ci;
mysql> GRANT ALL PRIVILEGES ON ironic.* TO 'ironic'@'localhost' \
       IDENTIFIED BY 'IRONIC_DBPASSWORD';
mysql> GRANT ALL PRIVILEGES ON ironic.* TO 'ironic'@'%' \
       IDENTIFIED BY 'IRONIC_DBPASSWORD';
```

Note

Ironic requires `utf8mb4` character encoding (4-byte UTF-8) for full Unicode support. This requires **MySQL 8.0+** or **MariaDB 10.3+**, which use the DYNAMIC row format by default and support the required index key lengths. Older MySQL/MariaDB versions are not supported.

Running on SQLite

It is possible to run the Bare Metal service with SQLite as the database backend. However, take into account the following limitations:

- You have to run Ironic in the all-in-one mode (see *configuring single-process ironic*). You cannot have multiple conductors this way.
- You should use WAL mode for the database. Ironic will try to enable it for you, but it can only be done for a fresh database.
- Even in the WAL mode, SQLite has limited write concurrency. If you experience database is locked errors, try reducing the frequency of periodic tasks. If that still does not help, you may need to use a server-based database.
- Not all database migrations are supported on SQLite. You may need to re-create the database on upgrades.

To use SQLite, configure your connection like this:

```
[database]
connection = sqlite:////full/path/to/ironic.sqlite
```

Note

This is not a typo! A full path requires 4 slashes.

If in doubt, use MySQL/MariaDB instead.

Install and configure components

Using DNF on RHEL/CentOS Stream and RDO packages:

```
# dnf install openstack-ironic-api openstack-ironic-conductor openstack-
↪ironic-novncproxy python3-ironicclient
```

On Ubuntu/Debian:

```
# apt-get install ironic-api ironic-conductor ironic-novncproxy python3-
↪ironicclient
```

The Bare Metal service is configured via its configuration file. This file is typically located at `/etc/ironic/ironic.conf`.

Although some configuration options are mentioned here, it is recommended that you review all the *Sample Configuration File* so that the Bare Metal service is configured for your needs.

It is possible to set up an ironic-api and an ironic-conductor services on the same host or different hosts. Users also can add new ironic-conductor hosts to deal with an increasing number of bare metal nodes. But the additional ironic-conductor services should be at the same version as that of existing ironic-conductor services.

Configuring ironic-api service

1. The Bare Metal service stores information in a database. This guide uses the MySQL database that is used by other OpenStack services.

   Configure the location of the database via the `connection` option. In the following, replace `IRONIC_DBPASSWORD` with the password of your `ironic` user, and replace `DB_IP` with the IP address where the DB server is located:

   ```
   [database]

   # The SQLAlchemy connection string used to connect to the
   # database (string value)
   connection=mysql+pymysql://ironic:IRONIC_DBPASSWORD@DB_IP/ironic?
   ↪charset=utf8
   ```

2. Configure the ironic-api service to use the RabbitMQ message broker by setting the following option. Replace `RPC_*` with appropriate address details and credentials of RabbitMQ server:

```
[DEFAULT]

# A URL representing the messaging driver to use and its full
# configuration. (string value)
transport_url = rabbit://RPC_USER:RPC_PASSWORD@RPC_HOST:RPC_PORT/
```

Alternatively, you can use JSON RPC for interactions between ironic-conductor and ironic-api. Enable it in the configuration and provide the keystone credentials to use for authentication:

```
[DEFAULT]

rpc_transport = json-rpc

[json_rpc]

# Authentication type to load (string value)
auth_type = password

# Authentication URL (string value)
auth_url=https://IDENTITY_IP:5000/

# Username (string value)
username=ironic

# User's password (string value)
password=IRONIC_PASSWORD

# Project name to scope to (string value)
project_name=service

# Domain ID containing project (string value)
project_domain_id=default

# User's domain id (string value)
user_domain_id=default
```

If you use port other than the default 8089 for JSON RPC, you have to configure it, for example:

```
[json_rpc]
port = 9999
```

3. Configure the ironic-api service to use these credentials with the Identity service. Replace `PUBLIC_IDENTITY_IP` with the public IP of the Identity server, `PRIVATE_IDENTITY_IP` with the private IP of the Identity server, replace `IRONIC_PASSWORD` with the password you chose for the `ironic` user in the Identity service and replace `MEMCACHED_SERVER` with the hostname of the memcached server:

```
[DEFAULT]

# Authentication strategy used by ironic-api: one of
# "keystone" or "noauth". "noauth" should not be used in a
```

(continues on next page)

(continued from previous page)

```
# production environment because all authentication will be
# disabled. (string value)
auth_strategy=keystone

[keystone_authtoken]

# Authentication type to load (string value)
auth_type=password

# Complete public Identity API endpoint (string value)
www_authenticate_uri=http://PUBLIC_IDENTITY_IP:5000

# Complete admin Identity API endpoint. (string value)
auth_url=http://PRIVATE_IDENTITY_IP:5000

# Service username. (string value)
username=ironic

# Service account password. (string value)
password=IRONIC_PASSWORD

# Service tenant name. (string value)
project_name=service

# Domain name containing project (string value)
project_domain_name=Default

# User's domain name (string value)
user_domain_name=Default

# memcached setting (string value)
memcached_servers=MEMCACHED_SERVER:11211
```

4. Create the Bare Metal service database tables:

```
$ ironic-dbsync --config-file /etc/ironic/ironic.conf create_schema
```

5. Restart the ironic-api service:

   RHEL/CentOS:

```
sudo systemctl restart openstack-ironic-api
```

   Ubuntu/Debian:

```
sudo service ironic-api restart
```

Configuring ironic-api behind a WSGI server

Bare Metal service can be configured to run behind any WSGI-capable web server like uWSGI or Gunicorn for better performance and scalability.

Note

This is optional, the ironic APIs can be run using the standalone `ironic-api` command. However, for production deployments, it is recommended to use a proper WSGI server for better performance, multiple workers, and integration with existing infrastructure.

The WSGI application

Ironic provides a WSGI application at `ironic.wsgi:application` that can be used with any WSGI server. The below example uses uWSGI, which is used in Ironic CI jobs.

Using uWSGI

1. Install uWSGI:

```
pip install uwsgi
```

2. Create a uWSGI configuration file (e.g., `/etc/uwsgi/ironic.ini`):

```
[uwsgi]
module = ironic.wsgi:application
http-socket = 127.0.0.1:6385
processes = 2
; allow 60 seconds for graceful shutdown on SIGTERM
die-on-term = true
exit-on-reload = false
hook-master-start = unix_signal:15 gracefully_kill_them_all
worker-reload-mercy = 60
; disallow connection reuse
add-header = Connection: close
; Prevent thundering herd on accept()
thunder-lock = true
; ensure file descriptors aren't shared between processes
lazy-apps = true
enable-threads = true
master = true
```

Note

This uWSGI configuration and comments are based on devstack configuration. You may need to modify settings depending on your deployment method and scale.

1. Start uWSGI:

```
uwsgi --ini /etc/uwsgi/ironic.ini
```

Reverse Proxy Configuration

WSGI servers are typically deployed behind a reverse proxy like nginx or Apache for SSL termination, load balancing, and serving static files.

Example nginx configuration:

```
upstream ironic-api {
    server 127.0.0.1:6385;
}

server {
    listen 443 ssl;
    server_name ironic.example.com;

    ssl_certificate /etc/ssl/certs/ironic.crt;
    ssl_certificate_key /etc/ssl/private/ironic.key;

    location / {
        proxy_pass http://ironic-api;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

For detailed reverse proxy configurations, see:

- nginx documentation
- Apache mod_proxy documentation
- HAProxy documentation

Passing Configuration Options

By default, Ironic will use its standard configuration file search paths. See https://docs.openstack.org/oslo.config/latest/configuration/options.html for more info.

If you need to specify a custom configuration file, you can set the `IRONIC_CONFIG_FILE` and/or `IRONIC_CONFIG_DIR` environment variable:

```
[uwsgi]
...
env = IRONIC_CONFIG_DIR=/etc/mycustomdir/
```

Important Considerations

1. Stop and disable the standalone ironic-api service before starting the WSGI server to avoid port conflicts.
2. When behind a reverse proxy, ensure `[oslo_middleware]/enable_proxy_headers_parsing` is set to `True` in ironic.conf to properly handle X-Forwarded headers.

Configuring ironic-conductor service

1. Replace `HOST_IP` with IP of the conductor host.

```
[DEFAULT]

# IP address of this host. If unset, will determine the IP
# programmatically. If unable to do so, will use "127.0.0.1".
# (string value)
my_ip=HOST_IP
```

> **Note**
>
> If a conductor host has multiple IPs, `my_ip` should be set to the IP which is on the same network as the bare metal nodes.

2. Configure the location of the database. Ironic-conductor should use the same configuration as ironic-api. Replace `IRONIC_DBPASSWORD` with the password of your `ironic` user, and replace DB_IP with the IP address where the DB server is located:

```
[database]

# The SQLAlchemy connection string to use to connect to the
# database. (string value)
connection=mysql+pymysql://ironic:IRONIC_DBPASSWORD@DB_IP/ironic?
↪charset=utf8
```

3. Configure the ironic-conductor service to use the RabbitMQ message broker by setting the following option. Ironic-conductor should use the same configuration as ironic-api. Replace `RPC_*` with appropriate address details and credentials of RabbitMQ server:

```
[DEFAULT]

# A URL representing the messaging driver to use and its full
# configuration. (string value)
transport_url = rabbit://RPC_USER:RPC_PASSWORD@RPC_HOST:RPC_PORT/
```

Alternatively, you can use JSON RPC for interactions between ironic-conductor and ironic-api. Enable it in the configuration and provide the keystone credentials to use for authenticating incoming requests (can be the same as for the API):

```
[DEFAULT]

rpc_transport = json-rpc

[keystone_authtoken]

# Authentication type to load (string value)
auth_type=password

# Complete public Identity API endpoint (string value)
```

(continues on next page)

(continued from previous page)

```
www_authenticate_uri=http://PUBLIC_IDENTITY_IP:5000

# Complete admin Identity API endpoint. (string value)
auth_url=http://PRIVATE_IDENTITY_IP:5000

# Service username. (string value)
username=ironic

# Service account password. (string value)
password=IRONIC_PASSWORD

# Service tenant name. (string value)
project_name=service

# Domain name containing project (string value)
project_domain_name=Default

# User's domain name (string value)
user_domain_name=Default
```

You can optionally change the host and the port the JSON RPC service will bind to, for example:

```
[json_rpc]
host_ip = 192.168.0.10
port = 9999
```

Warning

Hostnames of ironic-conductor machines must be resolvable by ironic-api services when JSON RPC is used.

4. Configure credentials for accessing other OpenStack services.

   In order to communicate with other OpenStack services, the Bare Metal service needs to use service users to authenticate to the OpenStack Identity service when making requests to other services. These users credentials have to be configured in each configuration file section related to the corresponding service:

   - `[neutron]` - to access the OpenStack Networking service
   - `[glance]` - to access the OpenStack Image service
   - `[swift]` - to access the OpenStack Object Storage service
   - `[cinder]` - to access the OpenStack Block Storage service
   - `[service_catalog]` - a special section holding credentials the Bare Metal service will use to discover its own API URL endpoint as registered in the OpenStack Identity service catalog.

   For simplicity, you can use the same service user for all services. For backward compatibility, this should be the same user configured in the `[keystone_authtoken]` section for the ironic-api service (see Configuring ironic-api service). However, this is not necessary, and you can create and configure separate service users for each service.

Under the hood, Bare Metal service uses `keystoneauth` library together with `Authentication plugin`, `Session` and `Adapter` concepts provided by it to instantiate service clients. Please refer to Keystoneauth documentation for supported plugins, their available options as well as Session- and Adapter-related options for authentication, connection and endpoint discovery respectively.

In the example below, authentication information for user to access the OpenStack Networking service is configured to use:

- Networking service is deployed in the Identity service region named `RegionTwo`, with only its `public` endpoint interface registered in the service catalog.
- HTTPS connection with specific CA SSL certificate when making requests
- the same service user as configured for ironic-api service
- dynamic `password` authentication plugin that will discover appropriate version of Identity service API based on other provided options
  - replace `IDENTITY_IP` with the IP of the Identity server, and replace `IRONIC_PASSWORD` with the password you chose for the `ironic` user in the Identity service

```
[neutron]

# Authentication type to load (string value)
auth_type = password

# Authentication URL (string value)
auth_url=https://IDENTITY_IP:5000/

# Username (string value)
username=ironic

# User's password (string value)
password=IRONIC_PASSWORD

# Project name to scope to (string value)
project_name=service

# Domain ID containing project (string value)
project_domain_id=default

# User's domain id (string value)
user_domain_id=default

# PEM encoded Certificate Authority to use when verifying
# HTTPs connections. (string value)
cafile=/opt/stack/data/ca-bundle.pem

# The default region_name for endpoint URL discovery. (string
# value)
region_name = RegionTwo

# List of interfaces, in order of preference, for endpoint
# URL. (list value)
```

(continues on next page)

(continued from previous page)

```
valid_interfaces=public
```

By default, in order to communicate with another service, the Bare Metal service will attempt to discover an appropriate endpoint for that service via the Identity services service catalog. The relevant configuration options from that service group in the Bare Metal service configuration file are used for this purpose. If you want to use a different endpoint for a particular service, specify this via the `endpoint_override` configuration option of that service group, in the Bare Metal services configuration file. Taking the previous Networking service example, this would be

```
[neutron]
...
endpoint_override = <NEUTRON_API_ADDRESS>
```

(Replace *<NEUTRON_API_ADDRESS>* with actual address of a specific Networking service endpoint.)

5. Configure enabled drivers and hardware types as described in *Enabling drivers and hardware types*.

   A. If you enabled any driver that uses *Direct deploy*, Swift backend for the Image service must be installed and configured, see *Configure the Image service for temporary URLs*. Ceph Object Gateway (RADOS Gateway) is also supported as the Image services backend, see *Ceph Object Gateway support*.

6. Configure the network for ironic-conductor service to perform node cleaning, see *Node cleaning* from the admin guide.

7. If ironic-novncproxy is enabled, ironic-conductor must be configured to build valid console URLs and it also needs to be configured with a console container provider. Each enabled console has a corresponding running container which runs a headless X11 session, connects to the graphical console of the BMC, and exposes a VNC server for ironic-novncproxy to connect to.

   Replace `PUBLIC_IP` and `PUBLIC_PORT` with appropriate values:

```
[vnc]

# Base url used to build browser links to graphical consoles. If a
# reverse proxy is used the protocol, IP, and port needs to match how
# users will access the service. When there is no reverse proxy
# ``PUBLIC_IP`` should match ``[vnc]host_ip`` and ``PUBLIC_PORT`` should
# match ``[vnc]port``
public_url=http://PUBLIC_IP:PUBLIC_PORT/vnc_lite.html

# The only functional container provider included is the systemd
# provider which manages containers as Systemd Quadlet containers. This
# provider is appropriate to use when the Ironic services themselves are
# not containerised, otherwise a custom external provider may be
# required
container_provider=systemd

# For the ``container_provider=systemd``, set a valid container image
# reference available to the podman image storage of the user running
# ironic-conductor. See /usr/share/ironic/vnc-container for instructions
```

(continues on next page)

(continued from previous page)

```
# to build a compatible image.
console_image=localhost/vnc-container
```

When `[vnc]container_provider=systemd` then the `openstack-ironic-conductor` service needs to be able to make `systemctl --user` calls for the user which both ironic-conductor and ironic-novncproxy run as.

Assuming the services are running as user `ironic`, discover the <UID> for that user by running `id -u ironic`. Edit `openstack-ironic-conductor.service` to add environment variable `DBUS_SESSION_BUS_ADDRESS`, substituting the <UID>

```
[Service]
...
Environment = DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/<UID>/bus
```

8. Restart the ironic-conductor service:

   RHEL/CentOS:

   ```
   sudo systemctl restart openstack-ironic-conductor
   ```

   Ubuntu/Debian:

   ```
   sudo service ironic-conductor restart
   ```

Configuring ironic-novncproxy service

1. The NoVNC proxy service needs to look up nodes in the database, so `ironic-novncproxy` requires the same database configuration as `ironic-api` and `ironic-conductor`.

   Configure the location of the database via the `connection` option. In the following, replace `IRONIC_DBPASSWORD` with the password of your `ironic` user, and replace `DB_IP` with the IP address where the DB server is located:

   ```
   [database]

   # The SQLAlchemy connection string used to connect to the
   # database (string value)
   connection=mysql+pymysql://ironic:IRONIC_DBPASSWORD@DB_IP/ironic?
   ↪charset=utf8
   ```

2. Configure NoVNC and host graphical console options. Replace `PUBLIC_IP` and `PUBLIC_URL` with appropriate values:

   ```
   [vnc]

   # Enable VNC related features, required to allow the ironic-novncproxy␣
   ↪service to start
   enabled=True

   # Port to bind to for serving NoVNC web assets and websockets
   port=6090
   ```

(continues on next page)

(continued from previous page)

```
# IP address to bind to for serving NoVNC web assets and websockets
host_ip=PUBLIC_IP
```

3. Restart the ironic-novncproxy service:

   RHEL/CentOS:

   ```
   sudo systemctl restart openstack-ironic-novncproxy
   ```

   Ubuntu/Debian:

   ```
   sudo service ironic-novncproxy restart
   ```

Configuring single-process ironic

As an alternative to starting separate API and conductor instances, you can start `ironic` services that combine an API and a conductor in the same process. This may be particularly beneficial in environments with limited resources and low number of nodes to handle.

Note

This feature is available starting with the Yoga release series.

1. Start with setting up the environment as described in *Configuring ironic-api service*, *Configuring ironic-conductor service*, and *Configuring ironic-novncproxy service*, but do not start any services. Merge configuration options into a single configuration file.

   Note

   Any RPC settings will only take effect if you have more than one combined service started or if you have additional conductors.

   If you dont plan to have more than one conductor, you can disable the RPC completely:

   ```
   [DEFAULT]
   rpc_transport = none
   ```

2. Stop existing services if they are already started:

   RHEL/CentOS:

   ```
   sudo systemctl stop openstack-ironic-api
   sudo systemctl stop openstack-ironic-conductor
   sudo systemctl stop openstack-ironic-novncproxy
   ```

   Ubuntu/Debian:

   ```
   sudo service ironic-api stop
   sudo service ironic-conductor stop
   sudo service ironic-novncproxy stop
   ```

3. Start or restart the ironic service:

   RHEL/CentOS:

   ```
   sudo systemctl restart openstack-ironic
   ```

   Ubuntu/Debian:

   ```
   sudo service ironic restart
   ```

2.1.4 Building or downloading a deploy ramdisk image

Ironic depends on having an image with the ironic-python-agent (IPA) service running on it for controlling and deploying bare metal nodes. This image is not written *to* the storage of any given node, but provides the runtime tooling and communication for the inspection, cleaning, and ultimately deployment of a bare metal node managed by Ironic.

Ironic publishes images on every commit from every branch of ironic-python-agent (IPA)

- DIB images are suitable for production usage and can be downloaded from https://tarballs.openstack.org/ironic-python-agent/dib/files/.

> **Warning**
>
> The published images will not work for dhcp-less deployments since the simple-init element is not present. Check the DIB documentation to see how to build the image.

Building from source

Check the ironic-python-agent-builder project for information on how to build ironic-python-agent ramdisks.

2.1.5 Integration with other OpenStack services

Configure the Identity service for the Bare Metal service

1. Create the Bare Metal service user (for example, `ironic`). The service uses this to authenticate with the Identity service. Use the `service` tenant and give the user the `admin` role:

   ```
   $ openstack user create --password IRONIC_PASSWORD \
     --email ironic@example.com ironic
   $ openstack role add --project service --user ironic admin
   ```

2. You must register the Bare Metal service with the Identity service so that other OpenStack services can locate it. To register the service:

   ```
   $ openstack service create --name ironic --description \
     "Ironic baremetal provisioning service" baremetal
   ```

3. Use the `id` property that is returned from the Identity service when registering the service (above), to create the endpoint, and replace `IRONIC_NODE` with your Bare Metal services API node:

```
$ openstack endpoint create --region RegionOne \
    baremetal admin http://$IRONIC_NODE:6385
$ openstack endpoint create --region RegionOne \
    baremetal public http://$IRONIC_NODE:6385
$ openstack endpoint create --region RegionOne \
    baremetal internal http://$IRONIC_NODE:6385
```

4. You may delegate limited privileges related to the Bare Metal service to your Users by creating Roles with the OpenStack Identity service. By default, the Bare Metal service expects the baremetal_admin and baremetal_observer Roles to exist, in addition to the default admin Role. There is no negative consequence if you choose not to create these Roles. They can be created with the following commands:

   ```
   $ openstack role create baremetal_admin
   $ openstack role create baremetal_observer
   ```

   If you choose to customize the names of Roles used with the Bare Metal service, do so by changing the is_member, is_observer, and is_admin policy settings in `/etc/ironic/policy.yaml`.

   More complete documentation on managing Users and Roles within your OpenStack deployment are outside the scope of this document, but may be found here.

5. You can further restrict access to the Bare Metal service by creating a separate baremetal Project, so that Bare Metal resources (Nodes, Ports, etc) are only accessible to members of this Project:

   ```
   $ openstack project create baremetal
   ```

   At this point, you may grant read-only access to the Bare Metal service API without granting any other access by issuing the following commands:

   ```
   $ openstack user create \
     --domain default --project-domain default --project baremetal \
     --password PASSWORD USERNAME
   $ openstack role add \
     --user-domain default --project-domain default --project baremetal \
     --user USERNAME baremetal_observer
   ```

6. Further documentation is available elsewhere for the `openstack` command-line client and the Identity service. A *policy.yaml.sample* file, which enumerates the services default policies, is provided for your convenience with the Bare Metal Service.

Configure the Compute service to use the Bare Metal service

The Compute service needs to be configured to use the Bare Metal services driver. The configuration file for the Compute service is typically located at `/etc/nova/nova.conf`.

The following configuration file must be modified on the Compute services controller nodes and compute nodes.

1. Change these configuration options in the Compute service configuration file (for example, `/etc/nova/nova.conf`):

```
[default]

# Defines which driver to use for controlling virtualization.
# Enable the ironic virt driver for this compute instance.
compute_driver=ironic.IronicDriver

# Amount of memory in MB to reserve for the host so that it is always
# available to host processes.
# It is impossible to reserve any memory on bare metal nodes, so set
# this to zero.
reserved_host_memory_mb=0

[filter_scheduler]

# Enables querying of individual hosts for instance information.
# Not possible for bare metal nodes, so set it to False.
track_instance_changes=False

[scheduler]

# This value controls how often (in seconds) the scheduler should
# attempt to discover new hosts that have been added to cells.
# If negative (the default), no automatic discovery will occur.
# As each bare metal node is represented by a separate host, it has
# to be discovered before the Compute service can deploy on it.
# The value here has to be carefully chosen based on a compromise
# between the enrollment speed and the load on the Compute scheduler.
# The recommended value of 2 minutes matches how often the Compute
# service polls the Bare Metal service for node information.
discover_hosts_in_cells_interval=120
```

Note

The alternative to setting the `discover_hosts_in_cells_interval` option is to run the following command on any Compute controller node after each node is enrolled:

```
nova-manage cell_v2 discover_hosts --by-service
```

2. Consider enabling the following option on controller nodes:

```
[filter_scheduler]

# Enabling this option is beneficial as it reduces re-scheduling events
# for ironic nodes when scheduling is based on resource classes,
# especially for mixed hypervisor case with host_subset_size = 1.
# However enabling it will also make packing of VMs on hypervisors
# less dense even when scheduling weights are completely disabled.
#shuffle_best_same_weighed_hosts = false
```

3. Carefully consider the following option:

```
[compute]

# This option will cause nova-compute to set itself to a disabled state
# if a certain number of consecutive build failures occur. This will
# prevent the scheduler from continuing to send builds to a compute
# service that is consistently failing. In the case of bare metal
# provisioning, however, a compute service is rarely the cause of build
# failures. Furthermore, bare metal nodes, managed by a disabled
# compute service, will be remapped to a different one. That may cause
# the second compute service to also be disabled, and so on, until no
# compute services are active.
# If this is not the desired behavior, consider increasing this value or
# setting it to 0 to disable this behavior completely.
#consecutive_build_service_disable_threshold = 10
```

4. Change these configuration options in the `ironic` section. Replace:
   - `IRONIC_PASSWORD` with the password you chose for the `ironic` user in the Identity Service
   - `IRONIC_NODE` with the hostname or IP address of the ironic-api node
   - `IDENTITY_IP` with the IP of the Identity server

```
[ironic]

# Ironic authentication type
auth_type=password

# Keystone API endpoint
auth_url=http://IDENTITY_IP:5000/v3

# Ironic keystone project name
project_name=service

# Ironic keystone admin name
username=ironic

# Ironic keystone admin password
password=IRONIC_PASSWORD

# Ironic keystone project domain
# or set project_domain_id
project_domain_name=Default

# Ironic keystone user domain
# or set user_domain_id
user_domain_name=Default
```

5. On the Compute services controller nodes, restart the `nova-scheduler` process:

```
Fedora/RHEL/CentOS:
  sudo systemctl restart openstack-nova-scheduler
```

(continues on next page)

(continued from previous page)

```
Ubuntu:
  sudo service nova-scheduler restart
```

6. On the Compute services compute nodes, restart the `nova-compute` process:

```
Fedora/RHEL/CentOS:
  sudo systemctl restart openstack-nova-compute

Ubuntu:
  sudo service nova-compute restart
```

Configure the Networking service for bare metal provisioning

You need to configure Networking so that the bare metal server can communicate with the Networking service for DHCP, PXE/HTTP boot and other requirements. This section covers configuring Networking for a single flat network for bare metal provisioning. In more advanced configurations, we typically refer to the network upon which nodes undergo deployment as the provisioning network, as the underlying resources to provision the node must be available for successful operations.

Warning

This documentation is geared for use of OVS with Neutron along with the `neutron-dhcp-agent`. It *is* possible to use OVN with `neutron-dhcp-agent`, and depending on version of OVN and Neutron, OVNs own DHCP service for IPv4 clients, but that is considered an advanced topic, and we encourage operators interested in use of OVN to fully understand its capabilities and state before attempting to utilize such a configuration. Please see *Use of OVN Networking* for more details.

It is recommended to use the baremetal ML2 mechanism driver and L2 agent for proper integration with the Networking service. Documentation regarding installation and configuration of the baremetal mechanism driver and L2 agent is available here.

For use with routed networks the baremetal ML2 components are required.

Note

When the baremetal ML2 components are *not* used, ports in the Networking service will have status: `DOWN`, and binding_vif_type: `binding_failed`. This was always the status for Bare Metal service `flat` network interface ports prior to the introduction of the baremetal ML2 integration. For a non-routed network, bare metal servers can still be deployed and are functional, despite this port binding state in the Networking service.

You will also need to provide Bare Metal service with the MAC address(es) of each node that it is provisioning; Bare Metal service in turn will pass this information to Networking service for DHCP and PXE boot configuration. An example of this is shown in the *Enrolling hardware with Ironic* section.

1. Install the networking-baremetal ML2 mechanism driver and L2 agent in the Networking service.
2. Edit `/etc/neutron/plugins/ml2/ml2_conf.ini` and modify these:

```
[ml2]
type_drivers = flat
project_network_types = flat
mechanism_drivers = openvswitch,baremetal

[ml2_type_flat]
flat_networks = physnet1

[securitygroup]
firewall_driver = neutron.agent.linux.iptables_firewall.
↪OVSHybridIptablesFirewallDriver
enable_security_group = True

[ovs]
bridge_mappings = physnet1:br-eth2
# Replace eth2 with the interface on the neutron node which you
# are using to connect to the bare metal server
```

3. Restart the `neutron-server` service, to load the new configuration.
4. Create and edit `/etc/neutron/plugins/ml2/ironic_neutron_agent.ini` and add the required configuration. For example:

```
[ironic]
project_domain_name = Default
project_name = service
user_domain_name = Default
password = password
username = ironic
auth_url = http://identity-server.example.com/identity
auth_type = password
region_name = RegionOne
```

5. Make sure the `ironic-neutron-agent` service is started.
6. If neutron-openvswitch-agent runs with `ovs_neutron_plugin.ini` as the input config-file, edit `ovs_neutron_plugin.ini` to configure the bridge mappings by adding the [ovs] section described in the previous step, and restart the neutron-openvswitch-agent.
7. Add the integration bridge to Open vSwitch:

```
$ ovs-vsctl add-br br-int
```

8. Create the br-eth2 network bridge to handle communication between the OpenStack services (and the Bare Metal services) and the bare metal nodes using eth2. Replace eth2 with the interface on the network node which you are using to connect to the Bare Metal service:

```
$ ovs-vsctl add-br br-eth2
$ ovs-vsctl add-port br-eth2 eth2
```

9. Restart the Open vSwitch agent:

```
# service neutron-plugin-openvswitch-agent restart
```

10. On restarting the Networking service Open vSwitch agent, the veth pair between the bridges br-int and br-eth2 is automatically created.

    Your Open vSwitch bridges should look something like this after following the above steps:

```
$ ovs-vsctl show

    Bridge br-int
        fail_mode: secure
        Port "int-br-eth2"
            Interface "int-br-eth2"
                type: patch
                options: {peer="phy-br-eth2"}
        Port br-int
            Interface br-int
                type: internal
    Bridge "br-eth2"
        Port "phy-br-eth2"
            Interface "phy-br-eth2"
                type: patch
                options: {peer="int-br-eth2"}
        Port "eth2"
            Interface "eth2"
        Port "br-eth2"
            Interface "br-eth2"
                type: internal
    ovs_version: "2.3.0"
```

11. Create the flat network on which you are going to launch the instances:

```
$ openstack network create --project $TENANT_ID sharednet1 --share \
    --provider-network-type flat --provider-physical-network physnet1
```

12. Create the subnet on the newly created network:

```
$ openstack subnet create $SUBNET_NAME --network sharednet1 \
  --subnet-range $NETWORK_CIDR --ip-version 4 --gateway $GATEWAY_IP \
  --allocation-pool start=$START_IP,end=$END_IP --dhcp
```

Configuring services for bare metal provisioning using IPv6

Use of IPv6 addressing for baremetal provisioning requires additional configuration. This page covers the IPv6 specifics only. Please refer to *Configure tenant networks* and *Configure the Networking service for bare metal provisioning* for general networking configuration.

Warning

IPv6 Networking is not actively tested in upstream Continuous Integration pipelines, in large part because test virtual machines have limited support and known issues. Hardware vendors often have

better firmware which enables this functionality, but may require specific operating modes, such as `stateless` or `stateful` depending on the requirements placed upon them by their customers.

Configure ironic PXE driver for provisioning using IPv6 addressing

The PXE drivers operate in such a way that they are able to utilize both IPv4 and IPv6 addresses based upon the deployments operating state and configuration. Internally, the drivers attempt to prepare configuration options for both formats, which allows ports which are IPv6 only to automatically receive boot parameters. As a result of this, it is critical that the *DEFAULT.my_ipv6* configuration parameter is set to the conductors IPv6 address. This option is unique per conductor, and due to the nature of automatic address assignment, it cannot be guessed by the software.

Provisioning with IPv6 stateless addressing

When using stateless addressing DHCPv6 does not provide addresses to the client. DHCPv6 however provides other configuration via DHCPv6 options such as the bootfile-url and bootfile-parameters.

Once the PXE driver is set to operate in IPv6 mode no further configuration is required in the Baremetal Service.

Creating networks and subnets in the Networking Service

When creating the Baremetal Service network(s) and subnet(s) in the Networking Services, subnets should have `ipv6-address-mode` set to `dhcpv6-stateless` and `ip-version` set to `6`. Depending on whether a router in the Networking Service is providing RAs (Router Advertisements) or not, the `ipv6-ra-mode` for the subnet(s) should either be set to `dhcpv6-stateless` or be left unset.

Note

If `ipv6-ra-mode` is left unset, an external router on the network is expected to provide RAs with the appropriate flags set for automatic addressing and other configuration.

Provisioning with IPv6 stateful addressing

When using stateful addressing DHCPv6 is providing both addresses and other configuration via DHCPv6 options such as the bootfile-url and bootfile-parameters.

The identity-association (IA) construct used by DHCPv6 is challenging when booting over the network. Firmware, and ramdisks typically end up using different DUID/IAID combinations and it is not always possible for one chain-booting stage to release its address before giving control to the next step. In case the DHCPv6 server is configured with static reservations only the result is that booting will fail because the DHCPv6 server has no addresses available. To get past this issue either configure the DHCPv6 server with multiple address reservations for each host, or use a dynamic range.

Note

Support for multiple address reservations requires dnsmasq version 2.81 or later. Some distributions may backport this feature to earlier dnsmasq version as part of the packaging, check the distributions release notes.

If a different (not dnsmasq) DHCPv6 server backend is used with the Networking service, use of multiple address reservations might not work.

Using the `flat` network interface

Due to the identity-association challenges with DHCPv6 provisioning using the `flat` network interface is not recommended. When ironic operates with the `flat` network interface the server instance port is used for provisioning and other operations. Ironic will not use multiple address reservations in this scenario. Because of this **it will not work in most cases**.

Using the `neutron` network interface

When using the `neutron` network interface the Baremetal Service will allocate multiple IPv6 addresses (4 addresses per port by default) on the service networks used for provisioning, cleaning, rescue and introspection. The number of addresses allocated can be controlled via the `[neutron]/dhcpv6_stateful_address_count` option in the Bare Metal Services configuration file (`/etc/ironic/ironic.conf`). Using multiple address reservations ensures that the DHCPv6 server can lease addresses to each step.

To enable IPv6 provisioning on neutron *flat* provider networks with no switch management, the `local_link_connection` field of baremetal ports must be set to `{'network_type': 'unmanaged'}`. The following example shows how to set the local_link_connection for operation on unmanaged networks:

```
baremetal port set \
  --local-link-connection network_type=unmanaged <port-uuid>
```

The use of multiple IPv6 addresses must also be enabled in the Networking Services dhcp agent configuration (`/etc/neutron/dhcp_agent.ini`) by setting the option `[DEFAULT]/dnsmasq_enable_addr6_list` to `True` (default `False` in Ussuri release).

Note

Support for multiple IPv6 address reservations in the dnsmasq backend was added to the Networking Service Ussuri release. It was also backported to the stable Train release.

Creating networks and subnets in the Networking Service

When creating the ironic service network(s) and subnet(s) in the Networking Service, subnets should have `ipv6-address-mode` set to `dhcpv6-stateful` and `ip-version` set to `6`. Depending on whether a router in the Networking Service is providing RAs (Router Advertisements) or not, the `ipv6-ra-mode` for the subnet(s) should be set to either `dhcpv6-stateful` or be left unset.

Note

If `ipv6-ra-mode` is left unset, an external router on the network is expected to provide RAs with the appropriate flags set for managed addressing and other configuration.

Configure Trait Based Networking to Plan Networking Related Operations at vif Attach

Ironic has a feature called Trait Based Networking (or TBN) that allows operators to configure how a nodes network will be built and attached.

TBN applies to OpenStack installations utilizing Ironic, Neutron, and Nova that want dynamic port scheduling based on networks and flavors chosen by the instance creator.

In order to use this feature, a few steps must be completed.

1. Enable Trait Based Networking in the ironic-conductor service configuration:

```
[conductor]

# Enables Trait Based Networking, defaults to False
enable_trait_based_networking=True
```

2. Place a TBN configuration file in the configured location. The default location is: `/etc/ironic/trait_based_networking.yaml`.

   For discussion of the syntax and format of the configuration file refer to *Trait Based Networking Configuration File Reference*.

   The default configuration which ships with Ironic is reproduced below:

```
CUSTOM_TRAIT_NAME:
  order: 1
  actions:
    - action: group_and_attach_ports
      filter: port.vendor == 'vendor_string'
      min_count: 2
CUSTOM_DIRECT_ATTACH_A_PURPLE_TO_STORAGE:
  actions:
    - action: attach_port
      filter: port.vendor == 'purple' && network.name == 'storage
↪'
CUSTOM_BOND_PURPLE_BY_2:
  actions:
    - action: group_and_attach_ports
      filter: port.vendor == 'purple'
      min_count: 2
      max_count: 2
CUSTOM_BOND_GREEN_STORAGE_TO_STORAGE_BY_2:
  actions:
    - action: group_and_attach_ports
      filter: port.vendor == 'green' && port.category == 'storage
↪' && ( network.name =~ 'storage' || network.tags =~ 'storage' )
      max_count: 2
      min_count: 2
CUSTOM_USE_PHYSNET_A_OR_B:
  actions:
    - action: attach_port
      filter: port.physical_network == 'fabric_a' && network.
↪tags == 'a'
```

(continues on next page)

(continued from previous page)

```
    - action: attach_port
      filter: port.physical_network == 'fabric_b' && network.
↪tags == 'b'
```

3. Set desired TBN traits on a nodes `instance_info.traits`. Trait names must match exactly for a TBN trait to be applied.

Then, when `vif_attach` is called, TBN will plan networking operations based on the nodes `instance_info.traits` and supplied configured traits. If planning succeeds, then each network operation will be applied.

Configure the Image service for temporary URLs

Some drivers of the Baremetal service (in particular, any drivers using *Direct deploy* or *Ansible deploy* interfaces, and some virtual media drivers) require target user images to be available over clean HTTP(S) URL with no authentication involved (neither username/password-based, nor token-based).

When using the Baremetal service integrated in OpenStack, this can be achieved by specific configuration of the Image service and Object Storage service as described below.

1. Configure the Image service to have object storage as a backend for storing images. For more details, please refer to the Image service configuration guide.

> **Note**
>
> When using Ceph+RadosGW for Object Storage service, images stored in Image service must be available over Object Storage service as well.

2. Enable TempURLs for the Object Storage account used by the Image service for storing images in the Object Storage service.

    1. Check if TempURLs are enabled:

```
# executed under credentials of the user used by Image service
# to access Object Storage service
$ openstack object store account show
+------------+----------------------------------------+
| Field      | Value                                  |
+------------+----------------------------------------+
| Account    | AUTH_bc39f1d9dcf9486899088007789ae643  |
| Bytes      | 536661727                              |
| Containers | 1                                      |
| Objects    | 19                                     |
| properties | Temp-Url-Key='secret'                  |
+------------+----------------------------------------+
```

    2. If property `Temp-Url-Key` is set, note its value.

    3. If property `Temp-Url-Key` is not set, you have to configure it (`secret` is used in the example below for the value):

```
$ openstack object store account set --property Temp-Url-Key=secret
```

3. Optionally, configure the ironic-conductor service. The default configuration assumes that:

    1. the Object Storage service is implemented by swift,
    2. the Object Storage service URL is available from the service catalog,
    3. the project, used by the Image service to access the Object Storage, is the same as the project, used by the Bare Metal service to access it,
    4. the container, used by the Image service, is called `glance`.

    If any of these assumptions do not hold, you may want to change your configuration file (typically located at `/etc/ironic/ironic.conf`), for example:

```
[glance]

swift_endpoint_url = http://openstack/swift
swift_account = AUTH_bc39f1d9dcf9486899088007789ae643
swift_container = glance
swift_temp_url_key = secret
```

4. (Re)start the ironic-conductor service.

Enabling HTTPS

Enabling HTTPS in Swift

The drivers using virtual media use swift for storing boot images and node configuration information (contains sensitive information for Ironic conductor to provision bare metal hardware). By default, HTTPS is not enabled in swift. HTTPS is required to encrypt all communication between swift and Ironic conductor and swift and bare metal (via virtual media). It can be enabled in one of the following ways:

- Using an SSL termination proxy
- Using native SSL support in swift (recommended only for testing purpose by swift).

Enabling HTTPS in Image service

Ironic drivers usually use Image service during node provisioning. By default, image service does not use HTTPS, but it is required for secure communication. It can be enabled by making the following changes to `/etc/glance/glance-api.conf`:

1. Configuring SSL support
2. Restart the glance-api service:

```
Fedora/RHEL/CentOS:
    sudo systemctl restart openstack-glance-api

Debian/Ubuntu:
    sudo service glance-api restart
```

See the Glance documentation, for more details on the Image service.

Enabling HTTPS communication between Image service and Object storage

This section describes the steps needed to enable secure HTTPS communication between Image service and Object storage when Object storage is used as the Backend.

To enable secure HTTPS communication between Image service and Object storage follow these steps:

1. *Enabling HTTPS in Swift*
2. Configure Swift Storage Backend
3. *Enabling HTTPS in Image service*

Enabling HTTPS communication between Image service and Bare Metal service

This section describes the steps needed to enable secure HTTPS communication between Image service and Bare Metal service.

To enable secure HTTPS communication between Bare Metal service and Image service follow these steps:

1. Edit `/etc/ironic/ironic.conf`:

```
[glance]
...
glance_cafile=/path/to/certfile
```

> **Note**
>
> glance_cafile is an optional path to a CA certificate bundle to be used to validate the SSL certificate served by Image service.

2. If not using the keystone service catalog for the Image service API endpoint discovery, also edit the `endpoint_override` option to point to HTTPS URL of image service (replace `<GLANCE_API_ADDRESS>` with hostname[:port][path] of the Image service endpoint):

```
[glance]
...
endpoint_override = https://<GLANCE_API_ADDRESS>
```

3. Restart ironic-conductor service:

```
Fedora/RHEL/CentOS:
    sudo systemctl restart openstack-ironic-conductor

Debian/Ubuntu:
    sudo service ironic-conductor restart
```

Configure the Bare Metal service for cleaning

> **Note**

If you configured the Bare Metal service to do *Automated cleaning* (which is enabled by default), you will need to set the `cleaning_network` configuration option.

1. Note the network UUID (the `id` field) of the network you created in *Configure the Networking service for bare metal provisioning* or another network you created for cleaning:

```
$ openstack network list
```

2. Configure the cleaning network UUID via the `cleaning_network` option in the Bare Metal service configuration file (`/etc/ironic/ironic.conf`). In the following, replace `NETWORK_UUID` with the UUID you noted in the previous step:

```
[neutron]
cleaning_network = NETWORK_UUID
```

3. Restart the Bare Metal services ironic-conductor:

```
Fedora/RHEL/CentOS:
  sudo systemctl restart openstack-ironic-conductor

Ubuntu:
  sudo service ironic-conductor restart
```

Configure tenant networks

Below is an example flow of how to set up the Bare Metal service so that node provisioning will happen in a multi-tenant environment (which means using the `neutron` network interface as stated above):

Note

For standalone deployments without the Networking service (Neutron), the `ironic-networking` network interface provides an alternative approach for managing switch port configurations. See *Configure the Standalone Networking Service* for details.

1. Network interfaces can be enabled on ironic-conductor by adding them to the `enabled_network_interfaces` configuration option under the `default` section of the configuration file:

```
[DEFAULT]
...
enabled_network_interfaces=noop,flat,neutron
```

   Keep in mind that, ideally, all ironic-conductors should have the same list of enabled network interfaces, but it may not be the case during ironic-conductor upgrades. This may cause problems if one of the ironic-conductors dies and some node that is taken over is mapped to an ironic-conductor that does not support the nodes network interface. Any actions that involve calling the nodes driver will fail until that network interface is installed and enabled on that ironic-conductor.

2. It is recommended to set the default network interface via the `default_network_interface` configuration option under the `default` section of the configuration file:

```
[DEFAULT]
...
default_network_interface=neutron
```

This default value will be used for all nodes that dont have a network interface explicitly specified in the creation request.

If this configuration option is not set, the default network interface is determined by looking at the *dhcp.dhcp_provider* configuration option value. If it is `neutron`, then `flat` network interface becomes the default, otherwise `noop` is the default.

3. Define a provider network in the Networking service, which we shall refer to as the provisioning network. Using the `neutron` network interface requires that `provisioning_network` and `cleaning_network` configuration options are set to valid identifiers (UUID or name) of networks in the Networking service. If these options are not set correctly, cleaning or provisioning will fail to start. There are two ways to set these values:

   - Under the `neutron` section of ironic configuration file:

     ```
     [neutron]
     cleaning_network = $CLEAN_UUID_OR_NAME
     provisioning_network = $PROVISION_UUID_OR_NAME
     ```

   - Under `provisioning_network` and `cleaning_network` keys of the nodes `driver_info` field as `driver_info['provisioning_network']` and `driver_info['cleaning_network']` respectively.

Note

If these `provisioning_network` and `cleaning_network` values are not specified in nodes `driver_info` then ironic falls back to the configuration in the `neutron` section.

Please refer to *Configure the Bare Metal service for cleaning* for more information about cleaning.

Warning

Please make sure that the Bare Metal service has exclusive access to the provisioning and cleaning networks. Spawning instances by non-admin users in these networks and getting access to the Bare Metal services control plane is a security risk. For this reason, the provisioning and cleaning networks should be configured as non-shared networks in the `admin` tenant.

Note

When using the `flat` network interface, bare metal instances are normally spawned onto the provisioning network. This is not supported with the `neutron` interface and the deployment will fail. Please ensure a different network is chosen in the Networking service when a bare metal instance is booted from the Compute service.

Note

The provisioning and cleaning networks may be the same network or distinct networks. To ensure that communication between the Bare Metal service and the deploy ramdisk works, it is important to ensure that security groups are disabled for these networks, *or* that the default security groups allow:

- DHCP
- TFTP
- egress port used for the Bare Metal service (6385 by default)
- ingress port used for ironic-python-agent (9999 by default)
- if using *Direct deploy*, the egress port used for the Object Storage service or the local HTTP server (typically 80 or 443)
- if using iPXE, the egress port used for the HTTP server running on the ironic-conductor nodes (typically 80).

4. This step is optional and applicable only if you want to use security groups during provisioning and/or cleaning of the nodes. If not specified, default security groups are used.

   1. Define security groups in the Networking service, to be used for provisioning and/or cleaning networks.
   2. Add the list of these security group UUIDs under the `neutron` section of ironic-conductors configuration file as shown below:

      ```
      [neutron]
      ...
      cleaning_network=$CLEAN_UUID_OR_NAME
      cleaning_network_security_groups=[$LIST_OF_CLEAN_SECURITY_GROUPS]
      provisioning_network=$PROVISION_UUID_OR_NAME
      provisioning_network_security_groups=[$LIST_OF_PROVISION_SECURITY_
      ↪GROUPS]
      ```

      Multiple security groups may be applied to a given network, hence, they are specified as a list. The same security group(s) could be used for both provisioning and cleaning networks.

Warning

If security groups are configured as described above, do not set the port_security_enabled flag to False for the corresponding Networking services network or port. This will cause the deploy to fail.

For example: if `provisioning_network_security_groups` configuration option is used, ensure that port_security_enabled flag for the provisioning network is set to True. This flag is set to True by default; make sure not to override it by manually setting it to False.

5. Install and configure a compatible ML2 mechanism driver which supports bare metal provisioning for your switch. See ML2 plugin configuration manual for details.
6. Restart the ironic-conductor and ironic-api services after the modifications:

- Fedora/RHEL/CentOS:

```
sudo systemctl restart openstack-ironic-api
sudo systemctl restart openstack-ironic-conductor
```

- Ubuntu:

```
sudo service ironic-api restart
sudo service ironic-conductor restart
```

7. Make sure that the ironic-conductor is reachable over the provisioning network by trying to download a file from a TFTP server on it, from some non-control-plane server in that network:

```
tftp $TFTP_IP -c get $FILENAME
```

   where FILENAME is the file located at the TFTP server.

See *Networking with the Bare Metal service* for required node configuration.

Add images to the Image service

Supported Image Formats

Ironic officially supports and tests use of `qcow2` formatted images as well as `raw` format images. Other types of disk images, like `vdi`, and single file `vmdk` files have been reported by users as working in their specific cases, but are not tested upstream. We advise operators to convert the image and properly upload the image to Glance.

Ironic enforces the list of supported and permitted image formats utilizing the `[conductor]permitted_image_formats` option in ironic.conf. This setting defaults to raw and qcow2.

A detected format mismatch between Glance and what the actual contents of the disk image file are detected as will result in a failed deployment. To correct such a situation, the image must be re-uploaded with the declared `--disk-format` or actual image file format corrected.

Instance (end-user) images

Build or download the user images as described in *Creating instance images*.

Load all the created images into the Image service, and note the image UUIDs in the Image service for each one as it is generated.

Note

Images from Glance used by Ironic must be flagged as `public`, which requires administrative privileges with the Glance image service to set.

Note

Starting with the 2025.2 release, the Ironic conductor can access images that are shared with its project, in addition to those it owns.

To use this feature, ensure the required images are shared with the project associated with the conductors credentials.

- For *whole disk images* just upload the image:

```
$ openstack image create my-whole-disk-image --public \
  --disk-format qcow2 --container-format bare \
  --file my-whole-disk-image.qcow2
```

Warning

The kernel/ramdisk pair must not be set for whole disk images, otherwise theyll be mistaken for partition images.

- For *partition images* to be used only with *local boot* (the default) the `img_type` property must be set:

```
$ openstack image create my-image --public \
  --disk-format qcow2 --container-format bare \
  --property img_type=partition --file my-image.qcow2
```

- For *partition images* to be used with both *local* and *network* boot:

  Add the kernel and ramdisk images to the Image service:

```
$ openstack image create my-kernel --public \
  --disk-format raw --container-format bare --file my-image.vmlinuz
```

  Store the image uuid obtained from the above step as `MY_VMLINUZ_UUID`.

```
$ openstack image create my-image.initrd --public \
  --disk-format raw --container-format bare --file my-image.initrd
```

  Store the image UUID obtained from the above step as `MY_INITRD_UUID`.

  Add the *my-image* to the Image service which is going to be the OS that the user is going to run. Also associate the above created images with this OS image. These two operations can be done by executing the following command:

```
$ openstack image create my-image --public \
  --disk-format qcow2 --container-format bare --property \
  kernel_id=$MY_VMLINUZ_UUID --property \
  ramdisk_id=$MY_INITRD_UUID --file my-image.qcow2
```

- For images used with the *ramdisk deploy interface* (e.g. when deploying ramdisk nodes through Nova), you can use either `kernel_id` / `ramdisk_id` properties (with `ironic_ramdisk_deploy=True` to distinguish from partition images) or a single `boot_iso_id` property pointing to an ISO in Glance. See *Booting a Ramdisk or an ISO* for details.

Deploy ramdisk images

1. Build or download the deploy images

   The deploy images are used initially for preparing the server (creating disk partitions) before the actual OS can be deployed.

   There are several methods to build or download deploy images, please read the *Building or downloading a deploy ramdisk image* section.

2. Add the deploy images to the Image service

   Add the deployment kernel and ramdisk images to the Image service:

   ```
   $ openstack image create deploy-vmlinuz --public \
     --disk-format raw --container-format bare \
     --file ironic-python-agent.vmlinuz
   ```

   Store the image UUID obtained from the above step as `DEPLOY_VMLINUZ_UUID` (or a different name when using the parameter specified by node architecture).

   ```
   $ openstack image create deploy-initrd --public \
     --disk-format raw --container-format bare \
     --file ironic-python-agent.initramfs
   ```

   Store the image UUID obtained from the above step as `DEPLOY_INITRD_UUID` (or a different name when using the parameter specified by node architecture).

3. Configure the Bare Metal service to use the produced images. It can be done per node as described in *Enrolling hardware with Ironic* or in the configuration file either using a dictionary to specify them by architecture (matching the nodes `cpu_arch` property) as follows:

   ```
   [conductor]
   deploy_kernel_by_arch = x86_64:<DEPLOY_VMLINUZ_X86_64_UUID>,aarch64:
   ↪<DEPLOY_VMLINUZ_AARCH64_UUID>
   deploy_ramdisk_by_arch = x86_64:<DEPLOY_INITRD_X86_64_UUID>,aarch64:
   ↪<DEPLOY_INITRD_AARCH64_UUID>
   ```

   or globally using the general configuration parameters:

   ```
   [conductor]
   deploy_kernel = <insert DEPLOY_VMLINUZ_UUID>
   deploy_ramdisk = <insert DEPLOY_INITRD_UUID>
   ```

   In the case when both general parameters and parameters specified by architecture are defined, the parameters specified by architecture take priority.

Create flavors for use with the Bare Metal service

Youll need to create a special bare metal flavor in the Compute service. The flavor is mapped to the bare metal node through the nodes `resource_class` field (available starting with Bare Metal API version 1.21). A flavor can request *exactly one* instance of a bare metal resource class.

Note that when creating the flavor, its useful to add the `RAM_MB` and `CPU` properties as a convenience to users, although they are not used for scheduling. The `DISK_GB` property is also not used for scheduling, but is still used to determine the root partition size.

1. Change these to match your hardware:

```
$ RAM_MB=1024
$ CPU=2
$ DISK_GB=100
```

2. Create the bare metal flavor by executing the following command:

```
$ openstack flavor create --ram $RAM_MB --vcpus $CPU --disk $DISK_GB \
  my-baremetal-flavor
```

> **Note**
>
> You can add `--id <id>` to specify an ID for the flavor.

See the docs on this command for other options that may be specified.

After creation, associate each flavor with one custom resource class. The name of a custom resource class that corresponds to a nodes resource class (in the Bare Metal service) is:

- the bare metal nodes resource class all upper-cased
- prefixed with `CUSTOM_`
- all punctuation replaced with an underscore

For example, if the resource class is named `baremetal-small`, associate the flavor with this custom resource class via:

```
$ openstack flavor set --property resources:CUSTOM_BAREMETAL_SMALL=1 my-
↪baremetal-flavor
```

Another set of flavor properties must be used to disable scheduling based on standard properties for a bare metal flavor:

```
$ openstack flavor set --property resources:VCPU=0 my-baremetal-flavor
$ openstack flavor set --property resources:MEMORY_MB=0 my-baremetal-flavor
$ openstack flavor set --property resources:DISK_GB=0 my-baremetal-flavor
```

Example

If you want to define a class of nodes called `baremetal.with-GPU`, start with tagging some nodes with it:

```
$ baremetal node set <node> --resource-class baremetal.with-GPU
```

> **Warning**
>
> It is possible to **add** a resource class to `active` nodes, but it is not possible to **replace** an existing resource class on them.

Then you can update your flavor to request the resource class instead of the standard properties:

```
$ openstack flavor set --property resources:CUSTOM_BAREMETAL_WITH_GPU=1 my-
↪baremetal-flavor
$ openstack flavor set --property resources:VCPU=0 my-baremetal-flavor
$ openstack flavor set --property resources:MEMORY_MB=0 my-baremetal-flavor
$ openstack flavor set --property resources:DISK_GB=0 my-baremetal-flavor
```

Note how `baremetal.with-GPU` in the nodes `resource_class` field becomes `CUSTOM_BAREMETAL_WITH_GPU` in the flavors properties.

Scheduling based on traits

The Compute service supports scheduling based on qualitative attributes using traits. Starting with Bare Metal REST API version 1.37, it is possible to assign a list of traits to each bare metal node. Traits assigned to a bare metal node will be assigned to the corresponding resource provider in the Compute service placement API.

When creating a flavor in the Compute service, required traits may be specified via flavor properties. The Compute service will then schedule instances only to bare metal nodes with all of the required traits.

Traits can be either standard or custom. Standard traits are listed in the os_traits library. Custom traits must meet the following requirements:

- prefixed with `CUSTOM_`
- contain only upper case characters A to Z, digits 0 to 9, or underscores
- no longer than 255 characters in length

A bare metal node can have a maximum of 50 traits.

Example

To add the standard trait `HW_CPU_X86_VMX` and a custom trait `CUSTOM_TRAIT1` to a node:

```
$ baremetal node add trait <node> CUSTOM_TRAIT1 HW_CPU_X86_VMX
```

Then, update the flavor to require these traits:

```
$ openstack flavor set --property trait:CUSTOM_TRAIT1=required my-baremetal-
↪flavor
$ openstack flavor set --property trait:HW_CPU_X86_VMX=required my-baremetal-
↪flavor
```

2.1.6 Set up the drivers for the Bare Metal service

Enabling drivers and hardware types

Introduction

The Bare Metal service delegates actual hardware management to **drivers**. *Drivers*, also called *hardware types*, consist of *hardware interfaces*: sets of functionality dealing with some aspect of bare metal provisioning in a vendor-specific way. There are generic **hardware types** (eg. `redfish` and `ipmi`), and vendor-specific one (eg. `ilo`).

Note

The terminologies *driver*, *dynamic driver*, and *hardware type* have the same meaning in the scope of Bare Metal service.

Enabling hardware types

Hardware types are enabled in the configuration file of the **ironic-conductor** service by setting the `enabled_hardware_types` configuration option, for example:

```
[DEFAULT]
enabled_hardware_types = ipmi,redfish
```

Due to the drivers dynamic nature, they also require configuring enabled hardware interfaces.

Note

All available hardware types and interfaces are listed in setup.cfg file in the source code tree.

Enabling hardware interfaces

There are several types of hardware interfaces:

bios
: manages configuration of the BIOS settings of a bare metal node. This interface is vendor-specific and can be enabled via the `enabled_bios_interfaces` option:

  ```
  [DEFAULT]
  enabled_hardware_types = <hardware_type_name>
  enabled_bios_interfaces = <bios_interface_name>
  ```

  See *BIOS Configuration* for details.

boot
: manages booting of both the deploy ramdisk and the user instances on the bare metal node. See *Boot interfaces* for details.

  Boot interface implementations are often vendor specific, and can be enabled via the `enabled_boot_interfaces` option:

  ```
  [DEFAULT]
  enabled_hardware_types = ipmi,ilo
  enabled_boot_interfaces = pxe,ilo-virtual-media
  ```

  Boot interfaces with `pxe` in their name require *Configuring Network Boot*. There are also a few hardware-specific boot interfaces - see *Drivers, Hardware Types, and Hardware Interfaces for Ironic* for their required configuration.

console
: manages access to the serial console of a bare metal node. See *Configuring Consoles* for details.

deploy
: defines how the image gets transferred to the target disk. See *Deploy Interfaces* for an explanation

of the difference between supported deploy interfaces.

The deploy interfaces can be enabled as follows:

```
[DEFAULT]
enabled_hardware_types = ipmi,redfish
enabled_deploy_interfaces = direct,ramdisk
```

Note

The `direct` deploy interface requires the Object Storage service or an HTTP service

inspect
implements fetching hardware information from nodes. Can be implemented out-of-band (via contacting the nodes BMC) or in-band (via booting a ramdisk on a node, mainly achieved by the agent interface).

```
[DEFAULT]
enabled_hardware_types = ipmi,ilo
enabled_inspect_interfaces = ilo,agent
```

See /admin/inspection for more details.

management
provides additional hardware management actions, like getting or setting boot devices. This interface is usually vendor-specific, and its name often matches the name of the hardware type (with `ipmitool` being a notable exception). For example:

```
[DEFAULT]
enabled_hardware_types = ipmi,redfish,ilo
enabled_management_interfaces = ipmitool,redfish,ilo
```

Using `ipmitool` requires *Configuring IPMI support*. See *Drivers, Hardware Types, and Hardware Interfaces for Ironic* for the required configuration of each driver.

network
connects/disconnects bare metal nodes to/from virtual networks. See *Configure tenant networks* for more details.

power
runs power actions on nodes. Similar to the management interface, it is usually vendor-specific, and its name often matches the name of the hardware type (with `ipmitool` being again an exception). For example:

```
[DEFAULT]
enabled_hardware_types = ipmi,redfish,ilo
enabled_power_interfaces = ipmitool,redfish,ilo
```

Using `ipmitool` requires *Configuring IPMI support*. See *Drivers, Hardware Types, and Hardware Interfaces for Ironic* for the required configuration of each driver.

raid
manages building and tearing down RAID on nodes. Similar to inspection, it can be implemented

either out-of-band or in-band (via `agent` implementation). See *RAID Configuration* for details. For example:

```
[DEFAULT]
enabled_hardware_types = ipmi,redfish,ilo
enabled_raid_interfaces = agent,no-raid
```

storage

manages the interaction with a remote storage subsystem, such as the Block Storage service, and helps facilitate booting from a remote volume. This interface ensures that volume target and connector information is updated during the lifetime of a deployed instance. See *Boot From Volume* for more details.

This interface defaults to a `noop` driver as it is considered an opt-in interface which requires additional configuration by the operator to be usable.

For example:

```
[DEFAULT]
enabled_hardware_types = ipmi
enabled_storage_interfaces = cinder,noop
```

vendor

is a place for vendor extensions to be exposed in API. See *Vendor Methods* for details.

```
[DEFAULT]
enabled_hardware_types = ipmi,redfish,ilo
enabled_vendor_interfaces = ipmitool,no-vendor
```

Here is a complete configuration example, enabling two generic protocols, IPMI and Redfish, with a few additional features:

```
[DEFAULT]
enabled_hardware_types = ipmi,redfish
enabled_boot_interfaces = pxe
enabled_console_interfaces = ipmitool-socat,no-console
enabled_deploy_interfaces = direct
enabled_inspect_interfaces = agent
enabled_management_interfaces = ipmitool,redfish
enabled_network_interfaces = flat,neutron
enabled_power_interfaces = ipmitool,redfish
enabled_raid_interfaces = agent
enabled_storage_interfaces = cinder,noop
enabled_vendor_interfaces = ipmitool,no-vendor
```

Note that some interfaces have implementations named `no-<TYPE>` where `<TYPE>` is the interface type. These implementations do nothing and return errors when used from API.

Hardware interfaces in multi-conductor environments

When enabling hardware types and their interfaces, make sure that for every enabled hardware type, the whole set of enabled interfaces matches for all conductors. However, different conductors can have different hardware types enabled.

For example, you can have two conductors with the following configuration respectively:

```
[DEFAULT]
enabled_hardware_types = ipmi
enabled_deploy_interfaces = direct
enabled_power_interfaces = ipmitool
enabled_management_interfaces = ipmitool
```

```
[DEFAULT]
enabled_hardware_types = redfish
enabled_deploy_interfaces = ansible
enabled_power_interfaces = redfish
enabled_management_interfaces = redfish
```

But you cannot have two conductors with the following configuration respectively:

```
[DEFAULT]
enabled_hardware_types = ipmi,redfish
enabled_deploy_interfaces = direct
enabled_power_interfaces = ipmitool,redfish
enabled_management_interfaces = ipmitool,redfish
```

```
[DEFAULT]
enabled_hardware_types = redfish
enabled_deploy_interfaces = ansible
enabled_power_interfaces = redfish
enabled_management_interfaces = redfish
```

This is because the `redfish` hardware type will have different enabled *deploy* interfaces on these conductors. It would have been fine, if the second conductor had `enabled_deploy_interfaces = direct` instead of `ansible`.

This situation is not detected by the Bare Metal service, but it can cause inconsistent behavior in the API, when node functionality will depend on which conductor it gets assigned to.

Note

We dont treat this as an error, because such *temporary* inconsistency is inevitable during a rolling upgrade or a configuration update.

Configuring interface defaults

When an operator does not provide an explicit value for one of the interfaces (when creating a node or updating its driver), the default value is calculated as described in *Defaults for hardware interfaces*. It is also possible to override the defaults for any interfaces by setting one of the options named `default_<IFACE>_interface`, where `<IFACE>` is the interface name. For example:

```
[DEFAULT]
default_deploy_interface = direct
default_network_interface = neutron
```

This configuration forces the default *deploy* interface to be `direct` and the default *network* interface to be `neutron` for all hardware types.

The defaults are calculated and set on a node when creating it or updating its hardware type. Thus, changing these configuration options has no effect on existing nodes.

Warning

The default interface implementation must be configured the same way across all conductors in the cloud, except maybe for a short period of time during an upgrade or configuration update. Otherwise the default implementation will depend on which conductor handles which node, and this mapping is not predictable or even persistent.

Warning

These options should be used with care. If a hardware type does not support the provided default implementation, its users will have to always provide an explicit value for this interface when creating a node.

Configuring Network Boot

Ironics primary means of booting hardware to perform actions or work on a baremetal node is to perform network booting. Traditionally, this has meant the use of Preboot Execution Environment, or PXE. This support and functionality has evolve as time has gone on to include support for not just the `pxe` `boot_interface` in concern with hardware vendor specific variations, but also a distinct `ipxe` setting for `boot_interface` with default values to enable use of iPXE.

As time passed, `http` and `http-ipxe` values were also added as valid `boot_interface` options which may be used, which are functionally identical in behavior to `pxe` and `ipxe`, except HTTP is used as the transport mechanism. Not all hardware supports HTTPBoot, as it is often referred.

Note

Support for HTTPBoot interfaces was added during the 2024.1 development cycle. Prior versions of Ironic does not contain the `http` and `http-ipxe` boot interfaces.

DHCP server setup

A DHCP server is required for network boot clients. You need to follow steps below.

1. Set the `[dhcp]/dhcp_provider` to `neutron` in the Bare Metal Services configuration file (`/etc/ironic/ironic.conf`):

   Note

   Refer *Configure tenant networks* for details. The `dhcp_provider` configuration is already set by the configuration defaults, and when you create subnet, DHCP is also enabled if you do not add any dhcp options at openstack subnet create command.

2. Enable DHCP in the subnet of provisioning network to be used for network boot (PXE, iPXE, HTTPBoot) operations.
3. Set the ip address range in the subnet for DHCP.

> **Note**
>
> Refer *Configure the Networking service for bare metal provisioning* for details about the two precedent steps.

4. Connect the openstack DHCP agent to the external network through the OVS bridges and the interface `eth2`.

> **Note**
>
> Refer *Configure the Networking service for bare metal provisioning* for details. You do not require this part if br-int, br-eth2 and eth2 are already connected.

5. Configure the host ip at `br-eth2`. If it locates at `eth2`, do below:

```
ip addr del 192.168.2.10/24 dev eth2
ip addr add 192.168.2.10/24 dev br-eth2
```

> **Note**
>
> Replace eth2 with the interface on the network node which you are using to connect to the Bare Metal service.

TFTP Server Setup

In order to deploy instances via PXE, a TFTP server needs to be set up on the Bare Metal service nodes which run the `ironic-conductor`.

Debian or Ubuntu

In Debian or Ubuntu, xinetd can be used to run tftp server service.

1. Make sure the tftp root directory exists and can be written to by the user the `ironic-conductor` is running as. For example:

```
sudo mkdir -p /tftpboot
sudo chown -R ironic /tftpboot
```

2. Install tftp server:

```
sudo apt-get install xinetd tftpd-hpa
```

3. Using xinetd to provide a tftp server setup to serve `/tftpboot`. Create or edit `/etc/xinetd.d/tftp` as below:

```
service tftp
{
  protocol        = udp
  port            = 69
  socket_type     = dgram
  wait            = yes
  user            = root
  server          = /usr/sbin/in.tftpd
  server_args     = -v -v -v -v -v --map-file /tftpboot/map-file /tftpboot
  disable         = no
  # This is a workaround for Fedora, where TFTP will listen only on
  # IPv6 endpoint, if IPv4 flag is not used.
  flags           = IPv4
}
```

and restart the `xinetd` service:

```
sudo systemctl restart xinetd
```

Note

In certain environments the networks MTU may cause TFTP UDP packets to get fragmented. Certain PXE firmwares struggle to reconstruct the fragmented packets which can cause significant slow down or even prevent the server from PXE booting. In order to avoid this, TFTPd provides an option to limit the packet size so that it they do not get fragmented. To set this additional option in the server_args above:

```
--blocksize <MAX MTU minus 32>
```

4. Create a map file in the tftp boot directory (`/tftpboot`):

```
echo 're ^(/tftpboot/) /tftpboot/\2' > /tftpboot/map-file
echo 're ^/tftpboot/ /tftpboot/' >> /tftpboot/map-file
echo 're ^(^/) /tftpboot/\1' >> /tftpboot/map-file
echo 're ^([^/]) /tftpboot/\1' >> /tftpboot/map-file
```

RHEL or CentOS

Note

Starting with CentOS 9 and RHEL 9, xinetd is not available. Use systemd socket activation instead. See the DevStack TFTP setup for an example configuration.

For CentOS 8 and earlier, or RHEL 8 and earlier, use a dedicated dnsmasq instance to run the TFTP server service.

1. Make sure the tftp root directory exists and can be written to by the user the `ironic-conductor` is running as. For example:

```
sudo mkdir -p /tftpboot
sudo chown -R ironic /tftpboot
```

2. Install tftp server:

```
sudo dnf install openstack-ironic-dnsmasq-tftp-server
```

3. Using dnsmasq to provide a tftp server setup to serve `/tftpboot`. Edit `/etc/ironic/dnsmasq-tftp-server.conf` as below:

```
port=0
bind-interfaces
enable-tftp
tftp-root=/tftpboot
```

and restart the `openstack-ironic-dnsmasq-tftp-server` service:

```
sudo systemctl restart openstack-ironic-dnsmasq-tftp-server
```

4. Create a map file in the tftp boot directory (`/tftpboot`):

```
echo 're ^(/tftpboot/) /tftpboot/\2' > /tftpboot/map-file
echo 're ^/tftpboot/ /tftpboot/' >> /tftpboot/map-file
echo 're ^(^/) /tftpboot/\1' >> /tftpboot/map-file
echo 're ^([^/]) /tftpboot/\1' >> /tftpboot/map-file
```

UEFI PXE - Grub setup

In order to deploy instances with PXE on bare metal nodes which support UEFI, perform these additional steps on the ironic conductor node to configure the PXE UEFI environment.

Note

Most commercial Linux distributions have signed shim and grub binaries, which are required for Secure Boot.

1. Install Grub2 and shim packages:

Ubuntu (18.04LTS and later):

```
sudo apt-get install grub-efi-amd64-signed shim-signed
```

RHEL/CentOS/Fedora:

```
sudo dnf install grub2-efi-x64 shim
```

2. Copy grub and shim boot loader images to `/tftpboot` directory:

Ubuntu (18.04LTS and later):

```
sudo cp /usr/lib/shim/shimx64.efi.signed /tftpboot/bootx64.efi
sudo cp /usr/lib/grub/x86_64-efi-signed/grubnetx64.efi.signed /tftpboot/
↪grubx64.efi
```

Fedora:

```
sudo cp /boot/efi/EFI/fedora/shim.efi /tftpboot/bootx64.efi
sudo cp /boot/efi/EFI/fedora/grubx64.efi /tftpboot/grubx64.efi
```

CentOS:

```
sudo cp /boot/efi/EFI/centos/shim.efi /tftpboot/bootx64.efi
sudo cp /boot/efi/EFI/centos/grubx64.efi /tftpboot/grubx64.efi
```

RHEL:

```
sudo cp /boot/efi/EFI/rhel/shim.efi /tftpboot/bootx64.efi
sudo cp /boot/efi/EFI/rhel/grubx64.efi /tftpboot/grubx64.efi
```

3. Update the bare metal node with `boot_mode:uefi` capability in nodes properties field. See *Boot mode support* for details.
4. Make sure that bare metal node is configured to boot in UEFI boot mode and boot device is set to network/pxe.

> **Note**
>
> Some drivers, e.g. `ilo` and `redfish`, support automatic setting of the boot mode during deployment. This step is not required for them. Please check *Drivers, Hardware Types, and Hardware Interfaces for Ironic* for information on whether your driver requires manual UEFI configuration.

iPXE setup

If you will be using iPXE to boot instead of PXE, iPXE needs to be set up on the Bare Metal service node(s) where `ironic-conductor` is running.

1. Make sure these directories exist and can be written to by the user the `ironic-conductor` is running as. For example:

```
sudo mkdir -p /tftpboot
sudo mkdir -p /httpboot
sudo chown -R ironic /tftpboot
sudo chown -R ironic /httpboot
```

2. Create a map file in the tftp boot directory (`/tftpboot`):

```
echo 'r ^([^/]) /tftpboot/\1' > /tftpboot/map-file
echo 'r ^(/tftpboot/) /tftpboot/\2' >> /tftpboot/map-file
```

3. Set up TFTP and HTTP servers.

   These servers should be running and configured to use the local /tftpboot and /httpboot directories respectively, as their root directories. (Setting up these servers is outside the scope of this install guide.)

   These root directories need to be mounted locally to the `ironic-conductor` services, so that the services can access them.

The Bare Metal services configuration file (/etc/ironic/ironic.conf) should be edited accordingly to specify the TFTP and HTTP root directories and server addresses. For example:

```
[pxe]

# Ironic compute node's tftp root path. (string value)
tftp_root=/tftpboot

# IP address of Ironic compute node's tftp server. (string
# value)
tftp_server=192.168.0.2

[deploy]
# Ironic compute node's http root path. (string value)
http_root=/httpboot

# Ironic compute node's HTTP server URL. Example:
# http://192.1.2.3:8080 (string value)
http_url=http://192.168.0.2:8080
```

See also: *Deploying outside of the provisioning network*.

4. Install the iPXE package with the boot images:

   Ubuntu:

   ```
   apt-get install ipxe
   ```

   RHEL/CentOS/Fedora:

   ```
   dnf install ipxe-snponly-x86_64
   ```

Note

Some distributions have chosen to minimize iPXE packaging to the `snponly` build of the iPXE binaries. This is because the drivers provided in the legacy `ipxe.efi` binary are geared for BIOS booting machines and can create an unrealiable experience. Furtheremore, if the `snponly` version does not work on a particular piece of hardware, the fault likely lies with the UEFI firmware on the host or device itself. An alternative may be Grub2, however Grub2 is not designed around network boot as its first class use case.

Note

The Ironic project is unaware of any vendor signed iPXE binaries to enable use of iPXE with Secure Boot, unless you have implemented your own Secure Boot key signing and support for the Machine Owner Key settings on individual baremetal nodes.

1. Copy the iPXE boot image (`undionly.kpxe` for **BIOS** and `ipxe.efi` for **UEFI**) to `/tftpboot`. The binary might be found at:

   Ubuntu:

```
cp /usr/lib/ipxe/{undionly.kpxe,ipxe.efi,snponly.efi} /tftpboot
```

Fedora/RHEL/CentOS:

```
cp /usr/share/ipxe/{undionly.kpxe,ipxe-snponly-x86_64.efi} /tftpboot
```

Note

`snponly.efi` may not be available for all distributions, and similarly not all distributions may have `ipxe.efi`

2. Enable/Configure iPXE overrides in the Bare Metal Services configuration file **if required** (/etc/ironic/ironic.conf):

```
[pxe]

# Neutron bootfile DHCP parameter. (string value)
ipxe_bootfile_name=undionly.kpxe

# Bootfile DHCP parameter for UEFI boot mode. (string value)
uefi_ipxe_bootfile_name=ipxe.efi

# Template file for PXE configuration. (string value)
ipxe_config_template=$pybasedir/drivers/modules/ipxe_config.template
```

Note

Most UEFI systems have integrated networking which means the *pxe.uefi_ipxe_bootfile_name* setting should be set to `snponly.efi` or `ipxe-snponly-x86_64.efi` if its available for your distribution.

Note

Setting the iPXE parameters noted in the code block above to no value, in other words setting a line to something like `ipxe_bootfile_name=` will result in ironic falling back to the default values of the non-iPXE PXE settings. This is for backwards compatibility.

3. Ensure iPXE is the default PXE, if applicable.

   In earlier versions of ironic, a now deprecated and removed `[pxe]ipxe_enabled` setting allowed operators to declare the behavior of the conductor to exclusively operate as if only iPXE was to be used. As time moved on, iPXE functionality was moved to its own `ipxe` boot interface.

   If you want to emulate that same behavior, set the following in the configuration file (/etc/ironic/ironic.conf):

```
[DEFAULT]
default_boot_interface=ipxe
```

(continues on next page)

(continued from previous page)

```
enabled_boot_interfaces=ipxe,pxe
```

Note

The *DEFAULT.enabled_boot_interfaces* setting may be exclusively set to `ipxe`, however ironic has multiple interfaces available depending on the hardware types available for use.

4. It is possible to configure the Bare Metal service in such a way that nodes will boot into the deploy image directly from Object Storage. Doing this avoids having to cache the images on the ironic-conductor host and serving them via the ironic-conductors *HTTP server*. This can be done if:

   1. the Image Service is used for image storage;
   2. the images in the Image Service are internally stored in Object Storage;
   3. the Object Storage supports generating temporary URLs for accessing objects stored in it. Both the OpenStack Swift and RADOS Gateway provide support for this.

      - See *Ceph Object Gateway support* on how to configure the Bare Metal Service with RADOS Gateway as the Object Storage.

   Configure this by setting the `[pxe]/ipxe_use_swift` configuration option to `True` as follows:

```
[pxe]

# Download deploy images directly from swift using temporary
# URLs. If set to false (default), images are downloaded to
# the ironic-conductor node and served over its local HTTP
# server. Applicable only when 'ipxe_enabled' option is set to
# true. (boolean value)
ipxe_use_swift=True
```

   Although the *HTTP server* still has to be deployed and configured (as it will serve iPXE boot script and boot configuration files for nodes), such configuration will shift some load from ironic-conductor hosts to the Object Storage service which can be scaled horizontally.

   Note that when SSL is enabled on the Object Storage service you have to ensure that iPXE firmware on the nodes can indeed boot from generated temporary URLs that use HTTPS protocol.

5. Restart the `ironic-conductor` process:

   Fedora/RHEL/CentOS:

```
sudo systemctl restart openstack-ironic-conductor
```

   Ubuntu:

```
sudo service ironic-conductor restart
```

PXE multi-architecture setup

It is possible to deploy servers of different architecture by one conductor. To use this feature, architecture-specific boot and template files must be configured using the configuration options *pxe.pxe_bootfile_name_by_arch* (or *pxe.ipxe_bootfile_name_by_arch* for iPXE-based deployments) and *pxe.pxe_config_template_by_arch* respectively, in the Bare Metal services configuration file (/etc/ironic/ironic.conf).

The *_by_arch configuration options are dictionaries that map architecture names to bootfiles and templates. The key is the architecture and the value is the boot (or config template) file.

A nodes cpu_arch property is used as the key to get the appropriate boot file and template file. If the nodes cpu_arch is not in the dictionary, the configuration options (in [pxe] group) pxe_bootfile_name, pxe_config_template, uefi_pxe_bootfile_name and uefi_pxe_config_template will be used instead.

In the following example, since x86 and x86_64 keys are not in the pxe_bootfile_name_by_arch or pxe_config_template_by_arch options, x86 and x86_64 nodes will be deployed by undionly.kpxe or bootx64.efi, depending on the nodes boot_mode capability (bios or uefi). However, aarch64 nodes will be deployed by grubaa64.efi, and ppc64 nodes by bootppc64:

```
[pxe]

# Bootfile DHCP parameter. (string value)
pxe_bootfile_name=undionly.kpxe

# On ironic-conductor node, template file for PXE
# configuration. (string value)
pxe_config_template = $pybasedir/drivers/modules/ipxe_config.template

# Bootfile DHCP parameter for UEFI boot mode. (string value)
uefi_pxe_bootfile_name=bootx64.efi

# On ironic-conductor node, template file for PXE
# configuration for UEFI boot loader. (string value)
uefi_pxe_config_template=$pybasedir/drivers/modules/pxe_grub_config.template

# Bootfile DHCP parameter per node architecture. (dict value)
pxe_bootfile_name_by_arch=aarch64:grubaa64.efi,ppc64:bootppc64

# On ironic-conductor node, template file for PXE
# configuration per node architecture. For example:
# aarch64:/opt/share/grubaa64_pxe_config.template (dict value)
pxe_config_template_by_arch=aarch64:pxe_grubaa64_config.template,ppc64:pxe_
↪ppc64_config.template
```

Note

The grub implementation may vary on different architecture, you may need to tweak the pxe config template for a specific arch. For example, grubaa64.efi shipped with CentoOS7 does not support linuxefi and initrdefi commands, youll need to switch to use linux and initrd command instead.

Note

A *pxe.ipxe_bootfile_name_by_arch* setting is available for multi-arch iPXE-based deployment, and defaults to the same behavior as the comperable *pxe.pxe_bootfile_name_by_arch* setting for standard PXE. For example:

```
[pxe]

ipxe_bootfile_name_by_arch = aarch64:ipxe-aa64.efi,x86_64:ipxe.efi
```

Note

When booting PowerPC based machines, the firmware loader directly boots a kernel and ramdisk. It explicitly reads a pxelinux style template, and then directly retrieves the files defined in the file without a network boot program.

PXE timeouts tuning

Because of its reliance on UDP-based protocols (DHCP and TFTP), PXE is particularly vulnerable to random failures during the booting stage. If the deployment ramdisk never calls back to the bare metal conductor, the build will be aborted, and the node will be moved to the `deploy failed` state, after the deploy callback timeout. This timeout can be changed via the *conductor.deploy_callback_timeout* configuration option.

The Bare Metal service can retry PXE boot if it takes too long. The timeout is defined via *pxe.boot_retry_timeout* and must be smaller than the `deploy_callback_timeout`, otherwise it will have no effect.

For example, the following configuration sets the overall timeout to 60 minutes, allowing two retries after 20 minutes:

```
[conductor]
deploy_callback_timeout = 3600

[pxe]
boot_retry_timeout = 1200
```

PXE artifacts

Ironic features the capability to load PXE artifacts into the conductor startup, minimizing the need for external installation and configuration management tooling from having to do additional work to facilitate.

While this is an advanced feature, and destination file names must match existing bootloader configured filenames.

For example, if using iPXE and GRUB across interfaces, you may desire a configuration similar to this example.

```
[pxe]
loader_file_paths = ipxe.efi:/usr/share/ipxe/ipxe-snponly-x86_64.efi,undionly.
```

(continues on next page)

(continued from previous page)

```
↪kpxe:/usr/share/ipxe/undionly.kpxe,bootx64.efi,/boot/efi/EFI/boot/grubx64.
↪efi,bootx64.efi:/boot/efi/EFI/boot/BOOTX64.EFI
```

If you choose to use relative paths as part of your destination, those paths will be created using configuration parameter *pxe.dir_permission* where as actual files copied are set with the configuration parameter *pxe.file_permission*. Absolute destination paths are not supported and will result in ironic failing to start up as it is a misconfiguration of the deployment.

Configuring unmanaged in-band inspection

With PXE

After you followed *TFTP Server Setup*, you need to create the default PXE configuration. Populate `/tftpboot/pxelinux.cfg/default` with the following contents:

```
default introspect

label introspect
kernel ironic-python-agent.kernel
append initrd=ironic-python-agent.initramfs ipa-inspection-callback-url=http:/
↪/{IP}:6385/v1/continue_inspection systemd.journald.forward_to_console=yes

ipappend 3
```

Instead of `http://{IP}:6385/v1/continue_inspection`, insert the correct Bare Metal API endpoint, keeping the mandatory `/v1/continue_inspection` suffix. You may also populate other IPA options (e.g. `ipa-debug=1` for detailed logging, `ipa-inspection-collectors` to customize the inspection process, or `ipa-api-url` to enable *Fast-Track Deployment*).

Second, you need to configure DHCP for unknown hosts since the OpenStack Networking service wont be able to handle them. For instance, you can install **dnsmasq** and use the following `/etc/dnsmasq.conf`:

```
port=0
interface={INTERFACE}
bind-interfaces
dhcp-range={DHCP IP RANGE, e.g. 192.168.0.50,192.168.0.150}
enable-tftp
tftp-root=/tftpboot
dhcp-boot=pxelinux.0
dhcp-sequential-ip
```

If you need this dnsmasq instance to co-exist with the OpenStack Networking service, some measures must be taken to prevent them from clashing over DHCP requests. One way to do it is to physically separate the inspection network. Another - to configure the *PXE filter service*.

Finally, build or download IPA images into `/tftpboot/ironic-python-agent.kernel` and `/tftpboot/ironic-python-agent.initramfs`. These can be the same images that you use for deployment and cleaning.

With iPXE

iPXE configuration is pretty similar to PXE above, but differs in details. Start with *iPXE Setup*, then create a new file `/httpboot/inspection.ipxe` with the following contents:

```
#!ipxe

:retry_dhcp
dhcp || goto retry_dhcp

:retry_boot
imgfree
kernel --timeout 30000 http://{IP}:8080/ironic-python-agent.kernel ipa-
↪inspection-callback-url=http://{IP}:6385/v1/continue_inspection systemd.
↪journald.forward_to_console=yes BOOTIF=${mac} initrd=ironic-python-agent.
↪initramfs || goto retry_boot
initrd --timeout 30000 http://{IP}:8080/ironic-python-agent.initramfs || goto␣
↪retry_boot
boot
```

Just as *with PXE*, adjust `ipa-inspection-callback-url` to match your deployment and add any required IPA options. You also need to fix `{IP}:8080` to match the iPXE server you configured previously.

The DHCP configuration is much more complex. Since most hardware does not have an up-to-date iPXE firmware, you need to bootstrap it from TFTP. The **dnsmasq** configuration may look roughly like this:

```
port=0
interface={INTERFACE}
bind-interfaces
dhcp-range={DHCP IP RANGE, e.g. 192.168.0.50,192.168.0.150}
enable-tftp
tftp-root=/tftpboot
dhcp-sequential-ip
dhcp-match=ipxe,175
dhcp-match=set:efi,option:client-arch,7
dhcp-match=set:efi,option:client-arch,9
dhcp-match=set:efi,option:client-arch,11
# dhcpv6.option: Client System Architecture Type (61)
dhcp-match=set:efi6,option6:61,0007
dhcp-match=set:efi6,option6:61,0009
dhcp-match=set:efi6,option6:61,0011
dhcp-userclass=set:ipxe6,iPXE
# Client is already running iPXE; move to next stage of chainloading
dhcp-boot=tag:ipxe,http://{IP}:8080/inspection.ipxe
# Client is PXE booting over EFI without iPXE ROM,
# send EFI version of iPXE chainloader
dhcp-boot=tag:efi,tag:!ipxe,ipxe.efi
dhcp-option=tag:efi6,tag:!ipxe6,option6:bootfile-url,tftp://{IP}/ipxe.efi
# Client is running PXE over BIOS; send BIOS version of iPXE chainloader
dhcp-boot=undionly.kpxe,localhost.localdomain,{IP}
```

Note

Its not trivial to write such a configuration from scratch. In addition to this document, you may take some inspiration from Bifrost and Metal3.

Finally, put `ironic-python-agent.kernel` and `ironic-python-agent.initramfs` to `/httpboot`.

HTTPBoot

HTTPBoot interfaces in Ironic are built upon the underlying network boot substrate. This means much of the configuration in the `[pxe]` and `[deploy]` impacts the use of HTTPBoot, except when Ironic is setting DHCP parameters, it populates a HTTP(S) URL to the DHCP server, which is then transmitted to the client attempting to Network Boot. In large part, this is because HTTPBoot is an evolution of PXE Boot technique and technology.

This means a TFTP server is *not* required, but the HTTP server is required as if you are utilizing iPXE. This is largely because iPXE has traditionally been leveraged by Operators to limit the TFTP packets being transmitted via UDP across a network.

One aspect to keep in mind, is HTTPBoot is relatively new when compared to PXE boot, and not all bootloaders may support HTTPBoot, as the underlying UEFI standard upon which it was largely based, UEFI v2.5, was published in 2015.

Ironic contains two distinct flavors of HTTPBoot, largely based upon what configuration defaults are used in terms of boot loader, templates, and overall mechanism style.

- `http` is the boot interface based upon the `pxe` boot interface. This is the interface you would want to use if you had, for example, a signed GRUB2 bootloader chain to utilize. In this case it is up to the boot loader to understand how to extract and run with the URL, and then retrieves any additional configuration loader files and configuration templates created on disk.
- `http-ipxe` is the boot interface based upon the `ipxe` boot interface. This interface signals to the client to utilize the configured iPXE loader binary over HTTP, and then the boot sequence proceeds with the pattern and capabilities of iPXE.

To enable the boot interfaces, you will need to add them to your *`DEFAULT.enabled_boot_interfaces`* configuration entry.

```
[DEFAULT]
enabled_boot_interfaces=ipxe,http-ipxe,pxe,http
```

Configuring IPMI support

Installing ipmitool command

To enable one of the drivers that use IPMI protocol for power and management actions (for example, `ipmi`), the `ipmitool` command must be present on the service node(s) where `ironic-conductor` is running. On most distros, it is provided as part of the `ipmitool` package. Source code is available at http://ipmitool.sourceforge.net/.

Warning

Certain distros, notably Mac OS X and SLES, install `openipmi` instead of `ipmitool` by default. This driver is not compatible with `openipmi` as it relies on error handling options not provided by this tool.

Please refer to the *IPMI driver* for information on how to configure and use IPMItool-based drivers.

Configuring hardware

IPMI is a relatively old protocol and may require additional set up on the hardware side that the Bare Metal service cannot do automatically:

1. Make sure IPMI is enabled and the account you use have the permissions to change power and boot devices. By default the administrator rights are expected, you can change it: see *Using a different privilege level*.
2. Make sure the cipher suites are configured for maximum security. Suite 17 is recommended, 3 can be used if its not available. Cipher suite 0 **must** be disabled as it provides unauthenticated access to the BMC.

   See also

   Cipher suites

3. Make sure the boot mode correspond to the expected boot mode on the node (see *Boot mode support*). Some hardware is able to change the boot mode to the requested by Ironic, some does not.

Validation and troubleshooting

Check that you can connect to, and authenticate with, the IPMI controller in your bare metal server by running `ipmitool`:

```
ipmitool -I lanplus -H <ip-address> -U <username> -P <password> chassis power␣
↪status
```

where `<ip-address>` is the IP of the IPMI controller you want to access. This is not the bare metal nodes main IP. The IPMI controller should have its own unique IP.

If the above command doesnt return the power status of the bare metal server, check that

- `ipmitool` is installed and is available via the `$PATH` environment variable.
- The IPMI controller on your bare metal server is turned on.
- The IPMI controller credentials and IP address passed in the command are correct.
- The conductor node has a route to the IPMI controller. This can be checked by just pinging the IPMI controller IP from the conductor node.

IPMI configuration

If there are slow or unresponsive BMCs in the environment, the `min_command_interval` configuration option in the `[ipmi]` section may need to be raised. The default is fairly conservative, as setting this timeout too low can cause older BMCs to crash and require a hard-reset.

Collecting sensor data

Bare Metal service supports sending IPMI sensor data to Telemetry with certain hardware types, such as `ipmi` and `ilo`. By default, support for sending IPMI sensor data to Telemetry is disabled. If you want to enable it, you should make the following two changes in `ironic.conf`:

```
[conductor]
send_sensor_data = true
[oslo_messaging_notifications]
driver = messagingv2
```

If you want to customize the sensor types which will be sent to Telemetry, change the `send_sensor_data_types` option. For example, the below settings will send information about temperature, fan, voltage from sensors to the Telemetry service:

```
send_sensor_data_types=Temperature,Fan,Voltage
```

Supported sensor types are defined by the Telemetry service, currently these are `Temperature`, `Fan`, `Voltage`, `Current`. Special value `All` (the default) designates all supported sensor types.

Configuring an ESP image

An ESP image is an image that contains the necessary bootloader to boot the ISO in UEFI mode. You will need a GRUB2 image file, as well as Shim for secure boot. See *UEFI PXE - Grub setup* for an explanation how to get them.

Then the following script can be used to build an ESP image:

```
DEST=/path/to/esp.img
GRUB2=/path/to/grub.efi
SHIM=/path/to/shim.efi

dd if=/dev/zero of=$DEST bs=4096 count=1024
mkfs.msdos -F 12 -n ESP_IMAGE $DEST

# The following commands require mtools to be installed
mmd -i $DEST EFI EFI/BOOT
mcopy -i $DEST -v $SHIM ::EFI/BOOT/BOOTX64.efi
mcopy -i $DEST -v $GRUB2 ::EFI/BOOT/GRUBX64.efi
mdir -i $DEST ::EFI/BOOT
```

Note

If you use an architecture other than x86-64, youll need to adjust the destination paths.

Warning

If you are using secure boot, you *must* utilize the same SHIM and GRUB binaries matching your distributions kernel and ramdisk, otherwise the Secure Boot chain of trust will be broken. Additionally, if you encounter odd issues UEFI booting with virtual media which point to the bootloader, verify the appropriate distribution matching binaries are in use.

The resulting image should be provided via the `driver_info/bootloader` ironic node property in form of an image UUID or a URL:

```
baremetal node set --driver-info bootloader=<glance-uuid-or-url> node-0
```

Alternatively, set the bootloader UUID or URL in the configuration file:

```
[conductor]
bootloader = <glance-uuid-or-url>
```

Finally, you need to provide the correct GRUB2 configuration path for your image. In most cases this path will depend on your distribution, more precisely, the distribution you took the GRUB2 image from. For example:

CentOS:

```
[DEFAULT]
grub_config_path = EFI/centos/grub.cfg
```

Ubuntu:

```
[DEFAULT]
grub_config_path = EFI/ubuntu/grub.cfg
```

Note

Unlike in the script above, these paths are case-sensitive!

2.1.7 Enrolling hardware with Ironic

After all the services have been properly configured, you should enroll your hardware with the Bare Metal service, and confirm that the Compute service sees the available hardware. The nodes will be visible to the Compute service once they are in the `available` provision state.

Note

After enrolling nodes with the Bare Metal service, the Compute service will not be immediately notified of the new resources. The Compute services resource tracker syncs periodically, and so any changes made directly to the Bare Metal services resources will become visible in the Compute service only after the next run of that periodic task. More information is in the *Troubleshooting* section.

Note

Any bare metal node that is visible to the Compute service may have a workload scheduled to it, if both the `power` and `management` interfaces pass the `validate` check. If you wish to exclude a node from the Compute services scheduler, for instance so that you can perform maintenance on it, you can set the node to maintenance mode. For more information see the *Maintenance mode* section.

Choosing a driver

When enrolling a node, the most important information to supply is *driver*. See *Enabling drivers and hardware types* for a detailed explanation of bare metal drivers, hardware types and interfaces. The `driver list` command can be used to list all drivers enabled on all hosts:

```
baremetal driver list
+---------------------+------------------------+
| Supported driver(s) | Active host(s)         |
+---------------------+------------------------+
| ipmi                | localhost.localdomain  |
+---------------------+------------------------+
```

The specific driver to use should be picked based on actual hardware capabilities and expected features. See *Drivers, Hardware Types, and Hardware Interfaces for Ironic* for more hints on that.

Each driver has a list of *driver properties* that need to be specified via the nodes `driver_info` field, in order for the driver to operate on node. This list consists of the properties of the hardware interfaces that the driver uses. These driver properties are available with the `driver property list` command:

```
$ baremetal driver property list ipmi
+----------------------+-------------------------------------------------------
↪-----------------------------------------------------+
| Property             | Description                                           ␣
↪                                                     |
+----------------------+-------------------------------------------------------
↪-----------------------------------------------------+
| ipmi_address         | IP address or hostname of the node. Required.         ␣
↪                                                     |
| ipmi_password        | password. Optional.                                   ␣
↪                                                     |
| ipmi_username        | username; default is NULL user. Optional.             ␣
↪                                                     |
| ...                  | ...                                                   ␣
↪                                                     |
| deploy_kernel        | UUID (from Glance) of the deployment kernel.␣
↪Required.                                                          |
| deploy_ramdisk       | UUID (from Glance) of the ramdisk that is mounted at␣
↪boot time. Required.                                 |
+----------------------+-------------------------------------------------------
↪-----------------------------------------------------+
```

The properties marked as required must be supplied either during node creation or shortly after. Some properties may only be required for certain features.

Note on API versions

Starting with API version 1.11, the Bare Metal service added a new initial provision state of `enroll` to its state machine. When this or later API version is used, new nodes get this state instead of `available`.

Existing automation tooling that use an API version lower than 1.11 are not affected, since the initial provision state is still `available`. However, using API version 1.11 or above may break existing automation tooling with respect to node creation.

The `openstack baremetal` command line tool tries to negotiate the most recent supported version, which in virtually all cases will be newer than 1.11.

To set the API version for all commands, you can set the environment variable `IRONIC_API_VERSION`. For the OpenStackClient baremetal plugin, set the `OS_BAREMETAL_API_VERSION` variable to the same value. For example:

```
$ export IRONIC_API_VERSION=1.11
$ export OS_BAREMETAL_API_VERSION=1.11
```

Enrollment process

Creating a node

This section describes the main steps to enroll a node and make it available for provisioning. Some steps are shown separately for illustration purposes, and may be combined if desired.

1. Create a node in the Bare Metal service with the `node create` command. At a minimum, you must specify the driver name (for example, `ipmi`).

   This command returns the node UUID along with other information about the node. The nodes provision state will be `enroll`:

```
$ baremetal node create --driver ipmi
+--------------+--------------------------------------+
| Property     | Value                                |
+--------------+--------------------------------------+
| uuid         | dfc6189f-ad83-4261-9bda-b27258eb1987 |
| driver_info  | {}                                   |
| extra        | {}                                   |
| driver       | ipmi                                 |
| chassis_uuid |                                      |
| properties   | {}                                   |
| name         | None                                 |
+--------------+--------------------------------------+

$ baremetal node show dfc6189f-ad83-4261-9bda-b27258eb1987
+------------------------+--------------------------------------+
| Property               | Value                                |
+------------------------+--------------------------------------+
| target_power_state     | None                                 |
| extra                  | {}                                   |
| last_error             | None                                 |
| maintenance_reason     | None                                 |
| provision_state        | enroll                               |
```

(continues on next page)

(continued from previous page)

```
| uuid                   | dfc6189f-ad83-4261-9bda-b27258eb1987 |
| console_enabled        | False                                |
| target_provision_state | None                                 |
| provision_updated_at   | None                                 |
| maintenance            | False                                |
| power_state            | None                                 |
| driver                 | ipmi                                 |
| properties             | {}                                   |
| instance_uuid          | None                                 |
| name                   | None                                 |
| driver_info            | {}                                   |
| ...                    | ...                                  |
+------------------------+--------------------------------------+
```

A node may also be referred to by a logical name as well as its UUID. A name can be assigned to the node during its creation by adding the `-n` option to the `node create` command or by updating an existing node with the `node set` command. See *Logical Names* for examples.

2. Starting with API version 1.31 (and `python-ironicclient` 1.13), you can pick which hardware interface to use with nodes that use hardware types. Each interface is represented by a node field called `<IFACE>_interface` where `<IFACE>` in the interface type, e.g. `boot`. See *Enabling drivers and hardware types* for details on hardware interfaces.

   An interface can be set either separately:

```
$ baremetal node set $NODE_UUID --deploy-interface direct --raid-
↪interface agent
```

   or set during node creation:

```
$ baremetal node create --driver ipmi \
    --deploy-interface direct \
    --raid-interface agent
```

   If no value is provided for some interfaces, *Defaults for hardware interfaces* are used instead.

3. Update the node `driver_info` with the required driver properties, so that the Bare Metal service can manage the node:

```
$ baremetal node set $NODE_UUID \
    --driver-info ipmi_username=$USER \
    --driver-info ipmi_password=$PASS \
    --driver-info ipmi_address=$ADDRESS
```

Note

If IPMI is running on a port other than 623 (the default). The port must be added to `driver_info` by specifying the `ipmi_port` value. Example:

```
$ baremetal node set $NODE_UUID --driver-info ipmi_port=$PORT_NUMBER
```

You may also specify all `driver_info` parameters during node creation by passing the **driver-info** option multiple times:

```
$ baremetal node create --driver ipmi \
    --driver-info ipmi_username=$USER \
    --driver-info ipmi_password=$PASS \
    --driver-info ipmi_address=$ADDRESS
```

See *Choosing a driver* above for details on driver properties.

4. Specify a deploy kernel and ramdisk compatible with the nodes driver, for example:

```
$ baremetal node set $NODE_UUID \
    --driver-info deploy_kernel=$DEPLOY_VMLINUZ_UUID \
    --driver-info deploy_ramdisk=$DEPLOY_INITRD_UUID
```

See *Add images to the Image service* for details.

5. Optionally you can specify the provisioning and/or cleaning network UUID or name in the nodes `driver_info`. The `neutron` network interface requires both `provisioning_network` and `cleaning_network`, while the `flat` network interface requires the `cleaning_network` to be set either in the configuration or on the nodes. For example:

```
$ baremetal node set $NODE_UUID \
    --driver-info cleaning_network=$CLEAN_UUID_OR_NAME \
    --driver-info provisioning_network=$PROVISION_UUID_OR_NAME
```

If you use *managed inspection*, you may also configure `inspection_network` the same way.

See *Configure tenant networks* for details.

6. You must also inform the Bare Metal service of the network interface cards which are part of the node by creating a port with each NICs MAC address. These MAC addresses are passed to the Networking service during instance provisioning and used to configure the network appropriately:

```
$ baremetal port create $MAC_ADDRESS --node $NODE_UUID
```

Note

When it is time to remove the node from the Bare Metal service, the command used to remove the port is `baremetal port delete <port uuid>`. When doing so, it is important to ensure that the baremetal node is not in `maintenance` as guarding logic to prevent orphaning Neutron Virtual Interfaces (VIFs) will be overridden.

Adding scheduling information

1. Assign a *resource class* to the node. A *resource class* should represent a class of hardware in your data center, that corresponds to a Compute flavor.

   For example, lets split hardware into these three groups:

   1. nodes with a lot of RAM and powerful CPU for computational tasks,
   2. nodes with powerful GPU for OpenCL computing,

3. smaller nodes for development and testing.

We can define three resource classes to reflect these hardware groups, named `large-cpu`, `large-gpu` and `small` respectively. Then, for each node in each of the hardware groups, well set their `resource_class` appropriately via:

```
$ baremetal node set $NODE_UUID --resource-class $CLASS_NAME
```

The `--resource-class` argument can also be used when creating a node:

```
$ baremetal node create --driver $DRIVER --resource-class $CLASS_NAME
```

To use resource classes for scheduling you need to update your flavors as described in *Create flavors for use with the Bare Metal service*.

Note

This is not required for standalone deployments, only for those using the Compute service for provisioning bare metal instances.

2. Update the nodes properties to match the actual hardware of the node. These are optional. When provided, `memory_mb` can be used for checking if the instance image fits into the nodes memory:

```
$ baremetal node set $NODE_UUID \
    --property memory_mb=$RAM_MB \
    --property local_gb=$DISK_GB
```

As above, these can also be specified at node creation by passing the **property** option to `node create` multiple times:

```
$ baremetal node create --driver ipmi \
    --driver-info ipmi_username=$USER \
    --driver-info ipmi_password=$PASS \
    --driver-info ipmi_address=$ADDRESS \
    --property memory_mb=$RAM_MB \
    --property local_gb=$DISK_GB
```

These values can also be discovered during *Hardware Inspection*.

Note

The value provided for the `local_gb` property should match the size of the root device youre going to deploy on. By default **ironic-python-agent** picks the smallest disk which is not smaller than 4 GiB.

If you override this logic by using root device hints (see *Specifying the disk for deployment (root device hints)*), the `local_gb` value should match the size of picked target disk.

3. If you wish to perform more advanced scheduling of the instances based on hardware capabilities, you may add metadata to each node that will be exposed to the Compute scheduler (see: Compute-CapabilitiesFilter). A full explanation of this is outside of the scope of this document. It can be done through the special `capabilities` member of node properties:

```
$ baremetal node set $NODE_UUID \
    --property capabilities=key1:val1,key2:val2
```

Some capabilities can also be discovered during *Hardware Inspection*.

4. If you wish to perform advanced scheduling of instances based on qualitative attributes of bare metal nodes, you may add traits to each bare metal node that will be exposed to the Compute scheduler (see: *Scheduling based on traits* for a more in-depth discussion of traits in the Bare Metal service). For example, to add the standard trait `HW_CPU_X86_VMX` and a custom trait `CUSTOM_TRAIT1` to a node:

```
$ baremetal node add trait $NODE_UUID \
    CUSTOM_TRAIT1 HW_CPU_X86_VMX
```

Validating node information

1. To check if Bare Metal service has the minimum information necessary for a nodes driver to be functional, you may `validate` it:

```
$ baremetal node validate $NODE_UUID
+------------+--------+--------+
| Interface  | Result | Reason |
+------------+--------+--------+
| boot       | True   |        |
| console    | True   |        |
| deploy     | True   |        |
| inspect    | True   |        |
| management | True   |        |
| network    | True   |        |
| power      | True   |        |
| raid       | True   |        |
| storage    | True   |        |
+------------+--------+--------+
```

If the node fails validation, each driver interface will return information as to why it failed:

```
$ baremetal node validate $NODE_UUID
+------------+--------+-----------------------------------------------------------
↪-----------------------------------------------------------------------------
↪----------+
| Interface  | Result | Reason                                                    ␣
↪                                                                             ␣
↪          |
+------------+--------+-----------------------------------------------------------
↪-----------------------------------------------------------------------------
↪----------+
| boot       | True   |                                                           ␣
↪                                                                             ␣
↪          |
| console    | None   | not supported                                             ␣
↪                                                                             ␣
```

(continues on next page)

(continued from previous page)

```
↪           |
| deploy     | False  | Cannot validate iSCSI deploy. Some parameters␣
↪were missing in node's instance_info. Missing are: ['root_gb', 'image_
↪source']        |
| inspect    | True   |                                                  ␣
↪                                                                       ␣
↪           |
| management | False  | Missing the following IPMI credentials in node's␣
↪driver_info: ['ipmi_address'].                                         ␣
↪           |
| network    | True   |                                                  ␣
↪                                                                       ␣
↪           |
| power      | False  | Missing the following IPMI credentials in node's␣
↪driver_info: ['ipmi_address'].                                         ␣
↪           |
| raid       | None   | not supported                                    ␣
↪                                                                       ␣
↪           |
| storage    | True   |                                                  ␣
↪                                                                       ␣
↪           |
+------------+--------+--------------------------------------------------
↪-----------------------------------------------------------------------
↪----------+
```

When using the Compute Service with the Bare Metal service, it is safe to ignore the deploy interfaces validation error due to lack of image information. You may continue the enrollment process. This information will be set by the Compute Service just before deploying, when an instance is requested:

```
$ baremetal node validate $NODE_UUID
+------------+--------+--------------------------------------------------
↪-----------------------------------------------------------------------
↪-------------------------------------+
| Interface  | Result | Reason                                           ␣
↪                                                                       ␣
↪                                      |
+------------+--------+--------------------------------------------------
↪-----------------------------------------------------------------------
↪-------------------------------------+
| boot       | False  | Cannot validate image information for node␣
↪because one or more parameters are missing from its instance_info.␣
↪Missing are: ['ramdisk', 'kernel', 'image_source'] |
| console    | True   |                                                  ␣
↪                                                                       ␣
↪                                      |
| deploy     | False  | Cannot validate image information for node␣
↪because one or more parameters are missing from its instance_info.␣
↪Missing are: ['ramdisk', 'kernel', 'image_source'] |
```

(continues on next page)

(continued from previous page)

```
| inspect    | True   |                                                              ␣
↪                                                                               ␣
↪                                          |
| management | True   |                                                              ␣
↪                                                                               ␣
↪                                          |
| network    | True   |                                                              ␣
↪                                                                               ␣
↪                                          |
| power      | True   |                                                              ␣
↪                                                                               ␣
↪                                          |
| raid       | None   | not supported                                                ␣
↪                                                                               ␣
↪                                          |
| storage    | True   |                                                              ␣
↪                                                                               ␣
↪                                          |
+------------+--------+--------------------------------------------------------
↪--------------------------------------------------------------------------------
↪---------------------------------------+
```

Making node available for deployment

In order for nodes to be available for deploying workloads on them, nodes must be in the `available` provision state. To do this, nodes must be moved from the `enroll` state to the `manageable` state and then to the `available` state.

Note

This section can be skipped, if API version 1.10 or earlier is used.

After creating a node and before moving it from its initial provision state of `enroll`, basic power and port information needs to be configured on the node. The Bare Metal service needs this information because it verifies that it is capable of controlling the node when transitioning the node from `enroll` to `manageable` state.

To move a node from `enroll` to `manageable` provision state:

```
$ baremetal node manage $NODE_UUID
$ baremetal node show $NODE_UUID
+------------------------+------------------------------------------------------
↪----------------+
| Property               | Value                                                ␣
↪                |
+------------------------+------------------------------------------------------
↪----------------+
| ...                    | ...                                                  ␣
↪                |
```

(continues on next page)

(continued from previous page)

```
| provision_state         | manageable                                          ␣
↪                | <- verify correct state
| uuid                    | 0eb013bb-1e4b-4f4c-94b5-2e7468242611                ␣
↪                |
| ...                     | ...                                                 ␣
↪                |
+-------------------------+-----------------------------------------------------
↪----------------+
```

Note

Since it is an asynchronous call, the response for `baremetal node manage` will not indicate whether the transition succeeded or not. You can check the status of the operation via `baremetal node show`. If it was successful, `provision_state` will be in the desired state. If it failed, there will be information in the nodes `last_error`.

When a node is moved from the `manageable` to `available` provision state, the node will go through automated cleaning if configured to do so (see *Configure the Bare Metal service for cleaning*).

To move a node from `manageable` to `available` provision state:

```
$ baremetal node provide $NODE_UUID
$ baremetal node show $NODE_UUID
+-------------------------+-----------------------------------------------------
↪----------------+
| Property                | Value                                               ␣
↪                |
+-------------------------+-----------------------------------------------------
↪----------------+
| ...                     | ...                                                 ␣
↪                |
| provision_state         | available                                           ␣
↪                | < - verify correct state
| uuid                    | 0eb013bb-1e4b-4f4c-94b5-2e7468242611                ␣
↪                |
| ...                     | ...                                                 ␣
↪                |
+-------------------------+-----------------------------------------------------
↪----------------+
```

For more details on the Bare Metal services state machine, see the *Bare Metal State Machine* documentation.

Mapping nodes to Compute cells

If the Compute service is used for scheduling, and the `discover_hosts_in_cells_interval` was not set as described in *Configure the Compute service to use the Bare Metal service*, then log into any controller node and run the following command to map the new node(s) to Compute cells:

```
nova-manage cell_v2 discover_hosts
```

Logical names

A node may also be referred to by a logical name as well as its UUID. Names can be assigned either during its creation by adding the `-n` option to the `node create` command or by updating an existing node with the `node set` command.

Node names must be unique, and conform to:

- rfc952
- rfc1123
- wiki_hostname

The node is named example in the following examples:

```
$ baremetal node create --driver ipmi --name example
```

or

```
$ baremetal node set $NODE_UUID --name example
```

Once assigned a logical name, a node can then be referred to by name or UUID interchangeably:

```
$ baremetal node create --driver ipmi --name example
+--------------+--------------------------------------+
| Property     | Value                                |
+--------------+--------------------------------------+
| uuid         | 71e01002-8662-434d-aafd-f068f69bb85e |
| driver_info  | {}                                   |
| extra        | {}                                   |
| driver       | ipmi                                 |
| chassis_uuid |                                      |
| properties   | {}                                   |
| name         | example                              |
+--------------+--------------------------------------+

$ baremetal node show example
+------------------------+--------------------------------------+
| Property               | Value                                |
+------------------------+--------------------------------------+
| target_power_state     | None                                 |
| extra                  | {}                                   |
| last_error             | None                                 |
| updated_at             | 2015-04-24T16:23:46+00:00            |
| ...                    | ...                                  |
| instance_info          | {}                                   |
+------------------------+--------------------------------------+
```

Defaults for hardware interfaces

For *hardware types*, users can request one of enabled implementations when creating or updating a node as explained in *Creating a node*.

When no value is provided for a certain interface when creating a node, or changing a nodes hardware type, the default value is used. You can use the driver details command to list the current enabled and default interfaces for a hardware type (for your deployment):

```
$ baremetal driver show ipmi
+--------------------------------+----------------+
| Field                          | Value          |
+--------------------------------+----------------+
| default_boot_interface         | pxe            |
| default_console_interface      | no-console     |
| default_deploy_interface       | direct         |
| default_inspect_interface      | no-inspect     |
| default_management_interface   | ipmitool       |
| default_network_interface      | flat           |
| default_power_interface        | ipmitool       |
| default_raid_interface         | no-raid        |
| default_vendor_interface       | no-vendor      |
| enabled_boot_interfaces        | pxe            |
| enabled_console_interfaces     | no-console     |
| enabled_deploy_interfaces      | direct         |
| enabled_inspect_interfaces     | no-inspect     |
| enabled_management_interfaces  | ipmitool       |
| enabled_network_interfaces     | flat, noop     |
| enabled_power_interfaces       | ipmitool       |
| enabled_raid_interfaces        | no-raid, agent |
| enabled_vendor_interfaces      | no-vendor      |
| hosts                          | ironic-host-1  |
| name                           | ipmi           |
| type                           | dynamic        |
+--------------------------------+----------------+
```

The defaults are calculated as follows:

1. If the `default_<IFACE>_interface` configuration option (where `<IFACE>` is the interface name) is set, its value is used as the default.

   If this implementation is not compatible with the nodes hardware type, an error is returned to a user. An explicit value has to be provided for the nodes `<IFACE>_interface` field in this case.

2. Otherwise, the first supported implementation that is enabled by an operator is used as the default.

   A list of supported implementations is calculated by taking the intersection between the implementations supported by the nodes hardware type and implementations enabled by the `enabled_<IFACE>_interfaces` option (where `<IFACE>` is the interface name). The calculation preserves the order of items, as provided by the hardware type.

   If the list of supported implementations is not empty, the first one is used. Otherwise, an error is returned to a user. In this case, an explicit value has to be provided for the `<IFACE>_interface` field.

See *Enabling drivers and hardware types* for more details on configuration.

Example

Consider the following configuration (shortened for simplicity):

```
[DEFAULT]
enabled_hardware_types = ipmi,redfish
enabled_console_interfaces = no-console,ipmitool-socat
enabled_deploy_interfaces = direct
enabled_management_interfaces = ipmitool,redfish
enabled_power_interfaces = ipmitool,redfish
default_deploy_interface = ansible
```

A new node is created with the `ipmi` driver and no interfaces specified:

```
$ export OS_BAREMETAL_API_VERSION=1.31
$ baremetal node create --driver ipmi
+--------------+--------------------------------------+
| Property     | Value                                |
+--------------+--------------------------------------+
| uuid         | dfc6189f-ad83-4261-9bda-b27258eb1987 |
| driver_info  | {}                                   |
| extra        | {}                                   |
| driver       | ipmi                                 |
| chassis_uuid |                                      |
| properties   | {}                                   |
| name         | None                                 |
+--------------+--------------------------------------+
```

Then the defaults for the interfaces that will be used by the node in this example are calculated as follows:

deploy
: An explicit value of `ansible` is provided for `default_deploy_interface`, so it is used.

power
: No default is configured. The `ipmi` hardware type supports only `ipmitool` power. The intersection between supported power interfaces and values provided in the `enabled_power_interfaces` option has only one item: `ipmitool`. It is used.

console
: No default is configured. The `ipmi` hardware type supports the following console interfaces: `ipmitool-socat`, and `no-console` (in this order). Both are enabled: `no-console` and `ipmitool-socat` (order does not matter). The intersection contains `ipmitool-socat` and `no-console`. The first item is used, and it is `ipmitool-socat`.

management
: Following the same calculation as *power*, the `ipmitool` management interface is used.

Hardware Inspection

The Bare Metal service supports hardware inspection that simplifies enrolling nodes - please see /admin/inspection for details.

Tenant Networks and Port Groups

See *Networking with the Bare Metal service* and *Port groups support*.

2.1.8 Using Bare Metal service as a standalone service

This guide explains how to configure and use the Bare Metal service standalone, i.e. without other OpenStack services. In this mode users are interacting with the bare metal API directly, not though OpenStack Compute.

Configuration

This guide covers manual configuration of the Bare Metal service in the standalone mode. Alternatively, Bifrost can be used for automatic configuration.

Service settings

It is possible to use the Bare Metal service without other OpenStack services. You should make the following changes to `/etc/ironic/ironic.conf`:

1. Choose an authentication strategy which supports standalone, one option is `noauth`:

   ```
   [DEFAULT]
   auth_strategy=noauth
   ```

   Another option is `http_basic` where the credentials are stored in an Apache htpasswd format file:

   ```
   [DEFAULT]
   auth_strategy=http_basic
   http_basic_auth_user_file=/etc/ironic/htpasswd
   ```

   Only the `bcrypt` format is supported, and the Apache `htpasswd` utility can be used to populate the file with entries, for example:

   ```
   htpasswd -nbB myName myPassword >> /etc/ironic/htpasswd
   ```

2. If you want to disable the Networking service, you should have your network pre-configured to serve DHCP and TFTP for machines that youre deploying. To disable it, change the following lines:

   ```
   [dhcp]
   dhcp_provider=none
   ```

   Note

   If you disabled the Networking service and the driver that you use is supported by at most one conductor, PXE boot will still work for your nodes without any manual config editing. This is

because you know all the DHCP options that will be used for deployment and can set up your DHCP server appropriately.

If you have multiple conductors per driver, it would be better to use Networking since it will do all the dynamically changing configurations for you.

3. If you want to disable using a messaging broker between conductor and API processes, switch to JSON RPC instead:

```
[DEFAULT]
rpc_transport = json-rpc
```

JSON RPC also has its own authentication strategy. If it is not specified then the strategy defaults to `[DEFAULT] auth_strategy`. The following will set JSON RPC to `noauth`:

```
[json_rpc]
auth_strategy = noauth
```

For `http_basic` the conductor server needs a credentials file to validate requests:

```
[json_rpc]
auth_strategy = http_basic
http_basic_auth_user_file = /etc/ironic/htpasswd-json-rpc
```

The API server also needs client-side credentials to be specified:

```
[json_rpc]
auth_type = http_basic
username = myName
password = myPassword
```

4. To make graphical consoles available for local viewing, set the following, including an appropriate container image reference for console_image.

```
[vnc]
enabled=True
port=6090
host_ip=127.0.0.1
public_url=http://127.0.0.1:6090/vnc_lite.html
container_provider=systemd
console_image=<image reference>
```

5. Starting with the Yoga release series, you can use a combined API+conductor+novncproxy service and completely disable the RPC. Set

```
[DEFAULT]
rpc_transport = none
```

and use the `ironic` executable to start the combined service.

> **Note**
>
> The combined service also works with RPC enabled, which can be useful for some deployments, but may not be advisable for all security models.

Using CLI

To use the baremetal CLI, set up these environment variables. If the `noauth` authentication strategy is being used, the value `none` must be set for OS_AUTH_TYPE. OS_ENDPOINT is the URL of the ironic-api process. For example:

```
export OS_AUTH_TYPE=none
export OS_ENDPOINT=http://localhost:6385/
```

If the `http_basic` authentication strategy is being used, the value `http_basic` must be set for OS_AUTH_TYPE. For example:

```
export OS_AUTH_TYPE=http_basic
export OS_ENDPOINT=http://localhost:6385/
export OS_USERNAME=myUser
export OS_PASSWORD=myPassword
```

Enrollment

Preparing images

If you dont use Image service, its possible to provide images to Bare Metal service via a URL.

At the moment, only two types of URLs are acceptable instead of Image service UUIDs: HTTP(S) URLs (for example, http://my.server.net/images/img) and file URLs (file:///images/img).

There are however some limitations for different hardware interfaces:

- If youre using *Direct deploy* with HTTP(s) URLs, you have to provide the Bare Metal service with the a checksum of your instance image.

  MD5 is used by default for backward compatibility reasons. To compute an MD5 checksum, you can use the following command:

  ```
  $ md5sum image.qcow2
  ed82def8730f394fb85aef8a208635f6  image.qcow2
  ```

  Alternatively, use a SHA256 checksum or any other algorithm supported by the Pythons hashlib, e.g.:

  ```
  $ sha256sum image.qcow2
  9f6c942ad81690a9926ff530629fb69a82db8b8ab267e2cbd59df417c1a28060  image.
  ↪qcow2
  ```

- If youre using *Direct deploy* with `file://` URLs, you have to ensure the images meet all requirements:

  - File images must be accessible to every conductor

- File images must not be located in `/dev`, `/sys`, `/proc`, `/etc`, `/boot`, `/run` or other system paths.
- File images must be located in a path listed in *conductor.file_url_allowed_paths*

Warning

The Ironic CLI tool will not transfer the file from a local machine to the conductor(s). Operators should use shared file systems or configuration management to ensure consistent availability of images.

Note

The Bare Metal service tracks content changes for non-Glance images by checking their modification date and time. For example, for HTTP image, if Last-Modified header value from response to a HEAD request to http://my.server.net/images/deploy.ramdisk is greater than cached image modification time, Ironic will re-download the content. For file:// images, the file system modification time is used.

If the HTTP server does not provide the last modification date and time, the image will be redownloaded every time it is used.

Enrolling nodes

1. Create a node in Bare Metal service. At minimum, you must specify the driver name (for example, `ipmi`). You can also specify all the required driver parameters in one command. This will return the node UUID:

```
$ baremetal node create --driver ipmi \
    --driver-info ipmi_address=ipmi.server.net \
    --driver-info ipmi_username=user \
    --driver-info ipmi_password=pass \
    --driver-info deploy_kernel=file:///images/deploy.vmlinuz \
    --driver-info deploy_ramdisk=http://my.server.net/images/deploy.
↪ramdisk
+--------------+--------------------------------------------------------
↪---------------+
| Property     | Value                                                  ␣
↪                |
+--------------+--------------------------------------------------------
↪---------------+
| uuid         | be94df40-b80a-4f63-b92b-e9368ee8d14c                   ␣
↪                |
| driver_info  | {u'deploy_ramdisk': u'http://my.server.net/images/deploy.
↪ramdisk',       |
|              | u'deploy_kernel': u'file:///images/deploy.vmlinuz', u
↪'ipmi_address':     |
|              | u'ipmi.server.net', u'ipmi_username': u'user', u'ipmi_
↪password':          |
|              | u'******'}                                             ␣
↪                |
```

(continues on next page)

(continued from previous page)

```
| extra          | {}                                                                 ␣
↪                  |
| driver         | ipmi                                                               ␣
↪                  |
| chassis_uuid   |                                                                    ␣
↪                  |
| properties     | {}                                                                 ␣
↪                  |
+----------------+--------------------------------------------------------------------
↪----------------+
```

Note that here deploy_kernel and deploy_ramdisk contain links to images instead of Image service UUIDs.

2. As in case of Compute service, you can also provide `capabilities` to node properties, but they will be used only by Bare Metal service (for example, boot mode).
3. Then create a port to inform Bare Metal service of the network interface cards which are part of the node by creating a port with each NICs MAC address. In this case, theyre used for naming of PXE configs for a node:

```
baremetal port create $MAC_ADDRESS --node $NODE_UUID
```

Configure the Standalone Networking Service

The Standalone Networking service provides network switch management for bare metal provisioning in environments where Neutron is not available or not desired. This service manages switch port configurations through a dedicated `ironic-networking` process that communicates with the Ironic conductor via RPC. The service is capable of provisioning switch port attributes on the neighboring switches to include VLAN configuration information for both access and trunk ports.

Warning

The standalone networking feature is experimental and the API may change in future releases. Specifically, using the `port.extra` attribute to pass switchport configuration on each individual port will be re-visited and likely converted to use a more formal API structure.

Note

The expected starting condition before a node is enrolled is that its switch ports are configured onto the idle VLAN. The idle VLAN can be considered as the default VLAN for a port. The networking service will always set the switch port back to this VLAN whenever no specific VLAN is required based on the state of the node. It is expected that this VLAN is routable back to the Ironic API. Nodes that are not configured onto the idle VLAN will fail the initial inspection; therefore, no LLDP information will be available to identify which switch ports need to be configured for that node. In that case, `local_link_connection` information must be provided manually for each port.

Overview

The standalone networking architecture consists of:

- **Networking Service** (`ironic-networking`): A standalone service that manages network switch configurations via switch drivers
- **Switch Driver Framework**: Pluggable drivers for different network switch vendors (e.g., generic-switch implemented using networking-generic-switch)
- **ironic-networking Network Interface**: An Ironic network interface that integrates with the standalone networking service
- **RPC Communication**: Communication between conductor and networking service using JSON-RPC or oslo-messaging supported backends like rabbitmq

Configuring Driver Interfaces

Switch Driver Framework

The switch driver framework within the `ironic-networking` service provides:

- **Driver Factory**: Dynamically loads switch drivers via stevedore
- **Base Driver Interface**: Common API for all switch drivers
- **Generic Switch Driver**: Built-in driver, named `generic-switch`, using networking-generic-switch
- **Driver Translators**: Convert generic switch configs to driver-specific formats

ironic-networking Network Interface

The `ironic-networking` network interface (enabled per-node) provides:

- Integration between Ironic nodes and the networking service
- Switchport configuration validation via JSON schemas
- Support for ports, and planned support for portgroups (LAG)
- State-based network configuration (cleaning, provisioning, rescuing, etc.)

Installation and Dependencies

The standalone networking service requires the `networking-generic-switch` library for the generic-switch driver. Install it from the driver requirements file:

```
$ pip install -r driver-requirements.txt
```

Configuration

Authentication Strategies

The standalone networking service supports the following authentication strategies for RPC communication between the Ironic conductor and the networking service:

- `keystone` - Use OpenStack Identity service (Keystone) for authentication. Recommended for production OpenStack deployments.

- `http_basic` - HTTP Basic authentication using an Apache-format htpasswd file. Suitable for standalone deployments without Keystone.
- `noauth` - No authentication. **Not recommended for production** as it provides no security.

The authentication strategy is configured via the `auth_strategy` option, which can be set globally in `[DEFAULT]` or per-service in the `[ironic_networking_json_rpc]` section.

Creating the htpasswd File

When using `http_basic` authentication, create the htpasswd file:

```
$ htpasswd -c -B /etc/ironic/htpasswd-rpc networking-rpc
```

Set appropriate permissions:

```
$ chmod 600 /etc/ironic/htpasswd-rpc
$ chown ironic:ironic /etc/ironic/htpasswd-rpc
```

RPC Transport Options

Communication between the Ironic conductor and the networking service can use one of two transport mechanisms:

- `json-rpc` - Direct JSON-RPC over HTTP/HTTPS. Simpler to configure for standalone deployments.
- `oslo_messaging` - Oslo.messaging with a message broker (e.g., RabbitMQ). Recommended for production OpenStack deployments as it provides better reliability and is consistent with other OpenStack services.

Example: Production Configuration with Keystone and Oslo Messaging

This example shows a production-ready configuration using Keystone authentication and RabbitMQ for RPC transport.

Main Ironic Configuration (ironic.conf)

```
[DEFAULT]
enabled_network_interfaces = ironic-networking,noop
auth_strategy = keystone
rpc_transport = oslo_messaging
transport_url = rabbit://ironic:password@rabbit-host:5672/

[ironic_networking]
rpc_transport = oslo_messaging
idle_network = access/native_vlan=123
inspection_network = access/native_vlan=123

[keystone_authtoken]
www_authenticate_uri = http://keystone-host:5000
auth_url = http://keystone-host:5000
auth_type = password
```

(continues on next page)

(continued from previous page)

```
project_domain_name = Default
user_domain_name = Default
project_name = service
username = ironic
password = <ironic-service-password>

[oslo_messaging_rabbit]
rabbit_host = rabbit-host
rabbit_userid = ironic
rabbit_password = password
```

Networking Service Configuration (ironic-networking.conf)

```
[DEFAULT]
auth_strategy = keystone
rpc_transport = oslo_messaging
transport_url = rabbit://ironic:password@rabbit-host:5672/

[keystone_authtoken]
www_authenticate_uri = http://keystone-host:5000
auth_url = http://keystone-host:5000
auth_type = password
project_domain_name = Default
user_domain_name = Default
project_name = service
username = ironic-networking
password = <ironic-networking-service-password>

[oslo_messaging_rabbit]
rabbit_host = rabbit-host
rabbit_userid = ironic
rabbit_password = password

[ironic_networking]
enabled_switch_drivers = generic-switch
driver_config_dir = /var/lib/ironic/networking
switch_config_file = /etc/ironic/networking/switch-configs.conf
```

Example: Standalone Configuration with HTTP Basic Auth and JSON-RPC

This example shows a simpler configuration suitable for standalone deployments without Keystone or a message broker.

Main Ironic Configuration (ironic.conf)

```
[DEFAULT]
enabled_network_interfaces = ironic-networking,noop
auth_strategy = http_basic
http_basic_auth_user_file = /etc/ironic/htpasswd
```

(continues on next page)

(continued from previous page)

```
[ironic_networking_json_rpc]
auth_strategy = http_basic
http_basic_auth_user_file = /etc/ironic/htpasswd-rpc
host_ip = localhost
port = 6190
username = networking-rpc
password = <rpc-password>

[ironic_networking]
rpc_transport = json-rpc
idle_network = access/native_vlan=123
inspection_network = access/native_vlan=123
```

Networking Service Configuration (ironic-networking.conf)

```
[DEFAULT]
auth_strategy = http_basic
http_basic_auth_user_file = /etc/ironic/htpasswd
rpc_transport = json-rpc

[ironic_networking_json_rpc]
auth_strategy = http_basic
http_basic_auth_user_file = /etc/ironic/htpasswd-rpc
host_ip = localhost
port = 6190
username = networking-rpc
password = <rpc-password>

[ssl]
cert_file = /etc/ironic/ssl/tls.crt
key_file = /etc/ironic/ssl/tls.key

[ironic_networking]
enabled_switch_drivers = generic-switch
driver_config_dir = /var/lib/ironic/networking
switch_config_file = /etc/ironic/networking/switch-configs.conf
```

Switch Configuration File (switch-configs.conf)

Create `/etc/ironic/switch-configs.conf` with your switch definitions. This file uses a generic format that is translated to driver-specific configuration by the driver adapter.

Note

The networking service auto-generates driver-specific config files in `driver_config_dir` from this generic configuration. You should not manually create files in that directory.

Generic Format

Each switch is defined in its own section with a name of your choice:

```
[switch-name]
# Switch connection information
address = 192.168.1.10
username = admin
password = secretpassword

# Driver type.  Currently only "generic-switch"
driver_type = generic-switch

# Device type.  Value specific to the driver type.  (i.e., if driver type is
# generic-switch then device type should be one of the
# networking-generic-switch "generic_switch.devices" entry points.
# Example netmiko_dell_os10).
device_type = netmiko_cisco_ios

# Switch MAC address (for identification)
mac_address = 00:11:22:33:44:55

# Optional: SSH port (default: 22)
port = 22

# Optional: Enable secret for privileged mode
enable_secret = enablepassword

# Optional: SSH key file instead of password
# key_file = /path/to/ssh/key

# Optional: Save configuration after changes (default: false)
persist = false

# Optional: Filter which VLAN instance are allowed on the switch
allowed_vlans = 100,200-210
```

Example with multiple switches:

```
[tor-switch-1]
address = 192.168.1.10
username = admin
password = switch1pass
device_type = cisco_ios
enable_secret = enablepass
persist = true
mac_address = aa:bb:cc:dd:ee:01

[tor-switch-2]
address = 192.168.1.11
username = admin
```

(continues on next page)

(continued from previous page)

```
password = switch2pass
device_type = arista_eos
mac_address = aa:bb:cc:dd:ee:02
```

Supported Device Types

The generic-switch driver supports any device type supported by the `networking-generic-switch` library. Common types include:

- `cisco_ios` - Cisco IOS switches
- `cisco_nxos` - Cisco Nexus switches
- `arista_eos` - Arista EOS switches
- `dell_os10` - Dell OS10 switches
- `juniper_junos` - Juniper Junos switches
- `ovs_linux` - Open vSwitch (for testing)

See the networking-generic-switch documentation for the complete list of supported device types.

Node Configuration

Configuring Network Interface

Set nodes to use the `ironic-networking` network interface:

```
$ openstack baremetal node set <node> \
    --network-interface ironic-networking
```

Override Default Networks (Optional)

You can override the global network configuration per-node using `driver_info`:

```
$ openstack baremetal node set <node> \
    --driver-info provisioning_network=access/native_vlan=250 \
    --driver-info cleaning_network=access/native_vlan=150
```

Port Configuration with Switchport Information

Each port must have `local_link_connection` and switchport configuration in the `extra` field. The `local_link_connection` information is expected to be populated automatically from LLDP at the end of inspection, or manually if automatic inspection is not used or LLDP is not available.

The `local_link_connection` identifies the physical switch and port:

```
{
  "switch_id": "aa:bb:cc:dd:ee:01",
  "port_id": "GigabitEthernet1/0/1",
  "switch_info": "tor-switch-1"
}
```

The `switch_id` should match either the switchs MAC address or the name used in `switch-configs.conf`.

Switchport Configuration Schema

The switchport configuration in `port.extra.switchport` must conform to this schema:

Required fields:

- `mode` (string): Port mode - `access`, `trunk`, or `hybrid`
- `native_vlan` (integer): Native VLAN ID (1-4094)

Optional fields:

- `allowed_vlans` (array of integers): Allowed VLAN IDs for trunk/hybrid modes (required if mode is trunk or hybrid)
- `mtu` (integer): Maximum transmission unit (max: 9216)

Validation rules:

- Access mode: `allowed_vlans` must NOT be specified
- Trunk/Hybrid mode: `allowed_vlans` must be specified with at least one VLAN
- PXE-enabled ports: Must use `access` mode

Example: Access Mode Port

Note

These examples manually set the local-link-connection info, but as described earlier, these values could be received via LLDP during inspection if both are enabled and available.

```
$ openstack baremetal port create \
    --node <node-uuid> \
    --address aa:bb:cc:dd:ee:ff \
    --local-link-connection switch_id=aa:bb:cc:dd:ee:01 \
    --local-link-connection port_id=GigabitEthernet1/0/1 \
    --local-link-connection switch_info=tor-switch-1 \
    --extra switchport='{"mode": "access", "native_vlan": 100}' \
    --pxe-enabled true
```

Example: Trunk Mode Port

```
$ openstack baremetal port create \
    --node <node-uuid> \
    --address aa:bb:cc:dd:ee:ff \
    --local-link-connection switch_id=aa:bb:cc:dd:ee:01 \
    --local-link-connection port_id=GigabitEthernet1/0/2 \
    --local-link-connection switch_info=tor-switch-1 \
    --extra switchport='{"mode": "trunk", "native_vlan": 1, "allowed_vlans":␣
```

(continues on next page)

(continued from previous page)

```
↪[100, 200, 300]}' \
    --pxe-enabled false
```

Running the Networking Service

Standalone Mode

Run the networking service as a separate process:

```
$ ironic-networking --config-file /etc/ironic/ironic.conf
```

Verify Service Status

Check that the service initialized correctly by examining logs. You should see log messages indicating:

- Number of driver-specific config files generated
- List of loaded switch drivers
- Networking service initialization on the configured host

Example output:

```
Networking service initialized with switch drivers: generic-switch
Generated 3 driver-specific config files during init
```

Troubleshooting

Common Issues

No switch drivers loaded

If you see:

```
No switch drivers loaded - networking service will operate without switch
management capabilities
```

Check:

- `enabled_switch_drivers` is set in `[ironic_networking]` section
- Switch driver is installed (e.g., `networking-generic-switch`)
- Driver entry point exists in `ironic.networking.switch_drivers` namespace

Switch not found errors

If switch operations fail with `SwitchNotFound`:

- Verify `switch_id` in ports `local_link_connection` matches a switch in `switch-configs.conf`
- Check switch driver configuration was generated in `driver_config_dir`
- Ensure switch MAC address or name matches exactly

RPC communication failures

For JSON-RPC connection issues:

- Verify `host_ip` and `port` in `[ironic_networking_json_rpc]` section
- Ensure networking service is running and listening
- Check firewall rules allow traffic on the configured port

For Oslo Messaging issues:

- Verify message bus (RabbitMQ) is running and accessible
- Check credentials in `[oslo_messaging_rabbit]` section
- Ensure conductor and networking service use the same transport URL

Debugging

Enable debug logging in `ironic.conf`:

```
[DEFAULT]
debug = true
log_file = /var/log/ironic/ironic-networking.log
```

This will show detailed information about:

- Switch driver loading and configuration
- RPC method calls and parameters
- Switch operations and responses
- VLAN validation results

Advanced Configuration

VLAN Range Restrictions

The `allowed_vlans` configuration option restricts which VLAN IDs can be used for port configuration. This setting can be configured globally in `ironic.conf` and optionally overridden on a per-switch basis in the switch configuration file.

Allow All VLANs (Default)

If `allowed_vlans` is not specified or set to `None`, all VLAN IDs (1-4094) are permitted:

```
[ironic_networking]
# Not specified - all VLANs allowed (default behavior)
```

Block All VLANs

To prevent any VLAN configuration (useful for testing or maintenance):

```
[ironic_networking]
# Empty list - no VLANs allowed
allowed_vlans = []
```

Allow Specific VLANs

Restrict to specific VLAN IDs using individual values, ranges, or a combination of both. Values are specified as a comma-separated list without square brackets:

```
[ironic_networking]
# Allow individual VLANs (comma-separated)
allowed_vlans = 100,101,102,1000

# Allow VLAN ranges
allowed_vlans = 100-199,1000-1099

# Mixed individual and range notation
allowed_vlans = 100,101,102-104,106,200-299,1000
```

The last example allows VLANs: 100, 101, 102, 103, 104, 106, 200-299, and 1000, but denies 105 and all others outside the specified ranges.

Per-Switch VLAN Restrictions

Override global restrictions for specific switches in the switch configuration file:

```
[tor-switch-1]
address = 192.168.1.10
username = admin
password = switchpass
device_type = cisco_ios
# Only specific VLANs on this switch
allowed_vlans = 200-299

[tor-switch-2]
address = 192.168.1.11
username = admin
password = switchpass
device_type = cisco_ios
# Only allow a different set of VLANs on this switch
allowed_vlans = 100-199,1000
```

Validation Behavior

When a port configuration operation is attempted:

1. The service checks if switch-specific `allowed_vlans` is configured
2. If not found, falls back to global `[ironic_networking] allowed_vlans`
3. If neither is specified, all VLANs (1-4094) are allowed
4. If configured, any VLAN outside the allowed set will cause the operation to fail.

The validation applies to both `native_vlan` and all VLANs in the `allowed_vlans` list for trunk/hybrid ports.

Network State Transitions

The ironic-networking interface manages switch configurations based on node state and will configure the switch port to the corresponding network configuration.

- **Inspection**: Configures `inspection_network` (if set)
- **Cleaning**: Configures `cleaning_network` (if set)
- **Provisioning**: Configures `provisioning_network` (if set)
- **Rescuing**: Configures `rescuing_network` (if set)
- **Servicing**: Configures `servicing_network` (if set)
- **Idle**: Reverts to `idle_network` (if set)

Note

The `idle_network` represents the default VLAN configuration that ports should be set to when no specific network is required for the nodes current state. This network is used whenever a node transitions to a state that doesnt have an explicit network configuration defined (e.g., `available`, `enroll`, `manageable`). Its also used when exiting states like cleaning, provisioning, or rescuing if no other network applies.

The idle network should be configured to allow connectivity back to the Ironic API for inspection and management operations. If `idle_network` is not configured, switch ports will not be configured (or will assume the switchs own global default) when transitioning to idle states.

Security Considerations

Credential Management

- Store switch passwords securely, not in plain text
- Use SSH keys (`key_file`) instead of passwords when possible
- Restrict file permissions on `switch-configs.conf`:

```
$ chmod 600 /etc/ironic/switch-configs.conf
$ chown ironic:ironic /etc/ironic/switch-configs.conf
```

Switch Access Control

- Use dedicated service accounts with minimal privileges for switch access
- Enable audit logging on switches for Ironic service account actions
- Regularly rotate switch credentials

Migration from Neutron

If migrating from Neutron-based networking to standalone networking:

1. Ensure all nodes are in a stable state (`available` or `active`)
2. Update node configuration to use `ironic-networking` interface:

```
$ openstack baremetal node set <node> \
    --network-interface ironic-networking
```

3. Add switchport configuration to ports (see Port Configuration section)
4. Configure and start the `ironic-networking` service
5. Test with a single node before migrating all nodes

Note

Ironic supports using different network interfaces on different nodes. Therefore, it is possible to have some nodes migrated to the new `ironic-networking` network interface while others are still using the `neutron` network interface.

Limitations and Future Work

Current Limitations

- LAG operations are not yet implemented
- Limited to switches supported by networking-generic-switch
- No runtime reload of Ironic switch configuration file. Requires ironic-networking service restart.
- Running multiple instances of ironic-networking for high-availability is not supported.

References

- *Layer 3 or DHCP-less ramdisk booting*
- *Networking with the Bare Metal service*
- *Configure the Networking service for bare metal provisioning*
- networking-generic-switch documentation

Once the installation is done, please see *Deploying with Bare Metal service* for information on how to deploy bare metal machines.

Deploying

The content has been migrated, please see *Deploying with Bare Metal service*.

2.1.9 Enabling the configuration drive (configdrive)

The Bare Metal service supports exposing a configuration drive image to the instances.

The configuration drive is used to store instance-specific metadata and is present to the instance as a disk partition labeled `config-2`. The configuration drive has a maximum size of 64MB. One use case for using the configuration drive is to expose a networking configuration when you do not use DHCP to assign IP addresses to instances.

The configuration drive is usually used in conjunction with the Compute service, but the Bare Metal service also offers a standalone way of using it. The following sections will describe both methods.

When used with Compute service

To enable the configuration drive for a specific request, pass `--config-drive true` parameter to the **`nova boot`** command, for example:

```
openstack server create --use-config-drive --flavor baremetal --image test-
↪image instance-1
```

Its also possible to enable the configuration drive automatically on all instances by configuring the `OpenStack Compute service` to always create a configuration drive by setting the following option in the `/etc/nova/nova.conf` file, for example:

```
[DEFAULT]
...

force_config_drive=True
```

In some cases, you may wish to pass a user customized script when deploying an instance. To do this, pass `--user-data /path/to/file` to the **`nova boot`** command.

When used standalone

When used without the Compute service, the operator needs to create a configuration drive and provide the file or HTTP URL to the Bare Metal service. See *Deploying with a config drive* for details.

Configuration drive storage in an object store

Under normal circumstances, the configuration drive can be stored in the Bare Metal service when the size is less than 64KB. Optionally, if the size is larger than 64KB there is support to store it in a swift endpoint. Both swift and radosgw use swift-style APIs.

The following option in `/etc/ironic/ironic.conf` enables swift as an object store backend to store config drive. This uses the Identity service to establish a session between the Bare Metal service and the Object Storage service.

```
[deploy]
...

configdrive_use_object_store = True
```

Use the following options in `/etc/ironic/ironic.conf` to enable radosgw. Credentials in the swift section are needed because radosgw will not use the Identity service and relies on radosgws username and password authentication instead.

```
[deploy]
...

configdrive_use_object_store = True

[swift]
...

username = USERNAME
```

(continues on next page)

(continued from previous page)

```
password = PASSWORD
auth_url = http://RADOSGW_IP:8000/auth/v1
```

If the *Direct deploy* is being used, edit `/etc/glance/glance-api.conf` to store the instance images in respective object store (radosgw or swift) as well:

```
[glance_store]
...

swift_store_user = USERNAME
swift_store_key = PASSWORD
swift_store_auth_address = http://RADOSGW_OR_SWIFT_IP:PORT/auth/v1
```

Accessing the configuration drive data

When the configuration drive is enabled, the Bare Metal service will create a partition on the instance disk and write the configuration drive image onto it. The configuration drive must be mounted before use. This is performed automatically by many tools, such as cloud-init and cloudbase-init. To mount it manually on a Linux distribution that supports accessing devices by labels, simply run the following:

```
mkdir -p /mnt/config
mount /dev/disk/by-label/config-2 /mnt/config
```

If the guest OS doesnt support accessing devices by labels, you can use other tools such as `blkid` to identify which device corresponds to the configuration drive and mount it, for example:

```
CONFIG_DEV=$(blkid -t LABEL="config-2" -odevice)
mkdir -p /mnt/config
mount $CONFIG_DEV /mnt/config
```

Cloud-init integration

The configuration drive can be especially useful when used with cloud-init, but in order to use it we should follow some rules:

- `Cloud-init` data should be organized in the expected format.
- Since the Bare Metal service uses a disk partition as the configuration drive, it will only work with cloud-init version >= 0.7.5.
- `Cloud-init` has a collection of data source modules, so when building the image with disk-image-builder we have to define `DIB_CLOUD_INIT_DATASOURCES` environment variable and set the appropriate sources to enable the configuration drive, for example:

  ```
  DIB_CLOUD_INIT_DATASOURCES="ConfigDrive, OpenStack" disk-image-create -o␣
  ↪fedora-cloud-image fedora baremetal
  ```

  For more information see how to configure cloud-init data sources.

2.1.10 Graphical console support

The Bare Metal service supports displaying graphical consoles from a number of hardware vendors.

The following preconditions are required for a nodes graphical console to be viewable:

- Service ironic-conductor has a configured console container provider appropriate for the environment
- Service ironic-novncproxy is configured and running
- The nodes `console_interface` is set to a graphical driver such as `redfish-graphical`

When enabled and configured, the following sequence occurs when a graphical console is accessed when interacting with Bare Metal service directly:

- A REST API call is made to enable the console, for example via the CLI command `baremetal node console enable`
- ironic-conductor creates and stores a time-limited token with the node
- ironic-conductor triggers starting a container which runs a virtual X11 display, starts a web browser, and exposes a VNC server
- Once enabled, a REST API call is made to fetch the console URL, for example via the CLI command `baremetal node console show`
- The user accesses the console URL with a web browser
- ironic-novncproxy serves the NoVNC web assets to the browser
- A websocket is initiated with ironic-novncproxy, which looks up the node and validates the token
- ironic-novncproxy makes a VNC connection with the console container and proxies VNC traffic between the container and the browser
- The container initiates a connection with the nodes BMC Redfish endpoint and determines which vendor script to run
- The container makes Redfish calls and simulates a browser user to display an HTML5 console, which the end user can now view

Building a console container

The tools/vnc-container directory contains the files and instructions to build a console container. This directory will be where further development occurs, and currently only a CentOS Stream based image can be built.

Container providers

ironic-conductor must be configured with a container provider so that it can trigger starting and stopping console containers based on nodes console enabled state. Given the variety of deployment architectures for Ironic, an appropriate container provider needs to be configured.

In many cases this will require writing an external custom container provider, especially when Ironic itself is deployed in a containerized environment.

Systemd container provider

The `systemd` provider manages containers as Systemd Quadlet containers. This provider is appropriate to use when the Ironic services themselves are not containerised, and is also a good match when ironic-conductor itself is managed as a Systemd unit.

To start a container, this provider writes `.container` files to `/etc/containers/systemd/users/{uid}/containers/systemd` then calls `systemctl --user daemon-reload` to generate a unit file which is then started with `systemctl --user start {unit name}`.

Kubernetes container provider

The `kubernetes` provider manages containers as kubernetes pods and allows associated resources to also be managed. The provider requires the `kubectl` command, and valid kubernetes credentials be available to the running ironic-conductor. The current assumption with this driver is that ironic-conductor, ironic-novncproxy, and the console containers are all running in the same kubernetes cluster. Therefore, the credentials will be provided by the service account mechanism supplied to the ironic-conductor pod.

`ironic.conf [vnc]kubernetes_container_template` points to a template file which defines the kubernetes resources including the pod running the console container. The default template creates one Secret to store the app info (including BMC credentials) and one Pod to run the actual console container. This default template `ironic-console-pod.yaml.template` is functional but will likely need to be replaced with a variant that customises:

- The namespace the resources are deployed to
- The labels to match the conventions of the deployment

When ironic-conductor starts and stops it will stop any existing console container associated with that ironic-conductor. For this delete-all operation, the labels in the template are transformed into a kubectl selector, so this needs to be a consideration when choosing the labels in the template.

When ironic-conductor is using cluster service account credentials, a RoleBinding to a Role which allows appropriate resource management is required. For example, the default template would require at minimum the following role rules:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  # ...
rules:
- apiGroups:
  - ""
  resources:
  - pods
  verbs:
  - create
  - delete
- apiGroups:
  - ""
  resources:
  - secrets
  verbs:
```

(continues on next page)

(continued from previous page)

```
  - create
  - delete
```

The provider assumes that ironic-novnc is running in the cluster, and can connect a VNC server using the console containers `hostIP`.

Creating an external container provider

An external python library can contribute its own container provider by subclassing `ironic.console.container.base.BaseConsoleContainer` then adding it to the librarys `setup.cfg [entry_points]ironic.console.container`.

The `start_container` method must return the IP and port of the resulting running VNC server, which in most scenarios would mean blocking until the container is running.

Networking requirements

ironic-novncproxy

Like ironic-api, ironic-novncproxy presents a public endpoint. However unlike ironic-api, node console URLs are coupled to the ironic-conductor managing that node, so load balancing across all ironic-novncproxy instances is not appropriate.

A TLS enabled reverse proxy needs to support WebSockets, otherwise TLS can be enabled in the `ironic.conf [vnc]` section.

ironic-novncproxy needs to be able to connect to the VNC servers exposed by the console containers.

Console containers

The VNC servers exposed by console containers are unencrypted and unauthenticated, so public access *must* be restricted via another network configuration mechanism. The ironic-novncproxy service needs to access the VNC server exposed by these containers, and so does nova-novncproxy when Nova is using the Ironic driver.

For the `systemd` container the VNC server will be published on a random high port number. For the `kubernetes` pod the VNC server is running on port `5900` on the pods `hostIP`.

Console containers need access to the management network to access the BMC web interface. If driver_info `redfish_verify_ca=False` then web requests will not be verified by the browser. Setting `redfish_verify_ca` to a certificate path is not yet supported by the `systemd` container provider as the certificate is not bind-mounted into the container. This can be supported locally by building a container which includes the expected certificate files.

2.1.11 Advanced features

Specifying the disk for deployment (root device hints)

The Bare Metal service supports passing hints to the deploy ramdisk about which disk it should pick for the deployment. The list of supported hints is:

- model (STRING): device identifier
- vendor (STRING): device vendor

- serial (STRING): disk serial number

  Note

  Some RAID controllers will generate serial numbers to represent volumes provided to the operating system which do not match or align to physical disks in a system.

- size (INT or range): size of the device in GiB or range boundaries in GiB size accepts a single int value representing the size of the device in GiB, or a range of two values representing the lower and higher acceptable sizes in GiB for the device using the special operator <range-in>, e.g. size: <range-in> (10 100)

  Note

  A nodes local_gb property is often set to a value 1 GiB less than the actual disk size to account for partitioning. However, in this case `size` should be the actual size. For example, for a 128 GiB disk `local_gb` will be 127, but size hint will be 128.

- wwn (STRING): unique storage identifier and typically mapping to a device. This can be a single device, or a SAN storage controller, or a RAID controller.
- wwn_with_extension (STRING): unique storage identifier with the vendor extension appended
- wwn_vendor_extension (STRING): unique vendor storage identifier
- rotational (BOOLEAN): whether its a rotational device or not. This hint makes it easier to distinguish HDDs (rotational) and SSDs (not rotational) when choosing which disk Ironic should deploy the image onto.
- hctl (STRING): the SCSI address (Host, Channel, Target and Lun), e.g 1:0:0:0
- by_path (STRING): the alternate device name corresponding to a particular PCI or iSCSI path, e.g /dev/disk/by-path/pci-0000:00

  Note

  Device identification by-path may not be reliable on Linux kernels using asynchronous device initialization.

- name (STRING): the device name, e.g /dev/md0

  Warning

  The root device hint name should only be used for devices with constant names (e.g RAID volumes). For SATA, SCSI and IDE disk controllers this hint is not recommended because the order in which the device nodes are added in Linux is arbitrary, resulting in devices like /dev/sda and /dev/sdb switching around at boot time.

Warning

Furthermore, recent move to asynchronous device initialization among some Linux distribution kernels means that the actual device name string is entirely unreliable when multiple devices are present in the host, as the device name is claimed by the device which responded first, as opposed to the previous pattern where it was the first initialized device in a synchronous process.

To associate one or more hints with a node, update the nodes properties with a `root_device` key, for example:

```
baremetal node set <node-uuid> --property root_device='{"wwn":
↪"0x4000cca77fc4dba1"}'
```

That will guarantee that Bare Metal service will pick the disk device that has the `wwn` equal to the specified wwn value, or fail the deployment if it can not be found.

Note

Starting with the Ussuri release, root device hints can be specified per-instance, see *Using Bare Metal service as a standalone service*.

The hints can have an operator at the beginning of the value string. If no operator is specified the default is == (for numerical values) and `s==` (for string values). The supported operators are:

- For numerical values:
  - = equal to or greater than. This is equivalent to >= and is supported for legacy reasons
  - == equal to
  - != not equal to
  - >= greater than or equal to
  - > greater than
  - <= less than or equal to
  - < less than
- For strings (as python comparisons):
  - `s==` equal to
  - `s!=` not equal to
  - `s>=` greater than or equal to
  - `s>` greater than
  - `s<=` less than or equal to
  - `s<` less than
  - `<in>` substring
- For collections:

  - <all-in> all elements contained in collection
  - <or> find one of these
  - <range-in> range boundaries in GiB, boundary condition could be inclusive '[' or exclusive '('.

Examples are:

- Finding a disk larger or equal to 60 GiB and non-rotational (SSD):

```
baremetal node set <node-uuid> --property root_device='{"size": ">= 60",
↪"rotational": false}'
```

- Finding a disk whose vendor is `samsung` or `winsys`:

```
baremetal node set <node-uuid> --property root_device='{"vendor": "<or>␣
↪samsung <or> winsys"}'
```

Note

If multiple hints are specified, a device must satisfy all the hints.

Appending kernel parameters to boot instances

The Bare Metal service supports passing custom kernel parameters to boot instances to fit users requirements. The way to append the kernel parameters is depending on how to boot instances.

Network boot

Currently, the Bare Metal service supports assigning unified kernel parameters to PXE booted instances by:

- Modifying the `[pxe]/kernel_append_params` configuration option, for example:

```
[pxe]
kernel_append_params = quiet splash
```

Note

The option was called `pxe_append_params` before the Xena cycle.

- Copying a template from shipped templates to another place, for example:

```
https://opendev.org/openstack/ironic/src/branch/master/ironic/drivers/
↪modules/pxe_config.template
```

  Making the modifications and pointing to the custom template via the configuration options: `[pxe]/pxe_config_template` and `[pxe]/uefi_pxe_config_template`.

Local boot

For local boot instances, users can make use of configuration drive (see *Enabling the configuration drive (configdrive)*) to pass a custom script to append kernel parameters when creating an instance. This is more flexible and can vary per instance. Here is an example for grub2 with ubuntu, users can customize it to fit their use case:

```
#!/usr/bin/env python
import os

# Default grub2 config file in Ubuntu
grub_file = '/etc/default/grub'
# Add parameters here to pass to instance.
kernel_parameters = ['quiet', 'splash']
grub_cmd = 'GRUB_CMDLINE_LINUX'
old_grub_file = grub_file+'~'
os.rename(grub_file, old_grub_file)
cmdline_existed = False
with open(grub_file, 'w') as writer, \
       open(old_grub_file, 'r') as reader:
       for line in reader:
           key = line.split('=')[0]
           if key == grub_cmd:
               #If there is already some value:
               if line.strip()[-1] == '"':
                   line = line.strip()[:-1] + ' ' + ' '.join(kernel_
↪parameters) + '"'
               cmdline_existed = True
           writer.write(line)
       if not cmdline_existed:
           line = grub_cmd + '=' + '"' + ' '.join(kernel_parameters) + '"'
           writer.write(line)

os.remove(old_grub_file)
os.system('update-grub')
os.system('reboot')
```

Console

In order to change default console configuration in the Bare Metal service configuration file (`[pxe]` section in `/etc/ironic/ironic.conf`), include the serial port terminal and serial speed. Serial speed must be the same as the serial configuration in the BIOS settings, so that the operating system boot process can be seen in the serial console. Following examples represent possible parameters for serial console respectively.

- Node serial console. The console parameter `console=ttyS0,115200n8` uses `ttyS0` for console output at `115200bps, 8bit, non-parity`, e.g.:

  ```
  [pxe]

  # Additional append parameters for baremetal PXE boot.
  kernel_append_params = nofb vga=normal console=ttyS0,115200n8
  ```

For detailed information on how to add consoles see the reference documents kernel params and serial console. In case of local boot the Bare Metal service is not able to control kernel boot parameters. To configure console locally, follow Local boot section above.

Boot mode support

Some of the bare metal hardware types (namely, `redfish`, `ilo` and generic `ipmi`) support setting boot mode (Legacy BIOS or UEFI).

Note

Setting boot mode support in generic `ipmi` driver is coupled with setting boot device. That makes boot mode support in the `ipmi` driver incomplete.

Note

In this chapter we will distinguish *ironic node* from *bare metal node*. The difference is that *ironic node* refers to a logical node, as it is configured in ironic, while *bare metal node* indicates the hardware machine that ironic is managing.

The following rules apply in order when ironic manages node boot mode:

- If the hardware type (or bare metal node) does not implement reading current boot mode of the bare metal node, then ironic assumes that boot mode is not set on the bare metal node
- If boot mode is not set on ironic node and bare metal node boot mode is unknown (not set, cant be read etc.), ironic node boot mode is set to the value of the *[deploy]/default_boot_mode* option
- If boot mode is set on a bare metal node, but is not set on ironic node, bare metal node boot mode is set on ironic node
- If boot mode is set on ironic node, but is not set on the bare metal node, ironic node boot mode is attempted to be set on the bare metal node (failure to set boot mode on the bare metal node will not fail ironic node deployment)
- If different boot modes appear on to be set ironic node and on the bare metal node, ironic node boot mode is attempted to be set on the bare metal node (failure to set boot mode on the bare metal node will fail ironic node deployment)

Warning

If a bare metal node does not support setting boot mode, then the operator needs to make sure that boot mode configuration is consistent between ironic node and the bare metal node.

The boot modes can be configured in the Bare Metal service in the following way:

- Only one boot mode (either `uefi` or `bios`) can be configured for the node.
- If the operator wants a node to boot always in `uefi` mode or `bios` mode, then they may use `capabilities` parameter within `properties` field of an bare metal node. The operator must manually set the appropriate boot mode on the bare metal node.

  To configure a node in `uefi` mode, then set `capabilities` as below:

```
openstack baremetal node set <node-uuid> --property capabilities='boot_
↪mode:uefi'
```

Conversely, to configure a node in `bios` mode, then set the `capabilities` as below:

```
openstack baremetal node set <node-uuid> --property capabilities='boot_
↪mode:bios'
```

Note

The Ironic project changed the default boot mode setting for nodes from `bios` to `uefi` during the Yoga development cycle.

Nodes having `boot_mode` set to `uefi` may be requested by adding an `extra_spec` to the Compute service flavor:

```
openstack flavor set ironic-test-3 --property capabilities:boot_mode="uefi"
openstack server create boot --flavor ironic-test-3 --image test-image␣
↪instance-1
```

If `capabilities` is used in `extra_spec` as above, nova scheduler (`ComputeCapabilitiesFilter`) will match only bare metal nodes which have the `boot_mode` set appropriately in `properties/capabilities`. It will filter out rest of the nodes.

The above facility for matching in the Compute service can be used in heterogeneous environments where there is a mix of `uefi` and `bios` machines, and operator wants to provide a choice to the user regarding boot modes. If the flavor doesnt contain `boot_mode` and `boot_mode` is configured for bare metal nodes, then nova scheduler will consider all nodes and user may get either `bios` or `uefi` machine.

Some hardware support setting secure boot mode, see *UEFI secure boot mode* for details.

Choosing the disk label

Note

The term `disk label` is historically used in Ironic and was taken from parted. Apparently everyone seems to have a different word for `disk label` - these are all the same thing: disk type, partition table, partition map and so on

Ironic allows operators to choose which disk label they want their bare metal node to be deployed with when Ironic is responsible for partitioning the disk; therefore choosing the disk label does not apply when the image being deployed is a `whole disk image`.

There are some edge cases where someone may want to choose a specific disk label for the images being deployed, including but not limited to:

- For machines in `bios` boot mode with disks larger than 2 terabytes its recommended to use a `gpt` disk label. Thats because a capacity beyond 2 terabytes is not addressable by using the MBR partitioning type. But, although GPT claims to be backward compatible with legacy BIOS systems thats not always the case.

- Operators may want to force the partitioning to be always MBR (even if the machine is deployed with boot mode `uefi`) to avoid breakage of applications and tools running on those instances.

The disk label can be configured in two ways; when Ironic is used with the Compute service or in standalone mode. The following bullet points and sections will describe both methods:

- When no disk label is provided Ironic will configure it according to the boot mode (see *Boot mode support*); `bios` boot mode will use `msdos` and `uefi` boot mode will use `gpt`.
- Only one disk label - either `msdos` or `gpt` - can be configured for the node.

Warning

If the host is in UEFI boot mode, use of `disk_label` is redundant, and may cause deployments to fail unexpectedly if the node is *not* explicitly set to boot in UEFI mode. Use of appropriate boot mode is highly recommended.

When used with Compute service

When Ironic is used with the Compute service the disk label should be set to nodes `properties/capabilities` field and also to the flavor which will request such capability, for example:

```
baremetal node set <node-uuid> --property capabilities='disk_label:gpt'
```

As for the flavor:

```
openstack flavor set baremetal --property capabilities:disk_label="gpt"
```

When used in standalone mode

When used without the Compute service, the disk label should be set directly to the nodes `instance_info` field, as below:

```
baremetal node set <node-uuid> --instance-info capabilities='{"disk_label":
↪"gpt"}'
```

Notifications

The Bare Metal service supports the emission of notifications, which are messages sent on a message broker (like RabbitMQ or anything else supported by the oslo messaging library) that indicate various events which occur, such as when a node changes power states. These can be consumed by an external service reading from the message bus. For example, Searchlight is an OpenStack service that uses notifications to index (and make searchable) resources from the Bare Metal service.

Notifications are disabled by default. For a complete list of available notifications and instructions for how to enable them, see the *Notifications*.

2.1.12 Troubleshooting

Once all the services are running and configured properly, and a node has been enrolled with the Bare Metal service and is in the `available` provision state, the Compute service should detect the node as an available resource and expose it to the scheduler.

Note

There is a delay, and it may take up to a minute (one periodic task cycle) for the Compute service to recognize any changes in the Bare Metal services resources (both additions and deletions).

In addition to watching `nova-compute` log files, you can see the available resources by looking at the list of Compute hypervisors. The resources reported therein should match the bare metal node properties, and the Compute service flavor.

Here is an example set of commands to compare the resources in Compute service and Bare Metal service:

```
$ baremetal node list
+--------------------------------------+---------------+-------------+--------
↪-----------+-------------+
| UUID                                 | Instance UUID | Power State |␣
↪Provisioning State | Maintenance |
+--------------------------------------+---------------+-------------+--------
↪-----------+-------------+
| 86a2b1bb-8b29-4964-a817-f90031debddb | None          | power off   |␣
↪available          | False       |
+--------------------------------------+---------------+-------------+--------
↪-----------+-------------+

$ baremetal node show 86a2b1bb-8b29-4964-a817-f90031debddb
+------------------------+------------------------------------------------------
↪------------------+
| Property               | Value                                                ␣
↪                  |
+------------------------+------------------------------------------------------
↪------------------+
| instance_uuid          | None                                                 ␣
↪                  |
| properties             | {u'memory_mb': u'1024', u'cpu_arch': u'x86_64', u
↪'local_gb': u'10'}  |
| maintenance            | False                                                ␣
↪                  |
| driver_info            | { [SNIP] }                                           ␣
↪                  |
| extra                  | {}                                                   ␣
↪                  |
| last_error             | None                                                 ␣
↪                  |
| created_at             | 2014-11-20T23:57:03+00:00                            ␣
↪                  |
| target_provision_state | None                                                 ␣
↪                  |
| driver                 | ipmi                                                 ␣
↪                  |
| updated_at             | 2014-11-21T00:47:34+00:00                            ␣
↪                  |
```

(continues on next page)

(continued from previous page)

```
| instance_info          | {}                                                  ␣
↪                   |
| chassis_uuid           | 7b49bbc5-2eb7-4269-b6ea-3f1a51448a59                ␣
↪                   |
| provision_state        | available                                           ␣
↪                   |
| reservation            | None                                                ␣
↪                   |
| power_state            | power off                                           ␣
↪                   |
| console_enabled        | False                                               ␣
↪                   |
| uuid                   | 86a2b1bb-8b29-4964-a817-f90031debddb                ␣
↪                   |
+------------------------+-----------------------------------------------------
↪------------------+

$ nova hypervisor-list
+--------------------------------------+---------------------------------------
↪+-------+---------+
| ID                                   | Hypervisor hostname                 ␣
↪| State | Status  |
+--------------------------------------+---------------------------------------
↪+-------+---------+
| 584cfdc8-9afd-4fbb-82ef-9ff25e1ad3f3 | 86a2b1bb-8b29-4964-a817-f90031debddb␣
↪| up    | enabled |
+--------------------------------------+---------------------------------------
↪+-------+---------+

$ nova hypervisor-show 584cfdc8-9afd-4fbb-82ef-9ff25e1ad3f3
+-------------------------+--------------------------------------+
| Property                | Value                                |
+-------------------------+--------------------------------------+
| cpu_info                | baremetal cpu                        |
| current_workload        | 0                                    |
| disk_available_least    | -                                    |
| free_disk_gb            | 10                                   |
| free_ram_mb             | 1024                                 |
| host_ip                 | [ SNIP ]                             |
| hypervisor_hostname     | 86a2b1bb-8b29-4964-a817-f90031debddb |
| hypervisor_type         | ironic                               |
| hypervisor_version      | 1                                    |
| id                      | 1                                    |
| local_gb                | 10                                   |
| local_gb_used           | 0                                    |
| memory_mb               | 1024                                 |
| memory_mb_used          | 0                                    |
| running_vms             | 0                                    |
| service_disabled_reason | -                                    |
```

(continues on next page)

(continued from previous page)

```
| service_host            | my-test-host                           |
| service_id              | 6                                      |
| state                   | up                                     |
| status                  | enabled                                |
| vcpus                   | 1                                      |
| vcpus_used              | 0                                      |
+-------------------------+----------------------------------------+
```

Maintenance mode

Maintenance mode may be used if you need to take a node out of the resource pool. Putting a node in maintenance mode will prevent Bare Metal service from executing periodic tasks associated with the node. This will also prevent Compute service from placing a tenant instance on the node by not exposing the node to the nova scheduler. Nodes can be placed into maintenance mode with the following command.

```
$ baremetal node maintenance set $NODE_UUID
```

A maintenance reason may be included with the optional `--reason` command line option. This is a free form text field that will be displayed in the `maintenance_reason` section of the `node show` command.

```
$ baremetal node maintenance set $UUID --reason "Need to add ram."

$ baremetal node show $UUID

+------------------------+----------------------------------------+
| Property               | Value                                  |
+------------------------+----------------------------------------+
| target_power_state     | None                                   |
| extra                  | {}                                     |
| last_error             | None                                   |
| updated_at             | 2015-04-27T15:43:58+00:00              |
| maintenance_reason     | Need to add ram.                       |
| ...                    | ...                                    |
| maintenance            | True                                   |
| ...                    | ...                                    |
+------------------------+----------------------------------------+
```

To remove maintenance mode and clear any `maintenance_reason` use the following command.

```
$ baremetal node maintenance unset $NODE_UUID
```

2.1.13 Next steps

Your OpenStack environment now includes the Bare Metal service.

2.1.14 Create user images for the Bare Metal service

The content has been migrated, please see *Creating instance images*.

2.2 Bare Metal Service Upgrade Guide

This document outlines various steps and notes for operators to consider when upgrading their ironic-driven clouds from previous versions of OpenStack.

The Bare Metal (ironic) service is tightly coupled with the ironic driver that is shipped with the Compute (nova) service. Some special considerations must be taken into account when upgrading your cloud.

Both offline and rolling upgrades are supported.

2.2.1 Plan your upgrade

- Rolling upgrades are supported for the Bare Metal service, allowing upgrades with minimal to no downtime of the Bare Metal API.
- Upgrades are supported between named releases in the framing of SLURP.
- The release notes should always be read carefully when upgrading the Bare Metal service. Specific upgrade steps and considerations are documented there.
- The Bare Metal service should always be upgraded before the Compute service.

> **Note**
>
> The ironic virt driver in nova always uses a specific version of the ironic REST API. This API version may be one that was introduced in the same development cycle, so upgrading nova first may result in nova being unable to use the Bare Metal API.

- Make a backup of your database. Ironic does not support downgrading of the database. Hence, in case of upgrade failure, restoring the database from a backup is the only choice.
- Before starting your upgrade, it is best to ensure that all nodes have reached, or are in, a stable `provision_state`. Nodes in states with long running processes such as deploying or cleaning, may fail, and may require manual intervention to return them to the available hardware pool. This is most likely in cases where a timeout has occurred or a service was terminated abruptly. For a visual diagram detailing states and possible state transitions, please see *Bare Metal State Machine*.

2.2.2 Offline upgrades

In an offline (or cold) upgrade, the Bare Metal service is not available during the upgrade, because all the services have to be taken down.

When upgrading the Bare Metal service, the following steps should always be taken in this order:

1. upgrade the ironic-python-agent image
2. update ironic code, without restarting services
3. run database schema migrations via `ironic-dbsync upgrade`
4. restart ironic-conductor and ironic-api services

Once the above is done, do the following:

- update any applicable configuration options to stop using any deprecated features or options, and perform any required work to transition to alternatives. All the deprecated features and options will be supported for one release cycle, so should be removed before your next upgrade is performed.

- upgrade python-ironicclient along with any other services connecting to the Bare Metal service as a client, such as nova-compute
- run the `ironic-dbsync online_data_migrations` command to make sure that data migrations are applied. The command lets you limit the impact of the data migrations with the `--max-count` option, which limits the number of migrations executed in one run. You should complete all of the migrations as soon as possible after the upgrade.

> **Warning**
>
> You will not be able to start an upgrade to the release after this one, until this has been completed for the current release. You need to complete the current releases data migrations before upgrading to the next release. If this is not done, you will not be able to upgrade to the next release it will not be possible to execute the next releases database schema updates.

2.2.3 Rolling upgrades

To Reduce downtime, the services can be upgraded in a rolling fashion, meaning to upgrade one or a few services at a time to minimize impact.

Rolling upgrades are supported for the Bare Metal service. This feature makes it possible to upgrade between consecutive releases with minimal to no downtime of the Bare Metal API.

Requirements

To facilitate an upgrade in a rolling fashion, you need to have a highly-available deployment consisting of at least two ironic-api and two ironic-conductor services. Use of a load balancer to balance requests across the ironic-api services is recommended, as it allows for a minimal impact to end users.

Concepts

There are four aspects of the rolling upgrade process to keep in mind:

- API and RPC version pinning, and versioned object backports
- online data migrations
- graceful service shutdown
- API load balancer draining

API & RPC version pinning and versioned object backports

Through careful RPC versioning, newer services are able to talk to older services (and vice-versa). The `[DEFAULT]/pin_release_version` configuration option is used for this. It should be set (pinned) to the release version that the older services are using. The newer services will backport RPC calls and objects to their appropriate versions from the pinned release. If the `IncompatibleObjectVersion` exception occurs, it is most likely due to an incorrect or unspecified `[DEFAULT]/pin_release_version` configuration value. For example, when `[DEFAULT]/pin_release_version` is not set to the older release version, no conversion will happen during the upgrade.

For the ironic-api service, the API version is pinned via the same `[DEFAULT]/pin_release_version` configuration option as above. When pinned, the new ironic-api services will not service any API requests with Bare Metal API versions that are higher than what the old ironic-api services support. HTTP status

code 406 is returned for such requests. This prevents new features (available in new API versions) from being used until after the upgrade has been completed.

Online data migrations

To make database schema migrations less painful to execute, we have implemented process changes to facilitate upgrades.

- All data migrations are banned from schema migration scripts.
- Schema migration scripts only update the database schema.
- Data migrations must be done at the end of the rolling upgrade process, after the schema migration and after the services have been upgraded to the latest release.

All data migrations are performed using the `ironic-dbsync online_data_migrations` command. It can be run as a background process so that it does not interrupt running services; however it must be run to completion for a cold upgrade if the intent is to make use of new features immediately.

(You would also execute the same command with services turned off if you are doing a cold upgrade).

This data migration must be completed. If not, you will not be able to upgrade to future releases. For example, if you upgraded between releases but did not do the data migrations, you will not be able to upgrade to the next release. (More precisely, you will not be able to apply the next releases schema migrations.)

Graceful conductor service shutdown

The ironic-conductor service is a Python process listening for messages on a message queue. When the operator sends the `SIGTERM` signal to the process, the service stops consuming messages from the queue, so that no additional work is picked up. It completes any outstanding work and then terminates. During this process, messages can be left on the queue and will be processed after the Python process starts back up. This gives us a way to shutdown a service using older code, and start up a service using newer code with minimal impact.

Note

This was tested with RabbitMQ messaging backend and may vary with other backends.

Nodes that are being acted upon by an ironic-conductor process, which are not in a stable state, will be put into a failed state when *conductor.graceful_shutdown_timeout* is reached. Node failures that occur during an upgrade are likely due to timeouts, resulting from delays involving messages being processed and acted upon by a conductor during long running, multi-step processes such as deployment or cleaning.

Drain conductor service shutdown

A drain shutdown is similar to graceful shutdown, differing in the following ways:

- Triggered by sending signal `SIGUSR2` to the process instead of `SIGTERM`
- The timeout for process termination is determined by *DEFAULT.drain_shutdown_timeout* instead of *conductor.graceful_shutdown_timeout*

DEFAULT.drain_shutdown_timeout is set long enough so that any node in a not stable state will have time to reach a stable state (complete or failed) before the ironic-conductor process terminates.

API load balancer draining

If you are using a load balancer for the ironic-api services, we recommend that you redirect requests to the new API services and drain off of the ironic-api services that have not yet been upgraded.

Rolling upgrade process

Before maintenance window

- Upgrade the ironic-python-agent image
- Using the new release (ironic code), execute the required database schema updates by running the database upgrade command: `ironic-dbsync upgrade`. These schema change operations should have minimal or no effect on performance, and should not cause any operations to fail (but please check the release notes). You can:
  - install the new release on an existing system
  - install the new release in a new virtualenv or a container

  At this point, new columns and tables may exist in the database. These database schema changes are done in a way that both the old and new (N and N+1) releases can perform operations against the same schema.

Note

Ironic bases its API, RPC and object storage format versions on the `[DEFAULT]/pin_release_version` configuration option. It is advisable to automate the deployment of changes in configuration files to make the process less error prone and repeatable.

During maintenance window

1. All ironic-conductor services should be upgraded first. Ensure that at least one ironic-conductor service is running at all times. For every ironic-conductor, either one by one or a few at a time:
   - shut down the service. Messages from the ironic-api services to the conductors are load-balanced by the message queue and a hash-ring, so the only thing you need to worry about is to shut the service down gracefully (using `SIGTERM` signal) to make sure it will finish all the requests being processed before shutting down.
   - upgrade the installed version of ironic and dependencies
   - set the `[DEFAULT]/pin_release_version` configuration option value to the version you are upgrading from (that is, the old version). Based on this setting, the new ironic-conductor services will downgrade any RPC communication and data objects to conform to the old service. For example, when upgrading between consecutive releases, set this value to `ocata`.
   - start the service
2. The next service to upgrade is ironic-api. Ensure that at least one ironic-api service is running at all times. You may want to start another temporary instance of the older ironic-api to handle the load while you are upgrading the original ironic-api services. For every ironic-api service, either one by one or a few at a time:

- in HA deployment you are typically running them behind a load balancer (for example HAProxy), so you need to take the service instance out of the balancer
- shut it down
- upgrade the installed version of ironic and dependencies
- set the `[DEFAULT]/pin_release_version` configuration option value to the version you are upgrading from (that is, the old version). Based on this setting, the new ironic-api services will downgrade any RPC communication and data objects to conform to the old service. In addition, the new services will return HTTP status code 406 for any requests with newer API versions that the old services did not support. This prevents new features (available in new API versions) from being used until after the upgrade has been completed. For example, when upgrading between consecutive releases, set this value to `ocata`.
- restart the service
- add it back into the load balancer

After upgrading all the ironic-api services, the Bare Metal service is running in the new version but with downgraded RPC communication and database object storage formats. New features (in new API versions) are not supported, because they could fail when objects are in the downgraded object formats and some internal RPC API functions may still not be available.

3. For all the ironic-conductor services, one at a time:
   - remove the `[DEFAULT]/pin_release_version` configuration option setting
   - restart the ironic-conductor service
4. For all the ironic-api services, one at a time:
   - remove the `[DEFAULT]/pin_release_version` configuration option setting
   - restart the ironic-api service

After maintenance window

Now that all the services are upgraded, the system is able to use the latest version of the RPC protocol and able to access all the features of the new release.

- Update any applicable configuration options to stop using any deprecated features or options, and perform any required work to transition to alternatives. All the deprecated features and options will be supported for one release cycle, so should be removed before your next upgrade is performed.
- Upgrade `python-ironicclient` along with other services connecting to the Bare Metal service as a client, such as `nova-compute`.

> **Warning**
>
> A `nova-compute` instance tries to attach VIFs to all active instances on start up. Make sure that for all active nodes there is at least one running `ironic-conductor` process to manage them. Otherwise the instances will be moved to the `ERROR` state on the `nova-compute` start up.

- Run the `ironic-dbsync online_data_migrations` command to make sure that data migrations are applied. The command lets you limit the impact of the data migrations with the

`--max-count` option, which limits the number of migrations executed in one run. You should complete all of the migrations as soon as possible after the upgrade.

Warning

Note that you will not be able to start an upgrade to the next release after this one, until this has been completed for the current release. For example, as part of upgrading between releases, you need to complete the current releases data migrations. If this not done, you will not be able to upgrade to the next release it will not be possible to execute the next releases database schema updates.

CHAPTER

THREE

USER GUIDE

3.1 Bare Metal Service User Guide

Ironic is an OpenStack project which provisions bare metal (as opposed to virtual) machines. It may be used independently or as part of an OpenStack Cloud, and integrates with the OpenStack Identity (keystone), Compute (nova), Network (neutron), Image (glance) and Object (swift) services.

When the Bare Metal service is appropriately configured with the Compute and Network services, it is possible to provision both virtual and physical machines through the Compute services API. However, the set of instance actions is limited, arising from the different characteristics of physical servers and switch hardware. For example, live migration can not be performed on a bare metal instance.

The community maintains reference drivers that leverage open-source technologies (eg. PXE and IPMI) to cover a wide range of hardware. Ironics pluggable driver architecture also allows hardware vendors to write and contribute drivers that may improve performance or add functionality not provided by the community drivers.

3.1.1 Understanding Bare Metal service

This page has moved.

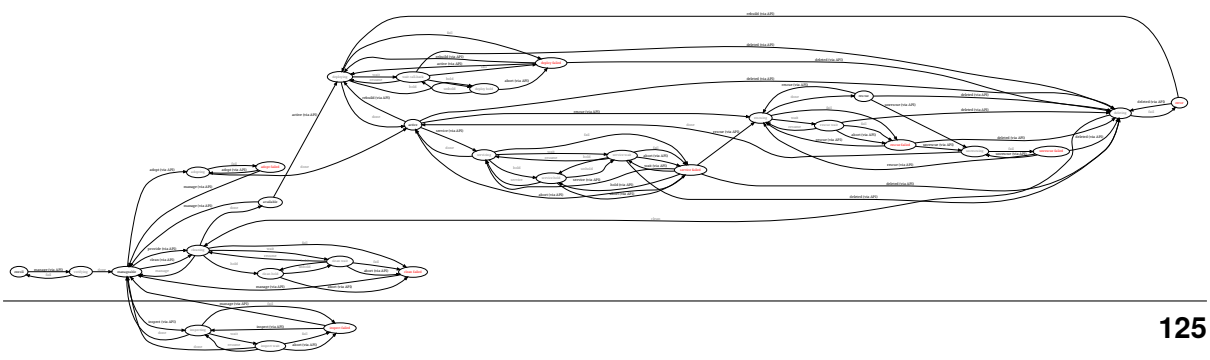
As part of the Ironic documentation improvements, this content has been consolidated. You can now find the updated content in the *Overview of Ironic* section.

3.1.2 Bare Metal State Machine

State Machine Diagram

The diagram below shows the provisioning states that an Ironic node goes through during the lifetime of a node. The diagram also depicts the events that transition the node to different states.

Stable states are highlighted with a thicker border. All transitions from stable states are initiated by API requests. There are a few other API-initiated-transitions that are possible from non-stable states. The events for these API-initiated transitions are indicated with (via API). Internally, the conductor initiates the other transitions (depicted in gray).



Please click the image above to view

the diagram at its full size, as the presence in the documentation results in it being scaled down.

Note

There are aliases for some transitions:

- `deploy` is an alias for `active`.
- `undeploy` is an alias for `deleted`.

Enrollment and Preparation

enroll (stable state)

This is the state that all nodes start off in when created using API version 1.11 or newer. When a node is in the `enroll` state, the only thing Ironic knows about it is that it exists, and Ironic cannot take any further action by itself. Once a node has its driver/interfaces and their required information set in `node.driver_info`, the node can be transitioned to the `verifying` state by setting the nodes provision state using the `manage` verb.

See *Enrolling hardware with Ironic* for information on enrolling nodes.

verifying

Ironic will validate that it can manage the node using the information given in `node.driver_info` and with either the driver/hardware type and interfaces it has been assigned. This involves going out and confirming that the credentials work to access whatever node control mechanism they talk to.

manageable (stable state)

Once Ironic has verified that it can manage the node using the driver/interfaces and credentials passed in at node create time, the node will be transitioned to the `manageable` state. From `manageable`, nodes can transition to:

- `manageable` (through `cleaning`) by setting the nodes provision state using the `clean` verb.
- `manageable` (through `inspecting`) by setting the nodes provision state using the `inspect` verb.
- `available` (through `cleaning` if automatic cleaning is enabled) by setting the nodes provision state using the `provide` verb.
- `active` (through `adopting`) by setting the nodes provision state using the `adopt` verb.

`manageable` is the state that a node should be moved into when any updates need to be made to it such as changes to fields in driver_info and updates to networking information on Ironic ports assigned to the node.

`manageable` is also the only stable state that can be transitioned to, from these failure states:

- `adopt failed`
- `clean failed`
- `inspect failed`

inspecting
`inspecting` will utilize node introspection to update hardware-derived node properties to reflect the current state of the hardware. Typically, the node will transition to `manageable` if inspection is synchronous, or `inspect wait` if asynchronous. The node will transition to `inspect failed` if error occurred.

See /admin/inspection for information about inspection.

inspect wait
This is the provision state used when an asynchronous inspection is in progress. A successfully inspected node shall transition to `manageable` state.

inspect failed
This is the state a node will move into when inspection of the node fails. From here the node can be transitioned to:

- `inspecting` by setting the nodes provision state using the `inspect` verb.
- `manageable` by setting the nodes provision state using the `manage` verb.

cleaning
Nodes in the `cleaning` state are being scrubbed and reprogrammed into a known configuration.

When a node is in the `cleaning` state it means that the conductor is executing the clean step (for out-of-band clean steps) or preparing the environment (building PXE configuration files, configuring the DHCP, etc) to boot the ramdisk for running in-band clean steps.

clean wait
Just like the `cleaning` state, the nodes in the `clean wait` state are being scrubbed and reprogrammed. The difference is that in the `clean wait` state the conductor is waiting for the ramdisk to boot or the clean step that is running in-band to finish.

The cleaning process of a node in the `clean wait` state can be interrupted by setting the nodes provision state using the `abort` verb if the task that is running allows it.

Deploy and Undeploy

available (stable state)
After nodes have been successfully preconfigured and cleaned, they are moved into the `available` state and are ready to be provisioned. From `available`, nodes can transition to:

- `active` (through `deploying`) by setting the nodes provision state using the `active` or `deploy` verbs.
- `manageable` by setting the nodes provision state using the `manage` verb.

deploying
Nodes in `deploying` are being prepared to run a workload on them. This consists of running a series of tasks, such as:

- Setting appropriate BIOS configurations
- Partitioning drives and laying down file systems.
- Creating any additional resources (node-specific network config, a config drive partition, etc.) that may be required by additional subsystems.

See *Deploying with Bare Metal service* and *Using deploy steps and templates* for information about deploying nodes.

wait call-back
: Just like the `deploying` state, the nodes in `wait call-back` are being deployed. The difference is that in `wait call-back` the conductor is waiting for the ramdisk to boot or execute parts of the deployment that need to run in-band on the node (for example, installing the bootloader, or writing the image to the disk).

  The deployment of a node in `wait call-back` can be interrupted by setting the nodes provision state using the `deleted` or `undeploy` verbs.

deploy failed
: This is the state a node will move into when a deployment fails, for example a timeout waiting for the ramdisk to PXE boot. From here the node can be transitioned to:

  - `active` (through `deploying`) by setting the nodes provision state using the `active`, `deploy` or `rebuild` verbs.
  - `available` (through `deleting` and `cleaning`) by setting the nodes provision state using the `deleted` or `undeploy` verbs.

active (stable state)
: Nodes in `active` have a workload running on them. Ironic may collect out-of-band sensor information (including power state) on a regular basis. Nodes in `active` can transition to:

  - `available` (through `deleting` and `cleaning`) by setting the nodes provision state using the `deleted` or `undeploy` verbs.
  - `active` (through `deploying`) by setting the nodes provision state using the `rebuild` verb.
  - `rescue` (through `rescuing`) by setting the nodes provision state using the `rescue` verb.

deleting
: Nodes in `deleting` state are being torn down from running an active workload. In `deleting`, Ironic tears down and removes any configuration and resources it added in `deploying` or `rescuing`.

error (stable state)
: This is the state a node will move into when deleting an active deployment fails. From `error`, nodes can transition to:

  - `available` (through `deleting` and `cleaning`) by setting the nodes provision state using the `deleted` or `undeploy` verbs.

adopting
: This state allows Ironic to take over management of a baremetal node with an existing workload on it. Ordinarily when a baremetal node is enrolled and managed by Ironic, it must transition through `cleaning` and `deploying` to reach `active` state. However, those baremetal nodes that have an existing workload on them, do not need to be deployed or cleaned again, so this transition allows these nodes to move directly from `manageable` to `active`.

  See *Node adoption* for information about this feature.

Rescue

rescuing
: Nodes in `rescuing` are being prepared to perform rescue operations. This consists of running a series of tasks, such as:

  - Setting appropriate BIOS configurations.

- Creating any additional resources (node-specific network config, etc.) that may be required by additional subsystems.

See *Rescue Mode* for information about this feature.

rescue wait
Just like the `rescuing` state, the nodes in `rescue wait` are being rescued. The difference is that in `rescue wait` the conductor is waiting for the ramdisk to boot or execute parts of the rescue that need to run in-band on the node (for example, setting the password for user named `rescue`).

The rescue operation of a node in `rescue wait` can be aborted by setting the nodes provision state using the `abort` verb.

rescue failed
This is the state a node will move into when a rescue operation fails, for example a timeout waiting for the ramdisk to PXE boot. From here the node can be transitioned to:

- `rescue` (through `rescuing`) by setting the nodes provision state using the `rescue` verb.
- `active` (through `unrescuing`) by setting the nodes provision state using the `unrescue` verb.
- `available` (through `deleting`) by setting the nodes provision state using the `deleted` verb.

rescue (stable state)
Nodes in `rescue` have a rescue ramdisk running on them. Ironic may collect out-of-band sensor information (including power state) on a regular basis. Nodes in `rescue` can transition to:

- `active` (through `unrescuing`) by setting the nodes provision state using the `unrescue` verb.
- `available` (through `deleting`) by setting the nodes provision state using the `deleted` verb.

unrescuing
Nodes in `unrescuing` are being prepared to transition to `active` state from `rescue` state. This consists of running a series of tasks, such as setting appropriate BIOS configurations such as changing boot device.

unrescue failed
This is the state a node will move into when an unrescue operation fails. From here the node can be transitioned to:

- `rescue` (through `rescuing`) by setting the nodes provision state using the `rescue` verb.
- `active` (through `unrescuing`) by setting the nodes provision state using the `unrescue` verb.
- `available` (through `deleting`) by setting the nodes provision state using the `deleted` verb.

Servicing

servicing
Nodes in the `servicing` state are nodes that are having service performed on them. This service is similar to cleaning, but is performed on nodes currently in `active` state and returns them to `active` state when complete.

When a node is in the `servicing` state it means that the conductor is executing the service step or preparing the environment to execute the step.

See *Node servicing* for more details on Node servicing.

service wait
Just like the `servicing` state, the nodes in the `service wait` state are being serviced with service

steps. The difference is that in the `service wait` state the conductor is waiting for the ramdisk to boot or the clean step, which is running in-band to finish.

The servicing of a node in the `service wait` state can be interrupted by setting the nodes provision state using the `abort` verb if the task that is running allows it.

service failed
This is the state a node will move into when a service operation fails, for example a timeout waiting for the ramdisk to PXE boot. From here the node can be transitioned to:

- `active` (through `servicing`) by setting the nodes provision state using the `service` verb.
- `rescue` (through `rescuing`) by setting the nodes provision state using the `rescue` verb.
- `active` by setting the nodes provision state using the `abort` verb.

Note

Before attempting to abort a servicing operation on a node either in `service wait` or `service failed` state, the user must check the remote console of the machine as a precaution and ensure there is no active firmware update running on the node. Aborting service may result in a power cycle operation that may interrupt the running update, causing irreversible damage to the hardware. Once it is confirmed no update operation is running, using `abort` verb is not expected to cause any issues.

3.1.3 Creating instance images

Bare Metal provisioning requires two sets of images: the deploy images and the user images. The *deploy images* are used by the Bare Metal service to prepare the bare metal server for actual OS deployment. Whereas the user images are installed on the bare metal server to be used by the end user. There are two types of user images:

partition images
contain only the contents of the root partition. Additionally, two more images are used together with them when booting from network: an image with a kernel and with an initramfs.

Warning

To use partition images, Grub2 must be installed in the image. This is *not* a recommended path.

whole disk images
contain a complete partition table with one or more partitions.

Warning

The kernel/initramfs pair must not be used with whole disk images, otherwise theyll be mistaken for partition images. Whole disk images are the recommended type of images to use.

Many distributions publish their own cloud images. These are usually whole disk images that are built for legacy boot mode (not UEFI), with Ubuntu being an exception (they publish images that work in both modes).

Supported Disk Image Formats

The following formats are tested by Ironic and are expected to work as long as no unknown or unsafe special features are being used

- raw - A file containing bytes as they would exist on a disk or other block storage device. This is the simplest format.
- qcow2 - An updated file format based upon the QEMU Copy-on-Write format.

A special mention exists for `iso` formatted CD images. While Ironic uses the ISO9660 filesystems in some of its processes for aspects such as virtual media, it does *not* support writing them to the remote block storage device.

Image formats we believe may work due to third party reports, but do not test:

- vmdk - A file format derived from the image format originally created by VMware for their hypervisor product line. Specifically we believe a single file VMDK formatted image should work. As there are are several subformats, some of which will not work and may result in unexpected behavior such as failed deployments.
- vdi - A file format used by Oracle VM Virtualbox hypervisor.

As Ironic does not support these formats, their usage is normally blocked due security considerations by default. Please consult with your Ironic Operator.

It is important to highlight that Ironic enforces and matches the file type based upon signature, and not file extension. If there is a mismatch, the input and or remote service records such as in the Image service must be corrected.

disk-image-builder

The disk-image-builder can be used to create user images required for deployment and the actual OS which the user is going to run.

- Install diskimage-builder package (use virtualenv, if you dont want to install anything globally):

```
# pip install diskimage-builder
```

- Build the image your users will run (Ubuntu image has been taken as an example):
  - Partition images

```
$ disk-image-create ubuntu baremetal dhcp-all-interfaces grub2 -o my-
↪image
```

  - Whole disk images

```
$ disk-image-create ubuntu vm dhcp-all-interfaces -o my-image
```

    with an EFI partition:

```
$ disk-image-create ubuntu vm block-device-efi dhcp-all-interfaces -
↪o my-image
```

The partition image command creates `my-image.qcow2`, `my-image.vmlinuz` and `my-image.initrd` files. The `grub2` element in the partition image creation command is only needed if local boot will be used to deploy `my-image.qcow2`, otherwise the images `my-image.vmlinuz` and `my-image.initrd`

will be used for PXE booting after deploying the bare metal with `my-image.qcow2`. For whole disk images only the main image is used.

If you want to use Fedora image, replace `ubuntu` with `fedora` in the chosen command.

To build images for distributions other than ubuntu, follow the build method for Whole disk images.

- Example for Rocky Linux

  If you use the rocky-container element, specify the `openssh-server` and `cloud-init` elements to enable ssh login with a key pair, and specify the `growroot` element to adjust the disk size.

```
$ export DIB_CONTAINERFILE_PODMAN_ROOT=1
$ disk-image-create rocky-container vm dhcp-all-interfaces \
> block-device-efi openssh-server cloud-init growroot -o my-image
```

If you want to use Alma Linux image, replace `rocky-container` with `almalinux-container` in the chosen command.

Virtual machine

Virtual machine software can also be used to build user images. There are different software options available, qemu-kvm is usually a good choice on linux platform, it supports emulating many devices and even building images for architectures other than the host machine by software emulation. VirtualBox is another good choice for non-linux host.

The procedure varies depending on the software used, but the steps for building an image are similar, the user creates a virtual machine, and installs the target system just like what is done for a real hardware. The system can be highly customized like partition layout, drivers or software shipped, etc.

Usually libvirt and its management tools are used to make interaction with qemu-kvm easier, for example, to create a virtual machine with `virt-install`:

```
$ virt-install --name centos8 --ram 4096 --vcpus=2 -f centos8.qcow2 \
> --cdrom CentOS-8-x86_64-1905-dvd1.iso
```

Graphic frontend like `virt-manager` can also be utilized.

The disk file can be used as user image after the system is set up and powered off. The path of the disk file varies depending on the software used, usually its stored in a user-selected part of the local file system. For qemu-kvm or GUI frontend building upon it, its typically stored at `/var/lib/libvirt/images`.

3.1.4 Deploying with Bare Metal service

This guide explains how to use Ironic to deploy nodes without any front-end service, such as OpenStack Compute (nova) or Metal3.

Allocations

Allocation is a way to find and reserve a node suitable for deployment. When an allocation is created, the list of available nodes is searched for a node with the given *resource class* and *traits*, similarly to how it is done in *OpenStack Compute flavors*. Only the resource class is mandatory, for example:

```
$ baremetal allocation create --resource-class baremetal --wait
+----------------+--------------------------------------+
| Field          | Value                                |
```

(continues on next page)

(continued from previous page)

```
+----------------+--------------------------------------+
| candidate_nodes | []                                   |
| created_at      | 2019-04-03T12:18:26+00:00            |
| extra           | {}                                   |
| last_error      | None                                 |
| name            | None                                 |
| node_uuid       | 5d946337-b1d9-4b06-8eda-4fb77e994a0d |
| resource_class  | baremetal                            |
| state           | active                               |
| traits          | []                                   |
| updated_at      | 2019-04-03T12:18:26+00:00            |
| uuid            | e84f5d60-84f1-4701-a635-10ff90e2f3b0 |
+----------------+--------------------------------------+
```

Note

The allocation processing is fast but nonetheless asynchronous. Use the `--wait` argument to wait for the results.

If an allocation is successful, it sets the nodes `instance_uuid` to the allocation UUID. The nodes UUID can be retrieved from the allocations `node_uuid` field.

An allocation is automatically deleted when the associated node is unprovisioned. If you dont provision the node, youre responsible for deleting the allocation.

See the allocation API reference for more information on how to use allocations.

Populating instance information

The nodes `instance_info` field is a JSON object that contains all information required for deploying an instance on bare metal. It has to be populated before deployment and is automatically cleared on tear down.

Image information

You need to specify image information in the nodes `instance_info` (see *Creating instance images*):

- `image_source` - URL of the whole disk or root partition image, mandatory. The following schemes are supported: `http://`, `https://` and `file://`. Files have to be accessible by the conductor. If the scheme is missing, an Image Service (glance) image UUID is assumed.
- In case the image source requires HTTP(s) Basic Authentication `RFC 7616` then the relevant authentication strategy has to be configured as `http_basic` and supplied with credentials in the ironic global config file. Further information about the authentication strategy selection can be found in *HTTP(s) Authentication strategy for user image servers*.
- `root_gb` - size of the root partition, required for partition images.

Note

Older versions of the Bare Metal service used to require a positive integer for `root_gb` even for whole-disk images.

- `image_checksum` - MD5 checksum of the image specified by `image_source`, only required for `http://` images when using *Direct deploy*.

  Other checksum algorithms are supported via the `image_os_hash_algo` and `image_os_hash_value` fields. They may be used instead of the `image_checksum` field.

  Warning

  If your operating system is running in FIPS 140-2 mode, MD5 will not be available, and you **must** use SHA256 or another modern algorithm.

  The checksum can also be a URL to a checksums file, e.g. one generated with:

```
$ cd /path/to/http/root
$ md5sum *.img > checksums
```

- `kernel`, `ramdisk` - HTTP(s) or file URLs of the kernel and initramfs of the target OS. Must be added **only** for partition images and only if network boot is required. Supports the same schemes as `image_source`.

An example for a partition image with local boot:

```
baremetal node set $NODE_UUID \
    --instance-info image_source=http://image.server/my-image.qcow2 \
    --instance-info image_checksum=1f9c0e1bad977a954ba40928c1e11f33 \
    --instance-info image_type=partition \
    --instance-info root_gb=10
```

With a SHA256 hash:

```
baremetal node set $NODE_UUID \
    --instance-info image_source=http://image.server/my-image.qcow2 \
    --instance-info image_os_hash_algo=sha256 \
    --instance-info image_os_hash_
↪value=a64dd95e0c48e61ed741ff026d8c89ca38a51f3799955097c5123b1705ef13d4 \
    --instance-info image_type=partition \
    --instance-info root_gb=10
```

With a whole disk image and a checksum URL:

```
baremetal node set $NODE_UUID \
    --instance-info image_source=http://image.server/my-image.qcow2 \
    --instance-info image_checksum=http://image.server/my-image.qcow2.CHECKSUM
```

Note

Certain hardware types and interfaces may require additional or different fields to be provided. See specific guides under *Drivers, Hardware Types, and Hardware Interfaces for Ironic*.

When using low RAM nodes with `http://` images that are not in the RAW format, you may want them cached locally, converted to raw and served from the conductors HTTP server:

```
baremetal node set $NODE_UUID --instance-info image_download_source=local
```

For software RAID with whole-disk images, the root UUID of the root partition has to be provided so that the bootloader can be correctly installed:

```
baremetal node set $NODE_UUID --instance-info image_rootfs_uuid=<uuid>
```

Capabilities

- *Boot mode* can be specified per instance:

  ```
  baremetal node set $NODE_UUID \
      --instance-info capabilities='{"boot_mode": "uefi"}'
  ```

  Otherwise, the `boot_mode` capability from the nodes `properties` will be used.

  Warning

  The two settings must not contradict each other.

  Note

  Previously ironic used a separate `instance_info/deploy_boot_mode` field instead.

- You can set *root device hints* per instance:

  ```
  baremetal node set $NODE_UUID \
      --instance-info root_device='{"wwn": "0x4000cca77fc4dba1"}'
  ```

  This setting overrides any previous setting in `properties` and will be removed on undeployment.

Overriding a hardware interface

Non-admins with temporary access to a node, may wish to specify different node interfaces. However, allowing them to set these interface values directly on the node is problematic, as there is no automated way to ensure that the original interface values are restored.

In order to temporarily override a hardware interface, simply set the appropriate value in `instance_info`. For example, if youd like to override a nodes storage interface, run the following:

```
baremetal node set $NODE_UUID --instance-info storage_interface=cinder
```

`instance_info` values persist until after a node is cleaned.

Note

This feature is available.

Attaching virtual interfaces

If using the OpenStack Networking service (neutron), you can attach its ports to a node before deployment as VIFs:

```
baremetal node vif attach $NODE_UUID $PORT_UUID
```

Warning

These are **neutron** ports, not **ironic** ports!

VIFs are automatically detached on deprovisioning.

Deployment

1. Validate that all parameters are correct:

```
$ baremetal node validate $NODE_UUID
+------------+--------+----------------------------------------------------
↪-------------+
| Interface  | Result | Reason                                             ␣
↪             |
+------------+--------+----------------------------------------------------
↪-------------+
| boot       | True   |                                                    ␣
↪             |
| console    | False  | Missing 'ipmi_terminal_port' parameter in node's␣
↪driver_info.  |
| deploy     | True   |                                                    ␣
↪             |
| inspect    | True   |                                                    ␣
↪             |
| management | True   |                                                    ␣
↪             |
| network    | True   |                                                    ␣
↪             |
| power      | True   |                                                    ␣
↪             |
| raid       | True   |                                                    ␣
↪             |
| storage    | True   |                                                    ␣
↪             |
+------------+--------+----------------------------------------------------
↪-------------+
```

2. Now you can start the deployment, run:

```
baremetal node deploy $NODE_UUID
```

3. You can also request custom deploy steps, see *Requesting steps* for details.

Deploying with a config drive

The configuration drive is a small image used to store instance-specific metadata and is present to the instance as a disk partition labeled `config-2`. See *Enabling the configuration drive (configdrive)* for a detailed explanation.

A configuration drive can be provided either as a whole ISO 9660 image or as JSON input for building an image. A first-boot service, such as cloud-init, must be running on the instance image for the configuration to be applied.

Building a config drive on the client side

For the format of the configuration drive, Bare Metal service expects a `gzipped` and `base64` encoded ISO 9660 file with a `config-2` label. The baremetal client can generate a configuration drive in the expected format. Pass a directory path containing the files that will be injected into it via the `--config-drive` parameter of the `baremetal node deploy` command, for example:

```
baremetal node deploy $NODE_UUID --config-drive /dir/configdrive_files
```

Note

A configuration drive could also be a data block with a VFAT filesystem on it instead of ISO 9660. But its unlikely that it would be needed since ISO 9660 is widely supported across operating systems.

Building a config drive on the conductor side

In modern versions of `python-ironicclient`, you can request building a configdrive on the server side by providing a JSON with keys `meta_data`, `user_data` and `network_data` (all optional), e.g.:

```
baremetal node deploy $node_identifier \
    --config-drive '{"meta_data": {"hostname": "server1.cluster"}}'
```

Note

When this feature is used, host name defaults to the nodes name or UUID.

SSH public keys can be provided as a mapping:

```
baremetal node deploy $NODE_UUID \
    --config-drive '{"meta_data": {"public_keys": {"0": "ssh key contents"}}}'
```

If using cloud-init, its configuration can be supplied as `user_data`, e.g.:

```
baremetal node deploy $NODE_UUID \
    --config-drive '{"user_data": "#cloud-config\n{\"users\": [{\"name\": ...}
↪]}"}'
```

Warning

User data is a string, not a JSON! Also note that a prefix, such as `#cloud-config`, is required, see user data format.

Some first-boot services support network configuration in the OpenStack network data format. It can be provided in the `network_data` field of the configuration drive.

Ramdisk booting

Advanced operators, specifically ones working with ephemeral workloads, may find it more useful to explicitly treat a node as one that would always boot from a Ramdisk. See *Booting a Ramdisk or an ISO* for details.

3.2 REST API Versioning for Ironic

3.2.1 Versioning

The ironic REST API supports two types of versioning:

- major versions, which have dedicated urls.
- microversions, which can be requested through the use of the `X-OpenStack-Ironic-API-Version` header.

There is only one major version supported currently, v1. As such, most URLs in this documentation are written with the /v1/ prefix.

Starting with the Kilo release, ironic supports microversions. In this context, a version is defined as a string of 2 integers separated by a dot: **X.Y**. Here `X` is a major version, always equal to `1`, and `Y` is a minor version. Server minor version is increased every time the API behavior is changed (note *Exceptions from Versioning*).

Note

Nova versioning documentation has a nice guide for developers on when to bump an API version.

The server indicates its minimum and maximum supported API versions in the `X-OpenStack-Ironic-API-Minimum-Version` and `X-OpenStack-Ironic-API-Maximum-Version` headers respectively, returned with every response. Client may request a specific API version by providing `X-OpenStack-Ironic-API-Version` header with request.

The requested microversion determines both the allowable requests and the response format for all requests. A resource may be represented differently based on the requested microversion.

If no version is requested by the client, the minimum supported version will be assumed. In this way, a client is only exposed to those API features that are supported in the requested (explicitly or implicitly) API version (again note *Exceptions from Versioning*, they are not covered by this rule).

We recommend clients that require a stable API to always request a specific version of API that they have been tested against.

Note

A special value `latest` can be requested instead a numerical microversion, which always requests the newest supported API version from the server.

REST API Versions History

REST API Version History

1.111 (Gazpacho)

Add `available_for_dynamic_portgroup` boolean field to the port object (defaults to `True`) and `dynamic_portgroup` boolean field to the portgroup object. The port field is user-editable via POST and PATCH requests. The portgroup field is read-only to API consumers.

1.110 (Gazpacho)

Add support for aborting deployment in the `wait call-back` (DEPLOYWAIT) state. This provides consistency with other wait states that already support the abort verb, allowing operators to issue `abort` on a node in DEPLOYWAIT state to transition it to DEPLOYFAIL. The node remains in DEPLOYFAIL state without triggering automated cleaning, allowing operators to investigate the issue or retry deployment.

1.109 (Gazpacho)

Add `health` field to the node object.

1.108 (Gazpacho)

Add support for `disable_ramdisk` parameter to provisioning endpoint `/v1/nodes/{node_ident}/states/provision` for the service verb.

1.107 (Gazpacho)

Add the `x-openstack-request-id` header to all responses. This header mirrors the `openstack-request-id` header.

1.106 (Gazpacho)

Add ability to filter on a node shard when listing portgroups.

1.105 (Gazpacho)

Removes support to set an OVN VTEP endpoint for `local_link_information`. Investigation by multiple contributors and maintainers realized that this experiemntal feature was not scalable and relies upon a manual ovn vtep endpoint service, and as such has been removed.

1.104 (Gazpacho)

Add a new `instance_name` field to the node resource. This field provides a human-readable name for the instance deployed on a node and is automatically synchronized with `instance_info.display_name` for backward compatibility with Nova.

1.103 (Gazpacho)

Add a category field to the Portgroup object.

1.102 (Gazpacho)

Add a physical_network field to the Portgroup object.

1.101 (Flamingo)

Add a category field to the Port object.

1.100 (Flamingo)

Add a vendor field to the Port object.

1.99 (Flamingo)

Add ability to filter on node conductor group when listing ports and portgroups.

1.98 (Flamingo)

Add support for patching object attributes with keys containing ~ or /.

1.97 (Flamingo)

Add a description field to the Port object.

1.95 (Epoxy)

Add support to set/unset disable_power_off on nodes.

1.94 (Epoxy)

Add support to create ports passing in either the node name or UUID.

1.92 (Dalmatian)

Adds runbooks, a predefined list of steps that can be run on nodes associated via traits and used in place of an explicit list of steps for manual cleaning or servicing, to enable self-service of maintenance items by project members.

- Adds a new REST API endpoint `/v1/runbooks/` with basic CRUD support.
- Extends the `/v1/nodes/<node>/states/provision` API to accept a runbook identifier (name or UUID) instead of `clean_steps` or `service_steps` for servicing or manual cleaning.

- Implements RBAC-aware lifecycle management for runbooks, allowing projects to limit who can CRUD and use a runbook.

1.91 (Dalmatian)

Removes special treatment of .json for API objects

- `/v1/nodes/test.json` will now only mean node with the name test.json
- `/v1/nodes/test.json`.json will mean a node with the name test.json.json and,
- `/v1/nodes/test` will mean a node with the name test.

So `/v1/nodes/test.json` will no longer default to test and will HTTP 404 unless a node with the name test actually exists.

This also removes the backward compatibility with the `guess_content_type_from_ext` feature

1.90 (Caracal)

API supports ovn vtep switches as a valid schema for `port.local_link_connection`. Ovn vtep switches are represented as the following:

```
{
  "port_id": "exampleportid",
  "vtep-logical-switch": "examplelogicalswitch",
  "vtep-physical-switch": "examplephysicalswitch"
}
```

1.89 (Caracal)

Adds support to attaching or detaching images from a nodes virtual media using the `/v1/nodes/{node_ident}/vmedia` endpoint. A `POST` request containing `device_type`, `image_url`, and `image_download_source` will attach the requested image to the nodes virtual media. A later `DELETE` request to the same endpoint will detach it.

1.88 (Bobcat)

Added the `name` field to the port API. It should be unique when set, and can be used to identify a port resource.

1.87 (Bobcat)

Adds the `service` provision state verb to allow modifications via the steps interface to occur with a baremetal node. With this functionality comes a `service_step` field on the `/v1/nodes` based resources, which indicates the current step.

1.86 (Bobcat)

Adds a `firmware_interface` field to the `/v1/nodes` resources.

1.85 (Bobcat, 22.1)

This version adds a new provision state change verb called `unhold` to be utilized with the new `provision_state` values `clean hold` and `deploy hold`. The verb instructs Ironic to remove the node from its present hold and to resume its prior cleaning or deployment process.

1.84 (Bobcat, 22.1)

Add callback endpoint for in-band inspection `/v1/continue_inspection`. This endpoint is not designed to be used by regular users.

1.83 (Bobcat, 22.0)

This version adds a concept of child nodes through the use of a `parent_node` field which can be set on a node.

Under normal conditions, child nodes are not visible in the normal node list, as they are more for nested resources and not machines which can be freely used outside of an integrated context of the parent node. Think of a child node as a node with its own BMC embedded inside of an existing node.

Additionally:

- Adds `GET /v1/nodes/{node_ident}/children` to return a list of node UUIDs which represent children, which can acted upon individually.
- Adds `GET /v1/nodes/?include_children=True` to return a list of all parent nodes and children.
- Adds `GET /v1/nodes?parent_node={node_ident}` to explicitly request a detailed list of nodes by parent relationship.

1.82 (Antelope, 21.4)

This version signifies the addition of node sharding endpoints.

- Adds support for get, set, and delete of shard key on Node object.
- Adds support for `GET /v1/shards` which returns a list of all shards and the count of nodes assigned to each.

1.81 (Antelope, 21.3)

Add endpoint to retrieve introspection data for nodes via the REST API.

- `GET /v1/nodes/{node_ident}/inventory/`

1.80 (Zed, 21.1)

This version is a signifier of additional RBAC functionality allowing a project scoped `admin` to create or delete nodes in Ironic.

1.79 (Zed, 21.0)

A node with the same name as the allocation `name` is moved to the start of the derived candidate list.

1.78 (Xena, 18.2)

Add endpoints to allow history events for nodes to be retrieved via the REST API.

- `GET /v1/nodes/{node_ident}/history/`
- `GET /v1/nodes/{node_ident}/history/{event_uuid}`

1.77 (Xena, 18.2)

Add a fields selector to the the Drivers list: * `GET /v1/drivers?fields=` Also add a fields selector to the the Driver detail: * `GET /v1/drivers/{driver_name}?fields=`

1.76 (Xena, 18.2)

Add endpoints for changing boot mode and secure boot state of node asynchronously:

- `PUT /v1/nodes/{node_ident}/states/boot_mode`
- `PUT /v1/nodes/{node_ident}/states/secure_boot`

1.75 (Xena, 18.1)

Add `boot_mode` and `secure_boot` to node object and expose their state at:

- `/v1/nodes/{node_ident}/states`

1.74 (Xena, 18.0)

Add support for BIOS registry fields which include details about the BIOS setting. Included in the `/v1/nodes/{node_ident}/bios/{setting}` response.

Add a new selector to include the fields in the BIOS settings list:

- `/v1/nodes/{node_ident}/bios/?detail=`

Also add a fields selector to the the BIOS settings list:

- `/v1/nodes/{node_ident}/bios/?fields=`

1.73 (Xena, 18.0)

Add a new `deploy` verb as an alias to `active` and `undeploy` verb as an alias to `deleted`.

1.72 (Wallaby, 17.0)

Add support for `agent_status` and `agent_status_message` to /v1/heartbeat. These fields are used for external installation tools, such as Anaconda, to report back status.

1.71 (Wallaby, 17.0)

Signifier of the API supporting keystone `system` scoped roles and access controls. This is an informational flag for clients to be aware of the servers capability.

1.70 (Wallaby, 17.0)

Add support for `disable_ramdisk` parameter to provisioning endpoint `/v1/nodes/{node_ident}/states/provision`.

1.69 (Wallaby, 16.2)

Add support for `deploy_steps` parameter to provisioning endpoint `/v1/nodes/{node_ident}/states/provision`. Available and optional when target is active or rebuild.

1.68 (Victoria, 16.0)

Added the `agent_verify_ca` parameter to the ramdisk heartbeat API.

1.67 (Victoria, 15.1)

Add support for the mutually exclusive `port_uuid` and `portgroup_uuid` fields by having the node vif_attach API accept those values within `vif_info`. If one is specified, then Ironic will attempt to attach a VIF to the relative port or portgroup.

1.66 (Victoria, 15.1)

Add `network_data` field to the node object, that will be used by stand-alone ironic to pass L3 network configuration information to ramdisk.

1.65 (Ussuri, 15.0)

Added `lessee` field to the node object. The field should match the `project_id` of the intended lessee. If an allocation has an owner, then the allocation process will only match the allocation with a node that has the same `owner` or `lessee`.

1.64 (Ussuri, 15.0)

Added the `network_type` to the port objects `local_link_connection` field. The `network_type` can be set to either `managed` or `unmanaged`. When the type is `unmanaged` other fields are not required. Use `unmanaged` when the neutron `network_interface` is required, but the network is in fact a flat network where no actual switch management is done.

1.63 (Ussuri, 15.0)

Added the following new endpoints for indicator management:

- `GET /v1/nodes/<node_ident>/management/indicators` to list all available indicators names for each of the hardware component. Currently known components are: `chassis`, `system`, `disk`, `power` and `nic`.
- `GET /v1/nodes/<node_ident>/management/indicators/<component>/<indicator_ident>` to retrieve all indicators and their states for the hardware component.
- `PUT /v1/nodes/<node_ident>/management/indicators/<component>/<indicator_ident>` change state of the desired indicators of the component.

1.62 (Ussuri, 15.0)

This version of the API is to signify capability of an ironic deployment to support the `agent token` functionality with the `ironic-python-agent`.

1.61 (Ussuri, 14.0)

Added `retired` field to the node object to mark nodes for retirement. If set, this flag will move nodes to `manageable` upon automatic cleaning. `manageable` nodes which have this flag set cannot be moved to available. Also added `retired_reason` to specify the retirement reason.

1.60 (Ussuri, 14.0)

Added `owner` field to the allocation object. The field should match the `project_id` of the intended owner. If the `owner` field is set, the allocation process will only match the allocation with a node that has the same `owner` field set.

1.59 (Ussuri, 14.0)

Added the ability to specify a `vendor_data` dictionary field in the `configdrive` parameter submitted with the deployment of a node. The value is a dictionary which is served as `vendor_data2.json` in the config drive.

1.58 (Train, 12.2.0)

Added the ability to backfill allocations for already deployed nodes by creating an allocation with `node` set.

1.57 (Train, 12.2.0)

Added the following new endpoint for allocation:

- `PATCH /v1/allocations/<allocation_ident>` that allows updating `name` and `extra` fields for an existing allocation.

1.56 (Stein, 12.1.0)

Added the ability for the `configdrive` parameter submitted with the deployment of a node, to include a `meta_data`, `network_data` and `user_data` dictionary fields. Ironic will now use the supplied data to create a configuration drive for the user. Prior uses of the `configdrive` field are unaffected.

1.55 (Stein, 12.1.0)

Added the following new endpoints for deploy templates:

- `GET /v1/deploy_templates` to list all deploy templates.
- `GET /v1/deploy_templates/<deploy template identifier>` to retrieve details of a deploy template.
- `POST /v1/deploy_templates` to create a deploy template.
- `PATCH /v1/deploy_templates/<deploy template identifier>` to update a deploy template.

- DELETE /v1/deploy_templates/<deploy template identifier> to delete a deploy template.

1.54 (Stein, 12.1.0)

Added new endpoints for external events:

- POST /v1/events for creating events. (This endpoint is only intended for internal consumption.)

1.53 (Stein, 12.1.0)

Added is_smartnic field to the port object to enable Smart NIC port creation in addition to local link connection attributes port_id and hostname.

1.52 (Stein, 12.1.0)

Added allocation API, allowing reserving a node for deployment based on resource class and traits. The new endpoints are:

- POST /v1/allocations to request an allocation.
- GET /v1/allocations to list all allocations.
- GET /v1/allocations/<ID or name> to retrieve the allocation details.
- GET /v1/nodes/<ID or name>/allocation to retrieve an allocation associated with the node.
- DELETE /v1/allocations/<ID or name> to remove the allocation.
- DELETE /v1/nodes/<ID or name>/allocation to remove an allocation associated with the node.

Also added a new field allocation_uuid to the node resource.

1.51 (Stein, 12.1.0)

Added description field to the node object to enable operators to store any information relates to the node. The field is limited to 4096 characters.

1.50 (Stein, 12.1.0)

Added owner field to the node object to enable operators to store information in relation to the owner of a node. The field is up to 255 characters and MAY be used in a later point in time to allow designation and deligation of permissions.

1.49 (Stein, 12.0.0)

Added new endpoints for retrieving conductors information, and added a conductor field to node object.

1.48 (Stein, 12.0.0)

Added protected field to the node object to allow protecting deployed nodes from undeploying, rebuilding or deletion. Also added protected_reason to specify the reason of making the node protected.

1.47 (Stein, 12.0.0)

Added `automated_clean` field to the node object, enabling cleaning per node.

1.46 (Rocky, 11.1.0)

Added `conductor_group` field to the node and the node response, as well as support to the API to return results by matching the parameter.

1.45 (Rocky, 11.1.0)

Added `reset_interfaces` parameter to nodes PATCH request, to specify whether to reset hardware interfaces to their defaults on drivers update.

1.44 (Rocky, 11.1.0)

Added `deploy_step` to the node object, to indicate the current deploy step (if any) being performed on the node.

1.43 (Rocky, 11.0.0)

Added `?detail=` boolean query to the API list endpoints to provide a more RESTful alternative to the existing `/nodes/detail` and similar endpoints.

1.42 (Rocky, 11.0.0)

Added `fault` to the node object, to indicate currently detected fault on the node.

1.41 (Rocky, 11.0.0)

Added support to abort inspection of a node in the `inspect wait` state.

1.40 (Rocky, 11.0.0)

Added BIOS properties as sub resources of nodes:

- GET /v1/nodes/<node_ident>/bios
- GET /v1/nodes/<node_ident>/bios/<setting_name>

Added `bios_interface` field to the node object to allow getting and setting the interface.

1.39 (Rocky, 11.0.0)

Added `inspect wait` to available provision states. A node is shown as `inspect wait` instead of `inspecting` during asynchronous inspection.

1.38 (Queens, 10.1.0)

Added provision_state verbs `rescue` and `unrescue` along with the following states: `rescue`, `rescue failed`, `rescue wait`, `rescuing`, `unrescue failed`, and `unrescuing`. After rescuing a node, it will be left in the `rescue` state running a rescue ramdisk, configured with the `rescue_password`, and listening with ssh on the specified network interfaces. Unrescuing a node will return it to `active`.

Added `rescue_interface` to the node object, to allow setting the rescue interface for a dynamic driver.

1.37 (Queens, 10.1.0)

Adds support for node traits, with the following new endpoints.

- GET /v1/nodes/<node identifier>/traits lists the traits for a node.
- PUT /v1/nodes/<node identifier>/traits sets all traits for a node.
- PUT /v1/nodes/<node identifier>/traits/<trait> adds a trait to a node.
- DELETE /v1/nodes/<node identifier>/traits removes all traits from a node.
- DELETE /v1/nodes/<node identifier>/traits/<trait> removes a trait from a node.

A nodes traits are also included the following node query and list responses:

- GET /v1/nodes/<node identifier>
- GET /v1/nodes/detail
- GET /v1/nodes?fields=traits

Traits cannot be specified on node creation, nor can they be updated via a PATCH request on the node.

1.36 (Queens, 10.0.0)

Added `agent_version` parameter to deploy heartbeat request for version negotiation with Ironic Python Agent features.

1.35 (Queens, 9.2.0)

Added ability to provide `configdrive` when node is updated to `rebuild` provision state.

1.34 (Pike, 9.0.0)

Adds a `physical_network` field to the port object. All ports in a portgroup must have the same value in their `physical_network` field.

1.33 (Pike, 9.0.0)

Added `storage_interface` field to the node object to allow getting and setting the interface.

Added `default_storage_interface` and `enabled_storage_interfaces` fields to the driver object to show the information.

1.32 (Pike, 9.0.0)

Added new endpoints for remote volume configuration:

- GET /v1/volume as a root for volume resources
- GET /v1/volume/connectors for listing volume connectors
- POST /v1/volume/connectors for creating a volume connector
- GET /v1/volume/connectors/<UUID> for showing a volume connector
- PATCH /v1/volume/connectors/<UUID> for updating a volume connector
- DELETE /v1/volume/connectors/<UUID> for deleting a volume connector

- GET /v1/volume/targets for listing volume targets
- POST /v1/volume/targets for creating a volume target
- GET /v1/volume/targets/<UUID> for showing a volume target
- PATCH /v1/volume/targets/<UUID> for updating a volume target
- DELETE /v1/volume/targets/<UUID> for deleting a volume target

Volume resources also can be listed as sub resources of nodes:

- GET /v1/nodes/<node identifier>/volume
- GET /v1/nodes/<node identifier>/volume/connectors
- GET /v1/nodes/<node identifier>/volume/targets

1.31 (Ocata, 7.0.0)

Added the following fields to the node object, to allow getting and setting interfaces for a dynamic driver:

- boot_interface
- console_interface
- deploy_interface
- inspect_interface
- management_interface
- power_interface
- raid_interface
- vendor_interface

1.30 (Ocata, 7.0.0)

Added dynamic driver APIs:

- GET /v1/drivers now accepts a `type` parameter (optional, one of `classic` or `dynamic`), to limit the result to only classic drivers or dynamic drivers (hardware types). Without this parameter, both classic and dynamic drivers are returned.
- GET /v1/drivers now accepts a `detail` parameter (optional, one of `True` or `False`), to show all fields for a driver. Defaults to `False`.
- GET /v1/drivers now returns an additional `type` field to show if the driver is classic or dynamic.
- GET /v1/drivers/<name> now returns an additional `type` field to show if the driver is classic or dynamic.
- GET /v1/drivers/<name> now returns additional fields that are null for classic drivers, and set as following for dynamic drivers:
  - The value of the default_<interface-type>_interface is the entrypoint name of the calculated default interface for that type:
    - default_boot_interface
    - default_console_interface

        * default_deploy_interface
        * default_inspect_interface
        * default_management_interface
        * default_network_interface
        * default_power_interface
        * default_raid_interface
        * default_vendor_interface
    - The value of the enabled_<interface-type>_interfaces is a list of entrypoint names of the enabled interfaces for that type:
        * enabled_boot_interfaces
        * enabled_console_interfaces
        * enabled_deploy_interfaces
        * enabled_inspect_interfaces
        * enabled_management_interfaces
        * enabled_network_interfaces
        * enabled_power_interfaces
        * enabled_raid_interfaces
        * enabled_vendor_interfaces

1.29 (Ocata, 7.0.0)

Add a new management API to support inject NMI, PUT /v1/nodes/(node_ident)/management/inject_nmi.

1.28 (Ocata, 7.0.0)

Add /v1/nodes/<node identifier>/vifs endpoint for attach, detach and list of VIFs.

1.27 (Ocata, 7.0.0)

Add `soft rebooting` and `soft power off` as possible values for the `target` field of the power state change payload, and also add `timeout` field to it.

1.26 (Ocata, 7.0.0)

Add portgroup `mode` and `properties` fields.

1.25 (Ocata, 7.0.0)

Add possibility to unset chassis_uuid from a node.

1.24 (Ocata, 7.0.0)

Added new endpoints /v1/nodes/<node>/portgroups and /v1/portgroups/<portgroup>/ports. Added new field `port.portgroup_uuid`.

1.23 (Ocata, 7.0.0)

Added /v1/portgroups/ endpoint.

1.22 (Newton, 6.1.0)

Added endpoints for deployment ramdisks.

1.21 (Newton, 6.1.0)

Add node `resource_class` field.

1.20 (Newton, 6.1.0)

Add node `network_interface` field.

1.19 (Newton, 6.1.0)

Add `local_link_connection` and `pxe_enabled` fields to the port object.

1.18 (Newton, 6.1.0)

Add `internal_info` readonly field to the port object, that will be used by ironic to store internal port-related information.

1.17 (Newton, 6.0.0)

Addition of provision_state verb `adopt` which allows an operator to move a node from `manageable` state to `active` state without performing a deployment operation on the node. This is intended for nodes that have already been deployed by external means.

1.16 (Mitaka, 5.0.0)

Add ability to filter nodes by driver.

1.15 (Mitaka, 5.0.0)

Add ability to do manual cleaning when a node is in the manageable provision state via PUT v1/nodes/<identifier>/states/provision, target:clean, clean_steps:[].

1.14 (Liberty, 4.2.0)

Make the following endpoints discoverable via Ironic API:

- /v1/nodes/<UUID or logical name>/states
- /v1/drivers/<driver name>/properties

1.13 (Liberty, 4.2.0)

Add a new verb `abort` to the API used to abort nodes in `CLEANWAIT` state.

1.12 (Liberty, 4.2.0)

This API version adds the following abilities:

- Get/set `node.target_raid_config` and to get `node.raid_config`.
- Retrieve the logical disk properties for the driver.

1.11 (Liberty, 4.0.0, breaking change)

Newly registered nodes begin in the `enroll` provision state by default, instead of `available`. To get them to the `available` state, the `manage` action must first be run to verify basic hardware control. On success the node moves to `manageable` provision state. Then the `provide` action must be run. Automated cleaning of the node is done and the node is made `available`.

1.10 (Liberty, 4.0.0)

Logical node names support all RFC 3986 unreserved characters. Previously only valid fully qualified domain names could be used.

1.9 (Liberty, 4.0.0)

Add ability to filter nodes by provision state.

1.8 (Liberty, 4.0.0)

Add ability to return a subset of resource fields.

1.7 (Liberty, 4.0.0)

Add node `clean_step` field.

1.6 (Kilo)

Add inspection process: introduce `inspecting` and `inspectfail` provision states, and `inspect` action that can be used when a node is in `manageable` provision state.

1.5 (Kilo)

Add logical node names that can be used to address a node in addition to the node UUID. Name is expected to be a valid fully qualified domain name in this version of API.

1.4 (Kilo)

Add `manageable` state and `manage` transition, which can be used to move a node to `manageable` state from `available`. The node cannot be deployed in `manageable` state. This change is mostly a preparation for future inspection work and introduction of `enroll` provision state.

1.3 (Kilo)

Add node `driver_internal_info` field.

1.2 (Kilo, breaking change)

Renamed NOSTATE (`None` in Python, `null` in JSON) node state to `available`. This is needed to reduce confusion around `None` state, especially when future additions to the state machine land.

1.1 (Kilo)

This was the initial version when API versioning was introduced. Includes the following changes from Kilo release cycle:

- Add node `maintenance_reason` field and an API endpoint to set/unset the node maintenance mode.
- Add sync and async support for vendor passthru methods.
- Vendor passthru endpoints support different HTTP methods, not only `POST`.
- Make vendor methods discoverable via the Ironic API.
- Add logic to store the config drive passed by Nova.

This has been the minimum supported version since versioning was introduced.

1.0 (Juno)

This version denotes Juno API and was never explicitly supported, as API versioning was not implemented in Juno, and 1.1 became the minimum supported version in Kilo.

Exceptions from Versioning

The following API-visible things are not covered by the API versioning:

- Current node state is always exposed as it is, even if not supported by the requested API version, with exception of `available` state, which is returned in version 1.1 as `None` (in Python) or `null` (in JSON).
- Data within free-form JSON attributes: `properties`, `driver_info`, `instance_info`, `driver_internal_info` fields on a node object; `extra` fields on all objects.
- Addition of new drivers.
- All vendor passthru methods.

CHAPTER

FOUR

ADMINISTRATOR GUIDE

4.1 Drivers, Hardware Types, and Hardware Interfaces for Ironic

4.1.1 Generic Interfaces

Boot interfaces

The boot interface manages booting of both the deploy ramdisk and the user instances on the bare metal node.

The *PXE boot* interface is generic and works with all hardware that supports booting from network. Alternatively, several vendors provide *virtual media* implementations of the boot interface. They work by pushing an ISO image to the nodes management controller, and do not require either PXE or iPXE. Check your driver documentation at *Drivers, Hardware Types, and Hardware Interfaces for Ironic* for details.

PXE boot

The `pxe` and `ipxe` boot interfaces uses PXE or iPXE accordingly to deliver the target kernel/ramdisk pair. PXE uses relatively slow and unreliable TFTP protocol for transfer, while iPXE uses HTTP. The downside of iPXE is that its less common, and usually requires bootstrapping using PXE first.

The `pxe` and `ipxe` boot interfaces work by preparing a PXE/iPXE environment for a node on the file system, then instructing the DHCP provider (for example, the Networking service) to boot the node from it. See *Example: PXE Boot and Direct Deploy Process* for a better understanding of the whole deployment process.

Note

Both PXE and iPXE are configured differently, when UEFI boot is used instead of conventional BIOS boot. This is particularly important for CPU architectures that do not have BIOS support at all.

The `ipxe` boot interface is used by default for many hardware types, including `ipmi`. Some hardware types, notably `ilo`, have their specific implementations of the PXE boot interface.

Additional configuration is required for this boot interface - see *Configuring Network Boot* for details.

HTTP Boot

The `http` and `http-ipxe` boot interfaces are based upon the Ironic implementation of the `pxe` and `ipxe` boot interfaces, respectively, and utilize HTTP in the transmission of the location to start the boot sequence from. These interfaces are specific to UEFI as they are rooted in the UEFI standard v2.5s support for booting from an HTTP URL.

One caveat to keep in mind is that these interfaces require hardware support and the ability to signal to the remote BMC that the node should boot utilizing `UEFIHTTP`. If a hardware type does not support that as an option, we will fallback and request `PXE` boot, but that realistically may only work if the firmware on the machine is smart enough to check and evaluate for an HTTP Boot URL instead of a PXE boot server and file name.

It should be noted, that these boot interfaces are available for the vendor independent, generic hardware types of `ipmi` and `redfish`. Hardware vendors typically only include additional interfaces after they have performed their own verification and qualification testing.

Kernel parameters

If you need to pass additional kernel parameters to the deployment/cleaning ramdisk (for example, to configure serial console), use the following configuration option:

```
[pxe]
kernel_append_params = nofb vga=normal
```

Note

The option was called `pxe_append_params` before the Xena cycle.

Per-node and per-instance overrides are also possible, for example:

```
baremetal node set node-0 \
  --driver-info kernel_append_params="nofb vga=normal"
baremetal node set node-0 \
  --instance-info kernel_append_params="nofb vga=normal"
```

Starting with the Zed cycle, you can combine the parameters from the configuration and from the node using the special `%default%` syntax:

```
baremetal node set node-0 \
  --driver-info kernel_append_params="%default% console=ttyS0,115200n8"
```

Together with the configuration above, the following parameters will be appended to the kernel command line:

```
nofb vga=normal console=ttyS0,115200n8
```

Note

Ironic does not do any de-duplication of the resulting kernel parameters. Both kernel itself and dracut seem to give priority to the last instance of the same parameter.

Warning

Previously our documentation listed the Linux kernel parameter `nomodeset` as an option. This option is intended for troubleshooting, and can greatly degrade performance with Matrox/Aspeed BMC Graphics controllers which is very commonly used on physical servers. The performance degradation can greatly reduce IO capacity upon every console graphics update being written to the screen.

Common options

Enable persistent boot device for deploy/clean operation

For (i)PXE booting, Ironic uses non-persistent boot order changes for clean/deploy by default. For some drivers, persistent changes are far more costly than non-persisent ones, so this approach can bring a performance benefit.

In order to control this behavior, however, Ironic provides the `force_persistent_boot_device` flag in the nodes `driver_info`. It allows the values `Default` (make all changes but the last one upon deployment non-persistent), `Always` (make all changes persistent), and `Never` (make all boot order changes non-persistent). For example in order to have only persistent changes one would need to set something like:

```
$ openstack baremetal node set --driver-info force_persistent_boot_device=
↪'Always' <node>
```

Note

It is recommended to check if the nodes state has not changed as there is no way of locking the node between these commands.

Note

The values True/False for the option force_persistent_boot_device in the nodes driver info for the (i)PXE drivers are deprecated and support for them may be removed in a future release. The former default value False is replaced by the new value Default, the value True is replaced by Always.

Deploy Interfaces

A *deploy* interface plays a critical role in the provisioning process. It orchestrates the whole deployment and defines how the image gets transferred to the target disk.

Direct deploy

With `direct` deploy interface, the deploy ramdisk fetches the image from an HTTP location. It can be an object storage (swift or RadosGW) temporary URL or a user-provided HTTP URL. The deploy ramdisk then copies the image to the target disk. See *direct deploy diagram* for a detailed explanation of how this deploy interface works.

You can specify this deploy interface when creating or updating a node:

```
baremetal node create --driver ipmi --deploy-interface direct
baremetal node set <NODE> --deploy-interface direct
```

Note

For historical reasons the `direct` deploy interface is sometimes called `agent`. This is because before the Kilo release **ironic-python-agent** used to only support this deploy interface.

Deploy with custom HTTP servers

The `direct` deploy interface can also be configured to use with custom HTTP servers set up at ironic conductor nodes, images will be cached locally and made accessible by the HTTP server.

To use this deploy interface with a custom HTTP server, set `image_download_source` to `http` in the `[agent]` section.

```
[agent]
...
image_download_source = http
...
```

This configuration affects *glance* and `file://` images. If you want `http(s)://` images to also be cached and served locally, use instead:

```
[agent]
image_download_source = local
```

Note

This option can also be set per node in `driver_info`:

```
baremetal node set <node> --driver-info image_download_source=local
```

or per instance in `instance_info`:

```
baremetal node set <node> --instance-info image_download_source=local
```

You need to set up a workable HTTP server at each conductor node which with `direct` deploy interface enabled, and check http related options in the ironic configuration file to match the HTTP server configurations.

```
[deploy]
http_url = http://example.com
http_root = /httpboot
```

Note

See also: *Deploying outside of the provisioning network*.

Each HTTP server should be configured to follow symlinks for images accessible from HTTP service. Please refer to configuration option `FollowSymLinks` if you are using Apache HTTP server, or `disable_symlinks` if Nginx HTTP server is in use.

Streaming raw images

The Bare Metal service is capable of streaming raw images directly to the target disk of a node, without caching them in the nodes RAM. When the source image is not already raw, the conductor will convert the image and calculate the new checksum.

Note

If no algorithm is specified via the `image_os_hash_algo` field, or if this field is set to `md5`, SHA256 is used for the updated checksum.

For HTTP or local file images that are already raw, you need to explicitly set the disk format to prevent the checksum from being unnecessarily re-calculated. For example:

```
baremetal node set <node> \
    --instance-info image_source=http://server/myimage.img \
    --instance-info image_os_hash_algo=sha512 \
    --instance-info image_os_hash_value=<SHA512 of the raw image> \
    --instance-info image_disk_format=raw
```

To disable this feature and cache images in the nodes RAM, set

```
[agent]
stream_raw_images = False
```

To disable the conductor-side conversion completely, set

```
[DEFAULT]
force_raw_images = False
```

Autodetect support

The `direct` deploy interface supports being auto-detected by the `autodetect` deploy interface (see *Autodetect deploy* for details). The `direct` interface will always claim to support switching to it for deployment regardless of the supplied node information. This makes it appropriate to use as the last fallback value in the `autodetect_deploy_interfaces` configuration option.

Autodetect deploy

The `autodetect` deploy interface automatically selects the most appropriate concrete deploy interface based on image metadata and node configuration. This simplifies node configuration by eliminating the need to manually specify different deploy interfaces for different image types.

When a deployment begins, the autodetect interface inspects the image and node configuration, then switches to the most appropriate deploy interface from the configured list. The detection logic checks interfaces in priority order using each interfaces `supports_deploy()` method. The first interface that returns `True` is selected. If no interfaces are detected as supported, the last interface in the configured list is used as a fallback.

You can specify this deploy interface when creating or updating a node:

```
baremetal node create --driver ipmi --deploy-interface autodetect
baremetal node set <NODE> --deploy-interface autodetect
```

Configuration

The autodetect interface uses the `autodetect_deploy_interfaces` configuration option to control which interfaces can be auto-selected and their priority order. The default configuration is:

```
[DEFAULT]
autodetect_deploy_interfaces = ramdisk,bootc,direct
```

This example configuration checks *Ramdisk deploy* first (for nodes with `kernel/ramdisk` or `boot_iso` in `instance_info`), then *Bootc deploy* (for OCI container images), and falls back to *Direct deploy* for standard disk images.

Note

The order of interfaces in the configuration list matters. Interfaces are checked in the order listed, and the first supported interface is selected. The last interface in the list serves as the fallback if no other interface is detected as supported.

Note

After deployment teardown, the nodes deploy interface is automatically restored to `autodetect` from whatever concrete interface was used during deployment.

Ansible deploy

This interface is similar to `direct` in the sense that the image is downloaded by the ramdisk directly from the image store (not from ironic-conductor host), but the logic of provisioning the node is held in a set of Ansible playbooks that are applied by the `ironic-conductor` service handling the node. While somewhat more complex to set up, this deploy interface provides greater flexibility in terms of advanced node preparation during provisioning.

This interface is supported by most but not all hardware types declared in ironic. However this deploy interface is not enabled by default. To enable it, add `ansible` to the list of enabled deploy interfaces in `enabled_deploy_interfaces` option in the `[DEFAULT]` section of ironics configuration file:

```
[DEFAULT]
...
enabled_deploy_interfaces = direct,ansible
...
```

Once enabled, you can specify this deploy interface when creating or updating a node:

```
baremetal node create --driver ipmi --deploy-interface ansible
baremetal node set <NODE> --deploy-interface ansible
```

For more information about this deploy interface, its features and how to use it, see *Ansible deploy interface*.

Ansible deploy interface

Ansible is a mature and popular automation tool, written in Python and requiring no agents running on the node being configured. All communications with the node are by default performed over secure SSH transport.

The `ansible` deploy interface uses Ansible playbooks to define the deployment logic. It is not based on Ironic Python Agent (IPA) and does not generally need IPA to be running in the deploy ramdisk.

Overview

The main advantage of this deploy interface is extended flexibility in regards to changing and adapting node deployment logic for specific use cases, via Ansible tooling that is already familiar to operators.

It can be used to shorten the usual feature development cycle of

- implementing logic in ironic,
- implementing logic in IPA,
- rebuilding deploy ramdisk,
- uploading deploy ramdisk to Glance/HTTP storage,
- reassigning deploy ramdisk to nodes,
- restarting ironic-conductor service(s) and
- running a test deployment

by using a stable deploy ramdisk and not requiring ironic-conductor restarts (see *Extending playbooks*).

The main disadvantage of this deploy interface is the synchronous manner of performing deployment/cleaning tasks. A separate `ansible-playbook` process is spawned for each node being provisioned or cleaned, which consumes one thread from the thread pool available to the `ironic-conductor` process and blocks this thread until the node provisioning or cleaning step is finished or fails. This has to be taken into account when planning an ironic deployment that enables this deploy interface.

Each action (deploy, clean) is described by a single playbook with roles, which is run whole during deployment, or tag-wise during cleaning. Control of cleaning steps is through tags and auxiliary clean steps file. The playbooks for actions can be set per-node, as can the clean steps file.

Features

Similar to deploy interfaces relying on Ironic Python Agent (IPA), this deploy interface also depends on the deploy ramdisk calling back to ironic APIs `heartbeat` endpoint.

However, the driver is currently synchronous, so only the first heartbeat is processed and is used as a signal to start `ansible-playbook` process.

User images

Supports whole-disk images and partition images:

- compressed images are downloaded to RAM and converted to disk device;
- raw images are streamed to disk directly.

For partition images the driver will create root partition, and, if requested, ephemeral and swap partitions as set in nodes `instance_info` by the Compute service or operator. The create partition table will be of `msdos` type by default, the nodes `disk_label` capability is honored if set in nodes `instance_info` (see also *Choosing the disk label*).

Configdrive partition

Creating a configdrive partition is supported for both whole disk and partition images, on both `msdos` and `GPT` labeled disks.

Root device hints

Root device hints are currently supported in their basic form only, with exact matches (see *Specifying the disk for deployment (root device hints)* for more details). If no root device hint is provided for the node, the first device returned as part of `ansible_devices` fact is used as root device to create partitions on or write the whole disk image to.

Node cleaning

Cleaning is supported, both automated and manual. The driver has two default clean steps:

- wiping device metadata
- disk shredding

Their priority can be overridden via `[deploy]\erase_devices_metadata_priority` and `[deploy]\erase_devices_priority` options, respectively, in the ironic configuration file.

As in the case of this driver all cleaning steps are known to the ironic-conductor service, booting the deploy ramdisk is completely skipped when there are no cleaning steps to perform.

Note

Aborting cleaning steps is not supported.

Logging

Logging is implemented as custom Ansible callback module, that makes use of `oslo.log` and `oslo.config` libraries and can reuse logging configuration defined in the main ironic configuration file to set logging for Ansible events, or use a separate file for this purpose.

It works best when `journald` support for logging is enabled.

Requirements

Ansible
Tested with, and targets, Ansible 2.5.x

Bootstrap image requirements

- password-less sudo permissions for the user used by Ansible
- python 2.7.x
- openssh-server
- GNU coreutils
- utils-linux
- parted
- gdisk
- qemu-utils
- python-requests (for ironic callback and streaming image download)
- python-netifaces (for ironic callback)

A set of scripts to build a suitable deploy ramdisk based on TinyCore Linux and `tinyipa` ramdisk, and an element for `diskimage-builder` can be found in ironic-staging-drivers project but will be eventually migrated to the new ironic-python-agent-builder project.

Setting up your environment

1. Install ironic (either as part of OpenStack or standalone)
   - If using ironic as part of OpenStack, ensure that the Image service is configured to use the Object Storage service as backend, and the Bare Metal service is configured accordingly, see *Configure the Image service for temporary URLs*.
2. Install Ansible version as specified in `ironic/driver-requirements.txt` file
3. Edit ironic configuration file
   A. Add `ansible` to the list of deploy interfaces defined in `[DEFAULT]\ enabled_deploy_interfaces` option.
   B. Ensure that a hardware type supporting `ansible` deploy interface is enabled in `[DEFAULT]\ enabled_hardware_types` option.
   C. Modify options in the `[ansible]` section of ironics configuration file if needed (see *Configuration file*).
4. (Re)start ironic-conductor service
5. Build suitable deploy kernel and ramdisk images
6. Upload them to Glance or put in your HTTP storage
7. Create new or update existing nodes to use the enabled driver of your choice and populate *Driver properties for the Node* when different from defaults.
8. Deploy the node as usual.

Ansible-deploy options

Configuration file

Driver options are configured in `[ansible]` section of ironic configuration file, for their descriptions and default values please see configuration file sample.

Driver properties for the Node

Set them per-node via `baremetal node set` command, for example:

```
baremetal node set <node> \
    --deploy-interface ansible \
    --driver-info ansible_username=stack \
    --driver-info ansible_key_file=/etc/ironic/id_rsa
```

ansible_username
: User name to use for Ansible to access the node. Default is taken from `[ansible]/default_username` option of the ironic configuration file (defaults to `ansible`).

ansible_key_file
: Private SSH key used to access the node. Default is taken from `[ansible]/default_key_file` option of the ironic configuration file. If neither is set, the default private SSH keys of the user running the `ironic-conductor` process will be used.

ansible_deploy_playbook
: Playbook to use when deploying this node. Default is taken from `[ansible]/default_deploy_playbook` option of the ironic configuration file (defaults to `deploy.yaml`).

ansible_shutdown_playbook
: Playbook to use to gracefully shutdown the node in-band. Default is taken from `[ansible]/default_shutdown_playbook` option of the ironic configuration file (defaults to `shutdown.yaml`).

ansible_clean_playbook
: Playbook to use when cleaning the node. Default is taken from `[ansible]/default_clean_playbook` option of the ironic configuration file (defaults to `clean.yaml`).

ansible_clean_steps_config
: Auxiliary YAML file that holds description of cleaning steps used by this node, and defines playbook tags in `ansible_clean_playbook` file corresponding to each cleaning step. Default is taken from `[ansible]/default_clean_steps_config` option of the ironic configuration file (defaults to `clean_steps.yaml`).

ansible_python_interpreter
: Absolute path to the python interpreter on the managed machine. Default is taken from `[ansible]/default_python_interpreter` option of the ironic configuration file. Ansible uses `/usr/bin/python` by default.

Customizing the deployment logic

Expected playbooks directory layout

The `[ansible]\playbooks_path` option in the ironic configuration file is expected to have a standard layout for an Ansible project with some additions:

```
<playbooks_path>
|
\_ inventory
\_ add-ironic-nodes.yaml
\_ roles
 \_ role1
 \_ role2
 \_ ...
|
\_callback_plugins
 \_ ...
|
\_ library
 \_ ...
```

The extra files relied by this driver are:

inventory
: Ansible inventory file containing a single entry of `conductor ansible_connection=local`. This basically defines an alias to `localhost`. Its purpose is to make logging for tasks performed by Ansible locally and referencing the localhost in playbooks more intuitive. This also suppresses warnings produced by Ansible about `hosts` file being empty.

add-ironic-nodes.yaml
: This file contains an Ansible play that populates in-memory Ansible inventory with access information received from the ansible-deploy interface, as well as some per-node variables. Include it in all your custom playbooks as the first play.

The default `deploy.yaml` playbook is using several smaller roles that correspond to particular stages of deployment process:

- `discover` - e.g. set root device and image target
- `prepare` - if needed, prepare system, for example create partitions
- `deploy` - download/convert/write user image and configdrive
- `configure` - post-deployment steps, e.g. installing the bootloader

Some more included roles are:

- `shutdown` - used to gracefully power the node off in-band
- `clean` - defines cleaning procedure, with each clean step defined as separate playbook tag.

Extending playbooks

Most probably youd start experimenting like this:

1. Create a copy of `deploy.yaml` playbook *in the same folder*, name it distinctively.
2. Create Ansible roles with your customized logic in `roles` folder.

    A. In your custom deploy playbook, replace the `prepare` role with your own one that defines steps to be run *before* image download/writing. This is a good place to set facts overriding those provided/omitted by the driver, like `ironic_partitions` or `ironic_root_device`, and create custom partitions or (software) RAIDs.

B. In your custom deploy playbook, replace the `configure` role with your own one that defines steps to be run *after* image is written to disk. This is a good place for example to configure the bootloader and add kernel options to avoid additional reboots.

C. Use those new roles in your new playbook.

3. Assign the custom deploy playbook youve created to the nodes `driver_info/ansible_deploy_playbook` field.

4. Run deployment.

A. No ironic-conductor restart is necessary.

B. A new deploy ramdisk must be built and assigned to nodes only when you want to use a command/script/package not present in the current deploy ramdisk and you can not or do not want to install those at runtime.

Variables you have access to

This driver will pass the single JSON-ified extra var argument to Ansible (as in `ansible-playbook -e ..`). Those values are then accessible in your plays as well (some of them are optional and might not be defined):

```
ironic:
  nodes:
  - ip: "<IPADDRESS>"
    name: "<NODE_UUID>"
    user: "<USER ANSIBLE WILL USE>"
    extra: "<COPY OF NODE's EXTRA FIELD>"
  image:
    url: "<URL TO FETCH THE USER IMAGE FROM>"
    disk_format: "<qcow2|raw|...>"
    container_format: "<bare|...>"
    checksum: "<hash-algo:hashstring>"
    mem_req: "<REQUIRED FREE MEMORY TO DOWNLOAD IMAGE TO RAM>"
    tags: "<LIST OF IMAGE TAGS AS DEFINED IN GLANCE>"
    properties: "<DICT OF IMAGE PROPERTIES AS DEFINED IN GLANCE>"
  configdrive:
    type: "<url|file>"
    location: "<URL OR PATH ON CONDUCTOR>"
  partition_info:
    label: "<msdos|gpt>"
    preserve_ephemeral: "<bool>"
    ephemeral_format: "<FILESYSTEM TO CREATE ON EPHEMERAL PARTITION>"
    partitions: "<LIST OF PARTITIONS IN FORMAT EXPECTED BY PARTED MODULE>"
  raid_config: "<COPY OF NODE's TARGET_RAID_CONFIG FIELD>"
```

`ironic.nodes`
List of dictionaries (currently of only one element) that will be used by `add-ironic-nodes.yaml` play to populate in-memory inventory. It also contains a copy of nodes `extra` field so you can access it in the playbooks. The Ansibles host is set to nodes UUID.

`ironic.image`
All fields of nodes `instance_info` that start with `image_` are passed inside this variable. Some extra notes and fields:

- `mem_req` is calculated from image size (if available) and config option *ansible.extra_memory*.
- if `checksum` is not in the form `<hash-algo>:<hash-sum>`, hashing algorithm is assumed to be `md5` (default in Glance).
- `validate_certs` - boolean (`yes/no`) flag that turns validating image store SSL certificate on or off (default is yes). Governed by *ansible.image_store_insecure* option in ironic configuration file.
- `cafile` - custom CA bundle to use for validating image store SSL certificate. Takes value of *ansible.image_store_cafile* if that is defined. Currently is not used by default playbooks, as Ansible has no way to specify the custom CA bundle to use for single HTTPS actions, however you can use this value in your custom playbooks to for example upload and register this CA in the ramdisk at deploy time.
- `client_cert` - cert file for client-side SSL authentication. Takes value of *ansible.image_store_certfile* option if defined. Currently is not used by default playbooks, however you can use this value in your custom playbooks.
- `client_key` - private key file for client-side SSL authentication. Takes value of *ansible.image_store_keyfile* option if defined. Currently is not used by default playbooks, however you can use this value in your custom playbooks.

ironic.partition_info.partitions

Optional. List of dictionaries defining partitions to create on the node in the form:

```
partitions:
- name: "<NAME OF PARTITION>"
  unit: "<UNITS FOR SIZE>"
  size: "<SIZE OF THE PARTITION>"
  type: "<primary|extended|logical>"
  align: "<ONE OF PARTED_SUPPORTED OPTIONS>"
  format: "<PARTITION TYPE TO SET>"
  flags:
    flag_name: "<bool>"
```

The driver will populate this list from `root_gb`, `swap_mb` and `ephemeral_gb` fields of `instance_info`. The driver will also prepend the `bios_grub`-labeled partition when deploying on GPT-labeled disk, and pre-create a 64 MiB partition for configdrive if it is set in `instance_info`.

Please read the documentation included in the `ironic_parted` modules source for more info on the module and its arguments.

ironic.partition_info.ephemeral_format

Optional. Taken from `instance_info`, it defines file system to be created on the ephemeral partition. Defaults to the value of `[pxe]\default_ephemeral_format` option in ironic configuration file.

ironic.partition_info.preserve_ephemeral

Optional. Taken from the `instance_info`, it specifies if the ephemeral partition must be preserved or rebuilt. Defaults to `no`.

ironic.raid_config

Taken from the `target_raid_config` if not empty, it specifies the RAID configuration to apply.

As usual for Ansible playbooks, you also have access to standard Ansible facts discovered by `setup` module.

Included custom Ansible modules

The provided `playbooks_path/library` folder includes several custom Ansible modules used by default implementation of `deploy` and `prepare` roles. You can use these modules in your playbooks as well.

`stream_url`
: Streaming download from HTTP(S) source to the disk device directly, tries to be compatible with Ansibles `get_url` module in terms of module arguments. Due to the low level of such operation it is not idempotent.

`ironic_parted`
: creates partition tables and partitions with `parted` utility. Due to the low level of such operation it is not idempotent. Please read the documentation included in the modules source for more information about this module and its arguments. The name is chosen so that the `parted` module included in Ansible is not shadowed.

Anaconda deploy

The `anaconda` deploy interface is another option for highly customized deployments. See *Deploying with anaconda deploy interface* for more details.

Ramdisk deploy

The ramdisk interface is intended to provide a mechanism to deploy an instance where the item to be deployed is in reality a ramdisk. It is documented separately, see *Booting a Ramdisk or an ISO*.

Autodetect support

The `ramdisk` deploy interface supports being auto-detected by the `autodetect` deploy interface (see *Autodetect deploy* for details). It checks whether the nodes `instance_info` contains `kernel` and `ramdisk` (without an `image_source`), or whether `boot_iso` is set. When either condition is met, autodetect will select the `ramdisk` deploy interface.

Custom agent deploy

The `custom-agent` deploy interface is designed for operators who want to completely orchestrate writing the instance image using in-band deploy steps from a custom agent image. If you use this deploy interface, you are responsible to provide all necessary deploy steps with priorities between 61 and 99 (see *Agent steps* for information on priorities).

Bootc Agent Deploy

The `bootc` deploy interface is designed to enable operators to deploy containers directly from a container image registry without intermediate conversion steps, such as creating custom disk images for modifications. This deployment interface utilizes the bootc project.

Ultimately this enables a streamlined flow, where a user of the deployment interface *can* create updated containers rapidly and the deployment interface will deploy that container image in a streamlined fashion

without the need to create intermediate disk images and post the disk images in a location where they can be accessed for deployment.

Ultimately this interface enables a streamlined flow, and offers limited flexibility in the model of deployment. As a result, this interface consumes the entire target disk on the host being deployed and offers no customization in terms of partitioning. This is largely because the overall security model of a bootc deployment, which leverages os-tree, is also fundamentally different than the model to leverage partition separation.

Note

This interface should be considered experimental and may evolve to include additional features as the Ironic project maintainers receive additional feedback.

Note

This interface is dependent upon the existence of `bootc` within a container image along with sufficient memory on the baremetal node being deployed to enable a complete download and extraction of image contents within system memory. It is this memory constraint which is why this interface is not actively tested in upstream CI. The possible failure modes of this interface are mainly focused upon the ability of the ramdisk being able to download, launch, and run bootc to trigger the installation which also isolates most risk to the actual bootc process execution.

Features

While this `deploy_interface` supports deploying configuration drives like most other Ironic supplied deploy interfaces, some additional parameters can be supplied via `instance_info` to enable tuning of deploy-time behavior by the user which cannot be modified post-deployment.

- `bootc_authorized_keys` - This option allows injection of a root user authorized keys file which is preserved inside of the deployed container on the host. This option is for actual key file content and can be one or more keys with a new line character.
- `bootc_tpm2_luks` - A boolean option, default False, enabling bootc to attempt to utilize auto-encryption of the deployed host filesystem upon which the container is deployed. This is not enabled by default due to a lack of software TPMs in Ironic CI. If operators would like this setting default changed, please discuss with Ironic developers.

Additionally, this interface also supports the passing of a pull secret to enable download from the remote image registry, which is part of the support for retrieval of artifacts from OCI Container registires. This parameter is `image_pull_secret`.

Autodetect support

The `bootc` deploy interface supports being auto-detected by the `autodetect` deploy interface (see *Autodetect deploy* for details). It checks whether the image source URL is an OCI container image (`oci://` URL), then introspects the image registry to confirm the artifact is a container image (OCI artifacts are handled by the `direct` deploy interface).

Caveats

- This deployment interface was not designed to be compatible with the OpenStack Compute service. This is because OpenStack focuses on disk images from Glance as to what to deploy, where as this interface is modeled to utilize a container image registry.
- Performance wise, this deployment interface performs many smaller actions, which at some times need to performed in a specific sequence, such as when unpacking layers. As a result, when comparing similar size containers to disk images, this interface is slower than the `direct` deploy interface.
- Container Images *must* have the bootc command present along with the applicable bootloader and artifacts required for whatever platform is being deployed.
- Because of how bootc works, there is no concept of image streaming directly to disk. This is because the way this interface works, podman is used to download all container image layer artifacts, along with extracting the layers. At which point `bootc` is executed and it begins to setup the disk for the host. As a result, most of the time a deploy is in progress will be observable as `deploy wait` while `bootc` executes.
- The memory requirements of the ramdisk, due to the way this interface works, requires the ability to download a container image, copy, and ultimatley extract all layers into the in-memory filesystem. Due to the way the kernel launches and allocates ramdisk memory for filesystem usage, a 600MB container image may require upwards of 10GB of RAM to be available on the overall host.
- This deployment interface explicitly signals to `bootc` that it should not execute its internal post-deployment fetch check to ensure upgrades are working. This is because this action may require authentication to succeed, **and** thus require credentials in the container to work. Configuration of credentials for **day-2** operations such as the execution of `bootc upgrade`, must be addressed post-deployment.
- If you intend SELinux to be enabled on the deployed host, it must also be enabled inside of the ironic-python-agent ramdisk. This is a design limitation of bootc outside of Ironics control.

Limitations

- At present, this interface does not support use of caching proxies. This may be addressed in the future.
- This deployment interface directly downloads artifacts from the requested Container Registry. Caching the container artifacts on the `ironic-conductor` host is not available. If you need the contaitainer content localized to the conductor, consider utilizing your own container registry.

Noop deploy

The `noop` deploy interface is a minimal deployment mechanism that performs no actual deployment operations. It is designed for scenarios where operators want to allocate nodes and mark them as `active` in Ironic without deploying an operating system.

This is useful when nodes are being managed by external systems but tracked in Ironics inventory.

You can specify this deploy interface when creating or updating a node:

```
baremetal node create --driver manual-management --deploy-interface noop
baremetal node set <NODE> --deploy-interface noop
```

Usage with Nova

For deployments using Ironic in conjunction with Nova, the `noop` deploy interface provides an alternative approach to adding production nodes that are already running workloads. Unlike the *node adoption* feature, which uses the `adopt` verb and is not suitable for Nova deployments, the `noop` deploy interface allows you to use the normal `deploy` verb instead.

With the `noop` deploy interface, nodes can be moved through the normal deployment workflow (from `manageable` to `available` to `deploying` to `active`) without any actual deployment operations taking place. This enables Nova to properly track the instances while Ironic skips all conductor-side deployment actions.

Configuration

Enable the interface in `ironic.conf`:

```
[DEFAULT]
enabled_deploy_interfaces = direct,ramdisk,noop
```

Warning

The `noop` deploy interface completely bypasses cleaning operations. Even if a user cannot update `Node.automated_clean`, as long as they set `Node.instance_info[deploy_interface]` to `noop`, cleaning will be skipped.

4.1.2 Hardware Types

iDRAC driver

Overview

The integrated Dell Remote Access Controller (iDRAC) is an out-of-band management platform on Dell EMC servers, and is supported directly by the `idrac` hardware type. This driver utilizes the Distributed Management Task Force (DMTF) Redfish protocol to perform all of its functions. In older versions of Ironic, this driver leveraged Web Services for Management (WSMAN) protocol.

iDRAC hardware is also supported by the generic `ipmi` and `redfish` hardware types, though with smaller feature sets.

Key features of the Dell iDRAC driver include:

- Out-of-band node inspection
- Boot device management and firmware management
- Power management
- RAID controller management and RAID volume configuration
- BIOS settings configuration

Ironic Features

The `idrac` hardware type extends the `redfish` hardware type and supports the following Ironic interfaces:

- *BIOS Interface*: BIOS management
- *Inspect Interface*: Hardware inspection
- *Management Interface*: Boot device and firmware management
- Power Interface: Power management
- *RAID Interface*: RAID controller and disk management
- *Vendor Interface*: eject virtual media (Redfish)

Prerequisites

The `idrac` hardware type requires the `sushy` library and the vendor extensions to be installed on the ironic conductor node(s) if an Ironic node is configured to use an `idrac-redfish` interface implementation, for example:

```
sudo pip install 'sushy>=5.6.0'
```

Enabling

The iDRAC driver supports Redfish for the bios, inspect, management, power, and raid interfaces.

The `idrac-redfish` implementation must be enabled to use Redfish for an interface.

To enable the `idrac` hardware type, add the following to your `/etc/ironic/ironic.conf`:

```
[DEFAULT]
enabled_hardware_types=idrac
enabled_management_interfaces=idrac-redfish
enabled_power_interfaces=redfish
```

To enable all optional features (BIOS, inspection, RAID, and vendor passthru), use the following configuration:

```
[DEFAULT]
enabled_hardware_types=idrac
enabled_bios_interfaces=redfish
enabled_firmware_interfaces=redfish
enabled_inspect_interfaces=idrac-redfish
enabled_management_interfaces=idrac-redfish
enabled_power_interfaces=idrac-redfish
enabled_raid_interfaces=idrac-redfish
enabled_vendor_interfaces=idrac-redfish
```

Below is the list of supported interface implementations in priority order:

Interface	Supported Implementations
bios	idrac-redfish, no-bios
boot	ipxe, pxe, http-ipxe, http, redfish-https, idrac-redfish-virtual-media
console	no-console
deploy	direct, ansible, ramdisk
firmware	redfish, no-firmware
inspect	idrac-redfish, agent, no-inspect
management	idrac-redfish
network	flat, neutron, noop
power	redfish, idrac-redfish
raid	idrac-redfish, no-raid
rescue	no-rescue, agent
storage	noop, cinder, external
vendor	redfish, idrac-redfish, no-vendor

Protocol-specific Properties

The Redfish protocols require different properties to be specified in the Ironic nodes `driver_info` field to communicate with the bare metal systems iDRAC.

The Redfish protocol requires the following properties:

- `redfish_username`: The Redfish user name to use when communicating with the iDRAC. Usually `root`.
- `redfish_password`: The password for the Redfish user to use when communicating with the iDRAC.
- `redfish_address`: The URL address of the iDRAC. It must include the authority portion of the URL, and can optionally include the scheme. If the scheme is missing, https is assumed.
- `redfish_system_id`: The Redfish ID of the server to be managed. This should always be: `/redfish/v1/Systems/System.Embedded.1`.

For other Redfish protocol parameters see *Redfish driver*.

Enrolling

The following command enrolls a bare metal node with the `idrac` hardware type using Redfish for all interfaces:

```
baremetal node create --driver idrac \
    --driver-info redfish_username=user \
    --driver-info redfish_password=pa$$w0rd \
    --driver-info redfish_address=drac.host \
    --driver-info redfish_system_id=/redfish/v1/Systems/System.Embedded.1 \
    --bios-interface redfish \
    --inspect-interface idrac-redfish \
    --management-interface idrac-redfish \
    --power-interface redfish \
    --raid-interface idrac-redfish \
    --vendor-interface redfish
```

BIOS Interface

The BIOS interface implementations supported by the `idrac` hardware type allows BIOS to be configured with the standard clean/deploy step approach.

Example

A clean step to enable `Virtualization` and `SRIOV` in BIOS of an iDRAC BMC would be as follows:

```
{
  "target":"clean",
  "clean_steps": [
    {
      "interface": "bios",
      "step": "apply_configuration",
      "args": {
        "settings": [
          {
            "name": "ProcVirtualization",
            "value": "Enabled"
          },
          {
            "name": "SriovGlobalEnable",
            "value": "Enabled"
          }
        ]
      }
    }
  ]
}
```

See the *Known Issues* for a known issue with `factory_reset` clean step. For additional details of BIOS configuration, see *BIOS Configuration*.

Inspect Interface

The Dell iDRAC out-of-band inspection process catalogs all the same attributes of the server as the IPMI driver. Unlike IPMI, it does this without requiring the system to be rebooted, or even to be powered on. Inspection is performed using the Redfish protocol directly without affecting the operation of the system being inspected.

The inspection discovers the following properties:

- `cpu_arch`: cpu architecture
- `local_gb`: disk size in gigabytes
- `memory_mb`: memory size in megabytes

Extra capabilities:

- `boot_mode`: UEFI or BIOS boot mode.
- `pci_gpu_devices`: number of GPU devices connected to the bare metal.

It also creates baremetal ports for each NIC port detected in the system. The `idrac-redfish` inspect interface does not currently set `pxe_enabled` on the ports. The user should ensure that `pxe_enabled` is set correctly on the ports following inspection with the `idrac-redfish` inspect interface.

Management Interface

The management interface for `idrac-redfish` supports:

- updating firmware on nodes using a manual cleaning step. See *Redfish driver* for more information on firmware update support.
- updating system and getting its inventory using configuration molds. For more information see *Import and export configuration*.

Import and export configuration

Warning

This feature has been deprecated and is anticipated to be removed once Ironic has a generalized interface for doing step template articulation for aspects beyond just deployment of baremetal nodes.

The clean and deploy steps provided in this section allow to configure the system and collect the system inventory using configuration mold files.

The introduction of this feature in the Wallaby release is experimental.

These steps are:

- `export_configuration` with the `export_configuration_location` input parameter to export the configuration from the existing system.
- `import_configuration` with the `import_configuration_location` input parameter to import the existing configuration mold into the system.
- `import_export_configuration` with the `export_configuration_location` and `import_configuration_location` input parameters. This step combines the previous two steps into one step that first imports existing configuration mold into system, then exports the resulting configuration.

The input parameters provided include the URL where the configuration mold is to be stored after the export, or the reference location for an import. For more information on setting up storage and available options see *Storage setup*.

Configuration molds are JSON files that contain three top-level sections: `bios`, `raid` and `oem`. The following is an example of a configuration mold:

```
{
  "bios": {
    "reset": false,
    "settings": [
      {
        "name": "ProcVirtualization",
        "value": "Enabled"
      },
```

(continues on next page)

(continued from previous page)

```
      {
        "name": "MemTest",
        "value": "Disabled"
      }
    ]
  }
  "raid": {
    "create_nonroot_volumes": true,
    "create_root_volume": true,
    "delete_existing": false,
    "target_raid_config": {
      "logical_disks": [
        {
          "size_gb": 50,
          "raid_level": "1+0",
          "controller": "RAID.Integrated.1-1",
          "volume_name": "root_volume",
          "is_root_volume": true,
          "physical_disks": [
            "Disk.Bay.0:Encl.Int.0-1:RAID.Integrated.1-1",
            "Disk.Bay.1:Encl.Int.0-1:RAID.Integrated.1-1"
          ]
        },
        {
          "size_gb": 100,
          "raid_level": "5",
          "controller": "RAID.Integrated.1-1",
          "volume_name": "data_volume",
          "physical_disks": [
            "Disk.Bay.2:Encl.Int.0-1:RAID.Integrated.1-1",
            "Disk.Bay.3:Encl.Int.0-1:RAID.Integrated.1-1",
            "Disk.Bay.4:Encl.Int.0-1:RAID.Integrated.1-1"
          ]
        }
      ]
    }
  }
  "oem": {
    "interface": "idrac-redfish",
    "data": {
      "SystemConfiguration": {
        "Model": "PowerEdge R640",
        "ServiceTag": "8CY9Z99",
        "TimeStamp": "Fri Jun 26 08:43:15 2020",
        "Components": [
          {
            [...]
            "FQDD": "NIC.Slot.1-1-1",
            "Attributes": [
```

(continues on next page)

(continued from previous page)

```
                {
                "Name": "BlnkLeds",
                "Value": "15",
                "Set On Import": "True",
                "Comment": "Read and Write"
                },
                {
                "Name": "VirtMacAddr",
                "Value": "00:00:00:00:00:00",
                "Set On Import": "False",
                "Comment": "Read and Write"
                },
                {
                "Name": "VirtualizationMode",
                "Value": "NONE",
                "Set On Import": "True",
                "Comment": "Read and Write"
                },
              [...]
              ]
            }
          ]
        }
    }
}
```

Currently, the OEM section is the only section that is supported. The OEM section uses the iDRAC Server Configuration Profile (SCP) and can be edited as necessary if it complies with the SCP. For more information about SCP and its capabilities, see SCP_Reference_Guide.

Note

iDRAC BMC connection settings are not exported to avoid overwriting these in another system when using unmodified exported configuration mold in import step. If need to replicate iDRAC BMC connection settings, then add these settings manually to configuration mold for import step.

To replicate the system configuration to that of a similar system, perform the following steps:

1. Configure a golden, or one to many, system.
2. Use the `export_configuration` step to export the configuration to the wanted location.
3. Adjust the exported configuration mold for other systems to replicate. For example, remove sections that do not need to be replicated such as iDRAC connection settings. The configuration mold can be accessed directly from the storage location.
4. Import the selected configuration mold into the other systems using the `import_configuration` step.

It is not mandatory to use `export_configuration` step to create a configuration mold. Upload the file to a designated storage location without using Ironic if it has been created manually or by other means.

Storage setup

To start using these steps, configure the storage location. The settings can be found in the `[molds]` section. Configure the storage type from the *`molds.storage`* setting. Currently, `swift`, which is enabled by default, and `http` are supported.

In the setup input parameters, the complete HTTP URL is used. This requires that the containers (for `swift`) and the directories (for `http`) are created beforehand, and that read/write access is configured accordingly.

Note

Use of TLS is strongly advised.

This setup configuration allows a user to access these locations outside of Ironic to list, create, update, and delete the configuration molds.

For more information see *Swift configuration* and *HTTP configuration*.

Swift configuration

To use Swift with configuration molds,

1. Create the containers to be used for configuration mold storage.
2. For Ironic Swift user that is configured in the `[swift]` section add read/write access to these containers.

HTTP configuration

To use HTTP server with configuration molds,

1. Enable HTTP PUT support.
2. Create the directory to be used for the configuration mold storage.
3. Configure read/write access for HTTP Basic access authentication and provide user credentials in *`molds.user`* and *`molds.password`* fields.

The HTTP web server does not support multitenancy and is intended to be used in a stand-alone Ironic, or single-tenant OpenStack environment.

RAID Interface

See *RAID Configuration* for more information on Ironic RAID support.

RAID interface of `redfish` hardware type can be used on iDRAC systems. Compared to `redfish` RAID interface, using `idrac-redfish` adds:

- Waiting for real-time operations to be available on RAID controllers. When using `redfish` this is not guaranteed and reboots might be intermittently required to complete,
- Converting non-RAID disks to RAID mode if there are any,
- Clearing foreign configuration, if any, after deleting virtual disks.

The following properties are supported by the Redfish RAID interface implementation:

Note

When using `idrac-redfish` for RAID interface iDRAC firmware greater than 4.40.00.00 is required.

Mandatory properties

- `size_gb`: Size in gigabytes (integer) for the logical disk. Use `MAX` as `size_gb` if this logical disk is supposed to use the rest of the space available.
- `raid_level`: RAID level for the logical disk. Valid values are `0`, `1`, `5`, `6`, `1+0`, `5+0` and `6+0`.

Note

`JBOD` and `2` are not supported, and will fail with reason: Cannot calculate spans for RAID level.

Optional properties

- `is_root_volume`: Optional. Specifies whether this disk is a root volume. By default, this is `False`.
- `volume_name`: Optional. Name of the volume to be created. If this is not specified, it will be auto-generated.

Backing physical disk hints

See *RAID Configuration* for more information on backing disk hints.

These are machine-independent information. The hints are specified for each logical disk to help Ironic find the desired disks for RAID configuration.

- `disk_type`
- `interface_type`
- `share_physical_disks`
- `number_of_physical_disks`

Backing physical disks

These are Dell RAID controller-specific values and must match the names provided by the iDRAC.

- `controller`: Mandatory. The name of the controller to use.
- `physical_disks`: Optional. The names of the physical disks to use.

Note

`physical_disks` is a mandatory parameter if the property `size_gb` is set to `MAX`.

Examples

Creation of RAID 1+0 logical disk with six disks on one controller:

```
{ "logical_disks":
  [ { "controller": "RAID.Integrated.1-1",
      "is_root_volume": "True",
      "physical_disks": [
        "Disk.Bay.0:Enclosure.Internal.0-1:RAID.Integrated.1-1",
        "Disk.Bay.1:Enclosure.Internal.0-1:RAID.Integrated.1-1",
        "Disk.Bay.2:Enclosure.Internal.0-1:RAID.Integrated.1-1",
        "Disk.Bay.3:Enclosure.Internal.0-1:RAID.Integrated.1-1",
        "Disk.Bay.4:Enclosure.Internal.0-1:RAID.Integrated.1-1",
        "Disk.Bay.5:Enclosure.Internal.0-1:RAID.Integrated.1-1"],
      "raid_level": "1+0",
      "size_gb": "MAX"}]}
```

Manual RAID Invocation

The following command can be used to delete any existing RAID configuration. It deletes all virtual disks/RAID volumes, unassigns all global and dedicated hot spare physical disks, and clears foreign configuration:

```
baremetal node clean --clean-steps \
  '[{"interface": "raid", "step": "delete_configuration"}]' ${node_uuid}
```

The following command shows an example of how to set the target RAID configuration:

```
baremetal node set --target-raid-config '{ "logical_disks":
  [ { "controller": "RAID.Integrated.1-1",
      "is_root_volume": true,
      "physical_disks": [
        "Disk.Bay.0:Enclosure.Internal.0-1:RAID.Integrated.1-1",
        "Disk.Bay.1:Enclosure.Internal.0-1:RAID.Integrated.1-1"],
      "raid_level": "0",
      "size_gb": "MAX"}]}' ${node_uuid}
```

The following command can be used to create a RAID configuration:

```
baremetal node clean --clean-steps \
  '[{"interface": "raid", "step": "create_configuration"}]' <node>
```

When the physical disk names or controller names are not known, the following Python code example shows how the `python-dracclient` can be used to fetch the information directly from the Dell bare metal:

```
import dracclient.client


client = dracclient.client.DRACClient(
    host="192.168.1.1",
    username="root",
```

(continues on next page)

(continued from previous page)

```
    password="calvin")
controllers = client.list_raid_controllers()
print(controllers)

physical_disks = client.list_physical_disks()
print(physical_disks)
```

Or using sushy with Redfish:

```
import sushy


client = sushy.Sushy('https://192.168.1.1', username='root', password='calvin
↪', verify=False)
for s in client.get_system_collection().get_members():
  print("System: %(id)s" % {'id': s.identity})
  for c in system1.storage.get_members():
      print("\tController: %(id)s" % {'id': c.identity})
      for d in c.drives:
        print("\t\tDrive: %(id)s" % {'id': d.identity})
```

Vendor Interface

idrac-redfish

Through the `idrac-redfish` vendor passthru interface these methods are available:

Method Name	HTTP Method	Description
`eject_`	`POST`	Eject a virtual media device. If no device is provided then all attached devices will be ejected. Optional argument: `boot_device` - the boot device to eject, either, `cd`, `dvd`, `usb` or `floppy`.

Known Issues

Nodes go into maintenance mode

After some period of time, nodes managed by the `idrac` hardware type may go into maintenance mode in Ironic. This issue can be worked around by changing the Ironic power state poll interval to 70 seconds. See *`conductor.sync_power_state_interval`* in `/etc/ironic/ironic.conf`.

PXE reset with factory_reset BIOS clean step

When using the `UEFI boot mode` with non-default PXE interface, the factory reset can cause the PXE interface to be reset to default, which doesnt allow the server to PXE boot for any further operations. This can cause a `clean_failed` state on the node or `deploy_failed` if you attempt to deploy a node after this step. For now, the only solution is for the operator to manually restore the PXE settings of the server for it to PXE boot again, properly. The problem is caused by the fact that with the `UEFI boot mode`, the `idrac` uses BIOS settings to manage PXE configuration. This is not the case with the `BIOS`

`boot mode` where the PXE configuration is handled as a configuration job on the integrated NIC itself, independently of the BIOS settings.

Timeout when powering off

Some servers might be slow when soft powering off and time out. The default retry count is 6, resulting in 30 seconds timeout (the default retry interval set by `post_deploy_get_power_state_retry_interval` is 5 seconds). To resolve this issue, increase the timeout to 90 seconds by setting the retry count to 18 as follows:

```
[agent]
post_deploy_get_power_state_retries = 18
```

Unable to mount remote share with iDRAC firmware before 4.40.40.00

When using iDRAC firmware 4.40.00.00 and consecutive versions before 4.40.40.00 with virtual media boot and new Virtual Console plug-in type eHTML5, there is an error: Unable to mount remote share. This is a known issue that is fixed in 4.40.40.00 iDRAC firmware release. If cannot upgrade, then adjust settings in iDRAC to use plug-in type HTML5. In iDRAC web UI go to Configuration -> Virtual Console and select Plug-in Type to HTML5.

During upgrade to 4.40.00.00 or newer iDRAC firmware eHTML5 is automatically selected if default plug-in type has been used and never changed. Systems that have plug-in type changed will keep selected plug-in type after iDRAC firmware upgrade.

Firmware update from Swift fails before 6.00.00.00

With iDRAC firmware prior to 6.00.00.00 and when using Swift to stage firmware update files in Management interface `firmware_update` clean step of `redfish` or `idrac` hardware type, the cleaning fails with error An internal error occurred. Unable to complete the specified operation. in iDRAC job. This is fixed in iDRAC firmware 6.00.00.00. If cannot upgrade, then use HTTP service to stage firmware files for iDRAC.

iLO driver

Overview

iLO driver enables to take advantage of features of iLO management engine in HPE ProLiant servers. The `ilo` hardware type is targeted for HPE ProLiant Gen8 and Gen9 systems which have iLO 4 management engine. The `ilo` hardware type supports ProLiant Gen10 systems which have iLO 5 management engine. iLO5 conforms to Redfish API and hence hardware type `redfish` (see *Redfish driver*) is also an option for this kind of hardware but it lacks the iLO specific features.

For more details and for up-to-date information (like tested platforms, known issues, etc), please check the iLO driver wiki page.

For enabling Gen10 systems and getting detailed information on Gen10 feature support in Ironic please check this Gen10 wiki section.

Warning

Starting from Gen11 servers and above (iLO6 and above) use `redfish` (see *Redfish driver*) hardware type for baremetal provisioning and management. You can use the `redfish` hardware type for

iLO5 hardware, however RAID configuration is not available via Redfish until the iLO6 baseboard management controllers.

The Ironic community does not anticipate new features to be added to the `ilo` and `ilo5` hardware types as `redfish` is superseding most vendor specific hardware types. These drivers are anticipated to be available in Ironic as long as the `proliantutils` library is maintained.

Hardware type

ProLiant hardware is primarily supported by the `ilo` hardware type. `ilo5` hardware type is only supported on ProLiant Gen10 and later systems. Both hardware can be used with reference hardware type `ipmi` (see *IPMI driver*) and `redfish` (see *Redfish driver*). For information on how to enable the `ilo` and `ilo5` hardware type, see *Enabling hardware types*.

Warning

It is important to note that while the HPE Edgeline series of servers may contain iLO adapters, they are known to not be compatible with the `ilo` hardware type. The `redfish` hardware type should be used instead.

The hardware type `ilo` supports following HPE server features:

- *Boot mode support*
- *UEFI Secure Boot Support*
- *Node Cleaning Support*
- *Node Deployment Customization*
- *Hardware Inspection Support*
- *Swiftless deploy for intermediate images*
- *HTTP(S) Based Deploy Support*
- *Support for iLO driver with Standalone Ironic*
- *RAID Support*
- *Disk Erase Support*
- *Initiating firmware update as manual clean step*
- *Smart Update Manager (SUM) based firmware update*
- *Updating security parameters as manual clean step*
- *Update Minimum Password Length security parameter as manual clean step*
- *Update Authentication Failure Logging security parameter as manual clean step*
- *Create Certificate Signing Request(CSR) as manual clean step*
- *Add HTTPS Certificate as manual clean step*
- *Activating iLO Advanced license as manual clean step*
- *Removing CA certificates from iLO as manual clean step*

- *Firmware based UEFI iSCSI boot from volume support*
- *Certificate based validation in iLO*
- *Rescue mode support*
- *Inject NMI support*
- *Soft power operation support*
- *BIOS configuration support*
- *IPv6 support*
- *Layer 3 or DHCP-less ramdisk booting*
- *Events subscription*

Apart from above features hardware type `ilo5` also supports following features:

- *Out of Band RAID Support*
- *Out of Band Sanitize Disk Erase Support*
- *Out of Band One Button Secure Erase Support*
- *UEFI-HTTPS Boot support*

Hardware interfaces

The `ilo` hardware type supports following hardware interfaces:

- **bios**
  Supports `ilo` and `no-bios`. The default is `ilo`. They can be enabled by using the *`DEFAULT.enabled_bios_interfaces`* option in `ironic.conf` as given below:

```
[DEFAULT]
enabled_hardware_types = ilo
enabled_bios_interfaces = ilo,no-bios
```

- **boot**
  Supports `ilo-virtual-media`, `ilo-pxe` and `ilo-ipxe`. The default is `ilo-virtual-media`. The `ilo-virtual-media` interface provides security enhanced PXE-less deployment by using iLO virtual media to boot up the bare metal node. The `ilo-pxe` and `ilo-ipxe` interfaces use PXE and iPXE respectively for deployment(just like *PXE boot*). These interfaces do not require iLO Advanced license. They can be enabled by using the *`DEFAULT.enabled_boot_interfaces`* option in `ironic.conf` as given below:

```
[DEFAULT]
enabled_hardware_types = ilo
enabled_boot_interfaces = ilo-virtual-media,ilo-pxe,ilo-ipxe
```

- **console**
  Only Supports `no-console`. It can be enabled by using the *`DEFAULT.enabled_console_interfaces`* option in `ironic.conf` as given below:

```
[DEFAULT]
enabled_hardware_types = ilo
enabled_console_interfaces = no-console
```

- **inspect**
  Supports `ilo` and `agent`. The default is `ilo`. They can be enabled by using the *DEFAULT.enabled_inspect_interfaces* option in `ironic.conf` as given below:

```
[DEFAULT]
enabled_hardware_types = ilo
enabled_inspect_interfaces = ilo,agent
```

- **management**
  Supports only `ilo`. It can be enabled by using the *DEFAULT.enabled_management_interfaces* option in `ironic.conf` as given below:

```
[DEFAULT]
enabled_hardware_types = ilo
enabled_management_interfaces = ilo
```

- **power**
  Supports only `ilo`. It can be enabled by using the *DEFAULT.enabled_power_interfaces* option in `ironic.conf` as given below:

```
[DEFAULT]
enabled_hardware_types = ilo
enabled_power_interfaces = ilo
```

- **raid**
  Supports `agent` and `no-raid`. The default is `no-raid`. They can be enabled by using the *DEFAULT.enabled_raid_interfaces* option in `ironic.conf` as given below:

```
[DEFAULT]
enabled_hardware_types = ilo
enabled_raid_interfaces = agent,no-raid
```

- **storage**
  Supports `cinder` and `noop`. The default is `noop`. They can be enabled by using the *DEFAULT.enabled_storage_interfaces* option in `ironic.conf` as given below:

```
[DEFAULT]
enabled_hardware_types = ilo
enabled_storage_interfaces = cinder,noop
```

  Note

  The storage interface `cinder` is supported only when corresponding boot interface of the `ilo` hardware type based node is `ilo-pxe` or `ilo-ipxe`. Please refer to *Boot From Volume* for configuring `cinder` as a storage interface.

- **rescue**
  Supports `agent` and `no-rescue`. The default is `no-rescue`. They can be enabled by using the *DEFAULT.enabled_rescue_interfaces* option in `ironic.conf` as given below:

```
[DEFAULT]
enabled_hardware_types = ilo
enabled_rescue_interfaces = agent,no-rescue
```

- **vendor**
  Supports `ilo`, `ilo-redfish` and `no-vendor`. The default is `ilo`. They can be enabled by using the *DEFAULT.enabled_vendor_interfaces* option in `ironic.conf` as given below:

```
[DEFAULT]
enabled_hardware_types = ilo
enabled_vendor_interfaces = ilo,ilo-redfish,no-vendor
```

The `ilo5` hardware type supports all the `ilo` interfaces described above, except for `boot` and `raid` interfaces. The details of `boot` and `raid` interfaces is as under:

- **raid**
  Supports `ilo5` and `no-raid`. The default is `ilo5`. They can be enabled by using the *DEFAULT.enabled_raid_interfaces* option in `ironic.conf` as given below:

```
[DEFAULT]
enabled_hardware_types = ilo5
enabled_raid_interfaces = ilo5,no-raid
```

- **boot**
  Supports `ilo-uefi-https` apart from the other boot interfaces supported by `ilo` hardware type. This can be enabled by using the *DEFAULT.enabled_boot_interfaces* option in `ironic.conf` as given below:

```
[DEFAULT]
enabled_hardware_types = ilo5
enabled_boot_interfaces = ilo-uefi-https,ilo-virtual-media
```

The `ilo` and `ilo5` hardware type support all standard `deploy` and `network` interface implementations, see *Enabling hardware interfaces* for details.

The following command can be used to enroll a ProLiant node with `ilo` hardware type:

```
baremetal node create \
    --driver ilo \
    --deploy-interface direct \
    --raid-interface agent \
    --rescue-interface agent \
    --driver-info ilo_address=<ilo-ip-address> \
    --driver-info ilo_username=<ilo-username> \
    --driver-info ilo_password=<ilo-password> \
    --driver-info deploy_iso=<glance-uuid-of-deploy-iso> \
    --driver-info rescue_iso=<glance-uuid-of-rescue-iso>
```

Note

The fields `deploy_iso` and `rescue_iso` used to be called `ilo_deploy_iso` and `ilo_rescue_iso` before the Xena release.

The following command can be used to enroll a ProLiant node with `ilo5` hardware type:

```
baremetal node create \
    --driver ilo5 \
    --deploy-interface direct \
    --raid-interface ilo5 \
    --rescue-interface agent \
    --driver-info ilo_address=<ilo-ip-address> \
    --driver-info ilo_username=<ilo-username> \
    --driver-info ilo_password=<ilo-password> \
    --driver-info deploy_iso=<glance-uuid-of-deploy-iso> \
    --driver-info rescue_iso=<glance-uuid-of-rescue-iso>
```

Please refer to *Enabling drivers and hardware types* for detailed explanation of hardware type.

Node configuration

- Each node is configured for `ilo` and `ilo5` hardware type by setting the following ironic node objects properties in `driver_info`:
  - `ilo_address`: IP address or hostname of the iLO.
  - `ilo_username`: Username for the iLO with administrator privileges.
  - `ilo_password`: Password for the above iLO user.
  - `client_port`: (optional) Port to be used for iLO operations if you are using a custom port on the iLO. Default port used is 443.
  - `client_timeout`: (optional) Timeout for iLO operations. Default timeout is 60 seconds.
  - `ca_file`: (optional) CA certificate file to validate iLO.
- The following properties are also required in node objects `driver_info` if `ilo-virtual-media` boot interface is used:
  - `deploy_iso`: The glance UUID of the deploy ramdisk ISO image.
  - `instance info/boot_iso` property to be either boot iso Glance UUID or a HTTP(S) URL. This is optional property and is used with *Booting a Ramdisk or an ISO*.

    Note

    The `boot_iso` property used to be called `ilo_boot_iso` before the Xena release.

  - `rescue_iso`: The glance UUID of the rescue ISO image. This is optional property and is used when `rescue` interface is set to `agent`.
- The following properties are also required in node objects `driver_info` if `ilo-pxe` or `ilo-ipxe` boot interface is used:
  - `deploy_kernel`: The glance UUID or a HTTP(S) URL of the deployment kernel.

  - `deploy_ramdisk`: The glance UUID or a HTTP(S) URL of the deployment ramdisk.
  - `rescue_kernel`: The glance UUID or a HTTP(S) URL of the rescue kernel. This is optional property and is used when `rescue` interface is set to `agent`.
  - `rescue_ramdisk`: The glance UUID or a HTTP(S) URL of the rescue ramdisk. This is optional property and is used when `rescue` interface is set to `agent`.

- The following properties are also required in node objects `driver_info` if `ilo-uefi-https` boot interface is used for `ilo5` hardware type:
  - `deploy_kernel`: The glance UUID or a HTTPS URL of the deployment kernel.
  - `deploy_ramdisk`: The glance UUID or a HTTPS URL of the deployment ramdisk.
  - `bootloader`: The glance UUID or a HTTPS URL of the bootloader.
  - `rescue_kernel`: The glance UUID or a HTTPS URL of the rescue kernel. This is optional property and is used when `rescue` interface is set to `agent`.
  - `rescue_ramdisk`: The glance UUID or a HTTP(S) URL of the rescue ramdisk. This is optional property and is used when `rescue` interface is set to `agent`.

Note

`ilo-uefi-https` boot interface is supported by only `ilo5` hardware type. If the images are not hosted in glance, the references must be HTTPS URLs hosted by secure webserver. This boot interface can be used only when the current boot mode is `UEFI`.

Note

The fields `deploy_kernel`, `deploy_ramdisk`, `rescue_kernel` `rescue_ramdisk` and `bootloader` used to have an `ilo_` prefix before the Xena release.

- The following parameters are mandatory in `driver_info` if `ilo-inspect` inspect interface is used and SNMPv3 inspection (`SNMPv3 Authentication` in HPE iLO4 User Guide) is desired:
  - `snmp_auth_user` : The SNMPv3 user.
  - `snmp_auth_prot_password` : The auth protocol pass phrase.
  - `snmp_auth_priv_password` : The privacy protocol pass phrase.

  The following parameters are optional for SNMPv3 inspection:
  - `snmp_auth_protocol` : The Auth Protocol. The valid values are MD5 and SHA. The iLO default value is MD5.
  - `snmp_auth_priv_protocol` : The Privacy protocol. The valid values are AES and DES. The iLO default value is DES.

Note

If configuration values for `ca_file`, `client_port` and `client_timeout` are not provided in the `driver_info` of the node, the corresponding config variables defined under `[ilo]` section in ironic.conf will be used.

Prerequisites

- proliantutils is a python package which contains a set of modules for managing HPE ProLiant hardware.

  Install `proliantutils` module on the ironic conductor node. Minimum version required is 2.8.0:

  ```
  $ pip install "proliantutils>=2.8.0"
  ```

- `ipmitool` command must be present on the service node(s) where `ironic-conductor` is running. On most distros, this is provided as part of the `ipmitool` package. Please refer to *Hardware Inspection Support* for more information on recommended version.

Different configuration for ilo hardware type

Glance Configuration

1. Configure Glance image service with its storage backend as Swift.
2. Set a temp-url key for Glance user in Swift. For example, if you have configured Glance with user `glance-swift` and tenant as `service`, then run the below command:

   ```
   swift --os-username=service:glance-swift post -m temp-url-
   ↪key:mysecretkeyforglance
   ```

3. Fill the required parameters in the `[glance]` section in `/etc/ironic/ironic.conf`. Normally you would be required to fill in the following details:

   ```
   [glance]
   swift_temp_url_key=mysecretkeyforglance
   swift_endpoint_url=https://10.10.1.10:8080
   swift_api_version=v1
   swift_account=AUTH_51ea2fb400c34c9eb005ca945c0dc9e1
   swift_container=glance
   ```

   The details can be retrieved by running the below command:

   ```
   $ swift --os-username=service:glance-swift stat -v | grep -i url

   StorageURL:     http://10.10.1.10:8080/v1/AUTH_
   ↪51ea2fb400c34c9eb005ca945c0dc9e1
   Meta Temp-Url-Key: mysecretkeyforglance
   ```

4. Swift must be accessible with the same admin credentials configured in Ironic. For example, if Ironic is configured with the below credentials in `/etc/ironic/ironic.conf`:

   ```
   [keystone_authtoken]
   admin_password = password
   admin_user = ironic
   admin_tenant_name = service
   ```

   Ensure `auth_version` in `keystone_authtoken` to 2.

   Then, the below command should work.:

```
$ swift --os-username ironic --os-password password --os-tenant-name␣
↪service --auth-version 2 stat

                     Account: AUTH_22af34365a104e4689c46400297f00cb
                  Containers: 2
                     Objects: 18
                       Bytes: 1728346241
Objects in policy "policy-0": 18
  Bytes in policy "policy-0": 1728346241
           Meta Temp-Url-Key: mysecretkeyforglance
                 X-Timestamp: 1409763763.84427
                  X-Trans-Id: tx51de96a28f27401eb2833-005433924b
                Content-Type: text/plain; charset=utf-8
               Accept-Ranges: bytes
```

5. Restart the Ironic conductor service:

```
$ service ironic-conductor restart
```

Web server configuration on conductor

- The HTTP(S) web server can be configured in many ways. For apache web server on Ubuntu, refer here
- Following config variables need to be set in `/etc/ironic/ironic.conf`:
  - `use_web_server_for_images` in `[ilo]` section:

```
[ilo]
use_web_server_for_images = True
```

  - `http_url` and `http_root` in `[deploy]` section:

```
[deploy]
# Ironic compute node's http root path. (string value)
http_root=/httpboot

# Ironic compute node's HTTP server URL. Example:
# http://192.1.2.3:8080 (string value)
http_url=http://192.168.0.2:8080
```

`use_web_server_for_images`: If the variable is set to `false`, the `ilo-virtual-media` boot interface uses swift containers to host the intermediate floppy image and the boot ISO. If the variable is set to `true`, it uses the local web server for hosting the intermediate files. The default value for `use_web_server_for_images` is False.

`http_url`: The value for this variable is prefixed with the generated intermediate files to generate a URL which is attached in the virtual media.

`http_root`: It is the directory location to which ironic conductor copies the intermediate floppy image and the boot ISO.

Note

HTTPS is strongly recommended over HTTP web server configuration for security enhancement. The `ilo-virtual-media` boot interface will send the instances configdrive over an encrypted channel if web server is HTTPS enabled. However for `ilo-uefi-https` boot interface HTTPS webserver is mandatory as this interface only supports HTTPS URLs.

Enable driver

1. Build a deploy ISO (and kernel and ramdisk) image, see *Building or downloading a deploy ramdisk image*
2. See *Glance Configuration* for configuring glance image service with its storage backend as `swift`.
3. Upload this image to Glance:

```
glance image-create --name deploy-ramdisk.iso --disk-format iso --
↪container-format bare < deploy-ramdisk.iso
```

4. Enable hardware type and hardware interfaces in `/etc/ironic/ironic.conf`:

```
[DEFAULT]
enabled_hardware_types = ilo
enabled_bios_interfaces = ilo
enabled_boot_interfaces = ilo-virtual-media,ilo-pxe,ilo-ipxe
enabled_power_interfaces = ilo
enabled_console_interfaces = no-console
enabled_raid_interfaces = agent
enabled_management_interfaces = ilo
enabled_inspect_interfaces = ilo
enabled_rescue_interfaces = agent
```

5. Restart the ironic conductor service:

```
$ service ironic-conductor restart
```

Optional functionalities for the `ilo` hardware type

Boot mode support

The hardware type `ilo` supports automatic detection and setting of boot mode (Legacy BIOS or UEFI).

- When boot mode capability is not configured:
  - If config variable `default_boot_mode` in `[ilo]` section of ironic configuration file is set to either bios or uefi, then iLO driver uses that boot mode for provisioning the baremetal ProLiant servers.
  - If the pending boot mode is set on the node then iLO driver uses that boot mode for provisioning the baremetal ProLiant servers.
  - If the pending boot mode is not set on the node then iLO driver uses uefi boot mode for UEFI capable servers and bios when UEFI is not supported.

- When boot mode capability is configured, the driver sets the pending boot mode to the configured value.
- Only one boot mode (either `uefi` or `bios`) can be configured for the node.
- If the operator wants a node to boot always in `uefi` mode or `bios` mode, then they may use `capabilities` parameter within `properties` field of an ironic node.

  To configure a node in `uefi` mode, then set `capabilities` as below:

```
baremetal node set <node> --property capabilities='boot_mode:uefi'
```

  Nodes having `boot_mode` set to `uefi` may be requested by adding an `extra_spec` to the nova flavor:

```
openstack flavor set ironic-test-3 --property capabilities:boot_mode="uefi
↪"
openstack server create --flavor ironic-test-3 --image test-image␣
↪instance-1
```

  If `capabilities` is used in `extra_spec` as above, nova scheduler (`ComputeCapabilitiesFilter`) will match only ironic nodes which have the `boot_mode` set appropriately in `properties/capabilities`. It will filter out rest of the nodes.

  The above facility for matching in nova can be used in heterogeneous environments where there is a mix of `uefi` and `bios` machines, and operator wants to provide a choice to the user regarding boot modes. If the flavor doesnt contain `boot_mode` then nova scheduler will not consider boot mode as a placement criteria, hence user may get either a BIOS or UEFI machine that matches with user specified flavors.

The automatic boot ISO creation for UEFI boot mode has been enabled in Kilo. The manual creation of boot ISO for UEFI boot mode is also supported. For the latter, the boot ISO for the deploy image needs to be built separately and the deploy images `boot_iso` property in glance should contain the glance UUID of the boot ISO. For building boot ISO, add `iso` element to the diskimage-builder command to build the image. For example:

```
disk-image-create ubuntu baremetal iso
```

UEFI Secure Boot Support

The hardware type `ilo` supports secure boot deploy, see *UEFI secure boot mode* for details.

iLO specific notes:

In UEFI secure boot, digitally signed bootloader should be able to validate digital signatures of kernel during boot process. This requires that the bootloader contains the digital signatures of the kernel. For the `ilo-virtual-media` boot interface, it is recommended that `boot_iso` property for user image contains the glance UUID of the boot ISO. If `boot_iso` property is not updated in glance for the user image, it would create the `boot_iso` using bootloader from the deploy iso. This `boot_iso` will be able to boot the user image in UEFI secure boot environment only if the bootloader is signed and can validate digital signatures of user image kernel.

For HPE ProLiant Gen9 servers, one can enroll public key using iLO System Utilities UI. Please refer to section `Accessing Secure Boot options` in HP UEFI System Utilities User Guide. One can also refer to white paper on Secure Boot for Linux on HP ProLiant servers for additional details.

For more up-to-date information, refer iLO driver wiki page

Node Cleaning Support

The hardware type `ilo` and `ilo5` supports node cleaning.

For more information on node cleaning, see *Node cleaning*

Supported Automated Cleaning Operations

- The automated cleaning operations supported are:
  - `reset_bios_to_default`: Resets system ROM settings to default. By default, enabled with priority 10. This clean step is supported only on Gen9 and above servers.
  - `reset_secure_boot_keys_to_default`: Resets secure boot keys to manufacturers defaults. This step is supported only on Gen9 and above servers. By default, enabled with priority 20 .
  - `reset_ilo_credential`: Resets the iLO password, if `ilo_change_password` is specified as part of nodes driver_info. By default, enabled with priority 30.
  - `clear_secure_boot_keys`: Clears all secure boot keys. This step is supported only on Gen9 and above servers. By default, this step is disabled.
  - `reset_ilo`: Resets the iLO. By default, this step is disabled.
  - `erase_devices`: An inband clean step that performs disk erase on all the disks including the disks visible to OS as well as the raw disks visible to Smart Storage Administrator (SSA). This step supports erasing of the raw disks visible to SSA in Proliant servers only with the ramdisk created using diskimage-builder from Ocata release. By default, this step is disabled. See *Disk Erase Support* for more details.
- For supported in-band cleaning operations, see *In-band vs out-of-band*.
- All the automated cleaning steps have an explicit configuration option for priority. In order to disable or change the priority of the automated clean steps, respective configuration option for priority should be updated in ironic.conf.
- Updating clean step priority to 0, will disable that particular clean step and will not run during automated cleaning.
- Configuration Options for the automated clean steps are listed under `[ilo]` and `[deploy]` section in ironic.conf

```
[ilo]
clean_priority_reset_ilo=0
clean_priority_reset_bios_to_default=10
clean_priority_reset_secure_boot_keys_to_default=20
clean_priority_clear_secure_boot_keys=0
clean_priority_reset_ilo_credential=30

[deploy]
erase_devices_priority=0
```

For more information on node automated cleaning, see *Automated cleaning*

Supported Manual Cleaning Operations

- The manual cleaning operations supported are:

  activate_license:
  Activates the iLO Advanced license. This is an out-of-band manual cleaning step associated with the `management` interface. See *Activating iLO Advanced license as manual clean step* for user guidance on usage. Please note that this operation cannot be performed using the `ilo-virtual-media` boot interface as it needs this type of advanced license already active to use virtual media to boot into to start cleaning operation. Virtual media is an advanced feature. If an advanced license is already active and the user wants to overwrite the current license key, for example in case of a multi-server activation key delivered with a flexible-quantity kit or after completing an Activation Key Agreement (AKA), then the driver can still be used for executing this cleaning step.

  clear_ca_certificates:
  Removes the CA certificates from iLO. See *Removing CA certificates from iLO as manual clean step* for user guidance on usage.

  apply_configuration:
  Applies given BIOS settings on the node. See *BIOS configuration support*. This step is part of the `bios` interface.

  factory_reset:
  Resets the BIOS settings on the node to factory defaults. See *BIOS configuration support*. This step is part of the `bios` interface.

  create_configuration:
  Applies RAID configuration on the node. See *RAID Configuration* for more information. This step is part of the `raid` interface.

  delete_configuration:
  Deletes RAID configuration on the node. See *RAID Configuration* for more information. This step is part of the `raid` interface.

  update_firmware:
  Updates the firmware of the devices. Also an out-of-band step associated with the `management` interface. See *Initiating firmware update as manual clean step* for user guidance on usage. The supported devices for firmware update are: `ilo`, `cpld`, `power_pic`, `bios` and `chassis`. Please refer to below table for their commonly used descriptions.

Device	Description
`ilo`	BMC for HPE ProLiant servers
`cpld`	System programmable logic device
`power_pic`	Power management controller
`bios`	HPE ProLiant System ROM
`chassis`	System chassis device

  Some devices firmware cannot be updated via this method, such as: storage controllers, host bus adapters, disk drive firmware, network interfaces and Onboard Administrator (OA).

  update_firmware_sum:
  Updates all or list of user specified firmware components on the node using Smart Update

Manager (SUM). It is an inband step associated with the `management` interface. See *Smart Update Manager (SUM) based firmware update* for more information on usage.

`security_parameters_update`:
Updates the Security Parameters. See *Updating security parameters as manual clean step* for user guidance on usage. The supported security parameters for this clean step are: `Password_Complexity`, `RequiredLoginForiLORBSU`, `IPMI/DCMI_Over_LAN`, `RequireHostAuthentication` and `Secure_Boot`.

`update_minimum_password_length`:
Updates the Minimum Password Length security parameter. See *Update Minimum Password Length security parameter as manual clean step* for user guidance on usage.

`update_auth_failure_logging_threshold`:
Updates the Authentication Failure Logging security parameter. See *Update Authentication Failure Logging security parameter as manual clean step* for user guidance on usage.

`create_csr`:
Creates the certificate signing request. See *Create Certificate Signing Request(CSR) as manual clean step* for user guidance on usage.

`add_https_certificate`:
Adds the signed HTTPS certificate to the iLO. See *Add HTTPS Certificate as manual clean step* for user guidance on usage.

- iLO with firmware version 1.5 is minimally required to support all the operations.

For more information on node manual cleaning, see *Manual cleaning*

Node Deployment Customization

The hardware type `ilo` and `ilo5` supports customization of node deployment via deploy templates, see *Using deploy steps and templates*.

The supported deploy steps are:

- **`apply_configuration`:**
  Applies given BIOS settings on the node. See *BIOS configuration support*. This step is part of the `bios` interface.
- **`factory_reset`:**
  Resets the BIOS settings on the node to factory defaults. See *BIOS configuration support*. This step is part of the `bios` interface.
- **`reset_bios_to_default`:**
  Resets system ROM settings to default. This step is supported only on Gen9 and above servers. This step is part of the `management` interface.
- **`reset_secure_boot_keys_to_default`:**
  Resets secure boot keys to manufacturers defaults. This step is supported only on Gen9 and above servers. This step is part of the `management` interface.
- **`reset_ilo_credential`:**
  Resets the iLO password. The password need to be specified in `ilo_password` argument of the step. This step is part of the `management` interface.
- **`clear_secure_boot_keys`:**
  Clears all secure boot keys. This step is supported only on Gen9 and above servers. This step

is part of the `management` interface.

- **`reset_ilo`:**
  Resets the iLO. This step is part of the `management` interface.
- **`update_firmware`:**
  Updates the firmware of the devices. This step is part of the `management` interface. See *Initiating firmware update as manual clean step* for user guidance on usage. The supported devices for firmware update are: `ilo`, `cpld`, `power_pic`, `bios` and `chassis`. This step is part of `management` interface. Please refer to below table for their commonly used descriptions.

Device	Description
`ilo`	BMC for HPE ProLiant servers
`cpld`	System programmable logic device
`power_pic`	Power management controller
`bios`	HPE ProLiant System ROM
`chassis`	System chassis device

  Some devices firmware cannot be updated via this method, such as: storage controllers, host bus adapters, disk drive firmware, network interfaces and Onboard Administrator (OA).
- **`flash_firmware_sum`:**
  Updates all or list of user specified firmware components on the node using Smart Update Manager (SUM). It is an inband step associated with the `management` interface. See *Smart Update Manager (SUM) based firmware update* for more information on usage.
- **`apply_configuration`:**
  Applies RAID configuration on the node. See *RAID Configuration* for more information. This step is part of the `raid` interface.

Example of using deploy template with the Compute service

Create a deploy template with a single step:

```
baremetal deploy template create \
    CUSTOM_HYPERTHREADING_ON \
    --steps '[{"interface": "bios", "step": "apply_configuration", "args": {
↪"settings": [{"name": "ProcHyperthreading", "value": "Enabled"}]}, "priority
↪": 150}]'
```

Add the trait `CUSTOM_HYPERTHREADING_ON` to the node represented by `$node_ident`:

```
baremetal node add trait $node_ident CUSTOM_HYPERTHREADING_ON
```

Update the flavor `bm-hyperthreading-on` in the Compute service with the following property:

```
openstack flavor set --property trait:CUSTOM_HYPERTHREADING_ON=required bm-
↪hyperthreading-on
```

Creating a Compute instance with this flavor will ensure that the instance is scheduled only to Bare Metal nodes with the `CUSTOM_HYPERTHREADING_ON` trait. When an instance is created using the `bm-hyperthreading-on` flavor, then the deploy steps of deploy template

CUSTOM_HYPERTHREADING_ON will be executed during the deployment of the scheduled node, causing Hyperthreading to be enabled in the nodes BIOS configuration.

Hardware Inspection Support

The hardware type `ilo` supports hardware inspection.

Note

- The disk size is returned by RIBCL/RIS only when RAID is preconfigured on the storage. If the storage is Direct Attached Storage, then RIBCL/RIS fails to get the disk size.
- The SNMPv3 inspection gets disk size for all types of storages. If RIBCL/RIS is unable to get disk size and SNMPv3 inspection is requested, the proliantutils does SNMPv3 inspection to get the disk size. If proliantutils is unable to get the disk size, it raises an error. This feature is available in proliantutils release version >= 2.2.0.
- The iLO must be updated with SNMPv3 authentication details. Please refer to the section `SNMPv3 Authentication` in HPE iLO4 User Guide for setting up authentication details on iLO. The following parameters are mandatory to be given in driver_info for SNMPv3 inspection:
  - `snmp_auth_user` : The SNMPv3 user.
  - `snmp_auth_prot_password` : The auth protocol pass phrase.
  - `snmp_auth_priv_password` : The privacy protocol pass phrase.

  The following parameters are optional for SNMPv3 inspection:
  - `snmp_auth_protocol` : The Auth Protocol. The valid values are MD5 and SHA. The iLO default value is MD5.
  - `snmp_auth_priv_protocol` : The Privacy protocol. The valid values are AES and DES. The iLO default value is DES.

The inspection process will discover the following properties:

- `memory_mb`: memory size
- `cpu_arch`: cpu architecture
- `local_gb`: disk size

Inspection can also discover the following extra capabilities for iLO driver:

- `ilo_firmware_version`: iLO firmware version
- `rom_firmware_version`: ROM firmware version
- `secure_boot`: secure boot is supported or not. The possible values are true or false. The value is returned as true if secure boot is supported by the server.
- `server_model`: server model
- `pci_gpu_devices`: number of gpu devices connected to the bare metal.
- `nic_capacity`: the max speed of the embedded NIC adapter.
- `sriov_enabled`: true, if server has the SRIOV supporting NIC.

- has_rotational: true, if server has HDD disk.
- has_ssd: true, if server has SSD disk.
- has_nvme_ssd: true, if server has NVME SSD disk.
- cpu_vt: true, if server supports cpu virtualization.
- hardware_supports_raid: true, if RAID can be configured on the server using RAID controller.
- nvdimm_n: true, if server has NVDIMM_N type of persistent memory.
- persistent_memory: true, if server has persistent memory.
- logical_nvdimm_n: true, if server has logical NVDIMM_N configured.
- rotational_drive_<speed>_rpm: The capabilities rotational_drive_4800_rpm, rotational_drive_5400_rpm, rotational_drive_7200_rpm, rotational_drive_10000_rpm and rotational_drive_15000_rpm are set to true if the server has HDD drives with speed of 4800, 5400, 7200, 10000 and 15000 rpm respectively.
- logical_raid_level_<raid_level>: The capabilities logical_raid_level_0, logical_raid_level_1, logical_raid_level_2, logical_raid_level_5, logical_raid_level_6, logical_raid_level_10, logical_raid_level_50 and logical_raid_level_60 are set to true if any of the raid levels among 0, 1, 2, 5, 6, 10, 50 and 60 are configured on the system.
- overall_security_status: Ok or Risk or Ignored as returned by iLO security dashboard. iLO computes the overall security status by evaluating the security status for each of the security parameters. Admin needs to fix the actual parameters and then re-inspect so that iLO can recompute the overall security status. If the all security params, whose security_status is Risk, have the Ignore field set to True, then iLO sets the overall security status value as Ignored. All the security params must have the security_status as Ok for the overall_security_status to have the value as Ok.
- last_firmware_scan_status: Ok or Risk as returned by iLO security dashboard. This denotes security status of the last firmware scan done on the system. If it is Risk, the recommendation is to run clean_step update_firmware_sum without any specific firmware components so that firmware is updated for all the components using latest SPP (Service Provider Pack) ISO and then re-inspect to get the security status again.
- security_override_switch: Ok or Risk as returned by iLO security dashboard. This is disable/enable login to the iLO using credentials. This can be toggled only by physical visit to the bare metal.
- gpu_<vendor>_count: Integer value. The capability name is dynamically formed as gpu_<vendor>_count. The vendor name is replaced in the <vendor>. If the vendor name is not returned by the hardware, then vendor ID in hexadecimal form is replaced in the capability name. Examples: {gpu_Nvidia_count: 1}, {gpu_0x102b_count: 1}.
- gpu_<vendor_device_name>_count: Integer value. The capability name is formed dynamically by replacing the gpu device name as returned by ilo in <vendor_device_name>. Examples: {gpu_Nvidia_Tesla_M10_count: 1}, {gpu_Embedded_Video_Controller_count: 1}
- gpu_<vendor_device_name>: Boolean. The capability name is formed dynamically by replacing the gpu device name as returned by ilo in <vendor_device_name>. Examples: {gpu_Nvidia_Tesla_M10: True}, {gpu_Embedded_Video_Controller: True}

Note

- The capability `nic_capacity` can only be discovered if ipmitool version >= 1.8.15 is used on the conductor. The latest version can be downloaded from here.
- The iLO firmware version needs to be 2.10 or above for nic_capacity to be discovered.
- To discover IPMI based attributes you need to enable iLO feature IPMI/DCMI over LAN Access on iLO4 and iLO5 management engine.
- The proliantutils returns only active NICs for Gen10 ProLiant HPE servers. The user would need to delete the ironic ports corresponding to inactive NICs for Gen8 and Gen9 servers as proliantutils returns all the discovered (active and otherwise) NICs for Gen8 and Gen9 servers and ironic ports are created for all of them. Inspection logs a warning if the node under inspection is Gen8 or Gen9.
- The security dashboard capabilities are applicable only for Gen10 ProLiant HPE servers and above. To fix the security dashboard parameters value from `Risk` to `Ok`, user need to fix the parameters separately and re-inspect to see the security status of the parameters.

The operator can specify these capabilities in nova flavor for node to be selected for scheduling:

```
openstack flavor set my-baremetal-flavor --property capabilities:server_model=
↪"<in> Gen8"

openstack flavor set my-baremetal-flavor --property capabilities:nic_capacity=
↪"10Gb"

openstack flavor set my-baremetal-flavor --property capabilities:ilo_firmware_
↪version="<in> 2.10"

openstack flavor set my-baremetal-flavor --property capabilities:has_ssd="true
↪"
```

See *Capabilities discovery* for more details and examples.

Swiftless deploy for intermediate images

The hardware type `ilo` with `ilo-virtual-media` as boot interface can deploy and boot the server with and without `swift` being used for hosting the intermediate temporary floppy image (holding metadata for deploy kernel and ramdisk) and the boot ISO. A local HTTP(S) web server on each conductor node needs to be configured. Please refer to *Web server configuration on conductor* for more information. The HTTPS web server needs to be enabled (instead of HTTP web server) in order to send management information and images in encrypted channel over HTTPS.

Note

This feature assumes that the user inputs are on Glance which uses swift as backend. If swift dependency has to be eliminated, please refer to *HTTP(S) Based Deploy Support* also.

Deploy Process

Please refer to *Swiftless deploy for intermediate images*.

HTTP(S) Based Deploy Support

The user input for the images given in `driver_info` like `deploy_iso`, `deploy_kernel` and `deploy_ramdisk` and in `instance_info` like `image_source`, `kernel`, `ramdisk` and `boot_iso` may also be given as HTTP(S) URLs.

The HTTP(S) web server can be configured in many ways. For the Apache web server on Ubuntu, refer here. The web server may reside on a different system than the conductor nodes, but its URL must be reachable by the conductor and the bare metal nodes.

Deploy Process

Please refer to *HTTP(S) based deploy*.

Support for iLO driver with Standalone Ironic

It is possible to use ironic as standalone services without other OpenStack services. The `ilo` hardware type can be used in standalone ironic. This feature is referred to as `iLO driver with standalone ironic` in this document.

Configuration

The HTTP(S) web server needs to be configured as described in *HTTP(S) Based Deploy Support* and *Web server configuration on conductor* needs to be configured for hosting intermediate images on conductor as described in *Swiftless deploy for intermediate images*.

Deploy Process

Glance and swift for partition images

Glance and swift with whole-disk images

Swiftless deploy

HTTP(S) based deploy

Standalone ironic

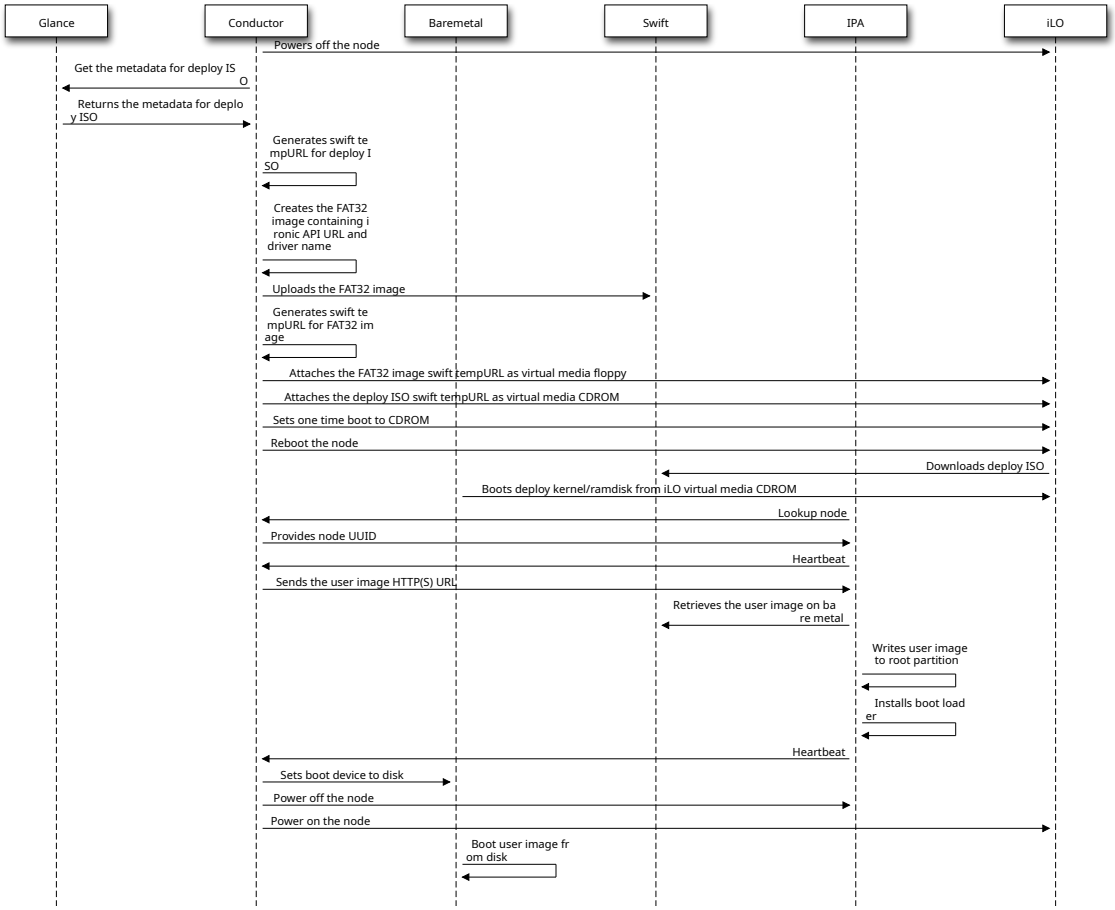
Activating iLO Advanced license as manual clean step

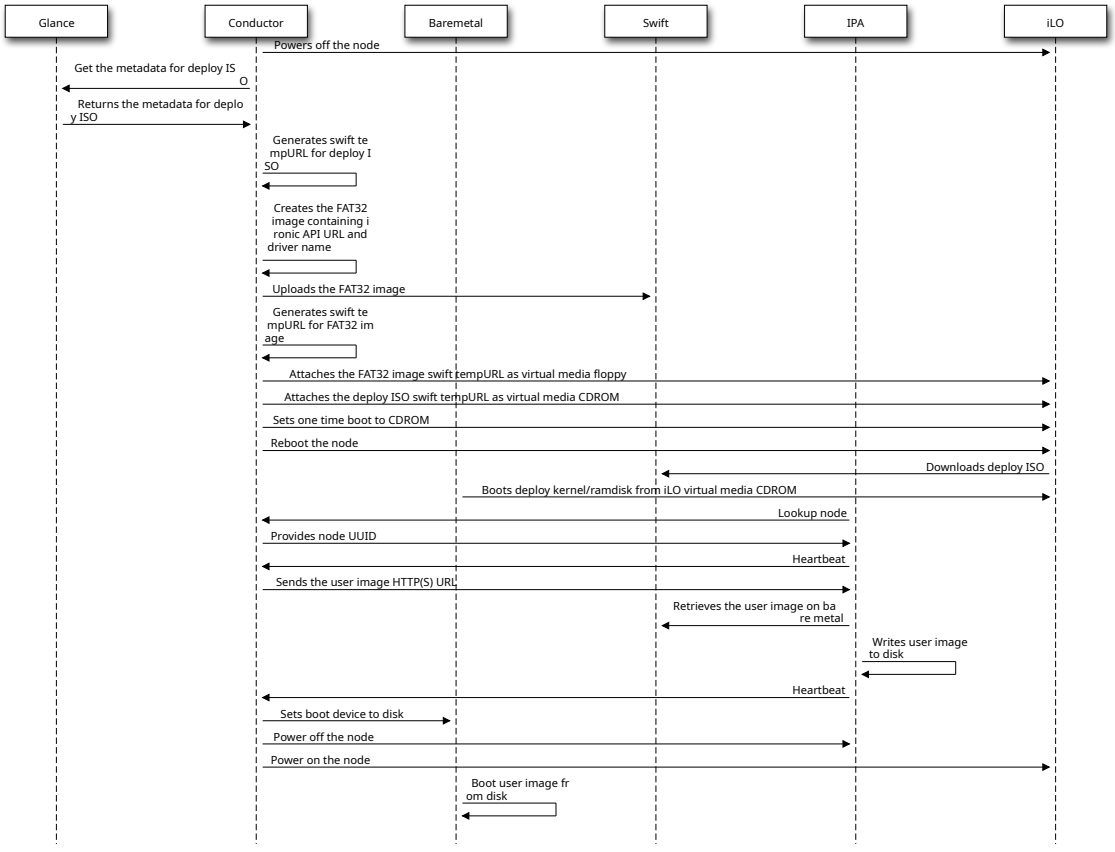
iLO driver can activate the iLO Advanced license key as a manual cleaning step. Any manual cleaning step can only be initiated when a node is in the `manageable` state. Once the manual cleaning is finished, the node will be put in the `manageable` state again. User can follow steps from *Manual cleaning* to initiate manual cleaning operation on a node.

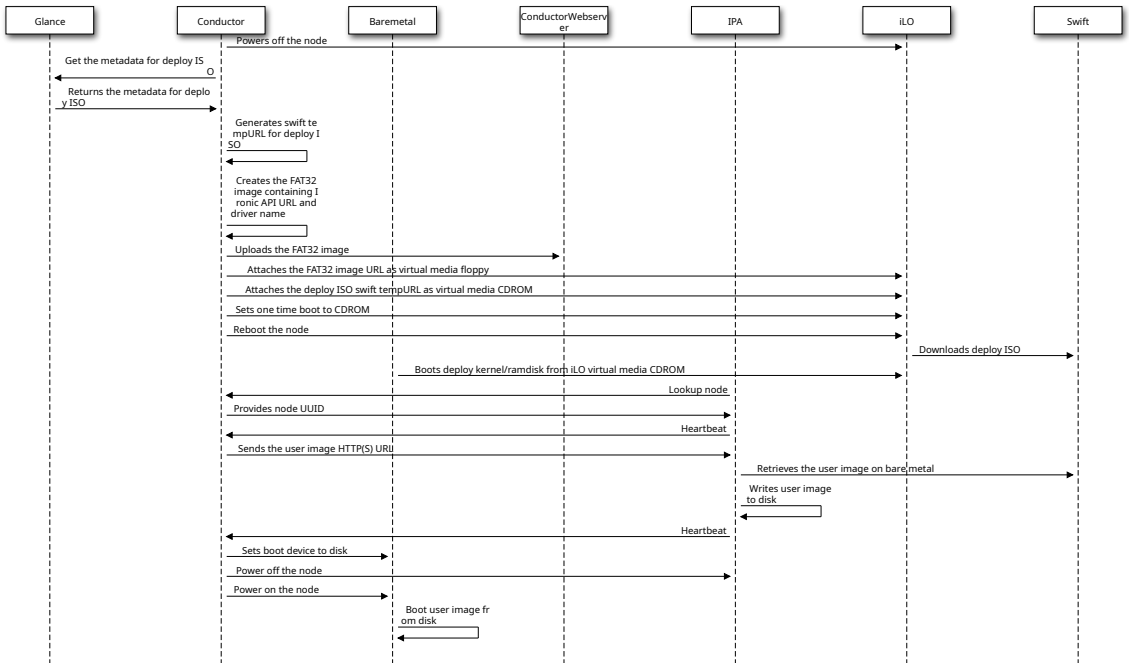
An example of a manual clean step with `activate_license` as the only clean step could be:

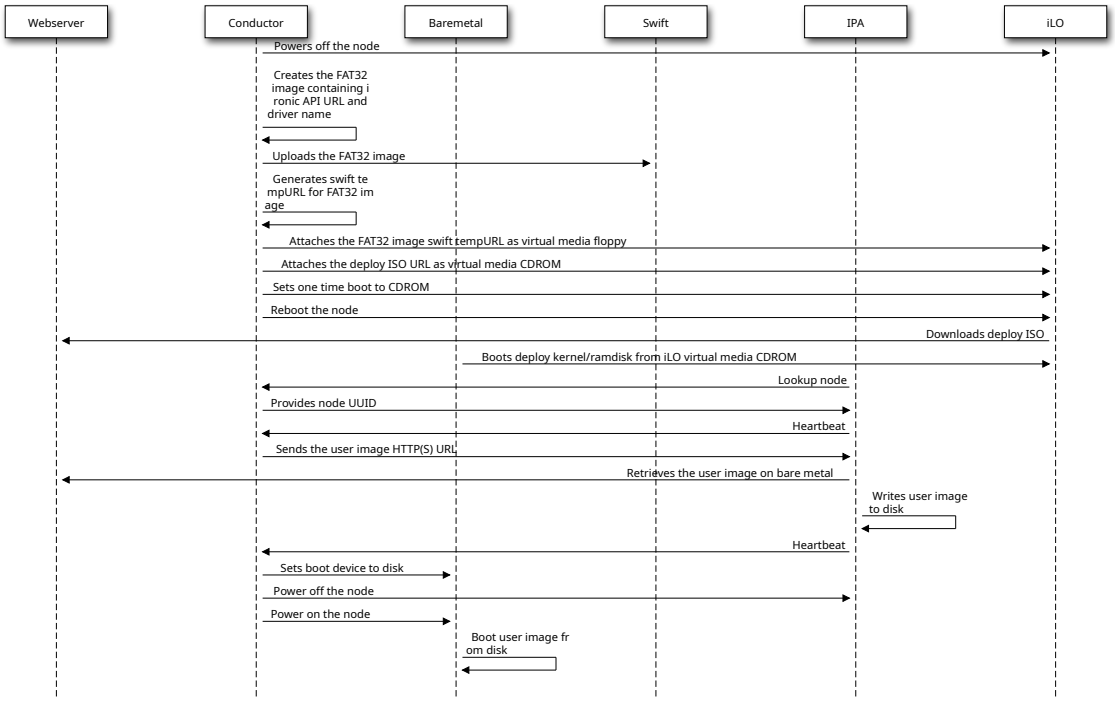
```
"clean_steps": [{
    "interface": "management",
    "step": "activate_license",
```

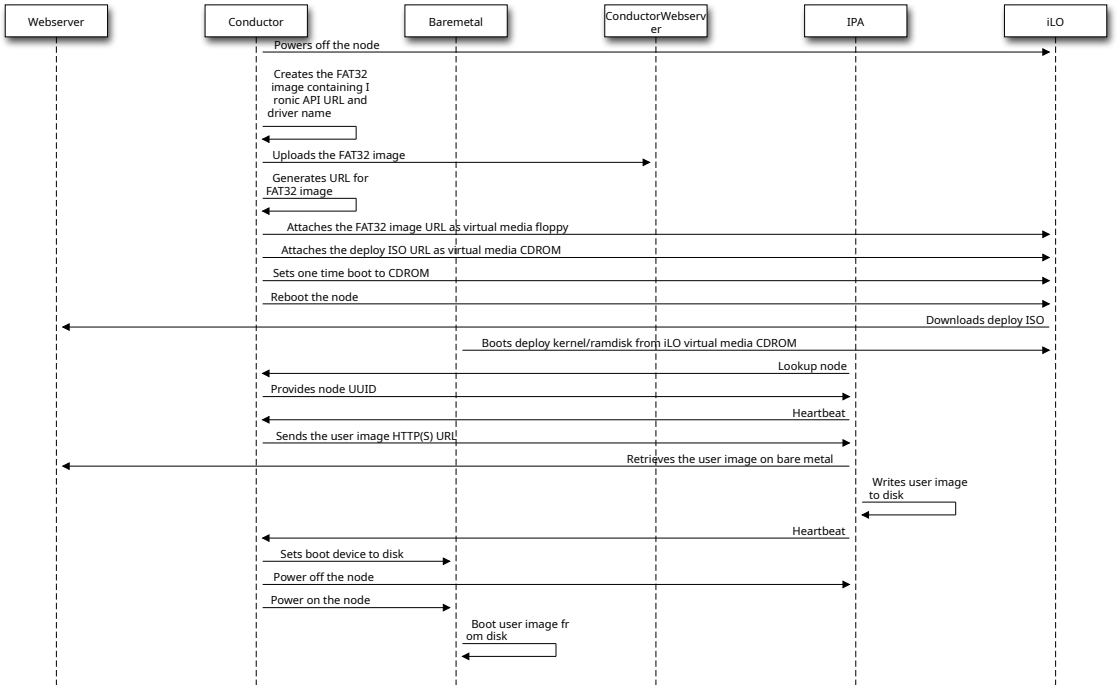
(continues on next page)


Glance
Conductor
Baremetal
Swift
IPA
iLO
Powers off the node
Get the metadata for deploy ISO
Returns the metadata for deploy ISO
Generates swift tempURL for deploy ISO
Creates the FAT32 image containing ironic API URL and driver name
Uploads the FAT32 image
Generates swift tempURL for FAT32 image
Attaches the FAT32 image swift tempURL as virtual media floppy
Attaches the deploy ISO swift tempURL as virtual media CDROM
Sets one time boot to CDROM
Reboot the node
Downloads deploy ISO
Boots deploy kernel/ramdisk from iLO virtual media CDROM
Lookup node
Provides node UUID
Heartbeat
Sends the user image HTTP(S) URL
Retrieves the user image on bare metal
Writes user image to root partition
Installs boot loader
Heartbeat
Sets boot device to disk
Power off the node
Power on the node
Boot user image from disk


Glance
Conductor
Baremetal
Swift
IPA
iLO
Powers off the node
Get the metadata for deploy ISO
Returns the metadata for deploy ISO
Generates swift tempURL for deploy ISO
Creates the FAT32 image containing ironic API URL and driver name
Uploads the FAT32 image
Generates swift tempURL for FAT32 image
Attaches the FAT32 image swift tempURL as virtual media floppy
Attaches the deploy ISO swift tempURL as virtual media CDROM
Sets one time boot to CDROM
Reboot the node
Downloads deploy ISO
Boots deploy kernel/ramdisk from iLO virtual media CDROM
Lookup node
Provides node UUID
Heartbeat
Sends the user image HTTP(S) URL
Retrieves the user image on bare metal
Writes user image to disk
Heartbeat
Sets boot device to disk
Power off the node
Power on the node
Boot user image from disk


Glance
Conductor
Baremetal
ConductorWebserver
IPA
iLO
Swift
Powers off the node
Get the metadata for deploy ISO
Returns the metadata for deploy ISO
Generates swift tempURL for deploy ISO
Creates the FAT32 image containing Ironic API URL and driver name
Uploads the FAT32 image
Attaches the FAT32 image URL as virtual media floppy
Attaches the deploy ISO swift tempURL as virtual media CDROM
Sets one time boot to CDROM
Reboot the node
Downloads deploy ISO
Boots deploy kernel/ramdisk from iLO virtual media CDROM
Lookup node
Provides node UUID
Heartbeat
Sends the user image HTTP(S) URL
Retrieves the user image on bare metal
Writes user image to disk
Heartbeat
Sets boot device to disk
Power off the node
Power on the node
Boot user image from disk


Webserver
Conductor
Baremetal
Swift
IPA
iLO
Powers off the node
Creates the FAT32 image containing ironic API URL and driver name
Uploads the FAT32 image
Generates swift tempURL for FAT32 image
Attaches the FAT32 image swift tempURL as virtual media floppy
Attaches the deploy ISO URL as virtual media CDROM
Sets one time boot to CDROM
Reboot the node
Downloads deploy ISO
Boots deploy kernel/ramdisk from iLO virtual media CDROM
Lookup node
Provides node UUID
Heartbeat
Sends the user image HTTP(S) URL
Retrieves the user image on bare metal
Writes user image to disk
Heartbeat
Sets boot device to disk
Power off the node
Power on the node
Boot user image from disk



(continued from previous page)

```
    "args": {
        "ilo_license_key": "ABC12-XXXXX-XXXXX-XXXXX-YZ345"
    }
}]
```

The different attributes of `activate_license` clean step are as follows:

Attribute	Description
`interface`	Interface of clean step, here `management`
`step`	Name of clean step, here `activate_license`
`args`	Keyword-argument entry (<name>: <value>) being passed to clean step
`args.ilo_license_key`	iLO Advanced license key to activate enterprise features. This is mandatory.

Removing CA certificates from iLO as manual clean step

iLO driver can remove the invalidated CA certificates as a manual step. Any manual cleaning step can only be initiated when a node is in the `manageable` state. Once the manual cleaning is finished, the node will be put in the `manageable` state again. User can follow steps from *Manual cleaning* to initiate manual cleaning operation on a node.

An example of a manual clean step with `clear_ca_certificates` as the only clean step could be:

```
"clean_steps": [{
    "interface": "management",
    "step": "clear_ca_certificates",
    "args": {
        "certificate_files" : ["/path/to/certsA", "/path/to/certsB"]
```

(continues on next page)

(continued from previous page)

```
    }
}]
```

The different attributes of `clear_ca_certificates` clean step are as follows:

Attribute	Description
`interface`	Interface of clean step, here `management`
`step`	Name of clean step, here `clear_ca_certificates`
`args`	Keyword-argument entry (<name>: <value>) being passed to clean step
`args.certificate_fil` This is mandatory.	List of CA certificates which are to be removed.

Initiating firmware update as manual clean step

iLO driver can invoke secure firmware update as a manual cleaning step. Any manual cleaning step can only be initiated when a node is in the `manageable` state. Once the manual cleaning is finished, the node will be put in the `manageable` state again. A user can follow steps from *Manual cleaning* to initiate manual cleaning operation on a node.

An example of a manual clean step with `update_firmware` as the only clean step could be:

```
"clean_steps": [{
    "interface": "management",
    "step": "update_firmware",
    "args": {
        "firmware_update_mode": "ilo",
        "firmware_images":[
            {
                "url": "file:///firmware_images/ilo/1.5/CP024444.scexe",
                "checksum": "a94e683ea16d9ae44768f0a65942234d",
                "component": "ilo"
            },
            {
                "url": "swift://firmware_container/cpld2.3.rpm",
                "checksum": "<md5-checksum-of-this-file>",
                "component": "cpld"
            },
            {
                "url": "http://my_address:port/firmwares/bios_vLatest.scexe",
                "checksum": "<sha256-checksum-of-this-file>",
                "component": "bios"
            },
            {
                "url": "https://my_secure_address_url/firmwares/chassis_
↪vLatest.scexe",
                "checksum": "<sha512-checksum-of-this-file>",
                "component": "chassis"
```

(continues on next page)

(continued from previous page)

```
            },
            {
                "url": "file:///home/ubuntu/firmware_images/power_pic/pmc_v3.
↪0.bin",
                "checksum": "<sha256-checksum-of-this-file>",
                "component": "power_pic"
            }
        ]
    }
}]
```

The different attributes of `update_firmware` clean step are as follows:

Attribute	Description
`interface`	Interface of clean step, here `management`
`step`	Name of clean step, here `update_firmware`
`args`	Keyword-argument entry (<name>: <value>) being passed to clean step
`args.firmware_update`	Mode (or mechanism) of out-of-band firmware update. Supported value is `ilo`. This is mandatory.
`args.firmware_images`	Ordered list of dictionaries of images to be flashed. This is mandatory.

Each firmware image block is represented by a dictionary (JSON), in the form:

```
{
  "url": "<url of firmware image file>",
  "checksum": "<SHA256, SHA512, or MD5 checksum of firmware image file to␣
↪verify the image>",
  "component": "<device on which firmware image will be flashed>"
}
```

All the fields in the firmware image block are mandatory.

- The different types of firmware url schemes supported are: `file`, `http`, `https` and `swift`.

> **Note**
>
> This feature assumes that while using `file` url scheme the file path is on the conductor controlling the node.

> **Note**
>
> The `swift` url scheme assumes the swift account of the `service` project. The `service` project (tenant) is a special project created in the Keystone system designed for the use of the core OpenStack services. When Ironic makes use of Swift for storage purpose, the account is generally `service` and the container is generally `ironic` and `ilo` driver uses a container named `ironic_ilo_container` for their own purpose.

> **Note**
>
> While using firmware files with a `.rpm` extension, make sure the commands `rpm2cpio` and `cpio` are present on the conductor, as they are utilized to extract the firmware image from the package.

- The firmware components that can be updated are: `ilo`, `cpld`, `power_pic`, `bios` and `chassis`.
- The firmware images will be updated in the order given by the operator. If there is any error during processing of any of the given firmware images provided in the list, none of the firmware updates will occur. The processing error could happen during image download, image checksum verification or image extraction. The logic is to process each of the firmware files and update them on the devices only if all the files are processed successfully. If, during the update (uploading and flashing) process, an update fails, then the remaining updates, if any, in the list will be aborted. But it is recommended to triage and fix the failure and re-attempt the manual clean step `update_firmware` for the aborted `firmware_images`.

  The devices for which the firmwares have been updated successfully would start functioning using their newly updated firmware.
- As a troubleshooting guidance on the complete process, check Ironic conductor logs carefully to see if there are any firmware processing or update related errors which may help in root causing or gain an understanding of where things were left off or where things failed. You can then fix or work around and then try again. A common cause of update failure is HPE Secure Digital Signature check failure for the firmware image file.
- To compute `sha256` checksum for your image file, you can use the following command:

```
$ sha256sum image.rpm
24f6abba6fb6921b05afdb4f9a671aed72af3add90c912b5e3989f51f1b359e5  image.
↪rpm
```

Smart Update Manager (SUM) based firmware update

The firmware update based on SUM is an inband clean/deploy step supported by iLO driver. The firmware update is performed on all or list of user specified firmware components on the node. Refer to SUM User Guide to get more information on SUM based firmware update.

> **Note**
>
> `update_firmware_sum` clean step requires the agent ramdisk with `Proliant Hardware Manager` from the proliantutils version 2.5.0 or higher. See *DIB support for Proliant Hardware Manager* to create the agent ramdisk with `Proliant Hardware Manager`.

> **Note**
>
> `flash_firmware_sum` deploy step requires the agent ramdisk with `Proliant Hardware Manager` from the proliantutils version 2.9.5 or higher. See *DIB support for Proliant Hardware Manager* to create the agent ramdisk with `Proliant Hardware Manager`.

The attributes of `update_firmware_sum/flash_firmware_sum` step are as follows:

Attribute	Description
`interface`	Interface of the clean step, here `management`
`step`	Name of the clean step, here `update_firmware_sum`
`args`	Keyword-argument entry (<name>: <value>) being passed to the clean step

The keyword arguments used for the step are as follows:

- `url`: URL of SPP (Service Pack for Proliant) ISO. It is mandatory. The URL schemes supported are `http`, `https` and `swift`.
- `checksum`: MD5 checksum of SPP ISO to verify the image. It is mandatory.
- `components`: List of filenames of the firmware components to be flashed. It is optional. If not provided, the firmware update is performed on all the firmware components.

The step performs an update on all or a list of firmware components and returns the SUM log files. The log files include `hpsum_log.txt` and `hpsum_detail_log.txt` which holds the information about firmware components, firmware version for each component and their update status. The log object will be named with the following pattern:

```
<node>[_<instance-uuid>]_update_firmware_sum_<timestamp yyyy-mm-dd-hh-mm-ss>.
↪tar.gz
or
<node>[_<instance-uuid>]_flash_firmware_sum_<timestamp yyyy-mm-dd-hh-mm-ss>.
↪tar.gz
```

Refer to *Retrieving logs from the deploy ramdisk* for more information on enabling and viewing the logs returned from the ramdisk.

An example of `update_firmware_sum` clean step:

```
{
    "interface": "management",
    "step": "update_firmware_sum",
    "args":
        {
            "url": "http://my_address:port/SPP.iso",
            "checksum": "abcdefxyz",
            "components": ["CP024356.scexe", "CP008097.exe"]
        }
}
```

The step fails if there is any error in the processing of step arguments. The processing error could happen during validation of components file extension, image download, image checksum verification or image extraction. In case of a failure, check Ironic conductor logs carefully to see if there are any validation or firmware processing related errors which may help in root cause analysis or gaining an understanding of where things were left off or where things failed. You can then fix or work around and then try again.

Warning

This feature is officially supported only with RHEL and SUSE based IPA ramdisk. Refer to SUM for supported OS versions for specific SUM version.

Note

Refer Guidelines for SPP ISO for steps to get SPP (Service Pack for ProLiant) ISO.

Updating security parameters as manual clean step

iLO driver can invoke security parameters update as a manual clean step. Any manual cleaning step can only be initiated when a node is in the `manageable` state. Once the manual cleaning is finished, the node will be put in the `manageable` state again. A user can follow steps from *Manual cleaning* to initiate manual cleaning operation on a node. This feature is only supported for iLO5 based hardware.

An example of a manual clean step with `security_parameters_update` as the only clean step could be:

```
"clean_steps": [{
    "interface": "management",
    "step": "security_parameters_update",
    "args": {
        "security_parameters":[
            {
                "param": "password_complexity",
                "enable": "True",
                "ignore": "False"
            },
            {
                "param": "require_login_for_ilo_rbsu",
                "enable": "True",
                "ignore": "False"
            },
            {
                "param": "ipmi_over_lan",
                "enable": "True",
                "ignore": "False"
            },
            {
                "param": "secure_boot",
                "enable": "True",
                "ignore": "False"
            },
            {
                "param": "require_host_authentication",
                "enable": "True",
                "ignore": "False"
            }
        ]
    }
```

(continues on next page)

(continued from previous page)

```
}]
```

The different attributes of `security_parameters_update` clean step are as follows:

Attribute	Description
`interface`	Interface of clean step, here `management`
`step`	Name of clean step, here `security_parameters_update`
`args`	Keyword-argument entry (<name>: <value>) being passed to clean step
`args.security_parame`	Ordered list of dictionaries of security parameters to be updated. This is mandatory.

Each security parameter block is represented by a dictionary (JSON), in the form:

```
{
  "param": "<security parameter name>",
  "enable": "security parameter to be enabled/disabled",
  "ignore": "security parameter status to be ignored or not"
}
```

In all of these fields, `param` field is mandatory. Remaining fields are boolean and are optional. If user doesnt pass any value then for `enable` field the default will be True and for `ignore` field default will be False.

- The Security Parameters which are supported for this clean step are: `Password_Complexity`, `RequiredLoginForiLORBSU`, `RequireHostAuthentication`, `IPMI/DCMI_Over_LAN` and `Secure_Boot`.

Update Minimum Password Length security parameter as manual clean step

iLO driver can invoke `Minimum Password Length` security parameter update as a manual clean step. This feature is only supported for iLO5 based hardware.

An example of a manual clean step with `update_minimum_password_length` as the only clean step could be:

```
"clean_steps": [{
    "interface": "management",
    "step": "update_minimum_password_length",
    "args": {
        "password_length": "8",
        "ignore": "False"
    }
}]
```

Both the arguments `password_length` and `ignore` are optional. The accepted values for password_length are 0 to 39. If user doesnt pass any value, the default value for password_length will be 8 and for ignore the default value be False.

Update Authentication Failure Logging security parameter as manual clean step

iLO driver can invoke `Authentication Failure Logging` security parameter update as a manual clean step. This feature is only supported for iLO5 based hardware.

An example of a manual clean step with `Authentication Failure Logging` as the only clean step could be:

```
"clean_steps": [{
    "interface": "management",
    "step": "update_auth_failure_logging_threshold",
    "args": {
        "logging_threshold": "1",
        "ignore": "False"
    }
}]
```

Both the arguments `logging_threshold` and `ignore` are optional. The accepted values for logging_threshold are 0 to 5. If user doesnt pass any value, the default value for logging_threshold will be 1 and for ignore the default value be False. If user passes the value of logging_threshold as 0, the Authentication Failure Logging security parameter will be disabled.

Create Certificate Signing Request(CSR) as manual clean step

iLO driver can invoke `create_csr` request as a manual clean step. This step is only supported for iLO5 based hardware.

An example of a manual clean step with `create_csr` as the only clean step could be:

```
"clean_steps": [{
    "interface": "management",
    "step": "create_csr",
    "args": {
        "csr_params": {
            "City": "Bengaluru",
            "CommonName": "1.1.1.1",
            "Country": "India",
            "OrgName": "HPE",
            "State": "Karnataka"
        }
    }
}]
```

The *ilo.cert_path* option in `ironic.conf` is used as the directory path for creating the CSR, which defaults to `/var/lib/ironic/ilo`. The CSR is created in the directory location given in *ilo.cert_path* in `node_uuid` directory as <node_uuid>.csr.

Add HTTPS Certificate as manual clean step

iLO driver can invoke `add_https_certificate` request as a manual clean step. This step is only supported for iLO5 based hardware.

An example of a manual clean step with `add_https_certificate` as the only clean step could be:

```
"clean_steps": [{
    "interface": "management",
    "step": "add_https_certificate",
    "args": {
        "cert_file": "/test1/iLO.crt"
    }
}]
```

Argument `cert_file` is mandatory. The `cert_file` takes the path or url of the certificate file. The url schemes supported are: `file`, `http` and `https`. The CSR generated in step `create_csr` needs to be signed by a valid CA and the resultant HTTPS certificate should be provided in `cert_file`. It copies the `cert_file` to *`ilo.cert_path`* under `node.uuid` as <node_uuid>.crt before adding it to iLO.

RAID Support

The inband RAID functionality is supported by iLO driver. See *RAID Configuration* for more information. Bare Metal service update node with following information after successful configuration of RAID:

- Node `properties/local_gb` is set to the size of root volume.
- Node `properties/root_device` is filled with `wwn` details of root volume. It is used by iLO driver as root device hint during provisioning.
- The value of raid level of root volume is added as `raid_level` capability to the nodes `capabilities` parameter within `properties` field. The operator can specify the `raid_level` capability in nova flavor for node to be selected for scheduling:

```
openstack flavor set ironic-test --property capabilities:raid_level="1+0"
openstack server create --flavor ironic-test --image test-image instance-1
```

DIB support for Proliant Hardware Manager

Install ironic-python-agent-builder

To create an agent ramdisk with `Proliant Hardware Manager`, use the `proliant-tools` element in DIB:

```
ironic-python-agent-builder -o proliant-agent-ramdisk -e proliant-tools fedora
```

Disk Erase Support

`erase_devices` is an inband clean step supported by iLO driver. It performs erase on all the disks including the disks visible to OS as well as the raw disks visible to the Smart Storage Administrator (SSA).

This inband clean step requires `ssacli` utility starting from version `2.60-19.0` to perform the erase on physical disks. See the ssacli documentation for more information on ssacli utility and different erase methods supported by SSA.

The disk erasure via `shred` is used to erase disks visible to the OS and its implementation is available in Ironic Python Agent. The raw disks connected to the Smart Storage Controller are erased using Sanitize erase which is a ssacli supported erase method. If Sanitize erase is not supported on the Smart Storage Controller the disks are erased using One-pass erase (overwrite with zeros).

This clean step is supported when the agent ramdisk contains the `Proliant Hardware Manager` from the proliantutils version 2.3.0 or higher. This clean step is performed as part of automated cleaning and it is disabled by default. See *In-band vs out-of-band* for more information on enabling/disabling a clean step.

Install ironic-python-agent-builder.

To create an agent ramdisk with `Proliant Hardware Manager`, use the `proliant-tools` element in DIB:

```
ironic-python-agent-builder -o proliant-agent-ramdisk -e proliant-tools fedora
```

See the proliant-tools for more information on creating agent ramdisk with `proliant-tools` element in DIB.

Firmware based UEFI iSCSI boot from volume support

With Gen9 (UEFI firmware version 1.40 or higher) and Gen10 HPE Proliant servers, the driver supports firmware based UEFI boot of an iSCSI cinder volume.

This feature requires the node to be configured to boot in `UEFI` boot mode, as well as user image should be `UEFI` bootable image, and `PortFast` needs to be enabled in switch configuration for immediate spanning tree forwarding state so it wouldnt take much time setting the iSCSI target as persistent device.

The driver does not support this functionality when in `bios` boot mode. In case the node is configured with `ilo-pxe` or `ilo-ipxe` as boot interface and the boot mode configured on the bare metal is `bios`, the iscsi boot from volume is performed using iPXE. See *Boot From Volume* for more details.

To use this feature, configure the boot mode of the bare metal to `uefi` and configure the corresponding ironic node using the steps given in *Boot From Volume*. In a cloud environment with nodes configured to boot from `bios` and `uefi` boot modes, the virtual media driver only supports uefi boot mode, and that attempting to use iscsi boot at the same time with a bios volume will result in an error.

BIOS configuration support

The `ilo` and `ilo5` hardware types support `ilo` BIOS interface. The support includes providing manual clean steps *apply_configuration* and *factory_reset* to manage supported BIOS settings on the node. See *BIOS Configuration* for more details and examples.

Note

Prior to the Stein release the user is required to reboot the node manually in order for the settings to take into effect. Starting with the Stein release, iLO drivers reboot the node after running clean steps related to the BIOS configuration. The BIOS settings are cached and the clean step is marked as success only if all the requested settings are applied without any failure. If application of any of the settings fails, the clean step is marked as failed and the settings are not cached.

Configuration

Following are the supported BIOS settings and the corresponding brief description for each of the settings. For a detailed description please refer to HPE Integrated Lights-Out REST API Documentation.

- `AdvancedMemProtection`: Configure additional memory protection with ECC (Error Checking and Correcting). Allowed values are `AdvancedEcc`, `OnlineSpareAdvancedEcc`,

MirroredAdvancedEcc.

- AutoPowerOn: Configure the server to automatically power on when AC power is applied to the system. Allowed values are AlwaysPowerOn, AlwaysPowerOff, RestoreLastState.
- BootMode: Select the boot mode of the system. Allowed values are Uefi, LegacyBios
- BootOrderPolicy: Configure how the system attempts to boot devices per the Boot Order when no bootable device is found. Allowed values are RetryIndefinitely, AttemptOnce, ResetAfterFailed.
- CollabPowerControl: Enables the Operating System to request processor frequency changes even if the Power Regulator option on the server configured for Dynamic Power Savings Mode. Allowed values are Enabled, Disabled.
- DynamicPowerCapping: Configure when the System ROM executes power calibration during the boot process. Allowed values are Enabled, Disabled, Auto.
- DynamicPowerResponse: Enable the System BIOS to control processor performance and power states depending on the processor workload. Allowed values are Fast, Slow.
- IntelligentProvisioning: Enable or disable the Intelligent Provisioning functionality. Allowed values are Enabled, Disabled.
- IntelPerfMonitoring: Exposes certain chipset devices that can be used with the Intel Performance Monitoring Toolkit. Allowed values are Enabled, Disabled.
- IntelProcVtd: Hypervisor or operating system supporting this option can use hardware capabilities provided by Intels Virtualization Technology for Directed I/O. Allowed values are Enabled, Disabled.
- IntelQpiFreq: Set the QPI Link frequency to a lower speed. Allowed values are Auto, MinQpiSpeed.
- IntelTxt: Option to modify Intel TXT support. Allowed values are Enabled, Disabled.
- PowerProfile: Set the power profile to be used. Allowed values are BalancedPowerPerf, MinPower, MaxPerf, Custom.
- PowerRegulator: Determines how to regulate the power consumption. Allowed values are DynamicPowerSavings, StaticLowPower, StaticHighPerf, OsControl.
- ProcAes: Enable or disable the Advanced Encryption Standard Instruction Set (AES-NI) in the processor. Allowed values are Enabled, Disabled.
- ProcCoreDisable: Disable processor cores using Intels Core Multi-Processing (CMP) Technology. Allowed values are Integers ranging from 0 to 24.
- ProcHyperthreading: Enable or disable Intel Hyperthreading. Allowed values are Enabled, Disabled.
- ProcNoExecute: Protect your system against malicious code and viruses. Allowed values are Enabled, Disabled.
- ProcTurbo: Enables the processor to transition to a higher frequency than the processors rated speed using Turbo Boost Technology if the processor has available power and is within temperature specifications. Allowed values are Enabled, Disabled.
- ProcVirtualization: Enables or Disables a hypervisor or operating system supporting this option to use hardware capabilities provided by Intels Virtualization Technology. Allowed values are Enabled, Disabled.

- `SecureBootStatus`: The current state of Secure Boot configuration. Allowed values are `Enabled`, `Disabled`.

  Note

  This setting is read-only and cant be modified with `apply_configuration` clean step.

- `Sriov`: If enabled, SR-IOV support enables a hypervisor to create virtual instances of a PCI-express device, potentially increasing performance. If enabled, the BIOS allocates additional resources to PCI-express devices. Allowed values are `Enabled`, `Disabled`.
- `ThermalConfig`: select the fan cooling solution for the system. Allowed values are `OptimalCooling`, `IncreasedCooling`, `MaxCooling`
- `ThermalShutdown`: Control the reaction of the system to caution level thermal events. Allowed values are `Enabled`, `Disabled`.
- `TpmState`: Current TPM device state. Allowed values are `NotPresent`, `PresentDisabled`, `PresentEnabled`.

  Note

  This setting is read-only and cant be modified with `apply_configuration` clean step.

- `TpmType`: Current TPM device type. Allowed values are `NoTpm`, `Tpm12`, `Tpm20`, `Tm10`.

  Note

  This setting is read-only and cant be modified with `apply_configuration` clean step.

- `UefiOptimizedBoot`: Enables or Disables the System BIOS boot using native UEFI graphics drivers. Allowed values are `Enabled`, `Disabled`.
- `WorkloadProfile`: Change the Workload Profile to accommodate your desired workload. Allowed values are `GeneralPowerEfficientCompute`, `GeneralPeakFrequencyCompute`, `GeneralThroughputCompute`, `Virtualization-PowerEfficient`, `Virtualization-MaxPerformance`, `LowLatency`, `MissionCritical`, `TransactionalApplicationProcessing`, `HighPerformanceCompute`, `DecisionSupport`, `GraphicProcessing`, `I/OThroughput`, `Custom`

  Note

  This setting is only applicable to ProLiant Gen10 servers with iLO 5 management systems.

Certificate based validation in iLO

The driver supports validation of certificates on the HPE Proliant servers. The path to certificate file needs to be appropriately set in `ca_file` in the nodes `driver_info`. To update SSL certificates into iLO, refer to HPE Integrated Lights-Out Security Technology Brief. Use iLO hostname or IP address as a Common Name (CN) while generating Certificate Signing Request (CSR). Use the same value as `ilo_address`

while enrolling node to Bare Metal service to avoid SSL certificate validation errors related to hostname mismatch.

Rescue mode support

The hardware type `ilo` supports rescue functionality. Rescue operation can be used to boot nodes into a rescue ramdisk so that the `rescue` user can access the node.

Please refer to *Rescue Mode* for detailed explanation of rescue feature.

Inject NMI support

The management interface `ilo` supports injection of non-maskable interrupt (NMI) to a bare metal. Following command can be used to inject NMI on a server:

```
baremetal node inject nmi <node>
```

Following command can be used to inject NMI via Compute service:

```
openstack server dump create <server>
```

Note

This feature is supported on HPE ProLiant Gen9 servers and beyond.

Soft power operation support

The power interface `ilo` supports soft power off and soft reboot operations on a bare metal. Following commands can be used to perform soft power operations on a server:

```
baremetal node reboot --soft \
    [--power-timeout <power-timeout>] <node>

baremetal node power off --soft \
    [--power-timeout <power-timeout>] <node>
```

Note

The configuration *`conductor.soft_power_off_timeout`* is used as a default timeout value when no timeout is provided while invoking hard or soft power operations.

Note

Server POST state is used to track the power status of HPE ProLiant Gen9 servers and beyond.

Out of Band RAID Support

With Gen10 HPE Proliant servers and later the `ilo5` hardware type supports firmware based RAID configuration as a clean step. This feature requires the node to be configured to `ilo5` hardware type and its raid interface to be `ilo5`. See *RAID Configuration* for more information.

After a successful RAID configuration, the Bare Metal service will update the node with the following information:

- Node `properties/local_gb` is set to the size of root volume.
- Node `properties/root_device` is filled with `wwn` details of root volume. It is used by iLO driver as root device hint during provisioning.

Later the value of raid level of root volume can be added in `baremetal-with-RAID10` (RAID10 for raid level 10) resource class. And consequently flavor needs to be updated to request the resource class to create the server using selected node:

```
baremetal node set test_node --resource-class \
baremetal-with-RAID10

openstack flavor set --property \
resources:CUSTOM_BAREMETAL_WITH_RAID10=1 test-flavor

openstack server create --flavor test-flavor --image test-image instance-1
```

Note

Supported raid levels for `ilo5` hardware type are: 0, 1, 5, 6, 10, 50, 60

IPv6 support

With the IPv6 support in `proliantutils>=2.8.0`, nodes can be enrolled into the baremetal service using the iLO IPv6 addresses.

```
baremetal node create --driver ilo  --deploy-interface direct \
    --driver-info ilo_address=2001:0db8:85a3:0000:0000:8a2e:0370:7334 \
    --driver-info ilo_username=test-user \
    --driver-info ilo_password=test-password \
    --driver-info deploy_iso=test-iso \
    --driver-info rescue_iso=test-iso
```

Note

No configuration changes (in e.g. ironic.conf) are required in order to support IPv6.

Out of Band Sanitize Disk Erase Support

With Gen10 HPE Proliant servers and later the `ilo5` hardware type supports firmware based sanitize disk erase as a clean step. This feature requires the node to be configured to `ilo5` hardware type and its management interface to be `ilo5`.

The possible erase pattern its supports are:

- For HDD - overwrite, zero, crypto
- For SSD - block, zero, crypto

The default erase pattern are, for HDD, overwrite and for SSD, block.

Note

In average 300GB HDD with default pattern overwrite would take approx. 9 hours and 300GB SSD with default pattern block would take approx. 30 seconds to complete the erase.

Out of Band One Button Secure Erase Support

With Gen10 HPE Proliant servers which have been updated with SPP version 2019.03.0 or later the `ilo5` hardware type supports firmware based one button secure erase as a clean step.

The One Button Secure Erase resets iLO and deletes all licenses stored there, resets BIOS settings, and deletes all Active Health System (AHS) and warranty data stored on the system. It also erases supported non-volatile storage data and deletes any deployment settings profiles. See HPE Gen10 Security Reference Guide for more information.

Below are the steps to perform this clean step:

- Perform the cleaning using one_button_secure_erase clean step

```
baremetal node clean $node_ident --clean-steps\
    '[{"interface": "management", "step": "one_button_secure_erase"}]'
```

- Once the clean step would triggered and node go to clean wait state and maintenance flag on node would be set to True, then delete the node

```
baremetal node delete $node_ident
```

Note

- Even after deleting the node, One Button Secure Erase operation would continue on the node.
- This clean step should be kept last if the multiple clean steps are to be executed. No clean step after this step would be executed.
- One Button Secure Erase should be used with extreme caution, and only when a system is being decommissioned. During the erase the iLO network would keep disconnecting and after the erase user will completely lose iLO access along with the credentials of the server, which needs to be regained by the administrator. The process can take up to a day or two to fully erase and reset all user data.
- When you activate One Button Secure Erase, iLO 5 does not allow firmware update or reset operations.

Note

Do not perform any iLO 5 configuration changes until this process is completed.

UEFI-HTTPS Boot support

The UEFI firmware on Gen10 HPE Proliant servers supports booting from secured URLs. With this capability `ilo5` hardware with `ilo-uefi-https` boot interface supports deploy/rescue features in more secured environments.

If swift is used as glance backend and ironic is configured to use swift to store temporary images, it is required that swift is configured on HTTPS so that the tempurl generated is HTTPS URL.

If the webserver is used for hosting the temporary images, then the webserver is required to serve requests on HTTPS.

If the images are hosted on a HTTPS webserver or swift configured with HTTPS with custom certificates, the user is required to export SSL certificates into iLO. Refer to HPE Integrated Lights-Out Security Technology Brief for more information.

The following command can be used to enroll a ProLiant node with `ilo5` hardware type and `ilo-uefi-https` boot interface:

```
baremetal node create \
    --driver ilo5 \
    --boot-interface ilo-uefi-https \
    --deploy-interface direct \
    --raid-interface ilo5 \
    --rescue-interface agent \
    --driver-info ilo_address=<ilo-ip-address> \
    --driver-info ilo_username=<ilo-username> \
    --driver-info ilo_password=<ilo-password> \
    --driver-info deploy_kernel=<glance-uuid-of-deploy-kernel> \
    --driver-info deploy_ramdisk=<glance-uuid-of-rescue-ramdisk> \
    --driver-info bootloader=<glance-uuid-of-bootloader>
```

Layer 3 or DHCP-less ramdisk booting

DHCP-less deploy is supported by `ilo` and `ilo5` hardware types. However it would work only with ilo-virtual-media boot interface. See *Layer 3 or DHCP-less ramdisk booting* for more information.

Events subscription

Events subscription is supported by `ilo` and `ilo5` hardware types with `ilo` vendor interface for Gen10 and Gen10 Plus servers. See *Node Vendor Passthru Methods* for more information.

Anaconda based deployment

Deployment with `anaconda` deploy interface is supported by `ilo` and `ilo5` hardware type and works with `ilo-pxe` and `ilo-ipxe` boot interfaces. See *Deploying with anaconda deploy interface* for more information.

Intel IPMI driver

Overview

The `intel-ipmi` hardware type is same as the *IPMI driver* hardware type except for the support of Intel Speed Select Performance Profile (Intel SST-PP) feature. Intel SST-PP allows a server to run different workloads by configuring the CPU to run at 3 distinct operating points or profiles.

Intel SST-PP supports three configuration levels:

- 0 - Intel SST-PP Base Config
- 1 - Intel SST-PP Config 1
- 2 - Intel SST-PP Config 2

The following table shows the list of active cores and their base frequency at different SST-PP config levels:

Config	Cores	Base Freq (GHz)
Base	24	2.4
Config 1	20	2.5
Config 2	16	2.7

This configuration is managed by the management interface `intel-ipmitool` for IntelIPMI hardware.

IntelIPMI manages nodes by using IPMI (Intelligent Platform Management Interface) protocol versions 2.0 or 1.5. It uses the IPMItool utility which is an open-source command-line interface (CLI) for controlling IPMI-enabled devices.

Glossary

- IPMI - Intelligent Platform Management Interface.
- Intel SST-PP - Intel Speed Select Performance Profile.

Enabling the IntelIPMI hardware type

Please see *Configuring IPMI support* for the required dependencies.

1. To enable `intel-ipmi` hardware, add the following configuration to your `ironic.conf`:

```
[DEFAULT]
enabled_hardware_types = intel-ipmi
enabled_management_interfaces = intel-ipmitool
```

2. Restart the Ironic conductor service:

```
sudo service ironic-conductor restart

# Or, for RDO:
sudo systemctl restart openstack-ironic-conductor
```

Registering a node with the IntelIPMI driver

Nodes configured to use the IntelIPMI drivers should have the `driver` field set to `intel-ipmi`.

All the configuration value required for IntelIPMI is the same as the IPMI hardware type except the management interface which is `intel-ipmitool`. Refer *IPMI driver* for details.

The `baremetal node create` command can be used to enroll a node with an IntelIPMI driver. For example:

```
baremetal node create --driver intel-ipmi \
    --driver-info ipmi_address=<address> \
    --driver-info ipmi_username=<username> \
    --driver-info ipmi_password=<password>
```

Features of the `intel-ipmi` hardware type

Intel SST-PP

A node with Intel SST-PP can be configured to use it via `configure_intel_speedselect` deploy step. This deploy accepts:

- `intel_speedselect_config`: Hexadecimal code of Intel SST-PP configuration. Accepted values are 0x00, 0x01, 0x02. These values correspond to `Intel SST-PP Config Base`, `Intel SST-PP Config 1`, `Intel SST-PP Config 2` respectively. The input value must be a string.
- `socket_count`: Number of sockets in the node. The input value must be a positive integer (1 by default).

The deploy step issues an IPMI command with the raw code for each socket in the node to set the requested configuration. A reboot is required to reflect the changes.

Each configuration profile is mapped to traits that Ironic understands. Please note that these names are used for example purpose only. Any name can be used. Only the parameter value should match the deploy step `configure_intel_speedselect`.

- 0 - `CUSTOM_INTEL_SPEED_SELECT_CONFIG_BASE`
- 1 - `CUSTOM_INTEL_SPEED_SELECT_CONFIG_1`
- 2 - `CUSTOM_INTEL_SPEED_SELECT_CONFIG_2`

Now to configure a node with Intel SST-PP while provisioning, create deploy templates for each profiles in Ironic.

```
baremetal deploy template create \
   CUSTOM_INTEL_SPEED_SELECT_CONFIG_BASE \
   --steps '[{"interface": "management", "step": "configure_intel_speedselect
↪", "args": {"intel_speedselect_config": "0x00", "socket_count": 2},
↪"priority": 150}]'

baremetal deploy template create \
    CUSTOM_INTEL_SPEED_SELECT_CONFIG_1 \
    --steps '[{"interface": "management", "step": "configure_intel_speedselect
↪", "args": {"intel_speedselect_config": "0x01", "socket_count": 2},
↪"priority": 150}]'
```

(continues on next page)

(continued from previous page)

```
baremetal deploy template create \
   CUSTOM_INTEL_SPEED_SELECT_CONFIG_2 \
   --steps '[{"interface": "management", "step": "configure_intel_speedselect
↪", "args": {"intel_speedselect_config": "0x02", "socket_count": 2},
↪"priority": 150}]'
```

All Intel SST-PP capable nodes should have these traits associated.

```
baremetal node add trait node-0 \
   CUSTOM_INTEL_SPEED_SELECT_CONFIG_BASE \
   CUSTOM_INTEL_SPEED_SELECT_CONFIG_1 \
   CUSTOM_INTEL_SPEED_SELECT_CONFIG_2
```

To trigger the Intel SST-PP configuration during node provisioning, one of the traits can be added to the flavor.

```
openstack flavor set baremetal --property trait:CUSTOM_INTEL_SPEED_SELECT_
↪CONFIG_1=required
```

Finally create a server with `baremetal` flavor to provision a baremetal node with Intel SST-PP profile *Config 1*.

IPMI driver

Overview

The `ipmi` hardware type manage nodes by using IPMI (Intelligent Platform Management Interface) protocol versions 2.0 or 1.5. It uses the IPMItool utility which is an open-source command-line interface (CLI) for controlling IPMI-enabled devices.

Glossary

- IPMI - Intelligent Platform Management Interface.
- IPMB - Intelligent Platform Management Bus/Bridge.
- BMC - Baseboard Management Controller.
- RMCP - Remote Management Control Protocol.

Enabling the IPMI hardware type

Please see *Configuring IPMI support* for the required dependencies.

1. The `ipmi` hardware type is enabled by default starting with the Ocata release. To enable it explicitly, add the following to your `ironic.conf`:

   ```
   [DEFAULT]
   enabled_hardware_types = ipmi
   enabled_management_interfaces = ipmitool,noop
   enabled_power_interfaces = ipmitool
   ```

   Optionally, enable the *vendor passthru interface* and either or both *console interfaces*:

```
[DEFAULT]
enabled_hardware_types = ipmi
enabled_console_interfaces = ipmitool-socat,no-console
enabled_management_interfaces = ipmitool,noop
enabled_power_interfaces = ipmitool
enabled_vendor_interfaces = ipmitool,no-vendor
```

2. Restart the Ironic conductor service.

Please see *Enabling drivers and hardware types* for more details.

Registering a node with the IPMI driver

Nodes configured to use the IPMItool drivers should have the `driver` field set to `ipmi`.

The following configuration value is required and has to be added to the nodes `driver_info` field:

- `ipmi_address`: The IP address or hostname of the BMC.

Other options may be needed to match the configuration of the BMC, the following options are optional, but in most cases, its considered a good practice to have them set:

- `ipmi_username`: The username to access the BMC; defaults to *NULL* user.
- `ipmi_password`: The password to access the BMC; defaults to *NULL*.
- `ipmi_port`: The remote IPMI RMCP port. By default ipmitool will use the port *623*.

Note

It is highly recommend that you setup a username and password for your BMC.

The `baremetal node create` command can be used to enroll a node with an IPMItool-based driver. For example:

```
baremetal node create --driver ipmi \
    --driver-info ipmi_address=<address> \
    --driver-info ipmi_username=<username> \
    --driver-info ipmi_password=<password>
```

Changing The Default IPMI Credential Persistence Method

- `store_cred_in_env`: *`ipmi.store_cred_in_env`*.

The `store_cred_in_env` configuration option allow users to switch between file-based and environment variable persistence methods for IPMI password.

For the temporary file option, long lived IPMI sessions, such as those for console support, leave files with credentials on the conductor disk for the duration of the session.

To switch to environment variable persistence, set the `store_cred_in_env` parameter to `True` in the configuration file:

```
[ipmi]
store_cred_in_env = True
```

Advanced configuration

When a simple configuration such as providing the `address`, `username` and `password` is not enough, the IPMItool driver contains many other options that can be used to address special usages.

Single/Double bridging functionality

Note

A version of IPMItool higher or equal to 1.8.12 is required to use the bridging functionality.

There are two different bridging functionalities supported by the IPMItool-based drivers: *single* bridge and *dual* bridge.

The following configuration values need to be added to the nodes `driver_info` field so bridging can be used:

- `ipmi_bridging`: The bridging type; default is *no*; other supported values are *single* for single bridge or *dual* for double bridge.
- **`ipmi_local_address`: The local IPMB address for bridged requests.**
  Required only if `ipmi_bridging` is set to *single* or *dual*. This configuration is optional, if not specified it will be auto discovered by IPMItool.
- `ipmi_target_address`: The destination address for bridged requests. Required only if `ipmi_bridging` is set to *single* or *dual*.
- `ipmi_target_channel`: The destination channel for bridged requests. Required only if `ipmi_bridging` is set to *single* or *dual*.

Double bridge specific options:

- `ipmi_transit_address`: The transit address for bridged requests. Required only if `ipmi_bridging` is set to *dual*.
- `ipmi_transit_channel`: The transit channel for bridged requests. Required only if `ipmi_bridging` is set to *dual*.

The parameter `ipmi_bridging` should specify the type of bridging required: *single* or *dual* to access the bare metal node. If the parameter is not specified, the default value will be set to *no*.

The `baremetal node set` command can be used to set the required bridging information to the Ironic node enrolled with the IPMItool driver. For example:

- Single Bridging:

```
baremetal node set <UUID or name> \
    --driver-info ipmi_local_address=<address> \
    --driver-info ipmi_bridging=single \
    --driver-info ipmi_target_channel=<channel> \
    --driver-info ipmi_target_address=<target address>
```

- Double Bridging:

```
baremetal node set <UUID or name> \
    --driver-info ipmi_local_address=<address> \
    --driver-info ipmi_bridging=dual \
    --driver-info ipmi_transit_channel=<transit channel> \
    --driver-info ipmi_transit_address=<transit address> \
    --driver-info ipmi_target_channel=<target channel> \
    --driver-info ipmi_target_address=<target address>
```

Changing the version of the IPMI protocol

The IPMItool-based drivers works with the versions *2.0* and *1.5* of the IPMI protocol. By default, the version *2.0* is used.

In order to change the IPMI protocol version in the bare metal node, the following option needs to be set to the nodes `driver_info` field:

- `ipmi_protocol_version`: The version of the IPMI protocol; default is *2.0*. Supported values are *1.5* or *2.0*.

The `baremetal node set` command can be used to set the desired protocol version:

```
baremetal node set <UUID or name> --driver-info ipmi_protocol_version=
↪<version>
```

Warning

Version *1.5* of the IPMI protocol does not support encryption. Therefore, it is highly recommended that version 2.0 is used.

Cipher suites

IPMI 2.0 introduces support for encryption and allows setting which cipher suite to use. Traditionally, `ipmitool` was using cipher suite 3 by default, but since SHA1 no longer complies with modern security requirement, recent versions (e.g. the one used in RHEL 8.2) are switching to suite 17.

Normally, the cipher suite to use is negotiated with the BMC using the special command. On some hardware the negotiation yields incorrect results and IPMI commands fail with

```
Error in open session response message : no matching cipher suite
Error: Unable to establish IPMI v2 / RMCP+ session
```

Another possible problem is `ipmitool` commands taking very long (tens of seconds or even minutes) because the BMC does not support cipher suite negotiation. In both cases you can specify the required suite yourself, e.g.

```
baremetal node set <UUID or name> --driver-info ipmi_cipher_suite=3
```

In scenarios where the operator cant specify the `ipmi_cipher_suite` for each node, the configuration parameter `[ipmi]/cipher_suite_versions` can be set to a list of cipher suites that will be used, Ironic will attempt to find a value that can be used from the list provided (from last to first):

```
[ipmi]
cipher_suite_versions = 1,2,3,6,7,8,11,12
```

To find the suitable values for this configuration, you can check the field `RMCP+ Cipher Suites` after running an `ipmitool` command, e.g:

```
$ ipmitool -I lanplus -H $HOST -U $USER -v -R 12 -N 5  lan print
# output
Set in Progress         : Set Complete
Auth Type Support       : NONE MD2 MD5 PASSWORD OEM
Auth Type Enable        : Callback : NONE MD2 MD5 PASSWORD OEM
IP Address Source       : Static Address
IP Address              : <IP>
Subnet Mask             : <Subnet>
MAC Address             : <MAC>
RMCP+ Cipher Suites     : 0,1,2,3,6,7,8,11,12
```

Warning

Only the cipher suites 3 and 17 are considered secure by the modern standards. Cipher suite 0 means no security at all.

Using a different privilege level

By default Ironic requests the `ADMINISTRATOR` privilege level of all commands. This is the easiest option, but if its not available for you, you can change it to `CALLBACK`, `OPERATOR` or `USER` this way:

```
baremetal node set <UUID or name> --driver-info ipmi_priv_level=OPERATOR
```

You must ensure that the user can still change power state and boot devices.

Static boot order configuration

See *Static boot order configuration*.

Vendor Differences

While the Intelligent Platform Management Interface (IPMI) interface is based upon a defined standard, the Ironic community is aware of at least one vendor which utilizes a non-standard boot device selector. In essence, this could be something as simple as different interpretation of the standard.

As of October 2020, the known difference is with Supermicro hardware where a selector of `0x24`, signifying a *REMOTE* boot device in the standard, must be used when a boot operation from the local disk subsystem is requested **in UEFI mode**. This is contrary to BIOS mode where the same BMCs expect the selector to be a value of `0x08`.

Because the BMC does not respond with any sort of error, nor do we want to risk BMC connectivity issues by explicitly querying all BMCs what vendor it may be before every operation, the vendor can automatically be recorded in the `properties` field `vendor`. When this is set to a value of `supermicro`, Ironic will navigate the UEFI behavior difference enabling the UEFI to be requested with boot to disk.

Example:

```
baremetal node set <UUID or name> \
    --properties vendor="supermicro"
```

Luckily, Ironic will attempt to perform this detection in power synchronization process, and record this value if not already set.

While similar issues may exist when setting the boot mode and target boot device in other vendors BMCs, we are not aware of them at present. Should you encounter such an issue, please feel free to report this via Launchpad, and be sure to include the `chassis bootparam get 5` output value along with the `mc info` output from your BMC.

Example:

```
ipmitool -I lanplus -H <BMC ADDRESS> -U <Username> -P <Password> \
    mc info
ipmitool -I lanplus -H <BMC ADDRESS> -U <Username> -P <Password> \
    chassis bootparam get 5
```

send_raw clean/deploy step

The `send_raw` vendor passthru method is available to be invoked as a clean or deployment step should raw bytes need to be transmitted to the remote BMC in order to facilitate some sort of action or specific state. In this case, the raw bytes to be set are conveyed with a `raw_bytes` argument on the requested clean or deploy step.

Example:

```
{
  "interface": "vendor",
  "step": "send_raw",
  "args": {
    "raw_bytes": "0x00 0x00 0x00 0x00"
  }
}
```

Redfish driver

Overview

The `redfish` driver enables managing servers compliant with the Redfish standard. While Redfish strives to provide a standard model of interaction for baseboard management controllers, vendors often do things slightly different or interpret the standard differently. Ironic attempts to support a number of features while also navigating these issues, however if you do encounter any issues, please do not hesitate to open a bug for the Ironic maintainers.

Supported features include:

- Network, *virtual media* and *HTTP(s)* boot.
- Additional virtual media features:
  - *Ramdisk deploy interface*.
  - *Layer 3 or DHCP-less ramdisk booting*.

  - Virtual media API.

- *Changing boot mode and secure boot status*.
- *In-band* and *out-of-band inspection*.
- Retrieving and changing *BIOS settings*.
- Applying *firmware updates*.
- Configuring *hardware RAID*.
- *Hardware metrics* and integration with ironic-prometheus-exporter.
- Event notifications configured via *Node Vendor Passthru Methods*.

Prerequisites

- The Sushy library should be installed on the ironic conductor node(s).

  For example, it can be installed with `pip`:

```
sudo pip install sushy
```

Enabling the Redfish driver

1. Add `redfish` to the list of `enabled_hardware_types`, `enabled_power_interfaces`, `enabled_management_interfaces` and `enabled_inspect_interfaces` as well as `redfish-virtual-media` and `redfish-https` to `enabled_boot_interfaces` in `/etc/ironic/ironic.conf`. For example:

```
[DEFAULT]
...
enabled_hardware_types = ipmi,redfish
enabled_boot_interfaces = ipxe,redfish-virtual-media,redfish-https
enabled_power_interfaces = ipmitool,redfish
enabled_management_interfaces = ipmitool,redfish
enabled_inspect_interfaces = agent,redfish
```

2. Restart the ironic conductor service:

```
sudo service ironic-conductor restart

# Or, for RDO:
sudo systemctl restart openstack-ironic-conductor
```

Registering a node with the Redfish driver

Nodes configured to use the driver should have the `driver` property set to `redfish`.

The following properties are specified in the nodes `driver_info` field:

`redfish_address`
: The URL address to the Redfish controller. It must include the authority portion of the URL, and can optionally include the scheme. If the scheme is missing, https is assumed. For example: `https://mgmt.vendor.com`. This is required.

redfish_system_id
: The canonical path to the ComputerSystem resource that the driver will interact with. It should include the root service, version and the unique resource path to the ComputerSystem. This property is only required if target BMC manages more than one ComputerSystem. Otherwise ironic will pick the only available ComputerSystem automatically. For example: `/redfish/v1/Systems/1`.

redfish_username
: User account with admin/server-profile access privilege. Although not required, it is highly recommended.

redfish_password
: User account password. Although not required, it is highly recommended.

redfish_verify_ca
: If `redfish_address` has the `https://` scheme, the driver will use a secure (TLS) connection when talking to the Redfish controller. By default (if this is not set or set to `True`), the driver will try to verify the host certificates. This can be set to the path of a certificate file or directory with trusted certificates that the driver will use for verification. To disable verifying TLS, set this to `False`. This is optional.

redfish_auth_type
: Redfish HTTP client authentication method. Can be `basic`, `session` or `auto`. The `auto` mode first tries `session` and falls back to `basic` if session authentication is not supported by the Redfish BMC. Default is set in ironic config as *`redfish.auth_type`*. Most operators should not need to leverage this setting. Session based authentication should generally be used in most cases as it prevents re-authentication every time a background task checks in with the BMC.

vmedia_transport_protocol
: Override transport protocol selection for virtual media. Valid values are `HTTP`, `NFS` (NFSv3), or `CIFS`. If not set, HTTP is used by default. This setting must be supported by the BMC.

> **Note**
>
> NFS uses NFSv3 and does not send credentials to the BMC. CIFS/SMB supports optional username/password authentication configured via `[cifs]username` and `[cifs]password`. Both NFS and CIFS require operators to mount the respective share at `[nfs]share_path` or `[cifs]share_path` on all conductor hosts.

> **Note**
>
> The `redfish_address`, `redfish_username`, `redfish_password`, and `redfish_verify_ca` fields, if changed, will trigger a new session to be established and cached with the BMC. The `redfish_auth_type` field will only be used for the creation of a new cached session, or should one be rejected by the BMC.

The `baremetal node create` command can be used to enroll a node with the `redfish` driver. For example:

```
baremetal node create --driver redfish --driver-info \
  redfish_address=https://example.com --driver-info \
  redfish_system_id=/redfish/v1/Systems/CX34R87 --driver-info \
```

(continues on next page)

(continued from previous page)

```
redfish_username=admin --driver-info redfish_password=password \
--name node-0
```

For more information about enrolling nodes see *Enrolling hardware with Ironic* in the install guide.

Boot mode support

The `redfish` hardware type can read current boot mode from the bare metal node as well as set it to either Legacy BIOS or UEFI.

Note

Boot mode management is the optional part of the Redfish specification. Not all Redfish-compliant BMCs might implement it. In that case it remains the responsibility of the operator to configure proper boot mode to their bare metal nodes.

UEFI secure boot

Secure boot mode can be automatically set and unset during deployment for nodes in UEFI boot mode, see *UEFI secure boot mode* for an explanation how to use it.

Two clean and deploy steps are provided for key management:

`management.reset_secure_boot_keys_to_default`
: resets secure boot keys to their manufacturing defaults.

`management.clear_secure_boot_keys`
: removes all secure boot keys from the node.

Rebooting on boot mode changes

While some hardware is able to change the boot mode or the *UEFI secure boot* state immediately, other models may require a reboot for such a change to be applied. Furthermore, some hardware models cannot change the boot mode and the secure boot state simultaneously, requiring several reboots.

The Bare Metal service refreshes the System resource after issuing a PATCH request against it. If the expected change is not observed, the node is rebooted, and the Bare Metal service waits until the change is applied. In the end, the previous power state is restored.

This logic makes changing boot configuration more robust at the expense of several reboots in the worst case.

Virtual media boot

The idea behind virtual media boot is that BMC gets hold of the boot image one way or the other (e.g. by HTTP GET, other methods are defined in the standard), then inserts it into nodes virtual drive as if it was burnt on a physical CD/DVD. The node can then boot from that virtual drive into the operating system residing on the image.

The major advantage of virtual media boot feature is that potentially unreliable TFTP image transfer phase of PXE protocol suite is fully eliminated.

Hardware types based on the `redfish` fully support booting deploy/rescue and user images over virtual media. Ironic builds bootable ISO images, for either UEFI or BIOS (Legacy) boot modes, at the moment of node deployment out of kernel and ramdisk images associated with the ironic node.

To boot a node managed by `redfish` hardware type over virtual media using BIOS boot mode, it suffice to set ironic boot interface to `redfish-virtual-media`, as opposed to `ipmitool`.

```
baremetal node set --boot-interface redfish-virtual-media node-0
```

Note

iDRAC firmware before 4.40.10.00 (on Intel systems) and 6.00.00.00 (on AMD systems) requires a non-standard Redfish call to boot from virtual media. Consider upgrading to 6.00.00.00, otherwise you **must** use the `idrac` hardware type and the `idrac-redfish-virtual-media` boot interface with older iDRAC firmware instead. For simplicity Ironic restricts both AMD and Intel systems before firmware version 6.00.00.00. See *iDRAC driver* for more details on this hardware type.

If UEFI boot mode is desired, the user should additionally supply EFI System Partition image (ESP), see *Configuring an ESP image* for details.

If `[driver_info]/config_via_floppy` boolean property of the node is set to `true`, ironic will create a file with runtime configuration parameters, place into on a FAT image, then insert the image into nodes virtual floppy drive.

When booting over PXE or virtual media, and user instance requires some specific kernel configuration, the nodes `instance_info[kernel_append_params]` or `driver_info[kernel_append_params]` properties can be used to pass user-specified kernel command line parameters.

```
baremetal node set node-0 \
  --driver-info kernel_append_params="nofb vga=normal"
```

Note

The `driver_info` field is supported starting with the Xena release.

Starting with the Zed cycle, you can combine the parameters from the configuration and from the node using the special `%default%` syntax:

```
baremetal node set node-0 \
  --driver-info kernel_append_params="%default% console=ttyS0,115200n8"
```

For ramdisk boot, the `instance_info[ramdisk_kernel_arguments]` property serves the same purpose (`%default%` is not supported since there is no default value in the configuration).

Pre-built ISO images

By default an ISO images is built per node using the deploy kernel and initramfs provided in the configuration or the nodes `driver_info`. Starting with the Wallaby release its possible to provide a pre-built ISO image:

```
baremetal node set node-0 \
  --driver_info deploy_iso=http://url/of/deploy.iso \
  --driver_info rescue_iso=http://url/of/rescue.iso
```

Note

OpenStack Image service (glance) image IDs and `file://` links are also accepted.

Note

NFS (NFSv3) and CIFS/SMB transport protocols are supported for virtual media attachment in addition to HTTP(S). HTTP is used by default. To use NFS or CIFS, configure the shares using `[nfs]base_url/[nfs]share_path` or `[cifs]base_url/[cifs]share_path`, and set `driver_info[vmedia_transport_protocol]` on each node. Operators are responsible for mounting the share on all conductor hosts.

Note

Before the Xena release the parameters were called `redfish_deploy_iso` and `redfish_rescue_iso` accordingly. The old names are still supported for backward compatibility.

No customization is currently done to the image, so e.g. *Layer 3 or DHCP-less ramdisk booting* wont work. *Configuring an ESP image* is also unnecessary.

Virtual Media Ramdisk

The `ramdisk` deploy interface can be used in concert with the `redfish-virtual-media` boot interface to facilitate the boot of a remote node utilizing pre-supplied virtual media. See *Booting a Ramdisk or an ISO* for information on how to enable and configure it.

Instead of supplying an `[instance_info]/image_source` parameter, a `[instance_info]/boot_iso` parameter can be supplied. The image will be downloaded by the conductor, and the instance will be booted using the supplied ISO image. In accordance with the `ramdisk` deployment interface behavior, once booted the machine will have a `provision_state` of `ACTIVE`.

```
baremetal node set <node name or UUID> \
    --boot-interface redfish-virtual-media \
    --deploy-interface ramdisk \
    --instance_info boot_iso=http://url/to.iso
```

This initial interface does not support bootloader configuration parameter injection, as such the `[instance_info]/kernel_append_params` setting is ignored.

Configuration drives are supported starting with the Wallaby release for nodes that have a free virtual USB slot:

```
baremetal node deploy <node name or UUID> \
    --config-drive '{"meta_data": {...}, "user_data": "..."}'
```

or via a link to a raw image:

```
baremetal node deploy <node name or UUID> \
    --config-drive http://example.com/config.img
```

Redfish HTTP(s) Boot

The `redfish-https` boot interface is very similar to the `redfish-virtual-media` boot interface. In this driver, we compose an ISO image, and request the BMC to inform the UEFI firmware to boot the Ironic ramdisk, or a other ramdisk image. This approach is intended to allow a pattern of engagement where we have minimal reliance on addressing and discovery of the Ironic deployment through autoconfiguration like DHCP, and somewhat mirrors vendor examples of booting from an HTTP URL.

This interface has some basic constraints.

- There is no configuration drive functionality, while Virtual Media did help provide such functionality.
- This interface *is* dependent upon BMC, EFI Firmware, and Bootloader, which means we may not see additional embedded files an contents in an ISO image. This is the same basic constraint over the `ramdisk` deploy interface when using Network Booting.
- This is a UEFI-Only boot interface. No legacy boot is possible with this interface.

A good starting point for this interface, is to think of it as higher security network boot, as we are explicitly telling the BMC where the node should boot from.

Like the `redfish-virtual-media` boot interface, you will need to create an EFI System Partition image (ESP), see *Configuring an ESP image* for details on how to do this.

Additionally, if you would like to use the `ramdisk` deployment interface, the same basic instructions covered in *Virtual Media Ramdisk* apply, just use `redfish-https` as the boot_interface, and keep in mind, no configuration drives exist with the `redfish-https` boot interface.

Limitations & Issues

Ironic contains two different ways of providing an HTTP(S) URL to a remote BMC. The first is Swift, enabled when *`redfish.use_swift`* is set to `true`. Ironic uploads files to Swift, which are then shared as Temporary Swift URLs. While highly scalable, this method does suffer from issues where some vendors BMCs reject URLs with **&** or **?** characters. There is no available workaround to leverage Swift in this state.

When the *`redfish.use_swift`* setting is set to `false`, Ironic will house the files locally in the *`deploy.http_root`* folder structure, and then generate a URL pointing the BMC to connect to the HTTP service configured via *`deploy.http_url`*.

Out-Of-Band inspection

The `redfish` hardware type can inspect the bare metal node by querying Redfish compatible BMC. This process is quick and reliable compared to the way the `agent` hardware type works i.e. booting bare metal node into the introspection ramdisk. The inspection collects various hardware information

including LLDP (Link Layer Discovery Protocol) data when available from the BMC, such as chassis ID, port ID, system name, system description, system capabilities, and management addresses.

Note

The `redfish` inspect interface relies on the optional parts of the Redfish specification. Not all Redfish-compliant BMCs might serve the required information, in which case bare metal node inspection will fail.

Note

The `local_gb` property cannot always be discovered, for example, when a node does not have local storage or the Redfish implementation does not support the required schema. In this case the property will be set to 0.

Retrieving BIOS Settings

When the *bios interface* is set to `redfish`, Ironic will retrieve the nodes BIOS settings as described in BIOS Configuration. In addition, via Sushy, Ironic will get the BIOS Attribute Registry (BIOS Registry) from the node which is a schema providing details on the settings. The following fields will be returned in the BIOS API (`/v1/nodes/{node_ident}/bios`) along with the setting name and value:

Field	Description
`attribute_typ`	The type of setting - `Enumeration`, `Integer`, `String`, `Boolean`, or `Password`
`allowable_val`	A list of allowable values when the attribute_type is `Enumeration`
`lower_bound`	The lowest allowed value when attribute_type is `Integer`
`upper_bound`	The highest allowed value when attribute_type is `Integer`
`min_length`	The shortest string length that the value can have when attribute_type is `String`
`max_length`	The longest string length that the value can have when attribute_type is `String`
`read_only`	The setting is ready only and cannot be modified
`unique`	The setting is specific to this node
`reset_require`	After changing this setting a node reboot is required

Further topics

Redfish hardware metrics

The `redfish` hardware type supports sending hardware metrics via the *notification system*. The `event_type` field of a notification will be set to `hardware.redfish.metrics` (where `redfish` may be replaced by a different driver name for hardware types derived from it).

Enabling redfish hardware metrics requires some ironic.conf configuration file updates:

```
[oslo_messaging_notifications]
# The Drivers(s) to handle sending notifications. Possible
# values are messaging, messagingv2, routing, log, test, noop,
# prometheus_exporter (multi valued)
# Example using the messagingv2 driver:
```

(continues on next page)

(continued from previous page)

```
driver = messagingv2

[sensor_data]
send_sensor_data = true

[metrics]
backend = collector
```

A full list of `[oslo_messaging_notifications]` configuration options can be found in the oslo.messaging documentation

The payload of each notification is a mapping where keys are sensor types (`Fan`, `Temperature`, `Power` or `Drive`) and values are also mappings from sensor identifiers to the sensor data.

Each `Fan` payload contains the following fields:

- `max_reading_range`, `min_reading_range` - the range of reading values.
- `reading`, `reading_units` - the current reading and its units.
- `serial_number` - the serial number of the fan sensor.
- `physical_context` - the context of the sensor, such as `SystemBoard`. Can also be `null` or just `Fan`.

Each `Temperature` payload contains the following fields:

- `max_reading_range_temp`, `min_reading_range_temp` - the range of reading values.
- `reading_celsius` - the current reading in degrees Celsius.
- `sensor_number` - the number of the temperature sensor.
- `physical_context` - the context of the sensor, usually reflecting its location, such as `CPU`, `Memory`, `Intake`, `PowerSupply` or `SystemBoard`. Can also be `null`.

Each `Power` payload contains the following fields:

- `power_capacity_watts`, `line_input_voltage`, `last_power_output_watts`
- `serial_number` - the serial number of the power source.
- `state` - the power source state: `enabled`, `absent` (`null` if unknown).
- `health` - the power source health status: `ok`, `warning`, `critical` (`null` if unknown).

Each `Drive` payload contains the following fields:

- `name` - the drive name in the BMC (this is **not** a Linux device name like `/dev/sda`).
- `model` - the drive model (if known).
- `capacity_bytes` - the drive capacity in bytes.
- `state` - the drive state: `enabled`, `absent` (`null` if unknown).
- `health` - the drive health status: `ok`, `warning`, `critical` (`null` if unknown).

Note

Drive payloads are often not available on real hardware.

Warning

Metrics collection works by polling several Redfish endpoints on the target BMC. Some older BMC implementations may have hard rate limits or misbehave under load. If this is the case for you, you need to reduce the metrics collection frequency or completely disable it.

Example (Dell)


```
{
    "message_id": "578628d2-9967-4d33-97ca-7e7c27a76abc",
    "publisher_id": "conductor-1.example.com",
    "event_type": "hardware.redfish.metrics",
    "priority": "INFO",
    "payload": {
        "message_id": "60653d54-87aa-43b8-a4ed-96d568dd4e96",
        "instance_uuid": null,
        "node_uuid": "aea161dc-2e96-4535-b003-ca70a4a7bb6d",
        "timestamp": "2023-10-22T15:50:26.841964",
        "node_name": "dell-430",
        "event_type": "hardware.redfish.metrics.update",
        "payload": {
            "Fan": {
                "0x17||Fan.Embedded.1A@System.Embedded.1": {
                    "identity": "0x17||Fan.Embedded.1A",
                    "max_reading_range": null,
                    "min_reading_range": 720,
                    "reading": 1680,
                    "reading_units": "RPM",
                    "serial_number": null,
                    "physical_context": "SystemBoard",
                    "state": "enabled",
                    "health": "ok"
                },
                "0x17||Fan.Embedded.2A@System.Embedded.1": {
                    "identity": "0x17||Fan.Embedded.2A",
                    "max_reading_range": null,
                    "min_reading_range": 720,
                    "reading": 3120,
                    "reading_units": "RPM",
                    "serial_number": null,
                    "physical_context": "SystemBoard",
                    "state": "enabled",
                    "health": "ok"
                },
```


(continues on next page)

(continued from previous page)


```
            "0x17||Fan.Embedded.2B@System.Embedded.1": {
                "identity": "0x17||Fan.Embedded.2B",
                "max_reading_range": null,
                "min_reading_range": 720,
                "reading": 3000,
                "reading_units": "RPM",
                "serial_number": null,
                "physical_context": "SystemBoard",
                "state": "enabled",
                "health": "ok"
            }
        },
        "Temperature": {
            "iDRAC.Embedded.1#SystemBoardInletTemp@System.Embedded.1": {
                "identity": "iDRAC.Embedded.1#SystemBoardInletTemp",
                "max_reading_range_temp": 47,
                "min_reading_range_temp": -7,
                "reading_celsius": 28,
                "physical_context": "SystemBoard",
                "sensor_number": 4,
                "state": "enabled",
                "health": "ok"
            },
            "iDRAC.Embedded.1#CPU1Temp@System.Embedded.1": {
                "identity": "iDRAC.Embedded.1#CPU1Temp",
                "max_reading_range_temp": 90,
                "min_reading_range_temp": 3,
                "reading_celsius": 63,
                "physical_context": "CPU",
                "sensor_number": 14,
                "state": "enabled",
                "health": "ok"
            }
        },
        "Power": {
            "PSU.Slot.1:Power@System.Embedded.1": {
                "power_capacity_watts": null,
                "line_input_voltage": 206,
                "last_power_output_watts": null,
                "serial_number": "CNLOD0075324D7",
                "state": "enabled",
                "health": "ok"
            },
            "PSU.Slot.2:Power@System.Embedded.1": {
                "power_capacity_watts": null,
                "line_input_voltage": null,
                "last_power_output_watts": null,
                "serial_number": "CNLOD0075324E5",
                "state": null,
```


(continues on next page)

(continued from previous page)


```
                    "health": "critical"
                }
            },
            "Drive": {
                "Solid State Disk 0:1:0:RAID.Integrated.1-1@System.Embedded.1
↪": {
                    "name": "Solid State Disk 0:1:0",
                    "capacity_bytes": 479559942144,
                    "state": "enabled",
                    "health": "ok"
                },
                "Physical Disk 0:1:1:RAID.Integrated.1-1@System.Embedded.1": {
                    "name": "Physical Disk 0:1:1",
                    "capacity_bytes": 1799725514752,
                    "state": "enabled",
                    "health": "ok"
                },
                "Physical Disk 0:1:2:RAID.Integrated.1-1@System.Embedded.1": {
                    "name": "Physical Disk 0:1:2",
                    "capacity_bytes": 1799725514752,
                    "state": "enabled",
                    "health": "ok"
                },
                "Backplane 1 on Connector 0 of Integrated RAID Controller␣
↪1:RAID.Integrated.1-1@System.Embedded.1": {
                    "name": "Backplane 1 on Connector 0 of Integrated RAID␣
↪Controller 1",
                    "capacity_bytes": null,
                    "state": "enabled",
                    "health": "ok"
                }
            }
        }
    },
    "timestamp": "2023-10-22 15:50:36.700458"
}
```


Node Vendor Passthru Methods

Method	Description
`create_subsc`	Create a new subscription on the Node
`delete_subsc`	Delete a subscription of a Node
`get_all_subs`	List all subscriptions of a Node
`get_subscrip`	Show a single subscription of a Node
`eject_vmedia`	Eject attached virtual media from a Node

Create Subscription

Table 1: Request

Name	In	Type	Description
Destination	body	string	The URI of the destination Event Service
EventTypes (optional)	body	array	List of types of events that shall be sent to the destination
Context (optional)	body	string	A client-supplied string that is stored with the event destination subscription
Protocol (optional)	body	string	The protocol type that the event will use for sending the event to the destination

Example JSON to use in `create_subscription`:

```
{
    "Destination": "https://someurl",
    "EventTypes": ["Alert"],
    "Context": "MyProtocol",
    "args": "Redfish"
}
```

Delete Subscription

Table 2: Request

Name	In	Type	Description
id	body	string	The Id of the subscription generated by the BMC

Example JSON to use in `delete_subscription`:

```
{
    "id": "<id of the subscription generated by the BMC>"
}
```

Get Subscription

Table 3: Request

Name	In	Type	Description
id	body	string	The Id of the subscription generated by the BMC

Example JSON to use in `get_subscription`:

```
{
    "id": "<id of the subscription generated by the BMC>"
}
```

Get All Subscriptions

The `get_all_subscriptions` doesnt require any parameters.

Eject Virtual Media

Table 4: Request

Name	In	Type	Description
boot_device (optional)	body	string	Type of the device to eject (all devices by default)

Internal Session Cache

The `redfish` hardware type, and derived interfaces, utilizes a built-in session cache which prevents Ironic from re-authenticating every time Ironic attempts to connect to the BMC for any reason.

This consists of cached connectors objects which are used and tracked by a unique consideration of `redfish_username`, `redfish_password`, `redfish_verify_ca`, and finally `redfish_address`. Changing any one of those values will trigger a new session to be created. The `redfish_system_id` value is explicitly not considered as Redfish has a model of use of one BMC to many systems, which is also a model Ironic supports.

The session cache default size is `1000` sessions per conductor. If you are operating a deployment with a larger number of Redfish BMCs, it is advised that you do appropriately tune that number. This can be tuned via the API service configuration file, *`redfish.connection_cache_size`*.

Session Cache Expiration

By default, sessions remain cached for as long as possible in memory, as long as they have not experienced an authentication, connection, or other unexplained error.

Under normal circumstances, the sessions will only be rolled out of the cache in order of oldest first when the cache becomes full. There is no time based expiration to entries in the session cache.

Of course, the cache is only in memory, and restarting the `ironic-conductor` will also cause the cache to be rebuilt from scratch. If this is due to any persistent connectivity issue, this may be sign of an unexpected condition, and please consider contacting the Ironic developer community for assistance.

Redfish Interoperability Profile

The Ironic project provides a Redfish Interoperability Profile located in `redfish-interop-profiles` folder at source code root. The Redfish Interoperability Profile is a JSON document written in a particular format that serves two purposes:

- It enables the creation of a human-readable document that merges the profile requirements with the Redfish schema into a single document for developers or users.

- It allows a conformance test utility to test a Redfish Service implementation for conformance with the profile.

The JSON document structure is intended to align easily with JSON payloads retrieved from Redfish Service implementations, to allow for easy comparisons and conformance testing. Many of the properties defined within this structure have assumed default values that correspond with the most common use case, so that those properties can be omitted from the document for brevity.

OpenStackIronicProfile 1.1.0

Specifies the OpenStack Ironic vendor-independent Redfish service requirements, typically offered by a baseboard management controller (BMC).

Bios

Allows reading or changing BIOS settings.

Properties

`Attributes` [required]
Current BIOS settings.

`AttributeRegistry`
Name of the registry with the schema of BIOS settings.

`@Redfish.Settings` [required]
`ETag`

`Messages`
Used to determine success or failure.

`SettingsObject` [required]
Provides a link to the actually updated object.

Link to a *Bios* resource.

`SupportedApplyTimes`
Determines whether update is immediate or needs a reboot.

Actions

`ResetBios`
Reset BIOS settings to their factory values.

Chassis

Allows collecting sensors data from the chassis.

Properties

`Power`
Provides a link to the power information.

Link to a *Power* resource.

Thermal
Provides a link to the thermal information.

Link to a *Thermal* resource.

UUID
Used as an ID for indicators.

ComputerSystem

Provides bare-metal node management. **[required]**

Properties

Bios
Reference to the corresponding Bios resource.

BiosVersion
Version of the system firmware.

Boot [required]
Allows changing boot devices and modes, which is fundamental for bare-metal provisioning.

BootSourceOverrideEnabled [required] [writable]
Manages whether the next boot device will be permanent or one-time.

BootSourceOverrideMode [required] [writable]
Allows switching boot mode to/from UEFI.

BootSourceOverrideTarget [required] [writable]
Allows changing the next boot device.

EthernetInterfaces
Provides a link to the nodes network interfaces.

Link to a collection of *EthernetInterface* resources.

IndicatorLED
Enables the bare-metal indicator API.

Links [required]

Chassis
Provides sensor data.

ManagedBy [required]
Provides a link from the node to its BMC.

Manufacturer
Provides the vendor property.

MemorySummary
Provides memory data during out-of-band inspection.

`TotalSystemMemoryGiB` **[required]**

PowerState [required]
Provides the current power state.

`Processors`
: Provides a link to the nodes CPUs.

  Link to a collection of *Processor* resources.

`SecureBoot`
: Provides a link to the nodes secure boot settings.

  Link to a *SecureBoot* resource.

`SimpleStorage`
: Provides disk data during out-of-band inspection.

  Link to a collection of *SimpleStorage* resources.

`Storage`
: Enables hardware RAID management.

  Link to a collection of *Storage* resources.

`VirtualMedia`
: Enables provisioning using virtual media.

  Link to a collection of *VirtualMedia* resources.

Actions

`Reset` **[required]**
: Provides an ability to execute power actions on the node.

  `ResetType` **[required]**

ComputerSystemCollection

At least one system is expected. **[required]**

Properties

`Members` **[required]**

Drive

Provides information about individual drives when configuring hardware RAID.

Properties

`CapacityBytes`

`MediaType` **[required]**

`Protocol` **[required]**

`Status` **[required]**
: `Health` **[required]**

  `State` **[required]**

EthernetInterface

Enables enrolling ports during inspection.

Properties

MACAddress [required]
MAC address is mandatory on ports.

Status [required]

Health [required]
Only healthy interfaces are considered.

State [required]
Enables filtering only enabled interfaces.

Manager

Provides access to the properties of the BMC.

Properties

FirmwareVersion
Provides the current firmware version of the BMC.

Power

Provides the current power information in the sensor data.

Properties

PowerSupplies [required]
Provides a list of the installed power supplies.

LastPowerOutputWatts

LineInputVoltage

PowerCapacityWatts

SerialNumber

Status [required]

Health
Health status to report in the sensors data.

State
Power state to report in the sensors data.

Processor

Provides CPU data during out-of-band inspection.

Properties

ProcessorArchitecture [required]
Used to determine the CPU architecture of the machine.

TotalThreads [required]
Used to estimate the core count.

SecureBoot

Allows turning secure boot mode on and off.

Properties

SecureBootEnable [required]
Allows reading and changing the secure boot state.

Actions

ResetKeys
Allows resetting secure boot keys via a step.

ServiceRoot

Provides links to all collections and services. **[required]**

Properties

Systems [required]
Provides a link to systems.

Link to a collection of *ComputerSystem* resources.

SessionService
Provides a link to the session service.

Link to a *SessionService* resource.

TaskService
Provides a link to the task service.

Link to a *TaskService* resource.

UpdateService
Provides a link to the update service.

Link to a *UpdateService* resource.

SessionService

Allows using sessions for authentication instead of HTTP basic authentication.

SimpleStorage

Provides information about disks during inspection as well as disk sensors.

Properties

`Devices` [required]

`CapacityBytes`
Disk capacity.

`Model`
Device model to report in the sensors data.

`Name`
Device name to report in the sensors data.

`Status` [required]

`Health`
Health status to report in the sensors data.

`State`
Device state to report in the sensors data.

Storage

Allows configuring hardware RAID.

Properties

`Drives` [required]
Provides a link to attached drives.

Link to a collection of *Drive* resources.

`StorageControllers` [required]
Provides information about storage controllers.

`SupportedRAIDTypes`
Defines which RAID types are supported.

`Volumes` [required]
Provides a link to existing volumes.

Link to a collection of *Volume* resources.

TaskService

Provides task management.

Thermal

Provides thermal information of a chassis as part of the sensors data.

Properties

Fans **[required]**

MaxReadingRange

MinReadingRange

Reading **[required]**

ReadingUnits **[required]**

SerialNumber

Status **[required]**

Health
Health status to report in the sensors data.

State
Device state to report in the sensors data.

Temperatures **[required]**

MaxReadingRangeTemp

MinReadingRangeTemp

ReadingCelsius **[required]**

PhysicalContext

SensorNumber

UpdateService

Actions

SimpleUpdate **[required]**
Enables firmware updates.

ImageURI **[required]**

Targets

TransferProtocol **[required]**

VirtualMedia

Enables provisioning using virtual media. **[required]**

Properties

Image **[required]**
URL of the image to attach.

Inserted

MediaTypes **[required]**
Supported media types for this virtual media slot.

WriteProtected

Actions

`EjectMedia` [required]
Enables ejecting virtual media devices.

`InsertMedia` [required]
Enables inserting virtual media devices.

`Image` **[required]**

`Inserted`

`TransferMethod`

`TransferProtocolType`

`WriteProtected`

Volume

Provides access to RAID volumes.

Properties

`CapacityBytes` **[required]**

`Name` **[required]**

`RAIDType`

VolumeCollection

Allows listing and creating RAID volumes.

Properties

`@Redfish.OperationApplyTimeSupport` **[required]**

Validation of Profiles using DMTF tool

An open source utility has been created by the Redfish Forum to verify that a Redfish Service implementation conforms to the requirements included in a Redfish Interoperability Profile. The Redfish Interop Validator is available for download from the DMTFs organization on Github at https://github.com/DMTF/Redfish-Interop-Validator. Refer to instructions in README on how to configure and run validation.

SNMP driver

The SNMP hardware type enables control of power distribution units of the type frequently found in data centre racks. PDUs frequently have a management ethernet interface and SNMP support enabling control of the power outlets.

The SNMP power interface works with the *PXE boot* interface for network deployment and network-configured boot.

Note

Unlike most of the other power interfaces, the SNMP power interface does not have a corresponding management interface. The SNMP hardware type uses the `noop` management interface instead.

List of supported devices

This is a non-exhaustive list of supported devices. Any device not listed in this table could possibly work using a similar driver.

Please report any device status.

Manufacturer	Model	Supported?	Driver name
APC	AP7920	Yes	apc_masterswitch
APC	AP9606	Yes	apc_masterswitch
APC	AP9225	Yes	apc_masterswitchplus
APC	AP7155	Yes	apc_rackpdu
APC	AP7900	Yes	apc_rackpdu
APC	AP7901	Yes	apc_rackpdu
APC	AP7902	Yes	apc_rackpdu
APC	AP7911a	Yes	apc_rackpdu
APC	AP7921	Yes	apc_rackpdu
APC	AP7922	Yes	apc_rackpdu
APC	AP7930	Yes	apc_rackpdu
APC	AP7931	Yes	apc_rackpdu
APC	AP7932	Yes	apc_rackpdu
APC	AP7940	Yes	apc_rackpdu
APC	AP7941	Yes	apc_rackpdu
APC	AP7951	Yes	apc_rackpdu
APC	AP7960	Yes	apc_rackpdu
APC	AP7990	Yes	apc_rackpdu
APC	AP7998	Yes	apc_rackpdu
APC	AP8941	Yes	apc_rackpdu
APC	AP8953	Yes	apc_rackpdu
APC	AP8959	Yes	apc_rackpdu
APC	AP8961	Yes	apc_rackpdu
APC	AP8965	Yes	apc_rackpdu
Aten	all?	Yes	aten
CyberPower	all?	Untested	cyberpower
EatonPower	all?	Untested	eatonpower
Teltronix	all?	Yes	teltronix
BayTech	MRP27	Yes	baytech_mrp27
Raritan	PX3-5547V-V2	Yes	raritan_pdu2
Raritan	PX3-5726V	Yes	raritan_pdu2
Raritan	PX3-5776U-N2	Yes	raritan_pdu2
Raritan	PX3-5969U-V2	Yes	raritan_pdu2
Raritan	PX3-5961I2U-V2	Yes	raritan_pdu2
Vertiv	NU30212	Yes	vertivgeist_pdu
ServerTech	CW-16VE-P32M	Yes	servertech_sentry3

continues on next page

Table 5 – continued from previous page

Manufacturer	Model	Supported?	Driver name
ServerTech	C2WG24SN	Yes	servertech_sentry4

Software Requirements

Additional python libraries to communicate with SNMP are required. Please see `driver-requirements.txt` for an updated list for your release.

Enabling the SNMP Hardware Type

1. Add `snmp` to the list of `enabled_hardware_types` in `ironic.conf`. Also update `enabled_management_interfaces` and `enabled_power_interfaces` in `ironic.conf` as shown below:

```
[DEFAULT]
enabled_hardware_types = snmp
enabled_management_interfaces = noop
enabled_power_interfaces = snmp
```

2. To enable the network boot fallback, update `enable_netboot_fallback` in `ironic.conf`:

```
[pxe]
enable_netboot_fallback = True
```

> **Note**
>
> It is important to enable the fallback as SNMP hardware type does not support setting of boot devices. When booting in legacy (BIOS) mode, the generated network booting artifact will force booting from local disk. In UEFI mode, Ironic will configure the boot order using UEFI variables.

3. Restart the Ironic conductor service.

```
service ironic-conductor restart
```

Ironic Node Configuration

Nodes configured to use the SNMP hardware type should have the `driver` field set to the hardware type `snmp`.

The following property values have to be added to the nodes `driver_info` field:

- `snmp_driver`: PDU manufacturer driver name or `auto` to automatically choose ironic snmp driver based on `SNMPv2-MIB::sysObjectID` value as reported by PDU.
- `snmp_address`: the IPv4 address of the PDU controlling this node.
- `snmp_port`: (optional) A non-standard UDP port to use for SNMP operations. If not specified, the default port (161) is used.
- `snmp_outlet`: The power outlet on the PDU (1-based indexing).

- `snmp_version`: (optional) SNMP protocol version (permitted values `1`, `2c` or `3`). If not specified, SNMPv1 is chosen.
- `snmp_community`: (Required for SNMPv1/SNMPv2c unless `snmp_community_read` and/or `snmp_community_write` properties are present in which case the latter take over) SNMP community name parameter for reads and writes to the PDU.
- `snmp_community_read`: SNMP community name parameter for reads to the PDU. Takes precedence over the `snmp_community` property.
- `snmp_community_write`: SNMP community name parameter for writes to the PDU. Takes precedence over the `snmp_community` property.
- `snmp_user`: (Required for SNMPv3) SNMPv3 User-based Security Model (USM) user name. Synonym for now obsolete `snmp_security` parameter.
- `snmp_auth_protocol`: SNMPv3 message authentication protocol ID. Valid values include: `none`, `md5`, `sha` for all pysnmp versions and additionally `sha224`, `sha256`, `sha384`, `sha512` for pysnmp versions 4.4.1 and later. Default is `none` unless `snmp_auth_key` is provided. In the latter case `md5` is the default.
- `snmp_auth_key`: SNMPv3 message authentication key. Must be 8+ characters long. Required when message authentication is used.
- `snmp_priv_protocol`: SNMPv3 message privacy (encryption) protocol ID. Valid values include: `none`, `des`, `3des`, `aes`, `aes192`, `aes256` for all pysnmp version and additionally `aes192blmt`, `aes256blmt` for pysnmp versions 4.4.3+. Note that message privacy requires using message authentication. Default is `none` unless `snmp_priv_key` is provided. In the latter case `des` is the default.
- `snmp_priv_key`: SNMPv3 message privacy (encryption) key. Must be 8+ characters long. Required when message encryption is used.
- `snmp_context_engine_id`: SNMPv3 context engine ID. Default is the value of authoritative engine ID.
- `snmp_context_name`: SNMPv3 context name. Default is an empty string.

The following command can be used to enroll a node with the `snmp` hardware type:

```
baremetal node create \
--driver snmp --driver-info snmp_driver=<pdu_manufacturer> \
--driver-info snmp_address=<ip_address> \
--driver-info snmp_outlet=<outlet_index> \
--driver-info snmp_community=<community_string>
```

Fake driver

Overview

The `fake-hardware` hardware type is what it claims to be: fake. Use of this type or the `fake` interfaces should be temporary or limited to non-production environments, as the `fake` interfaces do not perform any of the actions typically expected.

The `fake` interfaces can be configured to be combined with any of the real hardware interfaces, allowing you to effectively disable one or more hardware interfaces for testing by simply setting that interface to `fake`.

Use cases

Development

Developers can use `fake-hardware` hardware-type to mock out nodes for testing without those nodes needing to exist with physical or virtual hardware.

Scale testing

The `fake` drivers have a configurable delay in seconds which will result in those operations taking that long to complete. Two comma-delimited values will result in a delay with a triangular random distribution, weighted on the first value. These delays are applied to operations which typically block in other drivers. This allows more realistic scenarios to be arranged for performance and functional testing of an Ironic service without requiring real bare metal or faking at the BMC protocol level.

```
[fake]
power_delay = 5
boot_delay = 10
deploy_delay = 60,360
vendor_delay = 1
management_delay = 5
inspect_delay = 360,480
raid_delay = 10
bios_delay = 5
storage_delay = 10
rescue_delay = 120
```

Adoption

Some OpenStack deployers have used `fake` interfaces in Ironic to allow an adoption-style workflow with Nova. By setting a nodes hardware interfaces to `fake`, its possible to deploy to that node with Nova without causing any actual changes to the hardware or an OS already deployed on it.

This is generally an unsupported use case, but it is possible. For more information, see the relevant post from CERN TechBlog.

4.1.3 Changing Hardware Types and Interfaces

Hardware types and interfaces are enabled in the configuration as described in *Enabling drivers and hardware types*. Usually, a hardware type is configured on enrolling as described in *Enrolling hardware with Ironic*:

```
baremetal node create --driver <hardware type>
```

Any hardware interfaces can be specified on enrollment as well:

```
baremetal node create --driver <hardware type> \
    --deploy-interface direct --<other>-interface <other implementation>
```

For the remaining interfaces, the default value is assigned as described in *Defaults for hardware interfaces*. Both the hardware type and the hardware interfaces can be changed later via the node update API.

Changing Hardware Interfaces

Hardware interfaces can be changed by the following command:

```
baremetal node set <NODE> \
    --deploy-interface direct \
    --<other>-interface <other implementation>
```

The modified interfaces must be enabled and compatible with the current nodes hardware type.

Changing Hardware Type

Changing the nodes hardware type can pose a problem. When the `driver` field is updated, the final result must be consistent, that is, the resulting hardware interfaces must be compatible with the new hardware type. This will not work:

```
baremetal node create --name test --driver fake-hardware
baremetal node set test --driver ipmi
```

This is because the `fake-hardware` hardware type defaults to `fake` implementations for some or all interfaces, but the `ipmi` hardware type is incompatible with them. There are three ways to deal with this situation:

1. Provide new values for all incompatible interfaces, for example:

   ```
   baremetal node set test --driver ipmi \
       --boot-interface pxe \
       --deploy-interface direct \
       --management-interface ipmitool \
       --power-interface ipmitool
   ```

2. Request resetting some of the interfaces to their new defaults by using the `--reset-<IFACE>-interface` family of arguments, for example:

   ```
   baremetal node set test --driver ipmi \
       --reset-boot-interface \
       --reset-deploy-interface \
       --reset-management-interface \
       --reset-power-interface
   ```

3. Request resetting all interfaces to their new defaults:

   ```
   baremetal node set test --driver ipmi --reset-interfaces
   ```

   You can still specify explicit values for some interfaces:

   ```
   baremetal node set test --driver ipmi --reset-interfaces \
       --deploy-interface direct
   ```

Static boot order configuration

Some hardware is known to misbehave when changing the boot device through the BMC. To work around it you can use the `noop` management interface implementation with the `ipmi` and `redfish` hardware types. In this case the Bare Metal service will not change the boot device for you, leaving the pre-configured boot order.

For example, in the case of the *PXE boot*:

1. Via any available means configure the boot order on the node as follows:
    1. Boot from PXE/iPXE on the provisioning NIC.

    Warning

    If it is not possible to limit network boot to only provisioning NIC, make sure that no other DHCP/PXE servers are accessible by the node.

    2. Boot from the hard drive.
2. Make sure the `noop` management interface is enabled, for example:

```
[DEFAULT]
enabled_hardware_types = ipmi,redfish
enabled_management_interfaces = ipmitool,redfish,noop
```

3. Change the node to use the `noop` management interface:

```
baremetal node set <NODE> --management-interface noop
```

4.2 Bare Metal Service Features

4.2.1 Hardware Inspection

Overview

Inspection allows Bare Metal service to discover required node properties once required `driver_info` fields (for example, IPMI credentials) are set by an operator. Inspection will also create the Bare Metal service ports for the discovered ethernet MACs.

There are two kinds of inspection supported by Bare Metal service:

1. Out-of-band inspection is currently implemented by several hardware types, including `redfish`, `ilo` and `idrac`.
2. In-band inspection, also known as Agent inspection utilizing Ironic Python Agent to collect information.

The node should be in the `manageable` state before inspection is initiated. If it is in the `enroll` or `available` state, move it to `manageable` first:

```
baremetal node manage <node_UUID>
```

Then inspection can be initiated using the following command:

```
baremetal node inspect <node_UUID>
```

Functionality

Node auto-discovery

The Bare Metal service is capable of automatically enrolling new nodes that somehow (through external means, e.g. *Configuring unmanaged in-band inspection*) boot into an IPA ramdisk and call back with inspection data. This feature must be enabled explicitly in the configuration:

```
[DEFAULT]
default_inspect_interface = agent

[auto_discovery]
enabled = True
driver = ipmi
```

The newly created nodes will appear in the `enroll` provision state with the `driver` field set to the value specified in the configuration, as well as a boolean `auto_discovered` flag in the *Plugin data*.

After the node is enrolled, it will automatically go through the normal inspection process, which includes, among other things, creating ports. Any errors during this process will be reflected in the nodes `last_error` field (the node will not be deleted).

Limitations

- Setting BMC credentials is a manual task. The Bare Metal service does not generate new credentials for you even on those machines where its possible through `ipmitool`.
- Node uniqueness is checked using the supplied MAC addresses. In rare cases, it is possible to create duplicate nodes.
- Enabling discovery allows anyone with API access to create nodes with given MAC addresses and store inspection data of arbitrary size for them. This can be used for denial-of-service attacks.
- Setting `default_inspect_interface` is required for the inspection flow to continue correctly after the node creation.

Inspection data

The inspection process collects a lot of information about the node. This applies to both the `agent` (in-band) and `redfish` (out-of-band) inspection interfaces. The collected data consists of two parts:

- *Inventory* depends on the inspection interface used. See *Inventory*.
- *Plugin data* is data populated by ramdisk-side and server-side plug-ins. See *Plugin data*.

After a successful inspection, you can get both parts as JSON with:

```
$ baremetal node inventory save <NODE>
```

Use `jq` to filter the parts you need, e.g. only the inventory itself:

```
$ # System vendor information from the inventory
$ baremetal node inventory save <NODE> | jq .inventory.system_vendor
{
  "product_name": "KVM (9.2.0)",
  "serial_number": "",
  "manufacturer": "Red Hat",
  "firmware": {
    "vendor": "EDK II",
    "version": "edk2-20221207gitfff6d81270b5-7.el9",
    "build_date": "12/07/2022"
  }
}

$ # Interfaces used to create ports
$ baremetal node inventory save <NODE> | jq .plugin_data.valid_interfaces
{
  "eth0": {
    "name": "eth0",
    "mac_address": "52:54:00:5e:09:ff",
    "ipv4_address": "192.168.122.164",
    "ipv6_address": "fe80::5054:ff:fe5e:9ff",
    "has_carrier": true,
    "lldp": null,
    "vendor": "0x1af4",
    "product": "0x0001",
    "client_id": null,
    "biosdevname": null,
    "speed_mbps": null,
    "pxe_enabled": true
  }
}
```

Inventory

The shape of the `inventory` depends on the inspection interface used.

- *agent* - For information on what gets reported by this interface see hardware inventory.
- *redfish* - This interface aspires to the *agent* implementation but is known to be incomplete at this time.

Since many server-side plug-ins, known as *Inspection hooks* live in the Ironic source tree an effort is being made to define it here.

Top-Level

Key	Details
`cpu`	JSON object of CPU details
`memory`	JSON object of memory details
`bmc_address`	Optional IPv4 address as a string
`bmc_v6address`	Optional IPv6 address as a string
`disk`	List of JSON objects of disk details
`interfaces`	List of JSON objects of NIC details
`system_vendor`	JSON object of SMBIOS details
`boot`	JSON object of boot details
`lldp_raw`	Optional JSON object of LLDP data
`pci_devices`	Optional list of JSON objects of PCI details
`usb_devices`	Optional list of JSON objects of USB details

Plugin data

Plugin data is the storage for all information that is collected or processed by various plugins. Its format is not a part of the API stability promise and may change depending on your configuration.

Plugin data comes from two sources:

- inspection collectors - ramdisk-side inspection plug-ins.
- *Inspection hooks* - server-side inspection plug-ins.

Data storage

There are several options to store the inspection data, specified via the *`inventory.data_backend`* option:

`none`
: Do not store inspection data at all. The API will always return 404 NOT FOUND.

`database`
: Store inspection data in a separate table in the main database.

`swift`
: Store inspection data in the Object Store service (swift) in the container specified by the *`inventory.swift_data_container`* option.

Warning

There is currently no way to migrate data between backends. Changing the backend will remove access to existing data.

Inspection hooks

Inspection hooks are a type of the Bare Metal service plug-ins responsible for processing data from inspection. These hooks work with both the `agent` (in-band) and `redfish` (out-of-band) inspection interfaces. By configuring these hooks, an operator can fully customize the inspection processing phase.

For agent inspection, how the data is collected can be configured with inspection collectors.

Configuring hooks

Inspection hooks are configured separately for each inspection interface type.

Agent inspection hooks

For the `agent` inspection interface, two configuration options are responsible for inspection hooks: *inspector.default_hooks* defines which hooks run by default, while *inspector.hooks* defines which hooks to run in your deployment. Only the second option should be modified by operators, while the first one is to provide the defaults without hardcoding them:

```
[inspector]
hooks = $default_hooks
```

To make a hook run after the default ones, append it to the list, e.g.

```
[inspector]
hooks = $default_hooks,extra-hardware
```

Redfish inspection hooks

For the `redfish` inspection interface, the configuration is in the `[redfish]` section:

```
[redfish]
inspection_hooks = $default_inspection_hooks
```

To add additional hooks:

```
[redfish]
inspection_hooks = $default_inspection_hooks,extra-hardware
```

Default hooks

In the order they go in the *inspector.default_hooks* option:

ramdisk-error
: Processes the `error` field from the ramdisk, aborting inspection if it is not empty.

validate-interfaces
: Validates network interfaces and stores the result in the `plugin_data` in two fields:

  - `all_interfaces` - all interfaces that pass the basic functional check.
  - `valid_interfaces` - interfaces that satisfy the configuration in the *inspector.add_ports* option.

  In both cases, interfaces get an addition field:

  - `pxe_enabled` - whether PXE was enabled on this interface during the inspection boot.

ports
: Creates ports for interfaces in `valid_interfaces` as set by the `validate-interfaces` hook.

  Deletes ports that dont match the *inspector.keep_ports* setting.

architecture
Populates the `cpu_arch` property on the node.

Optional hooks

accelerators
Populates the `accelerators` property based on the reported PCI devices. The known accelerators are specified in the YAML file linked in the *inspector.known_accelerators* option. The default file is the following:

```
pci_devices:
  - vendor_id: "10de"
    device_id: "1eb8"
    type: GPU
    device_info: NVIDIA Corporation Tesla T4
  - vendor_id: "10de"
    device_id: "1df6"
    type: GPU
    device_info: NVIDIA Corporation GV100GL
  - vendor_id: "10de"
    device_id: "2236"
    type: GPU
    device_info: NVIDIA Corporation A10
  - vendor_id: "10de"
    device_id: "2235"
    type: GPU
    device_info: NVIDIA Corporation A40
  - vendor_id: "10de"
    device_id: "26b9"
    type: GPU
    device_info: NVIDIA Corporation L40S
  - vendor_id: "10de"
    device_id: "26ba"
    type: GPU
    device_info: NVIDIA Corporation L20
```

boot-mode
Sets the `boot_mode` capability based on the observed boot mode, see *Boot mode support*.

cpu-capabilities
Uses the CPU flags to *discover CPU capabilities*. The exact mapping can be customized via configuration:

```
[inspector]
cpu_capabilities = vmx:cpu_vt,svm:cpu_vt
```

See *inspector.cpu_capabilities* for the default mapping.

extra-hardware
Converts the data collected by python-hardware from its raw format into nested dictionaries under the `extra` plugin data field.

local-link-connection

Uses the LLDP information from the ramdisk to populate the `local_link_connection` field on ports with the physical switch information.

memory
Populates the `memory_mb` property based on physical RAM information from DMI.

parse-lldp
Parses the raw binary LLDP information from the ramdisk and populates the `parsed_lldp` dictionary in plugin data. The keys are network interface names, the values are dictionaries with LLDP values. Example:

```
"parsed_lldp": {
    "eth0": {
        "switch_chassis_id": "11:22:33:aa:bb:cc",
        "switch_system_name": "sw01-dist-1b-b12"
    }
}
```

pci-devices
Populates the capabilities based on PCI devices. The mapping is provided by the *inspector.pci_device_alias* option.

physical-network
Populates the `physical_network` port field for *Networking with the Bare Metal service* based on the detected IP addresses. The mapping is provided by the *inspector.physical_network_cidr_map* option.

raid-device
Detects the newly created RAID device and populates the `root_device` property used in *root device hints*. Requires two inspections: one before and one after the RAID creation.

root-device
Uses *root device hints* on the node and the storage device information from the ramdisk to calculate the expected root device and populate the `local_gb` property (taking the *inspector.disk_partitioning_spacing* option into account).

Inspection Rules

Inspection rules have now been migrated into Ironic as of the 2025.1 Epoxy release. This does not include support for reapplying inspection on already stored data, nor does it support the `"scope"` field.

The scope field allowed a rule to be applied only to specific nodes with matching scope value rather than all nodes where conditions are met.

An inspection rule consists of conditions to check, and actions to run. If conditions evaluate to true on the inspection data, then actions are run on a node.

Ironic provides an API to manage such rules. There are also built-in rules which are pre-saved and loaded from a YAML file and cannot be CRUD through the API.

Available conditions and actions are defined by an extendable set of plugins.

Refer to the Ironic API reference for inspection rules for information on how to CRUD inspection rules.

Actions & Conditions

Conditions and actions have the same base structure:

- `op` - operation: either boolean (conditions) or an action (actions).
- `args` - a list (in the sense of Python `*args`) or a dict (in the sense of Python `**kwargs`) with arguments.

Conditions

Available conditions include:

- `eq(*values, *, force_strings=False)` - Check if all values are equal. If force_strings, all values will be converted to strings first before the check.
- `lt(*values, *, force_strings=False)` - Check if all values are less than. If force_strings, all values will be converted to strings first before the check.
- `gt(*values, *, force_strings=False)` - Check if all values are greater than. If force_strings, all values will be converted to strings first before the check.
- `is-empty(value)` - Check if value is None or an empty string, list or a dictionary.
- `in-net(address, subnet)` - Check if the given address is in the provided subnet.
- `matches(value, regex)` - Check if the value fully matches the given regular expression.
- `contains(value, regex)` - Check if the value contains the given regular expression.
- `one-of(value, values)` - Check if the value is in the provided list. Similar to contains, but also works for non-string values.
- `is-none(value)` - Check if value is None.
- `is-true(value)` - Check if value evaluates to boolean True. This operator supports booleans, non-zero numbers and strings yes, true.
- `is-false(value)` - Check if value evaluates to boolean False. Supports booleans, zero, None and strings no, false.

To check for the inverse of any of these conditions, prefix the operator with an exclamation mark (with an optional space) before the op. E.g. `eq` - `!eq`.

Actions

Available actions include:

- `fail(msg)` - Fail inspection with the given message.
- `log(msg, level="info")` - Write the message to the Ironic logs.
- `set-attribute(path, value)` - Set the given path (in the sense of JSON patch) to the value.
- `extend-attribute(path, value, *, unique=False)` - Treat the given path as a list, append to it.
- `del-attribute(path)` - Unset the given path. Fails on invalid node attributes, but does not fail on missing subdict fields.
- `set-capability(name, value)` - Adds the given node capability with the supplied value.

- `unset-capability(name)` - Unsets the given node capability.
- `add-trait(name)` - Adds the given trait to the node.
- `remove-trait(name)` - Removes the given trait from the node.
- `set-plugin-data(path, value)` - Set a value in the plugin data.
- `extend-plugin-data(path, value, *, unique=False)` - Treat a value in the plugin data as a list, append to it. If unique is True, do not append if the item exists.
- `unset-plugin-data(path)` - Unset a value in the plugin data.
- `set-port-attribute(port_id, path, value)` - Set value on the port identified by a MAC or a UUID.
- `extend-port-attribute(port_id, path, value, *, unique=False)` - Treat the given path on the port as a list, append to it.
- `del-port-attribute(port_id, path)` - Unset value on the port identified by a MAC or a UUID.
- `api-call(url, *, headers=None, proxies=None, timeout=5, retries=3, backoff_factor=0.3)` - Performs an HTTP GET against the supplied URL.

Loops

Both conditions and actions accept an optional `loop` argument of list of items to iterate over for the same condition or action.

The `loop` field supports an Ansible-style loop (for reference, see Ansible loops documentation).

In conditions, theres an additional (and optional) `multiple` field which is only applicable when the loop field is present. It determines how the results of all loop iterations are combined:

- `any` (default) - returns `True` if any iterations result is `True`
- `all` - returns `True` only if all iterations results are `True`
- `first` - returns the result of the first iteration only, skipping remaining iterations if the first is `True`
- `last` - uses only the result from the last iteration, effectively ignoring previous iterations

For example, this condition check will return true if at any time of the iteration, the system is any of the models in the `loop` list:

```
- op: eq
  args: ["{inventory.system.product_name}", "{item}"]
  loop: ["HPE ProLiant DL380 Gen10", "PowerEdge R640", "Cisco UCS"]
  multiple: any
```

Whereas in actions, each iteration of the loop executes same action with the current item value.

Example of setting multiple attributes using loop:

```
- op: set-attribute
  args: ["{item[path]}", "{item[value]}"]
  loop:
    - {path: "/driver_info/ipmi_username", value: "admin"}
```

(continues on next page)

(continued from previous page)

```
    - {path: "/driver_info/ipmi_password", value: "password"}
    - {path: "/driver_info/ipmi_address", value: "{inventory[bmc_address]}"}
```

Note

Both dot (`"driver_info.ipmi_username"`) and slash (`"driver_info/ipmi_username"`) notation paths are supported.

Variable Interpolation

```
actions:
  - op: "set-attribute"
    args:
      path: "/driver_info/ipmi_address"
      value: "{data[inventory][bmc_address]}"
```

On a rule execution, values enclosed with braces, usually `value`, `msg`, `address`, and `subnet` fields in both actions and conditions, will be treated as replacement fields and formatted to a string using python string formatting notation.

If the value of any of these keys is a dict or list, strings nested at any level within the structure will be recursively formatted as well:

```
actions:
  - op: "set-attribute"
    args:
      path: "/properties/root_device"
      value: '{"serial": "{data[root_device][serial]}"}'
```

Capabilities discovery

This is an incomplete list of capabilities we want to discover during inspection. The exact support is hardware and hardware type specific though, the most complete list is provided by the iLO *Hardware Inspection Support*.

`secure_boot` (`true` or `false`)
: whether secure boot is supported for the node

`boot_mode` (`bios` or `uefi`)
: the boot mode the node is using

`cpu_vt` (`true` or `false`)
: whether the CPU virtualization is enabled

`cpu_aes` (`true` or `false`)
: whether the AES CPU extensions are enabled

`max_raid_level` (integer, 0-10)
: maximum RAID level supported by the node

`pci_gpu_devices` (non-negative integer)
: number of GPU devices on the node

The operator can specify these capabilities in nova flavor for node to be selected for scheduling:

```
openstack flavor set my-baremetal-flavor --property capabilities:pci_gpu_
↪devices="> 0"

openstack flavor set my-baremetal-flavor --property capabilities:secure_boot=
↪"true"
```

Please see a specific *hardware type page* for the exact list of capabilities this hardware type can discover.

Managed and unmanaged inspection

Agent inspection can be *managed* or *unmanaged*. This document explains the difference between these two concepts and applies both to the built-in agent inspection and to /admin/inspection.

Managed inspection

Inspection is *managed* when the Bare Metal conductor fully configures the node for inspection, including setting boot device, boot mode and power state. This is the only way to conduct inspection using *Virtual media boot* or with *Layer 3 or DHCP-less ramdisk booting*. This mode is engaged automatically when the node has sufficient information to configure boot (e.g. ports in case of iPXE).

For network interfaces based on OpenStack Networking (e.g. `flat` and `neutron`), the UUID or name of the inspection network must be provided via configuration or `driver_info`, for example:

```
[neutron]
inspection_network = <NETWORK UUID>
```

There are a few configuration options that tune managed inspection, the most important is `extra_kernel_params`, which allows adding kernel parameters for inspection specifically. This is where you can configure inspection collectors and other parameters, for example:

```
[inspector]
extra_kernel_params = ipa-inspection-collectors=default,logs ipa-collect-
↪lldp=1
```

For the built-in inspection, the bare metal API endpoint can be overridden instead:

```
[service_catalog]
endpoint_override = https://example.com/baremetal
```

Unmanaged inspection

Under *unmanaged* inspection we understand agent inspection where the boot configuration (iPXE scripts, DHCP options, etc) is not provided by the Bare Metal service. In this case, the node is simply set to boot from network and powered on. The operator is responsible for the correct network boot configuration, e.g. as explained in *Configuring unmanaged in-band inspection*.

Unmanaged inspection was the only inspection mode before the Ussuri release, and it is still used when the nodes boot cannot be configured by the conductor. The options described above do not affect unmanaged inspection.

Because of the complex installation and operation requirements, unmanaged inspection is disabled by default. To enable it, set `require_managed_boot` to `False`:

```
[inspector]
require_managed_boot = False
```

PXE filter service

The PXE filter service is responsible for managing the dnsmasq instance that is responsible for *Unmanaged inspection*. Running it allows this dnsmasq instance to co-exist with the OpenStack Networking services DHCP server on the same physical network.

How it works?

At the core of the PXE filter service is a periodic task that fetches all ports and compares the node IDs with the IDs of the nodes undergoing in-band inspection. All of the MAC addresses are added to the dnsmasq host files: to the allowlist of nodes on inspection and to the denylist for the rest.

Additionally, when any nodes are on inspection, unknown MACs are also allowed. Otherwise, access from unknown MACs to the dnsmasq service is denied.

Installation

Start with *Configuring unmanaged in-band inspection*. Then create a *hostsdir* writable by the PXE filter service and readable by dnsmasq. Configure it in the dnsmasq configuration file

```
dhcp-hostsdir=/var/lib/ironic/hostsdir
```

and in the Bare Metal service configuration

```
[pxe_filter]
dhcp_hostsdir = /var/lib/ironic/hostsdir
```

Then create a systemd service to start `ironic-pxe-filter` alongside dnsmasq, e.g.

```
[Unit]
Description=Ironic PXE filter

[Service]
Type=notify
Restart=on-failure
ExecStart=/usr/bin/ironic-pxe-filter --config-file /etc/ironic/ironic.conf
User=ironic
Group=ironic
```

Note that because of technical limitations, the PXE filter process cannot clean up the *hostsdir* itself. You may want to do it on the service start-up, e.g. like this (assuming the dnsmasq service is `ironic-dnsmasq` and its PID is stored in `/run/ironic/dnsmasq.pid`):

```
[Unit]
Description=Ironic PXE filter
Requires=ironic-dnsmasq.service
After=ironic-dnsmasq.service
```

(continues on next page)

(continued from previous page)

```
[Service]
Type=notify
Restart=on-failure
ExecStartPre=+/bin/bash -c "rm -f /usr/lib/ironic/hostsdir/* && kill -HUP
↪$(cat /run/ironic/dnsmasq.pid) || true"
ExecStart=/usr/bin/ironic-pxe-filter --config-file /etc/ironic/ironic.conf
User=ironic
Group=ironic
```

Scale considerations

The PXE filter service should be run once per each dnsmasq instance dedicated to unmanaged inspection. In most clouds, that will be 1 instance.

Migrating from ironic-inspector

This document outlines the process of migrating from a separate ironic-inspector service to the built-in /admin/inspection.

Note

This is a live document that is updated as more ironic-inspector features are supported in ironic. If youre upgrading to a branch other than `master`, use the version of this document from the target branch.

Understand the feature differences

Removed

Some rarely used or controversial features have not been migrated to ironic. This list currently includes:

- Retrieving unprocessed inspection data
- Reapplying the processing pipeline on new data
- *Node auto-discovery* is no longer based on plug-ins.
- Introspection of nodes in the `active` provision state.
- PXE filters based on `iptables`.
- Certain client commands are not available in ironicclient, for example, the ones that display the network interface information from the LLDP data.

ironic-inspector *Inspection rules* have now been implemented as of 2025.1 Epoxy release.

New defaults

- The database *`data storage backend`* is used by default.
- The list of *`default hooks`* is limited to only most commonly used ones (see also *Built-in hooks*).

Built-in hooks

Most of the ironic-inspector introspection hooks have been *migrated to ironic*, although many have been migrated for clarity and consistency.

Table 6: Hooks mapping

Inspector	ironic	default_hook	Notes
accelerators	accelerators	No	
capabilities	boot-mode, cpu-capabilities	No	Split into two logical parts.
extra_hardware	extra-hardware	No	python-hardware is not actively maintained any more.
lldp_basic	parse-lldp	No	
local_link_conn	local-link-connect:	No	
pci_devices	pci-devices	No	
physnet_cidr_ma	physical-network	No	
raid_device	raid-device	No	
root_device	root-device	No	
ramdisk_error	ramdisk-error	**Yes**	
scheduler	architecture, memory, cpus	Only architecture	Split, local_gb is enabled by the root-device hook.
validate_interf	validate-interfaces ports	**Yes**	Split into two logical parts.

Custom hooks

A custom hook (called *processing hook* in ironic-inspector) has to be derived from the base class *InspectionHook*. It differs from the older ironic-inspector ProcessingHook in a few important ways, requiring custom hooks to be adapted for ironic:

- Hooks operate on the regular *task* instead of the inspector-specific NodeInfo object.
- Since changes to nodes and ports no longer require an API call, hooks are expected to commit their changes immediately rather than letting them accumulate on the task object.
- The hook methods have been renamed: before_processing is called preprocess, the __call__ method is used instead of before_update.
- *Introspection data* has been split into its *inventory* part and *plugin data*. Hooks should not update the inventory.
- New hooks use the entry point ironic.inspection.hooks instead of ironic_inspector.hooks.processing.

Other concerns

- There is no way to migrate the inspection data automatically. You need to repeat inspections or copy the data over manually. For migration of data between Swift buckets, you could adapt *Copy inspection data between Swift buckets*.

Migration process

1. Make sure youre running at ironic 2024.1 or newer.
2. Enable the new inspection implementation as described in /admin/inspection.
3. Carefully research options in the `inventory` and `inspector` sections. Update options to match similar ones in the ironic-inspector configuration.
4. Enable the required *Built-in hooks*, taking into the account the new names and composition.
5. If using network boot and *unmanaged* inspection or auto-discovery, *configure unmanaged boot*.
6. If using the OpenStack Networking, consider configuring (but not starting yet) the *PXE filter service*.
7. Make sure no inspection are running.
8. Stop ironic-inspector or at least disable its PXE filter (it may conflict with the one used here).
9. Start *PXE filter service* if needed. Restart the Bare Metal service.
10. Change all nodes to use the new inspection interface, for example:

```
baremetal node list --fields uuid inspect_interface -f value | while read␣
↪uuid iface; do
    if [ "$iface" = "inspector" ]; then
        baremetal node set --inspect-interface agent "$uuid"
    fi
done
```

11. Make sure your scripts use ironicclient and the Bare Metal API in OpenStackSDK instead of the client API that is specific to ironic-inspector.

Copy inspection data between Swift buckets

This script assumes that you have S3 credentials for the buckets. You only have to configure the 5 configuration parameters.

```
import boto3
import json
from botocore.exceptions import ClientError

# Configure your S3 buckets and Ceph endpoint
SOURCE_BUCKET = ''
DEST_BUCKET = ''

# Ceph S3 configuration
CEPH_ENDPOINT = ''
CEPH_ACCESS_KEY = ''
CEPH_SECRET_KEY = ''

def get_s3_client():
    """Initialize and return S3 client for Ceph"""
    session = boto3.Session(
        aws_secret_access_key=CEPH_SECRET_KEY,
```

(continues on next page)

(continued from previous page)

```
        aws_access_key_id=CEPH_ACCESS_KEY)
    return session.client(
        's3',
        endpoint_url=CEPH_ENDPOINT)

def list_files_to_process(s3_client, bucket):
    """List all files in bucket that don't end with '-UNPROCESSED'"""
    files = []
    try:
        paginator = s3_client.get_paginator('list_objects_v2')
        for page in paginator.paginate(Bucket=bucket):
            if 'Contents' in page:
                for obj in page['Contents']:
                    key = obj['Key']
                    if not key.endswith('-UNPROCESSED'):
                        files.append(key)
    except ClientError as e:
        print(f"Error listing files: {e}")
        raise
    return files

def load_json_from_s3(s3_client, bucket, key):
    """Load and parse JSON file from S3"""
    try:
        response = s3_client.get_object(Bucket=bucket, Key=key)
        content = response['Body'].read().decode('utf-8')
        return json.loads(content)
    except ClientError as e:
        print(f"Error reading {key}: {e}")
        raise
    except json.JSONDecodeError as e:
        print(f"Error parsing JSON from {key}: {e}")
        raise

def save_json_to_s3(s3_client, bucket, key, data):
    """Save JSON data to S3"""
    try:
        s3_client.put_object(
            Bucket=bucket,
            Key=key,
            Body=json.dumps(data, indent=2),
            ContentType='application/json'
        )
        print(f"Saved: {key}")
    except ClientError as e:
        print(f"Error saving {key}: {e}")
        raise

def process_files():
```

(continues on next page)

(continued from previous page)

```
    """Main processing function"""
    s3_client = get_s3_client()
    print(f"Fetching files from {SOURCE_BUCKET}...")
    files = list_files_to_process(s3_client, SOURCE_BUCKET)
    print(f"Found {len(files)} files to process")

    # Process each file
    for file_key in files:
        print(f"\nProcessing: {file_key}")

        try:
            # Load JSON data
            data = load_json_from_s3(s3_client, SOURCE_BUCKET, file_key)

            # Split data
            inventory = data.pop('inventory', None)
            plugin = data

            # Check if inventory key existed
            if inventory is None:
                print(f"Warning: 'inventory' key not found in {file_key}")

            # Generate output filenames
            inventory_key = f"{file_key}-inventory"
            plugin_key = f"{file_key}-plugin"

            # Save split files
            if inventory is not None:
                save_json_to_s3(s3_client, DEST_BUCKET, inventory_key,␣
↪inventory)
            if plugin is not None:
                save_json_to_s3(s3_client, DEST_BUCKET, plugin_key, plugin)

        except Exception as e:
            print(f"Failed to process {file_key}: {e}")
            continue

    print("\nProcessing complete!")

if __name__ == "__main__":
    process_files()
```

Agent Inspection

Agent inspection (also known as in-band inspection) involves booting a ramdisk on the target node and fetching information directly from it. This process is more fragile and time-consuming than the out-of-band inspection, but it is not vendor-specific and works across a wide range of hardware.

Note

The implementation described in this document is not 100% compatible with the previous one (based on ironic-inspector). Check the documentation and the release notes for which features are currently available.

Configuration

Agent inspection is supported by all hardware types. The `agent` *inspect* interface has to be enabled to use it:

```
[DEFAULT]
enabled_inspect_interfaces = agent,redfish,no-inspect
```

You can make `agent` the default if you want all nodes to use it automatically:

```
[DEFAULT]
default_inspect_interface = agent
```

Of course, you can configure `agent` per node:

```
$ baremetal node set --inspect-interface agent <NODE>
```

4.2.2 Using deploy steps and templates

Contents

- *Using deploy steps and templates*
  - *Overview*
  - *Deploy Steps*
  - *Deploy Templates*

Overview

Node deployment is performed by the Bare Metal service to prepare a node for use by a workload. The exact work flow used depends on a number of factors, including the hardware type and interfaces assigned to a node.

Deploy Steps

The Bare Metal service implements deployment by collecting a list of deploy steps to perform on a node from the Power, Deploy, Management, BIOS, and RAID interfaces of the driver assigned to the node. These steps are then ordered by priority and executed on the node when the node is moved to the `deploying` state.

Nodes move to the `deploying` state when attempting to move to the `active` state (when the hardware is prepared for use by a workload). For a full understanding of all state transitions into deployment, please see *Bare Metal State Machine*.

The Bare Metal service added support for deploy steps in the Rocky release.

Order of execution

Deploy steps are ordered from higher to lower priority, where a larger integer is a higher priority. If the same priority is used by deploy steps on different interfaces, the following resolution order is used: Power, Management, Deploy, BIOS, and RAID interfaces.

Agent steps

All deploy interfaces based on ironic-python-agent (i.e. `direct`, `ansible` and any derivatives) expose the following deploy steps:

`deploy.deploy` (priority 100)
In this step the node is booted using a provisioning image.

`deploy.write_image` (priority 80)
An out-of-band (`ansible`) or in-band (`direct`) step that downloads and writes the image to the node.

`deploy.tear_down_agent` (priority 40)
In this step the provisioning image is shut down.

`deploy.switch_to_tenant_network` (priority 30)
In this step networking for the node is switched from provisioning to tenant networks.

`deploy.boot_instance` (priority 20)
In this step the node is booted into the user image.

Additionally, the `direct` deploy interfaces has:

`deploy.prepare_instance_boot` (priority 60)
In this step the boot device is configured and the bootloader is installed.

> **Note**
>
> For the `ansible` deploy interface these steps are done in `deploy.write_image`.

Accordingly, the following priority ranges can be used for custom deploy steps:

> 100
Out-of-band steps to run before deployment.

81 to 99
In-band deploy steps to run before the image is written.

61 to 79
In-band deploy steps to run after the image is written but before the bootloader is installed.

41 to 59
In-band steps to run after the image is written the bootloader is installed.

21 to 39
Out-of-band steps to run after the provisioning image is shut down.

1 to 19
Any steps that are run when the user instance is already running.

In-band steps

More deploy steps can be provided by the ramdisk, see IPA hardware managers documentation for a listing.

Requesting steps

Starting with Bare Metal API version 1.69 user can optionally supply deploy steps for node deployment when invoking deployment or rebuilding. Overlapping steps will take precedence over *Agent steps* and *Deploy Templates* steps.

Using baremetal client deploy steps can be passed via `--deploy-steps` argument. The argument `--deploy-steps` is one of:

- a JSON string
- path to a JSON file whose contents are passed to the API
- -, to read from stdin. This allows piping in the deploy steps.

An example by passing a JSON string:

```
baremetal node deploy <node> \
    --deploy-steps '[{"interface": "bios", "step": "apply_configuration",
↪"args": {"settings": [{"name": "LogicalProc", "value": "Enabled"}]},
↪"priority": 150}]'
```

Format of JSON for deploy steps argument is described in *Deploy step format* section.

Note

Starting with `ironicclient` 4.6.0 you can provide a YAML file for `--deploy-steps`.

Excluding the default steps

Starting with the Xena release, you can use the new *Custom agent deploy* interface to exclude the default step `write_image` and skip bootloader installation in the `prepare_instance_boot` step.

Writing a Deploy Step

Please refer to *Developing deploy and clean steps*.

FAQ

What deploy step is running?

To check what deploy step the node is performing or attempted to perform and failed, run the following command; it will return the value in the nodes `driver_internal_info` field:

```
baremetal node show <node> -f value -c driver_internal_info
```

The `deploy_steps` field will contain a list of all remaining steps with their priorities, and the first one listed is the step currently in progress or that the node failed before going into `deploy failed` state.

Troubleshooting

If deployment fails on a node, the node will be put into the `deploy failed` state until the node is deprovisioned. A deprovisioned node is moved to the `available` state after the cleaning process has been performed successfully.

Strategies for determining why a deploy step failed include checking the ironic conductor logs, checking logs from the ironic-python-agent that have been stored on the ironic conductor, or performing general hardware troubleshooting on the node.

Deploy Templates

Starting with the Stein release, with Bare Metal API version 1.55, deploy templates offer a way to define a set of one or more deploy steps to be executed with particular sets of arguments and priorities.

Each deploy template has a name, which must be a valid trait. Traits can be either standard or custom. Standard traits are listed in the os_traits library. Custom traits must meet the following requirements:

- prefixed with `CUSTOM_`
- contain only upper case characters A to Z, digits 0 to 9, or underscores
- no longer than 255 characters in length

Deploy step format

An invocation of a deploy step is defined in a deploy template as follows:

```
{
    "interface": "<name of the driver interface>",
    "step": "<name of the step>",
    "args": {
        "<arg1>": "<value1>",
        "<arg2>": "<value2>"
    },
    "priority": <priority of the step>
}
```

A deploy template contains a list of one or more such steps. Each combination of `interface` and `step` may only be specified once in a deploy template.

Matching deploy templates

During deployment, if any of the traits in a nodes `instance_info.traits` field match the name of a deploy template, then the steps from that deploy template will be added to the list of steps to be executed by the node.

When using the Compute service, any traits in the instances flavor properties or image properties are stored in `instance_info.traits` during deployment. See *Scheduling based on traits* for further information on how traits are used for scheduling when the Bare Metal service is used with the Compute service.

Note that there is no ongoing relationship between a node and any templates that are matched during deployment. The set of matching deploy templates is checked at deployment time. Any subsequent updates to or deletion of those templates will not be reflected in the nodes configuration unless it is redeployed or rebuilt. Similarly, if a node is rebuilt and the set of matching deploy templates has changed

since the initial deployment, then the resulting configuration of the node may be different from the initial deployment.

Overriding default deploy steps

A deploy step is enabled by default if it has a non-zero default priority. A default deploy step may be overridden in a deploy template. If the steps priority is a positive integer it will be executed with the specified priority and arguments. If the steps priority is zero, the step will not be executed.

If the *deploy.deploy step* is included in a deploy template, it can only be assigned a priority of zero to disable it.

Creating a deploy template via API

A deploy template can be created using the Bare Metal API:

```
POST /v1/deploy_templates
```

Here is an example of the body of a request to create a deploy template with a single step:

```
{
    "name": "CUSTOM_HYPERTHREADING_ON",
    "steps": [
        {
            "interface": "bios",
            "step": "apply_configuration",
            "args": {
                "settings": [
                    {
                        "name": "LogicalProc",
                        "value": "Enabled"
                    }
                ]
            },
            "priority": 150
        }
    ]
}
```

Further information on this API is available here.

Creating a deploy template via baremetal client

A deploy template can be created via the `baremetal deploy template create` command, starting with `python-ironicclient` 2.7.0.

The argument `--steps` must be specified. Its value is one of:

- a JSON string
- path to a JSON file whose contents are passed to the API
- -, to read from stdin. This allows piping in the deploy steps.

Example of creating a deploy template with a single step using a JSON string:

```
baremetal deploy template create \
    CUSTOM_HYPERTHREADING_ON \
    --steps '[{"interface": "bios", "step": "apply_configuration", "args": {
↪"settings": [{"name": "LogicalProc", "value": "Enabled"}]}, "priority": 150}
↪]'
```

Or with a file:

```
baremetal deploy template create \
    CUSTOM_HYPERTHREADING_ON \
    ---steps my-deploy-steps.txt
```

Or with stdin:

```
cat my-deploy-steps.txt | baremetal deploy template create \
    CUSTOM_HYPERTHREADING_ON \
    --steps -
```

Example of use with the Compute service

Note

The deploy steps used in this example are for example purposes only.

In the following example, we first add the trait `CUSTOM_HYPERTHREADING_ON` to the node represented by <node>:

```
baremetal node add trait <node> CUSTOM_HYPERTHREADING_ON
```

We also update the flavor `bm-hyperthreading-on` in the Compute service with the following property:

```
openstack flavor set --property trait:CUSTOM_HYPERTHREADING_ON=required bm-
↪hyperthreading-on
```

Creating a Compute instance with this flavor will ensure that the instance is scheduled only to Bare Metal nodes with the `CUSTOM_HYPERTHREADING_ON` trait.

We could then create a Bare Metal deploy template with the name `CUSTOM_HYPERTHREADING_ON` and a deploy step that enables Hyperthreading:

```
{
    "name": "CUSTOM_HYPERTHREADING_ON",
    "steps": [
        {
            "interface": "bios",
            "step": "apply_configuration",
            "args": {
                "settings": [
                    {
                        "name": "LogicalProc",
```

(continues on next page)

(continued from previous page)

```
                        "value": "Enabled"
                    }
                ]
            },
            "priority": 150
        }
    ]
}
```

When an instance is created using the `bm-hyperthreading-on` flavor, then the deploy steps of deploy template `CUSTOM_HYPERTHREADING_ON` will be executed during the deployment of the scheduled node, causing Hyperthreading to be enabled in the nodes BIOS configuration.

To make this example more dynamic, lets add a second trait `CUSTOM_HYPERTHREADING_OFF` to the node:

```
baremetal node add trait <node> CUSTOM_HYPERTHREADING_OFF
```

We could also update a second flavor, `bm-hyperthreading-off`, with the following property:

```
openstack flavor set --property trait:CUSTOM_HYPERTHREADING_OFF=required bm-
↪hyperthreading-off
```

Finally, we create a deploy template with the name `CUSTOM_HYPERTHREADING_OFF` and a deploy step that disables Hyperthreading:

```
{
    "name": "CUSTOM_HYPERTHREADING_OFF",
    "steps": [
        {
            "interface": "bios",
            "step": "apply_configuration",
            "args": {
                "settings": [
                    {
                        "name": "LogicalProc",
                        "value": "Disabled"
                    }
                ]
            },
            "priority": 150
        }
    ]
}
```

Creating a Compute instance with the `bm-hyperthreading-off` instance will cause the scheduled node to have Hyperthreading disabled in the BIOS during deployment.

We now have a way to create Compute instances with different configurations, by choosing between different Compute flavors, supported by a single Bare Metal node that is dynamically configured during deployment.

4.2.3 Node cleaning

Overview

Ironic provides two modes for node cleaning: `automated` and `manual`.

`Automated cleaning` is automatically performed before the first workload has been assigned to a node and when hardware is recycled from one workload to another.

`Manual cleaning` must be invoked by the operator.

Automated cleaning

When hardware is recycled from one workload to another, Ironic performs automated cleaning on the node to ensure its ready for another workload. This ensures the tenant will get a consistent bare metal node deployed every time.

With automated cleaning, nodes move to `cleaning` state when moving from `active -> available` state (when the hardware is recycled from one workload to another). Nodes also traverse cleaning when going from `manageable -> available` state (before the first workload is assigned to the nodes). For a full understanding of all state transitions into cleaning, please see *Bare Metal State Machine*.

Enabling automated cleaning

To enable automated cleaning, ensure that your ironic.conf is set as follows:

```
[conductor]
automated_clean=true
```

This will enable the default set of cleaning steps, based on your hardware and Ironic hardware types used for nodes. This includes, by default, erasing all of the previous tenants data.

You may also need to configure a *Cleaning Network*.

Cleaning steps

The way cleaning steps are determined depends on the value of *conductor.automated_cleaning_step_source*:

Autogenerated cleaning steps (autogenerated)
: This is the default mode of Ironic automated cleaning and provides the original Ironic behavior implemented originally in Kilo.

  Steps are collected from hardware interfaces and ordered from higher to lower priority, where a larger integer is a higher priority. In case of a conflict between priorities across interfaces, the following resolution order is used: Power, Management, Deploy, BIOS, and RAID interfaces.

  You can skip a cleaning step by setting the priority for that cleaning step to zero or `None`. You can reorder the cleaning steps by modifying the integer priorities of the cleaning steps.

Runbook-based cleaning steps (runbook)
: When using *Runbooks for Cleaning & Servicing* for automated cleaning, the exact steps and their order are defined in the runbook. Priority-based ordering does not apply; steps execute in the order specified in the runbook.

  If there is not a runbook assigned to perform cleaning on the node, and automated_cleaning is enabled, the machine will fail to clean and go into a `clean failed` state.

Hybrid (hybrid)
: This uses a runbook-based cleaning method if a cleaning runbook is configured for the node being cleaned. In this mode, if there is not a runbook configured for cleaning Ironic will fall-back to autogenerating cleaning steps.

See *How do I change the priority of a cleaning step?* for more information on changing the priority of an autogenerated cleaning step.

See *Configuring automated cleaning with runbooks* for full details on configuring cleaning runbooks.

Storage cleaning options

Warning

Ironics storage cleaning options by default will remove data from the disk permanently during automated cleaning.

Clean steps specific to storage are `erase_devices`, `erase_devices_metadata` and (added in Yoga) `erase_devices_express`.

`erase_devices` aims to ensure that the data is removed in the most secure way available. On devices that support hardware-assisted secure erasure (many NVMe and some ATA drives), this is the preferred option. If hardware-assisted secure erasure is not available and if *deploy.continue_if_disk_secure_erase_fails* is set to `True`, cleaning will fall back to using `shred` to overwrite the contents of the device. By default, if `erase_devices` is enabled and Ironic is unable to erase the device, cleaning will fail to ensure data security.

Note

`erase_devices` may take a very long time (hours or even days) to complete, unless fast, hardware-assisted data erasure is supported by all the devices in a system.

`erase_devices_metadata` clean step doesnt provide as strong assurance of irreversible destruction of data as `erase_devices`. However, it has the advantage of a reasonably quick runtime (seconds to minutes). It operates by destroying the metadata of the storage device without erasing every bit of the data itself. Attempts to restore data after running `erase_devices_metadata` may be successful but would certainly require relevant expertise and specialized tools.

Lastly, `erase_devices_express` combines some of the perks of both `erase_devices` and `erase_devices_metadata`. It attempts to utilize hardware-assisted data erasure features if available (currently only NVMe devices are supported). In case hardware-assisted data erasure is not available, it falls back to metadata erasure for the device (which is identical to `erase_devices_metadata`). It can be considered a time-optimized mode of storage cleaning, aiming to perform as thorough data erasure as it is possible within a short period of time. This clean step is particularly well suited for environments with hybrid NVMe-HDD storage configuration as it allows fast and secure erasure of data stored on NVMes combined with equally fast but more basic metadata-based erasure of data on commodity HDDs.

By default, Ironic will use `erase_devices_metadata` early in cleaning for reliability (ensuring a node cannot reboot into its old workload) and `erase_devices` later in cleaning to securely erase the drive; `erase_devices_express` is disabled.

Operators can use *deploy.erase_devices_priority* and *deploy.*

erase_devices_metadata_priority to change the priorities of the default device erase methods or disable them entirely by setting `0`. Other cleaning steps can have their priority modified via the *conductor.clean_step_priority_override* option. For example, the configuration snippet below disables `erase_devices_metadata` and `erase_devices` and instead performs an `erase_devices_express` erase step.

```
[deploy]
erase_devices_priority=0
erase_devices_metadata_priority=0

[conductor]
clean_step_priority_override=deploy.erase_devices_express:95
```

This ensures that `erase_devices` and `erase_devices_metadata` are disabled so that storage is not cleaned twice and then assigns a non-zero priority to `erase_devices_express`, hence enabling it. Any non-zero priority specified in the priority override will work; larger values will cause the disk erasure to run earlier in the cleaning process if multiple steps are enabled.

Other configurations that can modify how Ironic erases disks are below. This list may not be comprehensive. Please review ironic.conf.sample (linked) for more details:

- *deploy.enable_ata_secure_erase*, default `True`
- *deploy.enable_nvme_secure_erase*, default `True`
- *deploy.shred_random_overwrite_iterations*, default `1`
- *deploy.shred_final_overwrite_with_zeros*, default `True`
- *deploy.disk_erasure_concurrency*, default `4`

Warning

Ironic automated cleaning is defaulted to a secure configuration. You should not modify settings related to it unless you have special hardware needs or a unique use case. Misconfigurations can lead to data exposure vulnerabilities.

Configuring automated cleaning with runbooks

Starting with the 2025.2/Flamingo release, operators can configure Ironic to use runbooks for automated cleaning instead of relying on autogenerated steps. This provides more control over the cleaning process and ensures consistency across nodes.

Warning

When using runbooks for automated cleaning, ensure they include appropriate security measures such as disk erasure. Ironic does not validate that a runbook performs disk cleaning operations or any other specific cleaning step.

Trait matching

As always with runbooks, you must have a trait on the node which matches the runbook name. This allows a fail-safe to prevent dangerous, hardware-specific cleaning steps from running on incompatible

hardware.

You can disable this check by setting *conductor.automated_cleaning_runbook_validate_traits* to False.

```
openstack baremetal node add trait myNode CUSTOM_RB_EXAMPLE
```

Configure cleaning runbooks

Runbooks can be configured at three levels (from most to least specific):

1. **Per-node**:

   Operators can set a per-node cleaning runbook override using the following command:

   ```
   openstack baremetal node set myNode --driver-info cleaning_
   ↪runbook=CUSTOM_RB_EXAMPLE
   ```

   Warning

   Customizing cleaning per node requires setting *conductor.automated_cleaning_runbook_from_node* to True.

   Enabling node-level runbooks allows node owners to override cleaning behavior via use a noop runbook. Only enable this in trusted environments.

2. **Per-resource-class**:

   Operators can set a runbook per resource_class using *conductor.automated_cleaning_runbook_by_resource_class* to build a list of mappings of resource_class to runbook. These runbooks are used to clean any node in that resource class that do not have a node-level override.

   In this example, the large resource_class uses `CUSTOM_FULL_CLEAN` and the small resource_class uses `CUSTOM_QUICK_CLEAN`. Nodes in those resource classes would still be required to have traits matching the runbook name.

   ```
   [conductor]
   automated_cleaning_runbook_by_resource_class = large:CUSTOM_FULL_CLEAN,
   ↪small:CUSTOM_QUICK_CLEAN
   ```

3. **Global default**: Operators can also configure a global default, which is used for nodes which do not already have a more specific runbook configured, such as node-level overrides or a resource_class mapping.

   In this example, any node cleaned in the environment would use `CUSTOM_DEFAULT_CLEAN`. Unless trait mapping is disabled, all nodes would be required to have a trait also named `CUSTOM_DEFAULT_CLEAN` to successfully clean.

   ```
   [conductor]
   automated_cleaning_runbook = CUSTOM_DEFAULT_CLEAN
   ```

Create and assign runbooks

Create a runbook with the necessary cleaning steps:

```
baremetal runbook create --name CUSTOM_SECURE_ERASE \
  --steps '[{"interface": "deploy", "step": "erase_devices", "args": {},
↪"order": 0}]'
```

Ensure nodes have the matching trait:

```
baremetal node add trait <node> CUSTOM_SECURE_ERASE
```

Management Interface

Table 7: idrac-redfish cleaning steps

Name	Details	Priority	Stoppable	Arguments
clear_job_queue	Clear iDRAC job queue.	0	no	
clear_secure_bc	Clear all secure boot keys.	0	no	
export_configur	(Deprecated) Export the configuration of the server. Exports the configuration of the server against which the step is run and stores it in specific format in indicated location. Uses Dells Server Configuration Profile (SCP) from *sushy* oem extension to get ALL configuration for cloning.	0	no	export_configuration_location (*required*) URL of location to save the configuration to.
import_configur	(Deprecated) Import and apply the configuration to the server. Gets pre-created configuration from storage by given location and imports that into given server. Uses Dells Server Configuration Profile (SCP).	0	no	import_configuration_location (*required*) URL of location to fetch desired configuration from.
import_export_c	Import and export configuration in one go. Gets pre-created configuration from storage by given name and imports that into given server. After that exports the configuration of the server against which the step is run and stores it in specific format in indicated storage as configured by Ironic.	0	no	export_configuration_location (*required*) URL of location to save the configuration to. import_configuration_location (*required*) URL of location to fetch desired configuration from.
known_good_stat	Reset iDRAC to known good state. An iDRAC is reset to a known good state by resetting it and clearing its job queue.	0	no	
reset_idrac	Reset the iDRAC.	0	no	
reset_secure_bc	Reset secure boot keys to manufacturing defaults.	0	no	
set_bmc_clock	Set the BMC clock using Redfish Manager resource.	0	no	datetime_local_offset The local time offset from UTC target_datetime The datetime to set in ISO8601 format. If omitted, the current conductor UTC time

Table 8: ilo cleaning steps

Name	Details	Priority	Stoppable	Arguments
activate_licens	Activates iLO Advanced license.	0	no	ilo_license_key (*required*) The HPE iLO Advanced license key to activate enterprise features.
add_https_certi	Adds the signed HTTPS certificate to the iLO.	0	no	cert_file (*required*) This argument represents the path to the signed HTTPS certificate which will be added to the iLO.
clear_secure_bc	Clear all secure boot keys. Clears all the secure boot keys. This operation is supported only on HP Proliant Gen9 and above servers.	0	no	
create_csr	Creates the CSR.	0	no	csr_params (*required*) This arguments represents the information needed to create the CSR certificate. The keys to be provided are City, CommonName, OrgName, State.
reset_bios_to_d	Resets the BIOS settings to default values. Resets BIOS to default settings. This operation is currently supported only on HP Proliant Gen9 and above servers.	10	no	
reset_ilo	Resets the iLO.	0	no	
reset_ilo_crede	Resets the iLO password.	30	no	
reset_secure_bc	Reset secure boot keys to manufacturing defaults. Resets the secure boot keys to manufacturing defaults. This operation is supported only on HP Proliant Gen9 and above servers.	20	no	
security_parame	Updates the security parameters.	0	no	security_parameters (*required*) This argument represents the ordered list of JSON dictionaries of security parameters. Each security parameter consists of three fields, namely param, ignore and enable from which param field will be mandatory. These fields represent security parameter name, ignore flag and state of the security parameter. The supported security parameter names

Table 9: ilo5 cleaning steps

Name	Details	Priority	Stoppable	Arguments
activate_licens	Activates iLO Advanced license.	0	no	ilo_license_key (*required*) The HPE iLO Advanced license key to activate enterprise features.
add_https_certi	Adds the signed HTTPS certificate to the iLO.	0	no	cert_file (*required*) This argument represents the path to the signed HTTPS certificate which will be added to the iLO.
clear_ca_certif	Clears the certificates provided in the list of files to iLO.	0	no	certificate_files (*required*) The list of files containing the certificates to be cleared. If empty list is specified, all the certificates on the ilo will be cleared, except the certificates in the file configured with configuration parameter webserver_verify_ca are spared as they are required for booting the deploy image for some boot interfaces.
clear_secure_bc	Clear all secure boot keys. Clears all the secure boot keys. This operation is supported only on HP Proliant Gen9 and above servers.	0	no	
create_csr	Creates the CSR.	0	no	csr_params (*required*) This arguments represents the information needed to create the CSR certificate. The keys to be provided are City, CommonName, OrgName, State.
erase_devices	Erase all the drives on the node. This method performs out-of-band sanitize disk erase on all the supported physical drives in the node. This erase cannot be performed on logical drives.	0	no	erase_pattern Dictionary of disk type and corresponding erase pattern to be used to perform specific out-of-band sanitize disk erase. Supported values are, for hdd: (overwrite, crypto, zero), for ssd: (block, crypto, zero). Default pattern is: {hdd: overwrite, ssd: block}.
one_button_secu	Erase the whole system securely. The One-button secure erase process resets iLO and deletes all licenses stored there, resets BIOS settings, and deletes all Active Health	0	no	

Table 10: redfish cleaning steps

Name	Details	Priority	Stoppable	Arguments
`clear_secure_bc`	Clear all secure boot keys.	0	no	
`reset_secure_bc`	Reset secure boot keys to manufacturing defaults.	0	no	
`set_bmc_clock`	Set the BMC clock using Redfish Manager resource.	0	no	`datetime_local_offset` The local time offset from UTC `target_datetime` The datetime to set in ISO8601 format. If omitted, the current conductor UTC time is used.
`update_firmware`	Updates the firmware on the node.	0	no	`firmware_images` (*required*) A list of firmware images to apply.

Firmware Interface

Table 11: redfish cleaning steps

Name	Details	Priority	Stoppable	Arguments
`update`	Update the Firmware on the node using the settings for components.	0	no	`settings` (*required*) A list of dicts with firmware components to be updated

Bios Interface

Table 12: idrac-redfish cleaning steps

Name	Details	Priority	Stoppable	Arguments
`apply_configura`	Apply the BIOS settings to the node.	0	no	`settings` (*required*) A list of BIOS settings to be applied
`factory_reset`	Reset the BIOS settings of the node to the factory default.	0	no	

Table 13: ilo cleaning steps

Name	Details	Priority	Stoppable	Arguments
`apply_configura`	Applies the provided configuration on the node.	0	no	`settings` (*required*) Dictionary with current BIOS configuration.
`factory_reset`	Reset the BIOS settings to factory configuration.	0	no	

Table 14: redfish cleaning steps

Name	Details	Priority	Stoppable	Arguments
`apply_configura`	Apply the BIOS settings to the node.	0	no	`settings` (*required*) A list of BIOS settings to be applied
`factory_reset`	Reset the BIOS settings of the node to the factory default.	0	no	

Raid Interface

Table 15: agent cleaning steps

Name	Details	Priority	Stoppable	Arguments
`create_configur`	Create a RAID configuration on a bare metal using agent ramdisk. This method creates a RAID configuration on the given node.	0	no	
`delete_configur`	Deletes RAID configuration on the given node.	0	no	

Table 16: idrac-redfish cleaning steps

Name	Details	Priority	Stoppable	Arguments
`create_configur`	Create RAID configuration on the node. This method creates the RAID configuration as read from node.target_raid_config. This method by default will create all logical disks.	0	no	`create_nonroot_volumes` This specifies whether to create the non-root volumes. Defaults to *True*. `create_root_volume` This specifies whether to create the root volume. Defaults to *True*. `delete_existing` Setting this to *True* indicates to delete existing RAID configuration prior to creating the new configuration. Default value is *False*.
`delete_configur`	Delete RAID configuration on the node.	0	no	

Table 17: ilo5 cleaning steps

Name	Details	Priority	Stoppable	Arguments
`create_configur`	Create a RAID configuration on a bare metal using agent ramdisk. This method creates a RAID configuration on the given node.	0	no	`create_nonroot_volumes` This specifies whether to create the non-root volumes. Defaults to *True*. `create_root_volume` This specifies whether to create the root volume. Defaults to *True*.
`delete_configur`	Delete the RAID configuration.	0	no	

Table 18: redfish cleaning steps

Name	Details	Priority	Stoppable	Arguments
`create_configur`	Create RAID configuration on the node. This method creates the RAID configuration as read from node.target_raid_config. This method by default will create all logical disks.	0	no	`create_nonroot_volumes` This specifies whether to create the non-root volumes. Defaults to *True*. `create_root_volume` This specifies whether to create the root volume. Defaults to *True*. `delete_existing` Setting this to *True* indicates to delete existing RAID configuration prior to creating the new configuration. Default value is *False*.
`delete_configur`	Delete RAID configuration on the node.	0	no	

Manual cleaning

`Manual cleaning` is typically used to handle long-running, manual, or destructive tasks that an operator wishes to perform either before the first workload has been assigned to a node or between workloads. When initiating a manual clean, the operator specifies the cleaning steps to be performed. Manual cleaning can only be performed when a node is in the `manageable` state. Once the manual cleaning is finished, the node will be put in the `manageable` state again.

Ironic added support for manual cleaning in the 4.4 (Mitaka series) release.

Setup

In order for manual cleaning to work, you may need to configure a *Cleaning Network*.

Starting manual cleaning via API

Manual cleaning can only be performed when a node is in the `manageable` state. The REST API request to initiate it is available in API version 1.15 and higher:

```
PUT /v1/nodes/<node_ident>/states/provision
```

(Additional information is available here.)

This API will allow operators to put a node directly into `cleaning` provision state from `manageable` state via target: clean. The PUT will also require the argument clean_steps to be specified. This is an ordered list of cleaning steps. A cleaning step is represented by a dictionary (JSON), in the form:

```
{
    "interface": "<interface>",
    "step": "<name of cleaning step>",
```

(continues on next page)

(continued from previous page)

```
    "args": {"<arg1>": "<value1>", ..., "<argn>": "<valuen>"}
}
```

The interface and step keys are required for all steps. If a cleaning step method takes keyword arguments, the args key may be specified. It is a dictionary of keyword variable arguments, with each keyword-argument entry being `<name>: <value>`.

If any step is missing a required keyword argument, manual cleaning will not be performed and the node will be put in `clean failed` provision state with an appropriate error message.

If, during the cleaning process, a cleaning step determines that it has incorrect keyword arguments, all earlier steps will be performed and then the node will be put in `clean failed` provision state with an appropriate error message.

An example of the request body for this API:

```
{
  "target":"clean",
  "clean_steps": [{
    "interface": "raid",
    "step": "create_configuration",
    "args": {"create_nonroot_volumes": false}
  },
  {
    "interface": "deploy",
    "step": "erase_devices"
  }]
}
```

In the above example, the nodes RAID interface would configure hardware RAID without non-root volumes, and then all devices would be erased (in that order).

An example is setting the BMC clock using the Redfish management interface:

```
{
  "target": "clean",
  "clean_steps": [{
    "interface": "management",
    "step": "set_bmc_clock",
    "args": {"target_datetime": "2025-07-22T12:34:56+00:00"}
  }]
}
```

If `target_datetime` is omitted, Ironic uses the current UTC time on the conductor when the step executes. This makes the step suitable for automated cleaning runbooks without hardcoding a timestamp:

```
{
  "target": "clean",
  "clean_steps": [{
    "interface": "management",
    "step": "set_bmc_clock"
```

(continues on next page)

(continued from previous page)

```
  }]
}
```

This step requires the node to use the `redfish` management interface and that the Redfish service exposes the `DateTime` and `DateTimeLocalOffset` fields under the Manager Resource.

Alternatively, you can specify a runbook instead of clean_steps:

```
{
  "target":"clean",
  "runbook": "<runbook_name_or_uuid>"
}
```

The specified runbook must match one of the nodes traits to be used.

Starting manual cleaning via openstack baremetal CLI

Manual cleaning is available via the `baremetal node clean` command, starting with Bare Metal API version 1.15.

The argument `--clean-steps` must be specified. Its value is one of:

- a JSON string
- path to a JSON file whose contents are passed to the API
- `-` to read from stdin. This allows piping in the clean steps. Using - to signify stdin is common in Unix utilities.

The following examples assume that the Bare Metal API version was set via the `OS_BAREMETAL_API_VERSION` environment variable. (The alternative is to add `--os-baremetal-api-version 1.15` to the command.):

```
export OS_BAREMETAL_API_VERSION=1.15
```

Examples of doing this with a JSON string:

```
baremetal node clean <node> \
    --clean-steps '[{"interface": "deploy", "step": "erase_devices_metadata"}]
↪'

baremetal node clean <node> \
    --clean-steps '[{"interface": "deploy", "step": "erase_devices"}]'
```

Or with a file:

```
baremetal node clean <node> \
    --clean-steps my-clean-steps.txt
```

Or with stdin:

```
cat my-clean-steps.txt | baremetal node clean <node> \
    --clean-steps -
```

Runbooks for Manual Cleaning

Instead of passing a list of clean steps, operators can now use runbooks. Runbooks are curated lists of steps that can be associated with nodes via traits which simplifies the process of performing consistent cleaning operations across similar nodes.

To use a runbook for manual cleaning:

```
baremetal node clean <node> --runbook <runbook_name_or_uuid>
```

Runbooks must be created and associated with nodes beforehand. Only runbooks that match the nodes traits can be used for cleaning that node. For more information on the runbook API usage, see *Runbooks for Cleaning & Servicing*.

Cleaning Network

If you are using the Neutron DHCP provider (the default) you will also need to ensure you have configured a cleaning network. This network will be used to boot the ramdisk for in-band cleaning. You can use the same network as your tenant network. For steps to set up the cleaning network, please see *Configure the Bare Metal service for cleaning*.

In-band vs out-of-band

Ironic uses two main methods to perform actions on a node: in-band and out-of-band. Ironic supports using both methods to clean a node.

In-band

In-band steps are performed by Ironic making API calls to a ramdisk running on the node using a deploy interface. Currently, all the deploy interfaces support in-band cleaning. By default, ironic-python-agent ships with a minimal cleaning configuration, only erasing disks. However, you can add your own cleaning steps and/or override default cleaning steps with a custom Hardware Manager.

Out-of-band

Out-of-band are actions performed by your management controller, such as IPMI, iLO, or DRAC. Out-of-band steps will be performed by Ironic using a power or management interface. Which steps are performed depends on the hardware type and hardware itself.

For Out-of-Band cleaning operations supported by iLO hardware types, refer to *Node Cleaning Support*.

FAQ

How are cleaning steps ordered?

For automated cleaning, cleaning steps are ordered by integer priority, where a larger integer is a higher priority. In case of a conflict between priorities across hardware interfaces, the following resolution order is used:

1. Power interface
2. Management interface
3. Deploy interface
4. BIOS interface

5. RAID interface

For manual cleaning, the cleaning steps should be specified in the desired order.

How do I skip a cleaning step?

For automated cleaning, cleaning steps with a priority of zero or `None` are skipped.

How do I change the priority of a cleaning step?

For manual cleaning, or runbook-based cleaning, specify the cleaning steps in the desired order.

For automated cleaning, it depends on whether the cleaning steps are out-of-band or in-band.

Most out-of-band cleaning steps have an explicit configuration option for priority.

Changing the priority of an in-band (ironic-python-agent) cleaning step requires use of *conductor.clean_step_priority_override*, a configuration option that allows specifying the priority of each step using multiple configuration values:

```
[conductor]
clean_step_priority_override=deploy.erase_devices_metadata:123
clean_step_priority_override=management.reset_bios_to_default:234
clean_step_priority_override=management.clean_priority_reset_ilo:345
```

This parameter can be specified as many times as required to define priorities for several cleaning steps - the values will be combined.

What cleaning step is running?

To check what cleaning step the node is performing or attempted to perform and failed, run the following command; it will return the value in the nodes `driver_internal_info` field:

```
baremetal node show $node_ident -f value -c driver_internal_info
```

The `clean_steps` field will contain a list of all remaining steps with their priorities, and the first one listed is the step currently in progress or that the node failed before going into `clean failed` state.

Should I disable automated cleaning?

Automated cleaning is recommended for Ironic deployments, however, there are some tradeoffs to having it enabled. For instance, Ironic cannot deploy a new instance to a node that is currently cleaning, and cleaning can be a time consuming process. To mitigate this, we suggest using NVMe drives with support for NVMe Secure Erase (based on `nvme-cli` format command) or ATA drives with support for cryptographic ATA Security Erase, as typically the erase_devices step in the deploy interface takes the longest time to complete of all cleaning steps.

Why cant I power on/off a node while its cleaning?

During cleaning, nodes may be performing actions that shouldnt be interrupted, such as BIOS or Firmware updates. As a result, operators are forbidden from changing the power state via the Ironic API while a node is cleaning.

Advanced topics

Parent Nodes

The concept of a `parent_node` is where a node is configured to have a parent, and allows for actions upon the parent, to in some cases take into account child nodes. Mainly, the concept of executing clean steps in relation to child nodes.

In this context, a child node is primarily intended to be an embedded device with its own management controller. For example SmartNICs or Data Processing Units (DPUs) which may have their own management controller and power control.

The relationship between a parent node and a child node is established on the child node. Example:

```
baremetal node set --parent-node <parent_node_uuid> <child_node_uuid>
```

Child Node Clean Step Execution

You can execute steps that perform actions on child nodes. For example, turn them on (via step `power_on`), off (via step `power_off`), or to signal a BMC-controlled reboot (via step `reboot`).

For example, if you need to explicitly power off child node power, before performing another step, you can articulate it with a step such as:

```
[{
  "interface": "power",
  "step": "power_off",
  "execute_on_child_nodes": True,
  "limit_child_node_execution": ['f96c8601-0a62-4e99-97d6-1e0d8daf6dce']
},
{
  "interface": "deploy",
  "step": "erase_devices"
}]
```

As one would imagine, this step will power off a singular child node, as a limit has been expressed to a singular known node, and that child nodes power will be turned off via the management interface. Afterwards, the `erase_devices` step will be executed on the parent node.

Note

While the deployment step framework also supports the `execute_on_child_nodes` and `limit_child_node_execution` parameters, all of the step frameworks have a fundamental limitation in that child node step execution is intended for synchronous actions which do not rely upon the `ironic-python-agent` running on any child nodes. This constraint may be changed in the future.

Power Management with Child Nodes

The mix of child nodes and parent nodes has special power considerations, and these devices are evolving in the industry. That being said, the Ironic project has taken an approach of explicitly attempting to power on any parent node when a request comes in to power on a child node. This can be bypassed by setting a `driver_info` parameter `has_dedicated_power_supply` set to `True`, in recognition that

some hardware vendors are working on supplying independent power to these classes of devices to meet their customer use cases.

Similarly to the case of a power on request for a child node, when power is requested to be turned off for a parent node, Ironic will issue power off commands for all child nodes unless the child node has the `has_dedicated_power_supply` option set in the nodes `driver_info` field.

Troubleshooting

If cleaning fails on a node, the node will be put into `clean failed` state. If the failure happens while running a clean step, the node is also placed in maintenance mode to prevent Ironic from taking actions on the node. The operator should validate that no permanent damage has been done to the node and that no processes are still running on it before removing the maintenance mode.

> **Note**
>
> Older versions of Ironic may put the node to maintenance even when no clean step has been running.

Nodes in `clean failed` will not be powered off, as the node might be in a state such that powering it off could damage the node or remove useful information about the nature of the cleaning failure.

A `clean failed` node can be moved to `manageable` state, where it cannot be scheduled by Nova and you can safely attempt to fix the node. To move a node from `clean failed` to `manageable`:

```
baremetal node manage $node_ident
```

You can now take actions on the node, such as replacing a bad disk drive.

Strategies for determining why a cleaning step failed include checking the Ironic conductor logs, viewing logs on the still-running ironic-python-agent (if an in-band step failed), or performing general hardware troubleshooting on the node.

When the node is repaired, you can move the node back to `available` state, to allow it to be scheduled by Nova.

```
# First, move it out of maintenance mode
baremetal node maintenance unset $node_ident

# Now, make the node available for scheduling by Nova
baremetal node provide $node_ident
```

The node will begin automated cleaning from the start, and move to `available` state when complete.

4.2.4 Node adoption

Overview

As part of hardware inventory lifecycle management, it is not an unreasonable need to have the capability to be able to add hardware that should be considered in-use by the Bare Metal service, that may have been deployed by another Bare Metal service installation or deployed via other means.

As such, the node adoption feature allows a user to define a node as `active` while skipping the `available` and `deploying` states, which will prevent the node from being seen by the Compute service as ready for use.

This feature is leveraged as part of the state machine workflow, where a node in `manageable` can be moved to an `active` state via the provision_state verb `adopt`. To view the state transition capabilities, please see *Bare Metal State Machine*.

Note

For deployments using Ironic in conjunction with Nova, Ironics node adoption feature is not suitable. If you need to adopt production nodes into Ironic **and** Nova, you can find a high-level recipe in *Adoption with Nova*.

How it works

A node initially enrolled begins in the `enroll` state. An operator must then move the node to `manageable` state, which causes the nodes `power` interface to be validated. Once in `manageable` state, an operator can then explicitly choose to adopt a node.

Adoption of a node results in the validation of its `boot` interface, and upon success the process leverages what is referred to as the takeover logic. The takeover process is intended for conductors to take over the management of nodes for a conductor that has failed.

The takeover process involves the deploy interfaces `prepare` and `take_over` methods being called. These steps take specific actions such as downloading and staging the deployment kernel and ramdisk, ISO image, any required boot image, or boot ISO image and then places any PXE or virtual media configuration necessary for the node should it be required.

The adoption process makes no changes to the physical node, with the exception of operator-supplied configurations where virtual media is used to boot the node under normal circumstances. An operator should ensure that any supplied configuration defining the node is sufficient for the continued operation of the node moving forward.

Possible Risk

The main risk with this feature is that the supplied configuration may ultimately be incorrect or invalid which could result in potential operational issues:

- `rebuild` verb - Rebuild is intended to allow a user to re-deploy the node to a fresh state. The risk with adoption is that the image defined when an operator adopts the node may not be the valid image for the pre-existing configuration.

  If this feature is utilized for a migration from one deployment to another, and pristine original images are loaded and provided, then ultimately the risk is the same with any normal use of the `rebuild` feature, the server is effectively wiped.

- When deleting a node, the deletion or cleaning processes may fail if the incorrect deployment image is supplied in the configuration as the node may NOT have been deployed with the supplied image and driver or compatibility issues may exist as a result.

  Operators will need to be cognizant of that possibility and should plan accordingly to ensure that deployment images are known to be compatible with the hardware in their environment.

- Networking - Adoption will assert no new networking configuration to the newly adopted node as that would be considered modifying the node.

  Operators will need to plan accordingly and have network configuration such that the nodes will be able to network boot.

How to use

Note

The power state that the ironic-conductor observes upon the first successful power state check, as part of the transition to the `manageable` state will be enforced with a node that has been adopted. This means a node that is in `power off` state will, by default, have the power state enforced as `power off` moving forward, unless an administrator actively changes the power state using the Bare Metal service.

Requirements

Requirements for use are essentially the same as to deploy a node:

- Sufficient driver information to allow for a successful power management validation.
- Sufficient instance_info to pass deploy interface preparation.

Each driver may have additional requirements dependent upon the configuration that is supplied. An example of this would be defining a node to always boot from the network, which will cause the conductor to attempt to retrieve the pertinent files. Inability to do so will result in the adoption failing, and the node being placed in the `adopt failed` state.

Example

This is an example to create a new node, named `testnode`, with sufficient information to pass basic validation in order to be taken from the `manageable` state to `active` state:

```
# Explicitly set the client API version environment variable to
# 1.17, which introduces the adoption capability.
export OS_BAREMETAL_API_VERSION=1.17

baremetal node create --name testnode \
    --driver ipmi \
    --driver-info ipmi_address=<ip_address> \
    --driver-info ipmi_username=<username> \
    --driver-info ipmi_password=<password> \
    --driver-info deploy_kernel=<deploy_kernel_id_or_url> \
    --driver-info deploy_ramdisk=<deploy_ramdisk_id_or_url>

baremetal port create <node_mac_address> --node <node_uuid>

baremetal node set testnode \
    --instance-info image_source="http://localhost:8080/blankimage"

baremetal node manage testnode --wait

baremetal node adopt testnode --wait
```

Note

In the above example, the image_source setting must reference a valid image or file, however, that image or file can ultimately be empty.

Note

The above example utilizes a capability that defines the boot operation to be local. It is recommended to define the node as such unless network booting is desired.

Note

The above example will fail a re-deployment as a fake image is defined and no instance_info/image_checksum value is defined. As such any actual attempt to write the image out will fail as the image_checksum value is only validated at the time of an actual deployment operation.

Note

A user may wish to assign an instance_uuid to a node, which could be used to match an instance in the Compute service. Doing so is not required for the proper operation of the Bare Metal service.

baremetal node set <node name or uuid> instance-uuid <uuid>

Note

A user of this feature may wish to add new nodes with a `network_interface` value of `noop` and then change the interface at a later point and time.

Troubleshooting

Should an adoption operation fail for a node, the error that caused the failure will be logged in the nodes `last_error` field when viewing the node. This error, in the case of node adoption, will largely be due to the failure of a validation step. Validation steps are dependent upon what driver is selected for the node.

Any node that is in the `adopt failed` state can have the `adopt` verb re-attempted. Example:

```
baremetal node adopt <node name or uuid>
```

If a user wishes to cancel their attempt at adopting, they can then move the node back to `manageable` from `adopt failed` state by issuing the `manage` verb. Example:

```
baremetal node manage <node name or uuid>
```

If all else fails the hardware node can be removed from the Bare Metal service. The `node delete` command, which is **not** the same as setting the provision state to `deleted`, can be used while the node is in `adopt failed` state. This will delete the node without cleaning occurring to preserve the nodes current state. Example:

```
baremetal node delete <node name or uuid>
```

Adoption with Nova

Since there is no mechanism to create bare metal instances in Nova when nodes are adopted into Ironic, the node adoption feature described above cannot be used to add in production nodes to deployments that use Ironic together with Nova.

One option to add production nodes to an Ironic/Nova deployment is to use the fake drivers. The overall idea is that for Nova the nodes are instantiated normally to ensure the instances are properly created in the compute project while Ironic does not touch them.

Here are some high-level steps to be used as a guideline:

- create a bare metal flavor and a hosting project for the instances
- enroll the nodes into Ironic, create the ports, and move them to manageable
- change the hardware type and the interfaces to fake drivers
- provide the nodes to make them available
- one by one, add the nodes to the placement aggregate and create instances
- change the hardware type and the interfaces back to the real ones

Make sure you change the drivers to the fake ones **before** providing the nodes as cleaning will otherwise wipe your production servers!

The reason to make all nodes available and manage access via the aggregate is that this is much faster than providing nodes one by one and relying on the resource tracker to find them. Enabling them one by one is required to make sure the instance name and the (pre-adoption) name of the server match.

The above recipe does not cover Neutron which, depending on your deployment, may need to be handled in addition.

4.2.5 Retiring a node in Ironic

Overview

Retiring nodes is a natural part of a servers life cycle, for instance when the end of the warranty is reached and the physical space is needed for new deliveries to install replacement capacity.

However, depending on the type of the deployment, removing nodes from service can be a full workflow by itself as it may include steps like moving applications to other hosts, cleaning sensitive data from disks or the BMC, or tracking the dismantling of servers from their racks.

Ironic provides some means to support such workflows by allowing to tag nodes as `retired` which will prevent any further scheduling of instances, but will still allow for other operations, such as cleaning, to happen (this marks an important difference to nodes which have the `maintenance` flag set).

Requirements

The use of the retirement feature requires that automated cleaning be enabled. The default *conductor.automated_clean* setting must not be disabled as the retirement feature is only engaged upon the completion of cleaning as it sets forth the expectation of removing sensitive data from a node.

If youre uncomfortable with full cleaning, but want to make use of the the retirement feature, a compromise may be to explore use of metadata erasure, however this will leave additional data on

disk which you may wish to erase completely. Please consult the configuration for the *deploy.erase_devices_metadata_priority* and *deploy.erase_devices_priority* settings, and do note that clean steps can be manually invoked through manual cleaning should you wish to trigger the `erase_devices` clean step to completely wipe all data from storage devices. Alternatively, automated cleaning can also be enabled on an individual node level using the `baremetal node set --automated-clean` <node_id> command.

How to use

When it is known that a node shall be retired, set the `retired` flag on the node with:

```
baremetal node set --retired node-001
```

This can be done irrespective of the state the node is in, so in particular while the node is `active`.

Note

An exception are nodes which are in `available`. For backwards compatibility reasons, these nodes need to be moved to `manageable` first. Trying to set the `retired` flag for `available` nodes will result in an error.

Optionally, a reason can be specified when a node is retired, e.g.:

```
baremetal node set --retired node-001 \
  --retired-reason "End of warranty for delivery abc123"
```

Upon instance deletion, an `active` node with the `retired` flag set will not move to `available`, but to `manageable`. The node will hence not be eligible for scheduling of new instances.

Equally, nodes with `retired` set to True cannot move from `manageable` to `available`: the `provide` verb is blocked. This is to prevent accidental reuse of nodes tagged for removal from the fleet. In order to move these nodes to `available` none the less, the `retired` field needs to be removed first. This can be done via:

```
baremetal node unset --retired node-001
```

In order to facilitate the identification of nodes marked for retirement, e.g. by other teams, ironic also allows to list all nodes which have the `retired` flag set:

```
baremetal node list --retired
```

4.2.6 RAID Configuration

Overview

Ironic supports RAID configuration for bare metal nodes. It allows operators to specify the desired RAID configuration via the OpenStackClient CLI or REST API. The desired RAID configuration is applied on the bare metal during manual cleaning.

The examples described here use the OpenStackClient CLI; please see the REST API reference for their corresponding REST API requests.

Prerequisites

The bare metal node needs to use a hardware type that supports RAID configuration. RAID interfaces may implement RAID configuration either in-band or out-of-band. Software RAID is supported on all hardware, although with some caveats - see *Software RAID* for details.

In-band RAID configuration (including software RAID) is done using the Ironic Python Agent ramdisk. For in-band hardware RAID configuration, a hardware manager which supports RAID should be bundled with the ramdisk.

Whether a node supports RAID configuration could be found using the CLI command `baremetal node validate <node>`. In-band RAID is usually implemented by the `agent` RAID interface.

Build agent ramdisk which supports RAID configuration

For doing in-band hardware RAID configuration, Ironic needs an agent ramdisk bundled with a hardware manager which supports RAID configuration for your hardware. For example, the *DIB support for Proliant Hardware Manager* should be used for HPE Proliant Servers.

Note

For in-band software RAID, the agent ramdisk does not need to be bundled with a hardware manager as the generic hardware manager in the Ironic Python Agent already provides (basic) support for software RAID.

RAID configuration JSON format

The desired RAID configuration and current RAID configuration are represented in JSON format.

Target RAID configuration

This is the desired RAID configuration on the bare metal node. Using the OpenStackClient CLI (or REST API), the operator sets `target_raid_config` field of the node. The target RAID configuration will be applied during manual cleaning.

Target RAID configuration is a dictionary having `logical_disks` as the key. The value for the `logical_disks` is a list of JSON dictionaries. It looks like:

```
{
  "logical_disks": [
    {<desired properties of logical disk 1>},
    {<desired properties of logical disk 2>},
    ...
  ]
}
```

If the `target_raid_config` is an empty dictionary, it unsets the value of `target_raid_config` if the value was set with previous RAID configuration done on the node.

Each dictionary of logical disk contains the desired properties of logical disk supported by the hardware type. These properties are discoverable by:

```
baremetal driver raid property list <driver name>
```

Mandatory properties

These properties must be specified for each logical disk and have no default values:

- `size_gb` - Size (Integer) of the logical disk to be created in GiB. `MAX` may be specified if the logical disk should use all of the remaining space available. This can be used only when backing physical disks are specified (see below).
- `raid_level` - RAID level for the logical disk. Ironic supports the following RAID levels: 0, 1, 2, 5, 6, 1+0, 5+0, 6+0.

Optional properties

These properties have default values, and they may be overridden in the specification of any logical disk.

- `volume_name` - Name of the volume. Should be unique within the Node. If not specified, the volume name will be auto-generated.
- `is_root_volume` - Set to `true` if this is the root volume. At most one logical disk can have this set to `true`; the other logical disks must have this set to `false` or left unspecified.

  For hardware RAID, this value currently only affects the `create_root_volume` and `create_nonroot_volumes` parameters to the `create_configuration` step, but out-of-tree RAID interfaces may also use it to populate *root device hints*. Otherwise, you need to populate the hints yourself.

  For software RAID (since the Gazpacho release - through the change 962670), setting the parameter to `false`, prevents it from being picked up as the root volume. If both `root device hint`, and the `is_root_volume` (set to `true`) parameters are set, the `is_root_volume` parameter takes priority, and the volume should be picked up for the root device.

Backing physical disk hints

These hints are specified for each logical disk to let Ironic find the desired disks for RAID configuration. This is machine-independent information. This serves the use-case where the operator doesnt want to provide individual details for each bare metal node. None of these options are supported for software RAID.

- `share_physical_disks` - Set to `true` if this logical disk can share physical disks with other logical disks. The default value is `false`, except for software RAID which always shares disks.
- `disk_type` - `hdd` or `ssd`. If this is not specified, disk type will not be a criterion to find backing physical disks.
- `interface_type` - `sata` or `scsi` or `sas`. If this is not specified, interface type will not be a criterion to find backing physical disks.
- `number_of_physical_disks` - Integer, number of disks to use for the logical disk. Defaults to minimum number of disks required for the particular RAID level, except for software RAID which always spans all disks.

Backing physical disks

These are the actual machine-dependent information. This is suitable for environments where the operator wants to automate the selection of physical disks with a 3rd-party tool based on a wider range of attributes (eg. S.M.A.R.T. status, physical location). The values for these properties are hardware dependent.

- `controller` - The name of the controller as read by the RAID interface. In order to trigger the setup of a Software RAID via the Ironic Python Agent, the value of this property needs to be set to `software`.
- `physical_disks` - A list of physical disks to use as read by the RAID interface.

  For software RAID `physical_disks` is a list of device hints in the same format as used for *Specifying the disk for deployment (root device hints)*. The number of provided hints must match the expected number of backing devices (repeat the same hint if necessary).

Note

If properties from both Backing physical disk hints or Backing physical disks are specified, they should be consistent with each other. If they are not consistent, then the RAID configuration will fail (because the appropriate backing physical disks could not be found).

Examples for `target_raid_config`

Example 1. Single RAID disk of RAID level 5 with all of the space available. Make this the root volume to which Ironic deploys the image:

```
{
  "logical_disks": [
    {
      "size_gb": "MAX",
      "raid_level": "5",
      "is_root_volume": true
    }
  ]
}
```

Example 2. Two RAID disks. One with RAID level 5 of 100 GiB and use SSD. Another with RAID level 1 of 500 GiB and use HDD:

```
{
  "logical_disks": [
    {
      "size_gb": 100,
      "raid_level": "5",
      "disk_type": "ssd"
    },
    {
      "size_gb": 500,
      "raid_level": "1",
      "disk_type": "hdd"
    }
  ]
}
```

Example 3. Single RAID disk. I know which disks and controller to use:

```
{
  "logical_disks": [
    {
      "size_gb": 100,
      "raid_level": "5",
      "controller": "Smart Array P822 in Slot 3",
      "physical_disks": ["6I:1:5", "6I:1:6", "6I:1:7"]
    }
  ]
}
```

Example 4. Using backing physical disks:

```
{
  "logical_disks": [
    {
      "size_gb": 50,
      "raid_level": "1+0",
      "controller": "RAID.Integrated.1-1",
      "volume_name": "root_volume",
      "physical_disks": [
        "Disk.Bay.0:Encl.Int.0-1:RAID.Integrated.1-1",
        "Disk.Bay.1:Encl.Int.0-1:RAID.Integrated.1-1"
      ]
    },
    {
      "size_gb": 100,
      "raid_level": "5",
      "controller": "RAID.Integrated.1-1",
      "volume_name": "data_volume",
      "physical_disks": [
        "Disk.Bay.2:Encl.Int.0-1:RAID.Integrated.1-1",
        "Disk.Bay.3:Encl.Int.0-1:RAID.Integrated.1-1",
        "Disk.Bay.4:Encl.Int.0-1:RAID.Integrated.1-1"
      ]
    }
  ]
}
```

Example 5. Software RAID with two RAID devices. The smaller one should be picked up as a root device:

```
{
  "logical_disks": [
    {
      "size_gb": 100,
      "raid_level": "1",
      "is_root_volume": true,
      "controller": "software"
    },
```

(continues on next page)

(continued from previous page)

```
    {
      "size_gb": "MAX",
      "raid_level": "0",
      "controller": "software"
    }
  ]
}
```

Example 6. Software RAID, limiting backing block devices to exactly two devices with the size exceeding 100 GiB:

```
{
  "logical_disks": [
    {
      "size_gb": "MAX",
      "raid_level": "0",
      "controller": "software",
      "physical_disks": [
        {"size": "> 100"},
        {"size": "> 100"}
      ]
    }
  ]
}
```

Example 7. Single RAID disk of RAID level 1 which spans two disks. The `is_root_volume` set to false prevents the array from being picked up as the root volume. Therefore, unless there is a third disk present, this configuration would end up in an error:

```
{
  "logical_disks": [
    {
      "size_gb": "MAX",
      "raid_level": "1",
      "is_root_volume": false,
      "physical_disks": [
        {"size": "> 100"},
        {"size": "> 100"}
      ]
    }
  ]
}
```

Current RAID configuration

After target RAID configuration is applied on the bare metal node, Ironic populates the current RAID configuration. This is populated in the `raid_config` field in the Ironic node. This contains the details about every logical disk after they were created on the bare metal node. It contains details like RAID controller used, the backing physical disks used, WWN of each logical disk, etc. It also contains information about each physical disk found on the bare metal node.

To get the current RAID configuration:

```
baremetal node show <node-uuid-or-name>
```

Workflow

- Operator configures the bare metal node with a hardware type that has a `RAIDInterface` other than `no-raid`. For instance, for Software RAID, this would be `agent`.
- For in-band RAID configuration, operator builds an agent ramdisk which supports RAID configuration by bundling the hardware manager with the ramdisk. See *Build agent ramdisk which supports RAID configuration* for more information.
- Operator prepares the desired target RAID configuration as mentioned in *Target RAID configuration*. The target RAID configuration is set on the Ironic node:

  ```
  baremetal node set <node-uuid-or-name> \
    --target-raid-config <JSON file containing target RAID configuration>
  ```

  The CLI command can accept the input from standard input also:

  ```
  baremetal node set <node-uuid-or-name> \
    --target-raid-config -
  ```

- Create a JSON file with the RAID clean steps for manual cleaning. Add other clean steps as desired:

  ```
  [{
    "interface": "raid",
    "step": "delete_configuration"
  },
  {
    "interface": "raid",
    "step": "create_configuration"
  }]
  ```

  Note

  create_configuration doesnt remove existing disks. It is recommended to add delete_configuration before create_configuration to make sure that only the desired logical disks exist in the system after manual cleaning.

- Bring the node to `manageable` state and do a `clean` action to start cleaning on the node:

  ```
  baremetal node clean <node-uuid-or-name> \
    --clean-steps <JSON file containing clean steps created above>
  ```

- After manual cleaning is complete, the current RAID configuration is reported in the `raid_config` field when running:

  ```
  baremetal node show <node-uuid-or-name>
  ```

Software RAID

Building Linux software RAID in-band (via the Ironic Python Agent ramdisk) is supported starting with the Train release. It is requested by using the `agent` RAID interface and RAID configuration with all controllers set to `software`. You can find a software RAID configuration example in *Examples for target_raid_config*.

There are certain limitations to be aware of:

- Only the mandatory properties (plus the required `controller` property) from *Target RAID configuration* are currently supported.
- The number of created Software RAID devices must be 1 or 2. If there is only one Software RAID device, it has to be a RAID-1. If there are two, the first one has to be a RAID-1, while the RAID level for the second one can be 0, 1, 1+0, 5, or 6. As the first RAID device will be the deployment device, enforcing a RAID-1 reduces the risk of ending up with a non-booting node in case of a disk failure.
- Building RAID will fail if the target disks are already partitioned. Wipe the disks using e.g. the `erase_devices_metadata` clean step before building RAID:

```
[{
  "interface": "raid",
  "step": "delete_configuration"
},
{
  "interface": "deploy",
  "step": "erase_devices_metadata"
},
{
  "interface": "raid",
  "step": "create_configuration"
}]
```

- The final instance image must have the `mdadm` utility installed and needs to be able to detect software RAID devices at boot time (which is usually done by having the RAID drivers embedded in the images initrd).
- Regular cleaning will not remove RAID configuration (similarly to hardware RAID). To destroy RAID run the `delete_configuration` manual clean step.
- There is no support for partition images, only whole-disk images are supported with Software RAID. See *Add images to the Image service*. This includes flavors requesting dynamic creation of swap filesystems. Swap should be pre-allocated inside of a disk image partition layout.
- Images utilizing LVM for their root filesystem are not supported. Patches are welcome to explicitly support such functionality.
- If the root filesystem UUID is not known to Ironic via metadata, then the disk image layout **MUST** have the first partition consist of the root filesystem. Ironic is agnostic if the partition table is a DOS MBR or a GPT partition.

  Starting in Ironic 14.0.0 (Ussuri), the root filesystem UUID can be set and passed through to Ironic through the Glance Image Service `properties` sub-field `rootfs_uuid` for the image to be deployed.

  Starting in Ironic 16.1.0 (Wallaby), similar functionality is available via the baremetal node

`instance_info` field value `image_rootfs_uuid`. See *Using Bare Metal service as a standalone service* for more details on standalone usage including an example command.

- In UEFI mode, the Ironic Python Agent creates EFI system partitions (ESPs) for the bootloader and the boot configuration (grub.cfg or grubenv) on all holder devices. The content of these partitions is populated upon deployment from the deployed user image. Depending on how the partitions are mounted, the content of the partitions may get out of sync, e.g. when new kernels are installed or the bootloader is updated, so measures to keep these partitions in sync need to be taken. Note that starting with the Victoria release, the Ironic Python Agent configures a RAID-1 mirror for the ESPs, so no additional measures to ensure consistency of the ESPs should be required any longer.
- In BIOS mode, the Ironic Python Agent installs the boot loader onto all disks. While nothing is required for kernel or grub package updates, re-installing the bootloader on one disk, e.g. during a disk replacement, may require to re-install the bootloader on all disks. Otherwise, there is a risk of an incompatibility of the grub components stored on the device (i.e. stage1/boot.img in the MBR and stage1.5/core.img in the MBR gap) with the ones stored in /boot (stage2). This incompatibility can render the node unbootable if the wrong disk is selected for booting.
- Linux kernel device naming is not consistent across reboots for RAID devices and may be numbered in a distribution specific pattern. Operators will need to be mindful of this if a root device hint is utilized. A particular example of this is that the first md0 device on a Ubuntu based ramdisk may start as device md0, whereas on a Centos or Red Hat Enterprise Linux based ramdisk may start at device md127. After a reboot, these device names may change entirely.

Note

Root device hints should not be explicitly required to utilize software RAID. Candidate devices are chosen by sorting the usable device list looking for the smallest usable device which is then sorted by name. The secondary sort by name improves the odds for matching the first initialized block device. In the case of software RAID, they are always a little smaller than the primary block devices due to metadata overhead, which helps make them the most likely candidate devices.

Image requirements

Since Ironic needs to perform additional steps when deploying nodes with software RAID, there are some requirements the deployed images need to fulfill. Up to and including the Train release, the image needs to have its root file system on the first partition. Starting with Ussuri, the image can also have additional metadata to point Ironic to the partition with the root file system: for this, the image needs to set the `rootfs_uuid` property with the file system UUID of the root file system. One way to extract this UUID from an existing image is to download the image, mount it as a loopback device, and use `blkid`:

```
$ sudo losetup -f
$ sudo losetup /dev/loop0 /tmp/myimage.raw
$ sudo kpartx -a /dev/loop0
$ blkid
```

The pre-Ussuri approach, i.e. to have the root file system on the first partition, is kept as a fallback and hence allows software RAID deployments where Ironic does not have access to any image metadata (e.g. Ironic stand-alone).

Using RAID in nova flavor for scheduling

The operator can specify the `raid_level` capability in nova flavor for node to be selected for scheduling:

```
openstack flavor set my-baremetal-flavor --property capabilities:raid_level=
↪"1+0"
```

Developer documentation

In-band RAID configuration is done using IPA ramdisk. IPA ramdisk has support for pluggable hardware managers which can be used to extend the functionality offered by IPA ramdisk using stevedore plugins. For more information, see Ironic Python Agent Hardware Manager documentation.

The hardware manager that supports RAID configuration should do the following:

1. Implement a method named `create_configuration`. This method creates the RAID configuration as given in `target_raid_config`. After successful RAID configuration, it returns the current RAID configuration information which ironic uses to set `node.raid_config`.
2. Implement a method named `delete_configuration`. This method deletes all the RAID disks on the bare metal.
3. Return these two clean steps in `get_clean_steps` method with priority as 0. Example:

```
return [{'step': 'create_configuration',
         'interface': 'raid',
         'priority': 0},
        {'step': 'delete_configuration',
         'interface': 'raid',
         'priority': 0}]
```

4.2.7 BIOS Configuration

Overview

The Bare Metal service supports BIOS configuration for bare metal nodes. It allows administrators to retrieve and apply the desired BIOS settings via CLI or REST API. The desired BIOS settings are applied during manual cleaning.

Prerequisites

Bare metal servers must be configured by the administrator to be managed via ironic hardware type that supports BIOS configuration.

Enabling hardware types

Enable a specific hardware type that supports BIOS configuration. Refer to *Enabling drivers and hardware types* for how to enable a hardware type.

Enabling hardware interface

To enable the bios interface:

```
[DEFAULT]
enabled_bios_interfaces = no-bios
```

Append the actual bios interface name supported by the enabled hardware type to `enabled_bios_interfaces` with comma separated values in `ironic.conf`.

All available in-tree bios interfaces are listed in setup.cfg file in the source code tree, for example:

```
ironic.hardware.interfaces.bios =
    fake = ironic.drivers.modules.fake:FakeBIOS
    no-bios = ironic.drivers.modules.noop:NoBIOS
```

Retrieve BIOS settings

To retrieve the cached BIOS configuration from a specified node:

```
$ baremetal node bios setting list <node>
```

BIOS settings are cached on each node cleaning operation or when settings have been applied successfully via BIOS cleaning steps. The return of above command is a table of the last cached BIOS settings from the specified node. If `-f json` is added as a suffix to the above command, it returns BIOS settings as following:

```
[
  {
    "setting name":
      {
        "name": "setting name",
        "value": "value"
      }
  },
  {
    "setting name":
      {
        "name": "setting name",
        "value": "value"
      }
  },
  ...
]
```

To get a specified BIOS setting for a node:

```
$ baremetal node bios setting show <node> <setting-name>
```

If `-f json` is added as a suffix to the above command, it returns BIOS settings as following:

```
{
  "setting name":
    {
      "name": "setting name",
      "value": "value"
    }
}
```

Configure BIOS settings

Two *Manual cleaning* steps are available for managing nodes BIOS settings:

Factory reset

This cleaning step resets all BIOS settings to factory default for a given node:

```
{
  "target":"clean",
  "clean_steps": [
    {
      "interface": "bios",
      "step": "factory_reset"
    }
  ]
}
```

The `factory_reset` cleaning step does not require any arguments, as it resets all BIOS settings to factory defaults.

Apply BIOS configuration

This cleaning step applies a set of BIOS settings for a node:

```
{
  "target":"clean",
  "clean_steps": [
    {
      "interface": "bios",
      "step": "apply_configuration",
      "args": {
        "settings": [
          {
            "name": "name",
            "value": "value"
          },
          {
            "name": "name",
            "value": "value"
          }
        ]
      }
    }
  ]
}
```

The representation of `apply_configuration` cleaning step follows the same format of *Manual cleaning*. The desired BIOS settings can be provided via the `settings` argument which contains a list of BIOS options to be applied, each BIOS option is a dictionary with `name` and `value` keys.

To check whether the desired BIOS configuration is set properly, use the command mentioned in the *Retrieve BIOS settings* section.

Note

When applying BIOS settings to a node, vendor-specific driver may take the given BIOS settings from the argument and compare them with the current BIOS settings on the node and only apply when there is a difference.

4.2.8 Firmware update using manual cleaning

The firmware update cleaning step allows one or more firmware updates to be applied to a node. If multiple updates are specified, then they are applied sequentially in the order given. The server is rebooted once per update. If a failure occurs, the cleaning step immediately fails which may result in some updates not being applied. If the node is placed into maintenance mode while a firmware update cleaning step is running that is performing multiple firmware updates, the update in progress will complete, and processing of the remaining updates will pause. When the node is taken out of maintenance mode, processing of the remaining updates will continue.

Note

Only *Redfish driver* supports firmware updates currently.

When updating the BMC firmware, the BMC may become unavailable for a period of time as it resets. In this case, it may be desirable to have the cleaning step wait after the update has been applied before indicating that the update was successful. This allows the BMC time to fully reset before further operations are carried out against it. To cause the cleaning step to wait after applying an update, an optional `wait` argument may be specified in the firmware image dictionary. The value of this argument indicates the number of seconds to wait following the update. If the `wait` argument is not specified, then this is equivalent to `wait 0`, meaning that it will not wait and immediately proceed with the next firmware update if there is one, or complete the cleaning step if not.

How it works - via Management Interface

The `update_firmware` cleaning step accepts JSON in the following format:

```
[{
    "interface": "management",
    "step": "update_firmware",
    "args": {
        "firmware_images":[
            {
                "url": "<url_to_firmware_image1>",
                "checksum": "<checksum for image, uses SHA1, SHA256, or␣
↪SHA512>",
                "source": "<optional override source setting for image>",
                "wait": <number_of_seconds_to_wait>
            },
            {
                "url": "<url_to_firmware_image2>"
            },
            ...
        ]
```

(continues on next page)

(continued from previous page)

```
    }
}]
```

The different attributes of the `update_firmware` cleaning step are as follows:

Attribute	Description
`interface`	Interface of the cleaning step. Must be `management` for firmware update
`step`	Name of cleaning step. Must be `update_firmware` for firmware update
`args`	Keyword-argument entry (<name>: <value>) being passed to the step
`args.firmware_images`	Ordered list of dictionaries of firmware images to be applied

Each firmware image dictionary is of the form:

```
{
  "url": "<URL of firmware image file>",
  "checksum": "<checksum for image, uses SHA1>",
  "source": "<Optional override source setting for image>",
  "wait": <Optional time in seconds to wait after applying update>
}
```

The `url` and `checksum` arguments in the firmware image dictionary are mandatory, while the `source` and `wait` arguments are optional.

For `url` currently `http`, `https`, `swift` and `file` schemes are supported.

`source` corresponds to *`redfish.firmware_source`* and by setting it here, it is possible to override global setting per firmware image in clean step arguments.

Note

At the present time, targets for the firmware update cannot be specified. In testing, the BMC applied the update to all applicable targets on the node. It is assumed that the BMC knows what components a given firmware image is applicable to.

Applying updates

To perform a firmware update, first download the firmware to a web server, Swift or filesystem that the Ironic conductor or BMC has network access to. This could be the ironic conductor web server or another web server on the BMC network. Using a web browser, curl, or similar tool on a server that has network access to the BMC or Ironic conductor, try downloading the firmware to verify that the URLs are correct and that the web server is configured properly.

Next, construct the JSON for the firmware update cleaning step to be executed. When launching the firmware update, the JSON may be specified on the command line directly or in a file. The following example shows one cleaning step that installs four firmware updates. All except 3rd entry that has explicit `source` added, uses the setting from *`redfish.firmware_source`* to determine if and where to stage the files:

```
[{
    "interface": "management",
    "step": "update_firmware",
    "args": {
        "firmware_images":[
            {
                "url": "http://192.0.2.10/BMC_4_22_00_00.EXE",
                "checksum": "<sha1-checksum-of-the-file>",
                "wait": 300
            },
            {
                "url": "https://192.0.2.10/NIC_19.0.12_A00.EXE",
                "checksum": "<sha1-checksum-of-the-file>"
            },
            {
                "url": "file:///firmware_images/idrac/9/PERC_WN64_6.65.65.65_
↪A00.EXE",
                "checksum": "<sha1-checksum-of-the-file>",
                "source": "http"
            },
            {
                "url": "swift://firmware_container/BIOS_W8Y0W_WN64_2.1.7.EXE",
                "checksum": "<sha1-checksum-of-the-file>"
            }
        ]
    }
}]
```

Finally, launch the firmware update cleaning step against the node. The following example assumes the above JSON is in a file named `firmware_update.json`:

```
$ baremetal node clean <ironic_node_uuid> --clean-steps firmware_update.json
```

In the following example, the JSON is specified directly on the command line:

```
$ baremetal node clean <ironic_node_uuid> --clean-steps \
    '[{"interface": "management", "step": "update_firmware", "args": {
↪"firmware_images":[{"url": "http://192.0.2.10/BMC_4_22_00_00.EXE", "wait":␣
↪300}, {"url": "https://192.0.2.10/NIC_19.0.12_A00.EXE"}]}}]'
```

Note

Firmware updates may take some time to complete. If a firmware update cleaning step consistently times out, then consider performing fewer firmware updates in the cleaning step or increasing `clean_callback_timeout` in ironic.conf to increase the timeout value.

Warning

Warning: Removing power from a server while it is in the process of updating firmware may result

in devices in the server, or the server itself becoming inoperable.

How it works - via Firmware Interface

The `update` step can be used via cleaning or servicing, it accepts a JSON in the following format:

```
[{
    "interface": "firmware",
    "step": "update",
    "args": {
        "settings":[
            {
                "component": "bmc"
                "url": "<url_to_firmware_image1>"
                "wait": <number_of_seconds_to_wait>
            },
            {
                "component": "bios"
                "url": "<url_to_firmware_image2>"
            },
            {
                "component": "nic:AD0700",
                "url": "<url_to_firmware_image3>"
            },
            {
                "component": "nic:NIC.Slot.2",
                "url": "<url_to_firmware_image4>"
            }
            ...
        ]
    }
}]
```

The different attributes of the `update` step are as follows:

Attribute	Description
`interface`	Interface of the step. Must be `firmware` for firmware update
`step`	Name of the step. Must be `update` for firmware update
`args`	Keyword-argument entry (<name>: <value>) being passed to the step
`args.settings`	Ordered list of dictionaries of firmware updates to be applied

Each firmware image dictionary is of the form:

```
{
  "component": "The desired component to have the firmware updated, supported␣
↪components are listed below",
  "url": "<URL of firmware image file>",
  "wait": <Optional time in seconds to wait after applying update>
}
```

Supported Components	Description
bmc	The BMC firmware
bios	The BIOS firmware
nic:<NIC_REDFIS	Since machines can have multiple NICs, we use **nic:** as prefix plus the **NIC_REDFISH_ID** to identify the NIC to update

The `component` and `url` arguments in the firmware image dictionary are mandatory, while the `wait` argument is optional.

For `url` currently `http`, `https`, `swift` and `file` schemes are supported.

Applying updates

To perform a firmware update, first download the firmware to a web server, Swift or filesystem that the Ironic conductor or BMC has network access to. This could be the ironic conductor web server or another web server on the BMC network. Using a web browser, curl, or similar tool on a server that has network access to the BMC or Ironic conductor, try downloading the firmware to verify that the URLs are correct and that the web server is configured properly.

Next, construct the JSON for the firmware update step to be executed. When launching the firmware update, the JSON may be specified on the command line directly or in a file. The following example shows one step that installs two firmware updates.

```
[{
    "interface": "firmware",
    "step": "update",
    "args": {
        "settings":[
            {
                "component": "bmc",
                "url": "http://192.0.2.10/BMC_4_22_00_00.EXE",
                "wait": 300
            },
            {
                "component": "bios",
                "url": "https://192.0.2.10/BIOS_19.0.12_A00.EXE"
            },
            {
                "component": "nic:AD0700",
                "url": "http://192.0.2.10/NIC_19.0.12_AD0700.EXE"
            },
            {
                "component": "nic:NIC.Slot.2",
                "url": "http://192.0.2.10/NICSlot2_1.0.12.EXE"
            }
        ]
    }
}]
```

It is also possible to use `runbooks` for firmware updates.

```
$ baremetal runbook create --name <RUNBOOK> --steps \
    '[{"interface": "firmware", "step": "update", "args": {"settings":[{
↪"component": "bmc", "url":"http://192.168.0.8:8080/ilo5278.bin"}, {
↪"component": "nic:AD0700", "url":"http://192.168.0.8:8080/nic.bin"}, {
↪"component": "nic:NIC.Slot.2", "url":"http://192.168.0.8:8080/nic.bin"}]}}]'
$ baremetal node add trait <ironic_node_uuid> <RUNBOOK>
$ baremetal node <clean or service>  <ironic_node_uuid> --runbook <RUNBOOK>
```

Finally, launch the firmware update step against the node. The following example assumes the above JSON is in a file named `update.json`:

```
$ baremetal node clean <ironic_node_uuid> --clean-steps update.json
$ baremetal node service <ironic_node_uuid> --service-steps update.json
```

In the following example, the JSON is specified directly on the command line:

```
$ baremetal node clean <ironic_node_uuid> --clean-steps \
    '[{"interface": "firmware", "step": "update", "args": {"settings":[{
↪"component": "bmc", "url":"http://192.168.0.8:8080/ilo5278.bin"}]}}]'
$ baremetal node clean <ironic_node_uuid> --clean-steps \
    '[{"interface": "firmware", "step": "update", "args": {"settings":[{
↪"component": "bios", "url":"http://192.168.0.8:8080/bios.bin"}]}}]'
$ baremetal node service <ironic_node_uuid> --service-steps \
    '[{"interface": "firmware", "step": "update", "args": {"settings":[{
↪"component": "bmc", "url":"http://192.168.0.8:8080/ilo5278.bin"}]}}]'
$ baremetal node service <ironic_node_uuid> --service-steps \
    '[{"interface": "firmware", "step": "update", "args": {"settings":[{
↪"component": "bios", "url":"http://192.168.0.8:8080/bios.bin"}]}}]'
$ baremetal node clean <ironic_node_uuid> --clean-steps \
    '[{"interface": "firmware", "step": "update", "args": {"settings":[{
↪"component": "nic:AD0700", "url":"http://192.168.0.8:8080/nic.bin"}]}}]'
$ baremetal node clean <ironic_node_uuid> --clean-steps \
    '[{"interface": "firmware", "step": "update", "args": {"settings":[{
↪"component": "nic:NIC.Slot.2", "url":"http://192.168.0.8:8080/nic.bin"}]}}]'
```

Note

For Dell machines, you must extract the firmimgFIT.d9 from the iDRAC.exe This can be done using the command `7za e iDRAC_<VERSION>.exe`.

Note

For HPE machines you must extract the ilo5_<version>.bin from the ilo5_<version>.fwpkg This can be done using the command `7za e ilo<version>.fwpkg`.

4.2.9 Rescue Mode

Overview

The Bare Metal Service supports putting nodes in rescue mode using hardware types that support rescue interfaces. The hardware types utilizing ironic-python-agent with `PXE/Virtual Media` based boot interface can support rescue operation when configured appropriately.

Note

The rescue operation is currently supported only when tenant networks use DHCP to obtain IP addresses.

Rescue operation can be used to boot nodes into a rescue ramdisk so that the `rescue` user can access the node, in order to provide the ability to access the node in case access to OS is not possible. For example, if there is a need to perform manual password reset or data recovery in the event of some failure, rescue operation can be used.

Configuring The Bare Metal Service

Configure the Bare Metal Service appropriately so that the service has the information needed to boot the ramdisk before a user tries to initiate rescue operation. This will differ somewhat between different deploy environments, but an example of how to do this is outlined below:

1. Create and configure ramdisk that supports rescue operation. Please see *Building or downloading a deploy ramdisk image* for detailed instructions to build a ramdisk.
2. Configure a network to use for booting nodes into the rescue ramdisk in neutron, and note the UUID or name of this network. This is required if youre using the neutron DHCP provider and have Bare Metal Service managing ramdisk booting (the default). This can be the same network as your cleaning or tenant network (for flat network). For an example of how to configure new networks with Bare Metal Service, see the *Configure the Networking service for bare metal provisioning* documentation.
3. Add the unique name or UUID of your rescue network to `ironic.conf`:

   ```
   [neutron]
   rescuing_network=<RESCUE_UUID_OR_NAME>
   ```

   Note

   This can be set per node via driver_info[rescuing_network]

4. Restart the ironic conductor service.
5. Specify a rescue kernel and ramdisk or rescue ISO compatible with the nodes driver for pxe based boot interface or virtual-media based boot interface respectively.

   Example for pxe based boot interface:

   ```
   baremetal node set $NODE_UUID \
       --driver-info rescue_ramdisk=$RESCUE_INITRD_UUID \
       --driver-info rescue_kernel=$RESCUE_VMLINUZ_UUID
   ```

See *Add images to the Image service* for details. If you are not using Image service, it is possible to provide images to Bare Metal service via hrefs.

After this, The Bare Metal Service should be ready for `rescue` operation. Test it out by attempting to rescue an active node and connect to the instance using ssh, as given below:

```
baremetal node rescue $NODE_UUID \
    --rescue-password <PASSWORD> --wait

ssh rescue@$INSTANCE_IP_ADDRESS
```

To move a node back to active state after using rescue mode you can use `unrescue`. Please unmount any filesystems that were manually mounted before proceeding with unrescue. The node unrescue can be done as given below:

```
baremetal node unrescue $NODE_UUID
```

`rescue` and `unrescue` operations can also be triggered via the Compute Service using the following commands:

```
openstack server rescue --password <password> <server>

openstack server unrescue <server>
```

4.2.10 Boot From Volume

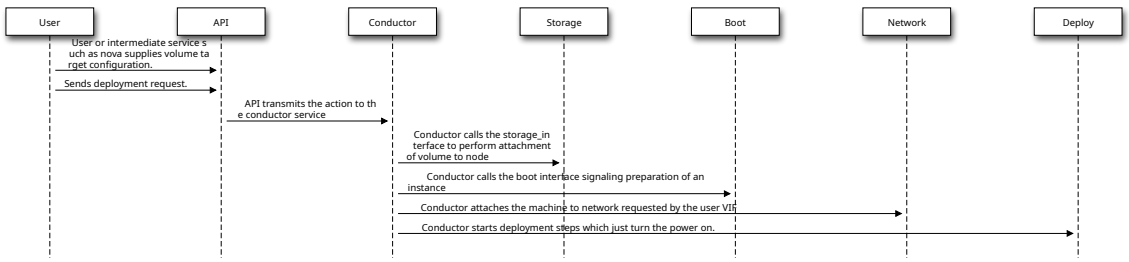
Overview

The Bare Metal service supports booting from a Cinder iSCSI volume as of the Pike release. This guide will primarily deal with this use case but will be updated as more paths for booting from a volume, such as FCoE, are introduced.

The boot from volume is supported on both legacy BIOS and UEFI (iPXE binary for EFI booting) boot mode. We need to perform with suitable images which will be created by diskimage-builder tool.

How this works - From Ironics point of view

In essence, ironic sets the stage for the process, by providing the required information to the boot interface to facilitate the configuration of the the node OR the iPXE boot templates such that the node CAN be booted.



In this example, the boot interface does the heavy lifting. For the `ilo` hardware type with its hardware type-specific boot interface, it is able to signal via an out-of-band mechanism to the baremetal nodes BMC that the integrated iSCSI initiators are to connect to the supplied volume target information.

In most hardware, this would be the network cards of the machine.

In the case of the `ipxe` boot interface, templates are created on disk which point to the iscsi target information that was either submitted as part of the volume target, or when integrated with Nova, what was requested as the baremetals boot from volume disk upon requesting the instance.

In terms of network access, both interface methods require connectivity to the iscsi target. In the vendor driver-specific path, additional network configuration options may be available to allow separation of standard network traffic and instance network traffic. In the iPXE case, this is not possible as the OS userspace re-configures the iSCSI connection after detection inside the OS ramdisk boot.

An iPXE user *may* be able to leverage multiple VIFs, one specifically set to be set with `pxe_enabled` to handle the initial instance boot and back-end storage traffic whereas external-facing network traffic occurs on a different interface. This is a common pattern in iSCSI based deployments in the physical realm.

Prerequisites

Currently booting from a volume requires:

- Bare Metal service version 9.0.0
- Bare Metal API microversion 1.33 or later
- A driver that utilizes the *PXE boot mechanism*. Currently booting from a volume is supported by the reference drivers that utilize PXE boot mechanisms when iPXE is enabled.
- iPXE is an explicit requirement, as it provides the mechanism that attaches and initiates booting from an iSCSI volume.
- Metadata services need to be configured and available for the instance images to obtain configuration such as keys. Configuration drives are not supported due to minimum disk extension sizes.

Conductor Configuration

In ironic.conf, you can specify a list of enabled storage interfaces. Check *DEFAULT.enabled_storage_interfaces* in your ironic.conf to ensure that your desired interface is enabled. For example, to enable the `cinder` and `noop` storage interfaces:

```
[DEFAULT]
enabled_storage_interfaces = cinder,noop
```

If you want to specify a default storage interface rather than setting the storage interface on a per node basis, set *DEFAULT.default_storage_interface* in ironic.conf. The `default_storage_interface` will be used for any node that doesnt have a storage interface defined.

Node Configuration

Storage Interface

You will need to specify what storage interface the node will use to handle storage operations. For example, to set the storage interface to `cinder` on an existing node:

```
baremetal node set --storage-interface cinder $NODE_UUID
```

A default storage interface can be specified in ironic.conf. See the *Conductor Configuration* section for details.

The storage interface is responsible for managing the mapping state of the volume to the host. If some changes need to be communicated from Cinder and then updated for Ironic to become aware of them, such as a change in iSCSI credentials, then the act of powering baremetal node off via Ironics API will trigger these values to be updated automatically as the `cinder` storage interface resets the volume attachments with power actions to ensure the latest information is used for each boot sequence.

iSCSI Configuration

In order for a bare metal node to boot from an iSCSI volume, the `iscsi_boot` capability for the node must be set to `True`. For example, if you want to update an existing node to boot from volume:

```
baremetal node set --property capabilities=iscsi_boot:True $NODE_UUID
```

You will also need to create a volume connector for the node, so the storage interface will know how to communicate with the node for storage operation. In the case of iSCSI, you will need to provide an iSCSI Qualifying Name (IQN) that is unique to your SAN. For example, to create a volume connector for iSCSI:

```
baremetal volume connector create \
         --node $NODE_UUID --type iqn --connector-id iqn.2017-08.org.
↪openstack.$NODE_UUID
```

Image Creation

We use `disk-image-create` in diskimage-builder tool to create images for boot from volume feature. Some required elements for this mechanism for corresponding boot modes are as following:

- Legacy BIOS boot mode: `iscsi-boot` element.
- UEFI boot mode: `iscsi-boot` and `block-device-efi` elements.

An example below:

```
export IMAGE_NAME=<image_name>
export DIB_CLOUD_INIT_DATASOURCES="ConfigDrive, OpenStack"
disk-image-create centos7 vm cloud-init-datasources dhcp-all-interfaces iscsi-
↪boot dracut-regenerate block-device-efi -o $IMAGE_NAME
```

Note

- For CentOS images, we must add dependent element named `dracut-regenerate` during image creation. Otherwise, the image creation will fail with an error.
- For Ubuntu images, we only support `iscsi-boot` element without `dracut-regenerate` element during image creation.

Advanced Topics

Use without the Compute Service

As discussed in other sections, the Bare Metal service has a concept of a `connector` that is used to represent an interface that is intended to be utilized to attach the remote volume.

In addition to the connectors, we have a concept of a `target` that can be defined via the API. While a user of this feature through the Compute service would automatically have a new target record created for them, it is not explicitly required and can be performed manually.

A target record can be created using a command similar to the example below:

```
baremetal volume target create \
          --node $NODE_UUID --type iscsi --boot-index 0 --volume $VOLUME_UUID
```

Note

A `boot-index` value of `0` represents the boot volume for a node. As the `boot-index` is per-node in sequential order, only one boot volume is permitted for each node.

Use Without Cinder

In the Rocky release, an `external` storage interface is available that can be utilized without a Block Storage Service installation.

Under normal circumstances the `cinder` storage interface interacts with the Block Storage Service to orchestrate and manage attachment and detachment of volumes from the underlying block service system.

The `external` storage interface contains the logic to allow the Bare Metal service to determine if the Bare Metal node has been requested with a remote storage volume for booting. This is in contrast to the default `noop` storage interface which does not contain logic to determine if the node should or could boot from a remote volume.

It must be noted that minimal configuration or value validation occurs with the `external` storage interface. The `cinder` storage interface contains more extensive validation, that is likely unnecessary in a `external` scenario.

Setting the external storage interface:

```
baremetal node set --storage-interface external $NODE_UUID
```

Setting a volume:

```
baremetal volume target create --node $NODE_UUID \
    --type iscsi --boot-index 0 --volume-id $VOLUME_UUID \
    --property target_iqn="iqn.2010-10.com.example:vol-X" \
    --property target_lun="0" \
    --property target_portal="192.168.0.123:3260" \
    --property auth_method="CHAP" \
    --property auth_username="ABC" \
    --property auth_password="XYZ" \
```

Ensure that no image_source is defined:

```
baremetal node unset \
    --instance-info image_source $NODE_UUID
```

Deploy the node:

```
baremetal node deploy $NODE_UUID
```

Upon deploy, the boot interface for the baremetal node will attempt to either create iPXE configuration OR set boot parameters out-of-band via the management controller. Such action is boot interface specific and may not support all forms of volume target configuration. As of the Rocky release, the bare metal service does not support writing an Operating System image to a remote boot from volume target, so that also must be ensured by the user in advance.

Records of volume targets are removed upon the node being undeployed, and as such are not persistent across deployments.

Cinder Multi-attach

Volume multi-attach is a function that is commonly performed in computing clusters where dedicated storage subsystems are utilized. For some time now, the Block Storage service has supported the concept of multi-attach. However, the Compute service, as of the Pike release, does not yet have support to leverage multi-attach. Concurrently, multi-attach requires the backend volume driver running as part of the Block Storage service to contain support for multi-attach volumes.

When support for storage interfaces was added to the Bare Metal service, specifically for the `cinder` storage interface, the concept of volume multi-attach was accounted for, however has not been fully tested, and is unlikely to be fully tested until there is a Compute service integration as well as volume driver support.

The data model for storage of volume targets in the Bare Metal service has no constraints on the same target volume from being utilized. When interacting with the Block Storage service, the Bare Metal service will prevent the use of volumes that are being reported as `in-use` if they do not explicitly support multi-attach.

4.2.11 Configuring Consoles

Overview

There are two types of consoles which are available in Bare Metal service:

- (*Node graphical console*) for a graphical console from a NoVNC web browser
- (*Node serial console*) for serial console support

Node graphical console

Graphical console drivers require a configured and running `ironic-novncproxy` service. Each supported driver is described below.

redfish-graphical

A driver for a subset of Redfish hosts. Starting the console will start a container which exposes a VNC server for `ironic-novncproxy` to attach to. When attached, a browser will start which displays an HTML5 based console on the following supported hosts:

- Dell iDRAC
- HPE iLO
- Supermicro

```
[DEFAULT]
enabled_hardware_types = redfish
enabled_console_interfaces = redfish-graphical,no-console
```

fake-graphical

A driver for demonstrating working graphical console infrastructure. Starting the console will start a container which exposes a VNC server for `ironic-novncproxy` to attach to. When attached, a browser will start which displays an animation.

```
[DEFAULT]
enabled_hardware_types = fake-hardware
enabled_console_interfaces = fake-graphical,no-console
```

Node serial console

Serial consoles for nodes are implemented using socat. It is supported by the `ipmi` and `redfish` hardware types.

Note

The use of the `ipmitool-socat` console interface on any hardware type requires the ipmi connection parameters to be set into the `driver_info` filed on the node.

Serial consoles can be configured in the Bare Metal service as follows:

- Install socat on the ironic conductor node. Also, `socat` needs to be in the $PATH environment variable that the ironic-conductor service uses.

  Installation example:

  Ubuntu:

  ```
  sudo apt-get install socat
  ```

  RHEL/CentOS/Fedora:

  ```
  sudo dnf install socat
  ```

- Append console parameters for bare metal PXE boot in the Bare Metal service configuration file. See the reference on how to configure them in *Appending kernel parameters to boot instances*.
- Enable the `ipmitool-socat` console interface, for example:

  ```
  [DEFAULT]
  enabled_console_interfaces = ipmitool-socat,no-console
  ```

- Configure node console.

  If the node uses a hardware type, for example `ipmi`, set the nodes console interface to `ipmitool-socat`:

```
baremetal node set <node> --console-interface ipmitool-socat
```

Enable the serial console, for example:

```
baremetal node set <node> --driver-info ipmi_terminal_port=<port>
baremetal node console enable <node>
```

Check whether the serial console is enabled, for example:

```
baremetal node validate <node>
```

Disable the serial console, for example:

```
baremetal node console disable  <node>
baremetal node unset <node> --driver-info <ipmi_terminal_port>
```

Serial console information is available from the Bare Metal service. Get serial console information for a node from the Bare Metal service as follows:

```
baremetal node console show <node>
+-----------------+--------------------------------------------------------------
↪----------+
| Property        | Value                                                        ␣
↪          |
+-----------------+--------------------------------------------------------------
↪----------+
| console_enabled | True                                                         ␣
↪          |
| console_info    | {u'url': u'tcp://<host>:<port>', u'type': u'socat'}          ␣
↪          |
+-----------------+--------------------------------------------------------------
↪----------+
```

If `console_enabled` is `false` or `console_info` is `None` then the serial console is disabled. If you want to launch serial console, see the `Configure node console`.

The node serial console of the Bare Metal service is compatible with the serial console of the Compute service. Hence, serial consoles to Bare Metal nodes can be seen and interacted with via the Dashboard service. In order to achieve that, you need to follow the documentation for Serial Console from the Compute service.

Configuring HA

When using Bare Metal serial console under High Availability (HA) configuration, you may consider some settings below.

- If you use HAProxy, you may need to set the timeout for both client and server sides with appropriate values. Here is an example of the configuration for the timeout parameter.

```
frontend nova_serial_console
  bind 192.168.20.30:6083
  timeout client 10m  # This parameter is necessary
```

(continues on next page)

(continued from previous page)

```
  use_backend nova_serial_console if <...>

backend nova_serial_console
  balance source
  timeout server 10m  # This parameter is necessary
  option  tcpka
  option  tcplog
  server  controller01 192.168.30.11:6083 check inter 2000 rise 2 fall 5
  server  controller02 192.168.30.12:6083 check inter 2000 rise 2 fall 5
```

- The Compute services caching feature may need to be enabled in order to make the Bare Metal serial console work under a HA configuration. Here is an example of a caching configuration in `nova.conf`.

```
[cache]
enabled = true
backend = dogpile.cache.memcached
memcache_servers = memcache01:11211,memcache02:11211,memcache03:11211
```

4.2.12 Notifications

Ironic, when configured to do so, will emit notifications over a message bus that indicate different events that occur within the service. These can be consumed by any external service. Examples may include a billing or usage system, a monitoring data store, or other OpenStack services. This page describes how to enable notifications and the different kinds of notifications that ironic may emit. The external consumer will see notifications emitted by ironic as JSON objects structured in the following manner:

```
{
    "priority": <string, defined by the sender>,
    "event_type": <string, defined by the sender>,
    "timestamp": <string, the isotime of when the notification emitted>,
    "publisher_id": <string, defined by the sender>,
    "message_id": <uuid, generated by oslo>,
    "payload": <json serialized dict, defined by the sender>
}
```

Configuration

To enable notifications with ironic, there are two configuration options in ironic.conf that must be adjusted.

The first option is the `notification_level` option in the `[DEFAULT]` section of the configuration file. This can be set to debug, info, warning, error, or critical, and determines the minimum priority level for which notifications are emitted. For example, if the option is set to warning, all notifications with priority level warning, error, or critical are emitted, but not notifications with priority level debug or info. For information about the semantics of each log level, see the OpenStack logging standards[1]. If this option is unset, no notifications will be emitted. The priority level of each available notification is documented below.

[1] https://wiki.openstack.org/wiki/LoggingStandards#Log_level_definitions

The second option is the `transport_url` option in the `[oslo_messaging_notifications]` section of the configuration. This determines the message bus used when sending notifications. If this is unset, the default transport used for RPC is used.

All notifications are emitted on the ironic_versioned_notifications topic in the message bus. Generally, each type of message that traverses the message bus is associated with a topic describing what the message is about. For more information, see the documentation of your chosen message bus, such as the RabbitMQ documentation[2].

Note that notifications may be lossy, and theres no guarantee that a notification will make it across the message bus to a consumer.

Versioning

Each notification has an associated version in the ironic_object.version field of the payload. Consumers are guaranteed that microversion bumps will add new fields, while macroversion bumps are backwards-incompatible and may have fields removed.

Versioned notifications are emitted by default to the `ironic_versioned_notifications` topic. This can be changed and it is configurable in the ironic.conf with the `versioned_notifications_topics` config option.

Available notifications

The notifications that ironic emits are described here. They are listed (alphabetically) by service first, then by event_type. All examples below show payloads before serialization to JSON.

ironic-api notifications

Resources CRUD notifications

These notifications are emitted from API service when ironic resources are modified as part of create, update, or delete (CRUD)[3] procedures. All CRUD notifications are emitted at INFO level, except for error status that is emitted at ERROR level.

List of CRUD notifications for chassis:

- `baremetal.chassis.create.start`
- `baremetal.chassis.create.end`
- `baremetal.chassis.create.error`
- `baremetal.chassis.update.start`
- `baremetal.chassis.update.end`
- `baremetal.chassis.update.error`
- `baremetal.chassis.delete.start`
- `baremetal.chassis.delete.end`
- `baremetal.chassis.delete.error`

Example of chassis CRUD notification:

[2] https://www.rabbitmq.com/documentation.html
[3] https://en.wikipedia.org/wiki/Create,_read,_update_and_delete

```
{
 "priority": "info",
 "payload":{
     "ironic_object.namespace":"ironic",
     "ironic_object.name":"ChassisCRUDPayload",
     "ironic_object.version":"1.0",
     "ironic_object.data":{
         "created_at": "2016-04-10T10:13:03+00:00",
         "description": "bare 28",
         "extra": {},
         "updated_at": "2016-04-27T21:11:03+00:00",
         "uuid": "1910f669-ce8b-43c2-b1d8-cf3d65be815e"
     }
 },
 "event_type":"baremetal.chassis.update.end",
 "publisher_id":"ironic-api.hostname02"
}
```


List of CRUD notifications for deploy template:

- `baremetal.deploy_template.create.start`
- `baremetal.deploy_template.create.end`
- `baremetal.deploy_template.create.error`
- `baremetal.deploy_template.update.start`
- `baremetal.deploy_template.update.end`
- `baremetal.deploy_template.update.error`
- `baremetal.deploy_template.delete.start`
- `baremetal.deploy_template.delete.end`
- `baremetal.deploy_template.delete.error`

Example of deploy template CRUD notification:


```
{
 "priority": "info",
 "payload":{
     "ironic_object.namespace":"ironic",
     "ironic_object.name":"DeployTemplateCRUDPayload",
     "ironic_object.version":"1.0",
     "ironic_object.data":{
         "created_at": "2019-02-10T10:13:03+00:00",
         "extra": {},
         "name": "CUSTOM_HYPERTHREADING_ON",
         "steps": [
             {
                 "interface": "bios",
                 "step": "apply_configuration",
                 "args": {
```


(continues on next page)

(continued from previous page)


```
                    "settings": [
                        {
                            "name": "LogicalProc",
                            "value": "Enabled"
                        }
                    ]
                },
                "priority": 150
            }
        ],
        "updated_at": "2019-02-27T21:11:03+00:00",
        "uuid": "1910f669-ce8b-43c2-b1d8-cf3d65be815e"
    }
},
"event_type":"baremetal.deploy_template.update.end",
"publisher_id":"ironic-api.hostname02"
}
```


List of CRUD notifications for node:

- `baremetal.node.create.start`
- `baremetal.node.create.end`
- `baremetal.node.create.error`
- `baremetal.node.update.start`
- `baremetal.node.update.end`
- `baremetal.node.update.error`
- `baremetal.node.delete.start`
- `baremetal.node.delete.end`
- `baremetal.node.delete.error`

Example of node CRUD notification:


```
{
"priority": "info",
"payload":{
    "ironic_object.namespace":"ironic",
    "ironic_object.name":"NodeCRUDPayload",
    "ironic_object.version":"1.13",
    "ironic_object.data":{
        "chassis_uuid": "db0eef9d-45b2-4dc0-94a8-fc283c01171f",
        "clean_step": None,
        "conductor_group": "",
        "console_enabled": False,
        "created_at": "2016-01-26T20:41:03+00:00",
        "deploy_step": None,
        "description": "my sample node",
```


(continues on next page)

(continued from previous page)


```
        "driver": "ipmi",
        "driver_info": {
            "ipmi_address": "192.168.0.111",
            "ipmi_username": "root"},
        "extra": {},
        "inspection_finished_at": None,
        "inspection_started_at": None,
        "instance_info": {},
        "instance_uuid": None,
        "last_error": None,
        "lessee": None,
        "maintenance": False,
        "maintenance_reason": None,
        "fault": None,
        "bios_interface": "no-bios",
        "boot_interface": "pxe",
        "console_interface": "no-console",
        "deploy_interface": "direct",
        "inspect_interface": "no-inspect",
        "management_interface": "ipmitool",
        "network_interface": "flat",
        "power_interface": "ipmitool",
        "raid_interface": "no-raid",
        "rescue_interface": "no-rescue",
        "storage_interface": "noop",
        "vendor_interface": "no-vendor",
        "name": None,
        "owner": None,
        "power_state": "power off",
        "properties": {
            "memory_mb":  4096,
            "cpu_arch":  "x86_64",
            "local_gb":  10,
            "cpus":  8},
        "protected": False,
        "protected_reason": None,
        "provision_state": "deploying",
        "provision_updated_at": "2016-01-27T20:41:03+00:00",
        "resource_class": None,
        "retired": None,
        "retired_reason": None,
        "target_power_state": None,
        "target_provision_state": "active",
        "traits": [
            "CUSTOM_TRAIT1",
            "HW_CPU_X86_VMX"],
        "updated_at": "2016-01-27T20:41:03+00:00",
        "uuid": "1be26c0b-03f2-4d2e-ae87-c02d7f33c123"
    }
```


(continues on next page)

(continued from previous page)

```
},
"event_type":"baremetal.node.update.end",
"publisher_id":"ironic-api.hostname02"
}
```

List of CRUD notifications for port:

- baremetal.port.create.start
- baremetal.port.create.end
- baremetal.port.create.error
- baremetal.port.update.start
- baremetal.port.update.end
- baremetal.port.update.error
- baremetal.port.delete.start
- baremetal.port.delete.end
- baremetal.port.delete.error

Example of port CRUD notification:

```
{
"priority": "info",
"payload":{
    "ironic_object.namespace":"ironic",
    "ironic_object.name":"PortCRUDPayload",
    "ironic_object.version":"1.3",
    "ironic_object.data":{
        "address": "77:66:23:34:11:b7",
        "created_at": "2016-02-11T15:23:03+00:00",
        "node_uuid": "5b236cab-ad4e-4220-b57c-e827e858745a",
        "extra": {},
        "is_smartnic": True,
        "local_link_connection": {},
        "physical_network": "physnet1",
        "portgroup_uuid": "bd2f385e-c51c-4752-82d1-7a9ec2c25f24",
        "pxe_enabled": True,
        "updated_at": "2016-03-27T20:41:03+00:00",
        "uuid": "1be26c0b-03f2-4d2e-ae87-c02d7f33c123"
    }
},
"event_type":"baremetal.port.update.end",
"publisher_id":"ironic-api.hostname02"
}
```

List of CRUD notifications for port group:

- baremetal.portgroup.create.start
- baremetal.portgroup.create.end

- baremetal.portgroup.create.error
- baremetal.portgroup.update.start
- baremetal.portgroup.update.end
- baremetal.portgroup.update.error
- baremetal.portgroup.delete.start
- baremetal.portgroup.delete.end
- baremetal.portgroup.delete.error

Example of portgroup CRUD notification:


```
{
 "priority": "info",
 "payload":{
     "ironic_object.namespace":"ironic",
     "ironic_object.name":"PortgroupCRUDPayload",
     "ironic_object.version":"1.0",
     "ironic_object.data":{
         "address": "11:44:32:87:61:e5",
         "created_at": "2017-01-11T11:33:03+00:00",
         "node_uuid": "5b236cab-ad4e-4220-b57c-e827e858745a",
         "extra": {},
         "mode": "7",
         "name": "portgroup-node-18",
         "properties": {},
         "standalone_ports_supported": True,
         "updated_at": "2017-01-31T11:41:07+00:00",
         "uuid": "db033a40-bfed-4c84-815a-3db26bb268bb",
     }
 },
 "event_type":"baremetal.portgroup.update.end",
 "publisher_id":"ironic-api.hostname02"
}
```


List of CRUD notifications for volume connector:

- baremetal.volumeconnector.create.start
- baremetal.volumeconnector.create.end
- baremetal.volumeconnector.create.error
- baremetal.volumeconnector.update.start
- baremetal.volumeconnector.update.end
- baremetal.volumeconnector.update.error
- baremetal.volumeconnector.delete.start
- baremetal.volumeconnector.delete.end
- baremetal.volumeconnector.delete.error

Example of volume connector CRUD notification:

```
{
 "priority": "info",
 "payload": {
     "ironic_object.namespace": "ironic",
     "ironic_object.name": "VolumeConnectorCRUDPayload",
     "ironic_object.version": "1.0",
     "ironic_object.data": {
        "connector_id": "iqn.2017-05.org.openstack:01:d9a51732c3f",
        "created_at": "2017-05-11T05:57:36+00:00",
        "extra": {},
        "node_uuid": "4dbb4e69-99a8-4e13-b6e8-dd2ad4a20caf",
        "type": "iqn",
        "updated_at": "2017-05-11T08:28:58+00:00",
        "uuid": "19b9f3ab-4754-4725-a7a4-c43ea7e57360"
     }
 },
 "event_type": "baremetal.volumeconnector.update.end",
 "publisher_id":"ironic-api.hostname02"
}
```


List of CRUD notifications for volume target:

- `baremetal.volumetarget.create.start`
- `baremetal.volumetarget.create.end`
- `baremetal.volumetarget.create.error`
- `baremetal.volumetarget.update.start`
- `baremetal.volumetarget.update.end`
- `baremetal.volumetarget.update.error`
- `baremetal.volumetarget.delete.start`
- `baremetal.volumetarget.delete.end`
- `baremetal.volumetarget.delete.error`

Example of volume target CRUD notification:


```
{
 "priority": "info",
 "payload": {
     "ironic_object.namespace": "ironic",
     "ironic_object.version": "1.0",
     "ironic_object.name": "VolumeTargetCRUDPayload"
     "ironic_object.data": {
         "boot_index": 0,
         "created_at": "2017-05-11T09:38:59+00:00",
         "extra": {},
         "node_uuid": "4dbb4e69-99a8-4e13-b6e8-dd2ad4a20caf",
         "properties": {
             "access_mode": "rw",
```


(continues on next page)

(continued from previous page)


```
            "auth_method": "CHAP"
            "auth_password": "***",
            "auth_username": "urxhQCzAKr4sjyE8DivY",
            "encrypted": false,
            "qos_specs": null,
            "target_discovered": false,
            "target_iqn": "iqn.2010-10.org.openstack:volume-f0d9b0e6-b242-
↪9105-91d4-a20331693ad8",
            "target_lun": 1,
            "target_portal": "192.168.12.34:3260",
            "volume_id": "f0d9b0e6-b042-4105-91d4-a20331693ad8",
        },
        "updated_at": "2017-05-11T09:52:04+00:00",
        "uuid": "82a45833-9c58-4ec1-943c-2091ab10e47b",
        "volume_id": "f0d9b0e6-b242-9105-91d4-a20331693ad8",
        "volume_type": "iscsi"
    }
},
"event_type": "baremetal.volumetarget.update.end",
"publisher_id":"ironic-api.hostname02"
}
```


Node maintenance notifications

These notifications are emitted from API service when maintenance mode is changed via API service. List of maintenance notifications for a node:

- `baremetal.node.maintenance_set.start`
- `baremetal.node.maintenance_set.end`
- `baremetal.node.maintenance_set.error`

start and end notifications have INFO level, error has ERROR. Example of node maintenance notification:


```
{
"priority": "info",
"payload":{
    "ironic_object.namespace":"ironic",
    "ironic_object.name":"NodePayload",
    "ironic_object.version":"1.15",
    "ironic_object.data":{
        "clean_step": None,
        "conductor_group": "",
        "console_enabled": False,
        "created_at": "2016-01-26T20:41:03+00:00",
        "deploy_step": None,
        "description": "my sample node",
        "driver": "ipmi",
        "extra": {},
        "inspection_finished_at": None,
```


(continues on next page)

(continued from previous page)


```
        "inspection_started_at": None,
        "instance_info": {},
        "instance_uuid": None,
        "last_error": None,
        "lessee": None,
        "maintenance": True,
        "maintenance_reason": "hw upgrade",
        "fault": None,
        "bios_interface": "no-bios",
        "boot_interface": "pxe",
        "console_interface": "no-console",
        "deploy_interface": "direct",
        "inspect_interface": "no-inspect",
        "management_interface": "ipmitool",
        "network_interface": "flat",
        "power_interface": "ipmitool",
        "raid_interface": "no-raid",
        "rescue_interface": "no-rescue",
        "storage_interface": "noop",
        "vendor_interface": "no-vendor",
        "name": None,
        "owner": None,
        "power_state": "power off",
        "properties": {
            "memory_mb":  4096,
            "cpu_arch":  "x86_64",
            "local_gb":  10,
            "cpus":  8},
        "protected": False,
        "protected_reason": None,
        "provision_state": "available",
        "provision_updated_at": "2016-01-27T20:41:03+00:00",
        "resource_class": None,
        "retired": None,
        "retired_reason": None,
        "target_power_state": None,
        "target_provision_state": None,
        "traits": [
            "CUSTOM_TRAIT1",
            "HW_CPU_X86_VMX"],
        "updated_at": "2016-01-27T20:41:03+00:00",
        "uuid": "1be26c0b-03f2-4d2e-ae87-c02d7f33c123"
    }
},
"event_type":"baremetal.node.maintenance_set.start",
"publisher_id":"ironic-api.hostname02"
}
```

ironic-conductor notifications

Node console notifications

These notifications are emitted by the ironic-conductor service when conductor service starts or stops console for the node. The notification event types for a node console are:

- `baremetal.node.console_set.start`
- `baremetal.node.console_set.end`
- `baremetal.node.console_set.error`
- `baremetal.node.console_restore.start`
- `baremetal.node.console_restore.end`
- `baremetal.node.console_restore.error`

`console_set` action is used when start or stop console is initiated. The `console_restore` action is used when the console was already enabled, but a driver must restart the console because an ironic-conductor was restarted. This may also be sent when an ironic-conductor takes over a node that was being managed by another ironic-conductor. start and end notifications have INFO level, error has ERROR. Example of node console notification:


```
{
 "priority": "info",
 "payload":{
     "ironic_object.namespace":"ironic",
     "ironic_object.name":"NodePayload",
     "ironic_object.version":"1.15",
     "ironic_object.data":{
         "clean_step": None,
         "conductor_group": "",
         "console_enabled": True,
         "created_at": "2016-01-26T20:41:03+00:00",
         "deploy_step": None,
         "description": "my sample node",
         "driver": "ipmi",
         "extra": {},
         "inspection_finished_at": None,
         "inspection_started_at": None,
         "instance_info": {},
         "instance_uuid": None,
         "last_error": None,
         "lessee": None,
         "maintenance": False,
         "maintenance_reason": None,
         "fault": None,
         "bios_interface": "no-bios",
         "boot_interface": "pxe",
         "console_interface": "no-console",
         "deploy_interface": "direct",
         "inspect_interface": "no-inspect",
         "management_interface": "ipmitool",
```


(continues on next page)

(continued from previous page)


```
        "network_interface": "flat",
        "power_interface": "ipmitool",
        "raid_interface": "no-raid",
        "rescue_interface": "no-rescue",
        "storage_interface": "noop",
        "vendor_interface": "no-vendor",
        "name": None,
        "owner": None,
        "power_state": "power off",
        "properties": {
            "memory_mb":  4096,
            "cpu_arch":  "x86_64",
            "local_gb":  10,
            "cpus":  8},
        "protected": False,
        "protected_reason": None,
        "provision_state": "available",
        "provision_updated_at": "2016-01-27T20:41:03+00:00",
        "resource_class": None,
        "retired": None,
        "retired_reason": None,
        "target_power_state": None,
        "target_provision_state": None,
        "traits": [
            "CUSTOM_TRAIT1",
            "HW_CPU_X86_VMX"],
        "updated_at": "2016-01-27T20:41:03+00:00",
        "uuid": "1be26c0b-03f2-4d2e-ae87-c02d7f33c123"
    }
},
"event_type":"baremetal.node.console_set.end",
"publisher_id":"ironic-conductor.hostname01"
}
```


baremetal.node.power_set

- `baremetal.node.power_set.start` is emitted by the ironic-conductor service when it begins a power state change. It has notification level info.
- `baremetal.node.power_set.end` is emitted when ironic-conductor successfully completes a power state change task. It has notification level info.
- `baremetal.node.power_set.error` is emitted by ironic-conductor when it fails to set a nodes power state. It has notification level error. This can occur when ironic fails to retrieve the old power state prior to setting the new one on the node, or when it fails to set the power state if a change is requested.

Here is an example payload for a notification with this event type. The to_power payload field indicates the power state to which the ironic-conductor is attempting to change the node:

```
{
 "priority": "info",
 "payload":{
     "ironic_object.namespace":"ironic",
     "ironic_object.name":"NodeSetPowerStatePayload",
     "ironic_object.version":"1.15",
     "ironic_object.data":{
         "clean_step": None,
         "conductor_group": "",
         "console_enabled": False,
         "created_at": "2016-01-26T20:41:03+00:00",
         "deploy_step": None,
         "description": "my sample node",
         "driver": "ipmi",
         "extra": {},
         "inspection_finished_at": None,
         "inspection_started_at": None,
         "instance_uuid": "d6ea00c1-1f94-4e95-90b3-3462d7031678",
         "last_error": None,
         "lessee": None,
         "maintenance": False,
         "maintenance_reason": None,
         "fault": None,
         "bios_interface": "no-bios",
         "boot_interface": "pxe",
         "console_interface": "no-console",
         "deploy_interface": "direct",
         "inspect_interface": "no-inspect",
         "management_interface": "ipmitool",
         "network_interface": "flat",
         "power_interface": "ipmitool",
         "raid_interface": "no-raid",
         "rescue_interface": "no-rescue",
         "storage_interface": "noop",
         "vendor_interface": "no-vendor",
         "name": None,
         "owner": None,
         "power_state": "power off",
         "properties": {
             "memory_mb":  4096,
             "cpu_arch":  "x86_64",
             "local_gb":  10,
             "cpus":  8},
         "protected": False,
         "protected_reason": None
         "provision_state": "available",
         "provision_updated_at": "2016-01-27T20:41:03+00:00",
         "resource_class": None,
         "retired": None,
         "retired_reason": None,
```


(continues on next page)

(continued from previous page)

```
        "target_power_state": None,
        "target_provision_state": None,
        "traits": [
            "CUSTOM_TRAIT1",
            "HW_CPU_X86_VMX"],
        "updated_at": "2016-01-27T20:41:03+00:00",
        "uuid": "1be26c0b-03f2-4d2e-ae87-c02d7f33c123",
        "to_power": "power on"
    }
},
"event_type":"baremetal.node.power_set.start",
"publisher_id":"ironic-conductor.hostname01"
}
```

baremetal.node.power_state_corrected

- `baremetal.node.power_state_corrected.success` is emitted by ironic-conductor when the power state on the baremetal hardware is different from the previous known power state of the node and the database is corrected to reflect this new power state. It has notification level info.

Here is an example payload for a notification with this event_type. The from_power payload field indicates the previous power state on the node, prior to the correction:

```
{
"priority": "info",
"payload":{
    "ironic_object.namespace":"ironic",
    "ironic_object.name":"NodeCorrectedPowerStatePayload",
    "ironic_object.version":"1.15",
    "ironic_object.data":{
        "clean_step": None,
        "conductor_group": "",
        "console_enabled": False,
        "created_at": "2016-01-26T20:41:03+00:00",
        "deploy_step": None,
        "description": "my sample node",
        "driver": "ipmi",
        "extra": {},
        "inspection_finished_at": None,
        "inspection_started_at": None,
        "instance_uuid": "d6ea00c1-1f94-4e95-90b3-3462d7031678",
        "last_error": None,
        "lessee": None,
        "maintenance": False,
        "maintenance_reason": None,
        "fault": None,
        "bios_interface": "no-bios",
        "boot_interface": "pxe",
        "console_interface": "no-console",
        "deploy_interface": "direct",
```

(continues on next page)

(continued from previous page)


```
        "inspect_interface": "no-inspect",
        "management_interface": "ipmitool",
        "network_interface": "flat",
        "power_interface": "ipmitool",
        "raid_interface": "no-raid",
        "rescue_interface": "no-rescue",
        "storage_interface": "noop",
        "vendor_interface": "no-vendor",
        "name": None,
        "owner": None,
        "power_state": "power off",
        "properties": {
            "memory_mb":  4096,
            "cpu_arch":  "x86_64",
            "local_gb":  10,
            "cpus":  8},
        "protected": False,
        "protected_reason": None,
        "provision_state": "available",
        "provision_updated_at": "2016-01-27T20:41:03+00:00",
        "resource_class": None,
        "retired": None,
        "retired_reason": None,
        "target_power_state": None,
        "target_provision_state": None,
        "traits": [
            "CUSTOM_TRAIT1",
            "HW_CPU_X86_VMX"],
        "updated_at": "2016-01-27T20:41:03+00:00",
        "uuid": "1be26c0b-03f2-4d2e-ae87-c02d7f33c123",
        "from_power": "power on"
    }
},
"event_type":"baremetal.node.power_state_corrected.success",
"publisher_id":"ironic-conductor.cond-hostname02"
}
```


baremetal.node.provision_set

- `baremetal.node.provision_set.start` is emitted by the ironic-conductor service when it begins a provision state transition. It has notification level INFO.
- `baremetal.node.provision_set.end` is emitted when ironic-conductor successfully completes a provision state transition. It has notification level INFO.
- `baremetal.node.provision_set.success` is emitted when ironic-conductor successfully changes provision state instantly, without any intermediate work required (example is AVAILABLE to MANAGEABLE). It has notification level INFO.
- `baremetal.node.provision_set.error` is emitted by ironic-conductor when it changes provision state as result of error event processing. It has notification level ERROR.

Here is an example payload for a notification with this event type. The previous_provision_state and previous_target_provision_state payload fields indicate a nodes provision states before state change, event is the FSM (finite state machine) event that triggered the state change:


```
{
 "priority": "info",
 "payload":{
     "ironic_object.namespace":"ironic",
     "ironic_object.name":"NodeSetProvisionStatePayload",
     "ironic_object.version":"1.16",
     "ironic_object.data":{
         "clean_step": None,
         "conductor_group": "",
         "console_enabled": False,
         "created_at": "2016-01-26T20:41:03+00:00",
         "deploy_step": None,
         "description": "my sample node",
         "driver": "ipmi",
         "driver_internal_info": {
             "is_whole_disk_image": True},
         "extra": {},
         "inspection_finished_at": None,
         "inspection_started_at": None,
         "instance_info": {},
         "instance_uuid": None,
         "last_error": None,
         "lessee": None,
         "maintenance": False,
         "maintenance_reason": None,
         "fault": None,
         "bios_interface": "no-bios",
         "boot_interface": "pxe",
         "console_interface": "no-console",
         "deploy_interface": "direct",
         "inspect_interface": "no-inspect",
         "management_interface": "ipmitool",
         "network_interface": "flat",
         "power_interface": "ipmitool",
         "raid_interface": "no-raid",
         "rescue_interface": "no-rescue",
         "storage_interface": "noop",
         "vendor_interface": "no-vendor",
         "name": None,
         "owner": None,
         "power_state": "power off",
         "properties": {
             "memory_mb":  4096,
             "cpu_arch":  "x86_64",
             "local_gb":  10,
             "cpus":  8},
         "protected": False,
```


(continues on next page)

(continued from previous page)

```
        "protected_reason": None,
        "provision_state": "deploying",
        "provision_updated_at": "2016-01-27T20:41:03+00:00",
        "resource_class": None,
        "retired": None,
        "retired_reason": None,
        "target_power_state": None,
        "target_provision_state": "active",
        "traits": [
            "CUSTOM_TRAIT1",
            "HW_CPU_X86_VMX"],
        "updated_at": "2016-01-27T20:41:03+00:00",
        "uuid": "1be26c0b-03f2-4d2e-ae87-c02d7f33c123",
        "previous_provision_state": "available",
        "previous_target_provision_state": None,
        "event": "deploy"
    }
 },
 "event_type":"baremetal.node.provision_set.start",
 "publisher_id":"ironic-conductor.hostname01"
}
```

4.2.13 Node Multi-Tenancy

This guide explains the steps needed to enable node multi-tenancy. This feature enables non-admins to perform API actions on nodes, limited by policy configuration. The Bare Metal service supports two kinds of non-admin users:

- Owner: owns specific nodes and performs administrative actions on them
- Lessee: receives temporary and limited access to a node

Setting the Owner and Lessee

Non-administrative access to a node is controlled through a nodes `owner` or `lessee` attribute:

```
baremetal node set --owner 080925ee2f464a2c9dce91ee6ea354e2  node-7
baremetal node set --lessee 2a210e5ff114c8f2b6e994218f51a904  node-10
```

Ironics API automatically grants visibility and access to these nodes when a user request comes in with a project scope, i.e. credentials which indicate the user is in a user within a project.

In older versions of Ironic, this functionality had to be manually configured and then set via custom policies, but now it is automatically available in modern versions of Ironic.

4.2.14 Booting a Ramdisk or an ISO

Ironic supports booting a user provided ramdisk or an ISO image (starting with the Victoria release) instead of deploying a node. Most commonly this is performed when an instance is booted via PXE, iPXE or Virtual Media, with the only local storage contents being those in memory. It is supported by `pxe`, `ipxe`, `redfish-virtual-media` and `ilo-virtual-media` boot interfaces.

Configuration

Ramdisk/ISO boot requires using the `ramdisk` deploy interface. It is enabled by default starting with the Zed release cycle. On an earlier release, it must be enabled explicitly:

```
[DEFAULT]
...
enabled_deploy_interfaces = direct,ramdisk
...
```

Once enabled and the conductor(s) have been restarted, the interface can be set upon creation of a new node:

```
baremetal node create --driver <driver> \
    --deploy-interface ramdisk \
    --boot-interface ipxe
```

or update an existing node:

```
baremetal node set <NODE> --deploy-interface ramdisk
```

You can also use it with *redfish virtual media* instead of iPXE.

Alternatively, if the node uses the `autodetect` deploy interface, the ramdisk interface is selected automatically when `kernel` and `ramdisk` are set in `instance_info` (without an `image_source`), or when `boot_iso` is set. To enable this, add `ramdisk` to the `autodetect_deploy_interfaces` configuration option. See *Autodetect deploy* for details.

Creating a ramdisk

A ramdisk can be created using the `ironic-ramdisk-base` element from ironic-python-agent-builder, e.g. with Debian:

```
export ELEMENTS_PATH=/opt/stack/ironic-python-agent-builder/dib
disk-image-create -o /output/ramdisk \
    debian-minimal ironic-ramdisk-base openssh-server dhcp-all-interfaces
```

You should consider using the following elements:

- openssh-server to install the SSH server since its not provided by default by some minimal images.
- devuser or dynamic-login to provide SSH access.
- dhcp-all-interfaces or simple-init to configure networking.

The resulting files (`/output/ramdisk.kernel` and `/output/ramdisk.initramfs` in this case) can then be used when *Booting a ramdisk*.

Booting a ramdisk

Pass the kernel and ramdisk as normally, also providing the ramdisk as an image source, for example,

```
baremetal node set <NODE> \
    --instance-info kernel=http://path/to/ramdisk.kernel \
    --instance-info ramdisk=http://path/to/ramdisk.initramfs
baremetal node deploy <NODE>
```

Booting an ISO

The `ramdisk` deploy interface can also be used to boot an ISO image. For example,

```
baremetal node set <NODE> \
    --instance-info boot_iso=http://path/to/boot.iso
baremetal node deploy <NODE>
```

Note

While this interface example utilizes a HTTP URL, as with all fields referencing file artifacts in the `instance_info` field, a user is able to request a file path URL, or an HTTPS URL, or as a Glance Image Service object UUID.

Warning

This feature, when utilized with the `ipxe boot_interface`, will only allow a kernel and ramdisk to be booted from the supplied ISO file. Any additional contents, such as additional ramdisk contents or installer package files will be unavailable after the boot of the Operating System. Operators wishing to leverage this functionality for actions such as OS installation should explore use of the standard `ramdisk deploy_interface` along with the `instance_info/kernel_append_params` setting to pass arbitrary settings such as a mirror URL for the initial ramdisk to load data from. This is a limitation of iPXE and the overall boot process of the operating system where memory allocated by iPXE is released.

When choosing your boot ISO, your ISO image will need to be sufficient to boot the hardware under normal conditions. For example, if the ISO is only compatible with BIOS booting, then a host in UEFI mode will not boot. This is not a limitation of Ironic, but an architectural limitation.

By default the Bare Metal service will cache the ISO locally and serve from its HTTP server. If you want to avoid that, set the following:

```
baremetal node set <NODE> \
    --instance-info ramdisk_image_download_source=http
```

ISO images are also cached across deployments, similarly to how it is done for normal instance images. The URL together with the last modified response header are used to determine if an image needs updating.

Using with Nova (Glance images)

When deploying ramdisk nodes through Nova, the user creates a Glance image with `kernel_id` and `ramdisk_id` properties the same AMI-style pattern used for partition images. Nova sets `image_source` in the nodes `instance_info` and Ironic resolves the kernel and ramdisk from Glance automatically.

1. Upload the kernel to Glance:

   ```
   openstack image create my-ramdisk-kernel --public \
     --disk-format raw --container-format bare \
     --file my-ramdisk.kernel
   ```

   Store the image UUID as `MY_KERNEL_UUID`.

2. Upload the ramdisk (initramfs) to Glance:

```
openstack image create my-ramdisk-initrd --public \
  --disk-format raw --container-format bare \
  --file my-ramdisk.initramfs
```

   Store the image UUID as `MY_RAMDISK_UUID`.

3. Create the Glance image with `kernel_id` and `ramdisk_id` properties. Since the ramdisk deploy interface does not write to disk, the disk image file is not used an empty placeholder file is sufficient.

   Include `ironic_ramdisk_deploy=True` so that the deploy interface autodetect mechanism can distinguish this image from a partition image (which also carries `kernel_id` / `ramdisk_id`):

```
touch /tmp/placeholder
openstack image create my-ramdisk-image --public \
  --disk-format raw --container-format bare \
  --property kernel_id=$MY_KERNEL_UUID \
  --property ramdisk_id=$MY_RAMDISK_UUID \
  --property ironic_ramdisk_deploy=True \
  --file /tmp/placeholder
```

4. Boot a Nova instance using this image:

```
openstack server create --image my-ramdisk-image \
  --flavor my-baremetal-flavor my-ramdisk-instance
```

Alternative: Using a boot ISO in Glance

Instead of separate kernel and ramdisk images, you can reference a single boot ISO stored in Glance via the `boot_iso_id` property:

1. Upload the boot ISO to Glance:

```
openstack image create my-ramdisk-boot-iso --public \
  --disk-format iso --container-format bare \
  --file my-ramdisk.iso
```

   Store the image UUID as `MY_BOOT_ISO_UUID`.

2. Create the Glance image with the `boot_iso_id` property:

```
touch /tmp/placeholder
openstack image create my-ramdisk-image --public \
  --disk-format raw --container-format bare \
  --property boot_iso_id=$MY_BOOT_ISO_UUID \
  --file /tmp/placeholder
```

3. Boot a Nova instance as above:

```
openstack server create --image my-ramdisk-image \
  --flavor my-baremetal-flavor my-ramdisk-instance
```

Note

The disk image file attached to the Glance image is **not** used by the ramdisk deploy interface. Only the kernel and ramdisk referenced by `kernel_id` / `ramdisk_id`, or the ISO referenced by `boot_iso_id`, are booted.

Limitations

The intended use case is for advanced scientific and ephemeral workloads where the step of writing an image to the local storage is not required or desired. As such, this interface does come with several caveats:

- Configuration drives are not supported with network boot, only with Redfish virtual media.
- Disk image contents are not written to the bare metal node.
- Users and Operators who intend to leverage this interface should expect to leverage a metadata service, custom ramdisk images, or the `instance_info/ramdisk_kernel_arguments` parameter to add options to the kernel boot command line.
- When using PXE/iPXE boot, bare metal nodes must continue to have network access to PXE and iPXE network resources. This is contrary to most tenant networking enabled configurations where this access is restricted to the provisioning and cleaning networks
- As with all deployment interfaces, automatic cleaning of the node will still occur with the contents of any local storage being wiped between deployments.

Common options

Disable persistent boot device for ramdisk iso boot

For iso boot, Ironic sets the boot target to continuously boot from the iso attached over virtual media. This behaviour may not always be desired e.g. if the vmedia is installing to hard drive and then rebooting. In order to instead set the virtual media to be one time boot Ironic provides the `force_persistent_boot_device` flag in the nodes `driver_info`. Which can be set to `Never`:

```
$ openstack baremetal node set --driver-info force_persistent_boot_device=
↪'Never' <node>
```

4.2.15 Hardware Burn-in

Overview

Workflows to onboard new hardware often include a stress-testing step to provoke early failures and to avoid that these load-triggered issues only occur when the nodes have already moved to production. These `burn-in` tests typically include CPU, GPU, memory, disk, and network. With the Xena release, Ironic supports such tests as part of the cleaning framework.

The burn-in steps rely on standard tools such as stress-ng for CPU and memory, fio for disk and network, or gpu-burn for GPU tests. The burn-in cleaning steps are part of the generic hardware manager in the Ironic Python Agent (IPA) and therefore the agent ramdisk does not need to be bundled with a specific IPA hardware manager to have them available.

Each burn-in step accepts (or in the case of network: needs) some basic configuration options, mostly to limit the duration of the test and to specify the amount of resources to be used. The options are set on a

nodes `driver-info` and prefixed with `agent_burnin_`. The options available for the individual tests will be outlined below.

CPU burn-in

The options, following a `agent_burnin_` + stress-ng stressor (`cpu`) + stress-ng option schema, are:

- `agent_burnin_cpu_timeout` (default: 24 hours)
- `agent_burnin_cpu_cpu` (default: 0, meaning all CPUs)

to limit the overall runtime and to pick the number of CPUs to stress.

For instance, to limit the time of the CPU burn-in to 10 minutes do:

```
baremetal node set --driver-info agent_burnin_cpu_timeout=600 \
    $NODE_NAME_OR_UUID
```

Then launch the test with:

```
baremetal node clean --clean-steps '[{"step": "burnin_cpu", \
    "interface": "deploy"}]' $NODE_NAME_OR_UUID
```

Memory burn-in

The options, following a `agent_burnin_` + stress-ng stressor (`vm`) + stress-ng option schema, are:

- `agent_burnin_vm_timeout` (default: 24 hours)
- `agent_burnin_vm_vm-bytes` (default: 98%)

to limit the overall runtime and to set the fraction of RAM to stress.

For instance, to limit the time of the memory burn-in to 1 hour and the amount of RAM to be used to 75% run:

```
baremetal node set --driver-info agent_burnin_vm_timeout=3600 \
    $NODE_NAME_OR_UUID
baremetal node set --driver-info agent_burnin_vm_vm-bytes=75% \
    $NODE_NAME_OR_UUID
```

Then launch the test with:

```
baremetal node clean --clean-steps '[{"step": "burnin_memory", \
    "interface": "deploy"}]' $NODE_NAME_OR_UUID
```

Disk burn-in

The options, following a `agent_burnin_` + fio stressor (`fio_disk`) + fio option schema, are:

- agent_burnin_fio_disk_runtime (default: 0, meaning no time limit)
- agent_burnin_fio_disk_loops (default: 4)

to set the time limit and the number of iterations when going over the disks.

For instance, to limit the number of loops to 2 set:

```
baremetal node set --driver-info agent_burnin_fio_disk_loops=2 \
    $NODE_NAME_OR_UUID
```

Then launch the test with:

```
baremetal node clean --clean-steps '[{"step": "burnin_disk", \
    "interface": "deploy"}]' $NODE_NAME_OR_UUID
```

To launch a parallel SMART self-test on all devices after the disk burn-in (which will fail the step if any of the tests fail), set:

```
baremetal node set --driver-info agent_burnin_fio_disk_smart_test=True \
    $NODE_NAME_OR_UUID
```

Network burn-in

Burning in the network needs a little more config since we need a pair of nodes to perform the test. This pairing can be done either in a static way, i.e. pairs are defined upfront, or dynamically via a distributed coordination backend which orchestrates the pair matching. While the static approach is more predictable in terms of which nodes test each other, the dynamic approach avoids nodes being blocked in case there are issues with servers and simply pairs all available nodes.

Static network burn-in configuration

To define pairs of nodes statically, each node can be assigned a `agent_burnin_fio_network_config` JSON which requires a `role` field (values: `reader`, `writer`) and a `partner` field (value is the hostname of the other node to test), like:

```
baremetal node set --driver-info agent_burnin_fio_network_config= \
    '{"role": "writer", "partner": "$HOST2"}' $NODE_NAME_OR_UUID1
baremetal node set --driver-info agent_burnin_fio_network_config= \
    '{"role": "reader", "partner": "$HOST1"}' $NODE_NAME_OR_UUID2
```

Dynamic network burn-in configuration

To use dynamic pair matching, a coordination backend is used via tooz. The corresponding backend URL then needs to be added to the node, e.g. for a Zookeeper backend it would look similar to:

```
baremetal node set --driver-info \
    agent_burnin_fio_network_pairing_backend_url= \
    'zookeeper://zk1.xyz.com:2181,zk2.xyz.com:2181,zk3.xyz.com:2181' \
    $NODE_NAME_OR_UUID1
baremetal node set --driver-info \
    agent_burnin_fio_network_pairing_backend_url= \
    'zookeeper://zk1.xyz.com:2181,zk2.xyz.com:2181,zk3.xyz.com:2181' \
    $NODE_NAME_OR_UUID2
...
baremetal node set --driver-info \
    agent_burnin_fio_network_pairing_backend_url= \
    'zookeeper://zk1.xyz.com:2181,zk2.xyz.com:2181,zk3.xyz.com:2181' \
    $NODE_NAME_OR_UUIDN
```

Different deliveries or network ports can be separated by creating different rooms on the backend with:

```
baremetal node set --driver-info \
agent_burnin_fio_network_pairing_group_name=$DELIVERY $NODE_NAME_OR_UUID
```

This allows to control which nodes (or interfaces) connect with which other nodes (or interfaces).

Launching network burn-in

In addition and similar to the other tests, there is a runtime option to be set (only on the writer):

```
baremetal node set --driver-info agent_burnin_fio_network_runtime=600 \
    $NODE_NAME_OR_UUID
```

The actual network burn-in can then be launched with:

```
baremetal node clean --clean-steps '[{"step": "burnin_network",\
    "interface": "deploy"}]' $NODE_NAME_OR_UUID1
baremetal node clean --clean-steps '[{"step": "burnin_network",\
    "interface": "deploy"}]' $NODE_NAME_OR_UUID2
```

Both nodes will wait for the other node to show up and block while waiting. If the partner does not show up, the cleaning timeout will step in.

GPU burn-in

The GPU burn-in tests come in two parts:

- Check that the correct number of GPUs are visible by the operating system
- GPU burn-in test using gpu-burn

The first check will only be performed if the option `agent_burnin_gpu_count` has been set to a value above 0.

The options, following a `agent_burnin_` + gpu stressor (`gpu`) option schema, are:

- `agent_burnin_gpu_install_dir` (default: /opt/gpu-burn)
- `agent_burnin_gpu_timeout` (default: 24 hours)
- `agent_burnin_gpu_memory` (default: 95%)
- `agent_burnin_gpu_count` (default: 0, The GPU count check is disabled by default)

For instance, in order to limit the time of the GPU burn-in to 10 minutes do:

```
baremetal node set --driver-info agent_burnin_gpu_timeout=600 \
    $NODE_NAME_OR_UUID
```

Then launch the test with:

```
baremetal node clean --clean-steps '[{"step": "burnin_gpu", \
    "interface": "deploy"}]' $NODE_NAME_OR_UUID
```

Logging

Since most of the burn-in steps are also providing information about the performance of the stressed components, keeping this information for verification or acceptance purposes may be desirable. By default, the output of the burn-in tools goes to the journal of the Ironic Python Agent and is therefore sent back as an archive to the conductor. In order to consume the output of the burn-in steps more easily, or even in real time, the nodes can be configured to store the output of the individual steps to files in the ramdisk (from where they can be picked up by a logging pipeline).

The configuration of the output file is done via one of `agent_burnin_cpu_outputfile`, `agent_burnin_vm_outputfile`, `agent_burnin_fio_disk_outputfile`, and `agent_burnin_fio_network_outputfile` parameters which need to be added to a node like:

```
baremetal node set --driver-info agent_burnin_cpu_outputfile=\
    '/var/log/burnin.cpu' $NODE_NAME_OR_UUID
```

Additional Information

All tests can be aborted at any moment with

```
baremetal node abort $NODE_NAME_OR_UUID
```

One can also launch multiple tests which will be run in sequence, e.g.:

```
baremetal node clean --clean-steps '[{"step": "burnin_cpu",\
   "interface": "deploy"}, {"step": "burnin_memory",\
   "interface": "deploy"}]' $NODE_NAME_OR_UUID
```

If desired, configuring `fast-track` may be helpful here as it allows to keep the node up between consecutive calls of `baremetal node clean`.

4.2.16 Vendor Passthru

The bare metal service allows drivers to expose vendor-specific API known as *vendor passthru*.

Node Vendor Passthru

Drivers may implement a passthrough API, which is accessible via the `/v1/nodes/<Node UUID or Name>/vendor_passthru?method={METHOD}` endpoint. Beyond basic checking, Ironic does not introspect the message body and simply passes it through to the relevant driver.

A method:

- can support one or more HTTP methods (for example, GET, POST)
- is asynchronous or synchronous
  - For asynchronous methods, a 202 (Accepted) HTTP status code is returned to indicate that the request was received, accepted and is being acted upon. No body is returned in the response.
  - For synchronous methods, a 200 (OK) HTTP status code is returned to indicate that the request was fulfilled. The response may include a body.
- can require an exclusive lock on the node. This only occurs if the method doesnt specify require_exclusive_lock=False in the decorator. If an exclusive lock is held on the node, other requests for the node will be delayed and may fail with an HTTP 409 (Conflict) error code.

This endpoint exposes a nodes driver directly, and as such, it is expressly not part of Ironics standard REST API. There is only a single HTTP endpoint exposed, and the semantics of the message body are determined solely by the driver. Ironic makes no guarantees about backwards compatibility; this is solely up to the discretion of each drivers author.

To get information about all the methods available via the vendor_passthru endpoint for a particular node, use CLI:

```
$ baremetal node passthru list <redfish-node>
+----------------------+-----------------------+-------+--------------------
↪--------------------------------------------------------------------------
↪--------------------------------------------------------------------------
↪---------------------------+-----------------------+
| Name                   | Supported HTTP methods | Async | Description       ␣
↪                                                                          ␣
↪                                                                          ␣
↪                            | Response is attachment |
+----------------------+-----------------------+-------+--------------------
↪--------------------------------------------------------------------------
↪--------------------------------------------------------------------------
↪---------------------------+-----------------------+
| create_subscription    | POST                  | False | Creates a␣
↪subscription on a node. Required argument: a dictionary of {'destination':
↪'destination_url'}                                                        ␣
↪                                        | False                 |
| delete_subscription    | DELETE                | False | Delete a␣
↪subscription on a node. Required argument: a dictionary of {'id':
↪'subscription_bmc_id'}                                                    ␣
↪                                                 | False                |
| eject_vmedia           | POST                  | True  | Eject a virtual␣
↪media device. If no device is provided then all attached devices will be␣
↪ejected. Optional arguments: 'boot_device' - the boot device to eject,␣
↪either 'cd', 'dvd', 'usb', or 'floppy' | False                 |
| get_all_subscriptions | GET                    | False | Returns all␣
↪subscriptions on the node.                                                ␣
↪                                                                          ␣
↪                                      | False                 |
| get_subscription       | GET                   | False | Get a subscription␣
↪on the node. Required argument: a dictionary of {'id': 'subscription_bmc_id
↪'}                                                                        ␣
↪                                | False                 |
+----------------------+-----------------------+-------+--------------------
↪--------------------------------------------------------------------------
↪--------------------------------------------------------------------------
↪---------------------------+-----------------------+
```

The response will contain information for each method, such as the methods name, a description, the HTTP methods supported, and whether its asynchronous or synchronous.

You can call a method with CLI, for example:

```
$ baremetal node passthru call <redfish-node> eject_vmedia
```

Driver Vendor Passthru

Drivers may implement an API for requests not related to any node, at `/v1/drivers/<driver name>/vendor_passthru?method={METHOD}`.

A method:

- can support one or more HTTP methods (for example, GET, POST)
- is asynchronous or synchronous
  - For asynchronous methods, a 202 (Accepted) HTTP status code is returned to indicate that the request was received, accepted and is being acted upon. No body is returned in the response.
  - For synchronous methods, a 200 (OK) HTTP status code is returned to indicate that the request was fulfilled. The response may include a body.

Note

Unlike methods in *Node Vendor Passthru*, a request does not lock any resource, so it will not delay other requests and will not fail with an HTTP 409 (Conflict) error code.

Ironic makes no guarantees about the semantics of the message BODY sent to this endpoint. That is left up to each drivers author.

To get information about all the methods available via the driver vendor_passthru endpoint, use CLI:

```
$ baremetal driver passthru list redfish
```

The response will contain information for each method, such as the methods name, a description, the HTTP methods supported, and whether its asynchronous or synchronous.

Warning

Currently only the methods available in the default interfaces of the hardware type are available.

You can call a method with CLI, for example:

```
$ baremetal driver passthru call <driver> <method>
```

4.2.17 Node servicing

Overview

In order to better enable operators to modify existing nodes, Ironic has introduced the model of Node Servicing, where you can take a node in `active` state, modify it using steps similar to Deploy Steps or manual cleaning through the Cleaning subsystem.

For more information on cleaning, please see *Node cleaning*.

Major differences

Service steps do not contain an automatic execution model, which is intrinisc to the standard deployment and automated cleaning workflows. This *may* change at some point in the future.

This also means that while a priority value *can* be supplied, it is not presently utilized.

Similarities to Cleaning and Deployment

Similar to Clean and Deploy steps, when invoked an operator can validate the current running steps by viewing the `driver_internal_info` field looking for a `service_steps` field. The *current* step being executed can be viewed using the baremetal node `service_step` field, which is a top level field.

Service steps are internally decorated on driver interface methods utilizing decorator. This means service steps do not *automatically* expose clean and deploy steps to be executed at any time. The Ironic development team took a cautious and intentional approach behind methods which are decorated. Besides, some clean and deployment steps are geared explicitly for operating in that mode, and would not be suitable to be triggered outside of the original workflow it was designed for use in.

Available Steps

Executing Service Steps

In order for manual cleaning to work, you may need to configure a *Servicing Network*.

Starting manual servicing via API

Servicing can only be performed when a node is in the `active` provision state. The REST API request to initiate it is available in API version 1.87 and higher:

```
PUT /v1/nodes/<node_ident>/states/provision
```

(Additional information is available here.)

This API will allow operators to put a node directly into `servicing` provision state from `active` provision state via target: service. The PUT will also require the argument service_steps to be specified. This is an ordered list of steps. A step is represented by a dictionary (JSON), in the form:

```
{
    "interface": "<interface>",
    "step": "<name of step>",
    "args": {"<arg1>": "<value1>", ..., "<argn>": <valuen>}
}
```

The interface and step keys are required for all steps. If a servicing step method takes keyword arguments, the args key may be specified. It is a dictionary of keyword variable arguments, with each keyword-argument entry being <name>: <value>.

If any step is missing a required keyword argument, servicing will not be performed and the node will be put in `service failed` provision state with an appropriate error message.

If, during the servicing process, a service step determines that it has incorrect keyword arguments, all earlier steps will be performed and then the node will be put in `service failed` provision state with an appropriate error message.

An example of the request body for this API:

```
{
  "target":"service",
  "service_steps": [{
    "interface": "raid",
    "step": "apply_configuration",
    "args": {"create_nonroot_volumes": True}
  },
  {
    "interface": "vendor",
    "step": "send_raw",
    "args": {"raw_bytes": "0x00 0x00 0x00 0x00"}
  }]
}
```

In the above example, the nodes RAID interface would apply the set RAID configuration, and then the vendor interfaces `send_raw` step would be called to send a raw command to the BMC. Please note, `send_raw` is only available for the `ipmi` hardware type.

Alternatively, you can specify a runbook instead of service_steps:

```
{
  "target":"service",
  "runbook": "<runbook_name_or_uuid>"
}
```

The specified runbook must match one of the nodes traits to be used.

Starting servicing via openstack baremetal CLI

Servicing is available via the `baremetal node service` command, starting with Bare Metal API version 1.87.

The argument `--service-steps` must be specified. Its value is one of:

- a JSON string
- path to a JSON file whose contents are passed to the API
- -, to read from stdin. This allows piping in the service steps. Using - to signify stdin is common in Unix utilities.

Examples of doing this with a JSON string:

```
baremetal node service <node> \
    --service-steps '[{"interface": "deploy", "step": "example_task"}]'
```

Or with a file:

```
baremetal node service <node> \
    --service-steps my-service-steps.txt
```

Or with stdin:

```
cat my-clean-steps.txt | baremetal node service <node> \
    --service-steps -
```

Using Runbooks for Servicing

Similar to manual cleaning, you can use runbooks for node servicing. Runbooks provide a predefined list of service steps associated with nodes via traits.

To use a runbook for servicing:

> baremetal node service <node> runbook <runbook_name_or_uuid>

Ensure that the runbook matches one of the nodes traits before using it for servicing. For more information on the runbook API usage, see *Runbooks for Cleaning & Servicing*.

Available Steps in Ironic

ipmi hardware type

vendor.send_raw

This step is covered in the *IPMI driver* documentation and is usable as a service step in addition to a deploy step.

redfish hardware type

bios.apply_configuration

This is covered in the *BIOS Configuration* configuration documentation as it started as a cleaning step. It is a standardized cross-interface name.

management.update_firmware

This step is covered in the *Redfish driver* and is intended to facilitate firmware updates via the BMC.

raid.apply_configuration

This step is covered in the *Redfish driver* and is intended to facilitate applying raid configuration.

raid.delete_configuration

This step is covered in the *Redfish driver* and is intended to delete configuration.

Agent

raid.apply_configuration

This is the standardized RAID passthrough interface for the agent, and can be leveraged like other RAID interfaces.

Neutron Network Interface

network.reattach_networking

This service step reattaches tenant networking on an active node without requiring a reboot or IPA deployment. This is useful for advanced networking scenarios such as VXLAN topology changes, network infrastructure migrations, or recovering from network binding failures.

Key characteristics:

- Does **not** require IPA ramdisk (`requires_ramdisk=False`)
- Does **not** reboot the instance
- Preserves VIF attachments while refreshing Neutron port bindings
- Supports Smart NIC ports with proper agent synchronization
- Typically completes in seconds

When to use:

- VXLAN network topology reconfiguration
- Physical network segment migrations
- Network binding state recovery
- Switch reconfiguration without downtime

Usage example:

```
$ openstack baremetal node service <node-uuid> \
    --service-steps '[{"interface": "network", "step": "reattach_networking"}]
↪'
```

For detailed troubleshooting guidance and additional information, see *Troubleshooting Ironic* section Reattaching networking on active instances.

Requirements:

- Node must be in `active` state
- Node must use `network_interface=neutron`
- Nodes ports must have existing tenant VIF attachments
- No arguments required

Available steps in Ironic-Python-Agent

> **Note**
>
> Steps available from the agent will be populated once support has merged in the agent to expose the steps to the ironic deployment.

Servicing Network

If you are using the Neutron DHCP provider (the default) you will also need to ensure you have configured a servicing network. This network will be used to boot the ramdisk for in-band service operations. This

setting is configured utilizing the *neutron.servicing_network* configuration parameter.

4.2.18 Building images for Windows

We can use `New-WindowsOnlineImage` in windows-openstack-imaging-tools tool as an option to create Windows images (whole disk images) corresponding boot modes which will support Windows NIC Teaming. And allow the utilization of link aggregation when the instance is spawned on hardware servers (Bare metals).

Requirements:

- A Microsoft Windows Server Operating System along with `Hyper-V virtualization` enabled, `PowerShell` version >=4 supported, `Windows Assessment and Deployment Kit`, in short `Windows ADK`.
- The Windows Server compatible drivers.
- Working git environment.

Preparation:

- Download a Windows Server 2012R2/ 2016 installation ISO.
- Install Windows Server 2012R2/ 2016 OS on the workstation PC along with following feature:
  - Enable Hyper-V virtualization.
  - Install PowerShell 4.0.
  - Install Git environment & import git proxy (if you have).
  - Create a new `Path` in the Microsoft Windows Server Operating System which support for submodule update via `git submodule update init` command:

```
- Variable name: Path
- Variable value: C:\Windows\System32\WindowsPowerShell\v1.0\;C:\
↪Program Files\Git\bin
```

  - Rename the virtual switch name in Windows Server 2012R2/ 2016 in `Virtual Switch Manager` into `external`.

Implementation:

- `Step 1`: Create folders: `C:\<folder_name_1>` where output images will be located, `C:\<folder_name_2>` where you need to place the necessary hardware drivers.
- `Step 2`: Copy and extract necessary hardware drivers in `C:\<folder_name_2>`.
- `Step 3`: Insert or burn Windows Server 2016 ISO to `D:\`.
- `Step 4`: Download `windows-openstack-imaging-tools` tools.

```
git clone https://github.com/cloudbase/windows-openstack-imaging-tools.git
```

- `Step 5`: Create & running script `create-windows-cloud-image.ps1`:

```
git submodule update --init
Import-Module WinImageBuilder.psm1
$windowsImagePath = "C:\<folder_name_1>\<output_file_name>.qcow2"
$VirtIOISOPath = "C:\<folder_name_1>\virtio.iso"
$virtIODownloadLink = "https://fedorapeople.org/groups/virt/virtio-win/
↪direct-downloads/archive-virtio/virtio-win-0.1.133-2/virtio-win.iso"
(New-Object System.Net.WebClient).DownloadFile($virtIODownloadLink,
↪$VirtIOISOPath)
$wimFilePath = "D:\sources\install.wim"
$extraDriversPath = "C:\<folder_name_2>\"
$image = (Get-WimFileImagesInfo -WimFilePath $wimFilePath)[1]
$switchName = 'external'
New-WindowsOnlineImage -WimFilePath $wimFilePath
  -ImageName $image.ImageName ` -WindowsImagePath $windowsImagePath -Type
↪'KVM' -ExtraFeatures @() `
  -SizeBytes 20GB -CpuCores 2 -Memory 2GB -SwitchName $switchName ` -
↪ProductKey $productKey -DiskLayout 'BIOS' `
  -ExtraDriversPath $extraDriversPath ` -InstallUpdates:$false -
↪AdministratorPassword 'Pa$$w0rd' `
  -PurgeUpdates:$true -DisableSwap:$true
```

After executing this command you will get two output files, first one being C:<folder_name_1><output_file_name>.qcow2, which is the resulting windows whole disk image and C:<folder_name_1>virtio.iso, which is virtio iso contains all the synthetic drivers for the KVM hypervisor.

See example_windows_images for more details and examples.

Note

We can change `SizeBytes`, `CpuCores`, and `Memory` depending on requirements.

4.2.19 Deploying without BMC Credentials

The Bare Metal service usually requires BMC credentials for all provisioning operations. Starting with the Victoria release series there is limited support for inspection, cleaning and deployments without the credentials.

Warning

This feature is experimental and only works in a limited scenario. When using it, you have to be prepared to provide BMC credentials in case of a failure or any non-supported actions.

How it works

The expected workflow is as follows:

1. The node is discovered by manually powering it on and gets the `manual-management` hardware type and `agent` power interface.

   If discovery is not used, a node can be enrolled through the API and then powered on manually.

2. The operator moves the node to `manageable`. It works because the `agent` power only requires to be able to connect to the agent.
3. The operator moves the node to `available`. Cleaning happens normally via the already running agent. If a reboot is needed, it is done by telling the agent to reboot the node in-band.
4. A user deploys the node. Deployment happens normally via the already running agent.
5. At the end of the deployment, the node is rebooted via the reboot command instead of power off+on.

Enabling

Fast-Track Deployment is a requirement for this feature to work. After enabling it, adds the `agent` power interface and the `manual-management` hardware type to the enabled list:

```
[DEFAULT]
enabled_hardware_types = manual-management
enabled_management_interfaces = noop
enabled_power_interfaces = agent

[deploy]
fast_track = true
```

As usual with the `noop` management, enable the networking boot fallback:

```
[pxe]
enable_netboot_fallback = true
```

Limitations

- Only the `noop` network interface is supported.
- Undeploy and rescue are not supported, you need to add BMC credentials first.
- If any errors happen in the process, recovery will likely require BMC credentials.
- Only rebooting is possible through the API, power on/off commands will fail.

4.2.20 Layer 3 or DHCP-less ramdisk booting

Booting nodes via PXE, while universally supported, suffers from one disadvantage: it requires a direct L2 connectivity between the node and the control plane for DHCP. Using virtual media it is possible to avoid not only the unreliable TFTP protocol but DHCP altogether.

When network data is provided for a node as explained below, the generated virtual media ISO will also serve as a configdrive, and the network data will be stored in the standard OpenStack location.

The simple-init element needs to be used when creating the deployment ramdisk. The Glean tool will look for a media labeled as `config-2`. If found, the network information from it will be read, and the nodes networking stack will be configured accordingly.

```
ironic-python-agent-builder -o /output/ramdisk \
    debian-minimal -e simple-init
```

Warning

Ramdisks based on distributions with NetworkManager require Glean 1.19.0 or newer to work.

Note

If desired, some interfaces can still be configured to use DHCP.

Hardware type support

This feature is known to work with the following hardware types:

- *Redfish* with `redfish-virtual-media` boot
- *iLO* with `ilo-virtual-media` boot

Configuring network data

When the Bare Metal service is running within OpenStack, no additional configuration is required - the network configuration will be fetched from the Network service.

Alternatively, the user can build and pass network configuration in the form of a network_data JSON to a node via the `network_data` field. Node-based configuration takes precedence over the configuration generated by the Network service and also works in standalone mode.

```
baremetal node set --network-data ~/network_data.json <node>
```

An example network data:

```
{
    "links": [
        {
            "id": "port-92750f6c-60a9-4897-9cd1-090c5f361e18",
            "type": "phy",
            "ethernet_mac_address": "52:54:00:d3:6a:71"
        }
    ],
    "networks": [
        {
            "id": "network0",
            "type": "ipv4",
            "link": "port-92750f6c-60a9-4897-9cd1-090c5f361e18",
            "ip_address": "192.168.122.42",
            "netmask": "255.255.255.0",
            "network_id": "network0",
            "routes": []
        }
    ],
    "services": []
}
```

Note

Some fields are redundant with the port information. Were looking into simplifying the format, but currently, all these fields are mandatory.

Youll need the deployed image to support network data, e.g. by pre-installing cloud-init or Glean on it (most cloud images have the former). Then you can provide the network data when deploying, for example:

```
baremetal node deploy <node> \
    --config-drive "{\"network_data\": $(cat ~/network_data.json)}"
```

Some first-boot services, such as Ignition, dont support network data. You can provide their configuration as part of user data instead:

```
baremetal node deploy <node> \
    --config-drive "{\"user_data\": \"... ignition config ...\"}"
```

Deploying outside of the provisioning network

If you need to combine traditional deployments using a provisioning network with virtual media deployments over L3, you may need to provide an alternative IP address for the remote nodes to connect to:

```
[deploy]
http_url = <HTTP server URL internal to the provisioning network>
external_http_url = <HTTP server URL with a routable IP address>
```

You may also need to override the callback URL, which is normally fetched from the service catalog or configured in the `[service_catalog]` section:

```
[deploy]
external_callback_url = <Bare Metal API URL with a routable IP address>
```

In case you need specific URLs for each node, you can use the `driver_info[external_http_url]` node property. When used it overrides the *deploy.http_url* and *deploy.external_http_url* settings in the configuration file.

```
baremetal node set node-0 \
  --driver-info external_http_url="<your_node_external_url>"
```

4.2.21 Deploying with anaconda deploy interface

Ironic supports deploying an OS with the anaconda installer. This anaconda deploy interface *ONLY* works with `pxe` and `ipxe` boot interfaces.

Configuration

The anaconda deploy interface is not enabled by default. To enable this, add `anaconda` to the value of the `enabled_deploy_interfaces` configuration option in ironic.conf. For example:

```
[DEFAULT]
...
enabled_deploy_interfaces = direct,anaconda
...
```

This change takes effect after all the ironic conductors have been restarted.

The default kickstart template is specified via the configuration option *anaconda.default_ks_template*. It is set to this ks.cfg.template but can be modified to be some other template.

```
[anaconda]
default_ks_template = /etc/ironic/ks.cfg.template
```

When creating an ironic node, specify `anaconda` as the deploy interface. For example:

```
baremetal node create --driver ipmi \
    --deploy-interface anaconda \
    --boot-interface ipxe
```

You can also set the anaconda deploy interface via `--deploy-interface` on an existing node:

```
baremetal node set <node> --deploy-interface anaconda
```

Creating an OS Image

While anaconda allows installing individual RPMs, the default kickstart file expects an OS tarball to be used as the OS image.

This `baremetal.yum` file contains all the yum/dnf commands that need to be run in order to generate the OS tarball. These commands install packages and package groups that need to be in the image:

```
group install 'Minimal Install'
install cloud-init
ts run
```

An OS tarball can be created using the following set of commands, along with the above `baremetal.yum` file:

```
export CHROOT=/home/<user>/os-image
mkdir -p $(CHROOT)
mkdir -p $(CHROOT)/{dev,proc,run,sys}
chown -hR root:root $(CHROOT)
mount --bind /var/cache/yum $(CHROOT)/var/cache/yum
mount --bind /dev $(CHROOT)/dev
mount -t proc proc $(CHROOT)/proc
mount -t tmpfs tmpfs $(CHROOT)/run
mount -t sysfs sysfs $(CHROOT)/sys
dnf -y --installroot=$(CHROOT) makecache
dnf -y --installroot=$(CHROOT) shell baremetal.yum
rpm --root $(CHROOT) --import $(CHROOT)/etc/pki/rpm-gpg/RPM-GPG-KEY-*
truncate -s 0 $(CHROOT)/etc/machine-id
```

(continues on next page)

(continued from previous page)

```
umount $(CHROOT)/var/cache/yum
umount $(CHROOT)/dev
umount $(CHROOT)/proc
umount $(CHROOT)/run
umount $(CHROOT)/sys
tar cpzf os-image.tar.gz --xattrs --acls --selinux -C $(CHROOT) .
```

Configuring the OS Image in glance

Anaconda is a two-stage installer stage 1 consists of the kernel and ramdisk and stage 2 lives in a squashfs file. All these components can be found in the CentOS/RHEL/Fedora ISO images.

The kernel and ramdisk can be found at `/images/pxeboot/vmlinuz` and `/images/pxeboot/initrd.img` respectively in the ISO. The stage 2 squashfs image can be normally found at `/LiveOS/squashfs.img` or `/images/install.img`.

The anaconda deploy driver uses the following image properties from glance, which are all optional depending on how you create your bare metal server:

- `kernel_id`
- `ramdisk_id`
- `stage2_id`
- `ks_template`
- `disk_file_extension`

All except `disk_file_extension` are glance image IDs. They can be prefixed with `glance://`.

Valid `disk_file_extension` values are:

- `.img`
- `.tar`
- `.tbz`
- `.tgz`
- `.txz`
- `.tar.gz`
- `.tar.bz2`
- `.tar.xz`

When the `disk_file_extension` property is not set to one of the above valid values the anaconda installer will assume that the image provided is a mountable OS disk.

An example of creating the necessary glance images with the anaconda files and the OS tarball and setting properties to refer to components can be seen below.

Note

The various images must be shared except for the OS image with the properties set. This image must be set to public. See bug 2099276 for more details.

```
# vmlinuz
openstack image create --container-format bare --disk-format raw --shared \
    --file ./vmlinuz anaconda-kernel-<version>

# initrd/initramfs/ramdisk
openstack image create --container-format bare --disk-format raw --shared \
    --file ./initrd.img anaconda-ramdisk-<version>

# squashfs/stage2
openstack image create --container-format bare --disk-format raw --shared \
    --file ./squashfs.img anaconda-stage2-<version>

KERNEL_ID=$(openstack image show -f value -c id anaconda-kernel-<version>)
RAMDISK_ID=$(openstack image show -f value -c id anaconda-ramdisk-<version>)
STAGE2_ID=$(openstack image show -f value -c id anaconda-stage2-<version>)

# the actual OS image we'll use as our source
openstack image create --container-format bare --disk-format raw --public \
    --property kernel_id=${KERNEL_ID} \
    --property ramdisk_id=${RAMDISK_ID} \
    --property stage2_id=${STAGE2_ID} \
    --property disk_file_extension=.tgz \
    --file ./os-image.tar.gz \
    my-anaconda-based-os-<version>
```

Deploying a node

To be able to deploy a node with the anaconda deploy interface the nodes `instance_info` must have an `image_source` at a minimum but depending on how your node is being deployed more fields must be populated.

If you are using Ironic via Nova then it will only set the `image_source` on `instance_info` so the following image properties are required:

- `kernel_id`
- `ramdisk_id`
- `stage2_id`

You may optionally upload a custom kickstart template to glance an associate it to the OS image via the `ks_template` property.

```
openstack server create --image my-anaconda-based-os-<version> ...
```

If you are not using Ironic via Nova then all properties except `disk_file_extension` can be supplied via `instance_info` or via the OS image properties. The values in `instance_info` will take precedence over those specified in the OS image. However most of their names are slightly altered.

- `kernel_id` OS image property is `kernel` in `instance_info`
- `ramdisk_id` OS image property is `ramdisk` in `instance_info`
- `stage2_id` OS image property is `stage2` in `instance_info`

Only the `ks_template` property remains the same in `instance_info`.

Note

If no `ks_template` is supplied then *anaconda.default_ks_template* will be used.

This is an example of how to set the kickstart template for a specific ironic node:

```
openstack baremetal node set <node> \
    --instance_info ks_template=glance://uuid
```

Ultimately to deploy your node it must be able to find the kernel, the ramdisk, the stage2 file, and your OS image via glance image properties or via `instance_info`.

```
openstack baremetal node set <node> \
    --instance_info image_source=glance://uuid
```

Warning

In the Ironic Project terminology, the word `template` often refers to a file that is supplied to the deployment, which Ironic supplies parameters to render a specific output. One critical example of this in the Ironic workflow, specifically with this driver, is that the generated `agent token` is conveyed to the booting ramdisk, facilitating it to call back to Ironic and indicate the state. This token is randomly generated for every deploy and is required. Specifically, this is leveraged in the templates `pre`, `onerror`, and `post` steps. For more information on Agent Token, please see *Agent Token*.

Standalone deployments

While this deployment interface driver was developed around the use of other OpenStack services, it is not explicitly required. For example, HTTP(S) URLs can be supplied by the API user to explicitly set the expected baremetal node `instance_info` fields

```
baremetal node set <node> \
   --instance_info image_source=<Mirror URL> \
   --instance_info kernel=<Kernel URL> \
   --instance_info ramdisk=<Initial Ramdisk URL> \
   --instance_info stage2=<Installer Stage2 Ramdisk URL>
```

When doing so, you may wish to also utilize a customized kickstart template, which can also be a URL. Please reference the ironic community provided template *ks.cfg.template* and use it as a basis for your own kickstart as it accounts for the particular stages and appropriate callbacks to Ironic.

Warning

The default template (for the kickstart liveimg command) expects an `instance_info\image_info` setting to be provided by the user, which serves as a base operating system image. In the context of the anaconda driver, it should be thought of almost like stage3. If youre using a custom template, it may not be required, but proceed with caution. See pykickstart documentation for more information on liveimg file format, structure, and use.

```
baremetal node set <node> \
    --instance_info ks_template=<URL>
```

If you do choose to use a liveimg with a customized template, or if you wish to use the stock template with a liveimg, you will need to provide this setting.

```
baremetal node set <node> \
    --instance_info image_info=<URL>
```

Warning

This is required if you do *not* utilize a customised template. As in use Ironics stock template.

The pattern of deployment in this case is identical to a deployment case where Ironic is integrated with OpenStack, however in this case Ironic collects the files, and stages them appropriately.

At this point, you should be able to request the baremetal node to deploy.

Standalone using a repository

Anaconda supports the concept of passing a repository as opposed to a dedicated URL path which has a `.treeinfo` file, which tells the initial boot scripts where to get various dependencies, such as what would be used as the anaconda `stage2` ramdisk. Unfortunately, this functionality is not well documented.

An example `.treeinfo` file can be found at http://mirror.stream.centos.org/9-stream/BaseOS/x86_64/os/.treeinfo.

Note

In the context of the `.treeinfo` file and the related folder structure for a deployment utilizing the `anaconda` deployment interface, `images/install.img` file represents a `stage2` ramdisk.

In the context of one wishing to deploy Centos Stream-9, the following may be useful.

```
baremetal node set <node> \
    --instance_info image_source=http://mirror.stream.centos.org/9-stream/
↪BaseOS/x86_64/os/ \
    --instance_info kernel=http://mirror.stream.centos.org/9-stream/BaseOS/
↪x86_64/os/images/pxeboot/vmlinuz \
    --instance_info ramdisk=http://mirror.stream.centos.org/9-stream/BaseOS/
↪x86_64/os/images/pxeboot/initrd.img
```

Once set, a kickstart template can be provided via an `instance_info` parameter, and the node deployed.

Deployment Process

At a high level, the mechanics of the anaconda driver work in the following flow, where we also note the stages and purpose of each part for informational purposes.

1. Network Boot Program (Such as iPXE) downloads the kernel and initial ramdisk.
2. Kernel launches, uncompresses initial ramdisk, and executes init inside of the ramdisk.

3. The initial ramdisk boot scripts, such as Dracut, recognize the kernel command line parameters Ironic supplied with the boot configuration, and downloads the second stage artifacts, in this case called the `stage2` image. This image contains Anaconda and base dependencies.
4. Anaconda downloads and parses the kickstart configuration which was also supplied on the kernel command line, and executes the commands as defined in the kickstart template.
5. The kickstart template, if specified in its contents, downloads a `liveimg` which is used as the base operating system image to start with.

Configuration Considerations

When using the `anaconda` deployment interface, some configuration parameters may need to be adjusted in your environment. This is in large part due to the general defaults being set to much lower values for image based deployments, but the way the anaconda deployment interface works, you may need to make some adjustments.

- *`conductor.deploy_callback_timeout`* likely needs to be adjusted for most `anaconda` deployment interface users. By default, this is a timer that looks for agents that have not checked in with Ironic, or agents which may have crashed or failed after they started. If the value is reached, then the current operation is failed. This value should be set to a number of seconds which exceeds your average anaconda deployment time.
- *`pxe.boot_retry_timeout`* can also be triggered and result in an anaconda deployment in progress getting reset as it is intended to reboot nodes that might have failed their initial PXE operation. Depending on the sizes of images, and the exact nature of what was deployed, it may be necessary to ensure this is a much higher value.

Limitations

- This deploy interface has only been tested with Red Hat based operating systems that use anaconda. Other systems are not supported.
- Runtime TLS certificate injection into ramdisks is not supported. Assets such as `ramdisk` or a `stage2` ramdisk image need to have trusted Certificate Authority certificates present within the images *or* the Ironic API endpoint utilized should utilize a known trusted Certificate Authority.
- The `anaconda` tooling deploying the instance/workload does not heartbeat to Ironic like the `ironic-python-agent` driven ramdisks. As such, you may need to adjust some timers. See *Configuration Considerations* for some details on this.

4.2.22 Node History

Overview

Ironic keeps a record of node error events in node history. This allows operators to track frequent failures and get additional context when troubleshooting failures.

How it works

Anytime a node would have its `last_error` populated, an entry is added to the history of the node containing severity of and details about the failure.

Node history can be completely disabled by setting *`conductor.node_history`* to `False`; it is enabled by default.

Since node history can grow unbounded over time, by default Ironic is configured to prune the data; this behavior is configurable with the following settings:

- *conductor.node_history_max_entries*
- *conductor.node_history_cleanup_interval*
- *conductor.node_history_cleanup_batch_count*
- *conductor.node_history_minimum_days*

Client usage

The baremetal CLI has full support for node history.

To view the list of entries in history for a node: `baremetal node history list $node`.

To see a specific node history entry you want to see in detail: `baremetal node history get $node $eventId`.

Node history entries cannot be removed manually. Use the configuration options listed in the previous section to control the automatic pruning behavior of Ironic.

4.2.23 Node Health Monitoring

Overview

Ironic can monitor and report the hardware health status of baremetal nodes by querying the Baseboard Management Controller (BMC). This provides operators with visibility into potential hardware issues before they cause failures.

The health status is stored in the nodes `health` field and can have one of the following values:

- **OK** - Hardware is functioning normally with no issues detected.
- **Warning** - Hardware has non-critical issues that may require attention.
- **Critical** - Hardware has critical issues requiring immediate attention.
- **None** - Health status is not available (driver doesnt support it or BMC didnt report it).

How it works

Health monitoring integrates with Ironics existing power state synchronization cycle. When the conductor queries a nodes power state, it also retrieves the hardware health status if the driver supports it.

The health status is cached in the nodes database record and updated periodically along with power state. No separate polling interval is required.

Drivers that do not implement health monitoring are silently skipped - nodes managed by such drivers will simply have a `None` health value.

Configuration

Health monitoring is **enabled by default**. To disable it, set the following in your Ironic configuration:

```
[conductor]
enable_health_monitoring = False
```

Supported Drivers

Currently, the following drivers support health monitoring:

- **Redfish** - Retrieves health from the Redfish System resource (`Status.Health` field). This reflects the overall rollup health status reported by the BMC, which typically aggregates the health of processors, memory, fans, power supplies, and storage.

Nodes using drivers without health monitoring support (e.g., IPMI) will have their health field set to `None`.

Client Usage

The health field is **not displayed by default** in `baremetal node list` output to maintain backward compatibility and keep the default output concise.

To view node health status, use one of the following methods:

Query specific fields including health:

```
$ baremetal node list --fields name health
+-------------+----------+
| Name        | Health   |
+-------------+----------+
| node-001    | OK       |
| node-002    | Warning  |
| node-003    | Critical |
| node-004    | None     |
+-------------+----------+
```

Include health alongside other common fields:

```
$ baremetal node list --fields name power_state provision_state maintenance␣
↪health
+-------------+-------------+--------------------+-------------+----------+
| Name        | Power State | Provisioning State | Maintenance | Health   |
+-------------+-------------+--------------------+-------------+----------+
| node-001    | power on    | active             | False       | OK       |
| node-002    | power on    | active             | False       | Warning  |
| node-003    | power off   | manageable         | True        | Critical |
+-------------+-------------+--------------------+-------------+----------+
```

Use long listing format:

```
$ baremetal node list --long
```

This includes health along with all other node fields.

View health in node details:

```
$ baremetal node show <node> --fields health
+--------+----------+
| Field  | Value    |
+--------+----------+
```

(continues on next page)

(continued from previous page)

```
| health | OK       |
+--------+----------+
```

API Version

The `health` field is available starting from API microversion **1.109**.

Clients using older API versions will not see this field in responses.

4.2.24 Use of an OCI Container Registry

What is an OCI Container Registry?

An OCI Container registry is an evolution of a docker container registry where layers which make up containers can be housed as individual data files, and then be retrieved to be reconstructed into a running container. OCI is short for Open Container Initiative, and you can learn more about about OCI at opencontainers.org.

Container registries are evolving to support housing other data files, and the focus in this context is the evolution to support those additional files to be housed in and served by a container registry.

Warning

This feature should be considered experimental.

Overview

A recent addition to Ironic is the ability to retrieve artifacts from an OCI Container Registry. This support is modeled such that it can be used by an Ironic deployment for both disk images, and underlying artifacts used for deployment, such as kernels and ramdisks. Different rules apply and as such, please review the next several sections carefully.

At present, this functionality is only available for users who are directly interacting with Ironic. Novas data model is *only* compatible with usage Glance at this present time, but that may change in the future.

How it works

An OCI registry has a layered data model which can be divided into three conceptual layers.

- Artifact Index - Higher level list which points to manifests and contains information like annotations, platform, architecture.
- Manifest - The intermediate structural location which contains the lower level details related to the blob (Binary Large OBject) data as well as the information where to find the blob data. When a container image is being referenced, this Manifest contains information on multiple layers which comprise a container.
- Blob data - The actual artifact, which can only be retrieved using the data provided in the manifest.

Ironic has a separate image service client which translates an an OCI style container URL in an `image_source` value, formatted such as `oci://host/user/container:tag`. When just a tag has been defined, which can be thought of as specific tagged view containing many artifacts, which Ironic will search through to find the best match.

Matching is performed with an attempt to weigh preference to file type based upon the configured `image_download_source`, where as with a `local` value, `qcow2` disk images are preferred. Otherwise `raw` is preferred. This uses the `disktype` annotation where `qcow2` and `qemu` are considered QCOW2 format images. A `disktype` annotation on the manifests of `raw` or `applehv`, are considered raw disk images. Once file types have been appropriately weighted, the code attempts to match the baremetal nodes CPU architecture to the listed platform `architecture` in the remote registry. Once the file identification process has been completed, Ironic automatically updates the `image_source` value to the matching artifact in the remote container registry.

Note

The code automatically attempts to handle differences in architecture naming which has been observed, where `x86_64` is sometimes referred to as `amd64`, and `aarch64` is sometimes referred to as `arm64`.

Warning

An `image_download_source` of `swift` is incompatible with this image service. Only `local` and `http` are supported.

When a URL is specific and pointing to a specific manifest, for example `oci://host/user/container@sha256:f00b...`, Ironic is only able to retrieve that specific file from the the container registry. Due to the data model, we also cannot learn additional details about that image such as annotations, as annotations are part of the structural data which points to manifests in the first place.

An added advantage to the use of container registries, is that the checksum *is confirmed* in transit based upon the supplied metadata from the container registry. For example, when you use a manifest URL, the digest portion of the URL is used to checksum the returned contents, and that manifests then contains the digest values for artifacts which also supplies sufficient information to identify the URL where to download the artifacts from.

Warning

The value of `image_checksum`, as well as `image_os_hash_value`, is ignored for OCI URLs. If you wish to restrict the checksum and matching artifact in this model, please use a sha digest URL which points to a specific manifest and is referenced by the manifests checksum.

Authentication

Authentication is an important topic for users of an OCI Image Registry.

While some public registries are fairly friendly to providing download access, other registries may have aggressive quotas in place which require users to be authenticated to download artifacts. Furthermore, private image registries may require authentication for any access.

As such, there are four available paths for providing configuration:

- A node `instance_info` value of `image_pull_secret`. This value may be utilized to retrieve an image artifact, but is not intended for pulling other artifacts like kernels or ramdisks used as part of a deployment process. As with all other `instance_info` field values, this value is deleted once the node has been unprovisioned. The way this field is used, is by supplying the pre-shared secret

token value. This is the same value which you would normally have in your Docker `config.json` file `auth` field for the top level domain your accessing.

- A node `driver_info` value of `image_pull_secret`. This setting is similar to the `instance_info` setting, but may be utilized by an administrator of a baremetal node to define the specific registry credential to utilize for the node.
- The *`oci.authentication_config`* which allows for a conductor process wide pre-shared secret configuration. This configuration value can be set to a file which parses the common auth configuration format used for container tooling in regards to the secret to utilize for container registry authentication. This value is only consulted *if* a specific secret has not been defined to utilize, and is intended to be compaitble with the the format used by docker `config.json` to store authentication detail.
- Basic authentication can also be used, but is not advised for production usage. In this context, a user:password string can be supplied as a pull secret value, or the `ironic.conf` configuration file `[deploy]` setting parameters section for basic authentication credentials can alternatively be used.

An example of the configuration file looks something like the following example.

```
{
  "auths": {
    "quay.io": {
      "auth": "<pull_secret_here>"
    },
    "private-registry.tld": {
      "auth": "<pull_secret_here>"
    }
  }
}
```

Note

The `image_pull_secret` values are not visible in the API surface due Ironics secret value santiization, which prevents sensitive values from being visible, and are instead returned as **.

Note

If you need to extract the pull secret from a config.json file, you may want to explore using the `jq` command with a syntax along the lines of *jq .auths.domain.tld.auth config.json* which will return the quoted string you can then populate. Other command line oriented ways exist for users to retrieve such a value once a login has completed to a container platform, meaning you can use that same token value if desired.

Available URL Formats

The following URL formats are available for use to download a disk image artifact. When a non-precise manifest URL is supplied, Ironic will attempt to identify and match the artifact. URLs for artifacts which are not disk images are required to be specific and point to a specific manifest.

Note

If no tag is defined, the tag `latest` will be attempted, however, if that is not found in the *list* of available tags returned by the container registry, an ImageNotFound error will be raised in Ironic.

- oci://host/path/container - Ironic assumes latest is the desired tag in this case.
- oci://host/path/container:tag - Ironic discoveres artifacts based upon the view provided by the defined tag.
- oci://host/path/container@sha256:f00f - This is a URL which defines a specific manifest. Should this be a container, this would be a manifest file with many layers to make a container, but for an artifact only a single file is represented by this manifest, and we retrieve this specific file.

Warning

The use of tag values to access an artifact, for example, `deploy_kernel` or `deploy_ramdisk`, is not possible. This is an intentional limitation which may addressed in a future version of Ironic.

Known Limitations

- For usage with disk images, only whole-disk images are supported. Ironic does not intend to support Partition images with this image service.
- IPA is unaware of remote container registries, as well as authentication to a remote registry. This is expected to be addressed in a future release of Ironic.
- Some artifacts may be compressed using Zstandard. Only disk images or artifacts which transit through the conductor may be appropriately decompressed. Unfortunately IPA wont be able to decompress such artifacts dynamically while streaming content.
- Authentication to container image registries is *only* available through the use of pre-shared token secrets.
- Use of tags may not be viable on some OCI Compliant image registries. This may result as an ImageNotFound error being raised when attempting to resolve a tag.

How do I upload files to my own registry?

While there are several different ways to do this, the easiest path is to leverage a tool called `ORAS`. You can learn more about ORAS at https://oras.land

The ORAS utility is able to upload arbitrary artifacts to a Container Registry along with the required manifest *and* then associates a tag for easy human reference. While the OCI data model *does* happily support a model of one tag in front of many manifests, ORAS does not. In the ORAS model, one tag is associated with one artifact.

In the examples below, you can see how this is achieved. Please be careful that these examples are *not* commands you can just cut and paste, but are intended to demonstrate the required step and share the concept of how to construct the URL for the artifact.

Note

These examples command lines may differ slightly based upon your remote registry, and underlying configuration, and as such leave out credential settings.

As a first step, we will demonstrate uploading an IPA Ramdisk kernel.

```
$ export HOST=my-container-host.domain.tld
$ export CONTAINER=my-project/my-container
$ oras push ${HOST}/${CONTAINER}:ipa_kernel tinyipa-master.vmlinuz
✓ Exists    tinyipa-master.vmlinuz                          5.65/5.65 MB 100.00
↪%     0s
   sha256:15ed5220a397e6960a9ac6f770a07e3cc209c6870c42cbf8f388aa409d11ea71
✓ Exists    application/vnd.oci.empty.v1+json                   2/2  B 100.00
↪%     0s
   sha256:44136fa355b3678a1146ad16f7e8649e94fb4fc21fe77e8310c060f61caaff8a
✓ Uploaded  application/vnd.oci.image.manifest.v1+json       606/606  B 100.00
↪%     0s
   sha256:2d408348dd6ff2e26efc1de03616ca91d76936a27028061bc314289cecdc895f
Pushed [registry] my-container-host.domain.tld/my-project/my-container:ipa_
↪kernel
ArtifactType: application/vnd.unknown.artifact.v1
Digest:␣
↪sha256:2d408348dd6ff2e26efc1de03616ca91d76936a27028061bc314289cecdc895f
$
$ export MY_IPA_KERNEL=oci://${HOST}/${CONTAINER}
↪:@sha256:2d408348dd6ff2e26efc1de03616ca91d76936a27028061bc314289cecdc895f
```

As you can see from this example, weve executed the command, and uploaded the file. The important aspect to highlight is the digest reported at the end. This is the manifest digest which you can utilize to generate your URL.

Warning

When constructing environment variables for your own use, specifically with digest values, please be mindful that you will need to utilize the digest value from your own upload, and not from the example.

```
$ oras push ${HOST}/${CONTAINER}:ipa_ramdisk tinyipa-master.gz
✓ Exists    tinyipa-master.gz                               91.9/91.9 MB 100.00
↪%     0s
   sha256:0d92eeb98483f06111a352b673d588b1aab3efc03690c1553ef8fd8acdde15fc
✓ Exists    application/vnd.oci.empty.v1+json                   2/2  B 100.00
↪%     0s
   sha256:44136fa355b3678a1146ad16f7e8649e94fb4fc21fe77e8310c060f61caaff8a
✓ Uploaded  application/vnd.oci.image.manifest.v1+json       602/602  B 100.00
↪%     0s
   sha256:b17e53ff83539dd6d49e714b09eeb3bd0a9bb7eee2ba8716f6819f2f6ceaad13
Pushed [registry] my-container-host.domain.tld/my-project/my-container:ipa_
↪ramdisk
ArtifactType: application/vnd.unknown.artifact.v1
Digest:␣
```

(continues on next page)

(continued from previous page)

```
↪sha256:b17e53ff83539dd6d49e714b09eeb3bd0a9bb7eee2ba8716f6819f2f6ceaad13
$
$ export MY_IPA_RAMDISK=oci://${HOST}/${CONTAINER}
↪:@sha256:b17e53ff83539dd6d49e714b09eeb3bd0a9bb7eee2ba8716f6819f2f6ceaad13
```

As a reminder, please remember to utilize *different* tags with ORAS.

For example, you can view the current tags in the remote registry by existing the following command.

```
$ oras repo tags --insecure $HOST/project/container
ipa_kernel
ipa_ramdisk
unrelated_item
$
```

Now that you have successfully uploaded an IPA kernel and ramdisk, the only item remaining is a disk image. In this example below, were generating a container tag based URL as well as direct manifest digest URL.

Note

The example below sets a manifest annotation of `disktype` and artifact platform. While not explicitly required, these are recommended should you allow Ironic to resolve the disk image utilizing the container tag as opposed to a digest URL.

```
$ oras push -a disktype=qcow2 --artifact-platform linux/x86_64 $HOST/
↪$CONTAINER:cirros-0.6.3 ./cirros-0.6.3-x86_64-disk.img
✓ Exists    cirros-0.6.3-x86_64-disk.img                  20.7/20.7 MB 100.00
↪%     0s
  sha256:7d6355852aeb6dbcd191bcda7cd74f1536cfe5cbf8a10495a7283a8396e4b75b
✓ Uploaded  application/vnd.oci.image.config.v1+json           38/38  B 100.00
↪%   43ms
  sha256:369358945e345b86304b802b704a7809f98ccbda56b0a459a269077169a0ac5a
✓ Uploaded  application/vnd.oci.image.manifest.v1+json       626/626  B 100.00
↪%     0s
  sha256:0a175cf13c651f44750d6a5cf0cf2f75d933bd591315d77e19105e5446b73a86
Pushed [registry] my-container-host.domain.tld/my-project/my-container:cirros-
↪0.6.3
ArtifactType: application/vnd.unknown.artifact.v1
Digest:␣
↪sha256:0a175cf13c651f44750d6a5cf0cf2f75d933bd591315d77e19105e5446b73a86
$ export MY_DISK_IMAGE_TAG_URL=oci://${HOST}/${CONTAINER}:cirros-0.6.3
$ export MY_DISK_IMAGE_DIGEST_URL=oci://${HOST}/${CONTAINER}
↪@sha256:0a175cf13c651f44750d6a5cf0cf2f75d933bd591315d77e19105e5446b73a86
```

4.2.25 Runbooks for Cleaning & Servicing

Overview

The Runbook resource represents a collection of steps that define a series of actions to be executed on a node during cleaning and servicing operations. Runbooks enable users to perform simple and complex operations in a consistent, predefined and automated manner.

A runbook is matched for a node if the runbooks name matches a trait in the node. So, runbooks can be created to match already existing traits for select nodes or the other way around; either way a valid runbook name must be unique and follow the traits naming convention since thats the basis of association with a node.

Hence, runbook names can be either standard or custom. Standard runbook names are listed in the os_traits library. Custom traits must be prefixed with `CUSTOM_`, contain only upper case characters A to Z, digits 0 to 9, or underscores and be no longer than 255 characters in length.

No two runbooks can have the same name, as runbook names must be unique within the system.

Access Control for Runbooks

Runbooks implements a role-based access control model that determines who can create, modify, and use them:

The `owner` and `public` fields determine a runbooks accessibility:

- If `owner` is non-null (`public` is automatically false), the runbook is scoped to that project and only usable on nodes owned or leased by that project
- If `owner` is null and `public` is false, only system-scoped users can access or use the runbook
- If `owner` is null and `public` is true, any project can use the runbook on compatible nodes, but only system-scoped users can modify it

Note

For design details and implementation specifics, please see the Runbooks specification.

Purpose of Trait Matching

The trait matching mechanism serves as an access control to ensure that runbooks are only executed on pre-approved nodes.

When executing a runbook, you must explicitly specify which runbook to run. Currently, there is no way to execute multiple runbooks on a single node with one command. However, you can include all necessary steps in a single comprehensive runbook if you need to perform multiple operations.

You can verify that a node has the required trait for a runbook:

```
baremetal node trait list <node>
```

Refer to the Ironic API reference for runbooks for information on how to create, and manage runbooks.

For more details about node cleaning and servicing operations, please see *Node cleaning* and *Node servicing*.

Example Runbook

```
baremetal runbook create --name CUSTOM_FIRMWARE_UPGRADE \
  --steps '[
     {
        "interface": "management",
        "step": "reset_bios_to_default",
        "args": {},
        "order": 1
     },
     {
        "interface": "management",
        "step": "update_firmware",
        "args": {
           "firmware_url": "https://example.com/firmware.bin",
           "component": "bios"
        },
        "order": 2
     },
     {
        "interface": "management",
        "step": "reboot",
        "args": {},
        "order": 3
     }
  ]'

The output of the create command would show the complete runbook details::

   +------------+--------------------------------------------------------------
↪-----------------------------------------+
   | Field      | Value                                                        ␣
↪                                         |
   +------------+--------------------------------------------------------------
↪-----------------------------------------+
   | created_at | 2025-03-12T14:16:26.054115+00:00                             ␣
↪                                         |
   | extra      | {}                                                           ␣
↪                                         |
   | name       | CUSTOM_FIRMWARE_UPGRADE                                      ␣
↪                                         |
   | owner      | None                                                         ␣
↪                                         |
   | public     | False                                                        ␣
↪                                         |
   | steps      | [{'interface': 'management', 'step': 'reset_bios_to_default
↪', 'args': {}, 'order': 1}, {'interface':    |
   |            | 'management', 'step': 'update_firmware', 'args': {'firmware_
↪url': 'https://example.com/firmware.bin',    |
   |            | 'component': 'bios'}, 'order': 2}, {'interface': 'management
↪', 'step': 'reboot', 'args': {}, 'order':    |
```

(continues on next page)

(continued from previous page)

```
   |             | 3}]                                                                 ␣
↪                                                                                    |
   | updated_at  | None                                                                ␣
↪                                                                                    |
   | uuid        | 160ff684-5216-4874-9a61-775c3a17c892                                ␣
↪                                                                                    |
   +-------------+--------------------------------------------------------------------
↪-----------------------------------------+
```

Cleaning and Servicing

Once a runbook is created and associated with a node via matching traits, it can be used in place of explicit cleaning or servicing steps.

For cleaning operations:

```
# Using a runbook name
baremetal node clean --runbook CUSTOM_FIRMWARE_UPGRADE node-0

# Or using a runbook UUID
baremetal node clean --runbook 160ff684-5216-4874-9a61-775c3a17c892 node-0
```

For servicing operations:

```
# Using a runbook name
baremetal node service --runbook CUSTOM_FIRMWARE_UPGRADE node-0

# Or using a runbook UUID
baremetal node service --runbook 160ff684-5216-4874-9a61-775c3a17c892 node-0
```

These commands will execute all the steps defined in the runbook in the specified order.

4.2.26 Container-Based Steps

Overview

The Container Hardware Manager in ironic-python-agent (IPA) allows running OCI-compatible containers as steps on bare metal nodes. This enables operators to package arbitrary tools firmware updaters, diagnostic suites, compliance scanners as container images and execute them during any step-based workflow, such as cleaning, deployment, or servicing.

Basics

A workflow for implementing container-based steps with runbooks is:

1. The operator builds an IPA ramdisk with the `ironic-python-agent-podman` diskimage-builder element, which installs podman and the Container Hardware Manager into the ramdisk.
2. The Ironic conductor sends `[agent_containers]` configuration to IPA via the lookup/heartbeat endpoint. This allows conductor-side settings to override any build-time defaults in the ramdisk.
3. When a runbook triggers a `container_clean_step`, IPA uses podman (or docker) to pull and run the specified container image on the bare metal node.

4. The container runs with host networking by default, executes its task, and exits. IPA reports the result back to the conductor.

Prerequisites

IPA ramdisk with podman support

The IPA ramdisk must be built with the `ironic-python-agent-podman` diskimage-builder (DIB) element. This element is currently **Debian-based only**.

```
export DIB_ALLOW_ARBITRARY_CONTAINERS=true
export DIB_RUNNER=podman

disk-image-create ironic-python-agent-ramdisk \
    ironic-python-agent-podman \
    debian -o ipa-with-podman
```

Key DIB environment variables:

DIB_ALLOW_ARBITRARY_CONTAINERS
: Set to `true` to allow any container image. Set to `false` (default) to restrict to a specific allowlist. Environments which permit non-admin roles to create and execute runbooks should not set this to `true` for security reasons.

DIB_ALLOWED_CONTAINERS
: Comma-separated list of allowed container image URLs. Only used when `DIB_ALLOW_ARBITRARY_CONTAINERS` is `false`.

DIB_RUNNER
: Container runtime: `podman` (default) or `docker`.

Container registry access

The container registry hosting your images must be accessible from the cleaning network. If using a private registry, ensure credentials and TLS certificates are configured in the ramdisk or passed via `pull_options`.

Ironic Conductor Configuration

The `[agent_containers]` configuration group controls how the conductor instructs IPA to handle containers. These settings are sent to IPA at lookup time, so changes take effect without rebuilding the ramdisk.

```
[agent_containers]
# Allow any container image (default: false)
allow_arbitrary_containers = false

# Allowlist of container images (used when above is false)
allowed_containers = docker://registry.example.com/firmware-tool:latest,
↪docker://registry.example.com/diag-suite:v2

# Container runtime (default: podman)
runner = podman
```

(continues on next page)

(continued from previous page)

```
# Options passed to the pull command
pull_options = --tls-verify=false

# Options passed to the run command
run_options = --rm --network=host --tls-verify=false
```

Warning

Setting `allow_arbitrary_containers = true` allows **any** container image to be pulled and executed with host-level network access on the bare metal node. Only enable this in trusted environments. Prefer using `allowed_containers` to maintain an explicit allowlist.

See also: *agent_containers.allow_arbitrary_containers*, *agent_containers.allowed_containers*, *agent_containers.runner*, *agent_containers.pull_options*, *agent_containers.run_options*.

Example Container-based Runbooks

The built-in step

The Container Hardware Manager exposes a built-in cleaning step called `container_clean_step` on the `deploy` interface. This step has a default priority of `0`, meaning it only runs when explicitly invoked via manual cleaning, servicing, or a runbook.

The step accepts the following arguments:

`container_url` (required)
: The full container image URL, e.g. `docker://registry.example.com/firmware-tool:latest`.

`pull_options` (optional)
: Override the default pull options for this specific container.

`run_options` (optional)
: Override the default run options for this specific container.

Single-container runbook

This example creates a runbook that runs a single firmware update container:

```
baremetal runbook create \
    --name CUSTOM_CONTAINER_FW_UPDATE \
    --steps '[
        {
            "interface": "deploy",
            "step": "container_clean_step",
            "args": {
                "container_url": "docker://registry.example.com/firmware-
↪tool:latest"
            },
```

(continues on next page)

(continued from previous page)

```
            "order": 1
        }
    ]'
```

Multi-container runbook

Runbooks can combine multiple container steps with traditional steps. This example runs a diagnostic container, then a firmware updater, and finishes with a standard disk metadata erase:

```
baremetal runbook create \
    --name CUSTOM_CONTAINER_CLEAN \
    --steps '[
        {
            "interface": "deploy",
            "step": "container_clean_step",
            "args": {
                "container_url": "docker://registry.example.com/diag-suite:v2"
            },
            "order": 1
        },
        {
            "interface": "deploy",
            "step": "container_clean_step",
            "args": {
                "container_url": "docker://registry.example.com/firmware-
↪tool:latest",
                "run_options": "--rm --network=host --privileged"
            },
            "order": 2
        },
        {
            "interface": "deploy",
            "step": "erase_devices_metadata",
            "args": {},
            "order": 3
        }
    ]'
```

Adding traits to nodes

Runbooks are matched to nodes via traits. Add the matching trait to all nodes that should use the runbook:

```
baremetal node add trait <node> CUSTOM_CONTAINER_CLEAN
```

Using the Runbook

Manual cleaning

Trigger the runbook on a node in `manageable` state:

```
baremetal node clean <node> --runbook CUSTOM_CONTAINER_CLEAN
```

Automated cleaning

To use container-based steps for automated cleaning, configure the conductor to use runbook-based or hybrid cleaning and assign the runbook. See *Configuring automated cleaning with runbooks* for full details on the available configuration levels (per-node, per-resource-class, global).

A minimal example using the global default:

```
[conductor]
automated_clean = true
automated_cleaning_step_source = runbook
automated_cleaning_runbook = CUSTOM_CONTAINER_CLEAN
```

All nodes must have the matching trait (`CUSTOM_CONTAINER_CLEAN`) unless trait validation is disabled via *conductor.automated_cleaning_runbook_validate_traits*.

Servicing

Container steps also work with *Node servicing*. Trigger a container runbook on an `active` node:

```
baremetal node service <node> --runbook CUSTOM_CONTAINER_CLEAN
```

Alternative Methods

Operators may utilize container-based steps that are hardcoded via configuration in-ramdisk.

Ironic-python-agent can be configured to expose arbitrary steps using containers for use in workflows, including automated cleaning, via a yaml configuration file.

For example:

```
steps:
  - name: manage_container_cleanup
    image: docker://172.24.4.1:5000/cleaning-image:latest
    interface: deploy
    reboot_requested: true
    pull_options:
      - --tls-verify=false
    run_options:
      - --rm
      - --network=host
      - --tls-verify=false
    abortable: true
    priority: 20
  - name: manage_container_cleanup2
    image: docker://172.24.4.1:5000/cleaning-image2:latest
    interface: deploy
    reboot_requested: true
    pull_options:
      - --tls-verify=false
```

(continues on next page)

(continued from previous page)

```
    run_options:
      - --rm
      - --network=host
      - --tls-verify=false
    abortable: true
    priority: 10
```

By placing a file in your IPA ramdisk with these contents in the path indicated by *agent_containers.container_steps_file*, cleaning steps `manage_container_cleanup` and `manage_container_cleanup2` will be reported as available cleaning steps at the indicated priority.

This is useful for high-security environments which would prefer the hassle of rebuilding a ramdisk to the risk of permitting runtime decisions around what containers to clean with.

Security Considerations

- **Prefer allowlisting** over `allow_arbitrary_containers = true`. The allowlist (`allowed_containers`) restricts which images IPA will accept, reducing the risk of running untrusted code.
- **TLS verification** the default `pull_options` and `run_options` include `--tls-verify=false` for development convenience. In production, remove this flag and ensure proper TLS certificates are available in the ramdisk.
- **Container privileges** by default, containers run with `--network=host`, giving them full access to the nodes network stack. Review `run_options` and consider adding `--read-only` or dropping capabilities where possible.

Troubleshooting

Container pull failures
: Check that the container registry is accessible from the cleaning network. Verify the image URL in the runbook step. If using TLS, ensure certificates are configured correctly in the ramdisk or add `--tls-verify=false` to `pull_options` for testing.

Step not found: container_clean_step
: The IPA ramdisk was not built with the `ironic-python-agent-podman` element. Rebuild the ramdisk with podman support as described in *Prerequisites*.

Container rejected by allowlist
: The container URL does not match any entry in `allowed_containers` and `allow_arbitrary_containers` is `false`. Either add the image to the allowlist or set `allow_arbitrary_containers = true` in `[agent_containers]`.

Trait mismatch
: The node does not have a trait matching the runbook name. Add the trait with `baremetal node add trait <node> <RUNBOOK_NAME>`.

4.2.27 Trait Based Networking

Introduction

Trait Based Networking, or TBN for short, is an Ironic feature that allows an Openstack installation utilizing Ironic, Neutron, and Nova to dynamically configure port scheduling for Ironic nodes.

Enable and Configure

To configure and enable TBN for your Ironic installation please see *Configure Trait Based Networking to Plan Networking Related Operations at vif Attach*

Terminology

First some terms:

- Node: An Ironic node.
- Trait: A set of actions referred to by a name.
- Action or Trait Action: A defined operation to apply to a node if the actions Filter Expression matches network objects associated with a node.
- Filter Expression: A boolean expression which filters for specific network objects like a port, portgroup or network.
- Port: A network interface.
- Portgroup: A set of ports, grouped together.
- Dynamic Portgroup: An ephemeral portgroup that is created by a traits action and is subsequently destroyed once detached.
- Network (aka VIF): A neutron network.

Trait Actions

The core of TBN functionality lies in actions that each trait defines. Each action allows the operator to define how exactly TBN will setup networking for the node.

The following actions are available:

- Attach port: Attach one or more ports belonging to a node to a network (aka vif).
- Attach portgroup: Attach one or more portgroups belonging to a node to a network (aka vif).
- Group and attach ports: Create a dynamic portgroup and attach it to a network.

In the future more actions may be added.

See *Trait Based Networking Configuration File Reference* for more information on configuring and applying trait actions.

Filter Expression Primer

Filter Expressions are how network related objects are filtered or matched with their applicable traits.

Lets look at a basic filter expression:

```
port.vendor == "purple"
```

This expression consists of three parts:

1. A named variable: `port.vendor` - This means the expression will consider a ports vendor field.
2. A comparator: == - In this case equality.
3. A string literal: `"purple"` - Gives a comparison value to compare against the contents of the named variable.

Taken together this expression can be read in plain English as: filter for ports with vendor that is exactly equal to the string purple.

All available variables and comparators are listed in the *Filter Expression Reference*.

Single expressions can be linked together using boolean operators:

- && boolean AND
- || boolean OR

Like so:

```
port.vendor == "purple" && port.category != "privatenet"
```

Again, in plain English this could be read as: filter for ports with vendor that is exactly equal to the string purple and whose category field is not privatenet.

Functions

Theres also a couple of functions available to filter ports vs portgroups:

- `port.is_port` will return true if the port is a port, and false otherwise.
- `port.is_portgroup` will return true if the port is a portgroup, and false otherwise.

Function expressions can be used in isolation:

```
port.is_port
```

Or linked with other expressions:

```
port.is_port && port.vendor == "green"
```

4.3 Configuration and Operation

4.3.1 Ironic Python Agent

Overview

Ironic Python Agent (also often called *IPA* or just *agent*) is a Python-based agent which handles *ironic* bare metal nodes in a variety of actions such as inspect, configure, clean and deploy images. IPA is distributed over nodes and runs, inside of a ramdisk, the process of booting this ramdisk on the node.

For more information see the ironic-python-agent documentation.

Drivers

Starting with the Kilo release all deploy interfaces (except for fake ones) are using IPA. For nodes using the *Direct deploy* interface, the conductor prepares a swift temporary URL or a local HTTP URL for the image. IPA then handles the whole deployment process: downloading an image from swift, putting it on the machine and doing any post-deploy actions.

Requirements

Using IPA requires it to be present and configured on the deploy ramdisk, see *Building or downloading a deploy ramdisk image*

Using proxies for image download

Overview

When using the *Direct deploy*, IPA supports using proxies for downloading the user image. For example, this could be used to speed up download by using a caching proxy.

Steps to enable proxies

1. Configure the proxy server of your choice (for example Squid, Apache Traffic Server). This will probably require you to configure the proxy server to cache the content even if the requested URL contains a query, and to raise the maximum cached file size as images can be pretty big. If you have HTTPS enabled in swift (see swift deployment guide), it is possible to configure the proxy server to talk to swift via HTTPS to download the image, store it in the cache unencrypted and return it to the node via HTTPS again. Because the image will be stored unencrypted in the cache, this approach is recommended for images that do not contain sensitive information. Refer to your proxy servers documentation to complete this step.
2. Set *glance.swift_temp_url_cache_enabled* in the ironic conductor config file to `True`. The conductor will reuse the cached swift temporary URLs instead of generating new ones each time an image is requested, so that the proxy server does not create new cache entries for the same image, based on the query part of the URL (as it contains some query parameters that change each time it is regenerated).
3. Set *glance.swift_temp_url_expected_download_start_delay* option in the ironic conductor config file to the value appropriate for your hardware. This is the delay (in seconds) from the time of the deploy request (when the swift temporary URL is generated) to when the URL is used for the image download. You can think of it as roughly the time needed for IPA ramdisk to startup and begin download. This value is used to check if the swift temporary URL duration is large enough to let the image download begin. Also if temporary URL caching is enabled, this will determine if a cached entry will still be valid when the download starts. It is used only if *glance.swift_temp_url_cache_enabled* is `True`.
4. Increase *glance.swift_temp_url_duration* option in the ironic conductor config file, as only non-expired links to images will be returned from the swift temporary URLs cache. This means that if `swift_temp_url_duration=1200` then after 20 minutes a new image will be cached by the proxy server as the query in its URL will change. The value of this option must be greater than or equal to *glance.swift_temp_url_expected_download_start_delay*.
5. Add one or more of `image_http_proxy`, `image_https_proxy`, `image_no_proxy` to driver_info properties in each node that will use the proxy.

Advanced configuration

Out-of-band vs. in-band power off on deploy

After deploying an image onto the nodes hard disk, Ironic will reboot the machine into the new image. By default this power action happens `in-band`, meaning that the ironic-conductor will instruct the IPA ramdisk to power itself off.

Some hardware may have a problem with the default approach and would require Ironic to talk directly to the management controller to switch the power off and on again. In order to tell Ironic to do that, you have to update the nodes `driver_info` field and set the `deploy_forces_oob_reboot` parameter with the value of **True**. For example, the below command sets this configuration in a specific node:

```
baremetal node set <UUID or name> --driver-info deploy_forces_oob_reboot=True
```

4.3.2 Networking with the Bare Metal service

Overview

Ironic contains several different networking use models and is largely built around an attachment being requested by the user, be it the `nova-compute` service on behalf of a Nova user, or directly using the vif attachment (`openstack baremetal node vif attach` or `baremetal node vif attach` commands).

Ironic manages the requested attachment state of the vif with the Networking service, and depending on the overall *network-interfaces* chosen, Ironic will perform additional actions such as attaching the node to an entirely separate provider network to improve the overall operational security.

The underlying `network_interface` chosen, covered in *network-interfaces* has significant power in the overall model and use of Ironic, and operators should choose accordingly.

Concepts

Terminology

- `physical network` - A physical network in the context of this document is a logical network fabric which works together. This *can* be represented for the purposes of modeling the infrastructure, and this terminology is somewhat over-used due to the similar filed name `physical_network` which refers to the named and configured representation of the network fabric.
- `vif` or `VIF` - Virtual Interface which is best described as a Neutron port. For Ironic, VIFs are always referred to utilizing the Neutron port ID value, which is a UUID value. A VIF may be available across a limited number of physical networks, dependent upon the clouds operating configuration and operating constraints.
- `ML2` - ML2 is a plugin model for Neutron, the Networking service. Advanced networking interactions including 3rd party plugins are utilized in this model along with some community plugins to achieve various actions.
- `provisioning network` - A separate logical network where bare metal nodes are deployed to isolate a deploying node from a tenant controlled, deployed node.
- `cleaning network` - Similar to the `provisioning network`, but a network where machines undergo cleaning operations.
- `binding profile` - A set of information sent to Neutron which includes the type of port, in Ironics case this is always `VNIC_BAREMETAL`, a `host_id` value matching the baremetal nodes

UUID, and any additional information like the `local_link_connection` information we will cover later on in this document, which tells an ML2 plugin where the physical baremetal port is attached, enabling network switch fabric configuration be appropriately updated.

- `port` - A port is the context of this document represents a physical or logical connection. For example, in the context of Ironic, a port is a possible physical connection which is connected for use to the underlying `physical network`. Whereas in the context of Neutron, a port is a virtual network interface, or in Ironic terminology, a `VIF`.
- `port group` - A composite set of ports, created utilizing Ironics API to represent LACP or otherwise bonded network interfaces which exist between a network fabric and the physical bare metal node.

Network interfaces

Network interface is one of the driver interfaces that manages network switching for nodes. There are 4 network interfaces available in the Ironic service:

- `noop` interface is used for standalone deployments, and does not perform any network switching;
- `flat` interface places all nodes into a single provider network that is pre-configured on the Networking service and physical equipment. Nodes remain physically connected to this network during their entire life cycle. The supplied VIF attachment record is updated with new DHCP records as needed. When using this network interface, the VIF needs to have been created on the same network upon which the bare metal node is physically attached.
- `neutron` interface provides tenant-defined networking through the Networking service, separating tenant networks from each other and from the provisioning and cleaning provider networks. Nodes will move between these networks during their life cycle. This interface requires Networking service support for the switches attached to the baremetal servers so they can be programmed. This interface generally requires use of ML2 plugins or other Neutron SDN integrations to facilitate the port configuration actions in the network fabric. When using IPv6, use of the `neutron` interface is highly recommended as use of the `dhcpv6-stateful` configuration model for IPv6 with Neutron also automatically creates multiple address records for stateful address resolution.
- `ironic-networking` interface provides network switching for standalone Ironic deployments without requiring the Networking service (Neutron). This interface communicates directly with the ironic-networking service to configure switch ports based on switchport configuration stored in each ports `extra` field. Nodes move between provisioning, cleaning, rescuing, inspection, and tenant networks during their life cycle. This interface is suitable for environments where Ironic manages the network fabric directly. This driver has different configuration requirements and doesnt follow the VIF attachment flow or other related workflows described in this document. Instead, see *Configure the Standalone Networking Service* for more details.

To use these interfaces, they need to be enabled in *ironic.conf* utilizing the *enabled_network_interfaces* setting.

VIF Attachment flow

When creating a virtual interface (VIF), the action occurs against the Neutron Networking Service, such as by using the `openstack port create` command, and then the port ID is submitted to Ironic to facilitate a VIF attachment.

Note

Instances created with Nova can have a Neutron port created under a variety of different circumstances. It is *highly* recommended, when using Nova, to explicitly declare the port(s) as part of your Nova instance creation request. When Nova asks Ironic to deploy a node, nova attempts to record all VIFs the user requested to be attached into Ironic, and then generate user friendly metadata as a result.

When a virtual interface (VIF) is requested to be attached to a node via the Ironic API, the following ordered criteria are used to select a suitable unattached port or port group:

- Require ports or port groups to not have a physical network or to have a physical network that matches one of the VIFs available physical networks which it may be configured across.
- Prefer ports and port groups which have a physical network to ports and port groups which do not have a physical network.
- Prefer port groups to ports, prefer ports with PXE enabled.

Note

This sequence also assumes the overall request was sent to Ironic without a user declared port or port group preference in the attachment request. Nova integration does not support explicit declaration of which physical port to attach a VIF to, which is a constraint model differing from virtual machines.

Users wishing to have explicit declariation of mappings should consider use of Ironic directly to manage Bare Metal resources. This is done with the use of the `--port-uuid <port_uuid>` option with the `baremetal node vif attach` command.

As all VIFs are required to be attached to a port group or independent ports, the maximum number of VIFs is determined by the number of configured and available ports represented in Ironic, as framed by the suitability criteria noted above.

Ironic goes to attach *any* supplied, selected, or even self-created VIFs, in two distinct steps.

If a host is *not* active:

The first step, as a result of bug 2106073, Ironic provides an initial `host_id` to Neutron so neutron can perform any deferred address assignment to enable appropriate network mapping. This initial binding lacks all detail to enable binding to an exact port. For a short period of time, the VIF binding may show as ACTIVE in Neutron.

Note

If the port was bound in *advance* of being submitted to Ironic, and we must perform an initial bind, the port will be unbound and rebound as part of the workflow.

Once were handing the instance over to the user, Ironic sets the physical ports MAC address, and provides additional context, such as the physical switch and switchport metadata which may be used. This action effectively re-binds the port, and the port state in Neutron can change as a result if binding is successful or fails. That may, ultimately, have no realistic impact to availability of ability to pass traffic over an interface, but can be rooted in the overall `network_interface` driver in use as well.

Note

Ironics `network_interface` driver selection does influence the base binding model behavior and as such the resulting state as reported by Neutron. Specifically `flat` assumes a pre-wired, static, never changing physical network. Neutron port states indicating a failure when using the `flat` interface is often more a sign of the `networking-baremetal` ML2 plugin not being configured in Neutron. The `neutron` interface is far more dynamic and the binding state can generally be relied upon if any operator configured ML2 plugins are functional.

If the host in in an *active* state:

Ironic explicitly sets the physical ports MAC address for which the VIF will be bound, and is immediately attached to the host with any required metadata for port binding, which is then transmitted to Neutron.

A port binding failure will, by default, forcibly abort deployment on the `neutron` interface to prevent scenarios where instances appear as `ACTIVE` but are actually connected to the wrong network (e.g., remaining on the provisioning network instead of their assigned VLAN), leading to non-functional networking.

This behavior can be adjusted using the *`neutron.fail_on_port_binding_failure`* configuration option (default: `true`). When set to `false`, failures will only log a warning.

For Smart NICs, port binding failures are treated as fatal regardless of the global configuration setting, as proper network configuration is essential for these devices.

Operators can override both the global configuration and Smart NIC default behavior on a per-node basis using the `fail_on_binding_failure` field in the nodes `driver_info`.

When Ironic goes to unbind the VIF, Ironic makes a request for Neutron to reset the assigned MAC address in order to avoid conflicts with Neutrons unique hardware MAC address requirement.

Note

For advanced networking scenarios where network bindings need to be refreshed on an active instance without rebooting, the `neutron` network interface provides a `reattach_networking` service step. This is useful for VXLAN topology changes, network infrastructure migrations, or recovering from stale binding states. See *Node servicing* for more details on using this service step, and *Troubleshooting Ironic* for detailed usage guidance in the Reattaching networking on active instances section.

Basic Provisioning flow

When provisioning, Ironic will attempt to attach all PXE enabled ports to the *provisioning network*. A modifier for this behavior is the *`neutron.add_all_ports`* option, where ironic will attempt to bind all ports to the required service network beyond the ironic ports with `pxe_enabled` set to `True`.

After provisioning work has been completed, and prior to the node being moved to the `ACTIVE` `provision_state`, the previously attached ports are unbound.

In the case of the `network_interface` set to `flat`, the requested VIF(s) utilized for all binding configurations in all states.

In the case of the `network_interface` set to `neutron`, the user requested VIFs are attached to the Ironic node for the first time, as the time spent in the *provisioning network* was utilizing VIFs which Ironic created and then deleted as part of the baremetal nodes movement through the state machine.

The same flow and logic applies to *cleaning*, *service*, and *rescue* workflows.

How are VIFs configured on the deployed machine

The general expectation is that the deployed operating system will utilize DHCP based autoconfiguration to establish the required configuration into running state for the newly provisioned machine automatically.

We do not suggest nor recommend attempting to utiize a mix of static configuration and dynamic configuration. That being said, tools like Glean and cloud-init may be useful to enable metadata translation to static system configuration in cases where it is needed.

Advanced Port and Network/VIF Attachment Flow with Trait Based Networking

Ironic can also be configured to dynamically setup a nodes ports and networks based on traits defined on a nodes `instance_info.traits` field at `vif_attach` time. Please see *Trait Based Networking* for more information.

Physical networks

Note

Physical networks work alongside *Conductor Groups* and *Availability Zones and Resource Isolation* to provide complete resource partitioning. Consider all three mechanisms when designing your network topology.

An Ironic port may be associated with a physical network using its `physical_network` field. Ironic uses this information when mapping between virtual ports in Neutron and physical ports and port groups. A ports physical network field is optional, and if not set then any VIF may be mapped to that port, provided that no free Ironic port with a suitable physical network assignment exists. When set, its value must be a name that corresponds to a physical network in Neutron for the networking-baremetal integration to properly reconcile the port. This has to be a name (not a UUID) since physical networks are configured in Neutron by name.

The physical network of a port group is explicitly defined and constituent ports should match. The Ironic service ensures that all ports in a port group have the same value in their physical network field.

The `physical_network` setting is used to have divided network fabrics which may carry different sets of traffic, and is intended to help model the reality multiple network fabrics into the overall operation with Neutron.

Local link connection

Use of the `neutron` or `ironic-networking` *network-interfaces* requires the Ironic port `local_link_connection` information to be populated for each Ironic port on a node in Ironic.

In the context of the `neutron` network interface, this information is provided to the networking services ML2 driver when a Virtual Interface (VIF) is attached. The ML2 driver uses the information to plug the specified port to the tenant network.

In the context of the `ironic-networking` network interface, this information is used by the Ironic Networking service to identify the switch port to be configured.

This information is typically populated through the introspection process by using LLDP data being broadcast from the switches, but may need to be manually set or changed in the case of a physical networking change, such as when a baremetal ports cable has been moved to a different port on a switch, or the switch has been replaced.

Note

For auto-discovery of values to work as part of introspection, switches must have LLDP enabled.

Note

Decoding LLDP data is performed as a best effort action. Some switch vendors, or changes in switch vendor firmware may impact field decoding. While this is rare, please report issues such as this to the Ironic project as bugs.

Table 19: `local_link_connection` fields

Field	Description
`switch_ic`	Required. Identifies a switch and can be a MAC address or an OpenFlow-based `datapath_id`.
`port_id`	Required. Port ID on the switch/Smart NIC, for example, Gig0/1, rep0-0.
`switch_ir`	Optional. Used to distinguish different switch models or other vendor-specific identifier. Some ML2 plugins may require this field.
`hostname`	Required in case of a Smart NIC port. Hostname of Smart NIC device.

Note

This isnt applicable to Infiniband ports because the network topology is discoverable by the Infiniband Subnet Manager. If specified, local_link_connection information will be ignored. If port is Smart NIC port then:

1. `port_id` is the representor port name on the Smart NIC.
2. `switch_id` is not mandatory.

Example setting of local link connection information

Below is an example command you can use as a basis to set the required information into Ironic.

```
baremetal port create <physical_mac_address> --node <node_uuid> \
     --local-link-connection switch_id=<switch_mac_address> \
     --local-link-connection switch_info=<switch_hostname> \
     --local-link-connection port_id=<switch_port_for_connection> \
     --pxe-enabled true \
     --physical-network physnet1
```

Warning

Depending on your ML2 plugin, you may need different or additional data to be provided as part of the `local_link_connection` information.

Alternatively, if you just need to update an existing value, such as the `port_id` value due to a cabling

change, you can use the *baremetal port set* command.

```
baremetal port set --node <node_uuid> \
    --local-link-connection port_id=<updated_switch_port_for_connection \
    <baremetal_port_uuid>
```

Example setting an Infiniband Port with local link connection information

Infiniband port requires require use of a client ID, where local link connection information is intended to be populated by the Infiniband Subnet Manager.

The client ID consists of <12-byte vendor prefix>:<8 byte port GUID>. There is no standard process for deriving the ports MAC address ($HW_MAC_ADDRESS); it is vendor specific.

For example, Mellanox ConnectX Family Devices prefix is ff:00:00:00:00:00:02:00:00:02:c9:00. If port GUID was f4:52:14:03:00:38:39:81 the client ID would be ff:00:00:00:00:00:02:00:00:02:c9:00:f4:52:14:03:00:38:39:81.

Mellanox ConnectX Family Devices HW_MAC_ADDRESS consists of 6 bytes; the port GUIDs lower 3 and higher 3 bytes. In this example it would be f4:52:14:38:39:81. Putting it all together, create an Infiniband port as follows.

```
baremetal port create <physical_mac_address> --node <node_uuid> \
     --pxe-enabled true \
     --extra client-id=<client_id> \
     --physical-network physnet1
```

Example setting a Smart NIC port

Smart NIC usage is a very specialized use case where an ML2 plugin as part of an infrastructure or the `neutron-l2-agent` is installed in the operating system on the Smart NIC *and* the service is configured to speak with the rest of the OpenStack deployment.

When a Smart NIC is present which is integrated in this fashion, Ironic needs to be aware to ensure overall chasiss power is in a state which is suitable to ensure that the port can be attached. i.e. The card can be programmed remotely.

To signal to Ironic the device and connection is supplied via a Smart NIC, use the following command. This requires the `hostname` of the operating system inside the Smart NIC to asserted along with the `port_id` value to match the internal port representation name.

```
baremetal port create <physical_mac_address> --node <node_uuid> \
     --local-link-connection hostname=<smartnic_hostname> \
     --local-link-connection port_id=<internal_port_name> \
     --pxe-enabled true \
     --physical-network physnet1 \
     --is-smartnic
```

Configuring and using Network Multi-tenancy

See the *Configure tenant networks* section in the installation guide for the Bare Metal (Ironic) service.

Configuring the Networking service

In addition to configuring Ironic, some additional configuration of the Neutron is required to ensure ports for bare metal servers are correctly programmed *and* represent a proper state, depending on your use model.

This configuration is determined by the Ironic network interface drivers you have enabled, which top of rack switches you have in your environment, and ultimately the structural model of your network, as in if your using `physical_network` values.

Physnet Mapping

When using physnet mapping, it is critical for proper instance scheduling for network resources to be informed of the physical network mappings which are represented in relation to the hosts in the deployment.

This takes the form of the `ironic-neutron-agent` which operators should deploy. Information on how to setup and configure this agent can be located at in the networking-baremetal installation documentation for the ironic-neutron-agent.

`flat` network interface

In order for Networking service ports to correctly operate with the Ironic service `flat` network interface the `baremetal` ML2 mechanism driver from networking-baremetal needs to be loaded into the Neutron configuration. This driver understands that the switch should be already configured by the admin, and will mark the networking service ports as successfully bound as nothing else needs to be done for the `VNIC_BAREMETAL` binding requests which made by Ironic on behalf of users seeking their ports to be attached.

1. Install the `networking-baremetal` library

   ```
   $ pip install networking-baremetal
   ```

2. Enable the `baremetal` driver in the Networking service ML2 configuration file

   ```
   [ml2]
   mechanism_drivers = ovs,baremetal
   ```

3. Restart your Neutron API service, which houses the ML2 mechanism drivers.

`neutron` network interface

The `neutron` network interface allows the Networking service to program the physical top of rack switches for the bare metal servers. To do this an ML2 mechanism driver which supports the `baremetal` VNIC type for the make and model of top of rack switch in the environment must be installed and enabled.

One case where you may wish to prefer the `neutron` network interface, even when your architecture is statically configured interfaces similar to `flat` networks, is when your using IPv6. Various hardware, bootloader, and Operating System DHCP clients utilize different techniques for generating the host identifier string which DHCP servers utilize to track IPv6 hosts. The `neutron` interface generates additional IPv6 DHCP entries to account for situations such as this, where as the `flat` interface is unable to do so.

This is a list of known top of rack ML2 mechanism drivers which work with the `neutron` network interface.

Community ML2 Drivers

Community ML2 drivers are drivers maintained by the community, and can be expected to generally focus on the minimum viable need to facilitate use cases.

Networking Generic Switch
: This ML2 mechanism driver is generally viewed as the go-to solution to get started. It is modeled upon remote switch configuration using text interfaces, and the minimum feature for each switch is setting a port on a vlan. This ML2 driver is tested in CI as it also supports management of some virtual machine networking as Ironic uses it in CI. It is also relatively simple to modify to enable support for newer models, or changes in vendor command lines. It also has some defects and issues, but is still viewed as the first go-to solution to get started. More information is available in the projects README. The projects documentation can also be found here.

Networking Baremetal
: This ML2 mechanism driver, which we briefly cover in the `flat` network interface settings earlier in this document, also has support for asserting configuration to remote switches using Netconf with the OpenConfig data model. This, similar to the issues with DMTF Redfish, means that it doesnt work for every Netconf supported switch. More information can be found at networking-baremetal documentation and device-drivers documentation with some additional detail covered on how to configure devices to manage.

Vendor ML2 Drivers

Cisco Nexus (networking-cisco)
: To install and configure this ML2 mechanism driver see Nexus Mechanism Driver Installation Guide. This driver does appear to be maintained by the vendor, but the Ironic community is unaware of its status otherwise.

Arista (networking-arista)
: The networking-arista project does appear to have some logic to handle the VNIC_BAREMETAL requests, and Arista was deeply involved when the overall model of ML2 switch orchestration was created. Limited information is available, but the repository can be found at on OpenDev in the x/networking-arista repository.

Previously in this list we included networking-fujitsu, however it no longer appears maintained. Customers of Fujitsu products should inquire with Fujitsu directly.

4.3.3 Port groups support

The Bare Metal service supports static configuration of port groups (bonds) in the instances via config-drive. See kernel documentation on bonding to see why it may be useful and how it is setup in linux. The sections below describe how to make use of them in the Bare Metal service.

Switch-side configuration

If port groups are desired in the ironic deployment, they need to be configured on the switches. It needs to be done manually, and the mode and properties configured on the switch have to correspond to the mode and properties that will be configured on the ironic side, as bonding mode and properties may be named differently on your switch, or have possible values different from the ones described in kernel documentation on bonding. Please refer to your switch configuration documentation for more details.

Provisioning and cleaning cannot make use of port groups if they need to boot the deployment ramdisk via (i)PXE. If your switches or desired port group configuration do not support port group fall-

back, which will allow port group members to be used by themselves, you need to set port groups `standalone_ports_supported` value to be `False` in ironic, as it is `True` by default.

Physical networks

If any port in a port group has a physical network, then all ports in that port group must have the same physical network.

Changing the physical network of a port group will cause the new value to be propagated to each member port of the group. The physical network of an individual port cannot be changed in isolation while it belongs to a port group.

See *physical networks* for further information on using physical networks in the Bare Metal service.

Port groups configuration in the Bare Metal service

Port group configuration is supported in ironic API microversions 1.26, the CLI commands below specify it for completeness.

1. When creating a port group, the node to which it belongs must be specified, along with, optionally, its name, address, mode, properties, and if it supports fallback to standalone ports:

```
baremetal port group create \
--node $NODE_UUID --name test --address fa:ab:25:48:fd:ba --mode 802.3ad \
--property miimon=100 --property xmit_hash_policy="layer2+3" \
--support-standalone-ports
```

   A port group can also be updated with `baremetal port group set` command, see its help for more details.

   If an address is not specified, the port group address on the deployed instance will be the same as the address of the neutron port that is attached to the port group. If the neutron port is not attached, the port group will not be configured.

   Note

   In standalone mode, port groups have to be configured manually. It can be done either statically inside the image, or by generating the configdrive and adding it to the nodes `instance_info`. For more information on how to configure bonding via configdrive, refer to cloud-init documentation and code. cloud-init version 0.7.7 or later is required for bonding configuration to work.

   The following is a simple sample for configuring bonding via configdrive:

   When booting an instance, it needs to add user-data file for configuring bonding via `--user-data` option. For example:

```
{
  "networks": [
    {
      "type": "physical",
      "name": "eth0",
      "mac_address": "fa:ab:25:48:fd:ba"
    },
    {
      "type": "physical",
```

```
      "name": "eth1",
      "mac_address": "fa:ab:25:48:fd:ab"
    },
    {
      "type": "bond",
      "name": "bond0",
      "bond_interfaces": [
        "eth0", "eth1"
        ],
        "mode": "active-backup"
    }
  ]
}
```

If the port groups address is not explicitly set in standalone mode, it will be set automatically by the process described in kernel documentation on bonding.

During interface attachment, port groups have higher priority than ports, so they will be used first. (It is not yet possible to specify which one is desired, a port group or a port, in an interface attachment request). Port groups that dont have any ports will be ignored.

The mode and properties values are described in the kernel documentation on bonding. The default port group mode is `active-backup`, and this default can be changed by setting the *DEFAULT.default_portgroup_mode* configuration option in the ironic API service configuration file.

2. Associate ports with the created port group.

   It can be done on port creation:

   ```
   baremetal port create \
   --node $NODE_UUID --address fa:ab:25:48:fd:ba --port-group test
   ```

   Or by updating an existing port:

   ```
   baremetal port set $PORT_UUID --port-group $PORT_GROUP_UUID
   ```

   When updating a port, the node associated with the port has to be in `enroll`, `manageable`, or `inspecting` states. A port group can have the same or different address as individual ports.

3. Boot an instance (or node directly, in case of using standalone ironic) providing an image that has cloud-init version 0.7.7 or later and supports bonding.

When the deployment is done, you can check that the port group is set up properly by running the following command in the instance:

```
cat /proc/net/bonding/bondX
```

where `X` is a number autogenerated by cloud-init for each configured port group, in no particular order. It starts with 0 and increments by 1 for every configured port group.

Link aggregation/teaming on windows

Portgroups are supported for Windows Server images, which can created by *Building images for Windows* instruction.

You can customise an instance after it is launched along with script file in `Configuration` of `Instance` and selected `Configuration Drive` option. Then ironic virt driver will generate network metadata and add all the additional information, such as bond mode, transmit hash policy, MII link monitoring interval, and of which links the bond consists. The information in InstanceMetadata will be used afterwards to generate the config drive.

4.3.4 Conductor Groups

See also

For a complete guide on achieving availability zone functionality, see *Availability Zones and Resource Isolation*.

Overview

Conductor groups provide **physical resource partitioning** in Ironic, similar to Novas availability zones but focused on conductor-level management. They work alongside *shards* to provide complete resource isolation and operational scaling capabilities.

Large-scale operators tend to have needs that involve creating well-defined and delineated resources. In some cases, these systems may reside close by or in faraway locations. The reasoning may be simple or complex, and yet is only known to the deployer and operator of the infrastructure.

A common case is the need for delineated high-availability domains where it would be much more efficient to manage a datacenter in Antarctica with a conductor in Antarctica, as opposed to a conductor in New York City.

How it works

Starting in ironic 11.1, each node has a `conductor_group` field which influences how the ironic conductor calculates (and thus allocates) baremetal nodes under ironics management. This calculation is performed independently by each operating conductor and as such if a conductor has a *`conductor.conductor_group`* configuration option defined in its `ironic.conf` configuration file, the conductor will then be limited to only managing nodes with a matching `conductor_group` string.

Note

Any conductor without a *`conductor.conductor_group`* setting will only manage baremetal nodes without a `conductor_group` value set upon node creation. If no such conductor is present when conductor groups are configured, node creation will fail unless a `conductor_group` is specified upon node creation.

Warning

Nodes without a `conductor_group` setting can only be managed when a conductor exists that does not have a *`conductor.conductor_group`* defined. If all conductors have been migrated to use a

conductor group, such nodes are effectively orphaned.

How to use

A conductor group value may be any case-insensitive string up to 255 characters long which matches the `^[a-zA-Z0-9_\-\.]*$` regular expression.

1. Set the *conductor.conductor_group* option in ironic.conf on one or more, but not all conductors:

```
[conductor]
conductor_group = OperatorDefinedString
```

2. Restart the ironic-conductor service.
3. Set the conductor group on one or more nodes:

```
baremetal node set \
    --conductor-group "OperatorDefinedString" <uuid>
```

4. As desired and as needed, the remaining conductors can be updated with the first two steps. Please be mindful of the constraints covered earlier in the document related to the ability to manage nodes.

Advanced Usage with Multiple Deployments

Conductor groups work within a single Ironic deployment. For complete service isolation across geographic regions or regulatory boundaries, consider using *multiple Ironic deployments* targeted by different Nova compute services.

See Also

- *Availability Zones and Resource Isolation* - Complete availability zone strategy
- *Networking with the Bare Metal service* - Physical network considerations
- *Reference Deploy Architectures* - Reference architectures

4.3.5 Security Overview

While the Bare Metal service is intended to be a secure application, it is important to understand what it does and does not cover today.

Deployers must properly evaluate their use case and take the appropriate actions to secure their environment(s). This document is intended to provide an overview of what risks an operator of the Bare Metal service should be aware of. It is not intended as a How-To guide for securing a data center or an OpenStack deployment.

Image Checksums

Ironic has long provided a capacity to supply and check a checksum for disk images being deployed. However, one aspect which Ironic has not asserted is Why? in terms of Is it for security? or Is it for data integrity?.

The answer is both to ensure a higher level of security with remote image files, *and* provide faster feedback should a image being transferred happens to be corrupted.

Normally checksums are verified by the `ironic-python-agent` **OR** the deployment interface responsible for overall deployment operation. That being said, not *every* deployment interface relies on disk images which have checksums, and those deployment interfaces are for specific use cases which Ironic users leverage, outside of the general use case capabilities provided by the `direct` deployment interface.

Note

Use of the node `instance_info/image_checksum` field is discouraged for integrated OpenStack Users as usage of the matching Glance Image Service field is also deprecated. That being said, Ironic retains this feature by popular demand while also enabling also retain simplified operator interaction. The newer field values supported by Glance are also specifically supported by Ironic as `instance_info/image_os_hash_value` for checksum values and `instance_info/image_os_hash_algo` field for the checksum algorithm.

Warning

Setting a checksum value to a URL is supported, *however* doing this is making a tradeoff with security as the remote checksum *can* change. Conductor support this functionality can be disabled using the *`conductor.disable_support_for_checksum_files`* setting.

REST API: user roles and policy settings

By default, users are authenticated and authorization details are provided to Ironic as part web APIs operating security model and interaction with keystone.

Default REST API user roles and policy settings have evolved, starting in the Wallaby development cycle, into a model often referred to in the OpenStack community as `Secure RBAC`. This model is intended balance usability, while leaning towards a secure-by-default state. You can find more information on this at *Secure RBAC*.

Operators may choose to override default, in-code, Role Based Access Control policies by utilizing override policies, which you can learn about at *Policies*.

Conductor Operation

Ironic relies upon the REST API to validate, authenticate, and authorize user requests and interactions. While the conductor service *can* be operated with the REST API in a single process, the normal operating mode is as separate services either connected to a Message bus or use of an authenticated JSON-RPC endpoint.

Multi-tenancy

There are two aspects of multitenancy to consider when evaluating a deployment of the Bare Metal Service: interactions between tenants on the network, and actions one tenant can take on a machine that will affect the next tenant.

Network Interactions

Interactions between tenants workloads running simultaneously on separate servers include, but are not limited to: IP spoofing, packet sniffing, and network man-in-the-middle attacks.

By default, the Bare Metal service provisions all nodes on a flat network, and does not take any precautions to avoid or prevent interaction between tenants. This can be addressed by integration with the OpenStack Identity, Compute, and Networking services, so as to provide tenant-network isolation. Additional documentation on network multi-tenancy is available.

Lingering Effects

Interactions between tenants placed sequentially on the same server include, but are not limited to: changes in BIOS settings, modifications to firmware, or files left on disk or peripheral storage devices (if these devices are not erased between uses).

By default, the Bare Metal service will erase (clean) the local disk drives during the cleaning phase, after deleting an instance. It *does not* reset BIOS or reflash firmware or peripheral devices. This can be addressed through customizing the utility ramdisk used during the cleaning phase. See details in the *Firmware security* section.

Firmware security

When the Bare Metal service deploys an operating system image to a server, that image is run natively on the server without virtualization. Any user with administrative access to the deployed instance has administrative access to the underlying hardware.

Most servers default settings do not prevent a privileged local user from gaining direct access to hardware devices. Such a user could modify device or firmware settings, and potentially flash new firmware to the device, before deleting their instance and allowing the server to be allocated to another user.

If the `[conductor]/automated_clean` configuration option is enabled (and the `[deploy]/erase_devices_priority` configuration option is not zero), the Bare Metal service will securely erase all local disk devices within a machine during instance deletion. However, the service does not ship with any code that will validate the integrity of, or make any modifications to, system or device firmware or firmware settings.

Operators are encouraged to write their own hardware manager plugins for the `ironic-python-agent` ramdisk. This should include custom `clean steps` that would be run during the *Node cleaning* process, as part of Node de-provisioning. The `clean steps` would perform the specific actions necessary within that environment to ensure the integrity of each servers firmware.

Ideally, an operator would work with their hardware vendor to ensure that proper firmware security measures are put in place ahead of time. This could include:

- installing signed firmware for BIOS and peripheral devices
- using a TPM (Trusted Platform Module) to validate signatures at boot time
- booting machines in *UEFI secure boot mode*, rather than BIOS mode, to validate kernel signatures
- disabling local (in-band) access from the host OS to the management controller (BMC)
- disabling modifications to boot settings from the host OS

Additional references:

- *Node cleaning*

UEFI secure boot mode

Secure Boot is an interesting topic because exists at an intersection of hardware, security, vendors, and what you are willing to put in place to in terms of process, controls, or further mechanisms to enable processes and capabilities.

At a high level, Secure Boot is where an artifact such as an operating system kernel or Preboot eXecution Environment (PXE) binary is read by the UEFI firmware, and executed if the artifact is signed with a trusted key. Once a piece of code has been loaded and executed, it may read more bytecode in and verify additional signed artifacts which were signed utilizing different keys.

This is fundamentally how most Linux operating systems boot today. A `shim` loader is signed by an authority, Microsoft, which is generally trusted by hardware vendors. The shim loader then loads a boot loader such as Grub, which then loads an operating system.

Underlying challenges

A major challenge for Secure Boot is the state of Preboot eXecution Environment binaries. Operating System distribution vendors tend not to request the authority with the general signing keys to sign these binary artifacts. The result of this, is that it is nearly impossible to network boot a machine which has Secure Boot enabled.

There are reports in the Open Source community that Microsoft has been willing to sign iPXE binaries, however the requirements are a bit steep for Open Source and largely means that Vendors would need to shoulder the burden for signed iPXE binaries to become common place. The iPXE developers provide further details on their website, but it provides the details which solidify why were unlikely to see a signed iPXE loader.

That is, unless, you sign iPXE yourself.

Which you can do, but you need to put in place your own key management infrastructure and teach the hardware to trust your signature, which is no simple feat in itself.

Note

The utility to manage keys in Linux on a local machine is *mokutil*, however its modeled for manual invocation. One may be able to manage keys via Baseboard Management Controller, and Ironic may add such capabilities at some point in time.

There is a possibility of utilizing shim and Grub2 to network boot a machine, however Grub2s capabilities for booting a machine are extremely limited when compared to a tool like iPXE. It is also worth noting the bulk of Ironics example configurations utilize iPXE, including whole activities like unmanaged hardware introspection with agent inspect interface.

Driver support for Deployment with Secure Boot

Some hardware types support turning UEFI secure boot dynamically when deploying an instance. Currently these are *iLO driver* and *Redfish driver*.

Other drivers, such as *IPMI driver*, may be able to be manually configured on the host, but as there is not standardization of Secure Boot support in the IPMI protocol, you may encounter unexpected behavior.

Support for the UEFI secure boot is declared by adding the `secure_boot` capability in the `capabilities` parameter in the `properties` field of a node. `secure_boot` is a boolean parameter and takes value as `true` or `false`.

To enable `secure_boot` on a node add it to `capabilities`:

```
baremetal node set <node> --property capabilities='secure_boot:true'
```

Alternatively use /admin/inspection to automatically populate the secure boot capability.

Warning

UEFI secure boot only works in UEFI boot mode, see *Boot mode support* for how to turn it on and off.

Compatible images

Most mainstream and vendor backed Linux based public cloud images are already compatible with use of secure boot.

Using Shim and Grub2 for Secure Boot

To utilize Shim and Grub to boot a baremetal node, actions are required by the administrator of the Ironic deployment as well as the user of Ironics API.

For the Ironic Administrator

To enable use of grub to network boot baremetal nodes for activities such as managed introspection, node cleaning, and deployment, some configuration is required in ironic.conf.:

```
[DEFAULT]
enabled_boot_interfaces = pxe
[pxe]
uefi_pxe_config_template = $pybasedir/drivers/modules/pxe_grub_config.template
tftp_root = /tftpboot
loader_file_paths = bootx64.efi:/usr/lib/shimx64.efi.signed,grubx64.efi:/usr/
↪lib/grub/x86_64-efi-signed/grubnetx64.efi.signed
```

Note

You may want to leverage the *pxe.loader_file_paths* feature, which automatically copies boot loaders into the `tftp_root` folder, but this functionality is not required if you manually copy the named files into the Preboot eXecution Environment folder(s), by default the [pxe]tftp_root, and [deploy]http_root folders.

Warning

Shim/Grub artifact paths will vary by distribution. The example above is taken from Ironics Continuous Integration test jobs where this functionality is exercised.

For the Ironic user

To set a node to utilize the `pxe` boot_interface, execute the baremetal command:

```
baremetal node set --boot-interface pxe <node>
```

Alternatively, if your hardware supports HttpBoot and your Ironic is at least 2023.2, you can set the `http` boot_interface instead:

```
baremetal node set --boot-interface http <node>
```

Enabling with OpenStack Compute

Nodes having `secure_boot` set to `true` may be requested by adding an `extra_spec` to the nova flavor:

```
openstack flavor set <flavor> --property capabilities:secure_boot="true"
openstack server create --flavor <flavor> --image <image> instance-1
```

If `capabilities` is used in `extra_spec` as above, nova scheduler (`ComputeCapabilitiesFilter`) will match only ironic nodes which have the `secure_boot` set appropriately in `properties/capabilities`. It will filter out rest of the nodes.

The above facility for matching in nova can be used in heterogeneous environments where there is a mix of machines supporting and not supporting UEFI secure boot, and operator wants to provide a choice to the user regarding secure boot. If the flavor doesnt contain `secure_boot` then nova scheduler will not consider secure boot mode as a placement criteria, hence user may get a secure boot capable machine that matches with user specified flavors but deployment would not use its secure boot capability. Secure boot deploy would happen only when it is explicitly specified through flavor.

Enabling standalone

To request secure boot for an instance in standalone mode (without OpenStack Compute), you must explicitly inform Ironic:

```
baremetal node set secure boot on <node>
```

Which can also be disabled by exeuting negative form of the command:

```
baremetal node set secure boot off <node>
```

Other considerations

Internal networks

Access to networks which the Bare Metal service uses internally should be prohibited from outside. These networks are the ones used for management (with the nodes BMC controllers), provisioning, cleaning (if used) and rescuing (if used).

This can be done with physical or logical network isolation, traffic filtering, etc.

While the Ironic project has made strives to enable the API to be utilized by end users directly, we still encourage operators to be as mindful as possible to ensure appropriate security controls are in place to also restrict access to the service.

Management interface technologies

Some nodes support more than one management interface technology (vendor and IPMI for example). If you use only one modern technology for out-of-band node access, it is recommended that you disable IPMI since the IPMI protocol is not secure. If IPMI is enabled, in most cases a local OS administrator is able to work in-band with IPMI settings without specifying any credentials, as this is a DCMI specification requirement.

Tenant network isolation

If you use tenant network isolation, services (TFTP or HTTP) that handle the nodes boot files should serve requests only from the internal networks that are used for the nodes being deployed and cleaned.

TFTP protocol does not support per-user access control at all.

For HTTP, there is no generic and safe way to transfer credentials to the node.

Also, tenant network isolation is not intended to work with network-booting a node by default, once the node has been provisioned.

API endpoints for RAM disk use

There are three (unauthorized) endpoints in the Bare Metal API that are intended for use by the ironic-python-agent RAM disk. They are not intended for public use.

These endpoints can potentially cause security issues even though the logic around these endpoints is intended to be defensive in nature. Access to these endpoints from external or untrusted networks should be prohibited. An easy way to do this is to:

- set up two groups of API services: one for external requests, the second for deploy RAM disks requests.
- to disable unauthorized access to these endpoints in the (first) API services group that serves external requests, the following lines should be added to the policy.yaml file:

```
# Send heartbeats from IPA ramdisk
"baremetal:node:ipa_heartbeat": "!"

# Access IPA ramdisk functions
"baremetal:driver:ipa_lookup": "!"

# Continue introspection IPA ramdisk endpoint
"baremetal:driver:ipa_continue_inspection": "!"
```

Rate Limiting

Ironic has a concept of a concurrent action limit, which allows operators to restrict concurrent, long running, destructive actions.

The overall use case this was implemented for was to help provide backstop for runaway processes and actions which one may apply to an environment, such as batch deletes of nodes. The appropriate settings for these settings are the *conductor.max_concurrent_deploy* with a default value of 250, and *conductor.max_concurrent_clean* with a default value of 50. These settings are reasonable defaults for medium to large deployments, but depending on load and usage patterns and can be safely tuned to be in line with an operators comfort level.

Memory Limiting

Because users of the Ironic API can request activities which can consume large amounts of memory, for example, disk image format conversions as part of a deployment operations. The Ironic conductor service has a minimum memory available check which is executed before launching these operations. It defaults to `1024` Megabytes, and can be tuned using the *`DEFAULT.minimum_required_memory`* setting.

Operators with a higher level of concurrency may wish to increase the default value.

BMC Credentials

Due to the nature of Ironic and the use of Baseboard Management Controllers to manage physical machines, Ironic has to be supplied with current credentials for each node it manages.

Ironic utilizes the credentials to perform actions on behalf of the API user in a manor which is both delineated and removes the need for external users to know the credentials. With this Ironic also monitors and helps enforce the requested power state of the remote machine, as well as to collect any sensor data when Ironic is configured to collect such data.

Should the credentials for baremetal nodes under Ironics management need to be changed, the credentials must also be updated in Ironic itself by updating the appropriate driver specific credential parameters on each baremetal node.

Under a normal sequence of events, Ironic will pickup the new credentials for the next management operation which is typically the power state sync. If the credentials are wrong or have the wrong level of access, then the node will enter a maintenance state automatically and a fault will be recorded. For more context on the power state sync and power fault recover, please see *Power Synchronization*. When this issue has been corrected with the credential, modern Ironic users should find the node recover automatically.

Warning

Operators reusing the same credentials with a centrally managed credential management system with separate BMC configuration should be careful about credential rotations. These operators risk causing the account to access the BMC to be locked out, and should likely explicitly set nodes to `maintenance` state prior to rotating credentials, and afterward unset the maintenance state. This is to avoid locking out access if the same username is utilized across the credential rotation. An alternative pattern is to create a *new* user, and rotate the nodes in Ironic by resetting the driver specific parameter with the new user name and password.

The Ironic project recommends:

- Unique passwords be used for each baremetal node.
- Operators leverage good password practices by not reusing the credentials for other tools or systems.

Disk Images

Ironic relies upon the `qemu-img` tool to convert images from a supplied disk image format, to a `raw` format in order to write the contents of a disk image to the remote device.

By default, only `qcow2` format is supported for this operation, however there have been reports other formats work when so enabled using the `[conductor]permitted_image_formats` configuration option.

Ironic takes several steps by default.

1. Ironic checks and compares supplied metadata with a remote authoritative source, such as the Glance Image Service, if available.
2. Ironic attempts to fingerprint the file type based upon available metadata and file structure. A file format which is not known to the image format inspection code may be evaluated as raw, which means the image would not be passed through `qemu-img`. When in doubt, use a `raw` image which you can verify is in the desirable and expected state.
3. The image then has a set of safety and sanity checks executed which look for unknown or unsafe feature usage in the base format which could permit an attacker to potentially leverage functionality in `qemu-img` which should not be utilized. This check, by default, occurs only through images which transverse *through* the conductor.
4. Ironic then checks if the fingerprint values and metadata values match. If they do not match, the requested image is rejected and the operation fails.
5. The image is then provided to the `ironic-python-agent`.

Images which are considered pass-through, as in they are supplied by an API user as a URL, or are translated to a temporary URL via available service configuration, are supplied as a URL to the `ironic-python-agent`.

Ironic can be configured to intercept this interaction and have the conductor download and inspect these items before the `ironic-python-agent` will do so, however this can increase the temporary disk utilization of the Conductor along with network traffic to facilitate the transfer. This check is disabled by default, but can be enabled using the `[conductor]conductor_always_validates_images` configuration option.

An option exists which forces all files to be served from the conductor, and thus force image inspection before involvement of the `ironic-python-agent` is the use of the `[agent]image_download_source` configuration parameter when set to `local` which proxies all disk images through the conductor. This setting is also available in the node `driver_info` and `instance_info` fields.

Mitigating Factors to disk images

In a fully integrated OpenStack context, Ironic requires images to be set to public in the Image Service.

A direct API user with sufficient elevated access rights *can* submit a URL for the baremetal node `instance_info` dictionary field with an `image_source` key value set to a URL. To do so explicitly requires elevated (trusted) access rights of a System scoped Member, or Project scoped Owner-Member, or a Project scoped Lessee-Admin via the `baremetal:node:update_instance_info` policy permission rule. Before the Wallaby release of OpenStack, this was restricted to `admin` and `baremetal_admin` roles and remains similarly restrictive in the newer Secure RBAC model.

4.3.6 Troubleshooting Ironic

Nova returns No valid host was found Error

Sometimes Nova Conductor log file nova-conductor.log or a message returned from Nova API contains the following error:

```
NoValidHost: No valid host was found. There are not enough hosts available.
```

No valid host was found means that the Nova Scheduler could not find a bare metal node suitable for booting the new instance.

This in turn usually means some mismatch between resources that Nova expects to find and resources that Ironic advertised to Nova.

A few things should be checked in this case:

1. Make sure that enough nodes are in `available` state, not in maintenance mode and not already used by an existing instance. Check with the following command:

   ```
   baremetal node list --provision-state available --no-maintenance --
   ↪unassociated
   ```

   If this command does not show enough nodes, use generic `baremetal node list` to check other nodes. For example, nodes in `manageable` state should be made available:

   ```
   baremetal node provide <IRONIC NODE>
   ```

   The Bare metal service automatically puts a node in maintenance mode if there are issues with accessing its management interface. See *Power fault and recovery* for details.

   The `node validate` command can be used to verify that all required fields are present. The following command should not return anything:

   ```
   baremetal node validate <IRONIC NODE> | grep -E '(power|management)\
   ↪W*False'
   ```

   Maintenance mode will be also set on a node if automated cleaning has failed for it previously.

2. Make sure that you have Compute services running and enabled:

   ```
   $ openstack compute service list --service nova-compute
   +----+--------------+-------------+------+---------+-------+--------------
   ↪--------------+
   | ID | Binary       | Host        | Zone | Status  | State | Updated At  ␣
   ↪              |
   +----+--------------+-------------+------+---------+-------+--------------
   ↪--------------+
   |  7 | nova-compute | example.com | nova | enabled | up    | 2017-09-
   ↪04T13:14:03.000000 |
   +----+--------------+-------------+------+---------+-------+--------------
   ↪--------------+
   ```

   By default, a Compute service is disabled after 10 consecutive build failures on it. This is to ensure that new build requests are not routed to a broken Compute service. If it is the case, make sure to fix the source of the failures, then re-enable it:

   ```
   openstack compute service set --enable <COMPUTE HOST> nova-compute
   ```

3. Starting with the Pike release, check that all your nodes have the `resource_class` field set using the following command:

   ```
   baremetal node list --fields uuid name resource_class
   ```

   Then check that the flavor(s) are configured to request these resource classes via their properties:

```
openstack flavor show <FLAVOR NAME> -f value -c properties
```

For example, if your node has resource class `baremetal-large`, it will be matched by a flavor with property `resources:CUSTOM_BAREMETAL_LARGE` set to `1`. See *Create flavors for use with the Bare Metal service* for more details on the correct configuration.

4. Upon scheduling, Nova will query the Placement API service for the available resource providers (in the case of Ironic: nodes with a given resource class). If placement does not have any allocation candidates for the requested resource class, the request will result in a No valid host was found error. It is hence sensible to check if Placement is aware of resource providers (nodes) for the requested resource class with:

```
$ openstack allocation candidate list --resource CUSTOM_BAREMETAL_LARGE='1
↪'
+---+------------------------------+--------------------------------------
↪+--------------------------------+
| # | allocation                   | resource provider                    ␣
↪| inventory used/capacity        |
+---+------------------------------+--------------------------------------
↪+--------------------------------+
| 1 | CUSTOM_BAREMETAL_LARGE=1     | 2f7b9c69-c1df-4e40-b94e-5821a4ea0453␣
↪| CUSTOM_BAREMETAL_LARGE=0/1     |
+---+------------------------------+--------------------------------------
↪+--------------------------------+
```

For Ironic, the resource provider is the UUID of the available Ironic node. If this command returns an empty list (or does not contain the targeted resource provider), the operator needs to understand first, why the resource tracker has not reported this provider to placement. Potential explanations include:

- the resource tracker cycle has not finished yet and the resource provider will appear once it has (the time to finish the cycle scales linearly with the number of nodes the corresponding `nova-compute` service manages);
- the node is in a state where the resource tracker does not consider it to be eligible for scheduling, e.g. when the node has `maintenance` set to `True`; make sure the target nodes are in `available` and `maintenance` is `False`;

5. The Nova flavor that you are using does not match any properties of the available Ironic nodes. Use

```
openstack flavor show <FLAVOR NAME>
```

to compare. The extra specs in your flavor starting with `capability:` should match ones in `node.properties['capabilities']`.

Note

The format of capabilities is different in Nova and Ironic. E.g. in Nova flavor:

```
$ openstack flavor show <FLAVOR NAME> -c properties
+------------+----------------------------------+
| Field      | Value                            |
```

```
+------------+-----------------------------------+
| properties | capabilities:boot_mode='uefi'     |
+------------+-----------------------------------+

But in Ironic node:
$ baremetal node show <IRONIC NODE> --fields properties
+------------+-------------------------------------------+
| Property   | Value                                     |
+------------+-------------------------------------------+
| properties | {u'capabilities': u'boot_mode:uefi'}      |
+------------+-------------------------------------------+
```

6. After making changes to nodes in Ironic, it takes time for those changes to propagate from Ironic to Nova. Check that

   ```
   openstack hypervisor stats show
   ```

   correctly shows total amount of resources in your system. You can also check `openstack hypervisor show <IRONIC NODE>` to see the status of individual Ironic nodes as reported to Nova.

7. Figure out which Nova Scheduler filter ruled out your nodes. Check the `nova-scheduler` logs for lines containing something like:

   ```
   Filter ComputeCapabilitiesFilter returned 0 hosts
   ```

   The name of the filter that removed the last hosts may give some hints on what exactly was not matched. See Nova filters documentation for more details.

8. If none of the above helped, check Ironic conductor log carefully to see if there are any conductor-related errors which are the root cause for No valid host was found. If there are any Error in deploy of node <IRONIC-NODE-UUID>: [Errno 28] error messages in Ironic conductor log, it means the conductor run into a special error during deployment. So you can check the log carefully to fix or work around and then try again.

Patching the Deploy Ramdisk

When debugging a problem with deployment and/or inspection you may want to quickly apply a change to the ramdisk to see if it helps. Of course you can inject your code and/or SSH keys during the ramdisk build (depends on how exactly youve built your ramdisk). But its also possible to quickly modify an already built ramdisk.

Create an empty directory and unpack the ramdisk content there:

```
$ mkdir unpack
$ cd unpack
$ gzip -dc /path/to/the/ramdisk | cpio -id
```

The last command will result in the whole Linux file system tree unpacked in the current directory. Now you can modify any files you want. The actual location of the files will depend on the way youve built the ramdisk.

Note

On a systemd-based system you can use the `systemd-nspawn` tool (from the `systemd-container` package) to create a lightweight container from the unpacked filesystem tree:

```
$ sudo systemd-nspawn --directory /path/to/unpacked/ramdisk/ /bin/bash
```

This will allow you to run commands within the filesystem, e.g. use package manager. If the ramdisk is also systemd-based, and you have login credentials set up, you can even boot a real ramdisk environment with

```
$ sudo systemd-nspawn --directory /path/to/unpacked/ramdisk/ --boot
```

After youve done the modifications, pack the whole content of the current directory back:

```
$ find . | cpio -H newc -o | gzip -c > /path/to/the/new/ramdisk
```

Note

You dont need to modify the kernel (e.g. `tinyipa-master.vmlinuz`), only the ramdisk part.

API Errors

The `debug_tracebacks_in_api` config option may be set to return tracebacks in the API response for all 4xx and 5xx errors.

Retrieving logs from the deploy ramdisk

When troubleshooting deployments (specially in case of a deploy failure) its important to have access to the deploy ramdisk logs to be able to identify the source of the problem. By default, Ironic will retrieve the logs from the deploy ramdisk when the deployment fails and save it on the local filesystem at `/var/log/ironic/deploy`.

To change this behavior, operators can make the following changes to `/etc/ironic/ironic.conf` under the `[agent]` group:

- `deploy_logs_collect`: Whether Ironic should collect the deployment logs on deployment. Valid values for this option are:
  - `on_failure` (**default**): Retrieve the deployment logs upon a deployment failure.
  - `always`: Always retrieve the deployment logs, even if the deployment succeed.
  - `never`: Disable retrieving the deployment logs.
- `deploy_logs_storage_backend`: The name of the storage backend where the logs will be stored. Valid values for this option are:
  - `local` (**default**): Store the logs in the local filesystem.
  - `swift`: Store the logs in Swift.
- `deploy_logs_local_path`: The path to the directory where the logs should be stored, used when the `deploy_logs_storage_backend` is configured to `local`. By default logs will be stored at **/var/log/ironic/deploy**.

- `deploy_logs_swift_container`: The name of the Swift container to store the logs, used when the deploy_logs_storage_backend is configured to swift. By default **ironic_deploy_logs_container**.
- `deploy_logs_swift_days_to_expire`: Number of days before a log object is marked as expired in Swift. If None, the logs will be kept forever or until manually deleted. Used when the deploy_logs_storage_backend is configured to swift. By default **30** days.

When the logs are collected, Ironic will store a *tar.gz* file containing all the logs according to the `deploy_logs_storage_backend` configuration option. All log objects will be named with the following pattern:

```
<node>[_<instance-uuid>]_<timestamp yyyy-mm-dd-hh:mm:ss>.tar.gz
```

Note

The *instance_uuid* field is not required for deploying a node when Ironic is configured to be used in standalone mode. If present it will be appended to the name.

Accessing the log data

When storing in the local filesystem

When storing the logs in the local filesystem, the log files can be found at the path configured in the `deploy_logs_local_path` configuration option. For example, to find the logs from the node `5e9258c4-cfda-40b6-86e2-e192f523d668`:

```
$ ls /var/log/ironic/deploy | grep 5e9258c4-cfda-40b6-86e2-e192f523d668
5e9258c4-cfda-40b6-86e2-e192f523d668_88595d8a-6725-4471-8cd5-c0f3106b6898_
↪2016-08-08-13:52:12.tar.gz
5e9258c4-cfda-40b6-86e2-e192f523d668_db87f2c5-7a9a-48c2-9a76-604287257c1b_
↪2016-08-08-14:07:25.tar.gz
```

Note

When saving the logs to the filesystem, operators may want to enable some form of rotation for the logs to avoid disk space problems.

When storing in Swift

When using Swift, operators can associate the objects in the container with the nodes in Ironic and search for the logs for the node `5e9258c4-cfda-40b6-86e2-e192f523d668` using the **prefix** parameter. For example:

```
$ swift list ironic_deploy_logs_container -p 5e9258c4-cfda-40b6-86e2-
↪e192f523d668
5e9258c4-cfda-40b6-86e2-e192f523d668_88595d8a-6725-4471-8cd5-c0f3106b6898_
↪2016-08-08-13:52:12.tar.gz
5e9258c4-cfda-40b6-86e2-e192f523d668_db87f2c5-7a9a-48c2-9a76-604287257c1b_
↪2016-08-08-14:07:25.tar.gz
```

To download a specific log from Swift, do:

```
$ swift download ironic_deploy_logs_container "5e9258c4-cfda-40b6-86e2-
↪e192f523d668_db87f2c5-7a9a-48c2-9a76-604287257c1b_2016-08-08-14:07:25.tar.gz
↪"
5e9258c4-cfda-40b6-86e2-e192f523d668_db87f2c5-7a9a-48c2-9a76-604287257c1b_
↪2016-08-08-14:07:25.tar.gz [auth 0.341s, headers 0.391s, total 0.391s, 0.
↪531 MB/s]
```

The contents of the log file

The log is just a `.tar.gz` file that can be extracted as:

```
$ tar xvf <file path>
```

The contents of the file may differ slightly depending on the distribution that the deploy ramdisk is using:

- For distributions using `systemd` there will be a file called **journal** which contains all the system logs collected via the `journalctl` command.
- For other distributions, the ramdisk will collect all the contents of the `/var/log` directory.

For all distributions, the log file will also contain the output of the following commands (if present): `ps`, `df`, `ip addr` and `iptables`.

Heres one example when extracting the content of a log file for a distribution that uses `systemd`:

```
$ tar xvf 5e9258c4-cfda-40b6-86e2-e192f523d668_88595d8a-6725-4471-8cd5-
↪c0f3106b6898_2016-08-08-13:52:12.tar.gz
df
ps
journal
ip_addr
iptables
```

DHCP during PXE or iPXE is inconsistent or unreliable

This can be caused by the spanning tree protocol delay on some switches. The delay prevents the switch port moving to forwarding mode during the nodes attempts to PXE, so the packets never make it to the DHCP server. To resolve this issue you should set the switch port that connects to your baremetal nodes as an edge or PortFast type port. Configured in this way the switch port will move to forwarding mode as soon as the link is established. An example on how to do that for a Cisco Nexus switch is:

```
$ config terminal
$ (config) interface eth1/11
$ (config-if) spanning-tree port type edge
```

Why does X issue occur when I am using LACP bonding with iPXE?

If you are using iPXE, an unfortunate aspect of its design and interaction with networking is an automatic response as a Link Aggregation Control Protocol (or LACP) peer to remote switches. iPXE does this for only the single port which is used for network booting.

In theory, this may help establish the port link-state faster with some switch vendors, but the official reasoning as far as the Ironic Developers are aware is not documented for iPXE. The end result of this is that once iPXE has stopped responding to LACP messages from the peer port, which occurs as part of the process of booting a ramdisk and iPXE handing over control to a full operating-system, switches typically begin a timer to determine how to handle the failure. This is because, depending on the mode of LACP, this can be interpreted as a switch or network fabric failure.

This may demonstrate as any number of behaviors or issues from ramdisks finding they are unable to acquire DHCP addresses over the network interface to downloads abruptly stalling, to even minor issues such as LLDP port data being unavailable in introspection.

Overall:

- Ironics agent doesnt officially support LACP and the Ironic community generally believes this may cause more problems than it would solve. During the Victoria development cycle, we added retry logic for most actions in an attempt to navigate the worst-known default hold-down timers to help ensure a deployment does not fail due to a short-lived transitory network connectivity failure in the form of a switch port having moved to a temporary blocking state. Where applicable and possible, many of these patches have been backported to supported releases. These patches also require that the switchport has an eventual fallback to a non-bonded mode. If the port remains in a blocking state, then traffic will be unable to flow and the deployment is likely to time out.
- If you must use LACP, consider `passive` LACP negotiation settings in the network switch as opposed to `active`. The difference being with passive the connected workload is likely a server where it should likely request the switch to establish the Link Aggregate. This is instead of being treated as if its possibly another switch.
- Consult your switch vendors support forums. Some vendors have recommended port settings for booting machines using iPXE with their switches.

IPMI errors

When working with IPMI, several settings need to be enabled depending on vendors.

Enable IPMI over LAN

Machines may not have IPMI access over LAN enabled by default. This could cause the IPMI port to be unreachable through ipmitool, as shown:

```
$ ipmitool -I lan -H ipmi_host -U ipmi_user -P ipmi_pass chassis power status
Error: Unable to establish LAN session
```

To fix this, enable `IPMI over lan` setting using your BMC tool or web app.

Troubleshooting lanplus interface

When working with lanplus interfaces, you may encounter the following error:

```
$ ipmitool -I lanplus -H ipmi_host -U ipmi_user -P ipmi_pass power status
Error in open session response message : insufficient resources for session
Error: Unable to establish IPMI v2 / RMCP+ session
```

To fix that issue, please enable `RMCP+ Cipher Suite3 Configuration` setting using your BMC tool or web app.

Why are my nodes stuck in a -ing state?

The Ironic conductor uses states ending with `ing` as a signifier that the conductor is actively working on something related to the node.

Often, this means there is an internal lock or `reservation` set on the node and the conductor is downloading, uploading, or attempting to perform some sort of Input/Output operation - see *Why does API return Node is locked by host?* for details.

In the case the conductor gets stuck, these operations should timeout, but there are cases in operating systems where operations are blocked until completion. These sorts of operations can vary based on the specific environment and operating configuration.

What can cause these sorts of failures?

Typical causes of such failures are going to be largely rooted in the concept of `iowait`, either in the form of downloading from a remote host or reading or writing to the disk of the conductor. An operator can use the iostat tool to identify the percentage of CPU time spent waiting on storage devices.

The fields that will be particularly important are the `iowait`, `await`, and `tps` ones, which can be read about in the `iostat` manual page.

In the case of network file systems, for backing components such as image caches or distributed `tftpboot` or `httpboot` folders, IO operations failing on these can, depending on operating system and underlying client settings, cause threads to be stuck in a blocking wait state, which is realistically undetectable short the operating system logging connectivity errors or even lock manager access errors.

For example with nfs, the underlying client recovery behavior, in terms of `soft`, `hard`, `softreval`, `nosoftreval`, will largely impact this behavior, but also NFS server settings can impact this behavior. A solid sign that this is a failure, is when an `ls /path/to/nfs` command hangs for a period of time. In such cases, the Storage Administrator should be consulted and network connectivity investigated for errors before trying to recover to proceed.

The bad news for IO related failures

If the node has a populated `reservation` field, and has not timed out or proceeded to a `fail` state, then the conductor process will likely need to be restarted. This is because the worker thread is hung with-in the conductor.

Manual intervention with-in Ironics database is *not* advised to try and un-wedge the machine in this state, and restarting the conductor is encouraged.

Note

Ironics conductor, upon restart, clears reservations for nodes which were previously managed by the conductor before restart.

If a distributed or network file system is in use, it is highly recommended that the operating system of the node running the conductor be rebooted as the running conductor may not even be able to exit in the state of an IO failure, again dependent upon site and server configuration.

File Size != Disk Size

An easy to make misconception is that a 2.4 GB file means that only 2.4 GB is written to disk. But if that files virtual size is 20 GB, or 100 GB things can become very problematic and extend the amount of time the node spends in `deploying` and `deploy wait` states.

Again, these sorts of cases will depend upon the exact configuration of the deployment, but hopefully these are areas where these actions can occur.

- Conversion to raw image files upon download to the conductor, from the *DEFAULT.force_raw_images* option. Users using Glance may also experience issues here as the conductor will cache the image to be written which takes place when the *agent.image_download_source* is set to `http` instead of `swift`.

Note

The QCOW2 image conversion utility does consume quite a bit of memory when converting images or writing them to the end storage device. This is because the files are not sequential in nature, and must be re-assembled from an internal block mapping. Internally Ironic limits this to 1GB of RAM. Operators performing large numbers of deployments may wish to disable raw images in these sorts of cases in order to minimize the conductor becoming a limiting factor due to memory and network IO.

Why are my nodes stuck in a wait state?

The Ironic conductor uses states containing `wait` as a signifier that the conductor is waiting for a callback from another component, such as the Ironic Python Agent or the Inspector. If this feedback does not arrive, the conductor will time out and the node will eventually move to a `failed` state. Depending on the configuration and the circumstances, however, a node can stay in a `wait` state for a long time or even never time out. The list of such wait states includes:

- `clean wait` for cleaning,
- `inspect wait` for introspection,
- `rescue wait` for rescuing, and
- `wait call-back` for deploying.

Communication issues between the conductor and the node

One of the most common issues when nodes seem to be stuck in a wait state occur when the node never received any instructions or does not react as expected: the conductor moved the node to a wait state but the node will never call back. Examples include wrong ciphers which will make ipmitool get stuck or BMCs in a state where they accept commands, but dont do the requested task (or only a part of it, like shutting off, but not starting). It is useful in these cases to see via a ping or the console if and which action the node is performing. If the node does not seem to react to the requests sent be the conductor, it may be worthwhile to try the corresponding action out-of-band, e.g. confirm that power on/off commands work when directly sent to the BMC. The section on *IPMI errors*. above gives some additional points to check. In some situations, a BMC reset may be necessary.

Ironic Python Agent stuck

Nodes can also get remain in a wait state when the component the conductor is waiting for gets stuck, e.g. when a hardware manager enters a loop or is waiting for an event that is never happening. In these cases, it might be helpful to connect to the IPA and inspect its logs, see the trouble shooting guide of the ironic-python-agent (IPA) on how to do this.

Stopping the operation

Cleaning, inspection, servicing and rescuing can be stopped while in `clean wait`, `inspect wait`, `service wait` and `rescue wait` states using the `abort` command. If the step running at the time has `abortable` parameter set to `True`, the abort will happen instantly, otherwise the process will be aborted once the currently running step is finished. The node will be moved to the corresponding failure state (`clean failed`, `inspect failed`, `service failed` or `rescue failed`):

```
baremetal node abort <node>
```

Deploying can be aborted while in the `wait call-back` state by starting an undeploy (normally resulting in cleaning):

```
baremetal node undeploy <node>
```

See *Bare Metal State Machine* for more details.

Note

Since the Bare Metal service is not doing anything actively in waiting states, the nodes are not moved to failed states on conductor restart.

Deployments fail with failed to update MAC address

The design of the integration with the Networking service (neutron) is such that once virtual ports have been created in the API, their MAC address must be updated in order for the DHCP server to be able to appropriately reply.

This can sometimes result in errors being raised indicating that the MAC address is already in use. This is because at some point in the past, a virtual interface was orphaned either by accident or by some unexpected glitch, and a previous entry is still present in Neutron.

This error looks something like this when reported in the ironic-conductor log output.:

> Failed to update MAC address on Neutron port 305beda7-0dd0-4fec-b4d2-78b7aa4e8e6a.: MacAddressInUseClient: Unable to complete operation for network 1e252627-6223-4076-a2b9-6f56493c9bac. The mac address 52:54:00:7c:c4:56 is in use.

Because we have no idea about this entry, we fail the deployment process as we cant make a number of assumptions in order to attempt to automatically resolve the conflict.

How did I get here?

Originally this was a fairly easy issue to encounter. The retry logic path which resulted between the Orchestration (heat) and Compute (nova) services, could sometimes result in additional un-necessary ports being created.

Most users of modern Ironic are unlikely to experience this except in situations where a Networking (neutron) VIF attachment was not removed or deleted prior to deleting and re-creating a port in the Bare Metal service. Users utilizing custom tooling may also experience a similar issue due to incorrect usage patterns, i.e. a port being pre-assigned a physical machines MAC address.

Ultimately, the key of this is that the port is being deleted, or a separate port was added outside of Ironics workflow. Under most operating circumstances, there really is no need to delete the port on a node, and VIF attachments are stored on the port object, so deleting the port *CAN* result in the VIF not being cleaned up from Neutron which ends up creating this sort of situation.

Under normal circumstances, when deleting ports, a node should be in a stable state, and the node should not be provisioned. If the `baremetal port delete` command fails, this may indicate that a known VIF is still attached. Generally if they are transitory from cleaning, provisioning, rescuing, or even inspection, getting the node to the `available` state will unblock your delete operation, that is unless there is a tenant VIF attahment. In that case, the vif will need to be removed from with-in the Bare Metal service using the `baremetal node vif detach` command.

A port can also be checked to see if there is a VIF attachment by consulting the ports `internal_info` field.

Warning

The `maintenance` flag can be used to force the nodes port to be deleted, however this will disable any check that would normally block the user from issuing a delete and accidentally orphaning the VIF attachment record.

How do I resolve this?

Generally, you need to identify the port with the offending MAC address. Example:

```
$ openstack port list --mac-address 52:54:00:7c:c4:56
```

From the commands output, you should be able to identify the `id` field. Using that, you can delete the port. Example:

```
$ openstack port delete <id>
```

Warning

Before deleting a port, you should always verify that it is no longer in use or no longer seems applicable/operable. If multiple deployments of the Bare Metal service with a single Neutron, the possibility that a inventory typo, or possibly even a duplicate MAC address exists, which could also produce the same basic error message. A final root cause is where users might pre-create a port in Neutron with a Bare Metal ports MAC address, which is an incorrect practice as Ironic attempts to assert the MAC address based upon the Node.

Reattaching networking on active instances

In advanced networking scenarios, operators may need to reconfigure the network binding for an active bare metal instance without rebooting it or interrupting the running workload. This is particularly useful for:

- **VXLAN network topology changes**: When the underlying VXLAN network configuration needs to be updated while instances remain online
- **Network infrastructure changes**: When physical network segments are reconfigured or migrated
- **Network binding recovery**: When Neutron port bindings become stale or inconsistent with the actual network state
- **Switch reconfiguration**: When network switches need to update their port configurations without instance downtime

The `reattach_networking` service step

The Neutron network interface provides a `reattach_networking` service step that can be invoked on nodes in the `active` state. This step:

1. Unbinds the tenant network ports in Neutron (clearing binding:host_id and binding:profile)
2. Rebinds the tenant network ports with updated binding information
3. Does **not** require booting the ironic-python-agent (IPA) ramdisk
4. Does **not** remove VIF attachments from the nodes ports
5. Does **not** require rebooting the instance

This allows network switches to reconfigure their ports based on the updated Neutron port binding information without any instance downtime.

How to use it

To reattach networking on an active node, use the following command:

```
$ openstack baremetal node service <node-uuid> \
    --service-steps '[{"interface": "network", "step": "reattach_networking"}]
↪'
```

The node will transition to `servicing` state, execute the reattach operation, and return to `active` state. The entire operation typically completes in seconds.

Example:

```
$ openstack baremetal node service 5e9258c4-cfda-40b6-86e2-e192f523d668 \
    --service-steps '[{"interface": "network", "step": "reattach_networking"}]
↪'
```

You can monitor the progress with:

```
$ openstack baremetal node show <node-uuid> -f value -c provision_state
```

When it returns to `active`, the network reattaching is complete.

Note

An operator may wish to utilize this feature via a runbook definition.

Important considerations

- **Only works with the Neutron network interface**: This service step is only available when using `network_interface=neutron`. It will not appear for `flat` or `noop` network interfaces.
- **Requires existing VIF attachments**: The nodes ports must already have tenant VIF attachments (`tenant_vif_port_id` in port.internal_info). This service step reattaches existing attachments; it does not create new ones.
- **Smart NIC support**: The step properly handles Smart NIC ports, including waiting for the Smart NIC host agent to be available before reattaching.
- **No network downtime expected**: In most cases, the network reattaching happens quickly enough that active network connections may continue working. However, brief disruption is possible depending on the network switch implementation and configuration.
- **Instance remains running**: The instance operating system and all running processes continue without interruption. No reboot occurs.

When NOT to use this

Do not use `reattach_networking` for:

- **Initial VIF attachment**: Use `openstack baremetal node vif attach` to attach VIFs to a node for the first time.
- **Removing VIFs**: Use `openstack baremetal node vif detach` to remove VIF attachments.
- **Nodes not in ACTIVE state**: This service step is designed for active instances. For nodes in other states, use the normal provisioning workflow.
- **Changing VIF assignments**: This step reattaches existing VIFs; it does not change which Neutron ports are attached to which node ports.

My test VM image does not deploy mount point does not exist

What is likely occurring

The image attempting to be deployed likely is a partition image where the file system that the user wishes to boot from lacks the required folders, such as `/dev` and `/proc`, which are required to install a bootloader for a Linux OS image

It should be noted that similar errors can also occur with whole disk images where we are attempting to setup the UEFI bootloader configuration. That being said, in this case, the image is likely invalid or contains an unexpected internal structure.

Users performing testing may choose something that they believe will work based on it working for virtual machines. These images are often attractive for testing as they are generic and include basic support for establishing networking and possibly installing user keys. Unfortunately, these images often lack drivers and firmware required for many different types of physical hardware which makes using them very problematic. Additionally, images such as Cirros do not have any contents in the root filesystem (i.e. an empty filesystem), as they are designed for the `ramdisk` to write the contents to disk upon boot.

How do I not encounter this issue?

We generally recommend using diskimage-builder or vendor supplied images. Centos, Ubuntu, Fedora, and Debian all publish operating system images which do generally include drivers and firmware for physical hardware. Many of these published cloud images, also support auto-configuration of networking AND population of user keys.

Issues with autoconfigured TLS

These issues will manifest as an error in `ironic-conductor` logs looking similar to (lines are wrapped for readability):

```
ERROR ironic.drivers.modules.agent_client [-]
Failed to connect to the agent running on node d7c322f0-0354-4008-92b4-
↪f49fb2201001
for invoking command clean.get_clean_steps. Error:
HTTPSConnectionPool(host='192.168.123.126', port=9999): Max retries exceeded␣
↪with url:
/v1/commands/?wait=true&agent_token=<token> (Caused by
SSLError(SSLError(1, '[SSL: CERTIFICATE_VERIFY_FAILED] certificate verify␣
↪failed (_ssl.c:897)'),)):
requests.exceptions.SSLError: HTTPSConnectionPool(host='192.168.123.126',␣
↪port=9999):
Max retries exceeded with url: /v1/commands/?wait=true&agent_token=<token>
(Caused by SSLError(SSLError(1, '[SSL: CERTIFICATE_VERIFY_FAILED] certificate␣
↪verify failed (_ssl.c:897)'),))
```

The cause of the issue is that the Bare Metal service cannot access the ramdisk with the TLS certificate provided by the ramdisk on first heartbeat. You can inspect the stored certificate in `/var/lib/ironic/certificates/<node>.crt`.

You can try connecting to the ramdisk using the IP address in the log message:

```
curl -vL https://<IP address>:9999/v1/commands \
    --cacert /var/lib/ironic/certificates/<node UUID>.crt
```

You can get the detailed information about the certificate using openSSL:

```
openssl x509 -text -noout -in /var/lib/ironic/certificates/<node UUID>.crt
```

Clock skew

One possible source of the problem is a discrepancy between the hardware clock on the node and the time on the machine with the Bare Metal service. It can be detected by comparing the `Not Before` field in the `openssl` output with the timestamp of a log message.

The recommended solution is to enable the NTP support in ironic-python-agent by passing the `ipa-ntp-server` argument with an address of an NTP server reachable by the node.

If it is not possible, you need to ensure the correct hardware time on the machine. Keep in mind a potential issue with timezones: an ability to store timezone in hardware is pretty recent and may not be available. Since ironic-python-agent is likely operating in UTC, the hardware clock should also be set in UTC.

Note

Microsoft Windows uses local time by default, so a machine that has previously run Windows will likely have wrong time.

Ive checked the clock skew, tried syncing with NTP, and certificate reports not valid yet

If you have encountered a case where you have tried to sync the clock of a system and deployment/cleaning operations are failing stating the certificate is not valid yet or has expired, there could be a few different things going on.

But first, we need to delineate an aspect. There is more than one clock in most systems.

Specifically, a Baseboard Management Controller has its own clock, which may or may not be syncing with the Host clock of the system. It is important to ensure that the Host clock in bios firmware settings is what is checked. The best way to do this is to, on the console, boot into firmware settings, and check the Date / Time Settings.

In most cases, simple clock skew can be addressed through use of the `ipa-ntp-server` setting, however the Ironic community has become aware of reports where even that does not remedy this issue. Investigation has found some hardware also allowing a timezone setting to be introduced with a UTC offset or localized time zone. It appears that when the ramdisk boots, the ramdisk clock does not properly calculate UTC as a result, and believes the time to be skewed based upon the timezone setting applied in firmware settings. The Ironic community recommends that the only timezone used set in a system clock is UTC, as Linux generally expects the system clock to be in UTC.

Another option is the `auto_tls_allowed_clock_skew` setting in the ironic-python-agent configuration file. It defaults to 1 hour. If you find you need to modify this setting in your deployment, please notify Ironic community.

I changed ironic.conf, and now I cant edit my nodes.

Whenever a node is created in ironic, default interfaces are identified as part of driver composition. This maybe sourced from explicit default values which have been set in `ironic.conf` or by the interface order for the enabled interfaces list. The result of this is that the `ironic-conductor` cannot spawn a `task` using the composed driver, as a portion of the driver is no longer enabled. This makes it difficult to edit or update the node if the settings have been changed.

For example, with networking interfaces, if you have `default_network_interface=neutron` or `enabled_network_interfaces=neutron,flat` in your `ironic.conf`, nodes would have been created with the `neutron` network interface.

This is because `default_network_interface` overrides the setting for new nodes, and that setting is **saved** to the database nodes table.

Similarly, the order of `enabled_network_interfaces` takes priority, and the first entry in the list is generally set to the default for the node upon creation, and that record is **saved** to the database nodes table.

The only case where driver composition does *not* calculate a default is if an explicit value is provided upon the creation of the node.

Example failure

A node in this state, when the `network_interface` was saved as `neutron`, yet the `neutron` interface is no longer enabled will fail basic state transition requests:

```
$ baremetal node manage 7164efca-37ab-1213-1112-b731cf795a5a
Could not find the following interface in the 'ironic.hardware.interfaces.
↪network' entrypoint: neutron. Valid interfaces are ['flat']. (HTTP 400)
```

How to fix this?

Revert the changes you made to `ironic.conf`.

This applies to any changes to any `default_*_interface` options or the order of interfaces in the for the `enabled_*_interfaces` options.

Once the conductor has been restarted with the updated configuration, you should now be able to update the interface using the `baremetal node set` command. In this example we use the `network_interface` as this is most commonly where it is encountered:

```
$ baremetal node set $NAME_OR_UUID --network-interface flat
```

Note

There are additional paths one can take to remedy this sort of issue, however we encourage operators to be mindful of operational consistency when making major configuration changes.

Once you have updated the saved interfaces, you should be able to safely return the `ironic.conf` configuration change in changing what interfaces are enabled by the conductor.

Im getting Out of Memory errors

This issue, also known as the the OOMKiller got my conductor case, is where your OS system memory reaches a point where the operating system engages measures to shed active memory consumption in order to prevent a complete failure of the machine. Unfortunately this can cause unpredictable behavior.

How did I get here?

One of the major consumers of memory in a host running an ironic-conductor is transformation of disk images using the `qemu-img` tool. This tool, because the disk images it works with are both compressed and out of linear block order, requires a considerable amount of memory to efficiently re-assemble and write-out a disk to a device, or to simply convert the format such as to a `raw` image.

By default, ironics configuration limits this conversion to 1 GB of RAM for the process, but each conversion does cause additional buffer memory to be used, which increases overall system memory pressure. Generally memory pressure alone from buffers will not cause an out of memory condition, but the multiple conversions or deployments running at the same time CAN cause extreme memory pressure and risk the system running out of memory.

How do I resolve this?

This can be addressed a few different ways:

- Use raw images, however these images can be substantially larger and require more data to be transmitted over the wire.
- Add more physical memory.
- Add swap space.
- Reduce concurrency, possibly via another conductor or changing the nova-compute.conf `max_concurrent_builds` parameter.
- Or finally, adjust the *DEFAULT.minimum_required_memory* parameter in your ironic.conf file. The default should be considered a default of last resort and you may need to reserve additional memory. You may also wish to adjust the *DEFAULT.minimum_memory_wait_retries* and *DEFAULT.minimum_memory_wait_time* parameters.

Why does API return Node is locked by host?

This error usually manifests as HTTP error 409 on the client side:

> Node d7e2aed8-50a9-4427-baaa-f8f595e2ceb3 is locked by host 192.168.122.1, please retry after the current operation is completed.

It happens, because an operation that modifies a node is requested, while another such operation is running. The conflicting operation may be user requested (e.g. a provisioning action) or related to the internal processes (e.g. changing power state during *Power Synchronization*). The reported host name corresponds to the conductor instance that holds the lock.

Normally, these errors are transient and safe to retry after a few seconds. If the lock is held for significant time, these are the steps you can take.

First of all, check the current `provision_state` of the node:

`verifying`
: means that the conductor is trying to access the nodes BMC. If it happens for minutes, it means that the BMC is either unreachable or misbehaving. Double-check the information in `driver_info`, especially the BMC address and credentials.

  If the access details seem correct, try resetting the BMC using, for example, its web UI.

`deploying/inspecting/cleaning`
: means that the conductor is doing some active work. It may include downloading or converting images, executing synchronous out-of-band deploy or clean steps, etc. A node can stay in this state for minutes, depending on various factors. Consult the conductor logs.

`available/manageable/wait call-back/clean wait`
: means that some background process is holding the lock. Most commonly its the power synchronization loop. Similarly to the `verifying` state, it may mean that the BMC access is broken or too slow. The conductor logs will provide you insights on what is happening.

To trace the process using conductor logs:

1. Isolate the relevant log parts. Lock messages come from the `ironic.conductor.task_manager` module. You can also check the `ironic.common.states` module for any state transitions:

```
$ grep -E '(ironic.conductor.task_manager|ironic.common.states|NodeLocked)
↪' \
    conductor.log > state.log
```

2. Find the first instance of `NodeLocked`. It may look like this (stripping timestamps and request IDs here and below for readability):

```
DEBUG ironic.conductor.task_manager [-] Attempting to get exclusive lock␣
↪on node d7e2aed8-50a9-4427-baaa-f8f595e2ceb3 (for node update) __init__␣
↪/usr/lib/python3.6/site-packages/ironic/conductor/task_manager.py:233
DEBUG ironic_lib.json_rpc.server [-] RPC error NodeLocked: Node d7e2aed8-
↪50a9-4427-baaa-f8f595e2ceb3 is locked by host 192.168.57.53, please␣
↪retry after the current operation is completed. _handle_error /usr/lib/
↪python3.6/site-packages/ironic_lib/json_rpc/server.py:179
```

   The events right before this failure will provide you a clue on why the lock is held.

3. Find the last successful **exclusive** locking event before the failure, for example:

```
DEBUG ironic.conductor.task_manager [-] Attempting to get exclusive lock␣
↪on node d7e2aed8-50a9-4427-baaa-f8f595e2ceb3 (for provision action␣
↪manage) __init__ /usr/lib/python3.6/site-packages/ironic/conductor/task_
↪manager.py:233
DEBUG ironic.conductor.task_manager [-] Node d7e2aed8-50a9-4427-baaa-
↪f8f595e2ceb3 successfully reserved for provision action manage (took 0.
↪01 seconds) reserve_node /usr/lib/python3.6/site-packages/ironic/
↪conductor/task_manager.py:350
DEBUG ironic.common.states [-] Exiting old state 'enroll' in response to␣
↪event 'manage' on_exit /usr/lib/python3.6/site-packages/ironic/common/
↪states.py:307
DEBUG ironic.common.states [-] Entering new state 'verifying' in response␣
↪to event 'manage' on_enter /usr/lib/python3.6/site-packages/ironic/
↪common/states.py:313
```

   This is your root cause, the lock is held because of the BMC credentials verification.

4. Find when the lock is released (if at all). The messages look like this:

```
DEBUG ironic.conductor.task_manager [-] Successfully released exclusive␣
↪lock for provision action manage on node d7e2aed8-50a9-4427-baaa-
↪f8f595e2ceb3 (lock was held 60.02 sec) release_resources /usr/lib/
↪python3.6/site-packages/ironic/conductor/task_manager.py:447
```

   The message tells you the reason the lock was held (`for provision action manage`) and the amount of time it was held (60.02 seconds, which is way too much for accessing a BMC).

Unfortunately, due to the way the conductor is designed, it is not possible to gracefully break a stuck lock held in `*-ing` states. As the last resort, you may need to restart the affected conductor. See *Why are my nodes stuck in a -ing state?*.

What is ConcurrentActionLimit?

ConcurrentActionLimit is an exception which is raised to clients when an operation is requested, but cannot be serviced at that moment because the overall threshold of nodes in concurrent Deployment or Cleaning operations has been reached.

These limits exist for two distinct reasons.

The first is they allow an operator to tune a deployment such that too many concurrent deployments cannot be triggered at any given time, as a single conductor has an internal limit to the number of overall concurrent tasks, this restricts only the number of running concurrent actions. As such, this accounts for the number of nodes in `deploy` and `deploy wait` states. In the case of deployments, the default value is relatively high and should be suitable for *most* larger operators.

The second is to help slow down the ability in which an entire population of baremetal nodes can be moved into and through cleaning, in order to help guard against authenticated malicious users, or accidental script driven operations. In this case, the total number of nodes in `deleting`, `cleaning`, and `clean wait` are evaluated. The default maximum limit for cleaning operations is *50* and should be suitable for the majority of baremetal operators.

These settings can be modified by using the *conductor.max_concurrent_deploy* and *conductor.max_concurrent_clean* settings from the ironic.conf file supporting the `ironic-conductor` service. Neither setting can be explicitly disabled, however there is also no upper limit to the setting.

Note

This was an infrastructure operator requested feature from actual lessons learned in the operation of Ironic in large scale production. The defaults may not be suitable for the largest scale operators.

Why do I have an error that an NVMe Partition is not a block device?

In some cases, you can encounter an error that suggests a partition that has been created on an NVMe block device, is not a block device.

Example:

> lsblk: /dev/nvme0n1p2: not a block device

What has happened is the partition contains a partition table inside of it which is confusing the NVMe device interaction. While basically valid in some cases to have nested partition tables, for example, with software raid, in the NVMe case the driver and possibly the underlying device gets quite confused. This is in part because partitions in NVMe devices are higher level abstracts.

The way this occurs is you likely had a `whole-disk` image, and it was configured as a partition image. If using glance, your image properties may have a `img_type` field, which should be `whole-disk`, or you have a `kernel_id` and `ramdisk_id` value in the glance image `properties` field. Definition of a kernel and ramdisk value also indicates that the image is of a `partition` image type. This is because a `whole-disk` image is bootable from the contents within the image, and partition images are unable to be booted without a kernel, and ramdisk.

If you are using Ironic in standalone mode, the optional `instance_info/image_type` setting may be advisable to be checked. Very similar to Glance usage above, if you have set Ironics node level `instance_info/kernel` and `instance_info/ramdisk` parameters, Ironic will proceed with deploying an image as if it is a partition image, and create a partition table on the new block device, and then write the contents of the image into the newly created partition.

Note

As a general reminder, the Ironic community recommends the use of whole disk images over the use of partition images.

Why cant I use Secure Erase/Wipe with RAID controllers?

Situations have been reported where an infrastructure operator is expecting particular device types to be Secure Erased or Wiped when they are behind a RAID controller.

For example, the server may have NVMe devices attached to a RAID controller which could be in pass-through or single disk volume mode. The same scenario exists basically regardless of the disk/storage medium/type.

The basic reason why is that RAID controllers essentially act as command translators with a buffer cache. They tend to offer a simplified protocol to the Operating System, and interact with the storage device in whatever protocol is native to the device. This is the root of the underlying problem.

Protocols such as SCSI are rooted in quite a bit of computing history, but never evolved to include primitives like Secure Erase which evolved in the ATA protocol.

The closest primitives in SCSI to ATA Secure Erase is the `FORMAT UNIT` and `UNMAP` commands.

`FORMAT UNIT` might be a viable solution, and a tool named sg_format exists, but there has not been a sufficient call upstream to implement this and test it sufficiently that the Ironic community would be comfortable shipping such a capability. The possibility also exists that a RAID controller might not translate this command through to an end device, just as some RAID controllers know how to handle and pass through ATA commands to disk devices which support them. It is entirely dependent upon the hardware configuration scenario.

The `UNMAP` command is similar to the ATA `TRIM` command. Unfortunately the SCSI protocol requires this be performed at block level, and similar to `FORMAT UNIT`, it may not be supported or just passed through.

If your interested in working on this area, or are willing to help test, please feel free to contact the *Ironic development community*. An additional option is the creation of your own custom Hardware Manager which can contain your preferred logic, however this does require some Python development experience.

One last item of note, depending on the RAID controller, the BMC, and a number of other variables, you may be able to leverage the RAID configuration interface to delete volumes/disks, and recreate them. This may have the same effect as a clean disk, however that too is RAID controller dependent behavior.

Im in clean failed state, what do I do?

There is only one way to exit the `clean failed` state. But before we visit the answer as to **how**, we need to stress the importance of attempting to understand **why** cleaning failed. On the simple side of things, this may be as simple as a DHCP failure, but on a complex side of things, it could be that a cleaning action failed against the underlying hardware, possibly due to a hardware failure.

As such, we encourage everyone to attempt to understand **why** before exiting the `clean failed` state, because you could potentially make things worse for yourself. For example if firmware updates were being performed, you may need to perform a rollback operation against the physical server, depending on what, and how the firmware was being updated. Unfortunately this also borders the territory of no simple answer.

This can be counter balanced with sometimes there is a transient networking failure and a DHCP address was not obtained. An example of this would be suggested by the `last_error` field indicating something about Timeout reached while cleaning the node, however we recommend following several basic troubleshooting steps:

- Consult the `last_error` field on the node, utilizing the `baremetal node show <uuid>` command.
- If the version of ironic supports the feature, consult the node history log, `baremetal node history list` and `baremetal node history get <uuid>`.
- Consult the actual console screen of the physical machine. *If* the ramdisk booted, you will generally want to investigate the controller logs and see if an uploaded agent log is being stored on the conductor responsible for the baremetal node. Consult *Retrieving logs from the deploy ramdisk*. If the node did not boot for some reason, you can typically just retry at this point and move on.

How to get out of the state, once youve understood **why** you reached it in the first place, is to utilize the `baremetal node manage <node_id>` command. This returns the node to `manageable` state, from where you can retry cleaning through automated cleaning with the `provide` command, or manual cleaning with `clean` command. or the next appropriate action in the workflow process you are attempting to follow, which may be ultimately be decommissioning the node because it could have failed and is being removed or replaced.

I cant seem to introspect newly added nodes in a large cluster

With larger clusters, the act of synchronizing DHCP for introspection and hardware discovery can take quite a bit of time because of the operational overhead. What happens is we spend so much time trying to perform the update that the processes stay continuously busy, which can have a side effect such as impacting the ability to successfully introspect nodes which were very recently added to the cluster.

To remedy this, try setting `[pxe_filter]sync_period` to be less frequent, i.e. a larger value to enable conductors to have time between running syncs.

Note

Ironic now has this functionality also merged into Ironic directly as part of the merge of the `ironic-inspector` service into `ironic` itself. This merger will result in a slightly more performant implementation, which may necessitate re-evaluation and tuning of the `[pxe_filter]sync_period` parameter.

Some or all of my baremetal nodes disappeared! Help?!

If you just upgraded, and this has occurred:

1) Dont Panic

2) Dont try to re-enroll the nodes. They should still be there, you just cant see them at the moment.

Over the past few years, Ironic and OpenStack project as a whole has been working to improve the model of Role Based Access Control. For users of Ironic, this means an extended role based access control model allowing delineation of nodes and the ability for projects to both self-manage.

The result is that users inside of a project are only permitted to see baremetal nodes, through the `owner` and `lessee` field, which has been granted access to the project.

However, as with any complex effort, there can be hiccups, and you have encountered one. Specifically that based upon large scale operator feedback, Ironic kept logic behind System scoped user usage, which OpenStack largely avoided due to concerns over effort.

As such, you have a couple different paths you can take, and your ideal path is also going to vary upon your model of usage and comfort level. We recommend reading the rest of this answer section before taking any further action.

A good starting point is obtaining a `system` scoped account with an `admin` or `member` role. Either of those roles will permit a nodes `owner` or `lessee` fields to be changed. Executing `baremetal node list` commands with this account should show you all baremetal nodes across all projects. Alternatively, if you just want to enable the legacy RBAC policies temporarily to change the fields, that is also an option, although not encouraged, and can be done utilizing the `[oslo_policy] enforce_scope` and `[oslo_policy] enforce_new_defaults` settings.

System Scoped Accounts

A `system` scoped account is one which has access and authority over the whole of the of an OpenStack deployment. A simplified way to think of this is when deployed, a username and password is utilized to bootstrap keystone. The rights granted to that user are inherently a system scoped `admin` role level of access. You can use this level of access to check the status, or run additional commands.

In this example below, which if successful, should return a list of all baremetal nodes known to Ironic, once the executing user supplies the valid password. In this case the admin account keystone was bootstrapped with. As a minor note, you will not be able to have any OS_* environment variables loaded into your current command shell, including OS_CLOUD for this command to be successful.

```
$ openstack --os-username admin --os-user-domain-name default --os-system-
↪scope all baremetal node list
```

You can alternatively issue a system-scoped token and reuse further commands with that token, or even generate a new system scoped account with a role of `member`.

Changing/Assigning an Owner

Ironic performs matching based upon Project ID. The owner field can be set to a projects ID value, which allows baremetal nodes to be visible.

```
$ PROJECT_ID=$(openstack project show -c id -f value $PROJECT_NAME)
$ baremetal node set --owner $PROJECT_ID $NODE_UUID_OR_NAME
```

Why am I only seeing *some* of the nodes?

During the Zed development cycle of Ironic, Ironic added an option which defaulted to True, which enabled project scoped `admin` users to be able to create their own baremetal nodes without needing higher level access. This default enabled option, `[api] project_admin_can_manage_own_nodes`, automatically stamps the requestors project ID on to a baremetal node if an `owner` is not otherwise specified upon creation. Obviously, this can create a mixed perception if an operator never paid attention to the `owner` field before now.

If your bare metal management processes require that full machine management is made using a project scoped account, please configure an appropriate node `owner` for the nodes which need to be managed. Ironic recognizes this is going to vary based upon processes and preferences.

Config Drives in Swift, but rebuilds fails?

When deploying instances, Ironic can be configured such that configuration drives are stored in Swift. The pointer to the configuration drive is saved in Ironic as a Temporary URL which has a time expiration.

When you issue the rebuild request for a node, Ironic expects that you will supply new configuration drive contents with your request, however this is also optional.

Because Swift has been set as the optional configuration drive storage location, a rebuild can fail if the prior configuration drive file is no longer accessible and no new configuration drive has been supplied to Ironic.

To resolve this case, you can either supply new configuration drive contents with your request, or disable configuration from being stored in Swift for new baremetal node deployments by changing setting *deploy.configdrive_use_object_store* to `false`.

Ironic says my Image is Invalid

As a result of security fixes which were added to Ironic, resulting from the security posture of the `qemu-img` utility, Ironic enforces certain aspects related to image files.

- Enforces that the file format of a disk image matches what Ironic is told by an API user. Any mismatch will result in the image being declared as invalid. A mismatch with the file contents and what is stored in the Image service will necessitate uploading a new image as that property cannot be changed in the image service *after* creation of an image.
- Enforces that the input file format to be written is `qcow2` or `raw`. This can be extended by modifying `[conductor]permitted_image_formats` in `ironic.conf`.
- Performs safety and sanity check assessment against the file, which can be disabled by modifying `[conductor]disable_deep_image_inspection` and setting it to `True`. Doing so is not considered safe and should only be done by operators accepting the inherent risk that the image they are attempting to use may have a bad or malicious structure. Image safety checks are generally performed as the deployment process begins and stages artifacts, however a late stage check is performed when needed by the ironic-python-agent.

Using /dev/sda does not write to the first disk

Alternative name: I chose /dev/sda but I found it as /dev/sdb after rebooting.

Historically, Linux users have grown accustom to a context where /dev/sda is the first device in a physical machine. Meaning, if you look at the device by_path information or the HCTL, or device LUN, the device ends with a zero.

For example, assuming 3 disks, two controllers, with a single disk on the second controller would look something like this:

- /dev/sda maps to a device with lun 0, HCTL 0:0:0:0
- /dev/sdb maps to a device with lun 1, HCTL 0:0:1:0
- /dev/sdc maps to a device with lun 2, HCTL 0:1:0:0

However, this was a pattern we grew accustom to because the order of device discovery was sequential *and* synchronous. In other words the kernel stepped through all possible devices one at a time. Where this breaks is when the kernel is operating in a mode where device initialization is asynchronous as some distributions have decided to adopt.

The result of a move to an asynchronous initialization is /dev/sda has always been the *first* device to initialize, *not* the first device in the system. As a result, we can end up with something looking like:

- /dev/sda maps to a device with lun 1, HCTL 0:0:1:0
- /dev/sdb maps to a device with lun 2, HCTL 0:1:0:0
- /dev/sdc maps to a device with lun 0, HCTL 0:0:0:0

Generally, most operators might then consider referencing the /dev/disk/by-path structure to match disk devices because that seems to imply a static order, *however* a kernel operating with asynchronous device initialization will order *everything*, including PCI devices the same way, meaning by-path can also be unreliable. Furthermore, if your server hardware is using multipath IO, you should be operating with multipath enabled such that the device is used.

The net result is the best criteria to match on is:

- Serial Number
- World Wide Name
- Device HCTL, which *does* appear to be static in these cases, but is not applicable for hosts using multipathing. It may, ultimately, not be static enough, just depending on the hardware in use.

I changed the BMC password, and now a node is in maintenance state.

Ironic automatically checks access to a remote BMC utilizing the configured credentials for the driver. Depending on the driver in use, this can vary in field name, but these fields are set in each nodes `driver_info` parameter and must be reset.

Once the credential issue has been resolved, Ironic automatically moves the node out of maintenance state upon the next attempt to check the power state. Please review *Power Synchronization* for more information, however an operator can also explicitly unset the maintenance state as well, and the examples below are detailed as such.

For example, with Redfish

```
$ openstack baremetal node set --driver-info redfish_username=<new_user> --
↪driver-info redfish_password=<new_pass> <node_uuid>
$ openstack baremetal node maintenance unset <node_uuid>
```

For example, with IPMI

```
$ openstack baremetal node set --driver-info ipmi_username=<new_user> --
↪driver-info ipmi_password=<new_pass> <node_uuid>
$ openstack baremetal node maintenance unset <node_uuid>
```

For example, with iDRAC drivers

```
$ openstack baremetal node set --driver-info drac_username=<new_user> --
↪driver-info drac_password=<new_pass> <node_uuid>
$ openstack baremetal node set --driver-info redfish_username=<new_user> --
↪driver-info redfish_password=<new_pass> <node_uuid>
$ openstack baremetal node maintenance unset <node_uuid>
```

Note

The `drac_username` and `drac_password` parameters were used by WSMAN Dell iDRAC drivers. Newer versions of the drivers utilize Redfish natively.

For example, with iLO drivers

```
$ openstack baremetal node set --driver-info ilo_username=<new_user> --driver-
↪info ilo_password=<new_pass> <node_uuid>
$ openstack baremetal node maintenance unset <node_uuid>
```

Note

Ironic generally does not manage or rotate the remote BMC credentials due to the risk of user lockout if the account is not for the specific use of Ironic. Some vendor drivers have added this functionality, however such actions should be made with caution.

Im attempting to use Redfish, and something weird is happening!

Redfish is a standard which is a union of an HTTPS based RESTful API interaction, coupled with individual resource data models. Occasionally vendors take a very different reading of the standard, which is to be expected, but at the same time the vendors sometimes also go directly against the standard, such as moving resources.

Ultimately Ironic tries to address as many of these issues that we are made aware of; however often we will need additional structural information to remedy these sorts of issues.

Often these sorts of issues might manifest as Virtual Media not working, or boot mode settings behaving oddly, or Ironic believe the UEFI state is None. While these are all past examples the Ironic has since addressed, most of them appeared on major vendor BMC firmware updates or major feature level updates in the BMC firmware.

The best way to address this is to open a bug in Launchpad and supply as much information for the Ironic community to be able to work this issue.

Commands which may provide helpful information for the Ironic community to navigate the issue you have encountered.

```
$ curl -u <bmc_user>:<bmc_password> -k https://<bmc_address>/redfish/v1/
↪Systems/
$ curl -u <bmc_user>:<bmc_password> -k https://<bmc_address>/redfish/v1/
↪Systems/<system>/
$ curl -u <bmc_user>:<bmc_password> -k https://<bmc_address>/redfish/v1/
↪Managers/
$ curl -u <bmc_user>:<bmc_password> -k https://<bmc_address>/redfish/v1/
↪Managers/<bmc>
```

Note

Because of the structurally nested model, the Ironic project may request additional commands to be executed against the BMC, but this will be on a case-by-case basis as we understand the exact issue

and what changes may be required in order to navigate this issue.

Warning

Please be careful to redact any usernames, passwords, and addresses which may be sensitive in any data provided to the Ironic community.

Unfortunately, there is no magical quick fix for these sorts of issues, but the Ironic community has a track record of attempting to navigate and address these issues as they arise.

Im using Boot-From-Volume, and the SAN passwords were changed.

In this event, the credential uses for an iSCSI attachment have been changed on a storage area network (SAN), and Ironic nodes have lost connectivity with iSCSI boot volumes.

The proper procedure here varies based upon if youre using the `cinder storage_interface` option, or if youre using the `external` option.

In the case of the `cinder` driver being used, if you power off the baremetal node via Ironics API, and then power the node back on. This will trigger Ironic to attempt to re-attach the volume to the baremetal node via Cinders API which will cause the most up-to-date configuration to be presented to the bare metal node being booted.

If youre using the `external` interface option, you will need to manually reset youre volume connector information values. Please refer to *Boot From Volume* for more information.

My configdrive download from tempurl fails with a 401 error!

This can happen because initial first sequence of deployment is close to the *conductor.deploy_callback_timeout* is set to a value, and *conductor.configdrive_swift_temp_url_duration* is not set to a value.

What is happening is the deployment process is running, but running to close to when the `deploy_callback_timeout` would have normally aborted the deployment, but it just got started deploying and thus would not be aborted.

Yet that timeout is the setting leverage when no other setting is set, and the result is that the configdrive temporary url download is rejected just after the temporary url timed out. Newer versions of ironic automatically add a slightly larger buffer to that window to try and prevent this issue from arising under normal conditions, however ideally you should set an explicit temporary url duration in your configuration.

4.3.7 Power Synchronization

Baremetal Power Sync

Each Baremetal conductor process runs a periodic task which synchronizes the power state of the nodes between its database and the actual hardware. If the value of the *conductor.force_power_state_during_sync* option is set to `true` the power state in the database will be forced on the hardware and if it is set to `false` the hardware state will be forced on the database. If this periodic task is enabled, it runs at an interval defined by the *conductor.sync_power_state_interval* config option for those nodes which are not in maintenance. The requests sent to Baseboard Management Controllers (BMCs) are done with a parallelism controlled by *conductor.sync_power_state_workers*.

The motivation to send out requests to BMCs in parallel is to handle misbehaving BMCs which may delay or even block the synchronization otherwise.

Note

In deployments with many nodes and IPMI as the configured BMC protocol, the default values of a 60 seconds power sync interval and 8 worker threads may lead to a high rate of required retries due to client-side UDP packet loss (visible via the corresponding warnings in the conductor logs). While Ironic automatically retries to get the power status for the affected nodes, the failure rate may be reduced by increasing the power sync cycle, e.g. to 300 seconds, and/or by reducing the number of power sync workers, e.g. to 2. Please keep in mind, however, that depending on the concrete setup increasing the power sync interval may have an impact on other components relying on up-to-date power states.

Compute-Baremetal Power Sync

Each `nova-compute` process in the Compute service runs a periodic task which synchronizes the power state of servers between its database and the compute driver. If enabled, it runs at an interval defined by the `sync_power_state_interval` config option on the `nova-compute` process. In case of the compute driver being baremetal driver, this sync will happen between the databases of the compute and baremetal services. Since the sync happens on the `nova-compute` process, the state in the compute database will be forced on the baremetal database in case of inconsistencies. Hence a node which was put down using the compute service API cannot be brought up through the baremetal service API since the power sync task will regard the compute services knowledge of the power state as the source of truth. In order to get around this disadvantage of the compute-baremetal power sync, baremetal service does power state change callbacks to the compute service using external events.

Power State Change Callbacks to the Compute Service

Whenever the Baremetal service changes the power state of a node, it can issue a notification to the Compute service. The Compute service will consume this notification and update the power state of the instance in its database. By conveying all the power state changes to the compute service, the baremetal service becomes the source of truth thus preventing the compute service from forcing wrong power states on the physical instance during the compute-baremetal power sync. It also adds the possibility of bringing up/down a physical instance through the baremetal service API even if it was put down/up through the compute service API.

This change requires the *nova* section and the necessary authentication options like the *nova.auth_url* to be defined in the configuration file of the baremetal service. If it is not configured the baremetal service will not be able to send notifications to the compute service and it will fall back to the behaviour of the compute service forcing power states on the baremetal service during the power sync. See *nova* group for more details on the available config options.

In case of baremetal stand alone deployments where there is no compute service running, the *nova.send_power_notifications* config option should be set to `False` to disable power state change callbacks to the compute service.

Note

The baremetal service sends notifications to the compute service only if the target power state is `power on` or `power off`. Other error and `None` states will be ignored. In situations where the power

state change is originally coming from the compute service, the notification will still be sent by the baremetal service and it will be a no-op on the compute service side with a debug log stating the node is already powering on/off.

Note

Although an exclusive lock is used when sending notifications to the compute service, there can still be a race condition if the compute-baremetal power sync happens to happen a nano-second before the power state change event is received from the baremetal service in which case the power state from compute services database will be forced on the node.

Power fault and recovery

When *Baremetal Power Sync* is enabled, and the Bare Metal service loses access to a node (usually because of invalid credentials due to rotation, BMC issues or networking interruptions), the node enters `maintenance` mode and its `fault` field is set to `power failure`. The exact reason is stored in the `maintenance_reason` field.

As always with maintenance mode, only a subset of operations will work on such nodes, and both the Compute service and the Ironics native allocation API will refuse to pick them. Any in-progress operations will either pause or fail.

The conductor responsible for the node will try to recover the connection periodically (with the interval configured by the *conductor.power_failure_recovery_interval* option). If the power sync is successful, the `fault` field is unset and the node leaves the maintenance mode.

Note

This only applies to automatic maintenance mode with the `fault` field set. Maintenance mode set manually is never left automatically.

Alternatively, you can disable maintenance mode yourself once the problem is resolved:

```
baremetal node maintenance unset <IRONIC NODE>
```

4.3.8 Fast-Track Deployment

Fast track is a mode of operation where the Bare Metal service keeps a machine powered on with the agent running between provisioning operations. It is first booted during in-band inspection or cleaning (whatever happens first) and is only shut down before rebooting into the final instance. Depending on the configuration, this mode can save several reboots and is particularly useful for scenarios where nodes are enrolled, prepared and provisioned within a short time.

Warning

Fast track deployment targets standalone use cases and is only tested with the `noop` networking. The case where inspection, cleaning, and provisioning networks are different is not supported.

Note

Fast track mode is very sensitive to long-running processes on the conductor side that may prevent agent heartbeats from being registered.

For example, converting a large image to the raw format may take long enough to reach the fast-track timeout. In this case, you can either *use raw images* or move the conversion to the agent side with:

```
[DEFAULT]
force_raw_images = False
```

Enabling

Fast track is off by default and should be enabled in the configuration:

```
[deploy]
fast_track = true
```

Starting with the Yoga release series, it can also be enabled or disabled per node:

```
baremetal node set <node> --driver-info fast_track=true
```

Inspection

If using *agent inspection*, aka in-band, you need to tell ironic not to power off nodes afterwards. Depending on the inspection mode (managed or unmanaged), you need to configure two places. In `ironic.conf`:

```
[inspector]
power_off = false
```

Finally, you need to update the *inspection PXE/iPXE configuration* to include the `ipa-api-url` kernel parameter, pointing at the **ironic** endpoint, in addition to the existing `ipa-inspection-callback-url`.

4.3.9 HTTP(s) Authentication strategy for user image servers

How to enable the feature via global configuration options

There are 3 variables that could be used to manage image server authentication strategy. The 3 variables are structured such a way that 1 of them `image_server_auth_strategy` (string) provides the option to specify the desired authentication strategy. Currently the only supported authentication strategy is `http_basic` that represents the HTTP(S) Basic Authentication also known as the `RFC 7616` internet standard.

The other two variables `image_server_password` and `image_server_user` provide username and password credentials for any authentication strategy that requires username and credentials to enable the authentication during image download processes. `image_server_auth_strategy` not just enables the feature but enforces checks on the values of the 2 related credentials. Currently only the `http_basic` strategy is utilizing the `image_server_password` and `image_server_user` variables.

When a authentication strategy is selected against the user image server an exception will be raised in case any of the credentials are None or an empty string. The variables belong to the `deploy` configuration group and could be configured via the global Ironic configuration file.

The authentication strategy configuration affects the download process for images downloaded by the conductor or the ironic-python-agent.

Example

Example of activating the `http-basic` strategy via `/etc/ironic/ironic.conf`:

```
[deploy]
...
image_server_auth_strategy = http_basic
image_server_user = username
image_server_password = password
...
```

Known limitations

This implementation of the authentication strategy for user image handling is implemented via the global Ironic configuration thus it doesnt provide node specific customization options.

When `image_server_auth_strategy` is set to any valid value all image sources will be treated with the same authentication strategy and Ironic will use the same credentials against all sources.

4.3.10 Use of OVN Networking

Overview

OVN is largely considered an evolution of OVS. While it is recommended that operators continue to utilize OVS with Ironic, OVN has an attractive a superset of capabilities and shifts some of the configuration of networking away from configuration files, towards a service modeled at serving a more scalable software defined networking experience. However, as with all newer technologies, there are caveats and issues. The purpose of this documentation is to help convey OVNs state, capabilities, and provide operators with the context required to navigate their path forward.

Warning

OVN is under quite a bit of active development, and this information may grow out of date quickly. Weve provided links to help spread the information and enable operators to learn the current status.

Challenges

DHCP

Historically, while OVN has included a DHCP server, this DHCP server has not had the capability to handle clients needing custom attributes such as those used by PXE and iPXE to enable network boot operations.

Typically, this has resulted in operators who use OVN with Bare Metal to continue to operate the `neutron-dhcp-agent` service, along with setting OVN configuration appropriate to disable OVN from responding to DHCP requests for baremetal ports. Please see routed networks for more information on this setting.

As of the 2023.2 Release of Ironic, The Ironic project *can* confirm that OVNs DHCP server does work for PXE and iPXE operations when using **IPv4**, OVS version **3.11**, and OVN version **23.06.0**.

Support for IPv6 is presently pending changes to Neutron, as IPv6 requires additional configuration options and a different pattern of behavior, and thus has not been tested. Your advised to continue to us the `neutron-dhcp-agent` if you need IPv6 at this time. Currently this support is being worked in Neutron change 890683 and bug 20305201.

Warning

Use of OVN with HTTPBoot interfaces has not been explicitly tested by the Ironic project, and is unlikely to take place until after integrated IPv6 support with Neutron is ready for use. The project does not expect any specific issues, but the OVN DHCP server is an entirely different server than the interfaces were tested upon.

Maximum Transmission Units

OVNs handling of MTUs has been identified by OVN as being incomplete. The reality is that it assumes the MTU is not further constrained beyond the gateway, which sort of works in some caess for virtual machines, but might not be applicable with baremetal because your traffic may pass through lower, or higher MTUs.

Ideally, your environment should have consistent MTUs. If you cannot have consistent MTUs, we recommend clamping the MTU and Maximum Segment Size (MSS) using your front end router to ensure igress traffic is sized and fragmented appropriately. Egress traffic should inherent its MTU size based upon the DHCP service configuration.

A items you can keep track of regarding MTU handling:

- Bug 2032817
- OVN TODO document

To clamp the MTU and MSS on a linux based router, you can utilize the following command:

```
ip route add $network via $OVN_ROUTER advmss $MAX_SEGMENT_SIZE mtu lock $MTU
```

Network Address Translation

Because the NAT and Connection Tracking layer gets applied differently with OVN, as the router doesnt appear as a namespace or to the local OS kernel, you will not be able to enable NAT translation for the Bare Metal Networks under the direct management of OVN which are used for provisioning, cleaning, servicing, rescuing, and any other Ironic driven activities.

In part, TFTP is also not advisable in these cases because the connection tracking layer which users of Linux networking may be familiar of, doesnt exist in the context of OVN. OVN is flows based which also prevents Ironic from being able to reach the private IP address of baremetal nodes in networks it utilizes when NAT is enabled.

Operator are advised to setup and leverage dedicated networks, and utilize firewall rules to restrict and limit communication flowing through OVN.

PXE boot of GRUB

Initial testing has revelaed that EFI booting Grub2 via OVN does not appear to work with OVN. For some reason, Grub2 believes the network mask is incorrect based upon the DHCP interaction, and results in a belief that the TFTP server is locally attached.

For example, if a client is assigned `10.1.0.13/28`, with a default gateway of `10.1.0.1`, and a tftp-sever of `10.203.101.230`, then grub2 believes its default route is 10.0.0.0/8.

This is being tracked as bug 2033430 until were better able to understand the root cause and file a bug with the appropriate project.

Required Configuration

OVN is designed to provide packet handling in a distributed fashion for a each compute hypervisor in a cloud of virtual machines. However with Bare Metal instances, you will likely need to have a pool of dedicated network nodes to handle OVN traffic.

Chassis as Gateway

The networking node chassis must be configured to operate as a gateway.

This can be configured manually, but *should* (as far as Ironic is aware) be configured by Neutron and set on interfaces matching the bridge mappings. At least, it works that way in Devstack.

Outbound SNAT

Outbound connectivity (that traverses the Neutron/OVN router) is known to be problematic due to a bug in the way External port priority is handled differently than port assignment than the Chassis Gateway. Ultimately this can create a mismatch between the ports and results in intermittent traffic loss. A known workaround is to manually update the HA Chassis Group and Gateway Chassis priorities so these are in sync for a given Neutron network and associated router.

This is being tracked as bug 1995078.

The Ironic community may integrate a fix into the `ironic-neutron-agent` at a future point in time if the bug in Neutron is not resolved.

ML2 Plugins

The `ovn-router` and `trunk` ml2 plugins as supplied with Neutron *must* be enabled.

If your additionally using the `baremetal-l2vni` mechanism driver, the `segments` plugin must also be enabled.

If you need to attach to the network

For example if you need to bind something into a network for baremetal, above and beyond a dedicated interface, you will need to make the attachment on the `br-ex` integration bridge, as opposed to `br-int` as one would have done with OVS.

Unknowns

It is presently unknown if it is possible for OVN to perform and enable VXLAN attachments to physical ports on integrated devices, thus operators are advised to continue to use `vlan` networking with their hosts with existing ML2 integrations.

4.3.11 Ceph Object Gateway support

Overview

Ceph project is a powerful distributed storage system. It contains object store and provides a RADOS Gateway Swift API which is compatible with OpenStack Swift API.

Ironic added support for RADOS Gateway temporary URL in the Mitaka release.

Configure Ironic and Glance with RADOS Gateway

1. Install Ceph storage with RADOS Gateway. See Ceph documentation.
2. Configure RADOS Gateway to use keystone for authentication. See Integrating with OpenStack Keystone
3. Register RADOS Gateway endpoint in the keystone catalog, with the same format swift uses, as the `object-store` service. URL example:

   `http://rados.example.com:8080/swift/v1/AUTH_$(project_id)s`.

   In the ceph configuration, make sure radosgw is configured with the following value:

   ```
   rgw swift account in url = True
   ```

4. Configure Glance API service for RADOS Swift API as backend. Edit the configuration file for the Glance API service (is typically located at `/etc/glance/glance-api.conf`):

   ```
   [glance_store]

   stores = file, http, swift
   default_store = swift
   default_swift_reference=ref1
   swift_store_config_file=/etc/glance/glance-swift-creds.conf
   swift_store_container = glance
   swift_store_create_container_on_put = True
   ```

   In the file referenced in `swift_store_config_file` option, add the following:

   ```
   [ref1]
   user = <service project>:<service user name>
   key = <service user password>
   user_domain_id = default
   project_domain_id = default
   auth_version = 3
   auth_address = http://keystone.example.com/identity
   ```

   Values for user and key options correspond to keystone credentials for RADOS Gateway service user.

Note: RADOS Gateway uses FastCGI protocol for interacting with HTTP server. Read your HTTP server documentation if you want to enable HTTPS support.

5. Restart Glance API service and upload all needed images.
6. If youre using custom container name in RADOS, change Ironic configuration file on the conductor host(s) as follows:

```
[glance]

swift_container = glance
```

7. Restart Ironic conductor service(s).

4.3.12 Emitting Software Metrics

Beginning with the Newton (6.1.0) release, the ironic services support emitting internal performance data to statsd. This allows operators to graph and understand performance bottlenecks in their system.

This guide assumes you have a statsd server setup. For information on using and configuring statsd, please see the statsd README and documentation.

These performance measurements, herein referred to as metrics, can be emitted from the Bare Metal service, including ironic-api, ironic-conductor, and ironic-python-agent. By default, none of the services will emit metrics.

It is important to stress that not only statsd is supported for metrics collection and transmission. This is covered later on in our documentation.

Configuring the Bare Metal Service to Enable Metrics with Statsd

Enabling metrics in ironic-api and ironic-conductor

The ironic-api and ironic-conductor services can be configured to emit metrics to statsd by adding the following to the ironic configuration file, usually located at `/etc/ironic/ironic.conf`:

```
[metrics]
backend = statsd
```

If a statsd daemon is installed and configured on every host running an ironic service, listening on the default UDP port (8125), no further configuration is needed. If you are using a remote statsd server, you must also supply connection information in the ironic configuration file:

```
[metrics_statsd]
# Point this at your environments' statsd host
statsd_host = 192.0.2.1
statsd_port = 8125
```

Enabling metrics in ironic-python-agent

The ironic-python-agent process receives its configuration in the response from the initial lookup request to the ironic-api service. This means to configure ironic-python-agent to emit metrics, you must enable the agent metrics backend in your ironic configuration file on all ironic-conductor hosts:

```
[metrics]
agent_backend = statsd
```

In order to reliably emit metrics from the ironic-python-agent, you must provide a statsd server that is reachable from both the configured provisioning and cleaning networks. The agent statsd connection information is configured in the ironic configuration file as well:

```
[metrics_statsd]
# Point this at a statsd host reachable from the provisioning and cleaning␣
↪nets
agent_statsd_host = 198.51.100.2
agent_statsd_port = 8125
```

Note

Use of a different metrics backend with the agent is not presently supported.

Transmission to the Message Bus Notifier

Regardless if youre using Ceilometer, ironic-prometheus-exporter, or some scripting you wrote to consume the message bus notifications, metrics data can be sent to the message bus notifier from the timer methods *and* additional gauge counters by utilizing the *`metrics.backend`* configuration option and setting it to `collector`. When this is the case, Information is cached locally and periodically sent along with the general sensor data update to the messaging notifier, which can consumed off of the message bus, or via notifier plugin (such as is done with ironic-prometheus-exporter).

Note

Transmission of timer data only works for the Conductor or `single-process` Ironic service model. A separate webserver process presently does not have the capability of triggering the call to retrieve and transmit the data.

Types of Metrics Emitted

The Bare Metal service emits timing metrics for every API method, as well as for most driver methods. These metrics measure how long a given method takes to execute.

A deployer with metrics enabled should expect between 100 and 500 distinctly named data points to be emitted from the Bare Metal service. This will increase if the metrics.preserve_host option is set to true or if multiple drivers are used in the Bare Metal deployment. This estimate may be used to determine if a deployer needs to scale their metrics backend to handle the additional load before enabling metrics. To see which metrics have changed names or have been removed between releases, refer to the ironic release notes.

Additional conductor metrics in the form of counts will also be generated in limited locations where petinant to the activity of the conductor.

Note

With the default statsd configuration, each timing metric may create additional metrics due to how statsd handles timing metrics. For more information, see statds documentation on metric types.

The ironic-python-agent ramdisk emits timing metrics for every API method.

Deployers who use custom HardwareManagers can emit custom metrics for their hardware. For more information on custom HardwareManagers, and emitting metrics from them, please see the ironic-python-agent documentation.

Adding New Metrics

If youre a developer, and would like to add additional metrics to ironic, please see the ironic-lib developer documentation for details on how to use the metrics library. A release note should also be created each time a metric is changed or removed to alert deployers of the change.

4.3.13 API Audit Logging

Audit middleware supports the delivery of CADF audit events via the Oslo messaging notifier capability. Based on the `notification_driver` configuration, audit event can be routed to messaging infrastructure (notification_driver = messagingv2) or can be routed to a log file (`[oslo_messaging_notifications]/driver = log`).

Audit middleware creates two events per REST API interaction. The first event has information extracted from request data and the second one has request outcome (response).

Enabling API Audit Logging

Audit middleware is available as part of `keystonemiddleware` (>= 1.6) library. For information regarding how audit middleware functions refer here.

Auditing can be enabled for the Bare Metal service by making the following changes to `/etc/ironic/ironic.conf`.

1. To enable audit logging of API requests:

```
[audit]
...
enabled=true
```

2. To customize auditing API requests, the audit middleware requires the audit_map_file setting to be defined. Update the value of the configuration setting audit_map_file to set its location. Audit map file configuration options for the Bare Metal service are included in the etc/ironic/ironic_api_audit_map.conf.sample file. To understand CADF format specified in ironic_api_audit_map.conf file, refer to CADF Format.:

```
[audit]
...
audit_map_file=/etc/ironic/api_audit_map.conf
```

3. Comma-separated list of Ironic REST API HTTP methods to be ignored during audit. It is used only when API audit is enabled. For example:

```
[audit]
...
ignore_req_list=GET,POST
```

Sample Audit Event

Following is the sample of the audit event for the ironic node list request.


```
{
   "event_type":"audit.http.request",
   "timestamp":"2016-06-15 06:04:30.904397",
   "payload":{
      "typeURI":"http://schemas.dmtf.org/cloud/audit/1.0/event",
      "eventTime":"2016-06-15T06:04:30.903071+0000",
      "target":{
         "id":"ironic",
         "typeURI":"unknown",
         "addresses":[
            {
               "url":"http://{ironic_admin_host}:6385",
               "name":"admin"
            },
            {
               "url":"http://{ironic_internal_host}:6385",
               "name":"private"
            },
            {
               "url":"http://{ironic_public_host}:6385",
               "name":"public"
            }
         ],
         "name":"ironic"
      },
      "observer":{
         "id":"target"
      },
      "tags":[
         "correlation_id?value=685f1abb-620e-5d5d-b74a-b4135fb32373"
      ],
      "eventType":"activity",
      "initiator":{
         "typeURI":"service/security/account/user",
         "name":"admin",
         "credential":{
            "token":"***",
            "identity_status":"Confirmed"
         },
         "host":{
            "agent":"python-ironicclient",
            "address":"10.1.200.129"
```


(continues on next page)

(continued from previous page)


```
        },
        "project_id":"d8f52dd7d9e1475dbbf3ba47a4a83313",
        "id":"8c1a948bad3948929aa5d5b50627a174"
      },
      "action":"read",
      "outcome":"pending",
      "id":"061b7aa7-5879-5225-a331-c002cf23cb6c",
      "requestPath":"/v1/nodes/?associated=True"
   },
   "priority":"INFO",
   "publisher_id":"ironic-api",
   "message_id":"2f61ebaa-2d3e-4023-afba-f9fca6f21fc2"
}
```


4.3.14 Custom API Middleware

Ironic supports loading custom WSGI middleware via stevedore entry points. This allows operators to add custom request processing such as validation, logging, rate limiting, or other functionality without modifying Ironic source code.

This follows the same plugin pattern used for inspection hooks and hardware interfaces.

Enabling Custom Middleware

To enable custom middleware, configure the `[api] middleware` option in `/etc/ironic/ironic.conf` with a comma-separated list of middleware names:

```
[api]
middleware = my-middleware,another-middleware
```

Middleware are applied in the order specified, wrapping the API application from inside out. The last middleware in the list will be the outermost wrapper and will process requests first.

Writing Custom Middleware

Custom middleware must be a callable that accepts a WSGI application and returns a wrapped WSGI application. The simplest form is a class with `__init__` and `__call__` methods:

```
class MyMiddleware:
    """Example WSGI middleware."""

    def __init__(self, application):
        self.application = application

    def __call__(self, environ, start_response):
        # Pre-process request here
        # ...

        # Call the wrapped application
        return self.application(environ, start_response)
```

For more complex middleware, you can use the `webob` library:

```
import webob
import webob.dec

class MyMiddleware:
    """Example middleware using webob."""

    def __init__(self, application):
        self.application = application

    @webob.dec.wsgify
    def __call__(self, req):
        # Validate request
        if not self._is_valid(req):
            return webob.Response(status=400, json={'error': 'Invalid'})

        # Pass through to the application
        return req.get_response(self.application)
```

Registering Middleware

Middleware are registered via the `ironic.api.middleware` entry point namespace. In your packages `pyproject.toml`:

```
[project.entry-points."ironic.api.middleware"]
my-middleware = "my_package.middleware:MyMiddleware"
```

Or in `setup.cfg`:

```
[entry_points]
ironic.api.middleware =
    my-middleware = my_package.middleware:MyMiddleware
```

After installing the package, the middleware can be enabled by adding `my-middleware` to the `[api] middleware` configuration option.

Example: Request Validation Middleware

Heres an example middleware that validates portgroup names:

```
import json
import re
import webob
import webob.dec

class PortgroupNameValidationMiddleware:
    """Validate portgroup names match a required format."""

    PATTERN = re.compile(r'^.+-port-channel(\d+)$')

    def __init__(self, application):
        self.application = application
```

(continues on next page)

(continued from previous page)

```
    @webob.dec.wsgify
    def __call__(self, req):
        if (req.method == 'POST'
                and req.path_info.rstrip('/') == '/v1/portgroups'):
            try:
                body = json.loads(req.body)
                name = body.get('name', '')
                if not self.PATTERN.match(name):
                    return webob.Response(
                        status=400,
                        json={'error': 'Invalid portgroup name format'}
                    )
            except json.JSONDecodeError:
                pass  # Let Ironic handle malformed JSON

        return req.get_response(self.application)
```

Register it in `pyproject.toml`:

```
[project.entry-points."ironic.api.middleware"]
portgroup-validation = "my_package:PortgroupNameValidationMiddleware"
```

Enable it in `ironic.conf`:

```
[api]
middleware = portgroup-validation
```

4.3.15 Tuning Ironic

Memory Utilization

Memory utilization is a difficult thing to tune in Ironic as largely we may be asked by API consumers to perform work for which the underlying tools require large amounts of memory.

The biggest example of this is image conversion. Images not in a raw format need to be written out to disk for conversion (when requested) which requires the conversion process to generate an in-memory map to re-assemble the image contents into a coherent stream of data. This entire process also stresses the kernel buffers and cache.

This ultimately comes down to a trade-off of Memory versus Performance, similar to the trade-off of Performance versus Cost.

On a plus side, an idle Ironic deployment does not need much in the way of memory. On the down side, a highly bursty environment where a large number of concurrent deployments may be requested should consider two aspects:

- How is the ironic-api service/process set up? Will more processes be launched automatically?
- Are images prioritized for storage size on disk? Or are they compressed and require format conversion?

API

Ironics API should have a fairly stable memory footprint with activity, however depending on how the webserver is running the API, additional processes can be launched.

Under normal conditions, as of Ironic 15.1, the `ironic-api` service/process consumes approximately 270MB of memory per worker. Depending on how the process is being launched, the number of workers and maximum request threads per worker may differ. Naturally there are configuration and performance trade-offs.

- Directly as a native python process, i.e. execute `ironic-api` processes. Each single worker allows for multiple requests to be handled and threaded at the same time which can allow high levels of request concurrency. As of the Victoria cycle, a direct invocation of the `ironic-api` program will only launch a maximum of four workers.
- Launched via a wrapper such as Apache+uWSGI may allow for multiple distinct worker processes, but these workers typically limit the number of request processing threads that are permitted to execute. This means requests can stack up in the front-end webserver and be released to the `ironic-api` as prior requests complete. In environments with long running synchronous calls, such as use of the vendor passthru interface, this can be very problematic.
- As a combined `ironic` process. In this case, a single primary process with two worker sub-processes are used.

When the webserver is launched by the API process directly, the default is based upon the number of CPU sockets in your machine.

When launching using uwsgi, this will entirely vary upon your configuration, but balancing workers/threads based upon your load and needs is highly advisable. Each worker process is unique and consumes far more memory than a comparable number of worker threads.

Note

Host operating systems featuring in-memory de-duplication should see an improvement in the overall memory footprint with multiple processes, but this is not something the development team has measured and will vary based upon multiple factors.

One important item to note: each Ironic API service/process *does* keep a copy of the hash ring as generated from the database *in-memory*. This is done to help allocate load across a cluster in-line with how individual nodes and their responsible conductors are allocated across the cluster. In other words, your amount of memory WILL increase corresponding to the number of nodes managed by each ironic conductor. It is important to understand that features such as conductor groups means that only matching portions of nodes will be considered for the hash ring if needed.

Conductor

A conductor process will launch a number of other processes, as required, in order to complete the requested work. Ultimately this means it can quickly consume large amounts of memory because it was asked to complete a substantial amount of work all at once.

The `ironic-conductor` from ironic 15.1 consumes by default about 340MB of RAM in an idle configuration. This process, by default, operates as a single process. Additional processes can be launched, but they must have unique resolvable hostnames and addresses for JSON-RPC or use a central oslo.messaging supported message bus in order for Webserver API to Conductor API communication to be functional.

Typically, the most memory intensive operation that can be triggered is a image conversion for deployment, which is limited to 1GB of RAM per conversion process.

Most deployments, by default, do have a concurrency limit depending on their Compute (See nova.conf setting `max_concurrent_builds`) configuration. However, this is only per `nova-compute` worker, so naturally this concurrency will scale with additional workers.

Stand-alone users can easily request deployments exceeding the Compute service default maximum concurrent builds. As such, if your environment is used this way, you may wish to carefully consider your deployment architecture.

With a single nova-compute process talking to a single conductor, asked to perform ten concurrent deployments of images requiring conversion, the memory needed may exceed 10GB. This does however, entirely depend upon image block structure and layout, and what deploy interface is being used.

Threads

The conductor uses python threads to enable concurrency. When a request comes from the API to the conductor over the RPC, the conductor verifies it, acquires a node-level lock (if needed) and launches a processing thread for further handling. The maximum number of such threads is limited to the value of *`conductor.workers_pool_size`* configuration option.

Note

Some workers are always or regularly occupied by internal processes, e.g. *Power Synchronization*.

Once the limit is reached, any new requests will be denied with HTTP code 503 (service unavailable). The clients are expected to be able to handle this code, most likely by retrying after a short delay or by throttling their requests. If you see a large number of this errors, you may try raising the limit gradually, while observing the conductor behavior and making sure the requests dont start to take longer because of switching between threads. A better alternative is to increase the number of conductors because it will also allow using more than one CPU core.

Note

Running more than one conductor on the same machine is a somewhat uncharted territory. You need to make sure they either have separate HTTP servers or share the same HTTP server without conflicting.

If you use JSON RPC, you also need to make sure the ports dont conflict by setting the *`json_rpc.port`* option.

Starting with the 2024.1 Caracal release cycle, a small proportion of the threads (specified by the *`conductor.reserved_workers_pool_percentage`* option) is reserved for API requests and other critical tasks. Periodic tasks and agent heartbeats cannot use them. This ensures that the API stays responsive even under extreme internal load.

Database

Query load upon the database is one of the biggest potential bottlenecks which can cascade across a deployment and ultimately degrade service to an Ironic user.

Often, depending on load, query patterns, periodic tasks, and so on and so forth, additional indexes may be needed to help provide hints to the database so it can most efficiently attempt to reduce the number of rows which need to be examined in order to return a result set.

Adding indexes

This example below is specific to MariaDB/MySQL, but the syntax should be easy to modify for operators using PostgreSQL.

```
use ironic;
create index owner_idx on nodes (owner) LOCK = SHARED;
create index lessee_idx on nodes (lessee) LOCK = SHARED;
create index driver_idx on nodes (driver) LOCK = SHARED;
create index provision_state_idx on nodes (provision_state) LOCK = SHARED;
create index reservation_idx on nodes (reservation) LOCK = SHARED;
create index conductor_group_idx on nodes (conductor_group) LOCK = SHARED;
create index resource_class_idx on nodes (resource_class) LOCK = SHARED;
```

Note

The indexes noted have been added automatically by Xena versions of Ironic and later. They are provided here as an example and operators can add them manually prior with versions of Ironic. The database upgrade for the Xena release of Ironic which adds these indexes are only aware of being able to skip index creation if it already exists on MySQL/MariaDB.

Note

It may be possible to use LOCK = NONE. Basic testing indicates this takes a little bit longer, but shouldnt result in the database table becoming write locked during the index creation. If the database engine cannot support this, then the index creation will fail.

Database platforms also have a concept of what is called a compound index where the index is aligned with the exact query pattern being submitted to the database. The database is able to use this compound index to attempt to drastically reduce the result set generation time for the remainder of the query. As of the composition of this document, we do not ship compound indexes in Ironic as we feel the most general benefit is single column indexes, and depending on data present, an operator may wish to explore compound indexes with their database administrator, as comound indexes can also have negative performance impacts if improperly constructed.

```
use ironic;
create index my_custom_app_query_index on nodes (reservation, provision_state,
↪ driver);
```

The risk, and *WHY* you should engage a Database Administrator, is depending on your configuration, the actual index may need to include one or more additional fields such as owner or lessee which may

be added on to the index. At the same time, queries with less field matches, or in different orders will exhibit different performance as the compound index may not be able to be consulted.

Indexes will not fix everything

Indexes are not a magical cure-all for all API or database performance issues, but they are an incredibly important part depending on data access and query patterns.

The underlying object layer and data conversions including record pagination do add a substantial amount of overhead to what may otherwise return as a result set on a manual database query. In Ironics case, due to the object model and the need to extract multiple pieces of data at varying levels of the data model to handle cases such as upgrades, the entire result set is downloaded and transformed which is an overhead you do not experience with a command line database client.

BMC interaction

In its default configuration, Ironic runs a periodic task to synchronize the power state of the managed physical nodes with the Ironic database. For the hardware type `ipmi` (see *IPMI driver*) and depending on the number of nodes, the network connectivity, and the parallelism of these queries, this synchronization may fail and retries will be triggered. Please find more details on the power synchronization and which options to adapt in case too many power sync failures occur in the section on *Power Synchronization*.

What can I do?

Previously in this document, weve already suggested some architectural constraints and limitations, but there are some things that can be done to maximize performance. Again, this will vary greatly depending on your use.

- Use the `direct` deploy interface. This offloads any final image conversion to the host running the `ironic-python-agent`. Additionally, if Swift or other object storage such as RadosGW is used, downloads can be completely separated from the host running the `ironic-conductor`.
- Use small/compact raw images. Qcow2 files are generally compressed and require substantial amounts of memory to decompress and stream.
- Tune the internal memory limit for the conductor using the `[DEFAULT]memory_required_minimum` setting. This will help the conductor throttle back memory intensive operations. The default should prevent Out-of-Memory operations, but under extreme memory pressure this may still be sub-optimal. Before changing this setting, it is highly advised to consult with your resident Unix wizard or even the Ironic development team in upstream IRC. This feature was added in the Wallaby development cycle.
- If network bandwidth is the problem you are seeking to solve for, you may wish to explore a mix of the `direct` deploy interface and caching proxies. Such a configuration can be highly beneficial in wide area deployments. See *Using proxies for image download*.
- If youre making use of large configuration drives, you may wish to ensure youre using Swift to store them as opposed to housing them inside of the database. The entire object and contents are returned whenever Ironic needs to evaluate the entire node, which can become a performance impact. For more information on configuration drives, please see *Enabling the configuration drive*.

4.3.16 Secure RBAC

Suggested Reading

It is likely an understatement to say that policy enforcement is a complex subject. It requires operational context to craft custom policy to meet general use needs. Part of this is why the Secure RBAC effort was started, to provide consistency and a good starting place for most users who need a higher level of granularity.

That being said, it would likely help anyone working to implement customization of these policies to consult some reference material in hopes of understanding the context.

- Keystone Administrator Guide - Service API Protection
- Ironic Scoped Role Based Access Control Specification

Historical Context - How we reached our access model

Ironic has reached the access model through an evolution the API and the data stored. Along with the data stored, the enforcement of policy based upon data stored in these fields.

- Ownership Information Storage
- Allow Node owners to Administer
- Allow Leasable Nodes

System Scoped

System scoped authentication is intended for administrative activities such as those crossing tenants/projects, as all tenants/projects should be visible to `system` scoped users in Ironic.

System scoped requests do not have an associated `project_id` value for the Keystone request authorization token utilized to speak with Ironic. These requests are translated through keystonemiddleware into values which tell Ironic what to do. Or to be more precise, tell the policy enforcement framework the information necessary to make decisions.

System scoped requests very much align with the access controls of Ironic before the Secure RBAC effort. The original custom role `baremetal_admin` privileges are identical to a system scoped `admin`s privileges. Similarly `baremetal_observer` is identical to a system scoped `reader`. In these concepts, the `admin` is allowed to create/delete objects/items. The `reader` is allowed to read details about items and is intended for users who may need an account with read-only access for or front-line support purposes.

In addition to these concepts, a `member` role exists in the Secure RBAC use model. Ironic does support this role, and in general `member` role users in a system scope are able to perform basic updates/changes, with the exception of special fields like those to disable cleaning.

Project Scoped

Project scoped authentication is when a request token and associated records indicate an associated `project_id` value.

The Secure RBAC model, since the introduction of the base capability has been extended as a result of an OpenStack community goal to include a `manager` role in the project scope. By default, this access is equivalent to a Project scoped `admin` user, however it may be delineated further as time moves forward.

Legacy Behavior

The legacy behavior of API service is that all requests are treated as project scoped requests where access is governed using an admin project. This behavior is *deprecated*. The new behavior is a delineation of access through `system` scoped and `project` scoped requests.

In essence, what would have served as an admin project, is now `system` scoped usage.

Previously, Ironic API, by default, responded with access denied or permitted based upon the admin project and associated role. These responses would generate an HTTP 403 if the project was incorrect or if a user role.

Note

While Ironic has had the concept of an `owner` and a `lessee`, they are *NOT* used by default. They require custom policy configuration files to be used in the legacy operating mode.

Supported Endpoints

- /nodes
- /nodes/<uuid>/ports
- /nodes/<uuid>/portgroups
- /nodes/<uuid>/volume/connectors
- /nodes/<uuid>/volume/targets
- /nodes/<uuid>/allocation
- /ports
- /portgroups
- /volume/connectors
- /volume/targets
- /allocations

How Project Scoped Works

Ironic has two project use models where access is generally more delegative to an `owner` and access to a `lessee` is generally more utilitarian.

The purpose of an owner, is more to enable the System Operator to delegate much of the administrative activity of a Node to the owner. This may be because they physically own the hardware, or they are in charge of the node. Regardless of the use model that the fields and mechanics support, these fields are to support humans, and possibly services where applicable.

The purpose of a lessee is more for a *tenant* in their *project* to be able to have access to perform basic actions with the API. In some cases that may be to reprovision or rebuild a node. Ultimately that is the lessees prerogative, but by default there are actions and field updates that cannot be performed by default. This is also governed by access level within a project.

These policies are applied in the way data is viewed and how data can be updated. Generally, an inability to view a node is an access permission issue in term of the project ID being correct for owner/lessee.

The ironic project has attempted to generally codify what we believe is reasonable, however operators may wish to override these policy settings. For details general policy setting details, please see *Policies*.

Field value visibility restrictions

Ironics API, by default has a concept of filtering node values to prevent sensitive data from being leaked. System scoped users are subjected to basic restrictions, whereas project scoped users are, by default, examined further and against additional policies. This threshold is controlled with the `baremetal:node:get:filter_threshold`.

By default, the following fields are masked on Nodes and are controlled by the associated policies. By default, owners are able to see insight into the infrastructure, whereas lessee users *CANNOT* view these fields by default.

- `last_error` - `baremetal:node:get:last_error`
- `reservation` - `baremetal:node:get:reservation`
- `driver_internal_info` - `baremetal:node:get:driver_internal_info`
- `driver_info` - `baremetal:node:get:driver_info`

Field update restrictions

Some of the fields in this list are restricted to System scoped users, or even only System Administrators. Some of these default restrictions are likely obvious. Owners cant change the owner. Lessees cant change the owner.

- `driver_info` - `baremetal:node:update:driver_info`
- `properties` - `baremetal:node:update:properties`
- `chassis_uuid` - `baremetal:node:update:chassis_uuid`
- `instance_uuid` - `baremetal:node:update:instance_uuid`
- `lessee` - `baremetal:node:update:lessee`
- `owner` - `baremetal:node:update:owner`
- `driver` - `baremetal:node:update:driver_interfaces`
- `*_interface` - `baremetal:node:update:driver_interfaces`
- `network_data` - `baremetal:node:update:network_data`
- `conductor_group` - `baremetal:node:update:conductor_group`
- `name` - `baremetal:node:update:name`
- `retired` - `baremetal:node:update:driver_info`
- `retired_reason` - `baremetal:node:update:retired`

Warning

The `chassis_uuid` field is a write-once-only field. As such it is restricted to system scoped administrators.

More information is available on these fields in *Policies*.

Allocations

The `allocations` endpoint of the API is somewhat different than other endpoints as it allows for the allocation of physical machines to an admin. In this context, there is not already an `owner` or `project_id` to leverage to control access for the creation process, any project member does have the inherent privilege of requesting an allocation. That being said, their allocation request will require physical nodes to be owned or leased to the `project_id` through the `node` fields `owner` or `lessee`.

Ability to override the owner is restricted to system scoped users by default and any new allocation being requested with a specific owner, if made in `project` scope, will have the `project_id` recorded as the owner of the allocation.

Ultimately, an operational behavior difference exists between the `owner` and `lessee` rights in terms of allocations. With the standard access rights, `lessee` users are able to create allocations if they own nodes which are not allocated or deployed, but they cannot reprovision nodes when using only a `member` role. This limitation is not the case for project-scoped users with the `admin` role.

Warning

The allocation endpoints use is restricted to project scoped interactions until `[oslo_policy]enforce_new_defaults` has been set to `True` using the `baremetal:allocation:create_pre_rbac` policy rule. This is in order to prevent endpoint misuse. Afterwards all project scoped allocations will automatically populate an owner. System scoped request are not subjected to this restriction, and operators may change the default restriction via the `baremetal:allocation:create_restricted` policy.

Practical differences

Most users, upon implementing the use of `system` scoped authentication should not notice a difference as long as their authentication token is properly scoped to `system` and with the appropriate role for their access level. For most users who used a `baremetal` project, or other custom project via a custom policy file, along with a custom role name such as `baremetal_admin`, this will require changing the user to be a `system` scoped user with `admin` privileges.

The most noticeable difference for API consumers is the HTTP 403 access code is now mainly a HTTP 404 access code. The access concept has changed from Does the user broadly have access to the API? to Does user have access to the node, and then do they have access to the specific resource?.

What is an owner or lessee?

An `owner` or `lessee` is the project which has been assigned baremetal resources. Generally these should be service projects as opposed to a project dedicated to a specific user. This will help prevent the need to involve a `system` scoped administrator from having to correct ownership records should a project need to be removed due to an individuals departure.

The underlying `project_id` is used to represent and associate the owner or lessee.

How do I assign an owner?

```
# baremetal node set --owner <project_id> <node>
```

Note

With the default access policy, an `owner` is able to change the assigned `lessee` of a node. However the `lessee` is unable to do the same.

How do I assign a lessee?

```
# baremetal node set --lessee <project_id> <node>
```

Ironic will, by default, automatically manage lessee at deployment time, setting the lessee field on deploy of a node and unset it before the node begins cleaning.

Operators can customize or disable this behavior via *conductor.automatic_lessee_source* configuration.

If *conductor.automatic_lessee_source* is set to `instance` (the default), this uses `node.instance_info['project_id']`, which is set when OpenStack Nova deploys an instance.

If *conductor.automatic_lessee_source* is set to `request`, the lessee is set to the project_id in the request context ideal for standalone Ironic deployments still utilizing OpenStack Keystone.

If *conductor.automatic_lessee_source* is set to to `none`, Ironic not will set a lessee on deploy.

What is the difference between an owner and lessee?

This is largely covered in *How Project Scoped Works* although as noted it is largely in means of access. A `lessee` is far more restrictive and an `owner` may revoke access to `lessee`.

Access to the underlying baremetal node is not exclusive between the `owner` and `lessee`, and this use model expects that some level of communication takes place between the appropriate parties.

Can I, a project admin, create a node?

Starting in API version `1.80`, the capability was added to allow users with an `admin` role to be able to create and delete their own nodes in Ironic.

This functionality is enabled by default, and automatically imparts `owner` privileges to the created Bare Metal node.

This functionality can be disabled by setting *api.project_admin_can_manage_own_nodes* to `False`.

Can I use a service role?

In later versions of Ironic, the `service` role has been added to enable delineation of accounts and access to Ironics API. As Ironics API was largely originally intended as an admin API service, the service role enables similar levels of access as a project-scoped user with the `admin` or `manager` roles.

In terms of access, this is likely best viewed as a user with the `manager` role, but with slight elevation in privilege to enable usage of the service via a service account.

A project scoped user with the `service` role is able to create baremetal nodes, but is not able to delete them. To disable the ability to create nodes, set the *api.project_admin_can_manage_own_nodes* setting to `False`. The nodes which can be accessed/managed in the project scope also align with the `owner` and `lessee` access model, and thus if nodes are not matching the users `project_id`, then Ironics API will appear not to have any enrolled baremetal nodes.

With the system scope, a user with the `service` role is able to create baremetal nodes, but also, not delete them. The access rights are modeled such an `admin` scoped is needed to delete baremetal nodes from Ironic.

4.3.17 Dashboard Integration

A plugin for the OpenStack Dashboard (horizon) service is under development. Documentation for that can be found within the ironic-ui project.

- Dashboard (horizon) plugin

4.4 Administrator Command References

Here are references for commands not elsewhere documented.

4.4.1 ironic-dbsync

The **ironic-dbsync** utility is used to create the database schema tables that the ironic services will use for storage. It can also be used to upgrade existing database tables when migrating between different versions of ironic.

The Alembic library is used to perform the database migrations.

Options

This is a partial list of the most useful options. To see the full list, run the following:

```
ironic-dbsync --help
```

-h, --help
Show help message and exit.

--config-dir <DIR>
Path to a config directory with configuration files.

--config-file <PATH>
Path to a configuration file to use.

-d, --debug
Print debugging output.

--version
Show the programs version number and exit.

upgrade, stamp, revision, version, create_schema, online_data_migrations
The *command* to run.

Usage

Options for the various *commands* for **ironic-dbsync** are listed when the *-h* or *--help* option is used after the command.

For example:

```
ironic-dbsync create_schema --help
```

Information about the database is read from the ironic configuration file used by the API server and conductor services. This file must be specified with the *--config-file* option:

```
ironic-dbsync --config-file /path/to/ironic.conf create_schema
```

The configuration file defines the database backend to use with the *connection* database option:

```
[database]
connection=mysql+pymysql://root@localhost/ironic
```

If no configuration file is specified with the *--config-file* option, **ironic-dbsync** assumes an SQLite database.

Command Options

ironic-dbsync is given a command that tells the utility what actions to perform. These commands can take arguments. Several commands are available:

create_schema

-h, --help

Show help for create_schema and exit.

This command will create database tables based on the most current version. It assumes that there are no existing tables.

An example of creating database tables with the most recent version:

```
ironic-dbsync --config-file=/etc/ironic/ironic.conf create_schema
```

online_data_migrations

-h, --help

Show help for online_data_migrations and exit.

--max-count <NUMBER>

The maximum number of objects (a positive value) to migrate. Optional. If not specified, all the objects will be migrated (in batches of 50 to avoid locking the database for long periods of time).

--option <MIGRATION.KEY=VALUE>

If a migration accepts additional parameters, they can be passed via this argument. It can be specified several times.

This command will migrate objects in the database to their most recent versions. This command must be successfully run (return code 0) before upgrading to a future release.

It returns:

- 1 (not completed) if there are still pending objects to be migrated. Before upgrading to a newer release, this command must be run until 0 is returned.
- 0 (success) after migrations are finished or there are no data to migrate

- 127 (error) if max-count is not a positive value or an option is invalid
- 2 (error) if the database is not compatible with this release. This command needs to be run using the previous release of ironic, before upgrading and running it with this release.

revision

-h, --help

Show help for revision and exit.

-m <MESSAGE>**, --message** <MESSAGE>

The message to use with the revision file.

--autogenerate

Compares table metadata in the application with the status of the database and generates migrations based on this comparison.

This command will create a new revision file. You can use the *--message* option to comment the revision.

This is really only useful for ironic developers making changes that require database changes. This revision file is used during database migration and will specify the changes that need to be made to the database tables. Further discussion is beyond the scope of this document.

stamp

-h, --help

Show help for stamp and exit.

--revision <REVISION>

The revision number.

This command will stamp the revision table with the version specified with the *--revision* option. It will not run any migrations.

upgrade

-h, --help

Show help for upgrade and exit.

--revision <REVISION>

The revision number to upgrade to.

This command will upgrade existing database tables to the most recent version, or to the version specified with the *--revision* option.

Before this upgrade is invoked, the command **ironic-dbsync online_data_migrations** must have been successfully run using the previous version of ironic (if you are doing an upgrade as opposed to a new installation of ironic). If it wasnt run, the database will not be compatible with this recent version of ironic, and this command will return 2 (error).

If there are no existing tables, then new tables are created, beginning with the oldest known version, and successively upgraded using all of the database migration files, until they are at the specified version. Note that this behavior is different from the *create_schema* command that creates the tables based on the most recent version.

An example of upgrading to the most recent table versions:

```
ironic-dbsync --config-file=/etc/ironic/ironic.conf upgrade
```

Note

This command is the default if no command is given to **ironic-dbsync**.

Warning

The upgrade command is not compatible with SQLite databases since it uses ALTER TABLE commands to upgrade the database tables. SQLite supports only a limited subset of ALTER TABLE.

version

-h, --help
: Show help for version and exit.

This command will output the current database version.

4.4.2 ironic-status

Synopsis

```
ironic-status <category> <command> [<args>]
```

Description

ironic-status is a tool that provides routines for checking the status of a Ironic deployment.

Options

The standard pattern for executing a **ironic-status** command is:

```
ironic-status <category> <command> [<args>]
```

Run without arguments to see a list of available command categories:

```
ironic-status
```

Categories are:

- `upgrade`

Detailed descriptions are below.

You can also run with a category argument such as `upgrade` to see a list of all commands in that category:

```
ironic-status upgrade
```

These sections describe the available categories and arguments for **ironic-status**.

Upgrade

`ironic-status upgrade check`

Performs a release-specific readiness check before restarting services with new code. This command expects to have complete configuration and access to databases and services.

Return Codes

Return code	Description
0	All upgrade readiness checks passed successfully and there is nothing to do.
1	At least one check encountered an issue and requires further investigation. This is considered a warning but the upgrade may be OK.
2	There was an upgrade status check failure that needs to be investigated. This should be considered something that stops an upgrade.
255	An unexpected error occurred.

History of Checks

12.0.0 (Stein)

- Adds a check for compatibility of the object versions with the release of ironic.

Wallaby

- Adds a check to validate the configured policy file is not JSON based as JSON based policies have been deprecated.

2024.2

- Adds a check that hardware types and interfaces in the configuration exist, and that nodes are configured with existing drivers and interfaces.

4.5 Configuration Reference for Ironic

Many aspects of the Bare Metal service are specific to the environment it is deployed in. The following pages describe configuration options that can be used to adjust the service to your particular situation.

4.5.1 Configuration Options

The following is an overview of all available configuration options in Ironic. For a sample configuration file, refer to *Sample Configuration File*.

DEFAULT

`run_external_periodic_tasks`

Type
boolean

Default
`True`

Some periodic tasks can be run in a separate process. Should we run them here?

`debug`

Type
boolean

Default
`False`

Mutable
This option can be changed without restarting.

If set to true, the logging level will be set to DEBUG instead of the default INFO level.

`log_config_append`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

The name of a logging configuration file. This file is appended to any existing logging configuration files. For details about logging configuration files, see the Python logging module documentation. Note that when logging configuration files are used then all logging configuration is set in the configuration file and other logging configuration options are ignored (for example, log-date-format).

Table 20: Deprecated Variations

Group	Name
DEFAULT	log-config
DEFAULT	log_config

`log_date_format`

Type
string

Default
`%Y-%m-%d %H:%M:%S`

Defines the format string for %(asctime)s in log records. Default: the value above . This option is ignored if log_config_append is set.

`log_file`

Type
string

Default
<None>

(Optional) Name of log file to send logging output to. If no default is set, logging will go to stderr as defined by use_stderr. This option is ignored if log_config_append is set.

Table 21: Deprecated Variations

Group	Name
DEFAULT	logfile

`log_dir`

Type
string

Default
<None>

(Optional) The base directory used for relative log_file paths. This option is ignored if log_config_append is set.

Table 22: Deprecated Variations

Group	Name
DEFAULT	logdir

`use_syslog`

Type
boolean

Default
`False`

Use syslog for logging. Existing syslog format is DEPRECATED and will be changed later to honor RFC5424. This option is ignored if log_config_append is set.

`use_journal`

Type
boolean

Default
`False`

Enable journald for logging. If running in a systemd environment you may wish to enable journal support. Doing so will use the journal native protocol which includes structured metadata in addition to log messages.This option is ignored if log_config_append is set.

`syslog_log_facility`

Type
string

Default
`LOG_USER`

Syslog facility to receive log lines. This option is ignored if log_config_append is set.

`use_json`

Type
boolean

Default
`False`

Use JSON formatting for logging. This option is ignored if log_config_append is set.

`use_stderr`

Type
boolean

Default
`False`

Log output to standard error. This option is ignored if log_config_append is set.

`log_color`

Type
boolean

Default
`False`

(Optional) Set the color key according to log levels. This option takes effect only when logging to stderr or stdout is used. This option is ignored if log_config_append is set.

`log_rotate_interval`

Type
integer

Default
1

The amount of time before the log files are rotated. This option is ignored unless log_rotation_type is set to interval.

`log_rotate_interval_type`

Type
string

Default
`days`

Valid Values
Seconds, Minutes, Hours, Days, Weekday, Midnight

Rotation interval type. The time of the last file change (or the time when the service was started) is used when scheduling the next rotation.

`max_logfile_count`

Type
integer

Default
30

Maximum number of rotated log files.

`max_logfile_size_mb`

Type
integer

Default
200

Log file maximum size in MB. This option is ignored if log_rotation_type is not set to size.

`log_rotation_type`

Type
string

Default
none

Valid Values
interval, size, none

Log rotation type.

Possible values

interval
Rotate logs at predefined time intervals.

size
Rotate logs once they reach a predefined size.

none
Do not rotate log files.

`logging_context_format_string`

Type
string

Default
```
%(asctime)s.%(msecs)03d %(process)d %(levelname)s %(name)s
[%(global_request_id)s %(request_id)s %(user_identity)s]
%(instance)s%(message)s
```

Format string to use for log messages with context. Used by oslo_log.formatters.ContextFormatter

`logging_default_format_string`

Type
string

Default
```
%(asctime)s.%(msecs)03d %(process)d %(levelname)s %(name)s [-]
%(instance)s%(message)s
```

Format string to use for log messages when context is undefined. Used by oslo_log.formatters.ContextFormatter

`logging_debug_format_suffix`

Type
string

Default
`%(funcName)s %(pathname)s:%(lineno)d`

Additional data to append to log message when logging level for the message is DEBUG. Used by oslo_log.formatters.ContextFormatter

`logging_exception_prefix`

Type
string

Default
`%(asctime)s.%(msecs)03d %(process)d ERROR %(name)s %(instance)s`

Prefix each line of exception output with this format. Used by oslo_log.formatters.ContextFormatter

`logging_user_identity_format`

Type
string

Default
`%(user)s %(project)s %(domain)s %(system_scope)s %(user_domain)s %(project_domain)s`

Defines the format string for %(user_identity)s that is used in logging_context_format_string. Used by oslo_log.formatters.ContextFormatter

`default_log_levels`

Type
list

Default
`['amqp=WARNING', 'amqplib=WARNING', 'qpid.messaging=INFO', 'oslo.messaging=INFO', 'oslo_messaging=INFO', 'stevedore=INFO', 'iso8601=WARNING', 'requests=WARNING', 'urllib3.connectionpool=WARNING', 'keystonemiddleware.auth_token=INFO', 'keystoneauth.session=INFO', 'openstack=WARNING', 'oslo_policy=WARNING', 'oslo_concurrency.lockutils=WARNING']`

List of package logging levels in logger=LEVEL pairs. This option is ignored if log_config_append is set.

`publish_errors`

Type
boolean

Default
`False`

Enables or disables publication of error events.

`instance_format`

Type
string

Default
`"[instance: %(uuid)s] "`

The format for an instance that is passed with the log message.

`instance_uuid_format`

Type
string

Default
`"[instance: %(uuid)s] "`

The format for an instance UUID that is passed with the log message.

`rate_limit_interval`

Type
integer

Default
`0`

Interval, number of seconds, of log rate limiting.

`rate_limit_burst`

Type
integer

Default
`0`

Maximum number of logged messages per rate_limit_interval.

`rate_limit_except_level`

Type
string

Default
`CRITICAL`

Valid Values
CRITICAL, ERROR, INFO, WARNING, DEBUG,

Log level name used by rate limiting. Logs with level greater or equal to rate_limit_except_level are not filtered. An empty string means that all levels are filtered.

`fatal_deprecations`

Type
boolean

Default
`False`

Enables or disables fatal status of deprecations.

`auth_strategy`

Type
string

Default
`keystone`

Valid Values
noauth, keystone, http_basic

Authentication strategy used by ironic-api. noauth should not be used in a production environment because all authentication will be disabled creating insecure operating conditions.

Possible values

noauth
no authentication

keystone
use the Identity service for authentication

http_basic
HTTP basic authentication

`http_basic_auth_user_file`

Type
string

Default
`/etc/ironic/htpasswd`

Path to Apache format user authentication file used when auth_strategy=http_basic

`allow_image_access_via_auth_token`

Type
boolean

Default
`False`

If True, Ironic allows access to Glance images if an auth_token is present in the request context.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

`ignore_project_check_for_admin_tasks`

Type
boolean

Default
`True`

If True, allows admin tasks to access image withoutmatching project_id

`debug_tracebacks_in_api`

Type
boolean

Default
`False`

Return server tracebacks in the API response for any error responses. WARNING: this is insecure and should not be used in a production environment.

`pecan_debug`

Type
boolean

Default
`False`

Enable pecan debug mode. WARNING: this is insecure and should not be used in a production environment.

`default_resource_class`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

Resource class to use for new nodes when no resource class is provided in the creation request.

`default_conductor_group`

Type
string

Default
`''`

Mutable
This option can be changed without restarting.

The conductor_group to use for new nodes when no conductor_group was defined in the creation request.

`use_rpc_for_database`

Type
boolean

Default
`False`

If the Ironic API should utilize the RPC layer for database interactions as opposed to directly connecting to the database API endpoint.

`enabled_hardware_types`

Type
list

Default
`['ipmi', 'redfish']`

Specify the list of hardware types to load during service initialization. Missing hardware types, or hardware types which fail to initialize, will prevent the conductor service from starting. This option defaults to a recommended set of production-oriented hardware types. A complete list of hardware types present on your system may be found by enumerating the ironic.hardware.types entrypoint.

`enabled_bios_interfaces`

Type
list

Default
`['no-bios', 'redfish']`

Specify the list of bios interfaces to load during service initialization. Missing bios interfaces, or bios interfaces which fail to initialize, will prevent the ironic-conductor service from starting. At least one bios interface that is supported by each enabled hardware type must be enabled here, or the ironic-conductor service will not start. Must not be an empty list. The default value is a recommended set of production-oriented bios interfaces. A complete list of bios interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.bios entrypoint. When setting this value, please make sure that every enabled hardware type will have the same set of enabled bios interfaces on every ironic-conductor service.

`default_bios_interface`

Type
string

Default
`<None>`

Default bios interface to be used for nodes that do not have bios_interface field set. A complete list of bios interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.bios entrypoint.

`enabled_boot_interfaces`

Type
list

Default
`['ipxe', 'pxe', 'redfish-virtual-media']`

Specify the list of boot interfaces to load during service initialization. Missing boot interfaces, or boot interfaces which fail to initialize, will prevent the ironic-conductor service from starting. At least one boot interface that is supported by each enabled hardware type must be enabled here, or the ironic-conductor service will not start. Must not be an empty list. The default value is a recommended set of production-oriented boot interfaces. A complete list of boot interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.boot entrypoint. When setting this value, please make sure that every enabled hardware type will have the same set of enabled boot interfaces on every ironic-conductor service.

`default_boot_interface`

Type
string

Default
<None>

Default boot interface to be used for nodes that do not have boot_interface field set. A complete list of boot interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.boot entrypoint.

`enabled_console_interfaces`

Type
list

Default
`['no-console']`

Specify the list of console interfaces to load during service initialization. Missing console interfaces, or console interfaces which fail to initialize, will prevent the ironic-conductor service from starting. At least one console interface that is supported by each enabled hardware type must be enabled here, or the ironic-conductor service will not start. Must not be an empty list. The default value is a recommended set of production-oriented console interfaces. A complete list of console interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.console entrypoint. When setting this value, please make sure that every enabled hardware type will have the same set of enabled console interfaces on every ironic-conductor service.

`default_console_interface`

Type
string

Default
<None>

Default console interface to be used for nodes that do not have console_interface field set. A complete list of console interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.console entrypoint.

`enabled_deploy_interfaces`

Type
list

Default
`['autodetect', 'direct', 'ramdisk', 'bootc']`

Specify the list of deploy interfaces to load during service initialization. Missing deploy interfaces, or deploy interfaces which fail to initialize, will prevent the ironic-conductor service from starting. At least one deploy interface that is supported by each enabled hardware type must be enabled here, or the ironic-conductor service will not start. Must not be an empty list. The default value is a recommended set of production-oriented deploy interfaces. A complete list of deploy interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.deploy entrypoint. When setting this value, please make sure that every enabled hardware type will have the same set of enabled deploy interfaces on every ironic-conductor service.

`default_deploy_interface`

Type
string

Default
<None>

Default deploy interface to be used for nodes that do not have deploy_interface field set. A complete list of deploy interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.deploy entrypoint.

`autodetect_deploy_interfaces`

Type
list

Default
`['ramdisk', 'bootc', 'direct']`

List of deploy interfaces that the autodetect deploy interface is allowed to switch to. The order of interfaces in the list indicates the order support will be checked for,with the first interface having the highest priority. The last interface in the list will be chosen if no interfaces are detected as supported. ramdisk and bootc have detection support and direct is an appropriate fallback for most cases. All interfaces listed here must also be in enabled_deploy_interfaces.

`enabled_firmware_interfaces`

Type
list

Default
`['no-firmware']`

Specify the list of firmware interfaces to load during service initialization. Missing firmware interfaces, or firmware interfaces which fail to initialize, will prevent the ironic-conductor service from starting. At least one firmware interface that is supported by each enabled hardware type must be enabled here, or the ironic-conductor service will not start. Must not be an empty list. The default value is a recommended set of production-oriented firmware interfaces. A complete list of firmware interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.firmware entrypoint. When setting this value, please make sure that every enabled hardware type will have the same set of enabled firmware interfaces on every ironic-conductor service.

`default_firmware_interface`

Type
string

Default
<None>

Default firmware interface to be used for nodes that do not have firmware_interface field set. A complete list of firmware interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.firmware entrypoint.

`enabled_inspect_interfaces`

Type
list

Default
`['no-inspect', 'redfish']`

Specify the list of inspect interfaces to load during service initialization. Missing inspect interfaces, or inspect interfaces which fail to initialize, will prevent the ironic-conductor service from starting. At least one inspect interface that is supported by each enabled hardware type must be enabled here, or the ironic-conductor service will not start. Must not be an empty list. The default value is a recommended set of production-oriented inspect interfaces. A complete list of inspect interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.inspect entrypoint. When setting this value, please make sure that every enabled hardware type will have the same set of enabled inspect interfaces on every ironic-conductor service.

`default_inspect_interface`

Type
string

Default
<None>

Default inspect interface to be used for nodes that do not have inspect_interface field set. A complete list of inspect interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.inspect entrypoint.

`enabled_management_interfaces`

Type
list

Default
<None>

Specify the list of management interfaces to load during service initialization. Missing management interfaces, or management interfaces which fail to initialize, will prevent the ironic-conductor service from starting. At least one management interface that is supported by each enabled hardware type must be enabled here, or the ironic-conductor service will not start. Must not be an empty list. The default value is a recommended set of production-oriented management interfaces. A complete list of management interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.management entrypoint. When setting this value, please make sure that every enabled hardware type will have the same set of enabled management interfaces on every ironic-conductor service.

`default_management_interface`

Type
string

Default
<None>

Default management interface to be used for nodes that do not have management_interface field set. A complete list of management interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.management entrypoint.

`enabled_network_interfaces`

Type
list

Default
`['flat', 'noop']`

Specify the list of network interfaces to load during service initialization. Missing network interfaces, or network interfaces which fail to initialize, will prevent the ironic-conductor service from starting. At least one network interface that is supported by each enabled hardware type must be enabled here, or the ironic-conductor service will not start. Must not be an empty list. The default value is a recommended set of production-oriented network interfaces. A complete list of network interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.network entrypoint. When setting this value, please make sure that every enabled hardware type will have the same set of enabled network interfaces on every ironic-conductor service.

`default_network_interface`

Type
string

Default
<None>

Default network interface to be used for nodes that do not have network_interface field set. A complete list of network interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.network entrypoint.

`enabled_power_interfaces`

Type
list

Default
<None>

Specify the list of power interfaces to load during service initialization. Missing power interfaces, or power interfaces which fail to initialize, will prevent the ironic-conductor service from starting. At least one power interface that is supported by each enabled hardware type must be enabled here, or the ironic-conductor service will not start. Must not be an empty list. The default value is a recommended set of production-oriented power interfaces. A complete list of power interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.power entrypoint. When setting this value, please make sure that every enabled hardware type will have the same set of enabled power interfaces on every ironic-conductor service.

`default_power_interface`

Type
string

Default
<None>

Default power interface to be used for nodes that do not have power_interface field set. A complete list of power interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.power entrypoint.

`enabled_raid_interfaces`

Type
list

Default
`['agent', 'no-raid', 'redfish']`

Specify the list of raid interfaces to load during service initialization. Missing raid interfaces, or raid interfaces which fail to initialize, will prevent the ironic-conductor service from starting. At least one raid interface that is supported by each enabled hardware type must be enabled here, or the ironic-conductor service will not start. Must not be an empty list. The default value is a recommended set of production-oriented raid interfaces. A complete list of raid interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.raid entrypoint. When setting this value, please make sure that every enabled hardware type will have the same set of enabled raid interfaces on every ironic-conductor service.

`default_raid_interface`

Type
string

Default
<None>

Default raid interface to be used for nodes that do not have raid_interface field set. A complete list of raid interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.raid entrypoint.

`enabled_rescue_interfaces`

Type
list

Default
`['no-rescue']`

Specify the list of rescue interfaces to load during service initialization. Missing rescue interfaces, or rescue interfaces which fail to initialize, will prevent the ironic-conductor service from starting. At least one rescue interface that is supported by each enabled hardware type must be enabled here, or the ironic-conductor service will not start. Must not be an empty list. The default value is a recommended set of production-oriented rescue interfaces. A complete list of rescue interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.rescue entrypoint. When setting this value, please make sure that every enabled hardware type will have the same set of enabled rescue interfaces on every ironic-conductor service.

`default_rescue_interface`

Type
string

Default
<None>

Default rescue interface to be used for nodes that do not have rescue_interface field set. A complete list of rescue interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.rescue entrypoint.

`enabled_storage_interfaces`

Type
list

Default
`['cinder', 'noop']`

Specify the list of storage interfaces to load during service initialization. Missing storage interfaces, or storage interfaces which fail to initialize, will prevent the ironic-conductor service from starting. At least one storage interface that is supported by each enabled hardware type must be enabled here, or the ironic-conductor service will not start. Must not be an empty list. The default value is a recommended set of production-oriented storage interfaces. A complete list of storage interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.storage entrypoint. When setting this value, please make sure that every enabled hardware type will have the same set of enabled storage interfaces on every ironic-conductor service.

`default_storage_interface`

Type
string

Default
noop

Default storage interface to be used for nodes that do not have storage_interface field set. A complete list of storage interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.storage entrypoint.

`enabled_vendor_interfaces`

Type
list

Default
`['ipmitool', 'redfish', 'no-vendor']`

Specify the list of vendor interfaces to load during service initialization. Missing vendor interfaces, or vendor interfaces which fail to initialize, will prevent the ironic-conductor service from starting. At least one vendor interface that is supported by each enabled hardware type must be enabled here, or the ironic-conductor service will not start. Must not be an empty list. The default value is a recommended set of production-oriented vendor interfaces. A complete list of vendor interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.vendor entrypoint. When setting this value, please make sure that every enabled hardware type will have the same set of enabled vendor interfaces on every ironic-conductor service.

`default_vendor_interface`

Type
string

Default
<None>

Default vendor interface to be used for nodes that do not have vendor_interface field set. A complete list of vendor interfaces present on your system may be found by enumerating the ironic.hardware.interfaces.vendor entrypoint.

`log_in_db_max_size`

Type
integer

Default
4096

Max number of characters of any node last_error/maintenance_reason pushed to database.

`hash_partition_exponent`

Type
integer

Default
5

Exponent to determine number of hash partitions to use when distributing load across conductors. Larger values will result in more even distribution of load and less load when rebalancing the ring, but more memory usage. Number of partitions per conductor is (2^hash_partition_exponent). This determines the granularity of rebalancing: given 10 hosts, and an exponent of the 2, there are 40 partitions in the ring.A few thousand partitions should make rebalancing smooth in most cases. The default is suitable for up to a few hundred conductors. Configuring for too many partitions has a negative impact on CPU usage.

`hash_ring_reset_interval`

Type
integer

Default
15

Time (in seconds) after which the hash ring is considered outdated and is refreshed on the next access.

`hash_ring_algorithm`

Type
string

Default
md5

Valid Values
shake_256, sha256, sha3_256, blake2s, blake2b, sha224, shake_128, sha3_224, sha3_384, sha512, sha3_512, md5, sha1, sha384

Advanced Option
Intended for advanced users and not used by the majority of users, and might have a significant effect on stability and/or performance.

Hash function to use when building the hash ring. If running on a FIPS system, do not use md5. WARNING: all ironic services in a cluster MUST use the same algorithm at all times. Changing the algorithm requires an offline update.

`force_raw_images`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

If True, convert backing images to raw disk image format.

`raw_image_growth_factor`

Type
floating point

Default
`2.0`

Minimum Value
1.0

The scale factor used for estimating the size of a raw image converted from compact image formats such as QCOW2. Default is 2.0, must be greater than 1.0.

`isolinux_bin`

Type
string

Default
`/usr/lib/syslinux/isolinux.bin`

Path to isolinux binary file.

`isolinux_config_template`

Type
string

Default
`$pybasedir/common/isolinux_config.template`

Template file for isolinux configuration file.

`grub_config_path`

Type
string

Default
`EFI/BOOT/grub.cfg`

GRUB2 configuration file location on the UEFI ISO images produced by ironic. The default value is usually incorrect and should not be relied on. If you use a GRUB2 image from a certain distribution, use a distribution-specific path here, e.g. EFI/ubuntu/grub.cfg

`grub_config_template`

Type
string

Default
`$pybasedir/common/grub_conf.template`

Template file for grub configuration file.

`ldlinux_c32`

Type
string

Default
<None>

Path to ldlinux.c32 file. This file is required for syslinux 5.0 or later. If not specified, the file is looked for in /usr/lib/syslinux/modules/bios/ldlinux.c32 and /usr/share/syslinux/ldlinux.c32.

`esp_image`

Type
string

Default
<None>

Path to EFI System Partition image file. This file is recommended for creating UEFI bootable ISO images efficiently. ESP image should contain a FAT12/16/32-formatted file system holding EFI boot loaders (e.g. GRUB2) for each hardware architecture ironic needs to boot. This option is only used when neither ESP nor ISO deploy image is configured to the node being deployed in which case ironic will attempt to fetch ESP image from the configured location or extract ESP image from UEFI-bootable deploy ISO image.

`parallel_image_downloads`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

Run image downloads and raw format conversions in parallel.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
Use image_download_concurrency

`image_download_concurrency`

Type
integer

Default
`20`

Minimum Value
1

How many image downloads and raw format conversions to run in parallel. Only affects image caches.

`my_ip`

Type
string

Default
`127.0.0.1`

This option has a sample default set, which means that its actual default value may vary from the one documented above.

IPv4 address of this host. If unset, will determine the IP programmatically. If unable to do so, will use 127.0.0.1. NOTE: This field does accept an IPv6 address as an override for templates and URLs, however it is recommended that [DEFAULT]my_ipv6 is used along with DNS names for service URLs for dual-stack environments.

`my_ipv6`

Type
string

Default
`2001:db8::1`

This option has a sample default set, which means that its actual default value may vary from the one documented above.

IP address of this host using IPv6. This value must be supplied via the configuration and cannot be adequately programmatically determined like the [DEFAULT]my_ip parameter for IPv4.

`notification_level`

Type
string

Default
<None>

Valid Values
debug, info, warning, error, critical

Specifies the minimum level for which to send notifications. If not set, no notifications will be sent. The default is for this option to be unset.

Possible values

debug
debug level

info
info level

warning
warning level

error
error level

critical
critical level

`versioned_notifications_topics`

Type
list

Default
`['ironic_versioned_notifications']`

Specifies the topics for the versioned notifications issued by Ironic.

The default value is fine for most deployments and rarely needs to be changed. However, if you have a third-party service that consumes versioned notifications, it might be worth getting a topic for that service. Ironic will send a message containing a versioned notification payload to each topic queue in this list.

The list of versioned notifications is visible in https://docs.openstack.org/ironic/latest/admin/notifications.html

`pybasedir`

Type
string

Default
`/usr/lib/python/site-packages/ironic/ironic`

This option has a sample default set, which means that its actual default value may vary from the one documented above.

Directory where the ironic python module is installed.

`bindir`

Type
string

Default
`$pybasedir/bin`

Directory where ironic binaries are installed.

`state_path`

Type
string

Default
`$pybasedir`

Top-level directory for maintaining ironics state.

`default_portgroup_mode`

Type
string

Default
`active-backup`

Mutable
This option can be changed without restarting.

Default mode for portgroups. Allowed values can be found in the linux kernel documentation on bonding: https://www.kernel.org/doc/Documentation/networking/bonding.txt.

`host`

Type
string

Default
`localhost`

This option has a sample default set, which means that its actual default value may vary from the one documented above.

Name of this node. This can be an opaque identifier. It is not necessarily a hostname, FQDN, or IP address. However, the node name must be valid within an AMQP key.

`pin_release_version`

Type
string

Default
<None>

Valid Values
zed, yoga, antelope, 9.2, 35.0, 34.0, 33.0, 32.0, 31.0, 30.0, 29.0, 28.0, 27.0, 26.1, 26.0, 25.0, 24.1, 24.0, 23.1, 23.0, 22.1, 22.0, 21.4, 21.3, 21.2, 21.1, 21.0, 2026.1, 2025.2, 2025.1, 2024.2, 2024.1, 2023.2, 2023.1, 20.2, 20.1, 20.0, 19.0, 18.2, 18.1, 18.0, 17.0, 16.2, 16.1, 16.0, 15.1, 15.0, 14.0, 13.0, 12.2, 12.1, 12.0, 11.1, 11.0, 10.1, 10.0

Mutable
This option can be changed without restarting.

Used for rolling upgrades. Setting this option downgrades (or pins) the Bare Metal API, the internal ironic RPC communication, and the database objects to their respective versions, so they are compatible with older services. When doing a rolling upgrade from version N to version N+1, set (to pin) this to N. To unpin (default), leave it unset and the latest versions will be used.

Possible values

zed
: zed release

yoga
: yoga release

antelope
: antelope release

9.2
: 9.2 release

35.0
: 35.0 release

34.0
: 34.0 release

33.0
: 33.0 release

32.0
: 32.0 release

31.0
: 31.0 release

30.0
: 30.0 release

29.0
: 29.0 release

28.0
: 28.0 release

27.0
: 27.0 release

26.1
: 26.1 release

26.0
: 26.0 release

25.0
: 25.0 release

24.1
: 24.1 release

24.0
: 24.0 release

23.1
: 23.1 release

23.0
: 23.0 release

22.1
22.1 release

22.0
22.0 release

21.4
21.4 release

21.3
21.3 release

21.2
21.2 release

21.1
21.1 release

21.0
21.0 release

2026.1
2026.1 release

2025.2
2025.2 release

2025.1
2025.1 release

2024.2
2024.2 release

2024.1
2024.1 release

2023.2
2023.2 release

2023.1
2023.1 release

20.2
20.2 release

20.1
20.1 release

20.0
20.0 release

19.0
19.0 release

18.2
18.2 release

18.1
18.1 release

18.0
18.0 release

17.0
17.0 release

16.2
16.2 release

16.1
16.1 release

16.0
16.0 release

15.1
15.1 release

15.0
15.0 release

14.0
14.0 release

13.0
13.0 release

12.2
12.2 release

12.1
12.1 release

12.0
12.0 release

11.1
11.1 release

11.0
11.0 release

10.1
10.1 release

10.0
10.0 release

`rpc_transport`

Type
string

Default
`oslo`

Valid Values
oslo, json-rpc, none

Which RPC transport implementation to use between conductor and API services

Possible values

oslo
use oslo.messaging transport

json-rpc
use JSON RPC transport

none
No RPC, only use local conductor

`minimum_memory_warning_only`

Type
boolean

Default
`False`

Mutable
This option can be changed without restarting.

Setting to govern if Ironic should only warn instead of attempting to hold back the request in order to prevent the exhaustion of system memory.

`minimum_required_memory`

Type
integer

Default
`1024`

Mutable
This option can be changed without restarting.

Minimum memory in MiB for the system to have available prior to starting a memory intensive process on the conductor.

`minimum_memory_wait_time`

Type
integer

Default
`15`

Mutable
This option can be changed without restarting.

Seconds to wait between retries for free memory before launching the process. This, combined with `memory_wait_retries` allows the conductor to determine how long we should attempt to directly retry.

`minimum_memory_wait_retries`

Type
integer

Default
`6`

Mutable
This option can be changed without restarting.

Number of retries to hold onto the worker before failing or returning the thread to the pool if the conductor can automatically retry.

`drain_shutdown_timeout`

Type
integer

Default
`1800`

Mutable
This option can be changed without restarting.

Timeout (seconds) after which a server will exit from a drain shutdown. Drain shutdowns are triggered by sending the signal SIGUSR2. Zero value means shutdown will never be triggered by a timeout.

`tempdir`

Type
string

Default
`/tmp`

This option has a sample default set, which means that its actual default value may vary from the one documented above.

Temporary working directory, default is Python temp dir.

`webserver_verify_ca`

Type
string

Default
`True`

Mutable
This option can be changed without restarting.

CA certificates to be used for certificate verification. This can be either a Boolean value or a path to a CA_BUNDLE file.If set to True, the certificates present in the standard path are used to verify the host certificates.If set to False, the conductor will ignore verifying the SSL certificate presented by the host.If its a path, conductor uses the specified certificate for SSL verification. If the path does not exist, the behavior is same as when this value is set to True i.e the certificates present in the standard path are used for SSL verification.

`webserver_connection_timeout`

Type
integer

Default
`60`

Connection timeout when accessing/interacting with remote web servers with images or other artifacts being accessed. An excessive value here is not advisable as excessive requests to an unreachable endpoint can result in Ironic service resources being consumed waiting for the connection to timeout.

`rbac_service_role_elevated_access`

Type
boolean

Default
`False`

Enable elevated access for users with service role belonging to the rbac_service_project_name project when using default policy. The default setting of disabled causes all service role requests to be scoped to the project the service account belongs to.

`rbac_service_project_name`

Type
string

Default
`service`

The project name utilized for Role Based Access Control checks for the reserved *service* project. This project is utilized for services to have accounts for cross-service communication. Often these accounts require higher levels of access, and effectively this permits accounts from the service to not be restricted to project scoping of responses. i.e. The service project user with a *service* role will be able to see nodes across all projects, similar to System scoped access. If not set to a value, and all service role access will be filtered matching an *owner* or *lessee*, if applicable. If an operator wishes to make behavior visible for all service role users across all projects, then a custom policy must be used to override the default service_role rule. It should be noted that the value of service is a default convention for OpenStack deployments, but the requisite access and details around end configuration are largely up to an operator if they are doing an OpenStack deployment manually.

`executor_thread_pool_size`

Type
integer

Default
`64`

Size of executor thread pool when executor is threading or eventlet.

Table 23: Deprecated Variations

Group	Name
DEFAULT	rpc_thread_pool_size

`rpc_response_timeout`

Type
integer

Default
60

Seconds to wait for a response from a call.

`transport_url`

Type
string

Default
rabbit://

The network address and optional user credentials for connecting to the messaging backend, in URL format. The expected format is:

driver://[user:pass@]host:port[,[userN:passN@]hostN:portN]/virtual_host?query

Example: rabbit://rabbitmq:password@127.0.0.1:5672//

For full details on the fields in the URL see the documentation of oslo_messaging.TransportURL at https://docs.openstack.org/oslo.messaging/latest/reference/transport.html

`control_exchange`

Type
string

Default
openstack

The default exchange under which topics are scoped. May be overridden by an exchange name specified in the transport_url option.

`rpc_ping_enabled`

Type
boolean

Default
False

Add an endpoint to answer to ping calls. Endpoint is named oslo_rpc_server_ping

`log_options`

Type
boolean

Default
True

Mutable
This option can be changed without restarting.

Enables or disables logging values of all registered options when starting a service (at DEBUG level).

`graceful_shutdown_timeout`

Type
integer

Default
60

Mutable
This option can be changed without restarting.

Specify a timeout after which a gracefully shutdown server will exit. Zero value means endless wait.

agent

`manage_agent_boot`

Type
boolean

Default
True

Whether Ironic will manage booting of the agent ramdisk. If set to False, you will need to configure your mechanism to allow booting the agent ramdisk. Deprecated for removal in 2025.2 release.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

`memory_consumed_by_agent`

Type
integer

Default
0

Mutable
This option can be changed without restarting.

The memory size in MiB consumed by agent when it is booted on a bare metal node. This is used for checking if the image can be downloaded and deployed on the bare metal node after booting agent ramdisk. This may be set according to the memory consumed by the agent ramdisk image.

`stream_raw_images`

Type
boolean

Default
True

Mutable
This option can be changed without restarting.

Whether the agent ramdisk should stream raw images directly onto the disk or not. By streaming raw images directly onto the disk the agent ramdisk will not spend time copying the image to a tmpfs partition (therefore consuming less memory) prior to writing it to the disk. Unless the disk where the image will be copied to is really slow, this option should be set to True. Defaults to True.

`post_deploy_get_power_state_retries`

Type
integer

Default
6

Number of times to retry getting power state to check if bare metal node has been powered off after a soft power off.

`post_deploy_get_power_state_retry_interval`

Type
integer

Default
5

Amount of time (in seconds) to wait between polling power state after trigger soft poweroff.

`agent_api_version`

Type
string

Default
v1

API version to use for communicating with the ramdisk agent.

`deploy_logs_collect`

Type
string

Default
on_failure

Valid Values
always, on_failure, never

Mutable
This option can be changed without restarting.

Whether Ironic should collect the deployment logs on deployment failure (on_failure), always or never.

Possible values

always
always collect the logs

on_failure
only collect logs if there is a failure

never
never collect logs

`deploy_logs_storage_backend`

Type
string

Default
`local`

Valid Values
local, swift

Mutable
This option can be changed without restarting.

The name of the storage backend where the logs will be stored.

Possible values

local
store the logs locally

swift
store the logs in Object Storage service

`deploy_logs_local_path`

Type
string

Default
`/var/log/ironic/deploy`

Mutable
This option can be changed without restarting.

The path to the directory where the logs should be stored, used when the deploy_logs_storage_backend is configured to local.

`deploy_logs_swift_container`

Type
string

Default
`ironic_deploy_logs_container`

Mutable
This option can be changed without restarting.

The name of the Swift container to store the logs, used when the deploy_logs_storage_backend is configured to swift.

`deploy_logs_swift_days_to_expire`

Type
integer

Default
`30`

Mutable
This option can be changed without restarting.

Number of days before a log object is marked as expired in Swift. If None, the logs will be kept forever or until manually deleted. Used when the deploy_logs_storage_backend is configured to swift.

`image_download_source`

Type
string

Default
`http`

Valid Values
swift, http, local

Mutable
This option can be changed without restarting.

Specifies whether direct deploy interface should try to use the image source directly or if ironic should cache the image on the conductor and serve it from ironics own http server.

Possible values

swift
IPA ramdisk retrieves instance image from the Object Storage service.

http
IPA ramdisk retrieves instance image from HTTP service served at conductor nodes.

local
Same as http, but HTTP images are also cached locally, converted and served from the conductor

`command_timeout`

Type
integer

Default
`60`

Mutable
This option can be changed without restarting.

Timeout (in seconds) for IPA commands. A large timeout value may result in the conductor free worker pool becoming exhausted should a multi-node network connectivity issue arise during in-band operations. These commands also cause the individual node lock to be held while in progress, which prevents new requests from being acted upon for the impacted nodes until the issue has been resolved.

`max_command_attempts`

Type
integer

Default
3

This is the maximum number of attempts that will be done for IPA commands that fails due to network problems.

`command_wait_attempts`

Type
integer

Default
100

Number of attempts to check for asynchronous commands completion before timing out.

`command_wait_interval`

Type
integer

Default
6

Number of seconds to wait for between checks for asynchronous commands completion.

`neutron_agent_poll_interval`

Type
integer

Default
2

Mutable
This option can be changed without restarting.

The number of seconds Neutron agent will wait between polling for device changes. This value should be the same as CONF.AGENT.polling_interval in Neutron configuration.

`neutron_agent_max_attempts`

Type
integer

Default
100

Max number of attempts to validate a Neutron agent status before raising network error for a dead agent.

`neutron_agent_status_retry_interval`

Type
integer

Default
10

Wait time in seconds between attempts for validating Neutron agent status.

`require_tls`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

If set to False, callback URLs without https:// will be permitted by the conductor, which may be needed for backwards compatibility outside of the supported version window.

`certificates_path`

Type
string

Default
`/var/lib/ironic/certificates`

Path to store auto-generated TLS certificates used to validate connections to the ramdisk.

`verify_ca`

Type
string

Default
`True`

Path to the TLS CA to validate connection to the ramdisk. Set to True to use the system default CA storage. Set to False to disable validation. Ignored when automatic TLS setup is used.

`api_ca_file`

Type
string

Default
<None>

Path to the TLS CA that is used to start the bare metal API. In some boot methods this file can be passed to the ramdisk.

`allow_md5_checksum`

Type
boolean

Default
`True`

When enabled, the agent will be notified it is permitted to consider MD5 checksums. This option is expected to change to a default of False in a 2024 release of Ironic.

agent_containers

allow_arbitrary_containers

Type
boolean

Default
`False`

Defines whether arbitrary containers are allowed for use in the IPA ramdisk. If set to False, only-containers in the allowed_containers list can be used.

allowed_containers

Type
list

Default
`[]`

List of allowed container images. Only used when allow_arbitrary_containers is set to False.Containers not in this list will be rejected.

container_steps_file

Type
string

Default
`/etc/ironic-python-agent.d/mysteps.yaml`

Path in the ramdisk to the YAML file containingcontainer steps to be executed.

runner

Type
string

Default
`podman`

Container runtime to use, such aspodman and docker.

pull_options

Type
string

Default
`--tls-verify=false`

Options to pass when pulling container images(e.g., tls-verify=false).

run_options

Type
string

Default
`--rm --network=host --tls-verify=false`

Options to pass when running containers(e.g., rm network=host).

`container_conf_file`

Type
string

Default
`/etc/containers/containers.conf`

Path to the container configuration filein the IPA ramdisk.

anaconda

`default_ks_template`

Type
string

Default
`$pybasedir/drivers/modules/ks.cfg.template`

Mutable
This option can be changed without restarting.

kickstart template to use when no kickstart template is specified in the instance_info or the glance OS image.

`insecure_heartbeat`

Type
boolean

Default
`False`

Mutable
This option can be changed without restarting.

Option to allow the kickstart configuration to be informed if SSL/TLS certificate verification should be enforced, or not. This option exists largely to facilitate easy testing and use of the `anaconda` deployment interface. When this option is set, heartbeat operations, depending on the contents of the utilized kickstart template, may not enforce TLS certificate verification.

ansible

`ansible_extra_args`

Type
string

Default
<None>

Extra arguments to pass on every invocation of Ansible.

`verbosity`

Type
integer

Default
<None>

Minimum Value
0

Maximum Value
4

Set ansible verbosity level requested when invoking ansible-playbook command. 4 includes detailed SSH session logging. Default is 4 when global debug is enabled and 0 otherwise.

`ansible_playbook_script`

Type
string

Default
`ansible-playbook`

Path to ansible-playbook script. Default will search the $PATH configured for user running ironic-conductor process. Provide the full path when ansible-playbook is not in $PATH or installed in not default location.

`playbooks_path`

Type
string

Default
`$pybasedir/drivers/modules/ansible/playbooks`

Path to directory with playbooks, roles and local inventory.

`config_file_path`

Type
string

Default
`$pybasedir/drivers/modules/ansible/playbooks/ansible.cfg`

Path to ansible configuration file. If set to empty, system default will be used.

`post_deploy_get_power_state_retries`

Type
integer

Default
6

Minimum Value
0

Number of times to retry getting power state to check if bare metal node has been powered off after a soft power off. Value of 0 means do not retry on failure.

`post_deploy_get_power_state_retry_interval`

Type
integer

Default
5

Minimum Value
0

Amount of time (in seconds) to wait between polling power state after trigger soft poweroff.

`extra_memory`

Type
integer

Default
10

Extra amount of memory in MiB expected to be consumed by Ansible-related processes on the node. Affects decision whether image will fit into RAM.

`image_store_insecure`

Type
boolean

Default
False

Skip verifying SSL connections to the image store when downloading the image. Setting it to True is only recommended for testing environments that use self-signed certificates.

`image_store_cafile`

Type
string

Default
<None>

Specific CA bundle to use for validating SSL connections to the image store. If not specified, CA available in the ramdisk will be used. Is not used by default playbooks included with the driver. Suitable for environments that use self-signed certificates.

`image_store_certfile`

Type
string

Default
<None>

Client cert to use for SSL connections to image store. Is not used by default playbooks included with the driver.

`image_store_keyfile`

Type
string

Default
<None>

Client key to use for SSL connections to image store. Is not used by default playbooks included with the driver.

`default_username`

Type
string

Default
`ansible`

Name of the user to use for Ansible when connecting to the ramdisk over SSH. It may be overridden by per-node ansible_username option in nodes driver_info field.

`default_key_file`

Type
string

Default
<None>

Absolute path to the private SSH key file to use by Ansible by default when connecting to the ramdisk over SSH. Default is to use default SSH keys configured for the user running the ironic-conductor service. Private keys with password must be pre-loaded into ssh-agent. It may be overridden by per-node ansible_key_file option in nodes driver_info field.

`default_deploy_playbook`

Type
string

Default
`deploy.yaml`

Path (relative to $playbooks_path or absolute) to the default playbook used for deployment. It may be overridden by per-node ansible_deploy_playbook option in nodes driver_info field.

`default_shutdown_playbook`

Type
string

Default
`shutdown.yaml`

Path (relative to $playbooks_path or absolute) to the default playbook used for graceful in-band shutdown of the node. It may be overridden by per-node ansible_shutdown_playbook option in nodes driver_info field.

`default_clean_playbook`

Type
string

Default
`clean.yaml`

Path (relative to $playbooks_path or absolute) to the default playbook used for node cleaning. It may be overridden by per-node ansible_clean_playbook option in nodes driver_info field.

`default_clean_steps_config`

Type
string

Default
`clean_steps.yaml`

Path (relative to $playbooks_path or absolute) to the default auxiliary cleaning steps file used during the node cleaning. It may be overridden by per-node ansible_clean_steps_config option in nodes driver_info field.

`default_python_interpreter`

Type
string

Default
<None>

Absolute path to the python interpreter on the managed machines. It may be overridden by per-node ansible_python_interpreter option in nodes driver_info field. By default, ansible uses /usr/bin/python

api

`host_ip`

Type
host address

Default
`0.0.0.0`

The IP address or hostname on which ironic-api listens.

`port`

Type
port number

Default
`6385`

Minimum Value
0

Maximum Value
65535

The TCP port on which ironic-api listens.

`unix_socket`

Type
string

Default
<None>

Unix socket to listen on. Disables host_ip and port.

`unix_socket_mode`

Type
unknown type

Default
<None>

File mode (an octal number) of the unix socket to listen on. Ignored if unix_socket is not set.

`max_limit`

Type
integer

Default
1000

Mutable
This option can be changed without restarting.

The maximum number of items returned in a single response from a collection resource.

`public_endpoint`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

Public URL to use when building the links to the API resources (for example, https://ironic.rocks:6384). If None the links will be built using the requests host URL. If the API is operating behind a proxy, you will want to change this to represent the proxys URL. Defaults to None. Ignored when proxy headers parsing is enabled via [oslo_middleware]enable_proxy_headers_parsing option.

`api_workers`

Type
integer

Default
<None>

Number of workers for OpenStack Ironic API service. The default is equal to the number of CPUs available, but not more than 4. One worker is used if the CPU number cannot be detected.

`enable_ssl_api`

Type
boolean

Default
`False`

Enable the integrated stand-alone API to service requests via HTTPS instead of HTTP. If there is a front-end service performing HTTPS offloading from the service, this option should be False; note, you will want to enable proxy headers parsing with [oslo_middleware]enable_proxy_headers_parsing option or configure [api]public_endpoint option to set URLs in responses to the SSL terminated one.

`restrict_lookup`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

Whether to restrict the lookup API to only nodes in certain states. Setting this to False can be insecure and is not advisable.

`ramdisk_heartbeat_timeout`

Type
integer

Default
`300`

Mutable
This option can be changed without restarting.

Maximum interval (in seconds) for agent heartbeats.

`network_data_schema`

Type
string

Default
`$pybasedir/api/controllers/v1/network-data-schema.json`

Schema for network data used by this deployment.

`project_admin_can_manage_own_nodes`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

If a project scoped administrative user is permitted to create/delete baremetal nodes in their project.

`disallowed_enrollment_boot_modes`

Type
list

Default
`[]`

Mutable
This option can be changed without restarting.

Specifies a list of boot modes that are not allowed during enrollment. Eg: [bios]

`cert_file`

Type
string

Default
<None>

Certificate file to use when starting the server securely.

`key_file`

Type
string

Default
<None>

Private key file to use when starting the server securely.

`middleware`

Type
list

Default
`[]`

Comma-separated list of WSGI middleware to load from the ironic.api.middleware entry point namespace. Middleware are applied in the order specified, wrapping the API application. This allows operators to add custom request processing (validation, logging, rate limiting, etc.) without modifying Ironic code. External packages can register middleware by adding entry points to the ironic.api.middleware namespace. Each middleware must be a callable that accepts a WSGI application and returns a wrapped application.

audit

`enabled`

Type
boolean

Default
`False`

Enable auditing of API requests (for ironic-api service).

`audit_map_file`

Type
string

Default
`/etc/ironic/api_audit_map.conf`

Path to audit map file for ironic-api service. Used only when API audit is enabled.

`ignore_req_list`

Type
string

Default
`''`

Comma separated list of Ironic REST API HTTP methods to be ignored during audit logging. For example: auditing will not be done on any GET or POST requests if this is set to GET,POST. It is used only when API audit is enabled.

audit_middleware_notifications

`use_oslo_messaging`

Type
boolean

Default
`True`

Indicate whether to use oslo_messaging as the notifier. If set to False, the local logger will be used as the notifier. If set to True, the oslo_messaging package must also be present. Otherwise, the local will be used instead.

`driver`

Type
string

Default
`<None>`

The Driver to handle sending notifications. Possible values are messaging, messagingv2, routing, log, test, noop. If not specified, then value from oslo_messaging_notifications conf section is used.

`topics`

Type
list

Default
`<None>`

List of AMQP topics used for OpenStack notifications. If not specified, then value from oslo_messaging_notifications conf section is used.

`transport_url`

Type
string

Default
<None>

A URL representing messaging driver to use for notification. If not specified, we fall back to the same configuration used for RPC.

auto_discovery

`enabled`

Type
boolean

Default
`False`

Mutable
This option can be changed without restarting.

Setting this to True enables automatic enrollment of inspected nodes that are not recognized. When enabling this feature, keep in mind that any machine hitting the inspection callback endpoint will be automatically enrolled. The driver must be set when setting this to True.

`driver`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

The default driver to use for newly enrolled nodes. Must be set when enabling auto-discovery.

`inspection_scope`

Type
string

Default
<None>

The default inspection scope for nodes enrolled via auto-discovery.

cifs

`base_url`

Type
string

Default
<None>

Base URL for CIFS/SMB shares (e.g., cifs://server/share or smb://server/share). Required for CIFS virtual media support. Must be used together with share_path.

`share_path`

Type
string

Default
<None>

Local mount point of the CIFS share where images will be placed. Required for CIFS virtual media support. Must be used together with base_url.

`username`

Type
string

Default
<None>

Username for CIFS authentication. If set, credentials will be passed to the BMC when inserting virtual media.

`password`

Type
string

Default
<None>

Password for CIFS authentication. If set, credentials will be passed to the BMC when inserting virtual media.

cinder

`action_retries`

Type
integer

Default
3

Number of retries in the case of a failed action (currently only used when detaching volumes).

`action_retry_interval`

Type
integer

Default
5

Retry interval in seconds in the case of a failed action (only specific actions are retried).

`auth_url`

Type
unknown type

Default
<None>

Authentication URL

`auth_type`

Type
unknown type

Default
<None>

Authentication type to load

Table 24: Deprecated Variations

Group	Name
cinder	auth_plugin

`cafile`

Type
string

Default
<None>

PEM encoded Certificate Authority to use when verifying HTTPs connections.

`certfile`

Type
string

Default
<None>

PEM encoded client certificate cert file

`collect_timing`

Type
boolean

Default
`False`

Collect per-API call timing information.

`connect_retries`

Type
integer

Default
<None>

The maximum number of retries that should be attempted for connection errors.

`connect_retry_delay`

Type
floating point

Default
<None>

Delay (in seconds) between two retries for connection errors. If not set, exponential retry starting with 0.5 seconds up to a maximum of 60 seconds is used.

`default_domain_id`

Type
unknown type

Default
<None>

Optional domain ID to use with v3 and v2 parameters. It will be used for both the user and project domain in v3 and ignored in v2 authentication.

`default_domain_name`

Type
unknown type

Default
<None>

Optional domain name to use with v3 API and v2 parameters. It will be used for both the user and project domain in v3 and ignored in v2 authentication.

`domain_id`

Type
unknown type

Default
<None>

Domain ID to scope to

`domain_name`

Type
unknown type

Default
<None>

Domain name to scope to

`endpoint_override`

Type
string

Default
<None>

Always use this endpoint URL for requests for this client. NOTE: The unversioned endpoint should be specified here; to request a particular API version, use the *version*, *min-version*, and/or *max-version* options.

`insecure`

Type
boolean

Default
`False`

Verify HTTPS connections.

`keyfile`

Type
string

Default
<None>

PEM encoded client certificate key file

`max_version`

Type
string

Default
<None>

The maximum major version of a given API, intended to be used as the upper bound of a range with min_version. Mutually exclusive with version.

`min_version`

Type
string

Default
<None>

The minimum major version of a given API, intended to be used as the lower bound of a range with max_version. Mutually exclusive with version. If min_version is given with no max_version it is as if max version is latest.

`password`

Type
unknown type

Default
<None>

Users password

`project_domain_id`

Type
unknown type

Default
<None>

Domain ID containing project

`project_domain_name`

Type
unknown type

Default
<None>

Domain name containing project

`project_id`

Type
unknown type

Default
<None>

Project ID to scope to

Table 25: Deprecated Variations

Group	Name
cinder	tenant-id
cinder	tenant_id

`project_name`

Type
unknown type

Default
<None>

Project name to scope to

Table 26: Deprecated Variations

Group	Name
cinder	tenant-name
cinder	tenant_name

`region_name`

Type
string

Default
<None>

The default region_name for endpoint URL discovery.

`retriable_status_codes`

Type
list

Default
<None>

List of retriable HTTP status codes that should be retried. If not set default to [503]

`retries`

Type
integer

Default
3

DEPRECATED: Client retries in the case of a failed request.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
Replaced by status_code_retries and status_code_retry_delay.

`service_name`

Type
string

Default
<None>

The default service_name for endpoint URL discovery.

`service_type`

Type
string

Default
volumev3

The default service_type for endpoint URL discovery.

`split_loggers`

Type
boolean

Default
False

Log requests to multiple loggers.

`status_code_retries`

Type
integer

Default
<None>

The maximum number of retries that should be attempted for retriable HTTP status codes.

`status_code_retry_delay`

Type
floating point

Default
<None>

Delay (in seconds) between two retries for retriable status codes. If not set, exponential retry starting with 0.5 seconds up to a maximum of 60 seconds is used.

`system_scope`

Type
unknown type

Default
<None>

Scope for system operations

`tenant_id`

Type
unknown type

Default
<None>

Tenant ID

`tenant_name`

Type
unknown type

Default
<None>

Tenant Name

`timeout`

Type
integer

Default
<None>

Timeout value for http requests

`trust_id`

Type
unknown type

Default
<None>

ID of the trust to use as a trustee use

`user_domain_id`

Type
unknown type

Default
<None>

Users domain id

`user_domain_name`

Type
unknown type

Default
<None>

Users domain name

`user_id`

Type
unknown type

Default
<None>

User id

`username`

Type
unknown type

Default
<None>

Username

Table 27: Deprecated Variations

Group	Name
cinder	user-name
cinder	user_name

`valid_interfaces`

Type
list

Default
`['internal', 'public']`

List of interfaces, in order of preference, for endpoint URL.

`version`

Type
string

Default
<None>

Minimum Major API version within a given Major API version for endpoint URL discovery. Mutually exclusive with min_version and max_version

conductor

`workers_pool_size`

Type
integer

Default
`300`

Minimum Value
3

The size of the workers thread pool. Note that 2 threads will be reserved by the conductor itself for handling heart beats and periodic tasks. On top of that, *sync_power_state_workers* will take up to 7 threads with the default value of 8.

`reserved_workers_pool_percentage`

Type
integer

Default
`5`

Minimum Value
0

Maximum Value
50

The percentage of the whole workers pool that will be kept for API requests and other important tasks. This part of the pool will not be used for periodic tasks or agent heartbeats. Set to 0 to disable.

`heartbeat_interval`

Type
integer

Default
`10`

Seconds between conductor heart beats.

`heartbeat_timeout`

Type
integer

Default
60

Maximum Value
315576000

Mutable
This option can be changed without restarting.

Maximum time (in seconds) since the last check-in of a conductor. A conductor is considered inactive when this time has been exceeded.

`sync_power_state_interval`

Type
integer

Default
120

Interval between syncing the node power state to the database, in seconds. Set to 0 to disable syncing.

`check_provision_state_interval`

Type
integer

Default
60

Minimum Value
0

Interval between checks of provision timeouts, in seconds. Set to 0 to disable checks.

`check_rescue_state_interval`

Type
integer

Default
60

Minimum Value
1

Interval (seconds) between checks of rescue timeouts.

`check_allocations_interval`

Type
integer

Default
60

Minimum Value
0

Interval between checks of orphaned allocations, in seconds. Set to 0 to disable checks.

`cache_clean_up_interval`

Type
integer

Default
`3600`

Minimum Value
0

Interval between cleaning up image caches, in seconds. Set to 0 to disable periodic clean-up.

`clear_image_cache_on_deploy_failure`

Type
boolean

Default
`False`

Mutable
This option can be changed without restarting.

Whether to clear cached instance images when deployment fails or aborted. When enabled, cached images are removed during deployment failure state transitions. When disabled (default), cached images are preserved for retry attempts and will be cleaned up eventually via periodic cache cleanup based on the defined TTL.

`deploy_callback_timeout`

Type
integer

Default
`1800`

Minimum Value
0

Timeout (seconds) to wait for a callback from a deploy ramdisk. Set to 0 to disable timeout.

`force_power_state_during_sync`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

During sync_power_state, should the hardware power state be set to the state recorded in the database (True) or should the database be updated based on the hardware state (False).

`power_state_sync_max_retries`

Type
integer

Default
3

During sync_power_state failures, limit the number of times Ironic should try syncing the hardware node power state with the node power state in DB

`sync_power_state_workers`

Type
integer

Default
8

Minimum Value
1

The maximum number of worker threads that can be started simultaneously to sync nodes power states from the periodic task.

`periodic_max_workers`

Type
integer

Default
8

Maximum number of worker threads that can be started simultaneously by a periodic task. Should be less than RPC thread pool size.

`node_locked_retry_attempts`

Type
integer

Default
3

Number of attempts to grab a node lock.

`node_locked_retry_interval`

Type
integer

Default
1

Seconds to sleep between node lock attempts.

`sync_local_state_interval`

Type
integer

Default
180

When conductors join or leave the cluster, existing conductors may need to update any persistent local state as nodes are moved around the cluster. This option controls how often, in seconds, each conductor will check for nodes that it should take over. Set it to 0 (or a negative value) to disable the check entirely.

`configdrive_swift_container`

Type
string

Default
ironic_configdrive_container

Name of the Swift container to store config drive data. Used when configdrive_use_object_store is True.

`configdrive_swift_temp_url_duration`

Type
integer

Default
<None>

Minimum Value
60

The timeout (in seconds) after which a configdrive temporary URL becomes invalid. Defaults to deploy_callback_timeout + 120 seconds if it is set, otherwise to 1800 seconds. Used when configdrive_use_object_store is True.

`inspect_wait_timeout`

Type
integer

Default
1800

Minimum Value
0

Timeout (seconds) for waiting for node inspection. 0 - unlimited.

`automated_clean`

Type
boolean

Default
True

Mutable
This option can be changed without restarting.

Enables or disables automated cleaning. Automated cleaning is a configurable set of steps, such as erasing disk drives, that are performed on the node to ensure it is in a baseline state and ready to be

deployed to. This is done after instance deletion as well as during the transition from a manageable to available state. When enabled, the particular steps performed to clean a node depend on which driver that node is managed by; see the individual drivers documentation for details.

`automated_cleaning_step_source`

Type
string

Default
`autogenerated`

Valid Values
autogenerated, runbook, hybrid

Mutable
This option can be changed without restarting.

Determines how automated_cleaning is performed; the default, autogenerated collects steps from hardware interfaces, then ordering by priority; runbook requires a runbook to be specified in config or driver_info, which is then used to clean thenode; hybrid uses a runbook if available, and falls-back to autogenerated cleaning steps if not.

Possible values

autogenerated
Collects steps from hardware interfaces and orders by priority. This provides the original Ironic cleaning behavior originally implemented in Kilo.

runbook
Runs cleaning via a runbook specified in configuration or node driver_info. If a runbook is not specified while automated_clean is enabled, cleaning will fail.

hybrid
Runs cleaning via a runbook if one is specified in configuration or node driver_info. If a runbook is not specified while automated_clean is enabled, Ironic will fallback to autogenerated cleaning steps.

`automated_cleaning_runbook`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

If set and [conductor]/automated_clean_step_source is set to hybrid or runbook, the runbook UUID or name provided here will be used during automated_cleaning for nodes which do not have a resource_class-specific runbook or runbook set in driver_info.

`automated_cleaning_runbook_by_resource_class`

Type
dict

Default
{}

Mutable
This option can be changed without restarting.

A dictionary of key-value pairs of node resource_class and runbook UUID or name which will be used to clean the node if [conductor]automated_clean_step_source is set to hybrid or runbook and a more specific runbook has not been configured in driver_info.

`automated_cleaning_runbook_from_node`

Type
boolean

Default
False

Mutable
This option can be changed without restarting.

When enabled, allows an administrator to configure a runbook in node[driver_info][cleaning_runbook] to use for that node when [conductor]automated_clean_step_source is set to hybrid or runbook. NOTE: This will permit any user with access to edit node[driver_info] to circumvent cleaning.

`automated_cleaning_runbook_validate_traits`

Type
boolean

Default
True

Mutable
This option can be changed without restarting.

When enabled, this option requires validation of a runbook before its used for automated cleaning. Nodes configured with a runbook that is not validated for use via trait matching will fail to clean.

`allow_provisioning_in_maintenance`

Type
boolean

Default
True

Mutable
This option can be changed without restarting.

Whether to allow nodes to enter or undergo deploy or cleaning when in maintenance mode. If this option is set to False, and a node enters maintenance during deploy or cleaning, the process will be aborted after the next heartbeat. Automated cleaning or making a node available will also fail. If True (the default), the process will begin and will pause after the node starts heartbeating. Moving it from maintenance will make the process continue.

`clean_callback_timeout`

Type
integer

Default
1800

Minimum Value
0

Timeout (seconds) to wait for a callback from the ramdisk doing the cleaning. If the timeout is reached the node will be put in the clean failed provision state. Set to 0 to disable timeout.

`service_callback_timeout`

Type
integer

Default
1800

Minimum Value
0

Timeout (seconds) to wait for a callback from the ramdisk doing the servicing. If the timeout is reached the node will be put in the service failed provision state. Set to 0 to disable timeout.

`rescue_callback_timeout`

Type
integer

Default
1800

Minimum Value
0

Timeout (seconds) to wait for a callback from the rescue ramdisk. If the timeout is reached the node will be put in the rescue failed provision state. Set to 0 to disable timeout.

`soft_power_off_timeout`

Type
integer

Default
600

Minimum Value
1

Mutable
This option can be changed without restarting.

Timeout (in seconds) of soft reboot and soft power off operation. This value always has to be positive.

`power_state_change_timeout`

Type
integer

Default
60

Minimum Value
2

Mutable
This option can be changed without restarting.

Number of seconds to wait for power operations to complete, i.e., so that a baremetal node is in the desired power state. If timed out, the power operation is considered a failure.

`power_failure_recovery_interval`

Type
integer

Default
300

Minimum Value
0

Interval (in seconds) between checking the power state for nodes previously put into maintenance mode due to power synchronization failure. A node is automatically moved out of maintenance mode once its power state is retrieved successfully. Set to 0 to disable this check.

`conductor_group`

Type
string

Default
''

Name of the conductor group to join. Can be up to 255 characters and is case insensitive. This conductor will only manage nodes with a matching conductor_group field set on the node.

`allow_deleting_available_nodes`

Type
boolean

Default
True

Mutable
This option can be changed without restarting.

Allow deleting nodes which are in state available.

`enable_mdns`

Type
boolean

Default
False

Whether to enable publishing the baremetal API endpoint via multicast DNS.

`deploy_kernel`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

Glance ID, http:// or file:// URL of the kernel of the default deploy image.

`deploy_ramdisk`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

Glance ID, http:// or file:// URL of the initramfs of the default deploy image.

`deploy_kernel_by_arch`

Type
dict

Default
{}

Mutable
This option can be changed without restarting.

A dictionary of key-value pairs of each architecture with the Glance ID, http:// or file:// URL of the kernel of the default deploy image.

`deploy_ramdisk_by_arch`

Type
dict

Default
{}

Mutable
This option can be changed without restarting.

A dictionary of key-value pairs of each architecture with the Glance ID, http:// or file:// URL of the initramfs of the default deploy image.

`rescue_kernel`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

Glance ID, http:// or file:// URL of the kernel of the default rescue image.

`rescue_ramdisk`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

Glance ID, http:// or file:// URL of the initramfs of the default rescue image.

`rescue_kernel_by_arch`

Type
dict

Default
{}

Mutable
This option can be changed without restarting.

A dictionary of key-value pairs of each architecture with the Glance ID, http:// or file:// URL of the kernel of the default rescue image.

`rescue_ramdisk_by_arch`

Type
dict

Default
{}

Mutable
This option can be changed without restarting.

A dictionary of key-value pairs of each architecture with the Glance ID, http:// or file:// URL of the initramfs of the default rescue image.

`rescue_password_hash_algorithm`

Type
string

Default
sha256

Valid Values
sha256, sha512

Mutable
This option can be changed without restarting.

Password hash algorithm to be used for the rescue password.

`require_rescue_password_hashed`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

Option to cause the conductor to not fallback to an un-hashed version of the rescue password, permitting rescue with older ironic-python-agent ramdisks.

`bootloader`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

Glance ID, http:// or file:// URL of the EFI system partition image containing EFI boot loader. This image will be used by ironic when building UEFI-bootable ISO out of kernel and ramdisk. Required for UEFI boot from partition images. Can be overridden per-architecture using the bootloader_by_arch option.

`clean_step_priority_override`

Type
unknown type

Default
`{}`

Priority to run automated clean steps for both in-band and out of band clean steps, provided in interface.step_name:priority format, e.g. deploy.erase_devices_metadata:123. The option can be specified multiple times to define priorities for multiple steps. If set to 0, this specific step will not run during cleaning. If unset for an inband clean step, will use the priority set in the ramdisk.

`node_history`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

Boolean value, default True, if node event history is to be recorded. Errors and other noteworthy events in relation to a node are journaled to a database table which incurs some additional load. A periodic task does periodically remove entries from the database. Please note, if this is disabled, the conductor will continue to purge entries as long as [conductor]node_history_cleanup_batch_count is not 0.

`node_history_max_entries`

Type
integer

Default
`300`

Minimum Value
0

Mutable
This option can be changed without restarting.

Maximum number of history entries which will be stored in the database per node. This setting excludes the minimum number of days retained using the [conductor]node_history_minimum_days setting.

`node_history_cleanup_interval`

Type
integer

Default
`86400`

Minimum Value
0

Interval in seconds at which node history entries can be cleaned up in the database. Setting to 0 disables the periodic task.

`node_history_cleanup_batch_count`

Type
integer

Default
`1000`

Minimum Value
0

The target number of node history records to purge from the database when performing clean-up. Deletes are performed by node, and a node with excess records for a node will still be deleted. Operators who find node history building up may wish to lower this threshold and decrease the time between cleanup operations using the `node_history_cleanup_interval` setting.

`node_history_minimum_days`

Type
integer

Default
`0`

Minimum Value
0

Mutable
This option can be changed without restarting.

The minimum number of days to explicitly keep on hand in the database history entries for nodes. This is exclusive from the [conductor]node_history_max_entries setting as users of this setting are anticipated to need to retain history by policy.

`conductor_cleanup_interval`

Type
integer

Default
`86400`

Minimum Value
0

Interval in seconds at which stale conductor entries can be cleaned up from the database. Setting to 0 disables the periodic task.

`conductor_cleanup_timeout`

Type
integer

Default
`1209600`

Minimum Value
60

Mutable
This option can be changed without restarting.

Timeout in seconds after which offline conductors are considered stale and can be cleaned up from the database. THis is always required to be at least 3x larger than [conductor]heartbeat_timeout since if otherwise, active conductors might be mistakenly removed from the database.

`conductor_cleanup_batch_size`

Type
integer

Default
`50`

Minimum Value
1

Mutable
This option can be changed without restarting.

The maximum number of stale conductor records to clean up from the database in a single cleanup operation.

`verify_step_priority_override`

Type
unknown type

Default
`{}`

Mutable
This option can be changed without restarting.

Priority to run automated verify steps provided in interface.step_name:priority format,e.g. management.clear_job_queue:123. The option can be specified multiple times to define priorities for multiple steps. If set to 0, this specific step will not run during verification.

`automatic_lessee`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

Deprecated. If Ironic should set the node.lessee field at deployment. Use [conductor]/automatic_lessee_source instead.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

`automatic_lessee_source`

Type
string

Default
`instance`

Valid Values
instance, request, none

Mutable
This option can be changed without restarting.

Source for Project ID the Ironic should record at deployment time in node.lessee field. If set to none, Ironic will not set a lessee field. If set to instance (default), uses Project ID indicated in instance metadata set by Nova or another external deployment service. If set to keystone, Ironic uses Project ID indicated by Keystone context.

Possible values

instance
Populates node.lessee field using metadata from node.instance_info[project_id] at deployment time. Useful for Nova-fronted deployments.

request
Populates node.lessee field using metadata from request context. Only useful for direct deployment requests to Ironic; not those proxied via an external service like Nova.

none
Ironic will not populate the node.lessee field.

`max_concurrent_deploy`

Type
integer

Default
`250`

Minimum Value
1

Mutable
This option can be changed without restarting.

The maximum number of concurrent nodes in deployment which are permitted in this Ironic system. If this limit is reached, new requests will be rejected until the number of deployments in progress is lower than this maximum. As this is a security mechanism requests are not queued, and this setting is a global setting applying to all requests this conductor receives, regardless of access rights. The concurrent deployment limit cannot be disabled.

`max_concurrent_clean`

Type
integer

Default
`50`

Minimum Value
1

Mutable
This option can be changed without restarting.

The maximum number of concurrent nodes in cleaning which are permitted in this Ironic system. If this limit is reached, new requests will be rejected until the number of nodes in cleaning is lower than this maximum. As this is a security mechanism requests are not queued, and this setting is a global setting applying to all requests this conductor receives, regardless of access rights. The concurrent clean limit cannot be disabled.

`max_concurrent_per_conductor_group`

Type
boolean

Default
`False`

Mutable
This option can be changed without restarting.

When set to True, the max_concurrent_deploy and max_concurrent_clean limits are applied per conductor group instead of globally. This allows different conductor groups to have their own independent concurrency limits. When False (the default), the limits apply to all nodes across the entire Ironic deployment.

`poweroff_in_cleanfail`

Type
boolean

Default
`False`

If True power off nodes in the `clean failed` state. Default False. Option may be unsafe when using Cleaning to perform hardware-transformative actions such as firmware upgrade.

`poweroff_in_servicefail`

Type
boolean

Default
`False`

If True power off nodes in the `service failed` state. Default False. Option may be unsafe when using service to perform hardware-transformative actions such as firmware upgrade.

`permit_child_node_step_async_result`

Type
boolean

Default
`False`

Mutable
This option can be changed without restarting.

This option allows child node steps to not error if the resulting step execution returned a wait state. Under normal conditions, child nodes are not expected to request a wait state. This option exists for operators to use if needed to perform specific tasks where this is known acceptable. Use at yourown risk!

`max_conductor_wait_step_seconds`

Type
integer

Default
`30`

Minimum Value
0

Maximum Value
1800

Mutable
This option can be changed without restarting.

The maximum number of seconds which a step can be requested to explicitly sleep or wait. This value should be changed sparingly as it holds a conductor thread and if used across many nodes at once can exhaust a conductors resources. Thiscapability has a hard coded maximum wait of 1800 seconds, or 30 minutes. If you need to wait longer than the maximum value, we recommend exploring hold steps.

`disallowed_deployment_boot_modes`

Type
list

Default
`[]`

Mutable
This option can be changed without restarting.

Specifies a list of boot modes that are not allowed during deployment. Eg: [bios]

`disable_deep_image_inspection`

Type
boolean

Default
`False`

Security Option to permit an operator to disable file content inspections. Under normal conditions, the conductor will inspect requested image contents which are transferred through the conductor. Disabling this option is not advisable and opens the risk of unsafe images being processed which may allow an attacker to leverage unsafe features in various disk image formats to perform a variety of unsafe and potentially compromising actions. This option is *not* mutable, and requires a service restart to change.

`conductor_always_validates_images`

Type
boolean

Default
`False`

Security Option to enable the conductor to *always* inspect the image content of any requested deploy, even if the deployment would have normally bypassed the conductors cache. When this is set to False, the Ironic-Python-Agent is responsible for any necessary image checks. Setting this to True will result in a higher utilization of resources (disk space, network traffic) as the conductor will evaluate *all* images. This option is *not* mutable, and requires a service restart to change. This option requires [conductor]disable_deep_image_inspection to be set to False.

`permitted_image_formats`

Type
list

Default
`['raw', 'gpt', 'qcow2', 'iso']`

Mutable
This option can be changed without restarting.

The supported list of image formats which are permitted for deployment with Ironic. If an image format outside of this list is detected, the image validation logic will fail the deployment process.

`disable_file_checksum`

Type
boolean

Default
`False`

Deprecated Security option: In the default case, image files have their checksums verified before undergoing additional conductor side actions such as image conversion. Enabling this option opens the risk of files being replaced at the source without the users knowledge.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

`disable_support_for_checksum_files`

Type
boolean

Default
`False`

Security option: By default Ironic will attempt to retrieve a remote checksum file via HTTP(S) URL in order to validate an image download. This is functionality aligning with ironic-python-agent support for standalone users. Disabling this functionality by setting this option to True will create a more secure environment, however it may break users in an unexpected fashion.

`disable_zstandard_decompression`

Type
boolean

Default
`False`

Option to enable disabling transparent decompression of files which are compressed with Zstandard compression. This option is provided should operators wish to disable this functionality, otherwise it is automatically applied by the conductor should a compressed artifact be detected.

`file_url_allowed_paths`

Type
list

Default
`['/var/lib/ironic', '/shared/html', '/templates', '/opt/cache/files', '/vagrant']`

List of paths that are allowed to be used as file:// URLs. Files in /boot, /dev, /etc, /proc, /sys and othersystem paths are always disallowed for security reasons. Any files in this path readable by ironic may be used as an image source when deploying. Setting this value to (empty) disables file:// URL support. Paths listed here are validated as absolute paths and will be rejectedif they contain path traversal mechanisms, such as ...

`graceful_shutdown_timeout`

Type
integer

Default
`60`

Specify a timeout after which a gracefully shutdown conductor will exit. Zero value means endless wait.

Table 28: Deprecated Variations

Group	Name
DEFAULT	graceful_shutdown_timeout

bootloader_by_arch

Type
dict

Default
{}

Bootloader ESP image parameter per node architecture. For example: x86_64:bootx64.efi,aarch64:grubaa64.efi. A nodes cpu_arch property is used as the key to get the appropriate bootloader ESP image. If the nodes cpu_arch is not in the dictionary, the [conductor]bootloader value will be used instead.

disable_configdrive_check

Type
boolean

Default
False

Mutable
This option can be changed without restarting.

Option to disable operations which check and potentially fix up configuration drive contents, such as invalid network metadata values. When these issues are detected, and Ironic is able to correct the data, Ironic will do so transparently. Setting this option to True will disable this functionality.

disable_metadata_mtu_check

Type
boolean

Default
False

Mutable
This option can be changed without restarting.

Option to disable consideration of supplied network_data.json link MTU values as basis to regenerate the supplied metadata.

error_on_ramdisk_config_inconsistency

Type
boolean

Default
False

Mutable
This option can be changed without restarting.

Option to determine if Ironic should fail to boot ramdisk in situations where configuration is ambiguous.e.g. if node[driver_info] contains an override for deploy_ramdisk but not deploy_kernel when ambiguous. When set to True, Ironic will raise and fail the provisioning action that required a ramdisk and kernel. When set to False, Ironic will fallback to the next valid, consistent configured ramdisk and kernel for the node.

`record_step_flows_in_history`

Type
boolean

Default
`True`

When True, the conductor writes a Node History entry at the start and end of every cleaning/servicing/deploy-steps flow. Disable this in very high-churn environments to reduce DB load.

`log_step_flows_to_syslog`

Type
boolean

Default
`False`

Log steps at the start/end of cleaning/servicing/deployment to the conductor service log (WARNING for aborted/failure, INFO otherwise.

`enable_trait_based_networking`

Type
boolean

Default
`False`

Enables Trait Based Networking (TBN) feature if True. When enabled, will apply traits defined in the TBN configuration file to networking actions when building instances with relevant traits defined.

`trait_based_networking_config_file`

Type
string

Default
`/etc/ironic/trait_based_networking.yaml`

The location of the configuration file for trait based configuration. Ironic will load this configuration file if TBN is enabled and apply the traits defined when building and attaching networks to node instances. Ironic will also reload this file if it detects the file has changed.

`trait_based_networking_raise_when_no_match`

Type
boolean

Default
`True`

If True: TBN will raise an exception when no traits (and actions) apply to a node when planning its network.

`trait_based_networking_enable_default_trait`

Type
boolean

Default
`True`

Enables the application of a default trait if no other TBN traits apply to a node during network planning. This trait is named CUSTOM_DEFAULT_TBN_TRAIT. If defined in the TBN configuration file supplied to ironic, that trait will be used. Otherwise, a pre-defined default trait will apply which will generate an action to attach the first available port or portgroup. To be clear: If this option is set to True and there are no matching traits, CUSTOM_DEFAULT_TBN_TRAIT is applied. This behavior does not require CUSTOM_DEFAULT_TBN_TRAIT to be added to node traits.

`enable_health_monitoring`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

Enable automatic hardware health monitoring for nodes. When enabled, the conductor will periodically query the management interface of nodes during power state synchronization to retrieve hardware health status from the BMC. Health information is stored in the node.health field and changes are recorded in node history. Drivers that do not support health monitoring are automatically skipped. Note: this adds one additional BMC query per node during each power sync cycle, which may impact performance in large deployments.

console

`terminal_pid_dir`

Type
string

Default
`<None>`

Directory for holding terminal pid files. If not specified, the temporary directory will be used.

`terminal_timeout`

Type
integer

Default
`600`

Minimum Value
0

Timeout (in seconds) for the terminal session to be closed on inactivity. Set to 0 to disable timeout. Used only by Socat console.

`subprocess_checking_interval`

Type
integer

Default
1

Time interval (in seconds) for checking the status of console subprocess.

`subprocess_timeout`

Type
integer

Default
10

Time (in seconds) to wait for the console subprocess to start.

`kill_timeout`

Type
integer

Default
1

Time (in seconds) to wait for the console subprocess to exit before sending SIGKILL signal.

`socat_address`

Type
ip address

Default
$my_ip

IP address of Socat service running on the host of ironic conductor. Used only by Socat console.

`port_range`

Type
list

Default
['10000:20000']

This option has a sample default set, which means that its actual default value may vary from the one documented above.

A range of ports available to be used for the console proxy service running on the host of ironic conductor, in the form of <start>:<stop> or comma-separated ranges like <start>:<stop>,<start>:<stop>. This option is used only by Socat console

cors

allowed_origin

Type
list

Default
<None>

Indicate whether this resource may be shared with the domain received in the requests origin header. Format: <protocol>://<host>[:<port>], no trailing slash. Example: https://horizon.example.com

allow_credentials

Type
boolean

Default
True

Indicate that the actual request can include user credentials

expose_headers

Type
list

Default
[]

Indicate which headers are safe to expose to the API. Defaults to HTTP Simple Headers.

max_age

Type
integer

Default
3600

Maximum cache age of CORS preflight requests.

allow_methods

Type
list

Default
['OPTIONS', 'GET', 'HEAD', 'POST', 'PUT', 'DELETE', 'TRACE', 'PATCH']

Indicate which methods can be used during the actual request.

allow_headers

Type
list

Default
[]

Indicate which header field names may be used during the actual request.

database

sqlite_synchronous

Type
boolean

Default
`True`

If True, SQLite uses synchronous mode.

backend

Type
string

Default
`sqlalchemy`

The back end to use for the database.

connection

Type
string

Default
<None>

The SQLAlchemy connection string to use to connect to the database.

slave_connection

Type
string

Default
<None>

The SQLAlchemy connection string to use to connect to the slave database.

asyncio_connection

Type
string

Default
<None>

The SQLAlchemy asyncio connection string to use to connect to the database.

asyncio_slave_connection

Type
string

Default
<None>

The SQLAlchemy asyncio connection string to use to connect to the slave database.

`synchronous_reader`

Type
boolean

Default
`True`

Whether or not to assume a reader context needs to guarantee it can read data committed by a writer assuming replication lag is present; defaults to True. When False, a reader context works the same as async_reader and will select the slave database if present. When using a galera cluster, this can be set to False only if you set mysql_wsrep_sync_wait to 1 (this will guarantee that the reader will wait until writesets are committed).Note that this may incur a performance degradation within the galera cluster. Note also that this parameter has no effect if you do not set any slave_connection.

`mysql_sql_mode`

Type
string

Default
`TRADITIONAL`

The SQL mode to be used for MySQL sessions. This option, including the default, overrides any server-set SQL mode. To use whatever SQL mode is set by the server configuration, set this to no value. Example: mysql_sql_mode=

`mysql_wsrep_sync_wait`

Type
integer

Default
<None>

For Galera only, configure wsrep_sync_wait causality checks on new connections. Default is None, meaning dont configure any setting.

`connection_recycle_time`

Type
integer

Default
`3600`

Connections which have been present in the connection pool longer than this number of seconds will be replaced with a new one the next time they are checked out from the pool.

`max_pool_size`

Type
integer

Default
`5`

Maximum number of SQL connections to keep open in a pool. Setting a value of 0 indicates no limit.

`max_retries`

Type
integer

Default
`10`

Maximum number of database connection retries during startup. Set to -1 to specify an infinite retry count.

`retry_interval`

Type
integer

Default
`10`

Interval between retries of opening a SQL connection.

`max_overflow`

Type
integer

Default
`50`

If set, use this value for max_overflow with SQLAlchemy.

`connection_debug`

Type
integer

Default
`0`

Minimum Value
0

Maximum Value
100

Verbosity of SQL debugging information: 0=None, 100=Everything.

`connection_trace`

Type
boolean

Default
`False`

Add Python stack traces to SQL as comment strings.

`pool_timeout`

Type
integer

Default
<None>

If set, use this value for pool_timeout with SQLAlchemy.

`use_db_reconnect`

Type
boolean

Default
`False`

Enable the experimental use of database reconnect on connection lost.

`db_retry_interval`

Type
integer

Default
`1`

Seconds between retries of a database transaction.

`db_inc_retry_interval`

Type
boolean

Default
`True`

If True, increases the interval between retries of a database operation up to db_max_retry_interval.

`db_max_retry_interval`

Type
integer

Default
`10`

If db_inc_retry_interval is set, the maximum seconds between retries of a database operation.

`db_max_retries`

Type
integer

Default
`20`

Maximum retries in case of connection error or deadlock error before error is raised. Set to -1 to specify an infinite retry count.

`connection_parameters`

Type
string

Default
''

Optional URL parameters to append onto the connection URL at connect time; specify as param1=value1&param2=value2&

`mysql_engine`

Type
string

Default
InnoDB

MySQL engine to use.

`sqlite_retries`

Type
boolean

Default
True

If SQLite database operation retry logic is enabled or not. Enabled by default.

`sqlite_max_wait_for_retry`

Type
integer

Default
10

Maximum number of seconds to retry SQLite database locks, after which the original exception will be returned to the caller. This does not presently apply to internal node lock release actions and DB actions centered around the completion of tasks.

deploy

`http_url`

Type
URI

Default
<None>

ironic-conductor nodes HTTP server URL. Example: http://192.1.2.3:8080

`http_root`

Type
string

Default
`/httpboot`

ironic-conductor nodes HTTP root path.

`image_server_auth_strategy`

Type
string

Default
`noauth`

Valid Values
noauth, http_basic

Mutable
This option can be changed without restarting.

Used to select authentication strategy against the image hosting HTTP(S) server. When set to http_basic it enables HTTP(S) Basic Authentication. Exception is thrown in case of missing credentials. When this option has a valid value such as http_basic, the same single set of credentials will be used against all user-image sources! Currently only the http_basic option has any functionality.

Possible values

noauth
No authentication

http_basic
HTTP Basic authentication

`image_server_user`

Type
string

Default
`<None>`

Mutable
This option can be changed without restarting.

Can be used by any authentication strategy that requires username credential. Currently utilized by the http_basic authentication strategy.

`image_server_password`

Type
string

Default
`<None>`

Mutable
This option can be changed without restarting.

Can be used by any authentication strategy that requires password credential. Currently utilized by the http_basic authentication strategy.

`external_http_url`

Type
URI

Default
<None>

URL of the ironic-conductor nodes HTTP server for boot methods such as virtual media, where images could be served outside of the provisioning network. Does not apply when Swift is used. Defaults to http_url.

`external_callback_url`

Type
URI

Default
<None>

Agent callback URL of the bare metal API for boot methods such as virtual media, where images could be served outside of the provisioning network. Defaults to the configuration from [service_catalog].

`enable_ata_secure_erase`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

Whether to support the use of ATA Secure Erase during the cleaning process.

`enable_nvme_secure_erase`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

Whether to support the use of NVMe Secure Erase during the cleaning process. Currently nvme-cli format command is supported with user-data and crypto modes, depending on device capabilities.

`erase_devices_priority`

Type
integer

Default
<None>

Mutable
This option can be changed without restarting.

Priority to run in-band erase devices via the Ironic Python Agent ramdisk. If unset, will use the priority set in the ramdisk (defaults to 10 for the GenericHardwareManager). If set to 0, will not run during cleaning.

`erase_devices_metadata_priority`

Type
integer

Default
<None>

Mutable
This option can be changed without restarting.

Priority to run in-band clean step that erases metadata from devices, via the Ironic Python Agent ramdisk. If unset, will use the priority set in the ramdisk (defaults to 99 for the GenericHardwareManager). If set to 0, will not run during cleaning.

`delete_configuration_priority`

Type
integer

Default
<None>

Mutable
This option can be changed without restarting.

Priority to run in-band clean step that erases RAID configuration from devices, via the Ironic Python Agent ramdisk. If unset, will use the priority set in the ramdisk (defaults to 0 for the GenericHardwareManager). If set to 0, will not run during cleaning.

`create_configuration_priority`

Type
integer

Default
<None>

Mutable
This option can be changed without restarting.

Priority to run in-band clean step that creates RAID configuration from devices, via the Ironic Python Agent ramdisk. If unset, will use the priority set in the ramdisk (defaults to 0 for the GenericHardwareManager). If set to 0, will not run during cleaning.

`shred_random_overwrite_iterations`

Type
integer

Default
1

Minimum Value
0

Mutable
This option can be changed without restarting.

During shred, overwrite all block devices N times with random data. This is only used if a device could not be ATA Secure Erased.

`shred_final_overwrite_with_zeros`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

Whether to write zeros to a nodes block devices after writing random data. This will write zeros to the device even when deploy.shred_random_overwrite_iterations is 0. This option is only used if a device could not be ATA Secure Erased.

`continue_if_disk_secure_erase_fails`

Type
boolean

Default
`False`

Mutable
This option can be changed without restarting.

Defines what to do if a secure erase operation (NVMe or ATA) fails during cleaning in the Ironic Python Agent. If False, the cleaning operation will fail and the node will be put in `clean failed` state. If True, shred will be invoked and cleaning will continue.

`disk_erasure_concurrency`

Type
integer

Default
`4`

Minimum Value
1

Mutable
This option can be changed without restarting.

Defines the target pool size used by Ironic Python Agent ramdisk to erase disk devices. The number of threads created to erase disks will not exceed this value or the number of disks to be erased.

`power_off_after_deploy_failure`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

Whether to power off a node after deploy failure.

`default_boot_mode`

Type
string

Default
`uefi`

Valid Values
uefi, bios

Mutable
This option can be changed without restarting.

Default boot mode to use when no boot mode is requested in nodes driver_info, capabilities or in the *instance_info* configuration. Currently the default boot mode is uefi, but it was bios previously in Ironic. It is recommended to set an explicit value for this option, and if the setting or default differs from nodes, to ensure that nodes are configured specifically for their desired boot mode.

Possible values

uefi
UEFI boot mode

bios
Legacy BIOS boot mode

`configdrive_use_object_store`

Type
boolean

Default
`False`

Mutable
This option can be changed without restarting.

Whether to upload the config drive to object store. Set this option to True to store config drive in a swift endpoint.

Table 29: Deprecated Variations

Group	Name
conductor	configdrive_use_swift

`http_image_subdir`

Type
string

Default
`agent_images`

The name of subdirectory under ironic-conductor nodes HTTP root path which is used to place instance images for the direct deploy interface, when local HTTP service is incorporated to provide instance image instead of swift tempurls.

`fast_track`

Type
boolean

Default
`False`

Mutable
This option can be changed without restarting.

Whether to allow deployment agents to perform lookup, heartbeat operations during initial states of a machine lifecycle and by-pass the normal setup procedures for a ramdisk. This feature also enables power operations which are part of deployment processes to be bypassed if the ramdisk has performed a heartbeat operation using the fast_track_timeout setting.

`fast_track_timeout`

Type
integer

Default
`300`

Minimum Value
0

Maximum Value
300

Mutable
This option can be changed without restarting.

Seconds for which the last heartbeat event is to be considered valid for the purpose of a fast track sequence. This setting should generally be less than the number of seconds for Power-On Self Test and typical ramdisk start-up. This value should not exceed the [api]ramdisk_heartbeat_timeout setting.

`erase_skip_read_only`

Type
boolean

Default
`False`

Mutable
This option can be changed without restarting.

If the ironic-python-agent should skip read-only devices when running the erase_devices clean step where block devices are zeroed out. This requires ironic-python-agent 6.0.0 or greater. By default a read-only device will cause non-metadata based cleaning operations to fail due to the possible operational security risk of data being retained between deployments of the bare metal node.

`ramdisk_image_download_source`

Type
string

Default
`local`

Valid Values
http, local, swift

Mutable
This option can be changed without restarting.

Specifies whether a boot iso image should be served from its own original location using the image source url directly, or if ironic should cache the image on the conductor and serve it from ironics own http server.

Possible values

http
In case the ramdisk is already a bootable iso, using this option it will be directly provided by an external HTTP service using its full url.

local
This is the default behavior. The image is downloaded, prepared and cached locally, to be served from the conductor.

swift
Same as http, but if the image is a Glance UUID, it is exposed via a Swift temporary URL.

`iso_master_path`

Type
string

Default
`/var/lib/ironic/master_iso_images`

On the ironic-conductor node, directory where master ISO images are stored on disk. Setting to the empty string disables image caching.

`iso_cache_size`

Type
integer

Default
`20480`

Maximum size (in MiB) of cache for master ISO images, including those in use.

`iso_cache_ttl`

Type
integer

Default
`10080`

Maximum TTL (in minutes) for old master ISO images in cache.

dhcp

dhcp_provider

Type
string

Default
`neutron`

DHCP provider to use. neutron uses Neutron, dnsmasq uses the Dnsmasq provider, and none uses a no-op provider.

disk_utils

image_convert_memory_limit

Type
integer

Default
`2048`

Memory limit for qemu-img convert in MiB. Implemented via the address space resource limit.

image_convert_attempts

Type
integer

Default
`3`

Number of attempts to convert an image.

drac

query_raid_config_job_status_interval

Type
integer

Default
`120`

Minimum Value
1

Interval (in seconds) between periodic RAID job status checks to determine whether the asynchronous RAID configuration was successfully finished or not.

boot_device_job_status_timeout

Type
integer

Default
`30`

Minimum Value
1

Maximum amount of time (in seconds) to wait for the boot device configuration job to transition to the correct state to allow a reboot or power on to complete.

`config_job_max_retries`

Type
integer

Default
`240`

Minimum Value
1

Maximum number of retries for the configuration job to complete successfully.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
This option has has no effect since 26.0.0

`query_import_config_job_status_interval`

Type
integer

Default
`60`

Minimum Value
0

Number of seconds to wait between checking for completed import configuration task

`bios_factory_reset_timeout`

Type
integer

Default
`600`

Minimum Value
1

Maximum time (in seconds) to wait for factory reset of BIOS settings to complete.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
This option has has no effect since 26.0.0

`raid_job_timeout`

Type
integer

Default
`300`

Minimum Value
1

Maximum time (in seconds) to wait for RAID job to complete

errors

`fatal_exception_format_errors`

Type
boolean

Default
`False`

Used if there is a formatting error when generating an exception message (a programming error). If True, raise an exception; if False, use the unformatted message.

Table 30: Deprecated Variations

Group	Name
ironic_lib	fatal_exception_format_errors

glance

`allowed_direct_url_schemes`

Type
list

Default
`[]`

A list of URL schemes that can be downloaded directly via the direct_url. Currently supported schemes: [file].

`auth_url`

Type
unknown type

Default
<None>

Authentication URL

`auth_type`

Type
unknown type

Default
<None>

Authentication type to load

Table 31: Deprecated Variations

Group	Name
glance	auth_plugin

`cafile`

Type
string

Default
<None>

PEM encoded Certificate Authority to use when verifying HTTPs connections.

`certfile`

Type
string

Default
<None>

PEM encoded client certificate cert file

`collect_timing`

Type
boolean

Default
`False`

Collect per-API call timing information.

`connect_retries`

Type
integer

Default
<None>

The maximum number of retries that should be attempted for connection errors.

`connect_retry_delay`

Type
floating point

Default
<None>

Delay (in seconds) between two retries for connection errors. If not set, exponential retry starting with 0.5 seconds up to a maximum of 60 seconds is used.

`default_domain_id`

Type
unknown type

Default
<None>

Optional domain ID to use with v3 and v2 parameters. It will be used for both the user and project domain in v3 and ignored in v2 authentication.

`default_domain_name`

Type
unknown type

Default
<None>

Optional domain name to use with v3 API and v2 parameters. It will be used for both the user and project domain in v3 and ignored in v2 authentication.

`domain_id`

Type
unknown type

Default
<None>

Domain ID to scope to

`domain_name`

Type
unknown type

Default
<None>

Domain name to scope to

`endpoint_override`

Type
string

Default
<None>

Always use this endpoint URL for requests for this client. NOTE: The unversioned endpoint should be specified here; to request a particular API version, use the *version*, *min-version*, and/or *max-version* options.

`insecure`

Type
boolean

Default
`False`

Verify HTTPS connections.

`keyfile`

Type
string

Default
<None>

PEM encoded client certificate key file

`max_version`

Type
string

Default
<None>

The maximum major version of a given API, intended to be used as the upper bound of a range with min_version. Mutually exclusive with version.

`min_version`

Type
string

Default
<None>

The minimum major version of a given API, intended to be used as the lower bound of a range with max_version. Mutually exclusive with version. If min_version is given with no max_version it is as if max version is latest.

`num_retries`

Type
integer

Default
`0`

Mutable
This option can be changed without restarting.

Number of retries when downloading an image from glance.

`password`

Type
unknown type

Default
<None>

Users password

`project_domain_id`

Type
unknown type

Default
<None>

Domain ID containing project

`project_domain_name`

Type
unknown type

Default
<None>

Domain name containing project

`project_id`

Type
unknown type

Default
<None>

Project ID to scope to

Table 32: Deprecated Variations

Group	Name
glance	tenant-id
glance	tenant_id

`project_name`

Type
unknown type

Default
<None>

Project name to scope to

Table 33: Deprecated Variations

Group	Name
glance	tenant-name
glance	tenant_name

`region_name`

Type
string

Default
<None>

The default region_name for endpoint URL discovery.

`retriable_status_codes`

Type
list

Default
<None>

List of retriable HTTP status codes that should be retried. If not set default to [503]

`service_name`

Type
string

Default
<None>

The default service_name for endpoint URL discovery.

`service_type`

Type
string

Default
`image`

The default service_type for endpoint URL discovery.

`split_loggers`

Type
boolean

Default
`False`

Log requests to multiple loggers.

`status_code_retries`

Type
integer

Default
<None>

The maximum number of retries that should be attempted for retriable HTTP status codes.

`status_code_retry_delay`

Type
floating point

Default
<None>

Delay (in seconds) between two retries for retriable status codes. If not set, exponential retry starting with 0.5 seconds up to a maximum of 60 seconds is used.

`swift_account`

Type
string

Default
<None>

The account that Glance uses to communicate with Swift. The format is AUTH_uuid. uuid is the UUID for the account configured in the glance-api.conf. For example: AUTH_a422b2-91f3-2f46-74b7-d7c9e8958f5d30. If not set, the default value is calculated based on the ID of the project used to access Swift (as set in the [swift] section). Swift temporary URL format: endpoint_url/api_version/account/container/object_id

`swift_account_prefix`

Type
string

Default
AUTH

The prefix added to the project uuid to determine the swift account.

`swift_api_version`

Type
string

Default
v1

The Swift API version to create a temporary URL for. Swift temporary URL format: endpoint_url/api_version/account/container/object_id

`swift_container`

Type
string

Default
glance

The Swift container Glance is configured to store its images in. Defaults to glance, which is the default in glance-api.conf. Swift temporary URL format: endpoint_url/api_version/account/container/object_id

`swift_endpoint_url`

Type
URI

Default
<None>

The endpoint (scheme, hostname, optional port) for the Swift URL of the form endpoint_url/api_version/account/container/object_id. Do not include trailing /. For example, use https://swift.example.com. If using RADOS Gateway, endpoint may also contain /swift path; if it does not, it will be appended. Used for temporary URLs, will be fetched from the service catalog, if not provided.

`swift_store_multiple_containers_seed`

Type
integer

Default
`0`

This should match a config by the same name in the Glance configuration file. When set to 0, a single-tenant store will only use one container to store all images. When set to an integer value between 1 and 32, a single-tenant store will use multiple containers to store images, and this value will determine how many containers are created.

`swift_temp_url_cache_enabled`

Type
boolean

Default
`False`

Whether to cache generated Swift temporary URLs. Setting it to true is only useful when an image caching proxy is used.

`swift_temp_url_duration`

Type
integer

Default
`1200`

The length of time in seconds that the temporary URL will be valid for. If some deploys get a 401 response code when trying to download from the temporary URL, try raising this duration. This value must be greater than or equal to the value for swift_temp_url_expected_download_start_delay

`swift_temp_url_expected_download_start_delay`

Type
integer

Default
`0`

Minimum Value
0

This is the delay (in seconds) from the time of the deploy request (when the Swift temporary URL is generated) to when the IPA ramdisk starts up and URL is used for the image download. This value is used to check if the Swift temporary URL duration is large enough to let the image download begin. Also if temporary URL caching is enabled this will determine if a cached entry will still be valid when the download starts. swift_temp_url_duration value must be greater than or equal to this options value.

`swift_temp_url_key`

Type
string

Default
<None>

The secret token given to Swift to allow temporary URL downloads. Required for temporary URLs. For the Swift backend, the key on the service project (as set in the [swift] section) is used by default.

`system_scope`

Type
unknown type

Default
<None>

Scope for system operations

`tenant_id`

Type
unknown type

Default
<None>

Tenant ID

`tenant_name`

Type
unknown type

Default
<None>

Tenant Name

`timeout`

Type
integer

Default
<None>

Timeout value for http requests

`trust_id`

Type
unknown type

Default
<None>

ID of the trust to use as a trustee use

`user_domain_id`

Type
unknown type

Default
<None>

Users domain id

`user_domain_name`

Type
unknown type

Default
<None>

Users domain name

`user_id`

Type
unknown type

Default
<None>

User id

`username`

Type
unknown type

Default
<None>

Username

Table 34: Deprecated Variations

Group	Name
glance	user-name
glance	user_name

`valid_interfaces`

Type
list

Default
`['internal', 'public']`

List of interfaces, in order of preference, for endpoint URL.

`version`

Type
string

Default
<None>

Minimum Major API version within a given Major API version for endpoint URL discovery. Mutually exclusive with min_version and max_version

healthcheck

enabled

Type
boolean

Default
`False`

Enable the health check endpoint at /healthcheck. Note that this is unauthenticated. More information is available at https://docs.openstack.org/oslo.middleware/latest/reference/healthcheck_plugins.html.

detailed

Type
boolean

Default
`False`

Show more detailed information as part of the response. Security note: Enabling this option may expose sensitive details about the service being monitored. Be sure to verify that it will not violate your security policies.

backends

Type
list

Default
`[]`

Additional backends that can perform health checks and report that information back as part of a request.

allowed_source_ranges

Type
list

Default
`[]`

A list of network addresses to limit source ip allowed to access healthcheck information. Any request from ip outside of these network addresses are ignored.

ignore_proxied_requests

Type
boolean

Default
`False`

Ignore requests with proxy headers.

`disable_by_file_path`

Type
string

Default
<None>

Check the presence of a file to determine if an application is running on a port. Used by DisableByFileHealthcheck plugin.

`disable_by_file_paths`

Type
list

Default
[]

Check the presence of a file based on a port to determine if an application is running on a port. Expects a port:path list of strings. Used by DisableByFilesPortsHealthcheck plugin.

`enable_by_file_paths`

Type
list

Default
[]

Check the presence of files. Used by EnableByFilesHealthcheck plugin.

ilo

`client_timeout`

Type
integer

Default
60

Timeout (in seconds) for iLO operations

`client_port`

Type
port number

Default
443

Minimum Value
0

Maximum Value
65535

Port to be used for iLO operations

`swift_ilo_container`

Type
string

Default
`ironic_ilo_container`

The Swift iLO container to store data.

`swift_object_expiry_timeout`

Type
integer

Default
`900`

Amount of time in seconds for Swift objects to auto-expire.

`use_web_server_for_images`

Type
boolean

Default
`False`

Set this to True to use http web server to host floppy images and generated boot ISO. This requires http_root and http_url to be configured in the [deploy] section of the config file. If this is set to False, then Ironic will use Swift to host the floppy images and generated boot_iso.

`clean_priority_reset_ilo`

Type
integer

Default
`0`

Priority for reset_ilo clean step.

`clean_priority_reset_bios_to_default`

Type
integer

Default
`10`

Priority for reset_bios_to_default clean step.

`clean_priority_reset_secure_boot_keys_to_default`

Type
integer

Default
`20`

Priority for reset_secure_boot_keys clean step. This step will reset the secure boot keys to manufacturing defaults.

`clean_priority_clear_secure_boot_keys`

Type
integer

Default
0

Priority for clear_secure_boot_keys clean step. This step is not enabled by default. It can be enabled to clear all secure boot keys enrolled with iLO.

`clean_priority_reset_ilo_credential`

Type
integer

Default
30

Priority for reset_ilo_credential clean step. This step requires ilo_change_password parameter to be updated in nodess driver_info with the new password.

`power_wait`

Type
integer

Default
2

Amount of time in seconds to wait in between power operations

`oob_erase_devices_job_status_interval`

Type
integer

Default
300

Minimum Value
10

Interval (in seconds) between periodic erase-devices status checks to determine whether the asynchronous out-of-band erase-devices was successfully finished or not. On an average, a 300GB HDD with default pattern overwrite would take approximately 9 hours and 300GB SSD with default pattern block would take approx. 30 seconds to complete sanitize disk erase.

`ca_file`

Type
string

Default
<None>

CA certificate file to validate iLO.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
Its being replaced by new configuration parameter verify_ca.

`verify_ca`

Type
string

Default
`True`

CA certificate to validate iLO. This can be either a Boolean value, a path to a CA_BUNDLE file or directory with certificates of trusted CAs. If set to True the driver will verify the host certificates; if False the driver will ignore verifying the SSL certificate. If its a path the driver will use the specified certificate or one of the certificates in the directory.

`default_boot_mode`

Type
string

Default
`auto`

Valid Values
auto, bios, uefi

Default boot mode to be used in provisioning when boot_mode capability is not provided in the properties/capabilities of the node. The default is auto for backward compatibility. When auto is specified, default boot mode will be selected based on boot mode settings on the system.

Possible values

auto
based on boot mode settings on the system

bios
BIOS boot mode

uefi
UEFI boot mode

`file_permission`

Type
integer

Default
`420`

File permission for swift-less image hosting with the octal permission representation of file access permissions. This setting defaults to `644`, or as the octal number `0o644` in Python. This setting must be set to the octal number representation, meaning starting with `0o`.

`kernel_append_params`

Type
string

Default
`nofb vga=normal`

Mutable
This option can be changed without restarting.

Additional kernel parameters to pass down to the instance kernel. These parameters can be consumed by the kernel or by the applications by reading /proc/cmdline. Mind severe cmdline size limit! Can be overridden by *instance_info/kernel_append_params* property.

`cert_path`

Type
string

Default
`/var/lib/ironic/ilo/`

On the ironic-conductor node, directory where ilo driver stores the CSR and the cert.

inspection_rules

`built_in_rules`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

Path to YAML file of built-in inspection rules.

`mask_secrets`

Type
string

Default
`always`

Valid Values
always, never, sensitive

Whether to mask secrets in the node information passed to the rules.

inspector

`status_check_period`

Type
integer

Default
`60`

period (in seconds) to check status of nodes on inspection

`force_dhcp`

Type
boolean

Default
`False`

If True, during managed inspection force the inspection ramdisk to use DHCP on all available interfaces and avoid injecting any static network configuration into the virtual media ISO. This ensures LLDP collection across all interfaces.

`extra_kernel_params`

Type
string

Default
`''`

extra kernel parameters to pass to the inspection ramdisk when boot is managed by ironic. Pairs key=value separated by spaces.

`power_off`

Type
boolean

Default
`True`

whether to power off a node after inspection finishes. Ignored for nodes that have fast track mode enabled. Note that the node may be powered off at the end of inspection for other reasons, e.g. for safe ejection of virtual media without corrupting the filesystem.

`callback_endpoint_override`

Type
string

Default
`<None>`

endpoint to use as a callback for posting back introspection data when boot is managed by ironic. Standard keystoneauth options are used by default.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
This option was used by inspector inspect interface, which was removed.

`require_managed_boot`

Type
boolean

Default
`True`

require that the in-band inspection boot is fully managed by the nodes boot interface. Set this to False if your installation has a separate (i)PXE boot environment for node discovery or unmanaged inspection. You may need to set it to False to inspect nodes that are not supported by boot interfaces (e.g. because they dont have ports).

`add_ports`

Type
string

Default
`pxe`

Valid Values
all, active, pxe, disabled

Which MAC addresses to add as ports during inspection.

Possible values

all
all MAC addresses

active
MAC addresses of NICs with IP addresses

pxe
only the MAC address of the PXE NIC

disabled
do not create any ports

`keep_ports`

Type
string

Default
`all`

Valid Values
all, present, added

Which ports (already present on a node) to keep after inspection.

Possible values

all
keep all ports, even ones with MAC addresses that are not present in the inventory

present
keep only ports with MAC addresses present in the inventory

added
keep only ports determined by the add_ports option

`update_pxe_enabled`

Type
boolean

Default
`True`

Whether to update the ports pxe_enabled field according to the inspection data.

`default_hooks`

Type
string

Default
`ramdisk-error,validate-interfaces,ports,architecture`

A comma-separated lists of inspection hooks that are run by default for the agent inspection interface. In most cases, the operators will not modify this. The default (somewhat conservative) hooks will raise an exception in case the ramdisk reports an error, validate interfaces in the inventory, create ports and set the nodes cpu architecture property.

`hooks`

Type
string

Default
`$default_hooks`

Comma-separated list of enabled hooks for processing pipeline for the agent inspection interface. The default for this is $default_hooks. Hooks can be added before or after the defaults like this: prehook,$default_hooks,posthook.

`known_accelerators`

Type
string

Default
`$pybasedir/drivers/modules/inspector/hooks/known_accelerators.yaml`

Path to the file which contains the known accelerator devices, to be used by the accelerators inspection hook.

`cpu_capabilities`

Type
dict

Default

```
{'vmx': 'cpu_vt', 'svm': 'cpu_vt', 'aes': 'cpu_aes', 'pse':
'cpu_hugepages', 'pdpe1gb': 'cpu_hugepages_1g', 'smx':
'cpu_txt'}
```

Mapping between a CPU flag and a node capability to set if this CPU flag is present. This configuration option is used by the cpu-capabilities inspection hook.

`extra_hardware_strict`

Type
boolean

Default
`False`

If True, refuse to parse extra data (in plugin_data) if at least one record is too short. Additionally, remove the incoming data even if parsing failed. This configuration option is used by the extra-hardware inspection hook.

`pci_device_alias`

Type
multi-valued

Default
`''`

An alias for a PCI device identified by vendor_id and product_id fields. Format: {vendor_id: 1234, product_id: 5678, name: pci_dev1}. Use double quotes for the keys and values.

`physical_network_cidr_map`

Type
list

Default
`10.10.10.0/24:physnet_a,2001:db8::/64:physnet_b`

This option has a sample default set, which means that its actual default value may vary from the one documented above.

Mapping of IP subnet CIDR to physical network. When the phyical-network inspection hook is enabled, the physical_network property of corresponding baremetal ports is populated based on this mapping.

`disk_partitioning_spacing`

Type
boolean

Default
`True`

Whether to leave 1 GiB of disk size untouched for partitioning. Only has effect when used with the IPA as a ramdisk, for older ramdisk local_gb is calculated on the ramdisk side. This configuration option is used by the root-device inspection hook.

inventory

data_backend

Type
string

Default
`database`

Valid Values
none, database, swift

The storage backend for storing inspection data.

Possible values

none
do not store inspection data

database
store in the service database

swift
store in the Object Storage (swift)

swift_data_container

Type
string

Default
`introspection_data_container`

The Swift container prefix to store the inspection data (separately inventory and plugin data).

ipmi

command_retry_timeout

Type
integer

Default
`60`

Mutable
This option can be changed without restarting.

Maximum time in seconds to retry retryable IPMI operations. (An operation is retryable, for example, if the requested operation fails because the BMC is busy.) Setting this too high can cause the sync power state periodic task to hang when there are slow or unresponsive BMCs.

min_command_interval

Type
integer

Default
`5`

Mutable
This option can be changed without restarting.

Minimum time, in seconds, between IPMI operations sent to a server. There is a risk with some hardware that setting this too low may cause the BMC to crash. Recommended setting is 5 seconds.

`use_ipmitool_retries`

Type
boolean

Default
`False`

When set to True and the parameters are supported by ipmitool, the number of retries and the retry interval are passed to ipmitool as parameters, and ipmitool will do the retries. When set to False, ironic will retry the ipmitool commands. Recommended setting is False

`kill_on_timeout`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

Kill *ipmitool* process invoked by ironic to read node power state if *ipmitool* process does not exit after *command_retry_timeout* timeout expires. Recommended setting is True. Setting to False may present an operational issue and will result in unexpected and undesirable behavior.

`disable_boot_timeout`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

Default timeout behavior whether ironic sends a raw IPMI command to disable the 60 second timeout for booting. Setting this option to False will NOT send that command, the default value is True. It may be overridden by per-node ipmi_disable_boot_timeout option in nodes driver_info field.

`additional_retryable_ipmi_errors`

Type
multi-valued

Default
`''`

Mutable
This option can be changed without restarting.

Additional errors ipmitool may encounter, specific to the environment it is run in.

debug

Type
boolean

Default
`False`

Mutable
This option can be changed without restarting.

Enables all ipmi commands to be executed with an additional debugging output. This is a separate option as ipmitool can log a substantial amount of misleading text when in this mode.

store_cred_in_env

Type
boolean

Default
`False`

Boolean flag to determine IPMI password persistence method. When True, credentials are stored in environemnt variables, Otherwise, credentials are stored in a file

cipher_suite_versions

Type
list

Default
`[]`

List of possible cipher suites versions that can be supported by the hardware in case the field *cipher_suite* is not set for the node.

ironic_networking

rpc_transport

Type
string

Default
<None>

Valid Values
json-rpc, oslo_messaging

The transport mechanism used for RPC communication. This can be set to json-rpc for JSON-RPC, oslo_messaging for Oslo Messaging, or none for no transport.

switch_config_file

Type
string

Default
`''`

Path to the switch configuration file that defines switches to be acted upon. The config file should be in INI format. For syntax refer to the user guide.

`driver_config_dir`

Type
string

Default
`/var/lib/ironic/networking`

The path to the driver configuration directory. This is used to dynamically write driver config files that are derived from entries in the file specified by the switch_config_file option. This directory should not be populated with files manually.

`enabled_switch_drivers`

Type
list

Default
`[]`

A list of switch drivers to load and make available for managing network switches. Switch drivers are loaded from external projects via entry points in the ironic.networking.switch_drivers namespace. Only drivers listed here will be loaded and made available for use. An empty list means no switch drivers will be loaded.

`allowed_vlans`

Type
list

Default
<None>

A list of VLAN IDs that are allowed to be used for port configuration. If not specified (None), all VLAN IDs are allowed. If set to an empty list ([]), no VLANs are allowed. If set to a list of values, only the specified VLAN IDs are allowed. The list is a comma separated list of VLAN ID values or range of values. For example, 100,101,102-104,106 would allow VLANs 100, 101, 102, 103, 104, and 106, but not 105. This setting can be overridden on a per-switch basis in the switch configuration file.

`cleaning_network`

Type
string

Default
`''`

The network to use for cleaning nodes. This should be expressed as {access|trunk}/native_vlan=VLAN_ID. Can be overridden on a per-node basis using the driver_info attribute and specifying this as *cleaning_network*

`rescuing_network`

Type
string

Default
''

The network to use for rescuing nodes. This should be expressed as {access|trunk}/native_vlan=VLAN_ID. Can be overridden on a per-node basis using the driver_info attribute and specifying this as *rescuing_network*

`provisioning_network`

Type
string

Default
''

The network to use for provisioning nodes. This should be expressed as {access|trunk}/native_vlan=VLAN_ID. Can be overridden on a per-node basis using the driver_info attribute and specifying this as *provisioning_network*

`servicing_network`

Type
string

Default
''

The network to use for servicing nodes. This should be expressed as {access|trunk}/native_vlan=VLAN_ID. Can be overridden on a per-node basis using the driver_info attribute and specifying this as *servicing_network*

`inspection_network`

Type
string

Default
''

The network to use for inspecting nodes. This should be expressed as {access|trunk}/native_vlan=VLAN_ID. Can be overridden on a per-node basis using the driver_info attribute and specifying this as *inspection_network*

`idle_network`

Type
string

Default
''

The network to use for initial inspecting of nodes. If provided switch ports will be configured back to this network whenever any of the other networks are removed/unconfigured. This should be expressed as {access|trunk}/native_vlan=VLAN_ID. Can be overridden on a per-node basis using the driver_info attribute and specifying this as *idle_network*

ironic_networking_json_rpc

allowed_roles

Type
list

Default
['admin']

List of roles allowed to use JSON RPC

auth_url

Type
unknown type

Default
<None>

Authentication URL

auth_strategy

Type
string

Default
<None>

Valid Values
noauth, keystone, http_basic

Authentication strategy used by JSON RPC. Defaults to the global auth_strategy setting.

Possible values

noauth
no authentication

keystone
use the Identity service for authentication

http_basic
HTTP basic authentication

auth_type

Type
unknown type

Default
<None>

Authentication type to load

Table 35: Deprecated Variations

Group	Name
ironic_networking_json_rpc	auth_plugin

`cafile`

Type
string

Default
<None>

PEM encoded Certificate Authority to use when verifying HTTPs connections.

`cert_file`

Type
string

Default
<None>

Certificate file the JSON-RPC listener will present to clients when [json_rpc]use_ssl=True.

`certfile`

Type
string

Default
<None>

PEM encoded client certificate cert file

`client_use_ssl`

Type
boolean

Default
`False`

Set to True to force TLS connections in the client even if use_ssl is set to False. Only makes sense if server-side TLS is provided outside of Ironic (e.g. with httpd acting as a reverse proxy).

`collect_timing`

Type
boolean

Default
`False`

Collect per-API call timing information.

`debug_log_request_id_only`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

When debug logging is enabled, log only the request ID instead of the full request and response for JSON RPC calls. This reduces log verbosity while still providing some traceability for performance analysis.

`default_domain_id`

Type
unknown type

Default
<None>

Optional domain ID to use with v3 and v2 parameters. It will be used for both the user and project domain in v3 and ignored in v2 authentication.

`default_domain_name`

Type
unknown type

Default
<None>

Optional domain name to use with v3 API and v2 parameters. It will be used for both the user and project domain in v3 and ignored in v2 authentication.

`domain_id`

Type
unknown type

Default
<None>

Domain ID to scope to

`domain_name`

Type
unknown type

Default
<None>

Domain name to scope to

`host_ip`

Type
host address

Default
::

The IP address or hostname on which JSON RPC will listen.

`http_basic_auth_user_file`

Type
string

Default
`/etc/ironic/htpasswd-json-rpc`

Path to Apache format user authentication file used when auth_strategy=http_basic

`http_basic_password`

Type
string

Default
<None>

Password to use for HTTP Basic authentication client requests.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
Use password instead

`http_basic_username`

Type
string

Default
<None>

Name of the user to use for HTTP Basic authentication client requests.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
Use username instead

`insecure`

Type
boolean

Default
`False`

Verify HTTPS connections.

`key_file`

Type
string

Default
<None>

Private key file matching cert_file.

`keyfile`

Type
string

Default
<None>

PEM encoded client certificate key file

`password`

Type
unknown type

Default
<None>

Users password

`port`

Type
port number

Default
8089

Minimum Value
0

Maximum Value
65535

The port to use for JSON RPC

`project_domain_id`

Type
unknown type

Default
<None>

Domain ID containing project

`project_domain_name`

Type
unknown type

Default
<None>

Domain name containing project

`project_id`

Type
unknown type

Default
<None>

Project ID to scope to

Table 36: Deprecated Variations

Group	Name
ironic_networking_json_rpc	tenant-id
ironic_networking_json_rpc	tenant_id

`project_name`

Type
unknown type

Default
<None>

Project name to scope to

Table 37: Deprecated Variations

Group	Name
ironic_networking_json_rpc	tenant-name
ironic_networking_json_rpc	tenant_name

`split_loggers`

Type
boolean

Default
`False`

Log requests to multiple loggers.

`system_scope`

Type
unknown type

Default
<None>

Scope for system operations

`tenant_id`

Type
unknown type

Default
<None>

Tenant ID

`tenant_name`

Type
unknown type

Default
<None>

Tenant Name

`timeout`

Type
integer

Default
<None>

Timeout value for http requests

`trust_id`

Type
unknown type

Default
<None>

ID of the trust to use as a trustee use

`unix_socket`

Type
string

Default
<None>

Unix socket to listen on. Disables host_ip and port.

`unix_socket_mode`

Type
unknown type

Default
<None>

File mode (an octal number) of the unix socket to listen on. Ignored if unix_socket is not set.

`use_ssl`

Type
boolean

Default
`False`

Whether to use TLS for JSON RPC

`user_domain_id`

Type
unknown type

Default
<None>

Users domain id

`user_domain_name`

Type
unknown type

Default
<None>

Users domain name

`user_id`

Type
unknown type

Default
<None>

User id

`username`

Type
unknown type

Default
<None>

Username

Table 38: Deprecated Variations

Group	Name
ironic_networking_json_rpc	user-name
ironic_networking_json_rpc	user_name

json_rpc

`allowed_roles`

Type
list

Default
`['admin']`

List of roles allowed to use JSON RPC

`auth_url`

Type
unknown type

Default
<None>

Authentication URL

`auth_strategy`

Type
string

Default
<None>

Valid Values
noauth, keystone, http_basic

Authentication strategy used by JSON RPC. Defaults to the global auth_strategy setting.

Possible values

noauth
no authentication

keystone
use the Identity service for authentication

http_basic
HTTP basic authentication

`auth_type`

Type
unknown type

Default
<None>

Authentication type to load

Table 39: Deprecated Variations

Group	Name
json_rpc	auth_plugin

`cafile`

Type
string

Default
<None>

PEM encoded Certificate Authority to use when verifying HTTPs connections.

`cert_file`

Type
string

Default
<None>

Certificate file the JSON-RPC listener will present to clients when [json_rpc]use_ssl=True.

`certfile`

Type
string

Default
<None>

PEM encoded client certificate cert file

`client_use_ssl`

Type
boolean

Default
`False`

Set to True to force TLS connections in the client even if use_ssl is set to False. Only makes sense if server-side TLS is provided outside of Ironic (e.g. with httpd acting as a reverse proxy).

`collect_timing`

Type
boolean

Default
`False`

Collect per-API call timing information.

`debug_log_request_id_only`

Type
boolean

Default
`True`

Mutable
This option can be changed without restarting.

When debug logging is enabled, log only the request ID instead of the full request and response for JSON RPC calls. This reduces log verbosity while still providing some traceability for performance analysis.

`default_domain_id`

Type
unknown type

Default
<None>

Optional domain ID to use with v3 and v2 parameters. It will be used for both the user and project domain in v3 and ignored in v2 authentication.

`default_domain_name`

Type
unknown type

Default
<None>

Optional domain name to use with v3 API and v2 parameters. It will be used for both the user and project domain in v3 and ignored in v2 authentication.

`domain_id`

Type
unknown type

Default
<None>

Domain ID to scope to

`domain_name`

Type
unknown type

Default
<None>

Domain name to scope to

`host_ip`

Type
host address

Default
::

The IP address or hostname on which JSON RPC will listen.

`http_basic_auth_user_file`

Type
string

Default
/etc/ironic/htpasswd-json-rpc

Path to Apache format user authentication file used when auth_strategy=http_basic

`http_basic_password`

Type
string

Default
<None>

Password to use for HTTP Basic authentication client requests.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
Use password instead

`http_basic_username`

Type
string

Default
<None>

Name of the user to use for HTTP Basic authentication client requests.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
Use username instead

`insecure`

Type
boolean

Default
`False`

Verify HTTPS connections.

`key_file`

Type
string

Default
<None>

Private key file matching cert_file.

`keyfile`

Type
string

Default
<None>

PEM encoded client certificate key file

`password`

Type
unknown type

Default
<None>

Users password

`port`

Type
port number

Default
8089

Minimum Value
0

Maximum Value
65535

The port to use for JSON RPC

`project_domain_id`

Type
unknown type

Default
<None>

Domain ID containing project

`project_domain_name`

Type
unknown type

Default
<None>

Domain name containing project

`project_id`

Type
unknown type

Default
<None>

Project ID to scope to

Table 40: Deprecated Variations

Group	Name
json_rpc	tenant-id
json_rpc	tenant_id

`project_name`

Type
unknown type

Default
<None>

Project name to scope to

Table 41: Deprecated Variations

Group	Name
json_rpc	tenant-name
json_rpc	tenant_name

`split_loggers`

Type
boolean

Default
`False`

Log requests to multiple loggers.

`system_scope`

Type
unknown type

Default
<None>

Scope for system operations

`tenant_id`

Type
unknown type

Default
<None>

Tenant ID

`tenant_name`

Type
unknown type

Default
<None>

Tenant Name

`timeout`

Type
integer

Default
<None>

Timeout value for http requests

`trust_id`

Type
unknown type

Default
<None>

ID of the trust to use as a trustee use

`unix_socket`

Type
string

Default
<None>

Unix socket to listen on. Disables host_ip and port.

`unix_socket_mode`

Type
unknown type

Default
<None>

File mode (an octal number) of the unix socket to listen on. Ignored if unix_socket is not set.

`use_ssl`

Type
boolean

Default
`False`

Whether to use TLS for JSON RPC

`user_domain_id`

Type
unknown type

Default
<None>

Users domain id

`user_domain_name`

Type
unknown type

Default
<None>

Users domain name

user_id

Type
unknown type

Default
<None>

User id

username

Type
unknown type

Default
<None>

Username

Table 42: Deprecated Variations

Group	Name
json_rpc	user-name
json_rpc	user_name

keystone_authtoken

www_authenticate_uri

Type
string

Default
<None>

Complete public Identity API endpoint. This endpoint should not be an admin endpoint, as it should be accessible by all end users. Unauthenticated clients are redirected to this endpoint to authenticate. Although this endpoint should ideally be unversioned, client support in the wild varies. If youre using a versioned v2 endpoint here, then this should *not* be the same endpoint the service user utilizes for validating tokens, because normal end users may not be able to reach that endpoint.

Table 43: Deprecated Variations

Group	Name
keystone_authtoken	auth_uri

auth_uri

Type
string

Default
<None>

Complete public Identity API endpoint. This endpoint should not be an admin endpoint, as it should be accessible by all end users. Unauthenticated clients are redirected to this endpoint to authenticate. Although this endpoint should ideally be unversioned, client support in the wild varies. If youre using a versioned v2 endpoint here, then this should *not* be the same endpoint the service user utilizes for validating tokens, because normal end users may not be able to reach that endpoint. This option is deprecated in favor of www_authenticate_uri and will be removed in the S release.

Warning

This option is deprecated for removal since Queens. Its value may be silently ignored in the future.

Reason
The auth_uri option is deprecated in favor of www_authenticate_uri and will be removed in the S release.

`auth_version`

Type
string

Default
<None>

API version of the Identity API endpoint.

`interface`

Type
string

Default
`internal`

Interface to use for the Identity API endpoint. Valid values are public, internal (default) or admin.

`delay_auth_decision`

Type
boolean

Default
`False`

Do not handle authorization requests within the middleware, but delegate the authorization decision to downstream WSGI components.

`http_connect_timeout`

Type
integer

Default
<None>

Request timeout value for communicating with Identity API server.

`http_request_max_retries`

Type
integer

Default
3

How many times are we trying to reconnect when communicating with Identity API Server.

`cache`

Type
string

Default
<None>

Request environment key where the Swift cache object is stored. When auth_token middleware is deployed with a Swift cache, use this option to have the middleware share a caching backend with swift. Otherwise, use the `memcached_servers` option instead.

`certfile`

Type
string

Default
<None>

Required if identity server requires client certificate

`keyfile`

Type
string

Default
<None>

Required if identity server requires client certificate

`cafile`

Type
string

Default
<None>

A PEM encoded Certificate Authority to use when verifying HTTPs connections. Defaults to system CAs.

`insecure`

Type
boolean

Default
False

Verify HTTPS connections.

`region_name`

Type
string

Default
<None>

The region in which the identity server can be found.

`memcached_servers`

Type
list

Default
<None>

Optionally specify a list of memcached server(s) to use for caching. If left undefined, tokens will instead be cached in-process.

Table 44: Deprecated Variations

Group	Name
keystone_authtoken	memcache_servers

`token_cache_time`

Type
integer

Default
300

In order to prevent excessive effort spent validating tokens, the middleware caches previously-seen tokens for a configurable duration (in seconds). Set to -1 to disable caching completely.

`memcache_security_strategy`

Type
string

Default
None

Valid Values
None, MAC, ENCRYPT

(Optional) If defined, indicate whether token data should be authenticated or authenticated and encrypted. If MAC, token data is authenticated (with HMAC) in the cache. If ENCRYPT, token data is encrypted and authenticated in the cache. If the value is not one of these options or empty, auth_token will raise an exception on initialization.

`memcache_secret_key`

Type
string

Default
<None>

(Optional, mandatory if memcache_security_strategy is defined) This string is used for key derivation.

`memcache_tls_enabled`

Type
boolean

Default
`False`

(Optional) Global toggle for TLS usage when comunicating with the caching servers.

`memcache_tls_cafile`

Type
string

Default
<None>

(Optional) Path to a file of concatenated CA certificates in PEM format necessary to establish the caching servers authenticity. If tls_enabled is False, this option is ignored.

`memcache_tls_certfile`

Type
string

Default
<None>

(Optional) Path to a single file in PEM format containing the clients certificate as well as any number of CA certificates needed to establish the certificates authenticity. This file is only required when client side authentication is necessary. If tls_enabled is False, this option is ignored.

`memcache_tls_keyfile`

Type
string

Default
<None>

(Optional) Path to a single file containing the clients private key in. Otherwhise the private key will be taken from the file specified in tls_certfile. If tls_enabled is False, this option is ignored.

`memcache_tls_allowed_ciphers`

Type
string

Default
<None>

(Optional) Set the available ciphers for sockets created with the TLS context. It should be a string in the OpenSSL cipher list format. If not specified, all OpenSSL enabled ciphers will be available.

`memcache_pool_dead_retry`

Type
integer

Default
`300`

(Optional) Number of seconds memcached server is considered dead before it is tried again.

`memcache_pool_maxsize`

Type
integer

Default
`10`

(Optional) Maximum total number of open connections to every memcached server.

`memcache_pool_socket_timeout`

Type
integer

Default
`3`

(Optional) Socket timeout in seconds for communicating with a memcached server.

`memcache_pool_unused_timeout`

Type
integer

Default
`60`

(Optional) Number of seconds a connection to memcached is held unused in the pool before it is closed.

`memcache_pool_conn_get_timeout`

Type
integer

Default
`10`

(Optional) Number of seconds that an operation will wait to get a memcached client connection from the pool.

`memcache_use_advanced_pool`

Type
boolean

Default
`True`

(Optional) Use the advanced (eventlet safe) memcached client pool.

`include_service_catalog`

Type
boolean

Default
`True`

(Optional) Indicate whether to set the X-Service-Catalog header. If False, middleware will not ask for service catalog on token validation and will not set the X-Service-Catalog header.

`enforce_token_bind`

Type
string

Default
`permissive`

Used to control the use and type of token binding. Can be set to: disabled to not check token binding. permissive (default) to validate binding information if the bind type is of a form known to the server and ignore it if not. strict like permissive but if the bind type is unknown the token will be rejected. required any form of token binding is needed to be allowed. Finally the name of a binding method that must be present in tokens.

`service_token_roles`

Type
list

Default
`['service']`

A choice of roles that must be present in a service token. Service tokens are allowed to request that an expired token can be used and so this check should tightly control that only actual services should be sending this token. Roles here are applied as an ANY check so any role in this list must be present. For backwards compatibility reasons this currently only affects the allow_expired check.

`service_token_roles_required`

Type
boolean

Default
`False`

For backwards compatibility reasons we must let valid service tokens pass that dont pass the service_token_roles check as valid. Setting this true will become the default in a future release and should be enabled if possible.

`service_type`

Type
string

Default
<None>

The name or type of the service as it appears in the service catalog. This is used to validate tokens that have restricted access rules.

memcache_sasl_enabled

Type
boolean

Default
`False`

Enable the SASL(Simple Authentication and Security Layer) if the SASL_enable is true, else disable.

memcache_username

Type
string

Default
`''`

the user name for the SASL

memcache_password

Type
string

Default
`''`

the username password for SASL

auth_type

Type
unknown type

Default
<None>

Authentication type to load

Table 45: Deprecated Variations

Group	Name
keystone_authtoken	auth_plugin

auth_section

Type
unknown type

Default
<None>

Config Section from which to load plugin specific options

mdns

`registration_attempts`

Type
integer

Default
5

Minimum Value
1

Number of attempts to register a service. Currently has to be larger than 1 because of race conditions in the zeroconf library.

`lookup_attempts`

Type
integer

Default
3

Minimum Value
1

Number of attempts to lookup a service.

`params`

Type
unknown type

Default
{}

Additional parameters to pass for the registered service.

`interfaces`

Type
list

Default
<None>

List of IP addresses of interfaces to use for mDNS. Defaults to all interfaces on the system.

metrics

`backend`

Type
string

Default
noop

Valid Values
noop, statsd, collector

Backend to use for the metrics system.

Possible values

noop
Do nothing in relation to metrics.

statsd
Transmits metrics data to a statsd backend.

collector
Collects metrics data and saves it in memory for use by the running application.

`prepend_host`

Type
boolean

Default
`False`

Prepend the hostname to all metric names. The format of metric names is [global_prefix.][host_name.]prefix.metric_name.

`prepend_host_reverse`

Type
boolean

Default
`True`

Split the prepended host value by . and reverse it (to better match the reverse hierarchical form of domain names).

`global_prefix`

Type
string

Default
<None>

Prefix all metric names with this value. By default, there is no global prefix. The format of metric names is [global_prefix.][host_name.]prefix.metric_name.

`agent_backend`

Type
string

Default
`noop`

Backend for the agent ramdisk to use for metrics. Default possible backends are noop and statsd.

`agent_prepend_host`

Type
boolean

Default
`False`

Prepend the hostname to all metric names sent by the agent ramdisk. The format of metric names is [global_prefix.][uuid.][host_name.]prefix.metric_name.

`agent_prepend_uuid`

Type
boolean

Default
`False`

Prepend the nodes Ironic uuid to all metric names sent by the agent ramdisk. The format of metric names is [global_prefix.][uuid.][host_name.]prefix.metric_name.

`agent_prepend_host_reverse`

Type
boolean

Default
`True`

Split the prepended host value by . and reverse it for metrics sent by the agent ramdisk (to better match the reverse hierarchical form of domain names).

`agent_global_prefix`

Type
string

Default
<None>

Prefix all metric names sent by the agent ramdisk with this value. The format of metric names is [global_prefix.][uuid.][host_name.]prefix.metric_name.

metrics_statsd

`statsd_host`

Type
string

Default
`localhost`

Host for use with the statsd backend.

`statsd_port`

Type
port number

Default
`8125`

Minimum Value
0

Maximum Value
65535

Port to use with the statsd backend.

`agent_statsd_host`

Type
string

Default
`localhost`

Host for the agent ramdisk to use with the statsd backend. This must be accessible from networks the agent is booted on.

`agent_statsd_port`

Type
port number

Default
`8125`

Minimum Value
0

Maximum Value
65535

Port for the agent ramdisk to use with the statsd backend.

molds

`storage`

Type
string

Default
`swift`

Valid Values
swift, http

Configuration mold storage location.

`user`

Type
string

Default
<None>

User for http Basic auth. By default set empty.

`password`

Type
string

Default
<None>

Password for http Basic auth. By default set empty.

`retry_attempts`

Type
integer

Default
3

Retry attempts for saving or getting configuration molds.

`retry_interval`

Type
integer

Default
3

Retry interval for saving or getting configuration molds.

neutron

`add_all_ports`

Type
boolean

Default
False

Mutable
This option can be changed without restarting.

Option to enable transmission of all ports to neutron when creating ports for provisioning, cleaning, or rescue. This is done without IP addresses assigned to the port, and may be useful in some bonded network configurations.

`allow_disabling_power_off`

Type
boolean

Default
False

By default, nodes with disable_power_off set to True cannot be used with the Neutron network interface because during tear-down they will be left with the instance image still running. Set this option to True to disable this validation.

`auth_url`

Type
unknown type

Default
<None>

Authentication URL

`auth_type`

Type
unknown type

Default
<None>

Authentication type to load

Table 46: Deprecated Variations

Group	Name
neutron	auth_plugin

`cafile`

Type
string

Default
<None>

PEM encoded Certificate Authority to use when verifying HTTPs connections.

`certfile`

Type
string

Default
<None>

PEM encoded client certificate cert file

`cleaning_network`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

Neutron network UUID or name for the ramdisk to be booted into for cleaning nodes. Required for neutron network interface. It is also required if cleaning nodes when using flat network interface or neutron DHCP provider. If a name is provided, it must be unique among all networks or cleaning will fail.

Table 47: Deprecated Variations

Group	Name
neutron	cleaning_network_uuid

`cleaning_network_security_groups`

Type
list

Default
`[]`

Mutable
This option can be changed without restarting.

List of Neutron Security Group UUIDs to be applied during cleaning of the nodes. Optional for the neutron network interface and not used for the flat or noop network interfaces. If not specified, default security group is used.

`collect_timing`

Type
boolean

Default
`False`

Collect per-API call timing information.

`connect_retries`

Type
integer

Default
<None>

The maximum number of retries that should be attempted for connection errors.

`connect_retry_delay`

Type
floating point

Default
<None>

Delay (in seconds) between two retries for connection errors. If not set, exponential retry starting with 0.5 seconds up to a maximum of 60 seconds is used.

`default_domain_id`

Type
unknown type

Default
<None>

Optional domain ID to use with v3 and v2 parameters. It will be used for both the user and project domain in v3 and ignored in v2 authentication.

`default_domain_name`

Type
unknown type

Default
<None>

Optional domain name to use with v3 API and v2 parameters. It will be used for both the user and project domain in v3 and ignored in v2 authentication.

`dhcpv6_stateful_address_count`

Type
integer

Default
4

Mutable
This option can be changed without restarting.

Number of IPv6 addresses to allocate for ports created for provisioning, cleaning, rescue or inspection on DHCPv6-stateful networks. Different stages of the chain-loading process will request addresses with different CLID/IAID. Due to non-identical identifiers multiple addresses must be reserved for the host to ensure each step of the boot process can successfully lease addresses.

`domain_id`

Type
unknown type

Default
<None>

Domain ID to scope to

`domain_name`

Type
unknown type

Default
<None>

Domain name to scope to

`endpoint_override`

Type
string

Default
<None>

Always use this endpoint URL for requests for this client. NOTE: The unversioned endpoint should be specified here; to request a particular API version, use the *version*, *min-version*, and/or *max-version* options.

`fail_on_port_binding_failure`

Type
boolean

Default
True

Whether to fail or continue deployment if neutron port binding fails.

`insecure`

Type
boolean

Default
`False`

Verify HTTPS connections.

`inspection_network`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

Neutron network UUID or name for the ramdisk to be booted into for in-band inspection of nodes. If a name is provided, it must be unique among all networks or inspection will fail.

`inspection_network_security_groups`

Type
list

Default
`[]`

Mutable
This option can be changed without restarting.

List of Neutron Security Group UUIDs to be applied during the node inspection process. Optional for the neutron network interface and not used for the flat or noop network interfaces. If not specified, the default security group is used.

`keyfile`

Type
string

Default
<None>

PEM encoded client certificate key file

`max_version`

Type
string

Default
<None>

The maximum major version of a given API, intended to be used as the upper bound of a range with min_version. Mutually exclusive with version.

`min_version`

Type
string

Default
<None>

The minimum major version of a given API, intended to be used as the lower bound of a range with max_version. Mutually exclusive with version. If min_version is given with no max_version it is as if max version is latest.

`password`

Type
unknown type

Default
<None>

Users password

`port_setup_delay`

Type
integer

Default
0

Minimum Value
0

Mutable
This option can be changed without restarting.

Delay value to wait for Neutron agents to setup sufficient DHCP configuration for port.

`project_domain_id`

Type
unknown type

Default
<None>

Domain ID containing project

`project_domain_name`

Type
unknown type

Default
<None>

Domain name containing project

`project_id`

Type
unknown type

Default
<None>

Project ID to scope to

Table 48: Deprecated Variations

Group	Name
neutron	tenant-id
neutron	tenant_id

`project_name`

Type
unknown type

Default
<None>

Project name to scope to

Table 49: Deprecated Variations

Group	Name
neutron	tenant-name
neutron	tenant_name

`provisioning_network`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

Neutron network UUID or name for the ramdisk to be booted into for provisioning nodes. Required for neutron network interface. If a name is provided, it must be unique among all networks or deploy will fail.

Table 50: Deprecated Variations

Group	Name
neutron	provisioning_network_uuid

`provisioning_network_security_groups`

Type
list

Default
[]

Mutable
This option can be changed without restarting.

List of Neutron Security Group UUIDs to be applied during provisioning of the nodes. Optional for the neutron network interface and not used for the flat or noop network interfaces. If not specified, default security group is used.

`region_name`

Type
string

Default
<None>

The default region_name for endpoint URL discovery.

`request_timeout`

Type
integer

Default
45

Mutable
This option can be changed without restarting.

Timeout for request processing when interacting with Neutron. This value should be increased if neutron port action timeouts are observed as neutron performs pre-commit validation prior returning to the API client which can take longer than normal client/server interactions.

`rescuing_network`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

Neutron network UUID or name for booting the ramdisk for rescue mode. This is not the network that the rescue ramdisk will use post-boot the tenant network is used for that. Required for neutron network interface, if rescue mode will be used. It is not used for the flat or noop network interfaces. If a name is provided, it must be unique among all networks or rescue will fail.

`rescuing_network_security_groups`

Type
list

Default
[]

Mutable
This option can be changed without restarting.

List of Neutron Security Group UUIDs to be applied during the node rescue process. Optional for the neutron network interface and not used for the flat or noop network interfaces. If not specified, the default security group is used.

`retriable_status_codes`

Type
list

Default
<None>

List of retriable HTTP status codes that should be retried. If not set default to [503]

`retries`

Type
integer

Default
3

Mutable
This option can be changed without restarting.

DEPRECATED: Client retries in the case of a failed request.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
Replaced by status_code_retries and status_code_retry_delay.

`service_name`

Type
string

Default
<None>

The default service_name for endpoint URL discovery.

`service_type`

Type
string

Default
network

The default service_type for endpoint URL discovery.

`servicing_network`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

Neutron network UUID or name for booting the ramdisk for service mode. Required for neutron network interface, if service mode will be used. It is not used for the flat or noop network interfaces. If a name is provided, it must be unique among all networks or service will fail.

`servicing_network_security_groups`

Type
list

Default
[]

Mutable
This option can be changed without restarting.

List of Neutron Security Group UUIDs to be applied during the node service process. Optional for the neutron network interface and not used for the flat or noop network interfaces. If not specified, the default security group is used.

`split_loggers`

Type
boolean

Default
False

Log requests to multiple loggers.

`status_code_retries`

Type
integer

Default
<None>

The maximum number of retries that should be attempted for retriable HTTP status codes.

`status_code_retry_delay`

Type
floating point

Default
<None>

Delay (in seconds) between two retries for retriable status codes. If not set, exponential retry starting with 0.5 seconds up to a maximum of 60 seconds is used.

`system_scope`

Type
unknown type

Default
<None>

Scope for system operations

`tenant_id`

Type
unknown type

Default
<None>

Tenant ID

`tenant_name`

Type
unknown type

Default
<None>

Tenant Name

`timeout`

Type
integer

Default
<None>

Timeout value for http requests

`trust_id`

Type
unknown type

Default
<None>

ID of the trust to use as a trustee use

`user_domain_id`

Type
unknown type

Default
<None>

Users domain id

`user_domain_name`

Type
unknown type

Default
<None>

Users domain name

user_id

Type
unknown type

Default
<None>

User id

username

Type
unknown type

Default
<None>

Username

Table 51: Deprecated Variations

Group	Name
neutron	user-name
neutron	user_name

valid_interfaces

Type
list

Default
['internal', 'public']

List of interfaces, in order of preference, for endpoint URL.

version

Type
string

Default
<None>

Minimum Major API version within a given Major API version for endpoint URL discovery. Mutually exclusive with min_version and max_version

nfs

base_url

Type
string

Default
<None>

Base URL for NFS shares (e.g., nfs://server/export). Required for NFS virtual media support. Must be used together with share_path.

`share_path`

Type
string

Default
<None>

Local mount point of the NFS share where images will be placed. Required for NFS virtual media support. Must be used together with base_url.

nova

`auth_url`

Type
unknown type

Default
<None>

Authentication URL

`auth_type`

Type
unknown type

Default
<None>

Authentication type to load

Table 52: Deprecated Variations

Group	Name
nova	auth_plugin

`cafile`

Type
string

Default
<None>

PEM encoded Certificate Authority to use when verifying HTTPs connections.

`certfile`

Type
string

Default
<None>

PEM encoded client certificate cert file

`collect_timing`

Type
boolean

Default
`False`

Collect per-API call timing information.

`connect_retries`

Type
integer

Default
<None>

The maximum number of retries that should be attempted for connection errors.

`connect_retry_delay`

Type
floating point

Default
<None>

Delay (in seconds) between two retries for connection errors. If not set, exponential retry starting with 0.5 seconds up to a maximum of 60 seconds is used.

`default_domain_id`

Type
unknown type

Default
<None>

Optional domain ID to use with v3 and v2 parameters. It will be used for both the user and project domain in v3 and ignored in v2 authentication.

`default_domain_name`

Type
unknown type

Default
<None>

Optional domain name to use with v3 API and v2 parameters. It will be used for both the user and project domain in v3 and ignored in v2 authentication.

`domain_id`

Type
unknown type

Default
<None>

Domain ID to scope to

`domain_name`

Type
unknown type

Default
<None>

Domain name to scope to

`endpoint_override`

Type
string

Default
<None>

Always use this endpoint URL for requests for this client. NOTE: The unversioned endpoint should be specified here; to request a particular API version, use the *version*, *min-version*, and/or *max-version* options.

`insecure`

Type
boolean

Default
`False`

Verify HTTPS connections.

`keyfile`

Type
string

Default
<None>

PEM encoded client certificate key file

`max_version`

Type
string

Default
<None>

The maximum major version of a given API, intended to be used as the upper bound of a range with min_version. Mutually exclusive with version.

`min_version`

Type
string

Default
<None>

The minimum major version of a given API, intended to be used as the lower bound of a range with max_version. Mutually exclusive with version. If min_version is given with no max_version it is as if max version is latest.

password

Type
unknown type

Default
<None>

Users password

project_domain_id

Type
unknown type

Default
<None>

Domain ID containing project

project_domain_name

Type
unknown type

Default
<None>

Domain name containing project

project_id

Type
unknown type

Default
<None>

Project ID to scope to

Table 53: Deprecated Variations

Group	Name
nova	tenant-id
nova	tenant_id

project_name

Type
unknown type

Default
<None>

Project name to scope to

Table 54: Deprecated Variations

Group	Name
nova	tenant-name
nova	tenant_name

`region_name`

Type
string

Default
<None>

The default region_name for endpoint URL discovery.

`retriable_status_codes`

Type
list

Default
<None>

List of retriable HTTP status codes that should be retried. If not set default to [503]

`send_power_notifications`

Type
boolean

Default
True

Mutable
This option can be changed without restarting.

When set to True, it will enable the support for power state change callbacks to nova. This option should be set to False in deployments that do not have the openstack compute service.

`service_name`

Type
string

Default
<None>

The default service_name for endpoint URL discovery.

`service_type`

Type
string

Default
compute

The default service_type for endpoint URL discovery.

`split_loggers`

Type
boolean

Default
`False`

Log requests to multiple loggers.

`status_code_retries`

Type
integer

Default
<None>

The maximum number of retries that should be attempted for retriable HTTP status codes.

`status_code_retry_delay`

Type
floating point

Default
<None>

Delay (in seconds) between two retries for retriable status codes. If not set, exponential retry starting with 0.5 seconds up to a maximum of 60 seconds is used.

`system_scope`

Type
unknown type

Default
<None>

Scope for system operations

`tenant_id`

Type
unknown type

Default
<None>

Tenant ID

`tenant_name`

Type
unknown type

Default
<None>

Tenant Name

`timeout`

Type
integer

Default
<None>

Timeout value for http requests

`trust_id`

Type
unknown type

Default
<None>

ID of the trust to use as a trustee use

`user_domain_id`

Type
unknown type

Default
<None>

Users domain id

`user_domain_name`

Type
unknown type

Default
<None>

Users domain name

`user_id`

Type
unknown type

Default
<None>

User id

`username`

Type
unknown type

Default
<None>

Username

Table 55: Deprecated Variations

Group	Name
nova	user-name
nova	user_name

`valid_interfaces`

Type
list

Default
`['internal', 'public']`

List of interfaces, in order of preference, for endpoint URL.

`version`

Type
string

Default
<None>

Minimum Major API version within a given Major API version for endpoint URL discovery. Mutually exclusive with min_version and max_version

oci

`secure_cdn_registries`

Type
list

Default
`['registry.redhat.io', 'registry.access.redhat.com', 'docker.io', 'registry-1.docker.io']`

An option which signals to the OCI Container Registry client which remote endpoints are fronted by Content Distribution Networks which we may receive redirects to in order to download the requested artifacts, where the OCI client should go ahead and issue the download request with authentication headers before being asked by the remote server for user authentication.

`authentication_config`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

An option which allows pre-shared authorization keys to be utilized by the Ironic service to facilitate authentication with remote image registries which may require authentication for all interactions. Ironic will utilize these credentials to access general artifacts, but Ironic will *also* prefer

user credentials, if supplied, for disk images. This file is in the same format utilized in the container ecosystem for the same purpose. Structured as a JSON document with an `auths` key, with remote registry domain FQDNs as keys, and a nested `auth` key within that value which holds the actual pre-shared secret. Ironic does not cache the contents of this file at launch, and the file can be updated as Ironic operates in the event pre-shared tokens need to be regenerated.

permit_fallback_to_http_transport

Type
boolean

Default
`False`

Security-Insecure: By default, the OCI client code expects all OCI registry interactions to take place utilizing HTTPS as the underlying transport mechanism to communicate with the remote registry. In reality, that is not always the case in testing environments, and as such this setting may be utilized to allow the internal OCI mechanism to fallback to HTTP if the remote endpoint lacks support for HTTPS.

oslo_concurrency

disable_process_locking

Type
boolean

Default
`False`

Enables or disables inter-process locks.

lock_path

Type
string

Default
<None>

Directory to use for lock files. For security, the specified directory should only be writable by the user running the processes that need locking. Defaults to environment variable OSLO_LOCK_PATH. If external locks are used, a lock path must be set.

oslo_messaging_kafka

kafka_max_fetch_bytes

Type
integer

Default
`1048576`

Max fetch bytes of Kafka consumer

`kafka_consumer_timeout`

Type
floating point

Default
`1.0`

Default timeout(s) for Kafka consumers

`consumer_group`

Type
string

Default
`oslo_messaging_consumer`

Group id for Kafka consumer. Consumers in one group will coordinate message consumption

`producer_batch_timeout`

Type
floating point

Default
`0.0`

Upper bound on the delay for KafkaProducer batching in seconds

`producer_batch_size`

Type
integer

Default
`16384`

Size of batch for the producer async send

`compression_codec`

Type
string

Default
`none`

Valid Values
none, gzip, snappy, lz4, zstd

The compression codec for all data generated by the producer. If not set, compression will not be used. Note that the allowed values of this depend on the kafka version

`enable_auto_commit`

Type
boolean

Default
`False`

Enable asynchronous consumer commits

`max_poll_records`

Type
integer

Default
500

The maximum number of records returned in a poll call

`security_protocol`

Type
string

Default
PLAINTEXT

Valid Values
PLAINTEXT, SASL_PLAINTEXT, SSL, SASL_SSL

Protocol used to communicate with brokers

`sasl_mechanism`

Type
string

Default
PLAIN

Mechanism when security protocol is SASL

`ssl_cafile`

Type
string

Default
''

CA certificate PEM file used to verify the server certificate

`ssl_client_cert_file`

Type
string

Default
''

Client certificate PEM file used for authentication.

`ssl_client_key_file`

Type
string

Default
''

Client key PEM file used for authentication.

ssl_client_key_password

Type
string

Default
''

Client key password file used for authentication.

oslo_messaging_notifications

driver

Type
multi-valued

Default
''

The Drivers(s) to handle sending notifications. Possible values are messaging, messagingv2, routing, log, test, noop

transport_url

Type
string

Default
<None>

A URL representing the messaging driver to use for notifications. If not set, we fall back to the same configuration used for RPC.

topics

Type
list

Default
['notifications']

AMQP topic used for OpenStack notifications.

retry

Type
integer

Default
-1

The maximum number of attempts to re-send a notification message which failed to be delivered due to a recoverable error. 0 - No retry, -1 - indefinite

oslo_messaging_rabbit

amqp_durable_queues

Type
boolean

Default
`False`

Use durable queues in AMQP. If rabbit_quorum_queue is enabled, queues will be durable and this value will be ignored.

amqp_auto_delete

Type
boolean

Default
`False`

Auto-delete queues in AMQP.

rpc_conn_pool_size

Type
integer

Default
`30`

Minimum Value
1

Size of RPC connection pool.

conn_pool_min_size

Type
integer

Default
`2`

The pool size limit for connections expiration policy

conn_pool_ttl

Type
integer

Default
`1200`

The time-to-live in sec of idle connections in the pool

ssl

Type
boolean

Default
`False`

Connect over SSL.

`ssl_version`

Type
string

Default
`''`

SSL version to use (valid only if SSL enabled). Valid values are TLSv1 and SSLv23. SSLv2, SSLv3, TLSv1_1, and TLSv1_2 may be available on some distributions.

`ssl_key_file`

Type
string

Default
`''`

SSL key file (valid only if SSL enabled).

`ssl_cert_file`

Type
string

Default
`''`

SSL cert file (valid only if SSL enabled).

`ssl_ca_file`

Type
string

Default
`''`

SSL certification authority file (valid only if SSL enabled).

`ssl_enforce_fips_mode`

Type
boolean

Default
`False`

Global toggle for enforcing the OpenSSL FIPS mode. This feature requires Python support. This is available in Python 3.9 in all environments and may have been backported to older Python versions on select environments. If the Python executable used does not support OpenSSL FIPS mode, an exception will be raised.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
FIPS_mode_set API was removed in OpenSSL 3.0.0. This option has no effect now.

`heartbeat_in_pthread`

Type
boolean

Default
`False`

(DEPRECATED) It is recommend not to use this option anymore. Run the health check heartbeat thread through a native python thread by default. If this option is equal to False then the health check heartbeat will inherit the execution model from the parent process. For example if the parent process has monkey patched the stdlib by using eventlet/greenlet then the heartbeat will be run through a green thread. This option should be set to True only for the wsgi services.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
The option is related to Eventlet which will be removed. In addition this has never worked as expected with services using eventlet for core service framework.

`kombu_reconnect_delay`

Type
floating point

Default
`1.0`

Minimum Value
0.0

Maximum Value
4.5

How long to wait (in seconds) before reconnecting in response to an AMQP consumer cancel notification.

`kombu_reconnect_splay`

Type
floating point

Default
`0.0`

Minimum Value
0.0

Random time to wait for when reconnecting in response to an AMQP consumer cancel notification.

`kombu_compression`

Type
string

Default
<None>

EXPERIMENTAL: Possible values are: gzip, bz2. If not set compression will not be used. This option may not be available in future versions.

`kombu_missing_consumer_retry_timeout`

Type
integer

Default
60

How long to wait a missing client before abandoning to send it its replies. This value should not be longer than rpc_response_timeout.

Table 56: Deprecated Variations

Group	Name
oslo_messaging_rabbit	kombu_reconnect_timeout

`kombu_failover_strategy`

Type
string

Default
round-robin

Valid Values
round-robin, shuffle

Determines how the next RabbitMQ node is chosen in case the one we are currently connected to becomes unavailable. Takes effect only if more than one RabbitMQ node is provided in config.

`rabbit_login_method`

Type
string

Default
AMQPLAIN

Valid Values
PLAIN, AMQPLAIN, EXTERNAL, RABBIT-CR-DEMO

The RabbitMQ login method.

`rabbit_retry_interval`

Type
integer

Default
1

Minimum Value
1

How frequently to retry connecting with RabbitMQ.

`rabbit_retry_backoff`

Type
integer

Default
2

Minimum Value
0

How long to backoff for between retries when connecting to RabbitMQ.

`rabbit_interval_max`

Type
integer

Default
30

Minimum Value
1

Maximum interval of RabbitMQ connection retries.

`rabbit_ha_queues`

Type
boolean

Default
`False`

Try to use HA queues in RabbitMQ (x-ha-policy: all). If you change this option, you must wipe the RabbitMQ database. In RabbitMQ 3.0, queue mirroring is no longer controlled by the x-ha-policy argument when declaring a queue. If you just want to make sure that all queues (except those with auto-generated names) are mirrored across all nodes, run: rabbitmqctl set_policy HA ^(?!amq.).* {ha-mode: all}

`rabbit_quorum_queue`

Type
boolean

Default
`False`

Use quorum queues in RabbitMQ (x-queue-type: quorum). The quorum queue is a modern queue type for RabbitMQ implementing a durable, replicated FIFO queue based on the Raft consensus algorithm. It is available as of RabbitMQ 3.8.0. If set this option will conflict with the HA queues (`rabbit_ha_queues`) aka mirrored queues, in other words the HA queues should be disabled. Quorum queues are also durable by default so the amqp_durable_queues option is ignored when this option is enabled.

rabbit_transient_quorum_queue

Type
boolean

Default
`False`

Use quorum queues for transients queues in RabbitMQ. Enabling this option will then make sure those queues are also using quorum kind of rabbit queues, which are HA by default.

rabbit_quorum_delivery_limit

Type
integer

Default
`0`

Each time a message is redelivered to a consumer, a counter is incremented. Once the redelivery count exceeds the delivery limit the message gets dropped or dead-lettered (if a DLX exchange has been configured) Used only when rabbit_quorum_queue is enabled, Default 0 which means dont set a limit.

rabbit_quorum_max_memory_length

Type
integer

Default
`0`

By default all messages are maintained in memory if a quorum queue grows in length it can put memory pressure on a cluster. This option can limit the number of messages in the quorum queue. Used only when rabbit_quorum_queue is enabled, Default 0 which means dont set a limit.

rabbit_quorum_max_memory_bytes

Type
integer

Default
`0`

By default all messages are maintained in memory if a quorum queue grows in length it can put memory pressure on a cluster. This option can limit the number of memory bytes used by the quorum queue. Used only when rabbit_quorum_queue is enabled, Default 0 which means dont set a limit.

rabbit_transient_queues_ttl

Type
integer

Default
1800

Minimum Value
0

Positive integer representing duration in seconds for queue TTL (x-expires). Queues which are unused for the duration of the TTL are automatically deleted. The parameter affects only reply and fanout queues. Setting 0 as value will disable the x-expires. If doing so, make sure you have a rabbitmq policy to delete the queues or you deployment will create an infinite number of queue over time.In case rabbit_stream_fanout is set to True, this option will control data retention policy (x-max-age) for messages in the fanout queue rather then the queue duration itself. So the oldest data in the stream queue will be discarded from it once reaching TTL Setting to 0 will disable x-max-age for stream which make stream grow indefinitely filling up the diskspace

`rabbit_qos_prefetch_count`

Type
integer

Default
0

Specifies the number of messages to prefetch. Setting to zero allows unlimited messages.

`heartbeat_timeout_threshold`

Type
integer

Default
60

Number of seconds after which the Rabbit broker is considered down if heartbeats keep-alive fails (0 disables heartbeat).

`heartbeat_rate`

Type
integer

Default
3

How often times during the heartbeat_timeout_threshold we check the heartbeat.

`direct_mandatory_flag`

Type
boolean

Default
True

(DEPRECATED) Enable/Disable the RabbitMQ mandatory flag for direct send. The direct send is used as reply, so the MessageUndeliverable exception is raised in case the client queue does not

exist.MessageUndeliverable exception will be used to loop for a timeout to lets a chance to sender to recover.This flag is deprecated and it will not be possible to deactivate this functionality anymore

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
Mandatory flag no longer deactivable.

`enable_cancel_on_failover`

Type
boolean

Default
`False`

Enable x-cancel-on-ha-failover flag so that rabbitmq server will cancel and notify consumerswhen queue is down

`use_queue_manager`

Type
boolean

Default
`False`

Should we use consistant queue names or random ones

`hostname`

Type
string

Default
`node1.example.com`

This option has a sample default set, which means that its actual default value may vary from the one documented above.

Hostname used by queue manager. Defaults to the value returned by socket.gethostname().

`processname`

Type
string

Default
`nova-api`

This option has a sample default set, which means that its actual default value may vary from the one documented above.

Process name used by queue manager

rabbit_stream_fanout

Type
boolean

Default
`False`

Use stream queues in RabbitMQ (x-queue-type: stream). Streams are a new persistent and replicated data structure (queue type) in RabbitMQ which models an append-only log with non-destructive consumer semantics. It is available as of RabbitMQ 3.9.0. If set this option will replace all fanout queues with only one stream queue.

oslo_middleware

enable_proxy_headers_parsing

Type
boolean

Default
`False`

Whether the application is behind a proxy or not. This determines if the middleware should parse the headers or not.

oslo_policy

enforce_scope

Type
boolean

Default
`True`

This option controls whether or not to enforce scope when evaluating policies. If `True`, the scope of the token used in the request is compared to the `scope_types` of the policy being enforced. If the scopes do not match, an `InvalidScope` exception will be raised. If `False`, a message will be logged informing operators that policies are being invoked with mismatching scope.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
This configuration was added temporarily to facilitate a smooth transition to the new RBAC. OpenStack will always enforce scope checks. This configuration option is deprecated and will be removed in the 2025.2 cycle.

enforce_new_defaults

Type
boolean

Default
`True`

This option controls whether or not to use old deprecated defaults when evaluating policies. If `True`, the old deprecated defaults are not going to be evaluated. This means if any existing token is allowed for old defaults but is disallowed for new defaults, it will be disallowed. It is encouraged to enable this flag along with the `enforce_scope` flag so that you can get the benefits of new defaults and `scope_type` together. If `False`, the deprecated policy check string is logically ORd with the new policy check string, allowing for a graceful upgrade experience between releases with new policies, which is the default behavior.

`policy_file`

Type
string

Default
`policy.yaml`

The relative or absolute path of a file that maps roles to permissions for a given service. Relative paths must be specified in relation to the configuration file setting this option.

`policy_default_rule`

Type
string

Default
`default`

Default rule. Enforced when a requested rule is not found.

`policy_dirs`

Type
multi-valued

Default
`policy.d`

Directories where policy configuration files are stored. They can be relative to any directory in the search path defined by the config_dir option, or absolute paths. The file defined by policy_file must exist for these directories to be searched. Missing or empty directories are ignored.

`remote_content_type`

Type
string

Default
`application/x-www-form-urlencoded`

Valid Values
application/x-www-form-urlencoded, application/json

Content Type to send and receive data for REST based policy check

`remote_ssl_verify_server_crt`

Type
boolean

Default
`False`

server identity verification for REST based policy check

`remote_ssl_ca_crt_file`

Type
string

Default
<None>

Absolute path to ca cert file for REST based policy check

`remote_ssl_client_crt_file`

Type
string

Default
<None>

Absolute path to client cert for REST based policy check

`remote_ssl_client_key_file`

Type
string

Default
<None>

Absolute path client key file REST based policy check

`remote_timeout`

Type
floating point

Default
`60`

Minimum Value
0

Timeout in seconds for REST based policy check

oslo_reports

`log_dir`

Type
string

Default
<None>

Path to a log directory where to create a file

`file_event_handler`

Type
string

Default
<None>

The path to a file to watch for changes to trigger the reports, instead of signals. Setting this option disables the signal trigger for the reports. If application is running as a WSGI application it is recommended to use this instead of signals.

`file_event_handler_interval`

Type
integer

Default
1

How many seconds to wait between polls when file_event_handler is set

oslo_versionedobjects

`fatal_exception_format_errors`

Type
boolean

Default
`False`

Make exception message format errors fatal

profiler

`enabled`

Type
boolean

Default
`False`

Enable the profiling for all services on this node.

Default value is False (fully disable the profiling feature).

Possible values:

- True: Enables the feature
- False: Disables the feature. The profiling cannot be started via this project operations. If the profiling is triggered by another project, this project part will be empty.

Table 57: Deprecated Variations

Group	Name
profiler	profiler_enabled

`trace_sqlalchemy`

Type
boolean

Default
`False`

Enable SQL requests profiling in services.

Default value is False (SQL requests wont be traced).

Possible values:

- True: Enables SQL requests profiling. Each SQL query will be part of the trace and can the be analyzed by how much time was spent for that.
- False: Disables SQL requests profiling. The spent time is only shown on a higher level of operations. Single SQL queries cannot be analyzed this way.

`trace_requests`

Type
boolean

Default
`False`

Enable python requests package profiling.

Supported drivers: jaeger+otlp

Default value is False.

Possible values:

- True: Enables requests profiling.
- False: Disables requests profiling.

`hmac_keys`

Type
string

Default
`SECRET_KEY`

Secret key(s) to use for encrypting context data for performance profiling.

This string value should have the following format: <key1>[,<key2>,<keyn>], where each key is some random string. A user who triggers the profiling via the REST API has to set one of these keys in the headers of the REST API call to include profiling results of this node for this particular project.

Both enabled flag and hmac_keys config options should be set to enable profiling. Also, to generate correct profiling information across all services at least one key needs to be consistent between OpenStack projects. This ensures it can be used from client side to generate the trace, containing information from all possible resources.

`connection_string`

Type
string

Default
`messaging://`

Connection string for a notifier backend.

Default value is `messaging://` which sets the notifier to oslo_messaging.

Examples of possible values:

- `messaging://` - use oslo_messaging driver for sending spans.
- `redis://127.0.0.1:6379` - use redis driver for sending spans.
- `mongodb://127.0.0.1:27017` - use mongodb driver for sending spans.
- `elasticsearch://127.0.0.1:9200` - use elasticsearch driver for sending spans.
- `jaeger://127.0.0.1:6831` - use jaeger tracing as driver for sending spans.

`es_doc_type`

Type
string

Default
`notification`

Document type for notification indexing in elasticsearch.

`es_scroll_time`

Type
string

Default
`2m`

This parameter is a time value parameter (for example: es_scroll_time=2m), indicating for how long the nodes that participate in the search will maintain relevant resources in order to continue and support it.

`es_scroll_size`

Type
integer

Default
`10000`

Elasticsearch splits large requests in batches. This parameter defines maximum size of each batch (for example: es_scroll_size=10000).

`socket_timeout`

Type
floating point

Default
`0.1`

Redissentinel provides a timeout option on the connections. This parameter defines that timeout (for example: socket_timeout=0.1).

`sentinel_service_name`

Type
string

Default
`mymaster`

Redissentinel uses a service name to identify a master redis service. This parameter defines the name (for example: `sentinal_service_name=mymaster`).

`filter_error_trace`

Type
boolean

Default
`False`

Enable filter traces that contain error/exception to a separated place.

Default value is set to False.

Possible values:

- True: Enable filter traces that contain error/exception.
- False: Disable the filter.

profiler_jaeger

`service_name_prefix`

Type
string

Default
<None>

Set service name prefix to Jaeger service name.

`process_tags`

Type
dict

Default
`{}`

Set process tracer tags.

profiler_otlp

service_name_prefix

Type
string

Default
<None>

Set service name prefix to OTLP exporters.

pxe

kernel_append_params

Type
string

Default
`nofb vga=normal`

Mutable
This option can be changed without restarting.

Additional append parameters for baremetal PXE boot.

Table 58: Deprecated Variations

Group	Name
pxe	pxe_append_params

default_ephemeral_format

Type
string

Default
`ext4`

Mutable
This option can be changed without restarting.

Default file system format for ephemeral partition, if one is created.

images_path

Type
string

Default
`/var/lib/ironic/images/`

On the ironic-conductor node, directory where images are stored on disk.

instance_master_path

Type
string

Default
`/var/lib/ironic/master_images`

On the ironic-conductor node, directory where master instance images are stored on disk. Setting to the empty string disables image caching.

`image_cache_size`

Type
integer

Default
`20480`

Maximum size (in MiB) of cache for master images, including those in use.

`image_cache_ttl`

Type
integer

Default
`10080`

Maximum TTL (in minutes) for old master images in cache.

`pxe_config_template`

Type
string

Default
`$pybasedir/drivers/modules/pxe_config.template`

Mutable
This option can be changed without restarting.

On ironic-conductor node, template file for PXE loader configuration.

`ipxe_config_template`

Type
string

Default
`$pybasedir/drivers/modules/ipxe_config.template`

Mutable
This option can be changed without restarting.

On ironic-conductor node, template file for iPXE operations.

`uefi_pxe_config_template`

Type
string

Default
`$pybasedir/drivers/modules/pxe_grub_config.template`

Mutable
This option can be changed without restarting.

On ironic-conductor node, template file for PXE configuration for UEFI boot loader. Generally this is used for GRUB specific templates.

`pxe_config_template_by_arch`

Type
dict

Default
{}

Mutable
This option can be changed without restarting.

On ironic-conductor node, template file for PXE configuration per node architecture. For example: aarch64:/opt/share/grubaa64_pxe_config.template

`tftp_server`

Type
string

Default
$my_ip

IP address of ironic-conductor nodes TFTP server.

`tftp_root`

Type
string

Default
/tftpboot

ironic-conductor nodes TFTP root path. The ironic-conductor must have read/write access to this path.

`tftp_master_path`

Type
string

Default
/tftpboot/master_images

On ironic-conductor node, directory where master TFTP images are stored on disk. Setting to the empty string disables image caching.

`dir_permission`

Type
integer

Default
<None>

The permission that will be applied to the TFTP folders upon creation. This should be set to the permission such that the tftpserver has access to read the contents of the configured TFTP folder. This setting is only required when the operating systems umask is restrictive such that ironic-conductor is creating files that cannot be read by the TFTP server. Setting to <None> will result in

the operating systems umask to be utilized for the creation of new tftp folders. The system default umask is masked out on the specified value. It is required that an octal representation is specified. For example: 0o755

`file_permission`

Type
integer

Default
`420`

The permission which is used on files created as part of configuration and setup of file assets for PXE based operations. Defaults to a value of 0o644. This value must be specified as an octal representation. For example: 0o644

`pxe_bootfile_name`

Type
string

Default
`pxelinux.0`

Bootfile DHCP parameter.

`pxe_config_subdir`

Type
string

Default
`pxelinux.cfg`

Directory in which to create symbolic links which represent the MAC or IP address of the ports on a node and allow boot loaders to load the PXE file for the node. This directory name is relative to the PXE or iPXE folders.

`uefi_pxe_bootfile_name`

Type
string

Default
`bootx64.efi`

Bootfile DHCP parameter for UEFI boot mode.

`ipxe_bootfile_name`

Type
string

Default
`undionly.kpxe`

Bootfile DHCP parameter.

`uefi_ipxe_bootfile_name`

Type
string

Default
`snponly.efi`

Bootfile DHCP parameter for UEFI boot mode. If you experience problems with booting using it, try ipxe.efi.

`pxe_bootfile_name_by_arch`

Type
dict

Default
`{}`

Bootfile DHCP parameter per node architecture. For example: aarch64:grubaa64.efi

`ipxe_bootfile_name_by_arch`

Type
dict

Default
`{}`

Bootfile DHCP parameter per node architecture. For example: aarch64:ipxe_aa64.efi

`ipxe_boot_script`

Type
string

Default
`$pybasedir/drivers/modules/boot.ipxe`

On ironic-conductor node, the path to the main iPXE script file.

`ipxe_fallback_script`

Type
string

Default
<None>

File name (e.g. inspector.ipxe) of an iPXE script to fall back to when booting to a MAC-specific script fails. When not set, booting will fail in this case.

`ipxe_timeout`

Type
integer

Default
`0`

Timeout value (in seconds) for downloading an image via iPXE. Timeout is disabled when this value is 0

`boot_retry_timeout`

Type
integer

Default
<None>

Minimum Value
60

Timeout (in seconds) after which PXE boot should be retried. Must be less than [conductor]deploy_callback_timeout. Disabled by default.

`boot_retry_check_interval`

Type
integer

Default
90

Minimum Value
1

Interval (in seconds) between periodic checks on PXE boot retry. Has no effect if boot_retry_timeout is not set.

`ip_version`

Type
string

Default
4

Valid Values
4, 6

Mutable
This option can be changed without restarting.

The IP version that will be used for PXE booting. Defaults to 4. This option has been a no-op for in-treedrivers since the Ussuri development cycle.

Possible values

4
IPv4

6
IPv6

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

`ipxe_use_swift`

Type
boolean

Default
`False`

Mutable
This option can be changed without restarting.

Download deploy and rescue images directly from swift using temporary URLs. If set to false (default), images are downloaded to the ironic-conductor node and served over its local HTTP server. Applicable only when ipxe compatible boot interface is used.

`enable_netboot_fallback`

Type
boolean

Default
`False`

Mutable
This option can be changed without restarting.

If True, generate a PXE environment even for nodes that use local boot. This is useful when the driver cannot switch nodes to local boot, e.g. with SNMP or with Redfish on machines that cannot do persistent boot. Mostly useful for standalone ironic since Neutron will prevent incorrect PXE boot.

`loader_file_paths`

Type
unknown type

Default
`{}`

Dictionary describing the bootloaders to load into conductor PXE/iPXE boot folders values from the host operating system. Formatted as key of destination file name, and value of a full path to a file to be copied. File assets will have [pxe]file_permission applied, if set. If used, the file names should match established bootloader configuration settings for bootloaders. Use example: ipxe.efi:/usr/share/ipxe/ipxe-snponly-x86_64.efi,undionly.kpxe:/usr/share/ipxe/undionly.kpxe

`initial_grub_template`

Type
string

Default
`$pybasedir/drivers/modules/initial_grub_cfg.template`

On ironic-conductor node, the path to the initial grubconfiguration template for grub network boot.

pxe_filter

dhcp_hostsdir

Type
string

Default
<None>

The MAC address cache directory, exposed to dnsmasq.This directory is expected to be in exclusive control of the driver but must be purged by the operator. Required.

supported_inspect_interfaces

Type
list

Default
['agent']

Mutable
This option can be changed without restarting.

List of inspect interfaces that will be considered by the PXE filter. Only nodes with these interfaces will be enabled.

sync_period

Type
integer

Default
45

Mutable
This option can be changed without restarting.

Period (in seconds) between synchronizing the state of dnsmasq with the database.

redfish

connection_attempts

Type
integer

Default
5

Minimum Value
1

Maximum number of attempts to try to connect to Redfish

connection_retry_interval

Type
integer

Default
4

Minimum Value
1

Number of seconds to wait between attempts to connect to Redfish

`connect_timeout`

Type
integer

Default
30

Minimum Value
1

TCP connection timeout in seconds when connecting to the Redfish BMC. This controls how long to wait for the initial TCP connection to be established. A lower value allows faster failure when a BMC is unreachable, while the default read timeout (60 seconds) still applies for waiting on response data from responsive BMCs.

`connection_cache_size`

Type
integer

Default
1000

Minimum Value
0

Maximum Redfish client connection cache size. Redfish driver would strive to reuse authenticated BMC connections (obtained through Redfish Session Service). This option caps the maximum number of connections to maintain. The value of *0* disables client connection caching completely.

`auth_type`

Type
string

Default
auto

Valid Values
basic, session, auto

Redfish HTTP client authentication method.

Possible values

basic
Use HTTP basic authentication

session
Use HTTP session authentication

auto
Try HTTP session authentication first, fall back to basic HTTP authentication

`use_swift`

Type
boolean

Default
`False`

Mutable
This option can be changed without restarting.

Upload generated ISO images for virtual media boot to Swift, then pass temporary URL to BMC for booting the node. If set to false, images are placed on the ironic-conductor node and served over its local HTTP server.

`swift_container`

Type
string

Default
`ironic_redfish_container`

Mutable
This option can be changed without restarting.

The Swift container to store Redfish driver data. Applies only when *use_swift* is enabled.

`swift_object_expiry_timeout`

Type
integer

Default
`900`

Mutable
This option can be changed without restarting.

Amount of time in seconds for Swift objects to auto-expire. Applies only when *use_swift* is enabled.

`kernel_append_params`

Type
string

Default
`nofb vga=normal`

Mutable
This option can be changed without restarting.

Additional kernel parameters to pass down to the instance kernel. These parameters can be consumed by the kernel or by the applications by reading /proc/cmdline. Mind severe cmdline size limit! Can be overridden by *instance_info/kernel_append_params* property.

`file_permission`

Type
integer

Default
`420`

File permission for swift-less image hosting with the octal permission representation of file access permissions. This setting defaults to `644`, or as the octal number `0o644` in Python. This setting must be set to the octal number representation, meaning starting with `0o`.

`firmware_update_status_interval`

Type
integer

Default
`60`

Minimum Value
0

Number of seconds to wait between checking for completed firmware update tasks

`firmware_update_fail_interval`

Type
integer

Default
`60`

Minimum Value
0

Number of seconds to wait between checking for failed firmware update tasks

`firmware_update_wait_unresponsive_bmc`

Type
integer

Default
`0`

Minimum Value
0

Number of seconds to wait before proceeding with the reboot to finish the BMC firmware update step

`firmware_update_required_successes`

Type
integer

Default
3

Minimum Value
0

Number of successful responses required to consider post-upgrade BMC validation a success. Set to 0 to disable post-upgrade validation entirely.

`firmware_update_validation_interval`

Type
integer

Default
`30`

Minimum Value
0

Timeout (in seconds) to wait between validation attempts. Set to 0 for rapid succession retries with no delay.

`firmware_update_resource_validation_timeout`

Type
integer

Default
`480`

Minimum Value
0

Timeout (in seconds) to wait for BMC resources (System, Manager, NetworkAdapters) to become stable and consistently available after firmware update. Set to 0 to disable post-upgrade validation entirely.

`firmware_update_bmc_timeout`

Type
integer

Default
`300`

Minimum Value
0

Timeout (in seconds) for BMC firmware updates. BMC firmware updates may need extended time to handle BMC transitional states during the firmware update process.

`firmware_update_reboot_delay`

Type
integer

Default
`300`

Minimum Value
0

Default wait time (in seconds) for component-specific firmware update operations. Used for: BIOS firmware update wait before reboot, BMC firmware version check timeout, and NIC firmware task completion timeout.

`firmware_update_bmc_version_check_interval`

Type
integer

Default
`30`

Minimum Value
0

Interval (in seconds) for checking BMC firmware version after BMC firmware update. Used to verify if BMC firmware has been successfully applied.

`firmware_update_nic_starting_wait`

Type
integer

Default
`30`

Minimum Value
0

Time (in seconds) to wait for a NIC firmware update task to progress beyond the STARTING state before triggering a reboot. Some NICs need a reboot to start applying firmware, while others can begin immediately. This timeout helps determine which behavior the hardware exhibits.

`firmware_update_overall_timeout`

Type
integer

Default
`7200`

Minimum Value
0

Maximum time (in seconds) allowed for the entire firmware update operation to complete. This provides a safety net for firmware updates that get stuck. Set to 0 to disable this timeout (not recommended). Default is 7200 seconds (2 hours).

`firmware_source`

Type
string

Default
`http`

Valid Values
http, local, swift

Mutable
This option can be changed without restarting.

Specifies how firmware image should be served. Whether from its original location using the firmware source URL directly, or should serve it from ironics Swift or HTTP server.

Possible values

http
If firmware source URL is also HTTP, then serve from original location, otherwise copy to ironics HTTP server. Default.

local
Download from original location and server from ironics HTTP server.

swift
If firmware source URL is also Swift, serve from original location, otherwise copy to ironics Swift server.

`raid_config_status_interval`

Type
integer

Default
`60`

Minimum Value
0

Number of seconds to wait between checking for completed raid config tasks

`raid_config_fail_interval`

Type
integer

Default
`60`

Minimum Value
0

Number of seconds to wait between checking for failed raid config tasks

`boot_mode_config_timeout`

Type
integer

Default
`900`

Minimum Value
0

Number of seconds to wait for boot mode or secure boot status change to take effect after a reboot. Set to 0 to disable waiting.

`verify_ca`

Type
string

Default
<None>

The default verify_ca path when redfish_verify_ca in driver_info is missing or set to True.

`enable_verify_bmc_clock`

Type
boolean

Default
`False`

Whether to enable the automated verify step that checks and sets the BMC clock. When enabled, Ironic will automatically attempt to set the BMCs clock during node registration (in the verify phase) using Redfishs DateTime fields. This helps avoid TLS certificate issues caused by incorrect BMC time.

`default_inspection_hooks`

Type
string

Default
`validate-interfaces,ports,architecture`

A comma-separated lists of inspection hooks that are run by default for the agent inspection interface. In most cases, the operators will not modify this. The default (somewhat conservative) hooks validate interfaces in the inventory, create ports and set the nodes cpu architecture property.

`inspection_hooks`

Type
string

Default
`$default_inspection_hooks`

Comma-separated list of enabled hooks for processing pipeline when using the redfish inspection interface. The default for this is $default_inspection_hooks. Hooks can be added before or after the defaults like this: prehook,$default_hooks,posthook.

`post_boot_retry_attempts`

Type
integer

Default
6

Minimum Value
1

Maximum number of retry attempts when BMC rejects boot device changes during POST (Power-On Self-Test). Some BMCs (e.g. HPE iLO) reject boot device modifications while the system is in POST after a firmware update or reboot.

`post_boot_retry_delay`

Type
integer

Default
5

Minimum Value
1

Minimum delay in seconds between retry attempts for POST-related boot device errors. Exponential backoff is applied, starting from this value up to 6x this value.

sensor_data

send_sensor_data

Type
boolean

Default
False

Enable sending sensor data message via the notification bus.

Table 59: Deprecated Variations

Group	Name
conductor	send_sensor_data

interval

Type
integer

Default
600

Minimum Value
1

Seconds between conductor sending sensor data message via the notification bus. This was originally for consumption via ceilometer, but the data may also be consumed via a plugin like ironic-prometheus-exporter or any other message bus data collector.

Table 60: Deprecated Variations

Group	Name
conductor	send_sensor_data_interval

workers

Type
integer

Default
4

Minimum Value
1

The maximum number of workers that can be started simultaneously for send data from sensors periodic task.

Table 61: Deprecated Variations

Group	Name
conductor	send_sensor_data_workers

`wait_timeout`

Type
integer

Default
`300`

The time in seconds to wait for send sensors data periodic task to be finished before allowing periodic call to happen again. Should be less than send_sensor_data_interval value.

Table 62: Deprecated Variations

Group	Name
conductor	send_sensor_data_wait_timeout

`data_types`

Type
list

Default
`['ALL']`

List of comma separated meter types which need to be sent to Ceilometer. The default value, ALL, is a special value meaning send all the sensor data. This setting only applies to baremetal sensor data being processed through the conductor.

Table 63: Deprecated Variations

Group	Name
conductor	send_sensor_data_types

`enable_for_undeployed_nodes`

Type
boolean

Default
`False`

The default for sensor data collection is to only collect data for machines that are deployed, however operators may desire to know if there are failures in hardware that is not presently in use. When set

to true, the conductor will collect sensor information from all nodes when sensor data collection is enabled via the send_sensor_data setting.

Table 64: Deprecated Variations

Group	Name
conductor	send_sensor_data_for_undeployed_nodes

enable_for_conductor

Type
boolean

Default
`True`

If to include sensor metric data for the Conductor process itself in the message payload for sensor data which allows operators to gather instance counts of actions and states to better manage the deployment.

enable_for_nodes

Type
boolean

Default
`True`

If to transmit any sensor data for any nodes under this conductors management. This option supersedes the `send_sensor_data_for_undeployed_nodes` setting.

service_catalog

auth_url

Type
unknown type

Default
<None>

Authentication URL

auth_type

Type
unknown type

Default
<None>

Authentication type to load

Table 65: Deprecated Variations

Group	Name
service_catalog	auth_plugin

`cafile`

Type
string

Default
<None>

PEM encoded Certificate Authority to use when verifying HTTPs connections.

`certfile`

Type
string

Default
<None>

PEM encoded client certificate cert file

`collect_timing`

Type
boolean

Default
`False`

Collect per-API call timing information.

`connect_retries`

Type
integer

Default
<None>

The maximum number of retries that should be attempted for connection errors.

`connect_retry_delay`

Type
floating point

Default
<None>

Delay (in seconds) between two retries for connection errors. If not set, exponential retry starting with 0.5 seconds up to a maximum of 60 seconds is used.

`default_domain_id`

Type
unknown type

Default
<None>

Optional domain ID to use with v3 and v2 parameters. It will be used for both the user and project domain in v3 and ignored in v2 authentication.

`default_domain_name`

Type
unknown type

Default
<None>

Optional domain name to use with v3 API and v2 parameters. It will be used for both the user and project domain in v3 and ignored in v2 authentication.

`domain_id`

Type
unknown type

Default
<None>

Domain ID to scope to

`domain_name`

Type
unknown type

Default
<None>

Domain name to scope to

`endpoint_override`

Type
string

Default
<None>

Always use this endpoint URL for requests for this client. NOTE: The unversioned endpoint should be specified here; to request a particular API version, use the *version*, *min-version*, and/or *max-version* options.

`insecure`

Type
boolean

Default
`False`

Verify HTTPS connections.

`keyfile`

Type
string

Default
<None>

PEM encoded client certificate key file

`max_version`

Type
string

Default
<None>

The maximum major version of a given API, intended to be used as the upper bound of a range with min_version. Mutually exclusive with version.

`min_version`

Type
string

Default
<None>

The minimum major version of a given API, intended to be used as the lower bound of a range with max_version. Mutually exclusive with version. If min_version is given with no max_version it is as if max version is latest.

`password`

Type
unknown type

Default
<None>

Users password

`project_domain_id`

Type
unknown type

Default
<None>

Domain ID containing project

`project_domain_name`

Type
unknown type

Default
<None>

Domain name containing project

`project_id`

Type
unknown type

Default
<None>

Project ID to scope to

Table 66: Deprecated Variations

Group	Name
service_catalog	tenant-id
service_catalog	tenant_id

`project_name`

Type
unknown type

Default
<None>

Project name to scope to

Table 67: Deprecated Variations

Group	Name
service_catalog	tenant-name
service_catalog	tenant_name

`region_name`

Type
string

Default
<None>

The default region_name for endpoint URL discovery.

`retriable_status_codes`

Type
list

Default
<None>

List of retriable HTTP status codes that should be retried. If not set default to [503]

`service_name`

Type
string

Default
<None>

The default service_name for endpoint URL discovery.

`service_type`

Type
string

Default
`baremetal`

The default service_type for endpoint URL discovery.

`split_loggers`

Type
boolean

Default
`False`

Log requests to multiple loggers.

`status_code_retries`

Type
integer

Default
<None>

The maximum number of retries that should be attempted for retriable HTTP status codes.

`status_code_retry_delay`

Type
floating point

Default
<None>

Delay (in seconds) between two retries for retriable status codes. If not set, exponential retry starting with 0.5 seconds up to a maximum of 60 seconds is used.

`system_scope`

Type
unknown type

Default
<None>

Scope for system operations

`tenant_id`

Type
unknown type

Default
<None>

Tenant ID

`tenant_name`

Type
unknown type

Default
<None>

Tenant Name

`timeout`

Type
integer

Default
<None>

Timeout value for http requests

`trust_id`

Type
unknown type

Default
<None>

ID of the trust to use as a trustee use

`user_domain_id`

Type
unknown type

Default
<None>

Users domain id

`user_domain_name`

Type
unknown type

Default
<None>

Users domain name

`user_id`

Type
unknown type

Default
<None>

User id

`username`

Type
unknown type

Default
<None>

Username

Table 68: Deprecated Variations

Group	Name
service_catalog	user-name
service_catalog	user_name

`valid_interfaces`

Type
list

Default
`['internal', 'public']`

List of interfaces, in order of preference, for endpoint URL.

`version`

Type
string

Default
<None>

Minimum Major API version within a given Major API version for endpoint URL discovery. Mutually exclusive with min_version and max_version

snmp

`power_timeout`

Type
integer

Default
`10`

Seconds to wait for power action to be completed

`reboot_delay`

Type
integer

Default
`0`

Minimum Value
0

Time (in seconds) to sleep between when rebooting (powering off and on again)

`power_action_delay`

Type
integer

Default
`0`

Minimum Value
0

Time (in seconds) to sleep before power on and after powering off. Which may be needed with some PDUs as they may not honor toggling a specific power port in rapid succession without a delay. This option may be useful if the attached physical machine has a substantial power supply to hold it over in the event of a brownout.

`udp_transport_timeout`

Type
floating point

Default
`1.0`

Minimum Value
0.0

Response timeout in seconds used for UDP transport. Timeout should be a multiple of 0.5 seconds and is applicable to each retry.

`udp_transport_retries`

Type
integer

Default
`5`

Minimum Value
0

Maximum number of UDP request retries, 0 means no retries.

ssl

`ca_file`

Type
string

Default
`<None>`

CA certificate file to use to verify connecting clients.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
The ca_file option is deprecated and will be removed in a future release.

`cert_file`

Type
string

Default
<None>

Certificate file to use when starting the server securely.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
The cert_file option is deprecated and will be removed in a future release.

`key_file`

Type
string

Default
<None>

Private key file to use when starting the server securely.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
The key_file option is deprecated and will be removed in a future release.

`version`

Type
string

Default
<None>

SSL version to use (valid only if SSL enabled). Valid values are TLSv1 and SSLv23. SSLv2, SSLv3, TLSv1_1, and TLSv1_2 may be available on some distributions.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
The version option is deprecated and will be removed in a future release.

`ciphers`

Type
string

Default
<None>

Sets the list of available ciphers. value should be a string in the OpenSSL cipher list format.

Warning

This option is deprecated for removal. Its value may be silently ignored in the future.

Reason
The ciphers option is deprecated and will be removed in a future release.

swift

`auth_url`

Type
unknown type

Default
<None>

Authentication URL

`auth_type`

Type
unknown type

Default
<None>

Authentication type to load

Table 69: Deprecated Variations

Group	Name
swift	auth_plugin

`cafile`

Type
string

Default
<None>

PEM encoded Certificate Authority to use when verifying HTTPs connections.

`certfile`

Type
string

Default
<None>

PEM encoded client certificate cert file

`collect_timing`

Type
boolean

Default
`False`

Collect per-API call timing information.

`connect_retries`

Type
integer

Default
<None>

The maximum number of retries that should be attempted for connection errors.

`connect_retry_delay`

Type
floating point

Default
<None>

Delay (in seconds) between two retries for connection errors. If not set, exponential retry starting with 0.5 seconds up to a maximum of 60 seconds is used.

`default_domain_id`

Type
unknown type

Default
<None>

Optional domain ID to use with v3 and v2 parameters. It will be used for both the user and project domain in v3 and ignored in v2 authentication.

`default_domain_name`

Type
unknown type

Default
<None>

Optional domain name to use with v3 API and v2 parameters. It will be used for both the user and project domain in v3 and ignored in v2 authentication.

`domain_id`

Type
unknown type

Default
<None>

Domain ID to scope to

`domain_name`

Type
unknown type

Default
<None>

Domain name to scope to

`endpoint_override`

Type
string

Default
<None>

Always use this endpoint URL for requests for this client. NOTE: The unversioned endpoint should be specified here; to request a particular API version, use the *version*, *min-version*, and/or *max-version* options.

`insecure`

Type
boolean

Default
`False`

Verify HTTPS connections.

`keyfile`

Type
string

Default
<None>

PEM encoded client certificate key file

`max_version`

Type
string

Default
<None>

The maximum major version of a given API, intended to be used as the upper bound of a range with min_version. Mutually exclusive with version.

`min_version`

Type
string

Default
<None>

The minimum major version of a given API, intended to be used as the lower bound of a range with max_version. Mutually exclusive with version. If min_version is given with no max_version it is as if max version is latest.

`password`

Type
unknown type

Default
<None>

Users password

`project_domain_id`

Type
unknown type

Default
<None>

Domain ID containing project

`project_domain_name`

Type
unknown type

Default
<None>

Domain name containing project

`project_id`

Type
unknown type

Default
<None>

Project ID to scope to

Table 70: Deprecated Variations

Group	Name
swift	tenant-id
swift	tenant_id

`project_name`

Type
unknown type

Default
<None>

Project name to scope to

Table 71: Deprecated Variations

Group	Name
swift	tenant-name
swift	tenant_name

`region_name`

Type
string

Default
<None>

The default region_name for endpoint URL discovery.

`retriable_status_codes`

Type
list

Default
<None>

List of retriable HTTP status codes that should be retried. If not set default to [503]

`service_name`

Type
string

Default
<None>

The default service_name for endpoint URL discovery.

`service_type`

Type
string

Default
`object-store`

The default service_type for endpoint URL discovery.

`split_loggers`

Type
boolean

Default
`False`

Log requests to multiple loggers.

`status_code_retries`

Type
integer

Default
<None>

The maximum number of retries that should be attempted for retriable HTTP status codes.

`status_code_retry_delay`

Type
floating point

Default
<None>

Delay (in seconds) between two retries for retriable status codes. If not set, exponential retry starting with 0.5 seconds up to a maximum of 60 seconds is used.

`system_scope`

Type
unknown type

Default
<None>

Scope for system operations

`tenant_id`

Type
unknown type

Default
<None>

Tenant ID

`tenant_name`

Type
unknown type

Default
<None>

Tenant Name

`timeout`

Type
integer

Default
<None>

Timeout value for http requests

`trust_id`

Type
unknown type

Default
<None>

ID of the trust to use as a trustee use

`user_domain_id`

Type
unknown type

Default
<None>

Users domain id

`user_domain_name`

Type
unknown type

Default
<None>

Users domain name

`user_id`

Type
unknown type

Default
<None>

User id

`username`

Type
unknown type

Default
<None>

Username

Table 72: Deprecated Variations

Group	Name
swift	user-name
swift	user_name

`valid_interfaces`

Type
list

Default
`['internal', 'public']`

List of interfaces, in order of preference, for endpoint URL.

`version`

Type
string

Default
<None>

Minimum Major API version within a given Major API version for endpoint URL discovery. Mutually exclusive with min_version and max_version

vnc

`enabled`

Type
boolean

Default
`False`

Enable VNC related features. Guests will get created with graphical devices to support this. Clients (for example Horizon) can then establish a VNC connection to the guest.

`host_ip`

Type
host address

Default
`0.0.0.0`

The IP address or hostname on which ironic-novncproxy listens.

`port`

Type
port number

Default
`6090`

Minimum Value
0

Maximum Value
65535

The TCP port on which ironic-novncproxy listens.

`public_url`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

Public URL to use when building the links to the noVNC client browser page (for example, http://127.0.0.1:6090/vnc_auto.html). If the API is operating behind a proxy, you will want to change this to represent the proxys URL.

`enable_ssl`

Type
boolean

Default
`False`

Enable the integrated stand-alone noVNC to service requests via HTTPS instead of HTTP. If there is a front-end service performing HTTPS offloading from the service, this option should be False; note, you will want to configure [vnc]public_endpoint option to set URLs in responses to the SSL terminated one.

`novnc_web`

Type
string

Default
`/usr/share/novnc`

Path to directory with content which will be served by a web server.

`novnc_record`

Type
string

Default
<None>

Filename that will be used for storing websocket frames received and sent by a VNC proxy service running on this host. If this is not set, no recording will be done.

`novnc_auth_schemes`

Type
list

Default
['none']

The allowed authentication schemes to use with proxied VNC connections

`read_only`

Type
boolean

Default
False

When True, keyboard and mouse events will not be passed to the console.

`token_timeout`

Type
integer

Default
600

Minimum Value
10

The lifetime of a console auth token (in seconds).

`expire_console_session_interval`

Type
integer

Default
120

Minimum Value
1

Interval (in seconds) between periodic checks to determine whether active console sessions have expired and need to be closed.

`container_provider`

Type
string

Default
fake

Console container provider which manages the containers that expose a VNC service to ironic-novncproxy or nova-novncproxy. Each container runs an X11 session and a browser showing the actual BMC console. systemd manages containers as systemd units via podman Quadlet support. The default is fake which returns an unusable VNC host and port. This needs to be changed if enabled is True. kubernetes manages containers as pods using template driven resource creation.

`console_image`

Type
string

Default
<None>

Mutable
This option can be changed without restarting.

Container image reference for the systemd and kubernetes console container provider, and any other out-of-tree provider which requires a configurable image reference.

`systemd_container_template`

Type
string

Default
`$pybasedir/console/container/ironic-console.container.template`

Mutable
This option can be changed without restarting.

For the systemd provider, path to the template for defining a console container. The default template requires that console_image be set.

`systemd_container_publish_port`

Type
string

Default
`$my_ip::5900`

Equivalent to the podman run port argument for the mapping of VNC port 5900 to the host. An IP address is required to bind to, defaulting to $my_ip. The VNC port exposed on the host will be a randomly allocated high port. These containers expose VNC servers which must be accessible by ironic-novncproxy and/or nova-novncproxy. The VNC servers have no authentication or encryption so they also should not be exposed to public access. Additionally, the containers need to be able to access BMC management endpoints.

`kubernetes_container_template`

Type
string

Default
`$pybasedir/console/container/ironic-console-pod.yaml.template`

Mutable
This option can be changed without restarting.

For the kubernetes provider, path to the template for defining the console resources. The default template creates one Secret to store the app info, and one Pod to run a console container. A custom template must include namespace metadata, and must define labels which can be used as a delete-all selector.

`kubernetes_pod_timeout`

Type
integer

Default
120

For the kubernetes provider, the time (in seconds) to wait for the console pod to start.

`ssl_cert_file`

Type
string

Default
<None>

Certificate file to use when starting the server securely.

`ssl_key_file`

Type
string

Default
<None>

Private key file to use when starting the server securely.

`ssl_minimum_version`

Type
string

Default
<None>

The minimum SSL version to use.

`ssl_ciphers`

Type
string

Default
<None>

Sets the list of available ciphers. value should be a string in the OpenSSL cipher list format.

`wait_for_ready_timeout`

Type
integer

Default
10

Minimum Value
1

Maximum number of seconds to wait for a console container to be ready to accept VNC connections after starting.

4.5.2 Policies

Warning

JSON formatted policy files were deprecated in the Wallaby development cycle due to the Victoria deprecation by the `olso.policy` library. Use the oslopolicy-convert-json-to-yaml tool to convert the existing JSON to YAML formatted policy file in backward compatible way.

The following is an overview of all available policies in Ironic. For a sample configuration file, refer to *Ironic Policy*.

ironic.api

admin_api

Default

```
role:admin or role:administrator
```

Legacy rule for cloud admin access

public_api

Default

```
is_public_api:True
```

Internal flag for public API routes

show_password

Default

```
!
```

Show or mask secrets within node driver information in API responses. This setting should be used with the utmost care as its use can present a security risk.

show_instance_secrets

Default

```
!
```

Show or mask secrets within instance information in API responses. This setting should be used with the utmost care as its use can present a security risk.

service_role

Default

```
role:service and project_name:%(config.service_project_name)s
```

Rule to match service role usage with a service project, delineated as a separate rule to enable customization.

is_member

Default

```
(project_domain_id:default or project_domain_id:None) and
(project_name:demo or project_name:baremetal)
```

May be used to restrict access to specific projects

is_observer

Default
`rule:is_member and (role:observer or role:baremetal_observer)`

Read-only API access

`is_admin`

Default
`rule:admin_api or (rule:is_member and role:baremetal_admin)`

Full read/write API access

`is_node_owner`

Default
`project_id:%(node.owner)s`

Owner of node

`is_node_lessee`

Default
`project_id:%(node.lessee)s`

Lessee of node

`is_allocation_owner`

Default
`project_id:%(allocation.owner)s`

Owner of allocation

`baremetal:node:create`

Default
`(role:admin and system_scope:all) or (role:service and system_scope:all)`

Operations

- **POST** `/nodes`

Scope Types

- **system**
- **project**

Create Node records

`baremetal:node:create:self_owned_node`

Default
`(role:admin) or (role:service)`

Operations

- **POST** `/nodes`

Scope Types

- **system**
- **project**

Create node records which will be tracked as owned by the associated user project.

`baremetal:node:list`

Default

```
(role:reader) or (role:service)
```

Operations

- **GET** `/nodes`
- **GET** `/nodes/detail`

Scope Types

- **system**
- **project**

Retrieve multiple Node records, filtered by an explicit owner or the client project_id

`baremetal:node:list_all`

Default

```
(role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role
```

Operations

- **GET** `/nodes`
- **GET** `/nodes/detail`

Scope Types

- **system**
- **project**

Retrieve multiple Node records

`baremetal:node:get`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:reader and
(project_id:%(node.owner)s or project_id:%(node.lessee)s)) or
(role:service and project_id:%(node.owner)s)
```

Operations

- **GET** `/nodes/{node_ident}`

Scope Types

- **system**
- **project**

Retrieve a single Node record

`baremetal:node:get:filter_threshold`

Default

```
(role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role
```

Operations

- **GET** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Filter to allow operators to govern the threshold where information should be filtered. Non-authorized users will be subjected to additional API policy checks for API content response bodies.

`baremetal:node:get:last_error`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:service
and system_scope:all) or (role:reader and project_id:%(node.
owner)s) or (role:service and project_id:%(node.owner)s)
```

Operations

- **GET** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Governs if the node last_error field is masked from API clients with insufficient privileges.

`baremetal:node:get:reservation`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:service
and system_scope:all) or (role:reader and project_id:%(node.
owner)s) or (role:service and project_id:%(node.owner)s)
```

Operations

- **GET** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Governs if the node reservation field is masked from API clients with insufficient privileges.

`baremetal:node:get:driver_internal_info`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:service
```

```
and system_scope:all) or (role:reader and project_id:%(node.
owner)s) or (role:service and project_id:%(node.owner)s)
```

Operations

- **GET** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Governs if the node driver_internal_info field is masked from API clients with insufficient privileges.

`baremetal:node:get:driver_info`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:service
and system_scope:all) or (role:reader and project_id:%(node.
owner)s) or (role:service and project_id:%(node.owner)s)
```

Operations

- **GET** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Governs if the driver_info field is masked from API clients with insufficient privileges.

`baremetal:node:update:driver_info`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Governs if node driver_info field can be updated via the API clients.

`baremetal:node:update:properties`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Governs if node properties field can be updated via the API clients.

`baremetal:node:update:chassis_uuid`

Default

```
role:admin and system_scope:all
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Governs if node chassis_uuid field can be updated via the API clients.

`baremetal:node:update:instance_uuid`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Governs if node instance_uuid field can be updated via the API clients.

`baremetal:node:update:lessee`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**

- **project**

Governs if node lessee field can be updated via the API clients.

`baremetal:node:update:owner`

Default

```
(role:member and system_scope:all) or rule:service_role
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Governs if node owner field can be updated via the API clients.

`baremetal:node:update:driver_interfaces`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:admin
and project_id:%(node.owner)s) or (role:manager and
project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Governs if node driver and driver interfaces field can be updated via the API clients.

`baremetal:node:update:network_data`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Governs if node driver_info field can be updated via the API clients.

`baremetal:node:update:conductor_group`

Default

```
(role:member and system_scope:all) or rule:service_role
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Governs if node conductor_group field can be updated via the API clients.

`baremetal:node:update:name`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Governs if node name field can be updated via the API clients.

`baremetal:node:update:retired`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Governs if node retired and retired reason can be updated by API clients.

`baremetal:node:update`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:member and (project_id:%(node.owner)s or
project_id:%(node.lessee)s)) or (role:service and
system_scope:all)
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Generalized update of node records

`baremetal:node:update_extra`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:member and (project_id:%(node.owner)s or
project_id:%(node.lessee)s)) or (role:service and
system_scope:all)
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Update Node extra field

`baremetal:node:update_instance_info`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:admin and
project_id:%(node.lessee)s) or (role:manager and
project_id:%(node.lessee)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Update Node instance_info field

`baremetal:node:update_owner_provisioned`

Default

```
role:admin and system_scope:all
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**

Update Node owner even when Node is provisioned

`baremetal:node:delete`

Default

```
role:admin and system_scope:all
```

Operations

- **DELETE** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Delete Node records

`baremetal:node:delete:self_owned_node`

Default

```
role:admin and project_id:%(node.owner)s
```

Operations

- **DELETE** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Delete node records which are associated with the requesting project.

`baremetal:node:validate`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:admin and
project_id:%(node.lessee)s) or (role:manager and
project_id:%(node.lessee)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **GET** /nodes/{node_ident}/validate

Scope Types

- **system**
- **project**

Request active validation of Nodes

`baremetal:node:set_maintenance`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:admin and
```

```
project_id:%(node.lessee)s) or (role:manager and
project_id:%(node.lessee)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PUT** /nodes/{node_ident}/maintenance

Scope Types

- **system**
- **project**

Set maintenance flag, taking a Node out of service

`baremetal:node:clear_maintenance`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:admin and
project_id:%(node.lessee)s) or (role:manager and
project_id:%(node.lessee)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **DELETE** /nodes/{node_ident}/maintenance

Scope Types

- **system**
- **project**

Clear maintenance flag, placing the Node into service again

`baremetal:node:get_boot_device`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:admin
and project_id:%(node.owner)s) or (role:manager and
project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **GET** /nodes/{node_ident}/management/boot_device
- **GET** /nodes/{node_ident}/management/boot_device/supported

Scope Types

- **system**
- **project**

Retrieve Node boot device metadata

`baremetal:node:set_boot_device`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:admin
and project_id:%(node.owner)s) or (role:manager and
project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PUT** /nodes/{node_ident}/management/boot_device

Scope Types

- **system**
- **project**

Change Node boot device

`baremetal:node:get_indicator_state`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:reader and
(project_id:%(node.owner)s or project_id:%(node.lessee)s)) or
(role:service and project_id:%(node.owner)s)
```

Operations

- **GET** /nodes/{node_ident}/management/indicators/{component}/{indicator}
- **GET** /nodes/{node_ident}/management/indicators

Scope Types

- **system**
- **project**

Retrieve Node indicators and their states

`baremetal:node:set_indicator_state`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PUT** /nodes/{node_ident}/management/indicators/{component}/{indicator}

Scope Types

- **system**
- **project**

Change Node indicator state

`baremetal:node:inject_nmi`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:admin
and project_id:%(node.owner)s) or (role:manager and
project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PUT** /nodes/{node_ident}/management/inject_nmi

Scope Types

- **system**
- **project**

Inject NMI for a node

`baremetal:node:get_states`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:reader and
(project_id:%(node.owner)s or project_id:%(node.lessee)s)) or
(role:service and project_id:%(node.owner)s)
```

Operations

- **GET** /nodes/{node_ident}/states

Scope Types

- **system**
- **project**

View Node power and provision state

`baremetal:node:set_power_state`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:member and (project_id:%(node.owner)s or
project_id:%(node.lessee)s)) or (role:service and
system_scope:all)
```

Operations

- **PUT** /nodes/{node_ident}/states/power

Scope Types

- **system**
- **project**

Change Node power status

`baremetal:node:set_boot_mode`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:member and (project_id:%(node.owner)s or
project_id:%(node.lessee)s)) or (role:service and
system_scope:all)
```

Operations

- **PUT** /nodes/{node_ident}/states/boot_mode

Scope Types

- **system**
- **project**

Change Node boot mode

`baremetal:node:set_secure_boot`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:member and (project_id:%(node.owner)s or
project_id:%(node.lessee)s)) or (role:service and
system_scope:all)
```

Operations

- **PUT** /nodes/{node_ident}/states/secure_boot

Scope Types

- **system**
- **project**

Change Node secure boot state

`baremetal:node:set_provision_state`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:admin and
project_id:%(node.lessee)s) or (role:manager and
project_id:%(node.lessee)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PUT** /nodes/{node_ident}/states/provision

Scope Types

- **system**
- **project**

Change Node provision status

`baremetal:node:set_provision_state:clean_steps`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:admin and
project_id:%(node.lessee)s) or (role:manager and
project_id:%(node.lessee)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PUT** /nodes/{node_ident}/states/provision

Scope Types

- **system**
- **project**

Allow execution of arbitrary steps on a node

`baremetal:node:set_provision_state:service_steps`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:admin and
project_id:%(node.lessee)s) or (role:manager and
project_id:%(node.lessee)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PUT** /nodes/{node_ident}/states/provision

Scope Types

- **system**
- **project**

Allow execution of arbitrary steps on a node

`baremetal:node:set_raid_state`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PUT** /nodes/{node_ident}/states/raid

Scope Types

- **system**
- **project**

Change Node RAID status

`baremetal:node:get_console`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **GET** /nodes/{node_ident}/states/console

Scope Types

- **system**
- **project**

Get Node console connection information

`baremetal:node:set_console_state`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PUT** /nodes/{node_ident}/states/console

Scope Types

- **system**
- **project**

Change Node console status

`baremetal:node:vif:list`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:reader and
(project_id:%(node.owner)s or project_id:%(node.lessee)s)) or
(role:service and project_id:%(node.owner)s)
```

Operations

- **GET** /nodes/{node_ident}/vifs

Scope Types

- **system**
- **project**

List VIFs attached to node

`baremetal:node:vif:attach`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
```

```
and project_id:%(node.owner)s) or (role:admin and
project_id:%(node.lessee)s) or (role:manager and
project_id:%(node.lessee)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **POST** /nodes/{node_ident}/vifs

Scope Types

- **system**
- **project**

Attach a VIF to a node

`baremetal:node:vif:detach`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:admin and
project_id:%(node.lessee)s) or (role:manager and
project_id:%(node.lessee)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **DELETE** /nodes/{node_ident}/vifs/{node_vif_ident}

Scope Types

- **system**
- **project**

Detach a VIF from a node

`baremetal:node:traits:list`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:reader and
(project_id:%(node.owner)s or project_id:%(node.lessee)s)) or
(role:service and project_id:%(node.owner)s)
```

Operations

- **GET** /nodes/{node_ident}/traits

Scope Types

- **system**
- **project**

List node traits

`baremetal:node:traits:set`

Default

```
((role:member and system_scope:all) or rule:service_role)
```

```
or (role:service and system_scope:all) or (role:admin
and project_id:%(node.owner)s) or (role:manager and
project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PUT** /nodes/{node_ident}/traits
- **PUT** /nodes/{node_ident}/traits/{trait}

Scope Types

- **system**
- **project**

Add a trait to, or replace all traits of, a node

`baremetal:node:traits:delete`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:admin
and project_id:%(node.owner)s) or (role:manager and
project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **DELETE** /nodes/{node_ident}/traits
- **DELETE** /nodes/{node_ident}/traits/{trait}

Scope Types

- **system**
- **project**

Remove one or all traits from a node

`baremetal:node:bios:get`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:reader and
(project_id:%(node.owner)s or project_id:%(node.lessee)s)) or
(role:service and project_id:%(node.owner)s)
```

Operations

- **GET** /nodes/{node_ident}/bios
- **GET** /nodes/{node_ident}/bios/{setting}

Scope Types

- **system**
- **project**

Retrieve Node BIOS information

`baremetal:node:disable_cleaning`

Default

```
role:admin and system_scope:all
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Disable Node disk cleaning

`baremetal:node:history:get`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:service
and system_scope:all) or (role:reader and project_id:%(node.
owner)s) or (role:service and project_id:%(node.owner)s)
```

Operations

- **GET** /nodes/{node_ident}/history
- **GET** /nodes/{node_ident}/history/{event_ident}

Scope Types

- **system**
- **project**

Filter to allow operators to retrieve history records for a node.

`baremetal:node:inventory:get`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:service
and system_scope:all) or (role:reader and project_id:%(node.
owner)s) or (role:service and project_id:%(node.owner)s)
```

Operations

- **GET** /nodes/{node_ident}/inventory

Scope Types

- **system**
- **project**

Retrieve inspection data for a node.

`baremetal:node:update:shard`

Default

```
role:admin and system_scope:all
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Governs if node shard field can be updated via the API clients.

`baremetal:shards:get`

Default

```
(role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role
```

Operations

- **GET** /shards

Scope Types

- **system**
- **project**

Governs if shards can be read via the API clients.

`baremetal:node:update:parent_node`

Default

```
(role:member and system_scope:all) or rule:service_role
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Governs if node parent_node field can be updated via the API clients.

`baremetal:node:update:disable_power_off`

Default

```
role:admin and system_scope:all
```

Operations

- **PATCH** /nodes/{node_ident}

Scope Types

- **system**
- **project**

Governs if power off can be disabled via the API clients.

`baremetal:node:firmware:get`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:reader and
(project_id:%(node.owner)s or project_id:%(node.lessee)s)) or
(role:service and project_id:%(node.owner)s)
```

Operations

- **GET** /nodes/{node_ident}/firmware

Scope Types

- **system**
- **project**

Retrieve Node Firmware components information

`baremetal:node:vmedia:attach`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:member and (project_id:%(node.owner)s or
project_id:%(node.lessee)s)) or (role:service and
system_scope:all)
```

Operations

- **POST** /nodes/{node_ident}/vmedia

Scope Types

- **system**
- **project**

Attach a virtual media device to a node

`baremetal:node:vmedia:detach`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:member and (project_id:%(node.owner)s or
project_id:%(node.lessee)s)) or (role:service and
system_scope:all)
```

Operations

- **DELETE** /nodes/{node_ident}/vmedia

Scope Types

- **system**
- **project**

Detach a virtual media device from a node

`baremetal:node:vmedia:get`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:reader and
```

```
(project_id:%(node.owner)s or project_id:%(node.lessee)s)) or
(role:service and project_id:%(node.owner)s)
```

Operations

- **GET** /nodes/{node_ident}/vmedia

Scope Types

- **system**
- **project**

Get virtual media device details from a node

`baremetal:port:get`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:reader and
(project_id:%(node.owner)s or project_id:%(node.lessee)s)) or
(role:service and project_id:%(node.owner)s)
```

Operations

- **GET** /ports/{port_id}
- **GET** /nodes/{node_ident}/ports
- **GET** /nodes/{node_ident}/ports/detail
- **GET** /portgroups/{portgroup_ident}/ports
- **GET** /portgroups/{portgroup_ident}/ports/detail

Scope Types

- **system**
- **project**

Retrieve Port records

`baremetal:port:list`

Default

```
(role:reader) or (role:service)
```

Operations

- **GET** /ports
- **GET** /ports/detail

Scope Types

- **system**
- **project**

Retrieve multiple Port records, filtered by owner

`baremetal:port:list_all`

Default

```
(role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role
```

Operations

- **GET** /ports
- **GET** /ports/detail

Scope Types

- **system**
- **project**

Retrieve multiple Port records

`baremetal:port:create`

Default

```
(role:admin and system_scope:all) or (role:service and
system_scope:all) or (role:admin and project_id:%(node.
owner)s) or (role:manager and project_id:%(node.owner)s) or
(role:service and project_id:%(node.owner)s)
```

Operations

- **POST** /ports

Scope Types

- **system**
- **project**

Create Port records

`baremetal:port:delete`

Default

```
(role:admin and system_scope:all) or (role:service and
system_scope:all) or (role:admin and project_id:%(node.
owner)s) or (role:manager and project_id:%(node.owner)s) or
(role:service and project_id:%(node.owner)s)
```

Operations

- **DELETE** /ports/{port_id}

Scope Types

- **system**
- **project**

Delete Port records

`baremetal:port:update`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:admin
and project_id:%(node.owner)s) or (role:manager and
```

```
project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PATCH** /ports/{port_id}

Scope Types

- **system**
- **project**

Update Port records

`baremetal:portgroup:get`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:reader and
(project_id:%(node.owner)s or project_id:%(node.lessee)s)) or
(role:service and project_id:%(node.owner)s)
```

Operations

- **GET** /portgroups
- **GET** /portgroups/detail
- **GET** /portgroups/{portgroup_ident}
- **GET** /nodes/{node_ident}/portgroups
- **GET** /nodes/{node_ident}/portgroups/detail

Scope Types

- **system**
- **project**

Retrieve Portgroup records

`baremetal:portgroup:create`

Default

```
(role:admin and system_scope:all) or (role:service and
system_scope:all) or (role:admin and project_id:%(node.
owner)s) or (role:manager and project_id:%(node.owner)s) or
(role:service and project_id:%(node.owner)s)
```

Operations

- **POST** /portgroups

Scope Types

- **system**
- **project**

Create Portgroup records

`baremetal:portgroup:delete`

Default

```
(role:admin and system_scope:all) or (role:service and
system_scope:all) or (role:admin and project_id:%(node.
owner)s) or (role:manager and project_id:%(node.owner)s) or
(role:service and project_id:%(node.owner)s)
```

Operations

- **DELETE** /portgroups/{portgroup_ident}

Scope Types

- **system**
- **project**

Delete Portgroup records

`baremetal:portgroup:update`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:admin
and project_id:%(node.owner)s) or (role:manager and
project_id:%(node.owner)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PATCH** /portgroups/{portgroup_ident}

Scope Types

- **system**
- **project**

Update Portgroup records

`baremetal:portgroup:list`

Default

```
(role:reader) or (role:service)
```

Operations

- **GET** /portgroups
- **GET** /portgroups/detail

Scope Types

- **system**
- **project**

Retrieve multiple Port records, filtered by owner

`baremetal:portgroup:list_all`

Default

```
(role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role
```

Operations

- **GET** /portgroups
- **GET** /portgroups/detail

Scope Types

- **system**
- **project**

Retrieve multiple Port records

`baremetal:chassis:get`

Default

```
(role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role
```

Operations

- **GET** /chassis
- **GET** /chassis/detail
- **GET** /chassis/{chassis_id}

Scope Types

- **system**

Retrieve Chassis records

`baremetal:chassis:create`

Default

```
role:admin and system_scope:all
```

Operations

- **POST** /chassis

Scope Types

- **system**

Create Chassis records

`baremetal:chassis:delete`

Default

```
role:admin and system_scope:all
```

Operations

- **DELETE** /chassis/{chassis_id}

Scope Types

- **system**

Delete Chassis records

`baremetal:chassis:update`

Default

`(role:member and system_scope:all) or rule:service_role`

Operations

- **PATCH** `/chassis/{chassis_id}`

Scope Types

- **system**

Update Chassis records

`baremetal:driver:get`

Default

`(role:reader and system_scope:all) or (role:service and system_scope:all) or rule:service_role`

Operations

- **GET** `/drivers`
- **GET** `/drivers/{driver_name}`

Scope Types

- **system**

View list of available drivers

`baremetal:driver:get_properties`

Default

`(role:reader and system_scope:all) or (role:service and system_scope:all) or rule:service_role`

Operations

- **GET** `/drivers/{driver_name}/properties`

Scope Types

- **system**

View driver-specific properties

`baremetal:driver:get_raid_logical_disk_properties`

Default

`(role:reader and system_scope:all) or (role:service and system_scope:all) or rule:service_role`

Operations

- **GET** `/drivers/{driver_name}/raid/logical_disk_properties`

Scope Types

- **system**

View driver-specific RAID metadata

`baremetal:node:vendor_passthru`

Default

`role:admin and system_scope:all`

Operations

- **GET** `nodes/{node_ident}/vendor_passthru/methods`
- **GET** `nodes/{node_ident}/vendor_passthru?method={method_name}`
- **PUT** `nodes/{node_ident}/vendor_passthru?method={method_name}`
- **POST** `nodes/{node_ident}/vendor_passthru?method={method_name}`
- **PATCH** `nodes/{node_ident}/vendor_passthru?method={method_name}`
- **DELETE** `nodes/{node_ident}/vendor_passthru?method={method_name}`

Scope Types

- **system**
- **project**

Access vendor-specific Node functions

`baremetal:driver:vendor_passthru`

Default

`role:admin and system_scope:all`

Operations

- **GET** `drivers/{driver_name}/vendor_passthru/methods`
- **GET** `drivers/{driver_name}/vendor_passthru?method={method_name}`
- **PUT** `drivers/{driver_name}/vendor_passthru?method={method_name}`
- **POST** `drivers/{driver_name}/vendor_passthru?method={method_name}`
- **PATCH** `drivers/{driver_name}/vendor_passthru?method={method_name}`
- **DELETE** `drivers/{driver_name}/vendor_passthru?method={method_name}`

Scope Types

- **system**

Access vendor-specific Driver functions

`baremetal:node:ipa_heartbeat`

Default

<empty string>

Operations

- **POST** /heartbeat/{node_ident}

Receive heartbeats from IPA ramdisk

`baremetal:driver:ipa_lookup`

Default
<empty string>

Operations

- **GET** /lookup

Access IPA ramdisk functions

`baremetal:driver:ipa_continue_inspection`

Default
<empty string>

Operations

- **POST** /continue_inspection

Receive inspection data from the ramdisk

`baremetal:volume:list_all`

Default

```
(role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role
```

Operations

- **GET** /volume/connectors
- **GET** /volume/targets
- **GET** /nodes/{node_ident}/volume/connectors
- **GET** /nodes/{node_ident}/volume/targets

Scope Types

- **system**
- **project**

Retrieve a list of all Volume connector and target records

`baremetal:volume:list`

Default

```
(role:reader) or (role:service)
```

Operations

- **GET** /volume/connectors
- **GET** /volume/targets
- **GET** /nodes/{node_ident}/volume/connectors
- **GET** /nodes/{node_ident}/volume/targets

Scope Types

- **system**
- **project**

Retrieve a list of Volume connector and target records

`baremetal:volume:get`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:reader and
(project_id:%(node.owner)s or project_id:%(node.lessee)s)) or
(role:service and project_id:%(node.owner)s)
```

Operations

- **GET** `/volume`
- **GET** `/volume/connectors`
- **GET** `/volume/connectors/{volume_connector_id}`
- **GET** `/volume/targets`
- **GET** `/volume/targets/{volume_target_id}`
- **GET** `/nodes/{node_ident}/volume`
- **GET** `/nodes/{node_ident}/volume/connectors`
- **GET** `/nodes/{node_ident}/volume/targets`

Scope Types

- **system**
- **project**

Retrieve Volume connector and target records

`baremetal:volume:create`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:admin
and project_id:%(node.owner)s) or (role:manager
and project_id:%(node.owner)s) or (role:admin and
project_id:%(node.lessee)s) or (role:manager and
project_id:%(node.lessee)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **POST** `/volume/connectors`
- **POST** `/volume/targets`

Scope Types

- **system**
- **project**

Create Volume connector and target records

`baremetal:volume:delete`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:admin
and project_id:%(node.owner)s) or (role:manager
and project_id:%(node.owner)s) or (role:admin and
project_id:%(node.lessee)s) or (role:manager and
project_id:%(node.lessee)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **DELETE** `/volume/connectors/{volume_connector_id}`
- **DELETE** `/volume/targets/{volume_target_id}`

Scope Types

- **system**
- **project**

Delete Volume connector and target records

`baremetal:volume:update`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:service and system_scope:all) or (role:member
and project_id:%(node.owner)s) or (role:admin and
project_id:%(node.lessee)s) or (role:manager and
project_id:%(node.lessee)s) or (role:service and
project_id:%(node.owner)s)
```

Operations

- **PATCH** `/volume/connectors/{volume_connector_id}`
- **PATCH** `/volume/targets/{volume_target_id}`

Scope Types

- **system**
- **project**

Update Volume connector and target records

`baremetal:volume:view_target_properties`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:admin)
```

Operations

- **GET** `/volume/connectors/{volume_connector_id}`
- **GET** `/volume/targets/{volume_target_id}`

Scope Types

- **system**
- **project**

Ability to view volume target properties

`baremetal:conductor:get`

Default

```
(role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role
```

Operations

- **GET** `/conductors`
- **GET** `/conductors/{hostname}`

Scope Types

- **system**
- **project**

Retrieve Conductor records

`baremetal:allocation:get`

Default

```
((role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role) or (role:reader and
project_id:%(allocation.owner)s)
```

Operations

- **GET** `/allocations/{allocation_id}`
- **GET** `/nodes/{node_ident}/allocation`

Scope Types

- **system**
- **project**

Retrieve Allocation records

`baremetal:allocation:list`

Default

```
(role:reader) or (role:service)
```

Operations

- **GET** `/allocations`

Scope Types

- **system**
- **project**

Retrieve multiple Allocation records, filtered by owner

`baremetal:allocation:list_all`

Default
`(role:reader and system_scope:all) or (role:service and system_scope:all) or rule:service_role`

Operations

- **GET** `/allocations`

Scope Types

- **system**
- **project**

Retrieve multiple Allocation records

`baremetal:allocation:create`

Default
`((role:member and system_scope:all) or rule:service_role) or (role:member)`

Operations

- **POST** `/allocations`

Scope Types

- **system**
- **project**

Create Allocation records

`baremetal:allocation:create_restricted`

Default
`(role:member and system_scope:all) or rule:service_role`

Operations

- **POST** `/allocations`

Scope Types

- **system**
- **project**

Create Allocation records with a specific owner.

`baremetal:allocation:delete`

Default
`((role:member and system_scope:all) or rule:service_role) or (role:member and project_id:%(allocation.owner)s)`

Operations

- **DELETE** `/allocations/{allocation_id}`
- **DELETE** `/nodes/{node_ident}/allocation`

Scope Types

- **system**
- **project**

Delete Allocation records

`baremetal:allocation:update`

Default

```
((role:member and system_scope:all) or rule:service_role) or
(role:member and project_id:%(allocation.owner)s)
```

Operations

- **PATCH** `/allocations/{allocation_id}`

Scope Types

- **system**
- **project**

Change name and extra fields of an allocation

`baremetal:allocation:create_pre_rbac`

Default

```
(rule:is_member and role:baremetal_admin) or
(is_admin_project:True and role:admin)
```

Operations

- **PATCH** `/allocations/{allocation_id}`

Scope Types

- **project**

Logical restrictor to prevent legacy allocation rule missuse - Requires blank allocations to originate from the legacy baremetal_admin.

`baremetal:events:post`

Default

```
role:admin and system_scope:all
```

Operations

- **POST** `/events`

Scope Types

- **system**

Post events

`baremetal:deploy_template:get`

Default

```
(role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role
```

Operations

- **GET** `/deploy_templates`

- **GET** /deploy_templates/{deploy_template_ident}

Scope Types

- **system**
- **project**

Retrieve Deploy Template records

`baremetal:deploy_template:create`

Default

```
role:admin and system_scope:all
```

Operations

- **POST** /deploy_templates

Scope Types

- **system**
- **project**

Create Deploy Template records

`baremetal:deploy_template:delete`

Default

```
role:admin and system_scope:all
```

Operations

- **DELETE** /deploy_templates/{deploy_template_ident}

Scope Types

- **system**
- **project**

Delete Deploy Template records

`baremetal:deploy_template:update`

Default

```
role:admin and system_scope:all
```

Operations

- **PATCH** /deploy_templates/{deploy_template_ident}

Scope Types

- **system**
- **project**

Update Deploy Template records

`baremetal:runbook:get`

Default

```
((role:reader and system_scope:all) or (role:service and
```

```
system_scope:all) or rule:service_role) or (role:reader and
project_id:%(runbook.owner)s) or role:service
```

Operations

- **GET** /runbooks/{runbook_ident}

Scope Types

- **system**
- **project**

Retrieve a single runbook record

`baremetal:runbook:list`

Default

```
(role:reader) or (role:service)
```

Operations

- **GET** /runbooks

Scope Types

- **system**
- **project**

Retrieve multiple runbook records, filtered by an explicit owner or the client project_id

`baremetal:runbook:list_all`

Default

```
(role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role
```

Operations

- **GET** /runbooks

Scope Types

- **system**
- **project**

Retrieve all runbook records

`baremetal:runbook:create`

Default

```
((role:member and system_scope:all) or rule:service_role) or
role:manager or role:service
```

Operations

- **POST** /runbooks

Scope Types

- **system**
- **project**

Create Runbook records

`baremetal:runbook:delete`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:manager and project_id:%(runbook.owner)s) or
role:service
```

Operations

- **DELETE** /runbooks/{runbook_ident}

Scope Types

- **system**
- **project**

Delete a runbook record

`baremetal:runbook:update`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:manager and project_id:%(runbook.owner)s) or
role:service
```

Operations

- **PATCH** /runbooks/{runbook_ident}

Scope Types

- **system**
- **project**

Update a runbook record

`baremetal:runbook:update:public`

Default

```
(role:member and system_scope:all) or rule:service_role
```

Operations

- **PATCH** /runbooks/{runbook_ident}/public

Scope Types

- **system**
- **project**

Set and unset a runbook as public

`baremetal:runbook:update:owner`

Default

```
(role:member and system_scope:all) or rule:service_role
```

Operations

- **PATCH** /runbooks/{runbook_ident}/owner

Scope Types

- **system**
- **project**

Set and unset the owner of a runbook

`baremetal:runbook:use`

Default

```
((role:member and system_scope:all) or rule:service_role)
or (role:manager and project_id:%(runbook.owner)s) or
role:service
```

Operations

- **PUT** `/nodes/{node_ident}/states/provision`

Scope Types

- **system**
- **project**

Allowed to use a runbook for node operations

`baremetal:inspection_rule:get`

Default

```
(role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role
```

Operations

- **GET** `/inspection_rules`
- **GET** `/inspection_rules/{rule_id}`

Scope Types

- **system**
- **project**

Get inspection rule(s)

`baremetal:inspection_rule:list_all`

Default

```
(role:reader and system_scope:all) or (role:service and
system_scope:all) or rule:service_role
```

Operations

- **GET** `/inspection_rules`

Scope Types

- **system**
- **project**

Retrieve all inspection_rule records

`baremetal:inspection_rule:create`

Default
`role:admin and system_scope:all`

Operations

- **POST** `/inspection_rules`

Scope Types

- **system**
- **project**

Create inspection rule

`baremetal:inspection_rule:update`

Default
`role:admin and system_scope:all`

Operations

- **PATCH** `/inspection_rules/{rule_id}`

Scope Types

- **system**
- **project**

Update an inspection rule

`baremetal:inspection_rule:delete`

Default
`role:admin and system_scope:all`

Operations

- **DELETE** `/inspection_rules`
- **DELETE** `/inspection_rules/{rule_id}`

Scope Types

- **system**
- **project**

Delete an inspection rule

4.6 Architecture and Implementation Details

4.6.1 Agent Token

Purpose

The concept of agent tokens is to provide a mechanism by which the relationship between an operating deployment of the Bare Metal Service and an instance of the `ironic-python-agent` is verified. In a sense, this token can be viewed as a session identifier or authentication token.

Warning

This functionality does not remove the risk of a man-in-the-middle attack that could occur from connection intercept or when TLS is not used for all communication.

This becomes useful in the case of deploying an edge node where intermediate networks are not trustworthy.

How it works

These tokens are provided in one of two ways to the running agent.

1. A pre-generated token that is embedded into virtual media ISOs.
2. A one-time generated token that is provided upon the first lookup of the node.

In both cases, the tokens are randomly generated using the Python `secrets` library. As of mid-2020, the default length is 43 characters.

Once the token has been provided, the token cannot be retrieved or accessed. It remains available to the conductors and is stored in the memory of the `ironic-python-agent`.

Note

In the case of the token being embedded with virtual media, it is read from a configuration file within the image. Ideally, this should be paired with Swift temporary URLs.

With the token is available in memory in the agent, the token is embedded with `heartbeat` operations to the ironic API endpoint. This enables the API to authenticate the heartbeat request, and refuse heartbeat requests from the `ironic-python-agent`. As of the Victoria release, the use of Agent Token is required for all agents and the previously available setting to force this functionality to be mandatory, `[DEFAULT]require_agent_token` has been removed and no longer has any effect.

Warning

If the Bare Metal Service is updated, and the version of `ironic-python-agent` should be updated to enable this feature.

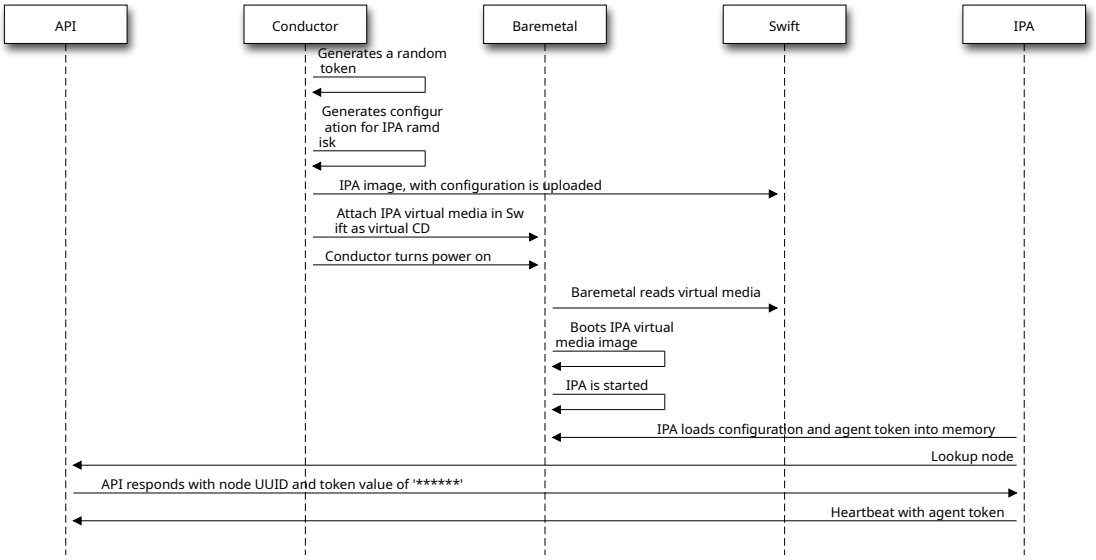
In addition to heartbeats being verified, commands from the `ironic-conductor` service to the `ironic-python-agent` also include the token, allowing the agent to authenticate the caller.

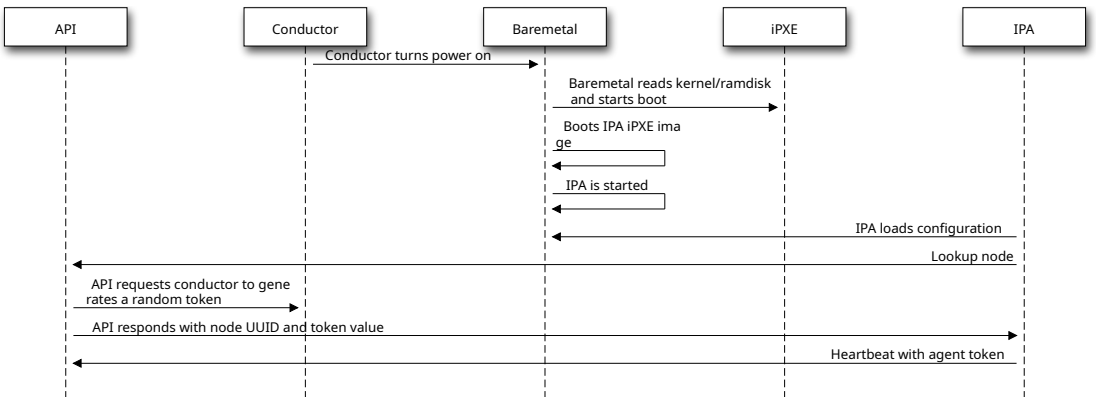
With Virtual Media

With PXE/iPXE/etc.

Agent Configuration

An additional setting that may be leveraged with the `ironic-python-agent` is a `agent_token_required` setting. Under normal circumstances, this setting can be asserted via the configuration supplied from the Bare Metal service deployment upon the `lookup` action but can be asserted via the embedded configuration for the agent in the ramdisk. This setting is also available via the kernel command line as `ipa-agent-token-required`.


API
Conductor
Baremetal
Swift
IPA
Generates a random token
Generates configuration for IPA ramdisk
IPA image, with configuration is uploaded
Attach IPA virtual media in Swift as virtual CD
Conductor turns power on
Baremetal reads virtual media
Boots IPA virtual media image
IPA is started
IPA loads configuration and agent token into memory
Lookup node
API responds with node UUID and token value of '******'
Heartbeat with agent token


API
Conductor
Baremetal
iPXE
IPA
Conductor turns power on
Baremetal reads kernel/ramdisk and starts boot
Boots IPA iPXE image
IPA is started
IPA loads configuration
Lookup node
API requests conductor to generates a random token
API responds with node UUID and token value
Heartbeat with agent token

4.6.2 Steps

What are steps?

Steps are exactly that, steps to achieve a goal, and in most cases, they are what an operator requested.

However, originally they were the internal list of actions to achieve to perform *automated cleaning*. The conductor would determine a list of steps or actions to take by generating a list of steps from data the conductor via drivers, the `ironic-python-agent`, and any loaded hardware managers determined to be needed.

As time passed and Ironics capabilities were extended, this was extended to *manual cleaning*, and later into *deploy steps*, and *deploy templates* allowing an operator to request for firmware to be updated by a driver, or RAID to be configured by the agent prior to the machine being released to the end user for use.

Reserved Functional Steps

In the execution of the cleaning, and deployment steps frameworks, some step names are reserved for specific functions which can be invoked by a user to perform specific actions.

Step Nam	Description
hold	Pauses the execution of the steps by moving the node from the current `deploy wait` or `clean wait` state to the appropriate hold state, such as `deploy hold` or `clean hold`. The process can be resumed by sending a `unhold` verb to the provision state API endpoint which will result in the process resuming upon the next heartbeat operation. During this time, heartbeat operations will continue be recorded by Ironic, but will not be acted upon, preventing the node from timing out. This step cannot be used against a child node in the context of being requested when executing against a parent node. The use case for this verb is if you have external automation or processes which need to be executed in the entire process to achieve the overall goal.
powe	Powers on the node, which may be useful if a nodes power must be toggled multiple times to enable embedded behavior such as to boot from network. This step can be executed against child nodes.
powe	Turn the node power off via the conductor. This step can be used against child nodes. When used outside of the context of a child node, any agent token metadata is also removed as so the machine can reboot back to the agent, if applicable.
re-boot	Reboot the node utilizing the conductor. This generally signals for power to be turned off and back on, however driver specific code may request an CPU interrupt based reset. This step can be executed on child nodes.
wait	Causes a brief pause in the overall step execution which pauses until the next heartbeat operation, unless a seconds argument is provided. If a *seconds* argument is provided, then the step execution will pause for the requested amount of time.

In the these cases, the interface upon which the method is expected is ignored, and the step is acted upon based upon just the steps name.

Example

In this example, we utilize the cleaning step `erase_devices` and then trigger hold of the node. In this specific case the node will enter a `clean hold` state.

```
{
  "target":"clean",
  "clean_steps": [{
    "interface": "deploy",
    "step": "erase_devices"
  },
  {
    "interface": "deploy",
    "step": "hold"
  }]
}
```

Once you have completed whatever action which needed to be performed while the node was in a held state, you will need to issue an unhold provision state command, via the API or command line to inform the node to proceed.

Set the environment

When using steps with the functionality to execute on child nodes, i.e. nodes who a populated `parent_node` field, you always want to ensure you have set the environment appropriately for your next action.

For example, if you are executing steps against a parent node, which then execute against a child node via the `execute_on_child_nodes` step option, and it requires power to be on, you will want to explicitly ensure the power is on for the parent node **unless** the child node can operate independently, as signaled through the driver_info option `has_dedicated_power_supply` on the child node. Power is an obvious case because Ironic has guarding logic internally to attempt to power-on the parent node, but it cannot be an after thought due to internal task locking.

Power specifically aside, the general principle applies to the execution of all steps. You need always want to build upon the prior step or existing existing known state of the system.

Note

Ironic will attempt to ensure power is active for a `parent_node` when powering on a child node. Conversely, Ironic will also attempt to power down child nodes if a parent node is requested to be turned off, unless the `has_dedicated_power_supply` option is set for the child node. This pattern of behavior prevents parent nodes from being automatically powered back on should a child node be left online.

BMC Clock Verification Step (Verify Phase)

The Redfish management interface includes a verify step called `verify_bmc_clock` which automatically checks and sets the BMC clock during node registration.

This step compares the system time on the conductor with the BMCs time reported via Redfish. If the clock differs by more than one second, Ironic updates the BMCs clock to match the conductors UTC time.

This step runs automatically if enabled and the node supports the Redfish interface.

How to Enable

The feature is controlled by the following configuration option in `ironic.conf`:

```
[redfish]
enable_verify_bmc_clock = true
```

By default, this option is set to `false`. To enable it, set it to `true` and restart the `ironic conductor`.

When It Runs

`verify_bmc_clock` step is triggered during the automated verify step of the node registration, before inspection and deployment.

Note:

- If the BMC does not support setting the clock via Redfish, the step will fail.
- If the time cannot be synchronized within 1 second, the step will raise a `NodeVerifyFailure`.
- If the configuration option is disabled, the step is skipped.
- `verify_bmc_clock` is defined with a priority of 1 and is not interruptible.
- This is different from the manual `clean` step `set_bmc_clock` which allows explicit datetime setting through the API, but also defaults to the current conductor UTC time when `target_datetime` is omitted.
- Release Notes

4.7 Administrators Guide

If you are a system administrator running Ironic, this section contains information that may help you understand how to operate and upgrade the services.

4.7.1 Availability Zones and Resource Isolation

Overview

While Ironic does not implement traditional OpenStack Availability Zones like Nova and Neutron, it provides a **three-tier approach** for resource partitioning and isolation that achieves comprehensive availability zone functionality:

- **Multiple Ironic Deployments: Completely separate Ironic services** targeted by different Nova compute nodes
- **Conductor Groups: Physical/geographical resource partitioning within** a deployment
- **Shards**: Logical grouping for operational scaling within a deployment

This document explains how these mechanisms work together and how to achieve sophisticated availability zone functionality across your infrastructure. This document does **not** cover similar effect which can be achieved through the use of API level Role Based Access Control through the `owner` and `lessee` fields.

Comparison with Other OpenStack Services

Service	Mechanism	Purpose
Nova	Availability Zones (host aggregates)	Instance placement across fault domains
Neutron	Agent AZs	Network service HA
Ironic	**Multiple Deployments + Conductor Groups + Shards**	**Complete service isolation + physical partitioning + operational scaling**

Ironics Three-Tier Approach

Tier 1: Multiple Ironic Deployments

The highest level of isolation involves running **completely separate Ironic services** that Nova and other API users can target independently.

Use Cases:

- Complete geographic separation (different regions/countries)
- Regulatory compliance requiring full data isolation
- Independent upgrade cycles and operational teams

Implementation: Configure separate Nova compute services to target different Ironic deployments using Novas Ironic driver configuration.

Benefits:

- Complete fault isolation - failure of one deployment doesnt affect others
- Independent scaling, upgrades, and maintenance
- Different operational policies per deployment
- Complete API endpoint separation

Tier 2: Conductor Groups (Physical Partitioning)

Within a single Ironic deployment, conductor groups provides **physical resource partitioning**.

Use Cases:

- Separate nodes by datacenter/availability zone within a region
- Separate nodes by conductor groups for conductor resource management
- Isolate hardware types or vendors
- Create fault domains for high availability
- Manage nodes with different network connectivity

Conductor groups control **which conductor manages which nodes**. Each conductor can be assigned to a specific group, and will only manage nodes that belong to the same group.

A classical challenge of Ironic is that it is able to manage far more Bare Metal nodes than a single `nova-compute` service is designed to support. The primary answer for this issue is to leverage Shards first, and then continue to evolve based upon operational needs.

See: *Conductor Groups* for detailed configuration.

Tier 3: Shards (Logical Partitioning)

The finest level of granularity for **operational and client-side grouping**.

Use Cases:

- Horizontal scaling of operations
- Parallelize maintenance tasks
- Create logical groupings for different teams

Shards can be used by clients, including Nova, to limit the scope of their requests to a logical and declared subset of nodes which prevents multiple `nova-compute` services from being able to see and work with the same node on multiple `nova-compute` services.

Note

Shards are client-side constructs - Ironic itself does not use shard values internally.

Added in version 1.82: Shard support was added in API version 1.82.

Warning

Once set, a shard should not be changed. Novas model of leveraging the Ironic API does not permit this value to be changed after the fact.

Common Deployment Patterns

Pattern 1: Multi-Region with Complete Isolation

Use Case: Global deployment with regulatory compliance

Implementation:

- **Multiple Deployments**: `ironic-us-east`, `ironic-eu-west`, `ironic-apac`
- **Nova Configuration**: Separate compute services per region
- **Conductor Groups**: Optional within each deployment
- **Shards**: Operational grouping within regions

Example Nova Configuration:

```
# nova-compute for US East region
[ironic]
auth_url = https://keystone-us-east.example.com/v3
endpoint_override = https://ironic-us-east.example.com

# nova-compute for EU West region
[ironic]
auth_url = https://keystone-eu-west.example.com/v3
endpoint_override = https://ironic-eu-west.example.com
```

Note

The above indicated `endpoint_override` configuration is provided for illustrative purposes to stress endpoints would be distinctly different.

Pattern 2: Single Region with Datacenter Separation

Use Case: Metro deployment across multiple datacenters

Implementation:

- **Single Deployment**: One Ironic service
- **Conductor Groups**: `datacenter-1`, `datacenter-2`, `datacenter-3`
- **Nova Configuration**: Target specific conductor groups
- **Shards**: Optional operational grouping

In this case, we dont expect BMC management network access to occur between datacenters. Thus each datacenter is configured with its own group of conductors.

Example Configuration:

```
# Configure Nova compute to target specific conductor group
[ironic]
conductor_group = datacenter-1

# Configure conductors (ironic.conf)
[conductor]
conductor_group = datacenter-1

# Assign nodes
baremetal node set --conductor-group datacenter-1 <node-uuid>
```

Note

Some larger operators who leverage conductor groups have suggested that it is sometimes logical to have a conductor set without a `conductor_group` set. This helps prevent orphaning nodes because Ironic routes all changes to the conductor which presently manages the node.

Pattern 3: Operational Scaling Within Datacenters

Use Case: Large deployment requiring parallel operations

Implementation:

- **Single Deployment**: One Ironic service
- **Conductor Groups**: By datacenter or hardware type
- **Shards**: Operational batches for maintenance/upgrades
- **Nova Configuration**: May target specific conductor groups

Example:

```
# Set up conductor groups by hardware
baremetal node set --conductor-group dell-servers <node-uuid-1>
baremetal node set --conductor-group hpe-servers <node-uuid-2>

# Create operational shards for maintenance
baremetal node set --shard maintenance-batch-1 <node-uuid-1>
baremetal node set --shard maintenance-batch-2 <node-uuid-2>
```

Pattern 4: Hybrid Multi-Tier Approach

Use Case: Complex enterprise deployment

Implementation: All three tiers working together

Example Architecture:

```
# Deployment 1: Production East Coast
# Nova compute service targets ironic-prod-east
[ironic]
endpoint_override = https://ironic-prod-east.example.com
conductor_group = datacenter-east

# Within this deployment:
baremetal node set --conductor-group datacenter-east --shard prod-batch-a
↪<node-uuid>

# Deployment 2: Production West Coast
# Nova compute service targets ironic-prod-west
[ironic]
endpoint_override = https://ironic-prod-west.example.com
conductor_group = datacenter-west
```

Nova Integration and Configuration

Targeting Multiple Ironic Deployments

Novas Ironic driver can be configured to target different Ironic services:

Per-Compute Service Configuration:

```
# /etc/nova/nova.conf on compute-service-1
[ironic]
auth_url = https://keystone-region1.example.com/v3
endpoint_override = https://ironic-region1.example.com
conductor_group = region1-zone1

# /etc/nova/nova.conf on compute-service-2
[ironic]
auth_url = https://keystone-region2.example.com/v3
endpoint_override = https://ironic-region2.example.com
conductor_group = region2-zone1
```

Advanced Options:

```
[ironic]
# Target specific conductor group within deployment
conductor_group = datacenter-east

# Target specific shard within deployment
shard = production-nodes

# Connection retry configuration
api_max_retries = 60
api_retry_interval = 2
```

See also

Nova Ironic Hypervisor Configuration for complete Nova configuration details.

Scaling Considerations

Nova Compute Service Scaling:

- Single nova-compute can handle several hundred Ironic nodes efficiently.
- Consider multiple compute services for >1000 nodes per deployment. Nova-compute is modeled on keeping a relatively small number of instances per nova-compute process. For example, 250 baremetal nodes.
- One nova-compute process per conductor group or shard is expected.
- A `conductor_group` which is independent of a nova-compute service configuration can be changed at any time. A shard should never be changed once it has been introduced to a nova-compute process.

Multi-Deployment Benefits:

- Independent scaling per deployment
- Isolated failure domains
- Different operational schedules

Integration Considerations

Network Considerations

Ironics partitioning works alongside physical network configuration:

- Physical networks can span multiple conductor groups
- Consider network topology when designing conductor group boundaries
- Ensure network connectivity between conductors and their assigned nodes

See also

Networking with the Bare Metal service for detailed network configuration guidance

Nova Placement and Scheduling

When using Ironic with Nova:

- Novas availability zones operate independently of Ironics partitioning
- Use resource classes and traits for capability-based scheduling

See also

Create flavors for use with the Bare Metal service for flavor and scheduling configuration

API Client Usage

Working Across Multiple Deployments

When managing multiple Ironic deployments, use separate client configurations:

```
# Configure client for deployment 1
export OS_AUTH_URL=https://keystone-east.example.com/v3
export OS_ENDPOINT_OVERRIDE=https://ironic-east.example.com
baremetal node list

# Configure client for deployment 2
export OS_AUTH_URL=https://keystone-west.example.com/v3
export OS_ENDPOINT_OVERRIDE=https://ironic-west.example.com
baremetal node list
```

Filtering by Conductor Group

```
# List nodes by conductor group
baremetal node list --conductor-group datacenter-east

# List ports by node conductor group
baremetal port list --conductor-group datacenter-east
```

Filtering by Shard

```
# List nodes by shard
baremetal node list --shard batch-a

# Get shard distribution
baremetal shard list

# Find nodes without a shard assignment
baremetal node list --unsharded
```

Combined Filtering Within Deployments

```
# Within a single deployment, filter by conductor group and shard
baremetal node list --conductor-group datacenter-1 --shard maintenance-batch-a

# Set both conductor group and shard on a node
baremetal node set --conductor-group datacenter-east --shard batch-a <node-
↪uuid>

# Get overview of resource distribution
baremetal shard list
baremetal conductor list
```

Best Practices

Deployment Strategy Planning

1. **Assess isolation requirements**: Determine if you need complete service separation
2. **Plan geographic distribution**: Use multiple deployments for true regional separation
3. **Design conductor groups**: Align with physical/network boundaries
4. **Implement shard strategy**: Plan for operational efficiency
5. **Configure Nova appropriately**: Match Nova compute services to your architecture

Operational Considerations

Multiple Deployments:

- Maintain consistent tooling across deployments
- Plan for cross-deployment migrations if needed
- Monitor each deployment independently
- Coordinate upgrade schedules

Within Deployments:

- Monitor conductor distribution: `baremetal shard list`
- Ensure conductor redundancy per group
- Align network topology with conductor groups
- Automate shard management for balance

Nova Integration:

- Plan compute service distribution across deployments
- Monitor nova-compute to Ironic node ratios
- Test failover scenarios between compute services

Naming Conventions

Naming patterns can be defined by the infrastructure operator and below are some basic suggestions which may be relevant based upon operational requirements.

Conductor Groups:

- Geographic: `datacenter-east`, `region-us-west`, `rack-01`
- Hardware: `dell-servers`, `hpe-gen10`, `gpu-nodes`
- Network: `vlan-100`, `isolated-network`

Shards:

- Operational: `maintenance-batch-1`, `upgrade-group-a`
- Size-based: `small-nodes`, `large-memory`
- Temporal: `weekend-maintenance`, `business-hours`

Decision Matrix

Choose your approach based on requirements:

Requirement	Multiple Deployments	Conductor **Groups	Shards
Complete isolation	✓ Best	✓ Good	No
Independent upgrades	✓ Complete	✓ Partial	No
Geographic separation	✓ Best	✓ Good	No
Operational scaling	Overhead	✓ Good	✓ Best
Resource efficiency	Lower	✓ Good	✓ Best

Troubleshooting

Multiple Deployment Issues

Connectivity Problems:

```
# Test connectivity to each deployment
baremetal --os-endpoint-override https://ironic-east.example.com node list
baremetal --os-endpoint-override https://ironic-west.example.com node list
```

Nova Configuration Issues:

```
# Check Nova compute service registration
openstack compute service list --service nova-compute

# Verify Nova can reach Ironic
grep -i ironic /var/log/nova/nova-compute.log
```

Cross-Deployment Node Migration:

```
# Export node data from source deployment
baremetal node show --fields all <node-uuid>
```

(continues on next page)

(continued from previous page)

```
# Import to destination deployment (manual process)
# Note: Requires careful planning and may need custom tooling
```

Common Issues Within Deployments

Orphaned nodes: Nodes without matching conductor groups cannot be managed

```
# Find nodes without conductor groups
baremetal node list --conductor-group ""

# List available conductor groups
baremetal conductor list
```

Unbalanced shards: Monitor node distribution across shards

```
# Check shard distribution
baremetal shard list

# Find heavily loaded shards
baremetal node list --shard <shard-name> | wc -l
```

Missing conductor groups: Ensure all groups have active conductors

```
# Check conductor status
baremetal conductor list

# Verify conductor group configuration
# Check ironic.conf [conductor] conductor_group setting
```

Migration Scenarios

Moving nodes between conductor groups:

```
# Move node to different conductor group
baremetal node set --conductor-group new-group <node-uuid>
```

Reassigning shards:

```
# Change node shard assignment
baremetal node set --shard new-shard <node-uuid>

# Remove shard assignment
baremetal node unset --shard <node-uuid>
```

Warning

Shards should never be changed once a nova-compute service has identified a node in Ironic. Changing a shard at this point is an unsupported action. As such, Ironics API RBAC policy restricts these

actions to a System-Scoped Admin user. Normal Admin users are denied this capability due the restriction and requirement on the nova-compute side of the consumption of shards.

See Also

- *Conductor Groups* - Detailed conductor group configuration
- *Networking with the Bare Metal service* - Physical network considerations
- *Reference Deploy Architectures* - Reference architectures
- *<no title>* - Multi-tenant deployments
- *Tuning Ironic* - Performance tuning considerations
- Nova Ironic Driver Documentation
- Nova Ironic Configuration Options

4.7.2 Deploy Steps

The deploy steps section has moved to *Using deploy steps and templates*.

4.7.3 VXLAN Networking

Overview

During the 2025.1 (Gazpacho) development cycle, the Ironic community merged initial support for using VXLAN networks with bare metal workloads. This capability enables bare metal hosts to attach to VXLAN tenant networks in the same way virtual machines do, while leveraging the physical network switch fabric to perform VXLAN termination and translation.

This document describes the architecture, requirements, and operational considerations for deploying VXLAN networking with Ironic.

What is VXLAN Support for Bare Metal?

This functionality enables bare metal nodes to participate in VXLAN overlay networks without acting as VXLAN Tunnel Endpoints (VTEPs) themselves. Instead, the physical network switches act as VTEPs, terminating VXLAN tunnels and translating them to VLANs on the physical switch ports where bare metal nodes are connected.

From the bare metal hosts perspective, it receives untagged Ethernet traffic on its network interface. However, this traffic is mapped to a VXLAN Network Identifier (VNI) within the network fabric, allowing it to share a logical network with virtual machines using VXLAN overlay networking.

Note

This implementation also supports Geneve network types in the same manner as VXLAN, translating them to VLAN attachments at the physical switch level. Such a configuration has received some basic testing in development as OVN treats the packets no differently than networks configured as VXLAN.

Why VXLAN Networking?

Network operators face several challenges that VXLAN addresses:

VLAN Limitations
: Traditional VLAN-based networking is constrained by the 4,096 VLAN limit (12-bit VLAN ID). In large-scale deployments, this constraint can become a significant limitation, especially when isolating tenant networks. VXLAN provides a 24-bit VNI space, supporting up to 16 million network segments.

Network Scalability
: VXLAN enables routed spine-leaf network architectures where overlay traffic can traverse Layer 3 boundaries. This eliminates the need for spanning tree protocols across large fabrics and enables more efficient traffic patterns.

Workload Integration
: In clouds running both virtual machines and bare metal nodes, VXLAN support enables seamless network integration. Tenant networks can span both virtualized and bare metal resources without requiring separate network infrastructure or administrative network creation.

Simplified Tenant Experience
: Unlike traditional VLAN networking in Ironic, which requires administrative privileges to create tenant networks (due to the need to specify physical network mappings), VXLAN networks can be created by regular users when the default tenant network type is configured for VXLAN or Geneve.

Terminology

VXLAN (Virtual eXtensible LAN)
: An overlay network protocol that encapsulates Layer 2 Ethernet frames within Layer 4 UDP packets, enabling Layer 2 networks to span Layer 3 boundaries.

VXLAN Network Identifier (VNI)
: A 24-bit identifier that uniquely identifies a VXLAN segment. Similar to a VLAN ID but with a much larger address space (16 million vs 4,096).

VXLAN Tunnel Endpoint (VTEP)
: A device (switch or host) that originates and/or terminates VXLAN tunnels, performing encapsulation and decapsulation of Ethernet frames.

Hierarchical Port Binding
: A Neutron mechanism that creates multiple binding segments for a single port. In this context, a top segment represents the VXLAN network and a bottom segment represents the physical VLAN used on the switch.

Lower Binding Segment
: The physical VLAN segment allocated by Neutron as part of hierarchical port binding. This VLAN is used internally by the switch fabric to map to the VXLAN VNI.

BGP EVPN (Ethernet VPN)
: A control plane protocol using BGP to distribute MAC address reachability information across a VXLAN fabric. This enables efficient forwarding and reduces broadcast, unknown unicast, and multicast (BUM) traffic.

Ingress Replication
: A method of handling broadcast, unknown unicast, and multicast traffic in VXLAN where the source VTEP replicates packets to each remote VTEP that is part of the VNI.

Architecture

The VXLAN support for Ironic consists of three main components:

1. **Mechanism Driver (baremetal-l2vni)** Handles hierarchical port binding for VXLAN and Geneve network types, allocating lower binding segments (VLANs) from the physical network pool. This driver also creates OVN localnet ports to bridge the logical overlay network to the physical network infrastructure.
2. **Physical Network Connectivity** Establishes the connection between the OpenStack cloud (specifically OVN network nodes) and the physical switch fabric. This can be implemented through hierarchical port binding with trunk ports, or through BGP EVPN Type-2 routes (under development in Neutron and should be expected as a result of the Hibiscus development cycle).

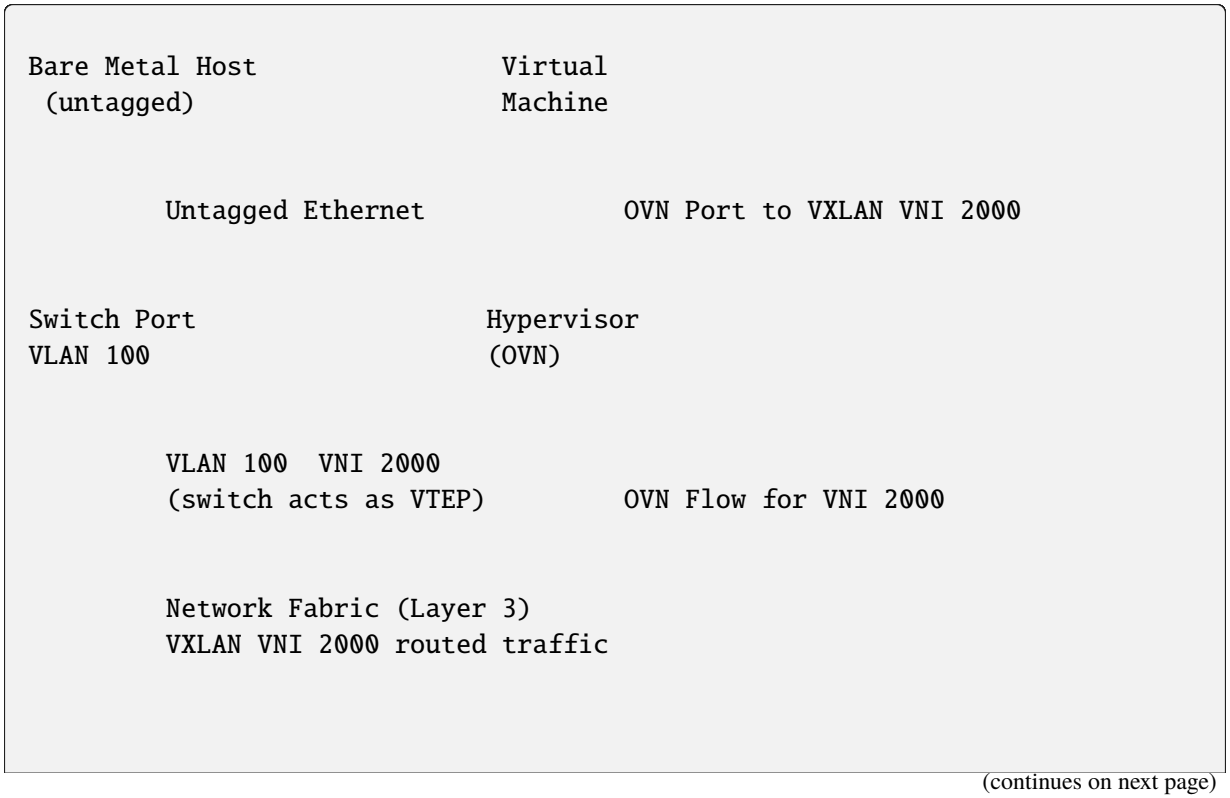
   Note

   The future integration of Neutron BGP EVPN Type-2 routes may not work for network types set to Geneve. Ultimately it works in the case where the baremetal-l2vni plugin creates a direct localnet port attachment, because internally OVN treats the packets with no difference as they traverse OVN itself. The Ironic project expects to test that integration once available and when time permits and is expected to then update documentation accordingly.

3. **Switch Configuration Driver** Programs the physical switches to create VNI-to-VLAN mappings and attach VLANs to physical ports. The networking-generic-switch ML2 plugin provides this functionality for common switch vendors. Third party ML2 plugins can be extended to support such a functionality by their maintainers.

Data Flow

The following diagram illustrates the packet flow for VXLAN bare metal networking:



(continues on next page)

(continued from previous page)

```
OVN Network Node
br-ex VLAN tag 100   VLAN 100 subinterface

localnet port        Bridges to OVN logical network

OVN Network
VNI 2000
```

The traffic flow works as follows:

1. Bare metal host sends untagged Ethernet frames
2. Physical switch tags frames with VLAN ID (e.g., VLAN 100)
3. Switch maps VLAN 100 to VXLAN VNI 2000 and encapsulates
4. VXLAN traffic is routed through the network fabric
5. Remote switches and OVN network nodes with the same VNI mapping receive the traffic
6. OVN network node receives VLAN 100 on br-ex trunk port, which is mapped to an OVN localnet port in the OVN logical network
7. Virtual machines on the same OVN network can communicate directly through the overlay network.

Note

In the diagram above, an OVN Network Node is being used. It is entirely possible to utilize hypervisors as well and perform some level of direct attachment. Such configurations are entirely dependent upon the operator applied configuration to OVN itself and all flows would exist within the constraints of OVN.

Note

When a tenant network is configured to be a `vxlan` network in an OVN enabled Neutron deployment, OVN does not utilize `vxlan` to transport the packets to the hypervisor. The packets are sent directly to the hypervisor over fabric which OVN orchestrates with additional header information which is VXLAN incompatible.

OVN Integration

In an OVN-based deployment, the mechanism driver creates localnet ports in the OVN logical network. These ports represent attachments to physical networks and are mapped to specific bridge mappings on OVN chassis nodes.

The OVN controller manages the lifecycle of these localnet ports, enabling them on nodes that have appropriate bridge mappings and disabling them when network topology changes. This provides automatic failover and load distribution across multiple network nodes.

Requirements

To deploy VXLAN networking for bare metal, the following components and configuration are required:

Infrastructure Requirements

- **VXLAN-capable physical switches** Your switch fabric must support VXLAN. Ingress replication is the recommended configuration and first targeted support case. Additional models of VXLAN traffic management within a VXLAN network are expected to evolve in the networking-generic-switch ML2 plugin as this feature set gets more adoption.
- **Neutron with OVN** The implementation uses OVN as the network virtualization platform. Other options are not presently supported.
- **Network nodes with physical connectivity** One or more network nodes (dedicated or compute nodes) with physical network interfaces connected to the switch fabric with matching `physical_network` network names.

Software Components

The following Neutron ML2 plugins must be installed and configured:

1. **networking-baremetal** Provides the `baremetal-l2vni` mechanism driver.
2. **networking-generic-switch** (or alternative switch ML2 plugin) Programs physical switches with VXLAN VNI and VLAN configuration.

Configuration Requirements

Mechanism Driver Ordering

In `/etc/neutron/plugins/ml2/ml2_conf.ini`, the mechanism drivers must be configured in the correct order:

```
[ml2]
mechanism_drivers = ovn,baremetal-l2vni,genericswitch,baremetal
```

The `baremetal-l2vni` driver must appear:

- After `ovn` (to ensure OVN processes the port binding first)
- Before `baremetal` (to create the hierarchical binding)
- Before the switch configuration driver (e.g., `genericswitch`)

Physical Network Configuration

Each bare metal port must be associated with a physical network. This can be configured either:

1. **Per-port** using the `physical_network` attribute:

   ```
   openstack baremetal port set --physical-network physnet1 <port-uuid>
   ```

2. **Default** in `/etc/neutron/plugins/ml2/ml2_conf.ini`:

   ```
   [l2vni]
   default_physical_network = physnet1
   ```

The physical network must have VLAN segments allocated. The mechanism driver will allocate VLANs from this pool as lower binding segments for VXLAN networks.

Warning

Physical network names and VLAN ranges must be consistently configured across all Neutron services. Inconsistent configuration will result in port binding failures.

Network Type Configuration

For users to create VXLAN networks without administrative privileges, set the tenant network type:

```
[ml2]
project_network_types = vxlan

[ml2_type_vxlan]
vni_ranges = 5000:10000
```

Note

The `project_network_types` parameter can be set to `geneve`, which is the default value for the setting. This is known to work for the `baremetal-l2vni` model of using `localnet` ports.

Warning

Provider networks (networks created with `--provider-*` options) are not supported with this model.

Physical Switch Configuration

Switches must be configured with:

- VXLAN support enabled
- BGP EVPN or multicast configuration for control plane to enable a VNI to be configured and the traffic to reach from one VTEP to another on the same network.
- Ingress replication configured (recommended)
- Appropriate interface addresses for VTEP functionality

The specific configuration varies by vendor and is typically done through the switch management interface or automation tools.

Connectivity Models

There are two primary models for connecting the OpenStack cloud to the physical switch fabric for VXLAN traffic.

Option 1: Hierarchical Port Binding with Trunk Ports

This is the currently implemented and recommended approach for most deployments.

In this model:

- Each OVN network node has one or more physical network interfaces connected to the switch fabric
- These interfaces are configured as 802.1Q trunk ports in Neutron
- For each VXLAN network that needs physical connectivity, a VLAN subport is created on the trunk
- The VLAN ID matches the lower binding segment allocated by the mechanism driver
- The physical switch maps the VLAN to the VXLAN VNI

Advantages:

- Leverages switch ASICs for VXLAN encapsulation/decapsulation, providing near wire-speed performance
- Proven approach already in use by downstream operators
- Straightforward configuration and troubleshooting

Limitations:

- Number of networks is limited by available VLANs per physical network (typically 4,096, but often lower due to switch constraints)
- May require careful planning of physical network assignments to avoid VLAN exhaustion
- Adds latency for VM-to-baremetal traffic as packets must traverse to network nodes

Best For:

- Bare metal-heavy deployments
- Environments requiring high bandwidth between bare metal nodes as traffic does not bridge across Network Nodes.
- Use cases where switch performance is critical

Option 2: BGP EVPN with Type-2 Routes

This approach is under development in the Neutron community and represents a future connectivity model.

In this model:

- Each compute node participates in BGP EVPN as a VTEP
- MAC address routes are distributed via BGP Type-2 routes
- VXLAN encapsulation/decapsulation occurs on hypervisors
- Physical switches also participate in the EVPN fabric

Advantages:

- Distributed encapsulation/decapsulation across compute nodes

- No VLAN constraints on network nodes
- Better scaling for VM-heavy environments

Limitations:

- CPU overhead on hypervisors for packet encapsulation/decapsulation
- Lower per-stream performance compared to switch ASICs
- More complex BGP configuration required

Best For:

- VM-heavy deployments with some bare metal nodes
- Environments prioritizing horizontal scalability
- Use cases where distributed processing is preferred

Choosing a Connectivity Model

The choice between connectivity models depends on your workload characteristics:

Use Hierarchical Port Binding if:

- Most traffic is between bare metal nodes
- You need maximum bandwidth and lowest latency
- Your switch fabric is designed for VXLAN/EVPN
- VLAN limits per physical network are acceptable
- Your Traffic flow performance is critical.

Use BGP EVPN (when available) if:

- Most traffic is between VMs with occasional bare metal communication
- You want to distribute encapsulation load across compute nodes
- VLAN constraints are problematic
- Your deployment is VM-centric with bare metal as a secondary workload

Note

The physical network modeling in OpenStack has historically represented entirely different broadcast or management domains. But ultimately they are a human management construct where a single node **can** have multiple physical networks, and it is just configuration to allow operators to model how they would prefer traffic to flow and define the overall structural bounds of a network.

Supported Switch Vendors

The networking-generic-switch plugin provides VXLAN support for the following switch operating systems:

- **Cisco Nexus** (NX-OS)
- **Arista EOS**

- **Juniper Junos**
- **SONiC** (vendor-neutral network OS)
- **Cumulus Networks NVUE**

Note

Additional switch vendors may be supported through third-party ML2 plugins. Check your switch vendors documentation for OpenStack integration options.

Unsupported Switch Platforms

Dell OS10
: While Dell OS10 supports VXLAN, it uses a three-tier mapping system (VLAN Virtual Network VNI) that requires a 16-bit virtual network ID in addition to the VLAN and VNI. This architecture cannot be easily automated with the current hierarchical port binding model and is therefore not supported. Furthermore, there are recommended limits to the number of VLAN on any given trunk interface which operators would need to be mindful of.

Switch Configuration Details

Each switch vendor requires specific configuration parameters. Common requirements include:

- **BGP ASN** - Autonomous System Number for BGP EVPN
- **VTEP interface** - Loopback or interface address used as the VXLAN source
- **Route distinguisher/target** - BGP EVPN routing parameters

Consult the networking-generic-switch documentation and your switch vendors VXLAN/EVPN configuration guides for specific parameters.

Operational Procedures

Creating Networks

With VXLAN support properly configured, regular (non-admin) users can create networks:

```
openstack network create my-network
```

The network will automatically:

1. Be assigned a VNI from the configured range
2. Have lower binding segments (VLANs) allocated per physical network
3. Be ready for bare metal and VM attachments

Attaching Bare Metal Nodes

1. Ensure the bare metal port has a `physical_network` set:

   ```
   openstack baremetal port set --physical-network physnet1 <port-uuid>
   ```

2. Create a port on the VXLAN network:

```
openstack port create --network my-network bare-port
```

3. Attach the port to a bare metal node:

```
openstack baremetal node vif attach <node> <port-uuid>
```

The mechanism driver will:

- Allocate a VLAN from the physical network if not already allocated
- Program the physical switch to create the VNI and map it to the VLAN
- Attach the VLAN to the physical switch port

Monitoring Network Segments

View the hierarchical binding segments for a network:

```
openstack network segment list --network my-network
```

You should see at least two segments:

- **Top segment**: network_type `vxlan` or `geneve` with the VNI
- **Bottom segment(s)**: network_type `vlan` with VLAN IDs per physical network

Troubleshooting

Port Binding Failures

Symptom: Baremetal node transitions from `deploying` to `deploy fail` and back to `available` due to the Neutron port binding failing.

Alternative Symptom: Bare metal node is stuck in wait call-back or deploying state for a period of time before going to deploy fail state and transitioning back to available.

Largely this pattern of behavior will be governed by the setting of *neutron.fail_on_port_binding_failure*, where the default value will represent the pattern observed in the first symptom, and the latter symptoms will represent the behavior when set to False.

Ultimately, this is due to one of the common causes noted below.

Common Causes:

1. **No physical_network configured**

   Check if the port has a physical_network:

   ```
   openstack baremetal port show <port-uuid> -c physical_network
   ```

   Set if missing:

   ```
   openstack baremetal port set --physical-network physnet1 <port-uuid>
   ```

Note

Networking-baremetal has a configuration option which can be placed into the [baremetal_l2vni] section of the Neutron ml2.conf files named `default_physical_network` which can be used in smaller scale cases where a physical network is not found on physical ports.

2. **VLAN exhaustion on physical network**

   Check available VLANs in `/etc/neutron/plugins/ml2/ml2_conf.ini`:

   ```
   [ml2_type_vlan]
   network_vlan_ranges = physnet1:100:200
   ```

   If the range is exhausted, either:

   - Expand the VLAN range (requires verification that the range can be expanded on the physical switch network.)
   - Delete unused networks to free VLANs
   - Use a different physical network. Physical networks are abstract in concept and can also co-exist, but must not overlap when it comes to the VLAN ranges allocated when they are not completely distinct physical networks.

Note

The Ironic project advises over-allocating the number of VLANs to be reserved for connecting workloads, as the effort to expand the range may seem trivial in configuration, but will require checking that the range is actually available on the switch which may yield additional unplanned work to make the new desired range available.

3. **Mechanism driver ordering incorrect**

   Verify the mechanism_drivers order in `ml2_conf.ini`, such that the baremetal-l2vni driver is *before* the switch ML2 driver managing switchports. Furthermore, generally baremetal should be last in the list of ports.:

   ```
   [ml2]
   mechanism_drivers = ovn,baremetal-l2vni,genericswitch,baremetal
   ```

4. **Physical network not configured consistently**

   Ensure the physical network name and VLAN ranges match across all Neutron configuration files on all nodes. A miss-matched configuration across Neutron nodes can result in configuration and available segments behaving unpredictably.

   Ideally, the physical network mapping and range data needs to be consistent across all neutron nodes with the exception of the actual bridge mapping data for OVN itself. That can be varied as required across a deployment to meet the needs of the cloud infrastructure.

Missing Lower Binding Segment

Symptom: Network segment list shows only the top VXLAN segment, no bottom VLAN segment.

Diagnosis:

Check the Neutron server logs for errors related to segment allocation:

```
sudo journalctl -u neutron-api.service | grep -i segment
```

Note

The service name may differ based upon distribution or packages, but generally neutron lots all such issues in the API service log.

Possible Causes:

- baremetal-l2vni driver not loaded or misconfigured
- No ports have been created on the network yet (lower segments are allocated lazily when first port is created)
- Physical network not defined in ml2_conf.ini or on the physical ports of baremetal nodes.

Switch Configuration Errors

Symptom: Lower segment exists but node cannot communicate on network.

Diagnosis:

1. Check switch logs for configuration errors
2. Verify VNI was created on the switch
3. Verify VLAN-to-VNI mapping
4. Check that VLAN is assigned to the correct physical ports
5. Check that OVN *IS* forwarding packets or not. OVN will not show the localnet port as up, but generally DHCP will be functional. You may need to utilize tools like `tcpdump` to verify OVN is properly attached, and that packets are flowing.
6. Check that a *router* is assigned to the OVN network, without a router OVN might not be locking the ports to a `HA_Chassis_Group` which can result in unexpected OVN behavior.

The specific troubleshooting steps vary by switch vendor. Consult the networking-generic-switch logs and your switch management interface.

Frequently Asked Questions

Can I use this on a single switch?

Yes. You dont need a multi-switch VXLAN fabric to use this functionality. The hierarchical port binding and VNI-to-VLAN mapping work on a single switch.

However, using VXLAN on a single switch doesnt provide the primary benefits of VXLAN (scaling beyond VLAN limits, Layer 3 fabric, etc.). It may still be useful for testing or as a migration path to a larger VXLAN deployment.

Warning

Allowing all VLANs to traverse your entire network creates a large broadcast domain and negates many benefits of VXLAN. Use appropriate VLAN pruning on trunk links.

Can I use this for QinQ (nested VLAN) traffic?

This is not a tested or supported use case. The VXLAN-to-VLAN translation model assumes standard Ethernet frames. QinQ (802.1ad) support would depend on switch capabilities and is not guaranteed to work.

How do I verify a lower segment was created?

Use the network segment list command:

```
openstack network segment list --network <network>
```

You should see multiple segments with the same network ID:

- One with `network_type: vxlan` or `geneve`
- One or more with `network_type: vlan`

What if DHCP doesnt work?

VXLAN networks with DHCP require the DHCP agent to bind to the network. This should happen automatically when the network is created.

Verify DHCP port exists:

```
openstack port list --network <network> --device-owner \
    network:dhcp
```

If missing, check the DHCP agent logs:

```
sudo journalctl -u neutron-dhcp-agent
```

Alternatively, you may also need to check the OVN port configuration as well if you are not presently utilizing the neturon-dhcp-agent and are instead leveraging native DHCP support in OVN.

Can I tune multicast/BUM traffic handling?

The current implementation is designed for ingress replication mode. While this is configurable on the switches themselves through BGP EVPN settings, the ML2 plugin does not expose these controls.

As use cases emerge, additional configuration options may be added to networking-generic-switch to provide vendor-specific tuning. Please feel free to open a bug in the networking-generic-switch launchpad if you have specific requirements or needs.

Can I mix VLAN and VXLAN networks on the same bare metal node?

Yes. A bare metal node can have multiple ports, some attached to traditional VLAN networks and others to VXLAN networks. Each port is handled independently during binding.

Known Limitations

VLAN Constraints per Physical Network

Each physical network has a limited number of VLANs available (typically 4,096, but often fewer due to switch vendor limits). Since lower binding segments consume VLANs, the number of unique VXLAN networks accessible per physical network is bounded by this limit.

Mitigation: Define multiple physical networks to distribute VLAN allocation across different switch fabrics or VLAN ranges.

Warning

Operationally, you should avoid mingling VXLAN and VLAN network types on larger switch networks because of the overall management overhead created in such setups.

Provider Networks Not Supported

VXLAN networks created with `--provider-*` options are not supported in this model. The hierarchical binding mechanism expects to allocate the lower segment automatically.

Mitigation: Use regular tenant networks with Geneve or VXLAN as the default network type.

Performance Depends on Connectivity Model

The hierarchical port binding model with trunk ports provides excellent bare metal-to-bare metal performance but introduces a hop through network nodes for VM-to-bare metal traffic. This may add latency compared to pure overlay approaches.

Mitigation: Size your network node capacity appropriately and use multiple network nodes with OVN HA chassis groups for load distribution.

Recovery from Hardware Failures

If a physical switch fails and is replaced, the VNI and VLAN configuration must be re-applied. Currently, there is no automatic mechanism to re-trigger port binding after switch replacement, although upstream development is underway to add such a capability, see change 973413 for more details.

Mitigation: Document your network-to-physical-network mappings and utilize switch configuration backup utilities.

No OVN Geneve Support for Physical Fabric

While this implementation translates OVN Geneve tenant networks to VXLAN VNIs on physical switches, it does not support native Geneve protocol on the physical switch fabric (as Geneve is not widely supported by switch vendors).

Additional Resources

Design Specifications

- **VXLAN Support Specification**: https://review.opendev.org/c/openstack/ironic-specs/+/959401

External Documentation

- **VXLAN RFC 7348**: https://datatracker.ietf.org/doc/html/rfc7348
- **OVN Documentation**: https://docs.ovn.org/
- **Neutron OVN Installation**: https://docs.openstack.org/neutron/latest/install/ovn/manual_install.html

This content has been moved to *Networking with the Bare Metal service*.

CHAPTER

FIVE

CONTRIBUTOR GUIDE

5.1 Developers Guide

5.1.1 Getting Started

If you are new to ironic, this section contains information that should help you get started as a developer working on the project or contributing to the project.

This guide assumes you have read the OpenDev getting started documentation. It will also be helpful to be familiar with OpenStack contributors documentation, which contains basic information about how to use many of the community tools and OpenStack practices.

Basic information about setting up development environments with devstack or bifrost, or getting unit tests running can be found here:

Developer Quick-Start

This is a quick walkthrough to get you started developing code for Ironic. This assumes you are already familiar with submitting code reviews to an OpenStack project. If you are not, please begin by following the steps in the OpenDev infra manual to get yourself familiar with the general git workflow we use.

This guide is primarily technical in nature; for information on how the Ironic team organizes work, please see Ironics contribution guide.

This document covers *Git hooks*, *Unit Testing Environment*, and *Integrated Testing Environments*. New contributors are recommended to setup git hooks, then continue with unit tests.

Git hooks

Ironic uses multiple software packages to ensure code is styled as expected. A convenient way to ensure your code complies with these rules is via the use of pre-commit.

To configure your environment for automatic checking of code spelling and linting before commit, install `pre-commit`:

```
pip install --user pre-commit
cd /path/to/ironic/checkout
pre-commit install --allow-missing-config
```

Now, before any commit, any changed lines will be run through our `pre-commit` checks. Contributors are still encouraged to run `pep8` and `codespell` tox environments before pushing code as an extra check.

More information about running tests in tox available in *Unit Testing Environment*.

Integrated Testing Environments

The ultimate in development environments for Ironic is a working system, with mock bare metal hardware and a fully functional API service. There are three ways to get environment, listed below.

Note

These environments may use automation that assume you are running on a VM. Please do not use these environments on a system that you are not willing to have wiped and reinstalled when complete.

Table 1: Testing Environments

Environment	Description/Uses	How-To
Devstack	Useful for testing Ironic with other OpenStack services. Also the environment required for running or building Ironics tempest tests. Recommended for new contributors.	*Deploying Ironic with DevStack*
Bifrost	Used for testing Ironic standalone with minimal setup or using real hardware, or testing bifrost changes directly.	*Bifrost Development Environment*
Local	Ironic services running locally, without any other OpenStack services. This can be useful for rapid prototyping, debugging, or testing database migrations.	*Exercising Ironic Services Locally*

Unit Testing Environment

For most people, unit testing is the quickest and easiest way to check the validity of a change. Unlike a fully integrated testing environment, unit tests can generally be safely run on a developers workstation.

Ironic uses tox to orchestrate unit tests and documentation building. Contributors are strongly encouraged to validate code passes unit tests under a supported version of python before pushing up a change. See the Project Testing Interface for the exact versions of python supported currently.

System Prerequisites

The following packages cover the prerequisites for a local development environment on most current distributions.

- Ubuntu/Debian:

```
sudo apt-get install build-essential python3-dev libssl-dev python3-pip␣
↪libmysqlclient-dev libxml2-dev libxslt-dev git git-review libffi-dev␣
↪gettext ipmitool psmisc graphviz libjpeg-dev qemu-utils
```

- RHEL/CentOS/Fedora:

```
sudo dnf install python3-devel openssl-devel python3-pip mysql-devel␣
↪libxml2-devel libxslt-devel git git-review libffi-devel gettext␣
↪ipmitool psmisc graphviz gcc libjpeg-turbo-devel qemu-img
```

To run the tests locally, it is a requirement that your terminal emulator supports unicode with the `en_US.UTF8` locale. If you use locale-gen to manage your locales, make sure you have enabled `en_US.UTF8` in `/etc/locale.gen` and rerun `locale-gen`.

Python Prerequisites

We suggest to use at least tox 3.9, if your distribution has an older version, you can install it using pip system-wise or better per user using the user option that by default will install the binary under $HOME/.local/bin, so you need to be sure to have that path in $PATH; for example:

```
pip install tox --user
```

will install tox as ~/.local/bin/tox

You may need to explicitly upgrade virtualenv if youve installed the one from your OS distribution and it is too old (tox will complain). You can upgrade it individually, if you need to:

```
pip install --upgrade virtualenv --user
```

Running Unit Tests Locally

If you havent already, Ironic source code should be pulled directly from git:

```
# from a user-writable directory, usually $HOME or $HOME/dev
git clone https://opendev.org/openstack/ironic
cd ironic
```

Most of the time, you will want to run codespell, unit tests, and pep8 checks. This can be done with the following command:

```
tox -e codespell,pep8,py3
```

Ironic has multiple test environments that can be run by tox. An incomplete list of environments and what they do is below. Please reference the `tox.ini` file in the project youre working on for a complete, up-to-date list.

Table 2: Tox Environments

Environment	Description
pep8	Run style checks on code, documentation, and release notes.
codespell	Check code against a list of known-misspelled words.
py<version>	Run unit tests with the specified python version. For example, `py310` will run the unit tests with python 3.10.
unit-with-driver-libs	Run unit tests with the default python3 on the system, but also includes driver-specific libraries and the tests they enable.
mysql-migrations	Run MySQL database migration unit tests. Setup database first using `tools/test-setup.sh` in Ironic repo.
docs	Build and validate documentation.
releasenotes	Build and validate release notes using `reno`.
api-ref	Build and validate API reference documentation.
genconfig	Generates example configuration file.
genpolicy	Generates example policy configuration file.
venv	Creates a venv, with dependencies installed, for running commands in e.g. `tox -evenv -- reno new my-release-note`

You may also pass options to the test programs using positional arguments. To run a specific unit test, this passes the desired test (regex string) to stestr:

```
# run a specific test for Python 3.10
tox -epy310 -- test_conductor
```

Debugging unit tests

In order to break into the debugger from a unit test we need to insert a breaking point to the code:

```
import pdb; pdb.set_trace()
```

Then run `tox` with the debug environment as one of the following:

```
tox -e debug
tox -e debug test_file_name
tox -e debug test_file_name.TestClass
tox -e debug test_file_name.TestClass.test_name
```

For more information see the oslotest documentation.

Other tests

Ironic also has a number of tests built with Tempest. For more information about writing or running those tests, see *Add Ironic Tempest Plugin*.

OSProfiler Tracing in Ironic

OSProfiler is an OpenStack cross-project profiling library. It is being used among OpenStack projects to look at performance issues and detect bottlenecks. For details on how OSProfiler works and how to use it in ironic, please refer to OSProfiler Support Documentation.

Building developer documentation

If you would like to build the documentation locally, eg. to test your documentation changes before uploading them for review, run these commands to build the documentation set:

- On the machine with the ironic checkout:

```
# change into the ironic source code directory
cd ~/ironic

# build the docs
tox -edocs
```

To view the built documentation locally, open up the top level index.html in your browser. For an example user named bob with the Ironic checkout in their homedir, the URL to put in the browser would be:

```
file:///home/bob/ironic/doc/build/html/index.html
```

If youre building docs on a remote VM, you can use pythons SimpleHTTPServer to setup a quick web-server to check your docs build:

```
# Change directory to the newly built HTML files
cd ~/ironic/doc/build/html/

# Create a server using python on port 8000
python3 -m SimpleHTTPServer 8000

# Now use your browser to open the top-level index.html located at:
http://remote_ip:8000
```

Deploying Ironic with DevStack

DevStack may be configured to deploy Ironic, setup Nova to use the Ironic driver and provide hardware resources (network, baremetal compute nodes) using a combination of OpenVSwitch and libvirt. It is highly recommended to deploy on an expendable virtual machine and not on your personal work station.

Ironic has a much smaller support envelope than devstack generally. For best results, use Ubuntu 24.04 (Noble) as the base OS for your devstack installation.

See also

https://docs.openstack.org/devstack/latest/

Note

The devstack demo tenant has read-only access to Ironics API. This is sufficient for all the examples below. Should you want to create or modify bare metal resources directly (ie. through Ironic rather than through Nova) you will need to use the devstack admin tenant.

Basic process

Create a stack user with proper permissions using script from devstack:

```
git clone https://opendev.org/openstack/devstack.git devstack
sudo ./devstack/tools/create-stack-user.sh
```

Switch to the stack user and clone DevStack:

```
sudo su - stack
git clone https://opendev.org/openstack/devstack.git devstack
```

From the *Configurations* section below, create a `local.conf` file.

Once you have the configuration in place and ready to go, you can deploy devstack with:

```
./stack.sh
```

Note

Devstack configurations change frequently. If you are having trouble getting one of the below configs to work, please file a bug against Ironic or ask on #openstack-ironic in OFTC.

Configurations

Ironic

Create devstack/local.conf with minimal settings required to enable Ironic. This does not configure Nova to operate with Ironic.

An example local.conf that enables the `direct` *deploy interface* and uses the `ipmi` hardware type by default:

```
cd devstack
cat >local.conf <<END
[[local|localrc]]
# Enable only minimal services
disable_all_services
enable_service g-api
enable_service key
enable_service memory_tracker
enable_service mysql
# NOTE(JayF): q-* is an old way of referring to neutron (fka quantum)
#             services. As of 2025-05-30, devstack won't work with all q-
#             configurations; it also won't work with all neutron-
#             service. Defining both gives us best possible behavior.
enable_service neutron-api q-api
enable_service neutron-agent q-agt
enable_service neutron-dhcp q-dhcp
enable_service neutron-l3 q-l3
enable_service neutron-metadata-agent q-meta
enable_service neutron-metering q-metering
```

(continues on next page)

(continued from previous page)

```
enable_service rabbit

# Credentials
ADMIN_PASSWORD=password
DATABASE_PASSWORD=password
RABBIT_PASSWORD=password
SERVICE_PASSWORD=password
SERVICE_TOKEN=password

# Set glance's default limit to be baremetal image friendly
GLANCE_LIMIT_IMAGE_SIZE_TOTAL=5000

# Enable Ironic plugin
enable_plugin ironic https://opendev.org/openstack/ironic

# Create 3 virtual machines to pose as Ironic's baremetal nodes.
IRONIC_VM_COUNT=3
IRONIC_BAREMETAL_BASIC_OPS=True
DEFAULT_INSTANCE_TYPE=baremetal

IRONIC_RPC_TRANSPORT=json-rpc
IRONIC_RAMDISK_TYPE=tinyipa

# Enable additional hardware types, if needed.
#IRONIC_ENABLED_HARDWARE_TYPES=ipmi,fake-hardware
# Don't forget that many hardware types require enabling of additional
# interfaces, most often power and management:
#IRONIC_ENABLED_MANAGEMENT_INTERFACES=ipmitool,fake
#IRONIC_ENABLED_POWER_INTERFACES=ipmitool,fake
#IRONIC_DEFAULT_DEPLOY_INTERFACE=direct

# Change this to alter the default driver for nodes created by devstack.
# This driver should be in the enabled list above.
IRONIC_DEPLOY_DRIVER="ipmi"

# The parameters below represent the minimum possible values to create
# functional nodes.
IRONIC_VM_SPECS_RAM=2500
IRONIC_VM_SPECS_DISK=3

# Size of the ephemeral partition in GB. Use 0 for no ephemeral partition.
IRONIC_VM_EPHEMERAL_DISK=0

# To build your own IPA ramdisk from source, set this to True
IRONIC_BUILD_DEPLOY_RAMDISK=False

INSTALL_TEMPEST=False
VIRT_DRIVER=ironic
```

(continues on next page)

(continued from previous page)

```
# By default, DevStack creates a 10.0.0.0/24 network for instances.
# If this overlaps with the hosts network, you may adjust with the
# following.
IP_VERSION=4
FIXED_RANGE=10.1.0.0/20
IPV4_ADDRS_SAFE_TO_USE=10.1.0.0/20
NETWORK_GATEWAY=10.1.0.1

Q_AGENT=openvswitch
Q_ML2_PLUGIN_MECHANISM_DRIVERS=openvswitch
Q_ML2_TENANT_NETWORK_TYPE=vxlan

# Log all output to files
LOGFILE=/opt/stack/devstack.log
LOGDIR=/opt/stack/logs
IRONIC_VM_LOG_DIR=/opt/stack/ironic-bm-logs

END
```

Ironic with Nova

With this config, Nova will be configured to use Ironics virt driver. Ironic will have the `direct` *deploy interface* enabled and use the `ipmi` hardware type with this config:

```
cd devstack
cat >local.conf <<END
[[local|localrc]]
# Credentials
ADMIN_PASSWORD=password
DATABASE_PASSWORD=password
RABBIT_PASSWORD=password
SERVICE_PASSWORD=password
SERVICE_TOKEN=password
SWIFT_HASH=password
SWIFT_TEMPURL_KEY=password

# Set glance's default limit to be baremetal image friendly
GLANCE_LIMIT_IMAGE_SIZE_TOTAL=5000

# Enable Ironic plugin
enable_plugin ironic https://opendev.org/openstack/ironic

# Disable nova novnc service, ironic does not support it anyway.
disable_service n-novnc

# Enable Swift for the direct deploy interface.
enable_service s-proxy s-object s-container s-account
```

(continues on next page)

(continued from previous page)

```
# Disable Horizon
disable_service horizon

# Disable Cinder
disable_service cinder c-sch c-api c-vol

# Configure networking by disabling OVN and enabling Neutron w/OVS.
disable_service ovn-controller
disable_service ovn-northd
disable_service neutron-ovn-metadata-agent

enable_service neutron-agent q-agt
enable_service neutron-dhcp q-dhcp
enable_service neutron-l3 q-l3
enable_service neutron-metadata-agent q-meta
enable_service neutron-api q-svc
Q_AGENT=openvswitch
Q_ML2_PLUGIN_MECHANISM_DRIVERS="openvswitch"
Q_ML2_TENANT_NETWORK_TYPE="vxlan"
Q_USE_SECGROUP="False"

# By default, devstack assumes you have IPv4 and IPv6 access. If you are on
# a host with IPv6 disabled, set the value below.
# IP_VERSION=4

# Swift temp URL's are required for the direct deploy interface
SWIFT_ENABLE_TEMPURLS=True

# Support via emulated BMC exists for the following hardware types, and
# VMs to back them will be created by default unless IRONIC_IS_HARDWARE is
# True.
#  - ipmi (VirtualBMC)
#  - redfish (sushy-tools)
#
# If you wish to change the default driver for nodes created by devstack,
# you can do so by setting IRONIC_DEPLOY_DRIVER to the name of the driver
# you wish used by default, and ensuring that driver (along with others) is
# enabled.
IRONIC_DEPLOY_DRIVER=ipmi

# Example: Uncommenting these will configure redfish by default
#IRONIC_ENABLED_HARDWARE_TYPES=redfish,ipmi,fake-hardware
#IRONIC_DEPLOY_DRIVER=redfish
# Don't forget that many hardware types require enabling of additional
# interfaces, most often power and management:
#IRONIC_ENABLED_MANAGEMENT_INTERFACES=redfish,ipmitool,fake
#IRONIC_ENABLED_POWER_INTERFACES=redfish,ipmitool,fake

IRONIC_VM_COUNT=3
```

(continues on next page)

(continued from previous page)

```
IRONIC_BAREMETAL_BASIC_OPS=True
DEFAULT_INSTANCE_TYPE=baremetal

# You can also change the default deploy interface used.
#IRONIC_DEFAULT_DEPLOY_INTERFACE=direct

# The parameters below represent the minimum possible values to create
# functional nodes.
IRONIC_VM_SPECS_RAM=2500
IRONIC_VM_SPECS_DISK=10

# Size of the ephemeral partition in GB. Use 0 for no ephemeral partition.
IRONIC_VM_EPHEMERAL_DISK=0

# To build your own IPA ramdisk from source, set this to True
IRONIC_BUILD_DEPLOY_RAMDISK=False

VIRT_DRIVER=ironic

# By default, DevStack creates a 10.0.0.0/24 network for instances.
# If this overlaps with the hosts network, you may adjust with the
# following.
# NETWORK_GATEWAY=10.1.0.1
# FIXED_RANGE=10.1.0.0/24
# FIXED_NETWORK_SIZE=256

# Log all output to files
LOGFILE=$HOME/devstack.log
LOGDIR=$HOME/logs
IRONIC_VM_LOG_DIR=$HOME/ironic-bm-logs

END
```

Note

For adding *Add Ironic Tempest Plugin* support to this configuration, see the *Add Ironic Tempest Plugin* section of this document.

Stable Branch Configuration

If you want to run Ironic in a branch other than master, you need to specify the correct version Ironic you want at the end of the enable_plugin line, e.g. `enable_plugin ironic https://opendev.org/openstack/ironic stable/2024.1`.

In almost all situations, youll also want any other associated OpenStack services to run in that version, too. Just add this config:

```
TARGET_BRANCH=stable/2024.1
```

VM Sizing and Config

Ironic devstack uses VMs to masquerade as baremetal for testing.

You can change the size and number of fake baremetal VMs spun up via `IRONIC_VM_COUNT`, `IRONIC_VM_SPECS_RAM` and, `IRONIC_VM_SPECS_DISK` in your devstack `local.conf`.

If you want to test fully-featured DIB ramdisks in one of these configs, increase the value of `IRONIC_VM_SPECS_RAM`. A value of `IRONIC_VM_SPECS_RAM=4096` should be sufficient. If necessary, you can reduce the `IRONIC_VM_COUNT` to free up additional ram for fewer, larger VMs.

Other Devstack Configurations

There are additional devstack configurations in other parts of contributor documentation:

Ironic Boot-from-Volume with DevStack

This guide shows how to setup DevStack for enabling boot-from-volume feature, which has been supported from the Pike release.

This scenario shows how to setup DevStack to enable nodes to boot from volumes managed by cinder with VMs as baremetal servers.

DevStack Configuration

The following is `local.conf` that will setup DevStack with 3 VMs that are registered in ironic. A volume connector with IQN is created for each node. These connectors can be used to connect volumes created by cinder. The detailed description for DevStack is at *Deploying Ironic with DevStack*.

```
[[local|localrc]]

enable_plugin ironic https://opendev.org/openstack/ironic

IRONIC_STORAGE_INTERFACE=cinder

# Credentials
ADMIN_PASSWORD=password
DATABASE_PASSWORD=password
RABBIT_PASSWORD=password
SERVICE_PASSWORD=password
SERVICE_TOKEN=password
SWIFT_HASH=password
SWIFT_TEMPURL_KEY=password

# Set glance's default limit to be baremetal image friendly
GLANCE_LIMIT_IMAGE_SIZE_TOTAL=5000

# Enable Neutron which is required by Ironic and disable nova-network.
disable_service n-net
disable_service n-novnc
enable_service q-svc
enable_service q-agt
enable_service q-dhcp
```

(continues on next page)

(continued from previous page)

```
enable_service q-l3
enable_service q-meta
enable_service neutron

# Enable Swift for the direct deploy interface.
enable_service s-proxy
enable_service s-object
enable_service s-container
enable_service s-account

# Disable Horizon
disable_service horizon

# Disable Heat
disable_service heat h-api h-api-cfn h-api-cw h-eng

# Swift temp URL's are required for the direct deploy interface.
SWIFT_ENABLE_TEMPURLS=True

# Create 3 virtual machines to pose as Ironic's baremetal nodes.
IRONIC_VM_COUNT=3
IRONIC_BAREMETAL_BASIC_OPS=True
DEFAULT_INSTANCE_TYPE=baremetal

# Enable additional hardware types, if needed.
#IRONIC_ENABLED_HARDWARE_TYPES=ipmi,fake-hardware
# Don't forget that many hardware types require enabling of additional
# interfaces, most often power and management:
#IRONIC_ENABLED_MANAGEMENT_INTERFACES=ipmitool,fake
#IRONIC_ENABLED_POWER_INTERFACES=ipmitool,fake
#IRONIC_DEFAULT_DEPLOY_INTERFACE=direct

# Change this to alter the default driver for nodes created by devstack.
# This driver should be in the enabled list above.
IRONIC_DEPLOY_DRIVER=ipmi

# The parameters below represent the minimum possible values to create
# functional nodes.
IRONIC_VM_SPECS_RAM=2500
IRONIC_VM_SPECS_DISK=10

# Size of the ephemeral partition in GB. Use 0 for no ephemeral partition.
IRONIC_VM_EPHEMERAL_DISK=0

# To build your own IPA ramdisk from source, set this to True
IRONIC_BUILD_DEPLOY_RAMDISK=False

VIRT_DRIVER=ironic
```

(continues on next page)

(continued from previous page)

```
# By default, DevStack creates a 10.0.0.0/24 network for instances.
# If this overlaps with the hosts network, you may adjust with the
# following.
NETWORK_GATEWAY=10.1.0.1
FIXED_RANGE=10.1.0.0/24
FIXED_NETWORK_SIZE=256

# Log all output to files
LOGFILE=$HOME/devstack.log
LOGDIR=$HOME/logs
IRONIC_VM_LOG_DIR=$HOME/ironic-bm-logs
```

After the environment is built, you can create a volume with cinder and request an instance with the volume to nova:

```
# set up the user for authentication purposes
# note that all users can be seen in /etc/openstack/clouds.yaml
export OS_CLOUD=devstack-admin-demo

# query the image id of the default cirros image
image=$(openstack image show $DEFAULT_IMAGE_NAME -f value -c id)

# create keypair
ssh-keygen
openstack keypair create --public-key ~/.ssh/id_rsa.pub default

# create volume
volume=$(openstack volume create --image $image --size 1 my-volume -f value -
↪c id)

# spawn instance
openstack server create --flavor baremetal --volume $volume --key-name␣
↪default testing
```

You can also run an integration test that an instance is booted from a remote volume with tempest in the environment:

```
cd /opt/stack/tempest
tox -e venv-tempest -- pip install (path to the ironic-tempest-plugin␣
↪directory)
tox -e all -- ironic_tempest_plugin.tests.scenario.test_baremetal_boot_from_
↪volume
```

Please note that the storage interface will only indicate errors based upon the state of the node and the configuration present. As such a node does not exclusively have to boot via a remote volume, and as such `validate` actions upon nodes may be slightly misleading. If an appropriate `volume target` is defined, no error should be returned for the boot interface.

Ironic multitenant networking and DevStack

This guide will walk you through using OpenStack Ironic/Neutron with the ML2 `networking-generic-switch` plugin. The intent is to provide context in order to help contributors who may be trying to use networking-generic-switch. This is *not* intended for production use, but purely for development purposes.

Using VMs as baremetal servers

This scenario shows how to setup Devstack to use Ironic/Neutron integration with VMs as baremetal servers and ML2 `networking-generic-switch` that interacts with OVS.

DevStack Configuration

The following is `local.conf` that will setup Devstack with 3 VMs that are registered in ironic. `networking-generic-switch` driver will be installed and configured in Neutron.

```
[[local|localrc]]

# Configure ironic from ironic devstack plugin.
enable_plugin ironic https://opendev.org/openstack/ironic

# Install networking-generic-switch Neutron ML2 driver that interacts with OVS
enable_plugin networking-generic-switch https://opendev.org/openstack/
↪networking-generic-switch

# Add link local info when registering Ironic node
IRONIC_USE_LINK_LOCAL=True

IRONIC_ENABLED_NETWORK_INTERFACES=flat,neutron
IRONIC_NETWORK_INTERFACE=neutron

#Networking configuration
OVS_PHYSICAL_BRIDGE=brbm
PHYSICAL_NETWORK=mynetwork
IRONIC_PROVISION_NETWORK_NAME=ironic-provision
IRONIC_PROVISION_SUBNET_PREFIX=10.0.5.0/24
IRONIC_PROVISION_SUBNET_GATEWAY=10.0.5.1

Q_PLUGIN=ml2
ENABLE_TENANT_VLANS=True
Q_ML2_TENANT_NETWORK_TYPE=vlan
TENANT_VLAN_RANGE=100:150

# Credentials
ADMIN_PASSWORD=password
RABBIT_PASSWORD=password
DATABASE_PASSWORD=password
SERVICE_PASSWORD=password
SERVICE_TOKEN=password
SWIFT_HASH=password
```

(continues on next page)

(continued from previous page)

```
SWIFT_TEMPURL_KEY=password

# Enable Ironic API and Ironic Conductor
enable_service ironic
enable_service ir-api
enable_service ir-cond

# Disable nova novnc service, ironic does not support it anyway.
disable_service n-novnc

# Enable Swift for the direct deploy interface.
enable_service s-proxy
enable_service s-object
enable_service s-container
enable_service s-account

# Disable Horizon
disable_service horizon

# Disable Cinder
disable_service cinder c-sch c-api c-vol

# Disable Tempest
disable_service tempest

# Set glance's default limit to be baremetal image friendly
GLANCE_LIMIT_IMAGE_SIZE_TOTAL=5000

# Swift temp URL's are required for the direct deploy interface.
SWIFT_ENABLE_TEMPURLS=True

# Create 3 virtual machines to pose as Ironic's baremetal nodes.
IRONIC_VM_COUNT=3
IRONIC_BAREMETAL_BASIC_OPS=True

# Enable additional hardware types, if needed.
#IRONIC_ENABLED_HARDWARE_TYPES=ipmi,fake-hardware
# Don't forget that many hardware types require enabling of additional
# interfaces, most often power and management:
#IRONIC_ENABLED_MANAGEMENT_INTERFACES=ipmitool,fake
#IRONIC_ENABLED_POWER_INTERFACES=ipmitool,fake
#IRONIC_DEFAULT_DEPLOY_INTERFACE=direct

# Change this to alter the default driver for nodes created by devstack.
# This driver should be in the enabled list above.
IRONIC_DEPLOY_DRIVER=ipmi

# The parameters below represent the minimum possible values to create
# functional nodes.
```

(continues on next page)

(continued from previous page)

```
IRONIC_VM_SPECS_RAM=2500
IRONIC_VM_SPECS_DISK=10

# Size of the ephemeral partition in GB. Use 0 for no ephemeral partition.
IRONIC_VM_EPHEMERAL_DISK=0

# To build your own IPA ramdisk from source, set this to True
IRONIC_BUILD_DEPLOY_RAMDISK=False

VIRT_DRIVER=ironic

# By default, DevStack creates a 10.0.0.0/24 network for instances.
# If this overlaps with the hosts network, you may adjust with the
# following.
NETWORK_GATEWAY=10.1.0.1
FIXED_RANGE=10.1.0.0/24
FIXED_NETWORK_SIZE=256

# Log all output to files
LOGFILE=$HOME/devstack.log
LOGDIR=$HOME/logs
IRONIC_VM_LOG_DIR=$HOME/ironic-bm-logs
```

Use of Switch Simulators

The devstack plugin has logic to enable to exercising of simulated network switches. This support in devstack exists to aid developers in verification of networking-generic-switch or other ML2 plugin behavior, and is *not* intended to be used to actually exercise the end behavior of the virtalized switch.

To be very specific: This is not for Continuous Integration. Switch simulators are exceptionally slow and are designed largely for users to ping across, not attempt to perform deployments of machines across.

Generally, the starting point is to leverage the `IRONIC_NETWORK_SIMULATOR` setting and set it to a supported value. However, due to vendor limitations on accessing product firmware artifacts, you may need to perform some initial downloads of artifacts and even update filenames in the devstack plugin if you need to leverage a different version. Thinking of the simulator, you should mentally model it as an appliance.

Currently supported by the devstack plugin:

- cisco_nexus9k - Cisco Nexus 9000 series virtual switch simulator. This requires access to Ciscos Nexus switch simulator downloads, which you must download, and place in /opt/stack on the host upon which you are executing devstack.
- force10_9 - Dell Force10 OS9 based switches utilizing the OS9 simulator installation ISO image. Devstack is able to download this file.
- force10_10 - Dell Force10 OS10 (SmartFabric) switches utilizing a user downloaded switch simulator zip file. The simulator package must be downloaded and extracted into /opt/stack for this to be leveraged.

Warning

The support for these switch simulators in the devstack plugin generally revolves around the use of versioned files and downloads. You will likely need to consult the devstack plugin source if you wish to leverage this functionality.

The plugin generally work by attempting to plug the Virtual Machines used for CI testing through to networking for the virtual switch appliance, while also attaching the switch to the general networking of the host devstack is running on, and establishing the necessary configuration so networking-generic-switch may access the device and assert configuration.

Deploying Ironic on ARM64 with DevStack

The instructions here are specifically on how to configure for Deploying Ironic with DevStack on an ARM64 architecture.

Configurations

Create devstack/local.conf with the following content:

```
cat >local.conf <<END
[[local|localrc]]
# Enable and disable services
disable_all_services
# enable_service <service name>

# Credentials
ADMIN_PASSWORD=password
DATABASE_PASSWORD=password
RABBIT_PASSWORD=password
SERVICE_PASSWORD=password
SERVICE_TOKEN=password

# Set glance's default limit to be baremetal image friendly
GLANCE_LIMIT_IMAGE_SIZE_TOTAL=5000

# Enable Ironic plugin
enable_plugin ironic https://opendev.org/openstack/ironic

# Create a virtual machine to pose as Ironic's baremetal node.
IRONIC_VM_COUNT=1

# The parameters below represent the minimum possible values to create
# functional aarch64-based nodes.
IRONIC_VM_SPECS_RAM=4096
IRONIC_VM_SPECS_DISK=3

IRONIC_VM_SPECS_CPU=1
IRONIC_VM_VOLUME_COUNT=2
```

(continues on next page)

(continued from previous page)

```
# Enable hardware types and interfaces.
IRONIC_ENABLED_HARDWARE_TYPES=redfish
IRONIC_ENABLED_MANAGEMENT_INTERFACES=redfish
IRONIC_DEFAULT_RESCUE_INTERFACE=agent
IRONIC_ENABLED_BOOT_INTERFACES="ipxe,redfish-virtual-media,http-ipxe,pxe,http"
IRONIC_ENABLED_DEPLOY_INTERFACES="direct,ramdisk"
IRONIC_ENABLED_RESCUE_INTERFACES="agent,no-rescue"

# Specify deploy driver. This driver should be in the enabled list above.
IRONIC_DEPLOY_DRIVER=redfish

FORCE_CONFIG_DRIVE=False

# aarch64 + IRONIC_BUILD_DEPLOY_RAMDISK will be a bad mix
IRONIC_BUILD_DEPLOY_RAMDISK=False

IRONIC_AGENT_IMAGE_DOWNLOAD_SOURCE=http
IRONIC_AUTOMATED_CLEAN_ENABLED=False
IRONIC_BOOT_MODE=uefi
IRONIC_CALLBACK_TIMEOUT=800
IRONIC_GRUB2_SHIM_FILE=https://mirror.stream.centos.org/9-stream/BaseOS/
↪aarch64/os/EFI/BOOT/BOOTAA64.EFI
IRONIC_GRUB2_FILE=https://mirror.stream.centos.org/9-stream/BaseOS/aarch64/os/
↪EFI/BOOT/grubaa64.efi
IRONIC_HW_ARCH=aarch64
IRONIC_DIB_RAMDISK_OS=debian-arm64

INSTALL_TEMPEST=False
VIRT_DRIVER=ironic

# By default, DevStack creates a 10.0.0.0/24 network for instances.
# If this overlaps with the hosts network, you may adjust with the
# following.
#
# IP_VERSION=4
# FIXED_RANGE=10.1.0.0/20
# IPV4_ADDRS_SAFE_TO_USE=10.1.0.0/20
# NETWORK_GATEWAY=10.1.0.1

Q_AGENT=openvswitch
Q_ML2_PLUGIN_MECHANISM_DRIVERS=openvswitch
Q_ML2_TENANT_NETWORK_TYPE=vxlan

# Log all output to files
LOGFILE=/opt/stack/devstack.log
LOGDIR=/opt/stack/logs
IRONIC_VM_LOG_DIR=/opt/stack/ironic-bm-logs

END
```

This configuration sets up DevStack to work with ARM architecture hardware, using aarch64 images and appropriate hardware types, interfaces, and settings.

Refer to the Ironic on Devstack setup guide for more information on deploying Ironic with DevStack.

Deploying to Ironic node using Nova

This section assumes you already have a working, deployed Ironic with Nova configured as laid out above.

First, set the user to the admin demo (Note that all the user options can be seen in `/etc/openstack/clouds.yaml`):

```
export OS_CLOUD=devstack-admin-demo
```

We need to gather two more pieces of information before performing the deploy, we need to determine what image to use, and what network to use.

Determine the network:

```
net_id=$(openstack network list | awk '/private/ {print $2}')
```

We also need to choose an image to deploy. Devstack has both cirros partition and whole disk images by default. For this example, well use the whole disk image:

```
image=$(openstack image list | grep -- '-disk' | awk '{ print $2 }')
```

Source credentials and create a key:

```
# create keypair
ssh-keygen
openstack keypair create --public-key ~/.ssh/id_rsa.pub default
```

Now youre ready to build:

```
openstack server create --flavor baremetal --nic net-id=$net_id --image
↪$image --key-name default testing
```

You should now see a Nova instance building:

```
openstack server list --long
+----------+---------+--------+------------+-------------+----------+---------
↪---+----------+------------------+------+------------+
| ID       | Name    | Status | Task State | Power State | Networks | Image␣
↪Name | Image ID | Availability Zone | Host | Properties |
+----------+---------+--------+------------+-------------+----------+---------
↪---+----------+------------------+------+------------+
| a2c7f812 | testing | BUILD  | spawning   | NOSTATE     |          | cirros-
↪0.3 | 44d4092a | nova              |      |            |
| -e386-4a |         |        |            |             |          | .5-x86_
↪64- | -51ac-47 |                   |      |            |
| 22-b393- |         |        |            |             |          | disk   ␣
↪    | 51-9c50- |                   |      |            |
| fe1802ab |         |        |            |             |          |        ␣
↪    | fd6e2050 |                   |      |            |
```

(continues on next page)

(continued from previous page)

```
| d56e     |         |        |            |             |          |
↪    | faa1     |                   |      |            |
+----------+---------+--------+------------+-------------+----------+
↪---+----------+-------------------+------+------------+
```

Nova will be interfacing with Ironic conductor to spawn the node. On the Ironic side, you should see an Ironic node associated with this Nova instance. It should be powered on and in a wait call-back provisioning state:

```
# Note that a different user is required to see the Ironic nodes
OS_CLOUD=devstack-system-admin openstack baremetal node list
+--------------------------------------+--------+--------------------------
↪-------+-------------+-------------------+-------------+
| UUID                                 | Name   | Instance UUID
↪       | Power State | Provisioning State | Maintenance |
+--------------------------------------+--------+--------------------------
↪-------+-------------+-------------------+-------------+
| 9e592cbe-e492-4e4f-bf8f-4c9e0ad1868f | node-0 | None
↪       | power off   | None               | False       |
| ec0c6384-cc3a-4edf-b7db-abde1998be96 | node-1 | None
↪       | power off   | None               | False       |
| 4099e31c-576c-48f8-b460-75e1b14e497f | node-2 | a2c7f812-e386-4a22-b393-
↪fe1802abd56e | power on    | wait call-back     | False       |
+--------------------------------------+--------+--------------------------
↪-------+-------------+-------------------+-------------+
```

At this point, Ironic conductor has called to libvirt (via virtualbmc) to power on a virtual machine, which will PXE + TFTP boot from the conductor node and progress through the Ironic provisioning workflow. One libvirt domain should be active now:

```
sudo virsh list --all
 Id    Name                           State
----------------------------------------------------
 2     node-2                         running
 -     node-0                         shut off
 -     node-1                         shut off
```

This provisioning process may take some time depending on the performance of the host system, but Ironic should eventually show the node as having an active provisioning state:

```
OS_CLOUD=devstack-system-admin openstack baremetal node list
+--------------------------------------+--------+--------------------------
↪-------+-------------+-------------------+-------------+
| UUID                                 | Name   | Instance UUID
↪       | Power State | Provisioning State | Maintenance |
+--------------------------------------+--------+--------------------------
↪-------+-------------+-------------------+-------------+
| 9e592cbe-e492-4e4f-bf8f-4c9e0ad1868f | node-0 | None
↪       | power off   | None               | False       |
| ec0c6384-cc3a-4edf-b7db-abde1998be96 | node-1 | None
```

(continues on next page)

(continued from previous page)

```
↪          | power off    | None                | False       |
| 4099e31c-576c-48f8-b460-75e1b14e497f | node-2 | a2c7f812-e386-4a22-b393-
↪fe1802abd56e | power on    | active              | False       |
+--------------------------------------+--------+-----------------------------
↪---------+-------------+-------------------+-------------+
```

This should also be reflected in the Nova instance state, which at this point should be ACTIVE, Running and an associated private IP:

```
openstack server list --long
+----------+---------+--------+------------+-------------+---------------+----
↪--------+----------+-------------------+------+------------+
| ID       | Name    | Status | Task State | Power State | Networks      |␣
↪Image Name | Image ID | Availability Zone | Host | Properties |
+----------+---------+--------+------------+-------------+---------------+----
↪--------+----------+-------------------+------+------------+
| a2c7f812 | testing | ACTIVE | none       | Running     | private=10.1. |␣
↪cirros-0.3 | 44d4092a | nova              |      |            |
| -e386-4a |         |        |            |             | 0.4, fd7d:1f3 | .5-
↪x86_64- | -51ac-47 |                   |      |            |
| 22-b393- |         |        |            |             | c:4bf1:0:f816 |␣
↪disk       | 51-9c50- |                   |      |            |
| fe1802ab |         |        |            |             | :3eff:f39d:6d |  ␣
↪        | fd6e2050 |                   |      |            |
| d56e     |         |        |            |             | 94            |  ␣
↪        | faa1     |                   |      |            |
+----------+---------+--------+------------+-------------+---------------+----
↪--------+----------+-------------------+------+------------+
```

The server should now be accessible via SSH:

```
ssh cirros@10.1.0.4
$
```

Testing Ironic with Tempest

Add Ironic Tempest Plugin

Using the stack user, clone the ironic-tempest-plugin repository in the same directory you cloned DevStack:

```
git clone https://opendev.org/openstack/ironic-tempest-plugin.git
```

Then, add the following configuration to a working Ironic with Nova devstack configuration:

```
TEMPEST_PLUGINS=/opt/stack/ironic-tempest-plugin
```

Running tests

Note

Some tests may be skipped depending on the configuration of your environment, they may be reliant on a driver or a capability that you did not configure.

After deploying devstack including Ironic with the ironic-tempest-plugin enabled, one might want to run integration tests against the running cloud. The Tempest project is the project that offers an integration test suite for OpenStack.

First, navigate to Tempest directory:

```
cd /opt/stack/tempest
```

To run all tests from the Ironic plugin, execute the following command:

```
tox -e all -- ironic
```

To limit the amount of tests that you would like to run, you can use a regex. For instance, to limit the run to a single test file, the following command can be used:

```
tox -e all -- ironic_tempest_plugin.tests.scenario.test_baremetal_basic_ops
```

Debugging tests

It is sometimes useful to step through the test code, line by line, especially when the error output is vague. This can be done by running the tests in debug mode and using a debugger such as pdb.

For example, after editing the *test_baremetal_basic_ops* file and setting up the pdb traces you can invoke the `run_tempest.sh` script in the Tempest directory with the following parameters:

```
./run_tempest.sh -N -d ironic_tempest_plugin.tests.scenario.test_baremetal_
↪basic_ops
```

- The *-N* parameter tells the script to run the tests in the local environment (without a virtualenv) so it can find the Ironic tempest plugin.
- The *-d* parameter enables the debug mode, allowing it to be used with pdb.

For more information about the supported parameters see:

```
./run_tempest.sh --help
```

Note

Always be careful when running debuggers in time sensitive code, they may cause timeout errors that werent there before.

FAQ/Tips for development using devstack

VM logs are missing

When running QEMU as non-root user (e.g. `qemu` on Fedora or `libvirt-qemu` on Ubuntu), make sure `IRONIC_VM_LOG_DIR` points to a directory where QEMU will be able to write. You can verify this with, for example:

```
# on Fedora
sudo -u qemu touch $HOME/ironic-bm-logs/test.log
# on Ubuntu
sudo -u libvirt-qemu touch $HOME/ironic-bm-logs/test.log
```

Downloading an unmerged patch when stacking

To check out an in-progress patch for testing, you can add a Git ref to the `enable_plugin` line. For instance:

```
enable_plugin ironic https://opendev.org/openstack/ironic refs/changes/46/
↪295946/15
```

For a patch in review, you can find the ref to use by clicking the Download button in Gerrit. You can also specify a different git repo, or a branch or tag:

```
enable_plugin ironic https://github.com/openstack/ironic stable/kilo
```

For more details, see the devstack plugin interface documentation.

Custom Ironic-Python-Agent ramdisk raising kernel panic

The above devstack configurations configure VMs for 2GB ram, which is insufficient to run anything but the TinyIPA CI ramdisk. See *VM Sizing and Config* for instructions on raising this value.

Bifrost Development Environment

Bifrost is a project that deploys and operates Ironic using ansible. It is generally used standalone, without many other services running alongside. This makes it a good choice for a quick development environment for Ironic features that may not interact with other OpenStack services, even if you arent developing against bifrost directly

Bifrost maintains its own documentation on building a test environment with bifrost.

The testenv provided is ideal for quickly testing API changes in Ironic or features for client libraries. It is not the best choice for changes that interact with one or more OpenStack services or which require tempest testing.

Exercising Ironic Services Locally

It can sometimes be helpful to run Ironic services locally, without needing a full devstack environment or a server in a remote datacenter.

If you would like to exercise the Ironic services in isolation within your local environment, you can do this without starting any other OpenStack services. For example, this is useful for rapidly prototyping

and debugging interactions between client and API, exploring the Ironic API for the first time, and basic testing.

This guide assumes you have already installed all required Ironic prerequisites, as documented in the prerequisites section of *Unit Testing Environment*.

Using tox

Ironic provides a tox environment suitable for running a single-process Ironic against sqlite. This utilizes the config in `tools/ironic.conf.localdev` to setup a simple, all-in-one Ironic service useful for testing.

By default, this configuration uses sqlite with a backing file `ironic/ironic.sqlite`. Deleting this file and restarting ironic will reset you to a blank state.

Setup

1. If you havent already downloaded the source code, do that first:

```
cd ~
git clone https://opendev.org/openstack/ironic
cd ironic
```

2. Run the ironic all-in-one process:

```
tox -elocal-ironic-dev
```

3. In another window, utilize the client inside the tox venv:

```
. .tox/local-ironic-dev/bin/activate
export OS_AUTH_TYPE=none
export OS_ENDPOINT=http://127.0.0.1:6385
baremetal driver list
```

4. Press CTRL+C in the window running `tox -elocal-ironic-dev` when you are done.

Manually

You may wish to do this manually in order to give you more granular control over library versions and configurations, to enable usage of a database server backend, or to spin up a non-all-in-one Ironic.

Step 1: Create a Python virtualenv

1. If you havent already downloaded the source code, do that first:

```
cd ~
git clone https://opendev.org/openstack/ironic
cd ironic
```

2. Create the Python virtualenv:

```
tox -elocal-ironic-dev --notest --develop -r
```

3. Activate the virtual environment:

```
. .tox/local-ironic-dev/bin/activate
```

Note

This installs `python-openstackclient` and `python-ironicclient` from pypi. You can instead install them from source by cloning the git repository, activating the venv, and running *pip install -e .* while in the root of the git repo.

4. Export some ENV vars so the client will connect to the local services that youll start in the next section:

```
export OS_AUTH_TYPE=none
export OS_ENDPOINT=http://localhost:6385/
```

Step 2: Install System Dependencies Locally

This step will install MySQL on your local system. This may not be desirable in some situations (eg, youre developing from a laptop and do not want to run a MySQL server on it all the time).

1. Install mysql-server:

   Ubuntu/Debian:

```
sudo apt-get install mysql-server
```

   RHEL/CentOS/Fedora:

```
sudo dnf install mariadb mariadb-server
sudo systemctl start mariadb.service
```

   If using MySQL, you need to create the initial database:

```
mysql -u root -pMYSQL_ROOT_PWD -e "create schema ironic"
```

2. Use the localdev config as a template, and modify it:

```
# copy sample config and modify it as necessary
cp tools/ironic.conf.localdev etc/ironic/ironic.conf.local

# Add mysql database connection information to config
echo -e "\n[database]" >> etc/ironic/ironic.conf.local
echo -e "connection = mysql+pymysql://root:MYSQL_ROOT_PWD@localhost/ironic
↪" >> etc/ironic/ironic.conf.local

# disable single-process mode and enable json-rpc
sed -i "s/rpc_transport = none/rpc_transport = json-rpc/" etc/ironic/
↪ironic.conf.local
```

Step 3: Start the Services

From within the python virtualenv, run the following command to prepare the database before you start the ironic services:

```
# initialize the database for ironic
ironic-dbsync --config-file etc/ironic/ironic.conf.local create_schema
```

Next, open two new terminals for this section, and run each of the examples here in a separate terminal. In this way, the services will *not* be run as daemons; you can observe their output and stop them with Ctrl-C at any time.

1. Start the API service in debug mode and watch its output:

   ```
   cd ~/ironic
   . .tox/local-ironic-dev/bin/activate
   ironic-api -d --config-file etc/ironic/ironic.conf.local
   ```

2. Start the Conductor service in debug mode and watch its output:

   ```
   cd ~/ironic
   . .tox/local-ironic-dev/bin/activate
   ironic-conductor -d --config-file etc/ironic/ironic.conf.local
   ```

Step 4: Interact with the running services

You should now be able to interact with ironic via the python client, which is present in the python virtualenv, and observe both services debug outputs in the other two windows. This is a good way to test new features or play with the functionality without necessarily starting DevStack.

To get started, export the following variables to point the client at the local instance of ironic:

```
export OS_AUTH_TYPE=none
export OS_ENDPOINT=http://127.0.0.1:6385
```

Then list the available commands and resources:

```
# get a list of available commands
baremetal help

# get the list of drivers currently supported by the available conductor(s)
baremetal driver list

# get a list of nodes (should be empty at this point)
baremetal node list
```

Here is an example walkthrough of creating a node:

```
# enroll the node with the fake hardware type and IPMI-based power and
# management interfaces. Note that driver info may be added at node
# creation time with "--driver-info"
NODE=$(baremetal node create --driver fake-hardware -f value -c uuid)
```

(continues on next page)

(continued from previous page)

```
# node info may also be added or updated later on
baremetal node set $NODE --driver-info fake_driver_info=fake
baremetal node set $NODE --extra extradata=isfun

# view the information for the node
baremetal node show $NODE

# request that the node's driver validate the supplied information
baremetal node validate $NODE

# you have now enrolled a node sufficiently to be able to control
# its power state from ironic!
baremetal node power on $NODE
```

If you make some code changes and want to test their effects, simply stop the services with Ctrl-C and restart them.

Step 5: Fixing your test environment

If you are testing changes that add or remove python entrypoints, or making significant changes to ironics python modules, or simply keep the virtualenv around for a long time, your development environment may reach an inconsistent state. It may help to delete cached .pyc files, update dependencies, reinstall ironic, or even recreate the virtualenv. The following commands may help with that, but are not an exhaustive troubleshooting guide:

```
# clear cached pyc files
cd ~/ironic/ironic
find ./ -name '*.pyc' | xargs rm

# reinstall ironic modules
cd ~/ironic
. .tox/local-ironic-dev/bin/activate
pip uninstall ironic
pip install -e .

# install and upgrade ironic and all python dependencies
cd ~/ironic
. .tox/local-ironic-dev/bin/activate
pip install -U -e .
```

Changing the Ironic Localdev Microversion

Localdev supports testing against a specific microversion, which is useful for testing backwards compatibility or evaluating external tools with different versions of the API.

Steps to Change the Microversion

To change the microversion, modify the `ironic.conf.localdev` file. Instead of directly modifying the microversion, you should set the release version corresponding to the microversion you intend to test. The mappings between release versions and microversions (for the REST API) are maintained in `ironic/common/release_mappings.py`.

For example, to set the microversion to **16.2**, you would adjust the `pin_release_version` parameter:

```
[DEFAULT]
# Set the release version corresponding to microversion 16.2
pin_release_version = 16.2
```

You can review the full configuration and release version mappings in the sample configuration documentation.

Resetting Localdev Setup

After changing the microversion, you need to reset the local development environment.

1. Delete the existing database file:

```
rm -f ironic/ironic.sqlite
```

2. Reinstall Ironic in the virtual environment:

```
. .tox/local-ironic-dev/bin/activate
pip uninstall ironic
pip install -e .
```

This change ensures that the APIs from the selected Ironic version will be applied, based on the release-to-microversion mapping in the Ironic codebase.

Ironics State Machine

The content has been migrated, please see *Bare Metal State Machine*.

5.1.2 Bugs

Information about how ironic projects handle bugs can be found below.

Bug Reporting and Triaging Guide

Launchpad

All Ironic projects use Launchpad for tracking bugs.

> **Note**
>
> Ironic projects formerly used Storyboard for tracking bugs. Since April 2023, we have switched to Launchpad for bugtracking.

Reporting Guide

We are constantly receiving a lot of requests, so its important to file a meaningful one for it to be acted upon. A good request:

- specifies **why** a change is needed. In case of a bug - what you expected to happen.
- explains how to reproduce the described condition.

> **Note**
>
> Please try to provide a reproducer based on unit tests, *devstack* or bifrost. While we try our best to support users using other installers and distributions, it may be non-trivial without deep knowledge of them. If youre using a commercial distribution or a product, please try contacting support first.

- should be understandable without additional context. For example, if you see an exception, we will need the full traceback. Other commonly required things are:
  - the contents of the node in question (use `baremetal node show <uuid>`)
  - debug logging related to the event, ideally with logs from the ramdisk
  - versions of ironic, ironic-python-agent, and any other coupled components.
- should not be too verbose either. Unfortunately, we cannot process a few days worth of system logs to find the problems, we expect your collaboration.
- is not a question or a support request. Please see *So You Want to Contribute* for the ways to contact us.
- provides a way to contact the reporter. Please follow the comments and expect follow-up emails, but ideally also be on IRC for questions.

An enhancement request additionally:

- benefits the overall project, not just one consumer. If you have a case that is specific to your requirements, think about ways to make Ironic extensible to be able to cover it.
- does not unnecessary increase the project scope. Consider if your idea can be implemented without changing Ironic or its projects, maybe it actually should?
- should specify if you are willing to perform the work to enhance Ironic yourself, of if youre submitting a request to for the project team to executed your requested enhancement.

Triaging Guide

The bug triaging process involves checking new stories to make sure they are actionable by the team. This guide is mostly targeting the project team, but we would appreciate if reporters could partly self-triage their own requests.

- Determine if the request is valid and complete. Use the checklist in the *Reporting Guide* for that.
- Is the request a bug report or an enhancement request (an RFE)? The difference is often subtle, the key question to answer is if the described behavior is expected.

  Add an `rfe` tag to all enhancement requests and propose it for the RFE Review section of the weekly meeting.

- Does the RFE obviously require a spec? Usually this is decided when an RFE is reviewed during the meeting, but some requests are undoubtedly complex, involve changing a lot of critical parts and thus demand a spec.

  Add a `needs-spec` tag to enhancement requests that obviously need a spec. Otherwise leave it until the meeting.

- Apply additional tags:
  - All hardware type specific stories should receive a corresponding tag (e.g. `ipmi`, `idrac`, etc).
  - API-related stories should have an `api` tag.
  - CI issues should have a `gate` tag.

The next actions **must only** be done by a core team member (or an experienced full-time contributor appoined by the PTL):

- Can the RFE be automatically approved? It happens if the RFE requests an implementation of a driver feature that is already implemented for other drivers and does not pose additional complexity.

  If the RFE can be automatically approved, apply the `rfe-approved` tag. If unsure, never apply the tag! Talk to the PTL instead.

- Does the RFE have a corresponding spec approved? If yes, apply the `rfe-approved` tag.
- In the end, apply the `ironic-triaged` tag to make the story as triaged.

Expiring Bugs

While we hope to fix all issues that our consumers hit, it is unfortunately not realistic. Stories **may** be closed by marking all their tasks `INVALID` in the following cases:

- No solution has been proposed in 1 calendar year.
- Additional information has been requested from the reporter, and no update has been provided in 1 calendar month.
- The request no longer aligns with the direction of the project.

> **Note**
>
> As usual, common sense should be applied when closing stories.

Bug Deputy Guide

Ironic has a rotating bug deputy role with assigned responsibilities around ensuring recurring project maintenance occurs, with a specific focus on bug triage.

It is the intent that the time commitment of an upstream bug deputy be no more than two to four hours a week on average.

Schedule

Typically, a bug deputy will serve for a one week period, with the Ironic meeting marking the beginning and end of the term.

A bug deputy schedule will be built at the beginning of the OpenStack release cycle and populated by project volunteers. Contributors can select weeks to volunteer for stints as bug deputy.

If there are insufficient volunteers, to cover a majority of weeks, the bug deputy program will be cancelled.

Responsibilities

Bug Triage

Triage bugs opened in any Ironic project.

All Ironic project bugtrackers, filtered and sorted for triage:

- ironic
- ironic-inspector
- ironic-python-agent
- ironic-lib
- bifrost
- ironic-prometheus-exporter
- ironic-python-agent-builder
- ironic-ui
- networking-baremetal
- networking-generic-switch
- python-ironic-inspector-client
- python-ironicclient
- sushy
- sushy-tools
- virtualbmc

Bug Bash

A bug bash is an informal, synchronous meeting to triage bugs. A bug deputy runs one per week at a time and in a format convenient for them.

The selected time and venue for this should be announced on the mailing list and at the Ironic meeting when the bug deputy position is handed over.

Note

Bug bashes may be discontinued when the backlog of old, untriaged bugs have been worked through.

Review Periodic Stable CI Jobs

The bug deputy is responsible for reviewing the periodic stable CI jobs once during their week and notifying the community if one fails for a new, non-random reason. The bug deputy should also be prepared to help debug the issue, but is ultimately only responsible for documenting it.

- Periodic Zuul build failures for Ironic/IPA/Ironic-Prom-Exp/Bifrost

- Periodic Zuul build failures for Ironic UI/NBM/NGS
- Periodic Zuul build failures for inspector-client/sushy/sushy-tools/vbmc/vpdu/

As of this writing, no other projects under Ironic governance run periodic jobs.

Weekly Report

Once a week, at the end of the bug deputys term, they should deliver a report to the Ironic meeting and the mailing list. This report should include any concerning bugs or CI breakages, as well as any other issues that the bug deputy feels the community needs to know about.

For contributors who do not wish to attend the weekly meeting, a small written report in the meeting agenda is sufficient.

5.1.3 Community and Policies

Bare Metal Community

This document provides information on how to reach out to the community for questions, bug reports or new code contributions.

Useful Links

Bug/Task tracker
https://bugs.launchpad.net/ironic/+bugs

Code Hosting
https://opendev.org/openstack/ironic

Code Review
https://review.opendev.org/#/q/status:open+project:openstack/ironic,n,z

Weekly Meeting Agenda
https://wiki.openstack.org/wiki/Meetings/Ironic#Agenda_for_next_meeting

Ironic Contributors Whiteboard
https://etherpad.opendev.org/p/IronicWhiteBoard

Asking Questions

There are two many venues where all discussions happen: IRC and mailing lists.

Internet Relay Chat IRC

Daily contributor discussions take place on IRC in the `#openstack-ironic` channel on the OFTC IRC network. Please feel free to connect to `ircs://irc.oftc.net:6697` and join our channel!

Note that while we have community members from everywhere in the world, were the most active from roughly 6am to 12am. If you dont get an answer to your question, try the *Mailing list*.

Additional information on getting connected can be found in the OpenStack community contribution guide.

Mailing list

We use the *openstack-discuss* mailing list for asynchronous communications and longer discussions. Navigate to http://lists.openstack.org/cgi-bin/mailman/listinfo/openstack-discuss to subscribe or view the archives.

When sending a message please prefix the *Subject* line with `[ironic]` so that we dont miss it.

Reporting Bugs

See *Bug Reporting and Triaging Guide* for more details on how we track bugs.

LaunchPad

Most of the tools used for OpenStack require a Launchpad ID for authentication. Ironic tracks bugs via Launchpad.

Storyboard

The Ironic project moved from StoryBoard for work and task tracking in April, 2023. Ironic bugs in storyboard may remain valid, but there should not be additional issues filed there.

Contributing Code

See also

- *So You Want to Contribute* - basic information on new code contributions
- *Developers Guide*

So You Want to Contribute

This document provides some necessary points for developers to consider when writing and reviewing Ironic code. The checklist will help developers get things right. Please make sure to check the *community page* first.

Contributing Code

If youre completely new to OpenStack and want to contribute to the ironic project, please start by familiarizing yourself with the Infra Teams Developer Guide. This will help you get your accounts set up in Launchpad and Gerrit, familiarize you with the workflow for the OpenStack continuous integration and testing systems, and help you with your first commit.

Everything Ironic

Ironic is a community of projects centered around the primary project repository ironic, which help facilitate the deployment and management of bare metal resources.

This means there are a number of different repositories that fall into the responsibility of the project team and the community. Some of the repositories may not seem strictly hardware related, but they may be tools or things to just make an aspect easier.

Related Projects

There are several projects that are tightly integrated with ironic and which are developed by the same community.

See also

- Bifrost Documentation
- Ironic Lib Documentation
- Ironic Python Agent (IPA) Documentation
- Ironic Client Documentation

Adding New Features

Ironic tracks new features using RFEs (Requests for Feature Enhancements) instead of blueprints. These are stories with rfe tag, and they should be submitted before a spec or code is proposed.

When a member of the ironic-core team decides that the proposal is worth implementing, a spec (if needed) and code should be submitted, referencing the RFE task or bug number. Contributors are welcome to submit a spec and/or code before the RFE is approved, however those patches will not land until the RFE is approved.

Feature Submission Process

1. Submit a bug report on Launchpad. If you cant describe your feature in a sentence or two, it may mean that you are either trying to capture more than one RFE at once, or that you are having a hard time defining what you are trying to solve at all. This may also be a sign that your feature may require a specification document.
2. Describe the proposed change in the Description field. The description should provide enough details for a knowledgeable developer to understand what is the existing problem in the current platform that needs to be addressed, or what is the enhancement that would make the platform more capable, both from a functional and a non-functional standpoint.
3. Submit the bug, add an rfe tag to it and assign yourself or whoever is going to work on this feature.
4. As soon as a member of the team acknowledges the bug, we will move it to the Review state. As time goes on, Discussion about the RFE, and whether to approve it will occur. If the RFE has not been triaged and youd like it to receive immediate attention, add it to the Open Discussion section of our weekly meeting agenda, and, timezone permitting, attend the meeting to advocate for your RFE.
5. Contributors will evaluate the RFE and may advise the submitter to file a spec in the ironic-specs repository to elaborate on the feature request. Typically this is when an RFE requires extra scrutiny, more design discussion, etc. For the spec submission process, please see the *Ironic Specs Process*. A specific task should be created to track the creation of a specification.
6. If a spec is not required, once the discussion has happened and there is positive consensus among the ironic-core team on the RFE, the RFE is approved, and its tag will move from rfe to rfe-approved. This means that the feature is approved and the related code may be merged.
7. If a spec is required, the spec must be submitted (with a new task as part of the story referenced as Task in the commit message), reviewed, and merged before the RFE will be approved (and the tag

changed to rfe-approved).

8. If the RFE is rejected, the ironic-core team will move the story to Invalid status.

Change Tracking

Please ensure work related to a bug or RFE is tagged with the bug. This generally is a Closes-bug, Partial-bug or Related-bug tag as described in the Git Commit messages guide.

Note

RFEs may only be approved by members of the ironic-core team.

Note

While not strictly required for minor changes and fixes, it is highly preferred by the Ironic community that any change which needs to be backported, have a recorded bug.

Managing Change Sets

If you would like some help, or if you (or some members of your team) are unable to continue working on the feature, updating and maintaining the changes, please let the rest of the ironic community know. You could leave a comment in one or more of the changes/patches, bring it up in IRC, the weekly meeting, or on the OpenStack development email list. Communicating this will make other contributors aware of the situation and allow for others to step forward and volunteer to continue with the work.

In the event that a contributor leaves the community, do not expect the contributors changes to be continued unless someone volunteers to do so.

Getting Your Patch Merged

Within the Ironic project, we generally require two core reviewers to sign-off (+2) change sets. We also will generally recognize non-core (+1) reviewers, and sometimes even reverse our decision to merge code based upon their reviews.

We recognize that some repositories have less visibility, as such it is okay to ask for a review in our IRC channel. Please be prepared to stay in IRC for a little while in case we have questions.

Sometimes we may also approve patches with a single core reviewer. This is generally discouraged, but sometimes necessary. When we do so, we try to explain why we do so. As a patch submitter, it equally helps us to understand why the change is important. Generally, more detail and context helps us understand the change faster.

Timeline Expectations

As with any large project, it does take time for features and changes to be merged in any of the project repositories. This is largely due to limited review bandwidth coupled with varying reviewer priorities and focuses.

When establishing an understanding of complexity, the following things should be kept in mind.

- Generally, small and minor changes can gain consensus and merge fairly quickly. These sorts of changes would be: bug fixes, minor documentation updates, follow-up changes.

- Medium changes generally consist of driver feature parity changes, where one driver is working to match functionality of another driver.
  - These changes generally only require an RFE for the purposes of tracking and correlating the change.
  - Documentation updates are expected to be submitted with or immediately following the initial change set.
- Larger or controversial changes generally take much longer to merge. This is often due to the necessity of reviewers to gain additional context and for change sets to be iterated upon to reach a state where there is consensus. These sorts of changes include: database, object, internal interface additions, RPC, rest API changes.
  - These changes will very often require specifications to reach consensus, unless there are pre-existing patterns or code already present.
  - These changes may require many reviews and iterations, and can also expect to be impacted by merge conflicts as other code or features are merged.
  - These changes must typically be split into a series of changes. Reviewers typically shy away from larger single change sets due to increased difficulty in reviewing.
  - Do not expect any API or user-visible data model changes to merge after the API client freeze. Some substrate changes may merge if not user visible.
- You should expect complex features, such as cross-project features or integration, to take longer than a single development cycle to land.
  - Building consensus is vital.
  - Often these changes are controversial or have multiple considerations that need to be worked through in the specification process, which may cause the design to change. As such, it may take months to reach consensus over design.
  - These features are best broken into larger chunks and tackled in an incremental fashion.

Live Upgrade Related Concerns

See *Rolling Upgrades*.

Driver Internal Info

The `driver_internal_info` node field was introduced in the Kilo release. It allows driver developers to store internal information that can not be modified by end users. Here is the list of existing common and agent driver attributes:

- Common attributes:
  - `is_whole_disk_image`: A Boolean value to indicate whether the user image contains ramdisk/kernel.
  - `clean_steps`: An ordered list of clean steps that will be performed on the node.
  - `deploy_steps`: An ordered list of deploy steps that will be performed on the node. Support for deploy steps was added in the `11.1.0` release.
  - `instance`: A list of dictionaries containing the disk layout values.

  - `root_uuid_or_disk_id`: A String value of the bare metal nodes root partition uuid or disk id.
  - `persistent_boot_device`: A String value of device from `ironic.common.boot_devices`.
  - `is_next_boot_persistent`: A Boolean value to indicate whether the next boot device is `persistent_boot_device`.

- Agent driver attributes:
  - `agent_url`: A String value of IPA API URL so that Ironic can talk to IPA ramdisk.
  - `hardware_manager_version`: A String value of the version of the hardware manager in IPA ramdisk.
  - `target_raid_config`: A Dictionary containing the target RAID configuration. This is a copy of the same name attribute in Node object. But this one is never actually saved into DB and is only read by IPA ramdisk.

Note

These are only some fields in use. Other vendor drivers might expose more `driver_internal_info` properties, please check their development documentation and/or module docstring for details. It is important for developers to make sure these properties follow the precedent of prefixing their variable names with a specific interface name (e.g., ilo_bar, drac_xyz), so as to minimize or avoid any conflicts between interfaces.

Ironic Specs Process

Specifications must follow the template which can be found at specs/template.rst, which is quite self-documenting. Specifications are proposed by adding them to the `specs/approved` directory, adding a soft link to it from the `specs/not-implemented` directory, and posting it for review to Gerrit. For more information, please see the README.

The same Gerrit process as with source code, using the repository ironic-specs, is used to add new specifications.

All approved specifications are available at: https://specs.openstack.org/openstack/ironic-specs. If a specification has been approved but not completed within one or more releases since the approval, it may be re-reviewed to make sure it still makes sense as written.

Ironic specifications are part of the *RFE (Requests for Feature Enhancements) process*. You are welcome to submit patches associated with an RFE, but they will have a -2 (do not merge) until the specification has been approved. This is to ensure that the patches dont get accidentally merged beforehand. You will still be able to get reviewer feedback and push new patch sets, even with a -2. The list of core reviewers for the specifications is small but mighty. (This is not necessarily the same list of core reviewers for code patches.)

Changes to existing specs

For approved but not-completed specs:

- cosmetic cleanup, fixing errors, and changing the definition of a feature can be done to the spec.

For approved and completed specs:

- changing a previously approved and completed spec should only be done for cosmetic cleanup or fixing errors.
- changing the definition of the feature should be done in a new spec.

Please see the Ironic specs process wiki page for further reference.

Project Team Leader Duties

The `Project Team Leader` or `PTL` is elected each development cycle by the contributors to the ironic community.

Think of this person as your primary contact if you need to try and rally the project, or have a major issue that requires attention.

They serve a role that is mainly oriented towards trying to drive the technical discussion forward and managing the idiosyncrasies of the project. With this responsibility, they are considered a public face of the project and are generally obliged to try and provide project updates and outreach communication.

All common PTL duties are enumerated here in the PTL guide.

Tasks like release management or preparation for a release are generally delegated with-in the team. Even outreach can be delegated, and specifically there is no rule stating that any member of the community cant propose a release, clean-up release notes or documentation, or even get on the occasional stage.

Developer FAQ (frequently asked questions)

Here are some answers to frequently-asked questions from IRC and elsewhere.

- *How do I*
  - *create a migration script template?*
  - *know if a release note is needed for my change?*
  - *create a new release note?*
  - *update a release note?*
  - *get a decision on something?*
  - *get an Ironic themed pony shaped feature?*

How do I

create a migration script template?

Using the `ironic-dbsync revision` command, e.g:

```
$ cd ironic
$ tox -evenv -- ironic-dbsync revision -m \"create foo table\"
```

It will create an empty alembic migration. For more information see the alembic documentation.

know if a release note is needed for my change?

Reno documentation contains a description of what can be added to each section of a release note. If, after reading this, youre still unsure about whether to add a release note for your change or not, keep in mind that it is intended to contain information for deployers, so changes to unit tests or documentation are unlikely to require one.

create a new release note?

By running `reno` command via tox, e.g:

```
$ tox -e venv -- reno new version-foo
  venv create: /home/foo/ironic/.tox/venv
  venv installdeps: -r/home/foo/ironic/test-requirements.txt
  venv develop-inst: /home/foo/ironic
  venv runtests: PYTHONHASHSEED='0'
  venv runtests: commands[0] | reno new version-foo
  Created new notes file in releasenotes/notes/version-foo-ecb3875dc1cbf6d9.
↪yaml
    venv: commands succeeded
    congratulations :)

$ git status
  On branch test
  Untracked files:
    (use "git add <file>..." to include in what will be committed)

    releasenotes/notes/version-foo-ecb3875dc1cbf6d9.yaml
```

Then edit the result file. Note that:

- we prefer to use present tense in release notes. For example, a release note should say Adds support for feature foo, not Added support for feature foo. (We use adds instead of add because grammatically, it is ironic adds support, not ironic add support.)
- any variant of English spelling (American, British, Canadian, Australian) is acceptable. The release note itself should be consistent and not have different spelling variants of the same word.

For more information see the reno documentation.

update a release note?

If this is a release note that pertains to something that was fixed on master or an intermediary release (during a development cycle, that hasnt been branched yet), you can go ahead and update it by submitting a patch.

If it is the release note of an ironic release that has branched, it can be updated but we will only allow it in extenuating circumstances. (It can be updated by *only* updating the file in that branch. DO NOT update the file in master and cherry-pick it. If you do, see how the mess was cleaned up.)

get a decision on something?

You have an issue and would like a decision to be made. First, make sure that the issue hasnt already been addressed, by looking at documentation, stories, specifications, or asking. Information and links can be found on the Ironic wiki page.

There are several ways to solicit comments and opinions:

- bringing it up at the weekly Ironic meeting
- bringing it up on IRC
- bringing it up on the mailing list (add [Ironic] to the Subject of the email)

If there are enough core folks at the weekly meeting, after discussing an issue, voting could happen and a decision could be made. The problem with IRC or the weekly meeting is that feedback will only come from the people that are actually present.

To inform (and solicit feedback from) more people about an issue, the preferred process is:

1. bring it up on the mailing list
2. after some period of time has elapsed (and depending on the thread activity), someone should propose a solution via gerrit. (E.g. the person that started the thread if no one else steps up.) The proposal should be made in the git repository that is associated with the issue. (For instance, this decision process was proposed as a documentation patch to the ironic repository.)
3. In the email thread, dont forget to provide a link to the proposed patch!
4. The discussion then moves to the proposed patch. If this is a big decision, we could declare that some percentage of the cores should vote on it before landing it.

(This process was suggested in an email thread about process for making decisions.)

get an Ironic themed pony shaped feature?

Also known as: I want a pony! How do I get a pony?

Ironic operates via a consensus driven model of collaboration. Often, others want the similarly shaped features, but may not agree on the details. Or different words may be used for the exact same meaning. The best way to build awareness, consensus, and a common set of words to describe the feature which you desire is to join the weekly IRC meeting and bringing up the desire during the `Open Discussion` portion of the meeting. Often Ironic contributors will ask for an RFE or a specification documentation to be started which we can collaborate on.

Persistence and collaboration are key. Often contributors are quite busy with numerous other tasks. In other words, if we dont respond quickly or on your first attempt, it is not you. Just try again.

Contributor Vision

Background

During the Rocky Project Teams Gathering (February/March 2018), The contributors in the room at that time took a few minutes to write out each contributors vision of where they see ironic in five years time.

After everyone had a chance to spend a few minutes writing, we went around the room and gave every contributor the chance to read their vision and allow other contributors to ask questions to better understand what each individual contributor wrote. While we were doing that, we also took time to capture the common themes.

This entire exercise did result in some laughs and a common set of words, and truly helped to ensure that the entire team proceeded to use the same words to describe various aspects as the sessions progressed during the week. We also agreed that we should write a shared vision, to have something to reference and remind us of where we want to go as a community.

Rocky Vision: For 2022-2023

Common Themes

Below is an entirely unscientific summary of common themes that arose during the discussion among fourteen contributors.

- Contributors picked a time between 2020, and 2023.
- 4 Contributors foresee ironic being the leading Open Source baremetal deployment technology
- 2 Contributors foresee ironic reaching feature parity with Nova.
- 2 Contributors foresee users moving all workloads to the cloud
- 1 Contributor foresees Kubernetes and Container integration being the major focus of Bare Metal as a Service further down the road.
- 2 Contributors foresee greater composible hardware being more common.
- 1 Contributor foresees ironic growing into or supporting CMDBs.
- 2 Contributors foresee that features are more micro-service oriented.
- 2 Contributors foresee that ironic supported all of the possible baremetal management needs
- 1 Contributor foresees standalone use being more common.
- 2 Contributors foresee the ironics developer community growing
- 2 Contributors foresee that auto-discovery will be more common.
- 2 Contributors foresee ironic being used for devices beyond servers, such as lightbulbs, IOT, etc.

Vision Statement

The year is 2022. Were meeting to plan the Z release of Ironic. We stopped to reflect upon the last few years of Ironics growth, how we had come such a long way to become the defacto open source baremetal deployment technology. How we had grown our use cases, and support for consumers such as containers, and users who wished to managed specialized fleets of composed machines.

New contributors and their different use cases have brought us closer to parity with virtual machines. Everyday were gaining word of more operators adopting the ironic communitys CMDB integration to leverage hardware discovery. Weve heard of operators deploying racks upon racks of new hardware by just connecting the power and network cables, and from there the operators have discovered time to write the worlds greatest operator novel with the time saved in commissioning new racks of hardware.

Time has brought us closer and taught us to be more collaborative across the community, and we look forward to our next release together.

Comparison to the 2018 OpenStack Technical Vision

In late-2018, the OpenStack Technical composed a technical vision of what OpenStack clouds should look like. While every component differs, and cloudy interactions change dramatically the closer to physical hardware one gets, there are a few areas where Ironic could use some improvement.

This list is largely for the purposes of help wanted. It is also important to note that Ironic as a project has a vision document for itself.

The Pillars of Cloud - Self Service

- Ironics mechanisms and tooling are low level infrastructure mechanisms and as such there has never been a huge emphasis or need on making Ironic be capable of offering direct multi-tenant interaction. Most users interact with the bare metal managed by Ironic via Nova, which abstracts away many of these issues. Eventually, we should offer direct multi-tenancy which is not oriented towards admin-only.

Design Goals - Built-in Reliability and Durability

- Ironic presently considers in-flight operations as failed upon the restart of a controller that was previously performing a task, because we do not know the current status of the task upon re-start. In some cases, this makes sense, but potentially requires administrative intervention in the worst of cases. In a perfect universe, Ironic conductors would validate their perception, in case tasks actually finished.

Design Goals - Graphical User Interface

- While a graphical interface was developed for Horizon in the form of ironic-ui, currently ironic-ui receives only minimal housekeeping. As Ironic has evolved, ironic-ui is stuck on version `1.34` and knows nothing of our evolution since. Ironic ultimately needs a contributor with sufficient time to pick up `ironic-ui` or to completely replace it as a functional and customizable user interface.

5.1.4 Architecture and Implementation Details

The following pages describe the architecture of the Bare Metal service and may be helpful to anyone working on or with the service, but are written primarily for developers.

System Architecture

High Level description

An Ironic deployment will be composed of the following components:

- An admin-only RESTful API service, by which privileged users, such as cloud operators and other services within the cloud control plane, may interact with the managed bare metal servers.
- A Conductor service, which does the bulk of the work. Functionality is exposed via the API service. The Conductor and API services communicate via RPC.
- A Database and DB API for storing the state of the Conductor and Drivers.
- A Deployment Ramdisk or Deployment Agent, which provide control over the hardware which is not available remotely to the Conductor. A ramdisk should be built which contains one of these agents, eg. with diskimage-builder. This ramdisk can be booted on-demand.

> **Note**
>
> The agent is never run inside a tenant instance.

Drivers

The internal driver API provides a consistent interface between the Conductor service and the driver implementations. A driver is defined by a *hardware type* deriving from the AbstractHardwareType class, defining supported *hardware interfaces*. See *Enabling drivers and hardware types* for a more detailed explanation. See *Pluggable Drivers* for an explanation on how to write new hardware types and interfaces.

Driver-Specific Periodic Tasks

Drivers may run their own periodic tasks, i.e. actions run repeatedly after a certain amount of time. Such a task is created by using the periodic decorator on an interface method. For example

```
from futurist import periodics

class FakePower(base.PowerInterface):
    @periodics.periodic(spacing=42)
    def task(self, manager, context):
        pass  # do something
```

Here the `spacing` argument is a period in seconds for a given periodic task. For example spacing=5 means every 5 seconds.

Starting with the Yoga cycle, there is also a new decorator *`ironic.conductor.periodics.node_periodic()`* to create periodic tasks that handle nodes. See *deploy steps documentation* for an example.

Driver-Specific Steps

Drivers may have specific steps that may need to be executed or offered to a user to execute in order to perform specific configuration tasks.

These steps should ideally be located on the management interface to enable consistent user experience of the hardware type. What should be avoided is duplication of existing interfaces such as the deploy interface to enable vendor specific cleaning or deployment steps.

Message Routing

Each Conductor registers itself in the database upon start-up, and periodically updates the timestamp of its record. Contained within this registration is a list of the drivers which this Conductor instance supports. This allows all services to maintain a consistent view of which Conductors and which drivers are available at all times.

Based on their respective driver, all nodes are mapped across the set of available Conductors using a consistent hashing algorithm. Node-specific tasks are dispatched from the API tier to the appropriate conductor using conductor-specific RPC channels. As Conductor instances join or leave the cluster, nodes may be remapped to different Conductors, thus triggering various driver actions such as take-over or clean-up.

Developing New Notifications

Ironic notifications are events intended for consumption by external services. Notifications are sent to these services over a message bus by oslo.messagings Notifier class. For more information about configuring notifications and available notifications, see *Notifications*.

Ironic also has a set of base classes that assist in clearly defining the notification itself, the payload, and the other fields not auto-generated by oslo (level, event_type and publisher_id). Below describes how to use these base classes to add a new notification to ironic.

Adding a new notification to ironic

To add a new notification to ironic, a new versioned notification class should be created by subclassing the NotificationBase class to define the notification itself and the NotificationPayloadBase class to define which fields the new notification will contain inside its payload. You may also define a schema to allow the payload to be automatically populated by the fields of an ironic object. Heres an example:

```
# The ironic object whose fields you want to use in your schema
@base.IronicObjectRegistry.register
class ExampleObject(base.IronicObject):
    # Version 1.0: Initial version
    VERSION = '1.0'
    fields = {
        'id': fields.IntegerField(),
        'uuid': fields.UUIDField(),
        'a_useful_field': fields.StringField(),
        'not_useful_field': fields.StringField()
    }

# A class for your new notification
@base.IronicObjectRegistry.register
class ExampleNotification(notification.NotificationBase):
    # Version 1.0: Initial version
    VERSION = '1.0'
    fields = {
        'payload': fields.ObjectField('ExampleNotifPayload')
    }

# A class for your notification's payload
@base.IronicObjectRegistry.register
class ExampleNotifPayload(notification.NotificationPayloadBase):
    # Schemas are optional. They just allow you to reuse other objects'
    # fields by passing in that object and calling populate_schema with
    # a kwarg set to the other object.
    SCHEMA = {
        'a_useful_field': ('example_obj', 'a_useful_field')
    }

    # Version 1.0: Initial version
    VERSION = '1.0'

    fields = {
```

(continues on next page)

(continued from previous page)

```
        'a_useful_field': fields.StringField(),
        'an_extra_field': fields.StringField(nullable=True)
    }
```

Note that both the payload and notification classes are oslo versioned objects. Modifications to these require a version bump so that consumers of notifications know when the notifications have changed.

SCHEMA defines how to populate the payload fields. Its an optional attribute that subclasses may use to easily populate notifications with data from other objects.

It is a dictionary where every key value pair has the following format:

```
<payload_field_name>: (<data_source_name>,
                       <field_of_the_data_source>)
```

The `<payload_field_name>` is the name where the data will be stored in the payload object; this field has to be defined as a field of the payload. The `<data_source_name>` shall refer to name of the parameter passed as kwarg to the payloads `populate_schema()` call and this object will be used as the source of the data. The `<field_of_the_data_source>` shall be a valid field of the passed argument.

The SCHEMA needs to be applied with the `populate_schema()` call before the notification can be emitted.

The value of the `payload.<payload_field_name>` field will be set by the `<data_source_name>.<field_of_the_data_source>` field. The `<data_source_name>` will not be part of the payload object internal or external representation.

Payload fields that are not set by the SCHEMA can be filled in the same way as in any versioned object.

Then, to create a payload, you would do something like the following. Note that if you choose to define a schema in the SCHEMA class variable, you must populate the schema by calling `populate_schema(example_obj=my_example_obj)` before emitting the notification is allowed:

```
my_example_obj = ExampleObject(id=1,
                               a_useful_field='important',
                               not_useful_field='blah')

# an_extra_field is optional since it's not a part of the SCHEMA and is a
# nullable field in the class fields
my_notify_payload = ExampleNotifyPayload(an_extra_field='hello')
# populate the schema with the ExampleObject fields
my_notify_payload.populate_schema(example_obj=my_example_obj)
```

You then create the notification with the oslo required fields (event_type, publisher_id, and level, all sender fields needed by oslo that are defined in the ironic notification base classes) and emit it:

```
notify = ExampleNotification(
    event_type=notification.EventType(object='example_obj',
        action='do_something', status=fields.NotificationStatus.START),
    publisher=notification.NotificationPublisher(
        service='ironic-conductor',
        host='hostname01'),
    level=fields.NotificationLevel.DEBUG,
```

(continues on next page)

(continued from previous page)

```
    payload=my_notify_payload)
notify.emit(context)
```

When specifying the event_type, `object` will specify the object being acted on, `action` will be a string describing what action is being performed on that object, and `status` will be one of start, end, error, or success. start and end are used to indicate when actions that are not immediate begin and succeed. success is used to indicate when actions that are immediate succeed. error is used to indicate when any type of action fails, regardless of whether its immediate or not. As a result of specifying these parameters, event_type will be formatted as `baremetal.<object>.<action>.<status>` on the message bus.

This example will send the following notification over the message bus:

```
{
    "priority": "debug",
    "payload":{
        "ironic_object.namespace":"ironic",
        "ironic_object.name":"ExampleNotifyPayload",
        "ironic_object.version":"1.0",
        "ironic_object.data":{
            "a_useful_field":"important",
            "an_extra_field":"hello"
        }
    },
    "event_type":"baremetal.example_obj.do_something.start",
    "publisher_id":"ironic-conductor.hostname01"
}
```

About OSProfiler

OSProfiler is an OpenStack cross-project profiling library. Its API provides different ways to add a new trace point. Trace points contain two messages (start and stop). Messages like below are sent to a collector:

```
{
    "name": <point_name>-(start|stop),
    "base_id": <uuid>,
    "parent_id": <uuid>,
    "trace_id": <uuid>,
    "info": <dict>
}
```

The fields are defined as follows:

`base_id` - <uuid> that is same for all trace points that belong to one trace. This is used to simplify the process of retrieving all trace points (related to one trace) from the collector.

`parent_id` - <uuid> of parent trace point.

`trace_id` - <uuid> of current trace point.

`info` - the dictionary that contains user information passed when calling profiler start() & stop() methods.

The profiler uses ceilometer as a centralized collector. Two other alternatives for ceilometer are pure MongoDB driver and Elasticsearch.

A notifier is setup to send notifications to ceilometer using oslo.messaging and ceilometer API is used to retrieve all messages related to one trace.

OSProfiler has entry point that allows the user to retrieve information about traces and present it in HTML/JSON using CLI.

For more details see OSProfiler Cross-project profiling library.

How to Use OSProfiler with Ironic in Devstack

To use or test OSProfiler in ironic, the user needs to setup Devstack with OSProfiler and ceilometer. In addition to the setup described at *Deploying Ironic with DevStack*, the user needs to do the following:

Add the following to `localrc` to enable OSProfiler and ceilometer:

```
enable_plugin ceilometer https://opendev.org/openstack/ceilometer
enable_plugin osprofiler https://opendev.org/openstack/osprofiler

# Enable the following services
CEILOMETER_NOTIFICATION_TOPICS=notifications,profiler
ENABLED_SERVICES+=,ceilometer-acompute,ceilometer-acentral
ENABLED_SERVICES+=,ceilometer-anotification,ceilometer-collector
ENABLED_SERVICES+=,ceilometer-alarm-evaluator,ceilometer-alarm-notifier
ENABLED_SERVICES+=,ceilometer-api
```

Run stack.sh.

Once Devstack environment is setup, edit `ironic.conf` to set the following profiler options and restart ironic services:

```
[profiler]
enabled = True
hmac_keys = SECRET_KEY # default value used across several OpenStack projects
trace_sqlalchemy = True
```

In order to trace ironic using OSProfiler, use openstackclient to run baremetal commands with `--os-profile SECRET_KEY`.

For example, the following will cause a <trace-id> to be printed after node list:

```
$ openstack --os-profile SECRET_KEY baremetal node list
```

Output of the above command will include the following:

```
Trace ID: <trace-id>
Display trace with command:
osprofiler trace show --html <trace-id>
```

The trace results can be seen using this command:

```
$ osprofiler trace show --html <trace-id>
```

The trace results can be saved in a file with `--out file-name` option:

```
$ osprofiler trace show --html <trace-id> --out trace.html
```

The trace results show the time spent in ironic-api, ironic-conductor, and db calls. More detailed db tracing is enabled if `trace_sqlalchemy` is set to true.

References

- OSProfiler Cross-project profiling library
- *Deploying Ironic with DevStack*

Rolling Upgrades

The ironic (ironic-api and ironic-conductor) services support rolling upgrades. This describes the design of rolling upgrades, followed by notes for developing new features or modifying an IronicObject.

Design

Rolling upgrades between releases

Ironic follows the release-cycle-with-intermediary release model. The releases are semantic-versioned, in the form <major>.<minor>.<patch>. We refer to a `named release` of ironic as the release associated with a time based integrated release of OpenStack, for example 2025.1 which was named Epoxy. Please review OpenStack releases for the high level release version to name mappings.

In addition, ironic follows the standard deprecation policy, which says that the deprecation period must be at least three months and a cycle boundary. This means that there will never be anything that is both deprecated *and* removed between two named releases.

Rolling upgrades will be supported between:

- named release N to N+1
- any named release to its latest revision, containing backported bug fixes. Because those bug fixes can contain improvements to the upgrade process, the operator should patch the system before upgrading between named releases.
- most recent named release N (and semver releases newer than N) to master. As with the above bullet point, there may be a bug or a feature introduced on a master branch, that we want to remove before publishing a named release. Deprecation policy allows to do this in a 3 month time frame. If the feature was included and removed in intermediate releases, there should be a release note added, with instructions on how to do a rolling upgrade to master from an affected release or release span. This would typically instruct the operator to upgrade to a particular intermediate release, before upgrading to master.

Rolling upgrade process

Ironic supports rolling upgrades as described in the *upgrade guide*.

The upgrade process will cause the ironic services to be running the `FromVer` and `ToVer` releases in this order:

0. Upgrade ironic code and run database schema migrations via the `ironic-dbsync upgrade` command.
1. Upgrade code and restart ironic-conductor services, one at a time.

2. Upgrade code and restart ironic-api services, one at a time.

3. Unpin API, RPC and object versions so that the services can now use the latest versions in `ToVer`. This is done via updating the configuration option described below in *API, RPC and object version pinning* and then restarting the services. ironic-conductor services should be restarted first, followed by the ironic-api services. This is to ensure that when new functionality is exposed on the unpinned API service (via API micro version), it is available on the backend.

step	ironic-api	ironic-conductor
0	all FromVer	all FromVer
1.1	all FromVer	some FromVer, some ToVer-pinned
1.2	all FromVer	all ToVer-pinned
2.1	some FromVer, some ToVer-pinned	all ToVer-pinned
2.2	all ToVer-pinned	all ToVer-pinned
3.1	all ToVer-pinned	some ToVer-pinned, some ToVer
3.2	all ToVer-pinned	all ToVer
3.3	some ToVer-pinned, some ToVer	all ToVer
3.4	all ToVer	all ToVer

Policy for changes to the DB model

The policy for changes to the DB model is as follows:

- Adding new items to the DB model is supported.
- The dropping of columns or tables and corresponding objects fields is subject to ironics deprecation policy. But its alembic script has to wait one more deprecation period, otherwise an `unknown column` exception will be thrown when `FromVer` services access the DB. This is because **`ironic-dbsync upgrade`** upgrades the DB schema but `FromVer` services still contain the dropped field in their SQLAlchemy DB model.
- An `alembic.op.alter_column()` to rename or resize a column is not allowed. Instead, split it into multiple operations, with one operation per release cycle (to maintain compatibility with an old SQLAlchemy model). For example, to rename a column, add the new column in release N, then remove the old column in release N+1.
- Some implementations of SQLs `ALTER TABLE`, such as adding foreign keys in PostgreSQL, may impose table locks and cause downtime. If the change cannot be avoided and the impact is significant (e.g. the table can be frequently accessed and/or store a large dataset), these cases must be mentioned in the release notes.

API, RPC and object version pinning

For the ironic services to be running old and new releases at the same time during a rolling upgrade, the services need to be able to handle different API, RPC and object versions.

This versioning is handled via the configuration option: `[DEFAULT]/pin_release_version`. It is used to pin the API, RPC and IronicObject (e.g., Node, Conductor, Chassis, Port, and Portgroup) versions for all the ironic services.

The default value of empty indicates that ironic-api and ironic-conductor will use the latest versions of API, RPC and IronicObjects. Its possible values are releases, named (e.g. `2025.1`) or sem-versioned (e.g. `29.0`).

Internally, in common/release_mappings.py, ironic maintains a mapping that indicates the API, RPC and IronicObject versions associated with each release. This mapping is maintained manually.

During a rolling upgrade, the services using the new release will set the configuration option value to be the name (or version) of the old release. This will indicate to the services running the new release, which API, RPC and object versions that they should be compatible with, in order to communicate with the services using the old release.

Handling API versions

When the (newer) service is pinned, the maximum API version it supports will be the pinned version which the older service supports (as described above at *API, RPC and object version pinning*). The ironic-api service returns HTTP status code 406 for any requests with API versions that are higher than this maximum version.

Handling RPC versions

ConductorAPI.__init__() sets the `version_cap` variable to the desired (latest or pinned) RPC API version and passes it to the `RPCClient` as an initialization parameter. This variable is then used to determine the maximum requested message version that the `RPCClient` can send.

Each RPC call can customize the request according to this `version_cap`. The *Ironic RPC versions* section below has more details about this.

Handling IronicObject versions

Internally, ironic services deal with IronicObjects in their latest versions. Only at these boundaries, when the IronicObject enters or leaves the service, do we deal with object versioning:

- getting objects from the database: convert to latest version
- saving objects to the database: if pinned, save in pinned version; else save in latest version
- serializing objects (to send over RPC): if pinned, send pinned version; else send latest version
- deserializing objects (receiving objects from RPC): convert to latest version

The ironic-api service also has to handle API requests/responses based on whether or how a feature is supported by the API version and object versions. For example, when the ironic-api service is pinned, it can only allow actions that are available to the objects pinned version, and cannot allow actions that are only available for the latest version of that object.

To support this:

- All the database tables (SQLAlchemy models) of the IronicObjects have a column named `version`. The value is the version of the object that is saved in the database.
- The method `IronicObject.get_target_version()` returns the target version. If pinned, the pinned version is returned. Otherwise, the latest version is returned.
- The method `IronicObject.convert_to_version()` converts the object into the target version. The target version may be a newer or older version than the existing version of the object. The bulk of the work is done in the helper method `IronicObject._convert_to_version()`. Subclasses that have new versions redefine this to perform the actual conversions.

In the following,

- The old release is `FromVer`; it uses version 1.14 of a Node object.

- The new release is `ToVer`. It uses version 1.15 of a Node object this has a deprecated `extra` field and a new `meta` field that replaces `extra`.
- db_obj[meta] and db_obj[extra] are the database representations of those node fields.

Getting objects from the database (API/conductor < DB)

Both ironic-api and ironic-conductor services read values from the database. These values are converted to IronicObjects via the method `IronicObject._from_db_object()`. This method always returns the IronicObject in its latest version, even if it was in an older version in the database. This is done regardless of the service being pinned or not.

Note that if an object is converted to a later version, that IronicObject will retain any changes (in its `_changed_fields` field) resulting from that conversion. This is needed in case the object gets saved later, in the latest version.

For example, if the node in the database is in version 1.14 and has db_obj[extra] set:

- a `FromVer` service will get a Node with node.extra = db_obj[extra] (and no knowledge of node.meta since it doesnt exist)
- a `ToVer` service (pinned or unpinned), will get a Node with:
  - node.meta = db_obj[extra]
  - node.extra = None
  - node._changed_fields = [meta, extra]

Saving objects to the database (API/conductor > DB)

The version used for saving IronicObjects to the database is determined as follows:

- For an unpinned service, the object is saved in its latest version. Since objects are always in their latest version, no conversions are needed.
- For a pinned service, the object is saved in its pinned version. Since objects are always in their latest version, the object needs to be converted to the pinned version before being saved.

The method `IronicObject.do_version_changes_for_db()` handles this logic, returning a dictionary of changed fields and their new values (similar to the existing `oslo.versionedobjects.VersionedObject.obj_get_changes()`). Since we do not keep track internally, of the database version of an object, the objects `version` field will always be part of these changes.

The *Rolling upgrade process* (at step 3.1) ensures that by the time an object can be saved in its latest version, all services are running the newer release (although some may still be pinned) and can handle the latest object versions.

An interesting situation can occur when the services are as described in step 3.1. It is possible for an IronicObject to be saved in a newer version and subsequently get saved in an older version. For example, a `ToVer` unpinned conductor might save a node in version 1.5. A subsequent request may cause a `ToVer` pinned conductor to replace and save the same node in version 1.4!

Sending objects via RPC (API/conductor -> RPC)

When a service makes an RPC request, any IronicObjects that are sent as part of that request are serialized into entities or primitives via `IronicObjectSerializer.serialize_entity()`. The version used for objects being serialized is as follows:

- For an unpinned service, the object is serialized to its latest version. Since objects are always in their latest version, no conversions are needed.
- For a pinned service, the object is serialized to its pinned version. Since objects are always in their latest version, the object is converted to the pinned version before being serialized. The converted object includes changes that resulted from the conversion; this is needed so that the service at the other end of the RPC request has the necessary information if that object will be saved to the database.

Receiving objects via RPC (API/conductor <- RPC)

When a service receives an RPC request, any entities that are part of the request need to be deserialized (via `oslo.versionedobjects.VersionedObjectSerializer.deserialize_entity()`). For entities that represent IronicObjects, we want the deserialization process (via `IronicObjectSerializer._process_object()`) to result in IronicObjects that are in their latest version, regardless of the version they were sent in and regardless of whether the receiving service is pinned or not. Again, any objects that are converted will retain the changes that resulted from the conversion, useful if that object is later saved to the database.

For example, a `FromVer` ironic-api could issue an `update_node()` RPC request with a node in version 1.4, where node.extra was changed (so node._changed_fields = [extra]). This node will be serialized in version 1.4. The receiving `ToVer` pinned ironic-conductor deserializes it and converts it to version 1.5. The resulting node will have node.meta set (to the changed value from node.extra in v1.4), node.extra = None, and node._changed_fields = [meta, extra].

When developing a new feature or modifying an IronicObject

When adding a new feature or changing an IronicObject, they need to be coded so that things work during a rolling upgrade.

The following describe areas where the code may need to be changed, as well as some points to keep in mind when developing code.

ironic-api

During a rolling upgrade, the new, pinned ironic-api is talking to a new conductor that might also be pinned. There may also be old ironic-api services. So the new, pinned ironic-api service needs to act like it was the older service:

- New features should not be made available, unless they are somehow totally supported in the old and new releases. Pinning the API version is in place to handle this.
  - If, for whatever reason, the API version pinning doesnt prevent a request from being handled that cannot or should not be handled, it should be coded so that the response has HTTP status code 406 (Not Acceptable). This is the same response to requests that have an incorrect (old) version specified.

Ironic RPC versions

When the signature (arguments) of an RPC method is changed or new methods are added, the following needs to be considered:

- The RPC version must be incremented and be the same value for both the client (`ironic/conductor/rpcapi.py`, used by ironic-api) and the server (`ironic/conductor/manager.py`, used by ironic-conductor). It should also be updated in `ironic/common/release_mappings.py`.
- Until there is a major version bump, new arguments of an RPC method can only be added as optional. Existing arguments cannot be removed or changed in incompatible ways with the method in older RPC versions.
- ironic-api (client-side) sets a version cap (by passing the version cap to the constructor of oslo_messaging.RPCClient). This pinning is in place during a rolling upgrade when the `[DEFAULT]/pin_release_version` configuration option is set.
- New RPC methods are not available when the service is pinned to the older release version. In this case, the corresponding REST API function should return a server error or implement alternative behaviours.
- Methods which change arguments should run `client.can_send_version()` to see if the version of the request is compatible with the version cap of the RPC Client. Otherwise the request needs to be created to work with a previous version that is supported.
- ironic-conductor (server-side) should tolerate older versions of requests in order to keep working during the rolling upgrade process. The behaviour of ironic-conductor will depend on the input parameters passed from the client-side.
- Old methods can be removed only after they are no longer used by a previous named release.

Object versions

When subclasses of `ironic.objects.base.IronicObject` are modified, the following needs to be considered:

- Any change of fields or change in signature of remotable methods needs a bump of the object version. The object versions are also maintained in `ironic/common/release_mappings.py`.
- New objects must be added to `ironic/common/release_mappings.py`. Also for the first releases they should be excluded from the version check by adding their class names to the `NEW_MODELS` list in `ironic/command/dbsync.py`.
- The arguments of remotable methods (methods which are remoted to the conductor via RPC) can only be added as optional. They cannot be removed or changed in an incompatible way (to the previous release).
- Field types cannot be changed. Instead, create a new field and deprecate the old one.
- There is a unit test that generates the hash of an object using its fields and the signatures of its remotable methods. Objects that have a version bump need to be updated in the expected_object_fingerprints dictionary; otherwise this test will fail. A failed test can also indicate to the developer that their change(s) to an object require a version bump.
- When new version objects communicate with old version objects and when reading or writing to the database, `ironic.objects.base.IronicObject._convert_to_version()` will be called to

convert objects to the target version. Objects should implement their own ._convert_to_version() to remove or alter fields which were added or changed after the target version:

```
def _convert_to_version(self, target_version,
                        remove_unavailable_fields=True):
    """Convert to the target version.

    Subclasses should redefine this method, to do the conversion of the
    object to the target version.

    Convert the object to the target version. The target version may be
    the same, older, or newer than the version of the object. This is
    used for DB interactions as well as for serialization/deserialization.

    The remove_unavailable_fields flag is used to distinguish these two
    cases:

    1) For serialization/deserialization, we need to remove the␣
↪unavailable
       fields, because the service receiving the object may not know about
       these fields. remove_unavailable_fields is set to True in this␣
↪case.

    2) For DB interactions, we need to set the unavailable fields to their
       appropriate values so that these fields are saved in the DB. (If
       they are not set, the VersionedObject magic will not know to
       save/update them to the DB.) remove_unavailable_fields is set to
       False in this case.

    :param target_version: the desired version of the object
    :param remove_unavailable_fields: True to remove fields that are
        unavailable in the target version; set this to True when
        (de)serializing. False to set the unavailable fields to␣
↪appropriate
        values; set this to False for DB interactions.
```

This method must handle:

- converting from an older version to a newer version
- converting from a newer version to an older version
- making sure, when converting, that you take into consideration other object fields that may have been affected by a field (value) only available in a newer version. For example, if field new is only available in Node version 1.5 and Node.affected = Node.new+3, when converting to 1.4 (an older version), you may need to change the value of Node.affected too.

Online data migrations

The `ironic-dbsync online_data_migrations` command will perform online data migrations.

Keep in mind the *Policy for changes to the DB model*. Future incompatible changes in SQLAlchemy models, like removing or renaming columns and tables can break rolling upgrades (when ironic services

are run with different release versions simultaneously). It is forbidden to remove these database resources when they may still be used by the previous named release.

When creating new Alembic migrations which modify existing models, make sure that any new columns default to NULL. Test the migration out on a non-empty database to make sure that any new constraints dont cause the database to be locked out for normal operations.

You can find an overview on what DDL operations may cause downtime in https://dev.mysql.com/doc/refman/5.7/en/innodb-create-index-overview.html. (You should also check older, widely deployed InnoDB versions for issues.) In the case of PostgreSQL, adding a foreign key may lock a whole table for writes.

Make sure to add a release note if there are any downtime-related concerns.

Backfilling default values, and migrating data between columns or between tables must be implemented inside an online migration script. A script is a database API method (added to `ironic/db/api.py` and `ironic/db/sqlalchemy/api.py`) which takes two arguments:

- context: an admin context
- max_count: this is used to limit the query. It is the maximum number of objects to migrate; >= 0. If zero, all the objects will be migrated.

It returns a two-tuple:

- the total number of objects that need to be migrated, at the start of the method, and
- the number of migrated objects.

In this method, the version column can be used to select and update old objects.

The method name should be added to the list of `ONLINE_MIGRATIONS` in `ironic/command/dbsync.py`.

The method should be removed in the next named release after this one.

After online data migrations are completed and the SQLAlchemy models no longer contain old fields, old columns can be removed from the database. This takes at least 3 releases, since we have to wait until the previous named release no longer contains references to the old schema. Before removing any resources from the database by modifying the schema, make sure that your implementation checks that all objects in the affected tables have been migrated. This check can be implemented using the version column.

ironic-dbsync upgrade command

The `ironic-dbsync upgrade` command first checks that the versions of the objects are compatible with the (new) release of ironic, before it will make any DB schema changes. If one or more objects are not compatible, the upgrade will not be performed.

This check is done by comparing the objects `version` field in the database with the expected (or supported) versions of these objects. The supported versions are the versions specified in `ironic.common.release_mappings.RELEASE_MAPPING`. The newly created tables cannot pass this check and thus have to be excluded by adding their object class names (e.g. `Node`) to `ironic.command.dbsync.NEW_MODELS`.

Role Based Access Control - Testing

The Role Based Access control testing is a minor departure from the Ironic standard pattern of entirely python based unit testing. In part this was done for purposes of speed and to keep the declaration of the test context.

This also lended itself to be very useful due to the nature of A/B testing which is required to properly migrate the Ironic project from a project scoped universe where an `admin project` is utilized as the authenticating factor coupled with two custom roles, `baremetal_admin`, and `baremetal_observer`.

As a contributor looking back after getting a over a thousand additional tests in place using this method, it definitely helped the speed at which these were created, and then ported to support additional.

How these tests work

These tests execute API calls through the API layer, using the appropriate verb and header, which settings to prevent the `keystonemiddleware` from intercepting and replacing the headers were passing. Ultimately this is a feature, and it helps quite a bit.

The second aspect of how this works is were mocking the conductor RPC `get_topic_for` and `get_random_topic_for` methods. These calls raise Temporary Unavailable, since trying to execute the entire interaction into the conductor is moderately pointless because all policy enforement is located with-in the API layer.

At the same time wiring everything up to go from API to conductor code and back would have been a heavier lift. As such, the tests largely look for one of the following error codes.

- 200 - Got the item from the API - This is an database driven interaction.
- 201 - Created - This is database driven interaction. These are rare.
- 204 - Accepted - This is a database driven interaction. These are rare.
- **403 - Forbidden - This tells us the policy worked as expected where** access was denied.
- **404 - NotFound - This is typically when objects were not found. Before** Ironic becomes scope aware, these are generally only in the drivers API endpoints behavior. In System scope aware Project scoped configuration, i.e. later RBAC tests, this will become the dominant response for project scoped users as responding with a 403 if they could be an owner or lessee would provide insight into the existence of a node.
- **503 - Service Unavailable - In the context of our tests, we expect this** when a request *has* been successfully authenticated and would have been sent along to the conductor.

How to make changes or review these tests?

The tests cycle through the various endpoints, and repeating patterns are clearly visible. Typically this means a given endpoint is cycled through with the same basic test using slightly different parameters such as different authentication parameters. When it comes to system scope aware tests supporting node `owners` and `lessee`, these tests will cycle a little more with slightly different attributes as the operation is not general against a shared common node, but different nodes.

Some tests will test body contents, or attributes. some will validate the number of records returned. This is important later with `owner` and `lessee` having slightly different views of the universe.

Some general rules apply

- Admins can do things, at least as far as their scope or rights apply. Remember: owner and lessee admins are closer to System scoped Admin Members.
- Members can do some things, but not everything
- Readers can always read, but as we get into sensitive data later on such as fields containing infrastructure internal addresses, these values will become hidden and additional tests will examine this.
- Third party, or external/other Admins will find nothing but sadness in empty lists, 403, 404, or even 500 errors.

What is/will be tested?

The idea is to in essence test as much as possible, however as these tests Role Based Access Control related capabilities will come in a series of phases, styles vary a little.

The first phase is `"legacy"`. In essence these are partially programmatically generated and then human reviewed and values populated with expected values.

The second phase is remarkably similar to `legacy`. It is the safety net where we execute the `legacy` tests with the updated `oslo.policy` configuration to help enforce scopes. These tests will intentionally begin to fail in phase three.

The third phase is the implementation of System scope awareness for the API. In this process, as various portions of the API are made system scope aware. The `legacy` tests are marked as `deprecated` which signals to the second phase test sequences that they are **expected** to fail. New `system scoped` tests are also implemented which are matched up by name to the `legacy` tests. The major difference being some header values, and a user with a `member` role in the `system` scope now has some rights.

The forth phase, is implementation of `owner` and `lessee` aware project scoping. The testing approach is similar, however it is much more of a shotgun approach. We test what we know should work, and what know should not work, but we do not have redundant testing for each role as `admin` users are also `members`, and since the policy rules are designed around thresholds of access, it just made no sense to run the same test for admin and members, where member was the threshold. These thresholds will vary with the proposed default policy. The forth scope also tests a third party external admin as a negative test to ensure that we are also denying access to resources appropriately.

Metrics

Ironic provides a pluggable metrics library as of the 2.0.0 release. The metrics backend to be used is configured via `CONF.metrics.backend`. Not all backends support all metrics types or metric sources.

The typical usage of metrics is to initialize and cache a metrics logger, using the `get_metrics_logger()` method in `metrics_utils`, then use that object to decorate functions or create context managers to gather metrics. The general convention is to provide the name of the module as the first argument to set it as the prefix, then set the actual metric name to the method name. For example:

```
from ironic import metrics_utils

METRICS = metrics_utils.get_metrics_logger(__name__)

@METRICS.timer('my_simple_method')
def my_simple_method(arg, matey):
```

(continues on next page)

(continued from previous page)

```
    pass

def my_complex_method(arg, matey):
    with METRICS.timer('complex_method_pt_1'):
        do_some_work()

    with METRICS.timer('complex_method_pt_2'):
        do_more_work()
```

There are three different kinds of metrics:

- **Timers** measure how long the code in the decorated method or context manager takes to execute, and emits the value as a timer metric. These are useful for measuring performance of a given block of code.
- **Counters** increment a counter each time a decorated method or context manager is executed. These are useful for counting the number of times a method is called, or the number of times an event occurs.
- **Gauges** return the value of a decorated method as a metric. This is useful when you want to monitor the value returned by a method over time.

Additionally, metrics can be sent directly, rather than using a context manager or decorator, when appropriate. When used in this way, we will simply emit the value provided as the requested metric type. For example:

```
from ironic import metrics_utils

METRICS = metrics_utils.get_metrics_logger(__name__)

def my_node_failure_method(node):
    if node.failed:
        METRICS.send_counter(node.uuid, 1)
```

The provided statsd backend natively supports all three metric types. For more information about how statsd changes behavior based on the metric type, see statsd metric types

5.1.5 Governance and Processes

These pages contain information for PTLs, cross-project liaisons, and core reviewers.

Releasing Ironic Projects

Since the responsibility for releases will move between people, we document that process here.

A full list of projects that Ironic manages is available in the governance site.

Who is responsible for releases?

The current PTL is ultimately responsible for making sure code gets released. They may choose to delegate this responsibility to a liaison, which is documented in the cross-project liaison wiki.

Anyone may submit a release request per the process below, but the PTL or liaison must +1 the request for it to be processed.

Release process

Releases are managed by the OpenStack release team. The release process is documented in the Project Team Guide.

What do we have to release?

The Ironic project has a number of deliverables under its governance. The ultimate source of truth for this is projects.yaml in the governance repository. These deliverables have varying release models, and these are defined in the deliverables YAML files in the releases repository.

In general, Ironic deliverables follow the cycle-with-intermediary release model.

Non-client libraries

The following deliverables are non-client libraries:

- sushy

Client libraries

The following deliverables are client libraries:

- python-ironicclient

Normal release

The following deliverables are Neutron plugins:

- networking-baremetal
- networking-generic-switch

The following deliverables are Horizon plugins:

- ironic-ui

The following deliverables are Tempest plugins:

- ironic-tempest-plugin

The following deliverables are tools:

- ironic-python-agent-builder

> **Note**
>
> if ironic-python-agent-builder is branched after ironic-python-agent, we need to make sure all artifacts are built by the time of Ironic branching.

The following deliverables are services, or treated as such:

- bifrost
- ironic
- ironic-prometheus-exporter
- ironic-python-agent

Manual release

The ironic-staging-drivers follows a different procedure, see Releasing ironic-staging-drivers.

Independent

The following deliverables are released independently:

- sushy-tools
- virtualbmc

Not released

The following deliverables do not need to be released:

- ironic-specs

OpenStack Release Schedule

The Ironic project follows the OpenStack coordinated release schedule. Each OpenStack release cycle spans approximately 6 months (26 weeks) and follows a structured timeline with specific milestones and deadlines.

Based on the standard OpenStack release schedule, a typical release cycle includes the following key phases:

Release Timeline Overview

Development Phase (Weeks R-25 to R-12)

- **R-25 to R-21**: Early development, feature planning
- **R-20 (Week 6)**: First milestone
- **R-17**: Cycle-trailing release deadline for previous cycle
- **R-12 (Week 14)**: Second milestone and membership freeze

Feature Development Phase (Weeks R-11 to R-5)

- **R-11 to R-8**: Active feature development
- **R-7**: Extra Active Contributor (AC) freeze
- **R-6**: Final release for non-client libraries (sushy)
- **R-5 (Week 22)**: Third milestone, feature freeze, final release for client libraries (python-ironicclient), plugins (networking-baremetal, networking-generic-switch, ironic-ui, ironic-tempest-plugin), and tools (ironic-python-agent-builder), soft string freeze, requirements freeze

Stabilization Phase (Weeks R-4 to R-1)

- **R-4**: Cycle highlights deadline
- **R-3**: RC1 target week for services (bifrost, ironic, ironic-prometheus-exporter, ironic-python-agent), hard string freeze
- **R-2**: Bug fixing and stabilization

- **R-1**: Final RCs and intermediary releases

Release Week (R+0)

- **R+0**: Coordinated release for services (bifrost, ironic, ironic-prometheus-exporter, ironic-python-agent) (typically on a Wednesday)

Key Deadlines for Ironic Projects

Milestone Releases

- Projects following the cycle-with-intermediary model should aim to release at each milestone
- Milestone participation is required for projects to be considered part of the coordinated release

Library Releases

- Non-client libraries (sushy): Final release at R-6
- Client libraries (python-ironicclient): Final release at R-5

Plugin Releases

- Neutron plugins (networking-baremetal, networking-generic-switch): Final release at R-5
- Horizon plugins (ironic-ui): Final release at R-5
- Tempest plugins (ironic-tempest-plugin): Final release at R-5

Service Releases

- Services (bifrost, ironic, ironic-prometheus-exporter, ironic-python-agent): Release candidates at R-3, final release at R+0
- Tools (ironic-python-agent-builder): Final release at R-5

Feature Freeze (R-5)

- No new features accepted after the third milestone
- Focus shifts to bug fixes and stabilization
- Release notes should be finalized

String Freeze

- Soft string freeze at R-5: No new user-facing strings
- Hard string freeze at R-3: No string changes to allow translation work

Requirements Freeze (R-5)

- Only critical requirements changes allowed after feature freeze
- Helps downstream packagers prepare for the release

Planning Your Release

When planning Ironic releases within an OpenStack cycle:

1. **Early Cycle (R-25 to R-12)**: Plan major features, coordinate with dependencies
2. **Mid Cycle (R-11 to R-6)**: Complete feature development, release libraries

3. **Late Cycle (R-5 to R+0)**: Focus on stabilization, bug fixes, and release preparation

For the most current release schedule, always refer to the official OpenStack releases website at https://releases.openstack.org/.

Bugfix branches

The following projects have `bugfix/X.Y` branches in addition to standard openstack `stable/NAME` branches:

- ironic
- ironic-python-agent

These projects receive releases every six months as part of the coordinated OpenStack release that happens semi-annually. These releases can be found in a `stable/NAME` branch.

They are also evaluated for additional bugfix releases between scheduled stable releases at the two and four month milestone between stable releases (roughly every 2 months). These releases can be found in a `bugfix/X.Y` branch. A bugfix release is only created if there are significant beneficial changes and a known downstream operator or distributor will consume the release.

A bugfix branch is supported for 6 months after its creation date.

To leave some version space for releases from these branches, releases of these projects from the master branch always increase either the major or the minor version.

Currently retirement of bugfix branches cannot be automated and must be done by the ironic team manually as explained below.

There are usually 3 bugfix branches present at all time, the latest 2 are actively maintained, while the third one is considered unmaintained. After creating a new bugfix branch, the oldest bugfix branch should be tagged as EoL and deleted.

Only members of the ironic-core or ironic-release groups can delete branches, please refer to this procedure to remove the oldest bugfix branch after the creation of a new one:

- checkout locally the bugfix branch to move to EoL, if it does not exist locally its possible to checkout it from remote and switch to it using `git checkout -t`, for example for bugfix/24.0 use:

```
git checkout -t origin/bugfix/24.0
```

- fast forward to latest change using:

```
git pull --ff-only
```

- add a signed tag to the latest commit of the bugfix branch named `bugfix-X.Y-eol` and add EOL bugfix/X.Y as description, for example for bugfix/24.0 add the tag bugfix-24.0-eol; use the `git tag` command for that, for example for bugfix/24.0 the syntax would be:

```
git tag -s bugfix-24.0-eol -m "EOL bugfix/24.0"
```

- push the new tag to gerrit using `git push gerrit TAG_NAME`, for example for bugfix/24.0 use:

```
git push gerrit bugfix-24.0-eol
```

- delete the bugfix branch on gerrit using `git push gerrit --delete BUGFIX_BRANCH_NAME`, again for bugfix/24.0 would be:

```
git push gerrit --delete bugfix/24.0
```

After the creation of a bugfix branch it is highly recommended to update the upper-constraints link for the tests in the tox.ini file, plus override the branch for the requirements project to be sure to use the correct upper-constraints from the branch creation time; for example see the following change:

https://review.opendev.org/c/openstack/ironic/+/938660

It is also mandatory to comment out or remove the metal3 integration job as it is not supposed to run in stable or bugfix branches, and also comment out or remove the tox codespell job to avoid failures due to version changes, spelling is fixed in master and most recent stable branches only and backported as is.

Things to do before releasing

- Review the unreleased release notes, if the project uses them. Make sure they follow our *standards*, are coherent, and have proper grammar. Combine release notes if necessary (for example, a release note for a feature and another release note to add to that feature may be combined).
- For Ironic releases only: if any new API microversions have been added since the last release, update the REST API version history (`doc/source/contributor/webapi-version-history.rst`) to indicate that they were part of the new release.
- To support rolling upgrades, add this new release version (and release name if it is a named release) into `ironic/common/release_mappings.py`:
  - in `RELEASE_MAPPING` make a copy of the `master` entry, and rename the first `master` entry to the new semver release version.
  - If this is a named release, add a `RELEASE_MAPPING` entry for the named release. Its value should be the same as that of the latest semver one (that you just added above).

    It is important to do this before a stable/<release> branch is made (or if the grenade switch is made to use the latest release from stable as the old release). Otherwise, once it is made, CI (the grenade job that tests new-release -> master) will fail.
- Check for any open patches that are close to be merged or release critical.

  This usually includes important bug fixes and/or features that wed like to release, including the related documentation.

How to propose a release

The steps that lead to a release proposal are mainly manual, while proposing the release itself is almost a 100% automated process, accomplished by following the next steps:

- Clone the openstack/releases repository. This is where deliverables are tracked and all the automation resides.
  - Under the `deliverables` directory you can see yaml files for each deliverable (i.e. subproject) grouped by release cycles.
  - The `_independent` directory contains yaml files for deliverables that are not bound to (official) cycles (e.g. Ironic-python-agent-builder).
- To check the changes were about to release we can use the tox environment `list-unreleased-changes`, with this syntax:

```
tox -e venv -- list-unreleased-changes <series> <deliverable>
```

The `series` argument is a release series (i.e. master or train, not stable/ussuri or stable/train).

For example, assuming were in the main directory of the releases repository, to check the changes in the ussuri series for Ironic-python-agent type:

```
tox -e venv -- list-unreleased-changes ussuri openstack/ironic-python-
↪agent
```

- To update the deliverable file for the new release, we use a scripted process in the form of a tox environment called `new-release`.

  To get familiar with it and see all the options, type:

  ```
  tox -e venv -- new-release -h
  ```

  Now, based on the list of changes we found in the precedent step, and the release notes, we need to decide on whether the next version will be major, minor (feature) or patch (bugfix).

  Note that in this case `series` is a code name (train, ussuri), not a branch. That is also valid for the current development branch (master) that takes the code name of the future stable release, for example if the future stable release code name is wallaby, we need to use wallaby as `series`.

  The `--stable-branch argument` is used only for branching in the end of a cycle, independent projects are not branched this way though.

  The `--intermediate-branch` option is used to create an intermediate bugfix branch following the new release model for Ironic projects.

  To propose the release, use the script to update the deliverable file, then commit the change, and propose it for review.

  For example, to propose a minor release for Ironic in the master branch (current development branch), considering that the code name of the future stable release is wallaby, use:

  ```
  tox -e venv -- new-release -v wallaby ironic feature
  ```

  Remember to use a meaningful topic, usually using the name of the deliverable, the new version and the branch, if applicable.

  A good commit message title should also include the same, for example Release Ironic 1.2.3 for ussuri

- As an optional step, we can use `tox -e list-changes` to double-check the changes before submitting them for review.

  Also `tox -e validate` (it might take a while to run based on the number of changes) does some some sanity-checks, but since everything is scripted, there shouldnt be any issue.

  All the scripts are designed and maintained by the release team; in case of questions or doubts or if any errors should arise, you can reach to them in the IRC channel `#openstack-release`; all release liaisons should be present there.

- After the change is up for review, the PTL or a release liaison will have to approve it before it can get approved by the release team. Then, it will be processed automatically by zuul.

Things to do after releasing

When a release is done that results in a stable branch

When a release is done that results in a stable branch for the project, several changes need to be made.

The release automation will push a number of changes that need to be approved. This includes:

- In the new stable branch:

  Note

  OpenStack Release tooling does this automatically.

  - a change to point `.gitreview` at the branch
  - a change to update the upper constraints file used by `tox`

- In the master branch:

  - updating the release notes RST to include the new branch.

    The generated RST does not include the version range in the title, so we typically submit a follow-up patch to do that. An example of this patch is here.

  - update the `templates` in `.zuul.yaml` or `zuul.d/project.yaml`.

    The update is necessary to use the job for the next release `openstack-python3-<next_release>-jobs`. An example of this patch is here.

We need to submit patches for changes in the stable branch to:

- update the Ironic devstack plugin to point at the branched tarball for IPA. An example of this patch is here.
- set appropriate defaults for `TEMPEST_BAREMETAL_MIN_MICROVERSION` and `TEMPEST_BAREMETAL_MAX_MICROVERSION` in `devstack/lib/ironic` to make sure that unsupported API tempest tests are skipped on stable branches. E.g. patch 495319.
- remove any CI jobs which are *not* required. Mainly this revolves around the metal3-integration CI job, however other non-voting jobs can also be removed safely. This can be achieved by editing the `.zuul.d/project.yaml` file.

Note

It is normal to reduce the number of CI jobs present on a stable branch the longer the branch exists. This is a mix of challenges related to distributions, dependencies, and CI resources. Maintainers should anticipate this as a normal activity and should avoid heroic efforts.

We need to submit patches for changes on master to:

- to support rolling upgrades, since the release was a named release, we need to make these changes. Note that we need to wait until *after* the switch in grenade is made to test the latest release (N) with master (e.g. for stable/queens). Doing these changes sooner after the Ironic release and before the switch when grenade is testing the prior release (N-1) with master, will cause the tests to fail. (You may want to ask/remind infra/qa team, as to when they will do this switch.)

  - In `ironic/common/release_mappings.py`, delete any entries from `RELEASE_MAPPING` associated with the oldest named release. Since we support upgrades between adjacent named releases, the master branch will only support upgrades from the most recent named release to master.
  - remove any DB migration scripts from `ironic.command.dbsync.ONLINE_MIGRATIONS` and remove the corresponding code from Ironic. (These migration scripts are used to migrate from an old release to this latest release; they shouldnt be needed after that.)

When a release is done that results in a bugfix branch

In this case the release management only creates a change to point `.gitreview` at the branch, `tox.ini` is not modified.

After the release:

- update the Tempest microversions as explained above.
- the CI needs additional configuration, so that Zuul knows which branch to take jobs definitions from. See the following examples:
  - ironic 18.1
  - ironic-python-agent 8.1

Ironic Tempest plugin

As **ironic-tempest-plugin** is branchless, we need to submit a patch adding stable jobs to its master branch. Example for Queens.

Bifrost

Bifrost needs to be updated to install dependencies using the stable branch. Example for Victoria. The upper constraints file referenced in `scripts/install-deps.sh` needs to be updated to the new release.

For all releases

For all releases, whether or not it results in a stable branch:

- update the specs repo to mark any specs completed in the release as implemented.
- remove any -2s on patches that were blocked until after the release.

Ironic Governance Structure

The ironic project manages a number of repositories that contribute to our mission. The full list of repositories that ironic manages is available in the governance site.

What belongs in ironic governance?

For a repository to be part of the Ironic project:

- It must comply with the TCs rules for a new project.
- It must not be intended for use with only a single vendors hardware. A library that implements a standard to manage hardware from multiple vendors (such as IPMI or redfish) is okay.
- It must align with Ironics mission statement.

Lack of contributor diversity is a chicken-egg problem, and as such a repository where only a single company is contributing is okay, with the hope that other companies will contribute after joining the ironic project.

Repositories that are no longer maintained should be pruned from governance regularly.

Proposing a new project to ironic governance

Bring the proposal to the ironic weekly meeting to discuss with the team.

5.1.6 Writing Drivers

Ironics community includes many hardware vendors who contribute drivers that enable more advanced functionality when Ironic is used in conjunction with that hardware. To do this, the Ironic developer community is committed to standardizing on a Python Driver API that meets the common needs of all hardware vendors, and evolving this API without breaking backwards compatibility. However, it is sometimes necessary for driver authors to implement functionality - and expose it through the REST API - that can not be done through any existing API.

To facilitate that, we also provide the means for API calls to be passed through ironic and directly to the driver. Some guidelines on how to implement this are provided below. Driver authors are strongly encouraged to talk with the developer community about any implementation using this functionality.

Pluggable Drivers

Ironic supports a pluggable driver model. This allows contributors to easily add new drivers, and operators to use third-party drivers or write their own. A driver is built at runtime from a *hardware type* and *hardware interfaces*. See *Enabling drivers and hardware types* for a detailed explanation of these concepts.

Hardware types and interfaces are loaded by the `ironic-conductor` service during initialization from the setuptools entrypoints `ironic.hardware.types` and `ironic.hardware.interfaces.<INTERFACE>` where `<INTERFACE>` is an interface type (for example, `deploy`). Only hardware types listed in the configuration option `enabled_hardware_types` and interfaces listed in configuration options `enabled_<INTERFACE>_interfaces` are loaded. A complete list of hardware types available on the system may be found by enumerating this entrypoint by running the following python script:

```
#!/usr/bin/env python

import pkg_resources as pkg
print [p.name for p in pkg.iter_entry_points("ironic.hardware.types") if not␣
↪p.name.startswith("fake")]
```

A list of drivers enabled in a running Ironic service may be found by issuing the following command against that API end point:

```
baremetal driver list
```

Writing a hardware type

A hardware type is a Python class, inheriting *ironic.drivers.hardware_type.AbstractHardwareType* and listed in the setuptools entry point `ironic.hardware.types`. Most of the real world hardware types inherit *ironic.drivers.generic.GenericHardware* instead. This

helper class provides useful implementations for interfaces that are usually the same for all hardware types, such as `deploy`.

The minimum required interfaces are:

- *boot* that specifies how to boot ramdisks and instances on the hardware. A generic `pxe` implementation is provided by the `GenericHardware` base class.
- *deploy* that orchestrates the deployment. A few common implementations are provided by the `GenericHardware` base class.

  As of the Rocky release, a deploy interface should decorate its deploy method to indicate that it is a deploy step. Conventionally, the deploy method uses a priority of 100.

```
@ironic.drivers.base.deploy_step(priority=100)
def deploy(self, task):
```

  Note

  Most of the hardware types should not override this interface.

- `power` implements power actions for the hardware. These common implementations may be used, if supported by the hardware:
  - *ironic.drivers.modules.ipmitool.IPMIPower*
  - *ironic.drivers.modules.redfish.power.RedfishPower*

  Otherwise, you need to write your own implementation by subclassing *ironic.drivers.base.PowerInterface* and providing missing methods.

  Note

  Power actions in Ironic are blocking - methods of a power interface should not return until the power action is finished or errors out.

- `management` implements additional out-of-band management actions, such as setting a boot device. A few common implementations exist and may be used, if supported by the hardware:
  - *ironic.drivers.modules.ipmitool.IPMIManagement*
  - *ironic.drivers.modules.redfish.management.RedfishManagement*

  Some hardware types, such as `snmp` do not support out-of-band management. They use the fake implementation in *ironic.drivers.modules.fake.FakeManagement* instead.

  Otherwise, you need to write your own implementation by subclassing *ironic.drivers.base.ManagementInterface* and providing missing methods.

Combine the interfaces in a hardware type by populating the lists of supported interfaces. These lists are prioritized, with the most preferred implementation first. For example:

```
class MyHardware(generic.GenericHardware):

    @property
```

(continues on next page)

(continued from previous page)

```
    def supported_management_interfaces(self):
        """List of supported management interfaces."""
        return [MyManagement, ipmitool.IPMIManagement]

    @property
    def supported_power_interfaces(self):
        """List of supported power interfaces."""
        return [MyPower, ipmitool.IPMIPower]
```

Note

In this example, all interfaces, except for `management` and `power` are taken from the `GenericHardware` base class.

Finally, give the new hardware type and new interfaces human-friendly names and create entry points for them in the `setup.cfg` file:

```
ironic.hardware.types =
    my-hardware = ironic.drivers.my_hardware:MyHardware
ironic.hardware.interfaces.power =
    my-power = ironic.drivers.modules.my_hardware:MyPower
ironic.hardware.interfaces.management =
    my-management = ironic.drivers.modules.my_hardware:MyManagement
```

Deploy and clean steps

Significant parts of the bare metal functionality is implemented via *deploy steps* or *clean steps*. See *Developing deploy and clean steps* for information on how to write them.

Supported Drivers

For a list of supported drivers (those that are continuously tested on every upstream commit) please consult the *drivers page*.

Vendor Methods

This document is a quick tutorial on writing vendor specific methods to a driver.

The first thing to note is that the Ironic API supports two vendor endpoints: A driver vendor passthru and a node vendor passthru.

- The `VendorInterface` allows hardware types to expose a custom top-level functionality which is not specific to a Node. For example, lets say the driver `ipmi` exposed a method called `authentication_types` that would return what are the authentication types supported. It could be accessed via the Ironic API like:

  ```
  GET http://<address>:<port>/v1/drivers/ipmi/vendor_passthru/
  ↪authentication_types
  ```

> **Warning**
>
> The Bare Metal API currently only allows to use driver passthru for the default `vendor` interface implementation for a given hardware type. This limitation will be lifted in the future.

- The node vendor passthru allows drivers to expose custom functionality on per-node basis. For example the same driver `ipmi` exposing a method called `send_raw` that would send raw bytes to the BMC, the method also receives a parameter called `raw_bytes` which the value would be the bytes to be sent. It could be accessed via the Ironic API like:

```
POST {'raw_bytes': '0x01 0x02'} http://<address>:<port>/v1/nodes/<node␣
↪UUID>/vendor_passthru/send_raw
```

Writing Vendor Methods

Writing a custom vendor method in Ironic should be simple. The first thing to do is write a class inheriting from the VendorInterface class:

```
class ExampleVendor(VendorInterface)

    def get_properties(self):
        return {}

    def validate(self, task, **kwargs):
        pass
```

The `get_properties` is a method that all driver interfaces have, it should return a dictionary of <property>:<description> telling in the description whether that property is required or optional so the node can be manageable by that driver. For example, a required property for a `ipmi` driver would be `ipmi_address` which is the IP address or hostname of the node. We are returning an empty dictionary in our example to make it simpler.

The `validate` method is responsible for validating the parameters passed to the vendor methods. Ironic will not introspect into what is passed to the drivers, its up to the developers writing the vendor method to validate that data.

Lets extend the `ExampleVendor` class to support two methods, the `authentication_types` which will be exposed on the driver vendor passthru endpoint; And the `send_raw` method that will be exposed on the node vendor passthru endpoint:

```
class ExampleVendor(VendorInterface)

    def get_properties(self):
        return {}

    def validate(self, task, method, **kwargs):
        if method == 'send_raw':
            if 'raw_bytes' not in kwargs:
                raise MissingParameterValue()

    @base.driver_passthru(['GET'], async_call=False)
```

(continues on next page)

(continued from previous page)

```
    def authentication_types(self, context, **kwargs):
        return {"types": ["NONE", "MD5", "MD2"]}

    @base.passthru(['POST'])
    def send_raw(self, task, **kwargs):
        raw_bytes = kwargs.get('raw_bytes')
        ...
```

Thats it!

Writing a node or driver vendor passthru method is pretty much the same, the only difference is how you decorate the methods and the first parameter of the method (ignoring self). A method decorated with the `@passthru` decorator should expect a Task object as first parameter and a method decorated with the `@driver_passthru` decorator should expect a Context object as first parameter.

Both decorators accept these parameters:

- http_methods: A list of what the HTTP methods supported by that vendor function. To know what HTTP method that function was invoked with, a `http_method` parameter will be present in the `kwargs`. Supported HTTP methods are *POST*, *PUT*, *GET* and *PATCH*.
- method: By default the method name is the name of the python function, if you want to use a different name this parameter is where this name can be set. For example:

```
@passthru(['PUT'], method="alternative_name")
def name(self, task, **kwargs):
    ...
```

- description: A string containing a nice description about what that method is supposed to do. Defaults to (empty string).
- async_call: A boolean value to determine whether this method should run asynchronously or synchronously. Defaults to True (Asynchronously).

> **Note**
>
> This parameter was previously called async.

The node vendor passthru decorator (`@passthru`) also accepts the following parameter:

- require_exclusive_lock: A boolean value determining whether this method should require an exclusive lock on a node between validate() and the beginning of method execution. For synchronous methods, the lock on the node would also be kept for the duration of method execution. Defaults to True.

> **Warning**
>
> Please avoid having a synchronous method for slow/long-running operations **or** if the method does talk to a BMC; BMCs are flaky and very easy to break.

Warning

Each asynchronous request consumes a worker thread in the `ironic-conductor` process. This can lead to starvation of the thread pool, resulting in a denial of service.

Give the new vendor interface implementation a human-friendly name and create an entry point for it in the `setup.cfg`:

```
ironic.hardware.interfaces.vendor =
    example = ironic.drivers.modules.example:ExampleVendor
```

Finally, add it to the list of supported vendor interfaces for relevant hardware types, for example:

```
class ExampleHardware(generic.GenericHardware):
    ...

    @property
    def supported_vendor_interfaces(self):
        return [example.ExampleVendor]
```

Backwards Compatibility

There is no requirement that changes to a vendor method be backwards compatible. However, for your users sakes, we highly recommend that you do so.

If you are changing the exceptions being raised, you might want to ensure that the same HTTP code is being returned to the user.

For non-backwards compatibility, please make sure you add a release note that indicates this.

Developing BIOS Interface

To support a driver specific BIOS interface it is necessary to create a class inheriting from the `BIOSInterface` class:

```
from ironic.drivers import base

class ExampleBIOS(base.BIOSInterface):

    def get_properties(self):
        return {}

    def validate(self, task):
        pass
```

See *Pluggable Drivers* for a detailed explanation of hardware type and interface.

The `get_properties` and `validate` are methods that all driver interfaces have. The hardware interface that supports BIOS settings should also implement the following three methods:

- Implement a method named `cache_bios_settings`. This method stores BIOS settings to the `bios_settings` table during cleaning operations and updates the `bios_settings` table when `apply_configuration` or `factory_reset` are successfully called.

```
from ironic.drivers import base

driver_client = importutils.try_import('driver.client')

class ExampleBIOS(base.BIOSInterface):
    def __init__(self):
        if driver_client is None:
            raise exception.DriverLoadError(
                driver=self.__class__.__name__,
                reason=_("Unable to import driver library"))

    def cache_bios_settings(self, task):
        node_id = task.node.id
        node_info = driver_common.parse_driver_info(task.node)
        settings = driver_client.get_bios_settings(node_info)
        create_list, update_list, delete_list, nochange_list = (
            objects.BIOSSettingList.sync_node_setting(settings))

        if len(create_list) > 0:
            objects.BIOSSettingList.create(
                task.context, node_id, create_list)
        if len(update_list) > 0:
            objects.BIOSSettingList.save(
                task.context, node_id, update_list)
        if len(delete_list) > 0:
            delete_names = []
            for setting in delete_list:
                delete_names.append(setting.name)
            objects.BIOSSettingList.delete(
                task.context, node_id, delete_names)
```

Note

`driver.client` is vendor specific library to control and manage the bare metal hardware, for example: python-dracclient, sushy.

- Implement a method named `factory_reset`. This method needs to use the `clean_step` decorator. It resets BIOS settings to factory default on the given node. It calls `cache_bios_settings` automatically to update existing `bios_settings` table once successfully executed.

```
class ExampleBIOS(base.BIOSInterface):

    @base.clean_step(priority=0)
    def factory_reset(self, task):
        node_info = driver_common.parse_driver_info(task.node)
        driver_client.reset_bios_settings(node_info)
```

- Implement a method named `apply_configuration`. This method needs to use the clean_step decorator. It takes the given BIOS settings and applies them on the node. It also calls `cache_bios_settings` automatically to update existing `bios_settings` table after successfully

applying given settings on the node.

```
class ExampleBIOS(base.BIOSInterface):

    @base.clean_step(priority=0, argsinfo={
        'settings': {
            'description': (
                'A list of BIOS settings to be applied'
            ),
            'required': True
        }
    })
    def apply_configuration(self, task, settings):
        node_info = driver_common.parse_driver_info(task.node)
        driver_client.apply_bios_settings(node_info, settings)
```

The `settings` parameter is a list of BIOS settings to be configured. for example:

```
[
  {
    "setting name":
      {
        "name": "String",
        "value": "String"
      }
  },
  {
    "setting name":
      {
        "name": "String",
        "value": "String"
      }
  },
  ...
]
```

Third Party Continuous Integration

Note

This document is a work-in-progress. Unfilled sections will be worked in follow-up patchsets. This version is to get a basic outline and index done so that we can then build on it. (krtaylor)

This document provides tips and guidelines for third-party driver developers setting up their continuous integration test systems.

CI Architecture Overview

Requirements Cookbook

Sizing

Infrastructure

This section describes what changes youll need to make to a your CI system to add an ironic job.

jenkins changes

nodepool changes

neutron changes

pre-test hook

cleanup hook

Ironic

Hardware Pool Management

Problem

If you are using actual hardware as target machines for your CI testing then the problem of two jobs trying to use the name target arises. If you have one target machine and a maximum number of one jobs running on your ironic pipeline at a time, then you wont run into this problem. However, one target may not handle the load of ironics daily patch submissions.

Solutions

Zuul v3

Molten Iron

molteniron is a tool that allows you to reserve hardware from a pool at the last minute to use in your job. Once finished testing, you can unreserve the hardware making it available for the next test job.

Tips and Tricks

Optimize Run Time

Image Server

Other References

Developing deploy and clean steps

Deploy steps basics

To support customized deployment step, implement a new method in an interface class and use the decorator `deploy_step` defined in `ironic/drivers/base.py`. For example, we will implement a `do_nothing` deploy step in the `AgentDeploy` class.

```
from ironic.drivers.modules import agent

class AgentDeploy(agent.AgentDeploy):

    @base.deploy_step(priority=200, argsinfo={
        'test_arg': {
            'description': (
                "This is a test argument."
            ),
            'required': True
        }
    })
    def do_nothing(self, task, **kwargs):
        return None
```

If you want to completely replace the deployment procedure, but still have the agent up and running, inherit `CustomAgentDeploy`:

```
from ironic.drivers.modules import agent

class AgentDeploy(agent.CustomAgentDeploy):

    def validate(self, task):
        super().validate(task)
        # ... custom validation

    @base.deploy_step(priority=80)
    def my_write_image(self, task, **kwargs):
        pass  # ... custom image writing

    @base.deploy_step(priority=70)
    def my_configure_bootloader(self, task, **kwargs):
        pass  # ... custom bootloader configuration
```

After deployment of the baremetal node, check the updated deploy steps:

```
baremetal node show $node_ident -f json -c driver_internal_info
```

The above command outputs the `driver_internal_info` as following:

```
{
  "driver_internal_info": {
    ...
    "deploy_steps": [
      {
        "priority": 200,
        "interface": "deploy",
        "step": "do_nothing",
        "argsinfo":
          {
            "test_arg":
```

(continues on next page)

(continued from previous page)

```
                {
                  "required": True,
                  "description": "This is a test argument."
                }
            }
        },
        {
          "priority": 100,
          "interface": "deploy",
          "step": "deploy",
          "argsinfo": null
        }
    ],
    "deploy_step_index": 1
  }
}
```

In-band deploy steps (deploy steps that are run inside the ramdisk) have to be implemented in a custom IPA hardware manager. All in-band deploy steps must have priorities between 41 and 99, see *Agent steps* for details.

Clean steps basics

Clean steps are written similarly to deploy steps, but are executed during *cleaning*. Steps with priority > 0 are executed during automated cleaning, all steps can be executed explicitly during manual cleaning. Unlike deploy steps, clean steps are commonly found in these interfaces:

bios
: Steps that apply BIOS settings, see *Implementing BIOS settings*.

deploy
: Steps that undo the effect of deployment (e.g. erase disks).

management
: Additional steps that use the nodes BMC, such as out-of-band firmware update or BMC reset.

raid
: Steps that build or tear down RAID, see *Implementing RAID*.

Note

When designing a new step for your driver, try to make it consistent with existing steps on other drivers.

Just as deploy steps, in-band clean steps have to be implemented in a custom IPA hardware manager.

Asynchronous steps

If the step returns `None`, ironic assumes its execution is finished and proceeds to the next step. Many steps are executed asynchronously; in this case you need to inform ironic that the step is not finished. There are several possibilities:

Combined in-band and out-of-band step

If your step starts as out-of-band and then proceeds as in-band (i.e. inside the agent), you only need to return `CLEANWAIT/DEPLOYWAIT` from the step.

```
from ironic.drivers import base
from ironic.drivers.modules import agent
from ironic.drivers.modules import agent_base
from ironic.drivers.modules import agent_client
from ironic.drivers.modules import deploy_utils

class MyDeploy(agent.CustomAgentDeploy):
    ...

    @base.deploy_step(priority=80)
    def my_deploy(self, task):
        ...
        return deploy_utils.get_async_step_return_state(task.node)

    # Usually you can use a more high-level pattern:

    @base.deploy_step(priority=60)
    def my_deploy2(self, task):
        new_step = {'interface': 'deploy',
                    'step': 'my_deploy2',
                    'args': {...}}
        client = agent_client.get_client(task)
        return agent_base.execute_step(task, new_step, 'deploy',
                                       client=client)
```

Warning

This approach only works for steps implemented on a `deploy` interface that inherits agent deploy.

Warning

Steps generally should have a return value of None **unless** the a state is returned as part of an asynchronous workflow.

Please be mindful of this constraint when creating steps, as the step runner **will** error if a value aside from None is returned upon step completion.

Execution on reboot

Some steps are executed out-of-band, but require a reboot to complete. Use the following pattern:

```
from ironic.drivers import base
from ironic.drivers.modules import deploy_utils

```

(continues on next page)

(continued from previous page)

```
class MyManagement(base.ManagementInterface):
    ...

    @base.clean_step(priority=0)
    def my_action(self, task):
        ...

        # Tell ironic that...
        deploy_utils.set_async_step_flags(
            node,
            # ... we're waiting for IPA to come back after reboot
            reboot=True,
            # ... the current step is done
            skip_current_step=True)

        return deploy_utils.reboot_to_finish_step(task)
```

Polling for completion

Finally, you may want to poll the BMC until the operation is complete. Often enough, this also involves a reboot. In this case you can use the *ironic.conductor.periodics.node_periodic()* decorator to create a periodic task that operates on relevant nodes:

```
from ironic.common import states
from ironic.common import utils
from ironic.conductor import periodics
from ironic.drivers import base
from ironic.drivers.modules import deploy_utils

_STATUS_CHECK_INTERVAL = ...  # better use a configuration option

class MyManagement(base.ManagementInterface):
    ...

    @base.clean_step(priority=0)
    def my_action(self, task):
        ...

        reboot_required = ...  # your step may or may not need rebooting

        # Make this node as running my_action. Often enough you will store
        # some useful data rather than a boolean flag.
        utils.set_node_nested_field(task.node, 'driver_internal_info',
                                    'in_my_action', True)

        # Tell ironic that...
        deploy_utils.set_async_step_flags(
            node,
            # ... we're waiting for IPA to come back after reboot
```

(continues on next page)

(continued from previous page)

```
            reboot=reboot_required,
            # ... the current step shouldn't be entered again
            skip_current_step=True,
            # ... we'll be polling until the step is done
            polling=True)

        if reboot_required:
            return deploy_utils.reboot_to_finish_step(task)

    @periodics.node_periodic(
        purpose='checking my action status',
        spacing=_STATUS_CHECK_INTERVAL,
        filters={
            # Skip nodes that already have a lock
            'reserved': False,
            # Only consider nodes that are waiting for cleaning or failed
            # on timeout.
            'provision_state_in': [states.CLEANWAIT, states.CLEANFAIL],
        },
        # Load driver_internal_info from the database on listing
        predicate_extra_fields=['driver_internal_info'],
        # Only consider nodes with in_my_action
        predicate=lambda n: n.driver_internal_info.get('in_my_action'),
    )
    def check_my_action(self, task, manager, context):
        if not needs_actions():  # insert your checks here
            return

        task.upgrade_lock()

        ...  # do any required updates

        # Drop the flag so that this node is no longer considered
        utils.pop_node_nested_field(task.node, 'driver_internal_info',
                                    'in_my_action')
```

Note that creating a `task` involves an additional database query, so you want to avoid creating them for too many nodes in your periodic tasks. Instead:

- Try to use precise `filters` to filter out nodes on the database level. Using `reserved` and `provision_state`/`provision_state_in` are recommended in most cases. See *ironic.db.api.Connection.get_nodeinfo_list()* for a list of possible filters.
- Use `predicate` to filter on complex fields such as `driver_internal_info`. Predicates are checked before tasks are created.

Implementing RAID

RAID is implemented via deploy and clean steps in the `raid` interfaces. By convention they have the following signatures:

```
from ironic.drivers import base

class MyRAID(base.RAIDInterface):

    @base.clean_step(priority=0, abortable=False, argsinfo={
        'create_root_volume': {
            'description': (
                'This specifies whether to create the root volume. '
                'Defaults to `True`.'
            ),
            'required': False
        },
        'create_nonroot_volumes': {
            'description': (
                'This specifies whether to create the non-root volumes. '
                'Defaults to `True`.'
            ),
            'required': False
        },
        'delete_existing': {
            'description': (
                'Setting this to `True` indicates to delete existing RAID '
                'configuration prior to creating the new configuration. '
                'Default value is `False`.'
            ),
            'required': False,
        }
    })
    def create_configuration(self, task, create_root_volume=True,
                             create_nonroot_volumes=True,
                             delete_existing=False):
        pass

    @base.clean_step(priority=0)
    @base.deploy_step(priority=0)
    def delete_configuration(self, task):
        pass

    @base.deploy_step(priority=0,
                      argsinfo=base.RAID_APPLY_CONFIGURATION_ARGSINFO)
    def apply_configuration(self, task, raid_config,
                            create_root_volume=True,
                            create_nonroot_volumes=False,
                            delete_existing=False):
        pass
```

Notes:

- `create_configuration` only works as a clean step, during deployment `apply_configuration` is used instead.
- `apply_configuration` accepts the target RAID configuration explicitly, while `create_configuration` uses the nodes `target_raid_config` field.
- Priorities default to 0 since RAID should not be built by default.

Implementing BIOS settings

BIOS is implemented via deploy and clean steps in the `raid` interfaces. By convention they have the following signatures:

```
from ironic.drivers import base

_APPLY_CONFIGURATION_ARGSINFO = {
    'settings': {
        'description': (
            'A list of BIOS settings to be applied'
        ),
        'required': True
    }
}

class MyBIOS(base.BIOSInterface):

    @base.clean_step(priority=0)
    @base.deploy_step(priority=0)
    @base.cache_bios_settings
    def factory_reset(self, task):
        pass

    @base.clean_step(priority=0, argsinfo=_APPLY_CONFIGURATION_ARGSINFO)
    @base.deploy_step(priority=0, argsinfo=_APPLY_CONFIGURATION_ARGSINFO)
    @base.cache_bios_settings
    def apply_configuration(self, task, settings):
        pass
```

Notes:

- Both `factory_reset` and `apply_configuration` can be used as deploy and clean steps.
- The `cache_bios_settings` decorator is used to ensure that the settings cached in the ironic database is updated.
- Priorities default to 0 since BIOS settings should not be modified by default.

5.1.7 Full Ironic Server Python API Reference

ironic

ironic package

Subpackages

ironic.api package

Subpackages

ironic.api.controllers package

Subpackages

ironic.api.controllers.v1 package

Submodules

ironic.api.controllers.v1.allocation module

class ironic.api.controllers.v1.allocation.**AllocationsController**(**args*, ***kwargs*)

Bases: RestController

REST controller for allocations.

delete(*allocation_ident*)

Delete an allocation.

Parameters
• **allocation_ident** The UUID or name of the allocation.

get_all(*node=None*, *resource_class=None*, *state=None*, *marker=None*, *limit=None*, *sort_key='id'*, *sort_dir='asc'*, *fields=None*, *owner=None*)

Retrieve a list of allocations.

Parameters

- **node** Filter the list of allocations by the node UUID or name.
- **resource_class** The requested resource class for the allocation. Can only be missing when backfilling an allocation (will be set to the nodes "resource_class" in such case).
- **state** Filter the list of allocations by the allocation state, one of "active", "allocating" or "error".
- **marker** The ID of the last-seen item. Use the "limit" parameter to make an initial limited request and use the ID of the last-seen item from the response as the "marker" parameter value in a subsequent limited request.
- **limit** Requests a page size of items. Returns a number of items up to a limit value. Use the "limit" parameter to make an initial limited request and use the ID of the last-seen item from the response as the "marker" parameter value in a subsequent limited request. This value cannot be larger than the "max_limit" option in the "[api]" section of the configuration. If it is higher than "max_limit", only "max-limit" resources will be returned.
- **sort_key** Sorts the response by this attribute value. Default is "id". You can specify multiple pairs of sort key and sort direction query parameters. If you omit the sort direction in a pair, the API uses the natural sorting direction of the server attribute that is provided as the "sort_key".
- **sort_dir** Sorts the response by the requested sort direction. A valid value is "asc" (ascending) or "desc" (descending). Default is "asc". You can specify multiple pairs of sort key and sort direction query parameters. If you omit the sort direction in a pair, the API uses the natural sorting direction of the server attribute that is provided as the "sort_key".
- **fields** One or more fields to be returned in the response.
  For example, the following request returns only the "uuid" and "name" fields for each node:
  ::
  GET /v1/nodes?fields=uuid,name
- **owner** Filter the list of returned allocations, and only return those with the specified owner.

get_one(*allocation_ident*, *fields=None*)

Retrieve information about the given allocation.

Parameters

- **allocation_ident** The UUID or name of the allocation.
- **fields** One or more fields to be returned in the response.
  For example, the following request returns only the "uuid" and "name" fields for each node:
  ::
  GET /v1/nodes?fields=uuid,name

invalid_sort_key_list = ['extra', 'candidate_nodes', 'traits']

patch(*allocation_ident*, *patch*)

Update an existing allocation.

Parameters

- **allocation_ident** The UUID or name of the allocation.
- **patch** A JSON patch document to apply to the allocation.

post(*allocation*)

Create a new allocation.

Parameters

- **allocation** The unique name of the Allocation.

class ironic.api.controllers.v1.allocation.**NodeAllocationController**(**args*, ***kwargs*)

Bases: RestController

REST controller for allocations.

delete()

Delete an allocation.

get_all(*fields=None*)

Get all allocations.

Parameters

- **fields** One or more fields to be returned in the response.
  For example, the following request returns only the "uuid" and "name" fields for each node:
  ::
  GET /v1/nodes?fields=uuid,name

invalid_sort_key_list = ['extra', 'candidate_nodes', 'traits']

ironic.api.controllers.v1.allocation.**allocation_sanitize**(*allocation*, *fields*)

ironic.api.controllers.v1.allocation.**convert_with_links**(*rpc_allocation*, *fields=None*, *sanitize=True*)

ironic.api.controllers.v1.allocation.**hide_fields_in_newer_versions**(*allocation*)

ironic.api.controllers.v1.allocation.**list_convert_with_links**(*rpc_allocations*, *limit*, *url*, *fields=None*, ***kwargs*)

ironic.api.controllers.v1.bios module

class ironic.api.controllers.v1.bios.**NodeBiosController**(**args*, ***kwargs*)

Bases: RestController

REST controller for bios.

get_all(*detail=None*, *fields=None*)

List node bios settings.

get_one(*setting_name*)

Retrieve information about the given bios setting.

Parameters

setting_name Logical name of the setting to retrieve.

ironic.api.controllers.v1.bios.**collection_from_list**(*node_ident*, *bios_settings*, *detail=None*, *fields=None*)

ironic.api.controllers.v1.bios.**convert_with_links**(*rpc_bios*, *node_uuid*, *detail=None*, *fields=None*)

Build a dict containing a bios setting value.

ironic.api.controllers.v1.chassis module

class ironic.api.controllers.v1.chassis.**ChassisController**(**args*, ***kwargs*)

Bases: RestController

REST controller for Chassis.

delete(*chassis_uuid*)

Delete a chassis.

Parameters

chassis_uuid UUID of a chassis.

detail(*marker=None*, *limit=None*, *sort_key='id'*, *sort_dir='asc'*)

Retrieve a list of chassis with detail.

Parameters

- **marker** pagination marker for large data sets.
- **limit** maximum number of resources to return in a single result. This value cannot be larger than the value of max_limit in the [api] section of the ironic configuration, or only max_limit resources will be returned.
- **sort_key** column to sort results by. Default: id.
- **sort_dir** direction to sort. asc or desc. Default: asc.

get_all(*marker=None*, *limit=None*, *sort_key='id'*, *sort_dir='asc'*, *fields=None*, *detail=None*)

Retrieve a list of chassis.

Parameters

- **marker** pagination marker for large data sets.
- **limit** maximum number of resources to return in a single result. This value cannot be larger than the value of max_limit in the [api] section of the ironic configuration, or only max_limit resources will be returned.
- **sort_key** column to sort results by. Default: id.
- **sort_dir** direction to sort. asc or desc. Default: asc.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.

get_one(*chassis_uuid*, *fields=None*)

Retrieve information about the given chassis.

Parameters

- **chassis_uuid** UUID of a chassis.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.

invalid_sort_key_list = ['extra']

nodes = <ironic.api.controllers.v1.node.NodesController object>

Expose nodes as a sub-element of chassis

patch(*chassis_uuid*, *patch*)

Update an existing chassis.

Parameters

- **chassis_uuid** UUID of a chassis.
- **patch** a json PATCH document to apply to this chassis.

post(*chassis*)

Create a new chassis.

Parameters

chassis a chassis within the request body.

ironic.api.controllers.v1.chassis.**convert_with_links**(*rpc_chassis*, *fields=None*, *sanitize=True*)

ironic.api.controllers.v1.chassis.**list_convert_with_links**(*rpc_chassis_list*, *limit*, *url*, *fields=None*, ***kwargs*)

ironic.api.controllers.v1.collection module

ironic.api.controllers.v1.collection.**get_next**(*collection*, *limit*, *url*, *key_field='uuid'*, ***kwargs*)

Return a link to the next subset of the collection.

ironic.api.controllers.v1.collection.**has_next**(*collection*, *limit*)

Return whether collection has more items.

ironic.api.controllers.v1.collection.**list_convert_with_links**(*items*, *item_name*, *limit*, *url*, *fields=None*, *sanitize_func=None*, *key_field='uuid'*, *sanitizer_args=None*, ***kwargs*)

Build a collection dict including the next link for paging support.

Parameters

- **items** List of unsanitized items to include in the collection
- **item_name** Name of dict key for items value
- **limit** Paging limit
- **url** Base URL for building next link
- **fields** Optional fields to use for sanitize function
- **sanitize_func** Optional sanitize function run on each item, item changes will be done in-place
- **key_field** Key name for building next URL
- **sanitizer_args** Dictionary with additional arguments to be passed to the sanitizer.
- **kwargs** other arguments passed to `get_next`

Returns

A dict containing `item_name` and `next` values

ironic.api.controllers.v1.conductor module

class ironic.api.controllers.v1.conductor.**ConductorsController**(**args*, ***kwargs*)

Bases: RestController

REST controller for conductors.

get_all(*marker=None*, *limit=None*, *sort_key='id'*, *sort_dir='asc'*, *fields=None*, *detail=None*)

Retrieve a list of conductors.

Parameters

- **marker** pagination marker for large data sets.
- **limit** maximum number of resources to return in a single result. This value cannot be larger than the value of max_limit in the [api] section of the ironic configuration, or only max_limit resources will be returned.
- **sort_key** column to sort results by. Default: id.
- **sort_dir** direction to sort. asc or desc. Default: asc.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.
- **detail** Optional, boolean to indicate whether retrieve a list of conductors with detail.

get_one(*hostname*, *fields=None*)

Retrieve information about the given conductor.

Parameters

- **hostname** hostname of a conductor.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.

invalid_sort_key_list = ['alive', 'drivers']

ironic.api.controllers.v1.conductor.**convert_with_links**(*rpc_conductor*, *fields=None*, *sanitize=True*)

ironic.api.controllers.v1.conductor.**list_convert_with_links**(*rpc_conductors*, *limit*, *url=None*, *fields=None*, ***kwargs*)

ironic.api.controllers.v1.deploy_template module

class ironic.api.controllers.v1.deploy_template.**DeployTemplatesController**(**args*, ***kwargs*)

Bases: RestController

REST controller for deploy templates.

delete(*template_ident*)

Delete a deploy template.

Parameters
template_ident UUID or logical name of a deploy template.

get_all(*marker=None*, *limit=None*, *sort_key='id'*, *sort_dir='asc'*, *fields=None*, *detail=None*)
Retrieve a list of deploy templates.

Parameters

- **marker** pagination marker for large data sets.
- **limit** maximum number of resources to return in a single result. This value cannot be larger than the value of max_limit in the [api] section of the ironic configuration, or only max_limit resources will be returned.
- **sort_key** column to sort results by. Default: id.
- **sort_dir** direction to sort. asc or desc. Default: asc.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.
- **detail** Optional, boolean to indicate whether retrieve a list of deploy templates with detail.

get_one(*template_ident*, *fields=None*)
Retrieve information about the given deploy template.

Parameters

- **template_ident** UUID or logical name of a deploy template.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.

invalid_sort_key_list = ['extra', 'steps']

patch(*template_ident*, *patch=None*)
Update an existing deploy template.

Parameters

- **template_ident** UUID or logical name of a deploy template.
- **patch** a json PATCH document to apply to this deploy template.

post(*template*)
Create a new deploy template.

Parameters
template a deploy template within the request body.

ironic.api.controllers.v1.deploy_template.**convert_with_links**(*rpc_template*, *fields=None*, *sanitize=True*)
Add links to the deploy template.

ironic.api.controllers.v1.deploy_template.**list_convert_with_links**(*rpc_templates*, *limit*, *fields=None*, ***kwargs*)

ironic.api.controllers.v1.deploy_template.**step_sanitize**(*step*)

ironic.api.controllers.v1.deploy_template.**template_sanitize**(*template*, *fields*)

Removes sensitive and unrequested data.

Will only keep the fields specified in the `fields` parameter.

Parameters

fields (*list of str*) list of fields to preserve, or `None` to preserve them all

ironic.api.controllers.v1.driver module

class ironic.api.controllers.v1.driver.**DriverPassthruController**(**args*, ***kwargs*)

Bases: `RestController`

REST controller for driver passthru.

This controller allow vendors to expose cross-node functionality in the Ironic API. Ironic will merely relay the message from here to the specified driver, no introspection will be made in the message body.

methods(*driver_name*)

Retrieve information about vendor methods of the given driver.

Parameters

driver_name name of the driver.

Returns

dictionary with <vendor method name>:<method metadata> entries.

Raises

DriverNotFound if the driver name is invalid or the driver cannot be loaded.

class ironic.api.controllers.v1.driver.**DriverRaidController**(**args*, ***kwargs*)

Bases: `RestController`

logical_disk_properties(*driver_name*)

Returns the logical disk properties for the driver.

Parameters
• **driver_name** Name of the driver.

Returns

Success: A dictionary containing the properties that can be mentioned **Failure:**

- **UnsupportedDriverExtension (HTTP 400 Bad Request)** Driver %(driver)s does not support %(extension)s (disabled or not implemented). If the driver doesnt support RAID configuration.
- **NotAcceptable (HTTP 406 Not Acceptable)** Request not acceptable. If requested version of the API is less than 1.12.
- **DriverNotFound (HTTP 404 Not Found)** Could not find the following driver(s) or hardware type(s): %(driver_name)s. If driver is not loaded on any of the conductors.

class ironic.api.controllers.v1.driver.**DriversController**(**args*, ***kwargs*)

Bases: RestController

REST controller for Drivers.

get_all(*type=None*, *detail=None*, *fields=None*)

Retrieve a list of drivers.

get_one(*driver_name*, *fields=None*)

Retrieve a single driver.

properties(*driver_name*)

Retrieve property information of the given driver.

Parameters

driver_name name of the driver.

Returns

dictionary with <property name>:<property description> entries.

Raises

DriverNotFound **(HTTP 404)** if the driver name is invalid or the driver cannot be loaded.

raid = <ironic.api.controllers.v1.driver.DriverRaidController object>

Expose RAID as a sub-element of drivers

vendor_passthru = <ironic.api.controllers.v1.driver.DriverPassthruController object>

ironic.api.controllers.v1.driver.**convert_with_links**(*name*, *hosts*, *detail=False*, *interface_info=None*, *fields=None*, *sanitize=True*)

Convert driver/hardware type info to a dict.

Parameters

- **name** name of a hardware type.
- **hosts** list of conductor hostnames driver is active on.

- **detail** boolean, whether to include detailed info, such as the type field and default/enabled interfaces fields.
- **interface_info** optional list of dicts of hardware interface info.
- **fields** list of fields to preserve, or None to preserve default
- **sanitize** boolean, sanitize driver

Returns
dict representing the driver object.

ironic.api.controllers.v1.driver.**driver_sanitize**(*driver*, *fields=None*)

ironic.api.controllers.v1.driver.**hide_fields_in_newer_versions**(*driver*)

This method hides fields that were added in newer API versions.

Certain fields were introduced at certain API versions. These fields are only made available when the requests API version matches or exceeds the versions when these fields were introduced.

ironic.api.controllers.v1.driver.**list_convert_with_links**(*hardware_types*, *detail=False*, *fields=None*)

Convert drivers and hardware types to an API-serializable object.

Parameters

- **hardware_types** dict mapping hardware type names to conductor host-names.
- **detail** boolean, whether to include detailed info, such as the type field and default/enabled interfaces fields.
- **fields** list of fields to preserve, or None to preserve default

Returns
an API-serializable driver collection object.

ironic.api.controllers.v1.event module

class ironic.api.controllers.v1.event.**EventsController**(**args*, ***kwargs*)

Bases: RestController

REST controller for Events.

post(*evts*)

ironic.api.controllers.v1.event.**events_valid**(*name*, *value*)

Validator for events

ironic.api.controllers.v1.firmware module

class ironic.api.controllers.v1.firmware.**NodeFirmwareController**(**args*, ***kwargs*)

Bases: RestController

REST controller for Firmware.

get_all(*detail=None*, *fields=None*)

List node firmware components.

ironic.api.controllers.v1.firmware.**collection_from_list**(*node_ident*, *firmware_components*, *detail=None*, *fields=None*)

ironic.api.controllers.v1.firmware.**convert_with_links**(*rpc_firmware*, *node_uuid*, *detail=None*, *fields=None*)

Build a dict containing a firmware component.

ironic.api.controllers.v1.inspection_rule module

class ironic.api.controllers.v1.inspection_rule.**InspectionRuleController**(**args*, ***kwargs*)

Bases: RestController

REST controller for inspection rules.

delete(*inspection_rule_uuid*)

Delete an inspection rule.

Parameters

- **inspection_rule_uuid** UUID of an inspection rule.
- **confirm** Confirmation string. Must be true for bulk deletion.

get_all(*marker=None*, *limit=None*, *sort_key='id'*, *sort_dir='asc'*, *fields=None*, *detail=None*, *phase=None*)

Retrieve a list of inspection rules.

Parameters

- **marker** pagination marker for large data sets.
- **limit** maximum number of resources to return in a single result. This value cannot be larger than the value of max_limit in the [api] section of the ironic configuration, or only max_limit resources will be returned.
- **sort_key** column to sort results by. Default: id.
- **sort_dir** direction to sort. asc or desc. Default: asc.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.
- **detail** Optional, boolean to indicate whether retrieve a list of inspection rules with detail.

get_one(*inspection_rule_uuid*, *fields=None*)

Retrieve information about the given inspection rule.

Parameters

- **inspection_rule_uuid** UUID of an inspection rule.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.

invalid_sort_key_list = ['actions', 'conditions']

patch(*inspection_rule_uuid*, *patch=None*)

Update an existing inspection rule.

Parameters

- **inspection_rule_uuid** UUID of the rule to update.
- **patch** a json PATCH document to apply to this inspection rule.

post(*inspection_rule*)

Create a new inspection rule.

Parameters

inspection_rule a inspection rule within the request body.

ironic.api.controllers.v1.inspection_rule.**convert_actions**(*rpc_actions*)

ironic.api.controllers.v1.inspection_rule.**convert_conditions**(*rpc_conditions*)

ironic.api.controllers.v1.inspection_rule.**convert_with_links**(*rpc_rule*, *fields=None*, *sanitize=True*)

Add links to the inspection rule.

ironic.api.controllers.v1.inspection_rule.**list_convert_with_links**(*rpc_rules*, *limit*, *fields=None*, ***kwargs*)

ironic.api.controllers.v1.inspection_rule.**rules_sanitize**(*inspection_rule*, *fields*)

Removes sensitive and unrequested data.

Will only keep the fields specified in the `fields` parameter.

Parameters

fields (*list of str*) list of fields to preserve, or `None` to preserve them all

ironic.api.controllers.v1.node module

class ironic.api.controllers.v1.node.**BootDeviceController**(**args*, ***kwargs*)

Bases: `RestController`

get(*node_ident*)

Get the current boot device for a node.

Parameters

node_ident the UUID or logical name of a node.

Returns

a json object containing:

boot_device
the boot device, one of *ironic.common.boot_devices* or None if it is unknown.

persistent
Whether the boot device will persist to all future boots or not, None if it is unknown.

put(*node_ident*, *boot_device*, *persistent=False*)

Set the boot device for a node.

Set the boot device to use on next reboot of the node.

Parameters

- **node_ident** the UUID or logical name of a node.
- **boot_device** the boot device, one of *ironic.common.boot_devices*.
- **persistent** Boolean value. True if the boot device will persist to all future boots, False if not. Default: False.

supported(*node_ident*)

Get a list of the supported boot devices.

Parameters

node_ident the UUID or logical name of a node.

Returns

A json object with the list of supported boot devices.

class ironic.api.controllers.v1.node.**GetNodeAndTopicMixin**

Bases: object

class ironic.api.controllers.v1.node.**IndicatorAtComponent**(***kwargs*)

Bases: object

class ironic.api.controllers.v1.node.**IndicatorController**(**args*, ***kwargs*)

Bases: RestController

get_all(*node_ident*, ***kwargs*)

Get node hardware components and their indicators.

Parameters

node_ident the UUID or logical name of a node.

Returns

A json object of hardware components (*ironic.common.components*) as keys with indicator IDs (from *get_supported_indicators*) as values.

get_one(*node_ident*, *indicator*)

Get node hardware component indicator and its state.

Parameters

- **node_ident** the UUID or logical name of a node.
- **indicator** Indicator ID (as reported by *get_supported_indicators*).

Returns

a dict with the state key and one of mod:*ironic.common.indicator_states* as a value.

put(*node_ident*, *indicator*, *state*)

Set node hardware component indicator to the desired state.

Parameters

- **node_ident** the UUID or logical name of a node.

- **indicator** Indicator ID (as reported by *get_supported_indicators*).
- **state** Indicator state, one of mod:*ironic.common.indicator_states*.

class ironic.api.controllers.v1.node.**InjectNmiController**(**args*, ***kwargs*)

Bases: RestController

put(*node_ident*)

Inject NMI for a node.

Inject NMI (Non Maskable Interrupt) for a node immediately.

Parameters
node_ident the UUID or logical name of a node.

Raises
NotFound if requested version of the API doesnt support inject nmi.

Raises
HTTPForbidden if the policy is not authorized.

Raises
NodeNotFound if the node is not found.

Raises
NodeLocked if the node is locked by another conductor.

Raises
UnsupportedDriverExtension if the nodes driver doesnt support management or management.inject_nmi.

Raises
InvalidParameterValue when the wrong driver info is specified or an invalid boot device is specified.

Raises
MissingParameterValue if missing supplied info.

class ironic.api.controllers.v1.node.**NodeChildrenController**(**args*, ***kwargs*)

Bases: RestController

get_all()

class ironic.api.controllers.v1.node.**NodeConsoleController**(**args*, ***kwargs*)

Bases: RestController

get(*node_ident*)

Get connection information about the console.

Parameters
node_ident UUID or logical name of a node.

put(*node_ident*, *enabled*)

Start and stop the node console.

Parameters

- **node_ident** UUID or logical name of a node.
- **enabled** Boolean value; whether to enable or disable the console.

class ironic.api.controllers.v1.node.**NodeHistoryController**(**args*, ***kwargs*)

Bases: RestController

detail_fields = ['uuid', 'created_at', 'severity', 'event_type', 'event', 'conductor', 'user']

get_all(*detail=False*, *marker=None*, *limit=None*, *sort_key='created_at'*, *sort_dir='asc'*)

List node history.

get_one(*event*)

Get a node history entry

standard_fields = ['uuid', 'created_at', 'severity', 'event']

class ironic.api.controllers.v1.node.**NodeInventoryController**(**args*, ***kwargs*)

Bases: RestController

get()

Node inventory of the node.

Parameters

node_ident the UUID of a node.

class ironic.api.controllers.v1.node.**NodeMaintenanceController**(**args*, ***kwargs*)

Bases: RestController

delete(*node_ident*)

Remove the node from maintenance mode.

Parameters

node_ident the UUID or logical name of a node.

put(*node_ident*, *reason=None*)

Put the node in maintenance mode.

Parameters

- **node_ident** the UUID or logical_name of a node.
- **reason** Optional, the reason why its in maintenance.

class ironic.api.controllers.v1.node.**NodeManagementController**(**args*, ***kwargs*)

Bases: RestController

boot_device = <ironic.api.controllers.v1.node.BootDeviceController object>

Expose boot_device as a sub-element of management

indicators = <ironic.api.controllers.v1.node.IndicatorController object>

Expose indicators as a sub-element of management

inject_nmi = <ironic.api.controllers.v1.node.InjectNmiController object>

Expose inject_nmi as a sub-element of management

class ironic.api.controllers.v1.node.**NodeStatesController**(**args*, ***kwargs*)

Bases: RestController

boot_mode(*node_ident*, *target*)

Asynchronous set the boot mode of the node.

Parameters

- **node_ident** the UUID or logical name of a node.
- **target** The desired boot_mode for the node (uefi/bios).

Raises
InvalidParameterValue (HTTP 400) if the requested target state is not valid.

Raises
NotFound (HTTP 404) if requested version of the API is less than 1.76.

Raises
Conflict (HTTP 409) if a node is in adopting state or another transient state.

console = <ironic.api.controllers.v1.node.NodeConsoleController object>

Expose console as a sub-element of states

get(*node_ident*)

List the states of the node.

Parameters
node_ident the UUID or logical_name of a node.

power(*node_ident*, *target*, *timeout=None*)

Set the power state of the node.

Parameters

- **node_ident** the UUID or logical name of a node.
- **target** The desired power state of the node.
- **timeout** timeout (in seconds) positive integer (> 0) for any power state. None indicates to use default timeout.

Raises
ClientSideError (HTTP 409) if a power operation is already in progress.

Raises
InvalidStateRequested (HTTP 400) if the requested target state is not valid or if the node is in CLEANING state.

Raises
NotAcceptable (HTTP 406) for soft reboot, soft power off or timeout parameter, if requested version of the API is less than 1.27.

Raises
Invalid (HTTP 400) if timeout value is less than 1.

provision(*node_ident*, *target*, *configdrive=None*, *clean_steps=None*, *deploy_steps=None*, *rescue_password=None*, *disable_ramdisk=None*, *service_steps=None*, *runbook=None*)

Asynchronous trigger the provisioning of the node.

This will set the target provision state of the node, and a background task will begin which actually applies the state change. This call will return a 202 (Accepted) indicating the request

was accepted and is in progress; the client should continue to GET the status of this node to observe the status of the requested action.

Parameters

- **node_ident** UUID or logical name of a node.
- **target** The desired provision state of the node or verb.
- **configdrive** Optional. A gzipped and base64 encoded configdrive or a dict to build a configdrive from. Only valid when setting provision state to active or rebuild.
- **clean_steps** An ordered list of cleaning steps that will be performed on the node. A cleaning step is a dictionary with required keys interface and step, and optional key args. If specified, the value for args is a keyword variable argument dictionary that is passed to the cleaning step method.:

```
{ 'interface': <driver_interface>,
  'step': <name_of_clean_step>,
  'args': {<arg1>: <value1>, ..., <argn>: <valuen>} }
```

For example (this isnt a real example, this cleaning step doesnt exist):

```
{ 'interface': 'deploy',
  'step': 'upgrade_firmware',
  'args': {'force': True} }
```

This is required (and only valid) when target is clean.

- **deploy_steps** A list of deploy steps that will be performed on the node. A deploy step is a dictionary with required keys interface, step, priority and args. If specified, the value for args is a keyword variable argument dictionary that is passed to the deploy step method.:

```
{ 'interface': <driver_interface>,
  'step': <name_of_deploy_step>,
  'args': {<arg1>: <value1>, ..., <argn>: <valuen>}
  'priority': <integer>}
```

For example (this isnt a real example, this deploy step doesnt exist):

```
{ 'interface': 'deploy',
  'step': 'upgrade_firmware',
  'args': {'force': True},
  'priority': 90 }
```

This is used only when target is active or rebuild and is optional.

- **rescue_password** A string representing the password to be set inside the rescue environment. This is required (and only valid), when target is rescue.
- **disable_ramdisk** Whether to skip booting ramdisk for cleaning.
- **service_steps** A list of service steps that will be performed on the node. A service step is a dictionary with required keys interface, step, priority and

args. If specified, the value for args is a keyword variable argument dictionary that is passed to the service step method.:

```
{ 'interface': <driver_interface>,
  'step': <name_of_service_step>,
  'args': {<arg1>: <value1>, ..., <argn>: <valuen>}
  'priority': <integer>}
```

For example (this isnt a real example, this service step doesnt exist):

```
{ 'interface': 'deploy',
  'step': 'upgrade_firmware',
  'args': {'force': True},
  'priority': 90 }
```

- **runbook** UUID or logical name of a runbook.

Raises
NodeLocked (HTTP 409) if the node is currently locked.

Raises
ClientSideError (HTTP 409) if the node is already being provisioned.

Raises
InvalidParameterValue (HTTP 400), if validation of clean_steps, deploy_steps, service_steps or power driver interface fails.

Raises
InvalidStateRequested (HTTP 400) if the requested transition is not possible from the current state.

Raises
NodeInMaintenance (HTTP 400), if operation cannot be performed because the node is in maintenance mode.

Raises
NoFreeConductorWorker (HTTP 503) if no workers are available.

Raises
NotAcceptable (HTTP 406) if the API version specified does not allow the requested state transition or parameters.

raid(*node_ident*, *target_raid_config*)

Set the target raid config of the node.

Parameters

- **node_ident** the UUID or logical name of a node.
- **target_raid_config** Desired target RAID configuration of the node. It may be an empty dictionary as well.

Raises
UnsupportedDriverExtension, if the nodes driver doesnt support RAID configuration.

Raises
InvalidParameterValue, if validation of target raid config fails.

Raises
NotAcceptable, if requested version of the API is less than 1.12.

secure_boot(*node_ident*, *target*)

Asynchronous set the secure_boot state of the node.

Parameters

- **node_ident** the UUID or logical name of a node.
- **target** Should secure_boot be enabled on node (True/False).

Raises
InvalidParameterValue (HTTP 400) if the requested target state is not valid.

Raises
NotFound (HTTP 404) if requested version of the API is less than 1.76.

Raises
Conflict (HTTP 409) if a node is in adopting state.

class ironic.api.controllers.v1.node.**NodeTraitsController**(**args*, ***kwargs*)

Bases: RestController

delete(*trait=None*)

Remove one or all traits from a node.

Parameters
trait String value; trait to remove from a node, or None. If None, all traits are removed.

get_all()

List node traits.

put(*trait=None*, *body=None*)

Add a trait to a node.

Parameters

- **trait** String value; trait to add to a node, or None. Mutually exclusive with traits. If not None, adds this trait to the node.
- **traits** List of Strings; traits to set for a node, or None. Mutually exclusive with trait. If not None, replaces the nodes traits with this list.

class ironic.api.controllers.v1.node.**NodeVIFController**(**args*, ***kwargs*)

Bases: RestController, *GetNodeAndTopicMixin*

delete(*vif_id*)

Detach a VIF from this node

Parameters
vif_id The ID of a VIF to detach

get_all()

Get a list of attached VIFs

post(*vif*)

Attach a VIF to this node

Parameters
vif a dictionary of information about a VIF. It must have an id key, whose value is a unique identifier for that VIF.

class ironic.api.controllers.v1.node.**NodeVendorPassthruController**(**args*, ***kwargs*)

Bases: RestController

REST controller for VendorPassthru.

This controller allow vendors to expose a custom functionality in the Ironic API. Ironic will merely relay the message from here to the appropriate driver, no introspection will be made in the message body.

methods(*node_ident*)

Retrieve information about vendor methods of the given node.

Parameters
node_ident UUID or logical name of a node.

Returns
dictionary with <vendor method name>:<method metadata> entries.

Raises
NodeNotFound if the node is not found.

class ironic.api.controllers.v1.node.**NodeVmediaController**(**args*, ***kwargs*)

Bases: RestController, *GetNodeAndTopicMixin*

delete(*device_types=None*)

Detach a virtual media from this node

Parameters
device_types A collection of device types.

get()

Get virtual media details for this node

post(*vmedia*)

Attach a virtual media to this node

Parameters
vmedia a dictionary of information about the attachment.

class ironic.api.controllers.v1.node.**NodesController**(**args*, ***kwargs*)

Bases: RestController

REST controller for Nodes.

delete(*node_ident*, **args*)

Delete a node.

Parameters
node_ident UUID or logical name of a node.

detail(*chassis_uuid=None*, *instance_uuid=None*, *associated=None*, *maintenance=None*, *retired=None*, *provision_state=None*, *marker=None*, *limit=None*, *sort_key='id'*, *sort_dir='asc'*, *driver=None*, *resource_class=None*, *fault=None*, *conductor_group=None*, *conductor=None*, *owner=None*, *description_contains=None*, *lessee=None*, *project=None*, *shard=None*, *sharded=None*, *include_children=None*, *parent_node=None*, *instance_name=None*)

Retrieve a list of nodes with detail.

Parameters

- **chassis_uuid** Optional UUID of a chassis, to get only nodes for that chassis.
- **instance_uuid** Optional UUID of an instance, to find the node associated with that instance.
- **associated** Optional boolean whether to return a list of associated or unassociated nodes. May be combined with other parameters.
- **maintenance** Optional boolean value that indicates whether to get nodes in maintenance mode (True), or not in maintenance mode (False).
- **retired** Optional boolean value that indicates whether to get nodes which are retired.
- **provision_state** Optional string value to get only nodes in that provision state.
- **marker** pagination marker for large data sets.
- **limit** maximum number of resources to return in a single result. This value cannot be larger than the value of max_limit in the [api] section of the ironic configuration, or only max_limit resources will be returned.
- **sort_key** column to sort results by. Default: id.
- **sort_dir** direction to sort. asc or desc. Default: asc.
- **driver** Optional string value to get only nodes using that driver.
- **resource_class** Optional string value to get only nodes with that resource_class.
- **fault** Optional string value to get only nodes with that fault.
- **conductor_group** Optional string value to get only nodes with that conductor_group.
- **owner** Optional string value that set the owner whose nodes are to be retrurned.
- **lessee** Optional string value that set the lessee whose nodes are to be returned.
- **project** Optional string value that set the project - lessee or owner - whose nodes are to be returned.
- **shard** Optional - set the shards whose nodes are to be returned.
- **description_contains** Optional string value to get only nodes with description field contains matching value.

- **sharded** Optional boolean whether to return a list of nodes with or without a shard set. May be combined with other parameters.
- **instance_name** Optional string that sets an instance_name to search for

from_chassis = False

A flag to indicate if the requests to this controller are coming from the top-level resource Chassis

get_all(*chassis_uuid=None*, *instance_uuid=None*, *associated=None*, *maintenance=None*, *retired=None*, *provision_state=None*, *marker=None*, *limit=None*, *sort_key='id'*, *sort_dir='asc'*, *driver=None*, *fields=None*, *resource_class=None*, *fault=None*, *conductor_group=None*, *detail=None*, *conductor=None*, *owner=None*, *description_contains=None*, *lessee=None*, *project=None*, *shard=None*, *sharded=None*, *include_children=None*, *parent_node=None*, *instance_name=None*)

Retrieve a list of nodes.

Parameters

- **chassis_uuid** Optional UUID of a chassis, to get only nodes for that chassis.
- **instance_uuid** Optional UUID of an instance, to find the node associated with that instance.
- **associated** Optional boolean whether to return a list of associated or unassociated nodes. May be combined with other parameters.
- **maintenance** Optional boolean value that indicates whether to get nodes in maintenance mode (True), or not in maintenance mode (False).
- **retired** Optional boolean value that indicates whether to get retired nodes.
- **provision_state** Optional string value to get only nodes in that provision state.
- **marker** pagination marker for large data sets.
- **limit** maximum number of resources to return in a single result. This value cannot be larger than the value of max_limit in the [api] section of the ironic configuration, or only max_limit resources will be returned.
- **sort_key** column to sort results by. Default: id.
- **sort_dir** direction to sort. asc or desc. Default: asc.
- **driver** Optional string value to get only nodes using that driver.
- **resource_class** Optional string value to get only nodes with that resource_class.
- **conductor_group** Optional string value to get only nodes with that conductor_group.
- **conductor** Optional string value to get only nodes managed by that conductor.
- **owner** Optional string value that set the owner whose nodes are to be retrurned.

- **lessee** Optional string value that set the lessee whose nodes are to be returned.
- **project** Optional string value that set the project - lessee or owner - whose nodes are to be returned.
- **shard** Optional string value that set the shards whose nodes are to be returned.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.
- **fault** Optional string value to get only nodes with that fault.
- **description_contains** Optional string value to get only nodes with description field contains matching value.
- **sharded** Optional boolean whether to return a list of nodes with or without a shard set. May be combined with other parameters.
- **instance_name** Optional string value to get nodes with a matching instance_name

get_one(*node_ident*, *fields=None*)

Retrieve information about the given node.

Parameters

- **node_ident** UUID or logical name of a node.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.

invalid_sort_key_list = ['properties', 'driver_info', 'extra', 'instance_info', 'driver_internal_info', 'clean_step', 'deploy_step', 'raid_config', 'target_raid_config', 'traits', 'network_data', 'service_step']

maintenance = <ironic.api.controllers.v1.node.NodeMaintenanceController object>

Expose maintenance as a sub-element of nodes

management = <ironic.api.controllers.v1.node.NodeManagementController object>

Expose management as a sub-element of nodes

parent_node = None

An indicator to signal if this resource is being accessed by a sub-controller.

patch(*node_ident*, *reset_interfaces=None*, *patch=None*)

Update an existing node.

Parameters

- **node_ident** UUID or logical name of a node.
- **reset_interfaces** whether to reset hardware interfaces to their defaults. Only valid when updating the driver field.
- **patch** a json PATCH document to apply to this node.

post(*node*)

Create a new node.

Parameters

node a node within the request body.

Example Node creation request:

```
{
    "name": "test_node_dynamic",
    "driver": "ipmi",
    "driver_info": {
        "ipmi_username": "ADMIN",
        "ipmi_password": "password"
    },
    "power_interface": "ipmitool",
    "resource_class": "bm-large"
}
```

states = <ironic.api.controllers.v1.node.NodeStatesController object>

Expose the state controller action as a sub-element of nodes

validate(*node=None*, *node_uuid=None*)

Validate the driver interfaces, using the nodes UUID or name.

Note that the node_uuid interface is deprecated in favour of the node interface

Parameters

- **node** UUID or name of a node.
- **node_uuid** UUID of a node.

vendor_passthru = <ironic.api.controllers.v1.node.NodeVendorPassthruController object>

A resource used for vendors to expose a custom functionality in the API

ironic.api.controllers.v1.node.**get_nodes_controller_reserved_names**()

ironic.api.controllers.v1.node.**hide_fields_in_newer_versions**(*obj*)

This method hides fields that were added in newer API versions.

Certain node fields were introduced at certain API versions. These fields are only made available when the requests API version matches or exceeds the versions when these fields were introduced.

ironic.api.controllers.v1.node.**indicator_convert_with_links**(*node_uuid*, *rpc_component*, *rpc_name*, ***rpc_fields*)

Add links to the indicator.

ironic.api.controllers.v1.node.**indicator_list_from_dict**(*node_ident*, *indicators*)

ironic.api.controllers.v1.node.**network_data_schema**()

ironic.api.controllers.v1.node.**node_convert_with_links**(*rpc_node*, *fields=None*, *sanitize=True*)

ironic.api.controllers.v1.node.**node_list_convert_with_links**(*nodes*, *limit*, *url*, *fields=None*, ***kwargs*)

ironic.api.controllers.v1.node.**node_patch_schema**()

ironic.api.controllers.v1.node.**node_patch_validator**(*name*, *value*)

ironic.api.controllers.v1.node.**node_sanitize**(*node*, *fields*, *cdict=None*, *show_driver_secrets=None*, *show_instance_secrets=None*, *evaluate_additional_policies=None*)

Removes sensitive and unrequested data.

Will only keep the fields specified in the `fields` parameter.

Parameters

- **fields** (*list of str*) list of fields to preserve, or `None` to preserve them all
- **cdict** Context dictionary for policy values evaluation. If not provided, it will be executed by the method, however for enumerating node lists, it is more efficient to provide.
- **show_driver_secrets** A boolean value to allow external single evaluation of policy instead of once per node. Default None.
- **show_instance_secrets** A boolean value to allow external evaluation of policy instead of once per node. Default None.
- **evaluate_additional_policies** A boolean value to allow external evaluation of policy instead of once per node. Default None.

ironic.api.controllers.v1.node.**node_schema**()

ironic.api.controllers.v1.node.**node_states_convert**(*rpc_node*)

ironic.api.controllers.v1.node.**node_validator**(*name*, *value*)

ironic.api.controllers.v1.node.**reject_fields_in_newer_versions**(*obj*)

When creating an object, reject fields that appear in newer versions.

ironic.api.controllers.v1.node.**reject_patch_in_newer_versions**(*patch*)

ironic.api.controllers.v1.node.**update_state_in_older_versions**(*obj*)

Change provision state names for API backwards compatibility.

Parameters

obj The dict being returned to the API client that is to be updated by this method.

ironic.api.controllers.v1.node.**validate_network_data**(*network_data*)

Validates node network_data field.

This method validates network data configuration against JSON schema.

Parameters
network_data a network_data field to validate

Raises
Invalid if network data is not schema-compliant

ironic.api.controllers.v1.notification_utils module

ironic.api.controllers.v1.notification_utils.**emit_end_notification**(*context*, *obj*, *action*, ***kwargs*)

Helper for emitting API end notifications.

Parameters

- **context** request context.
- **obj** resource rpc object.
- **action** Action string to go in the EventType.
- **kwargs** kwargs to use when creating the notification payload.

ironic.api.controllers.v1.notification_utils.**emit_start_notification**(*context*, *obj*, *action*, ***kwargs*)

Helper for emitting API start notifications.

Parameters

- **context** request context.
- **obj** resource rpc object.
- **action** Action string to go in the EventType.
- **kwargs** kwargs to use when creating the notification payload.

ironic.api.controllers.v1.notification_utils.**handle_error_notification**(*context*, *obj*, *action*, ***kwargs*)

Context manager to handle any error notifications.

Parameters

- **context** request context.
- **obj** resource rpc object.
- **action** Action string to go in the EventType.
- **kwargs** kwargs to use when creating the notification payload.

ironic.api.controllers.v1.port module

class ironic.api.controllers.v1.port.**PortsController**(**args*, ***kwargs*)

Bases: RestController

REST controller for Ports.

`advanced_net_fields = ['pxe_enabled', 'local_link_connection']`

`delete`(*port_uuid*)

Delete a port.

Parameters
`port_uuid` UUID of a port.

Raises
MalformedRequestURI, HTTPNotFound

`detail`(*node=None*, *node_uuid=None*, *address=None*, *marker=None*, *limit=None*, *sort_key='id'*, *sort_dir='asc'*, *portgroup=None*, *shard=None*, *description_contains=None*, *conductor_groups=None*)

Retrieve a list of ports with detail.

Note that the node_uuid interface is deprecated in favour of the node interface

Parameters

- `node` UUID or name of a node, to get only ports for that node.
- `node_uuid` UUID of a node, to get only ports for that node.
- `address` MAC address of a port, to get the port which has this MAC address.
- `portgroup` UUID or name of a portgroup, to get only ports for that portgroup.
- `shard` comma separated list of shards, to only get ports associated with nodes in those shards.
- `marker` pagination marker for large data sets.
- `limit` maximum number of resources to return in a single result. This value cannot be larger than the value of max_limit in the [api] section of the ironic configuration, or only max_limit resources will be returned.
- `sort_key` column to sort results by. Default: id.
- `sort_dir` direction to sort. asc or desc. Default: asc.
- `description_contains` Optional string value to get only ports with description field contains matching value.
- `conductor_groups` conductor groups, to filter the request by.

Raises
NotAcceptable, HTTPNotFound

`get_all`(*node=None*, *node_uuid=None*, *address=None*, *marker=None*, *limit=None*, *sort_key='id'*, *sort_dir='asc'*, *fields=None*, *portgroup=None*, *detail=None*, *shard=None*, *description_contains=None*, *conductor_groups=None*)

Retrieve a list of ports.

Note that the node_uuid interface is deprecated in favour of the node interface

Parameters

- `node` UUID or name of a node, to get only ports for that node.
- `node_uuid` UUID of a node, to get only ports for that node.

- **address** MAC address of a port, to get the port which has this MAC address.
- **marker** pagination marker for large data sets.
- **limit** maximum number of resources to return in a single result. This value cannot be larger than the value of max_limit in the [api] section of the ironic configuration, or only max_limit resources will be returned.
- **sort_key** column to sort results by. Default: id.
- **sort_dir** direction to sort. asc or desc. Default: asc.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.
- **portgroup** UUID or name of a portgroup, to get only ports for that portgroup.
- **shard** Optional, a list of shard ids to filter by, only ports associated with nodes in these shards will be returned.
- **description_contains** Optional string value to get only ports with description field contains matching value.
- **conductor_groups** conductor groups, to filter the request by.

Raises
NotAcceptable, HTTPNotFound

get_one(*port_ident*, *fields=None*)

Retrieve information about the given port.

Parameters

- **port_ident** UUID or name of a port.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.

Raises
NotAcceptable, HTTPNotFound

invalid_sort_key_list = ['extra', 'internal_info', 'local_link_connection']

patch(*port_ident*, *patch*)

Update an existing port.

Parameters

- **port_ident** UUID or name of a port.
- **patch** a json PATCH document to apply to this port.

Raises
NotAcceptable, HTTPNotFound

post(*port*)

Create a new port.

Parameters
port a port within the request body.

Raises
NotAcceptable, HTTPNotFound, Conflict

ironic.api.controllers.v1.port.**convert_with_links**(*rpc_port*, *fields=None*, *sanitize=True*)

ironic.api.controllers.v1.port.**hide_fields_in_newer_versions**(*port*)

ironic.api.controllers.v1.port.**list_convert_with_links**(*rpc_ports*, *limit*, *url*, *fields=None*, ***kwargs*)

ironic.api.controllers.v1.port.**port_sanitize**(*port*, *fields=None*)

Removes sensitive and unrequested data.

Will only keep the fields specified in the `fields` parameter.

Parameters
fields (*list of str*) list of fields to preserve, or `None` to preserve them all

ironic.api.controllers.v1.portgroup module

class ironic.api.controllers.v1.portgroup.**PortgroupsController**(**args*, ***kwargs*)

Bases: `RestController`

REST controller for portgroups.

delete(*portgroup_ident*)

Delete a portgroup.

Parameters
portgroup_ident UUID or logical name of a portgroup.

detail(*node=None*, *address=None*, *marker=None*, *limit=None*, *sort_key='id'*, *sort_dir='asc'*, *shard=None*, *conductor_groups=None*)

Retrieve a list of portgroups with detail.

Parameters

- **node** UUID or name of a node, to get only portgroups for that node.
- **address** MAC address of a portgroup, to get the portgroup which has this MAC address.
- **marker** pagination marker for large data sets.
- **limit** maximum number of resources to return in a single result. This value cannot be larger than the value of max_limit in the [api] section of the ironic configuration, or only max_limit resources will be returned.
- **sort_key** column to sort results by. Default: id.
- **sort_dir** direction to sort. asc or desc. Default: asc.
- **shard** Optional, a list of shard ids to filter by, only portgroups associated with nodes in these shards will be returned.
- **conductor_groups** conductor groups, to filter the request by.

get_all(*node=None*, *address=None*, *marker=None*, *limit=None*, *sort_key='id'*, *sort_dir='asc'*, *fields=None*, *detail=None*, *shard=None*, *conductor_groups=None*)

Retrieve a list of portgroups.

Parameters

- **node** UUID or name of a node, to get only portgroups for that node.
- **address** MAC address of a portgroup, to get the portgroup which has this MAC address.
- **marker** pagination marker for large data sets.
- **limit** maximum number of resources to return in a single result. This value cannot be larger than the value of max_limit in the [api] section of the ironic configuration, or only max_limit resources will be returned.
- **sort_key** column to sort results by. Default: id.
- **sort_dir** direction to sort. asc or desc. Default: asc.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.
- **shard** Optional, a list of shard ids to filter by, only portgroups associated with nodes in these shards will be returned.
- **conductor_groups** conductor groups, to filter the request by.

get_one(*portgroup_ident*, *fields=None*)

Retrieve information about the given portgroup.

Parameters

- **portgroup_ident** UUID or logical name of a portgroup.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.

invalid_sort_key_list = ['extra', 'internal_info', 'properties']

patch(*portgroup_ident*, *patch*)

Update an existing portgroup.

Parameters

- **portgroup_ident** UUID or logical name of a portgroup.
- **patch** a json PATCH document to apply to this portgroup.

post(*portgroup*)

Create a new portgroup.

Parameters

portgroup a portgroup within the request body.

ironic.api.controllers.v1.portgroup.**convert_with_links**(*rpc_portgroup*, *fields=None*, *sanitize=True*)

Add links to the portgroup.

ironic.api.controllers.v1.portgroup.**hide_fields_in_newer_versions**(*portgroup*)

ironic.api.controllers.v1.portgroup.**list_convert_with_links**(*rpc_portgroups*, *limit*, *url*, *fields=None*, ***kwargs*)

ironic.api.controllers.v1.portgroup.**portgroup_sanitize**(*portgroup*, *fields=None*)

Removes sensitive and unrequested data.

Will only keep the fields specified in the `fields` parameter.

Parameters

fields (*list of str*) list of fields to preserve, or `None` to preserve them all

ironic.api.controllers.v1.ramdisk module

class ironic.api.controllers.v1.ramdisk.**ContinueInspectionController**(**args*, ***kwargs*)

Bases: `RestController`

Controller handling inspection data from deploy ramdisk.

post(*data*, *node_uuid=None*)

Process a introspection data from the deploy ramdisk.

Parameters

- **data** Introspection data.
- **node_uuid** UUID of a node.

Raises

InvalidParameterValue if node_uuid is a valid UUID.

Raises

NoValidHost if RPC topic for node could not be retrieved.

Raises

NotFound if requested API version does not allow this endpoint or if lookup fails.

class ironic.api.controllers.v1.ramdisk.**HeartbeatController**(**args*, ***kwargs*)

Bases: `RestController`

Controller handling heartbeats from deploy ramdisk.

post(*node_ident*, *callback_url*, *agent_version=None*, *agent_token=None*, *agent_verify_ca=None*, *agent_status=None*, *agent_status_message=None*)

Process a heartbeat from the deploy ramdisk.

Parameters

- **node_ident** the UUID or logical name of a node.
- **callback_url** the URL to reach back to the ramdisk.
- **agent_version** The version of the agent that is heartbeating. `None` indicates that the agent that is heartbeating is a version before sending

agent_version was introduced so agent v3.0.0 (the last release before sending agent_version was introduced) will be assumed.

- **agent_token** randomly generated validation token.
- **agent_verify_ca** TLS certificate to use to connect to the agent.
- **agent_status** Current status of the heartbeating agent. Used by anaconda ramdisk to send status back to Ironic. The valid states are start, end, error
- **agent_status_message** Optional status message describing current agent_status

Raises
NodeNotFound if node with provided UUID or name was not found.

Raises
InvalidUuidOrName if node_ident is not valid name or UUID.

Raises
NoValidHost if RPC topic for node could not be retrieved.

Raises
NotFound if requested API version does not allow this endpoint.

class ironic.api.controllers.v1.ramdisk.**LookupController**(**args*, ***kwargs*)

Bases: RestController

Controller handling node lookup for a deploy ramdisk.

get_all(*addresses=None*, *node_uuid=None*)

Look up a node by its MAC addresses and optionally UUID.

If the restrict_lookup option is set to True (the default), limit the search to nodes in certain transient states (e.g. deploy wait).

Parameters

- **addresses** list of MAC addresses for a node.
- **node_uuid** UUID of a node.

Raises
NotFound if requested API version does not allow this endpoint.

Raises
NotFound if suitable node was not found or nodes provision state is not allowed for the lookup.

Raises
IncompleteLookup if neither node UUID nor any valid MAC address was provided.

lookup_allowed(*node*)

ironic.api.controllers.v1.ramdisk.**config**(*token*, *node=None*)

ironic.api.controllers.v1.ramdisk.**convert_with_links**(*node*)

ironic.api.controllers.v1.ramdisk.**get_valid_mac_addresses**(*addresses*, *node_uuid=None*)

ironic.api.controllers.v1.runbook module

class ironic.api.controllers.v1.runbook.**RunbooksController**(**args*, ***kwargs*)

Bases: RestController

REST controller for runbooks.

delete(*runbook_ident*)

Delete a runbook.

Parameters

runbook_ident UUID or logical name of a runbook.

get_all(*marker=None*, *limit=None*, *sort_key='id'*, *sort_dir='asc'*, *fields=None*, *detail=None*, *project=None*)

Retrieve a list of runbooks.

Parameters

- **marker** pagination marker for large data sets.
- **limit** maximum number of resources to return in a single result. This value cannot be larger than the value of max_limit in the [api] section of the ironic configuration, or only max_limit resources will be returned.
- **project** Optional string value that set the project whose runbooks are to be returned.
- **sort_key** column to sort results by. Default: id.
- **sort_dir** direction to sort. asc or desc. Default: asc.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.
- **detail** Optional, boolean to indicate whether retrieve a list of runbooks with detail.

get_one(*runbook_ident*, *fields=None*)

Retrieve information about the given runbook.

Parameters

- **runbook_ident** UUID or logical name of a runbook.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.

invalid_sort_key_list = ['extra', 'steps']

patch(*runbook_ident*, *patch=None*)

Update an existing runbook.

Parameters

- **runbook_ident** UUID or logical name of a runbook.
- **patch** a json PATCH document to apply to this runbook.

post(*runbook*)

Create a new runbook.

Parameters

runbook a runbook within the request body.

ironic.api.controllers.v1.runbook.**convert_with_links**(*rpc_runbook*, *fields=None*, *sanitize=True*)

Add links to the runbook.

ironic.api.controllers.v1.runbook.**list_convert_with_links**(*rpc_runbooks*, *limit*, *fields=None*, ***kwargs*)

ironic.api.controllers.v1.runbook.**runbook_sanitize**(*runbook*, *fields*)

Removes sensitive and unrequested data.

Will only keep the fields specified in the `fields` parameter.

Parameters

fields (*list of str*) list of fields to preserve, or `None` to preserve them all

ironic.api.controllers.v1.runbook.**step_sanitize**(*step*)

ironic.api.controllers.v1.shard module

class ironic.api.controllers.v1.shard.**ShardController**(**args*, ***kwargs*)

Bases: `RestController`

REST controller for shards.

get_all()

Retrieve a list of shards.

Returns

A list of shards.

get_one(*_*)

Explicitly do not support getting one.

ironic.api.controllers.v1.utils module

class ironic.api.controllers.v1.utils.**PassthruResponse**(*obj*, *status_code=None*)

Bases: `object`

Object to hold the response from a passthru call

obj

Store the result object from the view

status_code

Store an optional status_code

ironic.api.controllers.v1.utils.**allow_agent_token**()

Check if agent token is available.

ironic.api.controllers.v1.utils.**allow_agent_version_in_heartbeat**()

Check if agent version is allowed to be passed into heartbeat.

Version 1.36 of the API added the ability for agents to pass their version information to Ironic on heartbeat.

ironic.api.controllers.v1.utils.**allow_allocation_backfill**()

Check if backfilling allocations is allowed.

Version 1.58 of the API added support for backfilling allocations.

ironic.api.controllers.v1.utils.**allow_allocation_owner**()

Check if allocation owner field is allowed.

Version 1.60 of the API added the owner field to the allocation object.

ironic.api.controllers.v1.utils.**allow_allocations**()

Check if accessing allocation endpoints is allowed.

Version 1.52 of the API exposed allocation endpoints and allocation_uuid field for the node.

ironic.api.controllers.v1.utils.**allow_attach_detach_vmedia**()

Check if we should support virtual media actions.

ironic.api.controllers.v1.utils.**allow_bios_interface**()

Check if we should support bios interface and endpoints.

Version 1.40 of the API added support for bios interface.

ironic.api.controllers.v1.utils.**allow_build_configdrive**()

Check if building configdrive is allowed.

Version 1.56 of the API added support for building configdrive.

ironic.api.controllers.v1.utils.**allow_configdrive_vendor_data**()

Check if configdrive can contain a vendor_data key.

Version 1.59 of the API added support for configdrive vendor_data.

ironic.api.controllers.v1.utils.**allow_continue_inspection_endpoint**()

Check if /v1/continue_inspection endpoint is available.

As a special exception, we allow it in the base version so that the API can be used as a drop-in replacement for the Inspectors API.

ironic.api.controllers.v1.utils.**allow_deploy_abort**()

Check if deployment abort is allowed.

Version 1.110 of the API added support for deployment abort in DEPLOYWAIT

ironic.api.controllers.v1.utils.**allow_deploy_steps**()

Check if deploy_steps are available.

ironic.api.controllers.v1.utils.**allow_deploy_templates**()

Check if accessing deploy template endpoints is allowed.

Version 1.55 of the API exposed deploy template endpoints.

ironic.api.controllers.v1.utils.**allow_detail_query**()

Check if passing a detail=True query string is allowed.

Version 1.43 allows a user to pass the detail query string to list the resource with all the fields.

ironic.api.controllers.v1.utils.**allow_dynamic_drivers**()

Check if dynamic driver API calls are allowed.

Version 1.30 of the API added support for all of the driver composition related calls in the /v1/drivers API.

ironic.api.controllers.v1.utils.**allow_dynamic_interfaces**()

Check if dynamic interface fields are allowed.

Version 1.31 of the API added support for viewing and setting the fields in `V31_FIELDS` on the node object.

ironic.api.controllers.v1.utils.**allow_expose_conductors**()

Check if accessing conductor endpoints is allowed.

Version 1.49 of the API exposed conductor endpoints and conductor field for the node.

ironic.api.controllers.v1.utils.**allow_expose_events**()

Check if accessing events endpoint is allowed.

Version 1.54 of the API added the events endpoint.

ironic.api.controllers.v1.utils.**allow_field**(*field*)

Check if a field is allowed in the current version.

ironic.api.controllers.v1.utils.**allow_firmware_interface**()

Check if we should support firmware interface and endpoints.

Version 1.86 of the API added support for firmware interface.

ironic.api.controllers.v1.utils.**allow_get_vmedia**()

Check if we should support get virtual media action.

ironic.api.controllers.v1.utils.**allow_inject_nmi**()

Check if Inject NMI is allowed for the node.

Version 1.29 of the API allows Inject NMI for the node.

ironic.api.controllers.v1.utils.**allow_inspect_abort**()

Check if inspection abort is allowed.

Version 1.41 of the API added support for inspection abort

ironic.api.controllers.v1.utils.**allow_inspect_wait_state**()

Check if inspect wait is allowed for the node.

Version 1.39 of the API adds inspect wait state to substitute inspecting state during asynchronous hardware inspection.

ironic.api.controllers.v1.utils.**allow_inspection_rules**()

Check if accessing inspection rule endpoints is allowed.

Version 1.96 of the API exposed rule endpoints.

ironic.api.controllers.v1.utils.**allow_links_node_states_and_driver_properties**()
Check if links are displayable.

Version 1.14 of the API allows the display of links to node states and driver properties.

ironic.api.controllers.v1.utils.**allow_local_link_connection_network_type**()
Check if network_type is allowed in ports link_local_connection

ironic.api.controllers.v1.utils.**allow_node_history**()
Check if node history access is permitted by API version.

ironic.api.controllers.v1.utils.**allow_node_ident_as_param_for_port_creation**()
Check if node_ident parameter is allowed for port creation.

ironic.api.controllers.v1.utils.**allow_node_instance_name**()
Check if instance_name is allowed for nodes.

Version 1.104 of the API added instance_name field to the node object.

ironic.api.controllers.v1.utils.**allow_node_inventory**()
Check if node inventory is allowed.

ironic.api.controllers.v1.utils.**allow_node_logical_names**()

ironic.api.controllers.v1.utils.**allow_node_rebuild_with_configdrive**()
Check if we should support node rebuild with configdrive.

Version 1.35 of the API added support for node rebuild with configdrive.

ironic.api.controllers.v1.utils.**allow_ovn_vtep_version**()
Check if ovn vtep version is allowed.

Version 1.90 of the API added support for ovn vtep switches in port.local_link_connection.

Version 1.105 removed VTEP support logic.

ironic.api.controllers.v1.utils.**allow_port_advanced_net_fields**()
Check if we should return local_link_connection and pxe_enabled fields.

Version 1.19 of the API added support for these new fields in port object.

ironic.api.controllers.v1.utils.**allow_port_available_for_dynamic_portgroup**()
Check if available_for_dynamic_portgroup is allowed for ports.

Version 1.111 of the API added available_for_dynamic_portgroup field to the port object.

ironic.api.controllers.v1.utils.**allow_port_category**()
Check if category is allowed for ports.

Version 1.101 of the API added category field to the port object.

ironic.api.controllers.v1.utils.**allow_port_description**()
Check if description is allowed for ports.

Version 1.97 of the API added description field to the port object.

ironic.api.controllers.v1.utils.**allow_port_internal_info**()
Check if accessing internal_info is allowed for the port.

Version 1.18 of the API exposes internal_info readonly field for the port.

ironic.api.controllers.v1.utils.**allow_port_is_smartnic**()

Check if port is_smartnic field is allowed.

Version 1.53 of the API added is_smartnic field to the port object.

ironic.api.controllers.v1.utils.**allow_port_name**()

Check if name is allowed for ports.

Version 1.88 of the API added name field to the port object.

ironic.api.controllers.v1.utils.**allow_port_physical_network**()

Check if port physical network field is allowed.

Version 1.34 of the API added the physical network field to the port object. We also check whether the target version of the Port object supports the physical_network field as this may not be the case during a rolling upgrade.

ironic.api.controllers.v1.utils.**allow_port_vendor**()

Check if vendor is allowed for ports.

Version 1.100 of the API added vendor field to the port object.

ironic.api.controllers.v1.utils.**allow_portgroup_category**()

Check if category can be added to/queried from a portgroup.

Version 1.103 of the API added category field to portgroup object.

ironic.api.controllers.v1.utils.**allow_portgroup_dynamic_portgroup**()

Check if dynamic_portgroup is allowed for portgroups.

Version 1.111 of the API added dynamic_portgroup field to the portgroup object.

ironic.api.controllers.v1.utils.**allow_portgroup_mode_properties**()

Check if mode and properties can be added to/queried from a portgroup.

Version 1.26 of the API added mode and properties fields to portgroup object.

ironic.api.controllers.v1.utils.**allow_portgroup_physical_network**()

Check if physical_network can be added to/queried from a portgroup.

Version 1.102 of the API added physical_network field to portgroup object.

ironic.api.controllers.v1.utils.**allow_portgroups**()

Check if we should support portgroup operations.

Version 1.23 of the API added support for PortGroups.

ironic.api.controllers.v1.utils.**allow_portgroups_subcontrollers**()

Check if portgroups can be used as subcontrollers.

Version 1.24 of the API added support for Portgroups as subcontrollers

ironic.api.controllers.v1.utils.**allow_query_bios**()

Check if BIOS queries should be allowed based on version

ironic.api.controllers.v1.utils.**allow_raid_config**()

Check if RAID configuration is allowed for the node.

Version 1.12 of the API allows RAID configuration for the node.

ironic.api.controllers.v1.utils.**allow_ramdisk_endpoints**()

Check if heartbeat and lookup endpoints are allowed.

Version 1.22 of the API introduced them.

ironic.api.controllers.v1.utils.**allow_remove_chassis_uuid**()

Check if chassis_uuid can be removed from node.

Version 1.25 of the API added support for chassis_uuid removal

ironic.api.controllers.v1.utils.**allow_rescue_interface**()

Check if we should support rescue and unrescue operations and interface.

Version 1.38 of the API added support for rescue and unrescue.

ironic.api.controllers.v1.utils.**allow_reset_interfaces**()

Check if passing a reset_interfaces query string is allowed.

ironic.api.controllers.v1.utils.**allow_runbooks**()

Check if accessing runbook endpoints is allowed.

Version 1.92 of the API exposed runbook endpoints.

ironic.api.controllers.v1.utils.**allow_service_verb**()

Check if the service verb may be passed to the API.

ironic.api.controllers.v1.utils.**allow_shards_endpoint**()

Check if shards endpoint is available.

ironic.api.controllers.v1.utils.**allow_soft_power_off**()

Check if Soft Power Off is allowed for the node.

Version 1.27 of the API allows Soft Power Off, including Soft Reboot, for the node.

ironic.api.controllers.v1.utils.**allow_special_characters_in_patch_fields**()

Check if special characters are allowed in patch field keys.

ironic.api.controllers.v1.utils.**allow_status_in_heartbeat**()

Check if heartbeat accepts agent_status and agent_status_message.

ironic.api.controllers.v1.utils.**allow_storage_interface**()

Check if we should support storage_interface node and driver fields.

Version 1.33 of the API added support for storage interfaces.

ironic.api.controllers.v1.utils.**allow_traits**()

Check if traits are allowed for the node.

Version 1.37 of the API allows traits for the node.

ironic.api.controllers.v1.utils.**allow_unhold_verb**()

Check if the unhold verb may be passed to the API

ironic.api.controllers.v1.utils.**allow_verify_ca_in_heartbeat**()

Check if heartbeat accepts agent_verify_ca.

ironic.api.controllers.v1.utils.**allow_vifs_subcontroller**()

Check if node/vifs can be used.

Version 1.28 of the API added support for VIFs to be attached to Nodes.

ironic.api.controllers.v1.utils.**allow_volume**()

Check if volume connectors and targets are allowed.

Version 1.32 of the API added support for volume connectors and targets

ironic.api.controllers.v1.utils.**apply_jsonpatch**(*doc*, *patch*)

Apply a JSON patch, one operation at a time.

If the patch fails to apply, this allows us to determine which operation failed, making the error message a little less cryptic.

Parameters

- **doc** The JSON document to patch.
- **patch** The JSON patch to apply.

Returns
The result of the patch operation.

Raises
PatchError if the patch fails to apply.

Raises
exception.ClientSideError if the patch adds a new root attribute.

ironic.api.controllers.v1.utils.**check_allocation_policy_and_retrieve**(*policy_name*, *allocation_ident*)

Check if the specified policy authorizes request on allocation.

Param
policy_name: Name of the policy to check.

Param
allocation_ident: the UUID or logical name of a node.

Raises
HTTPForbidden if the policy forbids access.

Raises
AllocationNotFound if the node is not found.

Returns
RPC node identified by node_ident

ironic.api.controllers.v1.utils.**check_allow_boot_mode**(*node_capabilities*, *disallowed_boot_modes*)

Check if boot mode is allowed

ironic.api.controllers.v1.utils.**check_allow_child_node_params**(*include_children=None*, *parent_node=None*)

ironic.api.controllers.v1.utils.**check_allow_clean_disable_ramdisk**(*target*, *disable_ramdisk*)

ironic.api.controllers.v1.utils.**check_allow_configdrive**(*target*, *configdrive=None*)

ironic.api.controllers.v1.utils.**check_allow_deploy_steps**(*target*, *deploy_steps*)

Check if deploy steps are allowed

ironic.api.controllers.v1.utils.**check_allow_disable_ramdisk**(*target*, *disable_ramdisk*)

ironic.api.controllers.v1.utils.**check_allow_driver_detail**(*detail*)

Check if getting detailed driver info is allowed.

Version 1.30 of the API allows this.

ironic.api.controllers.v1.utils.**check_allow_filter_by_conductor**(*conductor*)

Check if filtering nodes by conductor is allowed.

Version 1.49 of the API allows filtering nodes by conductor.

ironic.api.controllers.v1.utils.**check_allow_filter_by_conductor_group**(*conductor_group*)

Check if filtering nodes by conductor_group is allowed.

Version 1.46 of the API allows filtering nodes by conductor_group.

ironic.api.controllers.v1.utils.**check_allow_filter_by_fault**(*fault*)

Check if filtering nodes by fault is allowed.

Version 1.42 of the API allows filtering nodes by fault.

ironic.api.controllers.v1.utils.**check_allow_filter_by_instance_name**(*instance_name*)

ironic.api.controllers.v1.utils.**check_allow_filter_by_lessee**(*lessee*)

Check if filtering nodes by lessee is allowed.

Version 1.62 of the API allows filtering nodes by lessee.

ironic.api.controllers.v1.utils.**check_allow_filter_by_owner**(*owner*)

Check if filtering nodes by owner is allowed.

Version 1.50 of the API allows filtering nodes by owner.

ironic.api.controllers.v1.utils.**check_allow_filter_by_shard**(*shard*)

Check if filtering nodes by shard is allowed.

Version 1.82 of the API allows filtering nodes by shard.

ironic.api.controllers.v1.utils.**check_allow_filter_driver_type**(*driver_type*)

Check if filtering drivers by classic/dynamic is allowed.

Version 1.30 of the API allows this.

ironic.api.controllers.v1.utils.**check_allow_management_verbs**(*verb*)

ironic.api.controllers.v1.utils.**check_allow_portgroup_filter_by_shard**(*shard*)

Check if filtering portgroups by shard is allowed.

Version 1.106 of the API allows filtering portgroups by shard.

ironic.api.controllers.v1.utils.**check_allow_specify_driver**(*driver*)

Check if filtering nodes by driver is allowed.

Version 1.16 of the API allows filter nodes by driver.

ironic.api.controllers.v1.utils.**check_allow_specify_fields**(*fields*)

Check if fetching a subset of the resource attributes is allowed.

Version 1.8 of the API allows fetching a subset of the resource attributes, this method checks if the required version is being requested.

ironic.api.controllers.v1.utils.**check_allow_specify_resource_class**(*resource_class*)

Check if filtering nodes by resource_class is allowed.

Version 1.21 of the API allows filtering nodes by resource_class.

ironic.api.controllers.v1.utils.**check_allowed_fields**(*fields*)

Check if fetching a particular field is allowed.

This method checks if the required version is being requested for fields that are only allowed to be fetched in a particular API version.

ironic.api.controllers.v1.utils.**check_allowed_portgroup_fields**(*fields*)

Check if fetching a particular field of a portgroup is allowed.

This method checks if the required version is being requested for fields that are only allowed to be fetched in a particular API version.

ironic.api.controllers.v1.utils.**check_and_retrieve_public_runbook**(*runbook_ident*)

If policy authorization check fails, check if runbook is public.

Param
runbook_ident: the UUID or logical name of a runbook.

Raises
HTTPForbidden if runbook is not public.

Returns
RPC runbook identified by runbook_ident

ironic.api.controllers.v1.utils.**check_for_invalid_fields**(*fields*, *object_fields*)

Check for requested non-existent fields.

Check if the user requested non-existent fields.

Parameters
fields A list of fields requested by the user

Object_fields
A list of fields supported by the object.

Raises
InvalidParameterValue if invalid fields were requested.

ironic.api.controllers.v1.utils.**check_for_invalid_state_and_allow_filter**(*provision_state*)

Check if filtering nodes by provision state is allowed.

Version 1.9 of the API allows filter nodes by provision state.

ironic.api.controllers.v1.utils.**check_list_policy**(*object_type*, *owner=None*)

Check if the list policy authorizes this request on an object.

Param
object_type: type of object being checked

Param
owner: owner filter for list query, if any

Raises
HTTPForbidden if the policy forbids access.

Returns
owner that should be used for list query, if needed

ironic.api.controllers.v1.utils.**check_multiple_node_policies_and_retrieve**(*policy_names*, *node_ident*, *with_suffix=False*)

Check if the specified policies authorize this request on a node.

Param
policy_names: List of policy names to check.

Param
node_ident: the UUID or logical name of a node.

Param
with_suffix: whether the RPC node should include the suffix

Raises
HTTPForbidden if the policy forbids access.

Raises
NodeNotFound if the node is not found.

Returns
RPC node identified by node_ident

ironic.api.controllers.v1.utils.**check_multiple_runbook_policies_and_retrieve**(*policy_names*, *runbook_ident*)

Check if the specified policies authorize this request on a runbook.

Param
policy_names: List of policy names to check.

Param
runbook_ident: the UUID or logical name of a runbook.

Raises
HTTPForbidden if the policy forbids access.

Raises
RunbookNotFound if the runbook is not found.

Returns
RPC runbook identified by runbook_ident

ironic.api.controllers.v1.utils.**check_node_policy_and_retrieve**(*policy_name*, *node_ident*, *with_suffix=False*)

Check if the specified policy authorizes this request on a node.

Param
policy_name: Name of the policy to check.

Param
node_ident: the UUID or logical name of a node.

Param
with_suffix: whether the RPC node should include the suffix

Raises
HTTPForbidden if the policy forbids access.

Raises
NodeNotFound if the node is not found.

Returns
RPC node identified by node_ident

ironic.api.controllers.v1.utils.**check_owner_policy**(*object_type*, *policy_name*, *owner*, *lessee=None*, *conceal_node=False*)

Check if the policy authorizes this request on an object.

Param
object_type: type of object being checked

Param
policy_name: Name of the policy to check.

Param
owner: the owner

Param
lessee: the lessee

Param
conceal_node: the UUID of the node IF we should conceal the existence of the node with a 404 Error instead of a 403 Error.

Raises
HTTPForbidden if the policy forbids access.

ironic.api.controllers.v1.utils.**check_policy**(*policy_name*)

Check if the specified policy is authorised for this request.

Policy_name
Name of the policy to check.

Raises
HTTPForbidden if the policy forbids access.

ironic.api.controllers.v1.utils.**check_policy_true**(*policy_name*)

Check if the specified policy is authorised for this request.

Policy_name
Name of the policy to check.

Returns
True if policy is matched, otherwise false.

ironic.api.controllers.v1.utils.**check_port_list_policy**(*portgroup=False*, *parent_node=None*, *parent_portgroup=None*)

Check if the specified policy authorizes this request on a port.

Parameters

- **portgroup** Boolean value, default false, indicating if the list policy check is for a portgroup as the policy names are different between ports and portgroups.
- **parent_node** The UUID of a node, if any, to apply a policy check to as well before applying other policy check operations.
- **parent_portgroup** The UUID of the parent portgroup if the list of ports was retrieved via the /v1/portgroups/<uuid>/ports.

Raises
HTTPForbidden if the policy forbids access.

Returns
owner that should be used for list query, if needed

ironic.api.controllers.v1.utils.**check_port_policy_and_retrieve**(*policy_name*, *port_ident*, *portgroup=False*)

Check if the specified policy authorizes this request on a port.

Param
policy_name: Name of the policy to check.

Param
port_ident: The name, uuid, or other valid ID value to find a port or portgroup by.

Raises
HTTPForbidden if the policy forbids access.

Raises
PortNotFound if the port is not found.

Raises
PortgroupNotFound if the portgroup is not found.

Returns
RPC port identified by port_ident associated node

ironic.api.controllers.v1.utils.**check_runbook_policy_and_retrieve**(*policy_name*, *runbook_ident*)

Check if the specified policy authorizes this request on a node.

Param
policy_name: Name of the policy to check.

Param
runbook_ident: the UUID or logical name of a runbook.

Raises
HTTPForbidden if the policy forbids access.

Raises
RunbookNotFound if the runbook is not found.

Returns
a runbook object

ironic.api.controllers.v1.utils.**check_volume_list_policy**(*parent_node=None*)

Check if the specified policy authorizes this request on a volume.

Parameters
parent_node The UUID of a node, if any, to apply a policy check to as well before applying other policy check operations.

Raises
HTTPForbidden if the policy forbids access.

Returns
owner that should be used for list query, if needed

ironic.api.controllers.v1.utils.**check_volume_policy_and_retrieve**(*policy_name*, *vol_ident*, *target=False*)

Check if the specified policy authorizes this request on a volume.

Param
policy_name: Name of the policy to check.

Param
vol_ident: The name, uuid, or other valid ID value to find a volume target or connector by.

Param
target: Boolean value to indicate if the check is for a volume target or connector. Default value is False, implying connector.

Raises
HTTPForbidden if the policy forbids access.

Raises
VolumeConnectorNotFound if the node is not found.

Raises
VolumeTargetNotFound if the node is not found.

Returns
RPC port identified by port_ident associated node

ironic.api.controllers.v1.utils.**convert_steps**(*rpc_steps*)

ironic.api.controllers.v1.utils.**decode_json_pointer**(*path*)

Decode JSON pointer as per RFC 6901 to support ~ and /.

ironic.api.controllers.v1.utils.**disallowed_fields**()

Generator of fields not allowed in the current request.

ironic.api.controllers.v1.utils.**duplicate_steps**(*name*, *value*)

Argument validator to check template for duplicate steps

ironic.api.controllers.v1.utils.**get_controller_reserved_names**(*cls*)

Get reserved names for a given controller.

Inspect the controller class and return the reserved names within it. Reserved names are names that can not be used as an identifier for a resource because the names are either being used as a custom action or is the name of a nested controller inside the given class.

Parameters

cls The controller class to be inspected.

ironic.api.controllers.v1.utils.**get_patch_values**(*patch*, *path*)

Get the patch values corresponding to the specified path.

If there are multiple values specified for the same path, for example

```
[{'op': 'add', 'path': '/name', 'value': 'abc'},
 {'op': 'add', 'path': '/name', 'value': 'bca'}]
```

return all of them in a list (preserving order)

Parameters

- **patch** HTTP PATCH request body.
- **path** the path to get the patch values for.

Returns

list of values for the specified path in the patch.

ironic.api.controllers.v1.utils.**get_request_return_fields**(*fields*, *detail*, *default_fields*, *check_detail_version=<function allow_detail_query>*, *check_fields_version=None*)

Calculate fields to return from an API request

The fields query and detail=True query can not be passed into a request at the same time. To use the detail query we need to be on a version of the API greater than expected, likewise some APIs require a certain version for the fields query. This function raises an InvalidParameterValue exception if any of these conditions are not met.

If these checks pass then this function will return either the fields passed in or the default fields provided.

Parameters

- **fields** The fields query passed into the API request.
- **detail** The detail query passed into the API request.
- **default_fields** The default fields to return if fields=None and detail=None.
- **check_detail_version** Function to check if detail query is allowed based on the version.
- **check_fields_version** Function to check if fields query is allowed based on the version.

Raises
InvalidParameterValue if there is an invalid combination of query strings or API version.

Returns
fields passed in value or default_fields

ironic.api.controllers.v1.utils.**get_rpc_allocation**(*allocation_ident*)

Get the RPC allocation from the allocation UUID or logical name.

Parameters
allocation_ident the UUID or logical name of an allocation.

Returns
The RPC allocation.

Raises
InvalidUuidOrName if the name or uuid provided is not valid.

Raises
AllocationNotFound if the allocation is not found.

ironic.api.controllers.v1.utils.**get_rpc_allocation_with_suffix**(*allocation_ident*)

Get the RPC allocation from the allocation UUID or logical name.

If HAS_JSON_SUFFIX flag is set in the pecan environment, try also looking for allocation_ident with .json suffix. Otherwise identical to get_rpc_allocation.

Parameters
allocation_ident the UUID or logical name of an allocation.

Returns
The RPC allocation.

Raises
InvalidUuidOrName if the name or uuid provided is not valid.

Raises
AllocationNotFound if the allocation is not found.

ironic.api.controllers.v1.utils.**get_rpc_deploy_template**(*template_ident*)

Get the RPC deploy template from the UUID or logical name.

Parameters
template_ident the UUID or logical name of a deploy template.

Returns
The RPC deploy template.

Raises
InvalidUuidOrName if the name or uuid provided is not valid.

Raises
DeployTemplateNotFound if the deploy template is not found.

ironic.api.controllers.v1.utils.**get_rpc_deploy_template_with_suffix**(*template_ident*)

Get the RPC deploy template from the UUID or logical name.

If HAS_JSON_SUFFIX flag is set in the pecan environment, try also looking for template_ident with .json suffix. Otherwise identical to get_rpc_deploy_template.

Parameters
template_ident the UUID or logical name of a deploy template.

Returns
The RPC deploy template.

Raises
InvalidUuidOrName if the name or uuid provided is not valid.

Raises
DeployTemplateNotFound if the deploy template is not found.

ironic.api.controllers.v1.utils.**get_rpc_node**(*node_ident*)

Get the RPC node from the node uuid or logical name.

Parameters
node_ident the UUID or logical name of a node.

Returns
The RPC Node.

Raises
InvalidUuidOrName if the name or uuid provided is not valid.

Raises
NodeNotFound if the node is not found.

ironic.api.controllers.v1.utils.**get_rpc_node_with_suffix**(*node_ident*)

Get the RPC node from the node uuid or logical name.

If HAS_JSON_SUFFIX flag is set in the pecan environment, try also looking for node_ident with .json suffix. Otherwise identical to get_rpc_node.

Parameters
node_ident the UUID or logical name of a node.

Returns
The RPC Node.

Raises
InvalidUuidOrName if the name or uuid provided is not valid.

Raises
NodeNotFound if the node is not found.

ironic.api.controllers.v1.utils.**get_rpc_portgroup**(*portgroup_ident*)

Get the RPC portgroup from the portgroup UUID or logical name.

Parameters
portgroup_ident the UUID or logical name of a portgroup.

Returns
The RPC portgroup.

Raises
InvalidUuidOrName if the name or uuid provided is not valid.

Raises
PortgroupNotFound if the portgroup is not found.

ironic.api.controllers.v1.utils.**get_rpc_portgroup_with_suffix**(*portgroup_ident*)

Get the RPC portgroup from the portgroup UUID or logical name.

If HAS_JSON_SUFFIX flag is set in the pecan environment, try also looking for portgroup_ident with .json suffix. Otherwise identical to get_rpc_portgroup.

Parameters
portgroup_ident the UUID or logical name of a portgroup.

Returns
The RPC portgroup.

Raises
InvalidUuidOrName if the name or uuid provided is not valid.

Raises
PortgroupNotFound if the portgroup is not found.

ironic.api.controllers.v1.utils.**get_rpc_runbook**(*runbook_ident*)

Get the RPC runbook from the UUID or logical name.

Parameters
runbook_ident the UUID or logical name of a runbook.

Returns
The RPC runbook.

Raises
InvalidUuidOrName if the name or uuid provided is not valid.

Raises
RunbookNotFound if the runbook is not found.

ironic.api.controllers.v1.utils.**initial_node_provision_state**()

Return node state to use by default when creating new nodes.

Previously the default state for new nodes was AVAILABLE. Starting with API 1.11 it is ENROLL.

ironic.api.controllers.v1.utils.**is_path_removed**(*patch*, *path*)

Returns whether the patch includes removal of the path (or subpath of).

Parameters

- **patch** HTTP PATCH request body.
- **path** the path to check.

Returns
True if path or subpath being removed, False otherwise.

ironic.api.controllers.v1.utils.**is_path_updated**(*patch*, *path*)

Returns whether the patch includes operation on path (or its subpath).

Parameters

- **patch** HTTP PATCH request body.
- **path** the path to check.

Returns
True if path or subpath being patched, False otherwise.

ironic.api.controllers.v1.utils.**is_valid_logical_name**(*name*)

Determine if the provided name is a valid hostname.

ironic.api.controllers.v1.utils.**is_valid_node_name**(*name*)

Determine if the provided name is a valid node name.

Check to see that the provided node name is valid, and isnt a UUID.

Parameters

name the node name to check.

Returns

True if the name is valid, False otherwise.

ironic.api.controllers.v1.utils.**local_link_normalize**(*name*, *value*)

ironic.api.controllers.v1.utils.**new_continue_inspection_endpoint**()

Check if /v1/continue_inspection endpoint is explicitly requested.

ironic.api.controllers.v1.utils.**object_to_dict**(*obj*, *include_created_at=True*, *include_updated_at=True*, *include_uuid=True*, *link_resource=None*, *link_resource_args=None*, *fields=None*)

Helper function to convert RPC objects to REST API dicts.

Parameters

- **obj** RPC object to convert to a dict
- **include_created_at** Whether to include standard base class attribute created_at
- **include_updated_at** Whether to include standard base class attribute updated_at
- **include_uuid** Whether to include standard base class attribute uuid
- **link_resource** When specified, generate a `links` value with a `self` and `bookmark` using this resource name
- **link_resource_args** Resource arguments to be added to generated links. When not specified, the object `uuid` will be used.
- **fields** Key names for dict values to populate directly from object attributes

Returns

A dict containing values from the object

ironic.api.controllers.v1.utils.**patch_update_changed_fields**(*from_dict*, *rpc_object*, *fields*, *schema*, *id_map=None*)

Update rpc object based on changed fields in a dict.

Only fields which have a corresponding schema field are updated when changed. Other values can be updated using the id_map.

Parameters

- **from_dict** Dict containing changed field values

- **rpc_object** Object to update changed fields on
- **fields** Field names on the rpc object
- **schema** jsonschema to get field names of the dict
- **id_map** Optional dict mapping object field names to arbitrary values when there is no matching field in the schema

ironic.api.controllers.v1.utils.**patch_validate_allowed_fields**(*patch*, *allowed_fields*)

Validate that a patch list only modifies allowed fields.

Parameters

- **patch** List of patch dicts to validate
- **allowed_fields** List of fields which are allowed to be patched

Returns

The list of fields which will be patched

Raises

exception.Invalid if any patch changes a field not in `allowed_fields`

ironic.api.controllers.v1.utils.**patched_validate_with_schema**(*patched_dict*, *schema*, *validator=None*)

Validate a patched dict object against a validator or schema.

This function has the side-effect of deleting any dict value which is not in the schema. This allows database-loaded objects to be pruned of their internal values before validation.

Parameters

- **patched_dict** dict representation of the object with patch updates applied
- **schema** Any dict key not in the schema will be deleted from the dict. If no validator is specified then the resulting `patched_dict` will be validated against the schema
- **validator** Optional validator to use if there is extra validation required beyond the schema

Raises

exception.Invalid if validation fails

ironic.api.controllers.v1.utils.**populate_node_uuid**(*obj*, *to_dict*)

Look up the node referenced in the object and populate a dict.

The node is fetched with the object `node_id` attribute and the dict `node_uuid` value is populated with the node uuid

Parameters

- **obj** object to get the node_id attribute
- **to_dict** dict to populate with a `node_uuid` value

Raises

exception.NodeNotFound if the node is not found

ironic.api.controllers.v1.utils.**replace_node_id_with_uuid**(*to_dict*)

Replace node_id dict value with node_uuid

node_uuid is found by fetching the node by id lookup.

Parameters
to_dict Dict to set node_uuid value on

Returns
The node object from the lookup

Raises
NodeNotFound with status_code set to 400 BAD_REQUEST when node is not found.

ironic.api.controllers.v1.utils.**replace_node_uuid_with_id**(*to_dict*)

Replace node_uuid dict value with node_id

node_id is found by fetching the node by uuid lookup.

Parameters
to_dict Dict to set node_id value on

Returns
The node object from the lookup

Raises
NodeNotFound with status_code set to 400 BAD_REQUEST when node is not found.

ironic.api.controllers.v1.utils.**sanitize_dict**(*to_sanitize*, *fields*)

Removes sensitive and unrequested data.

Will only keep the fields specified in the fields parameter (plus the links field).

Parameters

- **to_sanitize** dict to sanitize
- **fields** (*list of str*) list of fields to preserve, or None to preserve them all

ironic.api.controllers.v1.utils.**validate_limit**(*limit*)

ironic.api.controllers.v1.utils.**validate_sort_dir**(*sort_dir*)

ironic.api.controllers.v1.utils.**vendor_passthru**(*ident*, *method*, *topic*, *data=None*, *driver_passthru=False*)

Call a vendor passthru API extension.

Call the vendor passthru API extension and process the method response to set the right return code for methods that are asynchronous or synchronous; Attach the return value to the response object if its being served statically.

Parameters

- **ident** The resource identification. For nodes vendor passthru this is the nodes UUID, for drivers vendor passthru this is the drivers name.
- **method** The vendor method name.

- **topic** The RPC topic.
- **data** The data passed to the vendor method. Defaults to None.
- **driver_passthru** Boolean value. Whether this is a node or driver vendor passthru. Defaults to False.

Returns
A WSME response object to be returned by the API.

ironic.api.controllers.v1.versions module

ironic.api.controllers.v1.versions.**max_version_string**()

Returns the maximum supported API version (as a string).

If the service is pinned, the maximum API version is the pinned version. Otherwise, it is the maximum supported API version.

ironic.api.controllers.v1.versions.**min_version_string**()

Returns the minimum supported API version (as a string)

ironic.api.controllers.v1.volume module

class ironic.api.controllers.v1.volume.**VolumeController**(**args*, ***kwargs*)

Bases: RestController

REST controller for volume root

get()

ironic.api.controllers.v1.volume.**convert**(*node_ident=None*)

ironic.api.controllers.v1.volume_connector module

class ironic.api.controllers.v1.volume_connector.**VolumeConnectorsController**(**args*, ***kwargs*)

Bases: RestController

REST controller for VolumeConnectors.

delete(*connector_uuid*)

Delete a volume connector.

Parameters
connector_uuid UUID of a volume connector.

Raises
MalformedRequestURI if accessed with specifying a parent node.

Raises
NodeLocked if node is locked by another conductor

Raises
NodeNotFound if the node associated with the connector does not exist

Raises
VolumeConnectorNotFound if the volume connector cannot be found

Raises
InvalidStateRequested If a node associated with the volume connector is not powered off.

`get_all`(*node=None*, *marker=None*, *limit=None*, *sort_key='id'*, *sort_dir='asc'*, *fields=None*, *detail=None*, *project=None*)

Retrieve a list of volume connectors.

Parameters

- **node** UUID or name of a node, to get only volume connectors for that node.
- **marker** pagination marker for large data sets.
- **limit** maximum number of resources to return in a single result. This value cannot be larger than the value of max_limit in the [api] section of the ironic configuration, or only max_limit resources will be returned.
- **sort_key** column to sort results by. Default: id.
- **sort_dir** direction to sort. asc or desc. Default: asc.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.
- **detail** Optional, whether to retrieve with detail.

Returns
a list of volume connectors, or an empty list if no volume connector is found.

Raises
InvalidParameterValue if sort_key does not exist

Raises
InvalidParameterValue if sort key is invalid for sorting.

Raises
InvalidParameterValue if both fields and detail are specified.

`get_one`(*connector_uuid*, *fields=None*)

Retrieve information about the given volume connector.

Parameters

- **connector_uuid** UUID of a volume connector.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.

Returns
API-serializable volume connector object.

Raises
MalformedRequestURI if accessed with specifying a parent node.

Raises
VolumeConnectorNotFound if no volume connector exists with the specified UUID.

`invalid_sort_key_list = ['extra']`

patch(*connector_uuid*, *patch*)

Update an existing volume connector.

Parameters

- **connector_uuid** UUID of a volume connector.
- **patch** a json PATCH document to apply to this volume connector.

Returns
API-serializable volume connector object.

Raises
MalformedRequestURI if accessed with specifying a parent node.

Raises
PatchError if a given patch can not be applied.

Raises
VolumeConnectorNotFound if no volume connector exists with the specified UUID.

Raises
InvalidParameterValue if the volume connectors UUID is being changed

Raises
NodeLocked if node is locked by another conductor

Raises
NodeNotFound if the node associated with the connector does not exist

Raises
VolumeConnectorTypeAndIdAlreadyExists if another connector already exists with the same values for type and connector_id fields

Raises
InvalidUUID if invalid node UUID is passed in the patch.

Raises
InvalidStateRequested If a node associated with the volume connector is not powered off.

post(*connector*)

Create a new volume connector.

Parameters
connector a volume connector within the request body.

Returns
API-serializable volume connector object.

Raises
MalformedRequestURI if accessed with specifying a parent node.

Raises
VolumeConnectorTypeAndIdAlreadyExists if a volume connector already exists with the same type and connector_id

Raises
VolumeConnectorAlreadyExists if a volume connector with the same UUID already exists

ironic.api.controllers.v1.volume_connector.**convert_with_links**(*rpc_connector*, *fields=None*, *sanitize=True*)

ironic.api.controllers.v1.volume_connector.**list_convert_with_links**(*rpc_connectors*, *limit*, *url*, *fields=None*, *detail=None*, ***kwargs*)

ironic.api.controllers.v1.volume_target module

class ironic.api.controllers.v1.volume_target.**VolumeTargetsController**(**args*, ***kwargs*)

Bases: RestController

REST controller for VolumeTargets.

delete(*target_uuid*)

Delete a volume target.

Parameters
target_uuid UUID of a volume target.

Raises
MalformedRequestURI if accessed with specifying a parent node.

Raises
NodeLocked if node is locked by another conductor

Raises
NodeNotFound if the node associated with the target does not exist

Raises
VolumeTargetNotFound if the volume target cannot be found

Raises
InvalidStateRequested If a node associated with the volume target is not powered off.

get_all(*node=None*, *marker=None*, *limit=None*, *sort_key='id'*, *sort_dir='asc'*, *fields=None*, *detail=None*, *project=None*)

Retrieve a list of volume targets.

Parameters

- **node** UUID or name of a node, to get only volume targets for that node.
- **marker** pagination marker for large data sets.
- **limit** maximum number of resources to return in a single result. This value cannot be larger than the value of max_limit in the [api] section of the ironic configuration, or only max_limit resources will be returned.

- **sort_key** column to sort results by. Default: id.
- **sort_dir** direction to sort. asc or desc. Default: asc.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.
- **detail** Optional, whether to retrieve with detail.
- **project** Optional, an associated node project (owner, or lessee) to filter the query upon.

Returns
a list of volume targets, or an empty list if no volume target is found.

Raises
InvalidParameterValue if sort_key does not exist

Raises
InvalidParameterValue if sort key is invalid for sorting.

Raises
InvalidParameterValue if both fields and detail are specified.

get_one(*target_uuid*, *fields=None*)

Retrieve information about the given volume target.

Parameters

- **target_uuid** UUID of a volume target.
- **fields** Optional, a list with a specified set of fields of the resource to be returned.

Returns
API-serializable volume target object.

Raises
MalformedRequestURI if accessed with specifying a parent node.

Raises
VolumeTargetNotFound if no volume target with this UUID exists

invalid_sort_key_list = ['extra', 'properties']

patch(*target_uuid*, *patch*)

Update an existing volume target.

Parameters

- **target_uuid** UUID of a volume target.
- **patch** a json PATCH document to apply to this volume target.

Returns
API-serializable volume target object.

Raises
MalformedRequestURI if accessed with specifying a parent node.

Raises
PatchError if a given patch can not be applied.

Raises
InvalidParameterValue if the volume targets UUID is being changed

Raises
NodeLocked if the node is already locked

Raises
NodeNotFound if the node associated with the volume target does not exist

Raises
VolumeTargetNotFound if the volume target cannot be found

Raises
VolumeTargetBootIndexAlreadyExists if a volume target already exists with the same node ID and boot index values

Raises
InvalidUUID if invalid node UUID is passed in the patch.

Raises
InvalidStateRequested If a node associated with the volume target is not powered off.

post(*target*)

Create a new volume target.

Parameters
target a volume target within the request body.

Returns
API-serializable volume target object.

Raises
MalformedRequestURI if accessed with specifying a parent node.

Raises
VolumeTargetBootIndexAlreadyExists if a volume target already exists with the same node ID and boot index

Raises
VolumeTargetAlreadyExists if a volume target with the same UUID exists

ironic.api.controllers.v1.volume_target.**convert_with_links**(*rpc_target*, *fields=None*, *sanitize=True*)

ironic.api.controllers.v1.volume_target.**list_convert_with_links**(*rpc_targets*, *limit*, *url*, *fields=None*, *detail=None*, ***kwargs*)

Module contents

Version 1 of the Ironic API

Specification can be found at doc/source/webapi/v1.rst

class ironic.api.controllers.v1.**Controller**

Bases: object

Version 1 API controller root.

add_version_attributes()

index()

Submodules

ironic.api.controllers.base module

class ironic.api.controllers.base.**Version**(*headers*, *default_version*, *latest_version*)

Bases: object

API Version object.

max_string = 'X-OpenStack-Ironic-API-Maximum-Version'

HTTP response header

min_string = 'X-OpenStack-Ironic-API-Minimum-Version'

HTTP response header

static parse_headers(*headers*, *default_version*, *latest_version*)

Determine the API version requested based on the headers supplied.

Parameters

- **headers** webob headers
- **default_version** version to use if not specified in headers
- **latest_version** version to use if latest is requested

Returns

a tuple of (major, minor) version numbers

Raises

webob.HTTPNotAcceptable

string = 'X-OpenStack-Ironic-API-Version'

HTTP Header string carrying the requested version

ironic.api.controllers.link module

ironic.api.controllers.link.**build_url**(*resource*, *resource_args*, *bookmark=False*, *base_url=None*)

ironic.api.controllers.link.**make_link**(*rel_name*, *url*, *resource*, *resource_args*, *bookmark=False*, *type=None*)

Build a dict representing a link

ironic.api.controllers.root module

class ironic.api.controllers.root.**RootController**

Bases: object

index(**args*)

ironic.api.controllers.root.**root**()

ironic.api.controllers.version module

ironic.api.controllers.version.**all_versions**()

ironic.api.controllers.version.**default_version**()

Return a dict representing the current default version

id: The ID of the (major) version, also acts as the release number links: A list containing one link that points to the current version of the API

status: Status of the version, one of CURRENT, SUPPORTED, DEPRECATED

min_version: The current, maximum supported (major.minor) version of API.

version: Minimum supported (major.minor) version of API.

Module contents

ironic.api.middleware package

Submodules

ironic.api.middleware.auth_public_routes module

class ironic.api.middleware.auth_public_routes.**AuthPublicRoutes**(*app*, *auth*, *public_api_routes=None*)

Bases: object

A wrapper on authentication middleware.

Does not perform verification of authentication tokens for public routes in the API.

__call__(*env*, *start_response*)

Call self as a function.

ironic.api.middleware.json_ext module

class ironic.api.middleware.json_ext.**JsonExtensionMiddleware**(*app*)

Bases: object

Simplified processing of .json extension.

Previously Ironic API used the guess_content_type_from_ext feature. It was never needed, as we never allowed non-JSON content types anyway. Now that it is removed, this middleware strips .json extension for backward compatibility.

__call__(*env*, *start_response*)

Call self as a function.

transform_header(*version_string*)

Transforms version string to HTTP header format.

ironic.api.middleware.parsable_error module

Middleware to replace the plain text message body of an error response with one formatted so the client can parse it.

Based on pecan.middleware.errordocument

class ironic.api.middleware.parsable_error.**ParsableErrorMiddleware**(*app*)

Bases: object

Replace error body with something the client can parse.

__call__(*environ*, *start_response*)

Call self as a function.

ironic.api.middleware.request_log module

Middleware to log API request details including timing, status codes, and request information for debugging purposes.

class ironic.api.middleware.request_log.**RequestLogMiddleware**(*app*)

Bases: object

Middleware to log request details for debugging.

__call__(*environ*, *start_response*)

Call self as a function.

ironic.api.middleware.request_log.**get_real_ip**(*environ*)

Safely retrieves the real IP address from a WSGI request.

Module contents

class ironic.api.middleware.**AuthPublicRoutes**(*app*, *auth*, *public_api_routes=None*)

Bases: object

A wrapper on authentication middleware.

Does not perform verification of authentication tokens for public routes in the API.

__call__(*env*, *start_response*)

Call self as a function.

class ironic.api.middleware.**JsonExtensionMiddleware**(*app*)

Bases: object

Simplified processing of .json extension.

Previously Ironic API used the guess_content_type_from_ext feature. It was never needed, as we never allowed non-JSON content types anyway. Now that it is removed, this middleware strips .json extension for backward compatibility.

__call__(*env*, *start_response*)

Call self as a function.

transform_header(*version_string*)

Transforms version string to HTTP header format.

class ironic.api.middleware.**ParsableErrorMiddleware**(*app*)

Bases: object

Replace error body with something the client can parse.

__call__(*environ*, *start_response*)

Call self as a function.

ironic.api.validation package

Submodules

ironic.api.validation.validators module

Internal implementation of request/response validating middleware.

class ironic.api.validation.validators.**SchemaValidator**(*schema*, *is_body=True*)

Bases: object

A validator class

This class is changed from Draft202012Validator to add format checkers for data formats that are common in the Ironic API as well as add better error messages.

validate(**args*, ***kwargs*)

validator = None

validator_org

alias of Draft202012Validator

Module contents

API request/response validating middleware.

class ironic.api.validation.**Schemas**

Bases: object

A microversion-aware schema container.

Allow definition and retrieval of schemas on a microversion-aware basis.

__call__() → dict[str, object] | None

Call self as a function.

add_schema(*schema: dict[str, object]*, *min_version: int | None*, *max_version: int | None*) → None

validate_schemas() → None

Ensure there are no overlapping schemas.

ironic.api.validation.**api_version**(*min_version: int | None*, *max_version: int | None = None*, *message: str | None = None*, *exception_class: ~typing.Type[~webob.exc.HTTPException] = <class 'webob.exc.HTTPNotFound'>*)

Decorator for marking lower and upper bounds on API methods.

Parameters

- **min_version** An integer representing the minimum API version that the API is available under.
- **max_version** An integer representing the maximum API version that the API is available under.
- **message** A message to return if the API is not supported.
- **exception_class** The exception class to raise if the API version is not supported (default is HTTPNotFound).

ironic.api.validation.**request_body_schema**(*schema: Dict[str, Any]*, *min_version: int | None = None*, *max_version: int | None = None*)

Decorator for registering a request body schema on API methods.

`schema` will be used for validating the request body just before the API method is executed.

Parameters

- **schema** The JSON Schema schema used to validate the target.
- **min_version** An integer indicating the minimum API version `schema` applies against.
- **max_version** An integer indicating the maximum API version `schema` applies against.

ironic.api.validation.**request_parameter_schema**(*schema: Dict[str, Any]*, *min_version: int | None = None*, *max_version: int | None = None*)

Decorator for registering a request parameter schema on API methods.

`schema` will be used for validating request parameters just before the API method is executed.

Parameters

- **schema** The JSON Schema schema used to validate the target.
- **min_version** An integer indicating the minimum API version `schema` applies against.
- **max_version** An integer indicating the maximum API version `schema` applies against.

ironic.api.validation.**request_query_schema**(*schema: Dict[str, Any]*, *min_version: int | None = None*, *max_version: int | None = None*)

Decorator for registering a request query string schema on API methods.

`schema` will be used for validating request query strings just before the API method is executed.

Parameters

- **schema** The JSON Schema schema used to validate the target.

- **min_version** An integer indicating the minimum API version schema applies against.
- **max_version** An integer indicating the maximum API version schema applies against.

ironic.api.validation.**response_body_schema**(*schema: Dict[str, Any]*, *min_version: int | None = None*, *max_version: int | None = None*)

Decorator for registering a response body schema on API methods.

schema will be used for validating the response body just after the API method is executed.

Parameters

- **schema** The JSON Schema schema used to validate the target.
- **min_version** An integer indicating the minimum API version schema applies against.
- **max_version** An integer indicating the maximum API version schema applies against.

Submodules

ironic.api.app module

class ironic.api.app.**IronicCORS**(*application: WSGIApplication | None*, *conf: dict[str, ty.Any] | cfg.ConfigOpts | None = None*)

Bases: CORS

Ironic-specific CORS class

Were adding the Ironic-specific version headers to the list of simple headers in order that a request bearing those headers might be accepted by the Ironic REST API.

```
simple_headers = ['Accept', 'Accept-Language', 'Content-Type',
'Cache-Control', 'Content-Language', 'Expires', 'Last-Modified', 'Pragma',
'X-Auth-Token', 'X-OpenStack-Ironic-API-Maximum-Version',
'X-OpenStack-Ironic-API-Minimum-Version',
'X-OpenStack-Ironic-API-Version']
```

class ironic.api.app.**IronicRequestId**(*application: WSGIApplication | None*, *conf: dict[str, ty.Any] | cfg.ConfigOpts | None = None*)

Bases: RequestId

Ironic-specific request id middleware

Base request id middleware uses x-openstack-request-id but ironic has been using openstack-request-id historically. Replace the header for backward compatibility.

```
compat_headers:  list[str] = ['openstack-request-id']
```

class ironic.api.app.**VersionSelectorApplication**

Bases: object

__call__(*environ*, *start_response*)

Call self as a function.

ironic.api.app.**get_pecan_config**()

ironic.api.app.**setup_app**(*pecan_config=None*, *extra_hooks=None*)

ironic.api.config module

ironic.api.functions module

class ironic.api.functions.**FunctionArgument**(*name*, *datatype*, *mandatory*, *default*)

Bases: object

An argument definition of an api entry

datatype

Data type

default

Default value if argument is omitted

mandatory

True if the argument is mandatory

name

argument name

resolve_type(*registry*)

class ironic.api.functions.**FunctionDefinition**(*func*)

Bases: object

An api entry definition

arguments

The function arguments (list of *FunctionArgument*)

body_type

If the body carry the data of a single argument, its type

doc

Function documentation

extra_options

Dictionary of protocol-specific options.

static get(*func*)

Returns the *FunctionDefinition* of a method.

get_arg(*name*)

Returns a *FunctionArgument* from its name

ignore_extra_args

True if extra arguments should be ignored, NOT inserted in the kwargs of the function and not raise UnknownArgument exceptions

name

Function name

resolve_types(*registry*)

return_type

Return type

set_arg_types(*argspec*, *arg_types*)

set_options(*body=None*, *ignore_extra_args=False*, *status_code=200*, *rest_content_types=('json', 'xml')*, ***extra_options*)

status_code

Status code

ironic.api.functions.**getargspec**(*f*)

ironic.api.functions.**iswsmefunction**(*f*)

ironic.api.functions.**sig**

alias of *signature*

class ironic.api.functions.**signature**(**types*, ***options*)

Bases: object

Decorator that specify the argument types of an exposed function.

Parameters

- **return_type** Type of the value returned by the function
- **argN** Type of the Nth argument
- **body** If the function takes a final argument that is supposed to be the request body by itself, its type.
- **status_code** HTTP return status code of the function.
- **ignore_extra_args** Allow extra/unknown arguments (default to False)

Most of the time this decorator is not supposed to be used directly, unless you are not using WSME on top of another framework.

If an adapter is used, it will provide either a specialised version of this decororator, either a new decorator named @wsexpose that takes the same parameters (it will in addition expose the function, hence its name).

__call__(*func*)

Call self as a function.

ironic.api.functions.**wrapfunc**(*f*)

ironic.api.hooks module

class ironic.api.hooks.**ConfigHook**

Bases: PecanHook

Attach the config object to the request so controllers can get to it.

before(*state*)

Override this method to create a hook that gets called after routing, but before the request gets passed to your controller.

Parameters
state The Pecan `state` object for the current request.

class ironic.api.hooks.**ContextHook**(*public_api_routes*)

Bases: `PecanHook`

Configures a request context and attaches it to the request.

after(*state*)

Override this method to create a hook that gets called after the request has been handled by the controller.

Parameters
state The Pecan `state` object for the current request.

before(*state*)

Override this method to create a hook that gets called after routing, but before the request gets passed to your controller.

Parameters
state The Pecan `state` object for the current request.

class ironic.api.hooks.**DBHook**

Bases: `PecanHook`

Attach the dbapi object to the request so controllers can get to it.

after(*state*)

Override this method to create a hook that gets called after the request has been handled by the controller.

Parameters
state The Pecan `state` object for the current request.

before(*state*)

Override this method to create a hook that gets called after routing, but before the request gets passed to your controller.

Parameters
state The Pecan `state` object for the current request.

class ironic.api.hooks.**NoExceptionTracebackHook**

Bases: `PecanHook`

Workaround rpc.common: deserialize_remote_exception.

deserialize_remote_exception builds rpc exception traceback into error message which is then sent to the client. Such behavior is a security concern so this hook is aimed to cut-off traceback from the error message.

after(*state*)

Override this method to create a hook that gets called after the request has been handled by the controller.

Parameters
state The Pecan `state` object for the current request.

`class ironic.api.hooks.PublicUrlHook`

Bases: `PecanHook`

Attach the right public_url to the request.

Attach the right public_url to the request so resources can create links even when the API service is behind a proxy or SSL terminator.

`before(state)`

Override this method to create a hook that gets called after routing, but before the request gets passed to your controller.

Parameters
state The Pecan `state` object for the current request.

`class ironic.api.hooks.RPCHook`

Bases: `PecanHook`

Attach the rpcapi object to the request so controllers can get to it.

`before(state)`

Override this method to create a hook that gets called after routing, but before the request gets passed to your controller.

Parameters
state The Pecan `state` object for the current request.

`ironic.api.hooks.policy_deprecation_check()`

ironic.api.method module

`ironic.api.method.body(body_arg)`

Decorator which places HTTP request body JSON into a method argument

Parameters
body_arg Name of argument to populate with body JSON

`ironic.api.method.expose(status_code=None)`

`ironic.api.method.format_exception(excinfo, debug=False)`

Extract information that can be sent to the client.

ironic.api.wsgi module

WSGI script for Ironic API, installed by pbr.

`ironic.api.wsgi.initialize_wsgi_app(argv=['/home/zuul/src/opendev.org/openstack/ironic/.tox/pdf-docs/bin/sphinx-build', '-E', '-b', 'latex', 'doc/source', 'doc/build/pdf'])`

Module contents

ironic.command package

Submodules

ironic.command.api module

The Ironic Service API.

ironic.command.api.**main**()

ironic.command.conductor module

The Ironic Management Service

ironic.command.conductor.**issue_startup_warnings**(*conf*)

ironic.command.conductor.**main**()

ironic.command.conductor.**warn_about_inconsistent_kernel_ramdisk**(*conf*)

Check consistency of configurations around kernels and ramdisks

This method logs a warning if any of the paired CONF.conductor.*_ramdisk_by_arch and CONF.conductor.*_kernel_by_arch config dictionaries are inconsistent such as having a kernel configured for aarch64 without having a ramdisk configured.

Parameters
conf an ironic.conf.CONF oslo.config object

Returns
None

ironic.command.conductor.**warn_about_max_wait_parameters**(*conf*)

ironic.command.conductor.**warn_about_sqlite**()

ironic.command.conductor.**warn_about_unsafe_shred_parameters**(*conf*)

ironic.command.dbsync module

Run storage database migration.

class ironic.command.dbsync.**DBCommand**

Bases: object

check_obj_versions(*ignore_missing_tables=False*)

Check the versions of objects.

Check that the object versions are compatible with this release of ironic. It does this by comparing the objects .version field in the database, with the expected versions of these objects.

Returns None if compatible; a string describing the issue otherwise.

create_schema()

online_data_migrations()

revision()

stamp()

upgrade()

version()

ironic.command.dbsync.**add_command_parsers**(*subparsers*)

ironic.command.dbsync.**main**()

ironic.command.networking module

The Ironic Networking Service

ironic.command.networking.**issue_startup_warnings**(*conf*)

Issue any startup warnings for the networking service.

ironic.command.networking.**main**()

ironic.command.novncproxy module

Websocket proxy that is compatible with OpenStack Ironic noVNC consoles. Leverages websockify.py by Joel Martin

ironic.command.novncproxy.**main**()

ironic.command.pxe_filter module

class ironic.command.pxe_filter.**RPCService**(*host*, *manager_module*, *manager_class*)

Bases: *BaseRPCService*

stop()

Stop service.

ironic.command.pxe_filter.**main**()

ironic.command.singleprocess module

ironic.command.singleprocess.**main**()

ironic.command.status module

class ironic.command.status.**Checks**

Bases: UpgradeCommands

Upgrade checks for the ironic-status upgrade check command

Upgrade checks should be added as separate methods in this class and added to _upgrade_checks tuple.

ironic.command.status.**main**()

ironic.command.utils module

ironic.command.utils.**handle_drain**(*signo*, *frame*)

ironic.command.utils.**handle_no_deregister**(*signo*, *frame*)

ironic.command.utils.**handle_signal**()

Add a signal handler for SIGUSR1, SIGUSR2.

The SIGUSR1 handler ensures that the manager is not deregistered when it is shutdown.

The SIGUSR2 handler starts a drain shutdown.

Module contents

ironic.common package

Subpackages

ironic.common.glance_service package

Submodules

ironic.common.glance_service.image_service module

class ironic.common.glance_service.image_service.**GlanceImageService**(*client=None*, *context=None*)

Bases: object

call(*method*, **args*, ***kwargs*)

Call a glance client method.

If we get a connection error, retry the request according to CONF.num_retries.

Parameters

- **method** The method requested to be called.
- **args** A list of positional arguments for the method called
- **kwargs** A dict of keyword arguments for the method called

Raises

GlanceConnectionFailed

download(*image_href*, *data=None*, *checksum=None*, *checksum_algo=None*)

Calls out to Glance for data and writes data.

Parameters

- **image_href** The opaque image identifier.
- **data** (Optional) File object to write data to.
- **checksum** Expected checksum value for validation during transfer.
- **checksum_algo** Algorithm for checksum (e.g., md5, sha256).

property is_auth_set_needed

Property to notify the caller if it needs to set authentication.

member_list(*image_id: str*) → list[Member]

Calls out to Glance to find the members for an image_id

Parameters
image_id The image ID.

Returns
list of member IDs

show(*image_href*)

Returns a dict with image data for the given opaque image id.

Parameters
image_href The opaque image identifier.

Returns
A dict containing image metadata.

Raises
ImageNotFound

Raises
ImageUnacceptable if the image status is not active

swift_temp_url(*image_info*)

Generate a no-auth Swift temporary URL.

This function will generate (or return the cached one if temp URL cache is enabled) the temporary Swift URL using the image id from Glance and the config options: swift_endpoint_url, swift_api_version, swift_account and swift_container. The temporary URL will be valid for swift_temp_url_duration seconds. This allows Ironic to download a Glance image without passing around an auth_token.

Parameters
image_info The return from a GET request to Glance for a certain image_id. Should be a dictionary, with keys like name and checksum. See https://docs.openstack.org/glance/latest/user/glanceapi.html for examples.

Returns
A signed Swift URL from which an image can be downloaded, without authentication.

Raises
InvalidParameterValue if Swift config options are not set correctly.

Raises
MissingParameterValue if a required parameter is not set.

Raises
ImageUnacceptable if the image info from Glance does not have an image ID.

property transfer_verified_checksum

The transferred artifact checksum.

class ironic.common.glance_service.image_service.**TempUrlCacheElement**(*url*, *url_expires_at*)

Bases: tuple

url

Alias for field number 0

url_expires_at

Alias for field number 1

ironic.common.glance_service.image_service.**check_image_service**(*func*)

Creates a glance client if doesnt exists and calls the function.

ironic.common.glance_service.service_utils module

ironic.common.glance_service.service_utils.**get_conductor_project_id**()

ironic.common.glance_service.service_utils.**get_image_member_list**(*image_id*, *context*)

ironic.common.glance_service.service_utils.**is_glance_image**(*image_href*)

ironic.common.glance_service.service_utils.**is_image_active**(*image*)

Check the image status.

This check is needed in case the Glance image is stuck in queued status or pending_delete.

ironic.common.glance_service.service_utils.**is_image_available**(*context*, *image*)

Check image availability.

This check is needed in case Nova and Glance are deployed without authentication turned on.

ironic.common.glance_service.service_utils.**parse_image_id**(*image_href*)

Parse an image id from image href.

Parameters

image_href href of an image

Returns

image id parsed from image_href

Raises

InvalidImageRef when input image href is invalid

ironic.common.glance_service.service_utils.**translate_from_glance**(*image*)

Module contents

ironic.common.inspection_rules package

Submodules

ironic.common.inspection_rules.actions module

class ironic.common.inspection_rules.actions.**ActionBase**

Bases: *Base*

Abstract base class for rule action plugins.

FORMATTED_ARGS = []

List of params to be formatted with python format.

abstract __call__(*task*, **args*, ***kwargs*)

Run action on successful rule match.

execute_action(*task*, *action*, *inventory*, *plugin_data*, *loop_context=None*)

execute_with_loop(*task*, *action*, *inventory*, *plugin_data*)

class ironic.common.inspection_rules.actions.**AddTraitAction**

Bases: *ActionBase*

__call__(*task*, *name*)

Run action on successful rule match.

class ironic.common.inspection_rules.actions.**CallAPIHookAction**

Bases: *ActionBase*

FORMATTED_ARGS = ['url']

List of params to be formatted with python format.

OPTIONAL_PARAMS = ['headers', 'proxies', 'timeout', 'retries', 'backoff_factor']

__call__(*task*, *url*, *headers=None*, *proxies=None*, *timeout=5*, *retries=3*, *backoff_factor=0.3*)

Run action on successful rule match.

class ironic.common.inspection_rules.actions.**DelAttributeAction**

Bases: *ActionBase*

__call__(*task*, *path*)

Run action on successful rule match.

class ironic.common.inspection_rules.actions.**DelPortAttributeAction**

Bases: *ActionBase*

__call__(*task*, *port_id*, *path*)

Run action on successful rule match.

class ironic.common.inspection_rules.actions.**ExtendAttributeAction**

Bases: *ActionBase*

FORMATTED_ARGS = ['value']

List of params to be formatted with python format.

__call__(*task*, *path*, *value*, *unique=False*)

Run action on successful rule match.

class ironic.common.inspection_rules.actions.**ExtendPluginDataAction**

Bases: *ActionBase*

FORMATTED_ARGS = ['value']

List of params to be formatted with python format.

REQUIRES_PLUGIN_DATA = True

Flag to indicate if this action needs plugin_data as an arg.

__call__(*task*, *path*, *value*, *plugin_data*, *unique=False*)

Run action on successful rule match.

class ironic.common.inspection_rules.actions.**ExtendPortAttributeAction**

Bases: *ActionBase*

FORMATTED_ARGS = ['value']

List of params to be formatted with python format.

__call__(*task*, *port_id*, *path*, *value*, *unique=False*)

Run action on successful rule match.

class ironic.common.inspection_rules.actions.**FailAction**

Bases: *ActionBase*

__call__(*task*, *msg*)

Run action on successful rule match.

class ironic.common.inspection_rules.actions.**LogAction**

Bases: *ActionBase*

FORMATTED_ARGS = ['msg']

List of params to be formatted with python format.

VALID_LOG_LEVELS = {'critical', 'debug', 'error', 'info', 'warning'}

__call__(*task*, *msg*, *level='info'*)

Run action on successful rule match.

class ironic.common.inspection_rules.actions.**RemoveTraitAction**

Bases: *ActionBase*

__call__(*task*, *name*)

Run action on successful rule match.

class ironic.common.inspection_rules.actions.**SetAttributeAction**

Bases: *ActionBase*

FORMATTED_ARGS = ['value']

List of params to be formatted with python format.

__call__(*task*, *path*, *value*)

Run action on successful rule match.

class ironic.common.inspection_rules.actions.**SetCapabilityAction**

Bases: *ActionBase*

FORMATTED_ARGS = ['value']

List of params to be formatted with python format.

__call__(*task*, *name*, *value*)

Run action on successful rule match.

class ironic.common.inspection_rules.actions.**SetPluginDataAction**

Bases: *ActionBase*

FORMATTED_ARGS = ['value']

List of params to be formatted with python format.

REQUIRES_PLUGIN_DATA = True

Flag to indicate if this action needs plugin_data as an arg.

__call__(*task*, *path*, *value*, *plugin_data*)

Run action on successful rule match.

class ironic.common.inspection_rules.actions.**SetPortAttributeAction**

Bases: *ActionBase*

FORMATTED_ARGS = ['value']

List of params to be formatted with python format.

__call__(*task*, *port_id*, *path*, *value*)

Run action on successful rule match.

class ironic.common.inspection_rules.actions.**UnsetCapabilityAction**

Bases: *ActionBase*

__call__(*task*, *name*)

Run action on successful rule match.

class ironic.common.inspection_rules.actions.**UnsetPluginDataAction**

Bases: *ActionBase*

REQUIRES_PLUGIN_DATA = True

Flag to indicate if this action needs plugin_data as an arg.

__call__(*task*, *path*, *plugin_data*)

Run action on successful rule match.

ironic.common.inspection_rules.actions.**get_action**(*op_name*)

Get operator class by name.

ironic.common.inspection_rules.actions.**update_nested_dict**(*d*, *key_path*, *value*)

ironic.common.inspection_rules.base module

class ironic.common.inspection_rules.base.**Base**

Bases: object

REQUIRES_PLUGIN_DATA = False

Flag to indicate if this action needs plugin_data as an arg.

get_validation_signature()

Get the signature to validate against.

static interpolate_variables(*value*, *node*, *inventory*, *plugin_data*, *loop_context=None*, *op=None*)

validate(*op_args*)

Validate args passed during creation.

Default implementation checks for presence of required fields.

Parameters

op_args Operator args as a dictionary

Raises

InspectionRuleValidationFailure on validation failure

ironic.common.inspection_rules.engine module

ironic.common.inspection_rules.engine.**apply_actions**(*task*, *rule*, *inventory*, *plugin_data*)

ironic.common.inspection_rules.engine.**apply_rules**(*task*, *inventory*, *plugin_data*, *inspection_phase*)

Apply inspection rules to a node.

ironic.common.inspection_rules.engine.**check_conditions**(*task*, *rule*, *inventory*, *plugin_data*)

ironic.common.inspection_rules.engine.**get_built_in_rules**(*rules_file*)

Load built-in inspection rules.

ironic.common.inspection_rules.operators module

class ironic.common.inspection_rules.operators.**ContainsOperator**

Bases: *ReOperator*

__call__(*task*, *value*, *regex*)

Checks if condition holds for a given field.

class ironic.common.inspection_rules.operators.**EmptyOperator**

Bases: *OperatorBase*

FORMATTED_ARGS = ['value']

__call__(*task*, *value*)

Checks if condition holds for a given field.

class ironic.common.inspection_rules.operators.**EqOperator**

Bases: *SimpleOperator*

op(*b*, */*)

Same as a == b.

class ironic.common.inspection_rules.operators.**GeOperator**

Bases: *SimpleOperator*

op(*b*, */*)

Same as a >= b.

class ironic.common.inspection_rules.operators.**GtOperator**

Bases: *SimpleOperator*

op(*b*, */*)

Same as a > b.

class ironic.common.inspection_rules.operators.**IsFalseOperator**

Bases: *OperatorBase*

FORMATTED_ARGS = ['value']

__call__(*task*, *value*)

Checks if condition holds for a given field.

class ironic.common.inspection_rules.operators.**IsNoneOperator**

Bases: *OperatorBase*

FORMATTED_ARGS = ['value']

__call__(*task*, *value*)

Checks if condition holds for a given field.

class ironic.common.inspection_rules.operators.**IsTrueOperator**

Bases: *OperatorBase*

FORMATTED_ARGS = ['value']

__call__(*task*, *value*)

Checks if condition holds for a given field.

class ironic.common.inspection_rules.operators.**LeOperator**

Bases: *SimpleOperator*

op(*b*, */*)

Same as a <= b.

class ironic.common.inspection_rules.operators.**LtOperator**

Bases: *SimpleOperator*

op(*b*, */*)

Same as a < b.

class ironic.common.inspection_rules.operators.**MatchesOperator**

Bases: *ReOperator*

__call__(*task*, *value*, *regex*)

Checks if condition holds for a given field.

class ironic.common.inspection_rules.operators.**NeOperator**

Bases: *SimpleOperator*

op(*b*, */*)

Same as a != b.

class ironic.common.inspection_rules.operators.**NetOperator**

Bases: *OperatorBase*

FORMATTED_ARGS = ['address', 'subnet']

__call__(*task*, *address*, *subnet*)

Checks if condition holds for a given field.

class ironic.common.inspection_rules.operators.**OneOfOperator**

Bases: *OperatorBase*

FORMATTED_ARGS = ['value']

__call__(*task*, *value*, *values=[]*)

Checks if condition holds for a given field.

class ironic.common.inspection_rules.operators.**OperatorBase**

Bases: *Base*

Abstract base class for rule condition plugins.

abstract __call__(*task*, **args*, ***kwargs*)

Checks if condition holds for a given field.

check_condition(*task*, *condition*, *inventory*, *plugin_data*, *loop_context=None*)

Process condition arguments and apply the check logic.

Parameters

- **task** TaskManger instance
- **condition** condition to check
- **inventory** Node inventory data with hardware information
- **plugin_data** Data from inspection plugins
- **loop_context** Current loop item when called from check_with_loop

Raises

InspectionRuleExecutionFailure on unacceptable field value

Returns

True if check succeeded, otherwise False

check_with_loop(*task*, *condition*, *inventory*, *plugin_data*)

class ironic.common.inspection_rules.operators.**ReOperator**

Bases: *OperatorBase*

FORMATTED_ARGS = ['value']

validate_regex(*regex*)

class ironic.common.inspection_rules.operators.**SimpleOperator**

Bases: *OperatorBase*

__call__(*task*, *values*, *force_strings=False*)

Checks if condition holds for a given field.

op = None

ironic.common.inspection_rules.operators.**get_operator**(*op_name*)

Get operator class by name.

ironic.common.inspection_rules.utils module

Utilities and helper functions for rules.

class ironic.common.inspection_rules.utils.**ShallowMaskDict**(*data*, *sensitive_fields=None*, *mask_enabled=True*)

Bases: MutableMapping

Dictionary wrapper to mask sensitive fields on the fly.

This class implements the MutableMapping ABC to provide a complete dict-like interface while masking sensitive fields when accessed.

copy()

set_mask_enabled(*mask_enabled*)

class ironic.common.inspection_rules.utils.**ShallowMaskList**(*original_list*, *sensitive_fields=None*, *mask_enabled=True*)

Bases: MutableSequence

A proxy list to maintain original list and applies masking on the fly.

This class implements the MutableSequence ABC to provide a complete list-like interface while handling sensitive data masking consistently with ShallowMaskDict.

copy()

insert(*index*, *value*)

S.insert(index, value) — insert value before index

set_mask_enabled(*mask_enabled*)

ironic.common.inspection_rules.utils.**normalize_path**(*path*)

Convert a path (dot or slash notation) to a list of path parts

ironic.common.inspection_rules.utils.**parse_inverted_operator**(*op*)

Handle inverted operators.

Parses a logical condition operator to determine if it has been negated using a single leading exclamation mark (!). Ensures only one exclamation mark is allowed.

Example Usage:

```
parse_inverted_operator("!eq")   # Returns ("eq", True)
parse_inverted_operator(" eq ")  # Returns ("eq", False)
parse_inverted_operator("!!eq")  # Raises ValueError
```

Raises

ValueError: If multiple exclamation marks are present.

Returns

A tuple containing the cleaned operator and a boolean indicating whether negation was applied.

ironic.common.inspection_rules.validation module

class ironic.common.inspection_rules.validation.**InspectionPhase**(*value*, *names=<not given>*, **values*, *module=None*, *qualname=None*, *type=None*, *start=1*, *boundary=None*)

Bases: Enum

MAIN = 'main'

ironic.common.inspection_rules.validation.**validate_rule**(*rule*, *built_in=False*)

Validate an inspection rule using the JSON schema.

Parameters

- **rule** The inspection rule to validate.
- **built_in** Should this rule be treated as built in for validation

Raises

Invalid if the rule is invalid.

Module contents

ironic.common.json_rpc package

Submodules

ironic.common.json_rpc.client module

A simple JSON RPC client.

This client is compatible with any JSON RPC 2.0 implementation, including ours.

class ironic.common.json_rpc.client.**Client**(*serializer*, *version_cap=None*, *conf_group: str = 'json_rpc'*)

Bases: object

JSON RPC client with ironic exception handling.

allowed_exception_namespaces = ['ironic.common.exception.']

can_send_version(*version*)

prepare(*topic*, *version=None*)

Prepare the client to transmit a request.

Parameters

- **topic** Topic which is being addressed. Typically this is the hostname of the remote json-rpc service.
- **version** The RPC API version to utilize.

ironic.common.json_rpc.server module

Implementation of JSON RPC for communication between API and conductors.

This module implements a subset of JSON RPC 2.0 as defined in https://www.jsonrpc.org/specification. Main differences: * No support for batched requests. * No support for positional arguments passing. * No JSON RPC 1.0 fallback.

class ironic.common.json_rpc.server.**EmptyContext**(*src*)

Bases: object

request_id = None

to_dict()

exception ironic.common.json_rpc.server.**InvalidParams**(*message=None*, ***kwargs*)

Bases: *JsonRpcError*

code = -32602

exception ironic.common.json_rpc.server.**InvalidRequest**(*message=None*, ***kwargs*)

Bases: *JsonRpcError*

code = -32600

exception ironic.common.json_rpc.server.**JsonRpcError**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.json_rpc.server.**MethodNotFound**(*message=None*, ***kwargs*)

Bases: *JsonRpcError*

code = -32601

exception ironic.common.json_rpc.server.**ParseError**(*message=None*, ***kwargs*)

Bases: *JsonRpcError*

code = -32700

class ironic.common.json_rpc.server.**WSGIService**(*manager*, *serializer*, *context_class=<class 'ironic.common.json_rpc.server.EmptyContext'>*, *conf_group: str = 'json_rpc'*)

Bases: *BaseWSGIService*

Provides ability to launch JSON RPC as a WSGI application.

Module contents

ironic.common.trait_based_networking package

Subpackages

ironic.common.trait_based_networking.grammar package

Submodules

ironic.common.trait_based_networking.grammar.parser module

class ironic.common.trait_based_networking.grammar.parser.**FilterExpressionTransformer**(*visit_to* *bool* = *True*)

Bases: Transformer

compound_expression(*items*)

equality(*items*)

filter_expression(*items*)

function_expression(*items*)

gt(*items*)

gt_or_eq(*items*)

inequality(*items*)

lt(*items*)

lt_or_eq(*items*)

op_and(*items*)

op_or(*items*)

paren_expression(*items*)

port_is_port(*items*)

port_is_portgroup(*items*)

prefix_match(*items*)

single_expression(*items*)

string_literal(*items*)

variable_name(*items*)

Module contents

Submodules

ironic.common.trait_based_networking.base module

class ironic.common.trait_based_networking.base.**Actions**(*value*, *names=<not given>*, **values*, *module=None*, *qualname=None*, *type=None*, *start=1*, *boundary=None*)

Bases: Enum

Represents actions recognized by Trait Based Networking

ATTACH_PORT = 'attach_port'

ATTACH_PORTGROUP = 'attach_portgroup'

BOND_PORTS = 'bond_ports'

GROUP_AND_ATTACH_PORTS = 'group_and_attach_ports'

class ironic.common.trait_based_networking.base.**AttachAction**(*trait_action: TraitAction, node_uuid: str*)

Bases: *RenderedAction*

Base class for actions which will attach objects to networks

abstract get_portlike_object(*task*)

abstract portlike_uuid()

class ironic.common.trait_based_networking.base.**AttachPort**(*trait_action: TraitAction, node_uuid: str, port_uuid: str, network_id: str*)

Bases: *AttachAction*

Attach a port to a network

Contains all the necessary information to attach a port to a network (vif)

get_portlike_object(*task*)

network_id: str

port_uuid: str

portlike_uuid()

class ironic.common.trait_based_networking.base.**AttachPortgroup**(*trait_action: TraitAction, node_uuid: str, portgroup_uuid: str, network_id: str*)

Bases: *AttachAction*

Attach a portgroup to a network

Contains all the necessary information to attach a portgroup to a network (vif)

get_portlike_object(*task*)

network_id: str

portgroup_uuid: str

portlike_uuid()

class ironic.common.trait_based_networking.base.**Comparator**(*value*, *names=<not given>*, **values*, *module=None*, *qualname=None*, *type=None*, *start=1*, *boundary=None*)

Bases: Enum

A FilterExpression Comparator

Comparators test mathematical-esque relations between a variable and a string.

EQUALITY = '=='

GT = '>'

GT_OR_EQ = '>='

INEQUALITY = '!='

LT = '<'

LT_OR_EQ = '<='

PREFIX_MATCH = '=~'

eval(*variable*, *value*)

class ironic.common.trait_based_networking.base.**CompoundExpression**(*left_expression*, *operator*, *right_expression*)

Bases: object

A compound expression found within a FilterExpression

A compound expression consists of a left-hand expression and a right-hand expression joined by a boolean operator.

eval(*port*, *network*)

class ironic.common.trait_based_networking.base.**FilterExpression**(*expression*)

Bases: object

Encompasses filters found in TraitActions

Used to filter (port, network) pairs to apply actions which pass the filter.

Use FilterExpression.parse to transform a string containing a grammatically correct expression into a fully parsed FilterExpression object.

See FILTER_EXPRESSION_GRAMMAR in ironic.common.trait_based_networking.grammar.parser for full understanding of how these expressions are parsed.

eval(*port*, *network*)

classmethod parse(*expression*)

class ironic.common.trait_based_networking.base.**FunctionExpression**(*variable*)

Bases: object

A callable function from within a FilterExpression

Used to query objects to determine if they pass a FilterExpression or not.

eval(*port*, *network*)

class ironic.common.trait_based_networking.base.**GroupAndAttachPorts**(*trait_action: TraitAction*, *node_uuid: str*, *port_uuids: list[str]*, *network_id: str*)

Bases: *RenderedAction*

Assemble a group of ports as a dynamic portgroup and attach it

Contains all necessary information to assemble the new portgroup and attach it to a network (vif).

network_id: str

port_uuids: list[str]

class ironic.common.trait_based_networking.base.**Network**(*id: str*, *name: str*, *tags: frozenset[str]*)

Bases: object

A Network (aka vif)

Used to match against TraitAction FilterExpressions

classmethod from_vif_info(*vif_info*)

Helper method to create Networks from vif_info dictionaries

id: str

name: str

tags: frozenset[str]

classmethod universal_network()

class ironic.common.trait_based_networking.base.**NetworkTrait**(*name*, *actions*, *order=1*)

Bases: object

Represents an entire Trait for Trait Based Networking

Each trait can have many actions. Traits can be ordered explicitly if so desired.

class ironic.common.trait_based_networking.base.**NoMatch**(*trait_action: TraitAction*, *node_uuid: str*, *reason: str*)

Bases: *RenderedAction*

Returned by network planning when a trait action finds no matches

reason: str

class ironic.common.trait_based_networking.base.**NotImplementedAction**(*trait_action: TraitAction, node_uuid: str, action: Actions*)

Bases: *RenderedAction*

Returned by network planning if an action has not been implemented

action: *Actions*

class ironic.common.trait_based_networking.base.**Operator**(*value*, *names=<not given>*, **values*, *module=None*, *qualname=None*, *type=None*, *start=1*, *boundary=None*)

Bases: Enum

A FilterExpression Operator

Represents a boolean operator (AND, OR) between two expressions in a filter expression.

AND = '&&'

OR = '||'

eval(*variable*, *value*)

class ironic.common.trait_based_networking.base.**ParenExpression**(*expression*)

Bases: object

Represents an parentheses expression found in a FilterExpression

Aids in logically grouping and evaluating expressions before others.

eval(*port*, *network*)

class ironic.common.trait_based_networking.base.**Port**(*id: int, uuid: str, address: str, category: str | None, physical_network: str | None, vendor: str | None, available_for_dynamic_portgroup: bool*)

Bases: *PrimordialPort*

A Port used internally to query and match to TraitActions

available_for_dynamic_portgroup: bool

classmethod from_ironic_port(*ironic_port*)

is_port()

is_portgroup()

classmethod universal_port()

class ironic.common.trait_based_networking.base.**Portgroup**(*id: int*, *uuid: str*, *address: str*, *category: str | None*, *physical_network: str | None*, *vendor: str | None*, *dynamic_portgroup: bool*)

Bases: *PrimordialPort*

A Portgroup used internally to query and match to TraitActions

dynamic_portgroup: bool

classmethod from_ironic_portgroup(*ironic_portgroup*)

is_port()

is_portgroup()

class ironic.common.trait_based_networking.base.**PrimordialPort**(*id: int*, *uuid: str*, *address: str*, *category: str | None*, *physical_network: str | None*, *vendor: str | None*)

Bases: object

A set of common attributes belonging to both Ports and Portgroups

address: str

category: str | None

id: int

physical_network: str | None

uuid: str

vendor: str | None

class ironic.common.trait_based_networking.base.**RenderedAction**(*trait_action: TraitAction*, *node_uuid: str*)

Bases: object

A base class for Actions which are ready to apply

node_uuid: str

trait_action: *TraitAction*

class ironic.common.trait_based_networking.base.**SingleExpression**(*variable*, *comparator*, *literal*)

Bases: object

A single expression from within a FilterExpression

A single expression consists of a variable name, a comparator, and a string literal. For example:

port.vendor == purple

In this example, When eval()ed against a port whose vendor is purple this expression will return True. Otherwise the expression will return False.

eval(*port*, *network*)

class ironic.common.trait_based_networking.base.**TraitAction**(*trait_name: str*, *action:* Actions, *filter:* FilterExpression, *min_count: int | None = None*, *max_count: int | None = None*)

Bases: object

An action defined by a NetworkTrait

Each action contains a filter (FilterExpression) that determines which (port, network) pairs the action can apply to.

ALL_KEYS: ClassVar[list[str]] = ['action', 'filter', 'min_count', 'max_count']

NECESSARY_KEYS: ClassVar[list[str]] = ['action', 'filter']

action: *Actions*

filter: *FilterExpression*

matches(*portlike*, *network*)

Check if filter expression matches the port, network pairing.

max_count: int | None = None

min_count: int | None = None

trait_name: str

validate()

Check that the action is valid.

class ironic.common.trait_based_networking.base.**Variables**(*value*, *names=<not given>*, **values*, *module=None*, *qualname=None*, *type=None*, *start=1*, *boundary=None*)

Bases: Enum

Represents a variable (or function) used by FilterExpressions

Values of variables are drawn from network-related objects like ports, portgroups, or networks (aka vifs).

NETWORK_NAME = 'network.name'

NETWORK_TAGS = 'network.tags'

PORT_ADDRESS = 'port.address'

PORT_CATEGORY = 'port.category'

PORT_IS_PORT = 'port.is_port'

PORT_IS_PORTGROUP = 'port.is_portgroup'

PORT_PHYSICAL_NETWORK = 'port.physical_network'

PORT_VENDOR = 'port.vendor'

attribute_name()

object_name()

ironic.common.trait_based_networking.config_file module

class ironic.common.trait_based_networking.config_file.**ConfigFile**(*filename*)

Bases: object

Provides functionality to read TBN configuration files

Basic flow for use goes like:

```
cf = ConfigFile("some_tbn_config.yaml") # File is read().
valid, reasons = cf.validate()
if not valid:
    # Do something with reasons, like raise an exception and log.
    return reasons
cf.parse()  # If the file is valid, this *should* parse the config
traits = cf.traits() # Get the parsed traits as a list.
```

parse()

Render contents of configuration file as TBN objects

The result of this method can later be retrieved by calling traits().

read()

Read the YAML YBN configuration file

traits()

Return the parsed traits from the configuration file.

validate()

Check that contents conform to TBN expectations.

Returns

(valid, reasons): valid is a boolean representing if the contents passed validation or not, and reasons is a list of strings describing why the contents failed validation if they are not valid.

ironic.common.trait_based_networking.defaults module

ironic.common.trait_based_networking.defaults.**default_network_trait**()

Gets the default TBN trait

Returns

A single trait representing the default trait to apply if no other traits apply to a node. Will get the default defined in the configuration file if present. Otherwise returns a predefined default which matches the first available port or portgroup.

ironic.common.trait_based_networking.loader module

class ironic.common.trait_based_networking.loader.**ConfigLoader**

Bases: object

Extends ConfigFile and provides automatic config file change detection

Not intended for consumption outside of this module. Use the module level tbn_config_file_traits() function to retrieve configured and parsed traits.

refresh()

property traits

property valid

ironic.common.trait_based_networking.loader.**is_config_valid**()

ironic.common.trait_based_networking.loader.**tbn_config_file_traits**()

Get TBN traits from the configured YAML file.

Thread-safe way to retrieve the configured and parsed TBN traits.

ironic.common.trait_based_networking.plan module

ironic.common.trait_based_networking.plan.**all_no_match**(*actions: list[*RenderedAction*]*) → bool

Check if a list of actions contains only NoMatch actions

ironic.common.trait_based_networking.plan.**filter_out_attached_portlikes**(*portlikes: list[*PrimordialPort*]*, *actions: list[*AttachAction*]*) → list[*PrimordialPort*]

Filters out attached portlikes based on generated attach actions

ironic.common.trait_based_networking.plan.**filter_out_grouped_ports**(*portlikes: list[PrimordialPort]*, *actions: list[GroupAndAttachPorts]*) → list[*PrimordialPort*]

Filters out ports that have been used to form dynamic portgroups

ironic.common.trait_based_networking.plan.**filter_traits_for_node**(*node: Node*, *traits: list[NetworkTrait]*) → list[*NetworkTrait*]

Return a list of NetworkTraits that apply to a node

ironic.common.trait_based_networking.plan.**is_no_match_list**(*actions: list[RenderedAction]*) → bool

Check if a list contains only a NoMatch action

ironic.common.trait_based_networking.plan.**is_portlike_attached**(*portlike: Port | Portgroup*) → bool

Check if a portlike is attached or not

ironic.common.trait_based_networking.plan.**order_traits**(*traits: list[NetworkTrait]*) → list[*NetworkTrait*]

Sort a list of traits in ascending trait.order

ironic.common.trait_based_networking.plan.**plan_network**(*network_trait: NetworkTrait*, *node_uuid: str*, *node_ports: list[Port]*, *node_portgroups: list[Portgroup]*, *node_networks: list[Network]*) → list[*RenderedAction*]

Plan the network of a node based on TBN traits

Parameters

- **network_trait** A single NetworkTrait to consider for planning.
- **node_uuid** The UUID of the node to which this plan applies.
- **ports** A list of Ports available to this node.
- **portgroups** A list of Portgroups available to this node.
- **node_networks** A list of networks available to this node.

Returns

A list of RenderedActions which should be executed by the appropriate network driver.

ironic.common.trait_based_networking.plan.**plan_vif_attach**(*traits: list[NetworkTrait]*, *task: TaskManager*, *vif_info: dict*) → list[*RenderedAction*]

Main entry point of TBN from _vif_attach_tbn in NeutronVIFPortIDMixIn

Parameters

- **traits** A list of NetworkTraits that apply to the node being considered.
- **task** A TaskManager which contains important information about the node and network objects available.
- **vif_info** Information about the network (aka vif) which TBN will use to plan actions.

Returns

A list of RenderedActions which should be executed by the appropriate network driver.

Module contents

Submodules

ironic.common.args module

ironic.common.args.**and_valid**(**validators*)

Validates that every supplied validator passes

The value returned from each validator is passed as the value to the next one.

Parameters

- **name** Name of the argument
- **value** A value

Returns

The value transformed through every supplied validator

Raises

The error from the first failed validator

ironic.common.args.**boolean**(*name*, *value*)

Validate that the value is a string representing a boolean

Parameters

- **name** Name of the argument
- **value** A string value

Returns

The boolean representation of the value, or None if value is None

Raises

InvalidParameterValue if the value cannot be converted to a boolean

ironic.common.args.**dict_valid**(***validators*)

Return a validator function which validates dict fields

Validators will replace the value with the validation result. Any dict item which has no validator is ignored. When a key is missing in the value then the corresponding validator will not be run.

Param
validators dict where the key is a dict key to validate and the value is a validator function to run on that value

Returns
validator function which takes name and value arguments

ironic.common.args.**host_port**(*name*, *value*)

ironic.common.args.**integer**(*name*, *value*)

Validate that the value represents an integer

Parameters

- **name** Name of the argument
- **value** A value representing an integer

Returns
The value as an int, or None if value is None

Raises
InvalidParameterValue if the value does not represent an integer

ironic.common.args.**mac_address**(*name*, *value*)

Validate that the value represents a MAC address

Parameters

- **name** Name of the argument
- **value** A string value representing a MAC address

Returns
The value as a normalized MAC address, or None if value is None

Raises
InvalidParameterValue if the value is not a valid MAC address

ironic.common.args.**name**(*name*, *value*)

Validate that the value is a logical name

Parameters

- **name** Name of the argument
- **value** A logical name string value

Returns
The value, or None if value is None

Raises
InvalidParameterValue if the value is not a valid logical name

ironic.common.args.**or_valid**(**validators*)

Validates if at least one supplied validator passes

Parameters

- **name** Name of the argument
- **value** A value

Returns
The value returned from the first successful validator

Raises
The error from the last validator when every validation fails

ironic.common.args.**patch**(*name*, *value*, *, *schema={'items': {'additionalProperties': False, 'properties': {'op': {'enum': ['add', 'replace', 'remove'], 'type': 'string'}, 'path': {'pattern': '^(/[\\w-]+)+$', 'type': 'string'}, 'value': {}}, 'required': ['op', 'path'], 'type': 'object'}, 'type': 'array'}*)

Validate a patch API operation

ironic.common.args.**schema**(*schema*)

Return a validator function which validates the value with jsonschema

Param
schema dict representing jsonschema to validate with

Returns
validator function which takes name and value arguments

ironic.common.args.**string**(*name*, *value*)

Validate that the value is a string

Parameters

- **name** Name of the argument
- **value** A string value

Returns
The string value, or None if value is None

Raises
InvalidParameterValue if the value is not a string

ironic.common.args.**string_list**(*name*, *value*)

Validate and convert comma delimited string to a list.

Parameters

- **name** Name of the argument
- **value** A comma separated string of values

Returns
A list of unique values (lower-cased), maintaining the same order, or None if value is None

Raises
InvalidParameterValue if the value is not a string

ironic.common.args.**types**(**types*)

Return a validator function which checks the value is one of the types

Param
types one or more types to use for the isinstance test

Returns
validator function which takes name and value arguments

ironic.common.args.**uuid**(*name*, *value*)

Validate that the value is a UUID

Parameters

- **name** Name of the argument
- **value** A UUID string value

Returns
The value, or None if value is None

Raises
InvalidParameterValue if the value is not a valid UUID

ironic.common.args.**uuid_or_name**(*name*, *value*)

Validate that the value is a UUID or logical name

Parameters

- **name** Name of the argument
- **value** A UUID or logical name string value

Returns
The value, or None if value is None

Raises
InvalidParameterValue if the value is not a valid UUID or logical name

ironic.common.args.**validate**(**args*, ***kwargs*)

Decorator which validates and transforms function arguments

ironic.common.async_steps module

ironic.common.async_steps.**get_return_state**(*node*)

Returns state based on operation being invoked.

Parameters
node an ironic node object.

Returns
states.CLEANWAIT if cleaning operation in progress, or states.DEPLOYWAIT if deploy operation in progress, or states.SERVICEWAIT if servicing in progress.

ironic.common.async_steps.**prepare_node_for_next_step**(*node*, *step_type=None*)

Remove the flags responsible for the next step.

Cleans the polling and the skip-next step flags.

Parameters

- **node** A Node object
- **step_type** The type of steps to process: clean, service or deploy. If None, detected from the node.

Returns
The last value of the skip-next flag.

ironic.common.async_steps.**remove_node_flags**(*node*)

Remove all flags for the node.

Parameters
node A Node object

ironic.common.async_steps.**set_node_flags**(*node*, *reboot=None*, *skip_current_step=None*, *polling=None*, *step_type=None*)

Sets appropriate reboot flags in driver_internal_info based on operation

Parameters

- **node** an ironic node object.
- **reboot** Boolean value to set for nodes driver_internal_info flag cleaning_reboot, servicing_reboot or deployment_reboot based on the operation in progress. If it is None, corresponding reboot flag is not set in nodes driver_internal_info.
- **skip_current_step** Boolean value to set for nodes driver_internal_info flag skip_current_clean_step, skip_current_service_step or skip_current_deploy_step based on the operation in progress. If it is None, corresponding skip step flag is not set in nodes driver_internal_info.
- **polling** Boolean value to set for nodes driver_internal_info flag deployment_polling, servicing_polling or cleaning_polling. If it is None, the corresponding polling flag is not set in the nodes driver_internal_info. A polling flag is otherwise deleted from the nodes driver_internal_info.
- **step_type** The type of steps to process: clean, service or deploy. If None, detected from the node.

ironic.common.auth_basic module

class ironic.common.auth_basic.**BasicAuthMiddleware**(*app*, *auth_file*)

Bases: object

Middleware which performs HTTP basic authentication on requests

__call__(*env*, *start_response*)

Call self as a function.

format_exception(*e*)

ironic.common.auth_basic.**auth_entry**(*entry*, *password*)

Compare a password with a single user auth file entry

Param
entry: Line from auth user file to use for authentication

Param
password: Password encoded as bytes

Returns
A dictionary of WSGI environment values to append to the request

Raises
Unauthorized, if the entry doesnt match supplied password or if the entry is encrypted with a method other than bcrypt

ironic.common.auth_basic.**authenticate**(*auth_file*, *username*, *password*)

Finds username and password match in Apache style user auth file

The user auth file format is expected to comply with Apache documentation[1] however the bcrypt password digest is the *only* digest format supported.

[1] https://httpd.apache.org/docs/current/misc/password_encryptions.html

Param
auth_file: Path to user auth file

Param
username: Username to authenticate

Param
password: Password encoded as bytes

Returns
A dictionary of WSGI environment values to append to the request

Raises
Unauthorized, if no file entries match supplied username/password

ironic.common.auth_basic.**parse_entry**(*entry*)

Extrace the username and encrypted password from a user auth file entry

Param
entry: Line from auth user file to use for authentication

Returns
a tuple of username and encrypted password

Raises
ConfigInvalid if the password is not in the supported bcrypt format

ironic.common.auth_basic.**parse_header**(*env*)

Parse WSGI environment for Authorization header of type Basic

Param
env: WSGI environment to get header from

Returns
Token portion of the header value

Raises
Unauthorized, if header is missing or if the type is not Basic

ironic.common.auth_basic.**parse_token**(*token*)

Parse the token portion of the Authentication header value

Param
token: Token value from basic authorization header

Returns
tuple of username, password

Raises
Unauthorized, if username and password could not be parsed for any reason

ironic.common.auth_basic.**unauthorized**(*message=None*)

Raise an Unauthorized exception to prompt for basic authentication

Param
message: Optional message for esception

Raises
Unauthorized with WWW-Authenticate header set

ironic.common.auth_basic.**validate_auth_file**(*auth_file*)

Read the auth user file and validate its correctness

Param
auth_file: Path to user auth file

Raises
ConfigInvalid on validation error

ironic.common.automated_clean_methods module

Mapping of values for CONF.conductor.automated_clean_step_source, representing different possible sources for cleaning steps.

ironic.common.automated_clean_methods.**AUTOGENERATED = 'autogenerated'**

Use traditional Ironic method with automatically determined cleaning steps

ironic.common.automated_clean_methods.**HYBRID = 'hybrid'**

Use a runbook if configured; use autogenerated steps if not.

ironic.common.automated_clean_methods.**RUNBOOK = 'runbook'**

Use a runbook for automated cleaning; fail if no runbook is configured.

ironic.common.boot_devices module

Mapping of boot devices used when requesting the system to boot from an alternate device.

The options presented were based on the IPMItool chassis bootdev command. You can find the documentation at: http://linux.die.net/man/1/ipmitool

NOTE: This module does not include all the options from ipmitool because they dont make sense in the limited context of Ironic right now.

ironic.common.boot_devices.**BIOS = 'bios'**

Boot into BIOS setup

ironic.common.boot_devices.**CDROM = 'cdrom'**

Boot from CD/DVD

ironic.common.boot_devices.**DISK = 'disk'**

Boot from default Hard-drive

ironic.common.boot_devices.**FLOPPY = 'floppy'**

Boot from a floppy drive

ironic.common.boot_devices.**ISCSIBOOT = 'iscsiboot'**

Boot from iSCSI volume

ironic.common.boot_devices.**PXE = 'pxe'**

Boot from PXE boot

ironic.common.boot_devices.**SAFE = 'safe'**

Boot from default Hard-drive, request Safe Mode

ironic.common.boot_devices.**UEFIHTTP = 'uefihttp'**

Boot from a UEFI HTTP(s) URL

ironic.common.boot_devices.**VMEDIA_DEVICES = ['disk', 'cdrom', 'floppy']**

Devices that make sense for virtual media attachment.

ironic.common.boot_devices.**WANBOOT = 'wanboot'**

Boot from Wide Area Network

ironic.common.boot_modes module

Mapping of boot modes used when requesting the system to boot using alternative firmware interfaces.

The options presented were based on the Redfish protocol capabilities, specifically on the BootSourceOverrideMode property.

ironic.common.boot_modes.**LEGACY_BIOS = 'bios'**

Boot over legacy PC BIOS firmware interface

ironic.common.boot_modes.**UEFI = 'uefi'**

Boot over Unified Extensible Firmware Interface (UEFI) firmware interface

ironic.common.checksum_utils module

class ironic.common.checksum_utils.**TransferHelper**(*response*, *checksum_algo*, *expected_checksum*, *chunk_size=1048576*)

Bases: object

property bytes_transferred

Property value to return the number of bytes transferred.

property checksum_matches

Verifies the checksum matches and returns True/False.

property content_length

Property value to return the server indicated length.

ironic.common.checksum_utils.**compute_image_checksum**(*image_path*, *algorithm='md5'*)

Compute checksum by given image path and algorithm.

Parameters

- **image_path** The path to the file to undergo checksum calculation.
- **algorithm** The checksum algorithm to utilize. Defaults to md5 due to historical support reasons in Ironic.

Returns
The calculated checksum value.

Raises
ValueError when the checksum algorithm is not supported by the system.

ironic.common.checksum_utils.**get_checksum_and_algo**(*instance_info*)

Get and return the image checksum and algo.

Parameters
instance_info The node instance info, or newly updated/generated instance_info value.

Returns
A tuple containing two values, a checksum and algorithm, if available.

ironic.common.checksum_utils.**get_checksum_from_url**(*checksum*, *image_source*)

Gets a checksum value based upon a remote checksum URL file.

Parameters

- **checksum** The URL to the checksum URL content.
- **image_soource** The image source utilized to match with the contents of the URL payload file.

Raises
ImageDownloadFailed when the checksum file cannot be accessed or cannot be parsed.

ironic.common.checksum_utils.**is_checksum_url**(*checksum*)

Identify if checksum is not a url.

Parameters
checksum The user supplied checksum value.

Returns
True if the checksum is a url, otherwise False.

Raises
ImageChecksumURLNotSupported should the conductor have this support disabled.

ironic.common.checksum_utils.**validate_checksum**(*path*, *checksum*, *checksum_algo=None*)

Validate image checksum.

Parameters

- **path** File path in the form of a string to calculate a checksum which is compared to the checksum field.
- **checksum** The supplied checksum value, a string, which will be compared to the file.
- **checksum_algo** The checksum type of the algorithm.

Raises
ImageChecksumError if the supplied data cannot be parsed or if the supplied value does not match the supplied checksum value.

ironic.common.checksum_utils.**validate_text_checksum**(*payload*, *digest*)

Compares the checksum of a payload versus the digest.

The purpose of this method is to take the payload string data, and compare it to the digest value of the supplied input. The use of this is to validate the the data in cases where we have data and need to compare it. Useful in API responses, such as those from an OCI Container Registry.

Parameters

- **payload** The supplied string with an encode method.
- **digest** The checksum value in digest form of algorithm:checksum.

Raises

ImageChecksumError when the response payload does not match the supplied digest.

ironic.common.cinder module

ironic.common.cinder.**attach_volumes**(*task*, *volume_list*, *connector*)

Attach volumes to a node.

Enumerate through the provided list of volumes and attach the volumes to the node defined in the task utilizing the provided connector information.

If an attachment appears to already exist, we will skip attempting to attach the volume. If use of the volume fails, a user may need to remove any lingering pre-existing/unused attachment records since we have no way to validate if the connector profile data differs from what was provided to cinder.

Parameters

- **task** TaskManager instance representing the operation.
- **volume_list** List of volume_id UUID values representing volumes.
- **connector** Dictionary object representing the node sufficiently to attach a volume. This value can vary based upon the nodes configuration, capability, and ultimately the back-end storage driver. As cinder was designed around iSCSI, the ip and initiator keys are generally expected by cinder drivers. For FiberChannel, the key wwpns can be used with a list of port addresses. Some drivers support a multipath boolean key, although it is generally False. The host key is generally used for logging by drivers. Example:

```
{
'wwpns': ['list','of','port','wwns'],
'ip': 'ip address',
'initiator': 'initiator iqn',
'multipath': False,
'host': 'hostname',
}
```

Raises

StorageError If storage subsystem exception is raised.

Returns

List of connected volumes, including volumes that were already connected to desired nodes. The returned list can be relatively consistent depending on the end storage driver that the volume is configured for, however the driver_volume_type key should not be relied upon as it is a free-form value returned by the driver. The accompanying data key contains the actual target details which will indicate either target WWNs and a LUN or a target portal and IQN. It also always contains volume ID in cinder and ironic. Except for these two IDs, each driver may return somewhat different data although the same keys are used if the target is FC or iSCSI, so any logic should be based upon the returned contents. For already attached volumes, the structure contains already_attached: True key-value pair. In such case, connection info for the node is already in the database, data structure contains only basic info of volume ID in cinder and ironic, so any logic based on that should retrieve it from the database. Example:

```
[{
'driver_volume_type': 'fibre_channel'
'data': {
    'encrypted': False,
    'target_lun': 1,
    'target_wwn': ['1234567890123', '1234567890124'],
    'volume_id': '00000000-0000-0000-0000-000000000001',
    'ironic_volume_id':
    '11111111-0000-0000-0000-000000000001'}
},
{
'driver_volume_type': 'iscsi'
'data': {
    'target_iqn': 'iqn.2010-10.org.openstack:volume-000002',
    'target_portal': '127.0.0.0.1:3260',
    'volume_id': '00000000-0000-0000-0000-000000000002',
    'ironic_volume_id':
    '11111111-0000-0000-0000-000000000002',
    'target_lun': 2}
},
{
'already_attached': True
'data': {
    'volume_id': '00000000-0000-0000-0000-000000000002',
    'ironic_volume_id':
    '11111111-0000-0000-0000-000000000002'}
}]
```

ironic.common.cinder.**detach_volumes**(*task*, *volume_list*, *connector*, *allow_errors=False*)

Detach a list of volumes from a provided connector detail.

Enumerates through a provided list of volumes and issues detachment requests utilizing the connector information that describes the node.

Parameters

- **task** The TaskManager task representing the request.
- **volume_list** The list of volume id values to detach.

- **connector** Dictionary object representing the node sufficiently to attach a volume. This value can vary based upon the nodes configuration, capability, and ultimately the back-end storage driver. As cinder was designed around iSCSI, the ip and initiator keys are generally expected. For FiberChannel, the key wwpns can be used with a list of port addresses. Some drivers support a multipath boolean key, although it is generally False. The host key is generally used for logging by drivers. Example:

```
{
'wwpns': ['list','of','port','wwns']
'ip': 'ip address',
'initiator': 'initiator iqn',
'multipath': False,
'host': 'hostname'
}
```

- **allow_errors** Boolean value governing if errors that are returned are treated as warnings instead of exceptions. Default False.

Raises

StorageError

ironic.common.cinder.**get_client**(*context=None*, *auth_from_config=False*)

Retrieve a cinder client connection.

Parameters

- **context** request context, instance of ironic.common.context.RequestContext
- **auth_from_config** (boolean) When True, use auth values from conf parameters

Returns

A cinder client.

ironic.common.cinder.**is_volume_attached**(*node*, *volume*)

Check if a volume is attached to the supplied node.

Parameters

- **node** The object representing the node.
- **volume** The object representing the volume from cinder.

Returns

Boolean indicating if the volume is attached. Returns True if cinder shows the volume as presently attached, otherwise returns False.

ironic.common.cinder.**is_volume_available**(*volume*)

Check if a volume is available for a connection.

Parameters

volume The object representing the volume.

Returns

Boolean if volume is available.

ironic.common.components module

Mapping of common hardware components of a computer system.

ironic.common.components.**CHASSIS = 'chassis'**

Chassis enclosing one or more hardware components

ironic.common.components.**DISK = 'disk'**

Storage drive

ironic.common.components.**NIC = 'nic'**

Network interface

ironic.common.components.**POWER = 'power'**

Power supply unit

ironic.common.components.**SYSTEM = 'system'**

Computing system

ironic.common.config module

ironic.common.config.**parse_args**(*argv*, *default_config_files=None*)

ironic.common.console_factory module

class ironic.common.console_factory.**ConsoleContainerFactory**(***kwargs*)

Bases: object

property provider

ironic.common.context module

class ironic.common.context.**RequestContext**(*is_public_api=False*, *auth_token_info=None*, ***kwargs*)

Bases: RequestContext

Extends security contexts from the oslo.context library.

FROM_DICT_EXTRA_KEYS: list[str] = ['auth_token_info']

ensure_thread_contain_context()

Ensure threading contains context

For async/periodic tasks, the context of local thread is missing. Set it with request context and this is useful to log the request_id in log messages.

classmethod from_environ(*environ*, ***kwargs*)

Load a context object from a request environment.

If keyword arguments are provided then they override the values in the request environment, injecting the kwarg arguments used by ironic, as unknown values are filtered out from the final context object in the base oslo.context library.

Parameters

environ (*dict*) The environment dictionary associated with a request.

to_dict()

Return a dictionary of context attributes.

to_policy_values()

A dictionary of context attributes to enforce policy with.

oslo.policy enforcement requires a dictionary of attributes representing the current logged in user on which it applies policy enforcement. This dictionary defines a standard list of attributes that should be available for enforcement across services.

It is expected that services will often have to override this method with either deprecated values or additional attributes used by that service specific policy.

ironic.common.context.**get_admin_context**()

Create an administrator context.

ironic.common.dhcp_factory module

class ironic.common.dhcp_factory.**DHCPFactory**(***kwargs*)

Bases: object

clean_dhcp(*task*)

Clean up the DHCP BOOT options for this node.

Parameters

task A TaskManager instance.

property provider

update_dhcp(*task*, *dhcp_opts*, *ports=None*)

Send or update the DHCP BOOT options for this node.

Parameters

- **task** A TaskManager instance.
- **dhcp_opts** this will be a list of dicts, e.g.

```
[{'opt_name': '67',
  'opt_value': 'pxelinux.0'},
 {'opt_name': '66',
  'opt_value': '123.123.123.456'}]
```

- **ports** A dict with keys ports and portgroups and dicts as values. Each dict has key/value pairs of the form <ironic UUID>:<neutron port UUID>. e.g.

```
{'ports': {'port.uuid': vif.id},
 'portgroups': {'portgroup.uuid': vif.id}}
```

If the value is None, will get the list of ports/portgroups from the Ironic port/portgroup objects.

ironic.common.driver_factory module

class ironic.common.driver_factory.**BaseDriverFactory**

Bases: object

Discover, load and manage the drivers available.

This is subclassed to load both main drivers and extra interfaces.

get_driver(*name*)

Get a driver instance by name.

Parameters
name Name of the driver to retrieve.

Returns
Driver instance.

Raises
DriverNotFound if driver is not found.

Raises
DriverLoadError if drivers not initialized when required.

get_driver_classes()

Get driver classes without invoking them.

This is useful when drivers need to be loaded but not instantiated, such as when configuration must be prepared before driver initialization.

Returns
Dictionary mapping driver name to driver class.

classmethod initialize_drivers()

Initialize all loaded driver classes.

This method is used for two-phase initialization when drivers are loaded with invoke_on_load=False. Call this after any necessary configuration preprocessing has been completed.

This method is idempotent - calling it multiple times will only initialize drivers once.

Raises
DriverLoadError if driver initialization fails.

items()

Iterator over pairs (name, instance).

property names

The list of driver names available.

class ironic.common.driver_factory.**HardwareTypesFactory**

Bases: *BaseDriverFactory*

class ironic.common.driver_factory.**InterfaceFactory**

Bases: *BaseDriverFactory*

class ironic.common.driver_factory.**NetworkInterfaceFactory**

Bases: *InterfaceFactory*

class ironic.common.driver_factory.**StorageInterfaceFactory**

Bases: *InterfaceFactory*

ironic.common.driver_factory.**all_interfaces**()

Get all interfaces for all interface types.

Returns
Dictionary mapping interface type to dictionary mapping interface name to interface object.

ironic.common.driver_factory.**build_driver_for_task**(*task*)

Builds a composable driver for a given task.

Starts with a *BareDriver* object, and attaches implementations of the various driver interfaces to it. They come from separate driver factories and are configurable via the database.

Parameters
task The task containing the node to build a driver for.

Returns
A driver object for the task.

Raises
DriverNotFound if node.driver could not be found in the ironic.hardware.types namespaces.

Raises
InterfaceNotFoundInEntrypoint if some node interfaces are set to invalid or unsupported values.

Raises
IncompatibleInterface the requested implementation is not compatible with it with the hardware type.

ironic.common.driver_factory.**check_and_update_node_interfaces**(*node*, *hw_type=None*)

Ensure that node interfaces (e.g. for creation or updating) are valid.

Updates (but doesnt save to the database) hardware interfaces with calculated defaults, if they are not provided.

This function is run on node updating and creation, as well as each time a driver instance is built for a node.

Parameters

- **node** node object to check and potentially update
- **hw_type** hardware type instance object; will be detected from node.driver if missing

Returns
True if any changes were made to the node, otherwise False

Raises
InterfaceNotFoundInEntrypoint on validation failure

Raises
NoValidDefaultForInterface if the default value cannot be calculated and is not provided in the configuration

Raises
DriverNotFound if the nodes hardware type is not found

ironic.common.driver_factory.**default_interface**(*hw_type*, *interface_type*, *driver_name=None*, *node=None*)

Calculate and return the default interface implementation.

Finds the first implementation that is supported by the hardware type and is enabled in the configuration.

Parameters

- **hw_type** hardware type instance object.
- **interface_type** type of the interface (e.g. boot).
- **driver_name** entrypoint name of the hw_type object. Is used for exception message.
- **node** the identifier of a node. If specified, is used for exception message.

Returns
an entrypoint name of the calculated default implementation.

Raises
InterfaceNotFoundInEntrypoint if the entry point was not found.

Raises
NoValidDefaultForInterface if no default interface can be found.

ironic.common.driver_factory.**enabled_supported_interfaces**(*hardware_type*)

Get usable interfaces for a given hardware type.

For a given hardware type, find the intersection of enabled and supported interfaces for each interface type. This is the set of interfaces that are usable for this hardware type.

Parameters
hardware_type The hardware type object to search.

Returns
a dict mapping interface types to a list of enabled and supported interface names.

ironic.common.driver_factory.**get_hardware_type**(*hardware_type*)

Get a hardware type instance by name.

Parameters
hardware_type the name of the hardware type to find

Returns
An instance of ironic.drivers.hardware_type.AbstractHardwareType

Raises
DriverNotFound if requested hardware type cannot be found

ironic.common.driver_factory.**get_interface**(*hw_type*, *interface_type*, *interface_name*)

Get interface implementation instance.

For hardware types also validates compatibility.

Parameters

- **hw_type** a hardware type instance.
- **interface_type** name of the interface type (e.g. boot).
- **interface_name** name of the interface implementation from an appropriate entry point (ironic.hardware.interfaces.<interface type>).

Returns
instance of the requested interface implementation.

Raises
InterfaceNotFoundInEntrypoint if the entry point was not found.

Raises
IncompatibleInterface if hw_type is a hardware type and the requested implementation is not compatible with it.

ironic.common.driver_factory.**hardware_types**()

Get all hardware types.

Returns
Dictionary mapping hardware type name to hardware type object.

ironic.common.driver_factory.**interfaces**(*interface_type*)

Get all interfaces for a given interface type.

Parameters
interface_type String, type of interface to fetch for.

Returns
Dictionary mapping interface name to interface object.

ironic.common.exception module

Ironic specific exceptions list.

exception ironic.common.exception.**AgentAPIError**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**AgentCommandTimeout**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**AgentConnectionFailed**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**AgentInProgress**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**AllocationAlreadyExists**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**AllocationDuplicateName**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**AllocationFailed**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**AllocationNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**BIOSSettingAlreadyExists**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**BIOSSettingListNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**BIOSSettingNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

ironic.common.exception.**BadRequest**

alias of *Invalid*

exception ironic.common.exception.**BootModeNotAllowed**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**CatalogNotFound**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**ChassisAlreadyExists**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**ChassisNotEmpty**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**ChassisNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**ChildNodeLocked**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**ClientSideError**(*msg=None*, *status_code=400*, *faultcode='Client'*)

Bases: RuntimeError

property faultstring

exception ironic.common.exception.**CommunicationError**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**ConcurrentActionLimit**(*message=None*, ***kwargs*)

Bases: *TemporaryFailure*

exception ironic.common.exception.**ConductorAlreadyRegistered**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**ConductorHardwareInterfacesAlreadyRegistered**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**ConductorNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**ConfigDriveRegenerationFailure**(*message=None*, ***kwargs*)

Bases: *IronicException*

Raised when we fail to handle configuration drive corrections.

exception ironic.common.exception.**ConfigInvalid**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**ConfigNotFound**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**Conflict**(*message=None*, ***kwargs*)

Bases: *IronicException*

code = 409

exception ironic.common.exception.**ConsoleContainerError**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**ConsoleError**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**ConsoleSubprocessFailed**(*message=None*, ***kwargs*)

Bases: *ConsoleError*

exception ironic.common.exception.**DHCPLoadError**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**DatabaseVersionTooOld**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**DeployTemplateAlreadyExists**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**DeployTemplateDuplicateName**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**DeployTemplateNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**DirectoryNotWritable**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**DracOperationError**(*message=None*, ***kwargs*)

Bases: *DriverOperationError*

exception ironic.common.exception.**DriverLoadError**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**DriverNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**DriverNotFoundInEntrypoint**(*message=None*, ***kwargs*)

Bases: *DriverNotFound*

exception ironic.common.exception.**DriverOperationError**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**Duplicate**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**DuplicateName**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**DuplicateNodeOnLookup**(*message=None*, ***kwargs*)

Bases: *NodeNotFound*

exception ironic.common.exception.**ExclusiveLockRequired**(*message=None*, ***kwargs*)

Bases: *NotAuthorized*

exception ironic.common.exception.**FailedToCleanDHCPOpts**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**FailedToGetIPAddressOnPort**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**FailedToGetSensorData**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**FailedToParseSensorData**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**FailedToUpdateDHCPOptOnPort**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**FailedToUpdateMacOnPort**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**FileSystemNotSupported**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**FirmwareComponentAlreadyExists**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**FirmwareComponentNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**GlanceConnectionFailed**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**HTTPForbidden**(*message=None*, ***kwargs*)

Bases: *NotAuthorized*

ironic.common.exception.**HTTPNotFound**

alias of *NotFound*

exception ironic.common.exception.**HardwareInspectionFailure**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**IPMIFailure**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**IloOperationError**(*message=None*, ***kwargs*)

Bases: *DriverOperationError*

exception ironic.common.exception.**IloOperationNotSupported**(*message=None*, ***kwargs*)

Bases: *DriverOperationError*

exception ironic.common.exception.**ImageChecksumAlgorithmFailure**(*message=None*, ***kwargs*)

Bases: *InvalidImage*

Cannot load the requested or required checksum algorithm.

exception ironic.common.exception.**ImageChecksumError**(*message=None*, ***kwargs*)

Bases: *InvalidImage*

Exception indicating checksum failed to match.

exception ironic.common.exception.**ImageChecksumFileReadFailure**(*message=None*, ***kwargs*)

Bases: *InvalidImage*

An OSError was raised when trying to read the file.

code = 503

exception ironic.common.exception.**ImageChecksumURLNotSupported**(*message=None*, ***kwargs*)

Bases: *InvalidImage*

Exception indicating we cannot support the remote checksum file.

exception ironic.common.exception.**ImageConvertFailed**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**ImageCreationFailed**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**ImageDownloadFailed**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**ImageHostRateLimitFailure**(*message=None*, ***kwargs*)

Bases: *TemporaryFailure*

exception ironic.common.exception.**ImageMatchFailure**(*message=None*, ***kwargs*)

Bases: *InvalidImage*

exception ironic.common.exception.**ImageNotAuthorized**(*message=None*, ***kwargs*)

Bases: *NotAuthorized*

exception ironic.common.exception.**ImageNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**ImageRefValidationFailed**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**ImageServiceAuthenticationRequired**(*message=None*, ***kwargs*)

Bases: *ImageUnacceptable*

exception ironic.common.exception.**ImageUnacceptable**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**ImageUploadFailed**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**IncompatibleInterface**(*message=None*, ***kwargs*)

Bases: *InvalidParameterValue*

exception ironic.common.exception.**IncompleteLookup**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**IncorrectConfiguration**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**InputFileError**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**InspectionRuleAlreadyExists**(*message=None*, ***kwargs*)

Bases: *Conflict*

Rule requested already exists in the database.

exception ironic.common.exception.**InspectionRuleExecutionFailure**(*message=None*, ***kwargs*)

Bases: *HardwareInspectionFailure*

Raised when an inspection rule fails during execution.

exception ironic.common.exception.**InspectionRuleNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

The requested rule was not found.

exception ironic.common.exception.**InspectionRuleValidationFailure**(*message=None*, ***kwargs*)

Bases: *IronicException*

Inspection rule validation fails during creation or execution.

exception ironic.common.exception.**InstanceAssociated**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**InstanceDeployFailure**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**InstanceNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**InstanceRescueFailure**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**InstanceUnrescueFailure**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**InsufficientDiskSpace**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**InsufficientMemory**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**InterfaceNotFoundInEntrypoint**(*message=None*, ***kwargs*)

Bases: *InvalidParameterValue*

exception ironic.common.exception.**Invalid**(*message=None*, ***kwargs*)

Bases: *IronicException*

code = 400

exception ironic.common.exception.**InvalidConductorGroup**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**InvalidDatapathID**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**InvalidDeployTemplate**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**InvalidEndpoint**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**InvalidIPAddress**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**InvalidIPv4Address**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**InvalidIdentity**(*message=None*, ***kwargs*)
Bases: *Invalid*

exception ironic.common.exception.**InvalidImage**(*message=None*, ***kwargs*)
Bases: *ImageUnacceptable*

exception ironic.common.exception.**InvalidImageRef**(*message=None*, ***kwargs*)
Bases: *InvalidParameterValue*

exception ironic.common.exception.**InvalidInput**(*fieldname*, *value*, *msg=''*)
Bases: *ClientSideError*

property faultstring

exception ironic.common.exception.**InvalidKickstartFile**(*message=None*, ***kwargs*)
Bases: *Invalid*

exception ironic.common.exception.**InvalidKickstartTemplate**(*message=None*, ***kwargs*)
Bases: *Invalid*

exception ironic.common.exception.**InvalidMAC**(*message=None*, ***kwargs*)
Bases: *Invalid*

exception ironic.common.exception.**InvalidMetricConfig**(*message=None*, ***kwargs*)
Bases: *Invalid*

Error raised when a metric config is invalid.

message = 'Invalid value for metrics config option.'

exception ironic.common.exception.**InvalidName**(*message=None*, ***kwargs*)
Bases: *Invalid*

exception ironic.common.exception.**InvalidNodeInventory**(*message=None*, ***kwargs*)
Bases: *Invalid*

exception ironic.common.exception.**InvalidParameterValue**(*message=None*, ***kwargs*)
Bases: *Invalid*

exception ironic.common.exception.**InvalidRunbook**(*message=None*, ***kwargs*)
Bases: *Invalid*

exception ironic.common.exception.**InvalidState**(*message=None*, ***kwargs*)
Bases: *Conflict*

exception ironic.common.exception.**InvalidStateRequested**(*message=None*, ***kwargs*)
Bases: *Invalid*

exception ironic.common.exception.**InvalidSwitchID**(*message=None*, ***kwargs*)
Bases: *Invalid*

exception ironic.common.exception.**InvalidUUID**(*message=None*, ***kwargs*)
Bases: *Invalid*

exception ironic.common.exception.**InvalidUuidOrName**(*message=None*, ***kwargs*)
Bases: *Invalid*

exception ironic.common.exception.**IronicException**(*message=None*, ***kwargs*)

Bases: Exception

Base Ironic Exception

To correctly use this class, inherit from it and define a _msg_fmt property. That _msg_fmt will get printfd with the keyword arguments provided to the constructor.

If you need to access the message from an exception you should use str(exc)

code = 500

headers = {}

safe = False

exception ironic.common.exception.**KeystoneFailure**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**KeystoneUnauthorized**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**MACAlreadyExists**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**MalformedRequestURI**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**MetricsNotSupported**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**MissingParameterValue**(*message=None*, ***kwargs*)

Bases: *InvalidParameterValue*

exception ironic.common.exception.**NetworkError**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**NoConsolePid**(*message=None*, ***kwargs*)

Bases: *ConsoleError*

exception ironic.common.exception.**NoDriversLoaded**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**NoFreeConductorWorker**(*message=None*, ***kwargs*)

Bases: *TemporaryFailure*

code = 503

exception ironic.common.exception.**NoFreeIPMITerminalPorts**(*message=None*, ***kwargs*)

Bases: *TemporaryFailure*

exception ironic.common.exception.**NoFreePhysicalPorts**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**NoValidDefaultForInterface**(*message=None*, ***kwargs*)

Bases: *InvalidParameterValue*

exception ironic.common.exception.**NoValidHost**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**NodeAlreadyExists**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**NodeAssociated**(*message=None*, ***kwargs*)

Bases: *InvalidState*

exception ironic.common.exception.**NodeCleaningFailure**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**NodeConsoleNotEnabled**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**NodeHistoryNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**NodeInMaintenance**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**NodeInventoryAlreadyExists**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**NodeInventoryNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**NodeIsRetired**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**NodeLocked**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**NodeMaintenanceFailure**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**NodeNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**NodeNotLocked**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**NodeProtected**(*message=None*, ***kwargs*)

Bases: *HTTPForbidden*

exception ironic.common.exception.**NodeServicingFailure**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**NodeTagNotFound**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**NodeTraitNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**NodeVerifyFailure**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**NotAcceptable**(*message=None*, ***kwargs*)

Bases: *IronicException*

code = 406

exception ironic.common.exception.**NotAuthorized**(*message=None*, ***kwargs*)

Bases: *IronicException*

code = 403

exception ironic.common.exception.**NotFound**(*message=None*, ***kwargs*)

Bases: *IronicException*

code = 404

exception ironic.common.exception.**NotificationPayloadError**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**NotificationSchemaKeyError**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**NotificationSchemaObjectError**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**OciImageNotSpecific**(*message=None*, ***kwargs*)

Bases: *InvalidImage*

exception ironic.common.exception.**OciImageTagNotFound**(*message=None*, ***kwargs*)

Bases: *ImageNotFound*

exception ironic.common.exception.**OperationNotPermitted**(*message=None*, ***kwargs*)

Bases: *NotAuthorized*

exception ironic.common.exception.**ParentNodeLocked**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**PasswordFileFailedToCreate**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**PatchError**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**PathNotFound**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**PortAlreadyExists**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**PortDuplicateName**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**PortNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**PortgroupAlreadyExists**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**PortgroupDuplicateName**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**PortgroupMACAlreadyExists**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**PortgroupNotEmpty**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**PortgroupNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**PortgroupPhysnetInconsistent**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**PowerStateFailure**(*message=None*, ***kwargs*)

Bases: *InvalidState*

exception ironic.common.exception.**RFBAuthHandshakeFailed**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**RFBAuthNoAvailableScheme**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**RedfishConnectionError**(*message=None*, ***kwargs*)

Bases: *RedfishError*

exception ironic.common.exception.**RedfishError**(*message=None*, ***kwargs*)

Bases: *DriverOperationError*

exception ironic.common.exception.**RuleActionExecutionFailure**(*message=None*, ***kwargs*)

Bases: *InspectionRuleExecutionFailure*

Raised when an inspection rule action fails during execution.

exception ironic.common.exception.**RuleConditionCheckFailure**(*message=None*, ***kwargs*)

Bases: *InspectionRuleExecutionFailure*

Raised when an inspection rule condition fails during execution.

exception ironic.common.exception.**RunbookAlreadyExists**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**RunbookDuplicateName**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**RunbookNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**SNMPFailure**(*message=None*, ***kwargs*)

Bases: *DriverOperationError*

exception ironic.common.exception.**SecurityProxyNegotiationFailed**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**ServiceLookupFailure**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**ServiceRegistrationFailure**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**ServiceUnavailable**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**StorageError**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**SwiftObjectNotFoundError**(*message=None*, ***kwargs*)

Bases: *SwiftOperationError*

exception ironic.common.exception.**SwiftObjectStillExists**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**SwiftOperationError**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**SwitchNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**TBNAttributeRetrievalException**(*message=None*, ***kwargs*)

Bases: *TraitBasedNetworkingException*

Specified attribute could not be found.

exception ironic.common.exception.**TBNComparatorPrefixMatchTypeMismatch**(*message=None*, ***kwargs*)

Bases: *TraitBasedNetworkingException*

Comparator prefix match can only be used against a string.

exception ironic.common.exception.**TBNNoPortMatchesFound**(*message=None*, ***kwargs*)

Bases: *TraitBasedNetworkingException*

No matching ports were found when planning the network for this node

exception ironic.common.exception.**TBNNoTraitsApplicableToNode**(*message=None*, ***kwargs*)

Bases: *TraitBasedNetworkingException*

If no traits apply to a node, then nothing can be done.

exception ironic.common.exception.**TemporaryFailure**(*message=None*, ***kwargs*)

Bases: *IronicException*

code = 503

exception ironic.common.exception.**TraitBasedNetworkingException**(*message=None*, ***kwargs*)

Bases: *IronicException*

Raised when theres an issue with trait based networking feature.

exception ironic.common.exception.**Unauthorized**(*message=None*, ***kwargs*)

Bases: *IronicException*

code = 401

headers = {'WWW-Authenticate': 'Basic realm="Baremetal API"'}

exception ironic.common.exception.**UnknownArgument**(*argname*, *msg=''*)

Bases: *ClientSideError*

property faultstring

exception ironic.common.exception.**UnknownAttribute**(*fieldname*, *attributes*, *msg=''*)

Bases: *ClientSideError*

add_fieldname(*name*)

Add a fieldname to concatenate the full name.

Add a fieldname so that the whole hierarchy is displayed. Successive calls to this method will prepend `name` to the hierarchy of names.

property faultstring

exception ironic.common.exception.**UnsupportedDriverExtension**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**UnsupportedHardwareFeature**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**VendorPassthruException**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.common.exception.**VifAlreadyAttached**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**VifInvalidForAttach**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**VifNotAttached**(*message=None*, ***kwargs*)

Bases: *Invalid*

exception ironic.common.exception.**VolumeConnectorAlreadyExists**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**VolumeConnectorNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

exception ironic.common.exception.**VolumeConnectorTypeAndIdAlreadyExists**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**VolumeTargetAlreadyExists**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**VolumeTargetBootIndexAlreadyExists**(*message=None*, ***kwargs*)

Bases: *Conflict*

exception ironic.common.exception.**VolumeTargetNotFound**(*message=None*, ***kwargs*)

Bases: *NotFound*

ironic.common.faults module

Fault definitions.

ironic.common.faults.**CLEAN_FAILURE = 'clean failure'**

Node is moved to maintenance due to failure of a cleaning operation.

ironic.common.faults.**POWER_FAILURE = 'power failure'**

Node is moved to maintenance due to power synchronization failure.

ironic.common.faults.**RESCUE_ABORT_FAILURE = 'rescue abort failure'**

Node is moved to maintenance due to failure of cleaning up during rescue abort.

ironic.common.faults.**SERVICE_FAILURE = 'service failure'**

Node is moved to maintenance due to failure of a service operation.

ironic.common.fsm module

class ironic.common.fsm.**FSM**

Bases: FiniteMachine

An ironic state-machine class with some ironic specific additions.

add_state(*state*, *on_enter=None*, *on_exit=None*, *target=None*, *terminal=None*, *stable=False*)

Adds a given state to the state machine.

Parameters

- **stable** Use this to specify that this state is a stable/passive state. A state must have been previously defined as stable before it can be used as a target
- **target** The target state for state to go to. Before a state can be used as a target it must have been previously added and specified as stable

Further arguments are interpreted as for parent method `add_state`.

add_transition(*start*, *end*, *event*, *replace=False*)

Adds an allowed transition from start -> end for the given event.

Parameters

- **start** starting state
- **end** ending state
- **event** event that causes start state to transition to end state
- **replace** replace existing event instead of raising a `Duplicate` exception when the transition already exists.

initialize(*start_state=None*, *target_state=None*)

Initialize the FSM.

Parameters

- **start_state** the FSM is initialized to start from this state
- **target_state** if specified, the FSM is initialized to this target state. Otherwise use the default target state

is_stable(*state*)

Is the state stable?

Parameters
state the state of interest

Raises
InvalidState if the state is invalid

Returns
True if it is a stable state; False otherwise

process_event(*event*, *target_state=None*)

process the event.

Parameters

- **event** the event to be processed
- **target_state** if specified, the final target state for the event. Otherwise, use the default target state

property target_state

ironic.common.hash_ring module

class ironic.common.hash_ring.**HashRingManager**(*use_groups=True*, *cache=True*)

Bases: object

get_ring(*driver_name*, *conductor_group*)

classmethod reset()

property ring

ironic.common.health_states module

Mapping of hardware health states.

These states represent the overall health condition of the server hardware, as reported by the BMC. Drivers should convert vendor-specific health values to these Ironic-level constants.

class ironic.common.health_states.**HealthState**(*value*, *names=<not given>*, **values*, *module=None*, *qualname=None*, *type=None*, *start=1*, *boundary=None*)

Bases: Enum

Hardware health states reported by BMC.

CRITICAL = 'Critical'

Hardware has critical issues requiring immediate attention.

OK = 'OK'

Hardware is functioning normally with no issues detected.

WARNING = 'Warning'

Hardware has non-critical issues that may require attention.

ironic.common.i18n module

oslo.i18n integration module.

See https://docs.openstack.org/oslo.i18n/latest/user/

ironic.common.image_publisher module

class ironic.common.image_publisher.**AbstractPublisher**

Bases: object

Abstract base class for publishing images.

abstract publish(*source_path*, *file_name=None*)

Publish an image.

Parameters

- **source_path** Path to the source file.
- **file_name** Destination file name. If None, the file component of source_path is used.

Returns
The URL of the published image.

abstract unpublish(*file_name*)

Unpublish the image.

Parameters
file_name File name to unpublish.

class ironic.common.image_publisher.**CIFSPublisher**

Bases: *AbstractPublisher*

Image publisher using a CIFS/SMB share.

get_credentials()

Return credentials for CIFS authentication.

Returns
tuple of (username, password) or (None, None) if no credentials configured.

publish(*source_path*, *file_name=None*)

Publish an image.

Parameters

- **source_path** Path to the source file.
- **file_name** Destination file name. If None, the file component of source_path is used.

Returns
The URL of the published image.

unpublish(*file_name*)

Unpublish the image.

Parameters
file_name File name to unpublish.

class ironic.common.image_publisher.**LocalPublisher**(*image_subdir=None*, *file_permission=420*, *dir_permission=493*, *root_url=None*)

Bases: *AbstractPublisher*

Image publisher using a local web server.

publish(*source_path*, *file_name=None*)

Publish an image.

Parameters

- **source_path** Path to the source file.
- **file_name** Destination file name. If None, the file component of source_path is used.

Returns
The URL of the published image.

unpublish(*file_name*)

Unpublish the image.

Parameters

file_name File name to unpublish.

class ironic.common.image_publisher.**NFSPublisher**

Bases: *AbstractPublisher*

Image publisher using an NFS share.

publish(*source_path*, *file_name=None*)

Publish an image.

Parameters

- **source_path** Path to the source file.
- **file_name** Destination file name. If None, the file component of source_path is used.

Returns

The URL of the published image.

unpublish(*file_name*)

Unpublish the image.

Parameters

file_name File name to unpublish.

class ironic.common.image_publisher.**SwiftPublisher**(*container*, *delete_after*)

Bases: *AbstractPublisher*

Image publisher using OpenStack Swift.

publish(*source_path*, *file_name=None*)

Publish an image.

Parameters

- **source_path** Path to the source file.
- **file_name** Destination file name. If None, the file component of source_path is used.

Returns

The URL of the published image.

unpublish(*file_name*)

Unpublish the image.

Parameters

file_name File name to unpublish.

ironic.common.image_service module

class ironic.common.image_service.**BaseImageService**

Bases: `object`

Provides retrieval of disk images.

abstract download(*image_href*, *image_file*, *checksum=None*, *checksum_algo=None*)

Downloads image to specified location.

Parameters

- **image_href** Image reference.
- **image_file** File object to write data to.
- **checksum** Expected checksum value for validation during transfer.
- **checksum_algo** Algorithm for checksum (e.g., md5, sha256).

Raises

exception.ImageRefValidationFailed.

Raises

exception.ImageDownloadFailed.

property is_auth_set_needed

Property to notify the caller if it needs to set authentication.

abstract show(*image_href*)

Get dictionary of image properties.

Parameters

image_href Image reference.

Raises

exception.ImageRefValidationFailed.

Returns

dictionary of image properties. It has three of them: size, updated_at and properties. updated_at attribute is a naive UTC datetime object.

property transfer_verified_checksum

The transferred artifact checksum.

abstract validate_href(*image_href*)

Validate image reference.

Parameters

image_href Image reference.

Raises

exception.ImageRefValidationFailed.

Returns

Information needed to further operate with an image.

class ironic.common.image_service.**CifsImageService**

Bases: *BaseImageService*

Provides validation of CIFS/SMB image URLs.

download(*image_href*, *image_file*)

Download an image from CIFS.

Parameters

- **image_href** CIFS URL of the image.

- **image_file** File object to write the image to.

Raises
exception.ImageRefValidationFailed if CIFS URL is invalid.

Raises
NotImplementedError as the conductor does not download CIFS files. The conductor builds ISO images locally, copies them to the mounted CIFS share, and the BMC accesses files directly via CIFS.

show(*image_href*)

Get image metadata from CIFS URL.

Parameters
image_href CIFS URL of the image.

Raises
exception.ImageRefValidationFailed if CIFS URL is invalid.

Returns
Dictionary with image metadata.

validate_href(*image_href*, *secret=False*)

Validate CIFS image reference.

Parameters

- **image_href** Image reference.
- **secret** Specify if image_href being validated should not be shown in exception message.

Raises
exception.ImageRefValidationFailed if CIFS URL is malformed.

Returns
None

class ironic.common.image_service.**FileImageService**

Bases: *BaseImageService*

Provides retrieval of disk images available locally on the conductor.

download(*image_href*, *image_file*, *checksum=None*, *checksum_algo=None*)

Downloads image to specified location.

Parameters

- **image_href** Image reference.
- **image_file** File object to write data to.
- **checksum** Expected checksum value for validation during transfer. (Currently unused, for API compatibility.)
- **checksum_algo** Algorithm for checksum (e.g., md5, sha256). (Currently unused, for API compatibility.)

Raises
exception.ImageRefValidationFailed if source image file doesnt exist.

Raises
exception.ImageDownloadFailed if exceptions were raised while writing to file or creating hard link.

show(*image_href*)

Get dictionary of image properties.

Parameters
image_href Image reference.

Raises
exception.ImageRefValidationFailed if image file specified doesnt exist.

Returns
dictionary of image properties. It has three of them: size, updated_at and properties. updated_at attribute is a naive UTC datetime object.

validate_href(*image_href*)

Validate local image reference.

Parameters
image_href Image reference.

Raises
exception.ImageRefValidationFailed if source image file doesnt exist, is in a blocked path, or is not in an allowed path.

Returns
Path to image file if it exists and is allowed.

class ironic.common.image_service.**HttpImageService**

Bases: *BaseImageService*

Provides retrieval of disk images using HTTP.

download(*image_href*, *image_file*, *checksum=None*, *checksum_algo=None*)

Downloads image to specified location.

Parameters

- **image_href** Image reference.
- **image_file** File object to write data to.
- **checksum** Expected checksum value for validation during transfer.
- **checksum_algo** Algorithm for checksum (e.g., md5, sha256).

Raises
exception.ImageRefValidationFailed if GET request returned response code not equal to 200.

Raises
exception.ImageDownloadFailed if: * IOError happened during file write; * GET request failed.

Raises
exception.ImageChecksumError if checksum validation fails.

static gen_auth_from_conf_user_pass(*image_href*)

This function is used to pass the credentials to the chosen

credential verifier and in case the verification is successful generate the compatible authentication object that will be used with the request(s). This function handles the authentication object generation for authentication strategies that are username+password based. Credentials are collected from the oslo.config framework.

Parameters
image_href href of the image that is being acted upon

Returns
Authentication object used directly by the request library

Return type
requests.auth.HTTPBasicAuth

static get(*image_href*)

Downloads content and returns the response text.

Parameters
image_href Image reference.

Raises
exception.ImageRefValidationFailed if GET request returned response code not equal to 200.

Raises
exception.ImageDownloadFailed if: * IOError happened during file write; * GET request failed.

show(*image_href*)

Get dictionary of image properties.

Parameters
image_href Image reference.

Raises
exception.ImageRefValidationFailed if: * HEAD request failed; * HEAD request returned response code not equal to 200; * Content-Length header not found in response to HEAD request.

Returns
dictionary of image properties. It has three of them: size, updated_at and properties. updated_at attribute is a naive UTC datetime object.

property transfer_verified_checksum

The transferred artifact checksum.

validate_href(*image_href*, *secret=False*)

Validate HTTP image reference.

Parameters

- **image_href** Image reference.
- **secret** Specify if image_href being validated should not be shown in exception message.

Raises
exception.ImageRefValidationFailed if HEAD request failed or returned response code not equal to 200.

Returns
Response to HEAD request.

static verify_basic_auth_cred_format(*image_href*, *user=None*, *password=None*)

Verify basic auth credentials used for image head request.

Parameters

- **user** auth username
- **password** auth password

Raises
exception.ImageRefValidationFailed if the credentials are not present

class ironic.common.image_service.**NfsImageService**

Bases: *BaseImageService*

Provides retrieval of disk images using NFS.

download(*image_href*, *image_file*)

Download an image from NFS.

Parameters

- **image_href** NFS URL of the image.
- **image_file** File object to write the image to.

Raises
exception.ImageRefValidationFailed if NFS URL is invalid.

Raises
NotImplementedError as the conductor does not download NFS files. The conductor builds ISO images locally, copies them to the mounted NFS share, and the BMC accesses files directly via NFS.

show(*image_href*)

Get image metadata from NFS URL.

Parameters
image_href NFS URL of the image.

Raises
exception.ImageRefValidationFailed if NFS URL is invalid.

Returns
Dictionary with image metadata.

validate_href(*image_href*, *secret=False*)

Validate NFS image reference.

Parameters

- **image_href** Image reference.

- **secret** Specify if image_href being validated should not be shown in exception message.

Raises
exception.ImageRefValidationFailed if NFS URL is malformed.

Returns
None

class ironic.common.image_service.**OciImageService**

Bases: *BaseImageService*

Image Service class for accessing an OCI Container Registry.

download(*image_href*, *image_file*, *checksum=None*, *checksum_algo=None*)

Downloads image to specified location.

Parameters

- **image_href** Image reference.
- **image_file** File object to write data to.
- **checksum** Expected checksum value for validation during transfer. (Currently unused, OCI validates via manifest digest.)
- **checksum_algo** Algorithm for checksum (e.g., md5, sha256). (Currently unused, OCI validates via manifest digest.)

Raises
exception.ImageRefValidationFailed.

Raises
exception.ImageDownloadFailed.

Raises
exception.OciImageNotSpecific.

identify_specific_image(*image_href*, *image_download_source=None*, *cpu_arch=None*)

Identify a specific OCI Registry Artifact.

This method supports the caller, but is located in the image service code to provide it access to the Container Registry client code which holds the lower level methods.

The purpose of this method is to take the user requested image_href and identify the best matching artifact attached to a container registrys entry. This is because the container registry can contain many artifacts which can be distributed and allocated by different types. To achieve this goal, this method utilizes the image_download_source to weight the preference of type of file to look for, and the CPU architecture to enable support for mutli-arch container registries.

In order to inform the caller about the url, as well as related data, such as the manifest which points to the artifact, artifact digest, known original filename of the artifact, this method returns a dictionary with several fields which may be useful to aid in understanding of what artifact was chosen.

Parameters

- **image_href** The image URL as supplied by the Ironic user.

- **image_download_source** The Ironic image_download_source value, defaults to None. When a value of local is provided, this method prefers selection of qcow images over raw images. Otherwise, raw images are the preference.
- **cpu_arch** The Bare Metal nodes defined CPU architecture, if any. Defaults to None. When used, a direct match is sought in the remote container registry. If x86_64 or amd64 is used, the code searches for the values in the remote registry interchangeably due to OCI data model standardizing on *amd64* as the default value for 64bit x86 Architectures.

Returns

A dictionary with multiple values to the caller to aid in returning the required HTTP URL, but also metadata about the selected artifact including size, filename, blob digest, related manifest digest, the remote recorded mediaType value, if the file appears compressed, if the file appears to be a raw disk image, any HTTP Authorization secret, if applicable, and the OCI image manifest URL. As needs could be different based upon different selection algorithms and evolving standards/approaches in use of OCI registries, the dictionary can also be empty, or contain different values and any caller should defensively use information as needed. If a record is *not* found, a empty dictionary is the result set. Under normal circumstances, the result looks something like this example. { image_url: https://fqdn/path, image_size: 1234567, image_filename: filename.raw.zstd, image_checksum: f00f, image_container_blob_digest: sha256:f00f, image_media_type: application/zstd, image_compression_type: zstd, image_disk_format: raw, image_request_authorization_secret: None, oci_image_manifest_url: https://fqdn/path@sha256:123f, }

property is_auth_set_needed

Property to notify the caller if it needs to set authentication.

is_oci_artifact_image(*manifest*)

Detect if the container image has an OCI Artifact

Parameters

image_href The manifest to detect from

Returns

True if the manifest config is empty or the manifest artifactType is set to an actual value

set_image_auth(*image_url*, *auth_data*)

Sets the supplied auth_data dictionary on the class for use later.

Provides a mechanism to inform the image service of specific credentials without wiring this in as a first class citizen in all image service interfaces.

Parameters

auth_data The authentication data dictionary holding username, password, or other authentication data which may be used by this client class.

Returns

None

Raises

AssertionError should this method be called twice in the same workflow.

show(*image_href*)

Get dictionary of image properties.

Parameters
image_href Image reference.

Raises
exception.ImageRefValidationFailed.

Raises
exception.OciImageNotSpecific.

Returns
dictionary of image properties. It has three of them: size, checksum, and digest

property transfer_verified_checksum

Property to notify the caller if it needs to set authentication.

validate_href(*image_href*, *secret=None*)

Validate OCI image reference.

This method is an alias of the show method on this class, which exists only for API compatibility reasons. Ultimately, the show method performs all of the same validation required.

Parameters

- **image_href** Image reference.
- **secret** Unused setting.

Raises
exception.ImageRefValidationFailed

Raises
exception.OciImageNotSpecific

Returns
Identical output to the show method on this class as this method is an alias of the show.

ironic.common.image_service.**get_image_service**(*image_href*, *client=None*, *context=None*)

Get image service instance to download the image.

Parameters

- **image_href** String containing href to get image service for.
- **client** Glance client to be used for download, used only if image_href is Glance href.
- **context** request context, used only if image_href is Glance href.

Raises
exception.ImageRefValidationFailed if no image service can handle specified href.

Returns
Instance of an image service class that is able to download specified image.

ironic.common.image_service.**get_image_service_auth_override**(*node*, *permit_user_auth=True*)

Collect image service authentication overrides

This method is intended to collect authentication credentials together for submission to remote image services which may have authentication requirements which are not presently available, or where specific authentication details are required.

This method also explicitly prefers pull secrets and contents over `image_server_user` and `image_server_password` parameters.

Parameters

- **task** A Node object instance.
- **permit_user_auth** Option to allow the caller to indicate if user provided authentication should be permitted. For example, downloads of agent ramdisks should never be performed with user credentials, and only the caller of this method knows if that is the case.

Returns

A dictionary with username and password keys containing credential to utilize or None if no value found.

ironic.common.image_service.**is_container_registry_url**(*image_href*)

Determine if the supplied reference string is an OCI registry URL.

Parameters

image_href A string containing a url, sourced from the original user request.

Returns

True if the URL appears to be an OCI image registry URL. Otherwise, False.

ironic.common.images module

Handling of VM disk images.

ironic.common.images.**check_if_image_format_is_permitted**(*img_format*, *expected_format=None*, *node=None*)

Checks image format consistency.

Params img_format

The determined image format by name.

Params expected_format

Optional, the expected format based upon supplied configuration values.

Params node

A node object or None implying image cache.

Raises

InvalidImage if the requested image format is not permitted by configuration, or the expected_format does not match the determined format.

ironic.common.images.**converted_size**(*path*, *estimate=False*)

Get size of converted raw image.

The size of image converted to raw format can be growing up to the virtual size of the image.

Parameters

- **path** path to the image file.
- **estimate** Whether to estimate the size by scaling the original size

Returns

For *estimate=False*, return the size of the raw image file. For *estimate=True*, return the size of the original image scaled by the configuration value *raw_image_growth_factor*.

ironic.common.images.**create_boot_iso**(*context*, *output_filename*, *kernel_href*, *ramdisk_href*, *deploy_iso_href=None*, *esp_image_href=None*, *root_uuid=None*, *kernel_params=None*, *boot_mode=None*, *inject_files=None*, *publisher_id=None*)

Creates a bootable ISO image for a node.

Given the hrefs for kernel, ramdisk, root partitions UUID and kernel cmdline arguments, this method fetches the kernel and ramdisk, and builds a bootable ISO image that can be used to boot up the baremetal node.

Parameters

- **context** context
- **output_filename** the absolute path of the output ISO file
- **kernel_href** URL or glance uuid of the kernel to use
- **ramdisk_href** URL or glance uuid of the ramdisk to use
- **deploy_iso_href** URL or glance UUID of the deploy ISO image to extract EFI system partition image. If not specified, the *esp_image_href* option must be present if UEFI-bootable ISO is desired.
- **esp_image_href** URL or glance UUID of FAT12/16/32-formatted EFI system partition image containing the EFI boot loader (e.g. GRUB2) for each hardware architecture to boot. This image will be written onto the ISO image. If not specified, the *deploy_iso_href* option is only required for building UEFI-bootable ISO.
- **kernel_params** a string containing whitespace separated values kernel cmdline arguments of the form K=V or K (optional).
- **inject_files** Mapping of local source file paths to their location on the final ISO image.
- **publisher_id** A value to set as the publisher identifier string in the ISO image to be generated.

Boot_mode

the boot mode in which the deploy is to happen.

Raises

ImageCreationFailed, if creating boot ISO failed.

ironic.common.images.**create_esp_image_for_uefi**(*output_file*, *kernel*, *ramdisk*, *deploy_iso=None*, *esp_image=None*, *kernel_params=None*, *inject_files=None*, *publisher_id=None*)

Creates an ESP image on the specified file.

Copies the provided kernel, ramdisk and EFI system partition image (ESP) to a directory, generates the grub configuration file using kernel parameters and then generates a bootable ISO image for UEFI.

Parameters

- **output_file** the path to the file where the iso image needs to be created.
- **kernel** the kernel to use.
- **ramdisk** the ramdisk to use.
- **deploy_iso** deploy ISO image to extract EFI system partition image from. If not specified, the *esp_image* option is required.
- **esp_image** FAT12/16/32-formatted EFI system partition image containing the EFI boot loader (e.g. GRUB2) for each hardware architecture to boot. This image will be embedded into the ISO image. If not specified, the *deploy_iso* option is required.
- **kernel_params** a list of strings(each element being a string like K=V or K or combination of them like K1=V1,K2,) to be added as the kernel cmdline.
- **inject_files** Mapping of local source file paths to their location on the final ISO image.
- **publisher_id** A value to set as the publisher identifier string in the ISO image to be generated.

Raises

ImageCreationFailed, if image creation failed while copying files or while running command to generate iso.

ironic.common.images.**create_isolinux_image_for_bios**(*output_file*, *kernel*, *ramdisk*, *kernel_params=None*, *inject_files=None*, *publisher_id=None*)

Creates an isolinux image on the specified file.

Copies the provided kernel, ramdisk to a directory, generates the isolinux configuration file using the kernel parameters provided, and then generates a bootable ISO image.

Parameters

- **output_file** the path to the file where the iso image needs to be created.
- **kernel** the kernel to use.
- **ramdisk** the ramdisk to use.
- **kernel_params** a list of strings(each element being a string like K=V or K or combination of them like K1=V1,K2,) to be added as the kernel cmdline.
- **inject_files** Mapping of local source file paths to their location on the final ISO image.
- **publisher_id** A value to set as the publisher identifier string in the ISO image to be generated.

Raises

ImageCreationFailed, if image creation failed while copying files or while running command to generate iso.

ironic.common.images.**create_vfat_image**(*output_file*, *files_info=None*, *parameters=None*, *parameters_file='parameters.txt'*, *fs_size_kib=100*)

Creates the fat fs image on the desired file.

This method copies the given files to a root directory (optional), writes the parameters specified to the parameters file within the root directory (optional), and then creates a vfat image of the root directory.

Parameters

- **output_file** The path to the file where the fat fs image needs to be created.
- **files_info** A dict containing absolute path of file to be copied -> relative path within the vfat image. For example:

```
{
'/absolute/path/to/file' -> 'relative/path/within/root'
...
}
```

- **parameters** A dict containing key-value pairs of parameters.
- **parameters_file** The filename for the parameters file.
- **fs_size_kib** size of the vfat filesystem in KiB.

Raises

ImageCreationFailed, if image creation failed while doing any of filesystem manipulation activities like creating dirs, mounting, creating filesystem, copying files, etc.

ironic.common.images.**detect_file_format**(*path*)

Re-implementation of detect_file_format from oslo.utils

This implementation specifically allows for single multi-format combination of ISO+GPT, which it treats like ISO.

ironic.common.images.**download_size**(*context*, *image_href*, *image_service=None*, *image_auth_data=None*)

ironic.common.images.**fetch**(*context*, *image_href*, *path*, *force_raw=False*, *checksum=None*, *checksum_algo=None*, *image_auth_data=None*)

ironic.common.images.**fetch_into**(*context*, *image_href*, *image_file*, *image_auth_data=None*, *checksum=None*, *checksum_algo=None*)

Fetches image file contents into a file.

Parameters

- **context** A context object.
- **image_href** The Image URL or reference to attempt to retrieve.
- **image_file** The file handler or file name to write the requested file contents to.

- **image_auth_data** Optional dictionary for credentials to be conveyed from the original task to the image download process, if required.
- **checksum** Expected checksum value for validation during transfer.
- **checksum_algo** Algorithm for checksum (e.g., md5, sha256).

Returns

If a value is returned, that value was validated as the checksum. Otherwise None indicating the process had been completed.

ironic.common.images.**force_raw_will_convert**(*image_href*, *path_tmp*)

ironic.common.images.**get_image_properties**(*context*, *image_href*, *properties='all'*)

Returns the values of several properties of an image

Parameters

- **context** context
- **image_href** href of the image
- **properties** the properties whose values are required. This argument is optional, default value is all, so if not specified all properties will be returned.

Returns

a dict of the values of the properties. A property not on the glance metadata will have a value of None.

ironic.common.images.**get_source_format**(*image_href*, *path*)

ironic.common.images.**get_temp_url_for_glance_image**(*context*, *image_uuid*)

Returns the tmp url for a glance image.

Parameters

- **context** context
- **image_uuid** the UUID of the image in glance

Returns

the tmp url for the glance image.

ironic.common.images.**image_format_matches**(*actual_format*, *expected_format*)

ironic.common.images.**image_format_permitted**(*img_format*)

ironic.common.images.**image_show**(*context*, *image_href*, *image_service=None*, *image_auth_data=None*)

ironic.common.images.**image_to_raw**(*image_href*, *path*, *path_tmp*)

ironic.common.images.**is_source_a_path**(*ctx*, *image_source*)

Determine if the image source is a path.

This method determines if a supplied URL is a path.

Parameters

- **ctx** an admin/process context.

- **image_source** The supplied image source, expected to be a URL, which can be used to attempt to determine if the source is a path.

Returns

True if the image_source appears to be a path as opposed to an image to be downloaded. If the image source is not a path, False is returned. If any error is detected, None is returned.

ironic.common.images.**is_whole_disk_image**(*ctx*, *instance_info*)

Find out if the image is a partition image or a whole disk image.

Parameters

- **ctx** an admin context
- **instance_info** a nodes instance info dict

Returns

True for whole disk images and False for partition images and None on no image_source, the source being a path, or upon an Error.

ironic.common.images.**safety_check_image**(*image_path*, *node=None*)

Performs a safety check on the supplied image.

This method triggers the image format inspectors to both identify the type of the supplied file and safety check logic to identify if there are any known unsafe features being leveraged, and return the detected file format in the form of a string for the caller.

Parameters

- **image_path** A fully qualified path to an image which needs to be evaluated for safety.
- **node** A Node object, optional. When supplied logging indicates the node which triggered this issue, but the node is not available in all invocation cases.

Returns

a string representing the the image type which is used.

Raises

InvalidImage when the supplied image is detected as unsafe, or the image format inspector has failed to parse the supplied images contents.

ironic.common.indicator_states module

Mapping of the indicator LED states.

ironic.common.indicator_states.**BLINKING = 'blinking'**

LED is blinking

ironic.common.indicator_states.**OFF = 'off'**

LED is off

ironic.common.indicator_states.**ON = 'on'**

LED is on

ironic.common.indicator_states.**UNKNOWN = 'unknown'**

LED state is not known

ironic.common.keystone module

Central place for handling Keystone authorization and service lookup.

ironic.common.keystone.**add_auth_opts**(*options*, *service_type=None*)

Add auth options to sample config

As these are dynamically registered at runtime, this adds options for most used auth_plugins when generating sample config.

ironic.common.keystone.**get_adapter**(*group*, ***adapter_kwargs*)

Loads adapter from options in a configuration file section.

The adapter_kwargs will be passed directly to keystoneauth1 Adapter and will override the values loaded from config. Consult keystoneauth1 docs for available adapter options.

Parameters

group name of the config section to load adapter options from

ironic.common.keystone.**get_auth**(*group*, ***auth_kwargs*)

Loads auth plugin from options in a configuration file section.

The auth_kwargs will be passed directly to keystoneauth1 auth plugin and will override the values loaded from config. Note that the accepted kwargs will depend on auth plugin type as defined by [group]auth_type option. Consult keystoneauth1 docs for available auth plugins and their options.

Parameters

group name of the config section to load auth plugin options from

ironic.common.keystone.**get_endpoint**(*group*, ***adapter_kwargs*)

Get an endpoint from an adapter.

The adapter_kwargs will be passed directly to keystoneauth1 Adapter and will override the values loaded from config. Consult keystoneauth1 docs for available adapter options.

Parameters

group name of the config section to load adapter options from

Raises

CatalogNotFound if the endpoint is not found

ironic.common.keystone.**get_service_auth**(*context*, *endpoint*, *service_auth*, *only_service_auth=False*)

Create auth plugin wrapping both user and service auth.

When properly configured and using auth_token middleware, requests with valid service auth will not fail if the user token is expired.

Ideally we would use the plugin provided by auth_token middleware however this plugin isnt serialized yet.

Parameters

- **context** The RequestContext instance from which the user auth_token is extracted.
- **endpoint** The requested endpoint to be utilized.
- **service_auth** The service authentication credentals to be used.

- **only_service_auth** Boolean, default False. When set to True, the resulting Service token pair is generated as if it originates from the user itself. Useful to cast admin level operations which are launched by Ironic itself, as opposed to user initiated requests.

Returns
Returns a service token via the ServiceTokenAuthWrapper class.

ironic.common.keystone.**get_session**(*group*, ***session_kwargs*)

Loads session object from options in a configuration file section.

The session_kwargs will be passed directly to keystoneauth1 Session and will override the values loaded from config. Consult keystoneauth1 docs for available options.

Parameters
group name of the config section to load session options from

ironic.common.keystone.**ks_exceptions**(*f*)

Wraps keystoneclient functions and centralizes exception handling.

ironic.common.keystone.**register_auth_opts**(*conf*, *group*, *service_type=None*)

Register session- and auth-related options

Registers only basic auth options shared by all auth plugins. The rest are registered at runtime depending on auth plugin used.

ironic.common.kickstart_utils module

ironic.common.kickstart_utils.**decode_and_extract_config_drive_iso**(*config_drive_iso_gz*)

ironic.common.kickstart_utils.**prepare_config_drive**(*task*, *config_drive_path='/var/lib/cloud/seed/config_drive'*)

Prepare config_drive for writing to kickstart file

ironic.common.kickstart_utils.**read_iso9600_config_drive**(*config_drive*)

Read config drive and store its contents in a dict

Parameters
config_drive Config drive in iso9600 format

Returns
A dict containing path as key and contents of the configdrive file as value.

ironic.common.lessee_sources module

Mapping of values for CONF.conductor.automatic_lessee_source, representing different possible sources for lessee data.

ironic.common.lessee_sources.**INSTANCE = 'instance'**

Use instance_info[project_id]

ironic.common.lessee_sources.**NONE = 'none'**

Do not set lessee

ironic.common.lessee_sources.**REQUEST = 'request'**

Use metadata in request context

ironic.common.mdns module

Multicast DNS implementation for API discovery.

This implementation follows RFC 6763 as clarified by the API SIG guideline https://review.opendev.org/651222.

class ironic.common.mdns.**Zeroconf**

Bases: object

Multicast DNS implementation client and server.

Uses threading internally, so there is no start method. It starts automatically on creation.

> **Warning**
>
> The underlying library does not yet support IPv6.

close()

Shut down mDNS and unregister services.

> **Note**
>
> If another server is running for the same services, it will re-register them immediately.

register_service(*service_type*, *endpoint*, *params=None*)

Register a service.

This call announces the new services via multicast and instructs the built-in server to respond to queries about it.

Parameters

- **service_type** OpenStack service type, e.g. baremetal.
- **endpoint** full endpoint to reach the service.
- **params** optional properties as a dictionary.

Raises

ServiceRegistrationFailure if the service cannot be registered, e.g. because of conflicts.

ironic.common.metrics module

class ironic.common.metrics.**Counter**(*metrics*, *name*, *sample_rate*)

Bases: object

A counter decorator and context manager.

This metric type increments a counter every time the decorated method or context manager is executed. It is bound to this MetricLogger. For example:

```
from ironic.common import metrics_utils

METRICS = metrics_utils.get_metrics_logger()

@METRICS.counter('foo')
def foo(bar, baz):
    print bar, baz

with METRICS.counter('foo'):
    do_something()
```

__call__(*f*)

Call self as a function.

class ironic.common.metrics.**Gauge**(*metrics*, *name*)

Bases: object

A gauge decorator.

This metric type returns the value of the decorated method as a metric every time the method is executed. It is bound to this MetricLogger. For example:

```
from ironic.common import metrics_utils

METRICS = metrics_utils.get_metrics_logger()

@METRICS.gauge('foo')
def add_foo(bar, baz):
    return (bar + baz)
```

__call__(*f*)

Call self as a function.

class ironic.common.metrics.**MetricLogger**(*prefix=''*, *delimiter='.'*)

Bases: object

Abstract class representing a metrics logger.

A MetricLogger sends data to a backend (noop or statsd). The data can be a gauge, a counter, or a timer.

The data sent to the backend is composed of:

- a full metric name
- a numeric value

The format of the full metric name is:
_prefix<delim>name

where:

- _prefix: [global_prefix<delim>][uuid<delim>][host_name<delim>]prefix
- name: the name of this metric
- <delim>: the delimiter. Default is .

counter(*name*, *sample_rate=None*)

gauge(*name*)

get_metric_name(*name*)

Get the full metric name.

The format of the full metric name is:
_prefix<delim>name

where:

- _prefix: [global_prefix<delim>][uuid<delim>][host_name<delim>] prefix
- name: the name of this metric
- <delim>: the delimiter. Default is .

Parameters
name The metric name.

Returns
The full metric name, with logger prefix, as a string.

get_metrics_data()

Return the metrics collection, if available.

send_counter(*name*, *value*, *sample_rate=None*)

Send counter metric data.

Counters are used to count how many times an event occurred. The backend will increment the counter name by the value value.

Optionally, specify sample_rate in the interval [0.0, 1.0] to sample data probabilistically where:

```
P(send metric data) = sample_rate
```

If sample_rate is None, then always send metric data, but do not have the backend send sample rate information (if supported).

Parameters

- **name** Metric name
- **value** Metric numeric value that will be sent to the backend
- **sample_rate** Probabilistic rate at which the values will be sent. Value must be None or in the interval [0.0, 1.0].

send_gauge(*name*, *value*)

Send gauge metric data.

Gauges are simple values. The backend will set the value of gauge name to value.

Parameters

- **name** Metric name
- **value** Metric numeric value that will be sent to the backend

send_timer(*name*, *value*)

Send timer data.

Timers are used to measure how long it took to do something.

Parameters

- **m_name** Metric name
- **m_value** Metric numeric value that will be sent to the backend

timer(*name*)

class ironic.common.metrics.**NoopMetricLogger**(*prefix=''*, *delimiter='.'*)

Bases: *MetricLogger*

Noop metric logger that throws away all metric data.

class ironic.common.metrics.**Timer**(*metrics*, *name*)

Bases: object

A timer decorator and context manager.

This metric type times the decorated method or code running inside the context manager, and emits the time as the metric value. It is bound to this MetricLogger. For example:

```
from ironic.common import metrics_utils

METRICS = metrics_utils.get_metrics_logger()

@METRICS.timer('foo')
def foo(bar, baz):
    print bar, baz

with METRICS.timer('foo'):
    do_something()
```

__call__(*f*)

Call self as a function.

ironic.common.metrics_collector module

class ironic.common.metrics_collector.**DictCollectionMetricLogger**(*prefix*, *delimiter='.'*)

Bases: *MetricLogger*

Metric logger that collects internal counters.

COUNTER_TYPE = 'c'

GAUGE_TYPE = 'g'

TIMER_TYPE = 'ms'

get_metrics_data()

Return the metrics collection dictionary.

Returns

Dictionary containing the keys and values of data stored via the metrics collection hooks. The values themselves are dictionaries which contain a type field, indicating if the statistic is a counter, gauge, or timer. A counter has a *count* field, a gauge value has a *value* field, and a timer fiend las a count and sum fields. The multiple fields for for a timer type allows for additional statistics to be implied from the data once collected and compared over time.

ironic.common.metrics_statsd module

class ironic.common.metrics_statsd.**StatsdMetricLogger**(*prefix*, *delimiter='.'*, *host=None*, *port=None*)

Bases: *MetricLogger*

Metric logger that reports data via the statsd protocol.

COUNTER_TYPE = 'c'

GAUGE_TYPE = 'g'

TIMER_TYPE = 'ms'

ironic.common.metrics_utils module

ironic.common.metrics_utils.**get_metrics_logger**(*prefix=''*, *backend=None*, *host=None*, *delimiter='.'*)

Return a metric logger with the specified prefix.

The format of the prefix is: [global_prefix<delim>][host_name<delim>]prefix where <delim> is the delimiter (default is .)

Parameters

- **prefix** Prefix for this metric logger. Value should be a string or None.
- **backend** Backend to use for the metrics system. Possible values are noop and statsd.
- **host** Name of this node.
- **delimiter** Delimiter to use for the metrics name.

Returns

The new MetricLogger.

ironic.common.molds module

ironic.common.molds.**get_configuration**(*task*, *url*)

Gets configuration mold from indicated location.

Parameters

- **task** A TaskManager instance.
- **url** URL of the configuration item to get.

Returns

JSON configuration mold

Raises

- *IronicException* If using Swift storage and no authentication token found in tasks context.
- **HTTPError** If failed to complete HTTP request.

ironic.common.molds.**save_configuration**(*task*, *url*, *data*)

Store configuration mold to indicated location.

Parameters

- **task** A TaskManager instance.
- **name** URL of the configuration item to save to.
- **data** Content of JSON data to save.

Raises

- *IronicException* If using Swift storage and no authentication token found in tasks context.
- **HTTPError** If failed to complete HTTP request.

ironic.common.network module

ironic.common.network.**get_network_type_for_state**(*provision_state*)

Determine which network type should be active based on provision state.

Parameters

provision_state The nodes current provision state.

Returns

String indicating the network type, or IDLE_NETWORK if the state doesnt map to a specific network.

ironic.common.network.**get_node_vif_ids**(*task*)

Get all VIF ids for a node.

This function does not handle multi node operations.

Parameters

task a TaskManager instance.

Returns

A dict of Nodes neutron ports where keys are ports & portgroups and the values are dict of UUIDs and their associated VIFs, e.g.

```
{'ports': {'port.uuid': vif.id},
 'portgroups': {'portgroup.uuid': vif.id}}
```

ironic.common.network.**get_physnets_by_portgroup_id**(*task*, *portgroup_id*, *exclude_port=None*)

Return the set of physical networks associated with a portgroup.

Parameters

- **task** a TaskManager instance.

- **portgroup_id** ID of the portgroup.
- **exclude_port** A Port object to exclude from the determination of the portgroups physical network, or None.

Returns
The set of physical networks associated with the portgroup. The set will contain zero or one physical networks.

Raises
PortgroupPhysnetInconsistent if the portgroups ports are not assigned the same physical network.

ironic.common.network.**get_physnets_for_node**(*task*)

Return the set of physical networks for a node.

Returns the set of physical networks associated with a nodes ports. The physical network None is excluded from the set.

Parameters
task a TaskManager instance

Returns
A set of physical networks.

ironic.common.network.**get_portgroup_by_id**(*task*, *portgroup_id*)

Lookup a portgroup by ID on a task object.

Parameters

- **task** a TaskManager instance
- **portgroup_id** ID of the portgroup.

Returns
A Portgroup object or None.

ironic.common.network.**get_ports_by_portgroup_id**(*task*, *portgroup_id*)

Lookup ports by their portgroup ID on a task object.

Parameters

- **task** a TaskManager instance
- **portgroup_id** ID of the portgroup.

Returns
A list of Port objects.

ironic.common.network.**remove_vifs_from_node**(*task*)

Remove all vif attachment records from a node.

Parameters
task a TaskManager instance.

ironic.common.neutron module

class ironic.common.neutron.**NeutronNetworkInterfaceMixin**

Bases: object

get_cleaning_network_uuid(*task*)

get_inspection_network_uuid(*task*)

get_provisioning_network_uuid(*task*)

get_rescuing_network_uuid(*task*)

get_servicing_network_uuid(*task*)

validate_inspection(*task*)

Validate that the node has required properties for inspection.

Parameters
task A TaskManager instance with the node being checked

Raises
MissingParameterValue if node is missing one or more required parameters

Raises
UnsupportedDriverExtension

ironic.common.neutron.**PHYSNET_PARAM_NAME = 'provider:physical_network'**

Name of the neutron network API physical network parameter.

ironic.common.neutron.**add_ports_to_network**(*task*, *network_uuid*, *security_groups=None*)

Create neutron ports to boot the ramdisk.

Create neutron ports for each pxe_enabled port on task.node to boot the ramdisk.

If the config option neutron.add_all_ports is set, neutron ports for non-pxe-enabled ports are also created these neutron ports will not have any assigned IP addresses.

Parameters

- **task** a TaskManager instance.
- **network_uuid** UUID of a neutron network where ports will be created.
- **security_groups** List of Security Groups UUIDs to be used for network.

Raises
NetworkError

Returns
a dictionary in the form {port.uuid: neutron_port[id]}

ironic.common.neutron.**get_client**(*token=None*, *context=None*, *auth_from_config=False*)

Retrieve a neutron client connection.

Parameters

- **context** request context, instance of ironic.common.context.RequestContext
- **auth_from_config** (boolean) When True, use auth values from conf parameters

Returns
A neutron client.

ironic.common.neutron.**get_local_group_information**(*task*, *portgroup*)

Extract the portgroup information.

The information is returned in the form of:

```
{
    'id': portgroup.uuid,
    'name': portgroup.name,
    'bond_mode': portgroup.mode,
    'bond_properties': {
        'bond_propertyA': 'valueA',
        'bond_propertyB': 'valueB',
    }
}
```

Parameters

- **task** a task containing the Node object.
- **portgroup** Ironic portgroup object to extract data for.

Returns
port group information as a dict

ironic.common.neutron.**get_neutron_port_data**(*port_id*, *vif_id*, *client=None*, *context=None*, *mac_address=None*, *iface_type='vif'*, *bond_links=None*)

Gather Neutron port and network configuration

Query Neutron for port and network configuration, return whatever is available. Apply a mac_address override, which may be useful for the caller if this is being executed prior to port attachments.

Parameters

- **port_id** ironic port/portgroup ID.
- **vif_id** Neutron port ID.
- **client** Optional a Neutron client object.
- **context** (ironic.common.context.RequestContext) request context
- **mac_address** An override ethernet mac address.
- **iface_type** Default interface type to denote a link to be. Likely ignored by cloud-init, but still best to be populated. Other applicable option is phy, or bond.
- **bond_links** A list of dictionaries of additional links which indicate the port is a bind. This value being set overrides the iface_type value to bond

Raises
NetworkError

Returns
a dict holding network configuration information associated with this ironic or Neutron port.

ironic.common.neutron.**get_node_portmap**(*task*)

Extract the switch port information for the node.

The information is returned in the form of:

```
{
    port.uuid: {
        'switch_id': 'abc',
        'port_id': 'Po0/1',
        'other_llc_key': 'val'
    }
}
```

Parameters
task a task containing the Node object.

Returns
port information as a dict

ironic.common.neutron.**get_physnets_by_port_uuid**(*client*, *port_uuid*)

Return the set of physical networks associated with a neutron port.

Query the network to which the port is attached and return the set of physical networks associated with the segments in that network. If a port is assigned to a subnet with a direct segment mapping, return the physnet associated with its segment instead.

Parameters

- **client** A Neutron client object.
- **port_uuid** UUID of a Neutron port to query.

Returns
A set of physical networks.

Raises
NetworkError if the network query fails.

Raises
InvalidParameterValue for missing network.

ironic.common.neutron.**is_smartnic_port**(*port_data*)

Check that the port is Smart NIC port

Parameters
port_data an instance of ironic.objects.port.Port or port data as dict.

Returns
A boolean to indicate port as Smart NIC port.

ironic.common.neutron.**remove_neutron_ports**(*task*, *params*)

Deletes the neutron ports matched by params.

Parameters

- **task** a TaskManager instance.
- **params** Dict of params to filter ports.

Raises
NetworkError

ironic.common.neutron.**remove_ports_from_network**(*task*, *network_uuid*)

Deletes the neutron ports created for booting the ramdisk.

Parameters

- **task** a TaskManager instance.
- **network_uuid** UUID of a neutron network ports will be deleted from.

Raises
NetworkError

ironic.common.neutron.**rollback_ports**(*task*, *network_uuid*)

Attempts to delete any ports created by cleaning/provisioning

Purposefully will not raise any exceptions so error handling can continue.

Parameters

- **task** a TaskManager instance.
- **network_uuid** UUID of a neutron network.

ironic.common.neutron.**unbind_neutron_port**(*port_id*, *client=None*, *context=None*, *reset_mac=True*)

Unbind a neutron port

Remove a neutron ports binding profile and host ID so that it returns to an unbound state.

Parameters

- **port_id** Neutron port ID.
- **client** Optional a Neutron client object.
- **context** (ironic.common.context.RequestContext) request context
- **reset_mac** reset mac address

Raises
NetworkError

ironic.common.neutron.**unbind_neutron_port_if_bound**(*port_id*, *client=None*, *context=None*)

Check and if bound, unbind a neutron port.

A wrapper around unbind_neutron_port which checks if the port is already bound, and if so then triggers the port to be unbound. This is critical early on if a user has pre-bound the port.

If the port is missing from Neutron, then a NetworkError exception will also be raised because in this code path, we explicitly expect the port to already exist and be accessible.

Parameters

- **port_id** Neutron Port ID

- **client** Optional Neutron client.
- **context** (ironic.common.context.RequestContext) request context

Raises
NetworkError

ironic.common.neutron.**update_neutron_port**(*context*, *port_id*, *attrs*, *client=None*)

Undate a neutron port

Uses neutron client from conf client to update a neutron client an unbound state.

Parameters

- **context** request context, instance of ironic.common.context.RequestContext
- **port_id** Neutron port ID.
- **attrs** The attributes to update on the port
- **client** Optional Neutron client

ironic.common.neutron.**update_port_address**(*port_id*, *address*, *context=None*, *client=None*)

Update a ports mac address.

Parameters

- **port_id** Neutron port id.
- **address** new MAC address.
- **context** (ironic.common.context.RequestContext) request context
- **client** A neutron client object.

Raises
FailedToUpdateMacOnPort

ironic.common.neutron.**validate_network**(*uuid_or_name*, *net_type='network'*, *context=None*)

Check that the given network is present.

Parameters

- **uuid_or_name** network UUID or name
- **net_type** human-readable network type for error messages
- **context** (ironic.common.context.RequestContext) request context

Returns
network UUID

Raises
MissingParameterValue if uuid_or_name is empty

Raises
NetworkError on failure to contact Neutron

Raises
InvalidParameterValue for missing or duplicated network

ironic.common.neutron.**validate_port_info**(*node*, *port*)

Check that port contains enough information for deploy.

Neutron network interface requires that local_link_information field is filled before we can use this port.

Parameters

- **node** Ironic node object.
- **port** Ironic port object.

Returns

True if port info is valid, False otherwise.

ironic.common.neutron.**wait_for_host_agent**(*client*, *host_id*, *target_state='up'*)

Wait for neutron agent to become target state

Parameters

- **client** A Neutron client object.
- **host_id** Agent host_id
- **target_state** up: wait for up status, down: wait for down status

Returns

boolean indicates the agent state matches param value target_state_up.

Raises

exception.Invalid if target_state is not valid.

Raises

exception.NetworkError if host status didnt match the required status after max retry attempts.

ironic.common.neutron.**wait_for_port_status**(*client*, *port_id*, *status*, *fail_on_binding_failure=None*)

Wait for port status to be the desired status

Parameters

- **client** A Neutron client object.
- **port_id** Neutron port_id
- **status** Ports target status, can be ACTIVE, DOWN etc.
- **fail_on_binding_failure** Whether to raise exception if port binding fails.

Returns

boolean indicates that the port status matches the required value passed by param status.

Raises

InvalidParameterValue if the port does not exist.

Raises

exception.NetworkError if port status didnt match the required status after max retry attempts.

ironic.common.nova module

ironic.common.nova.**power_update**(*context*, *server_uuid*, *target_power_state*)

Creates and sends power state change for the provided server_uuid.

Parameters

- **context** request context, instance of ironic.common.context.RequestContext
- **server_uuid** The uuid of the node whose power state changed.
- **target_power_state** Targeted power state change i.e POWER_ON or POWER_OFF

Returns

A boolean which indicates if the power update was executed successfully (mainly for testing purposes).

ironic.common.oci_registry module

class ironic.common.oci_registry.**MakeSession**(*verify=True*)

Bases: object

Class method to uniformly create sessions.

Sessions created by this class will retry on errors with an exponential backoff before raising an exception. Because our primary interaction is with the container registries the adapter will also retry on 401 and 404. This is being done because registries commonly return 401 when an image is not found, which is commonly a cache miss. See the adapter definitions for more on retry details.

create()

class ironic.common.oci_registry.**OciClient**(*verify*)

Bases: object

authenticate(*image_url*, *username=None*, *password=None*)

Authenticate to the remote container registry.

Parameters

- **image_url** The URL to utilise for the remote container registry.
- **username** The username paraemter.
- **password** The password parameter.

Raises

AttributeError when an unknown authentication attribute has been specified by the remote service.

Raises

ImageServiceAuthenticationRequired when the remote Container registry requires authentication but we do not have a credentials.

download_blob_from_manifest(*manifest_url*, *image_file*)

Retrieves the requested blob from the manifest URL

And saves the requested manifests artifact as the requested image_file location, and then returns the verified checksum.

Parameters

- **manifest_url** The URL, in oci://host/user/container@digest formatted artifact manifest URL. This is *not* the digest value for the blob, which can only be discovered by retrieving the manifest.
- **image_file** The image file object to write the blob to.

Returns
The verified digest value matching the saved artifact.

get_artifact_index(*image*)

Retrieve an index of artifacts in the Container Registry.

Parameters
image The remote container registry URL in the form of oci://host/user/container:tag.

Returns
A dictionary object representing the index of artifacts present in the container registry, in the form of manifest references along with any other metadata per entry which the remote registry returns such as annotations, and platform labeling which aids in artifact selection.

get_blob_url(*image*, *blob_digest*)

Generates an HTTP representing an blob artifact.

Parameters

- **image** The OCI Container URL.
- **blob_digest** The digest value representing the desired blob artifact.

Returns
A HTTP URL string representing the blob URL which can be utilized by an HTTP client to retrieve the artifact.

get_cached_auth()

Retrieves the cached authentication header for reuse.

get_manifest(*image*, *digest=None*)

Retrieve a manifest from the remote API.

This method is a wrapper for the _get_manifest helper, which normalizes the input URL, performs basic sanity checking, and then calls the underlying method to retrieve the manifest.

The manifest is then returned to the caller in the form of a dictionary.

Parameters

- **image** The full URL to the desired manifest or the URL of the container and an accompanying digest parameter.
- **digest** The Digest value for the requested manifest.

Returns
A dictionary object representing the manifest as stored in the remote API.

class ironic.common.oci_registry.**RegistrySessionHelper**

Bases: object

Class with various registry session helpers

This class contains a bunch of static methods to be used when making session requests against a container registry. The methods are primarily used to handle authentication/reauthentication for the requests against registries that require auth.

static check_redirect_trusted(*request_response*, *request_session*, *stream=True*, *timeout=60*)

Check if weve been redirected to a trusted source

Because we may be using auth, we may not want to leak authentication keys to an untrusted source. If we get a redirect, we need to check that the redirect url is one of our sources that we trust. Otherwise we drop the Authorization header from the redirect request. Well add the header back into the request session after performing the request to ensure that future usage of the session.

Param
request_response: Response object of the request to check

Param
request_session: Session to use when redirecting

Param
stream: Should we stream the response of the redirect

Param
timeout: Timeout for the redirect request

static check_status(*session*, *request*, *allow_reauth=True*)

Check request status and trigger reauth

This function can be used to check if we need to perform authentication for a container registry request because weve gotten a 401.

static get(*request_session*, **args*, ***kwargs*)

Perform a get and retry if auth fails

This function is designed to be used when we perform a get to an authenticated source. This function will attempt a single re-authentication request if the first one fails.

static get_bearer_token(*session*, *username=None*, *password=None*, *realm=None*, *service=None*, *scope=None*)

get_token_from_config()

Takes a FQDN for a container registry and consults auth config.

This method evaluates named configuration parameter [oci]authentication_config and looks for pre-shared secrets in the supplied json file. It is written to defensively handle the file such that errors are not treated as fatal to the overall lookup process, but errors are logged.

The expected file format is along the lines of:

```
{
  "auths": {
```

(continues on next page)

(continued from previous page)

```
    "domain.name": {
      "auth": "pre-shared-secret-value"
    }
  }
}
```

Parameters

fqdn A fully qualified domain name for interacting with the remote image registry.

Returns

String value for the auth key which matches the supplied FQDN.

static parse_www_authenticate(*header*)

ironic.common.policy module

Policy Engine For Ironic.

ironic.common.policy.**authorize**(*rule*, *target*, *creds*, **args*, ***kwargs*)

A shortcut for policy.Enforcer.authorize()

Checks authorization of a rule against the target and credentials, and raises an exception if the rule is not defined. Always returns true if CONF.auth_strategy is not keystone.

ironic.common.policy.**check**(*rule*, *target*, *creds*, **args*, ***kwargs*)

A shortcut for policy.Enforcer.enforce()

Checks authorization of a rule against the target and credentials and returns True or False.

ironic.common.policy.**check_policy**(*rule*, *target*, *creds*, **args*, ***kwargs*)

Configuration aware role policy check wrapper.

Checks authorization of a rule against the target and credentials and returns True or False. Always returns true if CONF.auth_strategy is not keystone.

ironic.common.policy.**get_enforcer**()

Provides access to the single instance of Policy enforcer.

ironic.common.policy.**get_oslo_policy_enforcer**()

ironic.common.policy.**init_enforcer**(*policy_file=None*, *rules=None*, *default_rule=None*, *use_conf=True*)

Synchronously initializes the policy enforcer

Parameters

- **policy_file** Custom policy file to use, if none is specified, *CONF.oslo_policy.policy_file* will be used.
- **rules** Default dictionary / Rules to use. It will be considered just in the first instantiation.
- **default_rule** Default rule to use, CONF.oslo_policy.policy_default_rule will be used if none is specified.

- **use_conf** Whether to load rules from config file.

ironic.common.policy.**list_policies**()

ironic.common.profiler module

ironic.common.profiler.**setup**(*name*, *host='0.0.0.0'*)

Setup OSprofiler notifier and enable profiling.

Parameters

- **name** name of the service that will be profiled
- **host** hostname or host IP address that the service will be running on. By default host will be set to 0.0.0.0, but specifying host name / address usage is highly recommended.

Raises

TypeError in case of invalid connection string for a notifier backend, which is set in osprofiler.initializer.init_from_conf.

ironic.common.profiler.**trace_cls**(*name*, ***kwargs*)

Wrap the OSProfiler trace_cls decorator

Wrap the OSProfiler trace_cls decorator so that it will not try to patch the class unless OSProfiler is present and enabled in the config

Parameters

- **name** The name of action. For example, wsgi, rpc, db, etc..
- **kwargs** Any other keyword args used by profiler.trace_cls

ironic.common.pxe_utils module

class ironic.common.pxe_utils.**TFTPImageCache**

Bases: *ImageCache*

ironic.common.pxe_utils.**build_deploy_pxe_options**(*task*, *pxe_info*, *mode='deploy'*, *ipxe_enabled=False*)

ironic.common.pxe_utils.**build_extra_pxe_options**(*task*, *ramdisk_params=None*)

ironic.common.pxe_utils.**build_instance_pxe_options**(*task*, *pxe_info*, *ipxe_enabled=False*)

ironic.common.pxe_utils.**build_kickstart_config_options**(*task*)

Build the kickstart template options for a node

This method builds the kickstart template options for a node, given all the required parameters.

The options should then be passed to pxe_utils.create_kickstart_config to create the actual config files.

Parameters

task A TaskManager object

Returns

A dictionary of kickstart options to be used in the kickstart template.

ironic.common.pxe_utils.**build_pxe_config_options**(*task*, *pxe_info*, *service=False*, *ipxe_enabled=False*, *ramdisk_params=None*)

Build the PXE config options for a node

This method builds the PXE boot options for a node, given all the required parameters.

The options should then be passed to pxe_utils.create_pxe_config to create the actual config files.

Parameters

- **task** A TaskManager object
- **pxe_info** a dict of values to set on the configuration file
- **service** if True, build service mode pxe config for netboot-ed user image and skip adding deployment image kernel and ramdisk info to PXE options.
- **ipxe_enabled** Default false boolean to indicate if ipxe is in use by the caller.
- **ramdisk_params** the parameters to be passed to the ramdisk. as kernel command-line arguments.

Returns

A dictionary of pxe options to be used in the pxe bootfile template.

ironic.common.pxe_utils.**build_service_pxe_config**(*task*, *instance_image_info*, *root_uuid_or_disk_id*, *ramdisk_boot=False*, *ipxe_enabled=False*, *is_whole_disk_image=None*, *anaconda_boot=False*)

ironic.common.pxe_utils.**cache_ramdisk_kernel**(*task*, *pxe_info*, *ipxe_enabled=False*)

Fetch the necessary kernels and ramdisks for the instance.

ironic.common.pxe_utils.**clean_up_pxe_config**(*task*, *ipxe_enabled=False*)

Clean up the TFTP environment for the tasks node.

Parameters

task A TaskManager instance.

ironic.common.pxe_utils.**clean_up_pxe_env**(*task*, *images_info*, *ipxe_enabled=False*)

Cleanup PXE environment of all the images in images_info.

Cleans up the PXE environment for the mentioned images in images_info.

Parameters

- **task** a TaskManager object
- **images_info** A dictionary of images whose keys are the image names to be cleaned up (kernel, ramdisk, etc) and values are a tuple of identifier and absolute path.

ironic.common.pxe_utils.**create_ipxe_boot_script**()

Render the iPXE boot script into the HTTP root directory

ironic.common.pxe_utils.**create_pxe_config**(*task*, *pxe_options*, *template=None*, *ipxe_enabled=False*)

Generate PXE configuration file and MAC address links for it.

This method will generate the PXE configuration file for the tasks node under a directory named with the UUID of that node. For each MAC address or DHCP IP address (port) of that node, a symlink for the configuration file will be created under the PXE configuration directory, so regardless of which port boots first theyll get the same PXE configuration. If grub2 bootloader is in use, then its configuration will be created based on DHCP IP address in the form nn.nn.nn.nn.

Parameters

- **task** A TaskManager instance.
- **pxe_options** A dictionary with the PXE configuration parameters.
- **template** The PXE configuration template. If no template is given the node specific template will be used.

ironic.common.pxe_utils.**dhcp_options_for_instance**(*task*, *ipxe_enabled=False*, *url_boot=False*, *ip_version=None*, *http_boot_enabled=False*)

Retrieves the DHCP PXE boot options.

Parameters

- **task** A TaskManager instance.
- **ipxe_enabled** Default false boolean that signals if iPXE formatting should be returned by the method for DHCP server configuration.
- **url_boot** Default false boolean to inform the method if a URL should be returned to boot the node. If [pxe]ip_version is set to *6*, then this option has no effect as url_boot form is required by DHCPv6 standards.
- **ip_version** The IP version of options to return as values differ by IP version. Default to [pxe]ip_version. Possible options are integers 4 or 6.

Returns

Dictionary to be sent to the networking service describing the DHCP options to be set.

Raises

InvalidParameterValue if the underlying configuration cannot be conveyed to Neutron due to resulting value length.

ironic.common.pxe_utils.**ensure_tree**(*path*)

ironic.common.pxe_utils.**get_file_path_from_label**(*node_uuid*, *root_dir*, *label*)

Generate absolute paths to various images from their name(label)

This method generates absolute file system path on the conductor where various images need to be placed. For example the kickstart template, file and stage2 squashfs.img needs to be placed in the ipxe_root_dir since they will be transferred by anaconda ramdisk over http(s). The generated paths will be added to the image_info dictionary as values.

Parameters

- **node_uuid** the UUID of the node

- **root_dir** Directory in which the image must be placed
- **label** Name of the image

ironic.common.pxe_utils.**get_http_url_path_from_label**(*http_url*, *node_uuid*, *label*)

Generate http url path to various image artifacts

This method generates http(s) urls for various image artifacts int the webserver root. The generated urls will be added to the pxe_options dict and used to render pxe/ipxe configuration templates.

Parameters

- **http_url** URL to access the root of the webserver
- **node_uuid** the UUID of the node
- **label** Name of the image

ironic.common.pxe_utils.**get_image_info**(*node*, *mode='deploy'*, *ipxe_enabled=False*)

Generate the paths for TFTP files for deploy or rescue images.

This method generates the paths for the deploy (or rescue) kernel and deploy (or rescue) ramdisk.

Parameters

- **node** a node object
- **mode** Label indicating a deploy or rescue operation being carried out on the node. Supported values are deploy and rescue. Defaults to deploy, indicating deploy operation is being carried out.
- **ipxe_enabled** A default False boolean value to tell the method if the caller is using iPXE.

Returns

a dictionary whose keys are the names of the images (deploy_kernel, deploy_ramdisk, or rescue_kernel, rescue_ramdisk) and values are the absolute paths of them.

Raises

MissingParameterValue, if deploy_kernel/deploy_ramdisk or rescue_kernel/rescue_ramdisk is missing in nodes driver_info.

ironic.common.pxe_utils.**get_instance_image_info**(*task*, *ipxe_enabled=False*)

Generate the paths for TFTP files for instance related images.

This method generates the paths for instance kernel and instance ramdisk. This method also updates the node, so caller should already have a non-shared lock on the node.

Parameters

- **task** A TaskManager instance containing node and context.
- **ipxe_enabled** Default false boolean to indicate if ipxe is in use by the caller.

Returns

a dictionary whose keys are the names of the images (kernel, ramdisk) and values are the absolute paths of them. If its a whole disk image or node is configured for localboot, it returns an empty dictionary.

ironic.common.pxe_utils.**get_kernel_ramdisk_info**(*node_uuid*, *driver_info*, *mode='deploy'*, *ipxe_enabled=False*)

Get href and tftp path for deploy or rescue kernel and ramdisk.

Parameters

- **node_uuid** UUID of the node
- **driver_info** Nodes driver_info dict
- **mode** A label to indicate whether paths for deploy or rescue ramdisk are being requested. Supported values are deploy rescue. Defaults to deploy, indicating deploy paths will be returned.
- **ipxe_enabled** A default False boolean value to tell the method if the caller is using iPXE.

Returns

a dictionary whose keys are deploy_kernel and deploy_ramdisk or rescue_kernel and rescue_ramdisk and whose values are the absolute paths to them.

Note: driver_info should be validated outside of this method.

ironic.common.pxe_utils.**get_pxe_config_file_path**(*node_uuid*, *ipxe_enabled=False*, *http_boot_enabled=False*)

Generate the path for the nodes PXE configuration file.

Parameters

- **node_uuid** the UUID of the node.
- **ipxe_enabled** A default False boolean value to tell the method if the caller is using iPXE.

Returns

The path to the nodes PXE configuration file.

ironic.common.pxe_utils.**get_volume_pxe_options**(*task*)

Identify volume information for iPXE template generation.

ironic.common.pxe_utils.**is_ipxe_enabled**(*task*)

Return true if ipxe is set.

Parameters

task A TaskManager object

Returns

boolean true if the task driver instance is the iPXE driver.

ironic.common.pxe_utils.**parse_driver_info**(*node*, *mode='deploy'*)

Gets the driver specific Node deployment info.

This method validates whether the driver_info property of the supplied node contains the required information for this driver to deploy images to, or rescue, the node.

Parameters

- **node** a single Node.

- **mode** Label indicating a deploy or rescue operation being carried out on the node. Supported values are deploy and rescue. Defaults to deploy, indicating deploy operation is being carried out.

Returns
A dict with the driver_info values.

Raises
MissingParameterValue

ironic.common.pxe_utils.**place_common_config**()

Place template generated config which is not node specific.

Currently places the initial grub config for grub network boot.

ironic.common.pxe_utils.**place_loaders_for_boot**(*base_path: str | PathLike*)

Place configured bootloaders from the host OS.

Example: grubaa64.efi:/path/to/grub-aarch64.efi,

Parameters
base_path Destination path where files should be copied to.

ironic.common.pxe_utils.**prepare_instance_kickstart_config**(*task*, *image_info*, *anaconda_boot=False*)

Prepare to boot anaconda ramdisk by generating kickstart file

Parameters

- **task** a task from TaskManager.
- **image_info** a dict of values of instance image metadata to set on the configuration file.
- **anaconda_boot** if the boot is to a anaconda ramdisk configuration.

ironic.common.pxe_utils.**prepare_instance_pxe_config**(*task*, *image_info*, *iscsi_boot=False*, *ramdisk_boot=False*, *ipxe_enabled=False*, *anaconda_boot=False*, *http_boot_enabled=False*)

Prepares the config file for PXE boot

Parameters

- **task** a task from TaskManager.
- **image_info** a dict of values of instance image metadata to set on the configuration file.
- **iscsi_boot** if boot is from an iSCSI volume or not.
- **ramdisk_boot** if the boot is to a ramdisk configuration.
- **ipxe_enabled** Default false boolean to indicate if ipxe is in use by the caller.
- **anaconda_boot** if the boot is to a anaconda ramdisk configuration.
- **http_boot_enabled** If httpboot models of use are to be used with the underlying boot loaders.

Returns
None

ironic.common.pxe_utils.**validate_kickstart_file**(*ks_cfg*)

Check if the kickstart file is valid

Parameters
ks_cfg Contents of kickstart file to validate

Raises
InvalidKickstartFile

ironic.common.pxe_utils.**validate_kickstart_template**(*ks_template*)

Validate the kickstart template

Parameters
ks_template Path to the kickstart template

Raises
InvalidKickstartTemplate

ironic.common.qemu_img module

ironic.common.qemu_img.**convert_image**(*source*, *dest*, *out_format*, *run_as_root=False*, *cache=None*, *out_of_order=False*, *sparse_size=None*, *source_format='qcow2'*)

Convert image to other format.

ironic.common.raid module

ironic.common.raid.**SAS = 'sas'**

Serial Attached SCSI

ironic.common.raid.**SATA = 'sata'**

Serial AT Attachment

ironic.common.raid.**SCSI = 'scsi'**

Small Computer System Interface

ironic.common.raid.**filter_target_raid_config**(*node*, *create_root_volume=True*, *create_nonroot_volumes=True*)

Filter the target raid config based on root volume creation

This method can be used by any raid interface which wants to filter out target raid config based on condition whether the root volume will be created or not.

Parameters

- **node** a node object
- **create_root_volume** A boolean default value True governing if the root volume is returned else root volumes will be filtered out.
- **create_nonroot_volumes** A boolean default value True governing if the non root volume is returned else non-root volumes will be filtered out.

Raises
MissingParameterValue, if node.target_raid_config is missing or was found to be empty after skipping root volume and/or non-root volumes.

Returns
It will return filtered target_raid_config

ironic.common.raid.**get_logical_disk_properties**(*raid_config_schema*)

Get logical disk properties from RAID configuration schema.

This method reads the logical properties and their textual description from the schema that is passed.

Parameters
raid_config_schema A dictionary which is the schema to be used for getting properties that may be specified for the logical disk.

Returns
A dictionary containing the logical disk properties as keys and a textual description for them as values.

ironic.common.raid.**update_raid_info**(*node*, *raid_config*)

Update the nodes information based on the RAID config.

This method updates the nodes information to make use of the configured RAID for scheduling purposes (through properties[capabilities] and properties[local_gb]) and deploying purposes (using properties[root_device]).

Parameters

- **node** a node object
- **raid_config** The dictionary containing the current RAID configuration.

Raises
InvalidParameterValue, if raid_config has more than one root volume or if node.properties[capabilities] is malformed.

ironic.common.raid.**validate_configuration**(*raid_config*, *raid_config_schema*)

Validates the RAID configuration passed using JSON schema.

This method validates a RAID configuration against a RAID configuration schema.

Parameters

- **raid_config** A dictionary containing RAID configuration information
- **raid_config_schema** A dictionary which is the schema to be used for validation.

Raises
InvalidParameterValue, if validation of the RAID configuration fails.

ironic.common.release_mappings module

ironic.common.release_mappings.**get_object_versions**(*releases=None*, *objects=None*)

Gets the supported versions for all objects.

Supported versions are from the RELEASE_MAPPINGs.

Parameters

- **releases** a list of release names; if empty/None, versions from all releases are returned (the default).
- **objects** a list of names of objects of interest. If empty/None, versions of all objects are returned (the default).

Returns

a dictionary where the key is the object name and the value is a set of supported versions.

ironic.common.rpc module

class ironic.common.rpc.**RequestContextSerializer**(*base*)

Bases: Serializer

deserialize_context(*context*)

Deserialize a dictionary into a request context.

Parameters

ctxt Request context dictionary

Returns

Deserialized form of entity

deserialize_entity(*context*, *entity*)

Deserialize something from primitive form.

Parameters

- **ctxt** Request context, in deserialized form
- **entity** Primitive to be deserialized

Returns

Deserialized form of entity

serialize_context(*context*)

Serialize a request context into a dictionary.

Parameters

ctxt Request context

Returns

Serialized form of context

serialize_entity(*context*, *entity*)

Serialize something to primitive form.

Parameters

- **ctxt** Request context, in deserialized form
- **entity** Entity to be serialized

Returns

Serialized form of entity

ironic.common.rpc.**cleanup**()

ironic.common.rpc.**get_allowed_exmods**()

ironic.common.rpc.**get_client**(*target*, *version_cap=None*, *serializer=None*)

ironic.common.rpc.**get_sensors_notifier**(*service=None*, *host=None*, *publisher_id=None*)

ironic.common.rpc.**get_server**(*target*, *endpoints*, *serializer=None*)

ironic.common.rpc.**get_transport_url**(*url_str=None*)

ironic.common.rpc.**get_versioned_notifier**(*publisher_id=None*)

ironic.common.rpc.**init**(*conf*)

ironic.common.rpc.**set_defaults**(*control_exchange*)

ironic.common.rpc.**set_global_manager**(*manager*)

ironic.common.rpc_service module

class ironic.common.rpc_service.**BaseRPCService**(*host*, *manager_module*, *manager_class*)

Bases: Service

start()

Start service.

ironic.common.service module

ironic.common.service.**ensure_rpc_transport**(*conf=<oslo_config.cfg.ConfigOpts object>*)

ironic.common.service.**prepare_command**(*argv=None*)

Prepare any Ironic command for execution.

Sets up configuration and logging, registers objects.

ironic.common.service.**prepare_service**(*name*, *argv=None*,
conf=<oslo_config.cfg.ConfigOpts object>)

Prepare an Ironic service executable.

In addition to what *prepare_command* does, set up guru meditation reporting and profiling.

ironic.common.service.**process_launcher**(***kwargs*)

ironic.common.state_machine module

Mapping of bare metal node states.

Setting the node *power_state* is handled by the conductors power synchronization thread. Based on the power state retrieved from the driver for the node, the state is set to POWER_ON or POWER_OFF, accordingly. Should this fail, the *power_state* value is left unchanged, and the node is placed into maintenance mode.

The *power_state* can also be set manually via the API. A failure to change the state leaves the current state unchanged. The node is NOT placed into maintenance mode in this case.

ironic.common.state_machine.**on_enter**(*new_state*, *event*)

Used to log when entering a state.

ironic.common.state_machine.**on_exit**(*old_state*, *event*)

Used to log when a state is exited.

ironic.common.states module

Constants for bare metal node states.

This module contains only state constant definitions with no executable code. For the state machine implementation, see ironic.common.states.

ironic.common.states.**ACTIVE = 'active'**

Node is successfully deployed and associated with an instance.

ironic.common.states.**ADOPTFAIL = 'adopt failed'**

Node failed to complete the adoption process.

This state is the resulting state of a node that failed to complete adoption, potentially due to invalid or incompatible information being defined for the node.

ironic.common.states.**ADOPTING = 'adopting'**

Node is being adopted.

This provision state is intended for use to move a node from MANAGEABLE to ACTIVE state to permit designation of nodes as being managed by Ironic, however deployed previously by external means.

ironic.common.states.**AVAILABLE = 'available'**

Node is available for use and scheduling.

This state is replacing the NOSTATE state used prior to Kilo.

ironic.common.states.**CLEANFAIL = 'clean failed'**

Node failed cleaning. This requires operator intervention to resolve.

ironic.common.states.**CLEANHOLD = 'clean hold'**

Node is a holding state due to a clean step.

ironic.common.states.**CLEANING = 'cleaning'**

Node is being automatically cleaned to prepare it for provisioning.

ironic.common.states.**CLEANWAIT = 'clean wait'**

Node is waiting for a clean step to be finished.

This will be the nodes *provision_state* while the node is waiting for the driver to finish a cleaning step.

ironic.common.states.**DELETED = 'deleted'**

Node tear down was successful.

This is a transitory value of provision_state, and never represented in target_provision_state.

ironic.common.states.**DELETE_ALLOWED_STATES = ('manageable', 'enroll', 'adopt failed')**

States in which node deletion is allowed.

ironic.common.states.**DELETING = 'deleting'**

Node is actively being torn down.

ironic.common.states.**DEPLOY = 'deploy'**

Node is successfully deployed and associated with an instance. This is an alias for ACTIVE.

ironic.common.states.**DEPLOYDONE = 'deploy complete'**

Node was successfully deployed.

This is mainly a target provision state used during deployment. A successfully deployed node should go to ACTIVE status.

ironic.common.states.**DEPLOYFAIL = 'deploy failed'**

Node deployment failed.

ironic.common.states.**DEPLOYHOLD = 'deploy hold'**

Node is being held by a deploy step.

ironic.common.states.**DEPLOYING = 'deploying'**

Node is ready to receive a deploy request, or is currently being deployed.

A node will have its *provision_state* set to DEPLOYING briefly before it receives its initial deploy request. It will also move to this state from DEPLOYWAIT after the callback is triggered and deployment is continued (disk partitioning and image copying).

ironic.common.states.**DEPLOYWAIT = 'wait call-back'**

Node is waiting to be deployed.

This will be the node *provision_state* while the node is waiting for the driver to finish deployment.

ironic.common.states.**ENROLL = 'enroll'**

Node is enrolled.

This state indicates that Ironic is aware of a node, but is not managing it.

ironic.common.states.**ERROR = 'error'**

An error occurred during node processing.

The *last_error* attribute of the node details should contain an error message.

ironic.common.states.**FASTTRACK_LOOKUP_ALLOWED_STATES = frozenset({'available', 'clean wait', 'cleaning', 'deploying', 'enroll', 'inspect wait', 'inspecting', 'manageable', 'rescue wait', 'rescuing', 'service hold', 'service wait', 'servicing', 'wait call-back'})**

States where API lookups are permitted with fast track enabled.

ironic.common.states.**INSPECTFAIL = 'inspect failed'**

Node inspection failed.

ironic.common.states.**INSPECTING = 'inspecting'**

Node is under inspection.

This is the provision state used when inspection is started. A successfully inspected node shall transition to MANAGEABLE state. For asynchronous inspection, node shall transition to INSPECTWAIT state.

ironic.common.states.**INSPECTWAIT = 'inspect wait'**

Node is under inspection.

This is the provision state used when an asynchronous inspection is in progress. A successfully inspected node shall transition to MANAGEABLE state.

ironic.common.states.**LOOKUP_ALLOWED_STATES = frozenset({'clean wait', 'cleaning', 'deploying', 'inspect wait', 'inspecting', 'rescue wait', 'rescuing', 'service wait', 'wait call-back'})**

States when API lookups are normally allowed for nodes.

ironic.common.states.**MANAGEABLE = 'manageable'**

Node is in a manageable state.

This state indicates that Ironic has verified, at least once, that it had sufficient information to manage the hardware. While in this state, the node is not available for provisioning (it must be in the AVAILABLE state for that).

ironic.common.states.**NOSTATE = None**

No state information.

This state is used with power_state to represent a lack of knowledge of power state, and in target_*_state fields when there is no target.

ironic.common.states.**POWER_OFF = 'power off'**

Node is powered off.

ironic.common.states.**POWER_ON = 'power on'**

Node is powered on.

ironic.common.states.**REBOOT = 'rebooting'**

Node is rebooting.

ironic.common.states.**REBUILD = 'rebuild'**

Node is to be rebuilt.

This is not used as a state, but rather as a verb when changing the nodes provision_state via the REST API.

ironic.common.states.**RESCUE = 'rescue'**

Node is in rescue mode.

ironic.common.states.**RESCUEFAIL = 'rescue failed'**

Node rescue failed.

ironic.common.states.**RESCUEWAIT = 'rescue wait'**

Node is waiting on an external callback.

This will be the node *provision_state* while the node is waiting for the driver to finish rescuing the node.

ironic.common.states.**RESCUING = 'rescuing'**

Node is in process of being rescued.

ironic.common.states.**SERVICE = 'service'**

Node is being requested to be modified through a service step.

ironic.common.states.**SERVICEFAIL = 'service failed'**

Node has failed in a service step execution.

ironic.common.states.**SERVICEHOLD = 'service hold'**

Node is being held for direct intervention from a service step.

ironic.common.states.**SERVICEWAIT = 'service wait'**

Node is waiting for an operation to complete.

ironic.common.states.**SERVICING = 'servicing'**

Node is actively being changed by a service step.

ironic.common.states.**SERVICING_STATES = frozenset({'service failed', 'service hold', 'service wait', 'servicing'})**

All Node states related to inspection.

ironic.common.states.**SOFT_POWER_OFF = 'soft power off'**

Node is in the process of soft power off.

ironic.common.states.**SOFT_REBOOT = 'soft rebooting'**

Node is rebooting gracefully.

ironic.common.states.**STABLE_STATES = ('enroll', 'manageable', 'available', 'active', 'error', 'rescue')**

States that will not transition unless receiving a request.

ironic.common.states.**STUCK_STATES_TREATED_AS_FAIL = ('deploying', 'cleaning', 'verifying', 'inspecting', 'adopting', 'rescuing', 'unrescuing', 'deleting', 'servicing')**

States that cannot be resumed once a conductor dies.

If a node gets stuck with one of these states for some reason (eg. conductor goes down when executing task), node will be moved to fail state.

ironic.common.states.**UNDEPLOY = 'undeploy'**

Node tear down process has started.

This is an alias for DELETED.

ironic.common.states.**UNRESCUEFAIL = 'unrescue failed'**

Node unrescue failed.

ironic.common.states.**UNRESCUING = 'unrescuing'**

Node is being restored from rescue mode (to active state).

ironic.common.states.**UNSTABLE_STATES = ('deploying', 'wait call-back', 'cleaning', 'clean wait', 'verifying', 'deleting', 'inspecting', 'inspect wait', 'adopting', 'rescuing', 'rescue wait', 'unrescuing', 'servicing', 'service wait')**

States that can be changed without external request.

ironic.common.states.**UPDATE_ALLOWED_STATES = ('deploy failed', 'inspecting', 'inspect failed', 'inspect wait', 'clean failed', 'error', 'verifying', 'adopt failed', 'rescue failed', 'unrescue failed', 'service', 'service hold', 'service failed')**

Transitional states in which we allow updating a node.

ironic.common.states.**VERBS = {'abort': 'abort', 'active': 'deploy', 'adopt': 'adopt', 'clean': 'clean', 'deleted': 'delete', 'deploy': 'deploy', 'inspect': 'inspect', 'manage': 'manage', 'provide': 'provide', 'rescue': 'rescue', 'service': 'service', 'undeploy': 'delete', 'unhold': 'unhold', 'unrescue': 'unrescue'}**

Mapping of state-changing events that are PUT to the REST API

This is a mapping of target states which are PUT to the API, eg,
PUT /v1/node/states/provision {target: active}

The dict format is:
{target string used by the API: internal verb}

This provides a reference set of supported actions, and in the future may be used to support renaming these actions.

ironic.common.states.**VERIFYING = 'verifying'**

Node power management credentials are being verified.

ironic.common.swift module

class ironic.common.swift.**SwiftAPI**

Bases: object

API for communicating with Swift.

create_object(*container*, *obj*, *filename*, *object_headers=None*)

Uploads a given file to Swift.

Parameters

- **container** The name of the container for the object.
- **obj** The name of the object in Swift
- **filename** The file to upload, as the object data
- **object_headers** the headers for the object to pass to Swift

Returns
The Swift UUID of the object

Raises
SwiftOperationError, if any operation with Swift fails.

create_object_from_data(*obj*, *data*, *container*)

Uploads a given string to Swift.

Parameters

- **obj** The name of the object in Swift
- **data** string data to put in the object
- **container** The name of the container for the object. Defaults to the value set in the configuration options.

Returns
The Swift UUID of the object

Raises
utils.Error, if any operation with Swift fails.

delete_object(*container*, *obj*)

Deletes the given Swift object.

Parameters

- **container** The name of the container in which Swift object is placed.
- **obj** The name of the object in Swift to be deleted.

Raises
SwiftObjectNotFoundError, if object is not found in Swift.

Raises
SwiftOperationError, if operation with Swift fails.

generate_temp_url(*path*, *timeout*, *method*, *temp_url_key*)

Returns the temp url for a given path

get_object(*object*, *container*)

Downloads a given object from Swift.

Parameters

- **object** The name of the object in Swift
- **container** The name of the container for the object. Defaults to the value set in the configuration options.

Returns
Swift object

Raises
utils.Error, if the Swift operation fails.

get_temp_url(*container*, *obj*, *timeout*)

Returns the temp url for the given Swift object.

Parameters

- **container** The name of the container in which Swift object is placed.
- **obj** The name of the Swift object.
- **timeout** The timeout in seconds after which the generated url should expire.

Returns
The temp url for the object.

Raises
SwiftOperationError, if any operation with Swift fails.

get_temp_url_key()

Get the best temporary url key from the account headers.

ironic.common.swift.**get_swift_session**()

ironic.common.utils module

Utilities and helper functions.

ironic.common.utils.**check_dir**(*directory_to_check=None*, *required_space=1*)

Check a directory is usable.

This function can be used by drivers to check that directories they need to write to are usable. This should be called from the drivers init function. This function checks that the directory exists and then calls check_dir_writable and check_dir_free_space. If directory_to_check is not provided the default is to use the temp directory.

Parameters

- **directory_to_check** the directory to check.
- **required_space** amount of space to check for in MiB.

Raises
PathNotFound if directory can not be found

Raises
DirectoryNotWritable if user is unable to write to the directory

Raises
InsufficientDiskSpace if free space is < required space

ironic.common.utils.**create_link_without_raise**(*source*, *link*)

ironic.common.utils.**dd**(*src*, *dst*, **args*)

Execute dd from src to dst.

Parameters

- **src** the input file for dd command.
- **dst** the output file for dd command.
- **args** a tuple containing the arguments to be passed to dd command.

Raises
processutils.ProcessExecutionError if it failed to run the process.

ironic.common.utils.**execute**(**cmd*, *use_standard_locale=False*, *log_stdout=True*, ***kwargs*)

Convenience wrapper around oslos execute() method.

Executes and logs results from a system command. See docs for oslo_concurrency.processutils.execute for usage.

Parameters

- **cmd** positional arguments to pass to processutils.execute()
- **use_standard_locale** Defaults to False. If set to True, execute command with standard locale added to environment variables.
- **log_stdout** Defaults to True. If set to True, logs the output.
- **kwargs** keyword arguments to pass to processutils.execute()

Returns
(stdout, stderr) from process execution

Raises
UnknownArgumentError on receiving unknown arguments

Raises
ProcessExecutionError

Raises
OSError

ironic.common.utils.**fast_track_enabled**(*node*)

ironic.common.utils.**file_has_content**(*path*, *content*, *hash_algo='sha256'*)

Checks that content of the file is the same as provided reference.

Parameters

- **path** path to file
- **content** reference content to check against
- **hash_algo** hashing algo from hashlib to use, default is sha256

Returns
True if the hash of reference content is the same as the hash of files content, False otherwise

ironic.common.utils.**file_mime_type**(*path*)

Gets a mime type of the given file.

ironic.common.utils.**find_devices_by_hints**(*devices*, *root_device_hints*)

Find all devices that match the root device hints.

Try to find devices that match the root device hints. In order for a device to be matched it needs to satisfy all the given hints.

Parameters

- **devices**

A list of dictionaries representing the devices
containing one or more of the following keys:

name
(String) The device name, e.g /dev/sda

size
(Integer) Size of the device in *bytes*

model
(String) Device model

vendor
(String) Device vendor name

serial
(String) Device serial number

wwn
(String) Unique storage identifier

wwn_with_extension
(String): Unique storage identifier with the vendor extension appended

wwn_vendor_extension
(String): United vendor storage identifier

rotational
(Boolean) Whether its a rotational device or not. Useful to distinguish HDDs (rotational) and SSDs (not rotational).

hctl
(String): The SCSI address: Host, channel, target and lun. For example: 1:0:0:0.

by_path
(String): The alternative device name, e.g. /dev/disk/by-path/pci-0000:00

- **root_device_hints** A dictionary with the root device hints.

Raises
ValueError, if some information is invalid.

Returns
A generator with all matching devices as dictionaries.

ironic.common.utils.**get_route_source**(*dest*, *ignore_link_local=True*)

Get the IP address to send packages to destination.

ironic.common.utils.**get_updated_capabilities**(*current_capabilities*, *new_capabilities*)

Returns an updated capability string.

This method updates the original (or current) capabilities with the new capabilities. The original capabilities would typically be from a nodes properties[capabilities]. From new_capabilities, any new capabilities are added, and existing capabilities may have their values updated. This updated capabilities string is returned.

Parameters

- **current_capabilities** Current capability string
- **new_capabilities** the dictionary of capabilities to be updated.

Returns
An updated capability string. with new_capabilities.

Raises
ValueError, if current_capabilities is malformed or if new_capabilities is not a dictionary

ironic.common.utils.**is_fips_enabled**()

Check if FIPS mode is enabled in the system.

ironic.common.utils.**is_hostname_safe**(*hostname*)

Old check for valid logical node names.

Retained for compatibility with REST API < 1.10.

Nominally, checks that the supplied hostname conforms to:

- http://en.wikipedia.org/wiki/Hostname
- http://tools.ietf.org/html/rfc952
- http://tools.ietf.org/html/rfc1123

In practice, this check has several shortcomings and errors that are more thoroughly documented in bug #1468508.

Parameters
hostname The hostname to be validated.

Returns
True if valid. False if not.

ironic.common.utils.**is_http_url**(*url*)

ironic.common.utils.**is_ironic_using_sqlite**()

Return True if Ironic is configured to use SQLite

ironic.common.utils.**is_loopback**(*hostname_or_ip*)

Check if the provided hostname or IP address is a loopback.

ironic.common.utils.**is_memory_insufficient**(*raise_if_fail=False*)

Checks available system memory and holds the deployment process.

Evaluates the current system memory available, meaning can be allocated to a process by the kernel upon allocation request, and delays the execution until memory has been freed, or until it has timed out.

This method will issue a sleep, if the amount of available memory is insufficient. This is configured using the `[DEFAULT]minimum_memory_wait_time` and the `[DEFAULT]minimum_memory_wait_retries`.

Parameters
raise_if_fail Default False, but if set to true an InsufficientMemory exception is raised upon insufficient memory.

Returns
True if the check has timed out. Otherwise None is returned.

Raises
InsufficientMemory if the raise_if_fail parameter is set to True.

ironic.common.utils.**is_regex_string_in_file**(*path*, *string*)

ironic.common.utils.**is_valid_datapath_id**(*datapath_id*)

Verify the format of an OpenFlow datapath_id.

Check if a datapath_id is valid and contains 16 hexadecimal digits. Datapath ID format: the lower 48-bits are for a MAC address, while the upper 16-bits are implementer-defined.

Parameters
datapath_id OpenFlow datapath_id to be validated.

Returns
True if valid. False if not.

ironic.common.utils.**is_valid_logical_name**(*hostname*)

Determine if a logical name is valid.

The logical name may only consist of RFC3986 unreserved characters:

ALPHA / DIGIT / - / . / _ / ~

ironic.common.utils.**is_valid_no_proxy**(*no_proxy*)

Check no_proxy validity

Check if no_proxy value that will be written to environment variable by ironic-python-agent is valid.

Parameters

no_proxy the value that requires validity check. Expected to be a comma-separated list of host names, IP addresses and domain names (with optional :port).

Returns

True if no_proxy is valid, False otherwise.

ironic.common.utils.**match_root_device_hints**(*devices*, *root_device_hints*)

Try to find a device that matches the root device hints.

Try to find a device that matches the root device hints. In order for a device to be matched it needs to satisfy all the given hints.

Parameters

- **devices**

A list of dictionaries representing the devices
containing one or more of the following keys:

name
(String) The device name, e.g /dev/sda

size
(Integer) Size of the device in *bytes*

model
(String) Device model

vendor
(String) Device vendor name

serial
(String) Device serial number

wwn
(String) Unique storage identifier

wwn_with_extension
(String): Unique storage identifier with the vendor extension appended

wwn_vendor_extension
(String): United vendor storage identifier

rotational
(Boolean) Whether its a rotational device or not. Useful to distinguish HDDs (rotational) and SSDs (not rotational).

hctl
(String): The SCSI address: Host, channel, target and lun. For example: 1:0:0:0.

by_path
(String): The alternative device name, e.g. /dev/disk/by-path/pci-0000:00

- **root_device_hints** A dictionary with the root device hints.

Raises
ValueError, if some information is invalid.

Returns
The first device to match all the hints or None.

ironic.common.utils.**mkfs**(*fs*, *path*, *label=None*)

Format a file or block device

Parameters

- **fs** Filesystem type (examples include swap, ext3, ext4 btrfs, etc.)
- **path** Path to file or block device to format
- **label** Volume label to use

ironic.common.utils.**normalize_mac**(*address*)

Normalize a MAC address without validating it.

Parameters
address MAC address to be normalized.

Returns
Normalized MAC address.

ironic.common.utils.**parse_instance_info_capabilities**(*node*)

Parse the instance_info capabilities.

One way of having these capabilities set is via Nova, where the capabilities are defined in the Flavor extra_spec and passed to Ironic by the Nova Ironic driver.

NOTE: Although our API fully supports JSON fields, to maintain the backward compatibility with Juno the Nova Ironic driver is sending it as a string.

Parameters
node a single Node.

Raises
InvalidParameterValue if the capabilities string is not a dictionary or is malformed.

Returns
A dictionary with the capabilities if found, otherwise an empty dictionary.

ironic.common.utils.**parse_kernel_params**(*params*)

Parse kernel parameters into a dictionary.

`None` is used as a value for parameters that are not in the `key=value` format.

Parameters
params kernel parameters as a space-delimited string.

ironic.common.utils.**parse_root_device_hints**(*root_device*)

Parse the root_device property of a node.

Parses and validates the root_device property of a node. These are hints for how a nodes root device is created. The size hint should be a positive integer. The rotational hint should be a Boolean value.

Parameters
root_device the root_device dictionary from the nodes property.

Returns
a dictionary with the root device hints parsed or None if there are no hints.

Raises
ValueError, if some information is invalid.

ironic.common.utils.**pop_node_nested_field**(*node*, *collection*, *field*, *default=None*)

Pop a value from a dictionary field of a node.

Parameters

- **node** Node object.
- **collection** Name of the field with the dictionary.
- **field** Nested field name.
- **default** The default value to return.

Returns
The removed value or the default.

ironic.common.utils.**remove_large_keys**(*var*)

Remove specific keys from the var, recursing into dicts and lists.

ironic.common.utils.**render_template**(*template*, *params*, *is_file=True*, *strict=False*)

Renders Jinja2 template file with given parameters.

Parameters

- **template** full path to the Jinja2 template file
- **params** dictionary with parameters to use when rendering
- **is_file** whether template is file or string with template itself
- **strict** Enable strict template rendering. Default is False

Returns
Rendered template

Raises
jinja2.exceptions.UndefinedError

ironic.common.utils.**rmtree_without_raise**(*path*)

ironic.common.utils.**safe_rstrip**(*value*, *chars=None*)

Removes trailing characters from a string if that does not make it empty

Parameters

- **value** A string value that will be stripped.
- **chars** Characters to remove.

Returns
Stripped value.

ironic.common.utils.**set_node_nested_field**(*node*, *collection*, *field*, *value*)

Set a value in a dictionary field of a node.

Parameters

- **node** Node object.
- **collection** Name of the field with the dictionary.
- **field** Nested field name.
- **value** New value.

ironic.common.utils.**stop_after_retries**(*option*, *group=None*)

A tenacity retry helper that stops after retries specified in conf.

ironic.common.utils.**tempdir**(***kwargs*)

ironic.common.utils.**try_execute**(**cmd*, ***kwargs*)

The same as execute but returns None on error.

Executes and logs results from a system command. See docs for oslo_concurrency.processutils.execute for usage.

Instead of raising an exception on failure, this method simply returns None in case of failure.

Parameters

- **cmd** positional arguments to pass to processutils.execute()
- **kwargs** keyword arguments to pass to processutils.execute()

Raises
UnknownArgumentError on receiving unknown arguments

Returns
tuple of (stdout, stderr) or None in some error cases

ironic.common.utils.**unix_file_modification_datetime**(*file_name*)

ironic.common.utils.**unlink_without_raise**(*path*)

ironic.common.utils.**validate_and_normalize_datapath_id**(*datapath_id*)

Validate an OpenFlow datapath_id and return normalized form.

Checks whether the supplied OpenFlow datapath_id is formally correct and normalize it to all lower case.

Parameters
datapath_id OpenFlow datapath_id to be validated and normalized.

Returns
Normalized and validated OpenFlow datapath_id.

Raises
InvalidDatapathID If an OpenFlow datapath_id is not valid.

ironic.common.utils.**validate_and_normalize_mac**(*address*)

Validate a MAC address and return normalized form.

Checks whether the supplied MAC address is formally correct and normalize it to all lower case.

Parameters
address MAC address to be validated and normalized.

Returns
Normalized and validated MAC address.

Raises
InvalidMAC If the MAC address is not valid.

ironic.common.utils.**validate_conductor_group**(*conductor_group*)

ironic.common.utils.**validate_network_port**(*port*, *port_name='Port'*)

Validates the given port.

Parameters

- **port** TCP/UDP port.
- **port_name** Name of the port.

Returns
An integer port number.

Raises
InvalidParameterValue, if the port is invalid.

ironic.common.utils.**write_to_file**(*path*, *contents*, *permission=None*)

ironic.common.vnc module

Functions for VNC graphical console drivers and novncproxy

ironic.common.vnc.**get_console**(*node*)

Get the type and connection information about the console

Parameters
node the node object

Returns
A dict containing keys type, url

ironic.common.vnc.**novnc_authorize**(*node*)

Create and save a console token.

A random token is created and is stored in the node `driver_internal_info` along with creation time.

This is called by graphical console drivers when `start_console` is called.

Parameters
node the node object

Returns
an authorized token

ironic.common.vnc.**novnc_unauthorize**(*node*)

Clear any existing console token

Parameters
node the node object

ironic.common.vnc.**novnc_validate**(*node*, *token*)

Validate the token.

Parameters

- **node** the node object
- **token** the token for the authorization

Returns
The ConsoleAuthToken object if valid

The token is valid if the token is in the database and the expires time has not passed.

ironic.common.vnc.**token_valid_until**(*node*)

Calculate when the token will expire

Parameters
node the node object

Returns
a datetime object representing expiration time

Raises
NotAuthorized if no timestamp is stored in the node

ironic.common.wsgi_service module

class ironic.common.wsgi_service.**BaseWSGIService**(*name*, *app*, *conf*, *use_ssl=None*)

Bases: ServiceBase

reset()

No server greenpools to resize.

start()

Start serving this service using loaded configuration.

Returns
None

stop()

Stop serving this API.

Returns
None

wait()

Wait for the service to stop serving this API.

Returns
None

class ironic.common.wsgi_service.**WSGIService**(*name*, *use_ssl=False*)

Bases: *BaseWSGIService*

Provides ability to launch ironic API from wsgi app.

ironic.common.wsgi_service.**validate_cert_paths**(*cert_file*, *key_file*)

Module contents

ironic.conductor package

Submodules

ironic.conductor.allocations module

Functionality related to allocations.

ironic.conductor.allocations.**backfill_allocation**(*context*, *allocation*, *node_id*)

Assign the previously allocated node to the node allocation.

This is not the actual allocation process, but merely backfilling of allocation_uuid for a previously allocated node.

Parameters

- **context** an admin context
- **allocation** an allocation object associated with the node
- **node_id** An ID of the node.

Raises
AllocationFailed if the node does not match the allocation

Raises
NodeAssociated if the node is already associated with another instance or allocation.

Raises
InstanceAssociated if the allocations UUID is already used on another node as instance_uuid.

Raises
NodeNotFound if the node with the provided ID cannot be found.

ironic.conductor.allocations.**do_allocate**(*context*, *allocation*)

Process the allocation.

This call runs in a separate thread on a conductor. It finds suitable nodes for the allocation and reserves one of them.

This call does not raise exceptions since its designed to work asynchronously.

Parameters

- **context** an admin context
- **allocation** an allocation object

ironic.conductor.allocations.**verify_node_for_deallocation**(*node*, *allocation*)

Verify that allocation can be removed for the node.

Parameters

- **node** a node object
- **allocation** an allocation object associated with the node

ironic.conductor.base_manager module

Base conductor manager functionality.

class ironic.conductor.base_manager.**BaseConductorManager**(*host*, *topic*)

Bases: object

del_host(*deregister=True*, *clear_node_reservations=True*)

get_online_conductor_count()

Return a count of currently online conductors

has_reserved()

Determines if this host currently has any reserved nodes

Returns

True if this host has reserved nodes

init_host(*admin_context=None*, *start_consoles=True*, *start_allocations=True*)

Initialize the conductor host.

Parameters

- **admin_context** the admin context to pass to periodic tasks.
- **start_consoles** If consoles should be started in initialization.
- **start_allocations** If allocations should be started in initialization.

Raises

RuntimeError when conductor is already running.

Raises

NoDriversLoaded when no drivers are enabled on the conductor.

Raises

DriverNotFound if a driver is enabled that does not exist.

Raises

DriverLoadError if an enabled driver cannot be loaded.

Raises

DriverNameConflict if a classic driver and a dynamic driver are both enabled and have the same name.

Raises

IncorrectConfiguration if invalid configuration has been set which cannot permit proper operation.

iter_nodes(*fields=None*, ***kwargs*)

Iterate over nodes mapped to this conductor.

Requests node set from and filters out nodes that are not mapped to this conductor.

Yields tuples (node_uuid, driver, conductor_group,) where is derived from fields argument, e.g.: fields=None means yielding (uuid, driver, conductor_group), fields=[foo] means yielding (uuid, driver, conductor_group, foo).

Parameters

- **fields** list of fields to fetch in addition to uuid, driver, and conductor_group
- **kwargs** additional arguments to pass to dbapi when looking for nodes

Returns

generator yielding tuples of requested fields

keepalive_halt()

prepare_host()

Prepares host for initialization

Prepares the conductor for basic operation by removing any existing transitory node power states and reservations which were previously held by this host.

Under normal operation, this is also when the initial database connectivity is established for the conductors normal operation.

ironic.conductor.base_manager.**reject_when_reached**(*pool_size*)

Return a function to reject new work for Ironic.

Parameters

pool_size The maximum number of items in the execution pool.

Returns

A function which is executed by futurist when attempting to determine if work should be deleted, or not.

ironic.conductor.cleaning module

Functionality related to cleaning.

ironic.conductor.cleaning.**continue_node_clean**(*task*)

Continue cleaning after finishing an async clean step.

This function calculates which step has to run next and passes control into do_next_clean_step.

Parameters

task a TaskManager instance with an exclusive lock

ironic.conductor.cleaning.**do_next_clean_step**(*task*, *step_index*, *disable_ramdisk=None*)

Do cleaning, starting from the specified clean step.

Parameters

- **task** a TaskManager instance with an exclusive lock

- **step_index** The first clean step in the list to execute. This is the index (from 0) into the list of clean steps in the nodes driver_internal_info[clean_steps]. Is None if there are no steps to execute.
- **disable_ramdisk** Whether to skip booting ramdisk for cleaning.

ironic.conductor.cleaning.**do_node_clean**(*task*, *clean_steps=None*, *disable_ramdisk=False*, *automated_with_steps=False*)

Internal RPC method to perform cleaning of a node.

Parameters

- **task** a TaskManager instance with an exclusive lock on its node
- **clean_steps** For a manual clean, the list of clean steps to perform. Is None For automated cleaning (default). For more information, see the clean_steps parameter of `ConductorManager.do_node_clean()`.
- **disable_ramdisk** Whether to skip booting ramdisk for cleaning.
- **automated_with_steps** Indicates this is a declarative automated clean powered by a runbook.

ironic.conductor.cleaning.**do_node_clean_abort**(*task*)

Internal method to abort an ongoing operation.

Parameters

task a TaskManager instance with an exclusive lock

ironic.conductor.cleaning.**execute_step_on_child_nodes**(*task*, *step*)

Execute a requested step against a child node.

Parameters

- **task** The TaskManager object for the parent node.
- **step** The requested step to be executed.

Returns

None on Success, the resulting error message if a failure has occurred.

ironic.conductor.cleaning.**get_cleaning_flow**(*manual_clean=False*)

ironic.conductor.cleaning.**get_last_error**(*node*)

ironic.conductor.configdrive_utils module

ironic.conductor.configdrive_utils.**build_configdrive**(*node*, *configdrive*)

Build a configdrive from provided meta_data, network_data and user_data.

If uuid or name are not provided in the meta_data, theyre defaulted to the nodes uuid and name accordingly.

Parameters

- **node** an Ironic node object.
- **configdrive** A configdrive as a dict with keys `meta_data`, `network_data`, `user_data` and `vendor_data` (all optional).

Returns
A gzipped and base64 encoded configdrive as a string.

ironic.conductor.configdrive_utils.**check_and_fix_configdrive**(*task*, *configdrive*)

Check and fix the supplied config drive.

In certain cases, a caller may supply a horribly broken configuration drive which contains a list of empty dicts or even is an ISO in the same state.

The prupose of this method is to attempt to identify and correct those specific cases to help ensure the instance being deployed has what is necessary to boot.

If this method fails, an internal exception, ConfigDriveRegenerationFailure is raised by the various helper methods used by this method, however this method will just return the user supplied

Parameters

- **task** A Taskmanager object.
- **configdrive** The supplied configuration drive.

Returns
Returns a configuration drive aligning with the type which was originally supplied by the user.

ironic.conductor.configdrive_utils.**check_and_patch_configdrive**(*task*, *configdrive*)

Evaluates and patches an provided ISO9660 configuration drive file.

This method is responsible for taking the user requested configuration drive, performing basic evaluation, attempting to patch/fix the configuration drive to enable the latest metadata written to be valid and correct, which is intended to allow the overall deployment process to proceed with the correct data.

Parameters

- **task** A TaskManager object.
- **configdrive** The in-memory configuration drive file which was submitted by the requester.

Returns
The submitted configuration drive or a regenerated and patched configuration drive OR the original requested configuration drive *if* a failure is encountered in the overall process.

ironic.conductor.configdrive_utils.**generate_config_metadata**(*task*)

Generates new network config metadata.

This method wraps generate_instance_network_data and performs the logging actions in relation to it, and also performs a post generation sanity check on the network metadata.

When the data is invalid, this method *also* raises an internal ConfigDriveRegenerationFailure exception which is a signal to the caller method that an issue has occurred. The expectation being that the caller knows how to handle and navigate that issue without failing the deployment operation.

Parameters
task A TaskManager object.

Returns
A dictionary object with three keys, links, networks, and services which contains

various related entries as generated by the internal neutron get_neutron_port_data method which is then combined such that bond ports are also honored.

Raises

ConfigDriveRegenerationFailure when no metadata is generated or the configuration state of the drivers does not support the generation of network metadata. The purpose of this is to appropriately signal an error to the caller, not fail the overall requested operation by the end user.

ironic.conductor.configdrive_utils.**generate_instance_network_data**(*task*)

Generates OpenStack instance network metadata.

This method was added to help facilitate the correction of cases where bad metadata could be supplied to Ironic by a service such as Nova where an instance has been requested, but missing network configuration details harms the viability of the overall instances deployment.

This method works by taking the available information which Ironic manages and controls, and reconciles it together into an OpenStack style network_data document which can then be rendered as network_data.json.

Parameters

task A TaskManager class instance.

Returns

A dictionary object with three keys, links, networks, and services which contains various related entries as generated by the internal neutron get_neutron_port_data method which is then combined such that bond ports are also honored.

ironic.conductor.configdrive_utils.**get_configdrive_image**(*node*)

Get configdrive as an ISO image or a URL.

Converts the JSON representation into an image. URLs and raw contents are returned unchanged.

Parameters

node an Ironic node object.

Returns

A gzipped and base64 encoded configdrive as a string.

ironic.conductor.configdrive_utils.**is_invalid_network_metadata**(*network_data*)

Return True if network metadata is invalid.

Parameters

network_data A dictionary object containing network_metadata, with three subfields, links, networks, and services. Returns True if the document is invalid and missing network metadata, or when specific other issues such as null MTU values are detected.

Returns

True when the data supplied is invalid.

ironic.conductor.configdrive_utils.**regenerate_iso**(*source*, *dest*, *override_files*, *node_uuid=None*)

Utility method to regenerate a config drive ISO image.

This method takes the source, extracts the contents in a temporary folder, applies override contents, and then re-packages the contents into the dest file supplied as a parameter.

Parameters

- **source** A PyCdlib class object of an opened ISO file to extract.
- **dest** The destination filename to write the result to.
- **override_files** A dictionary of file name keys and contents stored as values paris, which will be used to override contents of the configuration drive for the purpose of patching metadata.
- **node_uuid** The node UUID, for logging purposes.

ironic.conductor.deployments module

Functionality related to deploying and undeploying.

ironic.conductor.deployments.**apply_automatic_lessee**(*task*)

Apply a automatic lessee to the node, if applicable

First of all, until removed next cycle, we check to see if CONF.automatic_lessee was explicitly set False by an operator if so, we do not apply a lessee.

When CONF.conductor.automatic_lessee_source is instance: - Take the lessee from instance_info[project_id] (e.g. as set by nova)

When CONF.conductor.automatic_lessee_source is request: - Take the lessee from request context (e.g. from keystone)

When CONF.conductor.automatic_lessee_source is none: OR the legacy CONF.automatic_lessee is explicitly set by an operator to False (regardless of lessee_source) - Dont apply a lessee to the node

Parameters
task a TaskManager instance.

Returns
True if node had a lessee applied

ironic.conductor.deployments.**continue_node_deploy**(*task*)

Continue deployment after finishing an async deploy step.

This function calculates which step has to run next and passes control into do_next_deploy_step. On the first run, deploy steps and templates are also validated.

Parameters
task a TaskManager instance with an exclusive lock

ironic.conductor.deployments.**do_next_deploy_step**(*task*, *step_index*)

Do deployment, starting from the specified deploy step.

Parameters

- **task** a TaskManager instance with an exclusive lock
- **step_index** The first deploy step in the list to execute. This is the index (from 0) into the list of deploy steps in the nodes driver_internal_info[deploy_steps]. Is None if there are no steps to execute.

ironic.conductor.deployments.**do_node_deploy**(*task*, *conductor_id=None*, *configdrive=None*, *deploy_steps=None*)

Prepare the environment and deploy a node.

ironic.conductor.deployments.**do_node_deploy_abort**(*task*)

Abort an ongoing deployment operation.

Parameters

task a TaskManager instance with an exclusive lock

ironic.conductor.deployments.**execute_step_on_child_nodes**(*task*, *step*)

Execute a requested step against a child node.

Parameters

- **task** The TaskManager object for the parent node.
- **step** The requested step to be executed.

Returns

None on Success, the resulting error message if a failure has occurred.

ironic.conductor.deployments.**start_deploy**(*task*, *manager*, *configdrive=None*, *event='deploy'*, *deploy_steps=None*)

Start deployment or rebuilding on a node.

This function does not check the node suitability for deployment, its left up to the caller.

Parameters

- **task** a TaskManager instance.
- **manager** a ConductorManager to run tasks on.
- **configdrive** a configdrive, if requested.
- **event** event to process: deploy or rebuild.
- **deploy_steps** Optional deploy steps.

ironic.conductor.deployments.**validate_deploy_steps**(*task*)

Validate the deploy steps after the ramdisk learns about them.

ironic.conductor.deployments.**validate_node**(*task*, *event='deploy'*)

Validate that a node is suitable for deployment/rebuilding.

Parameters

- **task** a TaskManager instance.
- **event** event to process: deploy or rebuild.

Raises

NodeInMaintenance, NodeProtected, InvalidStateRequested, BootModeNotAllowed

ironic.conductor.inspection module

Inspection implementation for the conductor.

ironic.conductor.inspection.**abort_inspection**(*task*)

Abort inspection for the node.

ironic.conductor.inspection.**continue_inspection**(*task*, *inventory*, *plugin_data*)

Continue inspection for the node.

ironic.conductor.inspection.**inspect_hardware**(*task*)

Initiates inspection.

Parameters

task a TaskManager instance with an exclusive lock on its node.

Raises

HardwareInspectionFailure if driver doesnt return the state as states.MANAGEABLE, states.INSPECTWAIT.

ironic.conductor.manager module

Conduct all activity related to bare-metal deployments.

A single instance of *ironic.conductor.manager.ConductorManager* is created within the *ironic-conductor* process, and is responsible for performing all actions on bare metal resources (Chassis, Nodes, and Ports). Commands are received via RPCs. The conductor service also performs periodic tasks, eg. to monitor the status of active deployments.

Drivers are loaded via entrypoints by the *ironic.common.driver_factory* class. Each driver is instantiated only once, when the ConductorManager service starts. In this way, a single ConductorManager may use multiple drivers, and manage heterogeneous hardware.

When multiple *ConductorManager* are run on different hosts, they are all active and cooperatively manage all nodes in the deployment. Nodes are locked by each conductor when performing actions which change the state of that node; these locks are represented by the *ironic.conductor.task_manager.TaskManager* class.

A tooz.hashring.HashRing is used to distribute nodes across the set of active conductors which support each nodes driver. Rebalancing this ring can trigger various actions by each conductor, such as building or tearing down the TFTP environment for a node, notifying Neutron of a change, etc.

class ironic.conductor.manager.**ConductorManager**(*host*, *topic='ironic.conductor_manager'*)

Bases: *BaseConductorManager*

Ironic Conductor manager main class.

RPC_API_VERSION = '1.62'

add_node_traits(***kwargs*)

attach_virtual_media(***kwargs*)

change_node_boot_mode(***kwargs*)

change_node_power_state(***kwargs*)

change_node_secure_boot(***kwargs*)

cleanup_stale_conductors(*context*)

Periodically clean up stale conductors from the database.

This task removes conductors that have been offline for longer than the configured timeout period. This helps prevent accumulation of stale conductor records in the database.

continue_inspection(***kwargs*)

continue_node_clean(*context*, *node_id*)

RPC method to continue cleaning a node.

This is useful for cleaning tasks that are async. When they complete, they call back via RPC, a new worker and lock are set up, and cleaning continues. This can also be used to resume cleaning on take_over.

Parameters

- **context** an admin context.
- **node_id** the id or uuid of a node.

Raises
InvalidStateRequested if the node is not in CLEANWAIT state

Raises
NoFreeConductorWorker when there is no free worker to start async task

Raises
NodeLocked if node is locked by another conductor.

Raises
NodeNotFound if the node no longer appears in the database

continue_node_deploy(*context*, *node_id*)

RPC method to continue deploying a node.

This is useful for deploying tasks that are async. When they complete, they call back via RPC, a new worker and lock are set up, and deploying continues. This can also be used to resume deploying on take_over.

Parameters

- **context** an admin context.
- **node_id** the ID or UUID of a node.

Raises
InvalidStateRequested if the node is not in DEPLOYWAIT state

Raises
NoFreeConductorWorker when there is no free worker to start async task

Raises
NodeLocked if node is locked by another conductor.

Raises
NodeNotFound if the node no longer appears in the database

continue_node_service(*context*, *node_id*)

RPC method to continue servicing a node.

This is useful for servicing tasks that are async. When they complete, they call back via RPC, a new worker and lock are set up, and servicing continues. This can also be used to resume servicing on take_over.

Parameters

- **context** an admin context.
- **node_id** the ID or UUID of a node.

Raises
InvalidStateRequested if the node is not in SERVICEWAIT state

Raises
NoFreeConductorWorker when there is no free worker to start async task

Raises
NodeLocked if node is locked by another conductor.

Raises
NodeNotFound if the node no longer appears in the database

create_allocation(***kwargs*)

create_node(***kwargs*)

create_port(***kwargs*)

destroy_allocation(***kwargs*)

destroy_node(***kwargs*)

destroy_port(***kwargs*)

destroy_portgroup(***kwargs*)

destroy_volume_connector(***kwargs*)

destroy_volume_target(***kwargs*)

detach_virtual_media(*context*, *node_id*, *device_types=None*)

Detach some or all virtual media devices from the node.

Parameters

- **context** request context.
- **node_id** node ID or UUID.
- **device_types** A collection of device type, ones from *ironic.common.boot_devices.VMEDIA_DEVICES*. If not provided, all devices are detached.

Raises
UnsupportedDriverExtension if the driver does not support this call.

Raises
InvalidParameterValue if validation of management driver interface failed.

Raises
NodeLocked if node is locked by another conductor.

Raises
NoFreeConductorWorker when there is no free worker to start async task.

do_node_clean(***kwargs*)

do_node_deploy(***kwargs*)

do_node_rescue(***kwargs*)

do_node_service(***kwargs*)

do_node_tear_down(***kwargs*)

do_node_unrescue(***kwargs*)

do_provisioning_action(***kwargs*)

driver_vendor_passthru(***kwargs*)

get_boot_device(***kwargs*)

get_console_information(***kwargs*)

get_driver_properties(***kwargs*)

get_driver_vendor_passthru_methods(***kwargs*)

get_indicator_state(***kwargs*)

get_node_vendor_passthru_methods(***kwargs*)

get_node_with_token(***kwargs*)

get_raid_logical_disk_properties(***kwargs*)

get_runbook(***kwargs*)

get_supported_boot_devices(***kwargs*)

get_supported_indicators(***kwargs*)

get_virtual_media(***kwargs*)

heartbeat(***kwargs*)

inject_nmi(***kwargs*)

inspect_hardware(***kwargs*)

manage_node_history(*context*)

object_action(*context*, *objinst*, *objmethod*, *args*, *kwargs*)
Perform an action on a VersionedObject instance.

Parameters

- **context** The context within which to perform the action

- **objinst** The object instance on which to perform the action
- **objmethod** The name of the action method to call
- **args** The positional arguments to the action method
- **kwargs** The keyword arguments to the action method

Returns
A tuple with the updates made to the object and the result of the action method

object_backport_versions(*context*, *objinst*, *object_versions*)

Perform a backport of an object instance.

The default behavior of the base VersionedObjectSerializer, upon receiving an object with a version newer than what is in the local registry, is to call this method to request a backport of the object.

Parameters

- **context** The context within which to perform the backport
- **objinst** An instance of a VersionedObject to be backported
- **object_versions** A dict of {objname: version} mappings

Returns
The downgraded instance of objinst

object_class_action_versions(*context*, *objname*, *objmethod*, *object_versions*, *args*, *kwargs*)

Perform an action on a VersionedObject class.

Parameters

- **context** The context within which to perform the action
- **objname** The registry name of the object
- **objmethod** The name of the action method to call
- **object_versions** A dict of {objname: version} mappings
- **args** The positional arguments to the action method
- **kwargs** The keyword arguments to the action method

Returns
The result of the action method, which may (or may not) be an instance of the implementing VersionedObject class.

remove_node_traits(***kwargs*)

set_boot_device(***kwargs*)

set_console_mode(***kwargs*)

set_indicator_state(***kwargs*)

set_target_raid_config(***kwargs*)

target = <Target version=1.62>

update_node(***kwargs*)

update_port(***kwargs*)

update_portgroup(***kwargs*)

update_portgroup_physical_network(***kwargs*)

update_volume_connector(***kwargs*)

update_volume_target(***kwargs*)

validate_driver_interfaces(***kwargs*)

vendor_passthru(***kwargs*)

vif_attach(***kwargs*)

vif_detach(***kwargs*)

vif_list(***kwargs*)

ironic.conductor.manager.**convert_steps**(*steps*)

Removes nonserializable fields from steps.

ironic.conductor.manager.**do_attach_virtual_media**(*task*, *device_type*, *image_url*, *image_download_source*)

ironic.conductor.manager.**do_detach_virtual_media**(*task*, *device_types*)

ironic.conductor.manager.**do_sync_power_state**(*task*, *count*)

Sync the power state for this node, incrementing the counter on failure.

When the limit of power_state_sync_max_retries is reached, the node is put into maintenance mode and the error recorded.

Parameters

- **task** a TaskManager instance
- **count** number of times this node has previously failed a sync

Raises

NodeLocked if unable to upgrade task lock to an exclusive one

Returns

Count of failed attempts. On success, the counter is set to 0. On failure, the count is incremented by one

ironic.conductor.manager.**get_vendor_passthru_metadata**(*route_dict*)

ironic.conductor.manager.**handle_sync_power_state_max_retries_exceeded**(*task*, *actual_power_state*, *exception=None*)

Handles power state sync exceeding the max retries.

When synchronizing the power state between a node and the DB has exceeded the maximum number of retries, change the DB power state to be the actual node power state and place the node in maintenance.

Parameters

- **task** a TaskManager instance with an exclusive lock
- **actual_power_state** the actual power state of the node; a power state from ironic.common.states
- **exception** the exception object that caused the sync power state to fail, if present.

ironic.conductor.notification_utils module

ironic.conductor.notification_utils.**emit_console_notification**(*task*, *action*, *status*)

Helper for conductor sending a set console state notification.

Parameters

- **task** a TaskManager instance.
- **action** Action string to go in the EventType. Must be either console_set or console_restore.
- **status** One of *ironic.objects.fields.NotificationStatus.START*, END or ERROR.

ironic.conductor.notification_utils.**emit_power_set_notification**(*task*, *level*, *status*, *to_power*)

Helper for conductor sending a set power state notification.

Parameters

- **task** a TaskManager instance.
- **level** Notification level. One of *ironic.objects.fields.NotificationLevel.ALL*
- **status** Status to go in the EventType. One of *ironic.objects.fields.NotificationStatus.SUCCESS* or ERROR. ERROR indicates that ironic-conductor couldnt retrieve the power state for this node, or that it couldnt set the power state of the node.
- **to_power** the power state the conductor is attempting to set on the node. This is used instead of the nodes target_power_state attribute since the baremetal.node.power_set.start notification is sent early, before target_power_state is set on the node.

ironic.conductor.notification_utils.**emit_power_state_corrected_notification**(*task*, *from_power*)

Helper for conductor sending a node power state corrected notification.

When ironic detects that the actual power state on a bare metal hardware is different from the power state on an ironic node (DB), the ironic nodes power state is corrected to be that of the bare metal hardware. A notification is emitted about this after the database is updated to reflect this correction.

Parameters

- **task** a TaskManager instance.
- **from_power** the power state of the node before this change was detected

ironic.conductor.notification_utils.**emit_provision_set_notification**(*task*, *level*, *status*, *prev_state*, *prev_target*, *event*)

Helper for conductor sending a set provision state notification.

Parameters

- **task** a TaskManager instance.
- **level** One of fields.NotificationLevel.
- **status** One of fields.NotificationStatus.
- **prev_state** Previous provision state.
- **prev_target** Previous target provision state.
- **event** FSM event that triggered provision state change.

ironic.conductor.periodics module

Conductor periodics.

exception ironic.conductor.periodics.**Stop**

Bases: `Exception`

A signal to stop the current iteration of a periodic task.

ironic.conductor.periodics.**node_periodic**(*purpose*, *spacing*, *enabled=True*, *filters=None*, *predicate=None*, *predicate_extra_fields=()*, *limit=None*, *shared_task=True*, *node_count_metric_name=None*)

A decorator to define a periodic task to act on nodes.

Defines a periodic task that fetches the list of nodes mapped to the current conductor which satisfy the provided filters.

The decorated function must be a method on either the conductor manager or a hardware interface. The signature is:

- for conductor manager: `(self, task, context)`
- for hardware interfaces: `(self, task, manager, context)`.

When the periodic is running on a hardware interface, only tasks using this interface are considered.

`NodeNotFound` and `NodeLocked` exceptions are ignored. Raise `Stop` to abort the current iteration of the task and reschedule it.

Parameters

- **purpose** a human-readable description of the activity, e.g. verifying that the cat is purring.
- **spacing** how often (in seconds) to run the periodic task.
- **enabled** whether the task is enabled; defaults to `spacing > 0`.
- **filters** database-level filters for the nodes.

- **predicate** a callable to run on the fetched nodes *before* creating a task for them. The only parameter will be a named tuple with fields `uuid`, `driver`, `conductor_group` plus everything from `predicate_extra_fields`. If the callable accepts a 2nd parameter, it will be the conductor manager instance.
- **predicate_extra_fields** extra fields to fetch on the initial request and pass into the `predicate`. Must not contain `uuid`, `driver` and `conductor_group` since they are always included.
- **limit** how many nodes to process before stopping the current iteration. If `predicate` returns `False`, the node is not counted. If the decorated function returns `False`, the node is not counted either. Can be a callable, in which case it will be called on each iteration to determine the limit.
- **shared_task** if `True`, the task will have a shared lock. It is recommended to start with a shared lock and upgrade it only if needed.
- **node_count_metric_name** A string value to identify a metric representing the count of matching nodes to be recorded upon the completion of the periodic.

ironic.conductor.periodics.**periodic**(*spacing*, *enabled=True*, ***kwargs*)

A decorator to define a periodic task.

Parameters

- **spacing** how often (in seconds) to run the periodic task.
- **enabled** whether the task is enabled; defaults to `spacing > 0`.

ironic.conductor.rpc_service module

class ironic.conductor.rpc_service.**RPCService**(*host*, *manager_module*, *manager_class*, *embed_api=False*)

Bases: *BaseRPCService*

property deregister_on_shutdown

is_draining()

stop()

Stop service.

ironic.conductor.rpcapi module

Client side of the conductor RPC API.

class ironic.conductor.rpcapi.**ConductorAPI**(*topic=None*)

Bases: `object`

Client side of the conductor RPC API.

API version history:

1.0 - Initial version.
Included get_node_power_status

1.1 - Added update_node and start_power_state_change.
1.2 - Added vendor_passthru.
1.3 - Rename start_power_state_change to change_node_power_state.
1.4 - Added do_node_deploy and do_node_tear_down.
1.5 - Added validate_driver_interfaces.
1.6 - change_node_power_state, do_node_deploy and do_node_tear_down
accept node id instead of node object.
1.7 - Added topic parameter to RPC methods.
1.8 - Added change_node_maintenance_mode.
1.9 - Added destroy_node.
1.10 - Remove get_node_power_state
1.11 - Added get_console_information, set_console_mode.
1.12 - validate_vendor_action, do_vendor_action replaced by single
vendor_passthru method.
1.13 - Added update_port.
1.14 - Added driver_vendor_passthru.
1.15 - Added rebuild parameter to do_node_deploy.
1.16 - Added get_driver_properties.
1.17 - Added set_boot_device, get_boot_device and
get_supported_boot_devices.
1.18 - Remove change_node_maintenance_mode.
1.19 - Change return value of vendor_passthru and
driver_vendor_passthru
1.20 - Added http_method parameter to vendor_passthru and
driver_vendor_passthru
1.21 - Added get_node_vendor_passthru_methods and
get_driver_vendor_passthru_methods
1.22 - Added configdrive parameter to do_node_deploy.
1.23 - Added do_provisioning_action
1.24 - Added inspect_hardware method
1.25 - Added destroy_port
1.26 - Added continue_node_clean
1.27 - Convert continue_node_clean to cast
1.28 - Change exceptions raised by destroy_node
1.29 - Change return value of vendor_passthru and
driver_vendor_passthru to a dictionary
1.30 - Added set_target_raid_config and
get_raid_logical_disk_properties
1.31 - Added Versioned Objects indirection API methods:
object_class_action_versions, object_action and
object_backport_versions
1.32 - Add do_node_clean
1.33 - Added update and destroy portgroup.
1.34 - Added heartbeat
1.35 - Added destroy_volume_connector and update_volume_connector

1.36 - Added create_node
1.37 - Added destroy_volume_target and update_volume_target
1.38 - Added vif_attach, vif_detach, vif_list
1.39 - Added timeout optional parameter to change_node_power_state
1.40 - Added inject_nmi
1.41 - Added create_port
1.42 - Added optional agent_version to heartbeat
1.43 - Added do_node_rescue, do_node_unrescue and can_send_rescue
1.44 - Added add_node_traits and remove_node_traits.
1.45 - Added continue_node_deploy
1.46 - Added reset_interfaces to update_node
1.47 - Added support for conductor groups
1.48 - Added allocation API
1.49 - Added get_node_with_token and agent_token argument to heartbeat
1.50 - Added set_indicator_state, get_indicator_state and
get_supported_indicators.
1.51 - Added agent_verify_ca to heartbeat.
1.52 - Added deploy steps argument to provisioning
1.53 - Added disable_ramdisk to do_node_clean.
1.54 - Added optional agent_status and agent_status_message to heartbeat
1.55 - Added change_node_boot_mode
1.56 - Added continue_inspection
1.57 - Added do_node_service
1.58 - Added support for json-rpc port usage
1.59 - Added support for attaching/detaching virtual media
1.60 - Added continue_node_service
1.61 - Added get virtual media support
1.62 - Added update_portgroup_physical_network

`RPC_API_VERSION = '1.62'`

`add_node_traits`(*context*, *node_id*, *traits*, *replace=False*, *topic=None*)

Add or replace traits for a node.

Parameters

- **context** request context.
- **node_id** node ID or UUID.
- **traits** a list of traits to add to the node.
- **replace** True to replace all of the nodes traits.
- **topic** RPC topic. Defaults to self.topic.

Raises

InvalidParameterValue if adding the traits would exceed the per-node traits limit.

Raises
NodeLocked if node is locked by another conductor.

Raises
NodeNotFound if the node does not exist.

`attach_virtual_media`(*context*, *node_id*, *device_type*, *image_url*, *image_download_source=None*, *topic=None*)

Attach a virtual media device to the node.

Parameters

- `context` request context.
- `node_id` node ID or UUID.
- `image_url` URL of the image to attach, HTTP or HTTPS.
- `image_download_source` Which way to serve the image to the BMC: http to serve it from the provided location, local to serve it from the local web server.
- `topic` RPC topic. Defaults to self.topic.

Raises
UnsupportedDriverExtension if the driver does not support this call.

Raises
InvalidParameterValue if validation of management driver interface failed.

Raises
NodeLocked if node is locked by another conductor.

Raises
NoFreeConductorWorker when there is no free worker to start async task.

`can_send_create_port`()

Return whether the RPCAPI supports the create_port method.

`can_send_rescue`()

Return whether the RPCAPI supports node rescue methods.

`change_node_boot_mode`(*context*, *node_id*, *new_state*, *topic=None*)

Change a nodes boot mode.

Synchronously, acquire lock and start the conductor background task to change boot mode of a node.

Parameters

- `context` request context.
- `node_id` node id or uuid.
- `new_state` one of ironic.common.boot_modes values (bios or uefi)
- `topic` RPC topic. Defaults to self.topic.

Raises
NoFreeConductorWorker when there is no free worker to start async task.

change_node_power_state(*context*, *node_id*, *new_state*, *topic=None*, *timeout=None*)

Change a nodes power state.

Synchronously, acquire lock and start the conductor background task to change power state of a node.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **new_state** one of ironic.common.states power state values
- **timeout** timeout (in seconds) positive integer (> 0) for any power state. None indicates to use default timeout.
- **topic** RPC topic. Defaults to self.topic.

Raises

NoFreeConductorWorker when there is no free worker to start async task.

change_node_secure_boot(*context*, *node_id*, *new_state*, *topic=None*)

Change a nodes secure_boot state.

Synchronously, acquire lock and start the conductor background task to change secure_boot state of a node.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **new_state** Target secure boot state (True => on or False => off)
- **topic** RPC topic. Defaults to self.topic.

Raises

NoFreeConductorWorker when there is no free worker to start async task.

continue_inspection(*context*, *node_id*, *inventory*, *plugin_data=None*, *topic=None*)

Continue in-band inspection.

Parameters

- **context** request context.
- **node_id** node ID or UUID.
- **inventory** hardware inventory from the node.
- **plugin_data** optional plugin-specific data.
- **topic** RPC topic. Defaults to self.topic.

Raises

NodeLocked if node is locked by another conductor.

continue_node_clean(*context*, *node_id*, *topic=None*)

Signal to conductor service to start the next cleaning action.

NOTE(JoshNang) this is an RPC cast, there will be no response or exception raised by the conductor for this RPC.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **topic** RPC topic. Defaults to self.topic.

continue_node_deploy(*context*, *node_id*, *topic=None*)

Signal to conductor service to start the next deployment action.

NOTE(rloo): this is an RPC cast, there will be no response or exception raised by the conductor for this RPC.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **topic** RPC topic. Defaults to self.topic.

continue_node_service(*context*, *node_id*, *topic=None*)

Signal to conductor service to start the next service action.

NOTE(janders): this is an RPC cast, there will be no response or exception raised by the conductor for this RPC.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **topic** RPC topic. Defaults to self.topic.

create_allocation(*context*, *allocation*, *topic=None*)

Create an allocation.

Parameters

- **context** request context.
- **allocation** an allocation object.
- **topic** RPC topic. Defaults to self.topic.

create_node(*context*, *node_obj*, *topic=None*)

Synchronously, have a conductor validate and create a node.

Create the nodes information in the database and return a node object.

Parameters

- **context** request context.
- **node_obj** a created (but not saved) node object.
- **topic** RPC topic. Defaults to self.topic.

Returns

created node object.

Raises
InterfaceNotFoundInEntrypoint if validation fails for any dynamic interfaces (e.g. network_interface).

Raises
NoValidDefaultForInterface if no default can be calculated for some interfaces, and explicit values must be provided.

`create_port`(*context*, *port_obj*, *topic=None*)

Synchronously, have a conductor validate and create a port.

Create the ports information in the database and return a port object. The conductor will lock related node and trigger specific driver actions if they are needed.

Parameters

- `context` request context.
- `port_obj` a created (but not saved) port object.
- `topic` RPC topic. Defaults to self.topic.

Returns
created port object.

`destroy_allocation`(*context*, *allocation*, *topic=None*)

Delete an allocation.

Parameters

- `context` request context.
- `allocation` an allocation object.
- `topic` RPC topic. Defaults to self.topic.

Raises
InvalidState if the associated node is in the wrong provision state to perform deallocation.

`destroy_node`(*context*, *node_id*, *topic=None*)

Delete a node.

Parameters

- `context` request context.
- `node_id` node id or uuid.
- `topic` RPC topic. Defaults to self.topic.

Raises
NodeLocked if node is locked by another conductor.

Raises
NodeAssociated if the node contains an instance associated with it.

Raises
InvalidState if the node is in the wrong provision state to perform deletion.

destroy_port(*context*, *port*, *topic=None*)

Delete a port.

Parameters

- **context** request context.
- **port** port object
- **topic** RPC topic. Defaults to self.topic.

Raises
NodeLocked if node is locked by another conductor.

Raises
NodeNotFound if the node associated with the port does not exist.

destroy_portgroup(*context*, *portgroup*, *topic=None*)

Delete a portgroup.

Parameters

- **context** request context.
- **portgroup** portgroup object
- **topic** RPC topic. Defaults to self.topic.

Raises
NodeLocked if node is locked by another conductor.

Raises
NodeNotFound if the node associated with the portgroup does not exist.

Raises
PortgroupNotEmpty if portgroup is not empty

destroy_volume_connector(*context*, *connector*, *topic=None*)

Delete a volume connector.

Delete the volume connector. The conductor will lock the related node during this operation.

Parameters

- **context** request context
- **connector** volume connector object
- **topic** RPC topic. Defaults to self.topic.

Raises
NodeLocked if node is locked by another conductor

Raises
NodeNotFound if the node associated with the connector does not exist

Raises
VolumeConnectorNotFound if the volume connector cannot be found

destroy_volume_target(*context*, *target*, *topic=None*)

Delete a volume target.

Parameters

- **context** request context
- **target** volume target object
- **topic** RPC topic. Defaults to self.topic.

Raises
NodeLocked if node is locked by another conductor

Raises
NodeNotFound if the node associated with the target does not exist

Raises
VolumeTargetNotFound if the volume target cannot be found

detach_virtual_media(*context*, *node_id*, *device_types=None*, *topic=None*)

Detach some or all virtual media devices from the node.

Parameters

- **context** request context.
- **node_id** node ID or UUID.
- **device_types** A collection of device type, ones from *ironic.common.boot_devices.VMEDIA_DEVICES*. If not provided, all devices are detached.
- **topic** RPC topic. Defaults to self.topic.

Raises
UnsupportedDriverExtension if the driver does not support this call.

Raises
InvalidParameterValue if validation of management driver interface failed.

Raises
NodeLocked if node is locked by another conductor.

Raises
NoFreeConductorWorker when there is no free worker to start async task.

do_node_clean(*context*, *node_id*, *clean_steps*, *disable_ramdisk=None*, *topic=None*)

Signal to conductor service to perform manual cleaning on a node.

Parameters

- **context** request context.
- **node_id** node ID or UUID.
- **clean_steps** a list of clean step dictionaries.
- **disable_ramdisk** Whether to skip booting ramdisk for cleaning.
- **topic** RPC topic. Defaults to self.topic.

Raises
InvalidParameterValue if validation of power driver interface failed.

Raises
InvalidStateRequested if cleaning can not be performed.

Raises
NodeInMaintenance if node is in maintenance mode.

Raises
NodeLocked if node is locked by another conductor.

Raises
NoFreeConductorWorker when there is no free worker to start async task.

do_node_deploy(*context*, *node_id*, *rebuild*, *configdrive*, *topic=None*, *deploy_steps=None*)

Signal to conductor service to perform a deployment.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **rebuild** True if this is a rebuild request.
- **configdrive** A gzipped and base64 encoded configdrive.
- **topic** RPC topic. Defaults to self.topic.
- **deploy_steps** Deploy steps

Raises
InstanceDeployFailure

Raises
InvalidParameterValue if validation fails

Raises
MissingParameterValue if a required parameter is missing

Raises
NoFreeConductorWorker when there is no free worker to start async task.

The node must already be configured and in the appropriate undeployed state before this method is called.

do_node_rescue(*context*, *node_id*, *rescue_password*, *topic=None*)

Signal to conductor service to perform a rescue.

Parameters

- **context** request context.
- **node_id** node ID or UUID.
- **rescue_password** A string representing the password to be set inside the rescue environment.
- **topic** RPC topic. Defaults to self.topic.

Raises
InstanceRescueFailure

Raises
NoFreeConductorWorker when there is no free worker to start async task.

The node must already be configured and in the appropriate state before this method is called.

do_node_service(*context*, *node_id*, *service_steps*, *disable_ramdisk=None*, *topic=None*)

Signal to conductor service to perform manual cleaning on a node.

Parameters

- **context** request context.
- **node_id** node ID or UUID.
- **service_steps** a list of service step dictionaries.
- **disable_ramdisk** Whether to skip booting ramdisk for service.
- **topic** RPC topic. Defaults to self.topic.

Raises
InvalidParameterValue if validation of power driver interface failed.

Raises
InvalidStateRequested if cleaning can not be performed.

Raises
NodeInMaintenance if node is in maintenance mode.

Raises
NodeLocked if node is locked by another conductor.

Raises
NoFreeConductorWorker when there is no free worker to start async task.

do_node_tear_down(*context*, *node_id*, *topic=None*)

Signal to conductor service to tear down a deployment.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **topic** RPC topic. Defaults to self.topic.

Raises
InstanceDeployFailure

Raises
InvalidParameterValue if validation fails

Raises
MissingParameterValue if a required parameter is missing

Raises
NoFreeConductorWorker when there is no free worker to start async task.

The node must already be configured and in the appropriate deployed state before this method is called.

do_node_unrescue(*context*, *node_id*, *topic=None*)

Signal to conductor service to perform an unrescue.

Parameters

- **context** request context.

- **node_id** node ID or UUID.
- **topic** RPC topic. Defaults to self.topic.

Raises
InstanceUnrescueFailure

Raises
NoFreeConductorWorker when there is no free worker to start async task.

The node must already be configured and in the appropriate state before this method is called.

do_provisioning_action(*context*, *node_id*, *action*, *topic=None*)

Signal to conductor service to perform the given action on a node.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **action** an action. One of ironic.common.states.VERBS
- **topic** RPC topic. Defaults to self.topic.

Raises
InvalidParameterValue

Raises
NoFreeConductorWorker when there is no free worker to start async task.

Raises
InvalidStateRequested if the requested action can not be performed.

This encapsulates some provisioning actions in a single call.

driver_vendor_passthru(*context*, *driver_name*, *driver_method*, *http_method*, *info*, *topic=None*)

Pass vendor-specific calls which dont specify a node to a driver.

Handles driver-level vendor passthru calls. These calls dont require a node UUID and are executed on a random conductor with the specified driver. If the method mode is async the conductor will start background worker to perform vendor action.

Parameters

- **context** request context.
- **driver_name** name of the driver on which to call the method.
- **driver_method** name of the vendor method, for use by the driver.
- **http_method** the HTTP method used for the request.
- **info** data to pass through to the driver.
- **topic** RPC topic. Defaults to self.topic.

Raises
InvalidParameterValue for parameter errors.

Raises
MissingParameterValue if a required parameter is missing

Raises
UnsupportedDriverExtension if the driver doesnt have a vendor interface, or if the vendor interface does not support the specified driver_method.

Raises
DriverNotFound if the supplied driver is not loaded.

Raises
NoFreeConductorWorker when there is no free worker to start async task.

Raises
InterfaceNotFoundInEntrypoint if the default interface for a hardware type is invalid.

Raises
NoValidDefaultForInterface if no default interface implementation can be found for this drivers vendor interface.

Returns

A dictionary containing:

return
The response of the invoked vendor method

async
Boolean value. Whether the method was invoked asynchronously (True) or synchronously (False). When invoked asynchronously the response will be always None.

attach
Boolean value. Whether to attach the response of the invoked vendor method to the HTTP response object (True) or return it in the response body (False).

`get_boot_device`(*context*, *node_id*, *topic=None*)

Get the current boot device.

Returns the current boot device of a node.

Parameters

- `context` request context.
- `node_id` node id or uuid.
- `topic` RPC topic. Defaults to self.topic.

Raises
NodeLocked if node is locked by another conductor.

Raises
UnsupportedDriverExtension if the nodes driver doesnt support management.

Raises
InvalidParameterValue when the wrong driver info is specified.

Raises
MissingParameterValue if missing supplied info.

Returns

a dictionary containing:

boot_device
the boot device, one of *ironic.common.boot_devices* or None if it is unknown.

persistent
Whether the boot device will persist to all future boots or not, None if it is unknown.

`get_conductor_for`(*node*)

Get the conductor which the node is mapped to.

Parameters
node a node object.

Returns
the conductor hostname.

Raises
NoValidHost

`get_console_information`(*context*, *node_id*, *topic=None*)

Get connection information about the console.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **topic** RPC topic. Defaults to self.topic.

Raises
UnsupportedDriverExtension if the nodes driver doesnt support console.

Raises
InvalidParameterValue when the wrong driver info is specified.

Raises
MissingParameterValue if a required parameter is missing

`get_current_topic`()

Get RPC topic name for the current conductor.

`get_driver_properties`(*context*, *driver_name*, *topic=None*)

Get the properties of the driver.

Parameters

- **context** request context.
- **driver_name** name of the driver.
- **topic** RPC topic. Defaults to self.topic.

Returns
a dictionary with <property name>:<property description> entries.

Raises
DriverNotFound.

`get_driver_vendor_passthru_methods`(*context*, *driver_name*, *topic=None*)

Retrieve information about vendor methods of the given driver.

Parameters

- **context** an admin context.
- **driver_name** name of the driver.
- **topic** RPC topic. Defaults to self.topic.

Raises
UnsupportedDriverExtension if current driver does not have vendor interface.

Raises
DriverNotFound if the supplied driver is not loaded.

Raises
InterfaceNotFoundInEntrypoint if the default interface for a hardware type is invalid.

Raises
NoValidDefaultForInterface if no default interface implementation can be found for this drivers vendor interface.

Returns
dictionary of <method name>:<method metadata> entries.

`get_indicator_state`(*context*, *node_id*, *component*, *indicator*, *topic=None*)

Get node hardware component indicator state.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **component** The hardware component, one of *ironic.common.components*.
- **indicator** Indicator IDs, as reported by *get_supported_indicators*)
- **topic** RPC topic. Defaults to self.topic.

Raises
NodeLocked if node is locked by another conductor.

Raises
UnsupportedDriverExtension if the nodes driver doesnt support management.

Raises
InvalidParameterValue when the wrong driver info is specified.

Raises
MissingParameterValue if missing supplied info.

Returns
Indicator state, one of mod:*ironic.common.indicator_states*.

get_node_vendor_passthru_methods(*context*, *node_id*, *topic=None*)

Retrieve information about vendor methods of the given node.

Parameters

- **context** an admin context.
- **node_id** the id or uuid of a node.
- **topic** RPC topic. Defaults to self.topic.

Returns

dictionary of <method name>:<method metadata> entries.

get_node_with_token(*context*, *node_id*, *topic=None*)

Request the node from the conductor with an agent token

Parameters

- **context** request context.
- **node_id** node ID or UUID.
- **topic** RPC topic. Defaults to self.topic.

Raises

NodeLocked if node is locked by another conductor.

Returns

A Node object with agent token.

get_raid_logical_disk_properties(*context*, *driver_name*, *topic=None*)

Get the logical disk properties for RAID configuration.

Gets the information about logical disk properties which can be specified in the input RAID configuration.

Parameters

- **context** request context.
- **driver_name** name of the driver
- **topic** RPC topic. Defaults to self.topic.

Raises

UnsupportedDriverExtension if the driver doesnt support RAID configuration.

Raises

InterfaceNotFoundInEntrypoint if the default interface for a hardware type is invalid.

Raises

NoValidDefaultForInterface if no default interface implementation can be found for this drivers RAID interface.

Returns

A dictionary containing the properties that can be mentioned for logical disks and a textual description for them.

`get_random_topic`()

Get an RPC topic for a random conductor service.

`get_supported_boot_devices`(*context*, *node_id*, *topic=None*)

Get the list of supported devices.

Returns the list of supported boot devices of a node.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **topic** RPC topic. Defaults to self.topic.

Raises

NodeLocked if node is locked by another conductor.

Raises

UnsupportedDriverExtension if the nodes driver doesnt support management.

Raises

InvalidParameterValue when the wrong driver info is specified.

Raises

MissingParameterValue if missing supplied info.

Returns

A list with the supported boot devices defined in *ironic.common.boot_devices*.

`get_supported_indicators`(*context*, *node_id*, *component=None*, *topic=None*)

Get node hardware components and their indicators.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **component** The hardware component, one of *ironic.common.components*.
- **topic** RPC topic. Defaults to self.topic.

Raises

NodeLocked if node is locked by another conductor.

Raises

UnsupportedDriverExtension if the nodes driver doesnt support management.

Raises

InvalidParameterValue when the wrong driver info is specified.

Raises

MissingParameterValue if missing supplied info.

Returns

A dictionary of hardware components (*ironic.common.components*) as keys with indicator IDs as values.

```
{
    'chassis': ['enclosure-0'],
    'system': ['blade-A']
    'drive': ['ssd0']
}
```

get_topic_for(*node*)

Get the RPC topic for the conductor service the node is mapped to.

Parameters
node a node object.

Returns
an RPC topic string.

Raises
NoValidHost

get_topic_for_driver(*driver_name*)

Get RPC topic name for a conductor supporting the given driver.

The topic is used to route messages to the conductor supporting the specified driver. A conductor is selected at random from the set of qualified conductors.

Parameters
driver_name the name of the driver to route to.

Returns
an RPC topic string.

Raises
DriverNotFound

get_virtual_media(*context*, *node_id*, *topic=None*)

Get all virtual media devices from the node.

Parameters

- **context** request context.
- **node_id** node ID or UUID.
- **topic** RPC topic. Defaults to self.topic.

Raises
UnsupportedDriverExtension if the driver does not support this call.

Raises
InvalidParameterValue if validation of management driver interface failed.

Raises
NodeLocked if node is locked by another conductor.

heartbeat(*context*, *node_id*, *callback_url*, *agent_version*, *agent_token=None*, *agent_verify_ca=None*, *agent_status=None*, *agent_status_message=None*, *topic=None*)

Process a node heartbeat.

Parameters

- **context** request context.
- **node_id** node ID or UUID.
- **callback_url** URL to reach back to the ramdisk.
- **topic** RPC topic. Defaults to self.topic.
- **agent_token** randomly generated validation token.
- **agent_version** the version of the agent that is heartbeating
- **agent_verify_ca** TLS certificate for the agent.
- **agent_status** The status of the agent that is heartbeating
- **agent_status_message** Optional message describing the agent status

Raises
InvalidParameterValue if an invalid agent token is received.

inject_nmi(*context*, *node_id*, *topic=None*)

Inject NMI for a node.

Inject NMI (Non Maskable Interrupt) for a node immediately. Be aware that not all drivers support this.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **topic** RPC topic. Defaults to self.topic.

Raises
NodeLocked if node is locked by another conductor.

Raises
UnsupportedDriverExtension if the nodes driver doesnt support management or management.inject_nmi.

Raises
InvalidParameterValue when the wrong driver info is specified or an invalid boot device is specified.

Raises
MissingParameterValue if missing supplied info.

inspect_hardware(*context*, *node_id*, *topic=None*)

Signals the conductor service to perform hardware introspection.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **topic** RPC topic. Defaults to self.topic.

Raises
NodeLocked if node is locked by another conductor.

Raises
HardwareInspectionFailure

Raises
NoFreeConductorWorker when there is no free worker to start async task.

Raises
UnsupportedDriverExtension if the nodes driver doesnt support inspection.

Raises
InvalidStateRequested if inspect is not a valid action to do in the current state.

`object_action`(*context*, *objinst*, *objmethod*, *args*, *kwargs*)

Perform an action on a VersionedObject instance.

We want any conductor to handle this, so it is intentional that there is no topic argument for this method.

Parameters

- **context** The context within which to perform the action
- **objinst** The object instance on which to perform the action
- **objmethod** The name of the action method to call
- **args** The positional arguments to the action method
- **kwargs** The keyword arguments to the action method

Raises
NotImplementedError when an operator makes an error during upgrade

Returns
A tuple with the updates made to the object and the result of the action method

`object_backport_versions`(*context*, *objinst*, *object_versions*)

Perform a backport of an object instance.

The default behavior of the base VersionedObjectSerializer, upon receiving an object with a version newer than what is in the local registry, is to call this method to request a backport of the object.

We want any conductor to handle this, so it is intentional that there is no topic argument for this method.

Parameters

- **context** The context within which to perform the backport
- **objinst** An instance of a VersionedObject to be backported
- **object_versions** A dict of {objname: version} mappings

Raises
NotImplementedError when an operator makes an error during upgrade

Returns
The downgraded instance of objinst

`object_class_action_versions`(*context*, *objname*, *objmethod*, *object_versions*, *args*, *kwargs*)

Perform an action on a VersionedObject class.

We want any conductor to handle this, so it is intentional that there is no topic argument for this method.

Parameters

- **context** The context within which to perform the action
- **objname** The registry name of the object
- **objmethod** The name of the action method to call
- **object_versions** A dict of {objname: version} mappings
- **args** The positional arguments to the action method
- **kwargs** The keyword arguments to the action method

Raises
NotImplementedError when an operator makes an error during upgrade

Returns
The result of the action method, which may (or may not) be an instance of the implementing VersionedObject class.

`remove_node_traits`(*context*, *node_id*, *traits*, *topic=None*)

Remove some or all traits from a node.

Parameters

- **context** request context.
- **node_id** node ID or UUID.
- **traits** a list of traits to remove from the node, or None. If None, all traits will be removed from the node.
- **topic** RPC topic. Defaults to self.topic.

Raises
NodeLocked if node is locked by another conductor.

Raises
NodeNotFound if the node does not exist.

Raises
NodeTraitNotFound if one of the traits is not found.

`set_boot_device`(*context*, *node_id*, *device*, *persistent=False*, *topic=None*)

Set the boot device for a node.

Set the boot device to use on next reboot of the node. Be aware that not all drivers support this.

Parameters

- **context** request context.
- **node_id** node id or uuid.

- **device** the boot device, one of *ironic.common.boot_devices*.
- **persistent** Whether to set next-boot, or make the change permanent. Default: False.
- **topic** RPC topic. Defaults to self.topic.

Raises
NodeLocked if node is locked by another conductor.

Raises
UnsupportedDriverExtension if the nodes driver doesnt support management.

Raises
InvalidParameterValue when the wrong driver info is specified or an invalid boot device is specified.

Raises
MissingParameterValue if missing supplied info.

set_console_mode(*context*, *node_id*, *enabled*, *topic=None*)

Enable/Disable the console.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **topic** RPC topic. Defaults to self.topic.
- **enabled** Boolean value; whether the console is enabled or disabled.

Raises
UnsupportedDriverExtension if the nodes driver doesnt support console.

Raises
InvalidParameterValue when the wrong driver info is specified.

Raises
MissingParameterValue if a required parameter is missing

Raises
NoFreeConductorWorker when there is no free worker to start async task.

set_indicator_state(*context*, *node_id*, *component*, *indicator*, *state*, *topic=None*)

Set node hardware components indicator to the desired state.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **component** The hardware component, one of *ironic.common.components*.
- **indicator** Indicator IDs, as reported by *get_supported_indicators*)
- **state** Indicator state, one of mod:*ironic.common.indicator_states*.
- **topic** RPC topic. Defaults to self.topic.

Raises
NodeLocked if node is locked by another conductor.

Raises
UnsupportedDriverExtension if the nodes driver doesnt support management.

Raises
InvalidParameterValue when the wrong driver info is specified or an invalid boot device is specified.

Raises
MissingParameterValue if missing supplied info.

`set_target_raid_config`(*context*, *node_id*, *target_raid_config*, *topic=None*)

Stores the target RAID configuration on the node.

Stores the target RAID configuration on node.target_raid_config

Parameters

- `context` request context.
- `node_id` node id or uuid.
- `target_raid_config` Dictionary containing the target RAID configuration. It may be an empty dictionary as well.
- `topic` RPC topic. Defaults to self.topic.

Raises
UnsupportedDriverExtension if the nodes driver doesnt support RAID configuration.

Raises
InvalidParameterValue, if validation of target raid config fails.

Raises
MissingParameterValue, if some required parameters are missing.

Raises
NodeLocked if node is locked by another conductor.

`update_node`(*context*, *node_obj*, *topic=None*, *reset_interfaces=False*)

Synchronously, have a conductor update the nodes information.

Update the nodes information in the database and return a node object. The conductor will lock the node while it validates the supplied information. If driver_info is passed, it will be validated by the core drivers. If instance_uuid is passed, it will be set or unset only if the node is properly configured.

Note that power_state should not be passed via this method. Use change_node_power_state for initiating driver actions.

Parameters

- `context` request context.
- `node_obj` a changed (but not saved) node object.
- `topic` RPC topic. Defaults to self.topic.
- `reset_interfaces` whether to reset hardware interfaces to their defaults.

Returns
updated node object, including all fields.

Raises
NoValidDefaultForInterface if no default can be calculated for some interfaces, and explicit values must be provided.

update_port(*context*, *port_obj*, *topic=None*)

Synchronously, have a conductor update the ports information.

Update the ports information in the database and return a port object. The conductor will lock related node and trigger specific driver actions if they are needed.

Parameters

- **context** request context.
- **port_obj** a changed (but not saved) port object.
- **topic** RPC topic. Defaults to self.topic.

Returns
updated port object, including all fields.

update_portgroup(*context*, *portgroup_obj*, *topic=None*)

Synchronously, have a conductor update the portgroups information.

Update the portgroups information in the database and return a portgroup object. The conductor will lock related node and trigger specific driver actions if they are needed.

Parameters

- **context** request context.
- **portgroup_obj** a changed (but not saved) portgroup object.
- **topic** RPC topic. Defaults to self.topic.

Returns
updated portgroup object, including all fields.

update_portgroup_physical_network(*context*, *portgroup_obj*, *new_physical_network*, *topic=None*)

Update the physical_network for a given portgroup and its ports.

Parameters

- **context** request context.
- **topic** RPC topic. Defaults to self.topic.

Raises
NodeLocked if node is locked by another conductor.

update_volume_connector(*context*, *connector*, *topic=None*)

Update the volume connectors information.

Update the volume connectors information in the database and return a volume connector object. The conductor will lock the related node during this operation.

Parameters

- **context** request context
- **connector** a changed (but not saved) volume connector object
- **topic** RPC topic. Defaults to self.topic.

Raises
InvalidParameterValue if the volume connectors UUID is being changed

Raises
NodeLocked if node is locked by another conductor

Raises
NodeNotFound if the node associated with the connector does not exist

Raises
VolumeConnectorNotFound if the volume connector cannot be found

Raises
VolumeConnectorTypeAndIdAlreadyExists if another connector already exists with the same values for type and connector_id fields

Returns
updated volume connector object, including all fields.

update_volume_target(*context*, *target*, *topic=None*)

Update the volume targets information.

Update the volume targets information in the database and return a volume target object. The conductor will lock the related node during this operation.

Parameters

- **context** request context
- **target** a changed (but not saved) volume target object
- **topic** RPC topic. Defaults to self.topic.

Raises
InvalidParameterValue if the volume targets UUID is being changed

Raises
NodeLocked if the node is already locked

Raises
NodeNotFound if the node associated with the volume target does not exist

Raises
VolumeTargetNotFound if the volume target cannot be found

Raises
VolumeTargetBootIndexAlreadyExists if a volume target already exists with the same node ID and boot index values

Returns
updated volume target object, including all fields

validate_driver_interfaces(*context*, *node_id*, *topic=None*)

Validate the *core* and *standardized* interfaces for drivers.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **topic** RPC topic. Defaults to self.topic.

Returns
a dictionary containing the results of each interface validation.

vendor_passthru(*context*, *node_id*, *driver_method*, *http_method*, *info*, *topic=None*)

Receive requests for vendor-specific actions.

Synchronously validate driver specific info or get driver status, and if successful invokes the vendor method. If the method mode is async the conductor will start background worker to perform vendor action.

Parameters

- **context** request context.
- **node_id** node id or uuid.
- **driver_method** name of method for driver.
- **http_method** the HTTP method used for the request.
- **info** info for node driver.
- **topic** RPC topic. Defaults to self.topic.

Raises
InvalidParameterValue if supplied info is not valid.

Raises
MissingParameterValue if a required parameter is missing

Raises
UnsupportedDriverExtension if current driver does not have vendor interface.

Raises
NoFreeConductorWorker when there is no free worker to start async task.

Raises
NodeLocked if node is locked by another conductor.

Returns

A dictionary containing:

return
The response of the invoked vendor method

async
Boolean value. Whether the method was invoked asynchronously (True) or synchronously (False). When invoked asynchronously the response will be always None.

attach
Boolean value. Whether to attach the response of the invoked vendor method to the HTTP response object (True) or return it in the response body (False).

`vif_attach`(*context*, *node_id*, *vif_info*, *topic=None*)

Attach VIF to a node

Parameters

- `context` request context.
- `node_id` node ID or UUID.
- `vif_info` a dictionary representing VIF object. It must have an id key, whose value is a unique identifier for that VIF.
- `topic` RPC topic. Defaults to self.topic.

Raises

NodeLocked, if node has an exclusive lock held on it

Raises

NetworkError, if an error occurs during attaching the VIF.

Raises

InvalidParameterValue, if a parameter thats required for VIF attach is wrong/missing.

`vif_detach`(*context*, *node_id*, *vif_id*, *topic=None*)

Detach VIF from a node

Parameters

- `context` request context.
- `node_id` node ID or UUID.
- `vif_id` an ID of a VIF.
- `topic` RPC topic. Defaults to self.topic.

Raises

NodeLocked, if node has an exclusive lock held on it

Raises

NetworkError, if an error occurs during detaching the VIF.

Raises

InvalidParameterValue, if a parameter thats required for VIF detach is wrong/missing.

`vif_list`(*context*, *node_id*, *topic=None*)

List attached VIFs for a node

Parameters

- `context` request context.
- `node_id` node ID or UUID.
- `topic` RPC topic. Defaults to self.topic.

Returns

List of VIF dictionaries, each dictionary will have an id entry with the ID of the VIF.

Raises
NetworkError, if an error occurs during listing the VIFs.

Raises
InvalidParameterValue, if a parameter thats required for VIF list is wrong/missing.

class ironic.conductor.rpcapi.**LocalContext**

Bases: object

Context to make calls to a local conductor.

call(*context*, *rpc_call_name*, ***kwargs*)
Make a local conductor call.

cast(*context*, *rpc_call_name*, ***kwargs*)
Make a local conductor call.

It is expected that the underlying call uses a thread to avoid blocking the caller.

Any exceptions are logged and ignored.

ironic.conductor.servicing module

Functionality related to servicing.

ironic.conductor.servicing.**continue_node_service**(*task*)
Continue servicing after finishing an async service step.

This function calculates which step has to run next and passes control into do_next_service_step.

Parameters
task a TaskManager instance with an exclusive lock

ironic.conductor.servicing.**do_next_service_step**(*task*, *step_index*, *disable_ramdisk=None*)
Do service, starting from the specified service step.

Parameters

- **task** a TaskManager instance with an exclusive lock
- **step_index** The first service step in the list to execute. This is the index (from 0) into the list of service steps in the nodes driver_internal_info[service_steps]. Is None if there are no steps to execute.
- **disable_ramdisk** Whether to skip booting ramdisk for service.

ironic.conductor.servicing.**do_node_service**(*task*, *service_steps=None*, *disable_ramdisk=False*)
Internal RPC method to perform servicing of a node.

Parameters

- **task** a TaskManager instance with an exclusive lock on its node
- **service_steps** The list of service steps to perform. If none, step validation will fail.
- **disable_ramdisk** Whether to skip booting ramdisk for servicing.

ironic.conductor.servicing.**do_node_service_abort**(*task*)

Internal method to abort an ongoing operation.

Parameters

task a TaskManager instance with an exclusive lock

ironic.conductor.servicing.**execute_step_on_child_nodes**(*task*, *step*)

Execute a requested step against a child node.

Parameters

- **task** The TaskManager object for the parent node.
- **step** The requested step to be executed.

Returns

None on Success, the resulting error message if a failure has occurred.

ironic.conductor.servicing.**get_last_error**(*node*)

ironic.conductor.steps module

ironic.conductor.steps.**all_steps_disable_ramdisk**(*task*, *user_steps*)

Check whether every user-requested clean step can run without ramdisk.

Parameters

- **task** A TaskManager object
- **user_steps** list of user-specified clean step dicts, each with at least interface and step keys.

Returns

True if all steps declare requires_ramdisk=False, else False.

ironic.conductor.steps.**find_step**(*steps*, *step*)

Find an identical step in the list of steps.

ironic.conductor.steps.**is_equivalent**(*step1*, *step2*)

Compare steps, ignoring their priority.

ironic.conductor.steps.**set_node_cleaning_steps**(*task*, *disable_ramdisk=False*, *use_existing_steps=False*)

Set up the node with clean step information for cleaning.

For automated cleaning, get the clean steps from the driver. For manual cleaning, the users clean steps are known but need to be validated against the drivers clean steps.

Parameters

- **task** A TaskManager object
- **disable_ramdisk** If *True*, only steps with requires_ramdisk=False are accepted.
- **use_existing_steps** If *True*, validate existing steps from driver_internal_info[clean_steps] which comes from manual cleaning API call or a runbook-based declarative automated cleaning. If *False*, retrieve cleaning steps from the driver (implicit automated cleaning).

Raises
InvalidParameterValue if there is a problem with the users clean steps.

Raises
NodeCleaningFailure if there was a problem getting the clean steps.

ironic.conductor.steps.**set_node_deployment_steps**(*task*, *reset_current=True*, *skip_missing=False*)

Set up the node with deployment step information for deploying.

Get the deploy steps from the driver.

Parameters
reset_current Whether to reset the current step to the first one.

Raises
InstanceDeployFailure if there was a problem getting the deployment steps.

ironic.conductor.steps.**set_node_service_steps**(*task*, *disable_ramdisk=False*)

Set up the node with clean step information for cleaning.

For automated cleaning, get the clean steps from the driver. For manual cleaning, the users clean steps are known but need to be validated against the drivers clean steps.

Parameters
disable_ramdisk If *True*, only steps with requires_ramdisk=False are accepted.

Raises
InvalidParameterValue if there is a problem with the users clean steps.

Raises
NodeServicingFailure if there was a problem getting the service steps.

ironic.conductor.steps.**step_id**(*step*)

Return the ID of a deploy step.

The ID is a string, <interface>.<step>.

Parameters
step the step dictionary.

Returns
the steps ID string.

ironic.conductor.steps.**use_reserved_step_handler**(*task*, *step*)

Returns guidance for reserved step execution or process is used.

This method is utilized to handle some specific cases with the execution of steps. For example, reserved step names, or reserved names which have specific meaning in the state machine.

Parameters

- **task** a TaskManager object.
- **step** The requested step.

ironic.conductor.steps.**validate_user_deploy_steps_and_templates**(*task*, *deploy_steps=None*, *skip_missing=False*)

Validate the user deploy steps and the deploy templates for a node.

Parameters

- **task** A TaskManager object
- **deploy_steps** Deploy steps to validate. Optional. If not provided then will check nodes driver internal info.
- **skip_missing** whether skip missing steps that are not yet available at the time of validation.

Raises

InvalidParameterValue if the instance has traits that map to deploy steps that are unsupported by the nodes driver interfaces or user deploy steps are unsupported by the nodes driver interfaces

Raises

InstanceDeployFailure if there was a problem getting the deploy steps from the driver.

ironic.conductor.task_manager module

A context manager to perform a series of tasks on a set of resources.

TaskManager is a context manager, created on-demand to allow synchronized access to a node and its resources.

The *TaskManager* will, by default, acquire an exclusive lock on a node for the duration that the TaskManager instance exists. You may create a TaskManager instance without locking by passing shared=True when creating it, but certain operations on the resources held by such an instance of TaskManager will not be possible. Requiring this exclusive lock guards against parallel operations interfering with each other.

A shared lock is useful when performing non-interfering operations, such as validating the driver interfaces.

An exclusive lock is stored in the database to coordinate between *ironic.conductor.manager* instances, that are typically deployed on different hosts.

TaskManager methods, as well as driver methods, may be decorated to determine whether their invocation requires an exclusive lock.

The TaskManager instance exposes certain node resources and properties as attributes that you may access:

task.context
The context passed to TaskManager()

task.shared
False if Node is locked, True if it is not locked. (The shared kwarg arg of TaskManager())

task.node
The Node object

task.ports
Ports belonging to the Node

task.portgroups
Portgroups belonging to the Node

task.volume_connectors
Storage connectors belonging to the Node

task.volume_targets
Storage targets assigned to the Node

task.driver
The Driver for the Node, or the Driver based on the driver_name kwarg of TaskManager().

Example usage:

```
with task_manager.acquire(context, node_id, purpose='power on') as task:
    task.driver.power.power_on(task.node)
```

If you need to execute task-requiring code in a background thread, the TaskManager instance provides an interface to handle this for you, making sure to release resources when the thread finishes (successfully or if an exception occurs). Common use of this is within the Manager like so:

```
with task_manager.acquire(context, node_id, purpose='some work') as task:
    <do some work>
    task.spawn_after(self._spawn_worker,
                     utils.node_power_action, task, new_state)
```

All exceptions that occur in the current GreenThread as part of the spawn handling are re-raised. You can specify a hook to execute custom code when such exceptions occur. For example, the hook is a more elegant solution than wrapping the with task_manager.acquire() with a try..exception block. (Note that this hook does not handle exceptions raised in the background thread.):

```
def on_error(e):
    if isinstance(e, Exception):
        ...

with task_manager.acquire(context, node_id, purpose='some work') as task:
    <do some work>
    task.set_spawn_error_hook(on_error)
    task.spawn_after(self._spawn_worker,
                     utils.node_power_action, task, new_state)
```

class ironic.conductor.task_manager.**TaskManager**(*context*, *node_id*, *shared=False*, *purpose='unspecified action'*, *retry=True*, *patient=False*, *load_driver=True*)

Bases: `object`

Context manager for tasks.

This class wraps the locking, driver loading, and acquisition of related resources (eg, Node and Ports) when beginning a unit of work.

downgrade_lock()

Downgrade the lock to a shared one.

load_driver()

property node

property portgroups

property ports

process_event(*event*, *callback=None*, *call_args=None*, *call_kwargs=None*, *err_handler=None*, *target_state=None*, *last_error=None*)

Process the given event for the tasks current state.

Parameters

- **event** the name of the event to process
- **callback** optional callback to invoke upon event transition
- **call_args** optional args to pass to the callback method
- **call_kwargs** optional kwargs to pass to the callback method
- **err_handler** optional error handler to invoke if the callback fails, eg. because there are no workers available (err_handler should accept arguments node, prev_prov_state, and prev_target_state)
- **target_state** if specified, the target provision state for the node. Otherwise, use the target state from the fsm
- **last_error** last error to set on the node together with the state transition.

Raises

InvalidState if the event is not allowed by the associated state machine

release_resources()

Unlock a node and release resources.

If an exclusive lock is held, unlock the node. Reset attributes to make it clear that this instance of TaskManager should no longer be accessed.

resume_cleaning()

A helper to resume cleaning with the right target state.

set_spawn_error_hook(*_on_error_method*, **args*, ***kwargs*)

Create a hook to handle exceptions when spawning a task.

Create a hook that gets called upon an exception being raised from spawning a background thread to do a task.

Parameters

- **_on_error_method** a callable object, its first parameter should accept the Exception object that was raised.
- **args** additional args passed to the callable object.
- **kwargs** additional kwargs passed to the callable object.

spawn_after(*_spawn_method*, **args*, ***kwargs*)

Call this to spawn a thread to complete the task.

The specified method will be called when the TaskManager instance exits.

Parameters

- **_spawn_method** a method that returns a GreenThread object
- **args** args passed to the method.
- **kwargs** additional kwargs passed to the method.

property tbn_traits

upgrade_lock(*purpose=None*, *retry=None*)

Upgrade a shared lock to an exclusive lock.

Also reloads node object from the database. If lock is already exclusive only changes the lock purpose when provided with one.

Parameters

- **purpose** optionally change the purpose of the lock
- **retry** whether to retry locking if it fails, the class-level value is used by default

Raises

NodeLocked if an exclusive lock remains on the node after node_locked_retry_attempts

property volume_connectors

property volume_targets

ironic.conductor.task_manager.**acquire**(*context*, **args*, ***kwargs*)

Shortcut for acquiring a lock on a Node.

Parameters

context Request context.

Returns

An instance of *TaskManager*.

ironic.conductor.task_manager.**require_exclusive_lock**(*f*)

Decorator to require an exclusive lock.

Decorated functions must take a *TaskManager* as the first parameter. Decorated class methods should take a *TaskManager* as the first parameter after self.

ironic.conductor.utils module

class ironic.conductor.utils.**StepFlow**(*value*, *names=<not given>*, **values*, *module=None*, *qualname=None*, *type=None*, *start=1*, *boundary=None*)

Bases: Enum

CLEANING_AUTO = 'automated_cleaning'

CLEANING_MANUAL = 'manual_cleaning'

DEPLOYMENT = 'deployment'

SERVICING = 'servicing'

ironic.conductor.utils.**abort_on_conductor_take_over**(*task*)

Set nodes state when a task was aborted due to conductor take over.

Parameters

task a TaskManager instance.

ironic.conductor.utils.**add_secret_token**(*node*, *pregenerated=False*)

Adds a secret token to driver_internal_info for IPA verification.

Parameters

- **node** Node object
- **pregenerated** Boolean value, default False, which indicates if the token should be marked as pregenerated in order to facilitate virtual media booting where the token is embedded into the configuration.

ironic.conductor.utils.**agent_is_alive**(*node*, *timeout=None*)

Check that the agent is likely alive.

The method then checks for the last agent heartbeat, and if it occurred within the timeout set by [deploy]fast_track_timeout, then agent is presumed alive.

Parameters

- **node** A node object.
- **timeout** Heartbeat timeout, defaults to *fast_track_timeout*.

ironic.conductor.utils.**cleaning_error_handler**(*task*, *logmsg*, *errmsg=None*, *traceback=False*, *tear_down_cleaning=True*, *set_fail_state=True*, *set_maintenance=None*)

Put a failed node in CLEANFAIL and maintenance (if needed).

Parameters

- **task** a TaskManager instance.
- **logmsg** Message to be logged.
- **errmsg** Message for the user. Optional, if not provided *logmsg* is used.
- **traceback** Whether to log a traceback. Defaults to False.
- **tear_down_cleaning** Whether to clean up the PXE and DHCP files after cleaning. Default to True.
- **set_fail_state** Whether to set node to failed state. Default to True.
- **set_maintenance** Whether to set maintenance mode. If None, maintenance mode will be set if and only if a clean step is being executed on a node.

ironic.conductor.utils.**cleanup_after_timeout**(*task*)

Cleanup deploy task after timeout.

Parameters

task a TaskManager instance.

ironic.conductor.utils.**cleanup_cleanwait_timeout**(*task*)

Cleanup a cleaning task after timeout.

Parameters
task a TaskManager instance.

ironic.conductor.utils.**cleanup_inspectwait_timeout**(*task*)

Cleanup an inspection task after timeout.

Parameters
task a TaskManager instance.

ironic.conductor.utils.**cleanup_rescuewait_timeout**(*task*)

Cleanup rescue task after timeout.

Parameters
task a TaskManager instance.

ironic.conductor.utils.**cleanup_servicewait_timeout**(*task*)

Cleanup a servicing task after timeout.

Parameters
task a TaskManager instance.

ironic.conductor.utils.**deploying_error_handler**(*task*, *logmsg*, *errmsg=None*, *traceback=False*, *clean_up=True*)

Put a failed node in DEPLOYFAIL.

Parameters

- **task** the task
- **logmsg** message to be logged
- **errmsg** message for the user
- **traceback** Boolean; True to log a traceback
- **clean_up** Boolean; True to clean up

ironic.conductor.utils.**exclude_current_conductor**(*current_conductor*, *offline_conductors*)

Wrapper to exclude current conductor from offline_conductors

In some cases the current conductor may have failed to update the heartbeat timestamp due to failure or resource starvation. When this occurs the dbapi get_offline_conductors method will include the current conductor in its return value.

Parameters

- **current_conductor** id or hostname of the current conductor
- **offline_conductors** List of offline conductors.

Returns
List of offline conductors, excluding current conductor

ironic.conductor.utils.**fail_on_error**(*error_callback*, *msg*, **error_args*, ***error_kwargs*)

A decorator for failing operation on failure.

ironic.conductor.utils.**fast_track_able**(*task*)

Checks if the operation can be a streamlined deployment sequence.

This is mainly focused on ensuring that we are able to quickly sequence through operations if we already have a ramdisk heartbeating through external means.

Parameters
task Taskmanager object

Returns
True if [deploy]fast_track is set to True, no iSCSI boot configuration is present, and no last_error is present for the node indicating that there was a recent failure.

ironic.conductor.utils.**get_node_next_clean_steps**(*task*, *skip_current_step=True*)

ironic.conductor.utils.**get_node_next_deploy_steps**(*task*, *skip_current_step=True*)

ironic.conductor.utils.**get_token_project_from_request**(*ctx*)

Identifies the request originator project via keystone token details.

This method evaluates the `auth_token_info` field, which is used to pass information returned from keystone as a tokens verification. This information is based upon the actual, original requester context provided `auth_token`.

When a service, such as Nova proxies a request, the request provided auth token value is intended to be from the original user.

Returns
The project ID value.

ironic.conductor.utils.**hash_password**(*password=''*)

Hashes a supplied password.

Parameters
password password to be hashed

ironic.conductor.utils.**inspecting_error_handler**(*task*, *logmsg*, *errmsg=None*, *traceback=False*, *tear_down_inspection=True*)

Handle inspection failures and clean up resources.

Parameters

- **task** a TaskManager instance.
- **logmsg** Message to be logged.
- **errmsg** Message for the user. Optional, if not provided *logmsg* is used.
- **traceback** Whether to log a traceback. Defaults to False.
- **tear_down_inspection** Whether to clean up the inspection network and boot configuration. Default to True.

ironic.conductor.utils.**is_agent_token_pregenerated**(*node*)

Determines if the token was generated for out of band configuration.

Ironic supports the ability to provide configuration data to the agent through the a virtual floppy or as part of the virtual media image which is attached to the BMC.

This method helps us identify WHEN we did so as we dont need to remove records of the token prior to rebooting the token. This is important as tokens provided through out of band means persist in the virtual media image, are loaded as part of the agent ramdisk, and do not require regeneration of the token upon the initial lookup, ultimately making the overall usage of virtual media and pregenerated tokens far more secure.

Parameters
node Node Object

Returns
True if the token was pregenerated as indicated by the nodes driver_internal_info field. False in all other cases.

ironic.conductor.utils.**is_agent_token_present**(*node*)

Determines if an agent token is present upon a node.

Parameters
node Node object

Returns
True if an agent_secret_token value is present in a node driver_internal_info field.

ironic.conductor.utils.**is_agent_token_valid**(*node*, *token*)

Validates if a supplied token is valid for the node.

Parameters

- **node** Node object
- **token** A token value to validate against the driver_internal_info field agent_secret_token.

Returns
True if the supplied token matches the token recorded in the supplied node object.

ironic.conductor.utils.**is_fast_track**(*task*)

Checks a fast track is available.

This method first ensures that the node and conductor configuration is valid to perform a fast track sequence meaning that we already have a ramdisk running through another means like discovery. If not valid, False is returned.

The method then checks for the last agent heartbeat, and if it occurred within the timeout set by [deploy]fast_track_timeout and the power state for the machine is POWER_ON, then fast track is permitted.

Parameters
task Taskmanager object

Returns
True if the last heartbeat that was recorded was within the [deploy]fast_track_timeout setting.

ironic.conductor.utils.**log_step_flow_history**(*node*, *flow*, *steps*, *status*, *aborted_step=None*)

Persist start/end milestones for cleaning / servicing / deploy flows.

Parameters

- **node** A Node object
- **flow** A StepFlow enum member, e.g. StepFlow.DEPLOYMENT
- **steps** full list of step dicts (may be empty/None)
- **status** 'start' or 'done'
- **aborted_step** name of failed step (optional)

ironic.conductor.utils.**make_salt**()

Generate a random salt with the indicator tag for password type.

Returns
a valid salt for use with crypt.crypt

ironic.conductor.utils.**node_cache_bios_settings**(*task*)

Do caching of bios settings if supported by driver

ironic.conductor.utils.**node_cache_boot_mode**(*task*)

Cache boot_mode and secure_boot state if supported by driver.

Cache current boot_mode and secure_boot in ironics node representation

Parameters
task a TaskManager instance containing the node to check.

ironic.conductor.utils.**node_cache_firmware_components**(*task*)

Do caching of firmware components if supported by driver

ironic.conductor.utils.**node_cache_health**(*task*)

Cache node health status from BMC if supported by the driver.

Attempts to retrieve hardware health status from the nodes management interface. If the driver supports health monitoring and the status has changed, the nodes health field is updated and an entry is added to node history.

Parameters
task a TaskManager instance containing the node to check.

ironic.conductor.utils.**node_cache_vendor**(*task*)

Cache the vendor if it can be detected.

ironic.conductor.utils.**node_change_boot_mode**(*task*, *target_boot_mode*)

Change boot mode to requested state for node

Parameters

- **task** a TaskManager instance containing the node to act on.
- **target_boot_mode** Any boot mode in *ironic.common.boot_modes*.

ironic.conductor.utils.**node_change_secure_boot**(*task*, *secure_boot_target*)

Change secure_boot state to requested state for node

Parameters

- **task** a TaskManager instance containing the node to act on.
- **secure_boot_target** (*boolean*) Target secure_boot state OneOf(True => on, False => off)

ironic.conductor.utils.**node_get_boot_mode**(*task*)

Read currently set boot mode from a node.

Reads the boot mode for a node. If boot mode cant be discovered, *None* is returned.

Parameters
task a TaskManager instance.

Raises
DriverOperationError or its derivative in case of driver runtime error.

Raises
UnsupportedDriverExtension if current driver does not have management interface or *get_boot_mode()* method is not supported.

Returns
Boot mode. One of `ironic.common.boot_mode` or *None* if boot mode cant be discovered

ironic.conductor.utils.**node_history_record**(*node*, *conductor=None*, *event=None*, *event_type=None*, *user=None*, *error=False*)

Records a node history record

Adds an entry to the node history table with the appropriate fields populated to ensure consistent experience by also updating the node `last_error` field. Please note the event is only recorded if the `[conductor]node_history_max_size` parameter is set to a value greater than `0`.

Parameters

- **node** A node object from a task object. Required.
- **conductor** The hostname of the conductor. If not specified this value is populated with the conductor FQDN.
- **event** The text to record to the node history table. If no value is supplied, the method silently returns to the caller.
- **event_type** The type activity where the event was encountered, either provisioning, monitoring, cleaning, or whatever text the a driver author wishes to supply based upon the activity. The purpose is to help guide an API consumer/operator to have a better contextual understanding of what was going on *when* the event occurred.
- **user** The user_id value which triggered the request, if available.
- **error** Boolean value, default false, to signify if the event is an error which should be recorded in the node `last_error` field.

Returns
None. No value is returned by this method.

ironic.conductor.utils.**node_power_action**(*task*, *new_state*, *timeout=None*)

Change power state or reset for a node.

Perform the requested power action if the transition is required.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **new_state** Any power state from ironic.common.states.

- **timeout** timeout (in seconds) positive integer (> 0) for any power state. None indicates to use default timeout.

Raises
InvalidParameterValue when the wrong state is specified or the wrong driver info is specified.

Raises
StorageError when a failure occurs updating the nodes storage interface upon setting power on.

Raises
other exceptions by the nodes power driver if something wrong occurred during the power action.

Raises
ChildNodeLocked when a child node must be acted upon due to this request, but we cannot act upon it due to a lock being held.

Raises
ParentNodeLocked when a parent node must be acted upon due to this request, but we cannot act upon it due to a lock being held.

ironic.conductor.utils.**node_set_boot_device**(*task*, *device*, *persistent=False*)

Set the boot device for a node.

If the node that the boot device change is being requested for is in ADOPTING state, the boot device will not be set as that change could potentially result in the future running state of an adopted node being modified erroneously.

Parameters

- **task** a TaskManager instance.
- **device** Boot device. Values are vendor-specific.
- **persistent** Whether to set next-boot, or make the change permanent. Default: False.

Raises
InvalidParameterValue if the validation of the ManagementInterface fails.

ironic.conductor.utils.**node_set_boot_mode**(*task*, *mode*)

Set the boot mode for a node.

Sets the boot mode for a node if the nodes driver interface contains a management interface.

If the node that the boot mode change is being requested for is in ADOPTING state, the boot mode will not be set as that change could potentially result in the future running state of an adopted node being modified erroneously.

Parameters

- **task** a TaskManager instance.
- **mode** Boot mode. Values are one of *ironic.common.boot_modes*

Raises
InvalidParameterValue if the validation of the ManagementInterface fails.

Raises
DriverOperationError or its derivative in case of driver runtime error.

Raises
UnsupportedDriverExtension if current driver does not have vendor interface or method is unsupported.

ironic.conductor.utils.**node_update_cache**(*task*)

Updates various cached information.

Includes vendor, boot mode, BIOS settings and firmware components.

Parameters
task A TaskManager instance containing the node to act on.

ironic.conductor.utils.**node_wait_for_power_state**(*task*, *new_state*, *timeout=None*)

Wait for node to be in new power state.

Parameters

- **task** a TaskManager instance.
- **new_state** the desired new power state, one of the power states in *ironic.common.states*.
- **timeout** number of seconds to wait before giving up. If not specified, uses the conductor.power_state_change_timeout config value.

Raises
PowerStateFailure if timed out

ironic.conductor.utils.**notify_conductor_resume_clean**(*task*)

ironic.conductor.utils.**notify_conductor_resume_deploy**(*task*)

ironic.conductor.utils.**notify_conductor_resume_operation**(*task*, *operation*)

Notify the conductor to resume an operation.

Parameters

- **task** the task
- **operation** the operation, a string

ironic.conductor.utils.**notify_conductor_resume_service**(*task*)

ironic.conductor.utils.**power_on_node_if_needed**(*task*)

Powers on node if it is powered off and has a Smart NIC port

Parameters
task A TaskManager object

Returns
the previous power state or None if no changes were made

Raises
exception.NetworkError if agent status didnt match the required status after max retry attempts.

ironic.conductor.utils.**power_state_error_handler**(*e*, *node*, *power_state*)

Set the nodes power states if error occurs.

This hook gets called upon an exception being raised when spawning the worker thread to change the power state of a node.

Parameters

- **e** the exception object that was raised.
- **node** an Ironic node object.
- **power_state** the power state to set on the node.

ironic.conductor.utils.**power_state_for_network_configuration**(*task*)

Handle the power state for a node reconfiguration.

Powers the node on if and only if it has a Smart NIC port. Yields for the actual reconfiguration, then restores the power state.

Parameters

task A TaskManager object.

ironic.conductor.utils.**provisioning_error_handler**(*e*, *node*, *provision_state*, *target_provision_state*)

Set the nodes provisioning states if error occurs.

This hook gets called upon an exception being raised when spawning the worker to do some provisioning to a node like deployment, tear down, or cleaning.

Parameters

- **e** the exception object that was raised.
- **node** an Ironic node object.
- **provision_state** the provision state to be set on the node.
- **target_provision_state** the target provision state to be set on the node.

ironic.conductor.utils.**remove_agent_url**(*node*)

Helper to remove the agent_url record.

ironic.conductor.utils.**remove_node_rescue_password**(*node*, *save=True*)

Helper to remove rescue password from a node.

Removes rescue password from node. It saves node by default. If node should not be saved, then caller needs to explicitly indicate it.

Parameters

- **node** an Ironic node object.
- **save** Boolean; True (default) to save the node; False otherwise.

ironic.conductor.utils.**rescuing_error_handler**(*task*, *msg*, *set_fail_state=True*)

Cleanup rescue task after timeout or failure.

Parameters

- **task** a TaskManager instance.

- **msg** a message to set into nodes last_error field
- **set_fail_state** a boolean flag to indicate if node needs to be transitioned to a failed state. By default node would be transitioned to a failed state.

ironic.conductor.utils.**restore_power_state_if_needed**(*task*, *power_state_to_restore*)

Change the nodes power state if power_state_to_restore is not None

Parameters

- **task** A TaskManager object
- **power_state_to_restore** power state

ironic.conductor.utils.**run_node_action**(*task*, *call*, *error_msg*, *success_msg=None*, ***kwargs*)

Run a node action and report any errors via last_error.

Parameters

- **task** A TaskManager instance containing the node to act on.
- **call** A callable object to invoke.
- **error_msg** A template for a failure message. Can use %(node)s, %(exc)s and any variables from kwargs.
- **success_msg** A template for a success message. Can use %(node)s and any variables from kwargs.
- **kwargs** Arguments to pass to the call.

ironic.conductor.utils.**servicing_error_handler**(*task*, *logmsg*, *errmsg=None*, *traceback=False*, *tear_down_service=True*, *set_fail_state=True*, *set_maintenance=None*)

Put a failed node in SERVICEFAIL and maintenance (if needed).

Parameters

- **task** a TaskManager instance.
- **logmsg** Message to be logged.
- **errmsg** Message for the user. Optional, if not provided *logmsg* is used.
- **traceback** Whether to log a traceback. Defaults to False.
- **tear_down_service** Whether to clean up the PXE and DHCP files after service. Default to True.
- **set_fail_state** Whether to set node to failed state. Default to True.
- **set_maintenance** Whether to set maintenance mode. If None, maintenance mode will be set if and only if a clean step is being executed on a node.

ironic.conductor.utils.**skip_automated_cleaning**(*node*)

Checks if node cleaning needs to be skipped for an specific node.

Parameters

node the node to consider

ironic.conductor.utils.**spawn_cleaning_error_handler**(*e*, *node*)

Handle spawning error for node cleaning.

ironic.conductor.utils.**spawn_deploying_error_handler**(*e*, *node*)

Handle spawning error for node deploying.

ironic.conductor.utils.**spawn_rescue_error_handler**(*e*, *node*)

Handle spawning error for node rescue.

ironic.conductor.utils.**spawn_servicing_error_handler**(*e*, *node*)

Handle spawning error for node servicing.

ironic.conductor.utils.**store_agent_certificate**(*node*, *agent_verify_ca*)

Store certificate received from the agent and return its path.

ironic.conductor.utils.**update_image_type**(*context*, *node*)

Updates is_whole_disk_image and image_type based on the node data.

Parameters

- **context** Request context.
- **node** Node object.

Returns

True if any changes have been done, else False.

ironic.conductor.utils.**update_next_step_index**(*task*, *step_type*)

Calculate the next step index and update the node.

Parameters

- **task** A TaskManager object
- **step_type** The type of steps to process: clean or deploy.

Returns

Index of the next step.

ironic.conductor.utils.**validate_instance_info_traits**(*node*)

Validate traits in instance_info.

All traits in instance_info must also exist as node traits.

Parameters

node an Ironic node object.

Raises

InvalidParameterValue if the instance traits are badly formatted, or contain traits that are not set on the node.

ironic.conductor.utils.**validate_port_physnet**(*task*, *port_obj*)

Validate the consistency of physical networks of ports in a portgroup.

Validate the consistency of a ports physical network with other ports in the same portgroup. All ports in a portgroup should have the same value (which may be None) for their physical_network field.

During creation or update of a port in a portgroup we apply the following validation criteria:

- If the portgroup has existing ports with different physical networks, we raise PortgroupPhysnetInconsistent. This shouldnt ever happen.
- If the port has a physical network that is inconsistent with other ports in the portgroup, we raise exception.Conflict.

If a ports physical network is None, this indicates that ironics VIF attachment mapping algorithm should operate in a legacy (physical network unaware) mode for this port or portgroup. This allows existing ironic nodes to continue to function after an upgrade to a release including physical network support.

Parameters

- **task** a TaskManager instance
- **port_obj** a port object to be validated.

Raises

Conflict if the port is a member of a portgroup which is on a different physical network.

Raises

PortgroupPhysnetInconsistent if the ports portgroup has ports which are not all assigned the same physical network.

ironic.conductor.utils.**value_within_timeout**(*value*, *timeout*)

Checks if the time is within the previous timeout seconds from now.

Parameters

- **value** a datetime or string representing date and time or None.
- **timeout** timeout in seconds.

ironic.conductor.utils.**verifying_error_handler**(*task*, *logmsg*, *errmsg=None*, *traceback=False*)

Handle errors during verification steps

Parameters

- **task** the task
- **logmsg** message to be logged
- **errmsg** message for the user
- **traceback** Boolean; True to log a traceback

ironic.conductor.utils.**wipe_cleaning_internal_info**(*task*)

Remove temporary cleaning fields from driver_internal_info.

ironic.conductor.utils.**wipe_deploy_internal_info**(*task*)

Remove temporary deployment fields from driver_internal_info.

ironic.conductor.utils.**wipe_internal_info_on_power_off**(*node*)

Wipe information that should not survive reboot/power off.

ironic.conductor.utils.**wipe_service_internal_info**(*task*)

Remove temporary servicing fields from driver_internal_info.

ironic.conductor.utils.**wipe_token_and_url**(*task*)

Remove agent URL and token from the task.

ironic.conductor.verify module

Functionality related to verify steps.

ironic.conductor.verify.**do_node_verify**(*task*)

Internal method to perform power credentials verification.

Module contents

ironic.conf package

Submodules

ironic.conf.agent module

ironic.conf.agent.**register_opts**(*conf*)

ironic.conf.agent_containers module

ironic.conf.agent_containers.**list_opts**()

ironic.conf.agent_containers.**register_opts**(*conf*)

ironic.conf.anaconda module

ironic.conf.anaconda.**register_opts**(*conf*)

ironic.conf.ansible module

ironic.conf.ansible.**register_opts**(*conf*)

ironic.conf.api module

class ironic.conf.api.**Octal**(*min=None*, *max=None*, *type_name='integer value'*, *choices=None*)

Bases: Integer

__call__(*value*)

Call self as a function.

ironic.conf.api.**register_opts**(*conf*)

ironic.conf.audit module

ironic.conf.audit.**register_opts**(*conf*)

ironic.conf.auth module

ironic.conf.auth.**add_auth_opts**(*options*, *service_type=None*)

Add auth options to sample config

As these are dynamically registered at runtime, this adds options for most used auth_plugins when generating sample config.

ironic.conf.auth.**register_auth_opts**(*conf*, *group*, *service_type=None*)

Register session- and auth-related options

Registers only basic auth options shared by all auth plugins. The rest are registered at runtime depending on auth plugin used.

ironic.conf.cifs module

ironic.conf.cifs.**register_opts**(*conf*)

ironic.conf.cinder module

ironic.conf.cinder.**list_opts**()

ironic.conf.cinder.**register_opts**(*conf*)

ironic.conf.conductor module

ironic.conf.conductor.**register_opts**(*conf*)

ironic.conf.console module

class ironic.conf.console.**PortRange**

Bases: String

__call__(*value*)

Call self as a function.

ironic.conf.console.**register_opts**(*conf*)

ironic.conf.database module

ironic.conf.database.**register_opts**(*conf*)

ironic.conf.default module

ironic.conf.default.**list_opts**()

ironic.conf.default.**register_opts**(*conf*)

ironic.conf.deploy module

ironic.conf.deploy.**register_opts**(*conf*)

ironic.conf.dhcp module

ironic.conf.dhcp.**register_opts**(*conf*)

ironic.conf.disk_utils module

ironic.conf.disk_utils.**register_opts**(*conf*)

ironic.conf.dnsmasq module

ironic.conf.dnsmasq.**register_opts**(*conf*)

ironic.conf.drac module

ironic.conf.drac.**register_opts**(*conf*)

ironic.conf.exception module

ironic.conf.exception.**register_opts**(*conf*)

ironic.conf.fake module

ironic.conf.fake.**register_opts**(*conf*)

ironic.conf.glance module

ironic.conf.glance.**list_opts**()

ironic.conf.glance.**register_opts**(*conf*)

ironic.conf.healthcheck module

ironic.conf.healthcheck.**register_opts**(*conf*)

ironic.conf.ilo module

ironic.conf.ilo.**register_opts**(*conf*)

ironic.conf.inspector module

ironic.conf.inspector.**register_opts**(*conf*)

ironic.conf.inventory module

ironic.conf.inventory.**register_opts**(*conf*)

ironic.conf.ipmi module

ironic.conf.ipmi.**register_opts**(*conf*)

ironic.conf.ironic_networking module

ironic.conf.ironic_networking.**list_opts**()

ironic.conf.ironic_networking.**register_opts**(*conf*)

ironic.conf.json_rpc module

ironic.conf.json_rpc.**auth_strategy**(*group='json_rpc'*)

Return effective auth strategy for a given group.

Parameters

group Configuration group name. Defaults to json_rpc.

Returns

Authentication strategy string.

ironic.conf.json_rpc.**list_opts**()

Return options for sample configuration generation.

ironic.conf.json_rpc.**register_opts**(*conf*, *group='json_rpc'*)

Register JSON-RPC options.

Parameters

- **conf** The global oslo.config object.
- **group** Configuration group name to register options under. Defaults to json_rpc. Allows reusing the same option set for different JSON-RPC clients (e.g., networking).

ironic.conf.mdns module

ironic.conf.mdns.**register_opts**(*conf*)

ironic.conf.metrics module

ironic.conf.metrics.**list_opts**()

ironic.conf.metrics.**register_opts**(*conf*)

ironic.conf.molds module

ironic.conf.molds.**register_opts**(*conf*)

ironic.conf.neutron module

ironic.conf.neutron.**list_opts**()

ironic.conf.neutron.**register_opts**(*conf*)

ironic.conf.nfs module

ironic.conf.nfs.**register_opts**(*conf*)

ironic.conf.nova module

ironic.conf.nova.**list_opts**()

ironic.conf.nova.**register_opts**(*conf*)

ironic.conf.oci module

ironic.conf.oci.**register_opts**(*conf*)

ironic.conf.opts module

ironic.conf.opts.**list_opts**()

Return a list of oslo.config options available in Ironic code.

The returned list includes all oslo.config options. Each element of the list is a tuple. The first element is the name of the group, the second element is the options.

The function is discoverable via the ironic entry point under the oslo.config.opts namespace.

The function is used by Oslo sample config file generator to discover the options.

Returns
a list of (group, options) tuples

ironic.conf.opts.**update_opt_defaults**()

ironic.conf.pxe module

ironic.conf.pxe.**register_opts**(*conf*)

ironic.conf.redfish module

ironic.conf.redfish.**register_opts**(*conf*)

ironic.conf.sensor_data module

ironic.conf.sensor_data.**register_opts**(*conf*)

ironic.conf.service_catalog module

ironic.conf.service_catalog.**list_opts**()

ironic.conf.service_catalog.**register_opts**(*conf*)

ironic.conf.snmp module

ironic.conf.snmp.**register_opts**(*conf*)

ironic.conf.swift module

ironic.conf.swift.**list_opts**()

ironic.conf.swift.**register_opts**(*conf*)

ironic.conf.types module

class ironic.conf.types.**ExplicitAbsolutePath**(*type_name='explicit absolute path'*)

Bases: ConfigType

Explicit Absolute path type.

Absolute path values do not get transformed and are returned as strings. They are validated to ensure they are absolute paths and are equal to os.path.abspath(value) protecting from path traversal issues.

Python path libraries generally define absolute path as anything starting with a /, so tools like path.PurePath(str).is_absolute() is not useful, because it will happily return that /tmp/../etc/resolv.conf is absolute. This type is to be used in cases where we require the path to be explicitly absolute.

Parameters

type_name Type name to be used in the sample config file.

__call__(*value*)

Call self as a function.

ironic.conf.vnc module

ironic.conf.vnc.**register_opts**(*conf*)

Module contents

ironic.console package

Subpackages

ironic.console.container package

Submodules

ironic.console.container.base module

Abstract base class for console container providers.

class ironic.console.container.base.**BaseConsoleContainer**

Bases: object

Base class for console container provider APIs.

abstract start_container(*task*, *app_name*, *app_info*)

Start a console container for a node.

Calling this will block until a consumable container host and port can be returned.

Parameters

- **task** A TaskManager instance.
- **app_name** Name of app to run in the container
- **app_info** Dict of app-specific info

Returns
Tuple of host IP address and published port

Raises
ConsoleContainerError

abstract stop_all_containers()

Stops all running console containers

This is run on conductor startup and graceful shutdown to ensure no console containers are running. :raises: ConsoleContainerError

abstract stop_container(*task*)

Stop a console container for a node.

Any existing running container for this node will be stopped.

Parameters
task A TaskManager instance.

Raises
ConsoleContainerError

ironic.console.container.fake module

Fake console container provider for disabling .

class ironic.console.container.fake.**FakeConsoleContainer**

Bases: *BaseConsoleContainer*

start_container(*task*, *app_name*, *app_info*)

Start a console container for a node.

Calling this will block until a consumable container host and port can be returned.

Parameters

- **task** A TaskManager instance.
- **app_name** Name of app to run in the container
- **app_info** Dict of app-specific info

Returns
Tuple of host IP address and published port

Raises
ConsoleContainerError

stop_all_containers()

Stops all running console containers

This is run on conductor startup and graceful shutdown to ensure no console containers are running. :raises: ConsoleContainerError

stop_container(*task*)

Stop a console container for a node.

Any existing running container for this node will be stopped.

Parameters
task A TaskManager instance.

Raises
ConsoleContainerError

ironic.console.container.kubernetes module

Kubernetes pod console container provider.

class ironic.console.container.kubernetes.**KubernetesConsoleContainer**

Bases: *BaseConsoleContainer*

Console container provider which uses kubernetes pods.

start_container(*task*, *app_name*, *app_info*)

Start a console container for a node.

Any existing running container for this node will be stopped.

Parameters
task A TaskManager instance.

Raises
ConsoleContainerError

stop_all_containers()

Stops all running console containers

This is run on conductor startup and graceful shutdown to ensure no console containers are running. :raises: ConsoleContainerError

stop_container(*task*)

Stop a console container for a node.

Any existing running container for this node will be stopped.

Parameters
task A TaskManager instance.

Raises
ConsoleContainerError

ironic.console.container.systemd module

Systemd Quadlet console container provider.

class ironic.console.container.systemd.**SystemdConsoleContainer**

Bases: *BaseConsoleContainer*

Console container provider which uses Systemd Quadlets.

start_container(*task*, *app_name*, *app_info*)

Start a console container for a node.

Starts a console container for the tasks node.

Parameters

- **task** A TaskManager instance.
- **app_name** Sets container environment APP value
- **app_info** Sets container environment APP_INFO value

Raises

ConsoleContainerError

stop_all_containers()

Stops all running console containers

This is run on conductor startup and graceful shutdown to ensure no console containers are running. :raises: ConsoleContainerError

stop_container(*task*)

Stop a console container for a node.

Any existing running container for this node will be stopped.

Parameters

task A TaskManager instance.

Raises

ConsoleContainerError

unit_dir = None

Module contents

ironic.console.rfb package

Submodules

ironic.console.rfb.auth module

class ironic.console.rfb.auth.**AuthType**(*value*, *names=<not given>*, **values*, *module=None*, *qualname=None*, *type=None*, *start=1*, *boundary=None*)

Bases: IntEnum

ARD = 30

INVALID = 0

MSLOGON = 4294967290

NONE = 1

RA2 = 5

RA2NE = 6

SASL = 20

TIGHT = 16

TLS = 18

ULTRA = 17

VENCRYPT = 19

VNC = 2

class ironic.console.rfb.auth.**RFBAuthScheme**

Bases: `object`

abstract security_handshake(*host_sock*)

Perform security-type-specific functionality.

This method is expected to return the socket-like object used to communicate with the server securely.

Should raise ironic.common.exception.RFBAuthHandshakeFailed if an error occurs

Parameters
host_sock socket connected to the host instance

abstract security_type()

Return the security type supported by this scheme

Returns the nova.console.rfb.auth.AuthType.XX constant representing the scheme implemented.

ironic.console.rfb.authnone module

class ironic.console.rfb.authnone.**RFBAuthSchemeNone**

Bases: *RFBAuthScheme*

security_handshake(*host_sock*, *password=None*)

Perform security-type-specific functionality.

This method is expected to return the socket-like object used to communicate with the server securely.

Should raise ironic.common.exception.RFBAuthHandshakeFailed if an error occurs

Parameters
host_sock socket connected to the host instance

security_type()

Return the security type supported by this scheme

Returns the nova.console.rfb.auth.AuthType.XX constant representing the scheme implemented.

ironic.console.rfb.auths module

class ironic.console.rfb.auths.**RFBAuthSchemeList**

Bases: object

AUTH_SCHEME_MAP = {'none': <class 'ironic.console.rfb.authnone.RFBAuthSchemeNone'>}

find_scheme(*desired_types*)

Find a suitable authentication scheme to use with compute node.

Identify which of the desired_types we can accept.

Parameters

desired_types A list of ints corresponding to the various authentication types supported.

Module contents

ironic.console.securityproxy package

Submodules

ironic.console.securityproxy.base module

class ironic.console.securityproxy.base.**SecurityProxy**

Bases: object

A console security Proxy Helper

Console security proxy helpers should subclass this class and implement a generic *connect* for the particular protocol being used.

Security drivers can then subclass the protocol-specific helper class.

abstract connect(*tenant_sock*, *host_sock*)

Initiate the console connection

This method performs the protocol specific negotiation, and returns the socket-like object to use to communicate with the host securely.

Parameters

- **tenant_sock** socket connected to the remote tenant user
- **host_sock** socket connected to the host node

Returns

a new host_sock for the node

ironic.console.securityproxy.rfb module

class ironic.console.securityproxy.rfb.**RFBSecurityProxy**

Bases: *SecurityProxy*

RFB Security Proxy Negotiation Helper.

This class proxies the initial setup of the RFB connection between the client and the server. Then, when the RFB security negotiation step arrives, it intercepts the communication, posing as a server with the None authentication type to the client, and acting as a client (via the methods below) to the server. After security negotiation, normal proxying can be used.

Note: this code mandates RFB version 3.8, since this is supported by any client and server impl written in the past 10+ years.

See the general RFB specification at:

https://tools.ietf.org/html/rfc6143

See an updated, maintained RDB specification at:

https://github.com/rfbproto/rfbproto/blob/master/rfbproto.rst

connect(*tenant_sock*, *host_sock*)

Initiate the RFB connection process.

This method performs the initial ProtocolVersion and Security messaging, and returns the socket-like object to use to communicate with the server securely. If an error occurs SecurityProxyNegotiationFailed will be raised.

Module contents

Submodules

ironic.console.novncproxy_service module

class ironic.console.novncproxy_service.**NoVNCProxyService**

Bases: Service

start()

Start service.

ironic.console.websocketproxy module

Websocket proxy that is compatible with OpenStack Ironic. Leverages websockify.py by Joel Martin

class ironic.console.websocketproxy.**IronicProxyRequestHandler**(**args*, ***kwargs*)

Bases: ProxyRequestHandler

new_websocket_client()

Called after a new WebSocket connection has been established.

send_head()

Common code for GET and HEAD commands.

This sends the response code and MIME headers.

Return value is either a file object (which has to be copied to the outputfile by the caller unless the command was HEAD, and must be closed by the caller under all circumstances), or None, in which case the caller has nothing further to do.

socket(**args*, ***kwargs*)

class ironic.console.websocketproxy.**IronicWebSocketProxy**(**args*, ***kwargs*)

Bases: WebSocketProxy

static get_logger()

terminate()

Override WebSockifyServer terminate

WebSocifyServer.Terminate exception is not handled by oslo_service, so raise SystemExit instead.

class ironic.console.websocketproxy.**TenantSock**(*reqhandler*)

Bases: object

A socket wrapper for communicating with the tenant.

This class provides a socket-like interface to the internal websockify send/receive queue for the client connection to the tenant user. It is used with the security proxy classes.

close()

finish_up()

recv(*cnt*)

sendall(*data*)

Module contents

ironic.db package

Subpackages

ironic.db.sqlalchemy package

Submodules

ironic.db.sqlalchemy.api module

SQLAlchemy storage backend.

class ironic.db.sqlalchemy.api.**Connection**

Bases: *Connection*

SqlAlchemy connection.

add_node_tag(*node_id*, *tag*)

Add tag to the node.

If the node_id and tag pair already exists, this should still succeed.

Parameters

- **node_id** The id of a node.
- **tag** A tag string.

Returns
the NodeTag object.

Raises
NodeNotFound if the node is not found.

add_node_trait(*node_id*, *trait*, *version*)

Add trait to the node.

If the node_id and trait pair already exists, this should still succeed.

Parameters

- **node_id** The id of a node.
- **trait** A trait string.
- **version** the version of the object.Trait.

Returns
the NodeTrait object.

Raises
InvalidParameterValue if adding the trait would exceed the per-node traits limit.

Raises
NodeNotFound if the node is not found.

bulk_delete_node_history_records(*entries*)

Utility method to bulk delete node history entries.

Parameters
entries A list of node history entry ids to be queried for deletion.

check_node_list(*idents*, *project=None*)

Check a list of node identities and map it to UUIDs.

This call takes a list of node names and/or UUIDs and tries to convert them to UUIDs. It fails early if any identities cannot possible be used as names or UUIDs.

Parameters
idents List of identities.

Returns
A mapping from requests identities to node UUIDs.

Raises
NodeNotFound if some identities were not found or cannot be valid names or UUIDs.

check_versions(*ignore_models=()*, *permit_initial_version=False*)

Checks the whole database for incompatible objects.

This scans all the tables in search of objects that are not supported; i.e., those that are not specified in *ironic.common.release_mappings.RELEASE_MAPPING*. This includes objects that have null version values.

Parameters

- **ignore_models** List of model names to skip.
- **permit_initial_version** Boolean, default False, to permit a NoSuchTableError exception to be raised by SQLAlchemy and accordingly bypass when an object has its initial object version.

Returns
A Boolean. True if all the objects have supported versions; False otherwise.

clear_node_reservations_for_conductor(*hostname*)

clear_node_target_power_state(*hostname*)

count_nodes_in_provision_state(*state*, *conductor_group=None*)
Count the number of nodes in given provision state.

Parameters

- **state** A provision_state value to match for the count operation. This can be a single provision state value or a list of values.
- **conductor_group** Optional conductor group to filter by. If provided, only nodes in this conductor group will be counted.

create_allocation(*values*)
Create a new allocation.

Parameters
values Dict of values to create an allocation with

Returns
An allocation

Raises
AllocationDuplicateName

Raises
AllocationAlreadyExists

create_bios_setting_list(*node_id*, *settings*, *version*)
Create a list of BIOSSetting records for a given node.

Parameters

- **node_id** The node id.
- **settings** A list of BIOS Settings to be created.

```
[
  {
   'name': String,
   'value': String,
   additional settings from BIOS registry
  },
  {
   'name': String,
   'value': String,
```

(continues on next page)

(continued from previous page)

```
    additional settings from BIOS registry
  },
  ...
]
```

- **version** the version of the object.BIOSSetting.

Returns
A list of BIOSSetting object.

Raises
NodeNotFound if the node is not found.

Raises
BIOSSettingAlreadyExists if any of the setting records already exists.

create_chassis(*values*)

Create a new chassis.

Parameters
values Dict of values.

create_deploy_template(*values*)

Create a deployment template.

Parameters
values A dict describing the deployment template. For example:

```
{
 'uuid': uuidutils.generate_uuid(),
 'name': 'CUSTOM_DT1',
}
```

Raises
DeployTemplateDuplicateName if a deploy template with the same name exists.

Raises
DeployTemplateAlreadyExists if a deploy template with the same UUID exists.

Returns
A deploy template.

create_firmware_component(*values*)

Create a FirmwareComponent record for a given node.

Parameters
values a dictionary with the necessary information to create a FirmwareComponent.

```
{
  'component': String,
  'initial_version': String,
  'current_version': String,
```

(continues on next page)

(continued from previous page)

```
  'last_version_flashed': String
}
```

Returns
A FirmwareComponent object.

Raises
FirmwareComponentAlreadyExists if any of the component records already exists.

create_inspection_rule(*values*)

Create new rule

create_node(*values*)

Create a new node.

Parameters
values A dict containing several items used to identify and track the node, and several dicts which are passed into the Drivers when managing this node. For example:

```
{
 'uuid': uuidutils.generate_uuid(),
 'instance_uuid': None,
 'power_state': states.POWER_OFF,
 'provision_state': states.AVAILABLE,
 'driver': 'ipmi',
 'driver_info': { ... },
 'properties': { ... },
 'extra': { ... },
}
```

Raises
InvalidParameterValue if values contains tags or traits.

Returns
A node.

create_node_history(*values*)

Create a new history record.

Parameters
values Dict of values.

create_node_inventory(*values*)

Create a new inventory record.

Parameters
values Dict of values.

create_port(*values*)

Create a new port.

Parameters
values Dict of values.

create_portgroup(*values*)

Create a new portgroup.

Parameters

values Dict of values with the following keys: id uuid name node_id address extra created_at updated_at

Returns

A portgroup

Raises

PortgroupDuplicateName

Raises

PortgroupMACAlreadyExists

Raises

PortgroupAlreadyExists

create_runbook(*values*)

Create a runbook.

Parameters

values A dict describing the runbook. For example:

```
{
 'uuid': uuidutils.generate_uuid(),
 'name': 'CUSTOM_DT1',
}
```

Raises

RunbookDuplicateName if a runbook with the same name exists.

Raises

RunbookAlreadyExists if a runbook with the same UUID exists.

Returns

A runbook.

create_volume_connector(*connector_info*)

Create a new volume connector.

Parameters

connector_info Dictionary containing information about the connector. Example:

```
{
    'uuid': '000000-..',
    'type': 'wwnn',
    'connector_id': '00:01:02:03:04:05:06',
    'node_id': 2
}
```

Returns

A volume connector.

Raises
VolumeConnectorTypeAndIdAlreadyExists If a connector already exists with a matching type and connector_id.

Raises
VolumeConnectorAlreadyExists If a volume connector with the same UUID already exists.

`create_volume_target`(*target_info*)

Create a new volume target.

Parameters
target_info Dictionary containing the information about the volume target. Example:

```
{
    'uuid': '000000-..',
    'node_id': 2,
    'boot_index': 0,
    'volume_id': '12345678-...'
    'volume_type': 'some type',
}
```

Returns
A volume target.

Raises
VolumeTargetBootIndexAlreadyExists if a volume target already exists with the same boot index and node ID.

Raises
VolumeTargetAlreadyExists if a volume target with the same UUID exists.

`delete_bios_setting_list`(*node_id*, *names*)

Delete a list of BIOS settings.

Parameters

- **node_id** The node id.
- **names** List of BIOS setting names to be deleted.

Raises
NodeNotFound if the node is not found.

Raises
BIOSSettingNotFound if any of BIOS setting name is not found.

`delete_conductor`(*hostname*)

Delete a conductor from the database.

Parameters
hostname The hostname of this conductor service.

Raises
ConductorNotFound if the conductor doesnt exist.

delete_node_tag(*node_id*, *tag*)

Delete specified tag from the node.

Parameters

- **node_id** The id of a node.
- **tag** A tag string.

Raises
NodeNotFound if the node is not found.

Raises
NodeTagNotFound if the tag is not found.

delete_node_trait(*node_id*, *trait*)

Delete specified trait from the node.

Parameters

- **node_id** The id of a node.
- **trait** A trait string.

Raises
NodeNotFound if the node is not found.

Raises
NodeTraitNotFound if the trait is not found.

destroy_allocation(*allocation_id*)

Destroy an allocation.

Parameters
allocation_id Allocation ID or UUID

Raises
AllocationNotFound

destroy_chassis(*chassis_id*)

Destroy a chassis.

Parameters
chassis_id The id or the uuid of a chassis.

destroy_deploy_template(*template_id*)

Destroy a deployment template.

Parameters
template_id ID of the deployment template to destroy.

Raises
DeployTemplateNotFound if the deploy template does not exist.

destroy_inspection_rule(*inspection_rule_id*)

Destroy an inspection rule.

Parameters
inspection_rule_id ID of the inspection_rule to destroy.

Raises
inspection_ruleNotFound if the inspection_rule does not exist.

`destroy_node`(*node_id*)
Destroy a node and its associated resources.

Destroy a node, including any associated ports, port groups, tags, traits, volume connectors, and volume targets.

Parameters
`node_id` The ID or UUID of a node.

`destroy_node_history_by_uuid`(*history_uuid*)
Destroy a history record.

Parameters
`history_uuid` The uuid of a history record

`destroy_node_inventory_by_node_id`(*node_id*)
Destroy a inventory record.

Parameters
`node_id` The node id value for records to be removed.

`destroy_port`(*port_id*)
Destroy an port.

Parameters
`port_id` The id or MAC of a port.

`destroy_portgroup`(*portgroup_id*)
Destroy a portgroup.

Parameters
`portgroup_id` The UUID or MAC of a portgroup.

Raises
PortgroupNotEmpty

Raises
PortgroupNotFound

`destroy_runbook`(*runbook_id*)
Destroy a runbook.

Parameters
`runbook_id` ID of the runbook to destroy.

Raises
RunbookNotFound if the runbook does not exist.

`destroy_volume_connector`(*ident*)
Destroy a volume connector.

Parameters
`ident` The UUID or integer ID of a volume connector.

Raises
VolumeConnectorNotFound If a volume connector with the specified ident does not exist.

destroy_volume_target(*ident*)

Destroy a volume target.

Parameters

ident The UUID or integer ID of a volume target.

Raises

VolumeTargetNotFound if a volume target with the specified ident does not exist.

get_active_hardware_type_dict(*use_groups=False*)

Retrieve hardware types for the registered and active conductors.

Parameters

use_groups Whether to factor conductor_group into the keys.

Returns

A dict which maps hardware type names to the set of hosts which support them. For example:

```
{hardware-type-a: set([host1, host2]),
 hardware-type-b: set([host2, host3])}
```

get_allocation_by_id(*allocation_id*)

Return an allocation representation.

Parameters

allocation_id The id of an allocation.

Returns

An allocation.

Raises

AllocationNotFound

get_allocation_by_name(*name*)

Return an allocation representation.

Parameters

name The logical name of an allocation.

Returns

An allocation.

Raises

AllocationNotFound

get_allocation_by_uuid(*allocation_uuid*)

Return an allocation representation.

Parameters

allocation_uuid The uuid of an allocation.

Returns

An allocation.

Raises

AllocationNotFound

get_allocation_list(*filters=None*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Return a list of allocations.

Parameters

- **filters** Filters to apply. Defaults to None.

  node_uuid
  uuid of node

  state
  allocation state

  resource_class
  requested resource class

- **limit** Maximum number of allocations to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** Direction in which results should be sorted. (asc, desc)

Returns
A list of allocations.

get_bios_setting(*node_id*, *name*)

Retrieve BIOS setting value.

Parameters

- **node_id** The node id.
- **name** String containing name of BIOS setting to be retrieved.

Returns
The BIOSSetting object.

Raises
NodeNotFound if the node is not found.

Raises
BIOSSettingNotFound if the BIOS setting is not found.

get_bios_setting_list(*node_id*)

Retrieve BIOS settings of a given node.

Parameters
node_id The node id.

Returns
A list of BIOSSetting objects.

Raises
NodeNotFound if the node is not found.

get_chassis_by_id(*chassis_id*)

Return a chassis representation.

Parameters
chassis_id The id of a chassis.

Returns
A chassis.

get_chassis_by_uuid(*chassis_uuid*)

Return a chassis representation.

Parameters
chassis_uuid The uuid of a chassis.

Returns
A chassis.

get_chassis_list(*limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Return a list of chassis.

Parameters

- **limit** Maximum number of chassis to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** direction in which results should be sorted. (asc, desc)

get_child_node_ids_by_parent_uuid(*node_uuid*, *exclude_dedicated_power=False*)

Retrieve a list of child node IDs for a given parent UUID.

This is an internal method, intended for use with power management logic to facilitate power actions upon nodes, where we obtain a list of nodes which requires additional actions, and return *only* the required node IDs in order to launch new tasks for power management activities.

Parameters

- **node_uuid** The uuid of the parent node, in order to directly match the parent_node field.
- **exclude_dedicated_power** Boolean, False, if the list should include child nodes with their own power supplies.

Returns
A list of tuples.

get_conductor(*hostname*, *online=True*)

Retrieve a conductors service record from the database.

Parameters

- **hostname** The hostname of the conductor service.
- **online** Specify the filter value on the *online* field when querying conductors. The `online` field is ignored if this value is set to None.

Returns
A conductor.

Raises
ConductorNotFound if the conductor with given hostname does not exist or doesnt meet the specified online expectation.

`get_conductor_list`(*limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Return a list of conductors.

Parameters

- `limit` Maximum number of conductors to return.
- `marker` the last item of the previous page; we return the next result set.
- `sort_key` Attribute by which results should be sorted.
- `sort_dir` direction in which results should be sorted. (asc, desc)

`get_deploy_template_by_id`(*template_id*)

Retrieve a deployment template by ID.

Parameters
`template_id` ID of the deployment template to retrieve.

Raises
DeployTemplateNotFound if the deploy template does not exist.

Returns
A deploy template.

`get_deploy_template_by_name`(*template_name*)

Retrieve a deployment template by name.

Parameters
`template_name` name of the deployment template to retrieve.

Raises
DeployTemplateNotFound if the deploy template does not exist.

Returns
A deploy template.

`get_deploy_template_by_uuid`(*template_uuid*)

Retrieve a deployment template by UUID.

Parameters
`template_uuid` UUID of the deployment template to retrieve.

Raises
DeployTemplateNotFound if the deploy template does not exist.

Returns
A deploy template.

`get_deploy_template_list`(*limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Retrieve a list of deployment templates.

Parameters

- `limit` Maximum number of deploy templates to return.
- `marker` The last item of the previous page; we return the next result set.

- **sort_key** Attribute by which results should be sorted.
- **sort_dir** Direction in which results should be sorted. (asc, desc)

Returns
A list of deploy templates.

get_deploy_template_list_by_names(*names*)

Return a list of deployment templates with one of a list of names.

Parameters
names List of names to filter by.

Returns
A list of deploy templates.

get_firmware_component(*node_id*, *name*)

Retrieve Firmware Component.

Parameters

- **node_id** The node id.
- **name** name of Firmware component.

Returns
The FirmwareComponent object.

Raises
NodeNotFound if the node is not found.

Raises
FirmwareComponentNotFound if the FirmwareComponent is not found.

get_firmware_component_list(*node_id*)

Retrieve Firmware Components of a given node.

Parameters
node_id The node id.

Returns
A list of FirmwareComponent objects.

Raises
NodeNotFound if the node is not found.

get_inspection_rule_by_id(*inspection_rule_id*)

Retrieve an inspection rule by id.

Parameters
inspection_rule_id id of the rule to retrieve.

Raises
InspectionRuleNotFound if the rule does not exist.

Returns
A rule.

get_inspection_rule_by_uuid(*inspection_rule_uuid*)

Retrieve an inspection rule by UUID.

Parameters
inspection_rule_uuid UUID of the rule to retrieve.

Raises
InspectionRuleNotFound if the rule does not exist.

Returns
A rule.

get_inspection_rule_list(*limit=None*, *marker=None*, *filters=None*, *sort_key=None*, *sort_dir=None*)

Retrieve a list of inspection rules.

Parameters

- **limit** Maximum number of rules to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** Direction in which results should be sorted. (asc, desc)

Returns
A list of inspection rules.

get_node_by_id(*node_id*)

Return a node.

Parameters
node_id The id of a node.

Returns
A node.

get_node_by_instance(*instance*)

Return a node.

Parameters
instance The instance uuid to search for.

Returns
A node.

Raises
InstanceNotFound if the instance is not found.

Raises
InvalidUUID if the instance uuid is invalid.

get_node_by_name(*node_name*)

Return a node.

Parameters
node_name The logical name of a node.

Returns
A node.

get_node_by_port_addresses(*addresses*)

Find a node by any matching port address.

Parameters
addresses list of port addresses (e.g. MACs).

Returns
Node object.

Raises
NodeNotFound if none or several nodes are found.

get_node_by_uuid(*node_uuid*)

Return a node.

Parameters
node_uuid The uuid of a node.

Returns
A node.

get_node_history_by_id(*history_id*)

Return a node history representation.

Parameters
history_id The id of a history record.

Returns
A history.

get_node_history_by_node_id(*node_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

List all the history records for a given node.

Parameters

- **node_id** The integer node ID.
- **limit** Maximum number of history records to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted
- **sort_dir** direction in which results should be sorted (asc, desc)

Returns
A list of histories.

get_node_history_by_uuid(*history_uuid*)

Return a node history representation.

Parameters
history_uuid The uuid of a history record

Returns
A history.

get_node_history_list(*limit=None*, *marker=None*, *sort_key='created_at'*, *sort_dir='asc'*)

Return a list of node history records

Parameters

- **limit** Maximum number of history records to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** direction in which results should be sorted. (asc, desc)

get_node_inventory_by_node_id(*node_id*)

Get the node inventory for a given node.

Parameters

node_id The integer node ID.

Returns

An inventory of a node.

get_node_list(*filters=None*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *fields=None*)

Return a list of nodes.

Parameters

- **filters** Filters to apply. Defaults to None.

  associated
  True | False

  reserved
  True | False

  maintenance
  True | False

  chassis_uuid
  uuid of chassis

  driver
  drivers name

  provision_state
  provision state of node

  provisioned_before
  nodes with provision_updated_at field before this interval in seconds

  shard
  nodes with the given shard

- **limit** Maximum number of nodes to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** direction in which results should be sorted. (asc, desc)
- **fields** Comma separated field list to return, to allow for only specific fields to be returned to have maximum API performance calls where not all columns are needed from the database.

get_node_list_columns(*columns=None*, *filters=None*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Get a node list with specific fields/columns.

Parameters

- **columns** A list of columns to retrieve from the database and populate into the object.
- **filters** The requested database field filters in the form of a dictionary with the applicable key, and filter value.
- **limit** Limit the number of returned nodes, default None.
- **marker** Starting marker to generate a paginated result set for the consumer.
- **sort_key** Sort key to apply to the result set.
- **sort_dir** Sort direction to apply to the result set.

Returns

A list of Node objects based on the data model from a SQLAlchemy result set, which the object layer can use to convert the node into an Node object list.

get_node_tags_by_node_id(*node_id*)

Get node tags based on its id.

Parameters

node_id The id of a node.

Returns

A list of NodeTag objects.

Raises

NodeNotFound if the node is not found.

get_node_traits_by_node_id(*node_id*)

Get node traits based on its id.

Parameters

node_id The id of a node.

Returns

A list of NodeTrait objects.

Raises

NodeNotFound if the node is not found.

get_nodeinfo_list(*columns=None*, *filters=None*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Get specific columns for matching nodes.

Return a list of the specified columns for all nodes that match the specified filters.

Parameters

- **columns** List of column names to return. Defaults to id column when columns == None.
- **filters** Filters to apply. Defaults to None.

associated
True | False

chassis_uuid
uuid of chassis

conductor_group
conductor group name

console_enabled
True | False

description_contains
substring in description

driver
drivers name

fault
current fault type

id
numeric ID

inspection_started_before
nodes with inspection_started_at field before this interval in seconds

instance_uuid
uuid of instance

instance_name
name of instance

lessee
nodes lessee (e.g. project ID)

maintenance
True | False

owner
nodes owner (e.g. project ID)

project
either owner or lessee

reserved
True | False

reserved_by_any_of
[conductor1, conductor2]

resource_class
resource class name

retired
True | False

shard_in
shard (multiple possibilities)

provision_state
provision state of node

provision_state_in
provision state of node (multiple possibilities)

provisioned_before
nodes with provision_updated_at field before this interval in seconds

uuid
uuid of node

uuid_in
uuid of node (multiple possibilities)

with_power_state
True | False

- **limit** Maximum number of nodes to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** direction in which results should be sorted. (asc, desc)

Returns
A list of tuples of the specified columns.

get_offline_conductors(*field='hostname'*)

Get a list conductors that are offline (dead).

Parameters
field A field to return, hostname by default.

Returns
A list of requested fields of offline conductors.

get_online_conductors()

Get a list conductor hostnames that are online and active.

Returns
A list of conductor hostnames.

get_port_by_address(*address*, *owner=None*, *project=None*)

Return a network port representation.

Parameters
address The MAC address of a port.

Returns
A port.

get_port_by_id(*port_id*)

Return a network port representation.

Parameters
port_id The id of a port.

Returns
A port.

get_port_by_name(*port_name*)

Return a network port representation.

Parameters
port_name The name of a port.

Returns
A port.

get_port_by_uuid(*port_uuid*)

Return a network port representation.

Parameters
port_uuid The uuid of a port.

Returns
A port.

get_port_list(*limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *owner=None*, *project=None*, *conductor_groups=None*, *filters=None*)

Return a list of ports.

Parameters

- **limit** Maximum number of ports to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** direction in which results should be sorted. (asc, desc)
- **conductor_groups** A list of conductor groups to filter by, defaults to None
- **filters** Filters to apply, defaults to None

get_portgroup_by_address(*address*, *project=None*)

Return a network portgroup representation.

Parameters

- **address** The MAC address of a portgroup.
- **project** A node owner or lessee to filter by.

Returns
A portgroup.

Raises
PortgroupNotFound

get_portgroup_by_id(*portgroup_id*, *project=None*)

Return a network portgroup representation.

Parameters
portgroup_id The id of a portgroup.

Returns
A portgroup.

Raises
PortgroupNotFound

get_portgroup_by_name(*name*)

Return a network portgroup representation.

Parameters
name The logical name of a portgroup.

Returns
A portgroup.

Raises
PortgroupNotFound

get_portgroup_by_uuid(*portgroup_uuid*)

Return a network portgroup representation.

Parameters
portgroup_uuid The uuid of a portgroup.

Returns
A portgroup.

Raises
PortgroupNotFound

get_portgroup_list(*limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*, *conductor_groups=None*, *filters=None*)

Return a list of portgroups.

Parameters

- **limit** Maximum number of portgroups to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** Direction in which results should be sorted. (asc, desc)
- **project** A node owner or lessee to filter by.
- **conductor_groups** A list of conductor groups to filter by, defaults to None
- **filters** Filters to apply, defaults to None

Returns
A list of portgroups.

get_portgroups_by_node_id(*node_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

List all the portgroups for a given node.

Parameters

- **node_id** The integer node ID.
- **limit** Maximum number of portgroups to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted
- **sort_dir** Direction in which results should be sorted (asc, desc)

- **project** A node owner or lessee to filter by.

Returns
A list of portgroups.

get_portgroups_by_shards(*shards*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Return a list of portgroups contained in the provided shards.

Parameters

- **shards** A list of shards to filter portgroups by.
- **limit** Maximum number of portgroups to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted
- **sort_dir** Direction in which results should be sorted (asc, desc)

Returns
A list of portgroups.

get_ports_by_node_id(*node_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *owner=None*, *project=None*, *filters=None*)

List all the ports for a given node.

Parameters

- **node_id** The integer node ID.
- **limit** Maximum number of ports to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted
- **sort_dir** direction in which results should be sorted (asc, desc)
- **filters** Filters to apply, defaults to None

Returns
A list of ports.

get_ports_by_portgroup_id(*portgroup_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *owner=None*, *project=None*, *filters=None*)

List all the ports for a given portgroup.

Parameters

- **portgroup_id** The integer portgroup ID.
- **limit** Maximum number of ports to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted
- **sort_dir** Direction in which results should be sorted (asc, desc)
- **filters** Filters to apply, defaults to None

Returns
A list of ports.

get_ports_by_shards(*shards*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *filters=None*)

Return a list of ports contained in the provided shards.

Parameters
shard_ids A list of shards to filter ports by.

get_runbook_by_id(*runbook_id*)

Retrieve a runbook by ID.

Parameters
runbook_id ID of the runbook to retrieve.

Raises
RunbookNotFound if the runbook does not exist.

Returns
A runbook.

get_runbook_by_name(*runbook_name*)

Retrieve a runbook by name.

Parameters
runbook_name name of the runbook to retrieve.

Raises
RunbookNotFound if the runbook does not exist.

Returns
A runbook.

get_runbook_by_uuid(*runbook_uuid*)

Retrieve a runbook by UUID.

Parameters
runbook_uuid UUID of the runbook to retrieve.

Raises
RunbookNotFound if the runbook does not exist.

Returns
A runbook.

get_runbook_list(*limit=None*, *marker=None*, *filters=None*, *sort_key=None*, *sort_dir=None*)

Retrieve a list of runbooks.

Parameters

- **limit** Maximum number of runbooks to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** Direction in which results should be sorted. (asc, desc)

Returns
A list of runbooks.

get_runbook_list_by_names(*names*)

Return a list of runbooks with one of a list of names.

Parameters
names List of names to filter by.

Returns
A list of runbooks.

get_shard_list()

Return a list of shards.

Returns
A list of dicts containing the keys name and count.

get_volume_connector_by_id(*db_id*)

Return a volume connector representation.

Parameters
db_id The integer database ID of a volume connector.

Returns
A volume connector with the specified ID.

Raises
VolumeConnectorNotFound If a volume connector with the specified ID is not found.

get_volume_connector_by_uuid(*connector_uuid*)

Return a volume connector representation.

Parameters
connector_uuid The UUID of a connector.

Returns
A volume connector with the specified UUID.

Raises
VolumeConnectorNotFound If a volume connector with the specified UUID is not found.

get_volume_connector_list(*limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

Return a list of volume connectors.

Parameters

- **limit** Maximum number of volume connectors to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** Direction in which results should be sorted. (asc, desc)
- **project** The associated node project to search with.

Returns
a list of `VolumeConnector` objects

Returns
A list of volume connectors.

Raises
InvalidParameterValue If sort_key does not exist.

`get_volume_connectors_by_node_id`(*node_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

List all the volume connectors for a given node.

Parameters

- **node_id** The integer node ID.
- **limit** Maximum number of volume connectors to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted
- **sort_dir** Direction in which results should be sorted (asc, desc)
- **project** The associated node project to search with.

Returns
a list of `VolumeConnector` objects

Returns
A list of volume connectors.

Raises
InvalidParameterValue If sort_key does not exist.

`get_volume_target_by_id`(*db_id*)

Return a volume target representation.

Parameters
db_id The database primary key (integer) ID of a volume target.

Returns
A volume target.

Raises
VolumeTargetNotFound if no volume target with this ID exists.

`get_volume_target_by_uuid`(*uuid*)

Return a volume target representation.

Parameters
uuid The UUID of a volume target.

Returns
A volume target.

Raises
VolumeTargetNotFound if no volume target with this UUID exists.

`get_volume_target_list`(*limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

Return a list of volume targets.

Parameters

- **limit** Maximum number of volume targets to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** direction in which results should be sorted. (asc, desc)
- **project** The associated node project to search with.

Returns
a list of `VolumeConnector` objects

Returns
A list of volume targets.

Raises
InvalidParameterValue if sort_key does not exist.

get_volume_targets_by_node_id(*node_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

List all the volume targets for a given node.

Parameters

- **node_id** The integer node ID.
- **limit** Maximum number of volume targets to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted
- **sort_dir** direction in which results should be sorted (asc, desc)
- **project** The associated node project to search with.

Returns
a list of `VolumeConnector` objects

Returns
A list of volume targets.

Raises
InvalidParameterValue if sort_key does not exist.

get_volume_targets_by_volume_id(*volume_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

List all the volume targets for a given volume id.

Parameters

- **volume_id** The UUID of the volume.
- **limit** Maximum number of volume targets to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted
- **sort_dir** direction in which results should be sorted (asc, desc)

Returns
A list of volume targets.

Raises
InvalidParameterValue if sort_key does not exist.

`list_conductor_hardware_interfaces`(*conductor_id*)

List all registered hardware interfaces for a conductor.

Parameters
conductor_id Database ID of conductor.

Returns
List of `ConductorHardwareInterfaces` objects.

`list_hardware_type_interfaces`(*hardware_types*)

List registered hardware interfaces for given hardware types.

This is restricted to only active conductors. :param hardware_types: list of hardware types to filter by. :returns: list of `ConductorHardwareInterfaces` objects.

`migrate_to_builtin_inspection`(*context*, *max_count*)

Handle the migration from inspector to agent inspection.

Parameters

- **context** the admin context
- **max_count** The maximum number of objects to migrate. Must be >= 0. If zero, all the objects will be migrated.

Returns
A 2-tuple, 1. the total number of objects that need to be migrated (at the beginning of this call) and 2. the number of migrated objects.

`node_tag_exists`(*node_id*, *tag*)

Check if the specified tag exist on the node.

Parameters

- **node_id** The id of a node.
- **tag** A tag string.

Returns
True if the tag exists otherwise False.

Raises
NodeNotFound if the node is not found.

`node_trait_exists`(*node_id*, *trait*)

Check if the specified trait exists on the node.

Parameters

- **node_id** The id of a node.
- **trait** A trait string.

Returns
True if the trait exists otherwise False.

Raises
NodeNotFound if the node is not found.

query_node_history_records_for_purge(*conductor_id*)

Utility method to identify nodes to clean history records for.

Parameters
conductor_id Id value for the conductor to perform this query on behalf of.

Returns
A dictionary with key values of node database ID values and a list of values associated with the node.

register_conductor(*values*, *update_existing=False*)

Register an active conductor with the cluster.

Parameters

- **values** A dict of values which must contain the following:

```
{
 'hostname': the unique hostname which identifies
             this Conductor service.
 'drivers': a list of supported drivers.
 'version': the version of the object.Conductor
}
```

- **update_existing** When false, registration will raise an exception when a conflicting online record is found. When true, will overwrite the existing record. Default: False.

Returns
A conductor.

Raises
ConductorAlreadyRegistered

register_conductor_hardware_interfaces(*conductor_id*, *interfaces*)

Registers hardware interfaces for a conductor.

Parameters

- **conductor_id** Database ID of conductor to register for.
- **hardware_type** Name of hardware type for the interfaces.
- **interface_type** Type of interfaces, e.g. deploy or boot.
- **interfaces** List of interface names to register.
- **default_interface** String, the default interface for this hardware type and interface type.

Raises
ConductorHardwareInterfacesAlreadyRegistered if at least one of the interfaces in the combination of all parameters is already registered.

release_node(*tag*, *node_id*)

Release the reservation on a node.

Parameters

- **tag** A string uniquely identifying the reservation holder.
- **node_id** A node id or uuid.

Raises
NodeNotFound if the node is not found.

Raises
NodeLocked if the node is reserved by another host.

Raises
NodeNotLocked if the node was found to not have a reservation at all.

reserve_node(*tag*, *node_id*)

Reserve a node.

To prevent other ManagerServices from manipulating the given Node while a Task is performed, mark it reserved by this host.

Parameters

- **tag** A string uniquely identifying the reservation holder.
- **node_id** A node id or uuid.

Returns
A Node object.

Raises
NodeNotFound if the node is not found.

Raises
NodeLocked if the node is already reserved.

set_node_tags(*node_id*, *tags*)

Replace all of the node tags with specified list of tags.

This ignores duplicate tags in the specified list.

Parameters

- **node_id** The id of a node.
- **tags** List of tags.

Returns
A list of NodeTag objects.

Raises
NodeNotFound if the node is not found.

set_node_traits(*node_id*, *traits*, *version*)

Replace all of the node traits with specified list of traits.

This ignores duplicate traits in the specified list.

Parameters

- **node_id** The id of a node.
- **traits** List of traits.

- **version** the version of the object.Trait.

Returns
A list of NodeTrait objects.

Raises
InvalidParameterValue if setting the traits would exceed the per-node traits limit.

Raises
NodeNotFound if the node is not found.

take_over_allocation(*allocation_id*, *old_conductor_id*, *new_conductor_id*)

Do a take over for an allocation.

The allocation is only updated if the old conductor matches the provided value, thus guarding against races.

Parameters

- **allocation_id** Allocation ID
- **old_conductor_id** The conductor ID we expect to be the current `conductor_affinity` of the allocation.
- **new_conductor_id** The conductor ID of the new `conductor_affinity`.

Returns
True if the take over was successful, False otherwise.

Raises
AllocationNotFound

touch_conductor(*hostname*, *online=True*)

Mark a conductor as active by updating its updated_at property.

Calling periodically with `online=False` will result in the conductor appearing unregistered, but recently enough to prevent other conductors failing orphan nodes. This improves the behaviour of graceful and drain shutdown.

Parameters

- **hostname** The hostname of this conductor service.
- **online** Whether the conductor is online.

Raises
ConductorNotFound

touch_node_provisioning(*node_id*)

Mark the nodes provisioning as running.

Mark the nodes provisioning as running by updating its provision_updated_at property.

Parameters
node_id The id of a node.

Raises
NodeNotFound

unregister_conductor(*hostname*)

Remove this conductor from the service registry immediately.

Parameters
hostname The hostname of this conductor service.

Raises
ConductorNotFound

unregister_conductor_hardware_interfaces(*conductor_id*)

Unregisters all hardware interfaces for a conductor.

Parameters
conductor_id Database ID of conductor to unregister for.

unset_node_tags(*node_id*)

Remove all tags of the node.

Parameters
node_id The id of a node.

Raises
NodeNotFound if the node is not found.

unset_node_traits(*node_id*)

Remove all traits of the node.

Parameters
node_id The id of a node.

Raises
NodeNotFound if the node is not found.

update_allocation(*allocation_id*, *values*, *update_node=True*)

Update properties of an allocation.

Parameters

- **allocation_id** Allocation ID
- **values** Dict of values to update.
- **update_node** If True and node_id is updated, update the node with instance_uuid and traits from the allocation

Returns
An allocation.

Raises
AllocationNotFound

Raises
AllocationDuplicateName

Raises
InstanceAssociated

Raises
NodeAssociated

update_bios_setting_list(*node_id*, *settings*, *version*)

Update a list of BIOSSetting records.

Parameters

- **node_id** The node id.
- **settings** A list of BIOS Settings to be updated.

```
[
  {
   'name': String,
   'value': String,
   additional settings from BIOS registry
  },
  {
   'name': String,
   'value': String,
   additional settings from BIOS registry
  },
  ...
]
```

- **version** the version of the object.BIOSSetting.

Returns

A list of BIOSSetting objects.

Raises

NodeNotFound if the node is not found.

Raises

BIOSSettingNotFound if any of the settings is not found.

update_chassis(*chassis_id*, *values*)

Update properties of an chassis.

Parameters

- **chassis_id** The id or the uuid of a chassis.
- **values** Dict of values to update.

Returns

A chassis.

update_deploy_template(*template_id*, *values*)

Update a deployment template.

Parameters

- **template_id** ID of the deployment template to update.
- **values** A dict describing the deployment template. For example:

```
{
 'uuid': uuidutils.generate_uuid(),
```

(continues on next page)

(continued from previous page)

```
 'name': 'CUSTOM_DT1',
}
```

Raises
DeployTemplateDuplicateName if a deploy template with the same name exists.

Raises
DeployTemplateNotFound if the deploy template does not exist.

Returns
A deploy template.

update_firmware_component(*node_id*, *component*, *values*)

Update a FirmwareComponent record.

Parameters

- **node_id** The node id.
- **component** The component of the node to update.
- **values** A dictionary with the new information about the FirmwareComponent.

```
{
  'current_version': String,
  'last_version_flashed': String
}
```

Returns
A FirmwareComponent object.

Raises
FirmwareComponentNotFound the component is not found.

update_inspection_rule(*rule_uuid*, *values*)

Update an existing inspection rule.

Parameters

- **values** Dict of values to update with.
- **rule_id** The rule id.

update_node(*node_id*, *values*)

Update properties of a node.

Parameters

- **node_id** The id or uuid of a node.
- **values** Dict of values to update. May be a partial list, eg. when setting the properties for a driver. For example:

```
{
 'driver_info':
```

(continues on next page)

(continued from previous page)

```
    {
     'my-field-1': val1,
     'my-field-2': val2,
    }
}
```

Returns
A node.

Raises
NodeAssociated

Raises
NodeNotFound

update_port(*port_id*, *values*)

Update properties of an port.

Parameters

- **port_id** The id or MAC of a port.
- **values** Dict of values to update.

Returns
A port.

update_portgroup(*portgroup_id*, *values*)

Update properties of a portgroup.

Parameters

- **portgroup_id** The UUID or MAC of a portgroup.
- **values** Dict of values to update. May contain the following keys: uuid name node_id address extra created_at updated_at

Returns
A portgroup.

Raises
InvalidParameterValue

Raises
PortgroupNotFound

Raises
PortgroupDuplicateName

Raises
PortgroupMACAlreadyExists

update_runbook(*runbook_id*, *values*)

Update a runbook.

Parameters

- **runbook_id** ID of the runbook to update.

- **values** A dict describing the runbook. For example:

```
{
 'uuid': uuidutils.generate_uuid(),
 'name': 'CUSTOM_DT1',
}
```

Raises
RunbookDuplicateName if a runbook with the same name exists.

Raises
RunbookNotFound if the runbook does not exist.

Returns
A runbook.

update_to_latest_versions(*context*, *max_count*)

Updates objects to their latest known versions.

This scans all the tables and for objects that are not in their latest version, updates them to that version.

Parameters

- **context** the admin context
- **max_count** The maximum number of objects to migrate. Must be >= 0. If zero, all the objects will be migrated.

Returns
A 2-tuple, 1. the total number of objects that need to be migrated (at the beginning of this call) and 2. the number of migrated objects.

update_volume_connector(*ident*, *connector_info*)

Update properties of a volume connector.

Parameters

- **ident** The UUID or integer ID of a volume connector.
- **connector_info** Dictionary containing the information about connector to update.

Returns
A volume connector.

Raises
VolumeConnectorTypeAndIdAlreadyExists If another connector already exists with a matching type and connector_id field.

Raises
VolumeConnectorNotFound If a volume connector with the specified ident does not exist.

Raises
InvalidParameterValue When a UUID is included in connector_info.

update_volume_target(*ident*, *target_info*)

Update information for a volume target.

Parameters

- **ident** The UUID or integer ID of a volume target.
- **target_info** Dictionary containing the information about volume target to update.

Returns
A volume target.

Raises
InvalidParameterValue if a UUID is included in target_info.

Raises
VolumeTargetBootIndexAlreadyExists if a volume target already exists with the same boot index and node ID.

Raises
VolumeTargetNotFound if no volume target with this ident exists.

ironic.db.sqlalchemy.api.**add_allocation_filter_by_conductor**(*query*, *value*)

ironic.db.sqlalchemy.api.**add_allocation_filter_by_node**(*query*, *value*)

ironic.db.sqlalchemy.api.**add_identity_filter**(*query*, *value*)

Adds an identity filter to a query.

Filters results by ID, if supplied value is a valid integer. Otherwise attempts to filter results by UUID.

Parameters

- **query** Initial query to add filter to.
- **value** Value for filtering results by.

Returns
Modified query.

ironic.db.sqlalchemy.api.**add_identity_where**(*op*, *model*, *value*)

Adds an identity filter to operation for where method.

Filters results by ID, if supplied value is a valid integer. Otherwise attempts to filter results by UUID.

Parameters

- **op** Initial operation to add filter to. i.e. a update or delete statement.
- **model** The SQLAlchemy model to apply.
- **value** Value for filtering results by.

Returns
Modified query.

ironic.db.sqlalchemy.api.**add_node_filter_by_chassis**(*query*, *value*)

ironic.db.sqlalchemy.api.**add_port_filter**(*query*, *value*)

Adds a port-specific filter to a query.

Filters results by address, if supplied value is a valid MAC address. Otherwise attempts to filter results by identity.

Parameters

- **query** Initial query to add filter to.
- **value** Value for filtering results by.

Returns

Modified query.

ironic.db.sqlalchemy.api.**add_port_filter_by_node**(*query*, *value*)

ironic.db.sqlalchemy.api.**add_port_filter_by_node_conductor_groups**(*query*, *conductor_groups*)

ironic.db.sqlalchemy.api.**add_port_filter_by_node_owner**(*query*, *value*)

ironic.db.sqlalchemy.api.**add_port_filter_by_node_project**(*query*, *value*)

ironic.db.sqlalchemy.api.**add_port_filter_by_portgroup**(*query*, *value*)

ironic.db.sqlalchemy.api.**add_port_filter_description_contains**(*query*, *filters*)

ironic.db.sqlalchemy.api.**add_portgroup_filter**(*query*, *value*)

Adds a portgroup-specific filter to a query.

Filters results by address, if supplied value is a valid MAC address. Otherwise attempts to filter results by identity.

Parameters

- **query** Initial query to add filter to.
- **value** Value for filtering results by.

Returns

Modified query.

ironic.db.sqlalchemy.api.**add_portgroup_filter_by_node**(*query*, *value*)

ironic.db.sqlalchemy.api.**add_portgroup_filter_by_node_conductor_groups**(*query*, *conductor_groups*)

ironic.db.sqlalchemy.api.**add_portgroup_filter_by_node_project**(*query*, *value*)

ironic.db.sqlalchemy.api.**add_volume_conn_filter_by_node_project**(*query*, *value*)

ironic.db.sqlalchemy.api.**add_volume_target_filter_by_node_project**(*query*, *value*)

ironic.db.sqlalchemy.api.**get_backend**()

The backend is this module itself.

ironic.db.sqlalchemy.api.**model_query**(*model*, **args*, ***kwargs*)

Query helper for simpler session usage.

WARNING: DO NOT USE, unless you know exactly what your doing AND are okay with a transaction possibly hanging.

Parameters
session if present, the session to use

ironic.db.sqlalchemy.api.**wrap_sqlite_retry**(*f*)

ironic.db.sqlalchemy.migration module

ironic.db.sqlalchemy.migration.**create_schema**(*config=None*, *engine=None*)

Create database schema from models description.

Can be used for initial installation instead of upgrade(head).

ironic.db.sqlalchemy.migration.**downgrade**(*revision*, *config=None*)

Used for downgrading database.

Parameters
version (*string*) Desired database version

ironic.db.sqlalchemy.migration.**revision**(*message=None*, *autogenerate=False*, *config=None*)

Creates template for migration.

Parameters

- **message** (*string*) Text that will be used for migration title
- **autogenerate** (*bool*) If True - generates diff based on current database state

ironic.db.sqlalchemy.migration.**stamp**(*revision*, *config=None*)

Stamps database with provided revision.

Dont run any migrations.

Parameters
revision (*string*) Should match one from repository or head - to stamp database with most recent revision

ironic.db.sqlalchemy.migration.**upgrade**(*revision*, *config=None*)

Used for upgrading database.

Parameters
version (*string*) Desired database version

ironic.db.sqlalchemy.migration.**version**(*config=None*, *engine=None*)

Current database version.

Returns
Database version

Return type
string

ironic.db.sqlalchemy.models module

SQLAlchemy models for baremetal data.

class ironic.db.sqlalchemy.models.**Allocation**(***kwargs*)

Bases: Base

Represents an allocation of a node for deployment.

candidate_nodes

conductor_affinity

created_at

extra

id

last_error

name

node_id

owner

resource_class

state

traits

updated_at

uuid

version

class ironic.db.sqlalchemy.models.**BIOSSetting**(***kwargs*)

Bases: Base

Represents a bios setting of a bare metal node.

allowable_values

attribute_type

created_at

lower_bound

max_length

min_length

name

node_id

read_only

reset_required

unique

updated_at

upper_bound

value

version

class ironic.db.sqlalchemy.models.**Chassis**(***kwargs*)

Bases: Base

Represents a hardware chassis.

created_at

description

extra

id

updated_at

uuid

version

class ironic.db.sqlalchemy.models.**Conductor**(***kwargs*)

Bases: Base

Represents a conductor service entry.

conductor_group

created_at

drivers

hostname

id

online

updated_at

version

class ironic.db.sqlalchemy.models.**ConductorHardwareInterfaces**(***kwargs*)

Bases: Base

Internal table used to track what is loaded on each conductor.

conductor_id

created_at

default

hardware_type

id

interface_name

interface_type

updated_at

version

class ironic.db.sqlalchemy.models.**DeployTemplate**(***kwargs*)

Bases: Base

Represents a deployment template.

created_at

extra

id

name

steps: Mapped[List[*DeployTemplateStep*]]

updated_at

uuid

version

class ironic.db.sqlalchemy.models.**DeployTemplateStep**(***kwargs*)

Bases: Base

Represents a deployment step in a deployment template.

args

created_at

deploy_template

deploy_template_id

id

interface

priority

step

updated_at

version

class ironic.db.sqlalchemy.models.**FirmwareComponent**(***kwargs*)

Bases: Base

Represents the firmware information of a bare metal node.

component

created_at

current_version

id

initial_version

last_version_flashed

node_id

updated_at

version

class ironic.db.sqlalchemy.models.**InspectionRule**(***kwargs*)

Bases: Base

actions

conditions

created_at

description

id

phase

priority

scope

sensitive

updated_at

uuid

version

class ironic.db.sqlalchemy.models.**IronicBase**

Bases: TimestampMixin, ModelBase

as_dict()

metadata = None

version = Column(None, String(length=15), table=None)

class ironic.db.sqlalchemy.models.**Node**(***kwargs*)

Bases: *NodeBase*

Represents a bare metal node.

allocation_id

automated_clean

bios_interface

boot_interface

boot_mode

chassis_id

clean_step

conductor_affinity

conductor_group

console_enabled

console_interface

created_at

deploy_interface

deploy_step

description

disable_power_off

driver

driver_info

driver_internal_info

extra

fault

firmware_interface

health

id

inspect_interface

inspection_finished_at

inspection_started_at

instance_info

instance_name

instance_uuid

last_error

lessee

maintenance

maintenance_reason

management_interface

name

network_data

network_interface

owner

parent_node

power_interface

power_state

properties

protected

protected_reason

provision_state

provision_updated_at

raid_config

raid_interface

rescue_interface

reservation

resource_class

retired

retired_reason

secure_boot

service_step

shard

storage_interface

tags: Mapped[List[*NodeTag*]]

target_power_state

target_provision_state

target_raid_config

traits: Mapped[List[*NodeTrait*]]

updated_at

uuid

vendor_interface

version

class ironic.db.sqlalchemy.models.**NodeBase**(***kwargs*)

Bases: Base

Represents a base bare metal node.

allocation_id

automated_clean

bios_interface

boot_interface

boot_mode

chassis_id

clean_step

conductor_affinity

conductor_group

console_enabled

console_interface

created_at

deploy_interface

deploy_step

description

disable_power_off

driver

driver_info

driver_internal_info

extra

fault

firmware_interface

health

id

inspect_interface

inspection_finished_at

inspection_started_at

instance_info

instance_name

instance_uuid

last_error

lessee

maintenance

maintenance_reason

management_interface

name

network_data

network_interface

owner

parent_node

power_interface

power_state

properties

protected

protected_reason

provision_state

provision_updated_at

raid_config

raid_interface

rescue_interface

reservation

resource_class

retired

retired_reason

secure_boot

service_step

shard

storage_interface

target_power_state

target_provision_state

target_raid_config

updated_at

uuid

vendor_interface

version

class ironic.db.sqlalchemy.models.**NodeHistory**(***kwargs*)

Bases: Base

Represents a history event of a bare metal node.

conductor

created_at

event

event_type

id

node_id

severity

updated_at

user

uuid

version

class ironic.db.sqlalchemy.models.**NodeInventory**(***kwargs*)

Bases: Base

Represents an inventory of a baremetal node.

created_at

id

inventory_data

node_id

plugin_data

updated_at

version

class ironic.db.sqlalchemy.models.**NodeTag**(***kwargs*)

Bases: Base

Represents a tag of a bare metal node.

created_at

node

node_id

tag

updated_at

version

class ironic.db.sqlalchemy.models.**NodeTrait**(***kwargs*)

Bases: Base

Represents a trait of a bare metal node.

created_at

node

node_id

trait

updated_at

version

class ironic.db.sqlalchemy.models.**Port**(***kwargs*)

Bases: Base

Represents a network port of a bare metal node.

address

available_for_dynamic_portgroup

category

created_at

description

extra

id

internal_info

is_smartnic

local_link_connection

name

node_id

node_uuid = ColumnAssociationProxyInstance(AssociationProxy('_node_uuid', 'uuid'))

physical_network

portgroup_id

pxe_enabled

updated_at

uuid

vendor

version

class ironic.db.sqlalchemy.models.**Portgroup**(***kwargs*)

Bases: Base

Represents a group of network ports of a bare metal node.

address

category

created_at

dynamic_portgroup

extra

id

internal_info

mode

name

node_id

node_uuid = ColumnAssociationProxyInstance(AssociationProxy('_node_uuid', 'uuid'))

physical_network

properties

standalone_ports_supported

updated_at

uuid

version

class ironic.db.sqlalchemy.models.**Runbook**(***kwargs*)

Bases: Base

Represents a runbook.

created_at

disable_ramdisk

extra

id

name

owner

public

steps: Mapped[List[*RunbookStep*]]

updated_at

uuid

version

class ironic.db.sqlalchemy.models.**RunbookStep**(***kwargs*)

Bases: Base

Represents a deployment step in a runbook.

args

created_at

id

interface

order

runbook

runbook_id

step

updated_at

version

class ironic.db.sqlalchemy.models.**VolumeConnector**(***kwargs*)

Bases: Base

Represents a volume connector of a bare metal node.

connector_id

created_at

extra

id

node_id

type

updated_at

uuid

version

class ironic.db.sqlalchemy.models.**VolumeTarget**(***kwargs*)

Bases: Base

Represents a volume target of a bare metal node.

boot_index

created_at

extra

id

node_id

properties

updated_at

uuid

version

volume_id

volume_type

ironic.db.sqlalchemy.models.**get_class**(*model_name*)

Returns the model class with the specified name.

Parameters
model_name the name of the class

Returns
the class with the specified name

Raises
Exception if there is no class associated with the name

ironic.db.sqlalchemy.models.**table_args**()

Module contents

Submodules

ironic.db.api module

Base classes for storage engines

class ironic.db.api.**Connection**

Bases: object

Base class for storage system connections.

abstract add_node_tag(*node_id*, *tag*)

Add tag to the node.

If the node_id and tag pair already exists, this should still succeed.

Parameters

- **node_id** The id of a node.
- **tag** A tag string.

Returns
the NodeTag object.

Raises
NodeNotFound if the node is not found.

abstract add_node_trait(*node_id*, *trait*, *version*)

Add trait to the node.

If the node_id and trait pair already exists, this should still succeed.

Parameters

- **node_id** The id of a node.
- **trait** A trait string.
- **version** the version of the object.Trait.

Returns
the NodeTrait object.

Raises
InvalidParameterValue if adding the trait would exceed the per-node traits limit.

Raises
NodeNotFound if the node is not found.

abstract bulk_delete_node_history_records(*node_id*, *limit*)

Utility method to bulk delete node history entries.

Parameters
entries A list of node history entry ids to be queried for deletion.

abstract check_node_list(*idents*)

Check a list of node identities and map it to UUIDs.

This call takes a list of node names and/or UUIDs and tries to convert them to UUIDs. It fails early if any identities cannot possible be used as names or UUIDs.

Parameters
idents List of identities.

Returns
A mapping from requests identities to node UUIDs.

Raises
NodeNotFound if some identities were not found or cannot be valid names or UUIDs.

abstract check_versions(*ignore_models=()*)

Checks the whole database for incompatible objects.

This scans all the tables in search of objects that are not supported; i.e., those that are not specified in *ironic.common.release_mappings.RELEASE_MAPPING*.

Parameters
ignore_models List of model names to skip.

Returns
A Boolean. True if all the objects have supported versions; False otherwise.

abstract count_nodes_in_provision_state(*state*, *conductor_group=None*)

Count the number of nodes in given provision state.

Parameters

- **state** A provision_state value to match for the count operation. This can be a single provision state value or a list of values.
- **conductor_group** Optional conductor group to filter by. If provided, only nodes in this conductor group will be counted.

abstract create_allocation(*values*)

Create a new allocation.

Parameters
values Dict of values to create an allocation with

Returns
An allocation

Raises
AllocationDuplicateName

Raises
AllocationAlreadyExists

abstract create_bios_setting_list(*node_id*, *settings*, *version*)

Create a list of BIOSSetting records for a given node.

Parameters

- **node_id** The node id.
- **settings** A list of BIOS Settings to be created.

```
[
  {
   'name': String,
   'value': String,
   additional settings from BIOS registry
  },
  {
   'name': String,
   'value': String,
   additional settings from BIOS registry
  },
  ...
]
```

- **version** the version of the object.BIOSSetting.

Returns
A list of BIOSSetting object.

Raises
NodeNotFound if the node is not found.

Raises
BIOSSettingAlreadyExists if any of the setting records already exists.

abstract create_chassis(*values*)

Create a new chassis.

Parameters
values Dict of values.

abstract create_deploy_template(*values*)

Create a deployment template.

Parameters

values A dict describing the deployment template. For example:

```
{
 'uuid': uuidutils.generate_uuid(),
 'name': 'CUSTOM_DT1',
}
```

Raises

DeployTemplateDuplicateName if a deploy template with the same name exists.

Raises

DeployTemplateAlreadyExists if a deploy template with the same UUID exists.

Returns

A deploy template.

abstract create_firmware_component(*values*)

Create a FirmwareComponent record for a given node.

Parameters

values a dictionary with the necessary information to create a FirmwareComponent.

```
{
  'component': String,
  'initial_version': String,
  'current_version': String,
  'last_version_flashed': String
}
```

Returns

A FirmwareComponent object.

Raises

FirmwareComponentAlreadyExists if any of the component records already exists.

abstract create_inspection_rule(*values*)

Create an inspection rule.

Parameters

values A dict describing the rule.

Raises

InspectionRuleAlreadyExists if an inspection rule with the same UUID exists.

Returns

A rule.

abstract create_node(*values*)

Create a new node.

Parameters

values A dict containing several items used to identify and track the node,

and several dicts which are passed into the Drivers when managing this node. For example:

```
{
 'uuid': uuidutils.generate_uuid(),
 'instance_uuid': None,
 'power_state': states.POWER_OFF,
 'provision_state': states.AVAILABLE,
 'driver': 'ipmi',
 'driver_info': { ... },
 'properties': { ... },
 'extra': { ... },
}
```

Raises
InvalidParameterValue if values contains tags or traits.

Returns
A node.

abstract create_node_history(*values*)

Create a new history record.

Parameters
values Dict of values.

abstract create_node_inventory(*values*)

Create a new inventory record.

Parameters
values Dict of values.

abstract create_port(*values*)

Create a new port.

Parameters
values Dict of values.

abstract create_portgroup(*values*)

Create a new portgroup.

Parameters
values Dict of values with the following keys: id uuid name node_id address extra created_at updated_at

Returns
A portgroup

Raises
PortgroupDuplicateName

Raises
PortgroupMACAlreadyExists

Raises
PortgroupAlreadyExists

abstract create_runbook(*values*)

Create a runbook.

Parameters

values A dict describing the runbook. For example:

```
{
 'uuid': uuidutils.generate_uuid(),
 'name': 'CUSTOM_DT1',
}
```

Raises

RunbookDuplicateName if a runbook with the same name exists.

Raises

RunbookAlreadyExists if a runbook with the same UUID exists.

Returns

A runbook.

abstract create_volume_connector(*connector_info*)

Create a new volume connector.

Parameters

connector_info Dictionary containing information about the connector. Example:

```
{
    'uuid': '000000-..',
    'type': 'wwnn',
    'connector_id': '00:01:02:03:04:05:06',
    'node_id': 2
}
```

Returns

A volume connector.

Raises

VolumeConnectorTypeAndIdAlreadyExists If a connector already exists with a matching type and connector_id.

Raises

VolumeConnectorAlreadyExists If a volume connector with the same UUID already exists.

abstract create_volume_target(*target_info*)

Create a new volume target.

Parameters

target_info Dictionary containing the information about the volume target. Example:

```
{
    'uuid': '000000-..',
    'node_id': 2,
```

(continues on next page)

(continued from previous page)

```
    'boot_index': 0,
    'volume_id': '12345678-...'
    'volume_type': 'some type',
}
```

Returns
A volume target.

Raises
VolumeTargetBootIndexAlreadyExists if a volume target already exists with the same boot index and node ID.

Raises
VolumeTargetAlreadyExists if a volume target with the same UUID exists.

abstract delete_bios_setting_list(*node_id*, *names*)

Delete a list of BIOS settings.

Parameters

- **node_id** The node id.
- **names** List of BIOS setting names to be deleted.

Raises
NodeNotFound if the node is not found.

Raises
BIOSSettingNotFound if any of BIOS setting name is not found.

abstract delete_conductor(*hostname*)

Delete a conductor from the database.

Parameters
hostname The hostname of this conductor service.

Raises
ConductorNotFound if the conductor doesnt exist.

abstract delete_node_tag(*node_id*, *tag*)

Delete specified tag from the node.

Parameters

- **node_id** The id of a node.
- **tag** A tag string.

Raises
NodeNotFound if the node is not found.

Raises
NodeTagNotFound if the tag is not found.

abstract delete_node_trait(*node_id*, *trait*)

Delete specified trait from the node.

Parameters

- **node_id** The id of a node.
- **trait** A trait string.

Raises
NodeNotFound if the node is not found.

Raises
NodeTraitNotFound if the trait is not found.

abstract destroy_allocation(*allocation_id*)

Destroy an allocation.

Parameters
allocation_id Allocation ID

Raises
AllocationNotFound

abstract destroy_chassis(*chassis_id*)

Destroy a chassis.

Parameters
chassis_id The id or the uuid of a chassis.

abstract destroy_deploy_template(*template_id*)

Destroy a deployment template.

Parameters
template_id ID of the deployment template to destroy.

Raises
DeployTemplateNotFound if the deploy template does not exist.

abstract destroy_inspection_rule(*inspection_rule_id*)

Destroy an inspection rule.

Parameters
inspection_rule_id ID of the inspection_rule to destroy.

Raises
inspection_ruleNotFound if the inspection_rule does not exist.

abstract destroy_node(*node_id*)

Destroy a node and its associated resources.

Destroy a node, including any associated ports, port groups, tags, traits, volume connectors, and volume targets.

Parameters
node_id The ID or UUID of a node.

abstract destroy_node_history_by_uuid(*history_uuid*)

Destroy a history record.

Parameters
history_uuid The uuid of a history record

abstract destroy_node_inventory_by_node_id(*node_id*)

Destroy a inventory record.

Parameters
node_id The node id value for records to be removed.

abstract destroy_port(*port_id*)

Destroy an port.

Parameters
port_id The id or MAC of a port.

abstract destroy_portgroup(*portgroup_id*)

Destroy a portgroup.

Parameters
portgroup_id The UUID or MAC of a portgroup.

Raises
PortgroupNotEmpty

Raises
PortgroupNotFound

abstract destroy_runbook(*runbook_id*)

Destroy a runbook.

Parameters
runbook_id ID of the runbook to destroy.

Raises
RunbookNotFound if the runbook does not exist.

abstract destroy_volume_connector(*ident*)

Destroy a volume connector.

Parameters
ident The UUID or integer ID of a volume connector.

Raises
VolumeConnectorNotFound If a volume connector with the specified ident does not exist.

abstract destroy_volume_target(*ident*)

Destroy a volume target.

Parameters
ident The UUID or integer ID of a volume target.

Raises
VolumeTargetNotFound if a volume target with the specified ident does not exist.

abstract get_active_hardware_type_dict(*use_groups=False*)

Retrieve hardware types for the registered and active conductors.

Parameters
use_groups Whether to factor conductor_group into the keys.

Returns

A dict which maps hardware type names to the set of hosts which support them. For example:

```
{hardware-type-a: set([host1, host2]),
 hardware-type-b: set([host2, host3])}
```

abstract get_allocation_by_id(*allocation_id*)

Return an allocation representation.

Parameters
allocation_id The id of an allocation.

Returns
An allocation.

Raises
AllocationNotFound

abstract get_allocation_by_name(*name*)

Return an allocation representation.

Parameters
name The logical name of an allocation.

Returns
An allocation.

Raises
AllocationNotFound

abstract get_allocation_by_uuid(*allocation_uuid*)

Return an allocation representation.

Parameters
allocation_uuid The uuid of an allocation.

Returns
An allocation.

Raises
AllocationNotFound

abstract get_allocation_list(*filters=None*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Return a list of allocations.

Parameters

- **filters** Filters to apply. Defaults to None.

  node_uuid
  uuid of node

  state
  allocation state

  resource_class
  requested resource class

- **limit** Maximum number of allocations to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** Direction in which results should be sorted. (asc, desc)

Returns
A list of allocations.

abstract get_bios_setting(*node_id*, *name*)

Retrieve BIOS setting value.

Parameters

- **node_id** The node id.
- **name** String containing name of BIOS setting to be retrieved.

Returns
The BIOSSetting object.

Raises
NodeNotFound if the node is not found.

Raises
BIOSSettingNotFound if the BIOS setting is not found.

abstract get_bios_setting_list(*node_id*)

Retrieve BIOS settings of a given node.

Parameters
node_id The node id.

Returns
A list of BIOSSetting objects.

Raises
NodeNotFound if the node is not found.

abstract get_chassis_by_id(*chassis_id*)

Return a chassis representation.

Parameters
chassis_id The id of a chassis.

Returns
A chassis.

abstract get_chassis_by_uuid(*chassis_uuid*)

Return a chassis representation.

Parameters
chassis_uuid The uuid of a chassis.

Returns
A chassis.

abstract get_chassis_list(*limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Return a list of chassis.

Parameters

- **limit** Maximum number of chassis to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** direction in which results should be sorted. (asc, desc)

abstract get_child_node_ids_by_parent_uuid(*node_uuid*, *exclude_dedicated_power=False*)

Retrieve a list of child node IDs for a given parent UUID.

This is an internal method, intended for use with power management logic to facilitate power actions upon nodes, where we obtain a list of nodes which requires additional actions, and return *only* the required node IDs in order to launch new tasks for power management activities.

Parameters

- **node_uuid** The uuid of the parent node, in order to directly match the parent_node field.
- **exclude_dedicated_power** Boolean, False, if the list should include child nodes with their own power supplies.

Returns

A list of tuples.

abstract get_conductor(*hostname*, *online=True*)

Retrieve a conductors service record from the database.

Parameters

- **hostname** The hostname of the conductor service.
- **online** Specify the filter value on the *online* field when querying conductors. The `online` field is ignored if this value is set to None.

Returns

A conductor.

Raises

ConductorNotFound if the conductor with given hostname does not exist or doesnt meet the specified online expectation.

abstract get_conductor_list(*limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Return a list of conductors.

Parameters

- **limit** Maximum number of conductors to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** direction in which results should be sorted. (asc, desc)

abstract get_deploy_template_by_id(*template_id*)

Retrieve a deployment template by ID.

Parameters
template_id ID of the deployment template to retrieve.

Raises
DeployTemplateNotFound if the deploy template does not exist.

Returns
A deploy template.

abstract get_deploy_template_by_name(*template_name*)

Retrieve a deployment template by name.

Parameters
template_name name of the deployment template to retrieve.

Raises
DeployTemplateNotFound if the deploy template does not exist.

Returns
A deploy template.

abstract get_deploy_template_by_uuid(*template_uuid*)

Retrieve a deployment template by UUID.

Parameters
template_uuid UUID of the deployment template to retrieve.

Raises
DeployTemplateNotFound if the deploy template does not exist.

Returns
A deploy template.

abstract get_deploy_template_list(*limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Retrieve a list of deployment templates.

Parameters

- **limit** Maximum number of deploy templates to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** Direction in which results should be sorted. (asc, desc)

Returns
A list of deploy templates.

abstract get_deploy_template_list_by_names(*names*)

Return a list of deployment templates with one of a list of names.

Parameters
names List of names to filter by.

Returns
A list of deploy templates.

abstract get_firmware_component(*node_id*, *name*)

Retrieve Firmware Component.

Parameters

- **node_id** The node id.
- **name** name of Firmware component.

Returns
The FirmwareComponent object.

Raises
NodeNotFound if the node is not found.

Raises
FirmwareComponentNotFound if the Firmware component is not found.

abstract get_firmware_component_list(*node_id*)

Retrieve a list Firmware Components of a given node.

Parameters
node_id The node id.

Returns
A list of FirmwareComponent objects.

Raises
NodeNotFound if the node is not found.

abstract get_inspection_rule_by_id(*inspection_rule_id*)

Retrieve an inspection rule by id.

Parameters
inspection_rule_id id of the rule to retrieve.

Raises
InspectionRuleNotFound if the rule does not exist.

Returns
A rule.

abstract get_inspection_rule_by_uuid(*inspection_rule_uuid*)

Retrieve an inspection rule by UUID.

Parameters
inspection_rule_uuid UUID of the rule to retrieve.

Raises
InspectionRuleNotFound if the rule does not exist.

Returns
A rule.

abstract get_inspection_rule_list(*limit=None*, *marker=None*, *filters=None*, *sort_key=None*, *sort_dir=None*)

Retrieve a list of inspection rules.

Parameters

- **limit** Maximum number of rules to return.

- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** Direction in which results should be sorted. (asc, desc)

Returns
A list of inspection rules.

abstract get_node_by_id(*node_id*)

Return a node.

Parameters
node_id The id of a node.

Returns
A node.

abstract get_node_by_instance(*instance*)

Return a node.

Parameters
instance The instance uuid to search for.

Returns
A node.

Raises
InstanceNotFound if the instance is not found.

Raises
InvalidUUID if the instance uuid is invalid.

abstract get_node_by_name(*node_name*)

Return a node.

Parameters
node_name The logical name of a node.

Returns
A node.

abstract get_node_by_port_addresses(*addresses*)

Find a node by any matching port address.

Parameters
addresses list of port addresses (e.g. MACs).

Returns
Node object.

Raises
NodeNotFound if none or several nodes are found.

abstract get_node_by_uuid(*node_uuid*)

Return a node.

Parameters
node_uuid The uuid of a node.

Returns
A node.

abstract get_node_history_by_id(*history_id*)

Return a node history representation.

Parameters
history_id The id of a history record.

Returns
A history.

abstract get_node_history_by_node_id(*node_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

List all the history records for a given node.

Parameters

- **node_id** The integer node ID.
- **limit** Maximum number of history records to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted
- **sort_dir** direction in which results should be sorted (asc, desc)

Returns
A list of histories.

abstract get_node_history_by_uuid(*history_uuid*)

Return a node history representation.

Parameters
history_uuid The uuid of a history record

Returns
A history.

abstract get_node_history_list(*limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Return a list of node history records

Parameters

- **limit** Maximum number of history records to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** direction in which results should be sorted. (asc, desc)

abstract get_node_inventory_by_node_id(*node_id*)

Get the node inventory for a given node.

Parameters
node_id The integer node ID.

Returns
An inventory of a node.

abstract get_node_list(*filters=None*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *fields=None*)

Return a list of nodes.

Parameters

- **filters** Filters to apply. Defaults to None.

  associated
  True | False

  reserved
  True | False

  maintenance
  True | False

  chassis_uuid
  uuid of chassis

  driver
  drivers name

  provision_state
  provision state of node

  provisioned_before
  nodes with provision_updated_at field before this interval in seconds

  shard
  nodes with the given shard

- **limit** Maximum number of nodes to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** direction in which results should be sorted. (asc, desc)
- **fields** Comma separated field list to return, to allow for only specific fields to be returned to have maximum API performance calls where not all columns are needed from the database.

abstract get_node_tags_by_node_id(*node_id*)

Get node tags based on its id.

Parameters
node_id The id of a node.

Returns
A list of NodeTag objects.

Raises
NodeNotFound if the node is not found.

abstract get_node_traits_by_node_id(*node_id*)

Get node traits based on its id.

Parameters
node_id The id of a node.

Returns
A list of NodeTrait objects.

Raises
NodeNotFound if the node is not found.

abstract get_nodeinfo_list(*columns=None*, *filters=None*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Get specific columns for matching nodes.

Return a list of the specified columns for all nodes that match the specified filters.

Parameters

- **columns** List of column names to return. Defaults to id column when columns == None.
- **filters** Filters to apply. Defaults to None.

  associated
  True | False

  chassis_uuid
  uuid of chassis

  conductor_group
  conductor group name

  console_enabled
  True | False

  description_contains
  substring in description

  driver
  drivers name

  fault
  current fault type

  id
  numeric ID

  inspection_started_before
  nodes with inspection_started_at field before this interval in seconds

  instance_uuid
  uuid of instance

  instance_name
  name of instance

  lessee
  nodes lessee (e.g. project ID)

  maintenance
  True | False

  owner
  nodes owner (e.g. project ID)

project
either owner or lessee

reserved
True | False

reserved_by_any_of
[conductor1, conductor2]

resource_class
resource class name

retired
True | False

shard_in
shard (multiple possibilities)

provision_state
provision state of node

provision_state_in
provision state of node (multiple possibilities)

provisioned_before
nodes with provision_updated_at field before this interval in seconds

uuid
uuid of node

uuid_in
uuid of node (multiple possibilities)

with_power_state
True | False

- `limit` Maximum number of nodes to return.
- `marker` the last item of the previous page; we return the next result set.
- `sort_key` Attribute by which results should be sorted.
- `sort_dir` direction in which results should be sorted. (asc, desc)

Returns
A list of tuples of the specified columns.

`abstract get_offline_conductors`(*field='hostname'*)

Get a list conductors that are offline (dead).

Parameters
`field` A field to return, hostname by default.

Returns
A list of requested fields of offline conductors.

`abstract get_online_conductors`()

Get a list conductor hostnames that are online and active.

Returns
A list of conductor hostnames.

abstract get_port_by_address(*address*)

Return a network port representation.

Parameters
address The MAC address of a port.

Returns
A port.

abstract get_port_by_id(*port_id*)

Return a network port representation.

Parameters
port_id The id of a port.

Returns
A port.

abstract get_port_by_name(*port_name*)

Return a network port representation.

Parameters
port_name The name of a port.

Returns
A port.

abstract get_port_by_uuid(*port_uuid*)

Return a network port representation.

Parameters
port_uuid The uuid of a port.

Returns
A port.

abstract get_port_list(*limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *conductor_groups=None*, *filters=None*)

Return a list of ports.

Parameters

- **limit** Maximum number of ports to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** direction in which results should be sorted. (asc, desc)
- **conductor_groups** A list of conductor groups to filter by, defaults to None
- **filters** Filters to apply, defaults to None

abstract get_portgroup_by_address(*address*, *project=None*)

Return a network portgroup representation.

Parameters

- **address** The MAC address of a portgroup.

- **project** A node owner or lessee to filter by.

Returns
A portgroup.

Raises
PortgroupNotFound

abstract get_portgroup_by_id(*portgroup_id*)

Return a network portgroup representation.

Parameters
portgroup_id The id of a portgroup.

Returns
A portgroup.

Raises
PortgroupNotFound

abstract get_portgroup_by_name(*name*)

Return a network portgroup representation.

Parameters
name The logical name of a portgroup.

Returns
A portgroup.

Raises
PortgroupNotFound

abstract get_portgroup_by_uuid(*portgroup_uuid*)

Return a network portgroup representation.

Parameters
portgroup_uuid The uuid of a portgroup.

Returns
A portgroup.

Raises
PortgroupNotFound

abstract get_portgroup_list(*limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*, *conductor_groups=None*, *filters=None*)

Return a list of portgroups.

Parameters

- **limit** Maximum number of portgroups to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** Direction in which results should be sorted. (asc, desc)
- **project** A node owner or lessee to filter by.
- **conductor_groups** A list of conductor groups to filter by, defaults to None

- **filters** Filters to apply, defaults to None

Returns
A list of portgroups.

abstract get_portgroups_by_node_id(*node_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

List all the portgroups for a given node.

Parameters

- **node_id** The integer node ID.
- **limit** Maximum number of portgroups to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted
- **sort_dir** Direction in which results should be sorted (asc, desc)
- **project** A node owner or lessee to filter by.

Returns
A list of portgroups.

abstract get_portgroups_by_shards(*shards*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Return a list of portgroups contained in the provided shards.

Parameters

- **shards** A list of shards to filter portgroups by.
- **limit** Maximum number of portgroups to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted
- **sort_dir** Direction in which results should be sorted (asc, desc)

Returns
A list of portgroups.

abstract get_ports_by_node_id(*node_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *filters=None*)

List all the ports for a given node.

Parameters

- **node_id** The integer node ID.
- **limit** Maximum number of ports to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted
- **sort_dir** direction in which results should be sorted (asc, desc)
- **filters** Filters to apply, defaults to None

Returns
A list of ports.

abstract get_ports_by_portgroup_id(*portgroup_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *filters=None*)

List all the ports for a given portgroup.

Parameters

- **portgroup_id** The integer portgroup ID.
- **limit** Maximum number of ports to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted
- **sort_dir** Direction in which results should be sorted (asc, desc)
- **filters** Filters to apply, defaults to None

Returns
A list of ports.

abstract get_ports_by_shards(*shards*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *filters=None*)

Return a list of ports contained in the provided shards.

Parameters
shard_ids A list of shards to filter ports by.

abstract get_runbook_by_id(*runbook_id*)

Retrieve a runbook by ID.

Parameters
runbook_id ID of the runbook to retrieve.

Raises
RunbookNotFound if the runbook does not exist.

Returns
A runbook.

abstract get_runbook_by_name(*runbook_name*)

Retrieve a runbook by name.

Parameters
runbook_name name of the runbook to retrieve.

Raises
RunbookNotFound if the runbook does not exist.

Returns
A runbook.

abstract get_runbook_by_uuid(*runbook_uuid*)

Retrieve a runbook by UUID.

Parameters
runbook_uuid UUID of the runbook to retrieve.

Raises
RunbookNotFound if the runbook does not exist.

Returns
A runbook.

abstract get_runbook_list(*limit=None*, *marker=None*, *filters=None*, *sort_key=None*, *sort_dir=None*)

Retrieve a list of runbooks.

Parameters

- **limit** Maximum number of runbooks to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** Direction in which results should be sorted. (asc, desc)

Returns
A list of runbooks.

abstract get_runbook_list_by_names(*names*)

Return a list of runbooks with one of a list of names.

Parameters
names List of names to filter by.

Returns
A list of runbooks.

abstract get_shard_list()

Retrieve a list of shards.

Returns
list of dicts containing shard names and count

abstract get_volume_connector_by_id(*db_id*)

Return a volume connector representation.

Parameters
db_id The integer database ID of a volume connector.

Returns
A volume connector with the specified ID.

Raises
VolumeConnectorNotFound If a volume connector with the specified ID is not found.

abstract get_volume_connector_by_uuid(*connector_uuid*)

Return a volume connector representation.

Parameters
connector_uuid The UUID of a connector.

Returns
A volume connector with the specified UUID.

Raises
VolumeConnectorNotFound If a volume connector with the specified UUID is not found.

abstract get_volume_connector_list(*limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

Return a list of volume connectors.

Parameters

- **limit** Maximum number of volume connectors to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** Direction in which results should be sorted. (asc, desc)
- **project** The associated node project to search with.

Returns
a list of `VolumeConnector` objects

Returns
A list of volume connectors.

Raises
InvalidParameterValue If sort_key does not exist.

abstract get_volume_connectors_by_node_id(*node_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

List all the volume connectors for a given node.

Parameters

- **node_id** The integer node ID.
- **limit** Maximum number of volume connectors to return.
- **marker** The last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted
- **sort_dir** Direction in which results should be sorted (asc, desc)
- **project** The associated node project to search with.

Returns
a list of `VolumeConnector` objects

Returns
A list of volume connectors.

Raises
InvalidParameterValue If sort_key does not exist.

abstract get_volume_target_by_id(*db_id*)

Return a volume target representation.

Parameters
db_id The database primary key (integer) ID of a volume target.

Returns
A volume target.

Raises
VolumeTargetNotFound if no volume target with this ID exists.

abstract get_volume_target_by_uuid(*uuid*)

Return a volume target representation.

Parameters
uuid The UUID of a volume target.

Returns
A volume target.

Raises
VolumeTargetNotFound if no volume target with this UUID exists.

abstract get_volume_target_list(*limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

Return a list of volume targets.

Parameters

- **limit** Maximum number of volume targets to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted.
- **sort_dir** direction in which results should be sorted. (asc, desc)
- **project** The associated node project to search with.

Returns
a list of `VolumeConnector` objects

Returns
A list of volume targets.

Raises
InvalidParameterValue if sort_key does not exist.

abstract get_volume_targets_by_node_id(*node_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

List all the volume targets for a given node.

Parameters

- **node_id** The integer node ID.
- **limit** Maximum number of volume targets to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted
- **sort_dir** direction in which results should be sorted (asc, desc)
- **project** The associated node project to search with.

Returns
a list of `VolumeConnector` objects

Returns
A list of volume targets.

Raises
InvalidParameterValue if sort_key does not exist.

abstract get_volume_targets_by_volume_id(*volume_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

List all the volume targets for a given volume id.

Parameters

- **volume_id** The UUID of the volume.
- **limit** Maximum number of volume targets to return.
- **marker** the last item of the previous page; we return the next result set.
- **sort_key** Attribute by which results should be sorted
- **sort_dir** direction in which results should be sorted (asc, desc)

Returns
A list of volume targets.

Raises
InvalidParameterValue if sort_key does not exist.

abstract list_conductor_hardware_interfaces(*conductor_id*)

List all registered hardware interfaces for a conductor.

Parameters
conductor_id Database ID of conductor.

Returns
List of `ConductorHardwareInterfaces` objects.

abstract list_hardware_type_interfaces(*hardware_types*)

List registered hardware interfaces for given hardware types.

This is restricted to only active conductors. :param hardware_types: list of hardware types to filter by. :returns: list of `ConductorHardwareInterfaces` objects.

abstract node_tag_exists(*node_id*, *tag*)

Check if the specified tag exist on the node.

Parameters

- **node_id** The id of a node.
- **tag** A tag string.

Returns
True if the tag exists otherwise False.

Raises
NodeNotFound if the node is not found.

abstract node_trait_exists(*node_id*, *trait*)

Check if the specified trait exists on the node.

Parameters

- **node_id** The id of a node.
- **trait** A trait string.

Returns

True if the trait exists otherwise False.

Raises

NodeNotFound if the node is not found.

abstract query_node_history_records_for_purge(*conductor_id*)

Utility method to identify nodes to clean history records for.

Parameters

conductor_id Id value for the conductor to perform this query on behalf of.

Returns

A dictionary with key values of node database ID values and a list of values associated with the node.

abstract register_conductor(*values*, *update_existing=False*)

Register an active conductor with the cluster.

Parameters

- **values** A dict of values which must contain the following:

```
{
 'hostname': the unique hostname which identifies
             this Conductor service.
 'drivers': a list of supported drivers.
 'version': the version of the object.Conductor
}
```

- **update_existing** When false, registration will raise an exception when a conflicting online record is found. When true, will overwrite the existing record. Default: False.

Returns

A conductor.

Raises

ConductorAlreadyRegistered

abstract register_conductor_hardware_interfaces(*conductor_id*, *hardware_type*, *interface_type*, *interfaces*, *default_interface*)

Registers hardware interfaces for a conductor.

Parameters

- **conductor_id** Database ID of conductor to register for.
- **hardware_type** Name of hardware type for the interfaces.

- **interface_type** Type of interfaces, e.g. deploy or boot.
- **interfaces** List of interface names to register.
- **default_interface** String, the default interface for this hardware type and interface type.

Raises
ConductorHardwareInterfacesAlreadyRegistered if at least one of the interfaces in the combination of all parameters is already registered.

abstract release_node(*tag*, *node_id*)

Release the reservation on a node.

Parameters
- **tag** A string uniquely identifying the reservation holder.
- **node_id** A node id or uuid.

Raises
NodeNotFound if the node is not found.

Raises
NodeLocked if the node is reserved by another host.

Raises
NodeNotLocked if the node was found to not have a reservation at all.

abstract reserve_node(*tag*, *node_id*)

Reserve a node.

To prevent other ManagerServices from manipulating the given Node while a Task is performed, mark it reserved by this host.

Parameters
- **tag** A string uniquely identifying the reservation holder.
- **node_id** A node id or uuid.

Returns
A Node object.

Raises
NodeNotFound if the node is not found.

Raises
NodeLocked if the node is already reserved.

abstract set_node_tags(*node_id*, *tags*)

Replace all of the node tags with specified list of tags.

This ignores duplicate tags in the specified list.

Parameters
- **node_id** The id of a node.
- **tags** List of tags.

Returns
A list of NodeTag objects.

Raises
NodeNotFound if the node is not found.

abstract set_node_traits(*node_id*, *traits*, *version*)

Replace all of the node traits with specified list of traits.

This ignores duplicate traits in the specified list.

Parameters

- **node_id** The id of a node.
- **traits** List of traits.
- **version** the version of the object.Trait.

Returns
A list of NodeTrait objects.

Raises
InvalidParameterValue if setting the traits would exceed the per-node traits limit.

Raises
NodeNotFound if the node is not found.

abstract take_over_allocation(*allocation_id*, *old_conductor_id*, *new_conductor_id*)

Do a take over for an allocation.

The allocation is only updated if the old conductor matches the provided value, thus guarding against races.

Parameters

- **allocation_id** Allocation ID
- **old_conductor_id** The conductor ID we expect to be the current `conductor_affinity` of the allocation.
- **new_conductor_id** The conductor ID of the new `conductor_affinity`.

Returns
True if the take over was successful, False otherwise.

Raises
AllocationNotFound

abstract touch_conductor(*hostname*, *online=True*)

Mark a conductor as active by updating its updated_at property.

Calling periodically with `online=False` will result in the conductor appearing unregistered, but recently enough to prevent other conductors failing orphan nodes. This improves the behaviour of graceful and drain shutdown.

Parameters

- **hostname** The hostname of this conductor service.
- **online** Whether the conductor is online.

Raises
ConductorNotFound

abstract touch_node_provisioning(*node_id*)

Mark the nodes provisioning as running.

Mark the nodes provisioning as running by updating its provision_updated_at property.

Parameters
node_id The id of a node.

Raises
NodeNotFound

abstract unregister_conductor(*hostname*)

Remove this conductor from the service registry immediately.

Parameters
hostname The hostname of this conductor service.

Raises
ConductorNotFound

abstract unregister_conductor_hardware_interfaces(*conductor_id*)

Unregisters all hardware interfaces for a conductor.

Parameters
conductor_id Database ID of conductor to unregister for.

abstract unset_node_tags(*node_id*)

Remove all tags of the node.

Parameters
node_id The id of a node.

Raises
NodeNotFound if the node is not found.

abstract unset_node_traits(*node_id*)

Remove all traits of the node.

Parameters
node_id The id of a node.

Raises
NodeNotFound if the node is not found.

abstract update_allocation(*allocation_id*, *values*, *update_node=True*)

Update properties of an allocation.

Parameters

- **allocation_id** Allocation ID
- **values** Dict of values to update.
- **update_node** If True and node_id is updated, update the node with instance_uuid and traits from the allocation

Returns
An allocation.

Raises
AllocationNotFound

Raises
AllocationDuplicateName

Raises
InstanceAssociated

Raises
NodeAssociated

abstract update_bios_setting_list(*node_id*, *settings*, *version*)

Update a list of BIOSSetting records.

Parameters

- **node_id** The node id.
- **settings** A list of BIOS Settings to be updated.

```
[
  {
   'name': String,
   'value': String,
   additional settings from BIOS registry
  },
  {
   'name': String,
   'value': String,
   additional settings from BIOS registry
  },
  ...
]
```

- **version** the version of the object.BIOSSetting.

Returns
A list of BIOSSetting objects.

Raises
NodeNotFound if the node is not found.

Raises
BIOSSettingNotFound if any of the settings is not found.

abstract update_chassis(*chassis_id*, *values*)

Update properties of an chassis.

Parameters

- **chassis_id** The id or the uuid of a chassis.
- **values** Dict of values to update.

Returns
A chassis.

abstract update_deploy_template(*template_id*, *values*)

Update a deployment template.

Parameters

- **template_id** ID of the deployment template to update.
- **values** A dict describing the deployment template. For example:

```
{
 'uuid': uuidutils.generate_uuid(),
 'name': 'CUSTOM_DT1',
}
```

Raises
DeployTemplateDuplicateName if a deploy template with the same name exists.

Raises
DeployTemplateNotFound if the deploy template does not exist.

Returns
A deploy template.

abstract update_firmware_component(*node_id*, *component*, *values*)

Update a FirmwareComponent record.

Parameters

- **node_id** The node id.
- **component** The component of the node to update.
- **values** A dictionary with the new information about the FirmwareComponent.

```
{
  'current_version': String,
  'last_version_flashed': String
}
```

Returns
A FirmwareComponent object.

Raises
FirmwareComponentNotFound the component is not found.

abstract update_inspection_rule(*inspection_rule_id*, *values*)

Update an inspection rule.

Parameters

- **inspection_rule_id** The id or uuid of an inspection rule.
- **values** Dict of values to update.

Raises
InspectionRuleNotFound if the rule does not exist.

Returns
A rule.

abstract update_node(*node_id*, *values*)

Update properties of a node.

Parameters

- **node_id** The id or uuid of a node.
- **values** Dict of values to update. May be a partial list, eg. when setting the properties for a driver. For example:

```
{
 'driver_info':
     {
      'my-field-1': val1,
      'my-field-2': val2,
     }
}
```

Returns
A node.

Raises
NodeAssociated

Raises
NodeNotFound

abstract update_port(*port_id*, *values*)

Update properties of an port.

Parameters

- **port_id** The id or MAC of a port.
- **values** Dict of values to update.

Returns
A port.

abstract update_portgroup(*portgroup_id*, *values*)

Update properties of a portgroup.

Parameters

- **portgroup_id** The UUID or MAC of a portgroup.
- **values** Dict of values to update. May contain the following keys: uuid name node_id address extra created_at updated_at

Returns
A portgroup.

Raises
InvalidParameterValue

Raises
PortgroupNotFound

Raises
PortgroupDuplicateName

Raises
PortgroupMACAlreadyExists

abstract update_runbook(*runbook_id*, *values*)

Update a runbook.

Parameters

- **runbook_id** ID of the runbook to update.
- **values** A dict describing the runbook. For example:

```
{
 'uuid': uuidutils.generate_uuid(),
 'name': 'CUSTOM_DT1',
}
```

Raises

RunbookDuplicateName if a runbook with the same name exists.

Raises

RunbookNotFound if the runbook does not exist.

Returns

A runbook.

abstract update_to_latest_versions(*context*, *max_count*)

Updates objects to their latest known versions.

This scans all the tables and for objects that are not in their latest version, updates them to that version.

Parameters

- **context** the admin context
- **max_count** The maximum number of objects to migrate. Must be >= 0. If zero, all the objects will be migrated.

Returns

A 2-tuple, 1. the total number of objects that need to be migrated (at the beginning of this call) and 2. the number of migrated objects.

abstract update_volume_connector(*ident*, *connector_info*)

Update properties of a volume connector.

Parameters

- **ident** The UUID or integer ID of a volume connector.
- **connector_info** Dictionary containing the information about connector to update.

Returns

A volume connector.

Raises

VolumeConnectorTypeAndIdAlreadyExists If another connector already exists with a matching type and connector_id field.

Raises

VolumeConnectorNotFound If a volume connector with the specified ident does not exist.

Raises
InvalidParameterValue When a UUID is included in connector_info.

abstract update_volume_target(*ident*, *target_info*)
Update information for a volume target.

Parameters
- **ident** The UUID or integer ID of a volume target.
- **target_info** Dictionary containing the information about volume target to update.

Returns
A volume target.

Raises
InvalidParameterValue if a UUID is included in target_info.

Raises
VolumeTargetBootIndexAlreadyExists if a volume target already exists with the same boot index and node ID.

Raises
VolumeTargetNotFound if no volume target with this ident exists.

ironic.db.api.**get_instance**()
Return a DB API instance.

ironic.db.migration module

Database setup and migration commands.

ironic.db.migration.**create_schema**()

ironic.db.migration.**get_backend**()

ironic.db.migration.**revision**(*message*, *autogenerate*)

ironic.db.migration.**stamp**(*version*)

ironic.db.migration.**upgrade**(*version=None*)
Migrate the database to *version* or the most recent version.

ironic.db.migration.**version**()

Module contents

ironic.dhcp package

Submodules

ironic.dhcp.base module

Abstract base class for dhcp providers.

class ironic.dhcp.base.**BaseDHCP**

Bases: object

Base class for DHCP provider APIs.

clean_dhcp_opts(*task*)

Clean up the DHCP BOOT options for all ports in *task*.

Parameters

task A TaskManager instance.

Raises

FailedToCleanDHCPOpts

get_ip_addresses(*task*)

Get IP addresses for all ports/portgroups in *task*.

Parameters

task A TaskManager instance.

Returns

List of IP addresses associated with tasks ports and portgroups.

supports_ipxe_tag()

Whether the provider will correctly apply the ipxe tag.

When iPXE makes a DHCP request, does this provider support adding the tag *ipxe* or *ipxe6* (for IPv6). When the provider returns True, options can be added which filter on these tags.

Returns

True when the driver supports tagging iPXE DHCP requests

abstract update_dhcp_opts(*task*, *options*, *vifs=None*)

Send or update the DHCP BOOT options for this node.

Parameters

- **task** A TaskManager instance.
- **options** this will be a list of dicts, e.g.

```
[{'opt_name': '67',
  'opt_value': 'pxelinux.0',
  'ip_version': 4},
 {'opt_name': '66',
  'opt_value': '123.123.123.456',
  'ip_version': 4}]
```

- **vifs** A dict with keys ports and portgroups and dicts as values. Each dict has key/value pairs of the form <ironic UUID>:<neutron port UUID>. e.g.

```
{'ports': {'port.uuid': vif.id},
 'portgroups': {'portgroup.uuid': vif.id}}
```

If the value is None, will get the list of ports/portgroups from the Ironic port/portgroup objects.

Raises
FailedToUpdateDHCPOptOnPort

abstract update_port_dhcp_opts(*port_id*, *dhcp_options*, *context=None*)
Update one or more DHCP options on the specified port.

Parameters

- **port_id** designate which port these attributes will be applied to.
- **dhcp_options** this will be a list of dicts, e.g.

```
[{'opt_name': '67',
  'opt_value': 'pxelinux.0',
  'ip_version': 4},
 {'opt_name': '66',
  'opt_value': '123.123.123.456',
  'ip_version': 4}]
```

- **context** (ironic.common.context.RequestContext) request context

Raises
FailedToUpdateDHCPOptOnPort

ironic.dhcp.dnsmasq module

class ironic.dhcp.dnsmasq.**DnsmasqDHCPApi**
Bases: *BaseDHCP*

API for managing host specific Dnsmasq configuration.

clean_dhcp_opts(*task*)
Clean up the DHCP BOOT options for the host in *task*.

Parameters
task A TaskManager instance.

Raises
FailedToCleanDHCPOpts

get_ip_addresses(*task*)
Get IP addresses for all ports/portgroups in *task*.

Parameters
task a TaskManager instance.

Returns
List of IP addresses associated with tasks ports/portgroups.

supports_ipxe_tag()
Whether the provider will correctly apply the ipxe tag.

When iPXE makes a DHCP request, does this provider support adding the tag *ipxe* or *ipxe6* (for IPv6). When the provider returns True, options can be added which filter on these tags.

The *dnsmasq* provider sets this to True on the assumption that the following is included in the dnsmasq.conf:

dhcp-match=set:ipxe,175

Returns
True

`update_dhcp_opts`(*task*, *options*, *vifs=None*)

Send or update the DHCP BOOT options for this node.

Parameters

- **task** A TaskManager instance.
- **options** this will be a list of dicts, e.g.

```
[{'opt_name': '67',
  'opt_value': 'pxelinux.0',
  'ip_version': 4},
 {'opt_name': '66',
  'opt_value': '123.123.123.456',
  'ip_version': 4}]
```

- **vifs** Ignored argument

`update_port_dhcp_opts`(*port_id*, *dhcp_options*, *context=None*)

Update one or more DHCP options on the specified port.

Parameters

- **port_id** designate which port these attributes will be applied to.
- **dhcp_options** this will be a list of dicts, e.g.

```
[{'opt_name': '67',
  'opt_value': 'pxelinux.0',
  'ip_version': 4},
 {'opt_name': '66',
  'opt_value': '123.123.123.456',
  'ip_version': 4}]
```

- **context** (ironic.common.context.RequestContext) request context

Raises
FailedToUpdateDHCPOptOnPort

ironic.dhcp.neutron module

`class ironic.dhcp.neutron.NeutronDHCPApi`

Bases: *BaseDHCP*

API for communicating to neutron 2.x API.

`get_ip_addresses`(*task*)

Get IP addresses for all ports/portgroups in *task*.

Parameters
task a TaskManager instance.

Returns
List of IP addresses associated with tasks ports/portgroups.

supports_ipxe_tag()

Whether the provider will correctly apply the ipxe tag.

When iPXE makes a DHCP request, does this provider support adding the tag *ipxe* or *ipxe6* (for IPv6). When the provider returns True, options can be added which filter on these tags.

Returns
True

update_dhcp_opts(*task*, *options*, *vifs=None*)

Send or update the DHCP BOOT options for this node.

Parameters

- **task** A TaskManager instance.
- **options** this will be a list of dicts, e.g.

```
[{'opt_name': '67',
  'opt_value': 'pxelinux.0',
  'ip_version': 4},
 {'opt_name': '66',
  'opt_value': '123.123.123.456',
  'ip_version': 4}]
```

- **vifs** a dict of Neutron port/portgroup dicts to update DHCP options on. The port/portgroup dict key should be Ironic port UUIDs, and the values should be Neutron port UUIDs, e.g.

```
{'ports': {'port.uuid': vif.id},
 'portgroups': {'portgroup.uuid': vif.id}}
If the value is None, will get the list of
ports/portgroups from the Ironic port/portgroup
objects.
```

update_port_dhcp_opts(*port_id*, *dhcp_options*, *context=None*)

Update a ports attributes.

Update one or more DHCP options on the specified port. For the relevant API spec, see https://docs.openstack.org/api-ref/network/v2/index.html#update-port

Parameters

- **port_id** designate which port these attributes will be applied to.
- **dhcp_options** this will be a list of dicts, e.g.

```
[{'opt_name': '67',
  'opt_value': 'pxelinux.0',
  'ip_version': 4},
 {'opt_name': '66',
  'opt_value': '123.123.123.456'},
  'ip_version': 4}]
```

- **context** (ironic.common.context.RequestContext) request context

Raises
FailedToUpdateDHCPOptOnPort

ironic.dhcp.none module

class ironic.dhcp.none.**NoneDHCPApi**

Bases: *BaseDHCP*

No-op DHCP API.

get_ip_addresses(*task*)

Get IP addresses for all ports/portgroups in *task*.

Parameters
task A TaskManager instance.

Returns
List of IP addresses associated with tasks ports and portgroups.

update_dhcp_opts(*task*, *options*, *vifs=None*)

Send or update the DHCP BOOT options for this node.

Parameters

- **task** A TaskManager instance.
- **options** this will be a list of dicts, e.g.

```
[{'opt_name': '67',
  'opt_value': 'pxelinux.0',
  'ip_version': 4},
 {'opt_name': '66',
  'opt_value': '123.123.123.456',
  'ip_version': 4}]
```

- **vifs** A dict with keys ports and portgroups and dicts as values. Each dict has key/value pairs of the form <ironic UUID>:<neutron port UUID>. e.g.

```
{'ports': {'port.uuid': vif.id},
 'portgroups': {'portgroup.uuid': vif.id}}
```

  If the value is None, will get the list of ports/portgroups from the Ironic port/portgroup objects.

Raises
FailedToUpdateDHCPOptOnPort

update_port_dhcp_opts(*port_id*, *dhcp_options*, *context=None*)

Update one or more DHCP options on the specified port.

Parameters

- **port_id** designate which port these attributes will be applied to.
- **dhcp_options** this will be a list of dicts, e.g.

```
[{'opt_name': '67',
  'opt_value': 'pxelinux.0',
  'ip_version': 4},
 {'opt_name': '66',
  'opt_value': '123.123.123.456',
  'ip_version': 4}]
```

- **context** (ironic.common.context.RequestContext) request context

Raises

FailedToUpdateDHCPOptOnPort

Module contents

ironic.drivers package

Subpackages

ironic.drivers.modules package

Subpackages

ironic.drivers.modules.ansible package

Submodules

ironic.drivers.modules.ansible.deploy module

Ansible deploy interface

class ironic.drivers.modules.ansible.deploy.**AnsibleDeploy**(**args*, ***kwargs*)

Bases: *HeartbeatMixin*, *AgentOobStepsMixin*, *DeployInterface*

Interface for deploy-related actions.

clean_up(*task*)

Clean up the deployment environment for this node.

collect_deploy_logs = False

deploy(*task*)

Perform a deployment to a node.

execute_clean_step(*task*, *step*)

Execute a clean step.

Parameters

- **task** a TaskManager object containing the node
- **step** a clean step dictionary to execute

Returns

None

get_clean_steps(*task*)

Get the list of clean steps from the file.

Parameters
task a TaskManager object containing the node

Returns
A list of clean step dictionaries

`get_properties()`

Return the properties of the interface.

`in_core_deploy_step(task)`

Check if we are in the deploy.deploy deploy step.

Assumes that we are in the DEPLOYWAIT state.

Parameters
task a TaskManager instance

Returns
True if the current deploy step is deploy.deploy.

`prepare(task)`

Prepare the deployment environment for this node.

`prepare_cleaning(task)`

Boot into the ramdisk to prepare for cleaning.

Parameters
task a TaskManager object containing the node

Raises
NodeCleaningFailure if the previous cleaning ports cannot be removed or if new cleaning ports cannot be created

Returns
None or states.CLEANWAIT for async prepare.

`process_next_step(task, step_type)`

Start the next clean/deploy step if the previous one is complete.

Parameters

- **task** a TaskManager instance
- **step_type** clean or deploy

`take_over(task)`

Take over management of this tasks node from a dead conductor.

If conductors hosts maintain a static relationship to nodes, this method should be implemented by the driver to allow conductors to perform the necessary work during the remapping of nodes to conductors when a conductor joins or leaves the cluster.

For example, the PXE driver has an external dependency:
Neutron must forward DHCP BOOT requests to a conductor which has prepared the tftpboot environment for the given node. When a conductor goes offline, another conductor must change this setting in Neutron as part of remapping that nodes control to itself. This is performed within the *takeover* method.

Parameters
task A TaskManager instance containing the node to act on.

tear_down(*task*)

Tear down a previous deployment on the tasks node.

tear_down_agent(*task*)

A deploy step to tear down the agent.

Shuts down the machine and removes it from the provisioning network.

Parameters
task a TaskManager object containing the node

tear_down_cleaning(*task*)

Clean up the PXE and DHCP files after cleaning.

Parameters
task a TaskManager object containing the node

Raises
NodeCleaningFailure if the cleaning ports cannot be removed

validate(*task*)

Validate the driver-specific Node deployment info.

write_image(*task*)

exception ironic.drivers.modules.ansible.deploy.**PlaybookNotFound**(*message=None*, ***kwargs*)

Bases: *IronicException*

Module contents

ironic.drivers.modules.drac package

Submodules

ironic.drivers.modules.drac.bios module

DRAC BIOS configuration specific methods

class ironic.drivers.modules.drac.bios.**DracRedfishBIOS**(**args*, ***kwargs*)

Bases: *RedfishBIOS*

iDRAC Redfish interface for BIOS settings-related actions.

Presently, this class entirely defers to its base class, a generic, vendor-independent Redfish interface. Future resolution of Dell EMC- specific incompatibilities and introduction of vendor value added should be implemented by this class.

supported = False

Indicates if an interface is supported.

This will be set to False for interfaces which are untested in first- or third-party CI, or in the process of being deprecated.

ironic.drivers.modules.drac.boot module

class ironic.drivers.modules.drac.boot.**DracRedfishVirtualMediaBoot**(**args*, ***kwargs*)

Bases: *RedfishVirtualMediaBoot*

iDRAC Redfish interface for virtual media boot-related actions.

Virtual Media allows booting the system from virtual CD/DVD drive containing user image that BMC inserts into the drive.

The CD/DVD images must be in ISO format and (depending on BMC implementation) could be pulled over HTTP, served as iSCSI targets or NFS volumes.

The baseline boot workflow is mostly based on the standard Redfish virtual media boot interface, which looks like this:

1. Pull kernel, ramdisk and ESP if UEFI boot is requested (FAT partition image with EFI boot loader) images
2. Create bootable ISO out of images (#1), push it to Glance and pass to the BMC as Swift temporary URL
3. Optionally create floppy image with desired system configuration data, push it to Glance and pass to the BMC as Swift temporary URL
4. Insert CD/DVD and (optionally) floppy images and set proper boot mode

For building deploy or rescue ISO, redfish boot interface uses *deploy_kernel*/*deploy_ramdisk* or *rescue_kernel*/*rescue_ramdisk* properties from *[instance_info]* or *[driver_info]*.

For building boot (user) ISO, redfish boot interface seeks *kernel_id* and *ramdisk_id* properties in the Glance image metadata found in *[instance_info]image_source* node property.

iDRAC virtual media boot interface only differs by the way how it sets the node to boot from a virtual media device - this is done via OEM action call implemented in Dell sushy OEM extension package.

VIRTUAL_MEDIA_DEVICES = {'cdrom': VirtualMediaType.CD, 'floppy': VirtualMediaType.FLOPPY}

ironic.drivers.modules.drac.inspect module

DRAC inspection interface

class ironic.drivers.modules.drac.inspect.**DracRedfishInspect**(**args*, ***kwargs*)

Bases: *RedfishInspect*

iDRAC Redfish interface for inspection-related actions.

inspect_hardware(*task*)

Inspect hardware to get the hardware properties.

Inspects hardware to get the essential properties. It fails if any of the essential properties are not received from the node.

Parameters

task a TaskManager instance.

Raises
HardwareInspectionFailure if essential properties could not be retrieved successfully.

Returns
The resulting state of inspection.

ironic.drivers.modules.drac.management module

DRAC management interface

class ironic.drivers.modules.drac.management.**DracRedfishManagement**(**args*, ***kwargs*)

Bases: *RedfishManagement*

iDRAC Redfish interface for management-related actions.

EXPORT_CONFIGURATION_ARGSINFO = {'export_configuration_location': {'description': 'URL of location to save the configuration to.', 'required': True}}

IMPORT_CONFIGURATION_ARGSINFO = {'import_configuration_location': {'description': 'URL of location to fetch desired configuration from.', 'required': True}}

IMPORT_EXPORT_CONFIGURATION_ARGSINFO = {'export_configuration_location': {'description': 'URL of location to save the configuration to.', 'required': True}, 'import_configuration_location': {'description': 'URL of location to fetch desired configuration from.', 'required': True}}

clear_job_queue(*task*)

Clear iDRAC job queue.

Parameters
task a TaskManager instance containing the node to act on.

Raises
RedfishError on an error.

export_configuration(*task*, *export_configuration_location*)

(Deprecated) Export the configuration of the server.

Exports the configuration of the server against which the step is run and stores it in specific format in indicated location.

Uses Dells Server Configuration Profile (SCP) from *sushy* oem extension to get ALL configuration for cloning.

Parameters

- **task** A task from TaskManager.
- **export_configuration_location** URL of location to save the configuration to.

Raises
MissingParameterValue if missing configuration name of a file to save the configuration to

Raises
DracOperatationError when no managagers for Redfish system found or configuration export from SCP failed

Raises
RedfishError when loading OEM extension failed

`import_configuration`(*task*, *import_configuration_location*)

(Deprecated) Import and apply the configuration to the server.

Gets pre-created configuration from storage by given location and imports that into given server. Uses Dells Server Configuration Profile (SCP).

Parameters

- **task** A task from TaskManager.
- **import_configuration_location** URL of location to fetch desired configuration from.

Raises
MissingParameterValue if missing configuration name of a file to fetch the configuration from

`import_export_configuration`(*task*, *import_configuration_location*, *export_configuration_location*)

Import and export configuration in one go.

Gets pre-created configuration from storage by given name and imports that into given server. After that exports the configuration of the server against which the step is run and stores it in specific format in indicated storage as configured by Ironic.

Parameters

- **import_configuration_location** URL of location to fetch desired configuration from.
- **export_configuration_location** URL of location to save the configuration to.

`known_good_state`(*task*)

Reset iDRAC to known good state.

An iDRAC is reset to a known good state by resetting it and clearing its job queue.

Parameters
task a TaskManager instance containing the node to act on.

Raises
RedfishError on an error.

`reset_idrac`(*task*)

Reset the iDRAC.

Parameters
task a TaskManager instance containing the node to act on.

Raises
RedfishError on an error.

ironic.drivers.modules.drac.power module

DRAC power interface

class ironic.drivers.modules.drac.power.**DracRedfishPower**(**args*, ***kwargs*)

Bases: *RedfishPower*

iDRAC Redfish interface for power-related actions.

Presently, this class entirely defers to its base class, a generic, vendor-independent Redfish interface. Future resolution of Dell EMC- specific incompatibilities and introduction of vendor value added should be implemented by this class.

supported = False

Indicates if an interface is supported.

This will be set to False for interfaces which are untested in first- or third-party CI, or in the process of being deprecated.

ironic.drivers.modules.drac.raid module

DRAC RAID specific methods

class ironic.drivers.modules.drac.raid.**DracRedfishRAID**(**args*, ***kwargs*)

Bases: *RedfishRAID*

iDRAC Redfish interface for RAID related actions.

Includes iDRAC specific adjustments for RAID related actions.

create_configuration(*task*, *create_root_volume=True*, *create_nonroot_volumes=True*, *delete_existing=False*)

Create RAID configuration on the node.

This method creates the RAID configuration as read from node.target_raid_config. This method by default will create all logical disks.

Parameters

- **task** TaskManager object containing the node.
- **create_root_volume** Setting this to False indicates not to create root volume that is specified in the nodes target_raid_config. Default value is True.
- **create_nonroot_volumes** Setting this to False indicates not to create nonroot volumes (all except the root volume) in the nodes target_raid_config. Default value is True.
- **delete_existing** Setting this to True indicates to delete RAID configuration prior to creating the new configuration. Default is False.

Returns
states.CLEANWAIT if RAID configuration is in progress asynchronously or None if it is complete.

Raises
RedfishError if there is an error creating the configuration

`delete_configuration`(*task*)
Delete RAID configuration on the node.

Parameters
task TaskManager object containing the node.

Returns
states.CLEANWAIT (cleaning) or states.DEPLOYWAIT (deployment) if deletion is in progress asynchronously or None if it is complete.

`post_delete_configuration`(*task*, *raid_configs*, *return_state=None*)
Perform post delete_configuration action to commit the config.

Clears foreign configuration for all RAID controllers. If no foreign configuration to clear, then checks if any controllers can be converted to RAID mode.

Parameters
- **task** a TaskManager instance containing the node to act on.
- **raid_configs** a list of dictionaries containing the RAID configuration operation details.
- **return_state** state to return based on operation being invoked

`pre_create_configuration`(*task*, *logical_disks_to_create*)
Perform required actions before creating config.

Converts any physical disks of selected controllers to RAID mode if in non-RAID mode.

Parameters
- **task** a TaskManager instance containing the node to act on.
- **logical_disks_to_create** list of logical disks to create.

Returns
updated list of logical disks to create

ironic.drivers.modules.drac.utils module

`ironic.drivers.modules.drac.utils.execute_oem_manager_method`(*task*, *process_name*, *lambda_oem_func*)
Loads OEM manager and executes passed method on it.

Known iDRAC Redfish systems has only one manager, but as Redfish schema allows a list this method iterates through all values in case this changes in future. If there are several managers, this will try starting from the first in the list until the first success.

Parameters
- **task** a TaskManager instance.
- **process_name** user friendly name of method to be executed. Used in exception and log messages.
- **lambda_oem_func** method to execute as lambda function with input parameter OEM extension manager. Example: lambda m: m.reset_idrac()

Returns
Returned value of lambda_oem_func

Raises
RedfishError if cant execute OEM function either because there are no managers to the system, failed to load OEM extension or execution of the OEM method failed itself.

ironic.drivers.modules.drac.vendor_passthru module

DRAC vendor-passthru interface

class ironic.drivers.modules.drac.vendor_passthru.**DracRedfishVendorPassthru**(**args*, ***kwargs*)

Bases: *RedfishVendorPassthru*

iDRAC Redfish interface for vendor_passthru.

Use the Redfish implementation for vendor passthru.

supported = False
Indicates if an interface is supported.

This will be set to False for interfaces which are untested in first- or third-party CI, or in the process of being deprecated.

Module contents

ironic.drivers.modules.ilo package

Submodules

ironic.drivers.modules.ilo.bios module

iLO BIOS Interface

class ironic.drivers.modules.ilo.bios.**IloBIOS**(**args*, ***kwargs*)

Bases: *BIOSInterface*

apply_configuration(*task*, *settings*)
Applies the provided configuration on the node.

Parameters
- **task** a TaskManager instance.
- **settings** Settings intended to be applied on the node.

Raises
NodeCleaningFailure, on failure to execute of clean step.

Raises
InstanceDeployFailure, on failure to execute of deploy step.

cache_bios_settings(*task*)
Store the BIOS settings in the database.

Parameters
task a TaskManager instance.

Raises
NodeCleaningFailure, on failure to execute of clean step.

Raises
InstanceDeployFailure, on failure to execute of deploy step.

`factory_reset`(*task*)

Reset the BIOS settings to factory configuration.

Parameters
task a TaskManager instance.

Raises
NodeCleaningFailure, on failure to execute of clean step.

Raises
InstanceDeployFailure, on failure to execute of deploy step.

`get_properties`()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

`validate`(*task*)

Check that driver_info contains required ILO credentials.

Validates whether the driver_info property of the supplied tasks node contains the required credentials information.

Parameters
task a task from TaskManager.

Raises
InvalidParameterValue if required iLO parameters are not valid.

Raises
MissingParameterValue if a required parameter is missing.

ironic.drivers.modules.ilo.boot module

Boot Interface for iLO drivers and its supporting methods.

`class ironic.drivers.modules.ilo.boot.IloPXEBoot`(**args*, ***kwargs*)

Bases: *PXEBoot*

`clean_up_instance`(*task*)

Cleans up the boot of instance.

This method cleans up the PXE environment that was setup for booting the instance. It unlinks the instance kernel/ramdisk in the nodes directory in tftproot and removes its PXE config. In case of UEFI iSCSI booting, it cleans up iSCSI target information from the node.

Parameters
task a task from TaskManager.

Returns
None

Raises
IloOperationError, if some operation on iLO failed.

prepare_instance(*task*)

Prepares the boot of instance.

This method prepares the boot of the instance after reading relevant information from the nodes instance_info. In case of localboot, it cleans up the PXE config. In case of boot from volume, it updates the iSCSI info onto iLO and sets the node to boot from UefiTarget boot device.

Parameters
task a task from TaskManager.

Returns
None

Raises
IloOperationError, if some operation on iLO failed.

prepare_ramdisk(*task*, *ramdisk_params*)

Prepares the boot of Ironic ramdisk using PXE.

This method prepares the boot of the deploy or rescue ramdisk after reading relevant information from the nodes driver_info and instance_info.

Parameters

- **task** a task from TaskManager.
- **ramdisk_params** the parameters to be passed to the ramdisk.

Returns
None

Raises
MissingParameterValue, if some information is missing in nodes driver_info or instance_info.

Raises
InvalidParameterValue, if some information provided is invalid.

Raises
IronicException, if some power or set boot boot device operation failed on the node.

Raises
IloOperationError, if some operation on iLO failed.

class ironic.drivers.modules.ilo.boot.**IloUefiHttpsBoot**(**args*, ***kwargs*)

Bases: *BootInterface*

capabilities = ['ramdisk_boot']

clean_up_instance(*task*)

Cleans up the boot of instance.

This method cleans up the environment that was setup for booting the instance.

Parameters
task A task from TaskManager.

Returns
None

`clean_up_ramdisk`(*task*)

Cleans up the boot of ironic ramdisk.

This method cleans up the environment that was setup for booting the deploy ramdisk.

Parameters
task A task from TaskManager.

Returns
None

`get_properties`()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

`prepare_instance`(*task*)

Prepares the boot of instance.

This method prepares the boot of the instance after reading relevant information from the nodes instance_info. It does the following depending on boot_option for deploy:

- If the boot_option requested for this deploy is local or image is a whole disk image, then it sets the node to boot from disk.
- Otherwise it finds/creates the boot ISO, sets the node boot option to UEFIHTTP and sets the URL as the boot ISO to boot the instance image.

Parameters
task a task from TaskManager.

Returns
None

Raises
IloOperationError, if some operation on iLO failed.

Raises
InstanceDeployFailure, if its try to boot iSCSI volume in BIOS boot mode.

`prepare_ramdisk`(*task*, *ramdisk_params*)

Prepares the boot of deploy ramdisk using UEFI-HTTPS boot.

This method prepares the boot of the deploy or rescue ramdisk after reading relevant information from the nodes driver_info and instance_info.

Parameters

- **task** a task from TaskManager.
- **ramdisk_params** the parameters to be passed to the ramdisk.

Returns
None

Raises
MissingParameterValue, if some information is missing in nodes driver_info or instance_info.

Raises
InvalidParameterValue, if some information provided is invalid.

Raises
IronicException, if some power or set boot boot device operation failed on the node.

Raises
IloOperationError, if some operation on iLO failed.

`validate`(*task*)

Validate the deployment information for the tasks node.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

`validate_inspection`(*task*)

Validate that the node has required properties for inspection.

Parameters
task A TaskManager instance with the node being checked

Raises
MissingParameterValue if node is missing one or more required parameters

Raises
UnsupportedDriverExtension

`validate_rescue`(*task*)

Validate that the node has required properties for rescue.

Parameters
task a TaskManager instance with the node being checked

Raises
MissingParameterValue if node is missing one or more required parameters

`class ironic.drivers.modules.ilo.boot.IloVirtualMediaBoot`(**args*, ***kwargs*)

Bases: *BootInterface*

`capabilities = ['iscsi_volume_boot', 'ramdisk_boot']`

clean_up_instance(*task*)

Cleans up the boot of instance.

This method cleans up the environment that was setup for booting the instance. It ejects virtual media. In case of UEFI iSCSI booting, it cleans up iSCSI target information from the node.

Parameters
task a task from TaskManager.

Returns
None

Raises
IloOperationError, if some operation on iLO failed.

clean_up_ramdisk(*task*)

Cleans up the boot of ironic ramdisk.

This method cleans up virtual media devices setup for the deploy or rescue ramdisk.

Parameters
task a task from TaskManager.

Returns
None

Raises
IloOperationError, if some operation on iLO failed.

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

prepare_instance(*task*)

Prepares the boot of instance.

This method prepares the boot of the instance after reading relevant information from the nodes instance_info. It does the following depending on boot_option for deploy:

- If the boot mode is uefi and its booting from volume, then it sets the iSCSI target info and node to boot from UefiTarget boot device.
- If not boot from volume and the boot_option requested for this deploy is local or image is a whole disk image, then it sets the node to boot from disk.
- Otherwise it finds/creates the boot ISO to boot the instance image, attaches the boot ISO to the bare metal and then sets the node to boot from CDROM.

Parameters
task a task from TaskManager.

Returns
None

Raises
IloOperationError, if some operation on iLO failed.

Raises
InstanceDeployFailure, if its try to boot iSCSI volume in BIOS boot mode.

`prepare_ramdisk`(*task*, *ramdisk_params*)

Prepares the boot of deploy ramdisk using virtual media.

This method prepares the boot of the deploy or rescue ramdisk after reading relevant information from the nodes driver_info and instance_info.

Parameters

- **task** a task from TaskManager.
- **ramdisk_params** the parameters to be passed to the ramdisk.

Returns
None

Raises
MissingParameterValue, if some information is missing in nodes driver_info or instance_info.

Raises
InvalidParameterValue, if some information provided is invalid.

Raises
IronicException, if some power or set boot boot device operation failed on the node.

Raises
IloOperationError, if some operation on iLO failed.

`validate`(*task*)

Validate the deployment information for the tasks node.

Parameters
task a TaskManager instance containing the node to act on.

Raises
InvalidParameterValue, if some information is invalid.

Raises
MissingParameterValue if kernel_id and ramdisk_id are missing in the Glance image or kernel and ramdisk not provided in instance_info for non-Glance image.

`validate_inspection`(*task*)

Validate that the node has required properties for inspection.

Parameters
task A TaskManager instance with the node being checked

Raises
MissingParameterValue if node is missing one or more required parameters

Raises
UnsupportedDriverExtension

validate_rescue(*task*)

Validate that the node has required properties for rescue.

Parameters
task a TaskManager instance with the node being checked

Raises
MissingParameterValue if node is missing one or more required parameters

class ironic.drivers.modules.ilo.boot.**IloiPXEBoot**(**args*, ***kwargs*)

Bases: *iPXEBoot*

clean_up_instance(*task*)

Cleans up the boot of instance.

This method cleans up the PXE environment that was setup for booting the instance. It unlinks the instance kernel/ramdisk in the nodes directory in tftproot and removes its PXE config. In case of UEFI iSCSI booting, it cleans up iSCSI target information from the node.

Parameters
task a task from TaskManager.

Returns
None

Raises
IloOperationError, if some operation on iLO failed.

prepare_instance(*task*)

Prepares the boot of instance.

This method prepares the boot of the instance after reading relevant information from the nodes instance_info. In case of localboot, it cleans up the PXE config. In case of boot from volume, it updates the iSCSI info onto iLO and sets the node to boot from UefiTarget boot device.

Parameters
task a task from TaskManager.

Returns
None

Raises
IloOperationError, if some operation on iLO failed.

prepare_ramdisk(*task*, *ramdisk_params*)

Prepares the boot of Ironic ramdisk using PXE.

This method prepares the boot of the deploy or rescue ramdisk after reading relevant information from the nodes driver_info and instance_info.

Parameters

- **task** a task from TaskManager.
- **ramdisk_params** the parameters to be passed to the ramdisk.

Returns
None

Raises
MissingParameterValue, if some information is missing in nodes driver_info or instance_info.

Raises
InvalidParameterValue, if some information provided is invalid.

Raises
IronicException, if some power or set boot boot device operation failed on the node.

Raises
IloOperationError, if some operation on iLO failed.

ironic.drivers.modules.ilo.boot.**parse_driver_info**(*node*, *mode='deploy'*)

Gets the driver specific Node deployment info.

This method validates whether the driver_info property of the supplied node contains the required information for this driver to deploy images to the node.

Parameters

- **node** a single Node.
- **mode** Label indicating a deploy or rescue operation being carried out on the node. Supported values are deploy and rescue. Defaults to deploy, indicating deploy operation is being carried out.

Returns
A dict with the driver_info values.

Raises
MissingParameterValue, if any of the required parameters are missing.

ironic.drivers.modules.ilo.boot.**prepare_node_for_deploy**(*task*)

Common preparatory steps for all iLO drivers.

This method performs common preparatory steps required for all drivers. 1. Power off node 2. Disables secure boot, if it is in enabled state. 3. Updates boot_mode capability to uefi if secure boot is requested. 4. Changes boot mode of the node if secure boot is disabled currently.

Parameters
task a TaskManager instance containing the node to act on.

Raises
IloOperationError, if some operation on iLO failed.

ironic.drivers.modules.ilo.common module

Common functionalities shared between different iLO modules.

ironic.drivers.modules.ilo.common.**POST_FINISHEDPOST_STATE = 'FinishedPost'**

Node is in FinishedPost post state.

ironic.drivers.modules.ilo.common.**POST_INPOSTDISCOVERY_STATE = 'InPostDiscoveryComplete'**

Node is in InPostDiscoveryComplete post state.

ironic.drivers.modules.ilo.common.**POST_INPOST_STATE = 'InPost'**

Node is in InPost post state.

ironic.drivers.modules.ilo.common.**POST_NULL_STATE = 'Null'**

Node is in Null post state.

ironic.drivers.modules.ilo.common.**POST_POWEROFF_STATE = 'PowerOff'**

Node is in PowerOff post state.

ironic.drivers.modules.ilo.common.**POST_RESET_STATE = 'Reset'**

Node is in Reset post state.

ironic.drivers.modules.ilo.common.**POST_UNKNOWN_STATE = 'Unknown'**

Node is in Unknown post state.

ironic.drivers.modules.ilo.common.**SUPPORTED_BOOT_MODE_LEGACY_BIOS_AND_UEFI = 'legacy bios and uefi'**

Node supports both legacy BIOS and UEFI boot mode.

ironic.drivers.modules.ilo.common.**SUPPORTED_BOOT_MODE_LEGACY_BIOS_ONLY = 'legacy bios only'**

Node supports only legacy BIOS boot mode.

ironic.drivers.modules.ilo.common.**SUPPORTED_BOOT_MODE_UEFI_ONLY = 'uefi only'**

Node supports only UEFI boot mode.

ironic.drivers.modules.ilo.common.**add_certificates**(*task*, *cert_file_list=None*)

Adds certificates to the node.

Adds certificates to the node based on the driver info provided.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **cert_file_list** List of certificates to be added to the node. If None, certificates from path configured in webserver_verify_ca will be added to the node.

Raises

IloOperationError on an error from IloClient library.

Raises

IloOperationNotSupported if retrieving post state is not supported on the server.

Raises

InvalidParameterValue, if any of the required parameters are invalid.

ironic.drivers.modules.ilo.common.**attach_vmedia**(*node*, *device*, *url*)

Attaches the given url as virtual media on the node.

Parameters

- **node** an ironic node object.
- **device** the virtual media device to attach
- **url** the http/https url to attach as the virtual media device

Raises

IloOperationError if insert virtual media failed.

ironic.drivers.modules.ilo.common.**cleanup_vmedia_boot**(*task*)

Cleans a node after a virtual media boot.

This method cleans up a node after a virtual media boot. It deletes the floppy image if it exists in CONF.ilo.swift_ilo_container or web server. It also ejects both virtual media cdrom and virtual media floppy.

Parameters
task a TaskManager instance containing the node to act on.

ironic.drivers.modules.ilo.common.**clear_certificates**(*task*, *cert_file_list=None*)

Clears any certificates added to the node.

Clears the certificates added to the node as part of any Ironic operation

Parameters

- **task** a TaskManager instance containing the node to act on.
- **cert_file_list** List of certificates to be removed from node. If None, all the certificates present on the node will be removed.

Raises
IloOperationError on an error from IloClient library.

Raises
IloOperationNotSupported if retrieving post state is not supported on the server.

ironic.drivers.modules.ilo.common.**copy_image_to_swift**(*source_file_path*, *destination_object_name*)

Uploads the given image to swift.

This method copies the given image to swift.

Parameters

- **source_file_path** The absolute path of the image file which needs to be copied to swift.
- **destination_object_name** The name of the object that will contain the copied image.

Raises
SwiftOperationError, if any operation with Swift fails.

Returns
temp url from swift after the source image is uploaded.

ironic.drivers.modules.ilo.common.**copy_image_to_web_server**(*source_file_path*, *destination*)

Copies the given image to the http web server.

This method copies the given image to the http_root location. It enables read-write access to the image else the deploy fails as the image file at the web_server url is inaccessible.

Parameters

- **source_file_path** The absolute path of the image file which needs to be copied to the web server root.
- **destination** The name of the file that will contain the copied image.

Raises
ImageUploadFailed exception if copying the source file to the web server fails.

Returns
image url after the source image is uploaded.

ironic.drivers.modules.ilo.common.**destroy_floppy_image_from_web_server**(*node*)

Removes the temporary floppy image.

It removes the floppy image created for deploy. :param node: an ironic node object.

ironic.drivers.modules.ilo.common.**download**(*target_file*, *file_url*)

Downloads file based on the scheme.

It downloads the file (url) to given location. The supported url schemes are file, http, and https. :param target_file: target file for copying the downloaded file. :param file_url: source file url from where file needs to be downloaded. :raises: ImageDownloadFailed, on failure to download the file.

ironic.drivers.modules.ilo.common.**eject_vmedia_devices**(*task*)

Ejects virtual media devices.

This method ejects virtual media floppy and cdrom.

Parameters
task a TaskManager instance containing the node to act on.

Returns
None

Raises
IloOperationError, if some error was encountered while trying to eject virtual media floppy or cdrom.

ironic.drivers.modules.ilo.common.**get_current_boot_mode**(*node*)

Get the current boot mode for a node.

Parameters
node an ironic node object.

Raises
IloOperationError if failed to fetch boot mode.

Raises
IloOperationNotSupported if node does not support getting pending boot mode.

ironic.drivers.modules.ilo.common.**get_ilo_object**(*node*)

Gets an IloClient object from proliantutils library.

Given an ironic node object, this method gives back a IloClient object to do operations on the iLO.

Parameters
node an ironic node object.

Returns
an IloClient object.

Raises
InvalidParameterValue on invalid inputs.

Raises
MissingParameterValue if some mandatory information is missing on the node

ironic.drivers.modules.ilo.common.**get_secure_boot_mode**(*task*)

Retrieves current enabled state of UEFI secure boot on the node

Returns the current enabled state of UEFI secure boot on the node.

Parameters
task a task from TaskManager.

Raises
MissingParameterValue if a required iLO parameter is missing.

Raises
IloOperationError on an error from IloClient library.

Raises
IloOperationNotSupported if UEFI secure boot is not supported.

Returns
Boolean value indicating current state of UEFI secure boot on the node.

ironic.drivers.modules.ilo.common.**get_server_post_state**(*node*)

Get the current state of system POST.

Parameters
node an ironic node object.

Returns
POST state of the server. The valida states are:- null, Unknown, Reset, PowerOff, InPost, InPostDiscoveryComplete and FinishedPost.

Raises
IloOperationError on an error from IloClient library.

Raises
IloOperationNotSupported if retrieving post state is not supported on the server.

ironic.drivers.modules.ilo.common.**parse_driver_info**(*node*)

Gets the driver specific Node info.

This method validates whether the driver_info property of the supplied node contains the required information for this driver.

Parameters
node an ironic Node object.

Returns
a dict containing information from driver_info (or where applicable, config values).

Raises
InvalidParameterValue if any parameters are incorrect

Raises
MissingParameterValue if some mandatory information is missing on the node

ironic.drivers.modules.ilo.common.**remove_image_from_swift**(*object_name*, *associated_with=None*)

Removes the given image from swift.

This method removes the given image name from swift. It deletes the image if it exists in CONF.ilo.swift_ilo_container

Parameters

- **object_name** The name of the object which needs to be removed from swift.
- **associated_with** string to depict the component/operation this object is associated to.

ironic.drivers.modules.ilo.common.**remove_image_from_web_server**(*object_name*)

Removes the given image from the configured web server.

This method removes the given image from the http_root location, if the image exists.

Parameters

object_name The name of the image file which needs to be removed from the web server root.

ironic.drivers.modules.ilo.common.**remove_single_or_list_of_files**(*file_location*)

Removes (deletes) the file or list of files.

This method only accepts single or list of files to delete. If single file is passed, this method removes (deletes) the file. If list of files is passed, this method removes (deletes) each of the files iteratively.

Parameters

file_location a single or a list of file paths

ironic.drivers.modules.ilo.common.**set_boot_mode**(*node*, *boot_mode*)

Sets the node to boot using boot_mode for the next boot.

Parameters

- **node** an ironic node object.
- **boot_mode** Next boot mode.

Raises

IloOperationError if setting boot mode failed.

ironic.drivers.modules.ilo.common.**set_secure_boot_mode**(*task*, *flag*)

Enable or disable UEFI Secure Boot for the next boot

Enable or disable UEFI Secure Boot for the next boot

Parameters

- **task** a task from TaskManager.
- **flag** Boolean value. True if the secure boot to be enabled in next boot.

Raises

IloOperationError on an error from IloClient library.

Raises

IloOperationNotSupported if UEFI secure boot is not supported.

ironic.drivers.modules.ilo.common.**setup_uefi_https**(*task*, *iso*, *persistent=False*)

Sets up system to boot from UEFIHTTP boot device.

Sets the one-time/persistent boot device to UEFIHTTP based on the argument supplied.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **iso** ISO URL to be set to boot from.
- **persistent** Indicates whether the system should be set to boot from the given device one-time or each time.

Raises

IloOperationError on an error from IloClient library.

Raises

IloOperationNotSupported if retrieving post state is not supported on the server.

ironic.drivers.modules.ilo.common.**setup_vmedia**(*task*, *iso*, *ramdisk_options=None*)

Attaches virtual media and sets it as boot device.

This method attaches the given bootable ISO as virtual media, prepares the arguments for ramdisk in virtual media floppy.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **iso** a bootable ISO image href to attach to. Should be either of below:
  - A Swift object - It should be of format `swift:<object-name>`. It is assumed that the image object is present in CONF.ilo.swift_ilo_container;
  - A Glance image - It should be format `glance://<glance-image-uuid>` or just `<glance-image-uuid>`;
  - An HTTP URL.
- **ramdisk_options** the options to be passed to the ramdisk in virtual media floppy.

Raises

ImageCreationFailed, if it failed while creating the floppy image.

Raises

IloOperationError, if some operation on iLO failed.

ironic.drivers.modules.ilo.common.**setup_vmedia_for_boot**(*task*, *boot_iso*, *parameters=None*)

Sets up the node to boot from the given ISO image.

This method attaches the given boot_iso on the node and passes the required parameters to it via virtual floppy image.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **boot_iso** a bootable ISO image to attach to. Should be either of below:

- A Swift object - It should be of format `swift:<object-name>`. It is assumed that the image object is present in CONF.ilo.swift_ilo_container;
- A Glance image - It should be format `glance://<glance-image-uuid>` or just `<glance-image-uuid>`;
- An HTTP URL.

- **parameters** the parameters to pass in the virtual floppy image in a dictionary. This is optional.

Raises
ImageCreationFailed, if it failed while creating the floppy image.

Raises
SwiftOperationError, if any operation with Swift fails.

Raises
IloOperationError, if attaching virtual media failed.

ironic.drivers.modules.ilo.common.**update_boot_mode**(*task*)

Update instance_info with boot mode to be used for deploy.

This method updates instance_info with boot mode to be used for deploy if node properties[capabilities] do not have boot_mode. It sets the boot mode on the node.

Parameters
task Task object.

Raises
IloOperationError if setting boot mode failed.

ironic.drivers.modules.ilo.common.**update_ipmi_properties**(*task*)

Update ipmi properties to node driver_info

Parameters
task a task from TaskManager.

ironic.drivers.modules.ilo.common.**update_redfish_properties**(*task*)

Update redfish properties to node driver_info

This method updates the nodes driver info with redfish driver driver_info. :param task: a task from TaskManager.

ironic.drivers.modules.ilo.common.**validate_security_parameter_values**(*sec_param_info*)

Validate security parameter with valid values.

Parameters
sec_param_info dict object containing the security parameter info

Raises
MissingParameterValue, for missing fields (or values) in security parameter info.

Raises
InvalidParameterValue, for unsupported security parameter

Returns
tuple of security param, ignore and enable parameters.

ironic.drivers.modules.ilo.common.**verify_image_checksum**(*image_location*, *expected_checksum*)

Verifies checksum of image file against the expected one.

This method generates the checksum of the image file on the fly and verifies it against the expected checksum provided as argument.

Parameters

- **image_location** location of image file whose checksum is verified.
- **expected_checksum** checksum to be checked against

Raises

ImageRefValidationFailed, if invalid file path or verification fails.

ironic.drivers.modules.ilo.firmware_processor module

Firmware file processor

class ironic.drivers.modules.ilo.firmware_processor.**FirmwareImageLocation**(*fw_image_location*, *fw_image_filename*)

Bases: object

Firmware image location class

This class acts as a wrapper class for the firmware image location. It primarily helps in removing the firmware files from their respective locations, made available for firmware update operation.

remove()

Exposed method to remove the wrapped firmware file

This method gets overridden by the remove method for the respective type of firmware file location it wraps.

class ironic.drivers.modules.ilo.firmware_processor.**FirmwareProcessor**(*url*)

Bases: object

Firmware file processor

This class helps in downloading the firmware file from url, extracting the firmware file (if its in compact format) and makes it ready for firmware update operation. In future, methods can be added as and when required to extend functionality for different firmware file types.

process_fw_on(*node*, *expected_checksum*)

Processes the firmware file from the url

This is the template method which downloads the firmware file from url, verifies checksum and extracts the firmware and makes it ready for firmware update operation. _download_fw_to method is set in the firmware processor object creation factory method, get_fw_processor(), based on the url type. :param node: a single Node. :param expected_checksum: checksum to be checked against. :returns: wrapper object of raw firmware image location :raises: IloOperationError, on failure to process firmware file. :raises: ImageDownloadFailed, on failure to download the original file. :raises: ImageRefValidationFailed, on failure to verify the checksum. :raises: SwiftOperationError, if upload to Swift fails. :raises: ImageUploadFailed, if upload to web server fails.

ironic.drivers.modules.ilo.firmware_processor.**get_and_validate_firmware_image_info**(*firmware_i*
firmware_u

Validates the firmware image info and returns the retrieved values.

Parameters
firmware_image_info dict object containing the firmware image info

Raises
MissingParameterValue, for missing fields (or values) in image info.

Raises
InvalidParameterValue, for unsupported firmware component

Returns
tuple of firmware url, checksum, component when the firmware update is ilo based.

ironic.drivers.modules.ilo.firmware_processor.**get_swift_url**(*parsed_url*)

Gets swift temp url.

It generates a temp url for the swift based firmware url to the target file. Expecting url as swift://containername/objectname.

Parameters
parsed_url Parsed url object.

Raises
SwiftOperationError, on failure to get url from swift.

ironic.drivers.modules.ilo.firmware_processor.**verify_firmware_update_args**(*func*)

Verifies the firmware update arguments.

ironic.drivers.modules.ilo.inspect module

iLO Inspect Interface

class ironic.drivers.modules.ilo.inspect.**IloInspect**(**args*, ***kwargs*)

Bases: *InspectInterface*

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

inspect_hardware(*task*)

Inspect hardware to get the hardware properties.

Inspects hardware to get the essential and additional hardware properties. It fails if any of the essential properties are not received from the node. It doesnt fail if node fails to return any capabilities as the capabilities differ from hardware to hardware mostly.

Parameters
task a TaskManager instance.

Raises
HardwareInspectionFailure if essential properties could not be retrieved successfully.

Raises
IloOperationError if system fails to get power state.

Returns
The resulting state of inspection.

`validate`(*task*)

Check that driver_info contains required ILO credentials.

Validates whether the driver_info property of the supplied tasks node contains the required credentials information.

Parameters
task a task from TaskManager.

Raises
InvalidParameterValue if required iLO parameters are not valid.

Raises
MissingParameterValue if a required parameter is missing.

ironic.drivers.modules.ilo.management module

iLO Management Interface

`class ironic.drivers.modules.ilo.management.`**Ilo5Management**(**args*, ***kwargs*)

Bases: *IloManagement*

`clear_ca_certificates`(*task*, *certificate_files*)

Clears the certificates provided in the list of files to iLO.

Parameters

- **task** a task from TaskManager.
- **certificate_files** a list of certificate files.

Raises
NodeCleaningFailure, on failure to execute of clean step.

Raises
InstanceDeployFailure, on failure to execute of deploy step.

`erase_devices`(*task*, ***kwargs*)

Erase all the drives on the node.

This method performs out-of-band sanitize disk erase on all the supported physical drives in the node. This erase cannot be performed on logical drives.

Parameters
task a TaskManager instance.

Raises
InvalidParameterValue, if any of the arguments are invalid.

Raises
IloError on an error from iLO.

one_button_secure_erase(*task*)

Erase the whole system securely.

The One-button secure erase process resets iLO and deletes all licenses stored there, resets BIOS settings, and deletes all Active Health System (AHS) and warranty data stored on the system. It also erases supported non-volatile storage data and deletes any deployment setting profiles.

Parameters
task a TaskManager instance.

Raises
IloError on an error from iLO.

class ironic.drivers.modules.ilo.management.**IloManagement**(**args*, ***kwargs*)

Bases: *ManagementInterface*

activate_license(*task*, ***kwargs*)

Activates iLO Advanced license.

Parameters
task a TaskManager object.

Raises
InvalidParameterValue, if any of the arguments are invalid.

Raises
NodeCleaningFailure, on failure to execute of clean step.

add_https_certificate(*task*, ***kwargs*)

Adds the signed HTTPS certificate to the iLO.

Parameters
task a TaskManager object.

clear_iscsi_boot_target(*task*)

Unset iSCSI details of the system in UEFI boot mode.

Parameters
task a task from TaskManager.

Raises
IloCommandNotSupportedInBiosError if system in BIOS boot mode.

Raises
IloError on an error from iLO.

clear_secure_boot_keys(*task*)

Clear all secure boot keys.

Clears all the secure boot keys. This operation is supported only on HP Proliant Gen9 and above servers.

Parameters
task a task from TaskManager.

Raises
NodeCleaningFailure, on failure to execute of clean step.

Raises
InstanceDeployFailure, on failure to execute of deploy step.

`create_csr`(*task*, ***kwargs*)

Creates the CSR.

Parameters
task a TaskManager object.

`flash_firmware_sum`(*task*, ***kwargs*)

Deploy step to Update the firmware using Smart Update Manager (SUM).

Parameters
task a TaskManager object.

Raises
InstanceDeployFailure, on failure to execute of deploy step.

Returns
states.DEPLOYWAIT to signify the step will be completed async

`get_boot_device`(*task*)

Get the current boot device for a node.

Returns the current boot device of the node.

Parameters
task a task from TaskManager.

Raises
MissingParameterValue if a required iLO parameter is missing.

Raises
IloOperationError on an error from IloClient library.

Returns

a dictionary containing:

boot_device
the boot device, one of the supported devices listed in *ironic.common.boot_devices* or None if it is unknown.

persistent
Whether the boot device will persist to all future boots or not, None if it is unknown.

`get_boot_mode`(*task*)

Get the current boot mode for a node.

Provides the current boot mode of the node.

Parameters
task A task from TaskManager.

Raises
IloOperationError on an error from IloClient library.

Returns
The boot mode, one of `ironic.common.boot_mode` or None if it is unknown.

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

get_secure_boot_state(*task*)

Get the current secure boot state for the node.

Parameters
task A task from TaskManager.

Raises
MissingParameterValue if a required parameter is missing

Raises
IloOperationError on an error from IloClient library.

Raises
UnsupportedDriverExtension if secure boot is not supported by the hardware

Returns
Boolean

get_sensors_data(*task*)

Get sensors data.

Parameters
task a TaskManager instance.

Raises
FailedToGetSensorData when getting the sensor data fails.

Raises
FailedToParseSensorData when parsing sensor data fails.

Raises
InvalidParameterValue if required ipmi parameters are missing.

Raises
MissingParameterValue if a required parameter is missing.

Returns
returns a dict of sensor data group by sensor type.

get_supported_boot_devices(*task*)

Get a list of the supported boot devices.

Parameters
task a task from TaskManager.

Returns
A list with the supported boot devices defined in *ironic.common.boot_devices*.

get_supported_boot_modes(*task*)

Get a list of the supported boot devices.

Parameters
task a task from TaskManager.

Raises
IloOperationError if any exception happens in proliantutils

Returns
A list with the supported boot devices defined in *ironic.common.boot_devices*.

inject_nmi(*task*)

Inject NMI, Non Maskable Interrupt.

Inject NMI (Non Maskable Interrupt) for a node immediately.

Parameters
task A TaskManager instance containing the node to act on.

Raises
IloCommandNotSupportedError if system does not support NMI injection.

Raises
IloError on an error from iLO.

Returns
None

reset_bios_to_default(*task*)

Resets the BIOS settings to default values.

Resets BIOS to default settings. This operation is currently supported only on HP Proliant Gen9 and above servers.

Parameters
task a task from TaskManager.

Raises
NodeCleaningFailure, on failure to execute of clean step.

Raises
InstanceDeployFailure, on failure to execute of deploy step.

reset_ilo(*task*)

Resets the iLO.

Parameters
task a task from TaskManager.

Raises
NodeCleaningFailure, on failure to execute of clean step.

Raises
InstanceDeployFailure, on failure to execute of deploy step.

reset_ilo_credential(*task*, *change_password=None*)

Resets the iLO password.

Parameters

- **task** a task from TaskManager.
- **change_password** Value for password to update on iLO.

Raises
NodeCleaningFailure, on failure to execute of clean step.

Raises
InstanceDeployFailure, on failure to execute of deploy step.

`reset_secure_boot_keys_to_default`(*task*)

Reset secure boot keys to manufacturing defaults.

Resets the secure boot keys to manufacturing defaults. This operation is supported only on HP Proliant Gen9 and above servers.

Parameters
task a task from TaskManager.

Raises
NodeCleaningFailure, on failure to execute of clean step.

Raises
InstanceDeployFailure, on failure to execute of deploy step.

`security_parameters_update`(*task*, ***kwargs*)

Updates the security parameters.

Parameters
task a TaskManager object.

`set_boot_device`(*task*, *device*, *persistent=False*)

Set the boot device for a node.

Set the boot device to use on next reboot of the node.

Parameters

- **task** a task from TaskManager.
- **device** the boot device, one of the supported devices listed in *ironic.common.boot_devices*.
- **persistent** Boolean value. True if the boot device will persist to all future boots, False if not. Default: False.

Raises
InvalidParameterValue if an invalid boot device is specified.

Raises
MissingParameterValue if a required parameter is missing.

Raises
IloOperationError on an error from IloClient library.

`set_boot_mode`(*task*, *mode*)

Set the boot mode for a node.

Set the boot mode to use on next reboot of the node.

Parameters

- **task** A task from TaskManager.
- **mode** The boot mode, one of *ironic.common.boot_modes*.

Raises
InvalidParameterValue if an invalid boot mode is specified.

Raises
IloOperationError if setting boot mode failed.

`set_iscsi_boot_target`(*task*)

Set iSCSI details of the system in UEFI boot mode.

The initiator is set with the target details like IQN, LUN, IP, Port etc. :param task: a task from TaskManager. :raises: MissingParameterValue if a required parameter is missing. :raises: IloCommandNotSupportedInBiosError if system in BIOS boot mode. :raises: IloError on an error from iLO.

`set_secure_boot_state`(*task*, *state*)

Set the current secure boot state for the node.

Parameters

- **task** A task from TaskManager.
- **state** A new state as a boolean.

Raises
MissingParameterValue if a required parameter is missing

Raises
IloOperationError on an error from IloClient library.

Raises
UnsupportedDriverExtension if secure boot is not supported by the hardware

`update_auth_failure_logging_threshold`(*task*, ***kwargs*)

Updates the Auth Failure Logging Threshold security parameter.

Parameters
task a TaskManager object.

`update_firmware`(*task*, ***kwargs*)

Updates the firmware.

Parameters
task a TaskManager object.

Raises
InvalidParameterValue if update firmware mode is not ilo. Even applicable for invalid input cases.

Raises
NodeCleaningFailure, on failure to execute of clean step.

Raises
InstanceDeployFailure, on failure to execute of deploy step.

`update_firmware_sum`(*task*, ***kwargs*)

Clean step to update the firmware using Smart Update Manager (SUM)

Parameters
task a TaskManager object.

Raises
NodeCleaningFailure, on failure to execute of clean step.

Returns
states.CLEANWAIT to signify the step will be completed async

`update_minimum_password_length`(*task*, ***kwargs*)

Updates the Minimum Password Length security parameter.

Parameters
task a TaskManager object.

`validate`(*task*)

Check that driver_info contains required ILO credentials.

Validates whether the driver_info property of the supplied tasks node contains the required credentials information.

Parameters
task a task from TaskManager.

Raises
InvalidParameterValue if required iLO parameters are not valid.

Raises
MissingParameterValue if a required parameter is missing.

ironic.drivers.modules.ilo.power module

iLO Power Driver

`class ironic.drivers.modules.ilo.power.IloPower`(**args*, ***kwargs*)

Bases: *PowerInterface*

`get_power_state`(*task*)

Gets the current power state.

Parameters

- **task** a TaskManager instance.
- **node** The Node.

Returns
one of *ironic.common.states* POWER_OFF, POWER_ON or ERROR.

Raises
InvalidParameterValue if required iLO credentials are missing.

Raises
IloOperationError on an error from IloClient library.

`get_properties`()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

get_supported_power_states(*task*)

Get a list of the supported power states.

Parameters

task A TaskManager instance containing the node to act on. currently not used.

Returns

A list with the supported power states defined in *ironic.common.states*.

reboot(*task*, *timeout=None*)

Reboot the node

Parameters

- **task** a TaskManager instance.
- **timeout** timeout (in seconds). Unsupported by this interface.

Raises

PowerStateFailure if the final state of the node is not POWER_ON.

Raises

IloOperationError on an error from IloClient library.

set_power_state(*task*, *power_state*, *timeout=None*)

Turn the current power state on or off.

Parameters

- **task** a TaskManager instance.
- **power_state** The desired power state POWER_ON,POWER_OFF or REBOOT from *ironic.common.states*.
- **timeout** timeout (in seconds). Unsupported by this interface.

Raises

InvalidParameterValue if an invalid power state was specified.

Raises

IloOperationError on an error from IloClient library.

Raises

PowerStateFailure if the power couldnt be set to power_state.

validate(*task*)

Check if node.driver_info contains the required iLO credentials.

Parameters

- **task** a TaskManager instance.
- **node** Single node object.

Raises

InvalidParameterValue if required iLO credentials are missing.

ironic.drivers.modules.ilo.raid module

iLO5 RAID specific methods

class ironic.drivers.modules.ilo.raid.**Ilo5RAID**(**args*, ***kwargs*)

Bases: *RAIDInterface*

Implementation of OOB RAIDInterface for iLO5.

apply_configuration(*task*, *raid_config*, *create_root_volume=True*, *create_nonroot_volumes=False*)

Applies RAID configuration on the given node.

Parameters

- **task** A TaskManager instance.
- **raid_config** The RAID configuration to apply.
- **create_root_volume** Setting this to False indicates not to create root volume that is specified in raid_config. Default value is True.
- **create_nonroot_volumes** Setting this to False indicates not to create non-root volumes (all except the root volume) in raid_config. Default value is True.
- **delete_existing** Setting this to True indicates to delete RAID configuration prior to creating the new configuration.

Raises

InvalidParameterValue, if the RAID configuration is invalid.

Returns

states.DEPLOYWAIT if RAID configuration is in progress asynchronously or None if it is complete.

create_configuration(*task*, *create_root_volume=True*, *create_nonroot_volumes=True*)

Create a RAID configuration on a bare metal using agent ramdisk.

This method creates a RAID configuration on the given node.

Parameters

- **task** a TaskManager instance.
- **create_root_volume** If True, a root volume is created during RAID configuration. Otherwise, no root volume is created. Default is True.
- **create_nonroot_volumes** If True, non-root volumes are created. If False, no non-root volumes are created. Default is True.

Raises

MissingParameterValue, if node.target_raid_config is missing or was found to be empty after skipping root volume and/or non-root volumes.

Raises

NodeCleaningFailure, on failure to execute clean step.

Raises

InstanceDeployFailure, on failure to execute deploy step.

delete_configuration(*task*)

Delete the RAID configuration.

Parameters
task a TaskManager instance containing the node to act on.

Raises
NodeCleaningFailure, on failure to execute clean step.

Raises
InstanceDeployFailure, on failure to execute deploy step.

get_properties()

Return the properties of the interface.

ironic.drivers.modules.ilo.vendor module

Vendor Interface for iLO drivers and its supporting methods.

class ironic.drivers.modules.ilo.vendor.**VendorPassthru**(**args*, ***kwargs*)

Bases: *RedfishVendorPassthru*

Vendor-specific interfaces for iLO deploy drivers.

boot_into_iso(*task*, ***kwargs*)

Attaches an ISO image in glance and reboots bare metal.

This method accepts an ISO image href (a Glance UUID or an HTTP(S) URL) attaches it as virtual media and then reboots the node. This is useful for debugging purposes. This can be invoked only when the node is in manage state.

Parameters

- **task** A TaskManager object.
- **kwargs** The arguments sent with vendor passthru. The expected kwargs are:

```
'boot_iso_href': href of the image to be booted. This␣
↪can be
    a Glance UUID or an HTTP(S) URL.
```

validate(*task*, *method*, ***kwargs*)

Validate vendor-specific actions.

Checks if a valid vendor passthru method was passed and validates the parameters for the vendor passthru method.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **method** method to be validated.
- **kwargs** kwargs containing the vendor passthru methods parameters.

Raises
MissingParameterValue, if some required parameters were not passed.

Raises
InvalidParameterValue, if any of the parameters have invalid value.

Raises

IloOperationNotSupported, if the driver does not support the given operation with ilo vendor interface.

Module contents

ironic.drivers.modules.inspector package

Subpackages

ironic.drivers.modules.inspector.hooks package

Submodules

ironic.drivers.modules.inspector.hooks.accelerators module

class ironic.drivers.modules.inspector.hooks.accelerators.**AcceleratorsHook**

Bases: *InspectionHook*

Hook to set the nodes accelerators property.

__call__(*task*, *inventory*, *plugin_data*)

Hook to run to process inspection data (before Ironic node update).

This hook is run after node is found and ports are created, just before the node is updated with the data.

Parameters

- **task** A TaskManager instance.
- **inventory** Hardware inventory information sent by the ramdisk. Must not be modified by the hook.
- **plugin_data** Plugin data sent by the ramdisk. May be modified by the hook.

Returns

nothing.

ironic.drivers.modules.inspector.hooks.architecture module

class ironic.drivers.modules.inspector.hooks.architecture.**ArchitectureHook**

Bases: *InspectionHook*

Hook to set the nodes cpu_arch property based on the inventory.

__call__(*task*, *inventory*, *plugin_data*)

Update node properties with CPU architecture.

ironic.drivers.modules.inspector.hooks.base module

Base code for inspection hooks support.

class ironic.drivers.modules.inspector.hooks.base.**InspectionHook**

Bases: `object`

Abstract base class for inspection hooks.

__call__(*task*, *inventory*, *plugin_data*)

Hook to run to process inspection data (before Ironic node update).

This hook is run after node is found and ports are created, just before the node is updated with the data.

Parameters

- **task** A TaskManager instance.
- **inventory** Hardware inventory information sent by the ramdisk. Must not be modified by the hook.
- **plugin_data** Plugin data sent by the ramdisk. May be modified by the hook.

Returns

nothing.

dependencies = []

An ordered list of hooks that must be enabled before this one.

The items here should be entry point names, not classes.

preprocess(*task*, *inventory*, *plugin_data*)

Hook to run before the main inspection data processing.

This hook is run even before sanity checks.

Parameters

- **task** A TaskManager instance.
- **inventory** Hardware inventory information sent by the ramdisk. Must not be modified by the hook.
- **plugin_data** Plugin data sent by the ramdisk. May be modified by the hook.

Returns

nothing.

ironic.drivers.modules.inspector.hooks.boot_mode module

class ironic.drivers.modules.inspector.hooks.boot_mode.**BootModeHook**

Bases: *InspectionHook*

Hook to set the nodes boot_mode capability in node properties.

__call__(*task*, *inventory*, *plugin_data*)

Hook to run to process inspection data (before Ironic node update).

This hook is run after node is found and ports are created, just before the node is updated with the data.

Parameters

- **task** A TaskManager instance.
- **inventory** Hardware inventory information sent by the ramdisk. Must not be modified by the hook.

- **plugin_data** Plugin data sent by the ramdisk. May be modified by the hook.

Returns
nothing.

ironic.drivers.modules.inspector.hooks.cpu_capabilities module

class ironic.drivers.modules.inspector.hooks.cpu_capabilities.**CPUCapabilitiesHook**

Bases: *InspectionHook*

Hook to set nodes capabilities based on cpu flags in the inventory.

__call__(*task*, *inventory*, *plugin_data*)

Hook to run to process inspection data (before Ironic node update).

This hook is run after node is found and ports are created, just before the node is updated with the data.

Parameters

- **task** A TaskManager instance.
- **inventory** Hardware inventory information sent by the ramdisk. Must not be modified by the hook.
- **plugin_data** Plugin data sent by the ramdisk. May be modified by the hook.

Returns
nothing.

ironic.drivers.modules.inspector.hooks.cpus module

class ironic.drivers.modules.inspector.hooks.cpus.**CPUSHook**

Bases: *InspectionHook*

Hook to set nodes CPUs count based on the inventory.

__call__(*task*, *inventory*, *plugin_data*)

Hook to run to process inspection data (before Ironic node update).

This hook is run after node is found and ports are created, just before the node is updated with the data.

Parameters

- **task** A TaskManager instance.
- **inventory** Hardware inventory information sent by the ramdisk. Must not be modified by the hook.
- **plugin_data** Plugin data sent by the ramdisk. May be modified by the hook.

Returns
nothing.

ironic.drivers.modules.inspector.hooks.extra_hardware module

class ironic.drivers.modules.inspector.hooks.extra_hardware.**ExtraHardwareHook**

Bases: *InspectionHook*

Hook to gather extra information about the node hardware.

__call__(*task*, *inventory*, *plugin_data*)

Store extra hardware information in plugin_data[extra]

Convert the extra collected data from the format of the hardware-detect tool (list of lists) to a nested dictionary. Remove the original `data` field from plugin_data, and save the converted data into a new field `extra` instead.

ironic.drivers.modules.inspector.hooks.local_link_connection module

class ironic.drivers.modules.inspector.hooks.local_link_connection.**LocalLinkConnectionHook**

Bases: *InspectionHook*

Hook to process mandatory LLDP packet fields

__call__(*task*, *inventory*, *plugin_data*)

Process LLDP data and patch Ironic port local link connection.

Process the non-vendor-specific LLDP packet fields for each NIC found for a baremetal node, port ID and chassis ID. These fields, if found and if valid, will be saved into the local link connection information (port id and switch id) fields on the Ironic port that represents that NIC.

dependencies = ['validate-interfaces']

An ordered list of hooks that must be enabled before this one.

The items here should be entry point names, not classes.

ironic.drivers.modules.inspector.hooks.memory module

class ironic.drivers.modules.inspector.hooks.memory.**MemoryHook**

Bases: *InspectionHook*

Hook to set the nodes memory_mb property based on the inventory.

__call__(*task*, *inventory*, *plugin_data*)

Update node properties with memory information.

ironic.drivers.modules.inspector.hooks.parse_lldp module

LLDP Processing Hook for basic TLVs

class ironic.drivers.modules.inspector.hooks.parse_lldp.**ParseLLDPHook**

Bases: *InspectionHook*

Process LLDP packet fields and store them in plugin_data[parsed_lldp]

Convert binary LLDP information into a readable form. Loop through raw LLDP TLVs and parse those from the basic management, 802.1, and 802.3 TLV sets. Store parsed data in the plugin_data as a new parsed_lldp dictionary with interface names as keys.

__call__(*task*, *inventory*, *plugin_data*)

Process LLDP data and update plugin_data with processed data

ironic.drivers.modules.inspector.hooks.pci_devices module

Gather and distinguish PCI devices from plugin_data.

class ironic.drivers.modules.inspector.hooks.pci_devices.**PciDevicesHook**

Bases: *InspectionHook*

Hook to count various PCI devices, and set the nodes capabilities.

This information can later be used by nova for node scheduling.

__call__(*task*, *inventory*, *plugin_data*)

Update node capabilities with PCI devices.

ironic.drivers.modules.inspector.hooks.physical_network module

Port Physical Network Hook

class ironic.drivers.modules.inspector.hooks.physical_network.**PhysicalNetworkHook**

Bases: *InspectionHook*

Hook to set the ports physical_network field.

Set the ironic ports physical_network field based on a CIDR to physical network mapping in the configuration.

__call__(*task*, *inventory*, *plugin_data*)

Process inspection data and patch the ports physical network.

dependencies = ['validate-interfaces']

An ordered list of hooks that must be enabled before this one.

The items here should be entry point names, not classes.

get_physical_network(*interface*)

Return a physical network to apply to an ironic port.

Parameters

interface The interface from the inventory.

Returns

The physical network to set, or None.

ironic.drivers.modules.inspector.hooks.ports module

class ironic.drivers.modules.inspector.hooks.ports.**PortsHook**

Bases: *InspectionHook*

Hook to create ironic ports.

__call__(*task*, *inventory*, *plugin_data*)

Hook to run to process inspection data (before Ironic node update).

This hook is run after node is found and ports are created, just before the node is updated with the data.

Parameters

- **task** A TaskManager instance.
- **inventory** Hardware inventory information sent by the ramdisk. Must not be modified by the hook.
- **plugin_data** Plugin data sent by the ramdisk. May be modified by the hook.

Returns
nothing.

dependencies = ['validate-interfaces']

An ordered list of hooks that must be enabled before this one.

The items here should be entry point names, not classes.

ironic.drivers.modules.inspector.hooks.ports.**add_ports**(*task*, *interfaces*)

Add ports for all previously validated interfaces.

ironic.drivers.modules.inspector.hooks.ports.**update_ports**(*task*, *all_interfaces*, *valid_macs*)

Update ports to match the valid MACs.

Depending on the value of `[inspector]keep_ports`, some ports may be removed.

ironic.drivers.modules.inspector.hooks.raid_device module

class ironic.drivers.modules.inspector.hooks.raid_device.**RaidDeviceHook**

Bases: *InspectionHook*

Hook for learning the root device after RAID creation.

This hook can figure out the root device in 2 runs. In the first run, the nodes inventory is saved as usual, and the hook does not do anything. The second run will check the difference between the recently discovered block devices (as reported by the inspection results) and the previously saved ones (from the previously saved inventory). If there is exactly one new block device, its serial number is saved in node.properties under the root_device key.

This way, it helps to figure out the root device hint in cases when Ironic doesnt have enough information to do so otherwise. One such usecase is DRAC RAID configuration, where the BMC doesnt provide any useful information about the created RAID disks. Using this hook immediately before and after creating the root RAID device will solve the issue of root device hints.

__call__(*task*, *inventory*, *plugin_data*)

Hook to run to process inspection data (before Ironic node update).

This hook is run after node is found and ports are created, just before the node is updated with the data.

Parameters

- **task** A TaskManager instance.
- **inventory** Hardware inventory information sent by the ramdisk. Must not be modified by the hook.
- **plugin_data** Plugin data sent by the ramdisk. May be modified by the hook.

Returns
nothing.

ironic.drivers.modules.inspector.hooks.ramdisk_error module

class ironic.drivers.modules.inspector.hooks.ramdisk_error.**RamdiskErrorHook**

Bases: *InspectionHook*

Hook to process error sent from the ramdisk.

__call__(*task*, *inventory*, *plugin_data*)

Hook to run to process inspection data (before Ironic node update).

This hook is run after node is found and ports are created, just before the node is updated with the data.

Parameters

- **task** A TaskManager instance.
- **inventory** Hardware inventory information sent by the ramdisk. Must not be modified by the hook.
- **plugin_data** Plugin data sent by the ramdisk. May be modified by the hook.

Returns
nothing.

preprocess(*task*, *inventory*, *plugin_data*)

Hook to run before the main inspection data processing.

This hook is run even before sanity checks.

Parameters

- **task** A TaskManager instance.
- **inventory** Hardware inventory information sent by the ramdisk. Must not be modified by the hook.
- **plugin_data** Plugin data sent by the ramdisk. May be modified by the hook.

Returns
nothing.

ironic.drivers.modules.inspector.hooks.root_device module

class ironic.drivers.modules.inspector.hooks.root_device.**RootDeviceHook**

Bases: *InspectionHook*

Smarter root disk selection using Ironic root device hints.

__call__(*task*, *inventory*, *plugin_data*)

Process root disk information.

ironic.drivers.modules.inspector.hooks.validate_interfaces module

class ironic.drivers.modules.inspector.hooks.validate_interfaces.**ValidateInterfacesHook**

Bases: *InspectionHook*

Hook to validate network interfaces.

__call__(*task*, *inventory*, *plugin_data*)

Hook to run to process inspection data (before Ironic node update).

This hook is run after node is found and ports are created, just before the node is updated with the data.

Parameters

- **task** A TaskManager instance.
- **inventory** Hardware inventory information sent by the ramdisk. Must not be modified by the hook.
- **plugin_data** Plugin data sent by the ramdisk. May be modified by the hook.

Returns

nothing.

preprocess(*task*, *inventory*, *plugin_data*)

Hook to run before the main inspection data processing.

This hook is run even before sanity checks.

Parameters

- **task** A TaskManager instance.
- **inventory** Hardware inventory information sent by the ramdisk. Must not be modified by the hook.
- **plugin_data** Plugin data sent by the ramdisk. May be modified by the hook.

Returns

nothing.

ironic.drivers.modules.inspector.hooks.validate_interfaces.**get_interfaces**(*node*, *inventory*)

Convert inventory to a dict with interfaces.

Returns
dict interface name -> interface (for valid interfaces).

ironic.drivers.modules.inspector.hooks.validate_interfaces.**get_pxe_mac**(*inventory*)
Get MAC address of the PXE interface.

ironic.drivers.modules.inspector.hooks.validate_interfaces.**validate_interfaces**(*node*, *inventory*, *interfaces*)
Validate interfaces on correctness and suitability.

Returns
dict interface name -> interface.

Module contents

Submodules

ironic.drivers.modules.inspector.agent module

In-band inspection implementation.

class ironic.drivers.modules.inspector.agent.**AgentInspect**(**args*, ***kwargs*)
Bases: *Common*

In-band inspection.

abort(*task*)
Abort hardware inspection.

Parameters
task a task from TaskManager.

continue_inspection(*task*, *inventory*, *plugin_data*)
Continue in-band hardware inspection.

Parameters

- **task** a task from TaskManager.
- **inventory** hardware inventory from the node.
- **plugin_data** optional plugin-specific data.

ironic.drivers.modules.inspector.interface module

class ironic.drivers.modules.inspector.interface.**Common**(**args*, ***kwargs*)
Bases: *InspectInterface*

get_properties()
Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

inspect_hardware(*task*)

Inspect hardware to obtain the hardware properties.

Results will be checked in a periodic task.

Parameters
task a task from TaskManager.

Returns
states.INSPECTWAIT

Raises
HardwareInspectionFailure on failure

validate(*task*)

Validate the driver-specific inspection information.

If invalid, raises an exception; otherwise returns None.

Parameters
task a task from TaskManager.

Raises
UnsupportedDriverExtension

ironic.drivers.modules.inspector.interface.**clean_up**(*task*, *finish=True*, *always_power_off=False*)

ironic.drivers.modules.inspector.interface.**inspection_error_handler**(*task*, *error*, *raise_exc=False*, *clean_up=True*)

ironic.drivers.modules.inspector.interface.**ironic_manages_boot**(*task*, *raise_exc=False*)

Whether ironic should manage boot for this node.

ironic.drivers.modules.inspector.interface.**prepare_managed_inspection**(*task*, *endpoint*)

Prepare the boot interface for managed inspection.

ironic.drivers.modules.inspector.interface.**tear_down_managed_boot**(*task*, *always_power_off=False*)

ironic.drivers.modules.inspector.lldp_parsers module

Names and mapping functions used to map LLDP TLVs to name/value pairs

class ironic.drivers.modules.inspector.lldp_parsers.**LLDPBasicMgmtParser**(*nv=None*)

Bases: *LLDPParser*

Class to handle parsing of 802.1AB Basic Management set

This class will also handle 802.1Q and 802.3 OUI TLVs.

add_capabilities(*struct*, *name*, *data*)

Handle LLDP_TLV_SYS_CAPABILITIES

add_mgmt_address(*struct*, *name*, *data*)

Handle LLDP_TLV_MGMT_ADDRESS

There can be multiple Mgmt Address TLVs, store in list.

handle_org_specific_tlv(*struct*, *name*, *data*)

Handle Organizationally Unique ID TLVs

This class supports 802.1Q and 802.3 OUI TLVs.

See http://www.ieee802.org/1/pages/802.1Q-2014.html, Annex D and http://standards.ieee.org/about/get/802/802.3.html

class ironic.drivers.modules.inspector.lldp_parsers.**LLDPParser**(*node_uuid*, *nv=None*)

Bases: object

Base class to handle parsing of LLDP TLVs

Each class that inherits from this base class must provide a parser map. Parser maps are used to associate a LLDP TLV with a function handler and arguments necessary to parse the TLV and generate one or more name/value pairs. Each LLDP TLV maps to a tuple with the following fields:

function - Handler function to generate name/value pairs

construct - Name of construct definition for TLV

name - User-friendly name of TLV. For TLVs that generate only one name/value pair, this is the name used

len_check - Boolean indicating if length check should be done on construct

It is valid to have a function handler of None, this is for TLVs that are not mapped to a name/value pair (e.g.LLDP_TLV_TTL).

add_dot1_link_aggregation(*struct*, *name*, *data*)

Add name/value pairs for TLV Dot1_LinkAggregationId

This is in the base class since it can be used by both dot1 and dot3.

add_nested_value(*struct*, *name*, *data*)

Add a single nested name/value pair to the dictionary

add_single_value(*struct*, *name*, *data*)

Add a single name/value pair to the nv dictionary

append_value(*name*, *value*)

Add value to a list mapped to name

parse_tlv(*tlv_type*, *data*)

Parse TLVs from mapping table

This functions takes the TLV type and the raw data for this TLV and gets a tuple from the parser_map. The construct field in the tuple contains the construct lib definition of the TLV which can be parsed to access individual fields. Once the TLV is parsed, the handler function for each TLV will store the individual fields as name/value pairs in nv_dict.

If the handler function does not exist, then no name/value pairs will be added to nv_dict, but since the TLV was handled, True will be returned.

Param
tlv_type - type identifier for TLV

Param
data - raw TLV value

Returns
True if TLV in parser_map and data is valid, otherwise False.

set_value(*name*, *value*)

Set name value pair in dictionary

The value for a name should not be changed if it exists.

class ironic.drivers.modules.inspector.lldp_parsers.**LLDPdot1Parser**(*node_uuid*, *nv=None*)

Bases: *LLDPParser*

Class to handle parsing of 802.1Q TLVs

add_dot1_port_protocol_vlan(*struct*, *name*, *data*)

Handle dot1_PORT_PROTOCOL_VLANID

add_dot1_protocol_identities(*struct*, *name*, *data*)

Handle dot1_PROTOCOL_IDENTITY

There can be multiple protocol ids TLVs, store in list

add_dot1_vlans(*struct*, *name*, *data*)

Handle dot1_VLAN_NAME

There can be multiple VLAN TLVs, add dictionary entry with id/vlan to list.

class ironic.drivers.modules.inspector.lldp_parsers.**LLDPdot3Parser**(*node_uuid*, *nv=None*)

Bases: *LLDPParser*

Class to handle parsing of 802.3 TLVs

add_dot3_macphy_config(*struct*, *name*, *data*)

Handle dot3_MACPHY_CONFIG_STATUS

ironic.drivers.modules.inspector.lldp_tlvs module

Link Layer Discovery Protocol TLVs

ironic.drivers.modules.inspector.lldp_tlvs.**bytes_to_int**(*obj*)

Convert bytes to an integer

Param
obj - array of bytes

ironic.drivers.modules.inspector.lldp_tlvs.**get_autoneg_cap**(*pmd*)

Get autonegotiated capability strings

This returns a list of capability strings from the Physical Media Dependent (PMD) capability bits.

Parameters
pmd PMD bits

Returns
Sorted list containing capability strings

ironic.drivers.modules.inspector.lldp_tlvs.**mapping_for_enum**(*mapping*)

Return tuple used for keys as a dict

Param
mapping - dict with tuple as keys

ironic.drivers.modules.inspector.lldp_tlvs.**mapping_for_switch**(*mapping*)

Return dict from values

Param
mapping - dict with tuple as keys

Module contents

class ironic.drivers.modules.inspector.**AgentInspect**(**args*, ***kwargs*)

Bases: *Common*

In-band inspection.

abort(*task*)

Abort hardware inspection.

Parameters
task a task from TaskManager.

continue_inspection(*task*, *inventory*, *plugin_data*)

Continue in-band hardware inspection.

Parameters

- **task** a task from TaskManager.
- **inventory** hardware inventory from the node.
- **plugin_data** optional plugin-specific data.

ironic.drivers.modules.intel_ipmi package

Submodules

ironic.drivers.modules.intel_ipmi.management module

Intel IPMI Hardware.

Supports Intel Speed Select Performance Profile.

class ironic.drivers.modules.intel_ipmi.management.**IntelIPMIManagement**(**args*, ***kwargs*)

Bases: *IPMIManagement*

configure_intel_speedselect(*task*, ***kwargs*)

Module contents

ironic.drivers.modules.network package

Submodules

ironic.drivers.modules.network.common module

class ironic.drivers.modules.network.common.**NetType**(*value*, *names=<not given>*, **values*, *module=None*, *qualname=None*, *type=None*, *start=1*, *boundary=None*)

Bases: Enum

CLEANING = 'cleaning'

INSPECTION = 'inspection'

PROVISIONING = 'provisioning'

RESCUING = 'rescuing'

SERVICING = 'servicing'

TENANT = 'tenant'

property vif_key

class ironic.drivers.modules.network.common.**NeutronVIFPortIDMixin**

Bases: *VIFPortIDMixin*

VIF port ID mixin class for neutron network interfaces.

Mixin class that provides VIF-related network interface methods for neutron network interfaces. On VIF attach/detach, the associated neutron port will be updated.

get_node_network_data(*task*)

Get network configuration data for node ports.

Pull network data from ironic node object if present, otherwise collect it for Neutron VIFs. The intended use of this method is to generate data *for* virtual media based deployments so a minimal configuration can be injected into the ramdisk.

Parameters
task A TaskManager instance.

Raises
InvalidParameterValue, if the network interface configuration is invalid.

Raises
MissingParameterValue, if some parameters are missing.

Returns
a dict holding network configuration information adhering Nova network metadata layout (*network_data.json*).

port_changed(*task*, *port_obj*)

Handle any actions required when a port changes

Parameters

- **task** a TaskManager instance.
- **port_obj** a changed Port object from the API before it is saved to database.

Raises

FailedToUpdateDHCPOptOnPort, Conflict

portgroup_changed(*task*, *portgroup_obj*)

Handle any actions required when a portgroup changes

Parameters

- **task** a TaskManager instance.
- **portgroup_obj** a changed Portgroup object from the API before it is saved to database.

Raises

FailedToUpdateDHCPOptOnPort, Conflict

vif_attach(*task*, *vif_info*)

Attach a virtual network interface to a node

Attach a virtual interface to a node. When selecting a port or portgroup to attach the virtual interface to, the following ordered criteria are applied:

- Require ports or portgroups to have a physical network that is either None or one of the VIFs allowed physical networks.
- Prefer ports or portgroups with a physical network field which is not None.
- Prefer portgroups to ports.
- Prefer ports with PXE enabled.

Parameters

- **task** A TaskManager instance.
- **vif_info** a dictionary of information about a VIF. It must have an id key, whose value is a unique identifier for that VIF.

Raises

NetworkError, VifAlreadyAttached, NoFreePhysicalPorts

Raises

PortgroupPhysnetInconsistent if one of the nodes portgroups has ports which are not all assigned the same physical network.

vif_detach(*task*, *vif_id*)

Detach a virtual network interface from a node

Parameters

- **task** A TaskManager instance.

- **vif_id** A VIF ID to detach

Raises
VifNotAttached if VIF not attached.

Raises
NetworkError if unbind Neutron port failed.

class ironic.drivers.modules.network.common.**VIFPortIDMixin**

Bases: object

VIF port ID mixin class for non-neutron network interfaces.

Mixin class that provides VIF-related network interface methods for non-neutron network interfaces. There are no effects due to VIF attach/detach that are external to ironic.

NOTE: This does not yet support the full set of VIF methods, as it does not provide vif_attach, vif_detach, port_changed, or portgroup_changed.

get_current_vif(*task*, *p_obj*)

Returns the currently used VIF associated with port or portgroup

We are booting the node only in one network at a time, and presence of cleaning_vif_port_id means were doing cleaning, of provisioning_vif_port_id - provisioning, of rescuing_vif_port_id - rescuing. Otherwise its a tenant network

Parameters

- **task** A TaskManager instance.
- **p_obj** Ironic port or portgroup object.

Returns
VIF ID associated with p_obj or None.

vif_list(*task*)

List attached VIF IDs for a node

Parameters
task A TaskManager instance.

Returns
List of VIF dictionaries, each dictionary will have an id entry with the ID of the VIF.

ironic.drivers.modules.network.common.**get_free_port_like_object**(*task*, *vif_id*, *physnets*, *vif_info={}*)

Find free port-like object (portgroup or port) VIF will be attached to.

Ensures that the VIF is not already attached to this node. When selecting a port or portgroup to attach the virtual interface to, the following ordered criteria are applied:

- Require ports or portgroups to have a physical network that is either None or one of the VIFs allowed physical networks.
- Prefer ports or portgroups with a physical network field which is not None.
- Prefer portgroups to ports.
- Prefer ports with PXE enabled.

Parameters

- **task** a TaskManager instance.
- **vif_id** Name or UUID of a VIF.
- **physnets** Set of physical networks on which the VIF may be attached. This is governed by the segments of the VIFs network. An empty set indicates that the ports physical networks should be ignored.
- **vif_info** dict that may contain extra information, such as port_uuid

Raises
VifAlreadyAttached, if VIF is already attached to the node.

Raises
NoFreePhysicalPorts, if there is no port-like object VIF can be attached to.

Raises
PortgroupPhysnetInconsistent if one of the nodes portgroups has ports which are not all assigned the same physical network.

Returns
port-like object VIF will be attached to.

ironic.drivers.modules.network.common.**plug_port_to_tenant_network**(*task*, *port_like_obj*, *client=None*)

Plug port like object to tenant network.

Parameters

- **task** A TaskManager instance.
- **port_like_obj** port-like object to plug.
- **client** Neutron client instance.

Raises
NetworkError if failed to update Neutron port.

Raises
VifNotAttached if tenant VIF is not associated with port_like_obj.

ironic.drivers.modules.network.common.**update_port_host_id**(*task*, *vif_id*, *client=None*)

Send an initial host_id value to Neutron to enable allocation.

Parameters

- **task** A TaskManager instance.
- **vif_id** The VIF ID to attach to the host.
- **client** A neutron client object

Raises
NetworkError if failed to update Neutron port.

Raises
VifNotAttached if tenant VIF is not associated with port_like_obj.

ironic.drivers.modules.network.flat module

Flat network interface. Useful for shared, flat networks.

class ironic.drivers.modules.network.flat.**FlatNetwork**(**args*, ***kwargs*)

Bases: *NeutronVIFPortIDMixin*, *NeutronNetworkInterfaceMixin*, *NetworkInterface*

Flat network interface.

add_cleaning_network(*task*)

Add the cleaning network to a node.

Parameters
task A TaskManager instance.

Returns
a dictionary in the form {port.uuid: neutron_port[id]}

Raises
NetworkError, InvalidParameterValue

add_inspection_network(*task*)

Add the inspection network to the node.

Parameters
task A TaskManager instance.

Returns
a dictionary in the form {port.uuid: neutron_port[id]}

Raises
NetworkError

Raises
InvalidParameterValue, if the network interface configuration is invalid.

add_provisioning_network(*task*)

Add the provisioning network to a node.

Parameters
task A TaskManager instance.

Raises
NetworkError when failed to set binding:host_id

add_rescuing_network(*task*)

Add the rescuing network to a node.

Flat network does not use the rescuing network. Bind the port again since unconfigure_tenant_network() unbound it.

Parameters
task A TaskManager instance.

Returns
a dictionary in the form {port.uuid: neutron_port[id]}

Raises
NetworkError, InvalidParameterValue

add_servicing_network(*task*)

Add the rescuing network to a node.

Flat network does not use the servicing network. Bind the port again since unconfigure_tenant_network() unbound it.

Parameters
task A TaskManager instance.

Returns
a dictionary in the form {port.uuid: neutron_port[id]}

Raises
NetworkError, InvalidParameterValue

capabilities = ['metadata']

configure_tenant_networks(*task*)

Configure tenant networks for a node.

Parameters
task A TaskManager instance.

remove_cleaning_network(*task*)

Remove the cleaning network from a node.

Parameters
task A TaskManager instance.

Raises
NetworkError

remove_inspection_network(*task*)

Removes the inspection network from a node.

Parameters
task A TaskManager instance.

Raises
InvalidParameterValue, if the network interface configuration is invalid.

Raises
MissingParameterValue, if some parameters are missing.

remove_provisioning_network(*task*)

Remove the provisioning network from a node.

Parameters
task A TaskManager instance.

remove_rescuing_network(*task*)

Remove the rescuing network from a node.

Flat network does not use the rescuing network. Unbind the port again since add_rescuing_network() bound it.

Parameters
task A TaskManager instance.

Raises
NetworkError

`remove_servicing_network`(*task*)

Remove the servicing network from a node.

Flat network does not use the servicing network. Unbind the port again since add_rescuing_network() bound it.

Parameters
task A TaskManager instance.

Raises
NetworkError

`unconfigure_tenant_networks`(*task*)

Unconfigure tenant networks for a node.

Unbind the port here/now to avoid the possibility of the ironic port being bound to the tenant and cleaning networks at the same time.

Parameters
task A TaskManager instance.

Raises
NetworkError

`validate`(*task*)

Validates the network interface.

Parameters
task a TaskManager instance.

Raises
InvalidParameterValue, if the network interface configuration is invalid.

Raises
MissingParameterValue, if some parameters are missing.

ironic.drivers.modules.network.ironic_networking module

Ironic Networking Network Interface

This is a network interface designed for standalone Ironic deployments that require minimal network configuration. It implements all required NetworkInterface methods but performs no actual operations, making it suitable for environments where external network configuration is handled separately or not required.

`class ironic.drivers.modules.network.ironic_networking.IronicNetworking`(**args*, ***kwargs*)

Bases: *NetworkInterface*

Ironic Networking network interface.

This network interface is designed for standalone Ironic deployments where configuration of switch ports should be handled by Ironic.

Port Configuration: This interface validates and processes ports that have an extra property containing a switchport sub-property. The switchport configuration must conform to the SWITCHPORT_SCHEMA defined in ironic_networking_schemas.py.

Expected switchport configuration format: - mode: access, trunk, or hybrid - native_vlan: VLAN ID (required) - allowed_vlans: List of VLAN IDs (required for trunk/hybrid modes only) - lag_name: Optional LAG name

Portgroup Configuration: This interface validates and processes portgroups that have an extra property containing a lag sub-property. The lag configuration must conform to the LAG_SCHEMA defined in ironic_networking_schemas.py.

add_cleaning_network(*task*)

Add the cleaning network to a node.

This method configures the cleaning network for each port on the node by applying the ports switchport configuration with the cleaning network VLAN/segment ID override if set.

Parameters
task A TaskManager instance.

Returns
Empty dictionary as no ports are configured.

add_inspection_network(*task*)

Add the inspection network to the node.

This method configures the inspection network for each port on the node by applying the ports switchport configuration with the inspection network VLAN/segment ID override if set.

Parameters
task A TaskManager instance.

Returns
Empty dictionary as no ports are configured.

add_provisioning_network(*task*)

Add the provisioning network to a node.

This method configures the provisioning network for each port on the node by applying the ports switchport configuration with the provisioning network VLAN/segment ID override if set.

Parameters
task A TaskManager instance.

add_rescuing_network(*task*)

Add the rescuing network to the node.

This method configures the rescuing network for each port on the node by applying the ports switchport configuration with the rescuing network VLAN/segment ID override if set.

Parameters
task A TaskManager instance.

Returns
Empty dictionary as no ports are configured.

add_servicing_network(*task*)

Add the servicing network to the node.

This method configures the servicing network for each port on the node by applying the ports switchport configuration with the servicing network VLAN/segment ID override if set.

Parameters
task A TaskManager instance.

Returns
Empty dictionary as no ports are configured.

configure_tenant_networks(*task*)

Configure tenant networks for a node.

This method configures tenant network connectivity for each port on the node by applying the ports switchport configuration with any tenant network overrides.

Parameters
task A TaskManager instance.

get_current_vif(*task*, *p_obj*)

Return the currently used VIF associated with port or portgroup.

In the ironic networking interface, no VIFs are managed, so this always returns None.

Parameters

- **task** A TaskManager instance.
- **p_obj** Ironic port or portgroup object.

Returns
None as no VIFs are managed by this interface.

get_node_network_data(*task*)

Return network configuration for node NICs.

This method returns network configuration data for the node. It first checks if static network data is configured on the node itself. If present, that takes precedence. Otherwise, it builds network data from the ports and portgroups attached to the node.

The network data is returned in Nova network metadata layout (*network_data.json*) format.

For the ironic-networking interface, this generates:

- Physical links (type: phy) for each port with a MAC address
- VLAN interfaces (type: vlan) for ports with allowed_vlans configured in their switchport settings

Parameters
task A TaskManager instance.

Returns
a dict holding network configuration information adhering to Nova network metadata layout (*network_data.json*).

need_power_on(*task*)

Check if node must be powered on before applying network changes.

Switch operations can be performed without powering on the node.

Parameters
task A TaskManager instance.

Returns
False as no power state changes are needed.

port_changed(*task*, *port_obj*)

Handle any actions required when a port changes.

In the ironic networking interface, this method processes ports that have switchport configuration in their extra field and calls the networking service API accordingly based on the nodes current provision state and what network should be active.

Parameters

- task A TaskManager instance.
- port_obj A changed Port object.

portgroup_changed(*task*, *portgroup_obj*)

Handle any actions required when a portgroup changes.

In the ironic networking interface, portgroup changes are not currently supported and will be logged but no action will be taken.

Parameters

- task A TaskManager instance.
- portgroup_obj A changed Portgroup object.

remove_cleaning_network(*task*)

Remove the cleaning network from a node.

This method resets the cleaning network configuration for each port on the node by restoring the ports default switchport configuration.

Parameters
task A TaskManager instance.

remove_inspection_network(*task*)

Remove the inspection network from a node.

This method resets the inspection network configuration for each port on the node by restoring the ports default switchport configuration.

Parameters
task A TaskManager instance.

remove_provisioning_network(*task*)

Remove the provisioning network from a node.

This method resets the provisioning network configuration for each port on the node by restoring the ports default switchport configuration.

Parameters
task A TaskManager instance.

remove_rescuing_network(*task*)

Remove the rescuing network from a node.

This method resets the rescuing network configuration for each port on the node by restoring the ports default switchport configuration.

Parameters
task A TaskManager instance.

remove_servicing_network(*task*)

Remove the servicing network from a node.

This method resets the servicing network configuration for each port on the node by restoring the ports default switchport configuration.

Parameters
task A TaskManager instance.

unconfigure_tenant_networks(*task*)

Unconfigure tenant networks for a node.

This method removes tenant network configuration for each port on the node by restoring the ports default switchport configuration.

Parameters
task A TaskManager instance.

validate(*task*)

Validate the network interface configuration.

For the ironic networking interface, this validates any ports that have switchport configuration in their extra field and any portgroups that have LAG configuration in their extra field.

Parameters
task A TaskManager instance.

Raises
InvalidParameterValue if switchport or lag configuration is invalid.

validate_inspection(*task*)

Validate that the node has required properties for inspection.

This method validates that at least one port has the required network configuration for inspection operations.

Parameters
task A TaskManager instance with the node being checked.

Raises
InvalidParameterValue if unable to parse network configuration

validate_rescue(*task*)

Validate the network interface for rescue operation.

This method validates that at least one port has the required network configuration for rescue operations.

Parameters
task A TaskManager instance.

Raises
InvalidParameterValue if unable to parse network configuration

vif_attach(*task*, *vif_info*)

Attach a virtual network interface to a node.

In the ironic networking interface, VIF attachment is a no-op. This allows the operation to complete successfully without performing any actual network configuration.

Parameters

- **task** A TaskManager instance.
- **vif_info** A dictionary of information about a VIF. It must have an id key, whose value is a unique identifier for that VIF.

vif_detach(*task*, *vif_id*)

Detach a virtual network interface from a node.

In the ironic networking interface, VIF detachment is a no-op. This allows the operation to complete successfully without performing any actual network configuration.

Parameters

- **task** A TaskManager instance.
- **vif_id** A VIF ID to detach.

vif_list(*task*)

List attached VIF IDs for a node.

In the ironic networking interface, no VIFs are ever attached, so this always returns an empty list.

Parameters
task A TaskManager instance.

Returns
Empty list as no VIFs are managed by this interface.

ironic.drivers.modules.network.ironic_networking_schemas module

JSON Schema definitions for the ironic-networking network interface.

This module contains the JSON schemas used to validate switchport and LAG configuration in port and portgroup extra fields.

ironic.drivers.modules.network.neutron module

class ironic.drivers.modules.network.neutron.**NeutronNetwork**(**args*, ***kwargs*)

Bases: *NeutronVIFPortIDMixin*, *NeutronNetworkInterfaceMixin*, *NetworkInterface*

Neutron v2 network interface

add_cleaning_network(*task*)

Create neutron ports for each port on task.node to boot the ramdisk.

Parameters
task a TaskManager instance.

Raises
NetworkError

Returns
a dictionary in the form {port.uuid: neutron_port[id]}

add_inspection_network(*task*)

Add the inspection network to the node.

Parameters
task A TaskManager instance.

Returns
a dictionary in the form {port.uuid: neutron_port[id]}

Raises
NetworkError

Raises
InvalidParameterValue, if the network interface configuration is invalid.

add_provisioning_network(*task*)

Add the provisioning network to a node.

Parameters
task A TaskManager instance.

Raises
NetworkError

add_rescuing_network(*task*)

Create neutron ports for each port to boot the rescue ramdisk.

Parameters
task a TaskManager instance.

Returns
a dictionary in the form {port.uuid: neutron_port[id]}

add_servicing_network(*task*)

Create neutron ports for each port to boot the servicing ramdisk.

Parameters
task a TaskManager instance.

Returns
a dictionary in the form {port.uuid: neutron_port[id]}

capabilities = ['metadata']

configure_tenant_networks(*task*)

Configure tenant networks for a node.

Parameters
task A TaskManager instance.

Raises
NetworkError

`need_power_on`(*task*)

Check if the node has any Smart NIC ports

Parameters
`task` A TaskManager instance.

Returns
A boolean to indicate Smart NIC port presence

`reattach_networking`(*task*)

Reattach tenant networking without rebooting the node.

This service step unbinds and rebinds the tenant network ports attached to the node without requiring a reboot or IPA deployment. This is useful for advanced networking scenarios such as:

- Reconfiguring VXLAN network topology
- Updating network binding after infrastructure changes
- Recovering from network binding failures
- Migrating network segments or physical networks

The VIF attachments are preserved (not removed), only the Neutron port bindings are cleared and re-established. This allows network switches to reconfigure the ports without instance downtime.

Parameters
`task` A TaskManager instance.

Raises
NetworkError if reattaching fails

`remove_cleaning_network`(*task*)

Deletes the neutron port created for booting the ramdisk.

Parameters
`task` a TaskManager instance.

Raises
NetworkError

`remove_inspection_network`(*task*)

Removes the inspection network from a node.

Parameters
`task` A TaskManager instance.

Raises
InvalidParameterValue, if the network interface configuration is invalid.

Raises
MissingParameterValue, if some parameters are missing.

`remove_provisioning_network`(*task*)

Remove the provisioning network from a node.

Parameters
task A TaskManager instance.

Raises
NetworkError

remove_rescuing_network(*task*)

Deletes neutron port created for booting the rescue ramdisk.

Parameters
task a TaskManager instance.

Raises
NetworkError

remove_servicing_network(*task*)

Deletes neutron port created for booting the servicing ramdisk.

Parameters
task a TaskManager instance.

Raises
NetworkError

unconfigure_tenant_networks(*task*)

Unconfigure tenant networks for a node.

Nova takes care of port removal from tenant network, we unbind it here/now to avoid the possibility of the ironic port being bound to the tenant and cleaning networks at the same time.

Parameters
task A TaskManager instance.

Raises
NetworkError

validate(*task*)

Validates the network interface.

Parameters
task a TaskManager instance.

Raises
InvalidParameterValue, if the network interface configuration is invalid.

Raises
MissingParameterValue, if some parameters are missing.

validate_rescue(*task*)

Validates the network interface for rescue operation.

Parameters
task a TaskManager instance.

Raises
InvalidParameterValue, if the network interface configuration is invalid.

Raises
MissingParameterValue, if some parameters are missing.

validate_servicing(*task*)

Validates the network interface for servicing operation.

Parameters
task a TaskManager instance.

Raises
InvalidParameterValue, if the network interface configuration is invalid.

Raises
MissingParameterValue, if some parameters are missing.

ironic.drivers.modules.network.noop module

class ironic.drivers.modules.network.noop.**NoopNetwork**(**args*, ***kwargs*)

Bases: *NetworkInterface*

Noop network interface.

add_cleaning_network(*task*)

Add the cleaning network to a node.

Parameters
task A TaskManager instance.

add_provisioning_network(*task*)

Add the provisioning network to a node.

Parameters
task A TaskManager instance.

configure_tenant_networks(*task*)

Configure tenant networks for a node.

Parameters
task A TaskManager instance.

get_current_vif(*task*, *p_obj*)

Returns the currently used VIF associated with port or portgroup

We are booting the node only in one network at a time, and presence of cleaning_vif_port_id means were doing cleaning, of provisioning_vif_port_id - provisioning of rescuing_vif_port_id - rescuing. Otherwise its a tenant network

Parameters

- **task** A TaskManager instance.
- **p_obj** Ironic port or portgroup object.

Returns
VIF ID associated with p_obj or None.

port_changed(*task*, *port_obj*)

Handle any actions required when a port changes

Parameters

- **task** a TaskManager instance.

- **port_obj** a changed Port object.

Raises
Conflict, FailedToUpdateDHCPOptOnPort

portgroup_changed(*task*, *portgroup_obj*)

Handle any actions required when a portgroup changes

Parameters

- **task** a TaskManager instance.
- **portgroup_obj** a changed Portgroup object.

Raises
Conflict, FailedToUpdateDHCPOptOnPort

remove_cleaning_network(*task*)

Remove the cleaning network from a node.

Parameters
task A TaskManager instance.

remove_provisioning_network(*task*)

Remove the provisioning network from a node.

Parameters
task A TaskManager instance.

unconfigure_tenant_networks(*task*)

Unconfigure tenant networks for a node.

Parameters
task A TaskManager instance.

validate_inspection(*task*)

Validate that the node has required properties for inspection.

Parameters
task A TaskManager instance with the node being checked

vif_attach(*task*, *vif_info*)

Attach a virtual network interface to a node

Parameters

- **task** A TaskManager instance.
- **vif_info** a dictionary of information about a VIF. It must have an id key, whose value is a unique identifier for that VIF.

Raises
NetworkError, VifAlreadyAttached, NoFreePhysicalPorts

vif_detach(*task*, *vif_id*)

Detach a virtual network interface from a node

Parameters

- **task** A TaskManager instance.

- **vif_id** A VIF ID to detach

Raises
NetworkError, VifNotAttached

vif_list(*task*)
List attached VIF IDs for a node.

Parameters
task A TaskManager instance.

Returns
List of VIF dictionaries, each dictionary will have an id entry with the ID of the VIF.

ironic.drivers.modules.network.switchport_config module

SwitchPortConfig - parsed representation of switchport configuration.

This module provides a dataclass for representing and parsing the switchport configuration format used by the Ironic Networking interface:

{access|trunk|hybrid}/native_vlan=VLAN_ID[/allowed_vlans=V1,V2,Vn-Vm,]

class ironic.drivers.modules.network.switchport_config.**SwitchPortConfig**(*mode: str*, *native_vlan: int | None = None*, *allowed_vlans: list | None = None*)

Bases: object

Parsed representation of a switchport configuration value.

Instances can be created from a configuration string via *from_string()* or from a switchport dict via *from_switchport()*.

allowed_vlans: list | None = None

classmethod from_string(*value*, *network_type='unknown'*)
Parse a network config string into a SwitchPortConfig.

Parameters

- **value** String in format {access|trunk|hybrid}/native_vlan=N[/allowed_vlans=V1,V2,Vn-Vm]
- **network_type** Network type label used in error messages.

Returns
A SwitchPortConfig instance.

Raises
InvalidParameterValue if the format is invalid.

classmethod from_switchport(*switchport*)

Create a SwitchPortConfig from a switchport configuration dict.

Parameters

switchport A dict with optional keys `mode`, `native_vlan`, and `allowed_vlans`.

Returns

A SwitchPortConfig instance, or None if the dict does not contain a `mode` key.

Raises

InvalidParameterValue if `mode` is not a valid mode or `native_vlan` is not an integer.

property is_valid

True if the config has the required fields for its mode.

For `access` mode, `native_vlan` must be set. For `trunk` or `hybrid` mode, `allowed_vlans` must be set.

mode: str

native_vlan: int | None = None

Module contents

ironic.drivers.modules.redfish package

Submodules

ironic.drivers.modules.redfish.bios module

class ironic.drivers.modules.redfish.bios.**RedfishBIOS**(**args*, ***kwargs*)

Bases: *BIOSInterface*

apply_configuration(*task*, *settings*)

Apply the BIOS settings to the node.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **settings** a list of BIOS settings to be updated.

Raises

RedfishConnectionError when it fails to connect to Redfish

Raises

RedfishError on an error from the Sushy library

cache_bios_settings(*task*)

Store or update the current BIOS settings for the node.

Get the current BIOS settings and store them in the bios_settings database table.

Parameters

task a TaskManager instance containing the node to act on.

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError on an error from the Sushy library

Raises
UnsupportedDriverExtension if the system does not support BIOS settings

`factory_reset`(*task*)

Reset the BIOS settings of the node to the factory default.

Parameters
task a TaskManager instance containing the node to act on.

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError on an error from the Sushy library

`get_properties`()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

`post_configuration`(*task*, *settings*)

Perform post configuration action to store the BIOS settings.

Extension point to allow vendor implementations to extend this class and override this method to perform a custom action to write the BIOS settings to the Redfish service. The default implementation performs a reboot.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **settings** a list of BIOS settings to be updated.

`post_reset`(*task*)

Perform post reset action to apply the BIOS factory reset.

Extension point to allow vendor implementations to extend this class and override this method to perform a custom action to apply the BIOS factory reset to the Redfish service. The default implementation performs a reboot.

Parameters
task a TaskManager instance containing the node to act on.

`validate`(*task*)

Validates the driver information needed by the redfish driver.

Parameters
task a TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

ironic.drivers.modules.redfish.boot module

class ironic.drivers.modules.redfish.boot.**RedfishHttpsBoot**(**args*, ***kwargs*)

Bases: *BootInterface*

A driver which utilizes UefiHttp like virtual media.

Utilizes the virtual media image build to craft a ISO image to signal to remote BMC to boot.

This interface comes with some constraints. For example, this interface is built under the operating assumption that DHCP is used. The UEFI Firmware needs to load some base configuration, regardless. Also depending on UEFI Firmware, and how it handles UefiHttp Boot, additional ISO contents, such as configuration drive materials might be unavailable. A similar constraint exists with `ramdisk` deployment.

capabilities = ['ramdisk_boot']

clean_up_instance(*task*)

Cleans up the boot of instance.

This method cleans up the environment that was setup for booting the instance.

Parameters
task A task from TaskManager.

Returns
None

clean_up_ramdisk(*task*)

Cleans up the boot of ironic ramdisk.

This method cleans up the environment that was setup for booting the deploy ramdisk.

Parameters
task A task from TaskManager.

Returns
None

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

prepare_instance(*task*)

Prepares the boot of instance over virtual media.

This method prepares the boot of the instance after reading relevant information from the nodes instance_info.

The internal logic is as follows:

- Cleanup any related files
- Sync the boot mode with the machine.

- Configure Secure boot, if required.
- If local boot, or a whole disk image was deployed, set the next boot device as disk.
- If ramdisk is the desired, then the UefiHttp boot option is set to the BMC with a request for this to be persistent.

Parameters
task a task from TaskManager.

Returns
None

Raises
InstanceDeployFailure, if its try to boot iSCSI volume in BIOS boot mode.

prepare_ramdisk(*task*, *ramdisk_params*)

Prepares the boot of the agent ramdisk.

This method prepares the boot of the deploy or rescue ramdisk after reading relevant information from the nodes driver_info and instance_info.

Parameters

- **task** A task from TaskManager.
- **ramdisk_params** the parameters to be passed to the ramdisk.

Returns
None

Raises
MissingParameterValue, if some information is missing in nodes driver_info or instance_info.

Raises
InvalidParameterValue, if some information provided is invalid.

Raises
IronicException, if some power or set boot boot device operation failed on the node.

validate(*task*)

Validate the deployment information for the tasks node.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

validate_inspection(*task*)

Validate that the node has required properties for inspection.

Parameters
task A TaskManager instance with the node being checked

Raises
MissingParameterValue if node is missing one or more required parameters

class ironic.drivers.modules.redfish.boot.**RedfishVirtualMediaBoot**(**args*, ***kwargs*)

Bases: *BootInterface*

Virtual media boot interface over Redfish.

Virtual Media allows booting the system from the virtual CD/DVD drive containing the user image that BMC inserts into the drive.

The CD/DVD images must be in ISO format and (depending on BMC implementation) could be pulled over HTTP, served as iSCSI targets or NFS volumes.

The baseline boot workflow looks like this:

1. Pull kernel, ramdisk and ESP (FAT partition image with EFI boot loader) images (ESP is only needed for UEFI boot)
2. Create bootable ISO out of images (#1), push it to Glance and pass to the BMC as Swift temporary URL
3. Optionally create floppy image with desired system configuration data, push it to Glance and pass to the BMC as Swift temporary URL
4. Insert CD/DVD and (optionally) floppy images and set proper boot mode

For building deploy or rescue ISO, redfish boot interface uses *deploy_kernel/deploy_ramdisk* or *rescue_kernel/rescue_ramdisk* properties from *[instance_info]* or *[driver_info]*.

For building boot (user) ISO, redfish boot interface seeks *kernel_id* and *ramdisk_id* properties in the Glance image metadata found in *[instance_info]image_source* node property.

capabilities = ['iscsi_volume_boot', 'ramdisk_boot', 'ramdisk_boot_configdrive']

clean_up_instance(*task*)

Cleans up the boot of instance.

This method cleans up the environment that was setup for booting the instance.

Parameters
task A task from TaskManager.

Returns
None

clean_up_ramdisk(*task*)

Cleans up the boot of ironic ramdisk.

This method cleans up the environment that was setup for booting the deploy ramdisk.

Parameters
task A task from TaskManager.

Returns
None

`get_properties`()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

`prepare_instance`(*task*)

Prepares the boot of instance over virtual media.

This method prepares the boot of the instance after reading relevant information from the nodes instance_info.

The internal logic is as follows:

- If *boot_option* requested for this deploy is local, then set the node to boot from disk.
- Unless *boot_option* requested for this deploy is ramdisk, pass root disk/partition ID to virtual media boot image
- Otherwise build boot image, insert it into virtual media device and set node to boot from CD.

Parameters
task a task from TaskManager.

Returns
None

Raises
InstanceDeployFailure, if its try to boot iSCSI volume in BIOS boot mode.

`prepare_ramdisk`(*task*, *ramdisk_params*)

Prepares the boot of deploy or rescue ramdisk over virtual media.

This method prepares the boot of the deploy or rescue ramdisk after reading relevant information from the nodes driver_info and instance_info.

Parameters

- **task** A task from TaskManager.
- **ramdisk_params** the parameters to be passed to the ramdisk.

Returns
None

Raises
MissingParameterValue, if some information is missing in nodes driver_info or instance_info.

Raises
InvalidParameterValue, if some information provided is invalid.

Raises
IronicException, if some power or set boot boot device operation failed on the node.

validate(*task*)

Validate the deployment information for the tasks node.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

validate_inspection(*task*)

Validate that the node has required properties for inspection.

Parameters
task A TaskManager instance with the node being checked

Raises
MissingParameterValue if node is missing one or more required parameters

validate_rescue(*task*)

Validate that the node has required properties for rescue.

Parameters
task a TaskManager instance with the node being checked

Raises
MissingParameterValue if node is missing one or more required parameters

ironic.drivers.modules.redfish.boot.**eject_vmedia**(*task*, *boot_device=None*)

Eject virtual CDs and DVDs

Parameters

- **task** A task from TaskManager.
- **boot_device** sushy boot device e.g. *VIRTUAL_MEDIA_CD*, *VIRTUAL_MEDIA_DVD* or *VIRTUAL_MEDIA_FLOPPY* or *None* to eject everything (default).

Raises
InvalidParameterValue, if no suitable virtual CD or DVD is found on the node.

ironic.drivers.modules.redfish.boot.**get_vmedia**(*task*)

Get the attached virtual CDs and DVDs for a node

Parameters
task A task from TaskManager.

Raises
InvalidParameterValue, if no suitable virtual CD or DVD is found on the node.

ironic.drivers.modules.redfish.boot.**insert_vmedia**(*task*, *image_url*, *device_type*)

Insert virtual CDs and DVDs

Parameters

- **task** A task from TaskManager.
- **image_url**
- **device_type** sushy boot device e.g. *VIRTUAL_MEDIA_CD*, *VIRTUAL_MEDIA_DVD* or *VIRTUAL_MEDIA_FLOPPY* or *None* to eject everything (default).

Raises
InvalidParameterValue, if no suitable virtual CD or DVD is found on the node.

ironic.drivers.modules.redfish.firmware module

class ironic.drivers.modules.redfish.firmware.**RedfishFirmware**(**args*, ***kwargs*)

Bases: *FirmwareInterface*

cache_firmware_components(*task*)

Store or update Firmware Components on the given node.

This method stores Firmware Components to the firmware_information table during cleaning operation. It will also update the timestamp of each Firmware Component.

Parameters
task a TaskManager instance.

Raises
UnsupportedDriverExtension, if the nodes driver doesnt support getting Firmware Components from bare metal.

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

retrieve_nic_components(*task*, *system*)

Helper function to retrieve all NICs components on a given node.

Parameters

- **task** a TaskManager instance.
- **system** a Redfish System object

Returns
a list of NIC components

update(*task*, *settings*)

Update the Firmware on the node using the settings for components.

Parameters

- **task** a TaskManager instance.
- **settings** a list of dictionaries, each dictionary contains the component name and the url that will be used to update the firmware.

Raises
UnsupportedDriverExtension, if the nodes driver doesnt support update via the interface.

Raises
InvalidParameterValue, if validation of the settings fails.

Raises
MissingParamterValue, if some required parameters are missing.

Returns
states.CLEANWAIT if Firmware update with the settings is in progress asynchronously of None if it is complete.

`validate`(*task*)

Validates the driver information needed by the redfish driver.

Parameters
task a TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

ironic.drivers.modules.redfish.firmware_utils module

`ironic.drivers.modules.redfish.firmware_utils.`**`cleanup`**(*node*)

Clean up staged files

Parameters
node Node for which to clean up. Should contain firmware_cleanup entry in *driver_internal_info* to indicate source(s) to be cleaned up.

`ironic.drivers.modules.redfish.firmware_utils.`**`download_to_temp`**(*node*, *url*)

Downloads to temporary location from given URL

Parameters

- **node** Node for which to download to temporary location
- **url** URL to download from

Returns
File path of temporary location file is downloaded to

`ironic.drivers.modules.redfish.firmware_utils.`**`get_swift_temp_url`**(*parsed_url*)

Gets Swift temporary URL

Parameters
parsed_url Parsed URL from URL in format swift://container/[sub-folder/]file

Returns
Swift temporary URL

`ironic.drivers.modules.redfish.firmware_utils.`**`stage`**(*node*, *source*, *temp_file*)

Stage temporary file to configured location

Parameters

- **node** Node for which to stage the file

- **source** Where to stage the file. Corresponds to CONF.redfish.firmware_source.
- **temp_file** File path of temporary file to stage

Returns
Tuple of staged URL and source (http or swift) that needs cleanup of staged files afterwards.

Raises
RedfishError If staging to HTTP server has failed.

ironic.drivers.modules.redfish.firmware_utils.**validate_firmware_interface_update_args**(*settings*

Validate `update` step input argument

Parameters
settings args to validate.

Raises
InvalidParameterValue When argument is not valid

ironic.drivers.modules.redfish.firmware_utils.**validate_update_firmware_args**(*firmware_images*)

Validate `update_firmware` step input argument

Parameters
firmware_images args to validate.

Raises
InvalidParameterValue When argument is not valid

ironic.drivers.modules.redfish.firmware_utils.**verify_checksum**(*node*, *checksum*, *file_path*)

Verify checksum.

Parameters

- **node** Node for which file to verify checksum
- **checksum** Expected checksum value
- **file_path** File path for which to verify checksum

Raises
RedfishError When checksum does not match

ironic.drivers.modules.redfish.graphical_console module

class ironic.drivers.modules.redfish.graphical_console.**RedfishGraphicalConsole**(**args*, ***kwargs*)

Bases: *GraphicalConsole*

get_app_info(*task*)

Information required by the app to connect to the console

For redfish based consoles the app info will be the parsed driver info.

Returns
dict containing parsed driver info

get_app_name()

Get the name of the app passed to the console container.

A single console container image is expected to support all known graphical consoles and each implementation is referred to as an app. Each graphical console driver will specify what app to load in the container.

Returns
String representing the app name to load in the console container

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

validate(*task*)

Validates the driver information needed by the redfish driver.

Parameters
task a TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

ironic.drivers.modules.redfish.inspect module

Redfish Inspect Interface

class ironic.drivers.modules.redfish.inspect.**RedfishInspect**(**args*, ***kwargs*)

Bases: *InspectInterface*

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

inspect_hardware(*task*)

Inspect hardware to get the hardware properties.

Inspects hardware to get the essential properties. It fails if any of the essential properties are not received from the node.

Parameters
task a TaskManager instance.

Raises
HardwareInspectionFailure if essential properties could not be retrieved successfully.

Returns
The resulting state of inspection.

validate(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the driver_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

ironic.drivers.modules.redfish.management module

class ironic.drivers.modules.redfish.management.**RedfishManagement**(**args*, ***kwargs*)

Bases: *ManagementInterface*

attach_virtual_media(*task*, *device_type*, *image_url*)

Attach a virtual media device to the node.

Parameters

- **task** A task from TaskManager.
- **device_type** A device type from *ironic.common.boot_devices.VMEDIA_DEVICES*.
- **image_url** URL of the image to attach, HTTP or HTTPS.

clear_secure_boot_keys(*task*)

Clear all secure boot keys.

Parameters
task a task from TaskManager.

Raises
UnsupportedDriverExtension if secure boot is now supported.

Raises
RedfishError on runtime driver error.

detach_virtual_media(*task*, *device_types=None*)

Detach some or all virtual media devices from the node.

Parameters

- **task** A task from TaskManager.
- **device_types** A list of device types from *ironic.common.boot_devices.VMEDIA_DEVICES*. If not provided, all devices are detached.

`detect_vendor`(*task*)

Detects and returns the hardware vendor.

Uses the Service Roots Vendor field to identify the BMC firmware vendor. Falls back to the Systems Manufacturer field if Vendor is not available in the Service Root.

Parameters
`task` A task from TaskManager.

Raises
InvalidParameterValue if an invalid component, indicator or state is specified.

Raises
MissingParameterValue if a required parameter is missing

Raises
RedfishError on driver-specific problems.

Returns
String representing the BMC reported Vendor or Manufacturer, otherwise returns None.

`get_boot_device`(*task*)

Get the current boot device for a node.

Parameters
`task` a task from TaskManager.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError on an error from the Sushy library

Returns

a dictionary containing:

boot_device
the boot device, one of *ironic.common.boot_devices* or None if it is unknown.

persistent
Boolean value or None, True if the boot device persists, False otherwise. None if its unknown.

`get_boot_mode`(*task*)

Get the current boot mode for a node.

Provides the current boot mode of the node.

Parameters
`task` A task from TaskManager.

Raises
MissingParameterValue if a required parameter is missing

Raises
DriverOperationError or its derivative in case of driver runtime error.

Returns
The boot mode, one of `ironic.common.boot_mode` or None if it is unknown.

`get_indicator_state`(*task*, *component*, *indicator*)

Get current state of the indicator of the hardware component.

Parameters

- **task** A task from TaskManager.
- **component** The hardware component, one of *ironic.common.components*.
- **indicator** Indicator ID (as reported by *get_supported_indicators*).

Raises
MissingParameterValue if a required parameter is missing

Raises
RedfishError on an error from the Sushy library

Returns
Current state of the indicator, one of *ironic.common.indicator_states*.

`get_mac_addresses`(*task*)

Get MAC address information for the node.

Parameters
task A TaskManager instance containing the node to act on.

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError on an error from the Sushy library

Returns
A list of MAC addresses for the node

`get_node_health`(*task*)

Get the current health status for a node.

Retrieves the hardware health status from the Redfish System resource. The health status represents the overall condition of the server hardware, including components like processors, memory, fans, power supplies, storage, etc.

Parameters
task A task from TaskManager.

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError on an error from the Sushy library

Returns
One of *ironic.common.health_states* constants (OK, WARNING, CRITICAL) or None if health status is not available.

`get_properties()`

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

`get_secure_boot_state(task)`

Get the current secure boot state for the node.

Parameters
task A task from TaskManager.

Raises
MissingParameterValue if a required parameter is missing

Raises
RedfishError or its derivative in case of a driver runtime error.

Raises
UnsupportedDriverExtension if secure boot is not supported by the hardware.

Returns
Boolean or None if status cannot be retrieved

`get_sensors_data(task)`

Get sensors data.

Collects sensor data from chassis (fans, temperature, power) and storage (drives) with minimal redfish API calls.

Parameters
task a TaskManager instance.

Returns
returns a dict of sensor data grouped by sensor type.

`get_supported_boot_devices(task)`

Get a list of the supported boot devices.

Parameters
task a task from TaskManager.

Returns
A list with the supported boot devices defined in *ironic.common.boot_devices*.

`get_supported_boot_modes(task)`

Get a list of the supported boot modes.

Parameters
task A task from TaskManager.

Returns
A list with the supported boot modes defined in *ironic.common.boot_modes*. If boot mode support cant be determined, empty list is returned.

get_supported_indicators(*task*, *component=None*)

Get a map of the supported indicators (e.g. LEDs).

Parameters

- **task** A task from TaskManager.
- **component** If not *None*, return indicator information for just this component, otherwise return indicators for all existing components.

Returns

A dictionary of hardware components (*ironic.common.components*) as keys with values being dictionaries having indicator IDs as keys and indicator properties as values.

```
{
    'chassis': {
        'enclosure-0': {
            "readonly": true,
            "states": [
                "OFF",
                "ON"
            ]
        }
    },
    'system':
        'blade-A': {
            "readonly": true,
            "states": [
                "OFF",
                "ON"
            ]
        }
    },
    'drive':
        'ssd0': {
            "readonly": true,
            "states": [
                "OFF",
                "ON"
            ]
        }
    }
}
```

get_virtual_media(*task*)

Get all virtual media devices from the node.

Parameters

task A task from TaskManager.

inject_nmi(*task*)

Inject NMI, Non Maskable Interrupt.

Inject NMI (Non Maskable Interrupt) for a node immediately.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError on an error from the Sushy library

reset_secure_boot_keys_to_default(*task*)

Reset secure boot keys to manufacturing defaults.

Parameters
task a task from TaskManager.

Raises
UnsupportedDriverExtension if secure boot is now supported.

Raises
RedfishError on runtime driver error.

restore_boot_device(*task*, *system*)

Restore boot device if needed.

Checks the redfish_boot_device internal flag and sets the one-time boot device accordingly. A warning is issued if it fails.

This method is supposed to be called from the Redfish power interface and should be considered private to the Redfish hardware type.

Parameters

- **task** a task from TaskManager.
- **system** a Redfish System object.

set_bmc_clock(*task*, *target_datetime=None*, *datetime_local_offset=None*)

Set the BMC clock using Redfish Manager resource.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **target_datetime** The datetime to set in ISO8601 format. Defaults to the current conductor UTC time.
- **datetime_local_offset** The local time offset from UTC (optional)

Raises
RedfishError if the operation fails

set_boot_device(*task*, *device*, *persistent=False*)

Set the boot device for a node.

Set the boot device to use on next reboot of the node.

Parameters

- **task** a task from TaskManager.
- **device** the boot device, one of *ironic.common.boot_devices*.
- **persistent** Boolean value. True if the boot device will persist to all future boots, False if not. Default: False.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError on an error from the Sushy library

set_boot_mode(*task*, *mode*)

Set the boot mode for a node.

Set the boot mode to use on next reboot of the node.

Parameters

- **task** A task from TaskManager.
- **mode** The boot mode, one of *ironic.common.boot_modes*.

Raises
InvalidParameterValue if an invalid boot mode is specified.

Raises
MissingParameterValue if a required parameter is missing

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError on an error from the Sushy library

set_indicator_state(*task*, *component*, *indicator*, *state*)

Set indicator on the hardware component to the desired state.

Parameters

- **task** A task from TaskManager.
- **component** The hardware component, one of *ironic.common.components*.
- **indicator** Indicator ID (as reported by *get_supported_indicators*).
- **state** Desired state of the indicator, one of *ironic.common.indicator_states*.

Raises
InvalidParameterValue if an invalid component, indicator or state is specified.

Raises
MissingParameterValue if a required parameter is missing

Raises
RedfishError on an error from the Sushy library

`set_secure_boot_state`(*task*, *state*)

Set the current secure boot state for the node.

Parameters

- **task** A task from TaskManager.
- **state** A new state as a boolean.

Raises
MissingParameterValue if a required parameter is missing

Raises
RedfishError or its derivative in case of a driver runtime error.

Raises
UnsupportedDriverExtension if secure boot is not supported by the hardware.

`update_firmware`(*task*, *firmware_images*)

Updates the firmware on the node.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **firmware_images** A list of firmware images are to apply.

Returns
None if it is completed.

Raises
RedfishError on an error from the Sushy library.

`validate`(*task*)

Validates the driver information needed by the redfish driver.

Parameters
task a TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

`verify_bmc_clock`(*task*)

Verify and auto-set the BMC clock to the current UTC time.

This step compares the system UTC time to the BMCs Redfish datetime. If the difference exceeds 1 second, it attempts to sync the time. Verification fails only if the BMC time remains incorrect after the update.

ironic.drivers.modules.redfish.power module

class ironic.drivers.modules.redfish.power.**RedfishPower**(**args*, ***kwargs*)

Bases: *PowerInterface*

get_power_state(*task*)

Get the current power state of the tasks node.

Parameters
task a TaskManager instance containing the node to act on.

Returns
a power state. One of *ironic.common.states*.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError on an error from the Sushy library

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

get_supported_power_states(*task*)

Get a list of the supported power states.

Parameters
task A TaskManager instance containing the node to act on. Not used by this driver at the moment.

Returns
A list with the supported power states defined in *ironic.common.states*.

reboot(*task*, *timeout=None*)

Perform a hard reboot of the tasks node.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **timeout** Time to wait for the node to become powered on.

Raises
MissingParameterValue if a required parameter is missing.

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError on an error from the Sushy library

set_power_state(*task*, *power_state*, *timeout=None*)

Set the power state of the tasks node.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **power_state** Any power state from *ironic.common.states*.
- **timeout** Time to wait for the node to reach the requested state.

Raises
MissingParameterValue if a required parameter is missing.

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError on an error from the Sushy library

validate(*task*)

Validates the driver information needed by the redfish driver.

Parameters
task a TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

ironic.drivers.modules.redfish.raid module

class ironic.drivers.modules.redfish.raid.**RedfishRAID**(**args*, ***kwargs*)

Bases: *RAIDInterface*

apply_configuration(*task*, *raid_config*, *create_root_volume=True*, *create_nonroot_volumes=False*, *delete_existing=False*)

Applies RAID configuration on the given node.

Parameters

- **task** A TaskManager instance.
- **raid_config** The RAID configuration to apply.
- **create_root_volume** Setting this to False indicates not to create root volume that is specified in raid_config. Default value is True.
- **create_nonroot_volumes** Setting this to False indicates not to create non-root volumes (all except the root volume) in raid_config. Default value is True.
- **delete_existing** Setting this to True indicates to delete RAID configuration prior to creating the new configuration.

Raises
InvalidParameterValue, if the RAID configuration is invalid.

Returns
states.DEPLOYWAIT if RAID configuration is in progress asynchronously or None if it is complete.

`create_configuration`(*task*, *create_root_volume=True*, *create_nonroot_volumes=True*, *delete_existing=False*)

Create RAID configuration on the node.

This method creates the RAID configuration as read from node.target_raid_config. This method by default will create all logical disks.

Parameters

- **task** TaskManager object containing the node.
- **create_root_volume** Setting this to False indicates not to create root volume that is specified in the nodes target_raid_config. Default value is True.
- **create_nonroot_volumes** Setting this to False indicates not to create nonroot volumes (all except the root volume) in the nodes target_raid_config. Default value is True.
- **delete_existing** Setting this to True indicates to delete RAID configuration prior to creating the new configuration. Default is False.

Returns
states.CLEANWAIT if RAID configuration is in progress asynchronously or None if it is complete.

Raises
RedfishError if there is an error creating the configuration

`delete_configuration`(*task*)

Delete RAID configuration on the node.

Parameters
task TaskManager object containing the node.

Returns
states.CLEANWAIT (cleaning) or states.DEPLOYWAIT (deployment) if deletion is in progress asynchronously or None if it is complete.

`get_properties`()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

`post_create_configuration`(*task*, *raid_configs*, *return_state=None*)

Perform post create_configuration action to commit the config.

Extension point to allow vendor implementations to extend this class and override this method to perform a custom action to commit the RAID create configuration to the Redfish service.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **raid_configs** a list of dictionaries containing the RAID configuration operation details.

- **return_state** state to return based on operation being invoked

post_delete_configuration(*task*, *raid_configs*, *return_state=None*)

Perform post delete_configuration action to commit the config.

Extension point to allow vendor implementations to extend this class and override this method to perform a custom action to commit the RAID delete configuration to the Redfish service.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **raid_configs** a list of dictionaries containing the RAID configuration operation details.
- **return_state** state to return based on operation being invoked

pre_create_configuration(*task*, *logical_disks_to_create*)

Perform required actions before creating config.

Extension point to allow vendor implementations to extend this class and override this method to perform custom actions prior to creating the RAID configuration on the Redfish service.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **logical_disks_to_create** list of logical disks to create.

Returns

updated list of logical disks to create.

pre_delete_configuration(*task*, *vols_to_delete*)

Perform required actions before deleting config.

Extension point to allow vendor implementations to extend this class and override this method to perform custom actions prior to deleting the RAID configuration on the Redfish service.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **vols_to_delete** list of volumes to delete.

validate_raid_config(*task*, *raid_config*)

Validates the given RAID configuration.

Parameters

- **task** A TaskManager instance.
- **raid_config** The RAID configuration to validate.

Raises

InvalidParameterValue, if the RAID configuration is invalid.

volume_create_error_handler(*task*, *exc*, *volume_collection*, *payload*)

Handle error from failed VolumeCollection.create()

Extension point to allow vendor implementations to extend this class and override this method to perform a custom action if the call to VolumeCollection.create() fails.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **exc** the exception raised by VolumeCollection.create().
- **volume_collection** the sushy VolumeCollection instance.
- **payload** the payload passed to the failed create().

Returns
Newly created Volume resource or TaskMonitor if async task.

Raises
RedfishError if there is an error creating the virtual disk.

ironic.drivers.modules.redfish.raid.**convert_drive_units**(*logical_disks*, *node*)

Convert size in logical_disks from gb to bytes

ironic.drivers.modules.redfish.raid.**create_virtual_disk**(*task*, *raid_controller*, *physical_disks*, *raid_level*, *size_bytes*, *disk_name=None*, *span_length=None*, *span_depth=None*, *error_handler=None*)

Create a single virtual disk on a RAID controller.

Parameters

- **task** TaskManager object containing the node.
- **raid_controller** id of the RAID controller.
- **physical_disks** ids of the physical disks.
- **raid_level** RAID level of the virtual disk.
- **size_bytes** size of the virtual disk.
- **disk_name** name of the virtual disk. (optional)
- **span_depth** Number of spans in virtual disk. (optional)
- **span_length** Number of disks per span. (optional)
- **error_handler** function to call if volume create fails. (optional)

Returns
Newly created Volume resource or TaskMonitor if async task.

Raises
RedfishConnectionError when it fails to connect to Redfish.

Raises
RedfishError if there is an error creating the virtual disk.

ironic.drivers.modules.redfish.raid.**get_physical_disks**(*node*)

Get the physical drives of the node for RAID controllers.

Parameters
node an ironic node object.

Returns
a list of Drive objects from sushy

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError if there is an error getting the drives via Redfish

ironic.drivers.modules.redfish.raid.**update_raid_config**(*node*)

Updates nodes raid_config field with current logical disks.

Parameters
node node for which to update the raid_config field

ironic.drivers.modules.redfish.utils module

ironic.drivers.modules.redfish.utils.**BIOS = 'bios'**

BIOS Firmware Component

ironic.drivers.modules.redfish.utils.**BMC = 'bmc'**

BMC Firmware Component

ironic.drivers.modules.redfish.utils.**FIRMWARE_COMPONENTS = ['bios', 'bmc']**

Firmware Components available to update

ironic.drivers.modules.redfish.utils.**NIC = 'nic'**

NIC Firmware Component type

ironic.drivers.modules.redfish.utils.**NIC_COMPONENT_PREFIX = 'nic:'**

Prefix for NIC Firmware Components

class ironic.drivers.modules.redfish.utils.**SessionCache**(*driver_info*)

Bases: object

Cache of HTTP sessions credentials

AUTH_CLASSES = {'auto': <class 'sushy.auth.SessionOrBasicAuth'>, 'basic': <class 'sushy.auth.BasicAuth'>, 'session': <class 'sushy.auth.SessionAuth'>}

ironic.drivers.modules.redfish.utils.**get_chassis**(*node*, *system*)

Get a Redfish Chassis associated with a System of a node

Parameters

- **node** an Ironic node object
- **system** the Sushy System object

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError if there is no Chassis registered

ironic.drivers.modules.redfish.utils.**get_component_type**(*component*)

Determine the type of firmware component.

Note: This helper exists primarily to handle NIC components which use a prefix pattern (e.g., nic:BCM57414, nic:adapter1) rather than exact string matches. This centralizes the component

type detection logic that is used in multiple places (update initiation, task monitoring, completion handling) and provides a single source of truth for component type classification.

Parameters
component The component name from settings

Returns
One of bios, bmc, nic, or None

ironic.drivers.modules.redfish.utils.**get_enabled_macs**(*task*, *system*)

Get information on MAC addresses of enabled ports using Redfish.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **system** a Redfish System object

Returns
a dictionary containing MAC addresses of enabled interfaces in a {mac: <state>} format, where <state> is a sushy constant

ironic.drivers.modules.redfish.utils.**get_event_service**(*node*)

Get a nodes event service.

Parameters
node an Ironic node object.

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError when the EventService is not registered in Redfish

ironic.drivers.modules.redfish.utils.**get_first_controller**(*storage*)

Get the first storage controller from a storage object.

Parameters
storage a storage object

Returns
the first storage controller or None

ironic.drivers.modules.redfish.utils.**get_manager**(*node*, *system*, *manager_id=None*)

Get a nodes manager.

Parameters

- **system** a Sushy system object
- **manager_id** the id of the manager

Returns
a sushy Manager

Raises
RedfishError when the System doesnt have Managers associated

ironic.drivers.modules.redfish.utils.**get_root_vendor**(*node*)

Get the BMC vendor from the Redfish Service Root.

Parameters
node an Ironic node object

Returns
The Vendor string from the ServiceRoot, or None

ironic.drivers.modules.redfish.utils.**get_system**(*node*)

Get a Redfish System that represents a node.

Parameters
node an Ironic node object

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError if the System is not registered in Redfish

ironic.drivers.modules.redfish.utils.**get_system_collection**(*node*)

Get a Redfish System Collection that includes the node

Parameters
node an Ironic node object

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError if the System is not registered in Redfish

ironic.drivers.modules.redfish.utils.**get_task_monitor**(*node*, *uri*)

Get a TaskMonitor for a node.

Parameters

- **node** an Ironic node object
- **uri** the URI of a TaskMonitor

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError when the TaskMonitor is not available in Redfish

ironic.drivers.modules.redfish.utils.**get_update_service**(*node*)

Get a nodes update service.

Parameters
node an Ironic node object

Raises
RedfishConnectionError when it fails to connect to Redfish

Raises
RedfishError when the UpdateService is not registered in Redfish

ironic.drivers.modules.redfish.utils.**parse_driver_info**(*node*)

Parse the information required for Ironic to connect to Redfish.

Parameters
node an Ironic node object

Returns
dictionary of parameters

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

ironic.drivers.modules.redfish.utils.**wait_until_get_system_ready**(*node*)

Wait until Redfish system is ready.

Parameters
node an Ironic node object

Raises
RedfishConnectionError on time out.

ironic.drivers.modules.redfish.vendor module

Vendor Interface for Redfish drivers and its supporting methods.

class ironic.drivers.modules.redfish.vendor.**RedfishVendorPassthru**(**args*, ***kwargs*)

Bases: *VendorInterface*

Vendor-specific interfaces for Redfish drivers.

create_subscription(*task*, ***kwargs*)

Creates a subscription.

Parameters

- **task** A TaskManager object.
- **kwargs** The arguments sent with vendor passthru.

Raises
RedfishError, if any problem occurs when trying to create a subscription.

delete_subscription(*task*, ***kwargs*)

Delete a subscription.

Parameters

- **task** A TaskManager object.
- **kwargs** The arguments sent with vendor passthru.

Raises
RedfishError, if any problem occurs when trying to delete the subscription.

eject_vmedia(*task*, ***kwargs*)

Eject a virtual media device.

Deprecated in favour of the generic API. This should be removed during the 2024.2 cycle.

Parameters

- **task** A TaskManager object.
- **kwargs** The arguments sent with vendor passthru. The optional kwargs are:: boot_device: the boot device to eject

get_all_subscriptions(*task*, ***kwargs*)

Get all Subscriptions on the node

Parameters

- **task** A TaskManager object.
- **kwargs** Not used.

Raises

RedfishError, if any problem occurs when retrieving all subscriptions.

get_properties()

Return the properties of the interface.

Returns

dictionary of <property name>:<property description> entries.

get_subscription(*task*, ***kwargs*)

Get a specific subscription on the node

Parameters

- **task** A TaskManager object.
- **kwargs** The arguments sent with vendor passthru.

Raises

RedfishError, if any problem occurs when retrieving the subscription.

validate(*task*, *method*, ***kwargs*)

Validate vendor-specific actions.

Checks if a valid vendor passthru method was passed and validates the parameters for the vendor passthru method.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **method** method to be validated.
- **kwargs** kwargs containing the vendor passthru methods parameters.

Raises

InvalidParameterValue, if any of the parameters have invalid value.

Module contents

ironic.drivers.modules.storage package

Submodules

ironic.drivers.modules.storage.cinder module

class ironic.drivers.modules.storage.cinder.**CinderStorage**(**args*, ***kwargs*)

Bases: *StorageInterface*

A storage_interface driver supporting Cinder.

attach_volumes(*task*)

Informs the storage subsystem to attach all volumes for the node.

Parameters
task The task object.

Raises
StorageError If an underlying exception or failure is detected.

detach_volumes(*task*, *connector=None*, *aborting_attach=False*)

Informs the storage subsystem to detach all volumes for the node.

This action is retried in case of failure.

Parameters

- **task** The task object.
- **connector** The dictionary representing a nodes connectivity as defined by _generate_connector(). Generated if not passed.
- **aborting_attach** Boolean representing if this detachment was requested to handle aborting a failed attachment

Raises
StorageError If an underlying exception or failure is detected.

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

should_write_image(*task*)

Determines if deploy should perform the image write-out.

Parameters
task The task object.

Returns
True if the deployment write-out process should be executed.

validate(*task*)

Validate storage_interface configuration for Cinder usage.

In order to provide fail fast functionality prior to nodes being requested to enter the active state, this method performs basic checks of the volume connectors, volume targets, and operator defined capabilities. These checks are to help ensure that we should have a compatible configuration prior to activating the node.

Parameters
task The task object.

Raises
InvalidParameterValue If a misconfiguration or mismatch exists that would prevent storage the cinder storage driver from initializing attachments.

ironic.drivers.modules.storage.external module

class ironic.drivers.modules.storage.external.**ExternalStorage**(**args*, ***kwargs*)

Bases: *StorageInterface*

Externally driven Storage Interface.

attach_volumes(*task*)

Informs the storage subsystem to attach all volumes for the node.

Parameters
task A TaskManager instance.

Raises
UnsupportedDriverExtension

detach_volumes(*task*)

Informs the storage subsystem to detach all volumes for the node.

Parameters
task A TaskManager instance.

Raises
UnsupportedDriverExtension

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

should_write_image(*task*)

Determines if deploy should perform the image write-out.

This enables the user to define a volume and Ironic understand that the image may already exist and we may be booting to that volume.

Parameters
task The task object.

Returns
True if the deployment write-out process should be executed.

validate(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

ironic.drivers.modules.storage.noop module

class ironic.drivers.modules.storage.noop.**NoopStorage**(**args*, ***kwargs*)

Bases: *StorageInterface*

No-op Storage Interface.

attach_volumes(*task*)

Informs the storage subsystem to attach all volumes for the node.

Parameters
task A TaskManager instance.

Raises
UnsupportedDriverExtension

detach_volumes(*task*)

Informs the storage subsystem to detach all volumes for the node.

Parameters
task A TaskManager instance.

Raises
UnsupportedDriverExtension

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

should_write_image(*task*)

Determines if deploy should perform the image write-out.

Parameters
task A TaskManager instance.

Returns
Boolean value to indicate if the interface expects the image to be written by Ironic.

Raises
UnsupportedDriverExtension

validate(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

Module contents

ironic.drivers.modules.switch package

Submodules

ironic.drivers.modules.switch.base module

Base classes for networking service switch drivers.

This module defines the base interface that all switch drivers must implement to provide vendor-specific network switch management functionality.

class ironic.drivers.modules.switch.base.**BaseTranslator**

Bases: object

Base class for configuration translators.

translate_config(*switch_name*, *config*)

Translate a single switch configuration.

Parameters

- **switch_name** Name of the switch
- **config** Dictionary of configuration options for the switch

Returns
Dictionary of section_name -> translated_config_dict

translate_configs(*switch_configs*)

Translate all switch configurations.

Parameters
switch_configs Dictionary of switch_name -> config_dict

Returns
Dictionary of section_name -> translated_config_dict

class ironic.drivers.modules.switch.base.**NoOpSwitchDriver**

Bases: *SwitchDriverBase*

No-operation switch driver for testing and development.

This driver implements all required methods but performs no actual switch operations. Its useful for testing the networking service without requiring actual switch hardware.

DRIVER_NAME = 'noop'

DRIVER_VERSION = '1.0.0'

SUPPORTED = True

delete_lag(*switch_ids*, *lag_name*)

LAG deletion without performing actual deletion.

get_switch_ids()

Return a list of switch identifiers.

get_switch_info(*switch_id*)

Return mock switch information.

classmethod get_translator()

Return the translator.

is_switch_configured(*switch_id*)

Check if NoOp driver can handle the specified switch.

For testing purposes, the NoOp driver always returns True, meaning it can handle any switch. This simplifies testing and provides a fallback for development environments.

Parameters

switch_id Identifier for the switch to check.

Returns

Always True for testing purposes.

reset_port(*switch_id*, *port_name*, *native_vlan=None*, *allowed_vlans=None*, *default_vlan=None*)

Log port reset operation without performing actual reset.

update_lag(*switch_ids*, *lag_name*, *description*, *mode*, *native_vlan*, *aggregation_mode*, *allowed_vlans=None*, *default_vlan=None*)

Log LAG update without performing actual configuration.

update_port(*switch_id*, *port_name*, *description*, *mode*, *native_vlan*, *allowed_vlans=None*, *default_vlan=None*, *lag_name=None*)

Log port update operation without performing actual change.

class ironic.drivers.modules.switch.base.**SwitchDriverBase**

Bases: object

Base class for all switch drivers.

Switch drivers provide vendor-specific implementations for managing network switches. They are loaded dynamically by the networking service based on configuration and handle operations like port configuration, VLAN management, and LAG operations.

DRIVER_NAME = None

DRIVER_VERSION = None

SUPPORTED = True

abstract delete_lag(*switch_ids*, *lag_name*)

Delete LAG configuration from switches.

Parameters

- **switch_ids** List of switch identifiers.
- **lag_name** Name of the LAG to delete.

Raises
ValueError on parameter validation errors.

Raises
SwitchNotFound if the specified switch does not exist.

Raises
SwitchDriverException on configuration failures.

abstract get_switch_ids()

Return a list of switch identifiers.

Returns
List of switch identifiers or an empty list.

abstract get_switch_info(*switch_id*)

Get information about a switch.

This method should return a dictionary containing information about the specified switch. The dictionary can include various attributes that control the behavior of the networking service for this switch.

Supported attributes: - switch_id: Identifier for the switch - device_name: The name of the switch device - device_type: The type of the switch device - ip: IP address of the switch device

Parameters
switch_id Identifier for the switch.

Returns
Dictionary with switch information or None.

Raises
SwitchNotFound if the specified switch does not exist.

abstract classmethod get_translator()

Return the translator for this drivers config entries.

is_switch_configured(*switch_id*)

Check if this driver is configured to manage the specified switch.

This method should return True if this driver is configured to handle the specified switch. This is used by the driver selection logic to determine which driver should handle a specific switch.

Parameters
switch_id Identifier for the switch to check.

Returns
True if this driver can manage the specified switch, False otherwise.

abstract reset_port(*switch_id*, *port_name*, *native_vlan=None*, *allowed_vlans=None*, *default_vlan=None*)

Reset a port configuration on a switch.

Parameters

- **switch_id** Identifier for the switch.
- **port_name** Name of the port on the switch.
- **native_vlan** VLAN ID to be removed from the port.
- **allowed_vlans** List of allowed VLAN IDs to be removed(optional).
- **default_vlan** VLAN ID to restore onto the port(optional).

Raises
ValueError on parameter validation errors.

Raises
SwitchNotFound if the specified switch does not exist.

Raises
SwitchDriverException on configuration failures.

property supported

Return whether this driver is supported.

abstract update_lag(*switch_ids*, *lag_name*, *description*, *mode*, *native_vlan*, *aggregation_mode*, *allowed_vlans=None*, *default_vlan=None*)

Update LAG configuration across switches.

Parameters

- **switch_ids** List of switch identifiers.
- **lag_name** Name of the LAG.
- **description** Description for the LAG.
- **mode** switchport mode (access or trunk).
- **native_vlan** VLAN ID to be set for the LAG.
- **aggregation_mode** Aggregation mode (e.g., lacp, static).
- **allowed_vlans** List of allowed VLAN IDs to be added (optional).
- **default_vlan** VLAN ID to removed from the port(optional).

Raises
ValueError on parameter validation errors.

Raises
SwitchNotFound if the specified switch does not exist.

Raises
SwitchDriverException on configuration failures.

abstract update_port(*switch_id*, *port_name*, *description*, *mode*, *native_vlan*, *allowed_vlans=None*, *default_vlan=None*, *lag_name=None*)

Update a port configuration on a switch.

Parameters

- **switch_id** Identifier for the switch.
- **port_name** Name of the port on the switch.

- **description** Description to set for the port.
- **mode** Port mode (access, trunk, or hybrid).
- **native_vlan** VLAN ID to be set on the port.
- **allowed_vlans** List of allowed VLAN IDs to be added(optional).
- **default_vlan** VLAN ID to removed from the port(optional).
- **lag_name** LAG name if port is part of a channel.

Raises
ValueError on parameter validation errors.

Raises
SwitchNotFound if the specified switch does not exist.

Raises
SwitchDriverException on configuration failures.

exception ironic.drivers.modules.switch.base.**SwitchDriverException**(*message=None*, ***kwargs*)

Bases: *IronicException*

exception ironic.drivers.modules.switch.base.**SwitchMethodNotImplemented**(*message=None*, ***kwargs*)

Bases: *SwitchDriverException*

exception ironic.drivers.modules.switch.base.**SwitchNotFound**(*message=None*, ***kwargs*)

Bases: *SwitchDriverException*

ironic.drivers.modules.switch.generic_switch module

class ironic.drivers.modules.switch.generic_switch.**GenericSwitchDriver**(**args*, ***kwargs*)

Bases: *SwitchDriverBase*

Generic Switch Standalone Driver implementation.

This driver provides a switch driver implementation that is decoupled from the Neutron ML2 interface. It provides the same access to the underlying generic switch device interface but with a non-Neutron specific API interface. It is intended to be used as a driver within Ironics Standalone Networking Service.

DRIVER_NAME = 'generic-switch'

DRIVER_VERSION = '1.0'

SUPPORTED = True

delete_lag(*switch_ids*, *lag_name*)

Delete LAG configuration from switches.

Parameters

- **switch_ids** List of switch identifiers.

- **lag_name** Name of the LAG to delete.

Raises
ValueError on parameter validation errors.

Raises
SwitchNotFound if the specified switch does not exist.

Raises
SwitchDriverException on configuration failures.

get_switch_ids()

Get a list of all switch IDs.

Returns
List of switch IDs

get_switch_info(*switch_id*)

Get information about a switch.

Parameters
switch_id MAC address or hostname of the switch

Returns
Dictionary containing switch information

Raises
SwitchNotFound if switch is not found

classmethod get_translator()

Return the translator for this drivers config entries.

is_switch_configured(*switch_id*)

Check if this driver is configured to manage the specified switch.

This method should return True if this driver is configured to handle the specified switch. This is used by the driver selection logic to determine which driver should handle a specific switch.

Parameters
switch_id Identifier for the switch to check.

Returns
True if this driver can manage the specified switch, False otherwise.

reset_port(*switch_id*, *port_name*, *native_vlan=None*, *allowed_vlans=None*, *default_vlan=None*)

Reset a port configuration on a switch.

Parameters

- **switch_id** Identifier for the switch.
- **port_name** Name of the port on the switch.
- **native_vlan** VLAN ID to be removed from the port.
- **allowed_vlans** List of allowed VLAN IDs to be removed(optional).
- **default_vlan** VLAN ID to restore onto the port(optional).

Raises
ValueError on parameter validation errors.

Raises
SwitchNotFound if the specified switch does not exist.

Raises
SwitchDriverException on configuration failures.

update_lag(*switch_ids*, *lag_name*, *description*, *mode*, *native_vlan*, *aggregation_mode*, *allowed_vlans=None*, *default_vlan=None*)

Update LAG configuration across switches.

Parameters

- **switch_ids** List of switch identifiers.
- **lag_name** Name of the LAG.
- **description** Description for the LAG.
- **mode** switchport mode (access or trunk).
- **native_vlan** VLAN ID to be set for the LAG.
- **aggregation_mode** Aggregation mode (e.g., lacp, static).
- **allowed_vlans** List of allowed VLAN IDs to be added (optional).
- **default_vlan** VLAN ID to removed from the port(optional).

Raises
ValueError on parameter validation errors.

Raises
SwitchNotFound if the specified switch does not exist.

Raises
SwitchDriverException on configuration failures.

update_port(*switch_id*, *port_name*, *description*, *mode*, *native_vlan*, *allowed_vlans=None*, *default_vlan=None*, *lag_name=None*, ***kwargs*)

Update a port configuration on a switch.

Parameters

- **switch_id** Identifier for the switch.
- **port_name** Name of the port on the switch.
- **description** Description to set for the port.
- **mode** Port mode (access, trunk, or hybrid).
- **native_vlan** VLAN ID to be set on the port.
- **allowed_vlans** List of allowed VLAN IDs to be added(optional).
- **default_vlan** VLAN ID to removed from the port(optional).
- **lag_name** LAG name if port is part of a LAG.

Raises
ValueError on parameter validation errors.

Raises
SwitchNotFound if the specified switch does not exist.

Raises
SwitchDriverException on configuration failures.

class ironic.drivers.modules.switch.generic_switch.**GenericSwitchTranslator**

Bases: *BaseTranslator*

Translates generic config to networking-generic-switch format.

Module contents

Submodules

ironic.drivers.modules.agent module

class ironic.drivers.modules.agent.**AgentDeploy**(**args*, ***kwargs*)

Bases: *CustomAgentDeploy*

Interface for deploy-related actions.

configure_local_boot(*task*, *root_uuid=None*, *efi_system_part_uuid=None*, *prep_boot_part_uuid=None*)

Helper method to configure local boot on the node.

This method triggers bootloader installation on the node. On successful installation of bootloader, this method sets the node to boot from disk.

Parameters

- **task** a TaskManager object containing the node
- **root_uuid** The UUID of the root partition. This is used for identifying the partition which contains the image deployed or None in case of whole disk images which we expect to already have a bootloader installed.
- **efi_system_part_uuid** The UUID of the efi system partition. This is used only in uefi boot mode.
- **prep_boot_part_uuid** The UUID of the PReP Boot partition. This is used only for booting ppc64* hardware.

Raises
InstanceDeployFailure if bootloader installation failed or on encountering error while setting the boot device on the node.

prepare_instance_boot(*task*)

Prepare instance for booting.

The base version only calls prepare_instance on the boot interface.

prepare_instance_to_boot(*task*, *root_uuid*, *efi_sys_uuid*, *prep_boot_part_uuid=None*)

Prepares instance to boot.

Parameters

- **task** a TaskManager object containing the node
- **root_uuid** the UUID for root partition

- **efi_sys_uuid** the UUID for the efi partition

Raises
InvalidState if fails to prepare instance

validate(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the properties of the supplied node contain the required information for this driver to deploy images to the node.

Parameters
task a TaskManager instance

Raises
MissingParameterValue, if any of the required parameters are missing.

Raises
InvalidParameterValue, if any of the parameters have invalid value.

write_image(*task*)

class ironic.drivers.modules.agent.**AgentRAID**(**args*, ***kwargs*)

Bases: *RAIDInterface*

Implementation of RAIDInterface which uses agent ramdisk.

apply_configuration(*task*, *raid_config*, *delete_existing=True*)

Applies RAID configuration on the given node.

Parameters

- **task** A TaskManager instance.
- **raid_config** The RAID configuration to apply.
- **delete_existing** Setting this to True indicates to delete RAID configuration prior to creating the new configuration.

Raises
InvalidParameterValue, if the RAID configuration is invalid.

Returns
states.DEPLOYWAIT if RAID configuration is in progress asynchronously or None if it is complete.

create_configuration(*task*, *create_root_volume=True*, *create_nonroot_volumes=True*)

Create a RAID configuration on a bare metal using agent ramdisk.

This method creates a RAID configuration on the given node.

Parameters

- **task** a TaskManager instance.
- **create_root_volume** If True, a root volume is created during RAID configuration. Otherwise, no root volume is created. Default is True.
- **create_nonroot_volumes** If True, non-root volumes are created. If False, no non-root volumes are created. Default is True.

Returns
states.CLEANWAIT if operation was successfully invoked.

Raises
MissingParameterValue, if node.target_raid_config is missing or was found to be empty after skipping root volume and/or non-root volumes.

`delete_configuration`(*task*)

Deletes RAID configuration on the given node.

Parameters
task a TaskManager instance.

Returns
states.CLEANWAIT if operation was successfully invoked

`get_clean_steps`(*task*)

Get the list of clean steps from the agent.

Parameters
task a TaskManager object containing the node

Raises
NodeCleaningFailure if the clean steps are not yet available (cached), for example, when a node has just been enrolled and has not been cleaned yet.

Returns
A list of clean step dictionaries

`get_deploy_steps`(*task*)

Get the list of deploy steps from the agent.

Parameters
task a TaskManager object containing the node

Raises
InstanceDeployFailure if the deploy steps are not yet available (cached), for example, when a node has just been enrolled and has not been deployed yet.

Returns
A list of deploy step dictionaries

`get_properties`()

Return the properties of the interface.

`class ironic.drivers.modules.agent.AgentRescue`(**args*, ***kwargs*)

Bases: *RescueInterface*

Implementation of RescueInterface which uses agent ramdisk.

`clean_up`(*task*)

Clean up after RESCUEWAIT timeout/failure or finishing rescue.

Rescue password should be removed from the node and ramdisk boot environment should be cleaned if Ironic is managing the ramdisk boot.

Parameters
task a TaskManager instance with the node.

Raises
NetworkError if the rescue ports cannot be removed.

`get_properties()`

Return the properties of the interface.

`rescue(task)`

Boot a rescue ramdisk on the node.

Parameters
task a TaskManager instance.

Raises
NetworkError if the tenant ports cannot be removed.

Raises
InvalidParameterValue when the wrong power state is specified or the wrong driver info is specified for power management.

Raises
other exceptions by the nodes power driver if something wrong occurred during the power action.

Raises
any boot interfaces prepare_ramdisk exceptions.

Returns
Returns states.RESCUEWAIT

`unrescue(task)`

Attempt to move a rescued node back to active state.

Parameters
task a TaskManager instance.

Raises
NetworkError if the rescue ports cannot be removed.

Raises
InvalidParameterValue when the wrong power state is specified or the wrong driver info is specified for power management.

Raises
other exceptions by the nodes power driver if something wrong occurred during the power action.

Raises
any boot interfaces prepare_instance exceptions.

Returns
Returns states.ACTIVE

`validate(task)`

Validate that the node has required properties for agent rescue.

Parameters
task a TaskManager instance with the node being checked

Raises
InvalidParameterValue if instance_info/rescue_password has empty password or rescuing network UUID config option has an invalid value.

Raises
MissingParameterValue if node is missing one or more required parameters

class ironic.drivers.modules.agent.**BootcAgentDeploy**(**args*, ***kwargs*)

Bases: *CustomAgentDeploy*

Interface for deploy-related actions.

execute_bootc_install(*task*)

set_boot_to_disk(*task*)

Sets the node to boot from disk.

In some cases, other steps may handle aspects like bootloaders and UEFI NVRAM entries required to boot. That leaves one last aspect, resetting the node to boot from disk.

This primarily exists for compatibility reasons of flow for Ironic, but we know some BMCs *really* need to be still told to boot from disk. The exception to this is Lenovo hardware, where we skip the action because it can create a UEFI NVRAM update failure case, which reverts the NVRAM state to last known good configuration.

Parameters
task A Taskmanager object.

supports_deploy(*task*)

Check if deploy is supported for the given node by this interface.

Parameters
task A TaskManager instance containing the node to act on.

Returns
True if the image_source has an oci:// URL scheme and repository introspection shows it is a bootc image.

validate(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the properties of the supplied node contain the required information for this driver to deploy images to the node.

Parameters
task a TaskManager instance

Raises
MissingParameterValue, if any of the required parameters are missing.

Raises
InvalidParameterValue, if any of the parameters have invalid value.

class ironic.drivers.modules.agent.**CustomAgentDeploy**(**args*, ***kwargs*)

Bases: *AgentBaseMixin*, *HeartbeatMixin*, *AgentOobStepsMixin*, *DeployInterface*

A deploy interface that relies on a custom agent to deploy.

Only provides the basic deploy steps to start the ramdisk, tear down the ramdisk and prepare the instance boot.

clean_up(*task*)

Clean up the deployment environment for this node.

If preparation of the deployment environment ahead of time is possible, this method should be implemented by the driver. It should erase anything cached by the *prepare* method.

If implemented, this method must be idempotent. It may be called multiple times for the same node on the same conductor, and it may be called by multiple conductors in parallel. Therefore, it must not require an exclusive lock.

This method is called before *tear_down*.

Parameters
task a TaskManager instance.

deploy(*task*)

Perform a deployment to a node.

Perform the necessary work to deploy an image onto the specified node. This method will be called after prepare(), which may have already performed any preparatory steps, such as pre-caching some data for the node.

Parameters
task a TaskManager instance.

Returns
status of the deploy. One of ironic.common.states.

execute_deploy_step(*task*, *step*)

Execute a deploy step.

Were trying to find a step among both out-of-band and in-band steps. In case of duplicates, out-of-band steps take priority. This property allows having an out-of-band deploy step that calls into a corresponding in-band step after some preparation (e.g. with additional input).

Parameters

- **task** a TaskManager object containing the node
- **step** a deploy step dictionary to execute

Raises
InstanceDeployFailure if the agent does not return a command status

Returns
states.DEPLOYWAIT to signify the step will be completed async

get_deploy_steps(*task*)

Get the list of deploy steps from the agent.

Parameters
task a TaskManager object containing the node

Raises
InstanceDeployFailure if the deploy steps are not yet available (cached), for example, when a node has just been enrolled and has not been deployed yet.

Returns
A list of deploy step dictionaries

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

prepare(*task*)

Prepare the deployment environment for this node.

Parameters
task a TaskManager instance.

Raises
NetworkError: if the previous cleaning ports cannot be removed or if new cleaning ports cannot be created.

Raises
InvalidParameterValue when the wrong power state is specified or the wrong driver info is specified for power management.

Raises
StorageError If the storage driver is unable to attach the configured volumes.

Raises
other exceptions by the nodes power driver if something wrong occurred during the power action.

Raises
exception.ImageRefValidationFailed if image_source is not Glance href and is not HTTP(S) URL.

Raises
exception.InvalidParameterValue if network validation fails.

Raises
any boot interfaces prepare_ramdisk exceptions.

prepare_instance_boot(*task*)

Prepare instance for booting.

The base version only calls prepare_instance on the boot interface.

should_manage_boot(*task*)

Whether agent boot is managed by ironic.

tear_down_agent(*task*)

A deploy step to tear down the agent.

Parameters
task a TaskManager object containing the node

validate(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the properties of the supplied node contain the required information for this driver to deploy images to the node.

Parameters
task a TaskManager instance

Raises
MissingParameterValue, if any of the required parameters are missing.

Raises
InvalidParameterValue, if any of the parameters have invalid value.

ironic.drivers.modules.agent.**set_boot_to_disk**(*task*, *target_boot_mode=None*)

Boot a node to disk.

This is a helper method to reduce duplication of code around handling vendor specifics for setting boot modes between multiple deployment interfaces inside of Ironic.

Parameters

- **task** A Taskmanager object.
- **target_boot_mode** The target boot_mode, defaults to UEFI.

ironic.drivers.modules.agent.**soft_power_off**(*task*, *client=None*)

Power off the node using the agent API.

ironic.drivers.modules.agent.**validate_http_provisioning_configuration**(*node*)

Validate configuration options required to perform HTTP provisioning.

Parameters
node an ironic node object

Raises
MissingParameterValue if required option(s) is not set.

ironic.drivers.modules.agent.**validate_image_proxies**(*node*)

Check that the provided proxy parameters are valid.

Parameters
node an Ironic node.

Raises
InvalidParameterValue if any of the provided proxy parameters are incorrect.

ironic.drivers.modules.agent_base module

class ironic.drivers.modules.agent_base.**AgentBaseMixin**

Bases: object

Mixin with base methods not relying on any deploy steps.

Provides full support for in-band and out-of-band cleaning and the machinery to support both deploy and clean in-band steps.

clean_up(*task*)

Clean up the deployment environment for the tasks node.

Unlinks TFTP and instance images and triggers image cache cleanup. Removes the TFTP configuration files for this node.

Parameters
task a TaskManager instance containing the node to act on.

`execute_clean_step`(*task*, *step*)

Execute a clean step asynchronously on the agent.

Parameters

- **task** a TaskManager object containing the node
- **step** a clean step dictionary to execute

Raises

NodeCleaningFailure if the agent does not return a command status

Returns

states.CLEANWAIT to signify the step will be completed async

`execute_service_step`(*task*, *step*)

Execute a service step asynchronously on the agent.

Parameters

- **task** a TaskManager object containing the node
- **step** a service step dictionary to execute

Raises

NodeServicingFailure if the agent does not return a command status

Returns

states.SERVICEWAIT to signify the step will be completed async

`get_clean_steps`(*task*)

Get the list of clean steps from the agent.

Parameters

task a TaskManager object containing the node

Raises

NodeCleaningFailure if the clean steps are not yet available (cached), for example, when a node has just been enrolled and has not been cleaned yet.

Returns

A list of clean step dictionaries

`get_service_steps`(*task*)

Get the list of clean steps from the agent.

Parameters

task a TaskManager object containing the node

Returns

A list of service step dictionaries, if an error occurs, then an empty list is returned.

`prepare_cleaning`(*task*)

Boot into the agent to prepare for cleaning.

Parameters

task a TaskManager object containing the node

Raises
NodeCleaningFailure, NetworkError if the previous cleaning ports cannot be removed or if new cleaning ports cannot be created.

Raises
InvalidParameterValue if cleaning network UUID config option has an invalid value.

Returns
states.CLEANWAIT to signify an asynchronous prepare

`prepare_service`(*task*)

Boot into the agent to prepare for service.

Parameters
task a TaskManager object containing the node

Raises
NodeServiceFailure, NetworkError if: the previous service ports cannot be removed or if new service ports cannot be created.

Raises
InvalidParameterValue if cleaning network UUID config option has an invalid value.

Returns
states.SERVICEWAIT to signify an asynchronous prepare

`process_next_step`(*task*, *step_type*, ***kwargs*)

Start the next clean/deploy step if the previous one is complete.

In order to avoid errors and make agent upgrades painless, the agent compares the version of all hardware managers at the start of the process (the agents get_clean|deploy_steps() call) and before executing each step. If the version has changed between steps, the agent is unable to tell if an ordering change will cause an issue so it returns VERSION_MISMATCH. For automated cleaning, we restart the entire cleaning cycle. For manual cleaning or deploy, we dont.

Additionally, if a step includes the reboot_requested property set to True, this method will coordinate the reboot once the step is completed.

`refresh_steps`(*task*, *step_type*)

Refresh the nodes cached clean/deploy steps from the booted agent.

Gets the nodes steps from the booted agent and caches them. The steps are cached to make get_clean_steps() calls synchronous, and should be refreshed as soon as the agent boots to start cleaning/deploy or if cleaning is restarted because of a hardware manager version mismatch.

Parameters

- **task** a TaskManager instance
- **step_type** clean or deploy

Raises
NodeCleaningFailure or InstanceDeployFailure if the agent returns invalid results

`restore_interface`(*task*)

Restore the original deploy interface for the node.

This restores the deploy interface to the original interface in case it was changed by switch_interface.

Parameters
`task` a TaskManager instance containing the node to act on.

`should_manage_boot`(*task*)

Whether agent boot is managed by ironic.

`take_over`(*task*)

Take over management of this node from a dead conductor.

Parameters
`task` a TaskManager instance.

`tear_down`(*task*)

Tear down a previous deployment on the tasks node.

Power off the node. All actual clean-up is done in the clean_up() method which should be called separately.

Parameters
`task` a TaskManager instance containing the node to act on.

Returns
deploy state DELETED.

Raises
NetworkError if the cleaning ports cannot be removed.

Raises
InvalidParameterValue when the wrong state is specified or the wrong driver info is specified.

Raises
StorageError when volume detachment fails.

Raises
other exceptions by the nodes power driver if something wrong occurred during the power action.

`tear_down_cleaning`(*task*)

Clean up the PXE and DHCP files after cleaning.

Parameters
`task` a TaskManager object containing the node

Raises
NodeCleaningFailure, NetworkError if the cleaning ports cannot be removed

`tear_down_service`(*task*)

Clean up the PXE and DHCP files after service.

Parameters
`task` a TaskManager object containing the node

Raises
NodeServiceFailure, NetworkError if the servicing ports cannot be removed

class ironic.drivers.modules.agent_base.**AgentOobStepsMixin**

Bases: object

Mixin with out-of-band deploy steps.

boot_instance(*task*)

Deploy step to boot the final instance.

Parameters
task a TaskManager object containing the node

switch_to_tenant_network(*task*)

Deploy step to switch the node to the tenant network.

Parameters
task a TaskManager object containing the node

class ironic.drivers.modules.agent_base.**HeartbeatMixin**

Bases: object

Mixin class implementing heartbeat processing.

collect_deploy_logs = True

continue_cleaning(*task*)

Start the next cleaning step if the previous one is complete.

Parameters
task a TaskManager instance

continue_servicing(*task*)

Start the next cleaning step if the previous one is complete.

Parameters
task a TaskManager instance

heartbeat(*task*, *callback_url*, *agent_version*, *agent_verify_ca=None*, *agent_status=None*, *agent_status_message=None*)

Process a heartbeat.

Parameters

- **task** task to work with.
- **callback_url** agent HTTP API URL.
- **agent_version** The version of the agent that is heartbeating
- **agent_verify_ca** TLS certificate for the agent.
- **agent_status** Status of the heartbeating agent
- **agent_status_message** Status message that describes the agent_status

heartbeat_allowed(*node*)

process_next_step(*task*, *step_type*)

Start the next step if the previous one is complete.

Parameters

- **task** a TaskManager instance
- **step_type** clean, deploy, service

reboot_to_instance(*task*)

Method invoked after the deployment is completed.

Parameters

task a TaskManager instance

refresh_clean_steps(*task*)

Refresh the nodes cached clean steps

Parameters

task a TaskManager instance

refresh_service_steps(*task*)

Refresh the nodes cached service steps

Parameters

task a TaskManager instance

refresh_steps(*task*, *step_type*)

Refresh the nodes cached clean steps

Parameters

- **task** a TaskManager instance
- **step_type** clean or deploy

ironic.drivers.modules.agent_base.**execute_clean_step**(*task*, *step*)

ironic.drivers.modules.agent_base.**execute_step**(*task*, *step*, *step_type*, *client=None*)

Execute a clean or deploy step asynchronously on the agent.

Parameters

- **task** a TaskManager object containing the node
- **step** a step dictionary to execute
- **step_type** clean or deploy
- **client** agent client (if available)

Raises

NodeCleaningFailure (clean step) or InstanceDeployFailure (deploy step) or NodeServicingFailure (service step) if the agent does not return a command status.

Returns

states.CLEANWAIT/DEPLOYWAIT/SERVICEWAIT to signify the step will be completed async

ironic.drivers.modules.agent_base.**find_step**(*task*, *step_type*, *interface*, *name*)

Find the given in-band step.

ironic.drivers.modules.agent_base.**get_steps**(*task*, *step_type*, *interface=None*, *override_priorities=None*)

Get the list of cached clean or deploy steps from the agent.

The steps cache is updated at the beginning of cleaning or deploy.

Parameters

- **task** a TaskManager object containing the node
- **step_type** clean or deploy
- **interface** The interface for which clean/deploy steps are to be returned. If this is not provided, it returns the steps for all interfaces.
- **override_priorities** a dictionary with keys being step names and values being new priorities for them. If a step isnt in this dictionary, the steps original priority is used.

Returns

A list of clean/deploy step dictionaries

ironic.drivers.modules.agent_base.**log_and_raise_deployment_error**(*task*, *msg*, *collect_logs=True*, *exc=None*)

Helper method to log the error and raise exception.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **msg** the message to set in last_error of the node.
- **collect_logs** Boolean indicating whether to attempt to collect logs from IPA-based ramdisk. Defaults to True. Actual log collection is also affected by CONF.agent.deploy_logs_collect config option.
- **exc** Exception that caused the failure.

ironic.drivers.modules.agent_base.**post_clean_step_hook**(*interface*, *step*)

Decorator method for adding a post clean step hook.

This is a mechanism for adding a post clean step hook for a particular clean step. The hook will get executed after the clean step gets executed successfully. The hook is not invoked on failure of the clean step.

Any method to be made as a hook may be decorated with @post_clean_step_hook mentioning the interface and step after which the hook should be executed. A TaskManager instance and the object for the last completed command (provided by agent) will be passed to the hook method. The return value of this method will be ignored. Any exception raised by this method will be treated as a failure of the clean step and the node will be moved to CLEANFAIL state.

Parameters

- **interface** name of the interface
- **step** The name of the step after which it should be executed.

Returns

A method which registers the given method as a post clean step hook.

ironic.drivers.modules.agent_base.**post_deploy_step_hook**(*interface*, *step*)

Decorator method for adding a post deploy step hook.

This is a mechanism for adding a post deploy step hook for a particular deploy step. The hook will get executed after the deploy step gets executed successfully. The hook is not invoked on failure of the deploy step.

Any method to be made as a hook may be decorated with @post_deploy_step_hook mentioning the interface and step after which the hook should be executed. A TaskManager instance and the object for the last completed command (provided by agent) will be passed to the hook method. The return value of this method will be ignored. Any exception raised by this method will be treated as a failure of the deploy step and the node will be moved to DEPLOYFAIL state.

Parameters

- **interface** name of the interface
- **step** The name of the step after which it should be executed.

Returns

A method which registers the given method as a post deploy step hook.

ironic.drivers.modules.agent_client module

class ironic.drivers.modules.agent_client.**AgentClient**

Bases: object

Client for interacting with nodes via a REST API.

collect_system_logs(*node*)

Collect and package diagnostic and support data from the ramdisk.

Parameters

node A Node object.

Raises

IronicException when failed to issue the request or there was a malformed response from the agent.

Raises

AgentAPIError when agent failed to execute specified command.

Raises

AgentInProgress when the command fails to execute as the agent is presently executing the prior command.

Returns

A dict containing command response from agent. See *get_commands_status()* for a command result sample.

execute_clean_step(*step*, *node*, *ports*)

Execute specified clean step.

Parameters

- **step** A clean step dictionary to execute.
- **node** A Node object.
- **ports** Ports associated with the node.

Raises
IronicException when failed to issue the request or there was a malformed response from the agent.

Raises
AgentAPIError when agent failed to execute specified command.

Raises
AgentInProgress when the command fails to execute as the agent is presently executing the prior command.

Returns

A dict containing command response from agent. See *get_commands_status()* for a command result sample. The value of key command_result is in the form of:

```
{
  'clean_result': <the result of execution, step specific>,
  'clean_step': <the clean step issued to agent>
}
```

execute_deploy_step(*step*, *node*, *ports*)

Execute specified deploy step.

Parameters

- **step** A deploy step dictionary to execute.
- **node** A Node object.
- **ports** Ports associated with the node.

Raises
IronicException when failed to issue the request or there was a malformed response from the agent.

Raises
AgentAPIError when agent failed to execute specified command.

Raises
AgentInProgress when the command fails to execute as the agent is presently executing the prior command.

Returns

A dict containing command response from agent. See *get_commands_status()* for a command result sample. The value of key command_result is in the form of:

```
{
  'deploy_result': <the result of execution, step specific>
↪,
  'deploy_step': <the deploy step issued to agent>
}
```

execute_service_step(*step*, *node*, *ports*)

Execute specified service step.

Parameters

- **step** A service step dictionary to execute.
- **node** A Node object.
- **ports** Ports associated with the node.

Raises

IronicException when failed to issue the request or there was a malformed response from the agent.

Raises

AgentAPIError when agent failed to execute specified command.

Raises

AgentInProgress when the command fails to execute as the agent is presently executing the prior command.

Returns

A dict containing command response from agent. See *get_commands_status()* for a command result sample. The value of key command_result is in the form of:

```
{
  'service_result': <the result of execution, step␣
↪specific>,
  'service_step': <the service step issued to agent>
}
```

finalize_rescue(*node*)

Instruct the ramdisk to finalize entering of rescue mode.

Parameters

node A Node object.

Raises

IronicException if rescue_password is missing, or when failed to issue the request, or there was a malformed response from the agent.

Raises

AgentAPIError when agent failed to execute specified command.

Raises

AgentInProgress when the command fails to execute as the agent is presently executing the prior command.

Raises

InstanceRescueFailure when the agent ramdisk is too old to support transmission of the rescue password.

Returns

A dict containing command response from agent. See *get_commands_status()* for a command result sample.

get_clean_steps(*node*, *ports*)

Get clean steps from agent.

Parameters

- **node** A node object.
- **ports** Ports associated with the node.

Raises

IronicException when failed to issue the request or there was a malformed response from the agent.

Raises

AgentAPIError when agent failed to execute specified command.

Raises

AgentInProgress when the command fails to execute as the agent is presently executing the prior command.

Returns

A dict containing command response from agent. See *get_commands_status()* for a command result sample. The value of key command_result is in the form of:

```
{
  'clean_steps': <a list of clean steps>,
  'hardware_manager_version': <manager version>
}
```

get_commands_status(*node*, *retry_connection=True*, *expect_errors=False*)

Get command status from agent.

Parameters

- **node** A Node object.
- **retry_connection** Whether to retry connection problems.
- **expect_errors** If True, do not log connection problems as errors.

Returns

A list of command results, each result is related to a command been issued to agent. A typical result can be:

```
{
  'command_name': <command name related to the result>,
  'command_params': <params related with the command>,
  'command_status': <current command status,
                     e.g. 'RUNNING', 'SUCCEEDED', 'FAILED'>,
  'command_error': <error message if command execution
                    failed>,
  'command_result': <command result if command execution
                     succeeded, the value is command␣
↪specific,
                     e.g.:
                     * a dictionary containing keys clean_
↪result
```

(continues on next page)

(continued from previous page)

```
                    and clean_step for the command
                    clean.execute_clean_step;
                  * a dictionary containing keys deploy_
↪result
                    and deploy_step for the command
                    deploy.execute_deploy_step;
                  * a string representing result message␣
↪for
                    the command standby.cache_image;
                  * None for the command standby.sync.>
}
```

get_deploy_steps(*node*, *ports*)

Get deploy steps from agent.

Parameters

- **node** A node object.
- **ports** Ports associated with the node.

Raises

IronicException when failed to issue the request or there was a malformed response from the agent.

Raises

AgentAPIError when agent failed to execute specified command.

Raises

AgentConnectionFailed when an a transient connection failure breaks the connection while the request is being processed.

Returns

A dict containing command response from agent. See *get_commands_status()* for a command result sample. The value of key command_result is in the form of:

```
{
  'deploy_steps': <a list of deploy steps>,
  'hardware_manager_version': <manager version>
}
```

get_last_command_status(*node*, *method*)

Get the last status for the given command.

Parameters

- **node** A Node object.
- **method** Command name.

Returns

A dict containing command status from agent or None if the command was not found.

get_partition_uuids(*node*)

Get deploy steps from agent.

Parameters

node A node object.

Raises

IronicException when failed to issue the request or there was a malformed response from the agent.

Raises

AgentAPIError when agent failed to execute specified command.

Raises

AgentInProgress when the command fails to execute as the agent is presently executing the prior command.

Returns

A dict containing command response from agent.

get_service_steps(*node*, *ports*)

Get service steps from agent.

Parameters

- **node** A node object.
- **ports** Ports associated with the node.

Raises

IronicException when failed to issue the request or there was a malformed response from the agent.

Raises

AgentAPIError when agent failed to execute specified command.

Raises

AgentInProgress when the command fails to execute as the agent is presently executing the prior command.

Returns

A dict containing command response from agent. See *get_commands_status()* for a command result sample. The value of key command_result is in the form of:

```
{
  'service_steps': <a list of service steps>,
  'hardware_manager_version': <manager version>
}
```

install_bootloader(*node*, *root_uuid*, *target_boot_mode*, *efi_system_part_uuid=None*, *prep_boot_part_uuid=None*, *software_raid=False*)

Install a boot loader on the image.

Parameters

- **node** A node object.

- **root_uuid** The UUID of the root partition.
- **target_boot_mode** The target deployment boot mode.
- **efi_system_part_uuid** The UUID of the efi system partition where the bootloader will be installed to, only used for uefi boot mode.
- **prep_boot_part_uuid** The UUID of the PReP Boot partition where the bootloader will be installed to when local booting a partition image on a ppc64* system.

Raises
IronicException when failed to issue the request or there was a malformed response from the agent.

Raises
AgentAPIError when agent failed to execute specified command.

Raises
AgentInProgress when the command fails to execute as the agent is presently executing the prior command.

Returns
A dict containing command response from agent. See *get_commands_status()* for a command result sample.

lockdown(*node*, *fail_if_unavailable=True*)

Lock down the agent so that its not usable any more.

Parameters

- **node** A Node object.
- **fail_if_unavailable** Whether to fail this call if the agent is already unavailable.

Raises
IronicException when failed to issue the request or there was a malformed response from the agent.

Raises
AgentAPIError when agent failed to execute specified command.

Raises
AgentInProgress when the command fails to execute as the agent is presently executing the prior command.

power_off(*node*)

Soft powers off the bare metal node by shutting down ramdisk OS.

Parameters
node A Node object.

Raises
IronicException when failed to issue the request or there was a malformed response from the agent.

Raises
AgentAPIError when agent failed to execute specified command.

Raises
AgentInProgress when the command fails to execute as the agent is presently executing the prior command.

Returns
A dict containing command response from agent. See *get_commands_status()* for a command result sample.

reboot(*node*)

Soft reboots the bare metal node by shutting down ramdisk OS.

Parameters
node A Node object.

Raises
IronicException when failed to issue the request or there was a malformed response from the agent.

Raises
AgentAPIError when agent failed to execute specified command.

Raises
AgentInProgress when the command fails to execute as the agent is presently executing the prior command.

Returns
A dict containing command response from agent. See *get_commands_status()* for a command result sample.

sync(*node*)

Flush file system buffers forcing changed blocks to disk.

Parameters
node A Node object.

Raises
IronicException when failed to issue the request or there was a malformed response from the agent.

Raises
AgentAPIError when agent failed to execute specified command.

Raises
AgentInProgress when the command fails to execute as the agent is presently executing the prior command.

Returns
A dict containing command response from agent. See *get_commands_status()* for a command result sample.

ironic.drivers.modules.agent_client.**get_client**(*task*)

Get client for this node.

ironic.drivers.modules.agent_client.**get_command_error**(*command*)

Extract an error string from the command result.

Parameters
command Command information from the agent.

Returns
Error string.

ironic.drivers.modules.agent_power module

The agent power interface.

class ironic.drivers.modules.agent_power.**AgentPower**(**args*, ***kwargs*)

Bases: *PowerInterface*

Power interface using the running agent for power actions.

get_power_state(*task*)

Return the power state of the tasks node.

Essentially, the only known state is POWER ON, everything else is an error (or more precisely `None`).

Parameters
task A TaskManager instance containing the node to act on.

Returns
A power state. One of *ironic.common.states*.

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

get_supported_power_states(*task*)

Get a list of the supported power states.

Only contains REBOOT.

Parameters
task A TaskManager instance containing the node to act on.

Returns
A list with the supported power states defined in *ironic.common.states*.

reboot(*task*, *timeout=None*)

Perform a reboot of the tasks node.

Only soft reboot is implemented.

Parameters

- **task** A TaskManager instance containing the node to act on.
- **timeout** timeout (in seconds) positive integer (> 0) for any power state. `None` indicates to use default timeout.

set_power_state(*task*, *power_state*, *timeout=None*)

Set the power state of the tasks node.

Parameters

- **task** A TaskManager instance containing the node to act on.

- **power_state** Power state from *ironic.common.states*. Only REBOOT and SOFT_REBOOT are supported and are synonymous.
- **timeout** timeout (in seconds) positive integer (> 0) for any power state. None indicates to use default timeout.

Raises
PowerStateFailure on non-supported power state.

supports_power_sync(*task*)

Check if power sync is supported for the given node.

Not supported for the agent power since it is not possible to power on/off nodes.

Parameters
task A TaskManager instance containing the node to act on with a **shared** lock.

Returns
boolean, whether power sync is supported.

validate(*task*)

Validate the driver-specific Node deployment info.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

ironic.drivers.modules.autodetect module

class ironic.drivers.modules.autodetect.**AutodetectDeploy**(**args*, ***kwargs*)

Bases: *DeployInterface*

Deploy interface that auto-detects the appropriate deployment method.

clean_up(*task*)

Clean up the deployment environment for the tasks node.

Parameters
task A TaskManager instance containing the node to act on.

deploy(*task*)

Perform a deployment to the tasks node.

This method should not be called directly as the autodetect interface is expected to switch to a concrete interface during switch_interface(). If this is called, it means the interface switch did not happen.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InstanceDeployFailure if deployment fails.

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

prepare(*task*)

Prepare the deployment environment for the tasks node.

switch_interface(*task*)

Switch the interface to use for deployment.

This calls supports_deploy() methods of deploy interfaces configured in the autodetect_deploy_interfaces option, in order, to determine which interface is supported for the current node/image. The first interface that returns True from supports_deploy() is chosen. If no interfaces are detected as supported, the last interface in the list is chosen as the fallback.

Raises
InvalidParameterValue if the interface is not enabled.

Parameters
task A TaskManager instance containing the node to act on.

take_over(*task*)

Take over management of this tasks node from a dead conductor.

Parameters
task A TaskManager instance containing the node to act on.

tear_down(*task*)

Tear down a previous deployment on the tasks node.

Parameters
task A TaskManager instance containing the node to act on.

Returns
deploy state DELETED.

validate(*task*)

Validate the driver-specific Node deployment info.

This method creates the deploy interface that would be switched to and calls its validate() method.

Parameters
task A TaskManager instance containing the node to act on.

Raises
MissingParameterValue if required parameters are missing.

ironic.drivers.modules.boot_mode_utils module

ironic.drivers.modules.boot_mode_utils.**configure_secure_boot_if_needed**(*task*)

Configures secure boot if it has been requested for the node.

ironic.drivers.modules.boot_mode_utils.**deconfigure_secure_boot_if_needed**(*task*)

Deconfigures secure boot if it has been requested for the node.

ironic.drivers.modules.boot_mode_utils.**get_boot_mode**(*node*)

Returns the boot mode.

Parameters
node an ironic node object.

Returns
bios or uefi

Raises
InvalidParameterValue, if the node boot mode disagrees with the boot mode set to node properties/capabilities

ironic.drivers.modules.boot_mode_utils.**get_boot_mode_for_deploy**(*node*)

Returns the boot mode that would be used for deploy.

This method returns boot mode to be used for deploy. It returns uefi if secure_boot is set to true in instance_info/capabilities of node. Otherwise it returns value of boot_mode in properties/capabilities of node if set. If that is not set, it returns boot mode in internal_driver_info/deploy_boot_mode for the node. If that is not set, it returns boot mode in instance_info/deploy_boot_mode for the node. It would return None if boot mode is present neither in capabilities of node properties nor in nodes internal_driver_info nor in nodes instance_info (which could also be None).

Parameters
node an ironic node object.

Returns
bios, uefi or None

Raises
InvalidParameterValue, if the node boot mode disagrees with the boot mode set to node properties/capabilities

ironic.drivers.modules.boot_mode_utils.**is_secure_boot_requested**(*node*)

Returns True if secure_boot is requested for deploy.

This method checks node property for secure_boot and returns True if it is requested.

Parameters
node a single Node.

Raises
InvalidParameterValue if the capabilities string is not a dictionary or is malformed.

Returns
True if secure_boot is requested.

ironic.drivers.modules.boot_mode_utils.**sync_boot_mode**(*task*)

Set nodes boot mode from bare metal configuration

Attempt to read currently set boot mode off the bare metal machine. Also read nodes boot mode configuration:

- If BM driver does not implement getting boot mode, assume BM boot mode is not set and apply the logic that follows
- If Ironic node boot mode is not set and BM node boot mode is not set - set Ironic boot mode to *[deploy]/default_boot_mode*
- If Ironic node boot mode is not set and BM node boot mode is set - set BM node boot mode on the Ironic node

- If Ironic node boot mode is set and BM node boot mode is not set - set Ironic boot mode to BM boot mode
- If both Ironic and BM node boot modes are set but they differ - try to set Ironic boot mode to BM boot mode and fail hard if underlying hardware type does not support setting boot mode

In the end, the new boot mode may be set in driver_internal_info/deploy_boot_mode.

Parameters
task a task object

ironic.drivers.modules.console_utils module

Ironic console utilities.

ironic.drivers.modules.console_utils.**acquire_port**(*host=None*)

Returns a free TCP port on current host.

Find and returns a free TCP port in the range(s) of CONF.console.port_range.

ironic.drivers.modules.console_utils.**get_socat_console_url**(*port*)

Get a URL to access the console via socat.

Parameters
port the terminal port (integer) for the node

Returns
an access URL to the socat console of the node

ironic.drivers.modules.console_utils.**make_persistent_password_file**(*path*, *password*)

Writes a file containing a password until deleted.

ironic.drivers.modules.console_utils.**release_port**(*port*)

Release specified TCP port.

ironic.drivers.modules.console_utils.**start_socat_console**(*node_uuid*, *port*, *console_cmd*, *env_variables=None*)

Open the serial console for a node.

Parameters

- **node_uuid** the uuid of the node
- **port** the terminal port for the node
- **console_cmd** the shell command that will be executed by socat to establish console to the node
- **env_variables** optional dict of environment variables to pass to the subprocess (e.g. IPMI_PASSWORD for ipmitool -E).

Raises

- *ConsoleError* if the directory for the PID file or the PID file cannot be created
- *ConsoleSubprocessFailed* when invoking the subprocess failed

ironic.drivers.modules.console_utils.**stop_socat_console**(*node_uuid*)

Close the serial console for a node.

Parameters
node_uuid the UUID of the node

Raises
ConsoleError if unable to stop the console process

ironic.drivers.modules.deploy_utils module

class ironic.drivers.modules.deploy_utils.**InstanceImageCache**

Bases: *ImageCache*

ironic.drivers.modules.deploy_utils.**agent_add_clean_params**(*task*)

Add required config parameters to nodes driver_internal_info.

Adds the required conf options to nodes driver_internal_info. It is Required to pass the information to IPA.

Parameters
task a TaskManager instance.

ironic.drivers.modules.deploy_utils.**build_agent_options**(*node*)

Build the options to be passed to the agent ramdisk.

Parameters
node an ironic node object

Returns
a dictionary containing the parameters to be passed to agent ramdisk.

ironic.drivers.modules.deploy_utils.**build_instance_info_for_deploy**(*task*)

Build instance_info necessary for deploying to a node.

Parameters
task a TaskManager object containing the node

Returns
a dictionary containing the properties to be updated in instance_info

Raises
exception.ImageRefValidationFailed if image_source is not Glance href and is not HTTP(S) URL.

ironic.drivers.modules.deploy_utils.**cache_instance_image**(*ctx*, *node*, *force_raw=None*, *expected_format=None*, *expected_checksum=None*, *expected_checksum_algo=None*)

Fetch the instances image from Glance

This method pulls the disk image and writes them to the appropriate place on local disk.

Parameters

- **ctx** context
- **node** an ironic node object

- **force_raw** whether convert image to raw format
- **expected_format** The expected format of the disk image contents.
- **expected_checksum** The expected image checksum, to be used if we need to convert the image to raw prior to deploying.
- **expected_checksum_algo** The checksum algo in use, if separately set.

Returns

a tuple containing the uuid of the image and the path in the filesystem where image is cached.

Raises

InvalidImage if the requested image is invalid and cannot be used for deployed based upon contents of the image or the metadata surrounding the image not matching the configured image.

ironic.drivers.modules.deploy_utils.**check_for_missing_params**(*info_dict*, *error_msg*, *param_prefix=''*)

Check for empty params in the provided dictionary.

Parameters

- **info_dict** The dictionary to inspect.
- **error_msg** The error message to prefix before printing the information about missing parameters.
- **param_prefix** Add this prefix to each parameter for error messages

Raises

MissingParameterValue, if one or more parameters are empty in the provided dictionary.

ironic.drivers.modules.deploy_utils.**check_image_size**(*node*, *image_info*)

Check if the requested image is larger than the ram size.

Parameters

- **node** a Node object containing the node to act on.
- **image_info** A dict containing image information.

Raises

InvalidParameterValue if size of the image is greater than the available ram size.

ironic.drivers.modules.deploy_utils.**check_interface_capability**(*interface*, *capability*)

Evaluate interface to determine if capability is present.

Parameters

- **interface** The interface object to check.
- **capability** The value representing the capability that the caller wishes to check if present.

Returns

True if capability found, otherwise False.

ironic.drivers.modules.deploy_utils.**compute_image_checksum**(*image_path*, *algorithm='md5'*)

Compute checksum by given image path and algorithm.

ironic.drivers.modules.deploy_utils.**destroy_http_instance_images**(*node*)

Delete instance image file and symbolic link refers to it.

ironic.drivers.modules.deploy_utils.**destroy_images**(*node_uuid*)

Delete instances image file.

Parameters

node_uuid the uuid of the ironic node.

ironic.drivers.modules.deploy_utils.**direct_deploy_should_convert_raw_image**(*node*)

Whether converts image to raw format for specified node.

Parameters

node ironic node object

Returns

Boolean, whether the direct deploy interface should convert image to raw.

ironic.drivers.modules.deploy_utils.**fetch_images**(*ctx*, *cache*, *images_info*, *force_raw=True*, *expected_format=None*, *expected_checksum=None*, *expected_checksum_algo=None*, *image_auth_data=None*)

Check for available disk space and fetch images using ImageCache.

Parameters

- **ctx** context
- **cache** ImageCache instance to use for fetching
- **images_info** list of tuples (image href, destination path)
- **force_raw** boolean value, whether to convert the image to raw format
- **expected_format** The expected format of the image.
- **expected_checksum** The expected image checksum, to be used if we need to convert the image to raw prior to deploying.
- **expected_checksum_algo** The checksum algo in use, if separately set.

Raises

InstanceDeployFailure if unable to find enough disk space

Raises

InvalidImage if the supplied image metadata or contents are deemed to be invalid, unsafe, or not matching the expectations asserted by configuration supplied or set.

ironic.drivers.modules.deploy_utils.**get_boot_option**(*node*)

Gets the boot option.

Parameters

node A single Node.

Raises
InvalidParameterValue if the capabilities string is not a dict or is malformed.

Returns
A string representing the boot option type. Defaults to configuration setting [deploy]default_boot_mode.

ironic.drivers.modules.deploy_utils.**get_disk_label**(*node*)

Return the disk label requested for deploy, if any.

Parameters
node a single Node.

Raises
InvalidParameterValue if the capabilities string is not a dictionary or is malformed.

Returns
the disk label or None if no disk label was specified.

ironic.drivers.modules.deploy_utils.**get_image_download_source**(*node*)

Get the effective value of image_download_source for the node.

ironic.drivers.modules.deploy_utils.**get_image_instance_info**(*node*)

Gets the image information from the node.

Get image information for the given node instance from its instance_info property.

Parameters
node a single Node.

Returns
A dict with required image properties retrieved from nodes instance_info.

Raises
MissingParameterValue, if image_source is missing in nodes instance_info. Also raises same exception if kernel/ramdisk is missing in instance_info for non-glance images.

ironic.drivers.modules.deploy_utils.**get_image_properties**(*ctx*, *image_href*)

Get properties of the image.

Parameters

- **ctx** security context
- **image_href** reference to the image

Returns
properties as a dictionary

Raises
InvalidParameterValue if the image cannot be accessed

ironic.drivers.modules.deploy_utils.**get_ipxe_boot_file**(*node*)

Return the iPXE boot file name requested for deploy.

This method returns iPXE boot file name to be used for deploy. Architecture specific boot file is searched first. BIOS/UEFI boot file is used if no valid architecture specific file found.

If no valid value is found, the default reverts to the `get_pxe_boot_file` method and thus the `[pxe]pxe_bootfile_name` and `[pxe]uefi_ipxe_bootfile_name` settings.

Parameters
node A single Node.

Returns
The iPXE boot file name.

`ironic.drivers.modules.deploy_utils.`**`get_ipxe_config_template`**`(`*`node`*`)`

Return the iPXE config template file name requested of deploy.

This method returns the iPXE configuration template file.

Parameters
node A single Node.

Returns
The iPXE config template file name.

`ironic.drivers.modules.deploy_utils.`**`get_ironic_api_url`**`()`

Resolve Ironic API endpoint

either from config of from Keystone catalog.

`ironic.drivers.modules.deploy_utils.`**`get_pxe_boot_file`**`(`*`node`*`)`

Return the PXE boot file name requested for deploy.

This method returns PXE boot file name to be used for deploy. Architecture specific boot file is searched first. BIOS/UEFI boot file is used if no valid architecture specific file found.

Parameters
node A single Node.

Returns
The PXE boot file name.

`ironic.drivers.modules.deploy_utils.`**`get_pxe_config_template`**`(`*`node`*`)`

Return the PXE config template file name requested for deploy.

This method returns PXE config template file to be used for deploy. First specific pxe template is searched in the node. After that architecture specific template file is searched. BIOS/UEFI template file is used if no valid architecture specific file found.

Parameters
node A single Node.

Returns
The PXE config template file name.

`ironic.drivers.modules.deploy_utils.`**`get_remote_boot_volume`**`(`*`task`*`)`

Identify a boot volume from any configured volumes.

Returns
None or the volume target representing the volume.

`ironic.drivers.modules.deploy_utils.`**`get_root_device_for_deploy`**`(`*`node`*`)`

Get a root device requested for deployment or None.

Raises
InvalidParameterValue on invalid hints.

Returns
Parsed root device hints or None if no hints were provided.

ironic.drivers.modules.deploy_utils.**get_single_nic_with_vif_port_id**(*task*)

Returns the MAC address of a port which has a VIF port id.

Parameters
task a TaskManager instance containing the ports to act on.

Returns
MAC address of the port connected to deployment network. None if it cannot find any port with vif id.

ironic.drivers.modules.deploy_utils.**is_anaconda_deploy**(*node*)

Determine if Anaconda deploy interface is in use for the deployment.

Parameters
node A single Node.

Returns
A boolean value of True when Anaconda deploy interface is in use otherwise False

ironic.drivers.modules.deploy_utils.**is_iscsi_boot**(*task*)

Return true if booting from an iscsi volume.

ironic.drivers.modules.deploy_utils.**is_ramdisk_deploy**(*node*)

ironic.drivers.modules.deploy_utils.**is_ramdisk_disabled**(*node*) → bool

Check if the ramdisk (IPA) is disabled for the node.

ironic.drivers.modules.deploy_utils.**is_software_raid**(*node*)

Determine if software raid is in use for the deployment.

Parameters
node A single Node.

Returns
A boolean value of True when software raid is in use, otherwise False

ironic.drivers.modules.deploy_utils.**needs_agent_ramdisk**(*node*, *mode='deploy'*)

Checks whether the node requires an agent ramdisk now.

ironic.drivers.modules.deploy_utils.**parse_instance_info**(*node*, *image_deploy=True*)

Gets the instance specific Node deployment info.

This method validates whether the instance_info property of the supplied node contains the required information for this driver to deploy images to the node.

Parameters

- **node** a single Node.
- **image_deploy** If the deployment interface is aware this is an image based deployment, default True.

Returns
A dict with the instance_info values.

Raises
MissingParameterValue, if any of the required parameters are missing.

Raises
InvalidParameterValue, if any of the parameters have invalid value.

ironic.drivers.modules.deploy_utils.**populate_storage_driver_internal_info**(*task*)

Set node driver_internal_info for boot from volume parameters.

Parameters
task a TaskManager object containing the node.

Raises
StorageError when a node has an iSCSI or FibreChannel boot volume defined but is not capable to support it.

ironic.drivers.modules.deploy_utils.**prepare_agent_boot**(*task*)

Prepare booting the agent on the node.

Parameters
task a TaskManager instance.

ironic.drivers.modules.deploy_utils.**prepare_inband_cleaning**(*task*, *manage_boot=True*)

Prepares the node to boot into agent for in-band cleaning.

This method does the following: 1. Prepares the cleaning ports for the bare metal node and updates the clean parameters in nodes driver_internal_info. 2. If manage_boot parameter is set to true, it also calls the prepare_ramdisk method of boot interface to boot the agent ramdisk. 3. Reboots the bare metal node.

Parameters

- **task** a TaskManager object containing the node
- **manage_boot** If this is set to True, this method calls the prepare_ramdisk method of boot interface to boot the agent ramdisk. If False, it skips preparing the boot agent ramdisk using boot interface, and assumes that the environment is setup to automatically boot agent ramdisk every time bare metal node is rebooted.

Returns
states.CLEANWAIT to signify an asynchronous prepare.

Raises
NetworkError, NodeCleaningFailure if the previous cleaning ports cannot be removed or if new cleaning ports cannot be created.

Raises
InvalidParameterValue if cleaning network UUID config option has an invalid value.

ironic.drivers.modules.deploy_utils.**prepare_inband_service**(*task*)

Boot a service ramdisk on the node.

Parameters
task a TaskManager instance.

Raises
NetworkError if the tenant ports cannot be removed.

Raises
InvalidParameterValue when the wrong power state is specified or the wrong driver info is specified for power management.

Raises
other exceptions by the nodes power driver if something wrong occurred during the power action.

Raises
any boot interfaces prepare_ramdisk exceptions.

Returns
Returns states.SERVICEWAIT

ironic.drivers.modules.deploy_utils.**reboot_to_finish_step**(*task*)

Reboot the node into IPA to finish a deploy/clean step.

Parameters
task a TaskManager instance.

Returns
states.CLEANWAIT if cleaning operation in progress or states.DEPLOYWAIT if deploy operation in progress.

ironic.drivers.modules.deploy_utils.**remove_http_instance_symlink**(*node_uuid*)

ironic.drivers.modules.deploy_utils.**rescue_or_deploy_mode**(*node*)

ironic.drivers.modules.deploy_utils.**set_failed_state**(*task*, *msg*, *collect_logs=True*)

Sets the deploy status as failed with relevant messages.

This method sets the deployment as fail with the given message. It sets nodes provision_state to DEPLOYFAIL and updates last_error with the given error message. It also powers off the baremetal node.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **msg** the message to set in logs and last_error of the node.
- **collect_logs** Boolean indicating whether to attempt to collect logs from IPA-based ramdisk. Defaults to True. Actual log collection is also affected by CONF.agent.deploy_logs_collect config option.

ironic.drivers.modules.deploy_utils.**step_error_handler**(*task*, *logmsg*, *errmsg=None*)

Run the correct handler for the current step.

Parameters

- **task** a TaskManager instance.
- **logmsg** Message to be logged.
- **errmsg** Message for the user. Optional, if not provided *logmsg* is used.

ironic.drivers.modules.deploy_utils.**switch_pxe_config**(*path*, *root_uuid_or_disk_id*, *boot_mode*, *is_whole_disk_image*, *iscsi_boot=False*, *ramdisk_boot=False*, *ipxe_enabled=False*, *anaconda_boot=False*)

Switch a pxe config from deployment mode to service mode.

Parameters

- **path** path to the pxe config file in tftpboot.
- **root_uuid_or_disk_id** root uuid in case of partition image or disk_id in case of whole disk image.
- **boot_mode** if boot mode is uefi or bios.
- **is_whole_disk_image** if the image is a whole disk image or not.
- **iscsi_boot** if boot is from an iSCSI volume or not.
- **ramdisk_boot** if the boot is to be to a ramdisk configuration.
- **ipxe_enabled** A default False boolean value to tell the method if the caller is using iPXE.
- **anaconda_boot** if the boot is to be to an anaconda configuration.

ironic.drivers.modules.deploy_utils.**tear_down_inband_cleaning**(*task*, *manage_boot=True*)

Tears down the environment setup for in-band cleaning.

This method does the following: 1. Powers off the bare metal node (unless the node is fast tracked or there was a cleaning failure). 2. If manage_boot parameter is set to true, it also calls the clean_up_ramdisk method of boot interface to clean up the environment that was set for booting agent ramdisk. 3. Deletes the cleaning ports which were setup as part of cleaning.

Parameters

- **task** a TaskManager object containing the node
- **manage_boot** If this is set to True, this method calls the clean_up_ramdisk method of boot interface to boot the agent ramdisk. If False, it skips this step.

Raises

NetworkError, NodeCleaningFailure if the cleaning ports cannot be removed.

ironic.drivers.modules.deploy_utils.**tear_down_inband_service**(*task*)

Tears down the environment setup for in-band service.

This method does the following: 1. Powers off the bare metal node (unless the node is fast tracked or there was a service failure). 2. If manage_boot parameter is set to true, it also calls the clean_up_ramdisk method of boot interface to clean up the environment that was set for booting agent ramdisk. 3. Deletes the cleaning ports which were setup as part of cleaning.

Parameters

task a TaskManager object containing the node

Raises
NetworkError, NodeServiceFailure if the cleaning ports cannot be removed.

ironic.drivers.modules.deploy_utils.**tear_down_storage_configuration**(*task*)

Clean up storage configuration.

Remove entries from driver_internal_info for storage and deletes the volume targets from the database. This is done to ensure a clean state for the next boot of the machine.

ironic.drivers.modules.deploy_utils.**try_set_boot_device**(*task*, *device*, *persistent=True*)

Tries to set the boot device on the node.

This method tries to set the boot device on the node to the given boot device. Under uefi boot mode, setting of boot device may differ between different machines. IPMI does not work for setting boot devices in uefi mode for certain machines. This method ignores the expected IPMI failure for uefi boot mode and just logs a message. In error cases, it is expected the operator has to manually set the node to boot from the correct device.

Parameters

- **task** a TaskManager object containing the node
- **device** the boot device
- **persistent** Whether to set the boot device persistently

Raises
Any exception from set_boot_device except IPMIFailure (setting of boot device using ipmi is expected to fail).

ironic.drivers.modules.deploy_utils.**validate_capabilities**(*node*)

Validates that specified supported capabilities have valid value

This method checks if the any of the supported capability is present in Node capabilities. For all supported capabilities specified for a Node, it validates that it has a valid value. The node can have capability as part of the properties or instance_info or both. Note that the actual value of a capability does not need to be the same in the nodes properties and instance_info.

Parameters
node an ironic node object.

Raises
InvalidParameterValue, if the capability is not set to a valid value.

ironic.drivers.modules.deploy_utils.**validate_image_properties**(*task*, *deploy_info*)

Validate the image.

For Glance images it checks that the image exists in Glance and its properties or deployment info contain the properties passed. If its not a Glance image, it checks that deployment info contains needed properties.

Parameters

- **task** TaskManager instance with a valid node
- **deploy_info** the deploy_info to be validated

Raises
InvalidParameterValue if: * connection to glance failed; * authorization for accessing image failed; * HEAD request to image URL failed or returned response code != 200; * HEAD request response does not contain Content-Length header; * the protocol specified in image URL is not supported.

Raises
MissingParameterValue if the image doesnt contain the mentioned properties.

ironic.drivers.modules.fake module

Fake driver interfaces used in testing.

This is also an example of some kinds of things which can be done within drivers. For instance, the MultipleVendorInterface class demonstrates how to load more than one interface and wrap them in some logic to route incoming vendor_passthru requests appropriately. This can be useful eg. when mixing functionality between a power interface and a deploy interface, when both rely on separate vendor_passthru methods.

class ironic.drivers.modules.fake.**FakeBIOS**(**args*, ***kwargs*)

Bases: *BIOSInterface*

Fake implementation of simple BIOSInterface.

apply_configuration(*task*, *settings*)

Validate & apply BIOS settings on the given node.

This method takes the BIOS settings from the settings param and applies BIOS settings on the given node. It may also validate the given bios settings before applying any settings and manage failures when setting an invalid BIOS config. In the case of needing password to update the BIOS config, it will be taken from the driver_info properties. After the BIOS configuration is done, cache_bios_settings will be called to update the nodes BIOS setting table with the BIOS configuration applied on the node.

Parameters

- **task** a TaskManager instance.
- **settings** Dictionary containing the BIOS configuration.

Raises
UnsupportedDriverExtension, if the nodes driver doesnt support BIOS configuration.

Raises
InvalidParameterValue, if validation of settings fails.

Raises
MissingParameterValue, if some required parameters are missing.

Returns
states.CLEANWAIT if BIOS configuration is in progress asynchronously or None if it is complete.

cache_bios_settings(*task*)

Store or update BIOS properties on the given node.

This method stores BIOS properties to the bios_settings table during cleaning operation and updates bios_settings table when apply_configuration() and factory_reset() are called to set new BIOS configurations. It will also update the timestamp of each bios setting.

Parameters
task a TaskManager instance.

Raises
UnsupportedDriverExtension, if the nodes driver doesnt support getting BIOS properties from bare metal.

Returns
None.

`factory_reset`(*task*)

Reset BIOS configuration to factory default on the given node.

This method resets BIOS configuration to factory default on the given node. After the BIOS reset action is done, cache_bios_settings will be called to update the nodes BIOS settings table with default bios settings.

Parameters
task a TaskManager instance.

Raises
UnsupportedDriverExtension, if the nodes driver doesnt support BIOS reset.

Returns
states.CLEANWAIT if BIOS configuration is in progress asynchronously or None if it is complete.

`get_properties`()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

`validate`(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

`class ironic.drivers.modules.fake.FakeBoot`(**args*, ***kwargs*)

Bases: *BootInterface*

Example implementation of a simple boot interface.

`capabilities = ['ipxe_boot', 'pxe_boot']`

`clean_up_instance`(*task*)

Cleans up the boot of instance.

This method cleans up the environment that was setup for booting the instance.

Parameters
`task` A task from TaskManager.

Returns
None

`clean_up_ramdisk`(*task*, *mode='deploy'*)

Cleans up the boot of ironic ramdisk.

This method cleans up the environment that was setup for booting the deploy or rescue ramdisk.

Parameters
`task` A task from TaskManager.

Returns
None

`get_properties`()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

`prepare_instance`(*task*)

Prepares the boot of instance.

This method prepares the boot of the instance after reading relevant information from the nodes database.

Parameters
`task` A task from TaskManager.

Returns
None

`prepare_ramdisk`(*task*, *ramdisk_params*, *mode='deploy'*)

Prepares the boot of Ironic ramdisk.

This method prepares the boot of the deploy or rescue ramdisk after reading relevant information from the nodes database.

Parameters

- `task` A task from TaskManager.
- `ramdisk_params` The options to be passed to the ironic ramdisk. Different implementations might want to boot the ramdisk in different ways by passing parameters to them. For example,

  When Agent ramdisk is booted to deploy a node, it takes the parameters ipa-api-url, etc.

Other implementations can make use of ramdisk_params to pass such information. Different implementations of boot interface will have different ways of passing parameters to the ramdisk.

Returns
None

`validate`(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

`class ironic.drivers.modules.fake.FakeConsole`(**args*, ***kwargs*)

Bases: *ConsoleInterface*

Example implementation of a simple console interface.

`get_console`(*task*)

Get connection information about the console.

This method should return the necessary information for the client to access the console.

Parameters
task A TaskManager instance containing the node to act on.

Returns
the console connection information.

`get_properties`()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

`start_console`(*task*)

Start a remote console for the tasks node.

This method should not raise an exception if console already started.

Parameters
task A TaskManager instance containing the node to act on.

`stop_console`(*task*)

Stop the remote console session for the tasks node.

Parameters
task A TaskManager instance containing the node to act on.

validate(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

class ironic.drivers.modules.fake.**FakeDeploy**(**args*, ***kwargs*)

Bases: *DeployInterface*

Class for a fake deployment driver.

Example implementation of a deploy interface that uses a separate power interface.

clean_up(*task*)

Clean up the deployment environment for the tasks node.

If preparation of the deployment environment ahead of time is possible, this method should be implemented by the driver. It should erase anything cached by the *prepare* method.

If implemented, this method must be idempotent. It may be called multiple times for the same node on the same conductor, and it may be called by multiple conductors in parallel. Therefore, it must not require an exclusive lock.

This method is called before *tear_down*.

Parameters
task A TaskManager instance containing the node to act on.

deploy(*task*)

Perform a deployment to the tasks node.

Perform the necessary work to deploy an image onto the specified node. This method will be called after prepare(), which may have already performed any preparatory steps, such as pre-caching some data for the node.

Parameters
task A TaskManager instance containing the node to act on.

Returns
status of the deploy. One of ironic.common.states.

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

prepare(*task*)

Prepare the deployment environment for the tasks node.

If preparation of the deployment environment ahead of time is possible, this method should be implemented by the driver.

If implemented, this method must be idempotent. It may be called multiple times for the same node on the same conductor.

This method is called before *deploy*.

Parameters
task A TaskManager instance containing the node to act on.

take_over(*task*)

Take over management of this tasks node from a dead conductor.

If conductors hosts maintain a static relationship to nodes, this method should be implemented by the driver to allow conductors to perform the necessary work during the remapping of nodes to conductors when a conductor joins or leaves the cluster.

For example, the PXE driver has an external dependency:
Neutron must forward DHCP BOOT requests to a conductor which has prepared the tftpboot environment for the given node. When a conductor goes offline, another conductor must change this setting in Neutron as part of remapping that nodes control to itself. This is performed within the *takeover* method.

Parameters
task A TaskManager instance containing the node to act on.

tear_down(*task*)

Tear down a previous deployment on the tasks node.

Given a node that has been previously deployed to, do all cleanup and tear down necessary to un-deploy that node.

Parameters
task A TaskManager instance containing the node to act on.

Returns
status of the deploy. One of ironic.common.states.

validate(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

class ironic.drivers.modules.fake.**FakeFirmware**(**args*, ***kwargs*)

Bases: *FirmwareInterface*

Example implementation of a simple firmware interface.

cache_firmware_components(*task*)

Store or update Firmware Components on the given node.

This method stores Firmware Components to the firmware_information table during cleaning operation. It will also update the timestamp of each Firmware Component.

Parameters
task a TaskManager instance.

Raises
UnsupportedDriverExtension, if the nodes driver doesnt support getting Firmware Components from bare metal.

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

update(*task*, *settings*)

Update the Firmware on the given using the settings for components.

Parameters

- **task** a TaskManager instance.
- **settings** a list of dictionaries, each dictionary contains the component name and the url that will be used to update the firmware.

Raises
UnsupportedDriverExtension, if the nodes driver doesnt support update via the interface.

Raises
InvalidParameterValue, if validation of the settings fails.

Raises
MissingParamterValue, if some required parameters are missing.

Returns
states.CLEANWAIT if Firmware update with the settings is in progress asynchronously of None if it is complete.

validate(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

class ironic.drivers.modules.fake.**FakeGraphicalConsole**(**args*, ***kwargs*)

Bases: *GraphicalConsole*

Console which displays the console container fake app.

get_app_name()

Get the name of the app passed to the console container.

A single console container image is expected to support all known graphical consoles and each implementation is referred to as an app. Each graphical console driver will specify what app to load in the container.

Returns
String representing the app name to load in the console container

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

validate(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

class ironic.drivers.modules.fake.**FakeInspect**(**args*, ***kwargs*)

Bases: *InspectInterface*

Example implementation of a simple inspect interface.

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

inspect_hardware(*task*)

Inspect hardware.

Inspect hardware to obtain the essential & additional hardware properties.

Parameters
task A task from TaskManager.

Raises
HardwareInspectionFailure, if unable to get essential hardware properties.

Returns
Resulting state of the inspection i.e. states.MANAGEABLE or None.

validate(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

class ironic.drivers.modules.fake.**FakeManagement**(**args*, ***kwargs*)

Bases: *ManagementInterface*

Example implementation of a simple management interface.

get_boot_device(*task*)

Get the current boot device for a node.

Provides the current boot device of the node. Be aware that not all drivers support this.

Parameters
task A task from TaskManager.

Raises
MissingParameterValue if a required parameter is missing

Returns

A dictionary containing:

boot_device
Ahe boot device, one of *ironic.common.boot_devices* or None if it is unknown.

persistent
Whether the boot device will persist to all future boots or not, None if it is unknown.

get_indicator_state(*task*, *component*, *indicator*)

Get current state of the indicator of the hardware component.

Parameters

- **task** A task from TaskManager.

- **component** The hardware component, one of *ironic.common.components*.
- **indicator** Indicator ID (as reported by *get_supported_indicators*).

Raises
InvalidParameterValue if an invalid component or indicator is specified.

Raises
MissingParameterValue if a required parameter is missing

Returns
Current state of the indicator, one of *ironic.common.indicator_states*.

`get_properties()`

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

`get_sensors_data(task)`

Get sensors data method.

Parameters
task A TaskManager instance.

Raises
FailedToGetSensorData when getting the sensor data fails.

Raises
FailedToParseSensorData when parsing sensor data fails.

Returns

Returns a consistent format dict of sensor data grouped by sensor type, which can be processed by Ceilometer. eg,

```
{
  'Sensor Type 1': {
    'Sensor ID 1': {
      'Sensor Reading': 'current value',
      'key1': 'value1',
      'key2': 'value2'
    },
    'Sensor ID 2': {
      'Sensor Reading': 'current value',
      'key1': 'value1',
      'key2': 'value2'
    }
  },
  'Sensor Type 2': {
    'Sensor ID 3': {
      'Sensor Reading': 'current value',
      'key1': 'value1',
      'key2': 'value2'
    },
```

(continues on next page)

(continued from previous page)

```
    'Sensor ID 4': {
      'Sensor Reading': 'current value',
      'key1': 'value1',
      'key2': 'value2'
    }
  }
}
```

get_supported_boot_devices(*task*)

Get a list of the supported boot devices.

Parameters

task A task from TaskManager.

Returns

A list with the supported boot devices defined in *ironic.common.boot_devices*.

get_supported_indicators(*task*, *component=None*)

Get a map of the supported indicators (e.g. LEDs).

Parameters

- **task** A task from TaskManager.
- **component** If not *None*, return indicator information for just this component, otherwise return indicators for all existing components.

Returns

A dictionary of hardware components (*ironic.common.components*) as keys with values being dictionaries having indicator IDs as keys and indicator properties as values.

```
{
    'chassis': {
        'enclosure-0': {
            "readonly": true,
            "states": [
                "off",
                "on"
            ]
        }
    },
    'system':
        'blade-A': {
            "readonly": true,
            "states": [
                "pff",
                "on"
            ]
        },
```

(continues on next page)

(continued from previous page)

```
    'drive':
        'ssd0': {
            "readonly": true,
            "states": [
                "off",
                "on"
            ]
        }
    }
}
```

set_boot_device(*task*, *device*, *persistent=False*)

Set the boot device for a node.

Set the boot device to use on next reboot of the node.

Parameters

- **task** A task from TaskManager.
- **device** The boot device, one of *ironic.common.boot_devices*.
- **persistent** Boolean value. True if the boot device will persist to all future boots, False if not. Default: False.

Raises

InvalidParameterValue if an invalid boot device is specified.

Raises

MissingParameterValue if a required parameter is missing

validate(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters

task A TaskManager instance containing the node to act on.

Raises

InvalidParameterValue on malformed parameter(s)

Raises

MissingParameterValue on missing parameter(s)

class ironic.drivers.modules.fake.**FakePower**(**args*, ***kwargs*)

Bases: *PowerInterface*

Example implementation of a simple power interface.

get_power_state(*task*)

Return the power state of the tasks node.

Parameters
task A TaskManager instance containing the node to act on.

Raises
MissingParameterValue if a required parameter is missing.

Returns
A power state. One of *ironic.common.states*.

`get_properties()`

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

`get_supported_power_states(task)`

Get a list of the supported power states.

Parameters
task A TaskManager instance containing the node to act on.

Returns
A list with the supported power states defined in *ironic.common.states*.

`reboot(task, timeout=None)`

Perform a hard reboot of the tasks node.

Drivers are expected to properly handle case when node is powered off by powering it on.

Parameters

- **task** A TaskManager instance containing the node to act on.
- **timeout** timeout (in seconds) positive integer (> 0) for any power state. `None` indicates to use default timeout.

Raises
MissingParameterValue if a required parameter is missing.

`set_power_state(task, power_state, timeout=None)`

Set the power state of the tasks node.

Parameters

- **task** A TaskManager instance containing the node to act on.
- **power_state** Any power state from *ironic.common.states*.
- **timeout** timeout (in seconds) positive integer (> 0) for any power state. `None` indicates to use default timeout.

Raises
MissingParameterValue if a required parameter is missing.

`validate(task)`

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

class ironic.drivers.modules.fake.**FakeRAID**(**args*, ***kwargs*)

Bases: *RAIDInterface*

Example implementation of simple RAIDInterface.

create_configuration(*task*, *create_root_volume=True*, *create_nonroot_volumes=True*)

Creates RAID configuration on the given node.

This method creates a RAID configuration on the given node. It assumes that the target RAID configuration is already available in node.target_raid_config. Implementations of this interface are supposed to read the RAID configuration from node.target_raid_config. After the RAID configuration is done (either in this method OR in a call-back method), ironic.common.raid.update_raid_info() may be called to sync the nodes RAID-related information with the RAID configuration applied on the node.

Parameters

- **task** A TaskManager instance.
- **create_root_volume** Setting this to False indicates not to create root volume that is specified in the nodes target_raid_config. Default value is True.
- **create_nonroot_volumes** Setting this to False indicates not to create non-root volumes (all except the root volume) in the nodes target_raid_config. Default value is True.
- **delete_existing** Setting this to True indicates to delete RAID configuration prior to creating the new configuration.

Returns
states.CLEANWAIT (cleaning) or states.DEPLOYWAIT (deployment) if RAID configuration is in progress asynchronously, or None if it is complete.

delete_configuration(*task*)

Deletes RAID configuration on the given node.

This method deletes the RAID configuration on the give node. After RAID configuration is deleted, node.raid_config should be cleared by the implementation.

Parameters
task A TaskManager instance.

Returns
states.CLEANWAIT (cleaning) or states.DEPLOYWAIT (deployment) if deletion is in progress asynchronously, or None if it is complete.

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

class ironic.drivers.modules.fake.**FakeRescue**(**args*, ***kwargs*)

Bases: *RescueInterface*

Example implementation of a simple rescue interface.

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

rescue(*task*)

Boot the tasks node into a rescue environment.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InstanceRescueFailure if node validation or rescue operation fails.

Returns
states.RESCUEWAIT if rescue is in progress asynchronously or states.RESCUE if it is complete.

unrescue(*task*)

Tear down the rescue environment, and return to normal.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InstanceUnrescueFailure if node validation or unrescue operation fails.

Returns
states.ACTIVE if it is successful.

validate(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

class ironic.drivers.modules.fake.**FakeStorage**(**args*, ***kwargs*)

Bases: *StorageInterface*

Example implementation of simple storage Interface.

attach_volumes(*task*)

Informs the storage subsystem to attach all volumes for the node.

Parameters
task A TaskManager instance.

Raises
UnsupportedDriverExtension

detach_volumes(*task*)

Informs the storage subsystem to detach all volumes for the node.

Parameters
task A TaskManager instance.

Raises
UnsupportedDriverExtension

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

should_write_image(*task*)

Determines if deploy should perform the image write-out.

Parameters
task A TaskManager instance.

Returns
Boolean value to indicate if the interface expects the image to be written by Ironic.

Raises
UnsupportedDriverExtension

validate(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

class ironic.drivers.modules.fake.**FakeVendorA**(**args*, ***kwargs*)

Bases: *VendorInterface*

Example implementation of a vendor passthru interface.

first_method(*task*, *http_method*, *bar*)

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

validate(*task*, *method*, ***kwargs*)

Validate vendor-specific actions.

If invalid, raises an exception; otherwise returns None.

Parameters

- **task** A task from TaskManager.
- **method** Method to be validated
- **kwargs** Info for action.

Raises
UnsupportedDriverExtension if method can not be mapped to the supported interfaces.

Raises
InvalidParameterValue if kwargs does not contain method.

Raises
MissingParameterValue

class ironic.drivers.modules.fake.**FakeVendorB**(**args*, ***kwargs*)

Bases: *VendorInterface*

Example implementation of a secondary vendor passthru.

fourth_method_shared_lock(*task*, *http_method*, *bar*)

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

log_passthrough(*task*, ***kwargs*)

second_method(*task*, *http_method*, *bar*)

third_method_sync(*task*, *http_method*, *bar*)

trigger_servicewait(*task*, ***kwargs*)

validate(*task*, *method*, ***kwargs*)

Validate vendor-specific actions.

If invalid, raises an exception; otherwise returns None.

Parameters

- **task** A task from TaskManager.
- **method** Method to be validated
- **kwargs** Info for action.

Raises
UnsupportedDriverExtension if method can not be mapped to the supported interfaces.

Raises
InvalidParameterValue if kwargs does not contain method.

Raises
MissingParameterValue

ironic.drivers.modules.fake.**parse_sleep_range**(*sleep_range*)

ironic.drivers.modules.fake.**sleep**(*sleep_range*)

ironic.drivers.modules.graphical_console module

class ironic.drivers.modules.graphical_console.**GraphicalConsole**(**args*, ***kwargs*)

Bases: *ConsoleInterface*

get_app_info(*task*)

Information required by the app to connect to the console

Returns
dict containing app-specific values

abstract get_app_name()

Get the name of the app passed to the console container.

A single console container image is expected to support all known graphical consoles and each implementation is referred to as an app. Each graphical console driver will specify what app to load in the container.

Returns
String representing the app name to load in the console container

get_console(*task*)

Get the type and connection information about the console.

start_console(*task*)

Start a remote console for the tasks node.

This method should not raise an exception if console already started.

Parameters
task A TaskManager instance containing the node to act on.

stop_console(*task*)

Stop the remote console session for the tasks node.

Parameters
task A TaskManager instance containing the node to act on.

ironic.drivers.modules.image_cache module

Utility for caching master images.

class ironic.drivers.modules.image_cache.**ImageCache**(*master_dir*, *cache_size*, *cache_ttl*, *disable_validation=False*, *force_raw=True*)

Bases: object

Class handling access to cache for master images.

clean_up(*amount=None*)

Clean up directory with images, keeping cache of the latest images.

Files with link count >1 are never deleted. Protected by global lock, so that no one messes with master images after we get listing and before we actually delete files.

Parameters

amount if present, amount of space to reclaim in bytes, cleaning will stop, if this goal was reached, even if it is possible to clean up more files

fetch_image(*href*, *dest_path*, *ctx=None*, *force_raw=None*, *expected_format=None*, *expected_checksum=None*, *expected_checksum_algo=None*, *image_auth_data=None*)

Fetch image by given href to the destination path.

Does nothing if destination path exists and is up to date with cache and href contents. Only creates a hard link (dest_path) to cached image if requested image is already in cache and up to date with href contents. Otherwise downloads an image, stores it in cache and creates a hard link (dest_path) to it.

Parameters

- **href** image UUID or href to fetch
- **dest_path** destination file path
- **ctx** context
- **force_raw** boolean value, whether to convert the image to raw format
- **expected_format** The expected image format.
- **expected_checksum** The expected image checksum
- **expected_checksum_algo** The expected image checksum algorithm, if needed/supplied.

ironic.drivers.modules.image_cache.**clean_up_all**()

Clean up all entries from all caches.

ironic.drivers.modules.image_cache.**clean_up_caches**(*ctx*, *directory*, *images_info*, *image_auth_data=None*)

Explicitly cleanup caches based on their priority (if required).

This cleans up the caches to free up the amount of space required for the images in images_info. The caches are cleaned up one after the other in the order of their priority. If we still cannot free up enough space after trying all the caches, this method throws exception.

Parameters

- **ctx** context
- **directory** the directory (of the cache) to be freed up.
- **images_info** a list of tuples of the form (image_uuid,path) for which space is to be created in cache.

Raises
InsufficientDiskSpace exception, if we cannot free up enough space after trying all the caches.

ironic.drivers.modules.image_cache.**cleanup**(*priority*)
Decorator method for adding cleanup priority to a class.

ironic.drivers.modules.image_utils module

class ironic.drivers.modules.image_utils.**ISOImageCache**
Bases: *ImageCache*

class ironic.drivers.modules.image_utils.**ImageHandler**(*driver*)
Bases: object

publish_image(*image_file*, *object_name*, *node_http_url=None*)
Make image file downloadable.

Depending on ironic settings, pushes given file into Swift or copies it over to local HTTP servers document root and returns publicly accessible URL leading to the given file.

Parameters
- **image_file** path to file to publish
- **object_name** name of the published file
- **node_http_url** a url to be used to publish the image. If set, the values from external_http_url and http_url from CONF.deploy wont be used.

Returns
a URL to download published file

unpublish_image(*object_name*)
Withdraw the image previously made downloadable.

Depending on ironic settings, removes previously published file from where it has been published - Swift or local HTTP servers document root.

Parameters
object_name name of the published file (optional)

classmethod unpublish_image_for_node(*node*, *prefix=''*, *suffix=''*)
Withdraw the image previously made downloadable.

Depending on ironic settings, removes previously published file from where it has been published - Swift or local HTTP servers document root.

Parameters
- **node** the node for which image was published.
- **prefix** object name prefix.

- **suffix** object name suffix.

update_driver_config(*driver*)

ironic.drivers.modules.image_utils.**cleanup_disk_image**(*task*, *prefix=None*)

Deletes the image if it was created for the node.

Parameters

- **task** an ironic node object.
- **prefix** Prefix to use for the object name.

ironic.drivers.modules.image_utils.**cleanup_floppy_image**(*task*)

Deletes the floppy image if it was created for the node.

Parameters

task an ironic node object.

ironic.drivers.modules.image_utils.**cleanup_iso_image**(*task*)

Deletes the ISO if it was created for the instance.

Parameters

task A task from TaskManager.

ironic.drivers.modules.image_utils.**cleanup_remote_image**(*task*, *file_name*)

Cleanup image created via prepare_remote_image.

ironic.drivers.modules.image_utils.**override_api_url**(*params*)

ironic.drivers.modules.image_utils.**prepare_boot_iso**(*task*, *d_info*, *root_uuid=None*)

Prepare boot ISO image

Build bootable ISO out of *[instance_info]/kernel*, *[instance_info]/ramdisk* and *[driver_info]/bootloader* if present. Otherwise, read *kernel_id* and *ramdisk_id* from *[instance_info]/image_source* Glance image metadata.

Push produced ISO image up to Glance and return temporary Swift URL to the image.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **d_info** Deployment information of the node
- **root_uuid** Root UUID

Returns

bootable ISO HTTP URL.

Raises

MissingParameterValue, if any of the required parameters are missing.

Raises

InvalidParameterValue, if any of the parameters have invalid value.

Raises

ImageCreationFailed, if creating ISO image failed.

ironic.drivers.modules.image_utils.**prepare_configdrive_image**(*task*, *content*)

Prepare an image with configdrive.

Decodes base64 contents and writes it into a disk image that can be attached e.g. to a virtual USB device. Images stored in Swift are downloaded first.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **content** Config drive as a base64-encoded string.

Raises
ImageCreationFailed, if it failed while creating the image.

Raises
SwiftOperationError, if any operation with Swift fails.

Returns
image URL for the image.

ironic.drivers.modules.image_utils.**prepare_deploy_iso**(*task*, *params*, *mode*, *d_info*)

Prepare deploy or rescue ISO image

Build bootable ISO out of *[driver_info]/deploy_kernel/[driver_info]/deploy_ramdisk* or *[driver_info]/rescue_kernel/[driver_info]/rescue_ramdisk* and *[driver_info]/bootloader*, then push built image up to Glance and return temporary Swift URL to the image.

If network interface supplies network configuration (*network_data*), a *network_data.json* will be written into an appropriate location on the final ISO.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **params** a dictionary containing parameter name->value mapping to be passed to kernel command line.
- **mode** either deploy or rescue.
- **d_info** Deployment information of the node

Returns
bootable ISO HTTP URL.

Raises
MissingParameterValue, if any of the required parameters are missing.

Raises
InvalidParameterValue, if any of the parameters have invalid value.

Raises
ImageCreationFailed, if creating ISO image failed.

ironic.drivers.modules.image_utils.**prepare_disk_image**(*task*, *content*, *prefix=None*)

Prepare an image with the given content.

If content is already an HTTP URL, return it unchanged.

Parameters

- **task** a TaskManager instance containing the node to act on.

- **content** Content as a string with a file name or bytes with contents.
- **prefix** Prefix to use for the object name.

Raises
ImageCreationFailed, if it failed while creating the image.

Raises
SwiftOperationError, if any operation with Swift fails.

Returns
image URL for the image.

ironic.drivers.modules.image_utils.**prepare_floppy_image**(*task*, *params=None*)

Prepares the floppy image for passing the parameters.

This method prepares a temporary VFAT filesystem image and adds a file into the image which contains parameters to be passed to the ramdisk. Then this method uploads built image to Swift [redfish]swift_container, setting it to auto expire after [redfish]swift_object_expiry_timeout seconds. Finally, a temporary Swift URL is returned addressing Swift object just created.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **params** a dictionary containing parameter name->value mapping to be passed to deploy or rescue image via floppy image.

Raises
ImageCreationFailed, if it failed while creating the floppy image.

Raises
SwiftOperationError, if any operation with Swift fails.

Returns
image URL for the floppy image.

ironic.drivers.modules.image_utils.**prepare_remote_image**(*task*, *image_url*, *file_name='boot.iso'*, *download_source='local'*, *cache=None*)

Generic function for publishing remote images.

Given the image provided by the user, generate a URL to pass to the BMC or a remote agent.

Parameters

- **task** TaskManager instance.
- **image_url** The original URL or a glance UUID.
- **file_name** File name to use when publishing.
- **download_source** How the image will be published: http (via a plain HTTP link, preverving remote links), local (via the local HTTP server even if the remote link is HTTP), swift (same as http, but Glance images are published via Swift temporary URLs).
- **cache** Image cache to use. Defaults to the ISO image cache.

Returns
The new URL (possibly the same as the old one).

ironic.drivers.modules.inspect_utils module

exception ironic.drivers.modules.inspect_utils.**AutoEnrollPossible**(*message=None*, ***kwargs*)

Bases: *IronicException*

Exception to indicate that the node can be enrolled.

The error message and code is the same as for NotFound to make sure we dont disclose any information when discovery is disabled.

code = 404

ironic.drivers.modules.inspect_utils.**cache_lookup_addresses**(*node*)

Cache lookup addresses for a quick access.

ironic.drivers.modules.inspect_utils.**clean_up_swift_entries**(*task*)

Delete swift entries containing inspection data.

Delete swift entries related to the node in task.node containing inspection data. The entries are `inspector_data-<task.node.uuid>-inventory` for hardware inventory and similar for `-plugin` containing the rest of the inspection data.

Parameters
task A TaskManager instance.

ironic.drivers.modules.inspect_utils.**clear_lookup_addresses**(*node*)

Remove lookup addresses cached on the node.

ironic.drivers.modules.inspect_utils.**create_ports_if_not_exist**(*task*, *macs=None*)

Create ironic ports from MAC addresses data dict.

Creates ironic ports from MAC addresses data returned with inspection or as requested by operator. Helper argument to detect the MAC address `get_mac_address` defaults to value part of MAC address dict key-value pair.

Parameters

- **task** A TaskManager instance.
- **macs** A sequence of MAC addresses. If `None`, fetched from the tasks management interface.

ironic.drivers.modules.inspect_utils.**get_inspection_callback**(*ironic_url*)

ironic.drivers.modules.inspect_utils.**get_inspection_data**(*node*, *context*)

Get inspection data.

Retrieve the inspection data for a node either from database or the Object Storage API (swift/radosgw) as configured.

Parameters

- **node** the Ironic node that the required data is about
- **context** an admin context

Returns
dictionary with `inventory` and `plugin_data` fields

Raises
NodeInventoryNotFound if no inventory has been saved

`ironic.drivers.modules.inspect_utils.`**lookup_node**(*context*, *mac_addresses*, *bmc_addresses*, *node_uuid=None*)

Do a node lookup by the information from the inventory.

Parameters

- **context** Request context
- **mac_addresses** List of MAC addresses.
- **bmc_addresses** List of BMC (realistically, IPMI) addresses.
- **node_uuid** Node UUID (if known).

Raises
NotFound with a generic message for all failures to avoid disclosing any information.

`ironic.drivers.modules.inspect_utils.`**missing_entrypoints_callback**(*names*)

Raise RuntimeError with comma-separated list of missing hooks

`ironic.drivers.modules.inspect_utils.`**run_inspection_hooks**(*task*, *inventory*, *plugin_data*, *hooks*, *on_error_plugin_data*)

Process data from the ramdisk using inspection hooks.

`ironic.drivers.modules.inspect_utils.`**store_inspection_data**(*node*, *inventory*, *plugin_data*, *context*)

Store inspection data.

Store the inspection data for a node. The storage is either the database or the Object Storage API (swift/radosgw) as configured.

Parameters

- **node** the Ironic node that the inspection data is about
- **inventory** the inventory to store
- **plugin_data** the plugin data (if any) to store
- **context** an admin context

`ironic.drivers.modules.inspect_utils.`**validate_inspection_hooks**(*intf: str*, *enabled_hooks: List[str]*)

Validate the enabled inspection hooks.

Parameters

- **intf** Inspection interface
- **enabled_hooks** Hooks to be enabled for the inspection interface

Raises
RuntimeError on missing or failed to load hooks

Returns
the list of hooks that passed validation

ironic.drivers.modules.ipmitool module

IPMI power manager driver.

Uses the ipmitool command (http://ipmitool.sourceforge.net/) to remotely manage hardware. This includes setting the boot device, getting a serial-over-LAN console, and controlling the power state of the machine.

NOTE THAT CERTAIN DISTROS MAY INSTALL openipmi BY DEFAULT, INSTEAD OF ipmitool, WHICH PROVIDES DIFFERENT COMMAND-LINE OPTIONS AND *IS NOT SUPPORTED* BY THIS DRIVER.

class ironic.drivers.modules.ipmitool.**IPMIConsole**(**args*, ***kwargs*)

Bases: *ConsoleInterface*

A base ConsoleInterface that uses ipmitool.

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

validate(*task*)

Validate the Node console info.

Parameters
task a task from TaskManager.

Raises
InvalidParameterValue

Raises
MissingParameterValue when a required parameter is missing

class ironic.drivers.modules.ipmitool.**IPMIManagement**(**args*, ***kwargs*)

Bases: *ManagementInterface*

detect_vendor(*task*)

Detects and returns the hardware vendor.

Parameters
task A task from TaskManager.

Raises
InvalidParameterValue if an invalid component, indicator or state is specified.

Raises
MissingParameterValue if a required parameter is missing

Returns
String representing the BMC reported Vendor or Manufacturer, otherwise returns None.

`get_boot_device`(*task*)

Get the current boot device for the tasks node.

Returns the current boot device of the node.

Parameters
`task` a task from TaskManager.

Raises
InvalidParameterValue if required IPMI parameters are missing.

Raises
IPMIFailure on an error from ipmitool.

Raises
MissingParameterValue if a required parameter is missing.

Returns

a dictionary containing:

boot_device
the boot device, one of *ironic.common.boot_devices* or None if it is unknown.

persistent
Whether the boot device will persist to all future boots or not, None if it is unknown.

`get_properties`()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

`get_sensors_data`(*task*)

Get sensors data.

Parameters
`task` a TaskManager instance.

Raises
FailedToGetSensorData when getting the sensor data fails.

Raises
FailedToParseSensorData when parsing sensor data fails.

Raises
InvalidParameterValue if required ipmi parameters are missing

Raises
MissingParameterValue if a required parameter is missing.

Returns
returns a dict of sensor data group by sensor type.

`get_supported_boot_devices`(*task*)

Get a list of the supported boot devices.

Parameters
`task` a task from TaskManager.

Returns
A list with the supported boot devices defined in *ironic.common.boot_devices*.

inject_nmi(*task*)

Inject NMI, Non Maskable Interrupt.

Inject NMI (Non Maskable Interrupt) for a node immediately.

Parameters
task A TaskManager instance containing the node to act on.

Raises
IPMIFailure on an error from ipmitool.

Returns
None

set_boot_device(*task*, *device*, *persistent=False*)

Set the boot device for the tasks node.

Set the boot device to use on next reboot of the node.

Parameters

- **task** a task from TaskManager.
- **device** the boot device, one of *ironic.common.boot_devices*.
- **persistent** Boolean value. True if the boot device will persist to all future boots, False if not. Default: False.

Raises
InvalidParameterValue if an invalid boot device is specified

Raises
MissingParameterValue if required ipmi parameters are missing.

Raises
IPMIFailure on an error from ipmitool.

validate(*task*)

Check that driver_info contains IPMI credentials.

Validates whether the driver_info property of the supplied tasks node contains the required credentials information.

Parameters
task a task from TaskManager.

Raises
InvalidParameterValue if required IPMI parameters are missing.

Raises
MissingParameterValue if a required parameter is missing.

class ironic.drivers.modules.ipmitool.**IPMIPower**(**args*, ***kwargs*)

Bases: *PowerInterface*

`get_power_state`(*task*)

Get the current power state of the tasks node.

Parameters
`task` a TaskManager instance containing the node to act on.

Returns
one of ironic.common.states POWER_OFF, POWER_ON or ERROR.

Raises
InvalidParameterValue if required ipmi parameters are missing.

Raises
MissingParameterValue if a required parameter is missing.

Raises
IPMIFailure on an error from ipmitool (from _power_status call).

`get_properties`()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

`get_supported_power_states`(*task*)

Get a list of the supported power states.

Parameters
`task` A TaskManager instance containing the node to act on. currently not used.

Returns
A list with the supported power states defined in *ironic.common.states*.

`reboot`(*task*, *timeout=None*)

Cycles the power to the tasks node.

Parameters

- `task` a TaskManager instance containing the node to act on.
- `timeout` timeout (in seconds) positive integer (> 0) for any power state. The timeout is counted once during power off and once during power on for reboots. `None` indicates that the default timeout will be used.

Raises
MissingParameterValue if required ipmi parameters are missing.

Raises
InvalidParameterValue if an invalid power state was specified.

Raises
PowerStateFailure if the final state of the node is not POWER_ON or the intermediate state of the node is not POWER_OFF.

`set_power_state`(*task*, *power_state*, *timeout=None*)

Turn the power on, off, soft reboot, or soft power off.

Parameters

- **task** a TaskManager instance containing the node to act on.
- **power_state** desired power state. one of ironic.common.states, POWER_ON, POWER_OFF, SOFT_POWER_OFF, or SOFT_REBOOT.
- **timeout** timeout (in seconds) positive integer (> 0) for any power state. The timeout is counted once during power off and once during power on for reboots. `None` indicates that the default timeout will be used.

Raises
InvalidParameterValue if an invalid power state was specified.

Raises
MissingParameterValue if required ipmi parameters are missing

Raises
PowerStateFailure if the power couldnt be set to pstate.

validate(*task*)

Validate driver_info for ipmitool driver.

Check that node[driver_info] contains IPMI credentials.

Parameters
task a TaskManager instance containing the node to act on.

Raises
InvalidParameterValue if required ipmi parameters are missing.

Raises
MissingParameterValue if a required parameter is missing.

class ironic.drivers.modules.ipmitool.**IPMISocatConsole**(**args*, ***kwargs*)

Bases: *IPMIConsole*

A ConsoleInterface that uses ipmitool and socat.

get_console(*task*)

Get the type and connection information about the console.

Parameters
task a task from TaskManager

start_console(*task*)

Start a remote console for the node.

Parameters
task a task from TaskManager

Raises
InvalidParameterValue if required ipmi parameters are missing

Raises
PasswordFileFailedToCreate if unable to create a file containing the password

Raises
ConsoleError if the directory for the PID file cannot be created

Raises
ConsoleSubprocessFailed when invoking the subprocess failed

stop_console(*task*)

Stop the remote console session for the node.

Parameters
task a task from TaskManager

Raises
ConsoleError if unable to stop the console

class ironic.drivers.modules.ipmitool.**VendorPassthru**(**args*, ***kwargs*)

Bases: *VendorInterface*

bmc_reset(*task*, *http_method*, *warm=True*)

Reset BMC with IPMI command bmc reset (warm|cold).

Parameters

- **task** a TaskManager instance.
- **http_method** the HTTP method used on the request.
- **warm** boolean parameter to decide on warm or cold reset.

Raises
IPMIFailure on an error from ipmitool.

Raises
MissingParameterValue if a required parameter is missing.

Raises
InvalidParameterValue when an invalid value is specified

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

send_raw(*task*, ***kwargs*)

Send raw bytes to the BMC. Bytes should be a string of bytes.

Parameters

- **task** a TaskManager instance.
- **raw_bytes** a string of raw bytes to send, e.g. 0x00 0x01 supplied as a kwargument.

Raises
IPMIFailure on an error from ipmitool.

Raises
MissingParameterValue if a required parameter is missing.

Raises
InvalidParameterValue when an invalid value is specified.

validate(*task*, *method*, ***kwargs*)

Validate vendor-specific actions.

If invalid, raises an exception; otherwise returns None.

Valid methods:

- send_raw
- bmc_reset

Parameters

- **task** a task from TaskManager.
- **method** method to be validated
- **kwargs** info for action.

Raises
InvalidParameterValue when an invalid parameter value is specified.

Raises
MissingParameterValue if a required parameter is missing.

ironic.drivers.modules.ipmitool.**check_cipher_suite_errors**(*cmd_stderr*)

Checks if the command stderr contains cipher suite errors.

Parameters
cmd_stderr The command stderr.

Returns
True if the cmd_stderr contains a cipher suite error, False otherwise.

ironic.drivers.modules.ipmitool.**choose_cipher_suite**(*actual_cipher_suite*)

Gives the possible next available cipher suite version.

Based on CONF.ipmi.cipher_suite_versions and the last cipher suite version used that failed. This function is only called if the node doesnt have cipher_suite set. Starts using the last element of the list and decreasing the index.

Parameters
actual_cipher_suite latest cipher suite used in the ipmi call.

Returns
the next possible cipher suite or None in case of empty configuration.

ironic.drivers.modules.ipmitool.**dump_sdr**(*task*, *file_path*)

Dump SDR data to a file.

Parameters

- **task** a TaskManager instance.
- **file_path** the path to SDR dump file.

Raises
IPMIFailure on an error from ipmitool.

Raises
MissingParameterValue if a required parameter is missing.

Raises
InvalidParameterValue when an invalid value is specified.

ironic.drivers.modules.ipmitool.**is_bridging_enabled**(*node*)

Check if IPMI bridging is enabled.

This call is used in the inspector lookup.

ironic.drivers.modules.ipmitool.**send_raw**(*task*, *raw_bytes*)

Send raw bytes to the BMC. Bytes should be a string of bytes.

Parameters

- **task** a TaskManager instance.
- **raw_bytes** a string of raw bytes to send, e.g. 0x00 0x01

Returns
a tuple with stdout and stderr.

Raises
IPMIFailure on an error from ipmitool.

Raises
MissingParameterValue if a required parameter is missing.

Raises
InvalidParameterValue when an invalid value is specified.

ironic.drivers.modules.ipmitool.**update_cipher_suite_cmd**(*actual_cs*, *args*)

Updates variables and the cipher suite cmd.

This function updates the values for all parameters so they can be used in the next retry of _exec_ipmitool.

Parameters

- **actual_cs** a string that represents the cipher suite that was used in the command.
- **args** a list that contains the ipmitool command that was executed, it will be modified in-place.

Returns
the next actual_cs

ironic.drivers.modules.ipxe module

iPXE Boot Interface

class ironic.drivers.modules.ipxe.**iPXEBoot**(**args*, ***kwargs*)

Bases: *PXEBaseMixin*, *BootInterface*

capabilities = ['iscsi_volume_boot', 'ramdisk_boot', 'ipxe_boot', 'pxe_boot']

ipxe_enabled = True

class ironic.drivers.modules.ipxe.**iPXEHttpBoot**(**args*, ***kwargs*)

Bases: *PXEBaseMixin*, *BootInterface*

```
capabilities = ['iscsi_volume_boot', 'ramdisk_boot', 'ipxe_boot',
'pxe_boot']

http_boot_enabled = True

ipxe_enabled = True
```

ironic.drivers.modules.noop module

Dummy interface implementations for use as defaults with optional interfaces.

Note that unlike fake implementations, these do not pass validation and raise exceptions for user-accessible actions.

class ironic.drivers.modules.noop.**FailMixin**

Bases: object

Mixin to add to an interface to make it fail validation.

get_properties()

validate(*task*, **args*, ***kwargs*)

class ironic.drivers.modules.noop.**NoBIOS**(**args*, ***kwargs*)

Bases: *FailMixin*, *BIOSInterface*

BIOS interface implementation that raises errors on all requests.

apply_configuration(*task*, *settings*)

Validate & apply BIOS settings on the given node.

This method takes the BIOS settings from the settings param and applies BIOS settings on the given node. It may also validate the given bios settings before applying any settings and manage failures when setting an invalid BIOS config. In the case of needing password to update the BIOS config, it will be taken from the driver_info properties. After the BIOS configuration is done, cache_bios_settings will be called to update the nodes BIOS setting table with the BIOS configuration applied on the node.

Parameters

- **task** a TaskManager instance.
- **settings** Dictionary containing the BIOS configuration.

Raises

UnsupportedDriverExtension, if the nodes driver doesnt support BIOS configuration.

Raises

InvalidParameterValue, if validation of settings fails.

Raises

MissingParameterValue, if some required parameters are missing.

Returns

states.CLEANWAIT if BIOS configuration is in progress asynchronously or None if it is complete.

cache_bios_settings(*task*)

Store or update BIOS properties on the given node.

This method stores BIOS properties to the bios_settings table during cleaning operation and updates bios_settings table when apply_configuration() and factory_reset() are called to set new BIOS configurations. It will also update the timestamp of each bios setting.

Parameters
task a TaskManager instance.

Raises
UnsupportedDriverExtension, if the nodes driver doesnt support getting BIOS properties from bare metal.

Returns
None.

factory_reset(*task*)

Reset BIOS configuration to factory default on the given node.

This method resets BIOS configuration to factory default on the given node. After the BIOS reset action is done, cache_bios_settings will be called to update the nodes BIOS settings table with default bios settings.

Parameters
task a TaskManager instance.

Raises
UnsupportedDriverExtension, if the nodes driver doesnt support BIOS reset.

Returns
states.CLEANWAIT if BIOS configuration is in progress asynchronously or None if it is complete.

class ironic.drivers.modules.noop.**NoConsole**(**args*, ***kwargs*)

Bases: *FailMixin*, *ConsoleInterface*

Console interface implementation that raises errors on all requests.

get_console(*task*, **args*, ***kwargs*)

Get connection information about the console.

This method should return the necessary information for the client to access the console.

Parameters
task A TaskManager instance containing the node to act on.

Returns
the console connection information.

start_console(*task*, **args*, ***kwargs*)

Start a remote console for the tasks node.

This method should not raise an exception if console already started.

Parameters
task A TaskManager instance containing the node to act on.

stop_console(*task*, **args*, ***kwargs*)

Stop the remote console session for the tasks node.

Parameters
task A TaskManager instance containing the node to act on.

class ironic.drivers.modules.noop.**NoDeploy**(**args*, ***kwargs*)

Bases: *DeployInterface*

Deploy interface that does nothing and succeeds.

clean_up(*task*)

Clean up the deployment environment for the tasks node.

If preparation of the deployment environment ahead of time is possible, this method should be implemented by the driver. It should erase anything cached by the *prepare* method.

If implemented, this method must be idempotent. It may be called multiple times for the same node on the same conductor, and it may be called by multiple conductors in parallel. Therefore, it must not require an exclusive lock.

This method is called before *tear_down*.

Parameters
task A TaskManager instance containing the node to act on.

deploy(*task*)

Perform a deployment to the tasks node.

Perform the necessary work to deploy an image onto the specified node. This method will be called after prepare(), which may have already performed any preparatory steps, such as pre-caching some data for the node.

Parameters
task A TaskManager instance containing the node to act on.

Returns
status of the deploy. One of ironic.common.states.

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

prepare(*task*)

Prepare the deployment environment for the tasks node.

If preparation of the deployment environment ahead of time is possible, this method should be implemented by the driver.

If implemented, this method must be idempotent. It may be called multiple times for the same node on the same conductor.

This method is called before *deploy*.

Parameters
task A TaskManager instance containing the node to act on.

take_over(*task*)

Take over management of this tasks node from a dead conductor.

If conductors hosts maintain a static relationship to nodes, this method should be implemented by the driver to allow conductors to perform the necessary work during the remapping of nodes to conductors when a conductor joins or leaves the cluster.

For example, the PXE driver has an external dependency:
Neutron must forward DHCP BOOT requests to a conductor which has prepared the tftpboot environment for the given node. When a conductor goes offline, another conductor must change this setting in Neutron as part of remapping that nodes control to itself. This is performed within the *takeover* method.

Parameters
task A TaskManager instance containing the node to act on.

tear_down(*task*)

Tear down a previous deployment on the tasks node.

Given a node that has been previously deployed to, do all cleanup and tear down necessary to un-deploy that node.

Parameters
task A TaskManager instance containing the node to act on.

Returns
status of the deploy. One of ironic.common.states.

validate(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

class ironic.drivers.modules.noop.**NoFirmware**(**args*, ***kwargs*)

Bases: *FailMixin*, *FirmwareInterface*

Firmware interface implementation that raises errors on all requests

cache_firmware_components(*task*)

Store or update Firmware Components on the given node.

This method stores Firmware Components to the firmware_information table during cleaning operation. It will also update the timestamp of each Firmware Component.

Parameters
task a TaskManager instance.

Raises
UnsupportedDriverExtension, if the nodes driver doesnt support getting Firmware Components from bare metal.

update(*task*, *settings*)

Update the Firmware on the given using the settings for components.

Parameters

- **task** a TaskManager instance.
- **settings** a list of dictionaries, each dictionary contains the component name and the url that will be used to update the firmware.

Raises
UnsupportedDriverExtension, if the nodes driver doesnt support update via the interface.

Raises
InvalidParameterValue, if validation of the settings fails.

Raises
MissingParamterValue, if some required parameters are missing.

Returns
states.CLEANWAIT if Firmware update with the settings is in progress asynchronously of None if it is complete.

class ironic.drivers.modules.noop.**NoInspect**(**args*, ***kwargs*)

Bases: *FailMixin*, *InspectInterface*

Inspect interface implementation that raises errors on all requests.

inspect_hardware(*task*, **args*, ***kwargs*)

Inspect hardware.

Inspect hardware to obtain the essential & additional hardware properties.

Parameters
task A task from TaskManager.

Raises
HardwareInspectionFailure, if unable to get essential hardware properties.

Returns
Resulting state of the inspection i.e. states.MANAGEABLE or None.

class ironic.drivers.modules.noop.**NoRAID**(**args*, ***kwargs*)

Bases: *FailMixin*, *RAIDInterface*

RAID interface implementation that raises errors on all requests.

create_configuration(*task*, **args*, ***kwargs*)

Creates RAID configuration on the given node.

This method creates a RAID configuration on the given node. It assumes that the target RAID configuration is already available in node.target_raid_config. Implementations of

this interface are supposed to read the RAID configuration from node.target_raid_config. After the RAID configuration is done (either in this method OR in a call-back method), ironic.common.raid.update_raid_info() may be called to sync the nodes RAID-related information with the RAID configuration applied on the node.

Parameters

- **task** A TaskManager instance.
- **create_root_volume** Setting this to False indicates not to create root volume that is specified in the nodes target_raid_config. Default value is True.
- **create_nonroot_volumes** Setting this to False indicates not to create non-root volumes (all except the root volume) in the nodes target_raid_config. Default value is True.
- **delete_existing** Setting this to True indicates to delete RAID configuration prior to creating the new configuration.

Returns

states.CLEANWAIT (cleaning) or states.DEPLOYWAIT (deployment) if RAID configuration is in progress asynchronously, or None if it is complete.

delete_configuration(*task*, **args*, ***kwargs*)

Deletes RAID configuration on the given node.

This method deletes the RAID configuration on the give node. After RAID configuration is deleted, node.raid_config should be cleared by the implementation.

Parameters

task A TaskManager instance.

Returns

states.CLEANWAIT (cleaning) or states.DEPLOYWAIT (deployment) if deletion is in progress asynchronously, or None if it is complete.

validate_raid_config(*task*, *raid_config*)

Validates the given RAID configuration.

This method validates the given RAID configuration. Driver implementations of this interface can override this method to support custom parameters for RAID configuration.

Parameters

- **task** A TaskManager instance.
- **raid_config** The RAID configuration to validate.

Raises

InvalidParameterValue, if the RAID configuration is invalid.

class ironic.drivers.modules.noop.**NoRescue**(**args*, ***kwargs*)

Bases: *FailMixin*, *RescueInterface*

Rescue interface implementation that raises errors on all requests.

rescue(*task*, **args*, ***kwargs*)

Boot the tasks node into a rescue environment.

Parameters

task A TaskManager instance containing the node to act on.

Raises
InstanceRescueFailure if node validation or rescue operation fails.

Returns
states.RESCUEWAIT if rescue is in progress asynchronously or states.RESCUE if it is complete.

unrescue(*task*, **args*, ***kwargs*)

Tear down the rescue environment, and return to normal.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InstanceUnrescueFailure if node validation or unrescue operation fails.

Returns
states.ACTIVE if it is successful.

class ironic.drivers.modules.noop.**NoVendor**(**args*, ***kwargs*)

Bases: *FailMixin*, *VendorInterface*

Vendor interface implementation that raises errors on all requests.

driver_validate(*method*, ***kwargs*)

Validate driver-vendor-passthru actions.

If invalid, raises an exception; otherwise returns None.

Parameters

- **method** method to be validated
- **kwargs** info for action.

Raises
MissingParameterValue if kwargs does not contain certain parameter.

Raises
InvalidParameterValue if parameter does not match.

ironic.drivers.modules.noop_mgmt module

No-op management interface implementation.

class ironic.drivers.modules.noop_mgmt.**NoopManagement**(**args*, ***kwargs*)

Bases: *ManagementInterface*

No-op management interface implementation.

Using this implementation requires the boot order to be preconfigured to first try PXE booting, then fall back to hard drives.

get_boot_device(*task*)

Get the current boot device for a node.

Provides the current boot device of the node. Be aware that not all drivers support this.

Parameters
task A task from TaskManager.

Raises
MissingParameterValue if a required parameter is missing

Returns

A dictionary containing:

boot_device
Ahe boot device, one of *ironic.common.boot_devices* or None if it is unknown.

persistent
Whether the boot device will persist to all future boots or not, None if it is unknown.

`get_properties()`

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

`get_sensors_data(task)`

Get sensors data method.

Parameters
task A TaskManager instance.

Raises
FailedToGetSensorData when getting the sensor data fails.

Raises
FailedToParseSensorData when parsing sensor data fails.

Returns

Returns a consistent format dict of sensor data grouped by sensor type, which can be processed by Ceilometer. eg,

```
{
  'Sensor Type 1': {
    'Sensor ID 1': {
      'Sensor Reading': 'current value',
      'key1': 'value1',
      'key2': 'value2'
    },
    'Sensor ID 2': {
      'Sensor Reading': 'current value',
      'key1': 'value1',
      'key2': 'value2'
    }
  },
  'Sensor Type 2': {
    'Sensor ID 3': {
      'Sensor Reading': 'current value',
      'key1': 'value1',
      'key2': 'value2'
```

(continues on next page)

(continued from previous page)

```
    },
    'Sensor ID 4': {
      'Sensor Reading': 'current value',
      'key1': 'value1',
      'key2': 'value2'
    }
  }
}
```

get_supported_boot_devices(*task*)

Get a list of the supported boot devices.

Parameters
task A task from TaskManager.

Returns
A list with the supported boot devices defined in *ironic.common.boot_devices*.

set_boot_device(*task*, *device*, *persistent=False*)

Set the boot device for a node.

Set the boot device to use on next reboot of the node.

Parameters

- **task** A task from TaskManager.
- **device** The boot device, one of *ironic.common.boot_devices*.
- **persistent** Boolean value. True if the boot device will persist to all future boots, False if not. Default: False.

Raises
InvalidParameterValue if an invalid boot device is specified.

Raises
MissingParameterValue if a required parameter is missing

validate(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

ironic.drivers.modules.pxe module

PXE Boot Interface

class ironic.drivers.modules.pxe.**HttpBoot**(**args*, ***kwargs*)

Bases: *PXEBaseMixin*, *BootInterface*

capabilities = ['ramdisk_boot', 'pxe_boot']

http_boot_enabled = True

class ironic.drivers.modules.pxe.**PXEAnacondaDeploy**(**args*, ***kwargs*)

Bases: *AgentBaseMixin*, *HeartbeatMixin*, *DeployInterface*

clean_up(*task*)

Clean up the deployment environment for the tasks node.

Unlinks TFTP and instance images and triggers image cache cleanup. Removes the TFTP configuration files for this node.

Parameters
task a TaskManager instance containing the node to act on.

deploy(*task*)

Perform a deployment to the tasks node.

Perform the necessary work to deploy an image onto the specified node. This method will be called after prepare(), which may have already performed any preparatory steps, such as pre-caching some data for the node.

Parameters
task A TaskManager instance containing the node to act on.

Returns
status of the deploy. One of ironic.common.states.

deploy_has_started(*task*)

deploy_is_done(*task*)

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

prepare(*task*)

Prepare the deployment environment for the tasks node.

If preparation of the deployment environment ahead of time is possible, this method should be implemented by the driver.

If implemented, this method must be idempotent. It may be called multiple times for the same node on the same conductor.

This method is called before *deploy*.

Parameters
task A TaskManager instance containing the node to act on.

reboot_to_instance(*task*)

Method invoked after the deployment is completed.

Parameters
task a TaskManager instance

should_manage_boot(*task*)

Whether agent boot is managed by ironic.

validate(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

class ironic.drivers.modules.pxe.**PXEBoot**(**args*, ***kwargs*)

Bases: *PXEBaseMixin*, *BootInterface*

capabilities = ['ramdisk_boot', 'pxe_boot']

ironic.drivers.modules.pxe_base module

Base PXE Interface Methods

class ironic.drivers.modules.pxe_base.**PXEBaseMixin**

Bases: object

clean_up_instance(*task*)

Cleans up the boot of instance.

This method cleans up the environment that was setup for booting the instance. It unlinks the instance kernel/ramdisk in nodes directory in tftproot and removes the PXE config.

Parameters
task a task from TaskManager.

Returns
None

clean_up_ramdisk(*task*)

Cleans up the boot of ironic ramdisk.

This method cleans up the PXE environment that was setup for booting the deploy or rescue ramdisk. It unlinks the deploy/rescue kernel/ramdisk in the nodes directory in tftproot and removes its PXE config.

Parameters

- **task** a task from TaskManager.
- **mode** Label indicating a deploy or rescue operation was carried out on the node. Supported values are deploy and rescue. Defaults to deploy, indicating deploy operation was carried out.

Returns
None

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

http_boot_enabled = False

ipxe_enabled = False

prepare_instance(*task*)

Prepares the boot of instance.

This method prepares the boot of the instance after reading relevant information from the nodes instance_info. In case of netboot, it updates the dhcp entries and switches the PXE config. In case of localboot, it cleans up the PXE config.

Parameters
task a task from TaskManager.

Returns
None

prepare_ramdisk(*task*, *ramdisk_params*)

Prepares the boot of Ironic ramdisk using PXE.

This method prepares the boot of the deploy or rescue kernel/ramdisk after reading relevant information from the nodes driver_info and instance_info.

Parameters

- **task** a task from TaskManager.
- **ramdisk_params** the parameters to be passed to the ramdisk. pxe driver passes these parameters as kernel command-line arguments.

Returns
None

Raises
MissingParameterValue, if some information is missing in nodes driver_info or instance_info.

Raises
InvalidParameterValue, if some information provided is invalid.

Raises
IronicException, if some power or set boot boot device operation failed on the node.

validate(*task*)

Validate the PXE-specific info for booting deploy/instance images.

This method validates the PXE-specific info for booting the ramdisk and instance on the node. If invalid, raises an exception; otherwise returns None.

Parameters
task a task from TaskManager.

Returns
None

Raises
InvalidParameterValue, if some parameters are invalid.

Raises
MissingParameterValue, if some required parameters are missing.

validate_inspection(*task*)

Validate that the node has required properties for inspection.

Parameters
task A TaskManager instance with the node being checked

Raises
UnsupportedDriverExtension

validate_rescue(*task*)

Validate that the node has required properties for rescue.

Parameters
task a TaskManager instance with the node being checked

Raises
MissingParameterValue if node is missing one or more required parameters

ironic.drivers.modules.ramdisk module

Ramdisk Deploy Interface

class ironic.drivers.modules.ramdisk.**RamdiskDeploy**(**args*, ***kwargs*)

Bases: *AgentBaseMixin*, *HeartbeatMixin*, *DeployInterface*

deploy(*task*)

Perform a deployment to the tasks node.

Perform the necessary work to deploy an image onto the specified node. This method will be called after prepare(), which may have already performed any preparatory steps, such as pre-caching some data for the node.

Parameters
task A TaskManager instance containing the node to act on.

Returns
status of the deploy. One of ironic.common.states.

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

`prepare`(*task*)

Prepare the deployment environment for the tasks node.

If preparation of the deployment environment ahead of time is possible, this method should be implemented by the driver.

If implemented, this method must be idempotent. It may be called multiple times for the same node on the same conductor.

This method is called before *deploy*.

Parameters
task A TaskManager instance containing the node to act on.

`supports_deploy`(*task*)

Check if this interface supports the given deployment.

Ramdisk deploy is appropriate when `ironic_ramdisk_deploy=True` is present **and** one of the following holds:

- `boot_iso` is set in instance_info, or
- `kernel` and `ramdisk` are set in instance_info, or
- `image_source` is a Glance image with `boot_iso_id` property, or
- `image_source` is a Glance image with `kernel_id` and `ramdisk_id` properties.

For instance_info cases the sentinel is looked up in `instance_info`; for Glance image cases it is looked up in the image properties.

Parameters
task A TaskManager instance containing the node to act on.

Returns
True if ramdisk deploy is appropriate.

`validate`(*task*)

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

ironic.drivers.modules.snmp module

Ironic SNMP power manager.

Provides basic power control using an SNMP-enabled smart power controller. Uses a pluggable driver model to support devices with different SNMP object models.

class ironic.drivers.modules.snmp.**SNMPClient**(*address*, *port*, *version*, *read_community=None*, *write_community=None*, *user=None*, *auth_proto=None*, *auth_key=None*, *priv_proto=None*, *priv_key=None*, *context_engine_id=None*, *context_name=None*)

Bases: object

SNMP client object.

Performs low level SNMP get and set operations. Encapsulates all interaction with PySNMP to simplify dynamic importing and unit testing.

get(*oid*)

Use PySNMP to perform an SNMP GET operation on a single object.

Parameters
oid The OID of the object to get.

Raises
SNMPFailure if an SNMP request fails.

Returns
The value of the requested object.

get_next(*oid*)

Use PySNMP to perform an SNMP GET NEXT operation on a table object.

Parameters
oid The OID of the object to get.

Raises
SNMPFailure if an SNMP request fails.

Returns
A list of values of the requested table object.

set(*oid*, *value*)

Use PySNMP to perform an SNMP SET operation on a single object.

Parameters

- **oid** The OID of the object to set.
- **value** The value of the object to set.

Raises
SNMPFailure if an SNMP request fails.

class ironic.drivers.modules.snmp.**SNMPDriverAPCMasterSwitch**(**args*, ***kwargs*)

Bases: *SNMPDriverSimple*

SNMP driver class for APC MasterSwitch PDU devices.

SNMP objects for APC SNMPDriverAPCMasterSwitch PDU: 1.3.6.1.4.1.318.1.1.4.4.2.1.3 sPDUOutletCtl Values: 1=On, 2=Off, 3=PowerCycle, [more options follow]

oid_device = (318, 1, 1, 4, 4, 2, 1, 3)

system_id = (318, 1, 1, 4)

value_power_off = 2

value_power_on = 1

class ironic.drivers.modules.snmp.**SNMPDriverAPCMasterSwitchPlus**(**args*, ***kwargs*)

Bases: *SNMPDriverSimple*

SNMP driver class for APC MasterSwitchPlus PDU devices.

SNMP objects for APC SNMPDriverAPCMasterSwitchPlus PDU: 1.3.6.1.4.1.318.1.1.6.5.1.1.5 sPDUOutletControlMSPOutletCommand Values: 1=On, 3=Off, [more options follow]

oid_device = (318, 1, 1, 6, 5, 1, 1, 5)

system_id = (318, 1, 1, 6)

value_power_off = 3

value_power_on = 1

class ironic.drivers.modules.snmp.**SNMPDriverAPCRackPDU**(**args*, ***kwargs*)

Bases: *SNMPDriverSimple*

SNMP driver class for APC RackPDU devices.

SNMP objects for APC SNMPDriverAPCRackPDU PDU: # 1.3.6.1.4.1.318.1.1.12.3.3.1.1.4 rPDUOutletControlOutletCommand Values: 1=On, 2=Off, 3=PowerCycle, [more options follow]

oid_device = (318, 1, 1, 12, 3, 3, 1, 1, 4)

system_id = (318, 1, 1, 12)

value_power_off = 2

value_power_on = 1

class ironic.drivers.modules.snmp.**SNMPDriverAten**(**args*, ***kwargs*)

Bases: *SNMPDriverSimple*

SNMP driver class for Aten PDU devices.

SNMP objects for Aten PDU: 1.3.6.1.4.1.21317.1.3.2.2.2.2 Outlet Power Values: 1=Off, 2=On, 3=Pending, 4=Reset

oid_device = (21317, 1, 3, 2, 2, 2, 2)

system_id = (21317,)

value_power_off = 1

value_power_on = 2

class ironic.drivers.modules.snmp.**SNMPDriverAuto**(**args*, ***kwargs*)

Bases: *SNMPDriverBase*

SYS_OBJ_OID = (1, 3, 6, 1, 2, 1, 1, 2)

class ironic.drivers.modules.snmp.**SNMPDriverBase**(*snmp_info*)

Bases: object

SNMP power driver base class.

The SNMPDriver class hierarchy implements manufacturer-specific MIB actions over SNMP to interface with different smart power controller products.

oid_enterprise = (1, 3, 6, 1, 4, 1)

power_off()

Set the power state to this node to OFF.

Raises
SNMPFailure if an SNMP request fails.

Returns
power state. One of *ironic.common.states*.

power_on()

Set the power state to this node to ON.

Raises
SNMPFailure if an SNMP request fails.

Returns
power state. One of *ironic.common.states*.

power_reset()

Reset the power to this node.

Raises
SNMPFailure if an SNMP request fails.

Returns
power state. One of *ironic.common.states*.

power_state()

Returns a nodes current power state.

Raises
SNMPFailure if an SNMP request fails.

Returns
power state. One of *ironic.common.states*.

retry_interval = 1

class ironic.drivers.modules.snmp.**SNMPDriverBaytechMRP27**(**args*, ***kwargs*)

Bases: *SNMPDriverSimple*

SNMP driver class for Baytech MRP27 PDU devices.

SNMP objects for Baytech MRP27 PDU: 4779, 1, 3, 5, 3, 1, 3, {unit_id} Outlet Power Values: 0=Off, 1=On, 2=Reboot

oid_device = (4779, 1, 3, 5, 3, 1, 3, 1)

unit_id = 1

value_power_off = 0

value_power_on = 1

class ironic.drivers.modules.snmp.**SNMPDriverCyberPower**(**args*, ***kwargs*)

Bases: *SNMPDriverSimple*

SNMP driver class for CyberPower PDU devices.

SNMP objects for CyberPower PDU: 1.3.6.1.4.1.3808.1.1.3.3.3.1.1.4 ePDUOutletControlOutletCommand Values: 1=On, 2=Off, 3=PowerCycle, [more options follow]

oid_device = (3808, 1, 1, 3, 3, 3, 1, 1, 4)

system_id = (3808,)

value_power_off = 2

value_power_on = 1

class ironic.drivers.modules.snmp.**SNMPDriverEatonPower**(**args*, ***kwargs*)

Bases: *SNMPDriverBase*

SNMP driver class for Eaton Power PDU.

The Eaton power PDU does not follow the model of SNMPDriverSimple as it uses multiple SNMP objects.

SNMP objects for Eaton Power PDU 1.3.6.1.4.1.534.6.6.7.6.6.1.2.<outlet ID> outletControlStatus Read 0=off, 1=on, 2=pending off, 3=pending on 1.3.6.1.4.1.534.6.6.7.6.6.1.3.<outlet ID> outletControlOffCmd Write 0 for immediate power off 1.3.6.1.4.1.534.6.6.7.6.6.1.4.<outlet ID> outletControlOnCmd Write 0 for immediate power on

oid_device = (534, 6, 6, 7, 6, 6, 1)

oid_poweroff = (4,)

oid_poweron = (3,)

oid_status = (2,)

status_off = 0

status_on = 1

status_pending_off = 2

status_pending_on = 3

system_id = (534,)

value_power_off = 0

value_power_on = 0

class ironic.drivers.modules.snmp.**SNMPDriverRaritanPDU2**(**args*, ***kwargs*)

Bases: *SNMPDriverBase*

SNMP driver class for Raritan PDU2 PDUs.

http://support.raritan.com/px2/version-2.4.1/mibs/pdu2-mib-020400-39592.txt http://cdn.raritan.com/download/PX/v1.5.20/PDU-MIB.txt

Command:

snmpset -v2c -c private -m+PDU2-MIB <pdu IP address> PDU2-MIB::switchingOperation.1.4 = cycle snmpset -v2c -c private <pdu IP address> .1.3.6.1.4.1.13742.6.4.1.2.1.2.1.4 i 2

Output:

PDU2-MIB::switchingOperation.1.4 = INTEGER: cycle(2)

aboveUpperCritical = 6

aboveUpperWarning = 5

alarmed = 11

belowLowerCritical = 2

belowLowerWarning = 3

detected = 9

fail = 14

inSync = 20

marginal = 13

no = 16

notDetected = 10

oid_device = (13742, 6, 4, 1, 2, 1)

oid_power_action = (2,)

oid_power_status = (3,)

oid_tower_infeed_idx = (1,)

ok = 12

one = 18

outOfSync = 21

standby = 17

status_closed = 1

status_normal = 4

status_off = 8

status_on = 7

status_open = 0

two = 19

unavailable = -1

value_power_off = 0

value_power_on = 1

yes = 15

class ironic.drivers.modules.snmp.**SNMPDriverServerTechSentry3**(**args*, ***kwargs*)

Bases: *SNMPDriverBase*

SNMP driver class for Server Technology Sentry 3 PDUs.

ftp://ftp.servertech.com/Pub/SNMP/sentry3/Sentry3.mib

SNMP objects for Server Technology Power PDU. 1.3.6.1.4.1.1718.3.2.3.1.5.1.1.<outlet ID> outletStatus Read 0=off, 1=on, 2=off wait, 3=on wait, [more options follow] 1.3.6.1.4.1.1718.3.2.3.1.11.1.1.<outlet ID> outletControlAction Write 0=no action, 1=on, 2=off, 3=reboot

oid_device = (1718, 3, 2, 3, 1)

oid_power_action = (11,)

oid_power_status = (5,)

oid_tower_infeed_idx = (1, 1)

status_off = 0

status_off_wait = 2

status_on = 1

status_on_wait = 3

value_power_off = 2

value_power_on = 1

class ironic.drivers.modules.snmp.**SNMPDriverServerTechSentry4**(**args*, ***kwargs*)

Bases: *SNMPDriverBase*

SNMP driver class for Server Technology Sentry 4 PDUs.

https://www.servertech.com/support/sentry-mib-oid-tree-downloads

SNMP objects for Server Technology Power PDU. 1.3.6.1.4.1.1718.4.1.8.5.1.1<outlet ID> outletStatus notSet (0) fixedOn (1) idleOff (2) idleOn (3) [more options follow] pendOn (8) pendOff (9) off (10) on (11) [more options follow] eventOff (16) eventOn (17) eventReboot (18) eventShutdown (19) 1.3.6.1.4.1.1718.4.1.8.5.1.2.<outlet ID> outletControlAction Write 0=no action, 1=on, 2=off, 3=reboot

fixedOn = 1

idleOff = 2

idleOn = 3

lockedOff = 14

lockedOn = 15

notSet = 0

ocpOff = 6

ocpOn = 7

oid_device = (1718, 4, 1, 8, 5, 1)

oid_power_action = (2,)

oid_power_status = (1,)

oid_tower_infeed_idx = (1, 1)

reboot = 12

shutdown = 13

status_off = 10

status_on = 11

status_pendOff = 9

status_pendOn = 8

value_power_off = 2

value_power_on = 1

wakeOff = 4

wakeOn = 5

class ironic.drivers.modules.snmp.**SNMPDriverSimple**(**args*, ***kwargs*)

Bases: *SNMPDriverBase*

SNMP driver base class for simple PDU devices.

Here, simple refers to devices which provide a single SNMP object for controlling the power state of an outlet.

The default OID of the power state object is of the form <enterprise OID>.<device OID>.<outlet ID>. A different OID may be specified by overriding the _snmp_oid method in a subclass.

abstract property oid_device

Device dependent portion of the power state object OID.

abstract property value_power_off

Value representing power off state.

abstract property value_power_on

Value representing power on state.

class ironic.drivers.modules.snmp.**SNMPDriverTeltronix**(**args*, ***kwargs*)

Bases: *SNMPDriverSimple*

SNMP driver class for Teltronix PDU devices.

SNMP objects for Teltronix PDU: 1.3.6.1.4.1.23620.1.2.2.1.4 Outlet Power Values: 1=Off, 2=On

oid_device = (23620, 1, 2, 2, 1, 4)

system_id = (23620,)

value_power_off = 1

value_power_on = 2

class ironic.drivers.modules.snmp.**SNMPDriverVertivGeistPDU**(**args*, ***kwargs*)

Bases: *SNMPDriverBase*

SNMP driver class for VertivGeist NU30017L/NU30019L PDU.

https://mibs.observium.org/mib/GEIST-V5-MIB/

off = 2

off2on = 4

oid_device = (21239, 5, 2, 3, 5, 1)

oid_power_action = (6,)

oid_power_status = (4,)

oid_tower_infeed_idx = (1,)

on = 1

on2off = 3

rebootOff = 5

rebootOn = 5

unavailable = 7

value_power_off = 4

value_power_on = 2

class ironic.drivers.modules.snmp.**SNMPPower**(**args*, ***kwargs*)

Bases: *PowerInterface*

SNMP Power Interface.

This PowerInterface class provides a mechanism for controlling the power state of a physical device using an SNMP-enabled smart power controller.

get_power_state(*task*)

Get the current power state.

Poll the SNMP device for the current power state of the node.

Parameters
task An instance of *ironic.manager.task_manager.TaskManager*.

Raises
MissingParameterValue if required SNMP parameters are missing.

Raises
InvalidParameterValue if SNMP parameters are invalid.

Raises
SNMPFailure if an SNMP request fails.

Returns
power state. One of *ironic.common.states*.

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

reboot(*task*, *timeout=None*)

Cycles the power to a node.

Parameters

- **task** An instance of *ironic.manager.task_manager.TaskManager*.
- **timeout** timeout (in seconds). Unsupported by this interface.

Raises
MissingParameterValue if required SNMP parameters are missing.

Raises
InvalidParameterValue if SNMP parameters are invalid.

Raises
PowerStateFailure if the final power state of the node is not POWER_ON after the timeout.

Raises
SNMPFailure if an SNMP request fails.

`set_power_state`(*task*, *pstate*, *timeout=None*)

Turn the power on or off.

Set the power state of a node.

Parameters

- **task** An instance of *ironic.manager.task_manager.TaskManager*.
- **pstate** Either POWER_ON or POWER_OFF from :class: *ironic.common.states*.
- **timeout** timeout (in seconds). Unsupported by this interface.

Raises
MissingParameterValue if required SNMP parameters are missing.

Raises
InvalidParameterValue if SNMP parameters are invalid or *pstate* is invalid.

Raises
PowerStateFailure if the final power state of the node is not as requested after the timeout.

Raises
SNMPFailure if an SNMP request fails.

`validate`(*task*)

Check that node.driver_info contains the requisite fields.

Raises
MissingParameterValue if required SNMP parameters are missing.

Raises
InvalidParameterValue if SNMP parameters are invalid.

ironic.drivers.modules.snmp.**memoize**(*f*)

ironic.drivers.modules.snmp.**retry_on_outdated_cache**(*f*)

Module contents

Submodules

ironic.drivers.base module

Abstract base classes for drivers.

ironic.drivers.base.**ALL_INTERFACES = {'bios', 'boot', 'console', 'deploy', 'firmware', 'inspect', 'management', 'network', 'power', 'raid', 'rescue', 'storage', 'vendor'}**

Constant holding all known interfaces.

class ironic.drivers.base.**BIOSInterface**(**args*, ***kwargs*)

Bases: *BaseInterface*

abstract apply_configuration(*task*, *settings*)

Validate & apply BIOS settings on the given node.

This method takes the BIOS settings from the settings param and applies BIOS settings on the given node. It may also validate the given bios settings before applying any settings and manage failures when setting an invalid BIOS config. In the case of needing password to update the BIOS config, it will be taken from the driver_info properties. After the BIOS configuration is done, cache_bios_settings will be called to update the nodes BIOS setting table with the BIOS configuration applied on the node.

Parameters

- **task** a TaskManager instance.
- **settings** Dictionary containing the BIOS configuration.

Raises

UnsupportedDriverExtension, if the nodes driver doesnt support BIOS configuration.

Raises

InvalidParameterValue, if validation of settings fails.

Raises

MissingParameterValue, if some required parameters are missing.

Returns

states.CLEANWAIT if BIOS configuration is in progress asynchronously or None if it is complete.

abstract cache_bios_settings(*task*)

Store or update BIOS properties on the given node.

This method stores BIOS properties to the bios_settings table during cleaning operation and updates bios_settings table when apply_configuration() and factory_reset() are called to set new BIOS configurations. It will also update the timestamp of each bios setting.

Parameters

task a TaskManager instance.

Raises

UnsupportedDriverExtension, if the nodes driver doesnt support getting BIOS properties from bare metal.

Returns

None.

abstract factory_reset(*task*)

Reset BIOS configuration to factory default on the given node.

This method resets BIOS configuration to factory default on the given node. After the BIOS reset action is done, cache_bios_settings will be called to update the nodes BIOS settings table with default bios settings.

Parameters

task a TaskManager instance.

Raises

UnsupportedDriverExtension, if the nodes driver doesnt support BIOS reset.

Returns
states.CLEANWAIT if BIOS configuration is in progress asynchronously or None if it is complete.

`interface_type = 'bios'`
Interface type, used for clean steps and logging.

`class ironic.drivers.base.BareDriver`
Bases: `object`

A bare driver object which will have interfaces attached later.

Any composable interfaces should be added as class attributes of this class, as well as appended to core_interfaces or standard_interfaces here.

`property all_interfaces`

`bios = None`
Standard attribute for BIOS related features.

A reference to an instance of :class:BIOSInterface.

`boot = None`
Standard attribute for boot related features.

A reference to an instance of :class:BootInterface.

`console = None`
Standard attribute for managing console access.

A reference to an instance of :class:ConsoleInterface.

`property core_interfaces`
Interfaces that are required to be implemented.

`deploy = None`
Core attribute for managing deployments.

A reference to an instance of :class:DeployInterface.

`firmware = None`
Standard attribute for inspection related features.

A reference to an instance of :class:FirmwareInterface.

`get_properties()`
Get the properties of the driver.

Returns
dictionary of <property name>:<property description> entries.

`inspect = None`
Standard attribute for inspection related features.

A reference to an instance of :class:InspectInterface.

`management = None`
Standard attribute for management related features.

A reference to an instance of :class:ManagementInterface.

network = None

Core attribute for network connectivity.

A reference to an instance of :class:NetworkInterface.

property non_vendor_interfaces

property optional_interfaces

Interfaces that can be no-op.

power = None

Core attribute for managing power state.

A reference to an instance of :class:PowerInterface.

raid = None

Standard attribute for RAID related features.

A reference to an instance of :class:RaidInterface.

rescue = None

Standard attribute for accessing rescue features.

A reference to an instance of :class:RescueInterface.

storage = None

Standard attribute for (remote) storage interface.

A reference to an instance of :class:StorageInterface.

vendor = None

Attribute for accessing any vendor-specific extensions.

A reference to an instance of :class:VendorInterface.

class ironic.drivers.base.**BaseInterface**(**args*, ***kwargs*)

Bases: object

A base interface implementing common functions for Driver Interfaces.

execute_clean_step(*task*, *step*)

Execute the clean step on task.node.

A clean step must take a single positional argument: a TaskManager object. It may take one or more keyword variable arguments (for use with manual cleaning only.)

A step can be executed synchronously or asynchronously. A step should return None if the method has completed synchronously or states.CLEANWAIT if the step will continue to execute asynchronously. If the step executes asynchronously, it should issue a call to the continue_node_clean RPC, so the conductor can begin the next clean step.

Parameters

- **task** A TaskManager object
- **step** The clean step dictionary representing the step to execute

Returns

None if this method has completed synchronously, or states.CLEANWAIT if the step will continue to execute asynchronously.

execute_deploy_step(*task*, *step*)

Execute the deploy step on task.node.

A deploy step must take a single positional argument: a TaskManager object. It may take one or more keyword variable arguments (for use in the future, when deploy steps can be specified via the API).

A step can be executed synchronously or asynchronously. A step should return None if the method has completed synchronously or states.DEPLOYWAIT if the step will continue to execute asynchronously. If the step executes asynchronously, it should issue a call to the continue_node_deploy RPC, so the conductor can begin the next deploy step.

Parameters

- **task** A TaskManager object
- **step** The deploy step dictionary representing the step to execute

Returns

None if this method has completed synchronously, or states.DEPLOYWAIT if the step will continue to execute asynchronously.

execute_service_step(*task*, *step*)

Execute the service step on task.node.

A service step must take a single positional argument: a TaskManager object. It does not take keyword variable arguments.

Parameters

- **task** A TaskManager object
- **step** The service step dictionary representing the step to execute

Returns

None if this method has completed synchronously

execute_verify_step(*task*, *step*)

Execute the verify step on task.node.

A verify step must take a single positional argument: a TaskManager object. It does not take keyword variable arguments.

Parameters

- **task** A TaskManager object
- **step** The verify step dictionary representing the step to execute

Returns

None if this method has completed synchronously

get_clean_steps(*task*)

Get a list of (enabled and disabled) clean steps for the interface.

This function will return all clean steps (both enabled and disabled) for the interface, in an unordered list.

Parameters

task A TaskManager object, useful for interfaces overriding this function

Raises

NodeCleaningFailure if there is a problem getting the steps from the driver. For example, when a node (using an agent driver) has just been enrolled and the agent isnt alive yet to be queried for the available clean steps.

Returns

A list of clean step dictionaries

`get_deploy_steps`(*task*)

Get a list of (enabled and disabled) deploy steps for the interface.

This function will return all deploy steps (both enabled and disabled) for the interface, in an unordered list.

Parameters

task A TaskManager object, useful for interfaces overriding this function

Raises

InstanceDeployFailure if there is a problem getting the steps from the driver. For example, when a node (using an agent driver) has just been enrolled and the agent isnt alive yet to be queried for the available deploy steps.

Returns

A list of deploy step dictionaries

`abstract get_properties`()

Return the properties of the interface.

Returns

dictionary of <property name>:<property description> entries.

`get_service_steps`(*task*)

Get a list of service steps for the interface.

This function will return all service steps (both enabled and disabled) for the interface, in an unordered list.

Parameters

task A TaskManager object, useful for interfaces overriding this function

Raises

NodeServiceFailure if there is a problem getting the steps from the driver. For example, when a node (using an agent driver) has just been enrolled and the agent isnt alive yet to be queried for the available clean steps.

Returns

A list of service step dictionaries

`get_verify_steps`(*task*)

Get a list of (enabled and disabled) verify steps for the interface.

This function will return all verify steps (both enabled and disabled) for the interface, in an unordered list.

Parameters

task A TaskManager object, useful for interfaces overriding this function

Raises

NodeVerifyFailure if there is a problem getting the steps from the driver.

For example, when a node (using an agent driver) has just been enrolled and the agent isnt alive yet to be queried for the available verify steps.

Returns
A list of verify step dictionaries

`interface_type = 'base'`

Interface type, used for clean steps and logging.

`supported = True`

Indicates if an interface is supported.

This will be set to False for interfaces which are untested in first- or third-party CI, or in the process of being deprecated.

`abstract validate(task)`

Validate the driver-specific Node deployment info.

This method validates whether the driver_info and/or instance_info properties of the tasks node contains the required information for this interface to function.

This method is often executed synchronously in API requests, so it should not conduct long-running checks.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue on malformed parameter(s)

Raises
MissingParameterValue on missing parameter(s)

`class ironic.drivers.base.BootInterface(*args, **kwargs)`

Bases: *BaseInterface*

Interface for boot-related actions.

`capabilities = []`

`abstract clean_up_instance(task)`

Cleans up the boot of instance.

This method cleans up the environment that was setup for booting the instance.

Parameters
task A task from TaskManager.

Returns
None

`abstract clean_up_ramdisk(task)`

Cleans up the boot of ironic ramdisk.

This method cleans up the environment that was setup for booting the deploy or rescue ramdisk.

Parameters
task A task from TaskManager.

Returns
None

`interface_type = 'boot'`

Interface type, used for clean steps and logging.

`abstract prepare_instance(task)`

Prepares the boot of instance.

This method prepares the boot of the instance after reading relevant information from the nodes database.

Parameters
task A task from TaskManager.

Returns
None

`abstract prepare_ramdisk(task, ramdisk_params)`

Prepares the boot of Ironic ramdisk.

This method prepares the boot of the deploy or rescue ramdisk after reading relevant information from the nodes database.

Parameters

- **task** A task from TaskManager.
- **ramdisk_params** The options to be passed to the ironic ramdisk. Different implementations might want to boot the ramdisk in different ways by passing parameters to them. For example,

  When Agent ramdisk is booted to deploy a node, it takes the parameters ipa-api-url, etc.

  Other implementations can make use of ramdisk_params to pass such information. Different implementations of boot interface will have different ways of passing parameters to the ramdisk.

Returns
None

`validate_inspection(task)`

Validate that the node has required properties for inspection.

Parameters
task A TaskManager instance with the node being checked

Raises
MissingParameterValue if node is missing one or more required parameters

Raises
UnsupportedDriverExtension

`validate_rescue(task)`

Validate that the node has required properties for rescue.

Parameters
task A TaskManager instance with the node being checked

Raises
MissingParameterValue if node is missing one or more required parameters

Raises
UnsupportedDriverExtension

class ironic.drivers.base.**ConsoleInterface**(**args*, ***kwargs*)

Bases: *BaseInterface*

Interface for console-related actions.

abstract get_console(*task*)

Get connection information about the console.

This method should return the necessary information for the client to access the console.

Parameters
task A TaskManager instance containing the node to act on.

Returns
the console connection information.

interface_type = 'console'

Interface type, used for clean steps and logging.

abstract start_console(*task*)

Start a remote console for the tasks node.

This method should not raise an exception if console already started.

Parameters
task A TaskManager instance containing the node to act on.

abstract stop_console(*task*)

Stop the remote console session for the tasks node.

Parameters
task A TaskManager instance containing the node to act on.

class ironic.drivers.base.**DeployInterface**(**args*, ***kwargs*)

Bases: *BaseInterface*

Interface for deploy-related actions.

abstract clean_up(*task*)

Clean up the deployment environment for the tasks node.

If preparation of the deployment environment ahead of time is possible, this method should be implemented by the driver. It should erase anything cached by the *prepare* method.

If implemented, this method must be idempotent. It may be called multiple times for the same node on the same conductor, and it may be called by multiple conductors in parallel. Therefore, it must not require an exclusive lock.

This method is called before *tear_down*.

Parameters
task A TaskManager instance containing the node to act on.

abstract deploy(*task*)

Perform a deployment to the tasks node.

Perform the necessary work to deploy an image onto the specified node. This method will be called after prepare(), which may have already performed any preparatory steps, such as pre-caching some data for the node.

Parameters
task A TaskManager instance containing the node to act on.

Returns
status of the deploy. One of ironic.common.states.

heartbeat(*task*, *callback_url*, *agent_version*, *agent_verify_ca=None*, *agent_status=None*, *agent_status_message=None*)

Record a heartbeat for the node.

Parameters

- **task** A TaskManager instance containing the node to act on.
- **callback_url** a URL to use to call to the ramdisk.
- **agent_version** The version of the agent that is heartbeating
- **agent_verify_ca** TLS certificate for the agent.
- **agent_status** Status of the heartbeating agent
- **agent_status_message** Message describing the agent status

Returns
None

interface_type = 'deploy'

Interface type, used for clean steps and logging.

abstract prepare(*task*)

Prepare the deployment environment for the tasks node.

If preparation of the deployment environment ahead of time is possible, this method should be implemented by the driver.

If implemented, this method must be idempotent. It may be called multiple times for the same node on the same conductor.

This method is called before *deploy*.

Parameters
task A TaskManager instance containing the node to act on.

prepare_cleaning(*task*)

Prepare the node for cleaning tasks.

For example, nodes that use the Ironic Python Agent will need to boot the ramdisk in order to do in-band cleaning tasks.

If the function is asynchronous, the driver will need to handle settings node.driver_internal_info[clean_steps] and node.clean_step, as they would be set in ironic.conductor.manager._do_node_clean, but cannot be set when this is asynchronous. After, the interface should make an RPC call to continue_node_cleaning to start cleaning.

NOTE(JoshNang) this should be moved to BootInterface when it gets implemented.

Parameters
task A TaskManager instance containing the node to act on.

Returns
If this function is going to be asynchronous, should return *states.CLEANWAIT*. Otherwise, should return *None*. The interface will need to call _get_cleaning_steps and then RPC to continue_node_cleaning

prepare_service(*task*)

Prepare the node for servicing tasks.

For example, nodes that use the Ironic Python Agent will need to boot the ramdisk in order to do in-band service tasks.

If the function is asynchronous, the driver will need to handle settings node.driver_internal_info[service_steps] and node.service_step, as they would be set in ironic.conductor.manager._do_node_service, but cannot be set when this is asynchronous. After, the interface should make an RPC call to continue_node_servicing to start cleaning.

Parameters
task A TaskManager instance containing the node to act on.

Returns
If this function is going to be asynchronous, should return *states.SERVICEWAIT*. Otherwise, should return *None*. The interface will need to call _get_cleaning_steps and then RPC to continue_node_service.

restore_interface(*task*)

Restore the original deploy interface for the node.

This restores the deploy interface to the original interface in case it was changed by switch_interface.

Parameters
task a TaskManager instance containing the node to act on.

supports_deploy(*task*)

Check if deploy is supported for the given node by this interface.

By default interfaces will claim to support deploy. Interfaces which can infer actual support using (for example) node instance_info should override this method.

Parameters
task A TaskManager instance containing the node to act on.

Returns
boolean, whether deploy is supported.

switch_interface(*task*)

Optionally switch the interface to use for deployment.

This method is called at the beginning of deployment before validation to accommodate interfaces which auto-detect another interface to use for deployment.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InvalidParameterValue if the interface is not enabled.

abstract take_over(*task*)

Take over management of this tasks node from a dead conductor.

If conductors hosts maintain a static relationship to nodes, this method should be implemented by the driver to allow conductors to perform the necessary work during the remapping of nodes to conductors when a conductor joins or leaves the cluster.

For example, the PXE driver has an external dependency:
Neutron must forward DHCP BOOT requests to a conductor which has prepared the tftpboot environment for the given node. When a conductor goes offline, another conductor must change this setting in Neutron as part of remapping that nodes control to itself. This is performed within the *takeover* method.

Parameters
task A TaskManager instance containing the node to act on.

abstract tear_down(*task*)

Tear down a previous deployment on the tasks node.

Given a node that has been previously deployed to, do all cleanup and tear down necessary to un-deploy that node.

Parameters
task A TaskManager instance containing the node to act on.

Returns
status of the deploy. One of ironic.common.states.

tear_down_cleaning(*task*)

Tear down after cleaning is completed.

Given that cleaning is complete, do all cleanup and tear down necessary to allow the node to be deployed to again.

NOTE(JoshNang) this should be moved to BootInterface when it gets implemented.

Parameters
task A TaskManager instance containing the node to act on.

tear_down_service(*task*)

Tear down after servicing is completed.

Given that servicing is complete, do all cleanup and tear down necessary to allow the node to be returned to an active state.

Parameters
task A TaskManager instance containing the node to act on.

class ironic.drivers.base.**FirmwareInterface**(**args*, ***kwargs*)

Bases: *BaseInterface*

Base class for firmware interface

abstract cache_firmware_components(*task*)

Store or update Firmware Components on the given node.

This method stores Firmware Components to the firmware_information table during cleaning operation. It will also update the timestamp of each Firmware Component.

Parameters
task a TaskManager instance.

Raises
UnsupportedDriverExtension, if the nodes driver doesnt support getting Firmware Components from bare metal.

interface_type = 'firmware'

Interface type, used for clean steps and logging.

abstract update(*task*, *settings*)

Update the Firmware on the given using the settings for components.

Parameters

- **task** a TaskManager instance.
- **settings** a list of dictionaries, each dictionary contains the component name and the url that will be used to update the firmware.

Raises
UnsupportedDriverExtension, if the nodes driver doesnt support update via the interface.

Raises
InvalidParameterValue, if validation of the settings fails.

Raises
MissingParamterValue, if some required parameters are missing.

Returns
states.CLEANWAIT if Firmware update with the settings is in progress asynchronously of None if it is complete.

class ironic.drivers.base.**InspectInterface**(**args*, ***kwargs*)

Bases: *BaseInterface*

Interface for inspection-related actions.

ESSENTIAL_PROPERTIES = {'cpu_arch', 'local_gb', 'memory_mb'}

The properties required by scheduler/deploy.

abort(*task*)

Abort asynchronized hardware inspection.

Abort an ongoing hardware introspection, this is only used for asynchronize based inspect interface.

NOTE: This interface is called with node exclusive lock held, the interface implementation is expected to be a quick processing.

Parameters
task a task from TaskManager.

Raises
UnsupportedDriverExtension, if the method is not implemented by specific inspect interface.

continue_inspection(*task*, *inventory*, *plugin_data=None*)
Continue in-band hardware inspection.

Should not be implemented for purely out-of-band implementations.

Parameters

- **task** a task from TaskManager.
- **inventory** hardware inventory from the node.
- **plugin_data** optional plugin-specific data.

Raises
UnsupportedDriverExtension, if the method is not implemented by specific inspect interface.

abstract inspect_hardware(*task*)
Inspect hardware.

Inspect hardware to obtain the essential & additional hardware properties.

Parameters
task A task from TaskManager.

Raises
HardwareInspectionFailure, if unable to get essential hardware properties.

Returns
Resulting state of the inspection i.e. states.MANAGEABLE or None.

interface_type = 'inspect'
Interface type, used for clean steps and logging.

class ironic.drivers.base.**ManagementInterface**(**args*, ***kwargs*)
Bases: *BaseInterface*

Interface for management related actions.

attach_virtual_media(*task*, *device_type*, *image_url*)
Attach a virtual media device to the node.

Parameters

- **task** A TaskManager instance containing the node to act on.
- **device_type** Device type, one of *ironic.common.boot_devices.VMEDIA_DEVICES*.
- **image_url** URL of the image to attach, HTTP or HTTPS.

Raises
UnsupportedDriverExtension

detach_virtual_media(*task*, *device_types=None*)
Detach some or all virtual media devices from the node.

Parameters

- **task** A TaskManager instance containing the node to act on.
- **device_types** A collection of device type, ones from *ironic.common.boot_devices.VMEDIA_DEVICES*. If not provided, all devices are detached.

Raises
UnsupportedDriverExtension

detect_vendor(*task*)

Detects, stores, and returns the hardware vendor.

If the Node object `properties` field does not already contain a `vendor` field, then this method is intended to query Detects the BMC hardware vendor and stores the returned value with-in the Node object `properties` field if detected.

Parameters
task A task from TaskManager.

Raises
InvalidParameterValue if an invalid component, indicator or state is specified.

Raises
MissingParameterValue if a required parameter is missing

Returns
String representing the BMC reported Vendor or Manufacturer, otherwise returns None.

abstract get_boot_device(*task*)

Get the current boot device for a node.

Provides the current boot device of the node. Be aware that not all drivers support this.

Parameters
task A task from TaskManager.

Raises
MissingParameterValue if a required parameter is missing

Returns

A dictionary containing:

boot_device
Ahe boot device, one of *ironic.common.boot_devices* or None if it is unknown.

persistent
Whether the boot device will persist to all future boots or not, None if it is unknown.

get_boot_mode(*task*)

Get the current boot mode for a node.

Provides the current boot mode of the node.

NOTE: Not all drivers support this method. Older hardware
may not implement that.

Parameters
task A task from TaskManager.

Raises
MissingParameterValue if a required parameter is missing

Raises
DriverOperationError or its derivative in case of driver runtime error.

Raises
UnsupportedDriverExtension if requested operation is not supported by the driver

Returns
The boot mode, one of `ironic.common.boot_mode` or None if it is unknown.

`get_indicator_state`(*task*, *component*, *indicator*)

Get current state of the indicator of the hardware component.

Parameters

- **task** A task from TaskManager.
- **component** The hardware component, one of *ironic.common.components*.
- **indicator** Indicator ID (as reported by *get_supported_indicators*).

Raises
InvalidParameterValue if an invalid component or indicator is specified.

Raises
MissingParameterValue if a required parameter is missing

Returns
Current state of the indicator, one of *ironic.common.indicator_states*.

`get_mac_addresses`(*task*)

Get MAC address information for the node.

Parameters
task A TaskManager instance containing the node to act on.

Raises
UnsupportedDriverExtension

Returns
A list of MAC addresses for the node

`get_node_health`(*task*)

Get the current health status for a node.

Retrieves hardware health status from the BMC if available. This provides insight into the overall hardware condition, including components like fans, power supplies, storage, etc.

NOTE: Not all drivers support this method. Only drivers with
BMC health monitoring capabilities implement this.

Parameters
task A task from TaskManager.

Raises
MissingParameterValue if a required parameter is missing

Raises
DriverOperationError or its derivative in case of driver runtime error.

Raises
UnsupportedDriverExtension if requested operation is not supported by the driver

Returns
One of *ironic.common.health_states* constants (OK, WARNING, CRITICAL) or None if health status is unknown or cannot be determined.

`get_secure_boot_state`(*task*)

Get the current secure boot state for the node.

NOTE: Not all drivers support this method. Older hardware may not implement that.

Parameters
task A task from TaskManager.

Raises
MissingParameterValue if a required parameter is missing

Raises
DriverOperationError or its derivative in case of driver runtime error.

Raises
UnsupportedDriverExtension if secure boot is not supported by the driver or the hardware

Returns
Boolean

`abstract get_sensors_data`(*task*)

Get sensors data method.

Parameters
task A TaskManager instance.

Raises
FailedToGetSensorData when getting the sensor data fails.

Raises
FailedToParseSensorData when parsing sensor data fails.

Returns

Returns a consistent format dict of sensor data grouped by sensor type, which can be processed by Ceilometer. eg,

```
{
  'Sensor Type 1': {
    'Sensor ID 1': {
      'Sensor Reading': 'current value',
```

(continues on next page)

(continued from previous page)

```
      'key1': 'value1',
      'key2': 'value2'
    },
    'Sensor ID 2': {
      'Sensor Reading': 'current value',
      'key1': 'value1',
      'key2': 'value2'
    }
  },
  'Sensor Type 2': {
    'Sensor ID 3': {
      'Sensor Reading': 'current value',
      'key1': 'value1',
      'key2': 'value2'
    },
    'Sensor ID 4': {
      'Sensor Reading': 'current value',
      'key1': 'value1',
      'key2': 'value2'
    }
  }
}
```

abstract get_supported_boot_devices(*task*)

Get a list of the supported boot devices.

Parameters
task A task from TaskManager.

Returns
A list with the supported boot devices defined in *ironic.common.boot_devices*.

get_supported_boot_modes(*task*)

Get a list of the supported boot modes.

NOTE: Not all drivers support this method. Older hardware may not implement that.

Parameters
task A task from TaskManager.

Raises
UnsupportedDriverExtension if requested operation is not supported by the driver

Raises
DriverOperationError or its derivative in case of driver runtime error.

Raises
MissingParameterValue if a required parameter is missing

Returns

A list with the supported boot modes defined in *ironic.common.boot_modes*. If boot mode support cant be determined, empty list is returned.

get_supported_indicators(*task*, *component=None*)

Get a map of the supported indicators (e.g. LEDs).

Parameters

- **task** A task from TaskManager.
- **component** If not *None*, return indicator information for just this component, otherwise return indicators for all existing components.

Returns

A dictionary of hardware components (*ironic.common.components*) as keys with values being dictionaries having indicator IDs as keys and indicator properties as values.

```
{
    'chassis': {
        'enclosure-0': {
            "readonly": true,
            "states": [
                "off",
                "on"
            ]
        }
    },
    'system':
        'blade-A': {
            "readonly": true,
            "states": [
                "pff",
                "on"
            ]
        }
    },
    'drive':
        'ssd0': {
            "readonly": true,
            "states": [
                "off",
                "on"
            ]
        }
    }
}
```

get_virtual_media(*task*)

Get all virtual media devices from the node.

Parameters

task A TaskManager instance containing the node to act on.

Raises
UnsupportedDriverExtension

`inject_nmi`(*task*)

Inject NMI, Non Maskable Interrupt.

Inject NMI (Non Maskable Interrupt) for a node immediately.

Parameters
task A TaskManager instance containing the node to act on.

Raises
UnsupportedDriverExtension

`interface_type = 'management'`

Interface type, used for clean steps and logging.

`abstract set_boot_device`(*task*, *device*, *persistent=False*)

Set the boot device for a node.

Set the boot device to use on next reboot of the node.

Parameters

- **task** A task from TaskManager.
- **device** The boot device, one of *ironic.common.boot_devices*.
- **persistent** Boolean value. True if the boot device will persist to all future boots, False if not. Default: False.

Raises
InvalidParameterValue if an invalid boot device is specified.

Raises
MissingParameterValue if a required parameter is missing

`set_boot_mode`(*task*, *mode*)

Set the boot mode for a node.

Set the boot mode to use on next reboot of the node.

Drivers implementing this method are required to implement the *get_supported_boot_modes* method as well.

NOTE: Not all drivers support this method. Hardware supporting only
one boot mode may not implement that.

Parameters

- **task** A task from TaskManager.
- **mode** The boot mode, one of *ironic.common.boot_modes*.

Raises
InvalidParameterValue if an invalid boot mode is specified.

Raises
MissingParameterValue if a required parameter is missing

Raises
UnsupportedDriverExtension if requested operation is not supported by the driver

Raises
DriverOperationError or its derivative in case of driver runtime error.

`set_indicator_state`(*task*, *component*, *indicator*, *state*)

Set indicator on the hardware component to the desired state.

Parameters

- **task** A task from TaskManager.
- **component** The hardware component, one of *ironic.common.components*.
- **indicator** Indicator ID (as reported by *get_supported_indicators*).

State
Desired state of the indicator, one of *ironic.common.indicator_states*.

Raises
InvalidParameterValue if an invalid component, indicator or state is specified.

Raises
MissingParameterValue if a required parameter is missing

`set_secure_boot_state`(*task*, *state*)

Set the current secure boot state for the node.

NOTE: Not all drivers support this method. Older hardware
may not implement that.

Parameters

- **task** A task from TaskManager.
- **state** A new state as a boolean.

Raises
MissingParameterValue if a required parameter is missing

Raises
DriverOperationError or its derivative in case of driver runtime error.

Raises
UnsupportedDriverExtension if secure boot is not supported by the driver or the hardware

`class ironic.drivers.base.NetworkInterface`(**args*, ***kwargs*)

Bases: *BaseInterface*

Base class for network interfaces.

`abstract add_cleaning_network`(*task*)

Add the cleaning network to a node.

Parameters
task A TaskManager instance.

Returns
a dictionary in the form {port.uuid: neutron_port[id]}

Raises
NetworkError

`add_inspection_network`(*task*)

Add the inspection network to the node.

Parameters
task A TaskManager instance.

Returns
a dictionary in the form {port.uuid: neutron_port[id]}

Raises
NetworkError

Raises
InvalidParameterValue, if the network interface configuration is invalid.

`abstract add_provisioning_network`(*task*)

Add the provisioning network to a node.

Parameters
task A TaskManager instance.

Raises
NetworkError

`add_rescuing_network`(*task*)

Add the rescuing network to the node.

Parameters
task A TaskManager instance.

Returns
a dictionary in the form {port.uuid: neutron_port[id]}

Raises
NetworkError

Raises
InvalidParameterValue, if the network interface configuration is invalid.

`add_servicing_network`(*task*)

Add the servicing network to the node.

Parameters
task A TaskManager instance.

Returns
a dictionary in the form {port.uuid: neutron_port[id]}

Raises
NetworkError

Raises
InvalidParameterValue, if the network interface configuration is invalid.

`capabilities = []`

`abstract configure_tenant_networks(task)`

Configure tenant networks for a node.

Parameters
`task` A TaskManager instance.

Raises
NetworkError

`abstract get_current_vif(task, p_obj)`

Returns the currently used VIF associated with port or portgroup

We are booting the node only in one network at a time, and presence of cleaning_vif_port_id means were doing cleaning, of provisioning_vif_port_id - provisioning, of rescuing_vif_port_id - rescuing. Otherwise its a tenant network.

Parameters

- `task` A TaskManager instance.
- `p_obj` Ironic port or portgroup object.

Returns
VIF ID associated with p_obj or None.

`get_node_network_data(task)`

Return network configuration for node NICs.

Gather L2 and L3 network settings from ironic port/portgroups objects and underlying network provider, then put together collected data in form of Nova network metadata (*network_data.json*) dict.

Ironic would eventually pass network configuration to the node being managed out-of-band.

Parameters
`task` A TaskManager instance.

Raises
InvalidParameterValue, if the network interface configuration is invalid.

Raises
MissingParameterValue, if some parameters are missing.

Returns
a dict holding network configuration information adhering Nova network metadata layout (*network_data.json*).

`get_properties()`

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

`interface_type = 'network'`

Interface type, used for clean steps and logging.

need_power_on(*task*)

Check if node must be powered on before applying network changes

Parameters
task A TaskManager instance.

Returns
Boolean.

abstract port_changed(*task*, *port_obj*)

Handle any actions required when a port changes

Parameters

- task A TaskManager instance.
- port_obj a changed Port object.

Raises
Conflict, FailedToUpdateDHCPOptOnPort

abstract portgroup_changed(*task*, *portgroup_obj*)

Handle any actions required when a port changes

Parameters

- task A TaskManager instance.
- portgroup_obj a changed Port object.

Raises
Conflict, FailedToUpdateDHCPOptOnPort

abstract remove_cleaning_network(*task*)

Remove the cleaning network from a node.

Parameters
task A TaskManager instance.

Raises
NetworkError

remove_inspection_network(*task*)

Removes the inspection network from a node.

Parameters
task A TaskManager instance.

Raises
NetworkError

Raises
InvalidParameterValue, if the network interface configuration is invalid.

Raises
MissingParameterValue, if some parameters are missing.

abstract remove_provisioning_network(*task*)

Remove the provisioning network from a node.

Parameters
task A TaskManager instance.

`remove_rescuing_network`(*task*)

Removes the rescuing network from a node.

Parameters
task A TaskManager instance.

Raises
NetworkError

Raises
InvalidParameterValue, if the network interface configuration is invalid.

Raises
MissingParameterValue, if some parameters are missing.

`remove_servicing_network`(*task*)

Removes the servicing network from a node.

Parameters
task A TaskManager instance.

Raises
NetworkError

Raises
InvalidParameterValue, if the network interface configuration is invalid.

Raises
MissingParameterValue, if some parameters are missing.

`abstract unconfigure_tenant_networks`(*task*)

Unconfigure tenant networks for a node.

Parameters
task A TaskManager instance.

`validate`(*task*)

Validates the network interface.

Parameters
task A TaskManager instance.

Raises
InvalidParameterValue, if the network interface configuration is invalid.

Raises
MissingParameterValue, if some parameters are missing.

`validate_inspection`(*task*)

Validate that the node has required properties for inspection.

Parameters
task A TaskManager instance with the node being checked

Raises
MissingParameterValue if node is missing one or more required parameters

Raises
UnsupportedDriverExtension

validate_rescue(*task*)

Validates the network interface for rescue operation.

Parameters
task A TaskManager instance.

Raises
InvalidParameterValue, if the network interface configuration is invalid.

Raises
MissingParameterValue, if some parameters are missing.

abstract vif_attach(*task*, *vif_info*)

Attach a virtual network interface to a node

Parameters

- **task** A TaskManager instance.
- **vif_info** a dictionary of information about a VIF. It must have an id key, whose value is a unique identifier for that VIF.

Raises
NetworkError, VifAlreadyAttached, NoFreePhysicalPorts

abstract vif_detach(*task*, *vif_id*)

Detach a virtual network interface from a node

Parameters

- **task** A TaskManager instance.
- **vif_id** A VIF ID to detach

Raises
NetworkError, VifNotAttached

abstract vif_list(*task*)

List attached VIF IDs for a node

Parameters
task A TaskManager instance.

Returns
List of VIF dictionaries, each dictionary will have an id entry with the ID of the VIF.

class ironic.drivers.base.**PowerInterface**(**args*, ***kwargs*)

Bases: *BaseInterface*

Interface for power-related actions.

abstract get_power_state(*task*)

Return the power state of the tasks node.

Parameters
task A TaskManager instance containing the node to act on.

Raises
MissingParameterValue if a required parameter is missing.

Returns
A power state. One of *ironic.common.states*.

`get_supported_power_states`(*task*)

Get a list of the supported power states.

Parameters
task A TaskManager instance containing the node to act on.

Returns
A list with the supported power states defined in *ironic.common.states*.

`interface_type = 'power'`

Interface type, used for clean steps and logging.

`abstract reboot`(*task*, *timeout=None*)

Perform a hard reboot of the tasks node.

Drivers are expected to properly handle case when node is powered off by powering it on.

Parameters

- **task** A TaskManager instance containing the node to act on.
- **timeout** timeout (in seconds) positive integer (> 0) for any power state. `None` indicates to use default timeout.

Raises
MissingParameterValue if a required parameter is missing.

`abstract set_power_state`(*task*, *power_state*, *timeout=None*)

Set the power state of the tasks node.

Parameters

- **task** A TaskManager instance containing the node to act on.
- **power_state** Any power state from *ironic.common.states*.
- **timeout** timeout (in seconds) positive integer (> 0) for any power state. `None` indicates to use default timeout.

Raises
MissingParameterValue if a required parameter is missing.

`supports_power_sync`(*task*)

Check if power sync is supported for the given node.

If `False`, the conductor will simply store whatever `get_power_state` returns in the database instead of trying to force the expected power state.

Parameters
task A TaskManager instance containing the node to act on.

Returns
boolean, whether power sync is supported.

class ironic.drivers.base.**RAIDInterface**(**args*, ***kwargs*)

Bases: *BaseInterface*

apply_configuration(*task*, *raid_config*, *create_root_volume=True*, *create_nonroot_volumes=True*, *delete_existing=True*)

Applies RAID configuration on the given node.

Parameters

- **task** A TaskManager instance.
- **raid_config** The RAID configuration to apply.
- **create_root_volume** Setting this to False indicates not to create root volume that is specified in raid_config. Default value is True.
- **create_nonroot_volumes** Setting this to False indicates not to create non-root volumes (all except the root volume) in raid_config. Default value is True.
- **delete_existing** Setting this to True indicates to delete RAID configuration prior to creating the new configuration.

Raises

InvalidParameterValue, if the RAID configuration is invalid.

Returns

states.DEPLOYWAIT if RAID configuration is in progress asynchronously or None if it is complete.

abstract create_configuration(*task*, *create_root_volume=True*, *create_nonroot_volumes=True*, *delete_existing=True*)

Creates RAID configuration on the given node.

This method creates a RAID configuration on the given node. It assumes that the target RAID configuration is already available in node.target_raid_config. Implementations of this interface are supposed to read the RAID configuration from node.target_raid_config. After the RAID configuration is done (either in this method OR in a call-back method), ironic.common.raid.update_raid_info() may be called to sync the nodes RAID-related information with the RAID configuration applied on the node.

Parameters

- **task** A TaskManager instance.
- **create_root_volume** Setting this to False indicates not to create root volume that is specified in the nodes target_raid_config. Default value is True.
- **create_nonroot_volumes** Setting this to False indicates not to create non-root volumes (all except the root volume) in the nodes target_raid_config. Default value is True.
- **delete_existing** Setting this to True indicates to delete RAID configuration prior to creating the new configuration.

Returns

states.CLEANWAIT (cleaning) or states.DEPLOYWAIT (deployment) if RAID configuration is in progress asynchronously, or None if it is complete.

abstract delete_configuration(*task*)

Deletes RAID configuration on the given node.

This method deletes the RAID configuration on the give node. After RAID configuration is deleted, node.raid_config should be cleared by the implementation.

Parameters
task A TaskManager instance.

Returns
states.CLEANWAIT (cleaning) or states.DEPLOYWAIT (deployment) if deletion is in progress asynchronously, or None if it is complete.

get_logical_disk_properties()

Get the properties that can be specified for logical disks.

This method returns a dictionary containing the properties that can be specified for logical disks and a textual description for them.

Returns
A dictionary containing properties that can be mentioned for logical disks and a textual description for them.

get_properties()

Return the properties of the interface.

Returns
dictionary of <property name>:<property description> entries.

interface_type = 'raid'

Interface type, used for clean steps and logging.

validate(*task*)

Validates the RAID Interface.

This method validates the properties defined by Ironic for RAID configuration. Driver implementations of this interface can override this method for doing more validations (such as BMCs credentials).

Parameters
task A TaskManager instance.

Raises
InvalidParameterValue, if the RAID configuration is invalid.

Raises
MissingParameterValue, if some parameters are missing.

validate_raid_config(*task*, *raid_config*)

Validates the given RAID configuration.

This method validates the given RAID configuration. Driver implementations of this interface can override this method to support custom parameters for RAID configuration.

Parameters

- **task** A TaskManager instance.
- **raid_config** The RAID configuration to validate.

Raises
InvalidParameterValue, if the RAID configuration is invalid.

ironic.drivers.base.**RAID_APPLY_CONFIGURATION_ARGSINFO** = {'create_nonroot_volumes': {'description': "Setting this to 'False' indicates not to create non-root volumes (all except the root volume) in 'raid_config'. Default value is 'True'.", 'required': False}, 'create_root_volume': {'description': "Setting this to 'False' indicates not to create root volume that is specified in 'raid_config'. Default value is 'True'.", 'required': False}, 'delete_existing': {'description': "Setting this to 'True' indicates to delete existing RAID configuration prior to creating the new configuration. Default value is 'True'.", 'required': False}, 'raid_config': {'description': 'The RAID configuration to apply.', 'required': True}}

This may be used as the deploy_step argsinfo argument for RAID interfaces implementing an apply_configuration deploy step.

class ironic.drivers.base.**RescueInterface**(**args*, ***kwargs*)

Bases: *BaseInterface*

Interface for rescue-related actions.

clean_up(*task*)

Clean up the rescue environment for the tasks node.

This is particularly useful for nodes where rescuing is asynchronous and a timeout occurs.

Parameters
task A TaskManager instance containing the node to act on.

Returns
None

interface_type = 'rescue'

Interface type, used for clean steps and logging.

abstract rescue(*task*)

Boot the tasks node into a rescue environment.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InstanceRescueFailure if node validation or rescue operation fails.

Returns
states.RESCUEWAIT if rescue is in progress asynchronously or states.RESCUE if it is complete.

abstract unrescue(*task*)

Tear down the rescue environment, and return to normal.

Parameters
task A TaskManager instance containing the node to act on.

Raises
InstanceUnrescueFailure if node validation or unrescue operation fails.

Returns
states.ACTIVE if it is successful.

class ironic.drivers.base.**StorageInterface**(**args*, ***kwargs*)

Bases: *BaseInterface*

Base class for storage interfaces.

abstract attach_volumes(*task*)

Informs the storage subsystem to attach all volumes for the node.

Parameters
task A TaskManager instance.

Raises
UnsupportedDriverExtension

abstract detach_volumes(*task*)

Informs the storage subsystem to detach all volumes for the node.

Parameters
task A TaskManager instance.

Raises
UnsupportedDriverExtension

interface_type = 'storage'

Interface type, used for clean steps and logging.

abstract should_write_image(*task*)

Determines if deploy should perform the image write-out.

Parameters
task A TaskManager instance.

Returns
Boolean value to indicate if the interface expects the image to be written by Ironic.

Raises
UnsupportedDriverExtension

class ironic.drivers.base.**VendorInterface**(**args*, ***kwargs*)

Bases: *BaseInterface*

Interface for all vendor passthru functionality.

Additional vendor- or driver-specific capabilities should be implemented as a method in the class inheriting from this class and use the @passthru or @driver_passthru decorators.

Methods decorated with @driver_passthru should be short-lived because it is a blocking call.

driver_validate(*method*, ***kwargs*)

Validate driver-vendor-passthru actions.

If invalid, raises an exception; otherwise returns None.

Parameters

- **method** method to be validated

- **kwargs** info for action.

Raises
MissingParameterValue if kwargs does not contain certain parameter.

Raises
InvalidParameterValue if parameter does not match.

interface_type = 'vendor'
Interface type, used for clean steps and logging.

abstract validate(*task*, *method=None*, ***kwargs*)
Validate vendor-specific actions.

If invalid, raises an exception; otherwise returns None.

Parameters

- **task** A task from TaskManager.
- **method** Method to be validated
- **kwargs** Info for action.

Raises
UnsupportedDriverExtension if method can not be mapped to the supported interfaces.

Raises
InvalidParameterValue if kwargs does not contain method.

Raises
MissingParameterValue

class ironic.drivers.base.**VendorMetadata**(*method*, *metadata*)
Bases: tuple

metadata
Alias for field number 1

method
Alias for field number 0

ironic.drivers.base.**cache_bios_settings**(*func*)
A decorator to cache bios settings after running the function.

Parameters
func Function or method to wrap.

ironic.drivers.base.**cache_firmware_components**(*func*)
A decorator to cache firmware components after running the function.

Parameters
func Function or method to wrap.

ironic.drivers.base.**clean_step**(*priority*, *abortable=False*, *argsinfo=None*, *requires_ramdisk=True*)
Decorator for cleaning steps.

Cleaning steps may be used in manual or automated cleaning.

For automated cleaning, only steps with priorities greater than 0 are used. These steps are ordered by priority from highest value to lowest value. For steps with the same priority, they are ordered by driver interface priority (see conductor.steps.CLEANING_INTERFACE_PRIORITY). execute_clean_step() will be called on each step.

For manual cleaning, the clean steps will be executed in a similar fashion to automated cleaning, but the steps and order of execution must be explicitly specified by the user when invoking the cleaning API.

Decorated clean steps must take as the only positional argument, a TaskManager object. Clean steps used in manual cleaning may also take keyword variable arguments (as described in argsinfo).

Clean steps can be either synchronous or asynchronous. If the step is synchronous, it should return *None* when finished, and the conductor will continue on to the next step. While the clean step is executing, the node will be in *states.CLEANING* provision state. If the step is asynchronous, the step should return *states.CLEANWAIT* to the conductor before it starts the asynchronous work. When the step is complete, the step should make an RPC call to *continue_node_clean* to move to the next step in cleaning. The node will be in *states.CLEANWAIT* provision state during the asynchronous work.

Examples:

```
class MyInterface(base.BaseInterface):
    # CONF.example_cleaning_priority should be an int CONF option
    @base.clean_step(priority=CONF.example_cleaning_priority)
    def example_cleaning(self, task):
        # do some cleaning

    @base.clean_step(priority=0, abortable=True, argsinfo=
                     {'size': {'description': 'size of widget (MB)',
                               'required': True}})
    def advanced_clean(self, task, **kwargs):
        # do some advanced cleaning
```

Parameters

- **priority** an integer priority, should be a CONF option
- **abortable** Boolean value. Whether the clean step is abortable or not; defaults to False.
- **argsinfo** a dictionary of keyword arguments where key is the name of the argument and value is a dictionary as follows:

```
'description': <description>. Required. This should␣
↪include
               possible values.
'required': Boolean. Optional; default is False. True if␣
↪this
            argument is required.  If so, it must be␣
↪specified in
            the clean request; false if it is optional.
```

- **requires_ramdisk** Whether this step requires the ramdisk to be running. Should be set to False for purely out-of-band steps.

Raises

InvalidParameterValue if any of the arguments are invalid

ironic.drivers.base.**deploy_step**(*priority*, *abortable=True*, *argsinfo=None*)

Decorator for deployment steps.

Only steps with priorities greater than 0 are used. These steps are ordered by priority from highest value to lowest value. For steps with the same priority, they are ordered by driver interface priority (see conductor.steps.DEPLOYING_INTERFACE_PRIORITY). execute_deploy_step() will be called on each step.

Decorated deploy steps must take as the only positional argument, a TaskManager object.

Deploy steps can be either synchronous or asynchronous. If the step is synchronous, it should return *None* when finished, and the conductor will continue on to the next step. While the deploy step is executing, the node will be in *states.DEPLOYING* provision state. If the step is asynchronous, the step should return *states.DEPLOYWAIT* to the conductor before it starts the asynchronous work. When the step is complete, the step should make an RPC call to *continue_node_deploy* to move to the next step in deployment. The node will be in *states.DEPLOYWAIT* provision state during the asynchronous work.

Examples:

```
class MyInterface(base.BaseInterface):
    @base.deploy_step(priority=100)
    def example_deploying(self, task):
        # do some deploying

    @base.deploy_step(priority=50, abortable=False)
    def non_abortable_deploy(self, task):
        # do some critical deploying that cannot be aborted
```

Parameters

- **priority** an integer (>=0) priority; used for determining the order in which the step is run in the deployment process.
- **abortable** Boolean value. Whether the deploy step is abortable or not; defaults to True.
- **argsinfo** a dictionary of keyword arguments where key is the name of the argument and value is a dictionary as follows:

```
'description': <description>. Required. This should␣
↪include
               possible values.
'required': Boolean. Optional; default is False. True if␣
↪this
            argument is required.  If so, it must be␣
↪specified in
            the deployment request; false if it is␣
↪optional.
```

Raises

InvalidParameterValue if any of the arguments are invalid

ironic.drivers.base.**driver_passthru**(*http_methods*, *method=None*, *async_call=True*, *description=None*, *attach=False*)

ironic.drivers.base.**passthru**(*http_methods*, *method=None*, *async_call=True*, *description=None*, *attach=False*, *require_exclusive_lock=True*)

ironic.drivers.base.**service_step**(*priority=None*, *abortable=False*, *argsinfo=None*, *requires_ramdisk=True*)

Decorator for service steps.

Service steps may be used in performing service upon a node.

For service, the steps will be executed in a similar fashion to cleaning, but the steps and order of execution must be explicitly specified by the user when invoking the servicing API.

Decorated service steps must take as the only a single positional argument, a TaskManager object, in addition to a keyword arguments variable (as described in argsinfo).

Service steps can be either synchronous or asynchronous. If the step is synchronous, it should return *None* when finished, and the conductor will continue on to the next step. While the clean step is executing, the node will be in *states.SERVICING* provision state. If the step is asynchronous, the step should return *states.SERVICEWAIT* to the conductor before it starts the asynchronous work. When the step is complete, the step should make an RPC call to *continue_node_service* to move to the next step in servicing. The node will be in *states.SERVICEWAIT* provision state during the asynchronous work.

Examples:

```
class MyInterface(base.BaseInterface):
    @base.service_step()
    def example_service(self, task):
        # do some service actions

    @base.service_step(priority=0, abortable=True, argsinfo=
                       {'size': {'description': 'size of widget (MB)',
                                 'required': True}})
    def advanced_service(self, task, **kwargs):
        # do some advanced magical service
```

Parameters

- **priority** an integer priority, defaults to None which maps to 0. Priorities are not considered, by default but exists should this functionality be adopted later on to align with the steps framework.
- **abortable** Boolean value. Whether the clean step is abortable or not; defaults to False.
- **argsinfo** a dictionary of keyword arguments where key is the name of the argument and value is a dictionary as follows:

```
'description': <description>. Required. This should␣
↪include
               possible values.
'required': Boolean. Optional; default is False. True if␣
```

(continues on next page)

(continued from previous page)

```
↪this
            argument is required.  If so, it must be␣
↪specified in
            the service request; false if it is optional.
```

- **requires_ramdisk** Whether this step requires the ramdisk to be running. Should be set to False for purely out-of-band steps.

Raises

InvalidParameterValue if any of the arguments are invalid

ironic.drivers.base.**verify_step**(*priority*)

Decorator for verify steps.

Only steps with priorities greater than 0 are used. These steps are ordered by priority from highest value to lowest value. For steps with the same priority, they are ordered by driver interface priority (see conductor.steps.VERIFY_INTERFACE_PRIORITY). execute_verify_step() will be called on each step.

Decorated verify steps must take as the only positional argument, a TaskManager object.

Verify steps are synchronous and should return *None* when finished, and the conductor will continue on to the next step. While the verify step is executing, the node will be in *states.VERIFYING* provision state.

Examples:

```
class MyInterface(base.BaseInterface):
    @base.verify_step(priority=100)
    def example_verifying(self, task):
        # do some verifying
```

Parameters

priority an integer (>=0) priority; used for determining the order in which the step is run in the verification process.

Raises

InvalidParameterValue if any of the arguments are invalid

ironic.drivers.drac module

DRAC Driver for remote system management using Dell Remote Access Card.

class ironic.drivers.drac.**IDRACHardware**

Bases: *RedfishHardware*

integrated Dell Remote Access Controller hardware type

property supported_bios_interfaces

List of supported bios interfaces.

property supported_boot_interfaces

List of supported boot interfaces.

property supported_inspect_interfaces

List of supported inspect interfaces.

property supported_management_interfaces

List of supported management interfaces.

property supported_power_interfaces

List of supported power interfaces.

property supported_raid_interfaces

List of supported raid interfaces.

property supported_vendor_interfaces

List of supported vendor interfaces.

ironic.drivers.fake_hardware module

Fake hardware type.

class ironic.drivers.fake_hardware.**FakeHardware**

Bases: *GenericHardware*

Fake hardware type.

This hardware type is special-cased in the driver factory to bypass compatibility verification. Thus, supported_* methods here are only for calculating the defaults, not for actual check.

All fake implementations are still expected to be enabled in the configuration.

property supported_bios_interfaces

List of classes of supported bios interfaces.

property supported_boot_interfaces

List of classes of supported boot interfaces.

property supported_console_interfaces

List of classes of supported console interfaces.

property supported_deploy_interfaces

List of classes of supported deploy interfaces.

property supported_firmware_interfaces

List of classes of supported bios interfaces.

property supported_inspect_interfaces

List of classes of supported inspect interfaces.

property supported_management_interfaces

List of classes of supported management interfaces.

property supported_power_interfaces

List of classes of supported power interfaces.

property supported_raid_interfaces

List of classes of supported raid interfaces.

property supported_rescue_interfaces

List of classes of supported rescue interfaces.

property supported_storage_interfaces

List of classes of supported storage interfaces.

property supported_vendor_interfaces

List of classes of supported rescue interfaces.

ironic.drivers.generic module

Generic hardware types.

class ironic.drivers.generic.**GenericHardware**

Bases: *AbstractHardwareType*

Abstract base class representing generic hardware.

This class provides reasonable defaults for all of the interfaces.

property supported_boot_interfaces

List of supported boot interfaces.

property supported_console_interfaces

List of classes of supported console interfaces.

property supported_deploy_interfaces

List of supported deploy interfaces.

property supported_firmware_interfaces

List of supported firmware interfaces.

property supported_inspect_interfaces

List of supported inspect interfaces.

property supported_network_interfaces

List of supported network interfaces.

property supported_raid_interfaces

List of supported raid interfaces.

property supported_rescue_interfaces

List of supported rescue interfaces.

property supported_storage_interfaces

List of supported storage interfaces.

class ironic.drivers.generic.**ManualManagementHardware**

Bases: *GenericHardware*

Hardware type that uses manual power and boot management.

Using this hardware type assumes that an operator manages reboot and setting boot devices manually. This hardware type should only be used when no suitable hardware type exists in ironic, or the existing hardware type misbehaves for any reason.

property supported_management_interfaces

List of supported management interfaces.

property supported_power_interfaces

List of supported power interfaces.

property supported_vendor_interfaces

List of supported vendor interfaces.

ironic.drivers.hardware_type module

Abstract base class for all hardware types.

class ironic.drivers.hardware_type.**AbstractHardwareType**

Bases: object

Abstract base class for all hardware types.

Hardware type is a family of hardware supporting the same set of interfaces from the ironic standpoint. This can be as wide as all hardware supporting the IPMI protocol or as narrow as several hardware models supporting some specific interfaces.

A hardware type defines an ordered list of supported implementations for each driver interface (power, deploy, etc).

get_properties()

Get the properties of the hardware type.

Note that this returns properties for the default interface of each type, for this hardware type. Since this is not node-aware, interface overrides cant be detected.

Returns

dictionary of <property name>:<property description> entries.

supported = True

Whether hardware is supported by the community.

property supported_bios_interfaces

List of supported bios interfaces.

abstract property supported_boot_interfaces

List of supported boot interfaces.

property supported_console_interfaces

List of supported console interfaces.

abstract property supported_deploy_interfaces

List of supported deploy interfaces.

property supported_firmware_interfaces

List of supported firmware interfaces.

property supported_inspect_interfaces

List of supported inspect interfaces.

abstract property supported_management_interfaces

List of supported management interfaces.

property supported_network_interfaces

List of supported network interfaces.

abstract property supported_power_interfaces

List of supported power interfaces.

property supported_raid_interfaces

List of supported raid interfaces.

property supported_rescue_interfaces

List of supported rescue interfaces.

property supported_storage_interfaces

List of supported storage interfaces.

property supported_vendor_interfaces

List of supported vendor interfaces.

ironic.drivers.ilo module

iLO Driver for managing HP Proliant Gen8 and above servers.

class ironic.drivers.ilo.**Ilo5Hardware**

Bases: *IloHardware*

iLO5 hardware type.

iLO5 hardware type is targeted for iLO5 based Proliant Gen10 servers.

property supported_boot_interfaces

List of supported boot interfaces.

property supported_management_interfaces

List of supported management interfaces.

property supported_raid_interfaces

List of supported raid interfaces.

class ironic.drivers.ilo.**IloHardware**

Bases: *GenericHardware*

iLO hardware type.

iLO hardware type is targeted for iLO 4 based Proliant Gen8 and Gen9 servers.

property supported_bios_interfaces

List of supported bios interfaces.

property supported_boot_interfaces

List of supported boot interfaces.

property supported_console_interfaces

List of supported console interfaces.

property supported_inspect_interfaces

List of supported inspect interfaces.

property supported_management_interfaces

List of supported management interfaces.

property supported_power_interfaces

List of supported power interfaces.

property supported_vendor_interfaces

List of supported vendor interfaces.

ironic.drivers.intel_ipmi module

class ironic.drivers.intel_ipmi.**IntelIPMIHardware**

Bases: *IPMIHardware*

Intel IPMI hardware type.

Uses `ipmitool` to implement power and management. Provides serial console implementations via `socat`. Supports Intel SST-PP feature.

property supported_management_interfaces

List of supported management interfaces.

ironic.drivers.ipmi module

Hardware type for IPMI (using ipmitool).

class ironic.drivers.ipmi.**IPMIHardware**

Bases: *GenericHardware*

IPMI hardware type.

Uses `ipmitool` to implement power and management. Provides serial console implementations via `socat`.

property supported_console_interfaces

List of supported console interfaces.

property supported_management_interfaces

List of supported management interfaces.

property supported_power_interfaces

List of supported power interfaces.

property supported_vendor_interfaces

List of supported vendor interfaces.

ironic.drivers.redfish module

class ironic.drivers.redfish.**RedfishHardware**

Bases: *GenericHardware*

Redfish hardware type.

property supported_bios_interfaces

List of supported bios interfaces.

property supported_boot_interfaces
List of supported boot interfaces.

property supported_console_interfaces
List of supported console interfaces.

property supported_firmware_interfaces
List of supported firmware interfaces.

property supported_inspect_interfaces
List of supported power interfaces.

property supported_management_interfaces
List of supported management interfaces.

property supported_power_interfaces
List of supported power interfaces.

property supported_raid_interfaces
List of supported raid interfaces.

property supported_vendor_interfaces
List of supported vendor interfaces.

ironic.drivers.snmp module

SNMP hardware types.

class ironic.drivers.snmp.**SNMPHardware**
Bases: *GenericHardware*

SNMP Hardware type

supported = False
Whether hardware is supported by the community.

property supported_management_interfaces
List of supported management interfaces.

property supported_power_interfaces
List of supported power interfaces.

ironic.drivers.utils module

class ironic.drivers.utils.**MixinVendorInterface**(**args*, ***kwargs*)
Bases: *VendorInterface*

Wrapper around multiple VendorInterfaces.

get_properties()
Return the properties from all the VendorInterfaces.

Returns
a dictionary of <property_name>:<property_description> entries.

validate(*task*, *method*, ***kwargs*)

Call validate on the appropriate interface only.

Raises
UnsupportedDriverExtension if method can not be mapped to the supported interfaces.

Raises
InvalidParameterValue if method is invalid.

Raises
MissingParameterValue if missing method or parameters in kwargs.

ironic.drivers.utils.**add_node_capability**(*task*, *capability*, *value*)

Add capability to nodes capabilities property.

If capability is already present, then a duplicate entry will be added.

Parameters

- **task** Task object.
- **capability** Capability key.
- **value** Capability value.

ironic.drivers.utils.**capabilities_to_dict**(*capabilities*)

Parse the capabilities string into a dictionary

Parameters
capabilities the capabilities of the node as a formatted string.

Raises
InvalidParameterValue if capabilities is not an string or has a malformed value

ironic.drivers.utils.**collect_ramdisk_logs**(*node*, *label=None*)

Collect and store the system logs from the IPA ramdisk.

Collect and store the system logs from the IPA ramdisk. This method makes a call to the IPA ramdisk to collect the logs and store it according to the configured storage backend.

Parameters

- **node** A node object.
- **label** A string to label the log file such as a clean step name.

ironic.drivers.utils.**ensure_next_boot_device**(*task*, *driver_info*)

Ensure boot from correct device if persistent is True

If ipmi_force_boot_device is True and is_next_boot_persistent, set to boot from correct device, else unset is_next_boot_persistent field.

Parameters

- **task** Node object.
- **driver_info** Node driver_info.

ironic.drivers.utils.**force_persistent_boot**(*task*, *device*, *persistent*)

Set persistent boot device to driver_internal_info

If persistent is True set persistent_boot_device field to the boot device and reset persistent to False, else set is_next_boot_persistent to False.

Parameters

- **task** Task object.
- **device** Boot device.
- **persistent** Whether next boot is persistent or not.

ironic.drivers.utils.**get_agent_iso**(*node*, *mode='deploy'*, *deprecated_prefix=None*)

Get the agent ISO image.

ironic.drivers.utils.**get_agent_kernel_ramdisk**(*node*, *mode='deploy'*, *deprecated_prefix=None*)

Get the agent kernel/ramdisk as a dictionary.

Get the agent kernel/ramdisk for a given $mode. This code enforces

ironic.drivers.utils.**get_field**(*node*, *name*, *deprecated_prefix=None*, *use_conf=False*, *collection='driver_info'*)

Get a driver_info field with deprecated prefix.

ironic.drivers.utils.**get_kernel_append_params**(*node*, *default*)

Get the applicable kernel params.

The locations are checked in this order:

1. The nodes instance_info.
2. The nodes driver_info.
3. Configuration.

Parameters

- **node** Node object.
- **default** Default value.

ironic.drivers.utils.**get_node_capability**(*node*, *capability*)

Returns capability value from nodes capabilities property.

Parameters

- **node** Node object.
- **capability** Capability key.

Returns

Capability value. If capability is not present, then return None

ironic.drivers.utils.**get_node_mac_addresses**(*task*)

Get all MAC addresses for the ports belonging to this tasks node.

Parameters

task a TaskManager instance containing the node to act on.

Returns
A list of MAC addresses in the format xx:xx:xx:xx:xx:xx.

ironic.drivers.utils.**get_ramdisk_logs_file_name**(*node*, *label=None*)

Construct the log file name.

Parameters

- **node** A node object.
- **label** A string to label the log file such as a clean step name.

Returns
The log file name.

ironic.drivers.utils.**get_verify_ca**(*node*, *verify_ca*)

Add verify_ca setting to driver_info if needed.

This function checks if verify_ca needs to be set based on configuration values and existing driver_info settings.

Parameters

- **node** The node object
- **verify_ca** The verify_ca settings in driver_info

Returns
Updated verify_ca setting if needed

ironic.drivers.utils.**need_prepare_ramdisk**(*node*)

Check if node needs preparing ramdisk

Parameters
node Node to check for

Returns
True if need to prepare ramdisk, otherwise False

ironic.drivers.utils.**power_off_and_on**(*task*)

A context manager that handles a reboot.

If disable_power_off is False, the node is powered off before yielding and powered back on afterwards. Otherwise, one reboot is issued in the end.

ironic.drivers.utils.**remove_node_capability**(*task*, *name*)

Remove capability from nodes capabilities property.

If capability is empty, do nothing.

Parameters

- **task** Task object.
- **capability** Capability key.

ironic.drivers.utils.**store_ramdisk_logs**(*node*, *logs*, *label=None*)

Store the ramdisk logs.

This method stores the ramdisk logs according to the configured storage backend.

Parameters

- **node** A node object.
- **logs** A gzipped and base64 encoded string containing the logs archive.
- **label** A string to label the log file such as a clean step name.

Raises
OSError if the directory to save the logs cannot be created.

Raises
IOError when the logs cant be saved to the local file system.

Raises
SwiftOperationError, if any operation with Swift fails.

Module contents

ironic.networking package

Subpackages

ironic.networking.switch_drivers package

Submodules

ironic.networking.switch_drivers.driver_adapter module

Driver Configuration Adapter

This module provides functionality to translate user-friendly switch configuration into driver-specific configuration formats. It allows users to configure switches using a generic format while supporting multiple switch driver implementations.

class ironic.networking.switch_drivers.driver_adapter.**NetworkingDriverAdapter**(*driver_classes*)

Bases: object

Adapter for translating switch config to driver-specific format.

preprocess_config(*output_file*)

Transform user config into driver-specific config files.

Scans oslo.config for switch configurations and generates driver-specific config files that then get written to a driver-specific config file.

Returns
Number of translations generated

register_translator(*driver_type*, *translator_instance*)

Register a custom translator for a driver type.

Parameters

- **driver_type** String identifier for the driver type
- **translator_instance** Instance of a translator class

reload_configuration(*output_file*)

Reload and regenerate switch configuration files.

This method re-extracts switch configurations from the config files and regenerates the driver-specific configuration files. It should be called when the switch configuration file has been modified.

Parameters
output_file Path to the output file for driver-specific configs

Returns
Number of translations generated

Raises
NetworkError if configuration reload fails

ironic.networking.switch_drivers.driver_factory module

Networking service driver factory for loading switch drivers.

This module provides a driver factory system for the networking service, allowing dynamic loading of switch drivers from external projects via entry points.

class ironic.networking.switch_drivers.driver_factory.**BaseSwitchDriverFactory**

Bases: *BaseDriverFactory*

Base factory for discovering, loading and managing switch drivers.

This factory loads switch drivers from entry points and manages their lifecycle. Switch drivers are loaded from external projects and provide vendor-specific implementations for network switch management.

Inherits from common.BaseDriverFactory to ensure consistency with Ironics standard driver factory pattern.

class ironic.networking.switch_drivers.driver_factory.**SwitchDriverFactory**

Bases: *BaseSwitchDriverFactory*

Factory for loading switch drivers from entry points.

ironic.networking.switch_drivers.driver_factory.**get_switch_driver**(*driver_name*)

Get a switch driver instance by name.

Parameters
driver_name Name of the switch driver to retrieve.

Returns
Instance of the switch driver.

Raises
DriverNotFound if the driver is not found.

ironic.networking.switch_drivers.driver_factory.**get_switch_driver_factory**()

Get the global switch driver factory instance.

ironic.networking.switch_drivers.driver_factory.**list_switch_drivers**()

Get a list of all available switch driver names.

Returns
List of switch driver names.

ironic.networking.switch_drivers.driver_factory.**switch_drivers**()

Get all switch drivers as a dictionary.

Returns
Dictionary mapping driver name to driver instance.

Module contents

Submodules

ironic.networking.api module

Networking API for other parts of Ironic to use.

ironic.networking.api.**delete_lag**(*context*, *switch_ids*, *lag_name*)

Delete a link aggregation group (LAG) configuration.

This is a convenience function that other parts of Ironic can use to delete LAG configurations.

Parameters

- **context** request context.
- **switch_ids** List of switch identifiers.
- **lag_name** Name of the LAG to delete.

Raises
InvalidParameterValue if validation fails.

Raises
NetworkError if the network operation fails.

Returns
Dictionary containing the deletion status.

ironic.networking.api.**get_networking_api**()

Get the networking API instance.

Returns
NetworkingAPI instance

ironic.networking.api.**reset_port**(*context*, *switch_id*, *port_name*, *native_vlan=None*, *allowed_vlans=None*, *default_vlan=None*)

Reset a network switch port to default configuration.

This is a convenience function that other parts of Ironic can use to reset network switch ports to their default configurations.

Parameters

- **context** request context.
- **switch_id** Identifier for the switch.
- **port_name** Name of the port on the switch.
- **native_vlan** VLAN ID to be removed from the port.
- **allowed_vlans** List of allowed VLAN IDs to be removed(optional).

- **default_vlan** VLAN ID to restore onto the port(optional).

Raises
InvalidParameterValue if validation fails.

Raises
NetworkError if the network operation fails.

Returns
Dictionary containing the reset port configuration.

ironic.networking.api.**update_lag**(*context*, *switch_ids*, *lag_name*, *description*, *mode*, *native_vlan*, *aggregation_mode*, *allowed_vlans=None*, *default_vlan=None*)

Update a link aggregation group (LAG) configuration.

This is a convenience function that other parts of Ironic can use to update LAG configurations.

Parameters

- **context** request context.
- **switch_ids** List of switch identifiers.
- **lag_name** Name of the LAG.
- **description** Description for the LAG.
- **mode** LAG mode (access or trunk).
- **native_vlan** VLAN ID to be set for the LAG.
- **aggregation_mode** Aggregation mode (e.g., lacp, static).
- **allowed_vlans** List of allowed VLAN IDs to be added (optional).
- **default_vlan** VLAN ID to removed from the port(optional).

Raises
InvalidParameterValue if validation fails.

Raises
NetworkError if the network operation fails.

Returns
Dictionary containing the updated LAG configuration.

ironic.networking.api.**update_port**(*context*, *switch_id*, *port_name*, *description*, *mode*, *native_vlan*, *allowed_vlans=None*, *lag_name=None*, *default_vlan=None*)

Update a network switch port configuration.

This is a convenience function that other parts of Ironic can use to update network switch port configurations.

Parameters

- **context** request context.
- **switch_id** Identifier for the switch.
- **port_name** Name of the port on the switch.

- **description** Description to set for the port.
- **mode** Port mode (access, trunk, or hybrid).
- **native_vlan** VLAN ID to be set on the port.
- **allowed_vlans** List of allowed VLAN IDs to be added(optional).
- **default_vlan** VLAN ID to removed from the port(optional).
- **lag_name** LAG name if port is part of a link aggregation group.

Raises
InvalidParameterValue if validation fails.

Raises
NetworkError if the network operation fails.

Returns
Dictionary containing the updated port configuration.

ironic.networking.manager module

Networking service manager for Ironic.

The networking service handles network-related operations for Ironic, providing RPC interfaces for configuring switch ports and network settings.

class ironic.networking.manager.**NetworkingManager**(*host*, *topic=None*)

Bases: object

Ironic Networking service manager.

RPC_API_VERSION = '1.0'

cleanup()

Clean up resources.

delete_lag(***kwargs*)

get_switches(***kwargs*)

init_host(*admin_context=None*)

Initialize the networking service host.

This method implements two-phase driver initialization: 1. Load driver classes (but dont initialize instances yet) 2. Get translators from driver classes and preprocess config 3. Initialize driver instances (config files now exist)

Parameters
admin_context admin context (unused but kept for compatibility)

prepare_host()

Prepare host for networking service initialization.

This method is called by the RPC service before starting the listener. Since networking host configuration is now handled in __init__, this method is a no-op.

reset_port(***kwargs*)

target = <Target version=1.0>

update_lag(***kwargs*)

update_port(***kwargs*)

ironic.networking.manager.**validate_vlan_configuration**(*operation*, *switch_id_arg_name='switch_id'*)

Decorator to validate VLAN configuration against allowed/denied lists.

This decorator extracts native_vlan and allowed_vlans from method arguments and validates them against the switchs VLAN configuration.

Parameters

- **operation** Description of the operation for error messages.
- **switch_id_arg_name** Name of the argument containing the switch ID. For multi-switch operations, this should be the argument containing the list of switch IDs.

Returns

Decorated function that applies VLAN validation.

ironic.networking.rpc_service module

class ironic.networking.rpc_service.**NetworkingRPCService**(*host*, *manager_module*, *manager_class*)

Bases: *BaseRPCService*

RPC service for the Ironic Networking Manager.

stop()

Stop the networking service.

ironic.networking.rpcapi module

Client side of the networking RPC API.

class ironic.networking.rpcapi.**NetworkingAPI**(*topic=None*)

Bases: object

Client side of the networking RPC API.

API version history:

1.0 - Initial version.

RPC_API_VERSION = '1.0'

delete_lag(*context*, *switch_ids*, *lag_name*, *topic=None*)

Delete a link aggregation group (LAG) configuration.

Parameters

- **context** request context.
- **switch_ids** List of switch identifiers.
- **lag_name** Name of the LAG to delete.
- **topic** RPC topic. Defaults to self.topic.

Raises
InvalidParameterValue if validation fails.

Raises
NetworkError if the network operation fails.

Returns
Dictionary containing the deletion status.

get_switches(*context*, *topic=None*)

Get information about all configured switches.

Parameters

- **context** request context.
- **topic** RPC topic. Defaults to self.topic.

Raises
NetworkError if the network operation fails.

Returns
Dictionary with switch_id as key and switch_info as value.

get_topic()

Get RPC topic name for the networking service.

reset_port(*context*, *switch_id*, *port_name*, *native_vlan*, *allowed_vlans=None*, *default_vlan=None*, *topic=None*)

Reset a network switch port to default configuration.

Parameters

- **context** request context.
- **switch_id** Identifier for the switch.
- **port_name** Name of the port on the switch.
- **native_vlan** VLAN ID to be removed from the port.
- **allowed_vlans** List of allowed VLAN IDs to be removed(optional).
- **default_vlan** VLAN ID to restore onto the port(optional).
- **topic** RPC topic. Defaults to self.topic.

Raises
InvalidParameterValue if validation fails.

Raises
NetworkError if the network operation fails.

Returns
Dictionary containing the reset port configuration.

update_lag(*context*, *switch_ids*, *lag_name*, *description*, *mode*, *native_vlan*, *aggregation_mode*, *allowed_vlans=None*, *default_vlan=None*, *topic=None*)

Update a link aggregation group (LAG) configuration.

Parameters

- **context** request context.
- **switch_ids** List of switch identifiers.
- **lag_name** Name of the LAG.
- **description** Description for the LAG.
- **mode** LAG mode (access or trunk).
- **native_vlan** VLAN ID to be set for the LAG.
- **aggregation_mode** Aggregation mode (e.g., lacp, static).
- **allowed_vlans** List of allowed VLAN IDs to be added (optional).
- **default_vlan** VLAN ID to removed from the port(optional).
- **topic** RPC topic. Defaults to self.topic.

Raises
InvalidParameterValue if validation fails.

Raises
NetworkError if the network operation fails.

Returns
Dictionary containing the updated LAG configuration.

update_port(*context*, *switch_id*, *port_name*, *description*, *mode*, *native_vlan*, *allowed_vlans=None*, *default_vlan=None*, *lag_name=None*, *topic=None*)

Update a port configuration on a switch.

Parameters

- **context** request context.
- **switch_id** Identifier for the switch.
- **port_name** Name of the port on the switch.
- **description** Description to set for the port.
- **mode** Port mode (access, trunk, or hybrid).
- **native_vlan** VLAN ID to be set on the port.
- **allowed_vlans** List of allowed VLAN IDs to be added(optional).
- **default_vlan** VLAN ID to removed from the port(optional).
- **lag_name** LAG name if port is part of a link aggregation group.
- **topic** RPC topic. Defaults to self.topic.

Raises
InvalidParameterValue if validation fails.

Raises
NetworkError if the network operation fails.

Returns
Dictionary containing the updated port configuration.

ironic.networking.switch_config module

Utilities for switch configuration and VLAN management.

ironic.networking.switch_config.**get_switch_vlan_config**(*switch_driver*, *switch_id*)

Get VLAN configuration for a switch.

Retrieves switch-specific VLAN configuration from the driver, with fallback to global configuration options.

Parameters

- **switch_driver** The switch driver instance.
- **switch_id** Identifier of the switch.

Returns
Dictionary containing allowed_vlans

Raises
Any exceptions from switch_driver.get_switch_info()

ironic.networking.switch_config.**validate_vlan_allowed**(*vlan_id*, *allowed_vlans_config=None*, *switch_config=None*)

Validate that a VLAN ID is allowed.

Parameters

- **vlan_id** The VLAN ID to validate
- **allowed_vlans_config** Global list of allowed vlans from config
- **switch_config** Optional switch-specific configuration dict that may contain an allowed_vlans key

Returns
True if the VLAN is allowed

Raises
InvalidParameterValue if the VLAN is not allowed

ironic.networking.switch_config.**validate_vlan_configuration**(*vlans_to_check*, *switch_driver*, *switch_id*, *operation_description='operation'*)

Validate VLAN configuration against allowed lists.

Checks if the specified VLANs are allowed according to the switch-specific or global configuration.

Parameters

- **vlans_to_check** List of VLAN IDs to validate.
- **switch_driver** The switch driver instance.

- **switch_id** Identifier of the switch.
- **operation_description** Description of the operation for error messages.

Raises
InvalidParameterValue if any VLAN is not allowed.

ironic.networking.utils module

Utilities for networking service operations.

ironic.networking.utils.**parse_vlan_ranges**(*vlan_spec*)

Parse VLAN specification into a set of VLAN IDs.

Parameters
vlan_spec List of VLAN IDs or ranges (e.g., [100, 102-104]) or a string representing a list of VLAN IDs (e.g., 100,102-104).

Returns
Set of integer VLAN IDs

Raises
InvalidParameterValue if the specification is invalid

ironic.networking.utils.**rpc_transport**()

Get the RPC transport type.

Module contents

ironic.objects package

Submodules

ironic.objects.allocation module

class ironic.objects.allocation.**Allocation**(*context=None*, ***kwargs*)

Bases: *IronicObject*, VersionedObjectDictCompat

VERSION = '1.3'

property candidate_nodes

classmethod check_node_list(*context*, *candidate_nodes*, *project*)

Provides a pass-through to the database for allocation node lists.

Calls the database directly as opposed to allowing the API to directly call the database when an indirection API is set.

property conductor_affinity

create(*context=None*)

Create a Allocation record in the DB.

Parameters
context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Allocation(context)

Raises
AllocationDuplicateName, AllocationAlreadyExists

property created_at

dbapi = <oslo_db.api.DBAPI object>

destroy(*context=None*)

Delete the Allocation from the DB.

Parameters
context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Allocation(context)

Raises
AllocationNotFound

property extra

fields = {'candidate_nodes': List(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'conductor_affinity': Integer(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'extra': FlexibleDict(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'id': Integer(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'last_error': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'name': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'node_id': Integer(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'owner': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'resource_class': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'state': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'traits': List(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid': UUID(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

classmethod get(*context*, *allocation_ident*)

Find an allocation by its ID, UUID or name.

Parameters

- **allocation_ident** The ID, UUID or name of an allocation.
- **context** Security context

Returns
An *Allocation* object.

Raises
InvalidIdentity

classmethod get_by_id(*context*, *allocation_id*)

Find an allocation by its integer ID.

Parameters

- **cls** the *Allocation*
- **context** Security context
- **allocation_id** The ID of an allocation.

Returns
An *Allocation* object.

Raises
AllocationNotFound

classmethod get_by_name(*context*, *name*)

Find an allocation based by its name.

Parameters

- **cls** the *Allocation*
- **context** Security context
- **name** The name of an allocation.

Returns
An *Allocation* object.

Raises
AllocationNotFound

classmethod get_by_uuid(*context*, *uuid*)

Find an allocation by its UUID.

Parameters

- **cls** the *Allocation*
- **context** Security context
- **uuid** The UUID of an allocation.

Returns
An *Allocation* object.

Raises
AllocationNotFound

property id

property last_error

classmethod list(*context*, *filters=None*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Return a list of Allocation objects.

Parameters

- **cls** the *Allocation*
- **context** Security context.
- **filters** Filters to apply.
- **limit** Maximum number of resources to return in a single result.
- **marker** Pagination marker for large data sets.
- **sort_key** Column to sort results by.
- **sort_dir** Direction to sort. asc or desc.

Returns
A list of *Allocation* object.

Raises
InvalidParameterValue

property name

property node_id

property owner

refresh(*context=None*)

Loads updates for this Allocation.

Loads an allocation with the same uuid from the database and checks for updated attributes. Updates are applied from the loaded allocation column by column, if there are any updates.

Parameters
context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Allocation(context)

Raises
AllocationNotFound

property resource_class

save(*context=None*)

Save updates to this Allocation.

Updates will be made column by column based on the result of self.what_changed().

Parameters
context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Allocation(context)

Raises
AllocationNotFound, AllocationDuplicateName

property state

property traits

property updated_at

property uuid

class ironic.objects.allocation.**AllocationCRUDNotification**(*context=None*, ***kwargs*)

Bases: *NotificationBase*

Notification when ironic creates, updates or deletes an allocation.

VERSION = '1.0'

property created_at

property event_type

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'event_type': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'level': NotificationLevel(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False,valid_values=('debug', 'info', 'warning', 'error', 'critical')), 'payload': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'publisher': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property level

property payload

property publisher

property updated_at

class ironic.objects.allocation.**AllocationCRUDPayload**(*allocation*, *node_uuid=None*)

Bases: *NotificationPayloadBase*

SCHEMA = {'candidate_nodes': ('allocation', 'candidate_nodes'), 'created_at': ('allocation', 'created_at'), 'extra': ('allocation', 'extra'), 'last_error': ('allocation', 'last_error'), 'name': ('allocation', 'name'), 'owner': ('allocation', 'owner'), 'resource_class': ('allocation', 'resource_class'), 'state': ('allocation', 'state'), 'traits': ('allocation', 'traits'), 'updated_at': ('allocation', 'updated_at'), 'uuid': ('allocation', 'uuid')}

```
VERSION = '1.1'

property candidate_nodes

property created_at

property extra

fields = {'candidate_nodes': List(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'extra': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'last_error': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'name':
String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'node_uuid': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'owner': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'resource_class': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'state': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'traits': List(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'updated_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid':
UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property last_error

property name

property node_uuid

property owner

property resource_class

property state

property traits

property updated_at

property uuid
```

ironic.objects.base module

Ironic common internal object model

class ironic.objects.base.**IronicObject**(*context=None*, ***kwargs*)

Bases: VersionedObject

Base class and object factory.

This forms the base of all objects that can be remoted or instantiated via RPC. Simply defining a class that inherits from this base class will make it remotely instantiatable. Objects should implement the necessary get classmethod routines as well as save object methods as appropriate.

OBJ_PROJECT_NAMESPACE = 'ironic'

OBJ_SERIAL_NAMESPACE = 'ironic_object'

as_dict()

Return the object represented as a dict.

The returned object is JSON-serialisable.

convert_to_version(*target_version*, *remove_unavailable_fields=True*)

Convert this object to the target version.

Convert the object to the target version. The target version may be the same, older, or newer than the version of the object. This is used for DB interactions as well as for serialization/deserialization.

The remove_unavailable_fields flag is used to distinguish these two cases:

1) For serialization/deserialization, we need to remove the unavailable fields, because the service receiving the object may not know about these fields. remove_unavailable_fields is set to True in this case.

2) For DB interactions, we need to set the unavailable fields to their appropriate values so that these fields are saved in the DB. (If they are not set, the VersionedObject magic will not know to save/update them to the DB.) remove_unavailable_fields is set to False in this case.

_convert_to_version() does the actual work.

Parameters

- **target_version** the desired version of the object
- **remove_unavailable_fields** True to remove fields that are unavailable in the target version; set this to True when (de)serializing. False to set the unavailable fields to appropriate values; set this to False for DB interactions.

do_version_changes_for_db()

Change the object to the version needed for the database.

If needed, this changes the object (modifies object fields) to be in the correct version for saving to the database.

The version used to save the object in the DB is determined as follows:

- If the object is pinned, we save the object in the pinned version. Since it is pinned, we must not save in a newer version, in case a rolling upgrade is happening and some services are still using the older version of ironic, with no knowledge of this newer version.
- If the object isnt pinned, we save the object in the latest version.

Because the object may be converted to a different object version, this method must only be called just before saving the object to the DB.

Returns
a dictionary of changed fields and their new values (could be an empty dictionary). These are the fields/values of the object that would be saved to the DB.

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

classmethod get_target_version()

Returns the target version for this object.

This is the version in which the object should be manipulated, e.g. sent over the wire via RPC or saved in the DB.

Returns
if pinned, returns the version of this object corresponding to the pin. Otherwise, returns the version of the object.

Raises
ovo_exception.IncompatibleObjectVersion

obj_refresh(*loaded_object*)

Applies updates for objects that inherit from base.IronicObject.

Checks for updated attributes in an object. Updates are applied from the loaded object column by column in comparison with the current object.

classmethod supports_version(*version*)

Return whether this object supports a particular version.

Check the requested version against the objects target version. The target version may not be the latest version during an upgrade, when object versions are pinned.

Parameters
version A tuple representing the version to check

Returns
Whether the version is supported

Raises
ovo_exception.IncompatibleObjectVersion

class ironic.objects.base.**IronicObjectListBase**(**args*, ***kwargs*)

Bases: ObjectListBase

as_dict()

Return the object represented as a dict.

The returned object is JSON-serialisable.

class ironic.objects.base.**IronicObjectRegistry**(**args*, ***kwargs*)

Bases: VersionedObjectRegistry

registration_hook(*cls*, *index*)

class ironic.objects.base.**IronicObjectSerializer**(*is_server=False*)

Bases: VersionedObjectSerializer

OBJ_BASE_CLASS

alias of *IronicObject*

serialize_entity(*context*, *entity*)

Serialize the entity.

This serializes the entity so that it can be sent over e.g. RPC. A serialized entity for an IronicObject is a dictionary with keys: ironic_object.namespace, ironic_object.data, ironic_object.name, ironic_object.version, and ironic_object.changes.

We assume that the client (ironic-API) is always talking to a server (ironic-conductor) that is running the same or a newer release than the client. The client doesnt need to downgrade any IronicObjects when sending them over RPC. The server, on the other hand, will need to do so if the server is pinned and the target version of an IronicObject is older than the latest version of that Object.

(Internally, the services deal with the latest versions of objects so we know that these objects are always in the latest versions.)

Parameters

- **context** security context
- **entity** the entity to be serialized; may be an IronicObject

Returns
the serialized entity

Raises
ovo_exception.IncompatibleObjectVersion (via .get_target_version())

ironic.objects.base.**max_version**(*versions*)

Return the maximum version in the list.

Parameters
versions a list of (string) versions; assumed to have at least one entry

Returns
the maximum version (string)

ironic.objects.bios module

class ironic.objects.bios.**BIOSSetting**(*context=None*, ***kwargs*)

Bases: *IronicObject*

VERSION = '1.2'

property allowable_values

property attribute_type

create(*context=None*)

Create a BIOS Setting record in DB.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: BIOSSetting(context)

Raises

NodeNotFound if the node id is not found.

Raises

BIOSSettingAlreadyExists if the setting record already exists.

property created_at

dbapi = <oslo_db.api.DBAPI object>

classmethod delete(*context*, *node_id*, *name*)

Delete a BIOS Setting based on its node_id and name.

Parameters

- **context** Security context.
- **node_id** The node id.
- **name** BIOS setting name to be deleted.

Raises

NodeNotFound if the node id is not found.

Raises

BIOSSettingNotFound if the bios setting name is not found.

```
fields = {'allowable_values': List(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'attribute_type': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'lower_bound': Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'max_length': Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'min_length': Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'name':
String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'node_id': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'read_only': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'reset_required': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'unique': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'updated_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'upper_bound': Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'value': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}
```

classmethod get(*context*, *node_id*, *name*)

Get a BIOS Setting based on its node_id and name.

Parameters

- **context** Security context.
- **node_id** The node id.
- **name** BIOS setting name to be retrieved.

Raises

NodeNotFound if the node id is not found.

Raises

BIOSSettingNotFound if the bios setting name is not found.

Returns

A :class:BIOSSetting object.

property lower_bound

property max_length

property min_length

property name

property node_id

property read_only

registry_fields = ('attribute_type', 'allowable_values', 'lower_bound', 'max_length', 'min_length', 'read_only', 'reset_required', 'unique', 'upper_bound')

property reset_required

save(*context=None*)

Save BIOS Setting update in DB.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: BIOSSetting(context)

Raises

NodeNotFound if the node id is not found.

Raises

BIOSSettingNotFound if the bios setting name is not found.

property unique

property updated_at

property upper_bound

property value

class ironic.objects.bios.**BIOSSettingList**(**args*, ***kwargs*)

Bases: *IronicObjectListBase*, *IronicObject*

VERSION = '1.1'

classmethod **create**(*context*, *node_id*, *settings*)

Create a list of BIOS Setting records in DB.

Parameters

- **context** Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: BIOSSetting(context)
- **node_id** The node id.
- **settings** A list of bios settings.

Raises

NodeNotFound if the node id is not found.

Raises

BIOSSettingAlreadyExists if any of the setting records already exists.

Returns
A list of BIOSSetting objects.

property created_at

dbapi = <oslo_db.api.DBAPI object>

classmethod delete(*context*, *node_id*, *names*)

Delete BIOS Settings based on node_id and names.

Parameters

- **context** Security context.
- **node_id** The node id.
- **names** List of BIOS setting names to be deleted.

Raises
NodeNotFound if the node id is not found.

Raises
BIOSSettingNotFound if any of BIOS setting fails to delete.

fields = {'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'objects': List(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'updated_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

classmethod get_by_node_id(*context*, *node_id*)

Get BIOS Setting based on node_id.

Parameters

- **context** Security context.
- **node_id** The node id.

Raises
NodeNotFound if the node id is not found.

Returns
A list of BIOSSetting objects.

property objects

classmethod save(*context*, *node_id*, *settings*)

Save a list of BIOS Setting updates in DB.

Parameters

- **context** Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: BIOSSetting(context)
- **node_id** The node id.
- **settings** A list of bios settings.

Raises
NodeNotFound if the node id is not found.

Raises
BIOSSettingNotFound if any of the bios setting names is not found.

Returns
A list of BIOSSetting objects.

classmethod sync_node_setting(*context*, *node_id*, *settings*)

Returns lists of create/update/delete/unchanged settings.

This method sync with bios_settings database table and sorts out four lists of create/update/delete/unchanged settings.

Parameters

- **context** Security context.
- **node_id** The node id.
- **settings** BIOS settings to be synced.

Returns
A 4-tuple of lists of BIOS settings to be created, updated, deleted and unchanged.

property updated_at

ironic.objects.chassis module

class ironic.objects.chassis.**Chassis**(*context=None*, ***kwargs*)

Bases: *IronicObject*, VersionedObjectDictCompat

VERSION = '1.4'

create(*context=None*)

Create a Chassis record in the DB.

Column-wise updates will be made based on the result of self.what_changed(). If target_power_state is provided, it will be checked against the in-database copy of the chassis before updates are made.

Parameters
context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Chassis(context)

property created_at

dbapi = <oslo_db.api.DBAPI object>

property description

destroy(*context=None*)

Delete the Chassis from the DB.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Chassis(context)

property extra

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'description': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'extra': FlexibleDict(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'id': Integer(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid': UUID(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

classmethod get(*context*, *chassis_id*)

Find a chassis based on its id or uuid and return a Chassis object.

Parameters

- **context** Security context
- **chassis_id** the id *or* uuid of a chassis.

Returns

a *Chassis* object.

classmethod get_by_id(*context*, *chassis_id*)

Find a chassis based on its integer ID and return a Chassis object.

Parameters

- **cls** the *Chassis*
- **context** Security context
- **chassis_id** the ID of a chassis.

Returns

a *Chassis* object.

classmethod get_by_uuid(*context*, *uuid*)

Find a chassis based on UUID and return a *Chassis* object.

Parameters

- **cls** the *Chassis*
- **context** Security context
- **uuid** the UUID of a chassis.

Returns

a *Chassis* object.

property id

classmethod list(*context*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Return a list of Chassis objects.

Parameters

- **cls** the *Chassis*
- **context** Security context.
- **limit** maximum number of resources to return in a single result.
- **marker** pagination marker for large data sets.
- **sort_key** column to sort results by.
- **sort_dir** direction to sort. asc or desc.

Returns

a list of *Chassis* object.

refresh(*context=None*)

Loads and applies updates for this Chassis.

Loads a *Chassis* with the same uuid from the database and checks for updated attributes. Updates are applied from the loaded chassis column by column, if there are any updates.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Chassis(context)

save(*context=None*)

Save updates to this Chassis.

Updates will be made column by column based on the result of self.what_changed().

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Chassis(context)

property updated_at

property uuid

class ironic.objects.chassis.**ChassisCRUDNotification**(*context=None*, ***kwargs*)

Bases: *NotificationBase*

Notification emitted when ironic creates, updates, deletes a chassis.

VERSION = '1.0'

property created_at

property event_type

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'event_type': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'level': NotificationLevel(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False,valid_values=('debug', 'info', 'warning', 'error', 'critical')), 'payload': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'publisher': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property level

property payload

property publisher

property updated_at

class ironic.objects.chassis.**ChassisCRUDPayload**(*chassis*, ***kwargs*)

Bases: *NotificationPayloadBase*

SCHEMA = {'created_at': ('chassis', 'created_at'), 'description': ('chassis', 'description'), 'extra': ('chassis', 'extra'), 'updated_at': ('chassis', 'updated_at'), 'uuid': ('chassis', 'uuid')}

VERSION = '1.0'

property created_at

property description

property extra

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'description': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'extra': FlexibleDict(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid': UUID(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False)}

property updated_at

property uuid

ironic.objects.conductor module

class ironic.objects.conductor.**Conductor**(*context=None*, ***kwargs*)

Bases: *IronicObject*, VersionedObjectDictCompat

VERSION = '1.6'

property conductor_group

property created_at

dbapi = <oslo_db.api.DBAPI object>

classmethod delete(*context*, *hostname*)

Delete a conductor from the database.

Parameters

- **cls** the *Conductor*
- **context** Security context
- **hostname** the hostname of the conductor to delete

Raises

ConductorNotFound if the conductor doesnt exist

property drivers

fields = {'conductor_group': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'drivers': List(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'hostname': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'id': Integer(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'online': Boolean(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

classmethod get_active_hardware_type_dict(*context*, *use_groups=False*)

Provides a hardware type list as it relates to the conductors.

This method provides a pass-through call mechanism on an attached object for insight into the state of hardware managers by conductors and does so as a direct call for compatibility with lightweight API method.

classmethod get_by_hostname(*context*, *hostname*, *online=True*)

Get a Conductor record by its hostname.

Parameters

- **cls** the *Conductor*

- **context** Security context
- **hostname** the hostname on which a Conductor is running
- **online** Specify the expected `online` field value for the conductor to be retrieved. The `online` field is ignored if this value is set to None.

Returns
a *Conductor* object.

classmethod get_shard_list(*context*)

Provides a shard list as it relates to conductors.

This method provides a pass-through all mechanism on an attached object for insight into the list of represented shards in a deployment which is sourced in the database combined with runtime configurations. The primary prupose of this being be lightweight and enable indirection_api call usage instead of trying to directly invoke the database.

property hostname

property id

classmethod list(*context*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Return a list of Conductor objects.

Parameters

- **cls** the *Conductor*
- **context** Security context.
- **limit** maximum number of resources to return in a single result.
- **marker** pagination marker for large data sets.
- **sort_key** column to sort results by.
- **sort_dir** direction to sort. asc or desc.

Returns
a list of *Conductor* object.

classmethod list_hardware_type_interfaces_dict(*context*, *names*)

Provides a list of hardware type interface names from conductors.

This method provides a pass-through call mechanism on an object as opposed to direct API call functionality.

property online

refresh(*context=None*)

Loads and applies updates for this Conductor.

Loads a *Conductor* with the same uuid from the database and checks for updated attributes. Updates are applied from the loaded chassis column by column, if there are any updates.

Parameters
context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Conductor(context)

classmethod register(*context*, *hostname*, *drivers*, *conductor_group*, *update_existing=False*)

Register an active conductor with the cluster.

Parameters

- **cls** the *Conductor*
- **context** Security context
- **hostname** the hostname on which the conductor will run
- **drivers** the list of drivers enabled in the conductor
- **conductor_group** conductor group to join, used for node:conductor affinity.
- **update_existing** When false, registration will raise an exception when a conflicting online record is found. When true, will overwrite the existing record. Default: False.

Raises

ConductorAlreadyRegistered

Returns

a *Conductor* object.

register_hardware_interfaces(*interfaces*)

Register hardware interfaces with the conductor.

Parameters

interfaces List of interface to register, each entry should be a dictionary containing hardware_type, interface_type, interface_name and default, e.g. {hardware_type: hardware-type, interface_type: deploy, interface_name: direct, default: True}

save(*context*)

Save is not supported by Conductor objects.

touch(*context=None*, *online=True*)

Touch this conductors DB record, marking it as up-to-date.

unregister(*context=None*)

Remove this conductor from the service registry.

unregister_all_hardware_interfaces()

Unregister all hardware interfaces for this conductor.

property updated_at

ironic.objects.deploy_template module

class ironic.objects.deploy_template.**DeployTemplate**(*context=None*, ***kwargs*)

Bases: *IronicObject*, VersionedObjectDictCompat

VERSION = '1.2'

create(*context=None*)

Create a DeployTemplate record in the DB.

Parameters

context security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: DeployTemplate(context).

Raises

DeployTemplateDuplicateName if a deploy template with the same name exists.

Raises

DeployTemplateAlreadyExists if a deploy template with the same UUID exists.

property created_at

dbapi = <oslo_db.api.DBAPI object>

destroy()

Delete the DeployTemplate from the DB.

Parameters

context security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: DeployTemplate(context).

Raises

DeployTemplateNotFound if the deploy template no longer appears in the database.

property extra

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'extra': FlexibleDict(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'id': Integer(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'name': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'steps': List(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid': UUID(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False)}

classmethod get_by_id(*context*, *template_id*)

Find a deploy template based on its integer ID.

Parameters

- **context** security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: DeployTemplate(context).
- **template_id** The ID of a deploy template.

Raises
DeployTemplateNotFound if the deploy template no longer appears in the database.

Returns
a *DeployTemplate* object.

classmethod get_by_name(*context*, *name*)

Find a deploy template based on its name.

Parameters

- **context** security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: DeployTemplate(context).
- **name** The name of a deploy template.

Raises
DeployTemplateNotFound if the deploy template no longer appears in the database.

Returns
a *DeployTemplate* object.

classmethod get_by_uuid(*context*, *uuid*)

Find a deploy template based on its UUID.

Parameters

- **context** security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: DeployTemplate(context).
- **uuid** The UUID of a deploy template.

Raises
DeployTemplateNotFound if the deploy template no longer appears in the database.

Returns
a *DeployTemplate* object.

property id

classmethod list(*context*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Return a list of DeployTemplate objects.

Parameters

- **context** security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: DeployTemplate(context).
- **limit** maximum number of resources to return in a single result.
- **marker** pagination marker for large data sets.
- **sort_key** column to sort results by.
- **sort_dir** direction to sort. asc or desc.

Returns
a list of *DeployTemplate* objects.

classmethod list_by_names(*context*, *names*)

Return a list of DeployTemplate objects matching a set of names.

Parameters

- **context** security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: DeployTemplate(context).
- **names** a list of names to filter by.

Returns
a list of *DeployTemplate* objects.

property name

refresh(*context=None*)

Loads updates for this deploy template.

Loads a deploy template with the same uuid from the database and checks for updated attributes. Updates are applied from the loaded template column by column, if there are any updates.

Parameters
context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Port(context)

Raises
DeployTemplateNotFound if the deploy template no longer appears in the database.

save(*context=None*)

Save updates to this DeployTemplate.

Column-wise updates will be made based on the result of self.what_changed().

Parameters
context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: DeployTemplate(context)

Raises
DeployTemplateDuplicateName if a deploy template with the same name exists.

Raises
DeployTemplateNotFound if the deploy template does not exist.

property steps

property updated_at

property uuid

class ironic.objects.deploy_template.**DeployTemplateCRUDNotification**(*context=None*, ***kwargs*)

Bases: *NotificationBase*

Notification emitted on deploy template API operations.

VERSION = '1.0'

property created_at

property event_type

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'event_type': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'level': NotificationLevel(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False,valid_values=('debug', 'info', 'warning', 'error', 'critical')), 'payload': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'publisher': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property level

property payload

property publisher

property updated_at

class ironic.objects.deploy_template.**DeployTemplateCRUDPayload**(*deploy_template*, ***kwargs*)

Bases: *NotificationPayloadBase*

SCHEMA = {'created_at': ('deploy_template', 'created_at'), 'extra': ('deploy_template', 'extra'), 'name': ('deploy_template', 'name'), 'steps': ('deploy_template', 'steps'), 'updated_at': ('deploy_template', 'updated_at'), 'uuid': ('deploy_template', 'uuid')}

VERSION = '1.0'

property created_at

property extra

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'extra': FlexibleDict(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'name': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'steps': List(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid': UUID(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False)}

property name

property steps

property updated_at

property uuid

ironic.objects.deployment module

class ironic.objects.deployment.**Deployment**(*context=None*, ***kwargs*)

Bases: *IronicObject*, VersionedObjectDictCompat

VERSION = '1.1'

create(*context=None*, *node=None*)

Create a Deployment.

Updates the corresponding node under the hood.

Parameters

- **context** Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Deployment(context)
- **node** Node object for deployment.

Raises

InstanceAssociated, NodeAssociated, NodeNotFound

property created_at

dbapi = <oslo_db.api.DBAPI object>

destroy(*context=None*, *node=None*)

Delete the Deployment.

Updates the corresponding node under the hood.

Parameters

- **context** Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Node(context)
- **node** Node object for deployment.

```
fields = {'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'image_checksum': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'image_ref': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'kernel_ref': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'node_uuid': UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'ramdisk_ref': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'root_device': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'root_gib': Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'state': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'swap_mib': Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'updated_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid':
UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}
```

classmethod get_by_node_uuid(*context*, *node_uuid*)

Find a deployment based by its nodes UUID.

Parameters

- **cls** the *Deployment*
- **context** Security context
- **node_uuid** The UUID of a corresponding node.

Returns

An *Deployment* object.

Raises

NodeNotFound

classmethod get_by_uuid(*context*, *uuid*)

Find a deployment by its UUID.

Parameters

- **cls** the *Deployment*

- **context** Security context
- **uuid** The UUID of a deployment.

Returns
An *Deployment* object.

Raises
InstanceNotFound

property image_checksum

property image_ref

instance_info_mapping = {'image_checksum': 'image_checksum', 'image_source': 'image_ref', 'kernel': 'kernel_ref', 'ramdisk': 'ramdisk_ref', 'root_device': 'root_device', 'root_gb': 'root_gib', 'swap_mb': 'swap_mib'}

instance_info_mapping_rev = {'image_checksum': 'image_checksum', 'image_ref': 'image_source', 'kernel_ref': 'kernel', 'ramdisk_ref': 'ramdisk', 'root_device': 'root_device', 'root_gib': 'root_gb', 'swap_mib': 'swap_mb'}

property kernel_ref

classmethod list(*context*, *filters=None*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Return a list of Deployment objects.

Parameters

- **cls** the *Deployment*
- **context** Security context.
- **filters** Filters to apply.
- **limit** Maximum number of resources to return in a single result.
- **marker** Pagination marker for large data sets.
- **sort_key** Column to sort results by.
- **sort_dir** Direction to sort. asc or desc.

Returns
A list of *Deployment* object.

Raises
InvalidParameterValue

node_mapping = {'instance_uuid': 'uuid', 'provision_state': 'state', 'uuid': 'node_uuid'}

property node_uuid

property ramdisk_ref

refresh(*context=None*)

Refresh the object by re-fetching from the DB.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Node(context)

property root_device

property root_gib

property state

property swap_mib

property updated_at

property uuid

ironic.objects.fields module

class ironic.objects.fields.**BooleanField**(***kwargs*)

Bases: BooleanField

class ironic.objects.fields.**DateTimeField**(*tzinfo_aware=True*, ***kwargs*)

Bases: DateTimeField

class ironic.objects.fields.**EnumField**(*valid_values*, ***kwargs*)

Bases: EnumField

class ironic.objects.fields.**FlexibleDict**

Bases: FieldType

static coerce(*obj*, *attr*, *value*)

This is called to coerce (if possible) a value on assignment.

This method should convert the value given into the designated type, or throw an exception if this is not possible.

Param:obj
The VersionedObject on which an attribute is being set

Param:attr
The name of the attribute being set

Param:value
The value being set

Returns
A properly-typed value

class ironic.objects.fields.**FlexibleDictField**(***kwargs*)

Bases: AutoTypedField

AUTO_TYPE = <ironic.objects.fields.FlexibleDict object>

class ironic.objects.fields.**IntegerField**(***kwargs*)

Bases: IntegerField

class ironic.objects.fields.**ListOfFlexibleDictsField**(***kwargs*)

Bases: AutoTypedField

AUTO_TYPE = <oslo_versionedobjects.fields.List object>

class ironic.objects.fields.**ListOfObjectsField**(*objtype*, *subclasses=False*, ***kwargs*)

Bases: ListOfObjectsField

class ironic.objects.fields.**ListOfStringsField**(***kwargs*)

Bases: ListOfStringsField

class ironic.objects.fields.**MACAddress**

Bases: FieldType

static coerce(*obj*, *attr*, *value*)

This is called to coerce (if possible) a value on assignment.

This method should convert the value given into the designated type, or throw an exception if this is not possible.

Param:obj
The VersionedObject on which an attribute is being set

Param:attr
The name of the attribute being set

Param:value
The value being set

Returns
A properly-typed value

class ironic.objects.fields.**MACAddressField**(***kwargs*)

Bases: AutoTypedField

AUTO_TYPE = <ironic.objects.fields.MACAddress object>

class ironic.objects.fields.**NotificationLevel**

Bases: Enum

ALL = ('debug', 'info', 'warning', 'error', 'critical')

CRITICAL = 'critical'

DEBUG = 'debug'

ERROR = 'error'

INFO = 'info'

WARNING = 'warning'

class ironic.objects.fields.**NotificationLevelField**(***kwargs*)

Bases: BaseEnumField

AUTO_TYPE = <ironic.objects.fields.NotificationLevel object>

class ironic.objects.fields.NotificationStatus

Bases: Enum

ALL = ('start', 'end', 'error', 'success')

END = 'end'

ERROR = 'error'

START = 'start'

SUCCESS = 'success'

class ironic.objects.fields.NotificationStatusField(**kwargs)

Bases: BaseEnumField

AUTO_TYPE = <ironic.objects.fields.NotificationStatus object>

class ironic.objects.fields.ObjectField(*objtype*, *subclasses=False*, ***kwargs*)

Bases: ObjectField

class ironic.objects.fields.StringAcceptsCallable

Bases: String

static coerce(*obj*, *attr*, *value*)

This is called to coerce (if possible) a value on assignment.

This method should convert the value given into the designated type, or throw an exception if this is not possible.

Param:obj
The VersionedObject on which an attribute is being set

Param:attr
The name of the attribute being set

Param:value
The value being set

Returns
A properly-typed value

class ironic.objects.fields.StringField(**kwargs)

Bases: StringField

class ironic.objects.fields.StringFieldThatAcceptsCallable(**kwargs)

Bases: StringField

Custom StringField object that allows for functions as default

In some cases we need to allow for dynamic defaults based on configuration options, this StringField object allows for a function to be passed as a default, and will only process it at the point the field is coerced

AUTO_TYPE = <ironic.objects.fields.StringAcceptsCallable object>

class ironic.objects.fields.UUIDField(**kwargs)

Bases: UUIDField

ironic.objects.firmware module

class ironic.objects.firmware.**FirmwareComponent**(*context=None*, ***kwargs*)

Bases: *IronicObject*

VERSION = '1.1'

property component

create(*context=None*)

Create a Firmware record in the DB.

Parameters
context Security context.

Raises
NodeNotFound if the node is not found.

Raises
FirmwareComponentAlreadyExists if the record already exists.

property created_at

property current_version

dbapi = <oslo_db.api.DBAPI object>

fields = {'component': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'current_version': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'id':
Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'initial_version': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'last_version_flashed': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'node_id': Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'updated_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

classmethod get(*context*, *node_id*, *name*)

Get a FirmwareComponent based on its node_id and name.

Parameters

- **context** Security context.
- **node_id** The node id.
- **name** The Firmware Component name.

Raises
NodeNotFound if the node id is not found.

Raises
FirmwareComponentNotFound if the Firmware Component name is not found.

Returns
A :class:FirmwareComponent object.

property id

property initial_version

property last_version_flashed

property node_id

save(*context=None*)

Save updates to this Firmware Component.

Updates will be made column by column based on the result of self.what_changed()

Parameters
context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: FirmwareComponent(context)

Raises
NodeNotFound if the node id is not found.

Raises
FirmwareComponentNotFound if the component is not found.

property updated_at

class ironic.objects.firmware.**FirmwareComponentList**(**args*, ***kwargs*)

Bases: *IronicObjectListBase*, *IronicObject*

VERSION = '1.1'

property created_at

dbapi = <oslo_db.api.DBAPI object>

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'objects': List(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

classmethod get_by_node_id(*context*, *node_id*)

Get FirmwareComponent based on node_id.

Parameters

- **context** Security context.
- **node_id** The node id.

Raises
NodeNotFound if the node is not found.

Returns
A list of FirmwareComponent objects.

property objects

classmethod sync_firmware_components(*context*, *node_id*, *components*)

Returns a list of create/update components.

This method sync with the firmware_information database table and sorts three lists - create / update / unchanged components.

Parameters

- **context** Security context.
- **node_id** The node id.
- **components** List of FirmwareComponents.

Returns
A 3-tuple of lists of Firmware Components to be created, updated and unchanged.

property updated_at

ironic.objects.indirection module

class ironic.objects.indirection.IronicObjectIndirectionAPI

Bases: VersionedObjectIndirectionAPI

object_action(*context*, *objinst*, *objmethod*, *args*, *kwargs*)

Perform an action on a VersionedObject instance.

When indirection_api is set on a VersionedObject (to a class implementing this interface), method calls on remotable methods will cause this to be executed to actually make the desired call. This often involves performing RPC.

Parameters

- **context** The context within which to perform the action
- **objinst** The object instance on which to perform the action
- **objmethod** The name of the action method to call
- **args** The positional arguments to the action method
- **kwargs** The keyword arguments to the action method

Returns
The result of the action method

object_backport_versions(*context*, *objinst*, *object_versions*)

Perform a backport of an object instance.

This method is basically just like object_backport() but instead of providing a specific target version for the toplevel object and relying on the service-side mapping to handle sub-objects, this sends a mapping of all the dependent objects and their client-supported versions. The

server will backport objects within the tree starting at objinst to the versions specified in object_versions, removing objects that have no entry. Use obj_tree_get_versions() to generate this mapping.

NOTE: This was not in the initial spec for this interface, so the base class raises NotImplementedError if you dont implement it. For backports, this method will be tried first, and if unimplemented, will fall back to object_backport().

Parameters

- **context** The context within which to perform the backport
- **objinst** An instance of a VersionedObject to be backported
- **object_versions** A dict of {objname: version} mappings

object_class_action_versions(*context*, *objname*, *objmethod*, *object_versions*, *args*, *kwargs*)

Perform an action on a VersionedObject class.

When indirection_api is set on a VersionedObject (to a class implementing this interface), classmethod calls on remotable_classmethod methods will cause this to be executed to actually make the desired call. This usually involves performing RPC.

This differs from object_class_action() in that it is provided with object_versions, a manifest of client-side object versions for easier nested backports. The manifest is the result of calling obj_tree_get_versions().

NOTE: This was not in the initial spec for this interface, so the base class raises NotImplementedError if you dont implement it. For backports, this method will be tried first, and if unimplemented, will fall back to object_class_action(). New implementations should provide this method instead of object_class_action()

Parameters

- **context** The context within which to perform the action
- **objname** The registry name of the object
- **objmethod** The name of the action method to call
- **object_versions** A dict of {objname: version} mappings
- **args** The positional arguments to the action method
- **kwargs** The keyword arguments to the action method

Returns

The result of the action method, which may (or may not) be an instance of the implementing VersionedObject class.

ironic.objects.inspection_rule module

class ironic.objects.inspection_rule.**InspectionRule**(*context=None*, ***kwargs*)

Bases: *IronicObject*, VersionedObjectDictCompat

VERSION = '1.1'

property actions

property conditions

create(*context=None*)

Create a InspectionRule record in the DB.

Parameters

context security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: InspectionRule(context).

Raises

InspectionRuleName if a inspection rule with the same name exists.

Raises

InspectionRuleAlreadyExists if a inspection rule with the same UUID exists.

property created_at

dbapi = <oslo_db.api.DBAPI object>

property description

destroy()

Delete the InspectionRule from the DB.

Parameters

context security context. NOTE: This should only be used internally by the indirection_api, but, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: InspectionRule(context).

Raises

InspectionRuleNotFound if the inspection_rule no longer appears in the database.

fields = {'actions': List(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'conditions': List(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'description': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'id':
Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'phase': String(default=main,nullable=True), 'priority':
Integer(default=0,nullable=False), 'scope': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'sensitive': Boolean(default=False,nullable=False), 'updated_at':
DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid':
UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False)}

classmethod get_by_uuid(*context*, *uuid*)

Find a inspection rule based on its UUID.

Parameters

- **context** security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: InspectionRule(context).
- **uuid** The UUID of a inspection rule.

Raises

InspectionRuleNotFound if the inspection rule no longer appears in the database.

Returns

a *InspectionRule* object.

property id

classmethod list(*context*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *filters=None*)

Return a list of InspectionRule objects.

Parameters

- **context** security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: InspectionRule(context).
- **limit** maximum number of resources to return in a single result.
- **marker** pagination marker for large data sets.
- **sort_key** column to sort results by.
- **sort_dir** direction to sort. asc or desc.

Returns

a list of *InspectionRule* objects.

property phase

property priority

refresh(*context=None*)

Loads updates for this inspection rule.

Loads a inspection rule with the same uuid from the database and checks for updated attributes. Updates are applied from the loaded rule column by column, if there are any updates.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Port(context)

Raises

InspectionRuleNotFound if the inspection rule no longer appears in the database.

save(*context=None*)

Save updates to this InspectionRule.

Column-wise updates will be made based on the result of self.what_changed().

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api, but, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: InspectionRule(context)

Raises

InspectionRuleNotFound if the inspection rule does not exist.

property scope

property sensitive

property updated_at

property uuid

class ironic.objects.inspection_rule.InspectionRuleCRUDNotification(*context=None*, ***kwargs*)

Bases: *NotificationBase*

Notification emitted on inspection rule API operations.

VERSION = '1.0'

property created_at

property event_type

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'event_type': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'level': NotificationLevel(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False,valid_values=('debug', 'info', 'warning', 'error', 'critical')), 'payload': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'publisher': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property level

property payload

property publisher

property updated_at

class ironic.objects.inspection_rule.**InspectionRuleCRUDPayload**(*inspection_rule*, ***kwargs*)

Bases: *NotificationPayloadBase*

SCHEMA = {'actions': ('inspection_rule', 'actions'), 'conditions': ('inspection_rule', 'conditions'), 'created_at': ('inspection_rule', 'created_at'), 'description': ('inspection_rule', 'description'), 'phase': ('inspection_rule', 'phase'), 'priority': ('inspection_rule', 'priority'), 'scope': ('inspection_rule', 'scope'), 'sensitive': ('inspection_rule', 'sensitive'), 'updated_at': ('inspection_rule', 'updated_at'), 'uuid': ('inspection_rule', 'uuid')}

VERSION = '1.0'

property actions

property conditions

property created_at

property description

fields = {'actions': List(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'conditions': List(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'description': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'phase': String(default=main,nullable=True), 'priority': Integer(default=0,nullable=False), 'scope': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'sensitive': Boolean(default=False,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid': UUID(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False)}

property phase

property priority

property scope

property sensitive

property updated_at

property uuid

ironic.objects.node module

class ironic.objects.node.**Node**(*context=None*, ***kwargs*)

Bases: *IronicObject*, VersionedObjectDictCompat

VERSION = '1.44'

property allocation_id

as_dict(*secure=False*, *mask_configdrive=True*)

Return the object represented as a dict.

The returned object is JSON-serialisable.

property automated_clean

property bios_interface

property boot_interface

property boot_mode

property chassis_id

property clean_step

property conductor_affinity

property conductor_group

property console_enabled

property console_interface

create(*context=None*)

Create a Node record in the DB.

Column-wise updates will be made based on the result of self.what_changed(). If target_power_state is provided, it will be checked against the in-database copy of the node before updates are made.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Node(context)

Raises

InvalidParameterValue if some property values are invalid.

property created_at

dbapi = <oslo_db.api.DBAPI object>

del_driver_internal_info(*key*, *default_value=None*)

Pop a value from the driver_internal_info.

Removing a *driver_internal_info* dict value via this method ensures that this field will be flagged for saving.

Parameters

- **key** Key of item to pop off the *driver_internal_info* dict
- **default_value** Value to return if the key doesnt exist

Returns

The removed value, or *default_value*

property deploy_interface

property deploy_step

property description

destroy(*context=None*)

Delete the Node from the DB.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Node(context)

property disable_power_off

property driver

property driver_info

property driver_internal_info

property extra

property fault

```
fields = {'allocation_id': Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'automated_clean': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'bios_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'boot_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'boot_mode': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'chassis_id': Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'clean_step': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'conductor_affinity': Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'conductor_group': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'console_enabled': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'console_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'deploy_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'deploy_step': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'description': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'disable_power_off': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'driver': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'driver_info': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'driver_internal_info': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'extra': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'fault': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'firmware_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'health': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'id':
Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'inspect_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'inspection_finished_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'inspection_started_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'instance_info': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'instance_name': String(default=<class
```

property firmware_interface

classmethod get(*context*, *node_id*)

Find a node based on its id or uuid and return a Node object.

Parameters

- **context** Security context
- **node_id** the id *or* uuid of a node.

Returns

a *Node* object.

classmethod get_by_id(*context*, *node_id*)

Find a node based on its integer ID and return a Node object.

Parameters

- **cls** the *Node*
- **context** Security context
- **node_id** the ID of a node.

Returns

a *Node* object.

classmethod get_by_instance_uuid(*context*, *instance_uuid*)

Find a node based on the instance UUID and return a Node object.

Parameters

- **cls** the *Node*
- **context** Security context
- **uuid** the UUID of the instance.

Returns

a *Node* object.

classmethod get_by_name(*context*, *name*)

Find a node based on name and return a Node object.

Parameters

- **cls** the *Node*
- **context** Security context
- **name** the logical name of a node.

Returns

a *Node* object.

classmethod get_by_port_addresses(*context*, *addresses*)

Get a node by associated port addresses.

Parameters

- **cls** the *Node*

- **context** Security context.
- **addresses** A list of port addresses.

Raises
NodeNotFound if the node is not found.

Returns
a *Node* object.

classmethod get_by_uuid(*context*, *uuid*)

Find a node based on UUID and return a Node object.

Parameters

- **cls** the *Node*
- **context** Security context
- **uuid** the UUID of a node.

Returns
a *Node* object.

get_interface(*iface*)

property health

property id

property inspect_interface

property inspection_finished_at

property inspection_started_at

property instance_info

property instance_name

property instance_uuid

property last_error

property lessee

classmethod list(*context*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *filters=None*, *fields=None*)

Return a list of Node objects.

Parameters

- **cls** the *Node*
- **context** Security context.
- **limit** maximum number of resources to return in a single result.
- **marker** pagination marker for large data sets.
- **sort_key** column to sort results by.
- **sort_dir** direction to sort. asc or desc.

- **filters** Filters to apply.
- **fields** Requested fields to be returned. Please note, some fields are mandatory for the data model and are automatically included. These are: id, version, updated_at, created_at, owner, and lessee.

Returns
a list of *Node* object.

list_child_node_ids(*exclude_dedicated_power=False*)

Returns a list of node IDs for child nodes, if any.

Parameters
exclude_dedicated_power Boolean, Default False, if the list should exclude nodes which are independently powered.

Returns
A list of any node_id values discovered in the database.

property maintenance

property maintenance_reason

property management_interface

property name

property network_data

property network_interface

property owner

property parent_node

property power_interface

property power_state

property properties

property protected

property protected_reason

property provision_state

property provision_updated_at

property raid_config

property raid_interface

refresh(*context=None*)

Refresh the object by re-fetching from the DB.

Parameters
context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even

though we dont use it. A context should be set when instantiating the object, e.g.: Node(context)

classmethod release(*context*, *tag*, *node_id*)

Release the reservation on a node.

Parameters

- **context** Security context.
- **tag** A string uniquely identifying the reservation holder.
- **node_id** A node id or uuid.

Raises

NodeNotFound if the node is not found.

property rescue_interface

property reservation

classmethod reserve(*context*, *tag*, *node_id*)

Get and reserve a node.

To prevent other ManagerServices from manipulating the given Node while a Task is performed, mark it reserved by this host.

Parameters

- **cls** the *Node*
- **context** Security context.
- **tag** A string uniquely identifying the reservation holder.
- **node_id** A node ID or UUID.

Raises

NodeNotFound if the node is not found.

Returns

a *Node* object.

property resource_class

property retired

property retired_reason

save(*context=None*)

Save updates to this Node.

Column-wise updates will be made based on the result of self.what_changed(). If target_power_state is provided, it will be checked against the in-database copy of the node before updates are made.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Node(context)

Raises
InvalidParameterValue if some property values are invalid.

property secure_boot

property service_step

set_driver_internal_info(*key*, *value*)
Set a *driver_internal_info* value.

Setting a *driver_internal_info* dict value via this method ensures that this field will be flagged for saving.

Parameters

- **key** Key of item to set
- **value** Value of item to set

set_instance_info(*key*, *value*)
Set an *instance_info* value.

Setting a *instance_info* dict value via this method ensures that this field will be flagged for saving.

Parameters

- **key** Key of item to set
- **value** Value of item to set

set_property(*key*, *value*)
Set a *properties* value.

Setting a *properties* dict value via this method ensures that this field will be flagged for saving.

Parameters

- **key** Key of item to set
- **value** Value of item to set

property shard

property storage_interface

property target_power_state

property target_provision_state

property target_raid_config

timestamp_driver_internal_info(*key*)
Set a *driver_internal_info* value with the current timestamp.

Setting a *driver_internal_info* timestamp value via this method ensures that this field will be flagged for saving.

Parameters
key Key of item to set the timestamp on

touch_provisioning(*context=None*)

Touch the database record to mark the provisioning as alive.

property traits

property updated_at

property uuid

property vendor_interface

class ironic.objects.node.**NodeCRUDNotification**(*context=None*, ***kwargs*)

Bases: *NotificationBase*

Notification emitted when ironic creates, updates or deletes a node.

VERSION = '1.0'

property created_at

property event_type

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'event_type': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'level': NotificationLevel(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False,valid_values=('debug', 'info', 'warning', 'error', 'critical')), 'payload': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'publisher': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property level

property payload

property publisher

property updated_at

class ironic.objects.node.**NodeCRUDPayload**(*node*, *chassis_uuid=None*)

Bases: *NodePayload*

Payload schema for when ironic creates, updates or deletes a node.

```
SCHEMA = {'bios_interface': ('node', 'bios_interface'), 'boot_interface':
('node', 'boot_interface'), 'boot_mode': ('node', 'boot_mode'),
'clean_step': ('node', 'clean_step'), 'conductor_group': ('node',
'conductor_group'), 'console_enabled': ('node', 'console_enabled'),
'console_interface': ('node', 'console_interface'), 'created_at':
('node', 'created_at'), 'deploy_interface': ('node', 'deploy_interface'),
'deploy_step': ('node', 'deploy_step'), 'description': ('node',
'description'), 'disable_power_off': ('node', 'disable_power_off'),
'driver': ('node', 'driver'), 'driver_info': ('node', 'driver_info'),
'extra': ('node', 'extra'), 'fault': ('node', 'fault'),
'inspect_interface': ('node', 'inspect_interface'),
'inspection_finished_at': ('node', 'inspection_finished_at'),
'inspection_started_at': ('node', 'inspection_started_at'),
'instance_info': ('node', 'instance_info'), 'instance_uuid': ('node',
'instance_uuid'), 'last_error': ('node', 'last_error'), 'lessee':
('node', 'lessee'), 'maintenance': ('node', 'maintenance'),
'maintenance_reason': ('node', 'maintenance_reason'),
'management_interface': ('node', 'management_interface'), 'name':
('node', 'name'), 'network_interface': ('node', 'network_interface'),
'owner': ('node', 'owner'), 'power_interface': ('node',
'power_interface'), 'power_state': ('node', 'power_state'), 'properties':
('node', 'properties'), 'protected': ('node', 'protected'),
'protected_reason': ('node', 'protected_reason'), 'provision_state':
('node', 'provision_state'), 'provision_updated_at': ('node',
'provision_updated_at'), 'raid_interface': ('node', 'raid_interface'),
'rescue_interface': ('node', 'rescue_interface'), 'resource_class':
('node', 'resource_class'), 'retired': ('node', 'retired'),
'retired_reason': ('node', 'retired_reason'), 'secure_boot': ('node',
'secure_boot'), 'storage_interface': ('node', 'storage_interface'),
'target_power_state': ('node', 'target_power_state'),
'target_provision_state': ('node', 'target_provision_state'),
'updated_at': ('node', 'updated_at'), 'uuid': ('node', 'uuid'),
'vendor_interface': ('node', 'vendor_interface')}

VERSION = '1.15'

property bios_interface

property boot_interface

property boot_mode

property chassis_uuid

property clean_step

property conductor_group

property console_enabled

property console_interface

property created_at
```

property deploy_interface

property deploy_step

property description

property disable_power_off

property driver

property driver_info

property extra

property fault

```
fields = {'bios_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'boot_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'boot_mode': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'chassis_uuid': UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'clean_step': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'conductor_group': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'console_enabled': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'console_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'deploy_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'deploy_step': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'description': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'disable_power_off': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'driver': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'driver_info': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'extra': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'fault': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'inspect_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'inspection_finished_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'inspection_started_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'instance_info': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'instance_uuid': UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'last_error': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'lessee': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'maintenance': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'maintenance_reason': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'management_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'name':
String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'network_interface': String(default=<class
```

property inspect_interface

property inspection_finished_at

property inspection_started_at

property instance_info

property instance_uuid

property last_error

property lessee

property maintenance

property maintenance_reason

property management_interface

property name

property network_interface

property owner

property power_interface

property power_state

property properties

property protected

property protected_reason

property provision_state

property provision_updated_at

property raid_interface

property rescue_interface

property resource_class

property retired

property retired_reason

property secure_boot

property storage_interface

property target_power_state

property target_provision_state

property traits

property updated_at

property uuid

property vendor_interface

class ironic.objects.node.**NodeConsoleNotification**(*context=None*, ***kwargs*)

Bases: *NotificationBase*

Notification emitted when node console state changed.

VERSION = '1.0'

property created_at

property event_type

fields = {'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'event_type': Object(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'level': NotificationLevel(default=<class 'oslo_versionedobjects.fields.
UnspecifiedDefault'>,nullable=False,valid_values=('debug', 'info',
'warning', 'error', 'critical')), 'payload': Object(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'publisher': Object(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'updated_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property level

property payload

property publisher

property updated_at

class ironic.objects.node.**NodeCorrectedPowerStateNotification**(*context=None*, ***kwargs*)

Bases: *NotificationBase*

Notification for when a nodes power state is corrected in the database.

This notification is emitted when ironic detects that the actual power state on a bare metal hardware is different from the power state on an ironic node (DB). This notification is emitted after the database is updated to reflect this correction.

VERSION = '1.0'

property created_at

property event_type

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'event_type': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'level': NotificationLevel(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False,valid_values=('debug', 'info', 'warning', 'error', 'critical')), 'payload': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'publisher': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property level

property payload

property publisher

property updated_at

class ironic.objects.node.**NodeCorrectedPowerStatePayload**(*node*, *from_power*)

Bases: *NodePayload*

Notification payload schema for when a nodes power state is corrected.

from_power indicates the previous power state on the ironic node before the node was updated.

VERSION = '1.17'

property bios_interface

property boot_interface

property boot_mode

property clean_step

property conductor_group

property console_enabled

property console_interface

property created_at

property deploy_interface

property deploy_step

property description

property disable_power_off

property driver

property extra

```
property fault
```

```
fields = {'bios_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'boot_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'boot_mode': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'clean_step': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'conductor_group': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'console_enabled': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'console_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'deploy_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'deploy_step': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'description': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'disable_power_off': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'driver': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'extra': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'fault': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'from_power': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'inspect_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'inspection_finished_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'inspection_started_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'instance_uuid': UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'last_error': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'lessee': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'maintenance': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'maintenance_reason': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'management_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'name':
String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'network_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'owner': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'power_interface': String(default=<class
```

property from_power

property inspect_interface

property inspection_finished_at

property inspection_started_at

property instance_uuid

property last_error

property lessee

property maintenance

property maintenance_reason

property management_interface

property name

property network_interface

property owner

property power_interface

property power_state

property properties

property protected

property protected_reason

property provision_state

property provision_updated_at

property raid_interface

property rescue_interface

property resource_class

property retired

property retired_reason

property secure_boot

property storage_interface

property target_power_state

property target_provision_state

property traits

property updated_at

property uuid

property vendor_interface

class ironic.objects.node.**NodeMaintenanceNotification**(*context=None*, ***kwargs*)

Bases: *NotificationBase*

Notification emitted when maintenance state changed via API.

VERSION = '1.0'

property created_at

property event_type

fields = {'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'event_type': Object(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'level': NotificationLevel(default=<class 'oslo_versionedobjects.fields.
UnspecifiedDefault'>,nullable=False,valid_values=('debug', 'info',
'warning', 'error', 'critical')), 'payload': Object(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'publisher': Object(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'updated_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property level

property payload

property publisher

property updated_at

class ironic.objects.node.**NodePayload**(*node*, ***kwargs*)

Bases: *NotificationPayloadBase*

Base class used for all notification payloads about a Node object.

SCHEMA = {'bios_interface': ('node', 'bios_interface'), 'boot_interface': ('node', 'boot_interface'), 'boot_mode': ('node', 'boot_mode'), 'clean_step': ('node', 'clean_step'), 'conductor_group': ('node', 'conductor_group'), 'console_enabled': ('node', 'console_enabled'), 'console_interface': ('node', 'console_interface'), 'created_at': ('node', 'created_at'), 'deploy_interface': ('node', 'deploy_interface'), 'deploy_step': ('node', 'deploy_step'), 'description': ('node', 'description'), 'disable_power_off': ('node', 'disable_power_off'), 'driver': ('node', 'driver'), 'extra': ('node', 'extra'), 'fault': ('node', 'fault'), 'inspect_interface': ('node', 'inspect_interface'), 'inspection_finished_at': ('node', 'inspection_finished_at'), 'inspection_started_at': ('node', 'inspection_started_at'), 'instance_uuid': ('node', 'instance_uuid'), 'last_error': ('node', 'last_error'), 'lessee': ('node', 'lessee'), 'maintenance': ('node', 'maintenance'), 'maintenance_reason': ('node', 'maintenance_reason'), 'management_interface': ('node', 'management_interface'), 'name': ('node', 'name'), 'network_interface': ('node', 'network_interface'), 'owner': ('node', 'owner'), 'power_interface': ('node', 'power_interface'), 'power_state': ('node', 'power_state'), 'properties': ('node', 'properties'), 'protected': ('node', 'protected'), 'protected_reason': ('node', 'protected_reason'), 'provision_state': ('node', 'provision_state'), 'provision_updated_at': ('node', 'provision_updated_at'), 'raid_interface': ('node', 'raid_interface'), 'rescue_interface': ('node', 'rescue_interface'), 'resource_class': ('node', 'resource_class'), 'retired': ('node', 'retired'), 'retired_reason': ('node', 'retired_reason'), 'secure_boot': ('node', 'secure_boot'), 'storage_interface': ('node', 'storage_interface'), 'target_power_state': ('node', 'target_power_state'), 'target_provision_state': ('node', 'target_provision_state'), 'updated_at': ('node', 'updated_at'), 'uuid': ('node', 'uuid'), 'vendor_interface': ('node', 'vendor_interface')}

VERSION = '1.17'

property bios_interface

property boot_interface

property boot_mode

property clean_step

property conductor_group

property console_enabled

property console_interface

property created_at

property deploy_interface

property deploy_step

property description

property disable_power_off

property driver

property extra

property fault

```
fields = {'bios_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'boot_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'boot_mode': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'clean_step': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'conductor_group': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'console_enabled': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'console_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'deploy_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'deploy_step': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'description': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'disable_power_off': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'driver': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'extra': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'fault': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'inspect_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'inspection_finished_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'inspection_started_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'instance_uuid': UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'last_error': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'lessee': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'maintenance': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'maintenance_reason': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'management_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'name':
String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'network_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'owner': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'power_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'power_state': String(default=<class
```

property inspect_interface

property inspection_finished_at

property inspection_started_at

property instance_uuid

property last_error

property lessee

property maintenance

property maintenance_reason

property management_interface

property name

property network_interface

property owner

property power_interface

property power_state

property properties

property protected

property protected_reason

property provision_state

property provision_updated_at

property raid_interface

property rescue_interface

property resource_class

property retired

property retired_reason

property secure_boot

property storage_interface

property target_power_state

property target_provision_state

property traits

property updated_at

property uuid

property vendor_interface

class ironic.objects.node.NodeSetPowerStateNotification(*context=None*, ***kwargs*)

Bases: *NotificationBase*

Notification emitted when ironic changes a nodes power state.

VERSION = '1.0'

property created_at

property event_type

fields = {'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'event_type': Object(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'level': NotificationLevel(default=<class 'oslo_versionedobjects.fields.
UnspecifiedDefault'>,nullable=False,valid_values=('debug', 'info',
'warning', 'error', 'critical')), 'payload': Object(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'publisher': Object(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'updated_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property level

property payload

property publisher

property updated_at

class ironic.objects.node.NodeSetPowerStatePayload(*node*, *to_power*)

Bases: *NodePayload*

Payload schema for when ironic changes a nodes power state.

VERSION = '1.17'

property bios_interface

property boot_interface

property boot_mode

property clean_step

property conductor_group

property console_enabled

property console_interface

property created_at

property deploy_interface

property deploy_step

property description

property disable_power_off

property driver

property extra

property fault

```
fields = {'bios_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'boot_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'boot_mode': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'clean_step': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'conductor_group': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'console_enabled': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'console_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'deploy_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'deploy_step': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'description': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'disable_power_off': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'driver': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'extra': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'fault': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'inspect_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'inspection_finished_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'inspection_started_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'instance_uuid': UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'last_error': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'lessee': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'maintenance': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'maintenance_reason': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'management_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'name':
String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'network_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'owner': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'power_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'power_state': String(default=<class
```

property inspect_interface

property inspection_finished_at

property inspection_started_at

property instance_uuid

property last_error

property lessee

property maintenance

property maintenance_reason

property management_interface

property name

property network_interface

property owner

property power_interface

property power_state

property properties

property protected

property protected_reason

property provision_state

property provision_updated_at

property raid_interface

property rescue_interface

property resource_class

property retired

property retired_reason

property secure_boot

property storage_interface

property target_power_state

property target_provision_state

property to_power

property traits

property updated_at

property uuid

property vendor_interface

class ironic.objects.node.**NodeSetProvisionStateNotification**(*context=None*, ***kwargs*)

Bases: *NotificationBase*

Notification emitted when ironic changes a node provision state.

VERSION = '1.0'

property created_at

property event_type

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'event_type': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'level': NotificationLevel(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False,valid_values=('debug', 'info', 'warning', 'error', 'critical')), 'payload': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'publisher': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property level

property payload

property publisher

property updated_at

class ironic.objects.node.**NodeSetProvisionStatePayload**(*node*, *prev_state*, *prev_target*, *event*)

Bases: *NodePayload*

Payload schema for when ironic changes a node provision state.

```
SCHEMA = {'bios_interface': ('node', 'bios_interface'), 'boot_interface':
('node', 'boot_interface'), 'boot_mode': ('node', 'boot_mode'),
'clean_step': ('node', 'clean_step'), 'conductor_group': ('node',
'conductor_group'), 'console_enabled': ('node', 'console_enabled'),
'console_interface': ('node', 'console_interface'), 'created_at':
('node', 'created_at'), 'deploy_interface': ('node', 'deploy_interface'),
'deploy_step': ('node', 'deploy_step'), 'description': ('node',
'description'), 'disable_power_off': ('node', 'disable_power_off'),
'driver': ('node', 'driver'), 'driver_internal_info': ('node',
'driver_internal_info'), 'extra': ('node', 'extra'), 'fault': ('node',
'fault'), 'inspect_interface': ('node', 'inspect_interface'),
'inspection_finished_at': ('node', 'inspection_finished_at'),
'inspection_started_at': ('node', 'inspection_started_at'),
'instance_info': ('node', 'instance_info'), 'instance_uuid': ('node',
'instance_uuid'), 'last_error': ('node', 'last_error'), 'lessee':
('node', 'lessee'), 'maintenance': ('node', 'maintenance'),
'maintenance_reason': ('node', 'maintenance_reason'),
'management_interface': ('node', 'management_interface'), 'name':
('node', 'name'), 'network_interface': ('node', 'network_interface'),
'owner': ('node', 'owner'), 'power_interface': ('node',
'power_interface'), 'power_state': ('node', 'power_state'), 'properties':
('node', 'properties'), 'protected': ('node', 'protected'),
'protected_reason': ('node', 'protected_reason'), 'provision_state':
('node', 'provision_state'), 'provision_updated_at': ('node',
'provision_updated_at'), 'raid_interface': ('node', 'raid_interface'),
'rescue_interface': ('node', 'rescue_interface'), 'resource_class':
('node', 'resource_class'), 'retired': ('node', 'retired'),
'retired_reason': ('node', 'retired_reason'), 'secure_boot': ('node',
'secure_boot'), 'storage_interface': ('node', 'storage_interface'),
'target_power_state': ('node', 'target_power_state'),
'target_provision_state': ('node', 'target_provision_state'),
'updated_at': ('node', 'updated_at'), 'uuid': ('node', 'uuid'),
'vendor_interface': ('node', 'vendor_interface')}

VERSION = '1.18'

property bios_interface

property boot_interface

property boot_mode

property clean_step

property conductor_group

property console_enabled

property console_interface

property created_at

property deploy_interface
```

property deploy_step

property description

property disable_power_off

property driver

property driver_internal_info

property event

property extra

property fault

```
fields = {'bios_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'boot_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'boot_mode': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'clean_step': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'conductor_group': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'console_enabled': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'console_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'deploy_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'deploy_step': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'description': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'disable_power_off': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'driver': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'driver_internal_info': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'event': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'extra': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'fault': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'inspect_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'inspection_finished_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'inspection_started_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'instance_info': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'instance_uuid': UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'last_error': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'lessee': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'maintenance': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'maintenance_reason': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'management_interface': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'name':
String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'network_interface': String(default=<class
```

property inspect_interface

property inspection_finished_at

property inspection_started_at

property instance_info

property instance_uuid

property last_error

property lessee

property maintenance

property maintenance_reason

property management_interface

property name

property network_interface

property owner

property power_interface

property power_state

property previous_provision_state

property previous_target_provision_state

property properties

property protected

property protected_reason

property provision_state

property provision_updated_at

property raid_interface

property rescue_interface

property resource_class

property retired

property retired_reason

property secure_boot

property storage_interface

property target_power_state

property target_provision_state

property traits

property updated_at

property uuid

property vendor_interface

ironic.objects.node_history module

class ironic.objects.node_history.**NodeHistory**(*context=None*, ***kwargs*)

Bases: *IronicObject*, VersionedObjectDictCompat

VERSION = '1.1'

property conductor

create(*context=None*)

Create a NodeHistory record in the DB.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: NodeHistory(context)

property created_at

dbapi = <oslo_db.api.DBAPI object>

destroy(*context=None*)

Delete the NodeHistory from the DB.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: NodeHistory(context)

Raises

NodeHistoryNotFound

property event

property event_type

```
fields = {'conductor': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'event': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'event_type': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'id':
Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'node_id': Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'severity': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'updated_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'user':
String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid':
UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}
```

classmethod get(*context*, *history_ident*)

Get a history based on its id or uuid.

Parameters

- **history_ident** The id or uuid of a history.
- **context** Security context

Returns

A *NodeHistory* object.

Raises

InvalidIdentity

classmethod get_by_id(*context*, *history_id*)

Get a NodeHistory object by its integer ID.

Parameters

- **cls** the *NodeHistory*
- **context** Security context
- **history_id** The ID of a history.

Returns

A *NodeHistory* object.

Raises

NodeHistoryNotFound

classmethod get_by_uuid(*context*, *uuid*)

Get a NodeHistory object by its UUID.

Parameters

- **cls** the *NodeHistory*
- **context** Security context
- **uuid** The UUID of a NodeHistory.

Returns
A *NodeHistory* object.

Raises
NodeHistoryNotFound

property id

classmethod list(*context*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Return a list of NodeHistory objects.

Parameters

- **cls** the *NodeHistory*
- **context** Security context.
- **limit** Maximum number of resources to return in a single result.
- **marker** Pagination marker for large data sets.
- **sort_key** Column to sort results by.
- **sort_dir** Direction to sort. asc or desc.

Returns
A list of *NodeHistory* object.

Raises
InvalidParameterValue

classmethod list_by_node_id(*context*, *node_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Return a list of NodeHistory objects belongs to a given node ID.

Parameters

- **cls** the *NodeHistory*
- **context** Security context.
- **node_id** The ID of the node.
- **limit** Maximum number of resources to return in a single result.
- **marker** Pagination marker for large data sets.
- **sort_key** Column to sort results by.
- **sort_dir** Direction to sort. asc or desc.

Returns
A list of *NodeHistory* object.

Raises
InvalidParameterValue

property node_id

property severity

property updated_at

property user

property uuid

ironic.objects.node_inventory module

class ironic.objects.node_inventory.**NodeInventory**(*context=None*, ***kwargs*)

Bases: *IronicObject*, VersionedObjectDictCompat

VERSION = '1.1'

create(*context=None*)

Create a NodeInventory record in the DB.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: NodeHistory(context)

property created_at

dbapi = <oslo_db.api.DBAPI object>

destroy(*context=None*)

Delete the NodeInventory from the DB.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: NodeInventory(context)

Raises

NodeInventoryNotFound

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'id': Integer(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'inventory_data': FlexibleDict(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'node_id': Integer(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'plugin_data': FlexibleDict(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

classmethod get_by_node_id(*context*, *node_id*)

Get a NodeInventory object by its node ID.

Parameters

- **cls** the *NodeInventory*
- **context** Security context
- **uuid** The UUID of a NodeInventory.

Returns

A *NodeInventory* object.

Raises

NodeInventoryNotFound

property id

property inventory_data

property node_id

property plugin_data

property updated_at

ironic.objects.notification module

class ironic.objects.notification.**EventType**(*context=None*, ***kwargs*)

Bases: *IronicObject*

Defines the event_type to be sent on the wire.

An EventType must specify the object being acted on, a string describing the action being taken on the notification, and the status of the action.

VERSION = '1.1'

property action

property created_at

fields = {'action': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'object': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'status': NotificationStatus(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False,valid_values=('start', 'end', 'error', 'success')), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property object

property status

to_event_type_field()

Constructs string for event_type to be sent on the wire.

The string is in the format: baremetal.<object>.<action>.<status>

Raises
ValueError if self.status is not one of `fields.NotificationStatusField`

Returns
event_type string

property updated_at

class ironic.objects.notification.**NotificationBase**(*context=None*, ***kwargs*)

Bases: *IronicObject*

Base class for versioned notifications.

Subclasses must define the payload field, which must be a subclass of NotificationPayloadBase.

VERSION = '1.0'

property created_at

emit(*context*)

Send the notification.

Raises
NotificationPayloadError

Raises
oslo_versionedobjects.exceptions.MessageDeliveryFailure

property event_type

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'event_type': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'level': NotificationLevel(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False,valid_values=('debug', 'info', 'warning', 'error', 'critical')), 'publisher': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property level

property publisher

property updated_at

class ironic.objects.notification.**NotificationPayloadBase**(**args*, ***kwargs*)

Bases: *IronicObject*

Base class for the payload of versioned notifications.

SCHEMA = {}

VERSION = '1.0'

property created_at

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

populate_schema(***kwargs*)

Populate the object based on the SCHEMA and the source objects

Parameters

kwargs A dict contains the source object and the keys defined in the SCHEMA

Raises

NotificationSchemaObjectError

Raises

NotificationSchemaKeyError

property updated_at

class ironic.objects.notification.**NotificationPublisher**(*context=None*, ***kwargs*)

Bases: *IronicObject*

VERSION = '1.0'

property created_at

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'host': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'service': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property host

property service

property updated_at

ironic.objects.notification.**mask_secrets**(*payload*)

Remove secrets from payload object.

ironic.objects.port module

class ironic.objects.port.**Port**(*context=None*, ***kwargs*)

Bases: *IronicObject*, VersionedObjectDictCompat

VERSION = '1.16'

property address

property available_for_dynamic_portgroup

property category

create(*context=None*)

Create a Port record in the DB.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Port(context)

Raises

MACAlreadyExists if address column is not unique

Raises

PortAlreadyExists if uuid column is not unique

property created_at

dbapi = <oslo_db.api.DBAPI object>

property description

destroy(*context=None*)

Delete the Port from the DB.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Port(context)

Raises

PortNotFound

property extra

```
fields = {'address': MACAddress(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'available_for_dynamic_portgroup': Boolean(default=True,nullable=False),
'category': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'description': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'extra': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'id':
Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'internal_info': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'is_smartnic': Boolean(default=False,nullable=True),
'local_link_connection': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'name':
String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'node_id': Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'node_uuid': UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'physical_network': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'portgroup_id': Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'pxe_enabled': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'updated_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid':
UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'vendor': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}
```

classmethod get(*context*, *port_id*)

Find a port.

Find a port based on its id or uuid or name or MAC address and return a Port object.

Parameters

- **context** Security context
- **port_id** the id *or* uuid *or* name *or* MAC address of a port.

Returns

a *Port* object.

Raises

InvalidIdentity

classmethod get_by_address(*context*, *address*, *owner=None*, *project=None*)

Find a port based on address and return a *Port* object.

Parameters

- **cls** the *Port*
- **context** Security context
- **address** the address of a port.
- **owner** DEPRECATED a node owner to match against
- **project** a node owner or lessee to match against

Returns

a *Port* object.

Raises

PortNotFound

classmethod get_by_id(*context*, *port_id*)

Find a port based on its integer ID and return a Port object.

Parameters

- **cls** the *Port*
- **context** Security context
- **port_id** the ID of a port.

Returns

a *Port* object.

Raises

PortNotFound

classmethod get_by_name(*context*, *name*)

Find a port based on name and return a *Port* object.

Parameters

- **cls** the *Port*
- **context** Security context
- **name** the name of a port.

Returns

a *Port* object.

Raises

PortNotFound

classmethod get_by_uuid(*context*, *uuid*)

Find a port based on UUID and return a *Port* object.

Parameters

- **cls** the *Port*
- **context** Security context

- **uuid** the UUID of a port.

Returns
a *Port* object.

Raises
PortNotFound

property id

property internal_info

property is_smartnic

classmethod list(*context*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *owner=None*, *project=None*, *conductor_groups=None*, *filters=None*)

Return a list of Port objects.

Parameters

- **context** Security context.
- **limit** maximum number of resources to return in a single result.
- **marker** pagination marker for large data sets.
- **sort_key** column to sort results by.
- **sort_dir** direction to sort. asc or desc.
- **owner** DEPRECATED a node owner to match against
- **project** a node owner or lessee to match against
- **conductor_groups** A list of conductor groups to filter by, defaults to None
- **filters** Filters to apply, defaults to None

Returns
a list of *Port* object.

Raises
InvalidParameterValue

classmethod list_by_node_id(*context*, *node_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *owner=None*, *project=None*, *filters=None*)

Return a list of Port objects associated with a given node ID.

Parameters

- **context** Security context.
- **node_id** the ID of the node.
- **limit** maximum number of resources to return in a single result.
- **marker** pagination marker for large data sets.
- **sort_key** column to sort results by.
- **sort_dir** direction to sort. asc or desc.
- **owner** DEPRECATED a node owner to match against

- **project** a node owner or lessee to match against
- **filters** Filters to apply, defaults to None

Returns
a list of *Port* object.

classmethod list_by_node_shards(*context*, *shards*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*, *filters=None*)

Return a list of Port objects associated with nodes in shards

Parameters

- **context** Security context.
- **shards** a list of shards
- **limit** maximum number of resources to return in a single result.
- **marker** pagination marker for large data sets.
- **sort_key** column to sort results by.
- **sort_dir** direction to sort. asc or desc.
- **project** a node owner or lessee to match against
- **filters** Filters to apply, defaults to None

Returns
a list of *Port* object.

classmethod list_by_portgroup_id(*context*, *portgroup_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *owner=None*, *project=None*, *filters=None*)

Return a list of Port objects associated with a given portgroup ID.

Parameters

- **context** Security context.
- **portgroup_id** the ID of the portgroup.
- **limit** maximum number of resources to return in a single result.
- **marker** pagination marker for large data sets.
- **sort_key** column to sort results by.
- **sort_dir** direction to sort. asc or desc.
- **owner** DEPRECATED a node owner to match against
- **project** a node owner or lessee to match against
- **filters** Filters to apply, defaults to None

Returns
a list of *Port* object.

property local_link_connection

property name

property node_id

property node_uuid

property physical_network

property portgroup_id

property pxe_enabled

refresh(*context=None*)

Loads updates for this Port.

Loads a port with the same uuid from the database and checks for updated attributes. Updates are applied from the loaded port column by column, if there are any updates.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Port(context)

Raises

PortNotFound

save(*context=None*)

Save updates to this Port.

Updates will be made column by column based on the result of self.what_changed().

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Port(context)

Raises

PortNotFound

Raises

MACAlreadyExists if address column is not unique

set_local_link_connection(*key*, *value*)

Set a *local_link_connection* value.

Setting a *local_link_connection* dict value via this method ensures that this field will be flagged for saving.

Parameters

- **key** Key of item to set
- **value** Value of item to set

classmethod supports_is_smartnic()

Return whether is_smartnic field is supported.

Returns
Whether is_smartnic field is supported

Raises
ovo_exception.IncompatibleObjectVersion

classmethod supports_physical_network()

Return whether the physical_network field is supported.

Returns
Whether the physical_network field is supported

Raises
ovo_exception.IncompatibleObjectVersion

property updated_at

property uuid

property vendor

class ironic.objects.port.**PortCRUDNotification**(*context=None*, ***kwargs*)

Bases: *NotificationBase*

Notification emitted when ironic creates, updates or deletes a port.

VERSION = '1.0'

property created_at

property event_type

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'event_type': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'level': NotificationLevel(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False,valid_values=('debug', 'info', 'warning', 'error', 'critical')), 'payload': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'publisher': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property level

property payload

property publisher

property updated_at

class ironic.objects.port.**PortCRUDPayload**(*port*, *node_uuid*, *portgroup_uuid*)

Bases: *NotificationPayloadBase*

```
SCHEMA = {'address': ('port', 'address'),
'available_for_dynamic_portgroup': ('port',
'available_for_dynamic_portgroup'), 'category': ('port', 'category'),
'created_at': ('port', 'created_at'), 'description': ('port',
'description'), 'extra': ('port', 'extra'), 'is_smartnic': ('port',
'is_smartnic'), 'local_link_connection': ('port',
'local_link_connection'), 'name': ('port', 'name'), 'physical_network':
('port', 'physical_network'), 'pxe_enabled': ('port', 'pxe_enabled'),
'updated_at': ('port', 'updated_at'), 'uuid': ('port', 'uuid'),
'vendor': ('port', 'vendor')}

VERSION = '1.8'

property address

property available_for_dynamic_portgroup

property category

property created_at

property description

property extra

fields = {'address': MACAddress(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'available_for_dynamic_portgroup': Boolean(default=True,nullable=False),
'category': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'description': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'extra': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'is_smartnic': Boolean(default=False,nullable=True),
'local_link_connection': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'name':
String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'node_uuid': UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'physical_network': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'portgroup_uuid': UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'pxe_enabled': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'updated_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid':
UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'vendor': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}
```

property is_smartnic

property local_link_connection

property name

property node_uuid

property physical_network

property portgroup_uuid

property pxe_enabled

property updated_at

property uuid

property vendor

ironic.objects.portgroup module

class ironic.objects.portgroup.**Portgroup**(*context=None*, ***kwargs*)

Bases: *IronicObject*, VersionedObjectDictCompat

VERSION = '1.9'

property address

property category

create(*context=None*)

Create a Portgroup record in the DB.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Portgroup(context)

Raises

DuplicateName, MACAlreadyExists, PortgroupAlreadyExists

property created_at

dbapi = <oslo_db.api.DBAPI object>

destroy(*context=None*)

Delete the Portgroup from the DB.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Portgroup(context)

Raises
PortgroupNotEmpty, PortgroupNotFound

property dynamic_portgroup

property extra

fields = {'address': MACAddress(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'category': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'dynamic_portgroup': Boolean(default=False,nullable=True), 'extra': FlexibleDict(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'id': Integer(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'internal_info': FlexibleDict(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'mode': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'name': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'node_id': Integer(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'node_uuid': UUID(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'physical_network': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'properties': FlexibleDict(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'standalone_ports_supported': Boolean(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid': UUID(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

classmethod get(*context*, *portgroup_ident*)

Find a portgroup based on its id, uuid, name or address.

Parameters

- **portgroup_ident** The id, uuid, name or address of a portgroup.
- **context** Security context

Returns
A *Portgroup* object.

Raises
InvalidIdentity

classmethod get_by_address(*context*, *address*, *project=None*)

Find portgroup by address and return a *Portgroup* object.

Parameters

- **cls** the *Portgroup*
- **context** Security context
- **address** The MAC address of a portgroup.
- **project** a node owner or lessee to match against.

Returns

A *Portgroup* object.

Raises

PortgroupNotFound

classmethod get_by_id(*context*, *portgroup_id*)

Find a portgroup by its integer ID and return a Portgroup object.

Parameters

- **cls** the *Portgroup*
- **context** Security context
- **portgroup_id** The ID of a portgroup.

Returns

A *Portgroup* object.

Raises

PortgroupNotFound

classmethod get_by_name(*context*, *name*)

Find portgroup based on name and return a *Portgroup* object.

Parameters

- **cls** the *Portgroup*
- **context** Security context
- **name** The name of a portgroup.

Returns

A *Portgroup* object.

Raises

PortgroupNotFound

classmethod get_by_uuid(*context*, *uuid*)

Find a portgroup by UUID and return a *Portgroup* object.

Parameters

- **cls** the *Portgroup*
- **context** Security context
- **uuid** The UUID of a portgroup.

Returns
A *Portgroup* object.

Raises
PortgroupNotFound

property id

property internal_info

classmethod list(*context*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*, *conductor_groups=None*, *filters=None*)

Return a list of Portgroup objects.

Parameters

- **cls** the *Portgroup*
- **context** Security context.
- **limit** Maximum number of resources to return in a single result.
- **marker** Pagination marker for large data sets.
- **sort_key** Column to sort results by.
- **sort_dir** Direction to sort. asc or desc.
- **project** a node owner or lessee to match against.
- **conductor_groups** A list of conductor groups to filter by, defaults to None
- **filters** Filters to apply, defaults to None

Returns
A list of *Portgroup* object.

Raises
InvalidParameterValue

classmethod list_by_node_id(*context*, *node_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

Return a list of Portgroup objects associated with a given node ID.

Parameters

- **cls** the *Portgroup*
- **context** Security context.
- **node_id** The ID of the node.
- **limit** Maximum number of resources to return in a single result.
- **marker** Pagination marker for large data sets.
- **sort_key** Column to sort results by.
- **sort_dir** Direction to sort. asc or desc.
- **project** a node owner or lessee to match against.

Returns
A list of *Portgroup* object.

Raises
InvalidParameterValue

classmethod list_by_node_shards(*context*, *shards*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*)

Return a list of Portgroup objects associated with nodes in shards.

Parameters

- **context** Security context.
- **shards** A list of shards.
- **limit** Maximum number of resources to return in a single result.
- **marker** Pagination marker for large data sets.
- **sort_key** Column to sort results by.
- **sort_dir** Direction to sort. asc or desc.

Returns
A list of *Portgroup* object.

property mode

property name

property node_id

property node_uuid

property physical_network

property properties

refresh(*context=None*)

Loads updates for this Portgroup.

Loads a portgroup with the same uuid from the database and checks for updated attributes. Updates are applied from the loaded portgroup column by column, if there are any updates.

Parameters
context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Portgroup(context)

Raises
PortgroupNotFound

save(*context=None*)

Save updates to this Portgroup.

Updates will be made column by column based on the result of self.what_changed().

Parameters
context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Portgroup(context)

Raises
PortgroupNotFound, DuplicateName, MACAlreadyExists

property standalone_ports_supported

update_physical_network(*new_physical_network*, *context=None*)

Updates the phyiscal_network for this Portgroup and its Ports.

Parameters
context Security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Portgroup(context)

Raises
PortgroupNotFound

property updated_at

property uuid

class ironic.objects.portgroup.**PortgroupCRUDNotification**(*context=None*, ***kwargs*)

Bases: *NotificationBase*

Notification when ironic creates, updates or deletes a portgroup.

VERSION = '1.0'

property created_at

property event_type

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'event_type': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'level': NotificationLevel(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False,valid_values=('debug', 'info', 'warning', 'error', 'critical')), 'payload': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'publisher': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property level

property payload

property publisher

property updated_at

class ironic.objects.portgroup.**PortgroupCRUDPayload**(*portgroup*, *node_uuid*)

Bases: *NotificationPayloadBase*

```
SCHEMA = {'address': ('portgroup', 'address'), 'category': ('portgroup',
'category'), 'created_at': ('portgroup', 'created_at'),
'dynamic_portgroup': ('portgroup', 'dynamic_portgroup'), 'extra':
('portgroup', 'extra'), 'mode': ('portgroup', 'mode'), 'name':
('portgroup', 'name'), 'physical_network': ('portgroup',
'physical_network'), 'properties': ('portgroup', 'properties'),
'standalone_ports_supported': ('portgroup',
'standalone_ports_supported'), 'updated_at': ('portgroup', 'updated_at'),
'uuid': ('portgroup', 'uuid')}

VERSION = '1.3'

property address

property category

property created_at

property dynamic_portgroup

property extra

fields = {'address': MACAddress(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'category': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'dynamic_portgroup': Boolean(default=False,nullable=True), 'extra':
FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'mode':
String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'name':
String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'node_uuid': UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'physical_network': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'properties': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'standalone_ports_supported': Boolean(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'updated_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid':
UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False)}

property mode

property name

property node_uuid
```

property physical_network

property properties

property standalone_ports_supported

property updated_at

property uuid

ironic.objects.runbook module

class ironic.objects.runbook.**Runbook**(*context=None*, ***kwargs*)

Bases: *IronicObject*, VersionedObjectDictCompat

VERSION = '1.1'

create(*context=None*)

Create a Runbook record in the DB.

Parameters

context security context. NOTE: This should only be used internally by the indirection_api, but, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Runbook(context).

Raises

RunbookDuplicateName if a runbook with the same name exists.

Raises

RunbookAlreadyExists if a runbook with the same UUID exists.

property created_at

dbapi = <oslo_db.api.DBAPI object>

destroy()

Delete the Runbook from the DB.

Parameters

context security context. NOTE: This should only be used internally by the indirection_api, but, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Runbook(context).

Raises

RunbookNotFound if the runbook no longer appears in the database.

property disable_ramdisk

property extra

```
fields = {'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'disable_ramdisk': Boolean(default=False,nullable=False), 'extra':
FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'id':
Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'name': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'owner': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'public': Boolean(default=False,nullable=False), 'steps':
List(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'updated_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid':
UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False)}
```

classmethod get_by_id(*context*, *runbook_id*)

Find a runbook based on its integer ID.

Parameters

- **context** security context. NOTE: This should only be used internally by the indirection_api, but, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Runbook(context).
- **runbook_id** The ID of a runbook.

Raises

RunbookNotFound if the runbook no longer appears in the database.

Returns

a *Runbook* object.

classmethod get_by_name(*context*, *name*)

Find a runbook based on its name.

Parameters

- **context** security context. NOTE: This should only be used internally by the indirection_api, but, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Runbook(context).
- **name** The name of a runbook.

Raises

RunbookNotFound if the runbook no longer appears in the database.

Returns

a *Runbook* object.

classmethod get_by_uuid(*context*, *uuid*)

Find a runbook based on its UUID.

Parameters

- **context** security context. NOTE: This should only be used internally by the indirection_api, but, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Runbook(context).
- **uuid** The UUID of a runbook.

Raises

RunbookNotFound if the runbook no longer appears in the database.

Returns

a *Runbook* object.

property id

classmethod list(*context*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *filters=None*)

Return a list of Runbook objects.

Parameters

- **context** security context. NOTE: This should only be used internally by the indirection_api, but, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Runbook(context).
- **limit** maximum number of resources to return in a single result.
- **marker** pagination marker for large data sets.
- **sort_key** column to sort results by.
- **sort_dir** direction to sort. asc or desc.
- **filters** Filters to apply.

Returns

a list of *Runbook* objects.

classmethod list_by_names(*context*, *names*)

Return a list of Runbook objects matching a set of names.

Parameters

- **context** security context. NOTE: This should only be used internally by the indirection_api, but, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Runbook(context).
- **names** a list of names to filter by.

Returns

a list of *Runbook* objects.

property name

property owner

property public

refresh(*context=None*)

Loads updates for this runbook.

Loads a runbook with the same uuid from the database and checks for updated attributes. Updates are applied from the loaded template column by column, if there are any updates.

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api, but, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Port(context)

Raises

RunbookNotFound if the runbook no longer appears in the database.

save(*context=None*)

Save updates to this Runbook.

Column-wise updates will be made based on the result of self.what_changed().

Parameters

context Security context. NOTE: This should only be used internally by the indirection_api, but, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Runbook(context)

Raises

RunbookDuplicateName if a runbook with the same name exists.

Raises

RunbookNotFound if the runbook does not exist.

property steps

property updated_at

property uuid

class ironic.objects.runbook.RunbookCRUDNotification(*context=None*, ***kwargs*)

Bases: *NotificationBase*

Notification emitted on runbook API operations.

VERSION = '1.0'

property created_at

property event_type

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'event_type': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'level': NotificationLevel(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False,valid_values=('debug', 'info', 'warning', 'error', 'critical')), 'payload': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'publisher': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property level

property payload

property publisher

property updated_at

class ironic.objects.runbook.RunbookCRUDPayload(*runbook*, ***kwargs*)

Bases: *NotificationPayloadBase*

SCHEMA = {'created_at': ('runbook', 'created_at'), 'disable_ramdisk': ('runbook', 'disable_ramdisk'), 'extra': ('runbook', 'extra'), 'name': ('runbook', 'name'), 'owner': ('runbook', 'owner'), 'public': ('runbook', 'public'), 'steps': ('runbook', 'steps'), 'updated_at': ('runbook', 'updated_at'), 'uuid': ('runbook', 'uuid')}

VERSION = '1.0'

property created_at

property disable_ramdisk

property extra

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'disable_ramdisk': Boolean(default=False,nullable=False), 'extra': FlexibleDict(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'name': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'owner': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'public': Boolean(default=False,nullable=False), 'steps': List(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid': UUID(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False)}

property name

property owner

property public

property steps

property updated_at

property uuid

ironic.objects.trait module

class ironic.objects.trait.**Trait**(*context=None*, ***kwargs*)

Bases: *IronicObject*

VERSION = '1.1'

create(*context=None*)

Create a Trait record in the DB.

Parameters

context security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Trait(context).

Raises

InvalidParameterValue if adding the trait would exceed the per-node traits limit.

Raises

NodeNotFound if the node no longer appears in the database.

property created_at

dbapi = <oslo_db.api.DBAPI object>

classmethod **destroy**(*context*, *node_id*, *trait*)

Delete the Trait from the DB.

Parameters

- **context** security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Trait(context).
- **node_id** The id of a node.
- **trait** A trait string.

Raises

NodeNotFound if the node no longer appears in the database.

Raises

NodeTraitNotFound if the trait is not found.

classmethod exists(*context*, *node_id*, *trait*)

Check whether a Trait exists in the DB.

Parameters

- **context** security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Trait(context).
- **node_id** The id of a node.
- **trait** A trait string.

Returns

True if the trait exists otherwise False.

Raises

NodeNotFound if the node no longer appears in the database.

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'node_id': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'trait': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property node_id

property trait

property updated_at

class ironic.objects.trait.**TraitList**(**args*, ***kwargs*)

Bases: *IronicObjectListBase*, *IronicObject*

VERSION = '1.2'

classmethod create(*context*, *node_id*, *traits*)

Replace all existing traits with the specified list.

Parameters

- **context** security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Trait(context).
- **node_id** The id of a node.
- **traits** List of Strings; traits to set.

Raises

InvalidParameterValue if adding the trait would exceed the per-node traits limit.

Raises

NodeNotFound if the node no longer appears in the database.

property created_at

dbapi = <oslo_db.api.DBAPI object>

classmethod destroy(*context*, *node_id*)

Delete all traits for the specified node.

Parameters

- **context** security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Trait(context).
- **node_id** The id of a node.

Raises

NodeNotFound if the node no longer appears in the database.

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'objects': List(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

classmethod get_by_node_id(*context*, *node_id*)

Return all traits for the specified node.

Parameters

- **context** security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: Trait(context).
- **node_id** The id of a node.

Raises

NodeNotFound if the node no longer appears in the database.

get_trait_names()

Return a list of names of the traits in this list.

property objects

property updated_at

ironic.objects.volume_connector module

class ironic.objects.volume_connector.**VolumeConnector**(*context=None*, ***kwargs*)

Bases: *IronicObject*, VersionedObjectDictCompat

VERSION = '1.1'

property connector_id

create(*context=None*)

Create a VolumeConnector record in the DB.

Parameters

context security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: VolumeConnector(context).

Raises

VolumeConnectorTypeAndIdAlreadyExists if a volume connector already exists with the same type and connector_id

Raises

VolumeConnectorAlreadyExists if a volume connector with the same UUID already exists

property created_at

dbapi = <oslo_db.api.DBAPI object>

destroy(*context=None*)

Delete the VolumeConnector from the DB.

Parameters

context security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: VolumeConnector(context).

Raises

VolumeConnectorNotFound if the volume connector cannot be found

property extra

fields = {'connector_id': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'extra': FlexibleDict(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'id': Integer(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'node_id': Integer(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'type': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid': UUID(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

classmethod get(*context*, *ident*)

Find a volume connector based on its ID or UUID.

Parameters

- **context** security context
- **ident** the database primary key ID *or* the UUID of a volume connector

Returns
a *VolumeConnector* object

Raises
InvalidIdentity if ident is neither an integer ID nor a UUID

Raises
VolumeConnectorNotFound if no volume connector exists with the specified ident

classmethod get_by_id(*context*, *db_id*)

Find a volume connector based on its integer ID.

Parameters

- **cls** the *VolumeConnector*
- **context** Security context.
- **db_id** The integer (database primary key) ID of a volume connector.

Returns
A *VolumeConnector* object.

Raises
VolumeConnectorNotFound if no volume connector exists with the specified ID.

classmethod get_by_uuid(*context*, *uuid*)

Find a volume connector based on its UUID.

Parameters

- **cls** the *VolumeConnector*
- **context** security context
- **uuid** the UUID of a volume connector

Returns
a *VolumeConnector* object

Raises
VolumeConnectorNotFound if no volume connector exists with the specified UUID

property id

classmethod list(*context*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

Return a list of VolumeConnector objects.

Parameters

- **context** security context
- **limit** maximum number of resources to return in a single result

- **marker** pagination marker for large data sets
- **sort_key** column to sort results by
- **sort_dir** direction to sort. asc or desc.
- **project** The associated node project to search with.

Returns
a list of *VolumeConnector* objects

Raises
InvalidParameterValue if sort_key does not exist

classmethod list_by_node_id(*context*, *node_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

Return a list of VolumeConnector objects related to a given node ID.

Parameters

- **context** security context
- **node_id** the integer ID of the node
- **limit** maximum number of resources to return in a single result
- **marker** pagination marker for large data sets
- **sort_key** column to sort results by
- **sort_dir** direction to sort. asc or desc.
- **project** The associated node project to search with.

Returns
a list of *VolumeConnector* objects

Returns
a list of *VolumeConnector* objects

Raises
InvalidParameterValue if sort_key does not exist

property node_id

refresh(*context=None*)

Load updates for this VolumeConnector.

Load a volume connector with the same UUID from the database and check for updated attributes. If there are any updates, they are applied from the loaded volume connector, column by column.

Parameters
context security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: VolumeConnector(context).

save(*context=None*)

Save updates to this VolumeConnector.

Updates will be made column by column based on the result of self.do_version_changes_for_db().

Parameters

context security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: VolumeConnector(context).

Raises

VolumeConnectorNotFound if the volume connector cannot be found

Raises

VolumeConnectorTypeAndIdAlreadyExists if another connector already exists with the same values for type and connector_id fields

Raises

InvalidParameterValue when the UUID is being changed

property type

property updated_at

property uuid

class ironic.objects.volume_connector.**VolumeConnectorCRUDNotification**(*context=None*, ***kwargs*)

Bases: *NotificationBase*

Notification emitted at CRUD of a volume connector.

VERSION = '1.0'

property created_at

property event_type

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'event_type': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'level': NotificationLevel(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False,valid_values=('debug', 'info', 'warning', 'error', 'critical')), 'payload': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'publisher': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property level

property payload

property publisher

property updated_at

class ironic.objects.volume_connector.**VolumeConnectorCRUDPayload**(*connector*, *node_uuid*)

Bases: *NotificationPayloadBase*

Payload schema for CRUD of a volume connector.

SCHEMA = {'connector_id': ('connector', 'connector_id'), 'created_at': ('connector', 'created_at'), 'extra': ('connector', 'extra'), 'type': ('connector', 'type'), 'updated_at': ('connector', 'updated_at'), 'uuid': ('connector', 'uuid')}

VERSION = '1.0'

property connector_id

property created_at

property extra

fields = {'connector_id': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'extra': FlexibleDict(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'node_uuid': UUID(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'type': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid': UUID(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False)}

property node_uuid

property type

property updated_at

property uuid

ironic.objects.volume_target module

class ironic.objects.volume_target.**VolumeTarget**(*context=None*, ***kwargs*)

Bases: *IronicObject*, VersionedObjectDictCompat

VERSION = '1.1'

property boot_index

create(*context=None*)

Create a VolumeTarget record in the DB.

Parameters

context security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: VolumeTarget(context).

Raises

VolumeTargetBootIndexAlreadyExists if a volume target already exists with the same node ID and boot index

Raises

VolumeTargetAlreadyExists if a volume target with the same UUID exists

property created_at

dbapi = <oslo_db.api.DBAPI object>

destroy(*context=None*)

Delete the VolumeTarget from the DB.

Parameters

context security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: VolumeTarget(context).

Raises

VolumeTargetNotFound if the volume target cannot be found

property extra

fields = {'boot_index': Integer(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'extra': FlexibleDict(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'id': Integer(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'node_id': Integer(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'properties': FlexibleDict(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid': UUID(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'volume_id': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'volume_type': String(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

classmethod get(*context*, *ident*)

Find a volume target based on its ID or UUID.

Parameters

- **context** security context
- **ident** the database primary key ID *or* the UUID of a volume target

Returns
a *VolumeTarget* object

Raises
InvalidIdentity if ident is neither an integer ID nor a UUID

Raises
VolumeTargetNotFound if no volume target with this ident exists

classmethod get_by_id(*context*, *db_id*)

Find a volume target based on its database ID.

Parameters

- **cls** the *VolumeTarget*
- **context** security context
- **db_id** the database primary key (integer) ID of a volume target

Returns
a *VolumeTarget* object

Raises
VolumeTargetNotFound if no volume target with this ID exists

classmethod get_by_uuid(*context*, *uuid*)

Find a volume target based on its UUID.

Parameters

- **cls** the *VolumeTarget*
- **context** security context
- **uuid** the UUID of a volume target

Returns
a *VolumeTarget* object

Raises
VolumeTargetNotFound if no volume target with this UUID exists

property id

classmethod list(*context*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

Return a list of VolumeTarget objects.

Parameters

- **context** security context
- **limit** maximum number of resources to return in a single result

- **marker** pagination marker for large data sets
- **sort_key** column to sort results by
- **sort_dir** direction to sort. asc or desc.
- **project** The associated node project to search with.

Returns
a list of `VolumeConnector` objects

Returns
a list of *VolumeTarget* objects

Raises
InvalidParameterValue if sort_key does not exist

classmethod list_by_node_id(*context*, *node_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

Return a list of VolumeTarget objects related to a given node ID.

Parameters

- **context** security context
- **node_id** the integer ID of the node
- **limit** maximum number of resources to return in a single result
- **marker** pagination marker for large data sets
- **sort_key** column to sort results by
- **sort_dir** direction to sort. asc or desc.
- **project** The associated node project to search with.

Returns
a list of `VolumeConnector` objects

Returns
a list of *VolumeTarget* objects

Raises
InvalidParameterValue if sort_key does not exist

classmethod list_by_volume_id(*context*, *volume_id*, *limit=None*, *marker=None*, *sort_key=None*, *sort_dir=None*, *project=None*)

Return a list of VolumeTarget objects related to a given volume ID.

Parameters

- **context** security context
- **volume_id** the UUID of the volume
- **limit** maximum number of volume targets to return in a single result
- **marker** pagination marker for large data sets
- **sort_key** column to sort results by
- **sort_dir** direction to sort. asc or desc.

Returns
a list of *VolumeTarget* objects

Raises
InvalidParameterValue if sort_key does not exist

property node_id

property properties

refresh(*context=None*)

Loads updates for this VolumeTarget.

Load a volume target with the same UUID from the database and check for updated attributes. If there are any updates, they are applied from the loaded volume target, column by column.

Parameters
context security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: VolumeTarget(context).

Raises
VolumeTargetNotFound if the volume target cannot be found

save(*context=None*)

Save updates to this VolumeTarget.

Updates will be made column by column based on the result of self.do_version_changes_for_db().

Parameters
context security context. NOTE: This should only be used internally by the indirection_api. Unfortunately, RPC requires context as the first argument, even though we dont use it. A context should be set when instantiating the object, e.g.: VolumeTarget(context).

Raises
InvalidParameterValue if the UUID is being changed

Raises
VolumeTargetBootIndexAlreadyExists if a volume target already exists with the same node ID and boot index values

Raises
VolumeTargetNotFound if the volume target cannot be found

property updated_at

property uuid

property volume_id

property volume_type

class ironic.objects.volume_target.**VolumeTargetCRUDNotification**(*context=None*, ***kwargs*)

Bases: *NotificationBase*

Notification emitted at CRUD of a volume target.

VERSION = '1.0'

property created_at

property event_type

fields = {'created_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'event_type': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'level': NotificationLevel(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False,valid_values=('debug', 'info', 'warning', 'error', 'critical')), 'payload': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'publisher': Object(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False), 'updated_at': DateTime(default=<class 'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property level

property payload

property publisher

property updated_at

class ironic.objects.volume_target.**VolumeTargetCRUDPayload**(*target*, *node_uuid*)

Bases: *NotificationPayloadBase*

SCHEMA = {'boot_index': ('target', 'boot_index'), 'created_at': ('target', 'created_at'), 'extra': ('target', 'extra'), 'properties': ('target', 'properties'), 'updated_at': ('target', 'updated_at'), 'uuid': ('target', 'uuid'), 'volume_id': ('target', 'volume_id'), 'volume_type': ('target', 'volume_type')}

VERSION = '1.0'

property boot_index

property created_at

property extra

```
fields = {'boot_index': Integer(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'created_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'extra': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'node_uuid': UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'properties': FlexibleDict(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'updated_at': DateTime(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True), 'uuid':
UUID(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=False),
'volume_id': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True),
'volume_type': String(default=<class
'oslo_versionedobjects.fields.UnspecifiedDefault'>,nullable=True)}

property node_uuid

property properties

property updated_at

property uuid

property volume_id

property volume_type
```

Module contents

ironic.objects.**register_all**()

ironic.pxe_filter package

Submodules

ironic.pxe_filter.dnsmasq module

ironic.pxe_filter.dnsmasq.**sync**(*allow_macs*, *deny_macs*, *allow_unknown*)

Conduct a complete sync of the state.

Unlike update, MACs not in either list are handled according to allow_unknown.

Parameters

- **allow_macs** MACs to allow in dnsmasq.
- **deny_macs** MACs to disallow in dnsmasq.
- **allow_unknown** Whether to allow access to dnsmasq to unknown MACs.

ironic.pxe_filter.dnsmasq.**update**(*allow_macs*, *deny_macs*, *allow_unknown=None*)

Update only the given MACs.

MACs not in either lists are ignored.

Parameters

- **allow_macs** MACs to allow in dnsmasq.
- **deny_macs** MACs to disallow in dnsmasq.
- **allow_unknown** If set to True, unknown MACs are also allowed. Setting it to False does nothing in this call.

ironic.pxe_filter.service module

class ironic.pxe_filter.service.**PXEFilterManager**(*host*)

Bases: object

del_host()

init_host(*admin_context*)

prepare_host()

topic = 'ironic.pxe_filter'

Module contents

ironic.wsgi package

Submodules

ironic.wsgi.api module

Module contents

Submodules

ironic.version module

Module contents

5.1.8 Understanding the Ironics CI

Its important to understand the role of each job in the CI, how to add new jobs and how to debug failures that may arise. To facilitate that, we have created the documentation below.

Jobs description

The description of each jobs that runs in the CI when you submit a patch for `openstack/ironic` is visible in *Table. OpenStack Ironic CI jobs description*.

Note

Ironic CI jobs are now documented using the description field in the job definition that they are created in.

Table 3: Table. OpenStack Ironic CI jobs description

Job name	Description
bifrost-integration-tinyipa-ubuntu-focal	Tests the integration between Ironic and Bifrost using a tinyipa image.
bifrost-integration-redfish-vmedia-uefi-centos-9	Tests the integration between Ironic and Bifrost using redfish vmedia and a dib image based on centos stream 9.
metal3-integration	Tests the integration between Ironic and Metal3 using the metal3-dev-env environment

Adding a new Job

Are you familiar with Zuul?

Before start trying to figure out how Zuul works, take some time and read about Zuul Config and the Zuul Best Practices.

Where can I find the existing jobs?

The jobs for the Ironic project are defined under the zuul.d folder in the root directory, that contains three files, whose function is described below.

- ironic-jobs.yaml: Contains the configuration of each Ironic Job converted to Zuul v3.
- legacy-ironic-jobs.yaml: Contains the configuration of each Ironic Job that havent been converted to Zuul v3 yet.
- project.yaml: Contains the jobs that will run during check and gate phase.

Create a new Job

Identify among the existing jobs the one that most closely resembles the scenario you want to test, the existing job will be used as `parent` in your job definition. Now you will only need to either overwrite or add variables to your job definition under the `vars` section to represent the desired scenario.

The code block below shows the minimal structure of a new job definition that you need to add to ironic-jobs.yaml.

```
- job:
    name: <name of the new job>
    description: <what your job does>
    parent: <Job that already exists>
    vars:
      <var1>: <new value>
```

After having the definition of your new job you just need to add the job name to the project.yaml under `check` and `gate`. Only jobs that are voting should be in the `gate` section.

```
- project:
    check:
      jobs:
        - <name of the new job>
```

(continues on next page)

(continued from previous page)

```
    gate:
      queue: ironic
      jobs:
        - <name of the new job>
```

Debugging CI failures

If you see `FAILURE` in one or more jobs for your patch please dont panic. This guide may help you to find the initial reason for the failure. When clicking in the failed job you will be redirect to the Zuul web page that contains all the information about the job build.

Zuul Web Page

The page has three tabs: `Summary`, `Logs` and `Console`.

- Summary: Contains overall information about the build of the job, if the job build failed it will contain a general output of the failure.
- Logs: Contains all configurations and log files about all services that were used in the job. This will give you an overall idea of the failures and you can identify services that may be involved. The `job-output` file can give an overall idea of the failures and what services may be involved.
- Console: Contains all the playbooks that were executed, by clicking in the arrow before each playbook name you can find the roles and commands that were executed.

Frequent Annoying Quirks (FAQ)

Networking (external/infra)

Ironic jobs, with more frequency than most, use external network resources and are more susceptible to failures caused by temporary connectivity issues.

Known issues may include:

- Failures building IPA images in any job ending in -src or jobs running against ironic-python-agent-builder. We should ensure the outage is resolved by testing the URL locally before rechceking.
- Unexpected/unexplainable failures in multinode may be caused by failed connectivity between two deployed devstack nodes. Any failures in the CI donor cloud causing network issues between two separate devstack VMs under coordinated test will cause failures.

Networking (OpenStack)

There are issues which can also cause networking failures to occur inside the job directly.

Known issues may include:

- Some dnsmasq versions have an issue which causes them to crash or segfault during automatic reconfiguration. Certain errors in the neutron dhcp agent (q-dhcp service in devstack) indicate that dnsmasq errored and must be respawned. One quick way to rule this out is to search `screen-q-dhcp.txt` for the string `'dnsmasq', '--no-hosts'` these spawn logs should only show up after an inability to find a PID file (Error: No such file or directory /opt/stack/data/neutron/UUID/pid). There should be no messages about respawning the process.
- EDK2 firmwares have known issues with IPv6, limiting our ability to test it.

Metal3 Continuous Integration

The Metal3 Continuous Integration job, named `metal3-integration` in the zuul configuration file, deploys a kubernetes cluster on emulated bare metal nodes, helping tests the ironic source code using the Metal3 Development Environment (metal3-dev-env) workflow from the Metal3 project.

Job configuration

While the metal3-integration job definition yaml file is in the same path of the other Ironic CI jobs, it uses separate configuration files under playbooks/metal3-ci in the Ironic repository. The configuration follows ansible syntax as for other CI jobs. The actual job is configured in run.yaml, where various environment variables for metal3-dev-env are defined under the `metal3_environment` entry. For more info about the metal3-dev-env environment variables definition and values please see the metal3-dev-env env variables page. In post.yaml we execute some post execution operations, like collecting logs and environment configuration, that are useful in case of troubleshooting.

Metal3 Development Environment Guide

To familiarize with the Metal3 Development Environment (metal3-dev-env), the Metal3 workflow, and in general with the project, its recommended to follow the TryIt section of the Metal3 User Guide. The metal3-dev-env workflow steps are explained in Section 1.2.

The Metal3 Development Environment (metal3-dev-env) is maintained by the Metal3 project community which is present in the `#cluster-api-baremetal` channel on Kubernetes Slack. For any questions or help on the project, or to escalate issues related to the `metal3-integration` job please contact the Metal3 community.

Troubleshooting Guide

The `metal3-integration` job logs are stored in the same way and following roughly the same path of the other Ironic CI jobs. In the main directory the `job-output.txt` file contains the console output of the job and, if any failure exists, the main reason of the breakage. Other useful logs are stored under the `controller` directory:

- `before_pivoting` directory stores services logs of the management cluster
- `libvirt` directory stores libvirt configuration and logs, including console logs of the emulated bare metal nodes
- `management_cluster` directory stores all configuration and logs of the metal3 services, such as the baremetal-operator (BMO)
- `system` directory stores information and logs from the operating system where metal3-dev-env is running

5.2 Contributing to Ironic Docs

This guide will help you get started with contributing to Ironic docs, whether you are fixing a typo or writing a new section.

5.2.1 Where the Docs Live

Ironic has multiple documentation sources located in different directories of the openstack/ironic repository.

- `doc/source/` Primary Ironic documentation
- `api-ref/source/` API Reference (https://docs.openstack.org/api-ref/baremetal/)
- `releasenotes/` Release notes for Ironic

5.2.2 How to Get Started

Clone the repository locally and create a new branch for your work.

For setup instructions, including creating a `Gerrit account` refer to the Developers Guide.

5.2.3 Make and Preview Documentation Changes

You can preview your documentation changes locally using the following command:

```
tox -e docs
```

This will build the HTML version in `doc/build/html/`. Open `index.html` in your browser to view your updates.

> **Note**
>
> If you have created a new file, make sure to include its path in the documentation table of contents.

To do this, edit `doc/source/index.rst` (or another relevant .rst file) and add the relative path of your new file to the `.. toctree::` directive. For example:

```
.. toctree::
   :maxdepth: 2

   contributor/doc_contributing
```

To open the generated HTML:

Option 1 Open the HTML file directly:

- On Linux:

```
xdg-open doc/build/html/index.html
```

- On macOS:

```
open doc/build/html/index.html
```

- On Windows:

Navigate to the `doc/build/html/` folder and double-click `index.html`.

Option 2 Start a local web server (Cross-platform):

```
cd doc/build/html
python -m http.server
```

Then navigate to URL.

5.2.4 Lint and Check Your Work

Before submitting, check your documentation and code formatting with:

```
tox -e pep8
```

This ensures `.rst` files follow style guidelines and catch common mistakes.

5.2.5 How to Submit Your Changes

After confirming your changes are correct, commit them with a clear message and submit for review with:

```
git review
```

Your patch will appear in the OpenStack Gerrit system for feedback and approval.

5.2.6 Writing Style and Guidelines

- Follow OpenStacks documentation style guide.
- Keep language clear, inclusive, and technical.
- Avoid passive voice and ambiguous words.
- Use reStructuredText (`.rst`) syntax.

5.2.7 Asking for Help

If you are stuck, dont hesitate to reach out to the ironic community.

Maintainers and review team information can be found in the `doc/OWNERS` file.

CHAPTER

SIX

SUPPORT RESOURCES

6.1 Support Resources

This section contains a table of the communitys platforms.

Resource Type	Ironic/OpenStack flavor	Common alternative
Code Repository	OpenDev	GitHub, GitLab
Code Contribution	Gerrit	Pull requests workflow (GitHub,GitLab), Email workflow (LKML)
Bug Tracking	Launchpad	GitHub issues, Jira
Chat	OFTC IRC	Slack
Video Conferencing	OpenDev MeetPad	Zoom

CHAPTER

SEVEN

REFERENCES

7.1 References

This section contains reference materials for Ironic, including API documentation, CLI tools, configuration details, and more.

7.1.1 Drivers, Hardware Types, and Hardware Interfaces

Drivers, Hardware Types, and Hardware Interfaces

7.1.2 Configuration Reference

Configuration Reference

7.1.3 API Reference

API Reference

7.1.4 CLI References

- Ironic CLI
- *ironic-dbsync*
- ironic-status

7.1.5 Trait Based Networking Reference

Trait Based Networking Configuration File Reference

Introduction

Trait Based Networkings trait configuration file is a YAML format file which defines a set of traits, and their corresponding actions. This file is ingested and validated by the Ironic conductor at start-up.

Trait Layout

Below is a valid YAML example trait:

```
CUSTOM_TRAIT_NAME:
  order: 1
  actions:
    - action: group_and_attach_ports
```

(continues on next page)

(continued from previous page)

```
    filter: port.vendor == 'vendor_string'
    min_count: 2
  - action: attach_port
    filter: port.vendor == 'vendor_string' && port.is_portgroup
    max_count: 1
```

`CUSTOM_TRAIT_NAME` is the traits name. Each trait is identified by a name which *must* start with `CUSTOM`. Its `order` is 1. Ordering is ascending, so lower orders will apply first.

`actions` is a list of actions to apply if this trait matches one defined in a nodes `instance_info.traits` field.

Each action has the following necessary keys:

- `action` - The action to take.
- `filter` - The Filter Expression to apply with this action.

Note

Refer to *Filter Expression Reference* for detailed explanations on how to write valid filter expressions.

and the following optional keys:

- `max_count` - The maximum number of objects that can match this action. There is no default maximum.
- `min_count` - The minimum number of objects that *must* match before this action applies. The default minimum is effectively 1.

Available Actions

The following actions are currently available:

- `attach_port` - Attach (port, network) pairs that pass this actions filter expression.
- `attach_portgroup` - Attach (portgroup, network) pairs that pass this actions filter expression.
- `group_and_attach_ports` - Select a set of ports. Create a dynamic portgroup comprised of the set of ports. Then attach the newly created dynamic portgroup to a suitable network. This action must set a `min_count` of at least 2. Also note that all ports selected for the portgroup must have the same `physical_network`.

Future actions are planned. This document will be updated as they become available.

Example Configuration File

An example Trait Based Networking configuration file is shipped with Ironic. A copy is available here. While backwards compatibility breaking changes are generally avoided where possible, please be aware that the linked copy may not be compatible with your version of Ironic.

Filter Expression Reference

This reference is for Trait Based Networkings Filter Expressions.

Note

If this document disagrees with `ironic.common.trait_based_networking.grammar.parser.FILTER_EXPRESSION_GRAMMAR` then this document is wrong. FILTER_EXPRESSION_GRAMMAR is the ultimate source of truth regarding the grammar and parsing of TBN filter expressions.

Filter Expressions

A filter expression is a boolean expression which evaluates to `True` if the objects under consideration match the filter, and `False` otherwise.

Filter expressions allow Ironic operators to create custom filtering logic for traits which will apply specific network actions or operations to nodes.

Filter expressions consider two basic network objects:

1. `portlike`: (aka `port` in this document) which can be either an Ironic port or portgroup.
2. `network`: Essentially a Neutron vif (virtual interface).

A filter expression that evaluates to `True` for a given tuple of `(portlike, network)` would cause a match to occur for the trait the filter belongs to. The traits defined actions would then apply if enough matches occur to satisfy the actions requirements.

Single Expression

A `single expression` has the form of:

```
variable_name comparator string_literal
```

Where `variable_name` is one of the available *variables*, `comparator` is one of the available *comparators*, and `string_literal` is a valid *string literal*.

A full example of a single expression:

```
port.category == 'public'
```

Which would evaluate to `True` whenever a portlike is considered that has a `category` that exactly equals `public`.

Function Expression

A `function expression` has the form of:

```
function
```

See *Functions* for available `functions`.

Compound Expression

A compound expression consists of two expressions joined by a *comparator*.

```
port.category == 'public' && port.vendor == 'green'
```

Parenthesis

Parenthesis can be used to group expressions together to guarantee evaluation precedence. For example:

```
port.category == 'private' || (port.vendor == 'purple' && network.name ==
↪'hypernet')
```

Would cause the right-hand side of `||` to be evaluated together before evaluating the result against the left side of `||`.

Comparators

Comparators allow comparisons between variables and string literals.

Comparator	Name	Explanation
==	Equality	Check for exact matches.
!=	Inequality	Check for any difference.
>=	Greater than or equal	Is the variable greater than or equal to the string literal?
>	Greater than	Is the variable greater than the string literal?
<=	Less than or equal	Is the variable less than or equal to the string literal?
<	Less than	Is the variable less than the string literal?
=~	Prefix match	Does the beginning of the variable match the string literal?

Examples

```
port.vendor == 'purple'
```

If a ports `vendor` is exactly `purple` then this expression will evaluate to `True` and `False` otherwise.

```
port.category =~ 'green'
```

If a ports `category` starts with the string `green` then this expression will evaluate to `True` and `False` otherwise.

```
network.name != 'private'
```

Match only networks if their name name is NOT `private`.

Boolean Operators

Used to join expressions to create complex filtering logic.

Operator	Name	Explanation
&&	And	If both expressions are `True`, then return `True`.
||	Or	If either expression is `True`, then return `True`.

Examples

```
port.vendor == 'purple' && port.category == 'private'
```

Will match a port if its `vendor` is `purple` and its category is `private`.

```
port.vendor == 'purple' || port.vendor == 'green'
```

Will match a port if its `vendor` is `purple` or `green`.

Functions

Functions allow basic querying of TBN related objects.

Function	Explanation
port.is_port	Returns `True` if the portlike under consideration is a `port`.
port.is_portgroup	Returns `True` if the portlike under consideration is a `portgroup`.

Examples

```
port.is_port
```

Will match portlikes which are a `port`.

String literal

String literals are enclosed by single quotes: `'`. String literals only allow alphanumeric characters, underscores, dashes, and periods.

The following regular expression encompasses valid string literals: `/\'[A-Za-z0-9_\-\.]*\'/`

Variables

Variables allow basic querying of network related objects in filter expressions. Available variables are listed below:

- network.name
- network.tags
- port.address
- port.category
- port.physical_network
- port.vendor

PYTHON MODULE INDEX

a

ironic.api, 903
ironic.api.app, 898
ironic.api.config, 899
ironic.api.controllers, 894
ironic.api.controllers.base, 893
ironic.api.controllers.link, 893
ironic.api.controllers.root, 894
ironic.api.controllers.v1, 892
ironic.api.controllers.v1.allocation, 833
ironic.api.controllers.v1.bios, 836
ironic.api.controllers.v1.chassis, 836
ironic.api.controllers.v1.collection, 838
ironic.api.controllers.v1.conductor, 839
ironic.api.controllers.v1.deploy_template, 839
ironic.api.controllers.v1.driver, 841
ironic.api.controllers.v1.event, 843
ironic.api.controllers.v1.firmware, 843
ironic.api.controllers.v1.inspection_rule, 844
ironic.api.controllers.v1.node, 845
ironic.api.controllers.v1.notification_utils, 859
ironic.api.controllers.v1.port, 859
ironic.api.controllers.v1.portgroup, 862
ironic.api.controllers.v1.ramdisk, 864
ironic.api.controllers.v1.runbook, 866
ironic.api.controllers.v1.shard, 867
ironic.api.controllers.v1.utils, 867
ironic.api.controllers.v1.versions, 887
ironic.api.controllers.v1.volume, 887
ironic.api.controllers.v1.volume_connector, 887
ironic.api.controllers.v1.volume_target, 890
ironic.api.controllers.version, 894
ironic.api.functions, 899
ironic.api.hooks, 900
ironic.api.method, 902
ironic.api.middleware, 895
ironic.api.middleware.auth_public_routes, 894
ironic.api.middleware.json_ext, 894
ironic.api.middleware.parsable_error, 895
ironic.api.middleware.request_log, 895
ironic.api.validation, 896
ironic.api.validation.validators, 896
ironic.api.wsgi, 902

c

ironic.command, 905
ironic.command.api, 903
ironic.command.conductor, 903
ironic.command.dbsync, 903
ironic.command.networking, 904
ironic.command.novncproxy, 904
ironic.command.pxe_filter, 904
ironic.command.singleprocess, 904
ironic.command.status, 904
ironic.command.utils, 905
ironic.common, 1022
ironic.common.args, 927
ironic.common.async_steps, 930
ironic.common.auth_basic, 931
ironic.common.automated_clean_methods, 933
ironic.common.boot_devices, 933
ironic.common.boot_modes, 934
ironic.common.checksum_utils, 934
ironic.common.cinder, 936
ironic.common.components, 939
ironic.common.config, 939
ironic.common.console_factory, 939
ironic.common.context, 939
ironic.common.dhcp_factory, 940
ironic.common.driver_factory, 941

ironic.common.exception, 944
ironic.common.faults, 958
ironic.common.fsm, 958
ironic.common.glance_service, 907
ironic.common.glance_service.image_service, 905
ironic.common.glance_service.service_utils, 907
ironic.common.hash_ring, 960
ironic.common.health_states, 960
ironic.common.i18n, 960
ironic.common.image_publisher, 960
ironic.common.image_service, 962
ironic.common.images, 971
ironic.common.indicator_states, 976
ironic.common.inspection_rules, 915
ironic.common.inspection_rules.actions, 907
ironic.common.inspection_rules.base, 910
ironic.common.inspection_rules.engine, 911
ironic.common.inspection_rules.operators, 911
ironic.common.inspection_rules.utils, 914
ironic.common.inspection_rules.validation, 915
ironic.common.json_rpc, 916
ironic.common.json_rpc.client, 915
ironic.common.json_rpc.server, 916
ironic.common.keystone, 977
ironic.common.kickstart_utils, 978
ironic.common.lessee_sources, 978
ironic.common.mdns, 979
ironic.common.metrics, 979
ironic.common.metrics_collector, 982
ironic.common.metrics_statsd, 983
ironic.common.metrics_utils, 983
ironic.common.molds, 983
ironic.common.network, 984
ironic.common.neutron, 986
ironic.common.nova, 992
ironic.common.oci_registry, 992
ironic.common.policy, 995
ironic.common.profiler, 996
ironic.common.pxe_utils, 996
ironic.common.qemu_img, 1002
ironic.common.raid, 1002
ironic.common.release_mappings, 1003
ironic.common.rpc, 1004
ironic.common.rpc_service, 1005
ironic.common.service, 1005
ironic.common.state_machine, 1005
ironic.common.states, 1006
ironic.common.swift, 1010
ironic.common.trait_based_networking, 927
ironic.common.trait_based_networking.base, 917
ironic.common.trait_based_networking.config_file, 924
ironic.common.trait_based_networking.defaults, 925
ironic.common.trait_based_networking.grammar, 917
ironic.common.trait_based_networking.grammar.parser, 916
ironic.common.trait_based_networking.loader, 925
ironic.common.trait_based_networking.plan, 925
ironic.common.utils, 1012
ironic.common.vnc, 1020
ironic.common.wsgi_service, 1021
ironic.conductor, 1084
ironic.conductor.allocations, 1022
ironic.conductor.base_manager, 1023
ironic.conductor.cleaning, 1024
ironic.conductor.configdrive_utils, 1025
ironic.conductor.deployments, 1028
ironic.conductor.inspection, 1030
ironic.conductor.manager, 1030
ironic.conductor.notification_utils, 1036
ironic.conductor.periodics, 1037
ironic.conductor.rpc_service, 1038
ironic.conductor.rpcapi, 1038
ironic.conductor.servicing, 1065
ironic.conductor.steps, 1066
ironic.conductor.task_manager, 1068
ironic.conductor.utils, 1071
ironic.conductor.verify, 1084
ironic.conf, 1089
ironic.conf.agent, 1084
ironic.conf.agent_containers, 1084
ironic.conf.anaconda, 1084
ironic.conf.ansible, 1084
ironic.conf.api, 1084
ironic.conf.audit, 1084
ironic.conf.auth, 1085

ironic.conf.cifs, 1085
ironic.conf.cinder, 1085
ironic.conf.conductor, 1085
ironic.conf.console, 1085
ironic.conf.database, 1085
ironic.conf.default, 1085
ironic.conf.deploy, 1085
ironic.conf.dhcp, 1086
ironic.conf.disk_utils, 1086
ironic.conf.dnsmasq, 1086
ironic.conf.drac, 1086
ironic.conf.exception, 1086
ironic.conf.fake, 1086
ironic.conf.glance, 1086
ironic.conf.healthcheck, 1086
ironic.conf.ilo, 1086
ironic.conf.inspector, 1086
ironic.conf.inventory, 1086
ironic.conf.ipmi, 1086
ironic.conf.ironic_networking, 1087
ironic.conf.json_rpc, 1087
ironic.conf.mdns, 1087
ironic.conf.metrics, 1087
ironic.conf.molds, 1087
ironic.conf.neutron, 1087
ironic.conf.nfs, 1088
ironic.conf.nova, 1088
ironic.conf.oci, 1088
ironic.conf.opts, 1088
ironic.conf.pxe, 1088
ironic.conf.redfish, 1088
ironic.conf.sensor_data, 1088
ironic.conf.service_catalog, 1088
ironic.conf.snmp, 1089
ironic.conf.swift, 1089
ironic.conf.types, 1089
ironic.conf.vnc, 1089
ironic.console, 1096
ironic.console.container, 1092
ironic.console.container.base, 1089
ironic.console.container.fake, 1090
ironic.console.container.kubernetes, 1091
ironic.console.container.systemd, 1092
ironic.console.novncproxy_service, 1095
ironic.console.rfb, 1094
ironic.console.rfb.auth, 1092
ironic.console.rfb.authnone, 1093
ironic.console.rfb.auths, 1094
ironic.console.securityproxy, 1095
ironic.console.securityproxy.base, 1094
ironic.console.securityproxy.rfb, 1095
ironic.console.websocketproxy, 1095

d

ironic.db, 1183
ironic.db.api, 1148
ironic.db.migration, 1183
ironic.db.sqlalchemy, 1148
ironic.db.sqlalchemy.api, 1096
ironic.db.sqlalchemy.migration, 1134
ironic.db.sqlalchemy.models, 1135
ironic.dhcp, 1189
ironic.dhcp.base, 1183
ironic.dhcp.dnsmasq, 1185
ironic.dhcp.neutron, 1186
ironic.dhcp.none, 1188
ironic.drivers, 1434
ironic.drivers.base, 1389
ironic.drivers.drac, 1424
ironic.drivers.fake_hardware, 1425
ironic.drivers.generic, 1426
ironic.drivers.hardware_type, 1427
ironic.drivers.ilo, 1428
ironic.drivers.intel_ipmi, 1429
ironic.drivers.ipmi, 1429
ironic.drivers.modules, 1389
ironic.drivers.modules.agent, 1296
ironic.drivers.modules.agent_base, 1303
ironic.drivers.modules.agent_client, 1310
ironic.drivers.modules.agent_power, 1318
ironic.drivers.modules.ansible, 1191
ironic.drivers.modules.ansible.deploy, 1189
ironic.drivers.modules.autodetect, 1319
ironic.drivers.modules.boot_mode_utils, 1320
ironic.drivers.modules.console_utils, 1322
ironic.drivers.modules.deploy_utils, 1323
ironic.drivers.modules.drac, 1197
ironic.drivers.modules.drac.bios, 1191
ironic.drivers.modules.drac.boot, 1192
ironic.drivers.modules.drac.inspect, 1192
ironic.drivers.modules.drac.management, 1193
ironic.drivers.modules.drac.power, 1195
ironic.drivers.modules.drac.raid, 1195

ironic.drivers.modules.drac.utils, 1196
ironic.drivers.modules.drac.vendor_passthru, 1197
ironic.drivers.modules.fake, 1333
ironic.drivers.modules.graphical_console, 1350
ironic.drivers.modules.ilo, 1226
ironic.drivers.modules.ilo.bios, 1197
ironic.drivers.modules.ilo.boot, 1198
ironic.drivers.modules.ilo.common, 1205
ironic.drivers.modules.ilo.firmware_processor, 1213
ironic.drivers.modules.ilo.inspect, 1214
ironic.drivers.modules.ilo.management, 1215
ironic.drivers.modules.ilo.power, 1222
ironic.drivers.modules.ilo.raid, 1224
ironic.drivers.modules.ilo.vendor, 1225
ironic.drivers.modules.image_cache, 1351
ironic.drivers.modules.image_utils, 1352
ironic.drivers.modules.inspect_utils, 1356
ironic.drivers.modules.inspector, 1238
ironic.drivers.modules.inspector.agent, 1234
ironic.drivers.modules.inspector.hooks, 1234
ironic.drivers.modules.inspector.hooks.accelerators, 1226
ironic.drivers.modules.inspector.hooks.architecture, 1226
ironic.drivers.modules.inspector.hooks.base, 1226
ironic.drivers.modules.inspector.hooks.boot_mode, 1227
ironic.drivers.modules.inspector.hooks.cpu_capabilities, 1228
ironic.drivers.modules.inspector.hooks.cpus, 1228
ironic.drivers.modules.inspector.hooks.extra_hardware, 1229
ironic.drivers.modules.inspector.hooks.local_link_connection, 1229
ironic.drivers.modules.inspector.hooks.memory, 1229
ironic.drivers.modules.inspector.hooks.parse_lldp, 1229
ironic.drivers.modules.inspector.hooks.pci_devices, 1230
ironic.drivers.modules.inspector.hooks.physical_network, 1230
ironic.drivers.modules.inspector.hooks.ports, 1230
ironic.drivers.modules.inspector.hooks.raid_device, 1231
ironic.drivers.modules.inspector.hooks.ramdisk_error, 1232
ironic.drivers.modules.inspector.hooks.root_device, 1233
ironic.drivers.modules.inspector.hooks.validate_interface, 1233
ironic.drivers.modules.inspector.interface, 1234
ironic.drivers.modules.inspector.lldp_parsers, 1235
ironic.drivers.modules.inspector.lldp_tlvs, 1237
ironic.drivers.modules.intel_ipmi, 1239
ironic.drivers.modules.intel_ipmi.management, 1238
ironic.drivers.modules.ipmitool, 1358
ironic.drivers.modules.ipxe, 1365
ironic.drivers.modules.network, 1257
ironic.drivers.modules.network.common, 1239
ironic.drivers.modules.network.flat, 1243
ironic.drivers.modules.network.ironic_networking, 1245
ironic.drivers.modules.network.ironic_networking_sc, 1250
ironic.drivers.modules.network.neutron, 1250
ironic.drivers.modules.network.noop, 1254
ironic.drivers.modules.network.switchport_config, 1255
ironic.drivers.modules.noop, 1366
ironic.drivers.modules.noop_mgmt, 1372
ironic.drivers.modules.pxe, 1375
ironic.drivers.modules.pxe_base, 1376
ironic.drivers.modules.ramdisk, 1378
ironic.drivers.modules.redfish, 1285
ironic.drivers.modules.redfish.bios, 1257
ironic.drivers.modules.redfish.boot, 1259
ironic.drivers.modules.redfish.firmware, 1264

ironic.drivers.modules.redfish.firmware_utils, 1265
ironic.drivers.modules.redfish.graphical_console, 1266
ironic.drivers.modules.redfish.inspect, 1267
ironic.drivers.modules.redfish.management, 1268
ironic.drivers.modules.redfish.power, 1276
ironic.drivers.modules.redfish.raid, 1277
ironic.drivers.modules.redfish.utils, 1281
ironic.drivers.modules.redfish.vendor, 1284
ironic.drivers.modules.snmp, 1380
ironic.drivers.modules.storage, 1289
ironic.drivers.modules.storage.cinder, 1285
ironic.drivers.modules.storage.external, 1287
ironic.drivers.modules.storage.noop, 1288
ironic.drivers.modules.switch, 1296
ironic.drivers.modules.switch.base, 1289
ironic.drivers.modules.switch.generic_switch, 1293
ironic.drivers.redfish, 1429
ironic.drivers.snmp, 1430
ironic.drivers.utils, 1430

i

ironic, 1554

n

ironic.networking, 1443
ironic.networking.api, 1436
ironic.networking.manager, 1438
ironic.networking.rpc_service, 1439
ironic.networking.rpcapi, 1439
ironic.networking.switch_config, 1442
ironic.networking.switch_drivers, 1436
ironic.networking.switch_drivers.driver_adapter, 1434
ironic.networking.switch_drivers.driver_factory, 1435
ironic.networking.utils, 1443

o

ironic.objects, 1553
ironic.objects.allocation, 1443
ironic.objects.base, 1449
ironic.objects.bios, 1451
ironic.objects.chassis, 1456
ironic.objects.conductor, 1460
ironic.objects.deploy_template, 1462
ironic.objects.deployment, 1467
ironic.objects.fields, 1470
ironic.objects.firmware, 1473
ironic.objects.indirection, 1475
ironic.objects.inspection_rule, 1476
ironic.objects.node, 1481
ironic.objects.node_history, 1513
ironic.objects.node_inventory, 1516
ironic.objects.notification, 1517
ironic.objects.port, 1519
ironic.objects.portgroup, 1528
ironic.objects.runbook, 1535
ironic.objects.trait, 1540
ironic.objects.volume_connector, 1542
ironic.objects.volume_target, 1547

p

ironic.pxe_filter, 1554
ironic.pxe_filter.dnsmasq, 1553
ironic.pxe_filter.service, 1554

v

ironic.version, 1554

w

ironic.wsgi, 1554
ironic.wsgi.api, 1554

INDEX

Symbols

__call__() (*ironic.api.app.VersionSelectorApplication method*), 898
__call__() (*ironic.api.functions.signature method*), 900
__call__() (*ironic.api.middleware.AuthPublicRoutes method*), 895
__call__() (*ironic.api.middleware.JsonExtensionMiddleware method*), 895
__call__() (*ironic.api.middleware.ParsableErrorMiddleware method*), 896
__call__() (*ironic.api.middleware.auth_public_routes.AuthPublicRoutes method*), 894
__call__() (*ironic.api.middleware.json_ext.JsonExtensionMiddleware method*), 894
__call__() (*ironic.api.middleware.parsable_error.ParsableErrorMiddleware method*), 895
__call__() (*ironic.api.middleware.request_log.RequestLogMiddleware method*), 895
__call__() (*ironic.api.validation.Schemas method*), 896
__call__() (*ironic.common.auth_basic.BasicAuthMiddleware method*), 931
__call__() (*ironic.common.inspection_rules.actions.ActionBase method*), 908
__call__() (*ironic.common.inspection_rules.actions.AddTraitAction method*), 908
__call__() (*ironic.common.inspection_rules.actions.CallAPIHookAction method*), 908
__call__() (*ironic.common.inspection_rules.actions.DelAttributeAction method*), 908
__call__() (*ironic.common.inspection_rules.actions.DelPortAttributeAction method*), 908
__call__() (*ironic.common.inspection_rules.actions.ExtendAttributeAction method*), 908
__call__() (*ironic.common.inspection_rules.actions.ExtendPluginDataAction method*), 909
__call__() (*ironic.common.inspection_rules.actions.ExtendPortAttributeAction method*), 909
__call__() (*ironic.common.inspection_rules.actions.FailAction method*), 909
__call__() (*ironic.common.inspection_rules.actions.LogAction method*), 909
__call__() (*ironic.common.inspection_rules.actions.RemoveTrait method*), 909
__call__() (*ironic.common.inspection_rules.actions.SetAttribute method*), 909
__call__() (*ironic.common.inspection_rules.actions.SetCapability method*), 909
__call__() (*ironic.common.inspection_rules.actions.SetPluginData method*), 910
__call__() (*ironic.common.inspection_rules.actions.SetPortAttribute method*), 910
__call__() (*ironic.common.inspection_rules.actions.UnsetCapability method*), 910
__call__() (*ironic.common.inspection_rules.actions.UnsetPluginData method*), 910
__call__() (*ironic.common.inspection_rules.operators.ContainsOperator method*), 911
__call__() (*ironic.common.inspection_rules.operators.EmptyOperator method*), 911
__call__() (*ironic.common.inspection_rules.operators.IsFalseOperator method*), 912
__call__() (*ironic.common.inspection_rules.operators.IsNoneOperator method*), 912
__call__() (*ironic.common.inspection_rules.operators.IsTrueOperator method*), 912
__call__() (*ironic.common.inspection_rules.operators.MatchesOperator method*), 912
__call__() (*ironic.common.inspection_rules.operators.NetOperatorMixin method*), 913
__call__() (*ironic.common.inspection_rules.operators.OneOfOperator method*), 913
__call__() (*ironic.common.inspection_rules.operators.OperatorBase method*), 913
__call__() (*ironic.common.inspection_rules.operators.SimpleOperator method*), 913
__call__() (*ironic.common.metrics.Counter method*), 980
__call__() (*ironic.common.metrics.Gauge method*), 980
__call__() (*ironic.common.metrics.Timer*

method), 982
__call__() (ironic.conf.api.Octal method), 1084
__call__() (ironic.conf.console.PortRange method), 1085
__call__() (ironic.conf.types.ExplicitAbsolutePath method), 1089
__call__() (ironic.drivers.modules.inspector.hooks.accelerators.AcceleratorsHook method), 1226
__call__() (ironic.drivers.modules.inspector.hooks.architecture.ArchitectureHook method), 1226
__call__() (ironic.drivers.modules.inspector.hooks.base_inspection.MandatoryHook method), 1226
__call__() (ironic.drivers.modules.inspector.hooks.boot_mode.BootModeHook method), 1227
__call__() (ironic.drivers.modules.inspector.hooks.cpu_capabilities.CPUCapabilitiesHook method), 1228
__call__() (ironic.drivers.modules.inspector.hooks.cpus.CPUsHook method), 1228
__call__() (ironic.drivers.modules.inspector.hooks.extra_hardware.ExtraHardwareHook method), 1229
__call__() (ironic.drivers.modules.inspector.hooks.local_link_connection.LocalLinkConnectionHook method), 1229
__call__() (ironic.drivers.modules.inspector.hooks.memory.MemoryHook method), 1229
__call__() (ironic.drivers.modules.inspector.hooks.parse_lldp.ParseLLDPHook method), 1229
__call__() (ironic.drivers.modules.inspector.hooks.pci_devices.PciDevicesHook method), 1230
__call__() (ironic.drivers.modules.inspector.hooks.physical_network.PhysicalNetworkHook method), 1230
__call__() (ironic.drivers.modules.inspector.hooks.ports.PortsHook method), 1230
__call__() (ironic.drivers.modules.inspector.hooks.raid_device.RaidDeviceHook method), 1231
__call__() (ironic.drivers.modules.inspector.hooks.ramdisk_error.RamdiskErrorHook method), 1232
__call__() (ironic.drivers.modules.inspector.hooks.root_device.RootDeviceHook method), 1233
__call__() (ironic.drivers.modules.inspector.hooks.validate_interfaces.ValidateInterfacesHook method), 1233
--autogenerate
revision command line option, 458
--config-dir
ironic-dbsync command line option, 456
--config-file
ironic-dbsync command line option, 456
--debug
ironic-dbsync command line option, 456
--help
create_schema command line option, 457
ironic-dbsync command line option, 456
online_data_migrations command line option, 457
revision command line option, 458
stamp command line option, 458
upgrade command line option, 458
version command line option, 459
--max-count
online_data_migrations command line option, 457
--message
revision command line option, 458
--online
online_data_migrations command line option, 457
--revision
stamp command line option, 458
upgrade command line option, 458
--version
ironic-dbsync command line option, 456
-d
ironic-dbsync command line option, 456
-h
create_schema command line option, 457
ironic-dbsync command line option, 456
online_data_migrations command line option, 457
revision command line option, 458
stamp command line option, 458
upgrade command line option, 458
version command line option, 459
-m
revision command line option, 458

A

abort() (ironic.drivers.base.InspectInterface method), 1401
abort() (ironic.drivers.modules.inspector.agent.AgentInspect method), 1234
abort() (ironic.drivers.modules.inspector.AgentInspect method), 1238
abort_inspection() (in module ironic.conductor.inspection), 1030

abort_on_conductor_take_over() (*in module ironic.conductor.utils*), 1071
aboveUpperCritical (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1384
aboveUpperWarning (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1384
AbstractHardwareType (*class in ironic.drivers.hardware_type*), 1427
AbstractPublisher (*class in ironic.common.image_publisher*), 960
AcceleratorsHook (*class in ironic.drivers.modules.inspector.hooks.accelerators*), 1226
acquire() (*in module ironic.conductor.task_manager*), 1071
acquire_port() (*in module ironic.drivers.modules.console_utils*), 1322
action (*ironic.common.trait_based_networking.base.Command attribute*), 921
action (*ironic.common.trait_based_networking.base.Trait attribute*), 923
action (*ironic.objects.notification.EventType property*), 1517
ActionBase (*class in ironic.common.inspection_rules.actions*), 907
Actions (*class in ironic.common.trait_based_networking.base*), 917
actions (*ironic.db.sqlalchemy.models.InspectionRule attribute*), 1138
actions (*ironic.objects.inspection_rule.InspectionRule property*), 1476
actions (*ironic.objects.inspection_rule.InspectionRuleCRUDPayload property*), 1480
activate_license() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1216
ACTIVE (*in module ironic.common.states*), 1006
add_allocation_filter_by_conductor() (*in module ironic.db.sqlalchemy.api*), 1132
add_allocation_filter_by_node() (*in module ironic.db.sqlalchemy.api*), 1132
add_auth_opts() (*in module ironic.common.keystone*), 977
add_auth_opts() (*in module ironic.conf.auth*), 1085
add_capabilities() (*ironic.drivers.modules.inspector.lldp_parsers.LLDPBasic method*), 1235
add_certificates() (*in module ironic.drivers.modules.ilo.common*), 1206
add_cleaning_network() (*ironic.drivers.base.NetworkInterface method*), 1409
add_cleaning_network() (*ironic.drivers.modules.network.flat.FlatNetwork method*), 1243
add_cleaning_network() (*ironic.drivers.modules.network.ironic_networking.IronicN method*), 1246
add_cleaning_network() (*ironic.drivers.modules.network.neutron.NeutronNetwork method*), 1250
add_cleaning_network() (*ironic.drivers.modules.network.noop.NoopNetwork method*), 1254
add_command_parsers() (*in module ironic.command.dbsync*), 904
add_dot1_link_aggregation() (*ironic.drivers.modules.inspector.lldp_parsers.LLDPParse method*), 1236
add_dot1_port_protocol_vlan() (*ironic.drivers.modules.inspector.lldp_parsers.LLDPdot11 method*), 1237
add_dot1_protocol_identities() (*ironic.drivers.modules.inspector.lldp_parsers.LLDPdot11 method*), 1237
add_dot1_vlans() (*ironic.drivers.modules.inspector.lldp_parsers.LLDPdot11 method*), 1237
add_dot3_macphy_config() (*ironic.drivers.modules.inspector.lldp_parsers.LLDPdot31 method*), 1237
add_fieldname() (*ironic.common.exception.UnknownAttribute method*), 957
add_https_certificate() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1216
add_identity_filter() (*in module ironic.db.sqlalchemy.api*), 1132
add_identity_where() (*in module ironic.db.sqlalchemy.api*), 1132
add_inspection_network() (*ironic.drivers.base.NetworkInterface method*), 1410
add_inspection_network()

(*ironic.drivers.modules.network.flat.FlatNetwork method*), 1243
add_inspection_network() (*ironic.drivers.modules.network.ironic_networking.IronicNetwork method*), 1246
add_inspection_network() (*ironic.drivers.modules.network.neutron.NeutronNetwork method*), 1251
add_mgmt_address() (*ironic.drivers.modules.inspector.lldp_parsers.LLDPBasicMgmtParser method*), 1235
add_nested_value() (*ironic.drivers.modules.inspector.lldp_parsers.LLDPParser method*), 1236
add_node_capability() (*in module ironic.drivers.utils*), 1431
add_node_filter_by_chassis() (*in module ironic.db.sqlalchemy.api*), 1132
add_node_tag() (*ironic.db.api.Connection method*), 1148
add_node_tag() (*ironic.db.sqlalchemy.api.Connection method*), 1096
add_node_trait() (*ironic.db.api.Connection method*), 1148
add_node_trait() (*ironic.db.sqlalchemy.api.Connection method*), 1097
add_node_traits() (*ironic.conductor.manager.ConductorManager method*), 1030
add_node_traits() (*ironic.conductor.rpcapi.ConductorAPI method*), 1040
add_port_filter() (*in module ironic.db.sqlalchemy.api*), 1132
add_port_filter_by_node() (*in module ironic.db.sqlalchemy.api*), 1133
add_port_filter_by_node_conductor_groups() (*in module ironic.db.sqlalchemy.api*), 1133
add_port_filter_by_node_owner() (*in module ironic.db.sqlalchemy.api*), 1133
add_port_filter_by_node_project() (*in module ironic.db.sqlalchemy.api*), 1133
add_port_filter_by_portgroup() (*in module ironic.db.sqlalchemy.api*), 1133
add_port_filter_description_contains() (*in module ironic.db.sqlalchemy.api*), 1133
add_portgroup_filter() (*in module ironic.db.sqlalchemy.api*), 1133
add_portgroup_filter_by_node() (*in module ironic.db.sqlalchemy.api*), 1133
add_portgroup_filter_by_node_conductor_groups() (*in module ironic.db.sqlalchemy.api*), 1133
add_portgroup_filter_by_node_project() (*in module ironic.db.sqlalchemy.api*), 1133
add_port_mac_mgmt_parser() (*in module ironic.drivers.modules.inspector.hooks.ports*), 1231
add_port_to_network() (*in module ironic.common.neutron*), 986
add_provisioning_network() (*ironic.drivers.base.NetworkInterface method*), 1410
add_provisioning_network() (*ironic.drivers.modules.network.flat.FlatNetwork method*), 1243
add_provisioning_network() (*ironic.drivers.modules.network.ironic_networking.IronicNetwork method*), 1246
add_provisioning_network() (*ironic.drivers.modules.network.neutron.NeutronNetwork method*), 1251
add_provisioning_network() (*ironic.drivers.modules.network.noop.NoopNetwork method*), 1254
add_rescuing_network() (*ironic.drivers.base.NetworkInterface method*), 1410
add_rescuing_network() (*ironic.drivers.modules.network.flat.FlatNetwork method*), 1243
add_rescuing_network() (*ironic.drivers.modules.network.ironic_networking.IronicNetwork method*), 1246
add_rescuing_network() (*ironic.drivers.modules.network.neutron.NeutronNetwork method*), 1251
add_schema() (*ironic.api.validation.Schemas method*), 896
add_secret_token() (*in module ironic.conductor.utils*), 1072
add_servicing_network() (*ironic.drivers.base.NetworkInterface method*), 1410
add_servicing_network() (*ironic.drivers.modules.network.flat.FlatNetwork method*), 1243

add_servicing_network() (*ironic.drivers.modules.network.ironic_networking.IronicNetwork method*), 1246
add_servicing_network() (*ironic.drivers.modules.network.neutron.NeutronNetwork method*), 1251
add_single_value() (*ironic.drivers.modules.inspector.lldp_parsers.LLDPParser method*), 1236
add_state() (*ironic.common.fsm.FSM method*), 958
add_transition() (*ironic.common.fsm.FSM method*), 959
add_version_attributes() (*ironic.api.controllers.v1.Controller method*), 893
add_volume_conn_filter_by_node_project() (*in module ironic.db.sqlalchemy.api*), 1133
add_volume_target_filter_by_node_project() (*in module ironic.db.sqlalchemy.api*), 1133
address (*ironic.common.trait_based_networking.base.PrimordialPort attribute*), 922
address (*ironic.db.sqlalchemy.models.Port attribute*), 1145
address (*ironic.db.sqlalchemy.models.Portgroup attribute*), 1145
address (*ironic.objects.port.Port property*), 1519
address (*ironic.objects.port.PortCRUDPayload property*), 1527
address (*ironic.objects.portgroup.Portgroup property*), 1528
address (*ironic.objects.portgroup.PortgroupCRUDPayload property*), 1534
AddTraitAction (*class in ironic.common.inspection_rules.actions*), 908
ADOPTFAIL (*in module ironic.common.states*), 1006
ADOPTING (*in module ironic.common.states*), 1006
advanced_net_fields (*ironic.api.controllers.v1.port.PortsController attribute*), 859
after() (*ironic.api.hooks.ContextHook method*), 901
after() (*ironic.api.hooks.DBHook method*), 901
after() (*ironic.api.hooks.NoExceptionTracebackHook method*), 901
agent_add_clean_params() (*in module ironic.drivers.modules.deploy_utils*), 1323
agent_is_alive() (*in module ironic.conductor.utils*), 1072
AgentAPIError, 944
AgentBaseMixin (*class in ironic.drivers.modules.agent_base*), 1303
AgentClient (*class in ironic.drivers.modules.agent_client*), 1310
AgentCommandTimeout, 944
AgentConnectionFailed, 944
AgentDeploy (*class in ironic.drivers.modules.agent*), 1296
AgentInProgress, 944
AgentInspect (*class in ironic.drivers.modules.inspector*), 1238
AgentInspect (*class in ironic.drivers.modules.inspector.agent*), 1234
AgentOobStepsMixin (*class in ironic.drivers.modules.agent_base*), 1307
AgentPower (*class in ironic.drivers.modules.agent_power*), 1318
AgentRAID (*class in ironic.drivers.modules.agent*), 1297
AgentRescue (*class in ironic.drivers.modules.agent*), 1298
alarmed (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1384
ALL (*ironic.objects.fields.NotificationLevel attribute*), 1471
ALL (*ironic.objects.fields.NotificationStatus attribute*), 1472
ALL_INTERFACES (*in module ironic.drivers.base*), 1389
all_interfaces (*ironic.drivers.base.BareDriver property*), 1391
all_interfaces() (*in module ironic.common.driver_factory*), 942
ALL_KEYS (*ironic.common.trait_based_networking.base.TraitAction attribute*), 923
all_no_match() (*in module ironic.common.trait_based_networking.plan*), 925
all_steps_disable_ramdisk() (*in module ironic.conductor.steps*), 1066
all_versions() (*in module ironic.api.controllers.version*), 894

Allocation (*class in ironic.db.sqlalchemy.models*), 1135
Allocation (*class in ironic.objects.allocation*), 1443
allocation_id (*ironic.db.sqlalchemy.models.Node attribute*), 1139
allocation_id (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1141
allocation_id (*ironic.objects.node.Node property*), 1481
allocation_sanitize() (*in module ironic.api.controllers.v1.allocation*), 836
AllocationAlreadyExists, 944
AllocationCRUDNotification (*class in ironic.objects.allocation*), 1447
AllocationCRUDPayload (*class in ironic.objects.allocation*), 1447
AllocationDuplicateName, 944
AllocationFailed, 944
AllocationNotFound, 944
AllocationsController (*class in ironic.api.controllers.v1.allocation*), 833
allow_agent_token() (*in module ironic.api.controllers.v1.utils*), 867
allow_agent_version_in_heartbeat() (*in module ironic.api.controllers.v1.utils*), 867
allow_allocation_backfill() (*in module ironic.api.controllers.v1.utils*), 868
allow_allocation_owner() (*in module ironic.api.controllers.v1.utils*), 868
allow_allocations() (*in module ironic.api.controllers.v1.utils*), 868
allow_attach_detach_vmedia() (*in module ironic.api.controllers.v1.utils*), 868
allow_bios_interface() (*in module ironic.api.controllers.v1.utils*), 868
allow_build_configdrive() (*in module ironic.api.controllers.v1.utils*), 868
allow_configdrive_vendor_data() (*in module ironic.api.controllers.v1.utils*), 868
allow_continue_inspection_endpoint() (*in module ironic.api.controllers.v1.utils*), 868
allow_deploy_abort() (*in module ironic.api.controllers.v1.utils*), 868
allow_deploy_steps() (*in module ironic.api.controllers.v1.utils*), 868
allow_deploy_templates() (*in module ironic.api.controllers.v1.utils*), 868
allow_detail_query() (*in module ironic.api.controllers.v1.utils*), 868
allow_dynamic_drivers() (*in module ironic.api.controllers.v1.utils*), 869
allow_dynamic_interfaces() (*in module ironic.api.controllers.v1.utils*), 869
allow_expose_conductors() (*in module ironic.api.controllers.v1.utils*), 869
allow_expose_events() (*in module ironic.api.controllers.v1.utils*), 869
allow_field() (*in module ironic.api.controllers.v1.utils*), 869
allow_firmware_interface() (*in module ironic.api.controllers.v1.utils*), 869
allow_get_vmedia() (*in module ironic.api.controllers.v1.utils*), 869
allow_inject_nmi() (*in module ironic.api.controllers.v1.utils*), 869
allow_inspect_abort() (*in module ironic.api.controllers.v1.utils*), 869
allow_inspect_wait_state() (*in module ironic.api.controllers.v1.utils*), 869
allow_inspection_rules() (*in module ironic.api.controllers.v1.utils*), 869
allow_links_node_states_and_driver_properties() (*in module ironic.api.controllers.v1.utils*), 869
allow_local_link_connection_network_type() (*in module ironic.api.controllers.v1.utils*), 870
allow_node_history() (*in module ironic.api.controllers.v1.utils*), 870
allow_node_ident_as_param_for_port_creation() (*in module ironic.api.controllers.v1.utils*), 870
allow_node_instance_name() (*in module ironic.api.controllers.v1.utils*), 870
allow_node_inventory() (*in module ironic.api.controllers.v1.utils*), 870
allow_node_logical_names() (*in module ironic.api.controllers.v1.utils*), 870
allow_node_rebuild_with_configdrive() (*in module ironic.api.controllers.v1.utils*), 870
allow_ovn_vtep_version() (*in module ironic.api.controllers.v1.utils*), 870
allow_port_advanced_net_fields() (*in module ironic.api.controllers.v1.utils*), 870
allow_port_available_for_dynamic_portgroup()

(*in module ironic.api.controllers.v1.utils*), 870
allow_port_category() (*in module ironic.api.controllers.v1.utils*), 870
allow_port_description() (*in module ironic.api.controllers.v1.utils*), 870
allow_port_internal_info() (*in module ironic.api.controllers.v1.utils*), 870
allow_port_is_smartnic() (*in module ironic.api.controllers.v1.utils*), 870
allow_port_name() (*in module ironic.api.controllers.v1.utils*), 871
allow_port_physical_network() (*in module ironic.api.controllers.v1.utils*), 871
allow_port_vendor() (*in module ironic.api.controllers.v1.utils*), 871
allow_portgroup_category() (*in module ironic.api.controllers.v1.utils*), 871
allow_portgroup_dynamic_portgroup() (*in module ironic.api.controllers.v1.utils*), 871
allow_portgroup_mode_properties() (*in module ironic.api.controllers.v1.utils*), 871
allow_portgroup_physical_network() (*in module ironic.api.controllers.v1.utils*), 871
allow_portgroups() (*in module ironic.api.controllers.v1.utils*), 871
allow_portgroups_subcontrollers() (*in module ironic.api.controllers.v1.utils*), 871
allow_query_bios() (*in module ironic.api.controllers.v1.utils*), 871
allow_raid_config() (*in module ironic.api.controllers.v1.utils*), 871
allow_ramdisk_endpoints() (*in module ironic.api.controllers.v1.utils*), 871
allow_remove_chassis_uuid() (*in module ironic.api.controllers.v1.utils*), 872
allow_rescue_interface() (*in module ironic.api.controllers.v1.utils*), 872
allow_reset_interfaces() (*in module ironic.api.controllers.v1.utils*), 872
allow_runbooks() (*in module ironic.api.controllers.v1.utils*), 872
allow_service_verb() (*in module ironic.api.controllers.v1.utils*), 872
allow_shards_endpoint() (*in module ironic.api.controllers.v1.utils*), 872
allow_soft_power_off() (*in module ironic.api.controllers.v1.utils*), 872
allow_special_characters_in_patch_fields() (*in module ironic.api.controllers.v1.utils*), 872
allow_status_in_heartbeat() (*in module ironic.api.controllers.v1.utils*), 872
allow_storage_interface() (*in module ironic.api.controllers.v1.utils*), 872
allow_traits() (*in module ironic.api.controllers.v1.utils*), 872
allow_unhold_verb() (*in module ironic.api.controllers.v1.utils*), 872
allow_verify_ca_in_heartbeat() (*in module ironic.api.controllers.v1.utils*), 872
allow_vifs_subcontroller() (*in module ironic.api.controllers.v1.utils*), 872
allow_volume() (*in module ironic.api.controllers.v1.utils*), 873
allowable_values (*ironic.db.sqlalchemy.models.BIOSSetting attribute*), 1135
allowable_values (*ironic.objects.bios.BIOSSetting property*), 1452
allowed_exception_namespaces (*ironic.common.json_rpc.client.Client attribute*), 915
allowed_vlans (*ironic.drivers.modules.network.switchport_config attribute*), 1256
AND (*ironic.common.trait_based_networking.base.Operator attribute*), 921
and_valid() (*in module ironic.common.args*), 927
AnsibleDeploy (*class in ironic.drivers.modules.ansible.deploy*), 1189
api_version() (*in module ironic.api.validation*), 896
append_value() (*ironic.drivers.modules.inspector.lldp_parsers.LLDPParser method*), 1236
apply_actions() (*in module ironic.common.inspection_rules.engine*), 911
apply_automatic_lessee() (*in module ironic.conductor.deployments*), 1028
apply_configuration() (*ironic.drivers.base.BIOSInterface method*), 1389
apply_configuration() (*ironic.drivers.base.RAIDInterface*

method), 1416
apply_configuration() (*ironic.drivers.modules.agent.AgentRAID method*), 1297
apply_configuration() (*ironic.drivers.modules.fake.FakeBIOS method*), 1333
apply_configuration() (*ironic.drivers.modules.ilo.bios.IloBIOS method*), 1197
apply_configuration() (*ironic.drivers.modules.ilo.raid.Ilo5RAID method*), 1224
apply_configuration() (*ironic.drivers.modules.noop.NoBIOS method*), 1366
apply_configuration() (*ironic.drivers.modules.redfish.bios.RedfishBIOS method*), 1257
apply_configuration() (*ironic.drivers.modules.redfish.raid.RedfishRAID method*), 1277
apply_jsonpatch() (*in module ironic.api.controllers.v1.utils*), 873
apply_rules() (*in module ironic.common.inspection_rules.engine*), 911
ArchitectureHook (*class in ironic.drivers.modules.inspector.hooks.architecture*), 1226
ARD (*ironic.console.rfb.auth.AuthType attribute*), 1092
args (*ironic.db.sqlalchemy.models.DeployTemplateStep attribute*), 1137
args (*ironic.db.sqlalchemy.models.RunbookStep attribute*), 1146
arguments (*ironic.api.functions.FunctionDefinition attribute*), 899
as_dict() (*ironic.db.sqlalchemy.models.IronicBase method*), 1138
as_dict() (*ironic.objects.base.IronicObject method*), 1449
as_dict() (*ironic.objects.base.IronicObjectListBase method*), 1450
as_dict() (*ironic.objects.node.Node method*), 1481
ATTACH_PORT (*ironic.common.trait_based_networking.base.Actions attribute*), 917
ATTACH_PORTGROUP (*ironic.common.trait_based_networking.base.Actions attribute*), 917
attach_virtual_media() (*ironic.conductor.manager.ConductorManager method*), 1030
attach_virtual_media() (*ironic.conductor.rpcapi.ConductorAPI method*), 1041
attach_virtual_media() (*ironic.drivers.base.ManagementInterface method*), 1402
attach_virtual_media() (*ironic.drivers.modules.redfish.management.RedfishMana method*), 1268
attach_vmedia() (*in module ironic.drivers.modules.ilo.common*), 1206
attach_volumes() (*in module ironic.common.cinder*), 936
attach_volumes() (*ironic.drivers.base.StorageInterface method*), 1419
attach_volumes() (*ironic.drivers.modules.fake.FakeStorage method*), 1348
attach_volumes() (*ironic.drivers.modules.storage.cinder.CinderStorage method*), 1286
attach_volumes() (*ironic.drivers.modules.storage.external.ExternalStorage method*), 1287
attach_volumes() (*ironic.drivers.modules.storage.noop.NoopStorage method*), 1288
AttachAction (*class in ironic.common.trait_based_networking.base*), 918
AttachPort (*class in ironic.common.trait_based_networking.base*), 918
AttachPortgroup (*class in ironic.common.trait_based_networking.base*), 918
attribute_name() (*ironic.common.trait_based_networking.base.Variables method*), 924
attribute_type (*ironic.db.sqlalchemy.models.BIOSSetting attribute*), 1135
attribute_type (*ironic.objects.bios.BIOSSetting property*), 1452
AUTH_CLASSES (*ironic.drivers.modules.redfish.utils.SessionCache*

attribute), 1281
auth_entry() (*in module ironic.common.auth_basic*), 931
AUTH_SCHEME_MAP (*ironic.console.rfb.auths.RFBAuthSchemeList attribute*), 1094
auth_strategy() (*in module ironic.conf.json_rpc*), 1087
authenticate() (*in module ironic.common.auth_basic*), 932
authenticate() (*ironic.common.oci_registry.OciClient method*), 992
authorize() (*in module ironic.common.policy*), 995
AuthPublicRoutes (*class in ironic.api.middleware*), 895
AuthPublicRoutes (*class in ironic.api.middleware.auth_public_routes*), 894
AuthType (*class in ironic.console.rfb.auth*), 1092
AUTO_TYPE (*ironic.objects.fields.FlexibleDictField attribute*), 1470
AUTO_TYPE (*ironic.objects.fields.ListOfFlexibleDictsField attribute*), 1471
AUTO_TYPE (*ironic.objects.fields.MACAddressField attribute*), 1471
AUTO_TYPE (*ironic.objects.fields.NotificationLevelField attribute*), 1471
AUTO_TYPE (*ironic.objects.fields.NotificationStatusField attribute*), 1472
AUTO_TYPE (*ironic.objects.fields.StringFieldThatAcceptsCallable attribute*), 1472
AutodetectDeploy (*class in ironic.drivers.modules.autodetect*), 1319
AutoEnrollPossible, 1356
AUTOGENERATED (*in module ironic.common.automated_clean_methods*), 933
automated_clean (*ironic.db.sqlalchemy.models.Node attribute*), 1139
automated_clean (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1141
automated_clean (*ironic.objects.node.Node property*), 1481
AVAILABLE (*in module ironic.common.states*), 1006
available_for_dynamic_portgroup (*ironic.common.trait_based_networking.base.Port attribute*), 921
available_for_dynamic_portgroup (*ironic.db.sqlalchemy.models.Port attribute*), 1145
available_for_dynamic_portgroup (*ironic.objects.port.Port property*), 1519
available_for_dynamic_portgroup (*ironic.objects.port.PortCRUDPayload property*), 1527

B

backfill_allocation() (*in module ironic.conductor.allocations*), 1022
BadRequest (*in module ironic.common.exception*), 945
BareDriver (*class in ironic.drivers.base*), 1391
Base (*class in ironic.common.inspection_rules.base*), 910
BaseConductorManager (*class in ironic.conductor.base_manager*), 1023
BaseConsoleContainer (*class in ironic.console.container.base*), 1089
BaseDHCP (*class in ironic.dhcp.base*), 1183
BaseDriverFactory (*class in ironic.common.driver_factory*), 941
BaseImageService (*class in ironic.common.image_service*), 962
BaseInterface (*class in ironic.drivers.base*), 1392
BaseRPCService (*class in ironic.common.rpc_service*), 1005
BaseSwitchDriverFactory (*class in ironic.networking.switch_drivers.driver_factory*), 1435
BaseTranslator (*class in ironic.drivers.modules.switch.base*), 1289
BaseWSGIService (*class in ironic.common.wsgi_service*), 1021
BasicAuthMiddleware (*class in ironic.common.auth_basic*), 931
before() (*ironic.api.hooks.ConfigHook method*), 900
before() (*ironic.api.hooks.ContextHook method*), 901
before() (*ironic.api.hooks.DBHook method*), 901
before() (*ironic.api.hooks.PublicUrlHook method*), 902

before() (*ironic.api.hooks.RPCHook method*), 902
belowLowerCritical (*ironic.drivers.modules.snmp.SNMPDriverAuto...PDU2 attribute*), 1384
belowLowerWarning (*ironic.drivers.modules.snmp.SNMPDriverAuto...PDU2 attribute*), 1384
BIOS (*in module ironic.common.boot_devices*), 933
BIOS (*in module ironic.drivers.modules.redfish.utils*), 1281
bios (*ironic.drivers.base.BareDriver attribute*), 1391
bios_interface (*ironic.db.sqlalchemy.models.Node attribute*), 1139
bios_interface (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1141
bios_interface (*ironic.objects.node.Node property*), 1481
bios_interface (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1495
bios_interface (*ironic.objects.node.NodeCRUDPayload property*), 1490
bios_interface (*ironic.objects.node.NodePayload property*), 1500
bios_interface (*ironic.objects.node.NodeSetPowerStatePayload property*), 1504
bios_interface (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1509
BIOSInterface (*class in ironic.drivers.base*), 1389
BIOSSetting (*class in ironic.db.sqlalchemy.models*), 1135
BIOSSetting (*class in ironic.objects.bios*), 1451
BIOSSettingAlreadyExists, 945
BIOSSettingList (*class in ironic.objects.bios*), 1454
BIOSSettingListNotFound, 945
BIOSSettingNotFound, 945
BLINKING (*in module ironic.common.indicator_states*), 976
BMC (*in module ironic.drivers.modules.redfish.utils*), 1281
bmc_reset() (*ironic.drivers.modules.ipmitool.VendorPassthru method*), 1363
body() (*in module ironic.api.method*), 902
body_type (*ironic.api.functions.FunctionDefinition attribute*), 899
BOND_PORTS (*ironic.common.trait_based_networking.base.Actions attribute*), 918
boolean() (*in module ironic.common.args*), 927
BooleanField (*class in ironic.objects.fields*), 1470
boot (*ironic.drivers.base.BareDriver attribute*), 1391
boot_device (*ironic.api.controllers.v1.node.NodeManagementCo attribute*), 848
boot_index (*ironic.db.sqlalchemy.models.VolumeTarget attribute*), 1147
boot_index (*ironic.objects.volume_target.VolumeTarget property*), 1547
boot_index (*ironic.objects.volume_target.VolumeTargetCRUDPay property*), 1552
boot_instance() (*ironic.drivers.modules.agent_base.AgentOobStepsMixin method*), 1307
boot_interface (*ironic.db.sqlalchemy.models.Node attribute*), 1139
boot_interface (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1141
boot_interface (*ironic.objects.node.Node property*), 1481
boot_interface (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1495
boot_interface (*ironic.objects.node.NodeCRUDPayload property*), 1490
boot_interface (*ironic.objects.node.NodePayload property*), 1500
boot_interface (*ironic.objects.node.NodeSetPowerStatePayload property*), 1504
boot_interface (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1509
boot_into_iso() (*ironic.drivers.modules.ilo.vendor.VendorPassthru method*), 1225
boot_mode (*ironic.db.sqlalchemy.models.Node*

attribute), 1139
boot_mode (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1141
boot_mode (*ironic.objects.node.Node property*), 1481
boot_mode (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1495
boot_mode (*ironic.objects.node.NodeCRUDPayload property*), 1490
boot_mode (*ironic.objects.node.NodePayload property*), 1500
boot_mode (*ironic.objects.node.NodeSetPowerStatePayload property*), 1504
boot_mode (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1509
boot_mode() (*ironic.api.controllers.v1.node.NodeStatesController method*), 848
BootcAgentDeploy (*class in ironic.drivers.modules.agent*), 1300
BootDeviceController (*class in ironic.api.controllers.v1.node*), 845
BootInterface (*class in ironic.drivers.base*), 1395
BootModeHook (*class in ironic.drivers.modules.inspector.hooks.boot_mode*), 1227
BootModeNotAllowed, 945
build_agent_options() (*in module ironic.drivers.modules.deploy_utils*), 1323
build_configdrive() (*in module ironic.conductor.configdrive_utils*), 1025
build_deploy_pxe_options() (*in module ironic.common.pxe_utils*), 996
build_driver_for_task() (*in module ironic.common.driver_factory*), 942
build_extra_pxe_options() (*in module ironic.common.pxe_utils*), 996
build_instance_info_for_deploy() (*in module ironic.drivers.modules.deploy_utils*), 1323
build_instance_pxe_options() (*in module ironic.common.pxe_utils*), 996
build_kickstart_config_options() (*in module ironic.common.pxe_utils*), 996
build_pxe_config_options() (*in module ironic.common.pxe_utils*), 997
build_service_pxe_config() (*in module ironic.common.pxe_utils*), 997
build_url() (*in module ironic.api.controllers.link*), 893
bulk_delete_node_history_records() (*ironic.db.api.Connection method*), 1149
bulk_delete_node_history_records() (*ironic.db.sqlalchemy.api.Connection method*), 1097
bytes_to_int() (*in module ironic.drivers.modules.inspector.lldp_tlvs*), 1237
bytes_transferred (*ironic.common.checksum_utils.TransferHelper property*), 934

C

cache_bios_settings() (*in module ironic.drivers.base*), 1420
cache_bios_settings() (*ironic.drivers.base.BIOSInterface method*), 1390
cache_bios_settings() (*ironic.drivers.modules.fake.FakeBIOS method*), 1333
cache_bios_settings() (*ironic.drivers.modules.ilo.bios.IloBIOS method*), 1197
cache_bios_settings() (*ironic.drivers.modules.noop.NoBIOS method*), 1366
cache_bios_settings() (*ironic.drivers.modules.redfish.bios.RedfishBIOS method*), 1257
cache_firmware_components() (*in module ironic.drivers.base*), 1420
cache_firmware_components() (*ironic.drivers.base.FirmwareInterface method*), 1400
cache_firmware_components() (*ironic.drivers.modules.fake.FakeFirmware method*), 1339
cache_firmware_components() (*ironic.drivers.modules.noop.NoFirmware method*), 1369
cache_firmware_components() (*ironic.drivers.modules.redfish.firmware.RedfishFirmware method*), 1264
cache_instance_image() (*in module ironic.drivers.modules.deploy_utils*), 1323
cache_lookup_addresses() (*in module ironic.drivers.modules.inspect_utils*),

1356
cache_ramdisk_kernel() (*in module ironic.common.pxe_utils*), 997
call() (*ironic.common.glance_service.image_service.GlanceImageService method*), 905
call() (*ironic.conductor.rpcapi.LocalContext method*), 1065
CallAPIHookAction (*class in ironic.common.inspection_rules.actions*), 908
can_send_create_port() (*ironic.conductor.rpcapi.ConductorAPI method*), 1041
can_send_rescue() (*ironic.conductor.rpcapi.ConductorAPI method*), 1041
can_send_version() (*ironic.common.json_rpc.client.Client method*), 915
candidate_nodes (*ironic.db.sqlalchemy.models.Allocation attribute*), 1135
candidate_nodes (*ironic.objects.allocation.Allocation property*), 1443
candidate_nodes (*ironic.objects.allocation.AllocationCRUDPayload property*), 1448
capabilities (*ironic.drivers.base.BootInterface attribute*), 1395
capabilities (*ironic.drivers.base.NetworkInterface attribute*), 1410
capabilities (*ironic.drivers.modules.fake.FakeBoot attribute*), 1334
capabilities (*ironic.drivers.modules.ilo.boot.IloUefiHttpsBoot attribute*), 1199
capabilities (*ironic.drivers.modules.ilo.boot.IloVirtualMediaBoot attribute*), 1201
capabilities (*ironic.drivers.modules.ipxe.iPXEBoot attribute*), 1365
capabilities (*ironic.drivers.modules.ipxe.iPXEHttpBoot attribute*), 1365
capabilities (*ironic.drivers.modules.network.flat.FlatNetwork attribute*), 1244
capabilities (*ironic.drivers.modules.network.neutron.NeutronNetwork attribute*), 1251
capabilities (*ironic.drivers.modules.pxe.HttpBoot attribute*), 1375
capabilities (*ironic.drivers.modules.pxe.PXEBoot attribute*), 1376
capabilities (*ironic.drivers.modules.redfish.boot.RedfishHttpsBoot attribute*), 1259
capabilities (*ironic.drivers.modules.redfish.boot.RedfishVirtualMediaBoot attribute*), 1261
capabilities_object() (*in module ironic.drivers.utils*), 1431
cast() (*ironic.conductor.rpcapi.LocalContext method*), 1065
CatalogNotFound, 945
category (*ironic.common.trait_based_networking.base.PrimordialPort attribute*), 922
category (*ironic.db.sqlalchemy.models.Port attribute*), 1145
category (*ironic.db.sqlalchemy.models.Portgroup attribute*), 1145
category (*ironic.objects.port.Port property*), 1520
category (*ironic.objects.port.PortCRUDPayload property*), 1527
category (*ironic.objects.portgroup.Portgroup property*), 1528
category (*ironic.objects.portgroup.PortgroupCRUDPayload property*), 1534
CDROM (*in module ironic.common.boot_devices*), 933
change_node_boot_mode() (*ironic.conductor.manager.ConductorManager method*), 1030
change_node_boot_mode() (*ironic.conductor.rpcapi.ConductorAPI method*), 1041
change_node_power_state() (*ironic.conductor.manager.ConductorManager method*), 1030
change_node_power_state() (*ironic.conductor.rpcapi.ConductorAPI method*), 1041
change_node_secure_boot() (*ironic.conductor.manager.ConductorManager method*), 1030
change_node_secure_boot() (*ironic.conductor.rpcapi.ConductorAPI method*), 1042
Chassis (*class in ironic.db.sqlalchemy.models*), 1136
ChassisCollection (*class in ironic.objects.chassis*), 1456
CHASSIS (*in module ironic.common.components*), 939
chassis_id (*ironic.db.sqlalchemy.models.Node attribute*), 1139
chassis_id (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1141

chassis_id (*ironic.objects.node.Node property*), 1481
chassis_uuid (*ironic.objects.node.NodeCRUDPayload property*), 1490
ChassisAlreadyExists, 945
ChassisController (*class in ironic.api.controllers.v1.chassis*), 836
ChassisCRUDNotification (*class in ironic.objects.chassis*), 1458
ChassisCRUDPayload (*class in ironic.objects.chassis*), 1459
ChassisNotEmpty, 945
ChassisNotFound, 945
check() (*in module ironic.common.policy*), 995
check_allocation_policy_and_retrieve() (*in module ironic.api.controllers.v1.utils*), 873
check_allow_boot_mode() (*in module ironic.api.controllers.v1.utils*), 873
check_allow_child_node_params() (*in module ironic.api.controllers.v1.utils*), 873
check_allow_clean_disable_ramdisk() (*in module ironic.api.controllers.v1.utils*), 873
check_allow_configdrive() (*in module ironic.api.controllers.v1.utils*), 874
check_allow_deploy_steps() (*in module ironic.api.controllers.v1.utils*), 874
check_allow_disable_ramdisk() (*in module ironic.api.controllers.v1.utils*), 874
check_allow_driver_detail() (*in module ironic.api.controllers.v1.utils*), 874
check_allow_filter_by_conductor() (*in module ironic.api.controllers.v1.utils*), 874
check_allow_filter_by_conductor_group() (*in module ironic.api.controllers.v1.utils*), 874
check_allow_filter_by_fault() (*in module ironic.api.controllers.v1.utils*), 874
check_allow_filter_by_instance_name() (*in module ironic.api.controllers.v1.utils*), 874
check_allow_filter_by_lessee() (*in module ironic.api.controllers.v1.utils*), 874
check_allow_filter_by_owner() (*in module ironic.api.controllers.v1.utils*), 874
check_allow_filter_by_shard() (*in module ironic.api.controllers.v1.utils*), 874
check_allow_filter_driver_type() (*in module ironic.api.controllers.v1.utils*), 874
check_allow_management_verbs() (*in module ironic.api.controllers.v1.utils*), 874
check_allow_portgroup_filter_by_shard() (*in module ironic.api.controllers.v1.utils*), 874
check_allow_specify_driver() (*in module ironic.api.controllers.v1.utils*), 874
check_allow_specify_fields() (*in module ironic.api.controllers.v1.utils*), 875
check_allow_specify_resource_class() (*in module ironic.api.controllers.v1.utils*), 875
check_allowed_fields() (*in module ironic.api.controllers.v1.utils*), 875
check_allowed_portgroup_fields() (*in module ironic.api.controllers.v1.utils*), 875
check_and_fix_configdrive() (*in module ironic.conductor.configdrive_utils*), 1026
check_and_patch_configdrive() (*in module ironic.conductor.configdrive_utils*), 1026
check_and_retrieve_public_runbook() (*in module ironic.api.controllers.v1.utils*), 875
check_and_update_node_interfaces() (*in module ironic.common.driver_factory*), 942
check_cipher_suite_errors() (*in module ironic.drivers.modules.ipmitool*), 1364
check_condition() (*ironic.common.inspection_rules.operators.OperatorBase method*), 913
check_conditions() (*in module ironic.common.inspection_rules.engine*), 911
check_dir() (*in module ironic.common.utils*), 1012
check_for_invalid_fields() (*in module ironic.api.controllers.v1.utils*), 875
check_for_invalid_state_and_allow_filter() (*in module ironic.api.controllers.v1.utils*), 875
check_for_missing_params() (*in module ironic.drivers.modules.deploy_utils*), 1324
check_if_image_format_is_permitted() (*in module ironic.common.images*), 971
check_image_service() (*in module ironic.common.glance_service.image_service*), 907

check_image_size() (*in module ironic.drivers.modules.deploy_utils*), 1324
check_interface_capability() (*in module ironic.drivers.modules.deploy_utils*), 1324
check_list_policy() (*in module ironic.api.controllers.v1.utils*), 875
check_multiple_node_policies_and_retrieve() (*in module ironic.api.controllers.v1.utils*), 876
check_multiple_runbook_policies_and_retrieve() (*in module ironic.api.controllers.v1.utils*), 876
check_node_list() (*ironic.db.api.Connection method*), 1149
check_node_list() (*ironic.db.sqlalchemy.api.Connection method*), 1097
check_node_list() (*ironic.objects.allocation.Allocation class method*), 1443
check_node_policy_and_retrieve() (*in module ironic.api.controllers.v1.utils*), 876
check_obj_versions() (*ironic.command.dbsync.DBCommand method*), 903
check_owner_policy() (*in module ironic.api.controllers.v1.utils*), 877
check_policy() (*in module ironic.api.controllers.v1.utils*), 877
check_policy() (*in module ironic.common.policy*), 995
check_policy_true() (*in module ironic.api.controllers.v1.utils*), 877
check_port_list_policy() (*in module ironic.api.controllers.v1.utils*), 878
check_port_policy_and_retrieve() (*in module ironic.api.controllers.v1.utils*), 878
check_redirect_trusted() (*ironic.common.oci_registry.RegistrySessionHelper static method*), 994
check_runbook_policy_and_retrieve() (*in module ironic.api.controllers.v1.utils*), 878
check_status() (*ironic.common.oci_registry.RegistrySessionHelper static method*), 994
check_versions() (*ironic.db.api.Connection method*), 1149
check_versions() (*ironic.db.sqlalchemy.api.Connection method*), 1097
check_volume_list_policy() (*in module ironic.api.controllers.v1.utils*), 879
check_volume_policy_and_retrieve() (*in module ironic.api.controllers.v1.utils*), 879
check_with_loop() (*ironic.common.inspection_rules.operators.OperatorBase method*), 913
Checks (*class in ironic.command.status*), 904
checksum_matches (*ironic.common.checksum_utils.TransferHelper property*), 934
ChildNodeLocked, 945
choose_cipher_suite() (*in module ironic.drivers.modules.ipmitool*), 1364
CifsImageService (*class in ironic.common.image_service*), 963
CIFSPublisher (*class in ironic.common.image_publisher*), 961
CinderStorage (*class in ironic.drivers.modules.storage.cinder*), 1285
clean_dhcp() (*ironic.common.dhcp_factory.DHCPFactory method*), 940
clean_dhcp_opts() (*ironic.dhcp.base.BaseDHCP method*), 1184
clean_dhcp_opts() (*ironic.dhcp.dnsmasq.DnsmasqDHCPApi method*), 1185
CLEAN_FAILURE (*in module ironic.common.faults*), 958
clean_step (*ironic.db.sqlalchemy.models.Node attribute*), 1139
clean_step (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1141
clean_step (*ironic.objects.node.Node property*), 1481
clean_step (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1495
clean_step (*ironic.objects.node.NodeCRUDPayload property*), 1490
clean_step (*ironic.objects.node.NodePayload property*), 1500
clean_step (*ironic.objects.node.NodeSetPowerStatePayload property*), 1504
clean_step (*ironic.objects.node.NodeSetProvisionStatePayload

property), 1509
clean_step() (in module ironic.drivers.base), 1420
clean_up() (in module ironic.drivers.modules.inspector.interface), 1235
clean_up() (ironic.drivers.base.DeployInterface method), 1397
clean_up() (ironic.drivers.base.RescueInterface method), 1418
clean_up() (ironic.drivers.modules.agent.AgentRescue method), 1298
clean_up() (ironic.drivers.modules.agent.CustomAgentDeploy method), 1300
clean_up() (ironic.drivers.modules.agent_base.AgentBaseMixin method), 1303
clean_up() (ironic.drivers.modules.ansible.deploy.AnsibleDeploy method), 1189
clean_up() (ironic.drivers.modules.autodetect.AutodetectDeploy method), 1319
clean_up() (ironic.drivers.modules.fake.FakeDeploy method), 1337
clean_up() (ironic.drivers.modules.image_cache.ImageCache method), 1351
clean_up() (ironic.drivers.modules.noop.NoDeploy method), 1368
clean_up() (ironic.drivers.modules.pxe.PXEAnacondaDeploy method), 1375
clean_up_all() (in module ironic.drivers.modules.image_cache), 1351
clean_up_caches() (in module ironic.drivers.modules.image_cache), 1351
clean_up_instance() (ironic.drivers.base.BootInterface method), 1395
clean_up_instance() (ironic.drivers.modules.fake.FakeBoot method), 1335
clean_up_instance() (ironic.drivers.modules.ilo.boot.IloiPXEBoot method), 1204
clean_up_instance() (ironic.drivers.modules.ilo.boot.IloPXEBoot method), 1198
clean_up_instance() (ironic.drivers.modules.ilo.boot.IloUefiHttpsBoot method), 1199
clean_up_instance() (ironic.drivers.modules.ilo.boot.IloVirtualMediaBoot method), 1201
clean_up_instance() (ironic.drivers.modules.pxe_base.PXEBaseMixin method), 1376
clean_up_instance() (ironic.drivers.modules.redfish.boot.RedfishHttpsBoot method), 1259
clean_up_instance() (ironic.drivers.modules.redfish.boot.RedfishVirtualMediaB method), 1261
clean_up_pxe_config() (in module ironic.common.pxe_utils), 997
clean_up_pxe_env() (in module ironic.common.pxe_utils), 997
clean_up_ramdisk() (ironic.drivers.base.BootInterface method), 1395
clean_up_ramdisk() (ironic.drivers.modules.fake.FakeBoot method), 1335
clean_up_ramdisk() (ironic.drivers.modules.ilo.boot.IloUefiHttpsBoot method), 1200
clean_up_ramdisk() (ironic.drivers.modules.ilo.boot.IloVirtualMediaBoot method), 1202
clean_up_ramdisk() (ironic.drivers.modules.pxe_base.PXEBaseMixin method), 1376
clean_up_ramdisk() (ironic.drivers.modules.redfish.boot.RedfishHttpsBoot method), 1259
clean_up_ramdisk() (ironic.drivers.modules.redfish.boot.RedfishVirtualMediaB method), 1261
clean_up_swift_entries() (in module ironic.drivers.modules.inspect_utils), 1356
CLEANFAIL (in module ironic.common.states), 1006
CLEANHOLD (in module ironic.common.states), 1006
CLEANING (in module ironic.common.states), 1006
CLEANING (ironic.drivers.modules.network.common.NetType attribute), 1239
CLEANING_AUTO (ironic.conductor.utils.StepFlow attribute), 1071
cleaning_error_handler() (in module ironic.conductor.utils), 1072
CLEANING_MANUAL (ironic.conductor.utils.StepFlow at-

tribute), 1071
cleanup() (*in module ironic.common.rpc*), 1004
cleanup() (*in module ironic.drivers.modules.image_cache*), 1352
cleanup() (*in module ironic.drivers.modules.redfish.firmware_utils*), 1265
cleanup() (*ironic.networking.manager.NetworkingManager method*), 1438
cleanup_after_timeout() (*in module ironic.conductor.utils*), 1072
cleanup_cleanwait_timeout() (*in module ironic.conductor.utils*), 1072
cleanup_disk_image() (*in module ironic.drivers.modules.image_utils*), 1353
cleanup_floppy_image() (*in module ironic.drivers.modules.image_utils*), 1353
cleanup_inspectwait_timeout() (*in module ironic.conductor.utils*), 1073
cleanup_iso_image() (*in module ironic.drivers.modules.image_utils*), 1353
cleanup_remote_image() (*in module ironic.drivers.modules.image_utils*), 1353
cleanup_rescuewait_timeout() (*in module ironic.conductor.utils*), 1073
cleanup_servicewait_timeout() (*in module ironic.conductor.utils*), 1073
cleanup_stale_conductors() (*ironic.conductor.manager.ConductorManager method*), 1031
cleanup_vmedia_boot() (*in module ironic.drivers.modules.ilo.common*), 1207
CLEANWAIT (*in module ironic.common.states*), 1006
clear_ca_certificates() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1215
clear_certificates() (*in module ironic.drivers.modules.ilo.common*), 1207
clear_iscsi_boot_target() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1216
clear_job_queue() (*ironic.drivers.modules.drac.management.DracRedfishManagement method*), 1193
clear_lookup_addresses() (*in module ironic.drivers.modules.inspect_utils*), 1356
clear_node_reservations_for_conductor() (*ironic.db.sqlalchemy.api.Connection method*), 1098
clear_node_target_power_state() (*ironic.db.sqlalchemy.api.Connection method*), 1098
clear_secure_boot_keys() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1216
clear_secure_boot_keys() (*ironic.drivers.modules.redfish.management.RedfishManag method*), 1268
Client (*class in ironic.common.json_rpc.client*), 915
ClientSideError, 945
close() (*ironic.common.mdns.Zeroconf method*), 979
close() (*ironic.console.websocketproxy.TenantSock method*), 1096
code (*ironic.common.exception.Conflict attribute*), 946
code (*ironic.common.exception.ImageChecksumFileReadFailure attribute*), 948
code (*ironic.common.exception.Invalid attribute*), 950
code (*ironic.common.exception.IronicException attribute*), 952
code (*ironic.common.exception.NoFreeConductorWorker attribute*), 952
code (*ironic.common.exception.NotAcceptable attribute*), 954
code (*ironic.common.exception.NotAuthorized attribute*), 954
code (*ironic.common.exception.NotFound attribute*), 954
code (*ironic.common.exception.TemporaryFailure attribute*), 957
code (*ironic.common.exception.Unauthorized attribute*), 957
code (*ironic.common.json_rpc.server.InvalidParams attribute*), 916
code (*ironic.common.json_rpc.server.InvalidRequest attribute*), 916
code (*ironic.common.json_rpc.server.MethodNotFound attribute*), 916
code (*ironic.common.json_rpc.server.ParseError attribute*), 916

code (*ironic.drivers.modules.inspect_utils.AutoEnrollPossible attribute*), 1356
coerce() (*ironic.objects.fields.FlexibleDict static method*), 1470
coerce() (*ironic.objects.fields.MACAddress static method*), 1471
coerce() (*ironic.objects.fields.StringAcceptsCallable static method*), 1472
collect_deploy_logs (*ironic.drivers.modules.agent_base.HeartbeatMixin attribute*), 1307
collect_deploy_logs (*ironic.drivers.modules.ansible.deploy.AnsibleDeploy attribute*), 1189
collect_ramdisk_logs() (*in module ironic.drivers.utils*), 1431
collect_system_logs() (*ironic.drivers.modules.agent_client.AgentClient method*), 1310
collection_from_list() (*in module ironic.api.controllers.v1.bios*), 836
collection_from_list() (*in module ironic.api.controllers.v1.firmware*), 843
Common (*class in ironic.drivers.modules.inspector.interface*), 1234
CommunicationError, 945
Comparator (*class in ironic.common.trait_based_networking.base*), 918
compat_headers (*ironic.api.app.IronicRequestId attribute*), 898
component (*ironic.db.sqlalchemy.models.FirmwareComponent attribute*), 1138
component (*ironic.objects.firmware.FirmwareComponent property*), 1473
compound_expression() (*ironic.common.trait_based_networking.grammar.AstBuilder method*), 917
CompoundExpression (*class in ironic.common.trait_based_networking.base*), 919
compute_image_checksum() (*in module ironic.common.checksum_utils*), 934
compute_image_checksum() (*in module ironic.drivers.modules.deploy_utils*), 1324
ConcurrentActionLimit, 945
conditions (*ironic.db.sqlalchemy.models.InspectionRule attribute*), 1138
conditions (*ironic.objects.inspection_rule.InspectionRule property*), 1476
conditions (*ironic.objects.inspection_rule.InspectionRuleCRUD property*), 1480
Conductor (*class in ironic.db.sqlalchemy.models*), 1136
conductor (*class in ironic.objects.conductor*), 1460
conductor (*ironic.db.sqlalchemy.models.NodeHistory attribute*), 1143
conductor (*ironic.objects.node_history.NodeHistory property*), 1513
conductor_affinity (*ironic.db.sqlalchemy.models.Allocation attribute*), 1135
conductor_affinity (*ironic.db.sqlalchemy.models.Node attribute*), 1139
conductor_affinity (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1141
conductor_affinity (*ironic.objects.allocation.Allocation property*), 1443
conductor_affinity (*ironic.objects.node.Node property*), 1481
conductor_group (*ironic.db.sqlalchemy.models.Conductor attribute*), 1136
conductor_group (*ironic.db.sqlalchemy.models.Node attribute*), 1139
conductor_group (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1141
conductor_group (*ironic.objects.conductor.Conductor property*), 1460
conductor_group (*ironic.objects.node.Node property*), 1481
conductor_group (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1495
conductor_group (*ironic.objects.node.NodeCRUDPayload property*), 1490
conductor_group (*ironic.objects.node.NodePayload property*), 1500
conductor_group (*ironic.objects.node.NodeSetPowerStatePayload*

property), 1504
conductor_group (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1509
conductor_id (*ironic.db.sqlalchemy.models.ConductorHardwareInterfaces attribute*), 1136
ConductorAlreadyRegistered, 945
ConductorAPI (*class in ironic.conductor.rpcapi*), 1038
ConductorHardwareInterfaces (*class in ironic.db.sqlalchemy.models*), 1136
ConductorHardwareInterfacesAlreadyRegistered, 945
ConductorManager (*class in ironic.conductor.manager*), 1030
ConductorNotFound, 945
ConductorsController (*class in ironic.api.controllers.v1.conductor*), 839
config() (*in module ironic.api.controllers.v1.ramdisk*), 865
ConfigDriveRegenerationFailure, 946
ConfigFile (*class in ironic.common.trait_based_networking.config*), 924
ConfigHook (*class in ironic.api.hooks*), 900
ConfigInvalid, 946
ConfigLoader (*class in ironic.common.trait_based_networking.loader*), 925
ConfigNotFound, 946
configure_intel_speedselect() (*ironic.drivers.modules.intel_ipmi.management.IntelIPMIManagement method*), 1238
configure_local_boot() (*ironic.drivers.modules.agent.AgentDeploy method*), 1296
configure_secure_boot_if_needed() (*in module ironic.drivers.modules.boot_mode_utils*), 1320
configure_tenant_networks() (*ironic.drivers.base.NetworkInterface method*), 1411
configure_tenant_networks() (*ironic.drivers.modules.network.flat.FlatNetwork method*), 1244
configure_tenant_networks() (*ironic.drivers.modules.network.ironic_networking.IronicNetworking method*), 1247
configure_tenant_networks() (*ironic.drivers.modules.network.neutron.NeutronNetwork method*), 1251
configure_tenant_networks() (*ironic.drivers.modules.network.noop.NoopNetwork method*), 1254
Conflict, 946
connect() (*ironic.console.securityproxy.base.SecurityProxy method*), 1094
connect() (*ironic.console.securityproxy.rfb.RFBSecurityProxy method*), 1095
Connection (*class in ironic.db.api*), 1148
Connection (*class in ironic.db.sqlalchemy.api*), 1096
connector_id (*ironic.db.sqlalchemy.models.VolumeConnector attribute*), 1147
connector_id (*ironic.objects.volume_connector.VolumeConnecto property*), 1542
connector_id (*ironic.objects.volume_connector.VolumeConnecto property*), 1547
console (*ironic.api.controllers.v1.node.NodeStatesController attribute*), 849
console (*ironic.drivers.base.BareDriver attribute*), 1391
console_enabled (*ironic.db.sqlalchemy.models.Node attribute*), 1139
console_enabled (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1141
console_enabled (*ironic.objects.node.Node property*), 1481
console_enabled (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1495
console_enabled (*ironic.objects.node.NodeCRUDPayload property*), 1490
console_enabled (*ironic.objects.node.NodePayload property*), 1500
console_enabled (*ironic.objects.node.NodeSetPowerStatePayload property*), 1504
console_enabled (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1509
console_interface (*ironic.db.sqlalchemy.models.Node attribute*), 1139
console_interface

(*ironic.db.sqlalchemy.models.NodeBase attribute*), 1141
console_interface (*ironic.objects.node.Node property*), 1481
console_interface (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1495
console_interface (*ironic.objects.node.NodeCRUDPayload property*), 1490
console_interface (*ironic.objects.node.NodePayload property*), 1500
console_interface (*ironic.objects.node.NodeSetPowerStatePayload property*), 1504
console_interface (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1509
ConsoleContainerError, 946
ConsoleContainerFactory (*class in ironic.common.console_factory*), 939
ConsoleError, 946
ConsoleInterface (*class in ironic.drivers.base*), 1397
ConsoleSubprocessFailed, 946
ContainsOperator (*class in ironic.common.inspection_rules.operators*), 911
content_length (*ironic.common.checksum_utils.TransferHelper property*), 934
ContextHook (*class in ironic.api.hooks*), 901
continue_cleaning() (*ironic.drivers.modules.agent_base.HeartbeatMixin method*), 1307
continue_inspection() (*in module ironic.conductor.inspection*), 1030
continue_inspection() (*ironic.conductor.manager.ConductorManager method*), 1031
continue_inspection() (*ironic.conductor.rpcapi.ConductorAPI method*), 1042
continue_inspection() (*ironic.drivers.base.InspectInterface method*), 1402
continue_inspection() (*ironic.drivers.modules.inspector.agent.AgentInspect method*), 1234
continue_inspection() (*ironic.drivers.modules.inspector.AgentInspect method*), 1238
continue_node_clean() (*in module ironic.conductor.cleaning*), 1024
continue_node_clean() (*ironic.conductor.manager.ConductorManager method*), 1031
continue_node_clean() (*ironic.conductor.rpcapi.ConductorAPI method*), 1042
continue_node_deploy() (*in module ironic.conductor.deployments*), 1028
continue_node_deploy() (*ironic.conductor.manager.ConductorManager method*), 1031
continue_node_deploy() (*ironic.conductor.rpcapi.ConductorAPI method*), 1043
continue_node_service() (*in module ironic.conductor.servicing*), 1065
continue_node_service() (*ironic.conductor.manager.ConductorManager method*), 1031
continue_node_service() (*ironic.conductor.rpcapi.ConductorAPI method*), 1043
continue_servicing() (*ironic.drivers.modules.agent_base.HeartbeatMixin method*), 1307
ContinueInspectionController (*class in ironic.api.controllers.v1.ramdisk*), 864
Controller (*class in ironic.api.controllers.v1*), 892
convert() (*in module ironic.api.controllers.v1.volume*), 887
convert_actions() (*in module ironic.api.controllers.v1.inspection_rule*), 845
convert_conditions() (*in module ironic.api.controllers.v1.inspection_rule*), 845
convert_drive_units() (*in module ironic.drivers.modules.redfish.raid*), 1280
convert_image() (*in module ironic.common.qemu_img*), 1002
convert_steps() (*in module ironic.api.controllers.v1.utils*), 879
convert_steps() (*in module ironic.conductor.manager*), 1035
convert_to_version()

(*ironic.objects.base.IronicObject method*), 1449
convert_with_links() (*in module ironic.api.controllers.v1.allocation*), 836
convert_with_links() (*in module ironic.api.controllers.v1.bios*), 836
convert_with_links() (*in module ironic.api.controllers.v1.chassis*), 838
convert_with_links() (*in module ironic.api.controllers.v1.conductor*), 839
convert_with_links() (*in module ironic.api.controllers.v1.deploy_template*), 840
convert_with_links() (*in module ironic.api.controllers.v1.driver*), 842
convert_with_links() (*in module ironic.api.controllers.v1.firmware*), 844
convert_with_links() (*in module ironic.api.controllers.v1.inspection_rule*), 845
convert_with_links() (*in module ironic.api.controllers.v1.port*), 862
convert_with_links() (*in module ironic.api.controllers.v1.portgroup*), 863
convert_with_links() (*in module ironic.api.controllers.v1.ramdisk*), 865
convert_with_links() (*in module ironic.api.controllers.v1.runbook*), 867
convert_with_links() (*in module ironic.api.controllers.v1.volume_connector*), 890
convert_with_links() (*in module ironic.api.controllers.v1.volume_target*), 892
converted_size() (*in module ironic.common.images*), 971
copy() (*ironic.common.inspection_rules.utils.ShallowMaskDict method*), 914
copy() (*ironic.common.inspection_rules.utils.ShallowMaskList method*), 914
copy_image_to_swift() (*in module ironic.drivers.modules.ilo.common*), 1207
copy_image_to_web_server() (*in module ironic.drivers.modules.ilo.common*), 1207
core_interfaces (*ironic.drivers.base.BareDriver property*), 1391
count_nodes_in_provision_state() (*ironic.db.api.Connection method*), 1149
count_nodes_in_provision_state() (*ironic.db.sqlalchemy.api.Connection method*), 1098
Counter (*class in ironic.common.metrics*), 979
counter() (*ironic.common.metrics.MetricLogger method*), 980
COUNTER_TYPE (*ironic.common.metrics_collector.DictCollectionM attribute*), 982
COUNTER_TYPE (*ironic.common.metrics_statsd.StatsdMetricLogger attribute*), 983
CPUCapabilitiesHook (*class in ironic.drivers.modules.inspector.hooks.cpu_capabilities*), 1228
CPUSHook (*class in ironic.drivers.modules.inspector.hooks.cpus*), 1228
create() (*ironic.common.oci_registry.MakeSession method*), 992
create() (*ironic.objects.allocation.Allocation method*), 1443
create() (*ironic.objects.bios.BIOSSetting method*), 1452
create() (*ironic.objects.bios.BIOSSettingList class method*), 1454
create() (*ironic.objects.chassis.Chassis method*), 1456
create() (*ironic.objects.deploy_template.DeployTemplate method*), 1462
create() (*ironic.objects.deployment.Deployment method*), 1467
create() (*ironic.objects.firmware.FirmwareComponent method*), 1473
create() (*ironic.objects.inspection_rule.InspectionRule method*), 1477
create() (*ironic.objects.node.Node method*), 1481
create() (*ironic.objects.node_history.NodeHistory method*), 1513
create() (*ironic.objects.node_inventory.NodeInventory method*), 1516
create() (*ironic.objects.port.Port method*), 1520
create() (*ironic.objects.portgroup.Portgroup method*), 1528
create() (*ironic.objects.runbook.Runbook*

method), 1535
create() (*ironic.objects.trait.Trait method*), 1540
create() (*ironic.objects.trait.TraitList class method*), 1541
create() (*ironic.objects.volume_connector.VolumeConnector method*), 1542
create() (*ironic.objects.volume_target.VolumeTarget method*), 1547
create_allocation() (*ironic.conductor.manager.ConductorManager method*), 1032
create_allocation() (*ironic.conductor.rpcapi.ConductorAPI method*), 1043
create_allocation() (*ironic.db.api.Connection method*), 1149
create_allocation() (*ironic.db.sqlalchemy.api.Connection method*), 1098
create_bios_setting_list() (*ironic.db.api.Connection method*), 1150
create_bios_setting_list() (*ironic.db.sqlalchemy.api.Connection method*), 1098
create_boot_iso() (*in module ironic.common.images*), 972
create_chassis() (*ironic.db.api.Connection method*), 1150
create_chassis() (*ironic.db.sqlalchemy.api.Connection method*), 1099
create_configuration() (*ironic.drivers.base.RAIDInterface method*), 1416
create_configuration() (*ironic.drivers.modules.agent.AgentRAID method*), 1297
create_configuration() (*ironic.drivers.modules.drac.raid.DracRedfishRAID method*), 1195
create_configuration() (*ironic.drivers.modules.fake.FakeRAID method*), 1346
create_configuration() (*ironic.drivers.modules.ilo.raid.Ilo5RAID method*), 1224
create_configuration() (*ironic.drivers.modules.noop.NoRAID method*), 1370
create_configuration() (*ironic.drivers.modules.redfish.raid.RedfishRAID method*), 1278
create_csr() (*ironic.drivers.modules.ilo.management.IloManage method*), 1217
create_deploy_template() (*ironic.db.api.Connection method*), 1150
create_deploy_template() (*ironic.db.sqlalchemy.api.Connection method*), 1099
create_esp_image_for_uefi() (*in module ironic.common.images*), 972
create_firmware_component() (*ironic.db.api.Connection method*), 1151
create_firmware_component() (*ironic.db.sqlalchemy.api.Connection method*), 1099
create_inspection_rule() (*ironic.db.api.Connection method*), 1151
create_inspection_rule() (*ironic.db.sqlalchemy.api.Connection method*), 1100
create_ipxe_boot_script() (*in module ironic.common.pxe_utils*), 997
create_isolinux_image_for_bios() (*in module ironic.common.images*), 973
create_link_without_raise() (*in module ironic.common.utils*), 1012
create_node() (*ironic.conductor.manager.ConductorManager method*), 1032
create_node() (*ironic.conductor.rpcapi.ConductorAPI method*), 1043
create_node() (*ironic.db.api.Connection method*), 1151
create_node() (*ironic.db.sqlalchemy.api.Connection method*), 1100
create_node_history() (*ironic.db.api.Connection method*), 1152
create_node_history() (*ironic.db.sqlalchemy.api.Connection method*), 1100
create_node_inventory() (*ironic.db.api.Connection method*), 1152
create_node_inventory() (*ironic.db.sqlalchemy.api.Connection method*), 1100

create_object() (*ironic.common.swift.SwiftAPI method*), 1010
create_object_from_data() (*ironic.common.swift.SwiftAPI method*), 1010
create_port() (*ironic.conductor.manager.ConductorManager method*), 1032
create_port() (*ironic.conductor.rpcapi.ConductorAPI method*), 1044
create_port() (*ironic.db.api.Connection method*), 1152
create_port() (*ironic.db.sqlalchemy.api.Connection method*), 1100
create_portgroup() (*ironic.db.api.Connection method*), 1152
create_portgroup() (*ironic.db.sqlalchemy.api.Connection method*), 1100
create_ports_if_not_exist() (*in module ironic.drivers.modules.inspect_utils*), 1356
create_pxe_config() (*in module ironic.common.pxe_utils*), 997
create_runbook() (*ironic.db.api.Connection method*), 1152
create_runbook() (*ironic.db.sqlalchemy.api.Connection method*), 1101
create_schema command line option
--help, 457
-h, 457
create_schema() (*in module ironic.db.migration*), 1183
create_schema() (*in module ironic.db.sqlalchemy.migration*), 1134
create_schema() (*ironic.command.dbsync.DBCommand method*), 903
create_schema, ironic-dbsync command line option, 456
create_subscription() (*ironic.drivers.modules.redfish.vendor.RedfishVendorPassthru method*), 1284
create_vfat_image() (*in module ironic.common.images*), 974
create_virtual_disk() (*in module ironic.drivers.modules.redfish.raid*), 1280
create_volume_connector() (*ironic.db.api.Connection method*), 1153
create_volume_connector() (*ironic.db.sqlalchemy.api.Connection method*), 1101
create_volume_target() (*ironic.db.api.Connection method*), 1153
create_volume_target() (*ironic.db.sqlalchemy.api.Connection method*), 1102
created_at (*ironic.db.sqlalchemy.models.Allocation attribute*), 1135
created_at (*ironic.db.sqlalchemy.models.BIOSSetting attribute*), 1135
created_at (*ironic.db.sqlalchemy.models.Chassis attribute*), 1136
created_at (*ironic.db.sqlalchemy.models.Conductor attribute*), 1136
created_at (*ironic.db.sqlalchemy.models.ConductorHardwareInt attribute*), 1137
created_at (*ironic.db.sqlalchemy.models.DeployTemplate attribute*), 1137
created_at (*ironic.db.sqlalchemy.models.DeployTemplateStep attribute*), 1137
created_at (*ironic.db.sqlalchemy.models.FirmwareComponent attribute*), 1138
created_at (*ironic.db.sqlalchemy.models.InspectionRule attribute*), 1138
created_at (*ironic.db.sqlalchemy.models.Node attribute*), 1139
created_at (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1141
created_at (*ironic.db.sqlalchemy.models.NodeHistory attribute*), 1143
created_at (*ironic.db.sqlalchemy.models.NodeInventory attribute*), 1144
created_at (*ironic.db.sqlalchemy.models.NodeTag attribute*), 1144
created_at (*ironic.db.sqlalchemy.models.NodeTrait attribute*), 1144
created_at (*ironic.db.sqlalchemy.models.Port attribute*), 1145
created_at (*ironic.db.sqlalchemy.models.Portgroup attribute*), 1145
created_at (*ironic.db.sqlalchemy.models.Runbook attribute*), 1146
created_at (*ironic.db.sqlalchemy.models.RunbookStep attribute*), 1147
created_at (*ironic.db.sqlalchemy.models.VolumeConnector attribute*), 1147

created_at (*ironic.db.sqlalchemy.models.VolumeTarget attribute*), 1147
created_at (*ironic.objects.allocation.Allocation property*), 1444
created_at (*ironic.objects.allocation.AllocationCRUDNotification property*), 1447
created_at (*ironic.objects.allocation.AllocationCRUDPayload property*), 1448
created_at (*ironic.objects.bios.BIOSSetting property*), 1452
created_at (*ironic.objects.bios.BIOSSettingList property*), 1455
created_at (*ironic.objects.chassis.Chassis property*), 1456
created_at (*ironic.objects.chassis.ChassisCRUDNotification property*), 1458
created_at (*ironic.objects.chassis.ChassisCRUDPayload property*), 1459
created_at (*ironic.objects.conductor.Conductor property*), 1460
created_at (*ironic.objects.deploy_template.DeployTemplate property*), 1463
created_at (*ironic.objects.deploy_template.DeployTemplateCRUDNotification property*), 1466
created_at (*ironic.objects.deploy_template.DeployTemplateCRUDPayload property*), 1466
created_at (*ironic.objects.deployment.Deployment property*), 1467
created_at (*ironic.objects.firmware.FirmwareComponent property*), 1473
created_at (*ironic.objects.firmware.FirmwareComponentList property*), 1474
created_at (*ironic.objects.inspection_rule.InspectionRule property*), 1477
created_at (*ironic.objects.inspection_rule.InspectionRuleCRUDNotification property*), 1479
created_at (*ironic.objects.inspection_rule.InspectionRuleCRUDPayload property*), 1480
created_at (*ironic.objects.node.Node property*), 1481
created_at (*ironic.objects.node.NodeConsoleNotification property*), 1494
created_at (*ironic.objects.node.NodeCorrectedPowerStateNotification property*), 1494
created_at (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1495
created_at (*ironic.objects.node.NodeCRUDNotification property*), 1489
created_at (*ironic.objects.node.NodeCRUDPayload property*), 1490
created_at (*ironic.objects.node.NodeMaintenanceNotification property*), 1499
created_at (*ironic.objects.node.NodePayload property*), 1500
created_at (*ironic.objects.node.NodeSetPowerStateNotification property*), 1504
created_at (*ironic.objects.node.NodeSetPowerStatePayload property*), 1505
created_at (*ironic.objects.node.NodeSetProvisionStateNotification property*), 1508
created_at (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1509
created_at (*ironic.objects.node_history.NodeHistory property*), 1513
created_at (*ironic.objects.node_inventory.NodeInventory property*), 1516
created_at (*ironic.objects.notification.EventType property*), 1517
created_at (*ironic.objects.notification.NotificationBase property*), 1518
created_at (*ironic.objects.notification.NotificationPayloadBase property*), 1519
created_at (*ironic.objects.notification.NotificationPublisher property*), 1520
created_at (*ironic.objects.port.Port property*), 1521
created_at (*ironic.objects.port.PortCRUDNotification property*), 1526
created_at (*ironic.objects.port.PortCRUDPayload property*), 1527
created_at (*ironic.objects.portgroup.Portgroup property*), 1528
created_at (*ironic.objects.portgroup.PortgroupCRUDNotification property*), 1533
created_at (*ironic.objects.portgroup.PortgroupCRUDPayload property*), 1534
created_at (*ironic.objects.runbook.Runbook property*), 1535
created_at (*ironic.objects.runbook.RunbookCRUDNotification property*), 1538
created_at (*ironic.objects.runbook.RunbookCRUDPayload property*), 1539
created_at (*ironic.objects.trait.Trait property*), 1540
created_at (*ironic.objects.trait.TraitList property*), 1541
created_at (*ironic.objects.volume_connector.VolumeConnector property*), 1543
created_at (*ironic.objects.volume_connector.VolumeConnectorC property*), 1546
created_at (*ironic.objects.volume_connector.VolumeConnectorC property*), 1547

created_at (*ironic.objects.volume_target.VolumeTarget property*), 1548
created_at (*ironic.objects.volume_target.VolumeTarget property*), 1552
created_at (*ironic.objects.volume_target.VolumeTarget property*), 1552
CRITICAL (*ironic.common.health_states.HealthState attribute*), 960
CRITICAL (*ironic.objects.fields.NotificationLevel attribute*), 1471
current_version (*ironic.db.sqlalchemy.models.FirmwareComponent attribute*), 1138
current_version (*ironic.objects.firmware.FirmwareComponent property*), 1473
CustomAgentDeploy (*class in ironic.drivers.modules.agent*), 1300

D

DatabaseVersionTooOld, 946
datatype (*ironic.api.functions.FunctionArgument attribute*), 899
DateTimeField (*class in ironic.objects.fields*), 1470
dbapi (*ironic.objects.allocation.Allocation attribute*), 1444
dbapi (*ironic.objects.bios.BIOSSetting attribute*), 1452
dbapi (*ironic.objects.bios.BIOSSettingList attribute*), 1455
dbapi (*ironic.objects.chassis.Chassis attribute*), 1456
dbapi (*ironic.objects.conductor.Conductor attribute*), 1460
dbapi (*ironic.objects.deploy_template.DeployTemplate attribute*), 1463
dbapi (*ironic.objects.deployment.Deployment attribute*), 1467
dbapi (*ironic.objects.firmware.FirmwareComponent attribute*), 1473
dbapi (*ironic.objects.firmware.FirmwareComponentList attribute*), 1474
dbapi (*ironic.objects.inspection_rule.InspectionRule attribute*), 1477
dbapi (*ironic.objects.node.Node attribute*), 1481
dbapi (*ironic.objects.node_history.NodeHistory attribute*), 1513
dbapi (*ironic.objects.node_inventory.NodeInventory attribute*), 1516
dbapi (*ironic.objects.port.Port attribute*), 1520
dbapi (*ironic.objects.portgroup.Portgroup attribute*), 1528
dbapi (*ironic.objects.runbook.Runbook attribute*), 1535
dbapi (*ironic.objects.trait.Trait attribute*), 1540
dbapi (*ironic.objects.trait.TraitList attribute*), 1542
dbapi (*ironic.objects.volume_connector.VolumeConnector attribute*), 1543
dbapi (*ironic.objects.volume_target.VolumeTarget attribute*), 1548
DBCommand (*class in ironic.command.dbsync*), 903
DBHook (*class in ironic.api.hooks*), 901
dd() (*in module ironic.common.utils*), 1012
DEBUG (*ironic.objects.fields.NotificationLevel attribute*), 1471
decode_and_extract_config_drive_iso() (*in module ironic.common.kickstart_utils*), 978
decode_json_pointer() (*in module ironic.api.controllers.v1.utils*), 879
deconfigure_secure_boot_if_needed() (*in module ironic.drivers.modules.boot_mode_utils*), 1320
default (*ironic.api.functions.FunctionArgument attribute*), 899
default (*ironic.db.sqlalchemy.models.ConductorHardwareInterfa attribute*), 1137
default_interface() (*in module ironic.common.driver_factory*), 943
default_network_trait() (*in module ironic.common.trait_based_networking.defaults*), 925
default_version() (*in module ironic.api.controllers.version*), 894
del_driver_internal_info() (*ironic.objects.node.Node method*), 1481
del_host() (*ironic.conductor.base_manager.BaseConductorMana method*), 1023
del_host() (*ironic.pxe_filter.service.PXEFilterManager method*), 1554
DelAttributeAction (*class in ironic.common.inspection_rules.actions*), 908
delete() (*ironic.api.controllers.v1.allocation.AllocationsControll method*), 833
delete() (*ironic.api.controllers.v1.allocation.NodeAllocationCon method*), 835
delete() (*ironic.api.controllers.v1.chassis.ChassisController*

method), 836
delete() (*ironic.api.controllers.v1.deploy_template.DeployTemplatesController method*), 839
delete() (*ironic.api.controllers.v1.inspection_rule.InspectionRulesController method*), 844
delete() (*ironic.api.controllers.v1.node.NodeMaintenanceController method*), 848
delete() (*ironic.api.controllers.v1.node.NodesController method*), 853
delete() (*ironic.api.controllers.v1.node.NodeTraitsController method*), 852
delete() (*ironic.api.controllers.v1.node.NodeVIFController method*), 852
delete() (*ironic.api.controllers.v1.node.NodeVmediaController method*), 853
delete() (*ironic.api.controllers.v1.port.PortsController method*), 860
delete() (*ironic.api.controllers.v1.portgroup.PortgroupsController method*), 862
delete() (*ironic.api.controllers.v1.runbook.RunbooksController method*), 866
delete() (*ironic.api.controllers.v1.volume_connector.VolumeConnectorsController method*), 887
delete() (*ironic.api.controllers.v1.volume_target.VolumeTargetsController method*), 890
delete() (*ironic.objects.bios.BIOSSetting class method*), 1452
delete() (*ironic.objects.bios.BIOSSettingList class method*), 1455
delete() (*ironic.objects.conductor.Conductor class method*), 1460
DELETE_ALLOWED_STATES (*in module ironic.common.states*), 1006
delete_bios_setting_list() (*ironic.db.api.Connection method*), 1154
delete_bios_setting_list() (*ironic.db.sqlalchemy.api.Connection method*), 1102
delete_conductor() (*ironic.db.api.Connection method*), 1154
delete_conductor() (*ironic.db.sqlalchemy.api.Connection method*), 1102
delete_configuration() (*ironic.drivers.base.RAIDInterface method*), 1416
delete_configuration() (*ironic.drivers.modules.agent.AgentRAID method*), 1298
delete_configuration() (*ironic.drivers.modules.drac.raid.DracRedfishRAID method*), 1196
delete_configuration() (*ironic.drivers.modules.fake.FakeRAID method*), 1346
delete_configuration() (*ironic.drivers.modules.ilo.raid.Ilo5RAID method*), 1224
delete_configuration() (*ironic.drivers.modules.noop.NoRAID method*), 1371
delete_configuration() (*ironic.drivers.modules.redfish.raid.RedfishRAID method*), 1278
delete_lag() (*in module ironic.networking.api*), 1436
delete_lag() (*ironic.drivers.modules.switch.base.NoOpSwitchDriver method*), 1290
delete_lag() (*ironic.drivers.modules.switch.base.SwitchDriverBase method*), 1290
delete_lag() (*ironic.drivers.modules.switch.generic_switch.GenericSwitchDriver method*), 1293
delete_lag() (*ironic.networking.manager.NetworkingManager method*), 1428
delete_lag() (*ironic.networking.rpcapi.NetworkingAPI method*), 1439
delete_node_tag() (*ironic.db.api.Connection method*), 1154
delete_node_tag() (*ironic.db.sqlalchemy.api.Connection method*), 1102
delete_node_trait() (*ironic.db.api.Connection method*), 1154
delete_node_trait() (*ironic.db.sqlalchemy.api.Connection method*), 1103
delete_object() (*ironic.common.swift.SwiftAPI method*), 1011
delete_subscription() (*ironic.drivers.modules.redfish.vendor.RedfishVendorPassthru method*), 1284
DELETED (*in module ironic.common.states*), 1006
DELETING (*in module ironic.common.states*), 1006
DelPortAttributeAction (*class in ironic.common.inspection_rules.actions*), 908
dependencies (*ironic.drivers.modules.inspector.hooks.base.InspectionHook attribute*), 1227
dependencies (*ironic.drivers.modules.inspector.hooks.local_link_

attribute), 1229
dependencies (*ironic.drivers.modules.inspector.hooks.physicalnetwork.PhysicalNetworkHook attribute*), 1230
dependencies (*ironic.drivers.modules.inspector.hooks.ports.PortsHook attribute*), 1231
DEPLOY (*in module ironic.common.states*), 1007
deploy (*ironic.drivers.base.BareDriver attribute*), 1391
deploy() (*ironic.drivers.base.DeployInterface method*), 1397
deploy() (*ironic.drivers.modules.agent.CustomAgentDeploy method*), 1301
deploy() (*ironic.drivers.modules.ansible.deploy.AnsibleDeploy method*), 1189
deploy() (*ironic.drivers.modules.autodetect.AutodetectDeploy method*), 1319
deploy() (*ironic.drivers.modules.fake.FakeDeploy method*), 1337
deploy() (*ironic.drivers.modules.noop.NoDeploy method*), 1368
deploy() (*ironic.drivers.modules.pxe.PXEAnacondaDeploy method*), 1375
deploy() (*ironic.drivers.modules.ramdisk.RamdiskDeploy method*), 1378
deploy_has_started() (*ironic.drivers.modules.pxe.PXEAnacondaDeploy method*), 1375
deploy_interface (*ironic.db.sqlalchemy.models.Node attribute*), 1139
deploy_interface (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1141
deploy_interface (*ironic.objects.node.Node property*), 1482
deploy_interface (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1495
deploy_interface (*ironic.objects.node.NodeCRUDPayload property*), 1490
deploy_interface (*ironic.objects.node.NodePayload property*), 1500
deploy_interface (*ironic.objects.node.NodeSetPowerStatePayload property*), 1505
deploy_interface (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1509
deploy_is_done() (*ironic.drivers.modules.pxe.PXEAnacondaDeploy method*), 1375
deploy_step (*ironic.db.sqlalchemy.models.Node attribute*), 1139
deploy_step (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1141
deploy_step (*ironic.objects.node.Node property*), 1482
deploy_step (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1495
deploy_step (*ironic.objects.node.NodeCRUDPayload property*), 1491
deploy_step (*ironic.objects.node.NodePayload property*), 1500
deploy_step (*ironic.objects.node.NodeSetPowerStatePayload property*), 1505
deploy_step (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1509
deploy_step() (*in module ironic.drivers.base*), 1422
deploy_template (*ironic.db.sqlalchemy.models.DeployTemplateStep attribute*), 1137
deploy_template_id (*ironic.db.sqlalchemy.models.DeployTemplateStep attribute*), 1137
DEPLOYDONE (*in module ironic.common.states*), 1007
DEPLOYFAIL (*in module ironic.common.states*), 1007
DEPLOYHOLD (*in module ironic.common.states*), 1007
DEPLOYING (*in module ironic.common.states*), 1007
deploying_error_handler() (*in module ironic.conductor.utils*), 1073
DeployInterface (*class in ironic.drivers.base*), 1397
Deployment (*class in ironic.objects.deployment*), 1467
DEPLOYMENT (*ironic.conductor.utils.StepFlow attribute*), 1071
DeployTemplate (*class in ironic.db.sqlalchemy.models*), 1137
DeployTemplate (*class in ironic.objects.deploy_template*), 1462
DeployTemplateAlreadyExists, 946
DeployTemplateCRUDNotification (*class in ironic.objects.deploy_template*), 1466
DeployTemplateCRUDPayload (*class in ironic.objects.deploy_template*), 1466

DeployTemplateDuplicateName, 946
DeployTemplateNotFound, 946
DeployTemplatesController (*class in ironic.api.controllers.v1.deploy_template*), 839
DeployTemplateStep (*class in ironic.db.sqlalchemy.models*), 1137
DEPLOYWAIT (*in module ironic.common.states*), 1007
deregister_on_shutdown (*ironic.conductor.rpc_service.RPCService property*), 1038
description (*ironic.db.sqlalchemy.models.Chassis attribute*), 1136
description (*ironic.db.sqlalchemy.models.InspectionRule attribute*), 1138
description (*ironic.db.sqlalchemy.models.Node attribute*), 1139
description (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1141
description (*ironic.db.sqlalchemy.models.Port attribute*), 1145
description (*ironic.objects.chassis.Chassis property*), 1456
description (*ironic.objects.chassis.ChassisCRUDPayload property*), 1459
description (*ironic.objects.inspection_rule.InspectionRule property*), 1477
description (*ironic.objects.inspection_rule.InspectionRuleCRUDPayload property*), 1480
description (*ironic.objects.node.Node property*), 1482
description (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1495
description (*ironic.objects.node.NodeCRUDPayload property*), 1491
description (*ironic.objects.node.NodePayload property*), 1500
description (*ironic.objects.node.NodeSetPowerStatePayload property*), 1505
description (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1510
description (*ironic.objects.port.Port property*), 1520
description (*ironic.objects.port.PortCRUDPayload property*), 1527
deserialize_context() (*ironic.common.rpc.RequestContextSerializer method*), 1004
deserialize_entity() (*ironic.common.rpc.RequestContextSerializer method*), 1004
destroy() (*ironic.objects.allocation.Allocation method*), 1444
destroy() (*ironic.objects.chassis.Chassis method*), 1456
destroy() (*ironic.objects.deploy_template.DeployTemplate method*), 1463
destroy() (*ironic.objects.deployment.Deployment method*), 1467
destroy() (*ironic.objects.inspection_rule.InspectionRule method*), 1477
destroy() (*ironic.objects.node.Node method*), 1482
destroy() (*ironic.objects.node_history.NodeHistory method*), 1513
destroy() (*ironic.objects.node_inventory.NodeInventory method*), 1516
destroy() (*ironic.objects.port.Port method*), 1520
destroy() (*ironic.objects.portgroup.Portgroup method*), 1528
destroy() (*ironic.objects.runbook.Runbook method*), 1535
destroy() (*ironic.objects.trait.Trait class method*), 1540
destroy() (*ironic.objects.trait.TraitList class method*), 1542
destroy() (*ironic.objects.volume_connector.VolumeConnector method*), 1543
destroy() (*ironic.objects.volume_target.VolumeTarget method*), 1548
destroy_allocation() (*ironic.conductor.manager.ConductorManager method*), 1032
destroy_allocation() (*ironic.conductor.rpcapi.ConductorAPI method*), 1044
destroy_allocation() (*ironic.db.api.Connection method*), 1155
destroy_allocation() (*ironic.db.sqlalchemy.api.Connection method*), 1103
destroy_chassis() (*ironic.db.api.Connection method*), 1155
destroy_chassis() (*ironic.db.sqlalchemy.api.Connection method*), 1103
destroy_deploy_template() (*ironic.db.api.Connection method*), 1155

destroy_deploy_template() (*ironic.db.sqlalchemy.api.Connection method*), 1103
destroy_floppy_image_from_web_server() (*in module ironic.drivers.modules.ilo.common*), 1208
destroy_http_instance_images() (*in module ironic.drivers.modules.deploy_utils*), 1325
destroy_images() (*in module ironic.drivers.modules.deploy_utils*), 1325
destroy_inspection_rule() (*ironic.db.api.Connection method*), 1155
destroy_inspection_rule() (*ironic.db.sqlalchemy.api.Connection method*), 1103
destroy_node() (*ironic.conductor.manager.ConductorManager method*), 1032
destroy_node() (*ironic.conductor.rpcapi.ConductorAPI method*), 1044
destroy_node() (*ironic.db.api.Connection method*), 1155
destroy_node() (*ironic.db.sqlalchemy.api.Connection method*), 1104
destroy_node_history_by_uuid() (*ironic.db.api.Connection method*), 1155
destroy_node_history_by_uuid() (*ironic.db.sqlalchemy.api.Connection method*), 1104
destroy_node_inventory_by_node_id() (*ironic.db.api.Connection method*), 1155
destroy_node_inventory_by_node_id() (*ironic.db.sqlalchemy.api.Connection method*), 1104
destroy_port() (*ironic.conductor.manager.ConductorManager method*), 1032
destroy_port() (*ironic.conductor.rpcapi.ConductorAPI method*), 1044
destroy_port() (*ironic.db.api.Connection method*), 1156
destroy_port() (*ironic.db.sqlalchemy.api.Connection method*), 1104
destroy_portgroup() (*ironic.conductor.manager.ConductorManager method*), 1032
destroy_portgroup() (*ironic.conductor.rpcapi.ConductorAPI method*), 1045
destroy_portgroup() (*ironic.db.api.Connection method*), 1156
destroy_portgroup() (*ironic.db.sqlalchemy.api.Connection method*), 1104
destroy_runbook() (*ironic.db.api.Connection method*), 1156
destroy_runbook() (*ironic.db.sqlalchemy.api.Connection method*), 1104
destroy_volume_connector() (*ironic.conductor.manager.ConductorManager method*), 1032
destroy_volume_connector() (*ironic.conductor.rpcapi.ConductorAPI method*), 1045
destroy_volume_connector() (*ironic.db.api.Connection method*), 1156
destroy_volume_connector() (*ironic.db.sqlalchemy.api.Connection method*), 1104
destroy_volume_target() (*ironic.conductor.manager.ConductorManager method*), 1032
destroy_volume_target() (*ironic.conductor.rpcapi.ConductorAPI method*), 1045
destroy_volume_target() (*ironic.db.api.Connection method*), 1156
destroy_volume_target() (*ironic.db.sqlalchemy.api.Connection method*), 1104
detach_virtual_media() (*ironic.conductor.manager.ConductorManager method*), 1032
detach_virtual_media() (*ironic.conductor.rpcapi.ConductorAPI method*), 1046
detach_virtual_media() (*ironic.drivers.base.ManagementInterface method*), 1402

detach_virtual_media() (*ironic.drivers.modules.redfish.management.RedfishManagement method*), 1268
detach_volumes() (*in module ironic.common.cinder*), 937
detach_volumes() (*ironic.drivers.base.StorageInterface method*), 1419
detach_volumes() (*ironic.drivers.modules.fake.FakeStorage method*), 1348
detach_volumes() (*ironic.drivers.modules.storage.cinder.CinderStorage method*), 1286
detach_volumes() (*ironic.drivers.modules.storage.external.ExternalStorage method*), 1287
detach_volumes() (*ironic.drivers.modules.storage.noop.NoopStorage method*), 1288
detail() (*ironic.api.controllers.v1.chassis.ChassisController method*), 837
detail() (*ironic.api.controllers.v1.node.NodesController method*), 853
detail() (*ironic.api.controllers.v1.port.PortsController method*), 860
detail() (*ironic.api.controllers.v1.portgroup.PortgroupController method*), 862
detail_fields (*ironic.api.controllers.v1.node.NodeHistoryController attribute*), 848
detect_file_format() (*in module ironic.common.images*), 974
detect_vendor() (*ironic.drivers.base.ManagementInterface method*), 1403
detect_vendor() (*ironic.drivers.modules.ipmitool.IPMIManagement method*), 1358
detect_vendor() (*ironic.drivers.modules.redfish.management.RedfishManagement method*), 1268
detected (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1384
dhcp_options_for_instance() (*in module ironic.common.pxe_utils*), 998
DHCPFactory (*class in ironic.common.dhcp_factory*), 940
DHCPLoadError, 946
dict_valid() (*in module ironic.common.args*), 927
DictCollectionMetricLogger (*class in ironic.common.metrics_collector*), 982
direct_deploy_should_convert_raw_image() (*in module ironic.drivers.modules.deploy_utils*), 1325
DirectoryNotWritable, 946
disable_power_off (*ironic.db.sqlalchemy.models.Node attribute*), 1139
disable_power_off (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1141
disable_power_off (*ironic.objects.node.Node property*), 1482
disable_power_off (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1495
disable_power_off (*ironic.objects.node.NodeCRUDPayload property*), 1491
disable_power_off (*ironic.objects.node.NodePayload property*), 1501
disable_power_off (*ironic.objects.node.NodeSetPowerStatePayload property*), 1505
disable_power_off (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1510
disable_ramdisk (*ironic.db.sqlalchemy.models.Runbook attribute*), 1146
disable_ramdisk (*ironic.objects.runbook.Runbook property*), 1535
disable_ramdisk (*ironic.objects.runbook.RunbookCRUDPayload property*), 1539
disallowed_fields() (*in module ironic.api.controllers.v1.utils*), 879
DISK (*in module ironic.common.boot_devices*), 939
DISK (*in module ironic.common.components*), 939
DnsmasqDHCPApi (*class in ironic.dhcp.dnsmasq*), 1185
do_allocate() (*in module ironic.conductor.allocations*), 1022
do_attach_virtual_media() (*in module ironic.conductor.manager*), 1035
do_detach_virtual_media() (*in module ironic.conductor.manager*), 1035

do_next_clean_step() (*in module ironic.conductor.cleaning*), 1024
do_next_deploy_step() (*in module ironic.conductor.deployments*), 1028
do_next_service_step() (*in module ironic.conductor.servicing*), 1065
do_node_clean() (*in module ironic.conductor.cleaning*), 1025
do_node_clean() (*ironic.conductor.manager.ConductorManager method*), 1033
do_node_clean() (*ironic.conductor.rpcapi.ConductorAPI method*), 1046
do_node_clean_abort() (*in module ironic.conductor.cleaning*), 1025
do_node_deploy() (*in module ironic.conductor.deployments*), 1028
do_node_deploy() (*ironic.conductor.manager.ConductorManager method*), 1033
do_node_deploy() (*ironic.conductor.rpcapi.ConductorAPI method*), 1047
do_node_deploy_abort() (*in module ironic.conductor.deployments*), 1029
do_node_rescue() (*ironic.conductor.manager.ConductorManager method*), 1033
do_node_rescue() (*ironic.conductor.rpcapi.ConductorAPI method*), 1047
do_node_service() (*in module ironic.conductor.servicing*), 1065
do_node_service() (*ironic.conductor.manager.ConductorManager method*), 1033
do_node_service() (*ironic.conductor.rpcapi.ConductorAPI method*), 1047
do_node_service_abort() (*in module ironic.conductor.servicing*), 1065
do_node_tear_down() (*ironic.conductor.manager.ConductorManager method*), 1033
do_node_tear_down() (*ironic.conductor.rpcapi.ConductorAPI method*), 1048
do_node_unrescue() (*ironic.conductor.manager.ConductorManager method*), 1033
do_node_unrescue() (*ironic.conductor.rpcapi.ConductorAPI method*), 1048
do_node_verify() (*in module ironic.conductor.verify*), 1084
do_provisioning_action() (*ironic.conductor.manager.ConductorManager method*), 1033
do_provisioning_action() (*ironic.conductor.rpcapi.ConductorAPI method*), 1049
do_sync_power_state() (*in module ironic.conductor.manager*), 1035
do_version_changes_for_db() (*ironic.objects.base.IronicObject method*), 1449
doc (*ironic.api.functions.FunctionDefinition attribute*), 899
downgrade() (*in module ironic.db.sqlalchemy.migration*), 1134
downgrade_lock() (*ironic.conductor.task_manager.TaskManager method*), 1069
download() (*in module ironic.drivers.modules.ilo.common*), 1208
download() (*ironic.common.glance_service.image_service.Glanc method*), 905
download() (*ironic.common.image_service.BaseImageService method*), 962
download() (*ironic.common.image_service.CifsImageService method*), 963
download() (*ironic.common.image_service.FileImageService method*), 964
download() (*ironic.common.image_service.HttpImageService method*), 965
download() (*ironic.common.image_service.NfsImageService method*), 967
download() (*ironic.common.image_service.OciImageService method*), 968
download_blob_from_manifest() (*ironic.common.oci_registry.OciClient method*), 992
download_size() (*in module ironic.common.images*), 974
download_to_temp() (*in module ironic.drivers.modules.redfish.firmware_utils*), 1265
DracOperationError, 946
DracRedfishBIOS (*class in ironic.drivers.modules.drac.bios*), 1191

DracRedfishInspect (*class in ironic.drivers.modules.drac.inspect*), 1192
DracRedfishManagement (*class in ironic.drivers.modules.drac.management*), 1193
DracRedfishPower (*class in ironic.drivers.modules.drac.power*), 1195
DracRedfishRAID (*class in ironic.drivers.modules.drac.raid*), 1195
DracRedfishVendorPassthru (*class in ironic.drivers.modules.drac.vendor_passthru*), 1197
DracRedfishVirtualMediaBoot (*class in ironic.drivers.modules.drac.boot*), 1192
driver (*ironic.db.sqlalchemy.models.Node attribute*), 1139
driver (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
driver (*ironic.objects.node.Node property*), 1482
driver (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1495
driver (*ironic.objects.node.NodeCRUDPayload property*), 1491
driver (*ironic.objects.node.NodePayload property*), 1501
driver (*ironic.objects.node.NodeSetPowerStatePayload property*), 1505
driver (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1510
driver_info (*ironic.db.sqlalchemy.models.Node attribute*), 1139
driver_info (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
driver_info (*ironic.objects.node.Node property*), 1482
driver_info (*ironic.objects.node.NodeCRUDPayload property*), 1491
driver_internal_info (*ironic.db.sqlalchemy.models.Node attribute*), 1139
driver_internal_info (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
driver_internal_info (*ironic.objects.node.Node property*), 1482
driver_internal_info (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1510
DRIVER_NAME (*ironic.drivers.modules.switch.base.NoOpSwitchDri attribute*), 1289
DRIVER_NAME (*ironic.drivers.modules.switch.base.SwitchDriverBa attribute*), 1290
DRIVER_NAME (*ironic.drivers.modules.switch.generic_switch.Gener attribute*), 1293
driver_passthru() (*in module ironic.drivers.base*), 1423
driver_sanitize() (*in module ironic.api.controllers.v1.driver*), 843
driver_validate() (*ironic.drivers.base.VendorInterface method*), 1419
driver_validate() (*ironic.drivers.modules.noop.NoVendor method*), 1372
driver_vendor_passthru() (*ironic.conductor.manager.ConductorManager method*), 1033
driver_vendor_passthru() (*ironic.conductor.rpcapi.ConductorAPI method*), 1049
DRIVER_VERSION (*ironic.drivers.modules.switch.base.NoOpSwitchDriver attribute*), 1289
DRIVER_VERSION (*ironic.drivers.modules.switch.base.SwitchDriverBase attribute*), 1290
DRIVER_VERSION (*ironic.drivers.modules.switch.generic_switch.GenericSwi attribute*), 1293
DriverLoadError, 946
DriverNotFound, 946
DriverNotFoundInEntrypoint, 947
DriverOperationError, 947
DriverPassthruController (*class in ironic.api.controllers.v1.driver*), 841
DriverRaidController (*class in ironic.api.controllers.v1.driver*), 841
drivers (*ironic.db.sqlalchemy.models.Conductor attribute*), 1136
drivers (*ironic.objects.conductor.Conductor property*), 1460
DriversController (*class in ironic.api.controllers.v1.driver*), 842
dump_sdr() (*in module ironic.drivers.modules.ipmitool*), 1364
Duplicate, 947
duplicate_steps() (*in module ironic.api.controllers.v1.utils*), 879
DuplicateName, 947

DuplicateNodeOnLookup, 947
dynamic_portgroup (*ironic.common.trait_based_networking.base.Portgroup attribute*), 922
dynamic_portgroup (*ironic.db.sqlalchemy.models.Portgroup attribute*), 1145
dynamic_portgroup (*ironic.objects.portgroup.Portgroup property*), 1529
dynamic_portgroup (*ironic.objects.portgroup.PortgroupCRUDPayload property*), 1534

E

eject_vmedia() (*in module ironic.drivers.modules.redfish.boot*), 1263
eject_vmedia() (*ironic.drivers.modules.redfish.vendor.RedfishVendorPassthru method*), 1284
eject_vmedia_devices() (*in module ironic.drivers.modules.ilo.common*), 1208
emit() (*ironic.objects.notification.NotificationBase method*), 1518
emit_console_notification() (*in module ironic.conductor.notification_utils*), 1036
emit_end_notification() (*in module ironic.api.controllers.v1.notification_utils*), 859
emit_power_set_notification() (*in module ironic.conductor.notification_utils*), 1036
emit_power_state_corrected_notification() (*in module ironic.conductor.notification_utils*), 1036
emit_provision_set_notification() (*in module ironic.conductor.notification_utils*), 1037
emit_start_notification() (*in module ironic.api.controllers.v1.notification_utils*), 859
EmptyContext (*class in ironic.common.json_rpc.server*), 916
EmptyOperator (*class in ironic.common.inspection_rules.operators*), 911
enabled_supported_interfaces() (*in module ironic.common.driver_factory*), 943
END (*ironic.objects.fields.NotificationStatus attribute*), 1472
ENROLL (*in module ironic.common.states*), 1007
ensure_next_boot_device() (*in module ironic.drivers.utils*), 1431
ensure_rpc_transport() (*in module ironic.common.service*), 1005
ensure_thread_contain_context() (*ironic.common.context.RequestContext method*), 939
ensure_tree() (*in module ironic.common.pxe_utils*), 998
EnumField (*class in ironic.objects.fields*), 1470
EqOperator (*class in ironic.common.inspection_rules.operators*), 911
EQUALITY (*ironic.common.trait_based_networking.base.Comparat attribute*), 919
equality() (*ironic.common.trait_based_networking.grammar.par method*), 917
erase_devices() (*ironic.drivers.modules.ilo.management.Ilo5Management method*), 1215
ERROR (*in module ironic.common.states*), 1007
ERROR (*ironic.objects.fields.NotificationLevel attribute*), 1471
ERROR (*ironic.objects.fields.NotificationStatus attribute*), 1472
ESSENTIAL_PROPERTIES (*ironic.drivers.base.InspectInterface attribute*), 1401
eval() (*ironic.common.trait_based_networking.base.Comparator method*), 919
eval() (*ironic.common.trait_based_networking.base.CompoundE method*), 919
eval() (*ironic.common.trait_based_networking.base.FilterExpres method*), 919
eval() (*ironic.common.trait_based_networking.base.FunctionExp method*), 920
eval() (*ironic.common.trait_based_networking.base.Operator method*), 921
eval() (*ironic.common.trait_based_networking.base.ParenExpres method*), 921
eval() (*ironic.common.trait_based_networking.base.SingleExpres method*), 923
event (*ironic.db.sqlalchemy.models.NodeHistory attribute*), 1143
event (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1510
event (*ironic.objects.node_history.NodeHistory property*), 1513

event_type (*ironic.db.sqlalchemy.models.NodeHistory attribute*), 1143
event_type (*ironic.objects.allocation.AllocationCRUDNotification property*), 1447
event_type (*ironic.objects.chassis.ChassisCRUDNotification property*), 1458
event_type (*ironic.objects.deploy_template.DeployTemplateCRUDNotification property*), 1466
event_type (*ironic.objects.inspection_rule.InspectionRuleCRUDNotification property*), 1479
event_type (*ironic.objects.node.NodeConsoleNotification property*), 1494
event_type (*ironic.objects.node.NodeCorrectedPowerStateNotification property*), 1494
event_type (*ironic.objects.node.NodeCRUDNotification property*), 1489
event_type (*ironic.objects.node.NodeMaintenanceNotification property*), 1499
event_type (*ironic.objects.node.NodeSetPowerStateNotification property*), 1504
event_type (*ironic.objects.node.NodeSetProvisionStateNotification property*), 1508
event_type (*ironic.objects.node_history.NodeHistory property*), 1513
event_type (*ironic.objects.notification.NotificationBase property*), 1518
event_type (*ironic.objects.port.PortCRUDNotification property*), 1526
event_type (*ironic.objects.portgroup.PortgroupCRUDNotification property*), 1533
event_type (*ironic.objects.runbook.RunbookCRUDNotification property*), 1538
event_type (*ironic.objects.volume_connector.VolumeConnectorCRUDNotification property*), 1546
event_type (*ironic.objects.volume_target.VolumeTargetCRUDNotification property*), 1552
events_valid() (*in module ironic.api.controllers.v1.event*), 843
EventsController (*class in ironic.api.controllers.v1.event*), 843
EventType (*class in ironic.objects.notification*), 1517
exclude_current_conductor() (*in module ironic.conductor.utils*), 1073
ExclusiveLockRequired, 947
execute() (*in module ironic.common.utils*), 1012
execute_action() (*ironic.common.inspection_rules.actions.ActionBase method*), 908
execute_bootc_install() (*ironic.drivers.modules.agent.BootcAgentDeploy method*), 1300
execute_clean_step() (*in module ironic.drivers.modules.agent_base*), 1308
execute_clean_step() (*ironic.drivers.base.BaseInterface method*), 1392
execute_clean_step() (*ironic.drivers.modules.agent_base.AgentBaseMixin method*), 1303
execute_clean_step() (*ironic.drivers.modules.agent_client.AgentClient method*), 1310
execute_clean_step() (*ironic.drivers.modules.ansible.deploy.AnsibleDeploy method*), 1189
execute_deploy_step() (*ironic.drivers.base.BaseInterface method*), 1392
execute_deploy_step() (*ironic.drivers.modules.agent.CustomAgentDeploy method*), 1301
execute_deploy_step() (*ironic.drivers.modules.agent_client.AgentClient method*), 1311
execute_oem_manager_method() (*in module ironic.drivers.modules.drac.utils*), 1196
execute_service_step() (*ironic.drivers.base.BaseInterface method*), 1393
execute_service_step() (*ironic.drivers.modules.agent_base.AgentBaseMixin method*), 1304
execute_service_step() (*ironic.drivers.modules.agent_client.AgentClient method*), 1311
execute_step() (*in module ironic.drivers.modules.agent_base*), 1308
execute_step_on_child_nodes() (*in module ironic.conductor.cleaning*), 1025
execute_step_on_child_nodes() (*in module ironic.conductor.deployments*), 1029
execute_step_on_child_nodes() (*in module ironic.conductor.servicing*), 1066
execute_verify_step() (*ironic.drivers.base.BaseInterface method*), 1393
execute_with_loop() (*ironic.common.inspection_rules.actions.ActionBase method*), 908

exists() (*ironic.objects.trait.Trait class method*), 1540
ExplicitAbsolutePath (*class in ironic.conf.types*), 1089
export_configuration() (*ironic.drivers.modules.drac.management.DracRedfishManagement method*), 1193
EXPORT_CONFIGURATION_ARGSINFO (*ironic.drivers.modules.drac.management.DracRedfishManagement attribute*), 1193
expose() (*in module ironic.api.method*), 902
ExtendAttributeAction (*class in ironic.common.inspection_rules.actions*), 908
ExtendPluginDataAction (*class in ironic.common.inspection_rules.actions*), 908
ExtendPortAttributeAction (*class in ironic.common.inspection_rules.actions*), 909
ExternalStorage (*class in ironic.drivers.modules.storage.external*), 1287
extra (*ironic.db.sqlalchemy.models.Allocation attribute*), 1135
extra (*ironic.db.sqlalchemy.models.Chassis attribute*), 1136
extra (*ironic.db.sqlalchemy.models.DeployTemplate attribute*), 1137
extra (*ironic.db.sqlalchemy.models.Node attribute*), 1139
extra (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
extra (*ironic.db.sqlalchemy.models.Port attribute*), 1145
extra (*ironic.db.sqlalchemy.models.Portgroup attribute*), 1146
extra (*ironic.db.sqlalchemy.models.Runbook attribute*), 1146
extra (*ironic.db.sqlalchemy.models.VolumeConnector attribute*), 1147
extra (*ironic.db.sqlalchemy.models.VolumeTarget attribute*), 1147
extra (*ironic.objects.allocation.Allocation property*), 1444
extra (*ironic.objects.allocation.AllocationCRUDPayload property*), 1448
extra (*ironic.objects.chassis.Chassis property*), 1457
extra (*ironic.objects.chassis.ChassisCRUDPayload property*), 1459
extra (*ironic.objects.deploy_template.DeployTemplate property*), 1463
extra (*ironic.objects.deploy_template.DeployTemplateCRUDPayload property*), 1467
extra (*ironic.objects.node.Node property*), 1482
extra (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1495
extra (*ironic.objects.node.NodeCRUDPayload property*), 1491
extra (*ironic.objects.node.NodePayload property*), 1501
extra (*ironic.objects.node.NodeSetPowerStatePayload property*), 1505
extra (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1510
extra (*ironic.objects.port.Port property*), 1520
extra (*ironic.objects.port.PortCRUDPayload property*), 1527
extra (*ironic.objects.portgroup.Portgroup property*), 1529
extra (*ironic.objects.portgroup.PortgroupCRUDPayload property*), 1534
extra (*ironic.objects.runbook.Runbook property*), 1535
extra (*ironic.objects.runbook.RunbookCRUDPayload property*), 1539
extra (*ironic.objects.volume_connector.VolumeConnector property*), 1543
extra (*ironic.objects.volume_connector.VolumeConnectorCRUDP property*), 1547
extra (*ironic.objects.volume_target.VolumeTarget property*), 1548
extra (*ironic.objects.volume_target.VolumeTargetCRUDPayload property*), 1552
extra_options (*ironic.api.functions.FunctionDefinition attribute*), 899
ExtraHardwareHook (*class in ironic.drivers.modules.inspector.hooks.extra_hardware*), 1229

F

factory_reset() (*ironic.drivers.base.BIOSInterface method*), 1390
factory_reset() (*ironic.drivers.modules.fake.FakeBIOS method*), 1334
factory_reset() (*ironic.drivers.modules.ilo.bios.IloBIOS method*), 1198
factory_reset()

(*ironic.drivers.modules.noop.NoBIOS method*), 1367
factory_reset() (*ironic.drivers.modules.redfish.bios.RedfishBIOS method*), 1258
fail (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU attribute*), 1384
fail_on_error() (*in module ironic.conductor.utils*), 1073
FailAction (*class in ironic.common.inspection_rules.actions*), 909
FailedToCleanDHCPOpts, 947
FailedToGetIPAddressOnPort, 947
FailedToGetSensorData, 947
FailedToParseSensorData, 947
FailedToUpdateDHCPOptOnPort, 947
FailedToUpdateMacOnPort, 947
FailMixin (*class in ironic.drivers.modules.noop*), 1366
FakeBIOS (*class in ironic.drivers.modules.fake*), 1333
FakeBoot (*class in ironic.drivers.modules.fake*), 1334
FakeConsole (*class in ironic.drivers.modules.fake*), 1336
FakeConsoleContainer (*class in ironic.console.container.fake*), 1090
FakeDeploy (*class in ironic.drivers.modules.fake*), 1337
FakeFirmware (*class in ironic.drivers.modules.fake*), 1339
FakeGraphicalConsole (*class in ironic.drivers.modules.fake*), 1340
FakeHardware (*class in ironic.drivers.fake_hardware*), 1425
FakeInspect (*class in ironic.drivers.modules.fake*), 1340
FakeManagement (*class in ironic.drivers.modules.fake*), 1341
FakePower (*class in ironic.drivers.modules.fake*), 1344
FakeRAID (*class in ironic.drivers.modules.fake*), 1346
FakeRescue (*class in ironic.drivers.modules.fake*), 1347
FakeStorage (*class in ironic.drivers.modules.fake*), 1347
FakeVendorA (*class in ironic.drivers.modules.fake*), 1348
FakeVendorB (*class in ironic.drivers.modules.fake*), 1349
fast_track_able() (*in module ironic.conductor.utils*), 1073
fast_track_enabled() (*in module ironic.common.utils*), 1013
FASTTRACK_LOOKUP_ALLOWED_STATES (*in module ironic.common.states*), 1007
fault (*ironic.db.sqlalchemy.models.Node attribute*), 1139
fault (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
fault (*ironic.objects.node.Node property*), 1482
fault (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1495
fault (*ironic.objects.node.NodeCRUDPayload property*), 1491
fault (*ironic.objects.node.NodePayload property*), 1501
fault (*ironic.objects.node.NodeSetPowerStatePayload property*), 1505
fault (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1510
faultstring (*ironic.common.exception.ClientSideError property*), 945
faultstring (*ironic.common.exception.InvalidInput property*), 951
faultstring (*ironic.common.exception.UnknownArgument property*), 957
faultstring (*ironic.common.exception.UnknownAttribute property*), 957
fetch() (*in module ironic.common.images*), 974
fetch_image() (*ironic.drivers.modules.image_cache.ImageCache method*), 1351
fetch_images() (*in module ironic.drivers.modules.deploy_utils*), 1325
fetch_into() (*in module ironic.common.images*), 974
fields (*ironic.objects.allocation.Allocation attribute*), 1444
fields (*ironic.objects.allocation.AllocationCRUDNotification attribute*), 1447
fields (*ironic.objects.allocation.AllocationCRUDPayload attribute*), 1448
fields (*ironic.objects.base.IronicObject attribute*), 1450
fields (*ironic.objects.bios.BIOSSetting attribute*), 1452
fields (*ironic.objects.bios.BIOSSettingList attribute*), 1455
fields (*ironic.objects.chassis.Chassis attribute*),

1457
fields (ironic.objects.chassis.ChassisCRUDNotification attribute), 1458
fields (ironic.objects.chassis.ChassisCRUDPayload attribute), 1459
fields (ironic.objects.conductor.Conductor attribute), 1460
fields (ironic.objects.deploy_template.DeployTemplate attribute), 1463
fields (ironic.objects.deploy_template.DeployTemplateCRUDNotification attribute), 1466
fields (ironic.objects.deploy_template.DeployTemplateCRUDPayload attribute), 1467
fields (ironic.objects.deployment.Deployment attribute), 1468
fields (ironic.objects.firmware.FirmwareComponent attribute), 1473
fields (ironic.objects.firmware.FirmwareComponentList attribute), 1474
fields (ironic.objects.inspection_rule.InspectionRule attribute), 1477
fields (ironic.objects.inspection_rule.InspectionRuleCRUDNotification attribute), 1479
fields (ironic.objects.inspection_rule.InspectionRuleCRUDPayload attribute), 1480
fields (ironic.objects.node.Node attribute), 1482
fields (ironic.objects.node.NodeConsoleNotification attribute), 1494
fields (ironic.objects.node.NodeCorrectedPowerStateNotification attribute), 1494
fields (ironic.objects.node.NodeCorrectedPowerStatePayload attribute), 1496
fields (ironic.objects.node.NodeCRUDNotification attribute), 1489
fields (ironic.objects.node.NodeCRUDPayload attribute), 1491
fields (ironic.objects.node.NodeMaintenanceNotification attribute), 1499
fields (ironic.objects.node.NodePayload attribute), 1501
fields (ironic.objects.node.NodeSetPowerStateNotification attribute), 1504
fields (ironic.objects.node.NodeSetPowerStatePayload attribute), 1505
fields (ironic.objects.node.NodeSetProvisionStateNotification attribute), 1508
fields (ironic.objects.node.NodeSetProvisionStatePayload attribute), 1510
fields (ironic.objects.node_history.NodeHistory attribute), 1513
fields (ironic.objects.node_inventory.NodeInventory attribute), 1516
fields (ironic.objects.notification.EventType attribute), 1517
fields (ironic.objects.notification.NotificationBase attribute), 1518
fields (ironic.objects.notification.NotificationPayloadBase attribute), 1519
fields (ironic.objects.notification.NotificationPublisher attribute), 1519
fields (ironic.objects.port.Port attribute), 1520
fields (ironic.objects.port.PortCRUDNotification attribute), 1526
fields (ironic.objects.port.PortCRUDPayload attribute), 1527
fields (ironic.objects.portgroup.Portgroup attribute), 1529
fields (ironic.objects.portgroup.PortgroupCRUDNotification attribute), 1533
fields (ironic.objects.portgroup.PortgroupCRUDPayload attribute), 1534
fields (ironic.objects.runbook.Runbook attribute), 1535
fields (ironic.objects.runbook.RunbookCRUDNotification attribute), 1538
fields (ironic.objects.runbook.RunbookCRUDPayload attribute), 1539
fields (ironic.objects.trait.Trait attribute), 1541
fields (ironic.objects.trait.TraitList attribute), 1542
fields (ironic.objects.volume_connector.VolumeConnector attribute), 1543
fields (ironic.objects.volume_connector.VolumeConnectorCRUD attribute), 1546
fields (ironic.objects.volume_connector.VolumeConnectorCRUD attribute), 1547
fields (ironic.objects.volume_target.VolumeTarget attribute), 1548
fields (ironic.objects.volume_target.VolumeTargetCRUDNotificat attribute), 1552
fields (ironic.objects.volume_target.VolumeTargetCRUDPayload attribute), 1552
file_has_content() (in module ironic.common.utils), 1013
file_mime_type() (in module ironic.common.utils), 1013
FileImageService (class in ironic.common.image_service), 964
FileSystemNotSupported, 947
filter (ironic.common.trait_based_networking.base.TraitAction attribute), 923
filter_expression()

(*ironic.common.trait_based_networking.grammar.parser.Parser method*), 917
filter_out_attached_portlikes() (*in module ironic.common.trait_based_networking.planner*), 925
filter_out_grouped_ports() (*in module ironic.common.trait_based_networking.planner*), 925
filter_target_raid_config() (*in module ironic.common.raid*), 1002
filter_traits_for_node() (*in module ironic.common.trait_based_networking.planner*), 926
FilterExpression (*class in ironic.common.trait_based_networking.base*), 919
FilterExpressionTransformer (*class in ironic.common.trait_based_networking.grammar.parser*), 916
finalize_rescue() (*ironic.drivers.modules.agent_client.AgentClient method*), 1312
find_devices_by_hints() (*in module ironic.common.utils*), 1013
find_scheme() (*ironic.console.rfb.auths.RFBAuthSchemeList method*), 1094
find_step() (*in module ironic.conductor.steps*), 1066
find_step() (*in module ironic.drivers.modules.agent_base*), 1308
finish_up() (*ironic.console.websocketproxy.TenantSock method*), 1096
firmware (*ironic.drivers.base.BareDriver attribute*), 1391
FIRMWARE_COMPONENTS (*in module ironic.drivers.modules.redfish.utils*), 1281
firmware_interface (*ironic.db.sqlalchemy.models.Node attribute*), 1139
firmware_interface (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
firmware_interface (*ironic.objects.node.Node property*), 1484
FirmwareComponent (*class in ironic.db.sqlalchemy.models*), 1138
FirmwareComponent (*class in ironic.objects.firmware*), 1473
FirmwareComponentAlreadyExists, 947
FirmwareComponentList (*class in ironic.objects.firmware*), 1474
FirmwareComponentNotFound, 947
FirmwareImageLocation (*class in ironic.drivers.modules.ilo.firmware_processor*), 1213
FirmwareInterface (*class in ironic.drivers.base*), 1400
FirmwareProcessor (*class in ironic.drivers.modules.ilo.firmware_processor*), 1213
first_method() (*ironic.drivers.modules.fake.FakeVendorA method*), 1349
FixedOn (*ironic.drivers.modules.snmp.SNMPDriverServerTechSen attribute*), 1386
flash_firmware_sum() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1217
FlatNetwork (*class in ironic.drivers.modules.network.flat*), 1243
FlexibleDict (*class in ironic.objects.fields*), 1470
FlexibleDictField (*class in ironic.objects.fields*), 1470
FLOPPY (*in module ironic.common.boot_devices*), 933
force_persistent_boot() (*in module ironic.drivers.utils*), 1431
force_raw_will_convert() (*in module ironic.common.images*), 975
format_exception() (*in module ironic.api.method*), 902
format_exception() (*ironic.common.auth_basic.BasicAuthMiddleware method*), 931
FORMATTED_ARGS (*ironic.common.inspection_rules.actions.ActionBase attribute*), 908
FORMATTED_ARGS (*ironic.common.inspection_rules.actions.CallAPIHookAc attribute*), 908
FORMATTED_ARGS (*ironic.common.inspection_rules.actions.ExtendAttributeA attribute*), 908
FORMATTED_ARGS (*ironic.common.inspection_rules.actions.ExtendPluginDa attribute*), 908
FORMATTED_ARGS

(*ironic.common.inspection_rules.actions.ExtendPortAttributeAction attribute*), 909
FORMATTED_ARGS (*ironic.common.inspection_rules.actions.LogAction attribute*), 909
FORMATTED_ARGS (*ironic.common.inspection_rules.actions.SetAttributeAction attribute*), 909
FORMATTED_ARGS (*ironic.common.inspection_rules.actions.SetCapabilityAction attribute*), 909
FORMATTED_ARGS (*ironic.common.inspection_rules.actions.SetPluginDataAction attribute*), 910
FORMATTED_ARGS (*ironic.common.inspection_rules.actions.SetPortAttributeAction attribute*), 910
FORMATTED_ARGS (*ironic.common.inspection_rules.operators.EmptyOperator attribute*), 911
FORMATTED_ARGS (*ironic.common.inspection_rules.operators.IsFalseOperator attribute*), 912
FORMATTED_ARGS (*ironic.common.inspection_rules.operators.IsNoneOperator attribute*), 912
FORMATTED_ARGS (*ironic.common.inspection_rules.operators.IsTrueOperator attribute*), 912
FORMATTED_ARGS (*ironic.common.inspection_rules.operators.NetOperator attribute*), 912
FORMATTED_ARGS (*ironic.common.inspection_rules.operators.OneOfOperator attribute*), 913
FORMATTED_ARGS (*ironic.common.inspection_rules.operators.ReOperator attribute*), 913
fourth_method_shared_lock() (*ironic.drivers.modules.fake.FakeVendorB method*), 1349
from_chassis (*ironic.api.controllers.v1.node.NodesController attribute*), 855
FROM_DICT_EXTRA_KEYS (*ironic.common.context.RequestContext attribute*), 939
from_environ() (*ironic.common.context.RequestContext class method*), 939
from_ironic_port() (*ironic.common.trait_based_networking.base.Port class method*), 921
from_ironic_portgroup() (*ironic.common.trait_based_networking.base.Portgroup class method*), 922
from_power (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
from_string() (*ironic.drivers.modules.network.switchport_config.SwitchportConfig class method*), 1256
from_switchport() (*ironic.drivers.modules.network.switchport_config.SwitchportConfig class method*), 1256
from_vif_info() (*ironic.common.trait_based_networking.base.Network class method*), 920
FSM (*class in ironic.common.fsm*), 958
function_expression() (*ironic.common.trait_based_networking.grammar.parser.IronicNetworkingParser method*), 917
FunctionArgument (*class in ironic.api.functions*), 899
FunctionDefinition (*class in ironic.api.functions*), 899
FunctionExpression (*class in ironic.common.trait_based_networking.base*), 910

G

GaugeObject (*class in ironic.common.metrics*), 980
gauge() (*ironic.common.metrics.MetricLogger method*), 981
GAUGE_TYPE (*ironic.common.metrics_collector.DictCollectionMetricsLogger attribute*), 982
GAUGE_TYPE (*ironic.common.metrics_statsd.StatsdMetricLogger attribute*), 983
gen_auth_from_conf_user_pass() (*ironic.common.image_service.HttpImageService static method*), 965
generate_config_metadata() (*in module ironic.conductor.configdrive_utils*), 1026
generate_instance_network_data() (*in module ironic.conductor.configdrive_utils*), 1027
generate_temp_url() (*ironic.common.swift.SwiftAPI method*), 1011
GenericHardware (*class in ironic.drivers.generic*), 1426
GenericSwitchDriver (*class in ironic.drivers.modules.switch.generic_switch*), 1293

GenericSwitchTranslator (*class in ironic.drivers.modules.switch.generic_switch*), 1296
GeOperator (*class in ironic.common.inspection_rules.operators*), 911
get() (*ironic.api.controllers.v1.node.BootDeviceController method*), 845
get() (*ironic.api.controllers.v1.node.NodeConsoleController method*), 847
get() (*ironic.api.controllers.v1.node.NodeInventoryController method*), 848
get() (*ironic.api.controllers.v1.node.NodeStatesController method*), 849
get() (*ironic.api.controllers.v1.node.NodeVmediaController method*), 853
get() (*ironic.api.controllers.v1.volume.VolumeController method*), 887
get() (*ironic.api.functions.FunctionDefinition static method*), 899
get() (*ironic.common.image_service.HttpImageService static method*), 966
get() (*ironic.common.oci_registry.RegistrySessionHelper static method*), 994
get() (*ironic.drivers.modules.snmp.SNMPClient method*), 1380
get() (*ironic.objects.allocation.Allocation class method*), 1444
get() (*ironic.objects.bios.BIOSSetting class method*), 1453
get() (*ironic.objects.chassis.Chassis class method*), 1457
get() (*ironic.objects.firmware.FirmwareComponent class method*), 1473
get() (*ironic.objects.node.Node class method*), 1484
get() (*ironic.objects.node_history.NodeHistory class method*), 1514
get() (*ironic.objects.port.Port class method*), 1521
get() (*ironic.objects.portgroup.Portgroup class method*), 1529
get() (*ironic.objects.volume_connector.VolumeConnector class method*), 1543
get() (*ironic.objects.volume_target.VolumeTarget class method*), 1548
get_action() (*in module ironic.common.inspection_rules.actions*), 910
get_active_hardware_type_dict() (*ironic.db.api.Connection method*), 1156
get_active_hardware_type_dict() (*ironic.db.sqlalchemy.api.Connection method*), 1105
get_active_hardware_type_dict() (*ironic.objects.conductor.Conductor class method*), 1460
get_adapter() (*in module ironic.common.keystone*), 977
get_admin_context() (*in module ironic.common.context*), 940
get_agent_iso() (*in module ironic.drivers.utils*), 1432
get_agent_kernel_ramdisk() (*in module ironic.drivers.utils*), 1432
get_all() (*ironic.api.controllers.v1.allocation.AllocationsContro method*), 833
get_all() (*ironic.api.controllers.v1.allocation.NodeAllocationCo method*), 835
get_all() (*ironic.api.controllers.v1.bios.NodeBiosController method*), 836
get_all() (*ironic.api.controllers.v1.chassis.ChassisController method*), 837
get_all() (*ironic.api.controllers.v1.conductor.ConductorsContro method*), 839
get_all() (*ironic.api.controllers.v1.deploy_template.DeployTemp method*), 840
get_all() (*ironic.api.controllers.v1.driver.DriversController method*), 842
get_all() (*ironic.api.controllers.v1.firmware.NodeFirmwareCont method*), 843
get_all() (*ironic.api.controllers.v1.inspection_rule.InspectionRu method*), 844
get_all() (*ironic.api.controllers.v1.node.IndicatorController method*), 846
get_all() (*ironic.api.controllers.v1.node.NodeChildrenControlle method*), 847
get_all() (*ironic.api.controllers.v1.node.NodeHistoryController method*), 848
get_all() (*ironic.api.controllers.v1.node.NodesController method*), 855
get_all() (*ironic.api.controllers.v1.node.NodeTraitsController method*), 852
get_all() (*ironic.api.controllers.v1.node.NodeVIFController method*), 852
get_all() (*ironic.api.controllers.v1.port.PortsController method*), 860
get_all() (*ironic.api.controllers.v1.portgroup.PortgroupsContro method*), 862
get_all() (*ironic.api.controllers.v1.ramdisk.LookupController method*), 865

get_all() (*ironic.api.controllers.v1.runbook.RunbooksController method*), 866
get_all() (*ironic.api.controllers.v1.shard.ShardController method*), 867
get_all() (*ironic.api.controllers.v1.volume_connector.VolumeConnectorController method*), 888
get_all() (*ironic.api.controllers.v1.volume_target.VolumeTargetController method*), 890
get_all_subscriptions() (*ironic.drivers.modules.redfish.vendor.RedfishVendorPassthru method*), 1285
get_allocation_by_id() (*ironic.db.api.Connection method*), 1157
get_allocation_by_id() (*ironic.db.sqlalchemy.api.Connection method*), 1105
get_allocation_by_name() (*ironic.db.api.Connection method*), 1157
get_allocation_by_name() (*ironic.db.sqlalchemy.api.Connection method*), 1105
get_allocation_by_uuid() (*ironic.db.api.Connection method*), 1157
get_allocation_by_uuid() (*ironic.db.sqlalchemy.api.Connection method*), 1105
get_allocation_list() (*ironic.db.api.Connection method*), 1157
get_allocation_list() (*ironic.db.sqlalchemy.api.Connection method*), 1105
get_allowed_exmods() (*in module ironic.common.rpc*), 1005
get_and_validate_firmware_image_info() (*in module ironic.drivers.modules.ilo.firmware_processor*), 1213
get_app_info() (*ironic.drivers.modules.graphical_console.GraphicalConsole method*), 1350
get_app_info() (*ironic.drivers.modules.redfish.graphical_console.RedfishGraphicalConsole method*), 1266
get_app_name() (*ironic.drivers.modules.fake.FakeGraphicalConsole method*), 1340
get_app_name() (*ironic.drivers.modules.graphical_console.GraphicalConsole method*), 1350
get_app_name() (*ironic.drivers.modules.redfish.graphical_console.RedfishGraphicalConsole method*), 1266
get_arg() (*ironic.api.functions.FunctionDefinition method*), 899
get_artifact_index() (*ironic.common.oci_registry.OciClient method*), 993
get_auth() (*in module ironic.common.keystone*), 977
get_autoneg_cap() (*in module ironic.drivers.modules.inspector.lldp_tlvs*), 1237
get_backend() (*in module ironic.db.migration*), 1183
get_backend() (*in module ironic.db.sqlalchemy.api*), 1133
get_bearer_token() (*ironic.common.oci_registry.RegistrySessionHelper static method*), 994
get_bios_setting() (*ironic.db.api.Connection method*), 1158
get_bios_setting() (*ironic.db.sqlalchemy.api.Connection method*), 1106
get_bios_setting_list() (*ironic.db.api.Connection method*), 1158
get_bios_setting_list() (*ironic.db.sqlalchemy.api.Connection method*), 1106
get_blob_url() (*ironic.common.oci_registry.OciClient method*), 993
get_boot_device() (*ironic.conductor.manager.ConductorManager method*), 1033
get_boot_device() (*ironic.conductor.rpcapi.ConductorAPI method*), 1050
get_boot_device() (*ironic.drivers.base.ManagementInterface method*), 1403
get_boot_device() (*ironic.drivers.modules.fake.FakeManagement method*), 1341
get_boot_device() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1217

get_boot_device() (*ironic.drivers.modules.ipmitool.IPMIManagement method*), 1358
get_boot_device() (*ironic.drivers.modules.noop_mgmt.NoopManagement method*), 1372
get_boot_device() (*ironic.drivers.modules.redfish.management.RedfishManagement method*), 1269
get_boot_mode() (*in module ironic.drivers.modules.boot_mode_utils*), 1320
get_boot_mode() (*ironic.drivers.base.ManagementInterface method*), 1403
get_boot_mode() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1217
get_boot_mode() (*ironic.drivers.modules.redfish.management.RedfishManagement method*), 1269
get_boot_mode_for_deploy() (*in module ironic.drivers.modules.boot_mode_utils*), 1321
get_boot_option() (*in module ironic.drivers.modules.deploy_utils*), 1325
get_built_in_rules() (*in module ironic.common.inspection_rules.engine*), 911
get_by_address() (*ironic.objects.port.Port class method*), 1521
get_by_address() (*ironic.objects.portgroup.Portgroup class method*), 1529
get_by_hostname() (*ironic.objects.conductor.Conductor class method*), 1460
get_by_id() (*ironic.objects.allocation.Allocation class method*), 1445
get_by_id() (*ironic.objects.chassis.Chassis class method*), 1457
get_by_id() (*ironic.objects.deploy_template.DeployTemplate class method*), 1463
get_by_id() (*ironic.objects.node.Node class method*), 1484
get_by_id() (*ironic.objects.node_history.NodeHistory class method*), 1514
get_by_id() (*ironic.objects.port.Port class method*), 1522
get_by_id() (*ironic.objects.portgroup.Portgroup class method*), 1530
get_by_id() (*ironic.objects.runbook.Runbook class method*), 1536
get_by_id() (*ironic.objects.volume_connector.VolumeConnector class method*), 1544
get_by_id() (*ironic.objects.volume_target.VolumeTarget class method*), 1549
get_by_uuid() (*ironic.objects.node.Node class method*), 1484
get_by_name() (*ironic.objects.allocation.Allocation class method*), 1445
get_by_name() (*ironic.objects.deploy_template.DeployTemplate class method*), 1464
get_by_name() (*ironic.objects.node.Node class method*), 1484
get_by_name() (*ironic.objects.port.Port class method*), 1522
get_by_name() (*ironic.objects.portgroup.Portgroup class method*), 1530
get_by_name() (*ironic.objects.runbook.Runbook class method*), 1536
get_by_node_id() (*ironic.objects.bios.BIOSSettingList class method*), 1455
get_by_node_id() (*ironic.objects.firmware.FirmwareComponentList class method*), 1474
get_by_node_id() (*ironic.objects.node_inventory.NodeInventory class method*), 1516
get_by_node_id() (*ironic.objects.trait.TraitList class method*), 1542
get_by_node_uuid() (*ironic.objects.deployment.Deployment class method*), 1468
get_by_port_addresses() (*ironic.objects.node.Node class method*), 1484
get_by_uuid() (*ironic.objects.allocation.Allocation class method*), 1445
get_by_uuid() (*ironic.objects.chassis.Chassis class method*), 1457
get_by_uuid() (*ironic.objects.deploy_template.DeployTemplate class method*), 1464
get_by_uuid() (*ironic.objects.deployment.Deployment class method*), 1468
get_by_uuid() (*ironic.objects.inspection_rule.InspectionRule class method*), 1477
get_by_uuid() (*ironic.objects.node.Node class method*), 1485

get_by_uuid() (*ironic.objects.node_history.NodeHistory class method*), 1514
get_by_uuid() (*ironic.objects.port.Port class method*), 1522
get_by_uuid() (*ironic.objects.portgroup.Portgroup class method*), 1530
get_by_uuid() (*ironic.objects.runbook.Runbook class method*), 1536
get_by_uuid() (*ironic.objects.volume_connector.VolumeConnector class method*), 1544
get_by_uuid() (*ironic.objects.volume_target.VolumeTarget class method*), 1549
get_cached_auth() (*ironic.common.oci_registry.OciClient method*), 993
get_chassis() (*in module ironic.drivers.modules.redfish.utils*), 1281
get_chassis_by_id() (*ironic.db.api.Connection method*), 1158
get_chassis_by_id() (*ironic.db.sqlalchemy.api.Connection method*), 1106
get_chassis_by_uuid() (*ironic.db.api.Connection method*), 1158
get_chassis_by_uuid() (*ironic.db.sqlalchemy.api.Connection method*), 1107
get_chassis_list() (*ironic.db.api.Connection method*), 1158
get_chassis_list() (*ironic.db.sqlalchemy.api.Connection method*), 1107
get_checksum_and_algo() (*in module ironic.common.checksum_utils*), 935
get_checksum_from_url() (*in module ironic.common.checksum_utils*), 935
get_child_node_ids_by_parent_uuid() (*ironic.db.api.Connection method*), 1159
get_child_node_ids_by_parent_uuid() (*ironic.db.sqlalchemy.api.Connection method*), 1107
get_class() (*in module ironic.db.sqlalchemy.models*), 1148
get_clean_steps() (*ironic.drivers.base.BaseInterface method*), 1393
get_clean_steps() (*ironic.drivers.modules.agent.AgentRAID method*), 1298
get_clean_steps() (*ironic.drivers.modules.agent_base.AgentBaseMixin method*), 1304
get_clean_steps() (*ironic.drivers.modules.agent_client.AgentClient method*), 1312
get_clean_steps() (*ironic.drivers.modules.ansible.deploy.AnsibleDeploy method*), 1189
get_cleaning_flow() (*in module ironic.conductor.cleaning*), 1025
get_cleaning_network_uuid() (*ironic.common.neutron.NeutronNetworkInterfaceMixin method*), 986
get_client() (*in module ironic.common.cinder*), 938
get_client() (*in module ironic.common.neutron*), 986
get_client() (*in module ironic.common.rpc*), 1005
get_client() (*in module ironic.drivers.modules.agent_client*), 1317
get_command_error() (*in module ironic.drivers.modules.agent_client*), 1317
get_commands_status() (*ironic.drivers.modules.agent_client.AgentClient method*), 1313
get_component_type() (*in module ironic.drivers.modules.redfish.utils*), 1281
get_conductor() (*ironic.db.api.Connection method*), 1159
get_conductor() (*ironic.db.sqlalchemy.api.Connection method*), 1107
get_conductor_for() (*ironic.conductor.rpcapi.ConductorAPI method*), 1051
get_conductor_list() (*ironic.db.api.Connection method*), 1159
get_conductor_list() (*ironic.db.sqlalchemy.api.Connection method*), 1108
get_conductor_project_id() (*in module ironic.common.glance_service.service_utils*), 907
get_configdrive_image() (*in module*

ironic.conductor.configdrive_utils), 1027
get_configuration() (*in module ironic.common.molds*), 983
get_console() (*in module ironic.common.vnc*), 1020
get_console() (*ironic.drivers.base.ConsoleInterface method*), 1397
get_console() (*ironic.drivers.modules.fake.FakeConsole method*), 1336
get_console() (*ironic.drivers.modules.graphical_console.GraphicalConsole method*), 1350
get_console() (*ironic.drivers.modules.ipmitool.IPMISocatConsole method*), 1362
get_console() (*ironic.drivers.modules.noop.NoConsole method*), 1367
get_console_information() (*ironic.conductor.manager.ConductorManager method*), 1033
get_console_information() (*ironic.conductor.rpcapi.ConductorAPI method*), 1051
get_controller_reserved_names() (*in module ironic.api.controllers.v1.utils*), 880
get_credentials() (*ironic.common.image_publisher.CIFSPublisher method*), 961
get_current_boot_mode() (*in module ironic.drivers.modules.ilo.common*), 1208
get_current_topic() (*ironic.conductor.rpcapi.ConductorAPI method*), 1051
get_current_vif() (*ironic.drivers.base.NetworkInterface method*), 1411
get_current_vif() (*ironic.drivers.modules.network.common.VIFPortIDMixin method*), 1241
get_current_vif() (*ironic.drivers.modules.network.ironic_networking_driver.IronicNetworking method*), 1247
get_current_vif() (*ironic.drivers.modules.network.noop.NoopNetwork method*), 1254
get_deploy_steps() (*ironic.drivers.base.BaseInterface method*), 1394
get_deploy_steps() (*ironic.drivers.modules.agent.AgentRAID method*), 1298
get_deploy_steps() (*ironic.drivers.modules.agent.CustomAgentDeploy method*), 1301
get_deploy_steps() (*ironic.drivers.modules.agent_client.AgentClient method*), 1314
get_deploy_template_by_id() (*ironic.db.api.Connection method*), 1159
get_deploy_template_by_id() (*ironic.db.sqlalchemy.api.Connection method*), 1108
get_deploy_template_by_name() (*ironic.db.api.Connection method*), 1160
get_deploy_template_by_name() (*ironic.db.sqlalchemy.api.Connection method*), 1108
get_deploy_template_by_uuid() (*ironic.db.api.Connection method*), 1160
get_deploy_template_by_uuid() (*ironic.db.sqlalchemy.api.Connection method*), 1108
get_deploy_template_list() (*ironic.db.api.Connection method*), 1160
get_deploy_template_list() (*ironic.db.sqlalchemy.api.Connection method*), 1108
get_deploy_template_list_by_names() (*ironic.db.api.Connection method*), 1160
get_deploy_template_list_by_names() (*ironic.db.sqlalchemy.api.Connection method*), 1109
get_disk_label() (*in module ironic.drivers.modules.deploy_utils*), 1326
get_driver() (*ironic.common.driver_factory.BaseDriverFactory method*), 941
get_driver_classes() (*ironic.common.driver_factory.BaseDriverFactory method*), 941
get_driver_properties() (*ironic.conductor.manager.ConductorManager method*), 1033
get_driver_properties() (*ironic.conductor.rpcapi.ConductorAPI method*), 1051
get_driver_vendor_passthru_methods() (*ironic.conductor.manager.ConductorManager*

method), 1033
get_driver_vendor_passthru_methods() (*ironic.conductor.rpcapi.ConductorAPI method*), 1052
get_enabled_macs() (*in module ironic.drivers.modules.redfish.utils*), 1282
get_endpoint() (*in module ironic.common.keystone*), 977
get_enforcer() (*in module ironic.common.policy*), 995
get_event_service() (*in module ironic.drivers.modules.redfish.utils*), 1282
get_field() (*in module ironic.drivers.utils*), 1432
get_file_path_from_label() (*in module ironic.common.pxe_utils*), 998
get_firmware_component() (*ironic.db.api.Connection method*), 1161
get_firmware_component() (*ironic.db.sqlalchemy.api.Connection method*), 1109
get_firmware_component_list() (*ironic.db.api.Connection method*), 1161
get_firmware_component_list() (*ironic.db.sqlalchemy.api.Connection method*), 1109
get_first_controller() (*in module ironic.drivers.modules.redfish.utils*), 1282
get_free_port_like_object() (*in module ironic.drivers.modules.network.common*), 1241
get_hardware_type() (*in module ironic.common.driver_factory*), 943
get_http_url_path_from_label() (*in module ironic.common.pxe_utils*), 999
get_ilo_object() (*in module ironic.drivers.modules.ilo.common*), 1208
get_image_download_source() (*in module ironic.drivers.modules.deploy_utils*), 1326
get_image_info() (*in module ironic.common.pxe_utils*), 999
get_image_instance_info() (*in module ironic.drivers.modules.deploy_utils*), 1326
get_image_member_list() (*in module ironic.common.glance_service.service_utils*), 907
get_image_properties() (*in module ironic.common.images*), 975
get_image_properties() (*in module ironic.drivers.modules.deploy_utils*), 1326
get_image_service() (*in module ironic.common.image_service*), 970
get_image_service_auth_override() (*in module ironic.common.image_service*), 970
get_indicator_state() (*ironic.conductor.manager.ConductorManager method*), 1033
get_indicator_state() (*ironic.conductor.rpcapi.ConductorAPI method*), 1052
get_indicator_state() (*ironic.drivers.base.ManagementInterface method*), 1404
get_indicator_state() (*ironic.drivers.modules.fake.FakeManagement method*), 1341
get_indicator_state() (*ironic.drivers.modules.redfish.management.RedfishMana method*), 1270
get_inspection_callback() (*in module ironic.drivers.modules.inspect_utils*), 1356
get_inspection_data() (*in module ironic.drivers.modules.inspect_utils*), 1356
get_inspection_network_uuid() (*ironic.common.neutron.NeutronNetworkInterfaceMixin method*), 986
get_inspection_rule_by_id() (*ironic.db.api.Connection method*), 1161
get_inspection_rule_by_id() (*ironic.db.sqlalchemy.api.Connection method*), 1109
get_inspection_rule_by_uuid() (*ironic.db.api.Connection method*), 1161
get_inspection_rule_by_uuid() (*ironic.db.sqlalchemy.api.Connection method*), 1109
get_inspection_rule_list() (*ironic.db.api.Connection method*),

1161
get_inspection_rule_list() (*ironic.db.sqlalchemy.api.Connection method*), 1110
get_instance() (*in module ironic.db.api*), 1183
get_instance_image_info() (*in module ironic.common.pxe_utils*), 999
get_interface() (*in module ironic.common.driver_factory*), 943
get_interface() (*ironic.objects.node.Node method*), 1485
get_interfaces() (*in module ironic.drivers.modules.inspector.hooks.validate_interfaces*), 1233
get_ip_addresses() (*ironic.dhcp.base.BaseDHCP method*), 1184
get_ip_addresses() (*ironic.dhcp.dnsmasq.DnsmasqDHCPApi method*), 1185
get_ip_addresses() (*ironic.dhcp.neutron.NeutronDHCPApi method*), 1186
get_ip_addresses() (*ironic.dhcp.none.NoneDHCPApi method*), 1188
get_ipxe_boot_file() (*in module ironic.drivers.modules.deploy_utils*), 1326
get_ipxe_config_template() (*in module ironic.drivers.modules.deploy_utils*), 1327
get_ironic_api_url() (*in module ironic.drivers.modules.deploy_utils*), 1327
get_kernel_append_params() (*in module ironic.drivers.utils*), 1432
get_kernel_ramdisk_info() (*in module ironic.common.pxe_utils*), 999
get_last_command_status() (*ironic.drivers.modules.agent_client.AgentClient method*), 1314
get_last_error() (*in module ironic.conductor.cleaning*), 1025
get_last_error() (*in module ironic.conductor.servicing*), 1066
get_local_group_information() (*in module ironic.common.neutron*), 987
get_logger() (*ironic.console.websocketproxy.IronicWebSocketProxy static method*), 1096
get_logical_disk_properties() (*in module ironic.common.raid*), 1003
get_logical_disk_properties() (*ironic.drivers.base.RAIDInterface method*), 1417
get_mac_addresses() (*ironic.drivers.base.ManagementInterface method*), 1404
get_mac_addresses() (*ironic.drivers.modules.redfish.management.RedfishMana method*), 1270
get_manager() (*in module ironic.drivers.modules.redfish.utils*), 1282
get_manifest() (*ironic.common.oci_registry.OciClient method*), 993
get_metric_name() (*ironic.common.metrics.MetricLogger method*), 981
get_metrics_data() (*ironic.common.metrics.MetricLogger method*), 981
get_metrics_data() (*ironic.common.metrics_collector.DictCollectionMetricLo method*), 982
get_metrics_logger() (*in module ironic.common.metrics_utils*), 983
get_network_type_for_state() (*in module ironic.common.network*), 984
get_networking_api() (*in module ironic.networking.api*), 1436
get_neutron_port_data() (*in module ironic.common.neutron*), 987
get_next() (*in module ironic.api.controllers.v1.collection*), 838
get_next() (*ironic.drivers.modules.snmp.SNMPClient method*), 1380
get_node_by_id() (*ironic.db.api.Connection method*), 1162
get_node_by_id() (*ironic.db.sqlalchemy.api.Connection method*), 1110
get_node_by_instance() (*ironic.db.api.Connection method*), 1162
get_node_by_instance() (*ironic.db.sqlalchemy.api.Connection method*), 1110
get_node_by_name() (*ironic.db.api.Connection method*), 1162

get_node_by_name() (*ironic.db.sqlalchemy.api.Connection method*), 1110
get_node_by_port_addresses() (*ironic.db.api.Connection method*), 1162
get_node_by_port_addresses() (*ironic.db.sqlalchemy.api.Connection method*), 1110
get_node_by_uuid() (*ironic.db.api.Connection method*), 1162
get_node_by_uuid() (*ironic.db.sqlalchemy.api.Connection method*), 1111
get_node_capability() (*in module ironic.drivers.utils*), 1432
get_node_health() (*ironic.drivers.base.ManagementInterface method*), 1404
get_node_health() (*ironic.drivers.modules.redfish.management.RedfishManagement method*), 1270
get_node_history_by_id() (*ironic.db.api.Connection method*), 1163
get_node_history_by_id() (*ironic.db.sqlalchemy.api.Connection method*), 1111
get_node_history_by_node_id() (*ironic.db.api.Connection method*), 1163
get_node_history_by_node_id() (*ironic.db.sqlalchemy.api.Connection method*), 1111
get_node_history_by_uuid() (*ironic.db.api.Connection method*), 1163
get_node_history_by_uuid() (*ironic.db.sqlalchemy.api.Connection method*), 1111
get_node_history_list() (*ironic.db.api.Connection method*), 1163
get_node_history_list() (*ironic.db.sqlalchemy.api.Connection method*), 1111
get_node_inventory_by_node_id() (*ironic.db.api.Connection method*), 1163
get_node_inventory_by_node_id() (*ironic.db.sqlalchemy.api.Connection method*), 1112
get_node_list() (*ironic.db.api.Connection method*), 1164
get_node_list() (*ironic.db.sqlalchemy.api.Connection method*), 1112
get_node_list_columns() (*ironic.db.sqlalchemy.api.Connection method*), 1112
get_node_mac_addresses() (*in module ironic.drivers.utils*), 1432
get_node_network_data() (*ironic.drivers.base.NetworkInterface method*), 1411
get_node_network_data() (*ironic.drivers.modules.network.common.NeutronVIFPort method*), 1239
get_node_network_data() (*ironic.drivers.modules.network.ironic_networking.IronicI method*), 1247
get_node_next_clean_steps() (*in module ironic.conductor.utils*), 1074
get_node_next_deploy_steps() (*in module ironic.conductor.utils*), 1074
get_node_portmap() (*in module ironic.common.neutron*), 988
get_node_tags_by_node_id() (*ironic.db.api.Connection method*), 1164
get_node_tags_by_node_id() (*ironic.db.sqlalchemy.api.Connection method*), 1113
get_node_traits_by_node_id() (*ironic.db.api.Connection method*), 1164
get_node_traits_by_node_id() (*ironic.db.sqlalchemy.api.Connection method*), 1113
get_node_vendor_passthru_methods() (*ironic.conductor.manager.ConductorManager method*), 1033
get_node_vendor_passthru_methods() (*ironic.conductor.rpcapi.ConductorAPI method*), 1052
get_node_vif_ids() (*in module ironic.common.network*), 984
get_node_with_token() (*ironic.conductor.manager.ConductorManager method*), 1033
get_node_with_token() (*ironic.conductor.rpcapi.ConductorAPI*

method), 1053
get_nodeinfo_list() (*ironic.db.api.Connection method*), 1165
get_nodeinfo_list() (*ironic.db.sqlalchemy.api.Connection method*), 1113
get_nodes_controller_reserved_names() (*in module ironic.api.controllers.v1.node*), 857
get_object() (*ironic.common.swift.SwiftAPI method*), 1011
get_object_versions() (*in module ironic.common.release_mappings*), 1003
get_offline_conductors() (*ironic.db.api.Connection method*), 1166
get_offline_conductors() (*ironic.db.sqlalchemy.api.Connection method*), 1115
get_one() (*ironic.api.controllers.v1.allocation.AllocationsController method*), 834
get_one() (*ironic.api.controllers.v1.bios.NodeBiosController method*), 836
get_one() (*ironic.api.controllers.v1.chassis.ChassisController method*), 837
get_one() (*ironic.api.controllers.v1.conductor.ConductorsController method*), 839
get_one() (*ironic.api.controllers.v1.deploy_template.DeployTemplatesController method*), 840
get_one() (*ironic.api.controllers.v1.driver.DriversController method*), 842
get_one() (*ironic.api.controllers.v1.inspection_rules.InspectionRulesController method*), 844
get_one() (*ironic.api.controllers.v1.node.IndicatorController method*), 846
get_one() (*ironic.api.controllers.v1.node.NodeHistoryController method*), 848
get_one() (*ironic.api.controllers.v1.node.NodesController method*), 856
get_one() (*ironic.api.controllers.v1.port.PortsController method*), 861
get_one() (*ironic.api.controllers.v1.portgroup.PortgroupsController method*), 863
get_one() (*ironic.api.controllers.v1.runbook.RunbooksController method*), 866
get_one() (*ironic.api.controllers.v1.shard.ShardController method*), 867
get_one() (*ironic.api.controllers.v1.volume_connector.VolumeConnectorsController method*), 888
get_one() (*ironic.api.controllers.v1.volume_target.VolumeTargetsController method*), 891
get_online_conductor_count() (*ironic.conductor.base_manager.BaseConductorManager method*), 1023
get_online_conductors() (*ironic.db.api.Connection method*), 1166
get_online_conductors() (*ironic.db.sqlalchemy.api.Connection method*), 1115
get_operator() (*in module ironic.common.inspection_rules.operators*), 913
get_oslo_policy_enforcer() (*in module ironic.common.policy*), 995
get_partition_uuids() (*ironic.drivers.modules.agent_client.AgentClient method*), 1314
get_patch_values() (*in module ironic.api.controllers.v1.utils*), 880
get_pecan_config() (*in module ironic.api.app*), 898
get_physical_disks() (*in module ironic.drivers.modules.redfish.raid*), 1280
get_physical_network() (*ironic.drivers.modules.inspector.hooks.physical_network.PhysicalNetworkHook method*), 1230
get_physnets_by_port_uuid() (*in module ironic.common.neutron*), 988
get_physnets_by_portgroup_id() (*in module ironic.common.network*), 984
get_physnets_for_node() (*in module ironic.common.network*), 985
get_port_by_address() (*ironic.db.api.Connection method*), 1166
get_port_by_address() (*ironic.db.sqlalchemy.api.Connection method*), 1115
get_port_by_id() (*ironic.db.api.Connection method*), 1167
get_port_by_id() (*ironic.db.sqlalchemy.api.Connection method*), 1115
get_port_by_name() (*ironic.db.api.Connection method*), 1167
get_port_by_name() (*ironic.db.sqlalchemy.api.Connection method*), 1115

get_port_by_uuid() (*ironic.db.api.Connection method*), 1167
get_port_by_uuid() (*ironic.db.sqlalchemy.api.Connection method*), 1116
get_port_list() (*ironic.db.api.Connection method*), 1167
get_port_list() (*ironic.db.sqlalchemy.api.Connection method*), 1116
get_portgroup_by_address() (*ironic.db.api.Connection method*), 1167
get_portgroup_by_address() (*ironic.db.sqlalchemy.api.Connection method*), 1116
get_portgroup_by_id() (*in module ironic.common.network*), 985
get_portgroup_by_id() (*ironic.db.api.Connection method*), 1168
get_portgroup_by_id() (*ironic.db.sqlalchemy.api.Connection method*), 1116
get_portgroup_by_name() (*ironic.db.api.Connection method*), 1168
get_portgroup_by_name() (*ironic.db.sqlalchemy.api.Connection method*), 1117
get_portgroup_by_uuid() (*ironic.db.api.Connection method*), 1168
get_portgroup_by_uuid() (*ironic.db.sqlalchemy.api.Connection method*), 1117
get_portgroup_list() (*ironic.db.api.Connection method*), 1168
get_portgroup_list() (*ironic.db.sqlalchemy.api.Connection method*), 1117
get_portgroups_by_node_id() (*ironic.db.api.Connection method*), 1169
get_portgroups_by_node_id() (*ironic.db.sqlalchemy.api.Connection method*), 1117
get_portgroups_by_shards() (*ironic.db.api.Connection method*), 1169
get_portgroups_by_shards() (*ironic.db.sqlalchemy.api.Connection method*), 1118
get_portlike_object() (*ironic.common.trait_based_networking.base.AttachActio method*), 918
get_portlike_object() (*ironic.common.trait_based_networking.base.AttachPort method*), 918
get_portlike_object() (*ironic.common.trait_based_networking.base.AttachPortg method*), 918
get_ports_by_node_id() (*ironic.db.api.Connection method*), 1169
get_ports_by_node_id() (*ironic.db.sqlalchemy.api.Connection method*), 1118
get_ports_by_portgroup_id() (*in module ironic.common.network*), 985
get_ports_by_portgroup_id() (*ironic.db.api.Connection method*), 1170
get_ports_by_portgroup_id() (*ironic.db.sqlalchemy.api.Connection method*), 1118
get_ports_by_shards() (*ironic.db.api.Connection method*), 1170
get_ports_by_shards() (*ironic.db.sqlalchemy.api.Connection method*), 1119
get_power_state() (*ironic.drivers.base.PowerInterface method*), 1414
get_power_state() (*ironic.drivers.modules.agent_power.AgentPower method*), 1318
get_power_state() (*ironic.drivers.modules.fake.FakePower method*), 1344
get_power_state() (*ironic.drivers.modules.ilo.power.IloPower method*), 1222
get_power_state() (*ironic.drivers.modules.ipmitool.IPMIPower method*), 1360
get_power_state() (*ironic.drivers.modules.redfish.power.RedfishPower method*), 1276
get_power_state()

(*ironic.drivers.modules.snmp.SNMPPower method*), 1388
get_properties() (*ironic.drivers.base.BareDriver method*), 1391
get_properties() (*ironic.drivers.base.BaseInterface method*), 1394
get_properties() (*ironic.drivers.base.NetworkInterface method*), 1411
get_properties() (*ironic.drivers.base.RAIDInterface method*), 1417
get_properties() (*ironic.drivers.hardware_type.AbstractHardwareType method*), 1427
get_properties() (*ironic.drivers.modules.agent.AgentRAID method*), 1298
get_properties() (*ironic.drivers.modules.agent.AgentRescue method*), 1299
get_properties() (*ironic.drivers.modules.agent.CustomAgentDeploy method*), 1301
get_properties() (*ironic.drivers.modules.agent_power.AgentPower method*), 1318
get_properties() (*ironic.drivers.modules.ansible.deploy.AnsibleDeploy method*), 1190
get_properties() (*ironic.drivers.modules.autodetect.AutodetectDeploy method*), 1319
get_properties() (*ironic.drivers.modules.fake.FakeBIOS method*), 1334
get_properties() (*ironic.drivers.modules.fake.FakeBoot method*), 1335
get_properties() (*ironic.drivers.modules.fake.FakeConsole method*), 1336
get_properties() (*ironic.drivers.modules.fake.FakeDeploy method*), 1337
get_properties() (*ironic.drivers.modules.fake.FakeFirmware method*), 1339
get_properties() (*ironic.drivers.modules.fake.FakeGraphicalConsole method*), 1340
get_properties() (*ironic.drivers.modules.fake.FakeInspect method*), 1340
get_properties() (*ironic.drivers.modules.fake.FakeManagement method*), 1342
get_properties() (*ironic.drivers.modules.fake.FakePower method*), 1345
get_properties() (*ironic.drivers.modules.fake.FakeRAID method*), 1346
get_properties() (*ironic.drivers.modules.fake.FakeRescue method*), 1347
get_properties() (*ironic.drivers.modules.fake.FakeStorage method*), 1348
get_properties() (*ironic.drivers.modules.fake.FakeVendorA method*), 1349
get_properties() (*ironic.drivers.modules.fake.FakeVendorB method*), 1349
get_properties() (*ironic.drivers.modules.ilo.bios.IloBIOS method*), 1198
get_properties() (*ironic.drivers.modules.ilo.boot.IloUefiHttpsBoot method*), 1200
get_properties() (*ironic.drivers.modules.ilo.boot.IloVirtualMediaBoot method*), 1202
get_properties() (*ironic.drivers.modules.ilo.inspect.IloInspect method*), 1214
get_properties() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1217
get_properties() (*ironic.drivers.modules.ilo.power.IloPower method*), 1222
get_properties() (*ironic.drivers.modules.ilo.raid.Ilo5RAID method*), 1225
get_properties() (*ironic.drivers.modules.inspector.interface.Common method*), 1234
get_properties()

(*ironic.drivers.modules.ipmitool.IPMIConsole method*), 1358
get_properties() (*ironic.drivers.modules.ipmitool.IPMIManagement method*), 1359
get_properties() (*ironic.drivers.modules.ipmitool.IPMIPower method*), 1361
get_properties() (*ironic.drivers.modules.ipmitool.VendorPassthru method*), 1363
get_properties() (*ironic.drivers.modules.noop.FailMixin method*), 1366
get_properties() (*ironic.drivers.modules.noop.NoDeploy method*), 1368
get_properties() (*ironic.drivers.modules.noop_mgmt.NoopManagement method*), 1373
get_properties() (*ironic.drivers.modules.pxe.PXEAnacondaDeploy method*), 1375
get_properties() (*ironic.drivers.modules.pxe_base.PXEBaseMixin method*), 1377
get_properties() (*ironic.drivers.modules.ramdisk.RamdiskDeploy method*), 1378
get_properties() (*ironic.drivers.modules.redfish.bios.RedfishBIOS method*), 1258
get_properties() (*ironic.drivers.modules.redfish.boot.RedfishHttpsBoot method*), 1259
get_properties() (*ironic.drivers.modules.redfish.boot.RedfishVirtualMediaBoot method*), 1261
get_properties() (*ironic.drivers.modules.redfish.firmware.RedfishFirmware method*), 1264
get_properties() (*ironic.drivers.modules.redfish.graphical_console.RedfishGraphicalConsole method*), 1267
get_properties() (*ironic.drivers.modules.redfish.inspect.RedfishInspect method*), 1267
get_properties() (*ironic.drivers.modules.redfish.management.RedfishManagement method*), 1271
get_properties() (*ironic.drivers.modules.redfish.power.RedfishPower method*), 1276
get_properties() (*ironic.drivers.modules.redfish.raid.RedfishRAID method*), 1278
get_properties() (*ironic.drivers.modules.redfish.vendor.RedfishVendorPassthru method*), 1285
get_properties() (*ironic.drivers.modules.snmp.SNMPPower method*), 1388
get_properties() (*ironic.drivers.modules.storage.cinder.CinderStorage method*), 1286
get_properties() (*ironic.drivers.modules.storage.external.ExternalStorage method*), 1287
get_properties() (*ironic.drivers.modules.storage.noop.NoopStorage method*), 1288
get_properties() (*ironic.drivers.utils.MixinVendorInterface method*), 1430
get_provisioning_network_uuid() (*ironic.common.neutron.NeutronNetworkInterfaceMixin method*), 986
get_pxe_boot_file() (*in module ironic.drivers.modules.deploy_utils*), 1327
get_pxe_config_file_path() (*in module ironic.common.pxe_utils*), 1000
get_pxe_config_template() (*in module ironic.drivers.modules.deploy_utils*), 1327
get_pxe_mac() (*in module ironic.drivers.modules.inspector.hooks.validate_interfaces*), 1234
get_raid_logical_disk_properties() (*ironic.conductor.manager.ConductorManager method*), 1033
get_raid_logical_disk_properties() (*ironic.conductor.rpcapi.ConductorAPI method*), 1053
get_ramdisk_logs_file_name() (*in module ironic.drivers.utils*), 1433
get_random_topic() (*ironic.conductor.rpcapi.ConductorAPI method*), 1053
get_request_id() (*in module ironic.api.middleware.request_log*), 895

get_remote_boot_volume() (*in module ironic.drivers.modules.deploy_utils*), 1327
get_request_return_fields() (*in module ironic.api.controllers.v1.utils*), 880
get_rescuing_network_uuid() (*ironic.common.neutron.NeutronNetworkInterfaceMixin method*), 986
get_return_state() (*in module ironic.common.async_steps*), 930
get_ring() (*ironic.common.hash_ring.HashRingManager method*), 960
get_root_device_for_deploy() (*in module ironic.drivers.modules.deploy_utils*), 1327
get_root_vendor() (*in module ironic.drivers.modules.redfish.utils*), 1282
get_route_source() (*in module ironic.common.utils*), 1014
get_rpc_allocation() (*in module ironic.api.controllers.v1.utils*), 881
get_rpc_allocation_with_suffix() (*in module ironic.api.controllers.v1.utils*), 881
get_rpc_deploy_template() (*in module ironic.api.controllers.v1.utils*), 881
get_rpc_deploy_template_with_suffix() (*in module ironic.api.controllers.v1.utils*), 881
get_rpc_node() (*in module ironic.api.controllers.v1.utils*), 882
get_rpc_node_with_suffix() (*in module ironic.api.controllers.v1.utils*), 882
get_rpc_portgroup() (*in module ironic.api.controllers.v1.utils*), 882
get_rpc_portgroup_with_suffix() (*in module ironic.api.controllers.v1.utils*), 882
get_rpc_runbook() (*in module ironic.api.controllers.v1.utils*), 883
get_runbook() (*ironic.conductor.manager.ConductorManager method*), 1033
get_runbook_by_id() (*ironic.db.api.Connection method*), 1170
get_runbook_by_id() (*ironic.db.sqlalchemy.api.Connection method*), 1119
get_runbook_by_name() (*ironic.db.api.Connection method*), 1170
get_runbook_by_name() (*ironic.db.sqlalchemy.api.Connection method*), 1119
get_runbook_by_uuid() (*ironic.db.api.Connection method*), 1170
get_runbook_by_uuid() (*ironic.db.sqlalchemy.api.Connection method*), 1119
get_runbook_list() (*ironic.db.api.Connection method*), 1171
get_runbook_list() (*ironic.db.sqlalchemy.api.Connection method*), 1119
get_runbook_list_by_names() (*ironic.db.api.Connection method*), 1171
get_runbook_list_by_names() (*ironic.db.sqlalchemy.api.Connection method*), 1119
get_secure_boot_mode() (*in module ironic.drivers.modules.ilo.common*), 1209
get_secure_boot_state() (*ironic.drivers.base.ManagementInterface method*), 1405
get_secure_boot_state() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1218
get_secure_boot_state() (*ironic.drivers.modules.redfish.management.RedfishMana method*), 1271
get_sensors_data() (*ironic.drivers.base.ManagementInterface method*), 1405
get_sensors_data() (*ironic.drivers.modules.fake.FakeManagement method*), 1342
get_sensors_data() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1218
get_sensors_data() (*ironic.drivers.modules.ipmitool.IPMIManagement method*), 1359
get_sensors_data() (*ironic.drivers.modules.noop_mgmt.NoopManagement method*), 1373
get_sensors_data() (*ironic.drivers.modules.redfish.management.RedfishMana method*), 1271
get_sensors_notifier() (*in module*

ironic.common.rpc), 1005
get_server() (*in module ironic.common.rpc*), 1005
get_server_post_state() (*in module ironic.drivers.modules.ilo.common*), 1209
get_service_auth() (*in module ironic.common.keystone*), 977
get_service_steps() (*ironic.drivers.base.BaseInterface method*), 1394
get_service_steps() (*ironic.drivers.modules.agent_base.AgentBaseMixin method*), 1304
get_service_steps() (*ironic.drivers.modules.agent_client.AgentClient method*), 1315
get_servicing_network_uuid() (*ironic.common.neutron.NeutronNetworkInterfaceMixin method*), 986
get_session() (*in module ironic.common.keystone*), 978
get_shard_list() (*ironic.db.api.Connection method*), 1171
get_shard_list() (*ironic.db.sqlalchemy.api.Connection method*), 1120
get_shard_list() (*ironic.objects.conductor.Conductor class method*), 1461
get_single_nic_with_vif_port_id() (*in module ironic.drivers.modules.deploy_utils*), 1328
get_socat_console_url() (*in module ironic.drivers.modules.console_utils*), 1322
get_source_format() (*in module ironic.common.images*), 975
get_steps() (*in module ironic.drivers.modules.agent_base*), 1308
get_subscription() (*ironic.drivers.modules.redfish.vendor.RedfishVendorPassthru method*), 1285
get_supported_boot_devices() (*ironic.conductor.manager.ConductorManager method*), 1033
get_supported_boot_devices() (*ironic.conductor.rpcapi.ConductorAPI method*), 1054
get_supported_boot_devices() (*ironic.drivers.base.ManagementInterface method*), 1406
get_supported_boot_devices() (*ironic.drivers.modules.fake.FakeManagement method*), 1343
get_supported_boot_devices() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1218
get_supported_boot_devices() (*ironic.drivers.modules.ipmitool.IPMIManagement method*), 1359
get_supported_boot_devices() (*ironic.drivers.modules.noop_mgmt.NoopManagement method*), 1374
get_supported_boot_devices() (*ironic.drivers.modules.redfish.management.RedfishMana method*), 1271
get_supported_boot_modes() (*ironic.drivers.base.ManagementInterface method*), 1406
get_supported_boot_modes() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1218
get_supported_boot_modes() (*ironic.drivers.modules.redfish.management.RedfishMana method*), 1271
get_supported_indicators() (*ironic.conductor.manager.ConductorManager method*), 1033
get_supported_indicators() (*ironic.conductor.rpcapi.ConductorAPI method*), 1054
get_supported_indicators() (*ironic.drivers.base.ManagementInterface method*), 1407
get_supported_indicators() (*ironic.drivers.modules.fake.FakeManagement method*), 1343
get_supported_indicators() (*ironic.drivers.modules.redfish.management.RedfishMana method*), 1271
get_supported_power_states() (*ironic.drivers.base.PowerInterface method*), 1415
get_supported_power_states() (*ironic.drivers.modules.agent_power.AgentPower method*), 1318
get_supported_power_states() (*ironic.drivers.modules.fake.FakePower method*), 1345

get_supported_power_states() (*ironic.drivers.modules.ilo.power.IloPower method*), 1222
get_supported_power_states() (*ironic.drivers.modules.ipmitool.IPMIPower method*), 1361
get_supported_power_states() (*ironic.drivers.modules.redfish.power.RedfishPower method*), 1276
get_swift_session() (*in module ironic.common.swift*), 1011
get_swift_temp_url() (*in module ironic.drivers.modules.redfish.firmware_utils*), 1265
get_swift_url() (*in module ironic.drivers.modules.ilo.firmware_processor*), 1214
get_switch_driver() (*in module ironic.networking.switch_drivers.driver_factory*), 1435
get_switch_driver_factory() (*in module ironic.networking.switch_drivers.driver_factory*), 1435
get_switch_ids() (*ironic.drivers.modules.switch.base.NoOpSwitchDriver method*), 1290
get_switch_ids() (*ironic.drivers.modules.switch.base.SwitchDriverBase method*), 1291
get_switch_ids() (*ironic.drivers.modules.switch.generic_switch.GenericSwitchDriver method*), 1294
get_switch_info() (*ironic.drivers.modules.switch.base.NoOpSwitchDriver method*), 1290
get_switch_info() (*ironic.drivers.modules.switch.base.SwitchDriverBase method*), 1291
get_switch_info() (*ironic.drivers.modules.switch.generic_switch.GenericSwitchDriver method*), 1294
get_switch_vlan_config() (*in module ironic.networking.switch_config*), 1442
get_switches() (*ironic.networking.manager.NetworkingManager method*), 1438
get_switches() (*ironic.networking.rpcapi.NetworkingAPI method*), 1440
get_system() (*in module ironic.drivers.modules.redfish.utils*), 1283
get_system_collection() (*in module ironic.drivers.modules.redfish.utils*), 1283
get_target_version() (*ironic.objects.base.IronicObject class method*), 1450
get_task_monitor() (*in module ironic.drivers.modules.redfish.utils*), 1283
get_temp_url() (*ironic.common.swift.SwiftAPI method*), 1011
get_temp_url_for_glance_image() (*in module ironic.common.images*), 975
get_temp_url_key() (*ironic.common.swift.SwiftAPI method*), 1011
get_token_from_config() (*ironic.common.oci_registry.RegistrySessionHelper method*), 994
get_token_project_from_request() (*in module ironic.conductor.utils*), 1074
get_topic() (*ironic.networking.rpcapi.NetworkingAPI method*), 1440
get_topic_for() (*ironic.conductor.rpcapi.ConductorAPI method*), 1055
get_topic_for_driver() (*ironic.conductor.rpcapi.ConductorAPI method*), 1055
get_trait_names() (*ironic.objects.trait.TraitList method*), 1542
get_translator() (*ironic.drivers.modules.switch.base.NoOpSwitchDriver class method*), 1290
get_translator() (*ironic.drivers.modules.switch.base.SwitchDriverBase class method*), 1291
get_translator() (*ironic.drivers.modules.switch.generic_switch.GenericSwitchDriver class method*), 1294
get_transport_url() (*in module ironic.common.rpc*), 1005
get_update_service() (*in module ironic.drivers.modules.redfish.utils*), 1283
get_updated_capabilities() (*in module ironic.common.utils*), 1014
get_valid_mac_addresses() (*in module ironic.api.controllers.v1.ramdisk*), 865

get_validation_signature() (*ironic.common.inspection_rules.base.Base method*), 910
get_vendor_passthru_metadata() (*in module ironic.conductor.manager*), 1035
get_verify_ca() (*in module ironic.drivers.utils*), 1433
get_verify_steps() (*ironic.drivers.base.BaseInterface method*), 1394
get_versioned_notifier() (*in module ironic.common.rpc*), 1005
get_virtual_media() (*ironic.conductor.manager.ConductorManager method*), 1033
get_virtual_media() (*ironic.conductor.rpcapi.ConductorAPI method*), 1055
get_virtual_media() (*ironic.drivers.base.ManagementInterface method*), 1407
get_virtual_media() (*ironic.drivers.modules.redfish.management.RedfishManagement method*), 1272
get_vmedia() (*in module ironic.drivers.modules.redfish.boot*), 1263
get_volume_connector_by_id() (*ironic.db.api.Connection method*), 1171
get_volume_connector_by_id() (*ironic.db.sqlalchemy.api.Connection method*), 1120
get_volume_connector_by_uuid() (*ironic.db.api.Connection method*), 1171
get_volume_connector_by_uuid() (*ironic.db.sqlalchemy.api.Connection method*), 1120
get_volume_connector_list() (*ironic.db.api.Connection method*), 1172
get_volume_connector_list() (*ironic.db.sqlalchemy.api.Connection method*), 1120
get_volume_connectors_by_node_id() (*ironic.db.api.Connection method*), 1172
get_volume_connectors_by_node_id() (*ironic.db.sqlalchemy.api.Connection method*), 1121
get_volume_pxe_options() (*in module ironic.common.pxe_utils*), 1000
get_volume_target_by_id() (*ironic.db.api.Connection method*), 1172
get_volume_target_by_id() (*ironic.db.sqlalchemy.api.Connection method*), 1121
get_volume_target_by_uuid() (*ironic.db.api.Connection method*), 1173
get_volume_target_by_uuid() (*ironic.db.sqlalchemy.api.Connection method*), 1121
get_volume_target_list() (*ironic.db.api.Connection method*), 1173
get_volume_target_list() (*ironic.db.sqlalchemy.api.Connection method*), 1121
get_volume_targets_by_node_id() (*ironic.db.api.Connection method*), 1173
get_volume_targets_by_node_id() (*ironic.db.sqlalchemy.api.Connection method*), 1122
get_volume_targets_by_volume_id() (*ironic.db.api.Connection method*), 1174
get_volume_targets_by_volume_id() (*ironic.db.sqlalchemy.api.Connection method*), 1122
getargspec() (*in module ironic.api.functions*), 900
GetNodeAndTopicMixin (*class in ironic.api.controllers.v1.node*), 846
GlanceConnectionFailed, 947
GlanceImageService (*class in ironic.common.glance_service.image_service*), 905
GraphicalConsole (*class in ironic.drivers.modules.graphical_console*), 1350
GROUP_AND_ATTACH_PORTS (*ironic.common.trait_based_networking.base.Actions attribute*), 918
GroupAndAttachPorts (*class in ironic.common.trait_based_networking.base*), 920
GT (*ironic.common.trait_based_networking.base.Comparator attribute*), 919
gt() (*ironic.common.trait_based_networking.grammar.parser.Filte method*), 917

GT_OR_EQ (*ironic.common.trait_based_networking.base.Comparator attribute*), 919
gt_or_eq() (*ironic.common.trait_based_networking.grammar.rules_v1 method*), 917
GtOperator (*class in ironic.common.inspection_rules.operators*), 911

H

handle_drain() (*in module ironic.command.utils*), 905
handle_error_notification() (*in module ironic.api.controllers.v1.notification_utils*), 859
handle_no_deregister() (*in module ironic.command.utils*), 905
handle_org_specific_tlv() (*ironic.drivers.modules.inspector.lldp_parsers.LLDPBasicMgmtParser method*), 1236
handle_signal() (*in module ironic.command.utils*), 905
handle_sync_power_state_max_retries_exceeded() (*in module ironic.conductor.manager*), 1035
hardware_type (*ironic.db.sqlalchemy.models.ConductorHardwareInterfaces attribute*), 1137
hardware_types() (*in module ironic.common.driver_factory*), 944
HardwareInspectionFailure, 948
HardwareTypesFactory (*class in ironic.common.driver_factory*), 941
has_next() (*in module ironic.api.controllers.v1.collection*), 838
has_reserved() (*ironic.conductor.base_manager.BaseConductorManager method*), 1023
hash_password() (*in module ironic.conductor.utils*), 1074
HashRingManager (*class in ironic.common.hash_ring*), 960
headers (*ironic.common.exception.IronicException attribute*), 952
headers (*ironic.common.exception.Unauthorized attribute*), 957
health (*ironic.db.sqlalchemy.models.Node attribute*), 1139
health (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
health (*ironic.objects.node.Node property*), 1485
HealthState (*class in ironic.common.health_states*), 960
heartbeat() (*ironic.conductor.manager.ConductorManager method*), 1033
heartbeat() (*ironic.conductor.rpcapi.ConductorAPI method*), 1055
heartbeat() (*ironic.drivers.base.DeployInterface method*), 1398
heartbeat() (*ironic.drivers.modules.agent_base.HeartbeatMixin method*), 1307
heartbeat_allowed() (*ironic.drivers.modules.agent_base.HeartbeatMixin method*), 1307
HeartbeatController (*class in ironic.api.controllers.v1.ramdisk*), 864
HeartbeatMixin (*class in ironic.drivers.modules.agent_base*), 1307
hide_fields_in_newer_versions() (*in module ironic.api.controllers.v1.allocation*), 836
hide_fields_in_newer_versions() (*in module ironic.api.controllers.v1.driver*), 843
hide_fields_in_newer_versions() (*in module ironic.api.controllers.v1.node*), 857
hide_fields_in_newer_versions() (*in module ironic.api.controllers.v1.port*), 862
hide_fields_in_newer_versions() (*in module ironic.api.controllers.v1.portgroup*), 863
host (*ironic.objects.notification.NotificationPublisher property*), 1519
host_port() (*in module ironic.common.args*), 928
hostname (*ironic.db.sqlalchemy.models.Conductor attribute*), 1136
hostname (*ironic.objects.conductor.Conductor property*), 1461
http_boot_enabled (*ironic.drivers.modules.ipxe.iPXEHttpBoot attribute*), 1366
http_boot_enabled (*ironic.drivers.modules.pxe.HttpBoot attribute*), 1375
http_boot_enabled (*ironic.drivers.modules.pxe_base.PXEBaseMixin attribute*), 1377
HttpBoot (*class in ironic.drivers.modules.pxe*), 1375
HTTPForbidden, 948
HttpImageService (*class in*

ironic.common.image_service), 965
HTTPNotFound (*in module ironic.common.exception*), 948
HYBRID (*in module ironic.common.automated_clean_methods*), 933

I

id (*ironic.common.trait_based_networking.base.NetworkInterface attribute*), 920
id (*ironic.common.trait_based_networking.base.PrimordialPort attribute*), 922
id (*ironic.db.sqlalchemy.models.Allocation attribute*), 1135
id (*ironic.db.sqlalchemy.models.Chassis attribute*), 1136
id (*ironic.db.sqlalchemy.models.Conductor attribute*), 1136
id (*ironic.db.sqlalchemy.models.ConductorHardwareInterfaces attribute*), 1137
id (*ironic.db.sqlalchemy.models.DeployTemplate attribute*), 1137
id (*ironic.db.sqlalchemy.models.DeployTemplateStep attribute*), 1137
id (*ironic.db.sqlalchemy.models.FirmwareComponent attribute*), 1138
id (*ironic.db.sqlalchemy.models.InspectionRule attribute*), 1138
id (*ironic.db.sqlalchemy.models.Node attribute*), 1139
id (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
id (*ironic.db.sqlalchemy.models.NodeHistory attribute*), 1143
id (*ironic.db.sqlalchemy.models.NodeInventory attribute*), 1144
id (*ironic.db.sqlalchemy.models.Port attribute*), 1145
id (*ironic.db.sqlalchemy.models.Portgroup attribute*), 1146
id (*ironic.db.sqlalchemy.models.Runbook attribute*), 1146
id (*ironic.db.sqlalchemy.models.RunbookStep attribute*), 1147
id (*ironic.db.sqlalchemy.models.VolumeConnector attribute*), 1147
id (*ironic.db.sqlalchemy.models.VolumeTarget attribute*), 1147
id (*ironic.objects.allocation.Allocation property*), 1445
id (*ironic.objects.chassis.Chassis property*), 1458
id (*ironic.objects.conductor.Conductor property*), 1461
id (*ironic.objects.deploy_template.DeployTemplate property*), 1464
id (*ironic.objects.firmware.FirmwareComponent property*), 1474
id (*ironic.objects.inspection_rule.InspectionRule property*), 1478
id (*ironic.objects.node.Node property*), 1485
id (*ironic.objects.node_history.NodeHistory property*), 1515
id (*ironic.objects.node_inventory.NodeInventory property*), 1517
id (*ironic.objects.port.Port property*), 1523
id (*ironic.objects.portgroup.Portgroup property*), 1531
id (*ironic.objects.runbook.Runbook property*), 1537
id (*ironic.objects.volume_connector.VolumeConnector property*), 1544
id (*ironic.objects.volume_target.VolumeTarget property*), 1549
identify_specific_image() (*ironic.common.image_service.OciImageService method*), 968
idleOff (*ironic.drivers.modules.snmp.SNMPDriverServerTechSent attribute*), 1386
idleOn (*ironic.drivers.modules.snmp.SNMPDriverServerTechSent attribute*), 1386
IDRACHardware (*class in ironic.drivers.drac*), 1424
ignore_extra_args (*ironic.api.functions.FunctionDefinition attribute*), 899
Ilo5Hardware (*class in ironic.drivers.ilo*), 1428
Ilo5Management (*class in ironic.drivers.modules.ilo.management*), 1215
Ilo5RAID (*class in ironic.drivers.modules.ilo.raid*), 1224
IloBIOS (*class in ironic.drivers.modules.ilo.bios*), 1197
IloHardware (*class in ironic.drivers.ilo*), 1428
IloInspect (*class in ironic.drivers.modules.ilo.inspect*), 1214
IloiPXEBoot (*class in ironic.drivers.modules.ilo.boot*), 1204
IloManagement (*class in ironic.drivers.modules.ilo.management*), 1216

IloOperationError, 948
IloOperationNotSupported, 948
IloPower (*class in ironic.drivers.modules.ilo.power*), 1222
IloPXEBoot (*class in ironic.drivers.modules.ilo.boot*), 1198
IloUefiHttpsBoot (*class in ironic.drivers.modules.ilo.boot*), 1199
IloVirtualMediaBoot (*class in ironic.drivers.modules.ilo.boot*), 1201
image_checksum (*ironic.objects.deployment.Deployment property*), 1469
image_format_matches() (*in module ironic.common.images*), 975
image_format_permitted() (*in module ironic.common.images*), 975
image_ref (*ironic.objects.deployment.Deployment property*), 1469
image_show() (*in module ironic.common.images*), 975
image_to_raw() (*in module ironic.common.images*), 975
ImageCache (*class in ironic.drivers.modules.image_cache*), 1351
ImageChecksumAlgorithmFailure, 948
ImageChecksumError, 948
ImageChecksumFileReadFailure, 948
ImageChecksumURLNotSupported, 948
ImageConvertFailed, 948
ImageCreationFailed, 948
ImageDownloadFailed, 948
ImageHandler (*class in ironic.drivers.modules.image_utils*), 1352
ImageHostRateLimitFailure, 948
ImageMatchFailure, 949
ImageNotAuthorized, 949
ImageNotFound, 949
ImageRefValidationFailed, 949
ImageServiceAuthenticationRequired, 949
ImageUnacceptable, 949
ImageUploadFailed, 949
import_configuration() (*ironic.drivers.modules.drac.management.DracRedfishManagement method*), 1194
IMPORT_CONFIGURATION_ARGSINFO (*ironic.drivers.modules.drac.management.DracRedfishManagement attribute*), 1193
import_export_configuration() (*ironic.drivers.modules.drac.management.DracRedfishManagement method*), 1194
IMPORT_EXPORT_CONFIGURATION_ARGSINFO (*ironic.drivers.modules.drac.management.DracRedfishManagement attribute*), 1193
in_core_deploy_step() (*ironic.drivers.modules.ansible.deploy.AnsibleDeploy method*), 1190
IncompatibleInterface, 949
IncompleteLookup, 949
IncorrectConfiguration, 949
index() (*ironic.api.controllers.root.RootController method*), 894
index() (*ironic.api.controllers.v1.Controller method*), 893
indicator_convert_with_links() (*in module ironic.api.controllers.v1.node*), 857
indicator_list_from_dict() (*in module ironic.api.controllers.v1.node*), 857
IndicatorAtComponent (*class in ironic.api.controllers.v1.node*), 846
IndicatorController (*class in ironic.api.controllers.v1.node*), 846
indicators (*ironic.api.controllers.v1.node.NodeManagementCon attribute*), 848
INEQUALITY (*ironic.common.trait_based_networking.base.Compa attribute*), 919
inequality() (*ironic.common.trait_based_networking.grammar.p method*), 917
INFO (*ironic.objects.fields.NotificationLevel attribute*), 1471
init() (*in module ironic.common.rpc*), 1005
init_enforcer() (*in module ironic.common.policy*), 995
init_host() (*ironic.conductor.base_manager.BaseConductorMa method*), 1023
init_host() (*ironic.networking.manager.NetworkingManager method*), 1438
init_host() (*ironic.pxe_filter.service.PXEFilterManager method*), 1554
initial_node_provision_state() (*in module ironic.api.controllers.v1.utils*), 883
initial_version (*ironic.db.sqlalchemy.models.FirmwareComponent attribute*), 1138
initial_version (*ironic.objects.firmware.FirmwareComponent property*), 1474
initialize() (*ironic.common.fsm.FSM method*), 959
initialize_drivers()

(*ironic.common.driver_factory.BaseDriverFactory class method*), 941
initialize_wsgi_app() (*in module ironic.api.wsgi*), 902
inject_nmi (*ironic.api.controllers.v1.node.NodeManagementController attribute*), 848
inject_nmi() (*ironic.conductor.manager.ConductorManager method*), 1033
inject_nmi() (*ironic.conductor.rpcapi.ConductorAPI method*), 1056
inject_nmi() (*ironic.drivers.base.ManagementInterface method*), 1408
inject_nmi() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1219
inject_nmi() (*ironic.drivers.modules.ipmitool.IPMIManagement method*), 1360
inject_nmi() (*ironic.drivers.modules.redfish.management.RedfishManagement method*), 1272
InjectNmiController (*class in ironic.api.controllers.v1.node*), 847
InputFileError, 949
insert() (*ironic.common.inspection_rules.utils.ShallowMaskDict method*), 914
insert_vmedia() (*in module ironic.drivers.modules.redfish.boot*), 1263
inspect (*ironic.drivers.base.BareDriver attribute*), 1391
inspect_hardware() (*in module ironic.conductor.inspection*), 1030
inspect_hardware() (*ironic.conductor.manager.ConductorManager method*), 1033
inspect_hardware() (*ironic.conductor.rpcapi.ConductorAPI method*), 1056
inspect_hardware() (*ironic.drivers.base.InspectInterface method*), 1402
inspect_hardware() (*ironic.drivers.modules.drac.inspect.DracRedfishInspect method*), 1192
inspect_hardware() (*ironic.drivers.modules.fake.FakeInspect method*), 1340
inspect_hardware() (*ironic.drivers.modules.ilo.inspect.IloInspect method*), 1214
inspect_hardware() (*ironic.drivers.modules.inspector.interface.CommonInspect method*), 1235
inspect_hardware() (*ironic.drivers.modules.noop.NoInspect method*), 1370
inspect_hardware() (*ironic.drivers.modules.redfish.inspect.RedfishInspect method*), 1267
inspect_interface (*ironic.db.sqlalchemy.models.Node attribute*), 1139
inspect_interface (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
inspect_interface (*ironic.objects.node.Node property*), 1485
inspect_interface (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1488
inspect_interface (*ironic.objects.node.NodeCRUDPayload property*), 1493
inspect_interface (*ironic.objects.node.NodePayload property*), 1503
inspect_interface (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
inspect_interface (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
INSPECTFAIL (*in module ironic.common.states*), 1007
INSPECTING (*in module ironic.common.states*), 1007
inspecting_error_handler() (*in module ironic.conductor.utils*), 1074
InspectInterface (*class in ironic.drivers.base*), 1401
INSPECTION (*ironic.drivers.modules.network.common.NetType attribute*), 1239
inspection_error_handler() (*in module ironic.drivers.modules.inspector.interface*), 1235
inspection_finished_at (*ironic.db.sqlalchemy.models.Node attribute*), 1139
inspection_finished_at (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
inspection_finished_at (*ironic.objects.node.Node property*), 1485

inspection_finished_at (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
inspection_finished_at (*ironic.objects.node.NodeCRUDPayload property*), 1493
inspection_finished_at (*ironic.objects.node.NodePayload property*), 1503
inspection_finished_at (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
inspection_finished_at (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
inspection_started_at (*ironic.db.sqlalchemy.models.Node attribute*), 1140
inspection_started_at (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
inspection_started_at (*ironic.objects.node.Node property*), 1485
inspection_started_at (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
inspection_started_at (*ironic.objects.node.NodeCRUDPayload property*), 1493
inspection_started_at (*ironic.objects.node.NodePayload property*), 1503
inspection_started_at (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
inspection_started_at (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
InspectionHook (*class in ironic.drivers.modules.inspector.hooks.base*), 1226
InspectionPhase (*class in ironic.common.inspection_rules.validation*), 915
InspectionRule (*class in ironic.db.sqlalchemy.models*), 1138
InspectionRule (*class in ironic.objects.inspection_rule*), 1476
InspectionRuleAlreadyExists, 949
InspectionRuleController (*class in ironic.api.controllers.v1.inspection_rule*), 844
InspectionRuleCRUDNotification (*class in ironic.objects.inspection_rule*), 1479
InspectionRuleCRUDPayload (*class in ironic.objects.inspection_rule*), 1480
InspectionRuleExecutionFailure, 949
InspectionRuleNotFound, 949
InspectionRuleValidationFailure, 949
INSPECTWAIT (*in module ironic.common.states*), 1007
install_bootloader() (*ironic.drivers.modules.agent_client.AgentClient method*), 1315
INSTANCE (*in module ironic.common.lessee_sources*), 978
instance_info (*ironic.db.sqlalchemy.models.Node attribute*), 1140
instance_info (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
instance_info (*ironic.objects.node.Node property*), 1485
instance_info (*ironic.objects.node.NodeCRUDPayload property*), 1493
instance_info (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
instance_info_mapping (*ironic.objects.deployment.Deployment attribute*), 1469
instance_info_mapping_rev (*ironic.objects.deployment.Deployment attribute*), 1469
instance_name (*ironic.db.sqlalchemy.models.Node attribute*), 1140
instance_name (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
instance_name (*ironic.objects.node.Node property*), 1485
instance_uuid (*ironic.db.sqlalchemy.models.Node attribute*), 1140
instance_uuid (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
instance_uuid (*ironic.objects.node.Node property*), 1485
instance_uuid (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
instance_uuid (*ironic.objects.node.NodeCRUDPayload property*), 1493
instance_uuid (*ironic.objects.node.NodePayload property*), 1503
instance_uuid (*ironic.objects.node.NodeSetPowerStatePayload*

property), 1507
instance_uuid (ironic.objects.node.NodeSetProvisionStatePayload property), 1512
InstanceAssociated, 950
InstanceDeployFailure, 950
InstanceImageCache (class in ironic.drivers.modules.deploy_utils), 1323
InstanceNotFound, 950
InstanceRescueFailure, 950
InstanceUnrescueFailure, 950
InsufficientDiskSpace, 950
InsufficientMemory, 950
inSync (ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute), 1384
integer() (in module ironic.common.args), 928
IntegerField (class in ironic.objects.fields), 1470
IntelIPMIHardware (class in ironic.drivers.intel_ipmi), 1429
IntelIPMIManagement (class in ironic.drivers.modules.intel_ipmi.management), 1238
interface (ironic.db.sqlalchemy.models.DeployTemplateStep attribute), 1137
interface (ironic.db.sqlalchemy.models.RunbookStep attribute), 1147
interface_name (ironic.db.sqlalchemy.models.ConductorHardwareInterfaces attribute), 1137
interface_type (ironic.db.sqlalchemy.models.ConductorHardwareInterfaces attribute), 1137
interface_type (ironic.drivers.base.BaseInterface attribute), 1395
interface_type (ironic.drivers.base.BIOSInterface attribute), 1391
interface_type (ironic.drivers.base.BootInterface attribute), 1396
interface_type (ironic.drivers.base.ConsoleInterface attribute), 1397
interface_type (ironic.drivers.base.DeployInterface attribute), 1398
interface_type (ironic.drivers.base.FirmwareInterface attribute), 1401
interface_type (ironic.drivers.base.InspectInterface attribute), 1402
interface_type (ironic.drivers.base.ManagementInterface attribute), 1408
interface_type (ironic.drivers.base.NetworkInterface attribute), 1411
interface_type (ironic.drivers.base.PowerInterface attribute), 1415
interface_type (ironic.drivers.base.RAIDInterface attribute), 1417
interface_type (ironic.drivers.base.RescueInterface attribute), 1418
interface_type (ironic.drivers.base.StorageInterface attribute), 1419
interface_type (ironic.drivers.base.VendorInterface attribute), 1420
InterfaceFactory (class in ironic.common.driver_factory), 941
InterfaceNotFoundInEntrypoint, 950
interfaces() (in module ironic.common.driver_factory), 944
internal_info (ironic.db.sqlalchemy.models.Port attribute), 1145
internal_info (ironic.db.sqlalchemy.models.Portgroup attribute), 1146
internal_info (ironic.objects.port.Port property), 1523
internal_info (ironic.objects.portgroup.Portgroup property), 1531
interpolate_variables() (ironic.common.inspection_rules.base.Base static method), 910
Invalid, 950
INVALID (ironic.console.rfb.auth.AuthType attribute), 1092
invalid_sort_key_list (ironic.api.controllers.v1.allocation.AllocationsController attribute), 835
invalid_sort_key_list (ironic.api.controllers.v1.allocation.NodeAllocationContr attribute), 836
invalid_sort_key_list (ironic.api.controllers.v1.chassis.ChassisController

attribute), 837
invalid_sort_key_list (*ironic.api.controllers.v1.conductor.ConductorsController attribute*), 839
invalid_sort_key_list (*ironic.api.controllers.v1.deploy_template.DeployTemplatesC attribute*), 840
invalid_sort_key_list (*ironic.api.controllers.v1.inspection_rule.InspectionRuleC attribute*), 844
invalid_sort_key_list (*ironic.api.controllers.v1.node.NodesControlle attribute*), 856
invalid_sort_key_list (*ironic.api.controllers.v1.port.PortsControlle attribute*), 861
invalid_sort_key_list (*ironic.api.controllers.v1.portgroup.PortgroupsControlle attribute*), 863
invalid_sort_key_list (*ironic.api.controllers.v1.runbook.RunbooksControlle attribute*), 866
invalid_sort_key_list (*ironic.api.controllers.v1.volume_connector.VolumeConnectorsC attribute*), 888
invalid_sort_key_list (*ironic.api.controllers.v1.volume_target.VolumeTargetsCon attribute*), 891
InvalidConductorGroup, 950
InvalidDatapathID, 950
InvalidDeployTemplate, 950
InvalidEndpoint, 950
InvalidIdentity, 950
InvalidImage, 951
InvalidImageRef, 951
InvalidInput, 951
InvalidIPAddress, 950
InvalidIPv4Address, 950
InvalidKickstartFile, 951
InvalidKickstartTemplate, 951
InvalidMAC, 951
InvalidMetricConfig, 951
InvalidName, 951
InvalidNodeInventory, 951
InvalidParameterValue, 951
InvalidParams, 916
InvalidRequest, 916
InvalidRunbook, 951
InvalidState, 951
InvalidStateRequested, 951
InvalidSwitchID, 951
InvalidUUID, 951
InvalidUuidOrName, 951
inventory_data (*ironic.db.sqlalchemy.models.NodeInventory attribute*), 1144
InventoryDataController (*ironic.objects.node_inventory.NodeInventory property*), 1517
IPMIConsoleController (*class in ironic.drivers.modules.ipmitool*), 1358
IPMIFailure, 948
IPMIHardware (*class in ironic.drivers.ipmi*), 1429
IPMIManagement (*class in ironic.drivers.modules.ipmitool*), 1358
IPMIPower (*class in ironic.drivers.modules.ipmitool*), 1360
IPMISocatConsole (*class in ironic.drivers.modules.ipmitool*), 1362
ipxe_enabled (*ironic.drivers.modules.ipxe.iPXEBoot attribute*), 1365
ipxe_enabled (*ironic.drivers.modules.ipxe.iPXEHttpBoot attribute*), 1366
ipxe_enabled (*ironic.drivers.modules.pxe_base.PXEBaseMixin attribute*), 1377
iPXEBoot (*class in ironic.drivers.modules.ipxe*), 1365
iPXEHttpBootController (*class in ironic.drivers.modules.ipxe*), 1365
ironic
    module, 1554
ironic.api
    module, 903
ironic.api.app
    module, 898
ironic.api.config
    module, 899
ironic.api.controllers
    module, 894
ironic.api.controllers.base
    module, 893
ironic.api.controllers.link
    module, 893
ironic.api.controllers.root
    module, 894
ironic.api.controllers.v1
    module, 892
ironic.api.controllers.v1.allocation
    module, 833
ironic.api.controllers.v1.bios
    module, 836
ironic.api.controllers.v1.chassis

module, 836
ironic.api.controllers.v1.collection
module, 838
ironic.api.controllers.v1.conductor
module, 839
ironic.api.controllers.v1.deploy_template
module, 839
ironic.api.controllers.v1.driver
module, 841
ironic.api.controllers.v1.event
module, 843
ironic.api.controllers.v1.firmware
module, 843
ironic.api.controllers.v1.inspection_rule
module, 844
ironic.api.controllers.v1.node
module, 845
ironic.api.controllers.v1.notification_utils
module, 859
ironic.api.controllers.v1.port
module, 859
ironic.api.controllers.v1.portgroup
module, 862
ironic.api.controllers.v1.ramdisk
module, 864
ironic.api.controllers.v1.runbook
module, 866
ironic.api.controllers.v1.shard
module, 867
ironic.api.controllers.v1.utils
module, 867
ironic.api.controllers.v1.versions
module, 887
ironic.api.controllers.v1.volume
module, 887
ironic.api.controllers.v1.volume_connector
module, 887
ironic.api.controllers.v1.volume_target
module, 890
ironic.api.controllers.version
module, 894
ironic.api.functions
module, 899
ironic.api.hooks
module, 900
ironic.api.method
module, 902
ironic.api.middleware
module, 895
ironic.api.middleware.auth_public_routes
module, 894
ironic.api.middleware.json_ext
module, 894
ironic.api.middleware.parsable_error
module, 895
ironic.api.middleware.request_log
module, 895
ironic.api.validation
module, 896
ironic.api.validation.validators
module, 896
ironic.api.wsgi
module, 902
ironic.command
module, 905
ironic.command.api
module, 903
ironic.command.conductor
module, 903
ironic.command.dbsync
module, 903
ironic.command.networking
module, 904
ironic.command.novncproxy
module, 904
ironic.command.pxe_filter
module, 904
ironic.command.singleprocess
module, 904
ironic.command.status
module, 904
ironic.command.utils
module, 905
ironic.common
module, 1022
ironic.common.args
module, 927
ironic.common.async_steps
module, 930
ironic.common.auth_basic
module, 931
ironic.common.automated_clean_methods
module, 933
ironic.common.boot_devices
module, 933
ironic.common.boot_modes
module, 934
ironic.common.checksum_utils
module, 934
ironic.common.cinder
module, 936
ironic.common.components

module, 939
ironic.common.config
module, 939
ironic.common.console_factory
module, 939
ironic.common.context
module, 939
ironic.common.dhcp_factory
module, 940
ironic.common.driver_factory
module, 941
ironic.common.exception
module, 944
ironic.common.faults
module, 958
ironic.common.fsm
module, 958
ironic.common.glance_service
module, 907
ironic.common.glance_service.image_service
module, 905
ironic.common.glance_service.service_utils
module, 907
ironic.common.hash_ring
module, 960
ironic.common.health_states
module, 960
ironic.common.i18n
module, 960
ironic.common.image_publisher
module, 960
ironic.common.image_service
module, 962
ironic.common.images
module, 971
ironic.common.indicator_states
module, 976
ironic.common.inspection_rules
module, 915
ironic.common.inspection_rules.actions
module, 907
ironic.common.inspection_rules.base
module, 910
ironic.common.inspection_rules.engine
module, 911
ironic.common.inspection_rules.operators
module, 911
ironic.common.inspection_rules.utils
module, 914
ironic.common.inspection_rules.validation
module, 915
ironic.common.json_rpc
module, 916
ironic.common.json_rpc.client
module, 915
ironic.common.json_rpc.server
module, 916
ironic.common.keystone
module, 977
ironic.common.kickstart_utils
module, 978
ironic.common.lessee_sources
module, 978
ironic.common.mdns
module, 979
ironic.common.metrics
module, 979
ironic.common.metrics_collector
module, 982
ironic.common.metrics_statsd
module, 983
ironic.common.metrics_utils
module, 983
ironic.common.molds
module, 983
ironic.common.network
module, 984
ironic.common.neutron
module, 986
ironic.common.nova
module, 992
ironic.common.oci_registry
module, 992
ironic.common.policy
module, 995
ironic.common.profiler
module, 996
ironic.common.pxe_utils
module, 996
ironic.common.qemu_img
module, 1002
ironic.common.raid
module, 1002
ironic.common.release_mappings
module, 1003
ironic.common.rpc
module, 1004
ironic.common.rpc_service
module, 1005
ironic.common.service
module, 1005
ironic.common.state_machine

module, 1005
ironic.common.states
module, 1006
ironic.common.swift
module, 1010
ironic.common.trait_based_networking
module, 927
ironic.common.trait_based_networking.base
module, 917
ironic.common.trait_based_networking.config_file
module, 924
ironic.common.trait_based_networking.defaults
module, 925
ironic.common.trait_based_networking.grammar
module, 917
ironic.common.trait_based_networking.grammar_parser
module, 916
ironic.common.trait_based_networking.loader
module, 925
ironic.common.trait_based_networking.plan
module, 925
ironic.common.utils
module, 1012
ironic.common.vnc
module, 1020
ironic.common.wsgi_service
module, 1021
ironic.conductor
module, 1084
ironic.conductor.allocations
module, 1022
ironic.conductor.base_manager
module, 1023
ironic.conductor.cleaning
module, 1024
ironic.conductor.configdrive_utils
module, 1025
ironic.conductor.deployments
module, 1028
ironic.conductor.inspection
module, 1030
ironic.conductor.manager
module, 1030
ironic.conductor.notification_utils
module, 1036
ironic.conductor.periodics
module, 1037
ironic.conductor.rpc_service
module, 1038
ironic.conductor.rpcapi
module, 1038
ironic.conductor.servicing
module, 1065
ironic.conductor.steps
module, 1066
ironic.conductor.task_manager
module, 1068
ironic.conductor.utils
module, 1071
ironic.conductor.verify
module, 1084
ironic.conf
module, 1089
ironic.conf.agent
module, 1084
ironic.conf.agent_containers
module, 1084
ironic.conf.anaconda
module, 1084
ironic.conf.ansible
module, 1084
ironic.conf.api
module, 1084
ironic.conf.audit
module, 1084
ironic.conf.auth
module, 1085
ironic.conf.cifs
module, 1085
ironic.conf.cinder
module, 1085
ironic.conf.conductor
module, 1085
ironic.conf.console
module, 1085
ironic.conf.database
module, 1085
ironic.conf.default
module, 1085
ironic.conf.deploy
module, 1085
ironic.conf.dhcp
module, 1086
ironic.conf.disk_utils
module, 1086
ironic.conf.dnsmasq
module, 1086
ironic.conf.drac
module, 1086
ironic.conf.exception
module, 1086
ironic.conf.fake

module, 1086
ironic.conf.glance
module, 1086
ironic.conf.healthcheck
module, 1086
ironic.conf.ilo
module, 1086
ironic.conf.inspector
module, 1086
ironic.conf.inventory
module, 1086
ironic.conf.ipmi
module, 1086
ironic.conf.ironic_networking
module, 1087
ironic.conf.json_rpc
module, 1087
ironic.conf.mdns
module, 1087
ironic.conf.metrics
module, 1087
ironic.conf.molds
module, 1087
ironic.conf.neutron
module, 1087
ironic.conf.nfs
module, 1088
ironic.conf.nova
module, 1088
ironic.conf.oci
module, 1088
ironic.conf.opts
module, 1088
ironic.conf.pxe
module, 1088
ironic.conf.redfish
module, 1088
ironic.conf.sensor_data
module, 1088
ironic.conf.service_catalog
module, 1088
ironic.conf.snmp
module, 1089
ironic.conf.swift
module, 1089
ironic.conf.types
module, 1089
ironic.conf.vnc
module, 1089
ironic.console
module, 1096
ironic.console.container
module, 1092
ironic.console.container.base
module, 1089
ironic.console.container.fake
module, 1090
ironic.console.container.kubernetes
module, 1091
ironic.console.container.systemd
module, 1092
ironic.console.novncproxy_service
module, 1095
ironic.console.rfb
module, 1094
ironic.console.rfb.auth
module, 1092
ironic.console.rfb.authnone
module, 1093
ironic.console.rfb.auths
module, 1094
ironic.console.securityproxy
module, 1095
ironic.console.securityproxy.base
module, 1094
ironic.console.securityproxy.rfb
module, 1095
ironic.console.websocketproxy
module, 1095
ironic.db
module, 1183
ironic.db.api
module, 1148
ironic.db.migration
module, 1183
ironic.db.sqlalchemy
module, 1148
ironic.db.sqlalchemy.api
module, 1096
ironic.db.sqlalchemy.migration
module, 1134
ironic.db.sqlalchemy.models
module, 1135
ironic.dhcp
module, 1189
ironic.dhcp.base
module, 1183
ironic.dhcp.dnsmasq
module, 1185
ironic.dhcp.neutron
module, 1186
ironic.dhcp.none

module, 1188
ironic.drivers
module, 1434
ironic.drivers.base
module, 1389
ironic.drivers.drac
module, 1424
ironic.drivers.fake_hardware
module, 1425
ironic.drivers.generic
module, 1426
ironic.drivers.hardware_type
module, 1427
ironic.drivers.ilo
module, 1428
ironic.drivers.intel_ipmi
module, 1429
ironic.drivers.ipmi
module, 1429
ironic.drivers.modules
module, 1389
ironic.drivers.modules.agent
module, 1296
ironic.drivers.modules.agent_base
module, 1303
ironic.drivers.modules.agent_client
module, 1310
ironic.drivers.modules.agent_power
module, 1318
ironic.drivers.modules.ansible
module, 1191
ironic.drivers.modules.ansible.deploy
module, 1189
ironic.drivers.modules.autodetect
module, 1319
ironic.drivers.modules.boot_mode_utils
module, 1320
ironic.drivers.modules.console_utils
module, 1322
ironic.drivers.modules.deploy_utils
module, 1323
ironic.drivers.modules.drac
module, 1197
ironic.drivers.modules.drac.bios
module, 1191
ironic.drivers.modules.drac.boot
module, 1192
ironic.drivers.modules.drac.inspect
module, 1192
ironic.drivers.modules.drac.management
module, 1193
ironic.drivers.modules.drac.power
module, 1195
ironic.drivers.modules.drac.raid
module, 1195
ironic.drivers.modules.drac.utils
module, 1196
ironic.drivers.modules.drac.vendor_passthru
module, 1197
ironic.drivers.modules.fake
module, 1333
ironic.drivers.modules.graphical_console
module, 1350
ironic.drivers.modules.ilo
module, 1226
ironic.drivers.modules.ilo.bios
module, 1197
ironic.drivers.modules.ilo.boot
module, 1198
ironic.drivers.modules.ilo.common
module, 1205
ironic.drivers.modules.ilo.firmware_processor
module, 1213
ironic.drivers.modules.ilo.inspect
module, 1214
ironic.drivers.modules.ilo.management
module, 1215
ironic.drivers.modules.ilo.power
module, 1222
ironic.drivers.modules.ilo.raid
module, 1224
ironic.drivers.modules.ilo.vendor
module, 1225
ironic.drivers.modules.image_cache
module, 1351
ironic.drivers.modules.image_utils
module, 1352
ironic.drivers.modules.inspect_utils
module, 1356
ironic.drivers.modules.inspector
module, 1238
ironic.drivers.modules.inspector.agent
module, 1234
ironic.drivers.modules.inspector.hooks
module, 1234
ironic.drivers.modules.inspector.hooks.accelerators
module, 1226
ironic.drivers.modules.inspector.hooks.architecture
module, 1226
ironic.drivers.modules.inspector.hooks.base
module, 1226
ironic.drivers.modules.inspector.hooks.boot_mode

module, 1227
ironic.drivers.modules.inspector.hooks.cpu_capabilities
module, 1228
ironic.drivers.modules.inspector.hooks.cpus module, 1228
ironic.drivers.modules.inspector.hooks.extra_hardware
module, 1229
ironic.drivers.modules.inspector.hooks.local_link_connection
module, 1229
ironic.drivers.modules.inspector.hooks.memory module, 1229
ironic.drivers.modules.inspector.hooks.parsers module, 1229
ironic.drivers.modules.inspector.hooks.pci_devices module, 1230
ironic.drivers.modules.inspector.hooks.physical_network
module, 1230
ironic.drivers.modules.inspector.hooks.ports module, 1230
ironic.drivers.modules.inspector.hooks.raid_device module, 1231
ironic.drivers.modules.inspector.hooks.ramdisk_error module, 1232
ironic.drivers.modules.inspector.hooks.root_device module, 1233
ironic.drivers.modules.inspector.hooks.validate_interfaces
module, 1233
ironic.drivers.modules.inspector.interface module, 1234
ironic.drivers.modules.inspector.lldp_parsers module, 1235
ironic.drivers.modules.inspector.lldp_tlvs module, 1237
ironic.drivers.modules.intel_ipmi module, 1239
ironic.drivers.modules.intel_ipmi.management module, 1238
ironic.drivers.modules.ipmitool module, 1358
ironic.drivers.modules.ipxe module, 1365
ironic.drivers.modules.network module, 1257
ironic.drivers.modules.network.common module, 1239
ironic.drivers.modules.network.flat module, 1243
ironic.drivers.modules.network.ironic_networking
module, 1245
ironic.drivers.modules.network.ironic_networking_schemas
module, 1250
ironic.drivers.modules.network.neutron module, 1250
ironic.drivers.modules.network.noop module, 1254
ironic.drivers.modules.network.switchport_config
module, 1256
ironic.drivers.modules.noop module, 1366
ironic.drivers.modules.noop_mgmt module, 1372
ironic.drivers.modules.pxe module, 1375
ironic.drivers.modules.pxe_base module, 1376
ironic.drivers.modules.ramdisk module, 1378
ironic.drivers.modules.redfish module, 1285
ironic.drivers.modules.redfish.bios module, 1257
ironic.drivers.modules.redfish.boot module, 1259
ironic.drivers.modules.redfish.firmware module, 1264
ironic.drivers.modules.redfish.firmware_utils
module, 1265
ironic.drivers.modules.redfish.graphical_console
module, 1266
ironic.drivers.modules.redfish.inspect module, 1267
ironic.drivers.modules.redfish.management module, 1268
ironic.drivers.modules.redfish.power module, 1276
ironic.drivers.modules.redfish.raid module, 1277
ironic.drivers.modules.redfish.utils module, 1281
ironic.drivers.modules.redfish.vendor module, 1284
ironic.drivers.modules.snmp module, 1380
ironic.drivers.modules.storage module, 1289
ironic.drivers.modules.storage.cinder module, 1285
ironic.drivers.modules.storage.external module, 1287
ironic.drivers.modules.storage.noop module, 1288
ironic.drivers.modules.switch

module, 1296
ironic.drivers.modules.switch.base
module, 1289
ironic.drivers.modules.switch.generic_switch
module, 1293
ironic.drivers.redfish
module, 1429
ironic.drivers.snmp
module, 1430
ironic.drivers.utils
module, 1430
ironic.networking
module, 1443
ironic.networking.api
module, 1436
ironic.networking.manager
module, 1438
ironic.networking.rpc_service
module, 1439
ironic.networking.rpcapi
module, 1439
ironic.networking.switch_config
module, 1442
ironic.networking.switch_drivers
module, 1436
ironic.networking.switch_drivers.driver_adapter
module, 1434
ironic.networking.switch_drivers.driver_factory
module, 1435
ironic.networking.utils
module, 1443
ironic.objects
module, 1553
ironic.objects.allocation
module, 1443
ironic.objects.base
module, 1449
ironic.objects.bios
module, 1451
ironic.objects.chassis
module, 1456
ironic.objects.conductor
module, 1460
ironic.objects.deploy_template
module, 1462
ironic.objects.deployment
module, 1467
ironic.objects.fields
module, 1470
ironic.objects.firmware
module, 1473
ironic.objects.indirection
module, 1475
ironic.objects.inspection_rule
module, 1476
ironic.objects.node
module, 1481
ironic.objects.node_history
module, 1513
ironic.objects.node_inventory
module, 1516
ironic.objects.notification
module, 1517
ironic.objects.port
module, 1519
ironic.objects.portgroup
module, 1528
ironic.objects.runbook
module, 1535
ironic.objects.trait
module, 1540
ironic.objects.volume_connector
module, 1542
ironic.objects.volume_target
module, 1547
ironic.pxe_filter
module, 1554
ironic.pxe_filter.dnsmasq
module, 1553
ironic.pxe_filter.service
module, 1554
ironic.version
module, 1554
ironic.wsgi
module, 1554
ironic.wsgi.api
module, 1554
ironic_manages_boot() (*in module ironic.drivers.modules.inspector.interface*), 1235
ironic-dbsync command line option
--config-dir, 456
--config-file, 456
--debug, 456
--help, 456
--version, 456
-d, 456
-h, 456
create_schema,, 456
online_data_migrations, 456
revision, 456
stamp, 456

upgrade, 456
version, 456
IronicBase (*class in ironic.db.sqlalchemy.models*), 1138
IronicCORS (*class in ironic.api.app*), 898
IronicException, 951
IronicNetworking (*class in ironic.drivers.modules.network.ironic_networking*), 1245
IronicObject (*class in ironic.objects.base*), 1449
IronicObjectIndirectionAPI (*class in ironic.objects.indirection*), 1475
IronicObjectListBase (*class in ironic.objects.base*), 1450
IronicObjectRegistry (*class in ironic.objects.base*), 1451
IronicObjectSerializer (*class in ironic.objects.base*), 1451
IronicProxyRequestHandler (*class in ironic.console.websocketproxy*), 1095
IronicRequestId (*class in ironic.api.app*), 898
IronicWebSocketProxy (*class in ironic.console.websocketproxy*), 1096
is_agent_token_pregenerated() (*in module ironic.conductor.utils*), 1074
is_agent_token_present() (*in module ironic.conductor.utils*), 1075
is_agent_token_valid() (*in module ironic.conductor.utils*), 1075
is_anaconda_deploy() (*in module ironic.drivers.modules.deploy_utils*), 1328
is_auth_set_needed (*ironic.common.glance_service.image_service.GlanceImageService property*), 905
is_auth_set_needed (*ironic.common.image_service.BaseImageService property*), 963
is_auth_set_needed (*ironic.common.image_service.OciImageService property*), 969
is_bridging_enabled() (*in module ironic.drivers.modules.ipmitool*), 1364
is_checksum_url() (*in module ironic.common.checksum_utils*), 935
is_config_valid() (*in module ironic.common.trait_based_networking.loader*), 925
is_container_registry_url() (*in module ironic.common.image_service*), 971
is_draining() (*ironic.conductor.rpc_service.RPCService method*), 1038
is_equivalent() (*in module ironic.conductor.steps*), 1066
is_fast_track() (*in module ironic.conductor.utils*), 1075
is_fips_enabled() (*in module ironic.common.utils*), 1014
is_glance_image() (*in module ironic.common.glance_service.service_utils*), 907
is_hostname_safe() (*in module ironic.common.utils*), 1014
is_http_url() (*in module ironic.common.utils*), 1015
is_image_active() (*in module ironic.common.glance_service.service_utils*), 907
is_image_available() (*in module ironic.common.glance_service.service_utils*), 907
is_invalid_network_metadata() (*in module ironic.conductor.configdrive_utils*), 1027
is_ipxe_enabled() (*in module ironic.common.pxe_utils*), 1000
is_ironic_using_sqlite() (*in module ironic.common.utils*), 1015
is_iscsi_boot() (*in module ironic.drivers.modules.deploy_utils*), 1328
is_loopback() (*in module ironic.common.utils*), 1015
is_memory_insufficient() (*in module ironic.common.utils*), 1015
is_mutable_list() (*in module ironic.common.trait_based_networking.plan*), 926
is_oci_artifact_image() (*ironic.common.image_service.OciImageService method*), 969
is_path_removed() (*in module ironic.api.controllers.v1.utils*), 883
is_path_updated() (*in module ironic.api.controllers.v1.utils*), 883
is_port() (*ironic.common.trait_based_networking.base.Port method*), 921
is_port() (*ironic.common.trait_based_networking.base.Portgroup method*), 922
is_portgroup() (*ironic.common.trait_based_networking.base.Port method*), 922
is_portgroup()

(*ironic.common.trait_based_networking.base.PortgroupInfo method*), 922
is_portlike_attached() (*in module ironic.common.trait_based_networking.planner*), 926
is_ramdisk_deploy() (*in module ironic.drivers.modules.deploy_utils*), 1328
is_ramdisk_disabled() (*in module ironic.drivers.modules.deploy_utils*), 1328
is_regex_string_in_file() (*in module ironic.common.utils*), 1015
is_secure_boot_requested() (*in module ironic.drivers.modules.boot_mode_utils*), 1321
is_smartnic (*ironic.db.sqlalchemy.models.Port attribute*), 1145
is_smartnic (*ironic.objects.port.Port property*), 1523
is_smartnic (*ironic.objects.port.PortCRUDPayload property*), 1528
is_smartnic_port() (*in module ironic.common.neutron*), 988
is_software_raid() (*in module ironic.drivers.modules.deploy_utils*), 1328
is_source_a_path() (*in module ironic.common.images*), 975
is_stable() (*ironic.common.fsm.FSM method*), 959
is_switch_configured() (*ironic.drivers.modules.switch.base.NoOpSwitchDriver method*), 1290
is_switch_configured() (*ironic.drivers.modules.switch.base.SwitchDriverBase method*), 1291
is_switch_configured() (*ironic.drivers.modules.switch.generic_switch.GenericSwitchDriver method*), 1294
is_valid (*ironic.drivers.modules.network.switchport_config.SwitchPortConfig property*), 1257
is_valid_datapath_id() (*in module ironic.common.utils*), 1015
is_valid_logical_name() (*in module ironic.api.controllers.v1.utils*), 883
is_valid_logical_name() (*in module ironic.common.utils*), 1015
is_valid_no_proxy() (*in module ironic.common.utils*), 1016
is_valid_node_name() (*in module ironic.api.controllers.v1.utils*), 884
is_volume_attached() (*in module ironic.common.cinder*), 938
is_volume_available() (*in module ironic.common.cinder*), 938
is_whole_disk_image() (*in module ironic.common.images*), 976
ISCSIBOOT (*in module ironic.common.boot_devices*), 933
IsFalseOperator (*class in ironic.common.inspection_rules.operators*), 912
IsNoneOperator (*class in ironic.common.inspection_rules.operators*), 912
ISOImageCache (*class in ironic.drivers.modules.image_utils*), 1352
issue_startup_warnings() (*in module ironic.command.conductor*), 903
issue_startup_warnings() (*in module ironic.command.networking*), 904
IsTrueOperator (*class in ironic.common.inspection_rules.operators*), 912
iswsmefunction() (*in module ironic.api.functions*), 900
items() (*ironic.common.driver_factory.BaseDriverFactory method*), 941
iter_nodes() (*ironic.conductor.base_manager.BaseConductorM method*), 1023

J

JsonExtensionMiddleware (*class in ironic.api.middleware*), 895
JsonExtensionMiddleware (*class in ironic.api.middleware.json_ext*), 894
JsonRpcError, 916

K

keepalive_halt() (*ironic.conductor.base_manager.BaseConductorManager method*), 1024
kernel_ref (*ironic.objects.deployment.Deployment property*), 1469
KeystoneFailure, 952
KeystoneUnauthorized, 952
known_good_state() (*ironic.drivers.modules.drac.management.DracRedfishMa method*), 1194
ks_exceptions() (*in module ironic.common.keystone*), 978

KubernetesConsoleContainer (*class in ironic.console.container.kubernetes*), 1091

L

last_error (*ironic.db.sqlalchemy.models.Allocation attribute*), 1135
last_error (*ironic.db.sqlalchemy.models.Node attribute*), 1140
last_error (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
last_error (*ironic.objects.allocation.Allocation property*), 1445
last_error (*ironic.objects.allocation.AllocationCRUDPayload property*), 1448
last_error (*ironic.objects.node.Node property*), 1485
last_error (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
last_error (*ironic.objects.node.NodeCRUDPayload property*), 1493
last_error (*ironic.objects.node.NodePayload property*), 1503
last_error (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
last_error (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
last_version_flashed (*ironic.db.sqlalchemy.models.FirmwareComponent attribute*), 1138
last_version_flashed (*ironic.objects.firmware.FirmwareComponent property*), 1474
LEGACY_BIOS (*in module ironic.common.boot_modes*), 934
LeOperator (*class in ironic.common.inspection_rules.operators*), 912
lessee (*ironic.db.sqlalchemy.models.Node attribute*), 1140
lessee (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
lessee (*ironic.objects.node.Node property*), 1485
lessee (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
lessee (*ironic.objects.node.NodeCRUDPayload property*), 1493
lessee (*ironic.objects.node.NodePayload property*), 1503
lessee (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
lessee (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
level (*ironic.objects.allocation.AllocationCRUDNotification property*), 1447
level (*ironic.objects.chassis.ChassisCRUDNotification property*), 1459
level (*ironic.objects.deploy_template.DeployTemplateCRUDNotif property*), 1466
level (*ironic.objects.inspection_rule.InspectionRuleCRUDNotifica property*), 1479
level (*ironic.objects.node.NodeConsoleNotification property*), 1494
level (*ironic.objects.node.NodeCorrectedPowerStateNotification property*), 1495
level (*ironic.objects.node.NodeCRUDNotification property*), 1489
level (*ironic.objects.node.NodeMaintenanceNotification property*), 1499
level (*ironic.objects.node.NodeSetPowerStateNotification property*), 1504
level (*ironic.objects.node.NodeSetProvisionStateNotification property*), 1508
level (*ironic.objects.notification.NotificationBase property*), 1518
level (*ironic.objects.port.PortCRUDNotification property*), 1526
level (*ironic.objects.portgroup.PortgroupCRUDNotification property*), 1533
level (*ironic.objects.runbook.RunbookCRUDNotification property*), 1539
level (*ironic.objects.volume_connector.VolumeConnectorCRUDN property*), 1546
level (*ironic.objects.volume_target.VolumeTargetCRUDNotificati property*), 1552
list() (*ironic.objects.allocation.Allocation class method*), 1446
list() (*ironic.objects.chassis.Chassis class method*), 1458
list() (*ironic.objects.conductor.Conductor class method*), 1461
list() (*ironic.objects.deploy_template.DeployTemplate class method*), 1464
list() (*ironic.objects.deployment.Deployment class method*), 1469
list() (*ironic.objects.inspection_rule.InspectionRule class method*), 1478
list() (*ironic.objects.node.Node class method*), 1485
list() (*ironic.objects.node_history.NodeHistory class method*), 1515
list() (*ironic.objects.port.Port class method*),

1523
list() (*ironic.objects.portgroup.Portgroup class method*), 1531
list() (*ironic.objects.runbook.Runbook class method*), 1537
list() (*ironic.objects.volume_connector.VolumeConnector class method*), 1544
list() (*ironic.objects.volume_target.VolumeTarget class method*), 1549
list_by_names() (*ironic.objects.deploy_template.DeployTemplate class method*), 1465
list_by_names() (*ironic.objects.runbook.Runbook class method*), 1537
list_by_node_id() (*ironic.objects.node_history.NodeHistory class method*), 1515
list_by_node_id() (*ironic.objects.port.Port class method*), 1523
list_by_node_id() (*ironic.objects.portgroup.Portgroup class method*), 1531
list_by_node_id() (*ironic.objects.volume_connector.VolumeConnector class method*), 1545
list_by_node_id() (*ironic.objects.volume_target.VolumeTarget class method*), 1550
list_by_node_shards() (*ironic.objects.port.Port class method*), 1524
list_by_node_shards() (*ironic.objects.portgroup.Portgroup class method*), 1532
list_by_portgroup_id() (*ironic.objects.port.Port class method*), 1524
list_by_volume_id() (*ironic.objects.volume_target.VolumeTarget class method*), 1550
list_child_node_ids() (*ironic.objects.node.Node method*), 1486
list_conductor_hardware_interfaces() (*ironic.db.api.Connection method*), 1174
list_conductor_hardware_interfaces() (*ironic.db.sqlalchemy.api.Connection method*), 1123
list_convert_with_links() (*in module ironic.api.controllers.v1.allocation*), 836
list_convert_with_links() (*in module ironic.api.controllers.v1.chassis*), 838
list_convert_with_links() (*in module ironic.api.controllers.v1.collection*), 838
list_convert_with_links() (*in module ironic.api.controllers.v1.conductor*), 839
list_convert_with_links() (*in module ironic.api.controllers.v1.deploy_template*), 840
list_convert_with_links() (*in module ironic.api.controllers.v1.driver*), 843
list_convert_with_links() (*in module ironic.api.controllers.v1.inspection_rule*), 845
list_convert_with_links() (*in module ironic.api.controllers.v1.port*), 862
list_convert_with_links() (*in module ironic.api.controllers.v1.portgroup*), 864
list_convert_with_links() (*in module ironic.api.controllers.v1.runbook*), 867
list_convert_with_links() (*in module ironic.api.controllers.v1.volume_connector*), 890
list_convert_with_links() (*in module ironic.api.controllers.v1.volume_target*), 892
list_hardware_type_interfaces() (*ironic.db.api.Connection method*), 1174
list_hardware_type_interfaces() (*ironic.db.sqlalchemy.api.Connection method*), 1123
list_hardware_type_interfaces_dict() (*ironic.objects.conductor.Conductor class method*), 1461
list_opts() (*in module ironic.conf.agent_containers*), 1084
list_opts() (*in module ironic.conf.cinder*), 1085
list_opts() (*in module ironic.conf.default*), 1085
list_opts() (*in module ironic.conf.glance*), 1086
list_opts() (*in module ironic.conf.ironic_networking*), 1087
list_opts() (*in module ironic.conf.json_rpc*), 1087
list_opts() (*in module ironic.conf.metrics*), 1087
list_opts() (*in module ironic.conf.neutron*), 1087

list_opts() (*in module ironic.conf.nova*), 1088
list_opts() (*in module ironic.conf.opts*), 1088
list_opts() (*in module ironic.conf.service_catalog*), 1088
list_opts() (*in module ironic.conf.swift*), 1089
list_policies() (*in module ironic.common.policy*), 996
list_switch_drivers() (*in module ironic.networking.switch_drivers.driver_factory*), 1435
ListOfFlexibleDictsField (*class in ironic.objects.fields*), 1471
ListOfObjectsField (*class in ironic.objects.fields*), 1471
ListOfStringsField (*class in ironic.objects.fields*), 1471
LLDPBasicMgmtParser (*class in ironic.drivers.modules.inspector.lldp_parsers*), 1235
LLDPdot1Parser (*class in ironic.drivers.modules.inspector.lldp_parsers*), 1237
LLDPdot3Parser (*class in ironic.drivers.modules.inspector.lldp_parsers*), 1237
LLDPParser (*class in ironic.drivers.modules.inspector.lldp_parsers*), 1236
load_driver() (*ironic.conductor.task_manager.TaskManager method*), 1069
local_link_connection (*ironic.db.sqlalchemy.models.Port attribute*), 1145
local_link_connection (*ironic.objects.port.Port property*), 1524
local_link_connection (*ironic.objects.port.PortCRUDPayload property*), 1528
local_link_normalize() (*in module ironic.api.controllers.v1.utils*), 884
LocalContext (*class in ironic.conductor.rpcapi*), 1065
LocalLinkConnectionHook (*class in ironic.drivers.modules.inspector.hooks.local_link_connection*), 1229
LocalPublisher (*class in ironic.common.image_publisher*), 961
lockdown() (*ironic.drivers.modules.agent_client.AgentClient method*), 1316
lockedOff (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry4 attribute*), 1386
lockedOn (*ironic.drivers.modules.snmp.SNMPDriverServerTechSe attribute*), 1386
log_and_raise_deployment_error() (*in module ironic.drivers.modules.agent_base*), 1309
log_passthrough() (*ironic.drivers.modules.fake.FakeVendorB method*), 1349
log_step_flow_history() (*in module ironic.conductor.utils*), 1075
LogAction (*class in ironic.common.inspection_rules.actions*), 909
logical_disk_properties() (*ironic.api.controllers.v1.driver.DriverRaidController method*), 841
lookup_allowed() (*ironic.api.controllers.v1.ramdisk.LookupController method*), 865
LOOKUP_ALLOWED_STATES (*in module ironic.common.states*), 1008
lookup_node() (*in module ironic.drivers.modules.inspect_utils*), 1357
LookupController (*class in ironic.api.controllers.v1.ramdisk*), 865
lower_bound (*ironic.db.sqlalchemy.models.BIOSSetting attribute*), 1135
lower_bound (*ironic.objects.bios.BIOSSetting property*), 1453
LT (*ironic.common.trait_based_networking.base.Comparator attribute*), 919
lt() (*ironic.common.trait_based_networking.grammar.parser.Filte method*), 917
LT_OR_EQ (*ironic.common.trait_based_networking.base.Comparat attribute*), 919
lt_or_eq() (*ironic.common.trait_based_networking.grammar.par method*), 917
LtOperator (*class in ironic.common.inspection_rules.operators*), 912

M

mac_address() (*in module ironic.common.args*), 928
MACAddress (*class in ironic.objects.fields*), 1471
MACAddressField (*class in ironic.objects.fields*), 1471

MACAlreadyExists, 952
MAIN (*ironic.common.inspection_rules.validation.InspectionPhase attribute*), 915
main() (*in module ironic.command.api*), 903
main() (*in module ironic.command.conductor*), 903
main() (*in module ironic.command.dbsync*), 904
main() (*in module ironic.command.networking*), 904
main() (*in module ironic.command.novncproxy*), 904
main() (*in module ironic.command.pxe_filter*), 904
main() (*in module ironic.command.singleprocess*), 904
main() (*in module ironic.command.status*), 904
maintenance (*ironic.api.controllers.v1.node.NodesController attribute*), 856
maintenance (*ironic.db.sqlalchemy.models.Node attribute*), 1140
maintenance (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
maintenance (*ironic.objects.node.Node property*), 1486
maintenance (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
maintenance (*ironic.objects.node.NodeCRUDPayload property*), 1493
maintenance (*ironic.objects.node.NodePayload property*), 1503
maintenance (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
maintenance (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
maintenance_reason (*ironic.db.sqlalchemy.models.Node attribute*), 1140
maintenance_reason (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
maintenance_reason (*ironic.objects.node.Node property*), 1486
maintenance_reason (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
maintenance_reason (*ironic.objects.node.NodeCRUDPayload property*), 1493
maintenance_reason (*ironic.objects.node.NodePayload property*), 1503
maintenance_reason (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
maintenance_reason (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
make_link() (*in module ironic.api.controllers.link*), 893
make_persistent_password_file() (*in module ironic.drivers.modules.console_utils*), 1322
make_salt() (*in module ironic.conductor.utils*), 1076
MakeSession (*class in ironic.common.oci_registry*), 992
MalformedRequestURI, 952
manage_node_history() (*ironic.conductor.manager.ConductorManager method*), 1033
MANAGEABLE (*in module ironic.common.states*), 1008
management (*ironic.api.controllers.v1.node.NodesController attribute*), 856
management (*ironic.drivers.base.BareDriver attribute*), 1391
management_interface (*ironic.db.sqlalchemy.models.Node attribute*), 1140
management_interface (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
management_interface (*ironic.objects.node.Node property*), 1486
management_interface (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
management_interface (*ironic.objects.node.NodeCRUDPayload property*), 1493
management_interface (*ironic.objects.node.NodePayload property*), 1503
management_interface (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
management_interface (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
ManagementInterface (*class in ironic.drivers.base*), 1402

mandatory (*ironic.api.functions.FunctionArgument attribute*), 899
ManualManagementHardware (*class in ironic.drivers.generic*), 1426
mapping_for_enum() (*in module ironic.drivers.modules.inspector.lldp_tlvs*), 1238
mapping_for_switch() (*in module ironic.drivers.modules.inspector.lldp_tlvs*), 1238
marginal (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1384
mask_secrets() (*in module ironic.objects.notification*), 1519
match_root_device_hints() (*in module ironic.common.utils*), 1016
matches() (*ironic.common.trait_based_networking.base.TraitAction method*), 923
MatchesOperator (*class in ironic.common.inspection_rules.operators*), 912
max_count (*ironic.common.trait_based_networking.base.TraitAction attribute*), 923
max_length (*ironic.db.sqlalchemy.models.BIOSSetting attribute*), 1135
max_length (*ironic.objects.bios.BIOSSetting property*), 1453
max_string (*ironic.api.controllers.base.Version attribute*), 893
max_version() (*in module ironic.objects.base*), 1451
max_version_string() (*in module ironic.api.controllers.v1.versions*), 887
member_list() (*ironic.common.glance_service.image_service.GlanceImageService method*), 906
memoize() (*in module ironic.drivers.modules.snmp*), 1389
MemoryHook (*class in ironic.drivers.modules.inspector.hooks.memory*), 1229
message (*ironic.common.exception.InvalidMetricConfig attribute*), 951
metadata (*ironic.db.sqlalchemy.models.IronicBase attribute*), 1138
metadata (*ironic.drivers.base.VendorMetadata attribute*), 1420
method (*ironic.drivers.base.VendorMetadata attribute*), 1420
MethodNotFound, 916
methods() (*ironic.api.controllers.v1.driver.DriverPassthruController method*), 841
methods() (*ironic.api.controllers.v1.node.NodeVendorPassthruController method*), 853
MetricLogger (*class in ironic.common.metrics*), 980
MetricsNotSupported, 952
migrate_to_builtin_inspection() (*ironic.db.sqlalchemy.api.Connection method*), 1123
min_count (*ironic.common.trait_based_networking.base.TraitAction attribute*), 923
min_length (*ironic.db.sqlalchemy.models.BIOSSetting attribute*), 1135
min_length (*ironic.objects.bios.BIOSSetting property*), 1453
min_string (*ironic.api.controllers.base.Version attribute*), 893
min_version_string() (*in module ironic.api.controllers.v1.versions*), 887
missing_entrypoints_callback() (*in module ironic.drivers.modules.inspect_utils*), 1357
MissingParameterValue, 952
MixinVendorInterface (*class in ironic.drivers.utils*), 1430
mkfs() (*in module ironic.common.utils*), 1017
mode (*ironic.db.sqlalchemy.models.Portgroup attribute*), 1146
mode (*ironic.drivers.modules.network.switchport_config.SwitchPort attribute*), 1257
mode (*ironic.objects.portgroup.Portgroup property*), 1532
mode (*ironic.objects.portgroup.PortgroupCRUDPayload property*), 1534
model_query() (*in module ironic.db.sqlalchemy.api*), 1133
module
  ironic, 1554
  ironic.api, 903
  ironic.api.app, 898
  ironic.api.config, 899
  ironic.api.controllers, 894
  ironic.api.controllers.base, 893
  ironic.api.controllers.link, 893
  ironic.api.controllers.root, 894
  ironic.api.controllers.v1, 892
  ironic.api.controllers.v1.allocation, 833
  ironic.api.controllers.v1.bios, 836
  ironic.api.controllers.v1.chassis,

836
ironic.api.controllers.v1.collection, 838
ironic.api.controllers.v1.conductor, 839
ironic.api.controllers.v1.deploy_template, 839
ironic.api.controllers.v1.driver, 841
ironic.api.controllers.v1.event, 843
ironic.api.controllers.v1.firmware, 843
ironic.api.controllers.v1.inspection_rule, 844
ironic.api.controllers.v1.node, 845
ironic.api.controllers.v1.notification_utils, 859
ironic.api.controllers.v1.port, 859
ironic.api.controllers.v1.portgroup, 862
ironic.api.controllers.v1.ramdisk, 864
ironic.api.controllers.v1.runbook, 866
ironic.api.controllers.v1.shard, 867
ironic.api.controllers.v1.utils, 867
ironic.api.controllers.v1.versions, 887
ironic.api.controllers.v1.volume, 887
ironic.api.controllers.v1.volume_connector, 887
ironic.api.controllers.v1.volume_target, 890
ironic.api.controllers.version, 894
ironic.api.functions, 899
ironic.api.hooks, 900
ironic.api.method, 902
ironic.api.middleware, 895
ironic.api.middleware.auth_public_routes, 894
ironic.api.middleware.json_ext, 894
ironic.api.middleware.parsable_error, 895
ironic.api.middleware.request_log, 895
ironic.api.validation, 896
ironic.api.validation.validators, 896
ironic.api.wsgi, 902
ironic.command, 905
ironic.command.api, 903
ironic.command.conductor, 903
ironic.command.dbsync, 903
ironic.command.networking, 904
ironic.command.novncproxy, 904
ironic.command.pxe_filter, 904
ironic.command.singleprocess, 904
ironic.command.status, 904
ironic.command.utils, 905
ironic.common, 1022
ironic.common.args, 927
ironic.common.async_steps, 930
ironic.common.auth_basic, 931
ironic.common.automated_clean_methods, 933
ironic.common.boot_devices, 933
ironic.common.boot_modes, 934
ironic.common.checksum_utils, 934
ironic.common.cinder, 936
ironic.common.components, 939
ironic.common.config, 939
ironic.common.console_factory, 939
ironic.common.context, 939
ironic.common.dhcp_factory, 940
ironic.common.driver_factory, 941
ironic.common.exception, 944
ironic.common.faults, 958
ironic.common.fsm, 958
ironic.common.glance_service, 907
ironic.common.glance_service.image_service, 905
ironic.common.glance_service.service_utils, 907
ironic.common.hash_ring, 960
ironic.common.health_states, 960
ironic.common.i18n, 960
ironic.common.image_publisher, 960
ironic.common.image_service, 962
ironic.common.images, 971
ironic.common.indicator_states, 976
ironic.common.inspection_rules, 915
ironic.common.inspection_rules.actions, 907
ironic.common.inspection_rules.base, 910
ironic.common.inspection_rules.engine, 911
ironic.common.inspection_rules.operators, 911
ironic.common.inspection_rules.utils, 914

ironic.common.inspection_rules.validation, 915
ironic.common.json_rpc, 916
ironic.common.json_rpc.client, 915
ironic.common.json_rpc.server, 916
ironic.common.keystone, 977
ironic.common.kickstart_utils, 978
ironic.common.lessee_sources, 978
ironic.common.mdns, 979
ironic.common.metrics, 979
ironic.common.metrics_collector, 982
ironic.common.metrics_statsd, 983
ironic.common.metrics_utils, 983
ironic.common.molds, 983
ironic.common.network, 984
ironic.common.neutron, 986
ironic.common.nova, 992
ironic.common.oci_registry, 992
ironic.common.policy, 995
ironic.common.profiler, 996
ironic.common.pxe_utils, 996
ironic.common.qemu_img, 1002
ironic.common.raid, 1002
ironic.common.release_mappings, 1003
ironic.common.rpc, 1004
ironic.common.rpc_service, 1005
ironic.common.service, 1005
ironic.common.state_machine, 1005
ironic.common.states, 1006
ironic.common.swift, 1010
ironic.common.trait_based_networking, 927
ironic.common.trait_based_networking.base, 917
ironic.common.trait_based_networking.configdrive, 924
ironic.common.trait_based_networking.default, 925
ironic.common.trait_based_networking.grammar, 917
ironic.common.trait_based_networking.grammar_parser, 916
ironic.common.trait_based_networking.loader, 925
ironic.common.trait_based_networking.plan, 925
ironic.common.utils, 1012
ironic.common.vnc, 1020
ironic.common.wsgi_service, 1021
ironic.conductor, 1084
ironic.conductor.allocations, 1022
ironic.conductor.base_manager, 1023
ironic.conductor.cleaning, 1024
ironic.conductor.configdrive_utils, 1025
ironic.conductor.deployments, 1028
ironic.conductor.inspection, 1030
ironic.conductor.manager, 1030
ironic.conductor.notification_utils, 1036
ironic.conductor.periodics, 1037
ironic.conductor.rpc_service, 1038
ironic.conductor.rpcapi, 1038
ironic.conductor.servicing, 1065
ironic.conductor.steps, 1066
ironic.conductor.task_manager, 1068
ironic.conductor.utils, 1071
ironic.conductor.verify, 1084
ironic.conf, 1089
ironic.conf.agent, 1084
ironic.conf.agent_containers, 1084
ironic.conf.anaconda, 1084
ironic.conf.ansible, 1084
ironic.conf.api, 1084
ironic.conf.audit, 1084
ironic.conf.auth, 1085
ironic.conf.cifs, 1085
ironic.conf.cinder, 1085
ironic.conf.conductor, 1085
ironic.conf.console, 1085
ironic.conf.database, 1085
ironic.conf.default, 1085
ironic.conf.deploy, 1085
ironic.conf.dhcp, 1086
ironic.conf.disk_utils, 1086
ironic.conf.dnsmasq, 1086
ironic.conf.drac, 1086
ironic.conf.exception, 1086
ironic.conf.fake, 1086
ironic.conf.glance, 1086
ironic.conf.healthcheck, 1086
ironic.conf.ilo, 1086
ironic.conf.inspector, 1086
ironic.conf.inventory, 1086
ironic.conf.ipmi, 1086
ironic.conf.ironic_networking, 1087
ironic.conf.json_rpc, 1087
ironic.conf.mdns, 1087
ironic.conf.metrics, 1087
ironic.conf.molds, 1087
ironic.conf.neutron, 1087
ironic.conf.nfs, 1088

ironic.conf.nova, 1088
ironic.conf.oci, 1088
ironic.conf.opts, 1088
ironic.conf.pxe, 1088
ironic.conf.redfish, 1088
ironic.conf.sensor_data, 1088
ironic.conf.service_catalog, 1088
ironic.conf.snmp, 1089
ironic.conf.swift, 1089
ironic.conf.types, 1089
ironic.conf.vnc, 1089
ironic.console, 1096
ironic.console.container, 1092
ironic.console.container.base, 1089
ironic.console.container.fake, 1090
ironic.console.container.kubernetes, 1091
ironic.console.container.systemd, 1092
ironic.console.novncproxy_service, 1095
ironic.console.rfb, 1094
ironic.console.rfb.auth, 1092
ironic.console.rfb.authnone, 1093
ironic.console.rfb.auths, 1094
ironic.console.securityproxy, 1095
ironic.console.securityproxy.base, 1094
ironic.console.securityproxy.rfb, 1095
ironic.console.websocketproxy, 1095
ironic.db, 1183
ironic.db.api, 1148
ironic.db.migration, 1183
ironic.db.sqlalchemy, 1148
ironic.db.sqlalchemy.api, 1096
ironic.db.sqlalchemy.migration, 1134
ironic.db.sqlalchemy.models, 1135
ironic.dhcp, 1189
ironic.dhcp.base, 1183
ironic.dhcp.dnsmasq, 1185
ironic.dhcp.neutron, 1186
ironic.dhcp.none, 1188
ironic.drivers, 1434
ironic.drivers.base, 1389
ironic.drivers.drac, 1424
ironic.drivers.fake_hardware, 1425
ironic.drivers.generic, 1426
ironic.drivers.hardware_type, 1427
ironic.drivers.ilo, 1428
ironic.drivers.intel_ipmi, 1429
ironic.drivers.ipmi, 1429
ironic.drivers.modules, 1389
ironic.drivers.modules.agent, 1296
ironic.drivers.modules.agent_base, 1303
ironic.drivers.modules.agent_client, 1310
ironic.drivers.modules.agent_power, 1318
ironic.drivers.modules.ansible, 1191
ironic.drivers.modules.ansible.deploy, 1189
ironic.drivers.modules.autodetect, 1319
ironic.drivers.modules.boot_mode_utils, 1320
ironic.drivers.modules.console_utils, 1322
ironic.drivers.modules.deploy_utils, 1323
ironic.drivers.modules.drac, 1197
ironic.drivers.modules.drac.bios, 1191
ironic.drivers.modules.drac.boot, 1192
ironic.drivers.modules.drac.inspect, 1192
ironic.drivers.modules.drac.management, 1193
ironic.drivers.modules.drac.power, 1195
ironic.drivers.modules.drac.raid, 1195
ironic.drivers.modules.drac.utils, 1196
ironic.drivers.modules.drac.vendor_passthru, 1197
ironic.drivers.modules.fake, 1333
ironic.drivers.modules.graphical_console, 1350
ironic.drivers.modules.ilo, 1226
ironic.drivers.modules.ilo.bios, 1197
ironic.drivers.modules.ilo.boot, 1198
ironic.drivers.modules.ilo.common, 1205
ironic.drivers.modules.ilo.firmware_processor, 1213
ironic.drivers.modules.ilo.inspect, 1214

ironic.drivers.modules.ilo.management, 1215
ironic.drivers.modules.ilo.power, 1222
ironic.drivers.modules.ilo.raid, 1224
ironic.drivers.modules.ilo.vendor, 1225
ironic.drivers.modules.image_cache, 1351
ironic.drivers.modules.image_utils, 1352
ironic.drivers.modules.inspect_utils, 1356
ironic.drivers.modules.inspector, 1238
ironic.drivers.modules.inspector.agent, 1234
ironic.drivers.modules.inspector.hooks, 1234
ironic.drivers.modules.inspector.hooks.accelerators, 1226
ironic.drivers.modules.inspector.hooks.architecture, 1226
ironic.drivers.modules.inspector.hooks.base, 1226
ironic.drivers.modules.inspector.hooks.boot_mode, 1227
ironic.drivers.modules.inspector.hooks.cpu_capabilities, 1228
ironic.drivers.modules.inspector.hooks.cpus, 1228
ironic.drivers.modules.inspector.hooks.extra_hardware, 1229
ironic.drivers.modules.inspector.hooks.local_link_connection, 1229
ironic.drivers.modules.inspector.hooks.memory, 1229
ironic.drivers.modules.inspector.hooks.parse_lldp, 1229
ironic.drivers.modules.inspector.hooks.pci_devices, 1230
ironic.drivers.modules.inspector.hooks.physical_network, 1230
ironic.drivers.modules.inspector.hooks.ports, 1230
ironic.drivers.modules.inspector.hooks.raid_device, 1231
ironic.drivers.modules.inspector.hooks.ramdisk_error, 1232
ironic.drivers.modules.inspector.hooks.root_device, 1233
ironic.drivers.modules.inspector.hooks.validate_, 1233
ironic.drivers.modules.inspector.interface, 1234
ironic.drivers.modules.inspector.lldp_parsers, 1235
ironic.drivers.modules.inspector.lldp_tlvs, 1237
ironic.drivers.modules.intel_ipmi, 1239
ironic.drivers.modules.intel_ipmi.management, 1238
ironic.drivers.modules.ipmitool, 1358
ironic.drivers.modules.ipxe, 1365
ironic.drivers.modules.network, 1257
ironic.drivers.modules.network.common, 1239
ironic.drivers.modules.network.flat, 1243
ironic.drivers.modules.network.ironic_networking, 1245
ironic.drivers.modules.network.ironic_networking, 1250
ironic.drivers.modules.network.neutron, 1250
ironic.drivers.modules.network.noop, 1251
ironic.drivers.modules.network.switchport_config, 1256
ironic.drivers.modules.noop, 1366
ironic.drivers.modules.noop_mgmt, 1372
ironic.drivers.modules.pxe, 1375
ironic.drivers.modules.pxe_base, 1376
ironic.drivers.modules.ramdisk, 1378
ironic.drivers.modules.redfish, 1285
ironic.drivers.modules.redfish.bios, 1257
ironic.drivers.modules.redfish.boot, 1259
ironic.drivers.modules.redfish.firmware, 1264
ironic.drivers.modules.redfish.firmware_utils, 1265
ironic.drivers.modules.redfish.graphical_console, 1266
ironic.drivers.modules.redfish.inspect, 1267

ironic.drivers.modules.redfish.management, 1268
ironic.drivers.modules.redfish.power, 1276
ironic.drivers.modules.redfish.raid, 1277
ironic.drivers.modules.redfish.utils, 1281
ironic.drivers.modules.redfish.vendor, 1284
ironic.drivers.modules.snmp, 1380
ironic.drivers.modules.storage, 1289
ironic.drivers.modules.storage.cinder, 1285
ironic.drivers.modules.storage.external, 1287
ironic.drivers.modules.storage.noop, 1288
ironic.drivers.modules.switch, 1296
ironic.drivers.modules.switch.base, 1289
ironic.drivers.modules.switch.generic_switch, 1293
ironic.drivers.redfish, 1429
ironic.drivers.snmp, 1430
ironic.drivers.utils, 1430
ironic.networking, 1443
ironic.networking.api, 1436
ironic.networking.manager, 1438
ironic.networking.rpc_service, 1439
ironic.networking.rpcapi, 1439
ironic.networking.switch_config, 1442
ironic.networking.switch_drivers, 1436
ironic.networking.switch_drivers.driver_adapter, 1434
ironic.networking.switch_drivers.driver_factory, 1435
ironic.networking.utils, 1443
ironic.objects, 1553
ironic.objects.allocation, 1443
ironic.objects.base, 1449
ironic.objects.bios, 1451
ironic.objects.chassis, 1456
ironic.objects.conductor, 1460
ironic.objects.deploy_template, 1462
ironic.objects.deployment, 1467
ironic.objects.fields, 1470
ironic.objects.firmware, 1473
ironic.objects.indirection, 1475
ironic.objects.inspection_rule, 1476
ironic.objects.node, 1481
ironic.objects.node_history, 1513
ironic.objects.node_inventory, 1516
ironic.objects.notification, 1517
ironic.objects.port, 1519
ironic.objects.portgroup, 1528
ironic.objects.runbook, 1535
ironic.objects.trait, 1540
ironic.objects.volume_connector, 1542
ironic.objects.volume_target, 1547
ironic.pxe_filter, 1554
ironic.pxe_filter.dnsmasq, 1553
ironic.pxe_filter.service, 1554
ironic.version, 1554
ironic.wsgi, 1554
ironic.wsgi.api, 1554
MSLOGON (*ironic.console.rfb.auth.AuthType attribute*), 1093

N

name (*ironic.api.functions.FunctionArgument attribute*), 899
name (*ironic.api.functions.FunctionDefinition attribute*), 899
name (*ironic.common.trait_based_networking.base.Network attribute*), 920
name (*ironic.db.sqlalchemy.models.Allocation attribute*), 1135
name (*ironic.db.sqlalchemy.models.BIOSSetting attribute*), 1135
name (*ironic.db.sqlalchemy.models.DeployTemplate attribute*), 1137
name (*ironic.db.sqlalchemy.models.Node attribute*), 1140
name (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
name (*ironic.db.sqlalchemy.models.Port attribute*), 1145
name (*ironic.db.sqlalchemy.models.Portgroup attribute*), 1146
name (*ironic.db.sqlalchemy.models.Runbook attribute*), 1146
name (*ironic.objects.allocation.Allocation property*), 1446
name (*ironic.objects.allocation.AllocationCRUDPayload property*), 1448
name (*ironic.objects.bios.BIOSSetting property*), 1453
name (*ironic.objects.deploy_template.DeployTemplate*

property), 1465
name (ironic.objects.deploy_template.DeployTemplateCRUDPayload property), 1467
name (ironic.objects.node.Node property), 1486
name (ironic.objects.node.NodeCorrectedPowerStatePayload property), 1498
name (ironic.objects.node.NodeCRUDPayload property), 1493
name (ironic.objects.node.NodePayload property), 1503
name (ironic.objects.node.NodeSetPowerStatePayload property), 1507
name (ironic.objects.node.NodeSetProvisionStatePayload property), 1512
name (ironic.objects.port.Port property), 1524
name (ironic.objects.port.PortCRUDPayload property), 1528
name (ironic.objects.portgroup.Portgroup property), 1532
name (ironic.objects.portgroup.PortgroupCRUDPayload property), 1534
name (ironic.objects.runbook.Runbook property), 1537
name (ironic.objects.runbook.RunbookCRUDPayload property), 1539
name() (in module ironic.common.args), 928
names (ironic.common.driver_factory.BaseDriverFactory property), 941
native_vlan (ironic.drivers.modules.network.switchport_config.SwitchportConfig attribute), 1257
NECESSARY_KEYS (ironic.common.trait_based_networking.base.TraitAction attribute), 923
need_power_on() (ironic.drivers.base.NetworkInterface method), 1411
need_power_on() (ironic.drivers.modules.network.ironic_networking.IronicNetworking method), 1247
need_power_on() (ironic.drivers.modules.network.neutron.NeutronNetwork method), 1252
need_prepare_ramdisk() (in module ironic.drivers.utils), 1433
needs_agent_ramdisk() (in module ironic.drivers.modules.deploy_utils), 1328
NeOperator (class in ironic.common.inspection_rules.operators), 912
NetOperator (class in ironic.common.inspection_rules.operators), 912
NetType (class in ironic.drivers.modules.network.common), 1239
Network (class in ironic.common.trait_based_networking.base), 920
network (ironic.drivers.base.BareDriver attribute), 1391
network_data (ironic.db.sqlalchemy.models.Node attribute), 1140
network_data (ironic.db.sqlalchemy.models.NodeBase attribute), 1142
network_data (ironic.objects.node.Node property), 1486
network_data_schema() (in module ironic.api.controllers.v1.node), 857
network_id (ironic.common.trait_based_networking.base.AttachP attribute), 918
network_id (ironic.common.trait_based_networking.base.AttachP attribute), 918
network_id (ironic.common.trait_based_networking.base.GroupA attribute), 920
network_interface (ironic.db.sqlalchemy.models.Node attribute), 1140
network_interface (ironic.db.sqlalchemy.models.NodeBase attribute), 1142
network_interface (ironic.objects.node.Node property), 1486
network_interface (ironic.objects.node.NodeCorrectedPowerStatePayload property), 1498
network_interface (ironic.objects.node.NodeCRUDPayload property), 1493
network_interface (ironic.objects.node.NodePayload property), 1503
network_interface (ironic.objects.node.NodeSetPowerStatePayload property), 1507
network_interface (ironic.objects.node.NodeSetProvisionStatePayload property), 1512
NETWORK_NAME (ironic.common.trait_based_networking.base.Vari attribute), 924
NETWORK_TAGS (ironic.common.trait_based_networking.base.Vari attribute), 924

NetworkError, 952
NetworkingAPI (*class in ironic.networking.rpcapi*), 1439
NetworkingDriverAdapter (*class in ironic.networking.switch_drivers.driver_adapter*), 1434
NetworkingManager (*class in ironic.networking.manager*), 1438
NetworkingRPCService (*class in ironic.networking.rpc_service*), 1439
NetworkInterface (*class in ironic.drivers.base*), 1409
NetworkInterfaceFactory (*class in ironic.common.driver_factory*), 941
NetworkTrait (*class in ironic.common.trait_based_networking.base*), 920
NeutronDHCPApi (*class in ironic.dhcp.neutron*), 1186
NeutronNetwork (*class in ironic.drivers.modules.network.neutron*), 1250
NeutronNetworkInterfaceMixin (*class in ironic.common.neutron*), 986
NeutronVIFPortIDMixin (*class in ironic.drivers.modules.network.common*), 1239
new_continue_inspection_endpoint() (*in module ironic.api.controllers.v1.utils*), 884
new_websocket_client() (*ironic.console.websocketproxy.IronicProxyRequestHandler method*), 1095
NfsImageService (*class in ironic.common.image_service*), 967
NFSPublisher (*class in ironic.common.image_publisher*), 962
NIC (*in module ironic.common.components*), 939
NIC (*in module ironic.drivers.modules.redfish.utils*), 1281
NIC_COMPONENT_PREFIX (*in module ironic.drivers.modules.redfish.utils*), 1281
no (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1384
NoBIOS (*class in ironic.drivers.modules.noop*), 1366
NoConsole (*class in ironic.drivers.modules.noop*), 1367
NoConsolePid, 952
Node (*class in ironic.db.sqlalchemy.models*), 1139
Node (*class in ironic.objects.node*), 1481
node (*ironic.conductor.task_manager.TaskManager property*), 1069
node (*ironic.db.sqlalchemy.models.NodeTag attribute*), 1144
node (*ironic.db.sqlalchemy.models.NodeTrait attribute*), 1144
node_cache_bios_settings() (*in module ironic.conductor.utils*), 1076
node_cache_boot_mode() (*in module ironic.conductor.utils*), 1076
node_cache_firmware_components() (*in module ironic.conductor.utils*), 1076
node_cache_health() (*in module ironic.conductor.utils*), 1076
node_cache_vendor() (*in module ironic.conductor.utils*), 1076
node_change_boot_mode() (*in module ironic.conductor.utils*), 1076
node_change_secure_boot() (*in module ironic.conductor.utils*), 1076
node_convert_with_links() (*in module ironic.api.controllers.v1.node*), 857
node_get_boot_mode() (*in module ironic.conductor.utils*), 1076
node_history_record() (*in module ironic.conductor.utils*), 1077
node_id (*ironic.db.sqlalchemy.models.Allocation attribute*), 1135
node_id (*ironic.db.sqlalchemy.models.BIOSSetting attribute*), 1135
node_id (*ironic.db.sqlalchemy.models.FirmwareComponent attribute*), 1138
node_id (*ironic.db.sqlalchemy.models.NodeHistory attribute*), 1143
node_id (*ironic.db.sqlalchemy.models.NodeInventory attribute*), 1144
node_id (*ironic.db.sqlalchemy.models.NodeTag attribute*), 1144
node_id (*ironic.db.sqlalchemy.models.NodeTrait attribute*), 1144
node_id (*ironic.db.sqlalchemy.models.Port attribute*), 1145
node_id (*ironic.db.sqlalchemy.models.Portgroup attribute*), 1146
node_id (*ironic.db.sqlalchemy.models.VolumeConnector attribute*), 1147
node_id (*ironic.db.sqlalchemy.models.VolumeTarget attribute*), 1147
node_id (*ironic.objects.allocation.Allocation property*), 1446

node_id (*ironic.objects.bios.BIOSSetting property*), 1454
node_id (*ironic.objects.firmware.FirmwareComponent property*), 1474
node_id (*ironic.objects.node_history.NodeHistory property*), 1515
node_id (*ironic.objects.node_inventory.NodeInventory property*), 1517
node_id (*ironic.objects.port.Port property*), 1525
node_id (*ironic.objects.portgroup.Portgroup property*), 1532
node_id (*ironic.objects.trait.Trait property*), 1541
node_id (*ironic.objects.volume_connector.VolumeConnector property*), 1545
node_id (*ironic.objects.volume_target.VolumeTarget property*), 1551
node_list_convert_with_links() (*in module ironic.api.controllers.v1.node*), 858
node_mapping (*ironic.objects.deployment.Deployment attribute*), 1469
node_patch_schema() (*in module ironic.api.controllers.v1.node*), 858
node_patch_validator() (*in module ironic.api.controllers.v1.node*), 858
node_periodic() (*in module ironic.conductor.periodics*), 1037
node_power_action() (*in module ironic.conductor.utils*), 1077
node_sanitize() (*in module ironic.api.controllers.v1.node*), 858
node_schema() (*in module ironic.api.controllers.v1.node*), 858
node_set_boot_device() (*in module ironic.conductor.utils*), 1078
node_set_boot_mode() (*in module ironic.conductor.utils*), 1078
node_states_convert() (*in module ironic.api.controllers.v1.node*), 858
node_tag_exists() (*ironic.db.api.Connection method*), 1174
node_tag_exists() (*ironic.db.sqlalchemy.api.Connection method*), 1123
node_trait_exists() (*ironic.db.api.Connection method*), 1174
node_trait_exists() (*ironic.db.sqlalchemy.api.Connection method*), 1123
node_update_cache() (*in module ironic.conductor.utils*), 1079
node_uuid (*ironic.common.trait_based_networking.base.Rendered attribute*), 922
node_uuid (*ironic.db.sqlalchemy.models.Port attribute*), 1145
node_uuid (*ironic.db.sqlalchemy.models.Portgroup attribute*), 1146
node_uuid (*ironic.objects.allocation.AllocationCRUDPayload property*), 1448
node_uuid (*ironic.objects.deployment.Deployment property*), 1469
node_uuid (*ironic.objects.port.Port property*), 1525
node_uuid (*ironic.objects.port.PortCRUDPayload property*), 1528
node_uuid (*ironic.objects.portgroup.Portgroup property*), 1532
node_uuid (*ironic.objects.portgroup.PortgroupCRUDPayload property*), 1534
node_uuid (*ironic.objects.volume_connector.VolumeConnectorCR property*), 1547
node_uuid (*ironic.objects.volume_target.VolumeTargetCRUDPayl property*), 1553
node_validator() (*in module ironic.api.controllers.v1.node*), 858
node_wait_for_power_state() (*in module ironic.conductor.utils*), 1079
NodeAllocationController (*class in ironic.api.controllers.v1.allocation*), 835
NodeAlreadyExists, 953
NodeAssociated, 953
NodeBase (*class in ironic.db.sqlalchemy.models*), 1141
NodeBiosController (*class in ironic.api.controllers.v1.bios*), 836
NodeChildrenController (*class in ironic.api.controllers.v1.node*), 847
NodeCleaningFailure, 953
NodeConsoleController (*class in ironic.api.controllers.v1.node*), 847
NodeConsoleNotEnabled, 953
NodeConsoleNotification (*class in ironic.objects.node*), 1494
NodeCorrectedPowerStateNotification (*class in ironic.objects.node*), 1494
NodeCorrectedPowerStatePayload (*class in ironic.objects.node*), 1495
NodeCRUDNotification (*class in ironic.objects.node*), 1489
NodeCRUDPayload (*class in ironic.objects.node*), 1489

NodeFirmwareController (*class in ironic.api.controllers.v1.firmware*), 843
NodeHistory (*class in ironic.db.sqlalchemy.models*), 1143
NodeHistory (*class in ironic.objects.node_history*), 1513
NodeHistoryController (*class in ironic.api.controllers.v1.node*), 847
NodeHistoryNotFound, 953
NodeInMaintenance, 953
NodeInventory (*class in ironic.db.sqlalchemy.models*), 1144
NodeInventory (*class in ironic.objects.node_inventory*), 1516
NodeInventoryAlreadyExists, 953
NodeInventoryController (*class in ironic.api.controllers.v1.node*), 848
NodeInventoryNotFound, 953
NodeIsRetired, 953
NodeLocked, 953
NodeMaintenanceController (*class in ironic.api.controllers.v1.node*), 848
NodeMaintenanceFailure, 953
NodeMaintenanceNotification (*class in ironic.objects.node*), 1499
NodeManagementController (*class in ironic.api.controllers.v1.node*), 848
NodeNotFound, 953
NodeNotLocked, 953
NodePayload (*class in ironic.objects.node*), 1499
NoDeploy (*class in ironic.drivers.modules.noop*), 1368
NodeProtected, 953
nodes (*ironic.api.controllers.v1.chassis.ChassisController attribute*), 837
NodesController (*class in ironic.api.controllers.v1.node*), 853
NodeServicingFailure, 953
NodeSetPowerStateNotification (*class in ironic.objects.node*), 1504
NodeSetPowerStatePayload (*class in ironic.objects.node*), 1504
NodeSetProvisionStateNotification (*class in ironic.objects.node*), 1508
NodeSetProvisionStatePayload (*class in ironic.objects.node*), 1508
NodeStatesController (*class in ironic.api.controllers.v1.node*), 848
NodeTag (*class in ironic.db.sqlalchemy.models*), 1144
NodeTagNotFound, 953
NodeTrait (*class in ironic.db.sqlalchemy.models*), 1144
NodeTraitNotFound, 953
NodeTraitsController (*class in ironic.api.controllers.v1.node*), 852
NodeVendorPassthruController (*class in ironic.api.controllers.v1.node*), 853
NodeVerifyFailure, 954
NodeVIFController (*class in ironic.api.controllers.v1.node*), 852
NodeVmediaController (*class in ironic.api.controllers.v1.node*), 853
NoDriversLoaded, 952
NoExceptionTracebackHook (*class in ironic.api.hooks*), 901
NoFirmware (*class in ironic.drivers.modules.noop*), 1369
NoFreeConductorWorker, 952
NoFreeIPMITerminalPorts, 952
NoFreePhysicalPorts, 952
NoInspect (*class in ironic.drivers.modules.noop*), 1370
NoMatch (*class in ironic.common.trait_based_networking.base*), 920
non_vendor_interfaces (*ironic.drivers.base.BareDriver property*), 1392
NONE (*in module ironic.common.lessee_sources*), 978
NONE (*ironic.console.rfb.auth.AuthType attribute*), 1093
NoneDHCPApi (*class in ironic.dhcp.none*), 1188
NoopManagement (*class in ironic.drivers.modules.noop_mgmt*), 1372
NoopMetricLogger (*class in ironic.common.metrics*), 982
NoopNetwork (*class in ironic.drivers.modules.network.noop*), 1254
NoopStorage (*class in ironic.drivers.modules.storage.noop*), 1288
NoOpSwitchDriver (*class in ironic.drivers.modules.switch.base*), 1289
NoRAID (*class in ironic.drivers.modules.noop*), 1370
NoRescue (*class in ironic.drivers.modules.noop*),

1371
normalize_mac() (*in module ironic.common.utils*), 1017
normalize_path() (*in module ironic.common.inspection_rules.utils*), 914
NOSTATE (*in module ironic.common.states*), 1008
NotAcceptable, 954
NotAuthorized, 954
notDetected (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1384
NotFound, 954
NotificationBase (*class in ironic.objects.notification*), 1518
NotificationLevel (*class in ironic.objects.fields*), 1471
NotificationLevelField (*class in ironic.objects.fields*), 1471
NotificationPayloadBase (*class in ironic.objects.notification*), 1518
NotificationPayloadError, 954
NotificationPublisher (*class in ironic.objects.notification*), 1519
NotificationSchemaKeyError, 954
NotificationSchemaObjectError, 954
NotificationStatus (*class in ironic.objects.fields*), 1472
NotificationStatusField (*class in ironic.objects.fields*), 1472
notify_conductor_resume_clean() (*in module ironic.conductor.utils*), 1079
notify_conductor_resume_deploy() (*in module ironic.conductor.utils*), 1079
notify_conductor_resume_operation() (*in module ironic.conductor.utils*), 1079
notify_conductor_resume_service() (*in module ironic.conductor.utils*), 1079
NotImplementedAction (*class in ironic.common.trait_based_networking.base*), 921
notSet (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry4 attribute*), 1386
NoValidDefaultForInterface, 952
NoValidHost, 953
NoVendor (*class in ironic.drivers.modules.noop*), 1372
novnc_authorize() (*in module ironic.common.vnc*), 1020
novnc_unauthorize() (*in module ironic.common.vnc*), 1020
novnc_validate() (*in module ironic.common.vnc*), 1021
NoVNCProxyService (*class in ironic.console.novncproxy_service*), 1095

O

obj (*ironic.api.controllers.v1.utils.PassthruResponse attribute*), 867
OBJ_BASE_CLASS (*ironic.objects.base.IronicObjectSerializer attribute*), 1451
OBJ_PROJECT_NAMESPACE (*ironic.objects.base.IronicObject attribute*), 1449
obj_refresh() (*ironic.objects.base.IronicObject method*), 1450
OBJ_SERIAL_NAMESPACE (*ironic.objects.base.IronicObject attribute*), 1449
object (*ironic.objects.notification.EventType property*), 1517
object_action() (*ironic.conductor.manager.ConductorManager method*), 1033
object_action() (*ironic.conductor.rpcapi.ConductorAPI method*), 1057
object_action() (*ironic.objects.indirection.IronicObjectIndirectionAPI method*), 1475
object_backport_versions() (*ironic.conductor.manager.ConductorManager method*), 1034
object_backport_versions() (*ironic.conductor.rpcapi.ConductorAPI method*), 1057
object_backport_versions() (*ironic.objects.indirection.IronicObjectIndirectionAPI method*), 1475
object_class_action_versions() (*ironic.conductor.manager.ConductorManager method*), 1034
object_class_action_versions() (*ironic.conductor.rpcapi.ConductorAPI method*), 1057
object_class_action_versions() (*ironic.objects.indirection.IronicObjectIndirectionAPI method*), 1476
object_name() (*ironic.common.trait_based_networking.base.Var method*), 924
object_to_dict() (*in module*

ironic.api.controllers.v1.utils), 884
ObjectField (*class in ironic.objects.fields*), 1472
objects (*ironic.objects.bios.BIOSSettingList property*), 1455
objects (*ironic.objects.firmware.FirmwareComponentList property*), 1475
objects (*ironic.objects.trait.TraitList property*), 1542
OciClient (*class in ironic.common.oci_registry*), 992
OciImageNotSpecific, 954
OciImageService (*class in ironic.common.image_service*), 968
OciImageTagNotFound, 954
ocpOff (*ironic.drivers.modules.snmp.SNMPDriverServerTechStromy4 attribute*), 1386
ocpOn (*ironic.drivers.modules.snmp.SNMPDriverServerTechStromy4 attribute*), 1386
Octal (*class in ironic.conf.api*), 1084
OFF (*in module ironic.common.indicator_states*), 976
off (*ironic.drivers.modules.snmp.SNMPDriverVertivGeistPDU attribute*), 1387
off2on (*ironic.drivers.modules.snmp.SNMPDriverVertivGeistPDU attribute*), 1387
oid_device (*ironic.drivers.modules.snmp.SNMPDriverAPCMasterSwitch attribute*), 1381
oid_device (*ironic.drivers.modules.snmp.SNMPDriverAPCMasterSwitchPlus attribute*), 1381
oid_device (*ironic.drivers.modules.snmp.SNMPDriverAPCRackPDU attribute*), 1381
oid_device (*ironic.drivers.modules.snmp.SNMPDriverAstute attribute*), 1381
oid_device (*ironic.drivers.modules.snmp.SNMPDriverBaytechMRP27 attribute*), 1383
oid_device (*ironic.drivers.modules.snmp.SNMPDriverCyberPower attribute*), 1383
oid_device (*ironic.drivers.modules.snmp.SNMPDriverEatonPower attribute*), 1383
oid_device (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1384
oid_device (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry3 attribute*), 1385
oid_device (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry4 attribute*), 1386
oid_device (*ironic.drivers.modules.snmp.SNMPDriverSimple property*), 1387
oid_device (*ironic.drivers.modules.snmp.SNMPDriverTeltronix attribute*), 1387
oid_device (*ironic.drivers.modules.snmp.SNMPDriverVertivGeistPDU attribute*), 1387
oid_enterprise (*ironic.drivers.modules.snmp.SNMPDriverBase attribute*), 1382
oid_power_action (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1384
oid_power_action (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry3 attribute*), 1385
oid_power_action (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry4 attribute*), 1386
oid_power_action (*ironic.drivers.modules.snmp.SNMPDriverVertivGeistPDU attribute*), 1387
oid_power_status (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1384
oid_power_status (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry3 attribute*), 1385
oid_power_status (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry4 attribute*), 1386
oid_power_status (*ironic.drivers.modules.snmp.SNMPDriverVertivGeistPDU attribute*), 1387
oid_power_status (*ironic.drivers.modules.snmp.SNMPDriverEatonPower attribute*), 1383
oid_power_status (*ironic.drivers.modules.snmp.SNMPDriverEatonPowerMIB attribute*), 1383
oid_power_status (*ironic.drivers.modules.snmp.SNMPDriverEatonPowerMIB attribute*), 1383
oid_tower_infeed_idx (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1384
oid_tower_infeed_idx (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry3 attribute*), 1385
oid_tower_infeed_idx (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry4 attribute*), 1386
oid_tower_infeed_idx (*ironic.drivers.modules.snmp.SNMPDriverVertivGeistPDU attribute*), 1387
OK (*ironic.common.health_states.HealthState attribute*), 960
ok (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1384
ON (*in module ironic.common.indicator_states*), 976

on (*ironic.drivers.modules.snmp.SNMPDriverVertivGeist4PDU attribute*), 1387
on2off (*ironic.drivers.modules.snmp.SNMPDriverVertivGeist4PDU attribute*), 1387
on_enter() (*in module ironic.common.state_machine*), 1005
on_exit() (*in module ironic.common.state_machine*), 1006
one (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1384
one_button_secure_erase() (*ironic.drivers.modules.ilo.management.Ilo5Management method*), 1215
OneOfOperator (*class in ironic.common.inspection_rules.operators*), 913
online (*ironic.db.sqlalchemy.models.Conductor attribute*), 1136
online (*ironic.objects.conductor.Conductor property*), 1461
online_data_migrations ironic-dbsync command line option, 456
online_data_migrations command line option
--help, 457
--max-count, 457
--option, 457
-h, 457
online_data_migrations() (*ironic.command.dbsync.DBCommand method*), 903
op (*ironic.common.inspection_rules.operators.SimpleOperator attribute*), 913
op() (*ironic.common.inspection_rules.operators.EqOperator method*), 911
op() (*ironic.common.inspection_rules.operators.GeOperator method*), 911
op() (*ironic.common.inspection_rules.operators.GtOperator method*), 912
op() (*ironic.common.inspection_rules.operators.LeOperator method*), 912
op() (*ironic.common.inspection_rules.operators.LtOperator method*), 912
op() (*ironic.common.inspection_rules.operators.NeOperator method*), 912
op_and() (*ironic.common.trait_based_networking.grammar.rule_parser.RuleExpressionTransformer method*), 917
op_or() (*ironic.common.trait_based_networking.grammar.rule_parser.RuleExpressionTransformer method*), 917
OperationNotPermitted, 954
Operator (*class in ironic.common.trait_based_networking.base*), 921
OperatorBase (*class in ironic.common.inspection_rules.operators*), 913
optional_interfaces (*ironic.drivers.base.BareDriver property*), 1392
OPTIONAL_PARAMS (*ironic.common.inspection_rules.actions.CallAPIHookAction attribute*), 908
OR (*ironic.common.trait_based_networking.base.Operator attribute*), 921
or_valid() (*in module ironic.common.args*), 928
order (*ironic.db.sqlalchemy.models.RunbookStep attribute*), 1147
order_traits() (*in module ironic.common.trait_based_networking.plan*), 926
outOfSync (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1384
override_api_url() (*in module ironic.drivers.modules.image_utils*), 1353
owner (*ironic.db.sqlalchemy.models.Allocation attribute*), 1135
owner (*ironic.db.sqlalchemy.models.Node attribute*), 1140
owner (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
owner (*ironic.db.sqlalchemy.models.Runbook attribute*), 1146
owner (*ironic.objects.allocation.Allocation property*), 1446
owner (*ironic.objects.allocation.AllocationCRUDPayload property*), 1448
owner (*ironic.objects.node.Node property*), 1486
owner (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
owner (*ironic.objects.node.NodeCRUDPayload property*), 1493
owner (*ironic.objects.node.NodePayload property*), 1503
owner (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
owner (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
owner (*ironic.objects.runbook.Runbook property*), 1537
owner (*ironic.objects.runbook.RunbookCRUDPayload*

property), 1540

P

paren_expression() (*ironic.common.trait_based_networking.grammar.parser.FilterExpressionTransformer method*), 917
ParenExpression (*class in ironic.common.trait_based_networking.base*), 921
parent_node (*ironic.api.controllers.v1.node.NodesController attribute*), 856
parent_node (*ironic.db.sqlalchemy.models.Node attribute*), 1140
parent_node (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
parent_node (*ironic.objects.node.Node property*), 1486
ParentNodeLocked, 954
ParsableErrorMiddleware (*class in ironic.api.middleware*), 896
ParsableErrorMiddleware (*class in ironic.api.middleware.parsable_error*), 895
parse() (*ironic.common.trait_based_networking.base.FilterExpression class method*), 919
parse() (*ironic.common.trait_based_networking.config.GenericConfigFile method*), 924
parse_args() (*in module ironic.common.config*), 939
parse_driver_info() (*in module ironic.common.pxe_utils*), 1000
parse_driver_info() (*in module ironic.drivers.modules.ilo.boot*), 1205
parse_driver_info() (*in module ironic.drivers.modules.ilo.common*), 1209
parse_driver_info() (*in module ironic.drivers.modules.redfish.utils*), 1283
parse_entry() (*in module ironic.common.auth_basic*), 932
parse_header() (*in module ironic.common.auth_basic*), 932
parse_headers() (*ironic.api.controllers.base.Version static method*), 893
parse_image_id() (*in module ironic.common.glance_service.service_utils*), 907
parse_instance_info() (*in module ironic.drivers.modules.deploy_utils*), 1328
parse_instance_info_capabilities() (*in module ironic.common.utils*), 1017
parse_inverted_operator() (*in module ironic.common.inspection_rules.utils*), 914
parse_kernel_params() (*in module ironic.common.utils*), 1017
parse_root_device_hints() (*in module ironic.common.utils*), 1018
parse_sleep_range() (*in module ironic.drivers.modules.fake*), 1350
parse_tlv() (*ironic.drivers.modules.inspector.lldp_parsers.LLDP method*), 1236
parse_token() (*in module ironic.common.auth_basic*), 932
parse_vlan_ranges() (*in module ironic.networking.utils*), 1443
parse_www_authenticate() (*ironic.common.oci_registry.RegistrySessionHelper static method*), 995
ParseError, 916
ParseLLDPHook (*class in ironic.drivers.modules.inspector.hooks.parse_lldp*), 1229
passthru() (*in module ironic.drivers.base*), 1423
PassthruResponse (*class in ironic.api.controllers.v1.utils*), 867
PasswordFileFailedToCreate, 954
patch() (*in module ironic.common.args*), 929
patch() (*ironic.api.controllers.v1.allocation.AllocationsController method*), 835
patch() (*ironic.api.controllers.v1.chassis.ChassisController method*), 837
patch() (*ironic.api.controllers.v1.deploy_template.DeployTemplat method*), 840
patch() (*ironic.api.controllers.v1.inspection_rule.InspectionRuleC method*), 844
patch() (*ironic.api.controllers.v1.node.NodesController method*), 856
patch() (*ironic.api.controllers.v1.port.PortsController method*), 861
patch() (*ironic.api.controllers.v1.portgroup.PortgroupsController method*), 863
patch() (*ironic.api.controllers.v1.runbook.RunbooksController method*), 866
patch() (*ironic.api.controllers.v1.volume_connector.VolumeConn method*), 888
patch() (*ironic.api.controllers.v1.volume_target.VolumeTargetsCo method*), 891
patch_update_changed_fields() (*in module*

ironic.api.controllers.v1.utils), 884
patch_validate_allowed_fields() (*in module ironic.api.controllers.v1.utils*), 885
patched_validate_with_schema() (*in module ironic.api.controllers.v1.utils*), 885
PatchError, 954
PathNotFound, 954
payload (*ironic.objects.allocation.AllocationCRUDNotification property*), 1447
payload (*ironic.objects.chassis.ChassisCRUDNotification property*), 1459
payload (*ironic.objects.deploy_template.DeployTemplateCRUDNotification property*), 1466
payload (*ironic.objects.inspection_rule.InspectionRuleCRUDNotification property*), 1479
payload (*ironic.objects.node.NodeConsoleNotification property*), 1494
payload (*ironic.objects.node.NodeCorrectedPowerStateNotification property*), 1495
payload (*ironic.objects.node.NodeCRUDNotification property*), 1489
payload (*ironic.objects.node.NodeMaintenanceNotification property*), 1499
payload (*ironic.objects.node.NodeSetPowerStateNotification property*), 1504
payload (*ironic.objects.node.NodeSetProvisionStateNotification property*), 1508
payload (*ironic.objects.port.PortCRUDNotification property*), 1526
payload (*ironic.objects.portgroup.PortgroupCRUDNotification property*), 1533
payload (*ironic.objects.runbook.RunbookCRUDNotification property*), 1539
payload (*ironic.objects.volume_connector.VolumeConnectorCRUDNotification property*), 1546
payload (*ironic.objects.volume_target.VolumeTargetCRUDNotification property*), 1552
PciDevicesHook (*class in ironic.drivers.modules.inspector.hooks.pci_devices*), 1230
periodic() (*in module ironic.conductor.periodics*), 1038
phase (*ironic.db.sqlalchemy.models.InspectionRule attribute*), 1138
phase (*ironic.objects.inspection_rule.InspectionRule property*), 1478
phase (*ironic.objects.inspection_rule.InspectionRuleCRUDPayload property*), 1480
physical_network (*ironic.common.trait_based_networking.base.PlannedPort attribute*), 922
physical_network (*ironic.db.sqlalchemy.models.Port attribute*), 1145
physical_network (*ironic.db.sqlalchemy.models.Portgroup attribute*), 1146
physical_network (*ironic.objects.port.Port property*), 1525
physical_network (*ironic.objects.port.PortCRUDPayload property*), 1528
physical_network (*ironic.objects.portgroup.Portgroup property*), 1532
physical_network (*ironic.objects.portgroup.PortgroupCRUDPayload property*), 1534
PhysicalNetworkHook (*class in ironic.drivers.modules.inspector.hooks.physical_network*), 1230
PHYSNET_PARAM_NAME (*in module ironic.common.neutron*), 986
place_common_config() (*in module ironic.common.pxe_utils*), 1001
place_loaders_for_boot() (*in module ironic.common.pxe_utils*), 1001
plan_network() (*in module ironic.common.trait_based_networking.plan*), 926
plan_vif_attach() (*in module ironic.common.trait_based_networking.plan*), 926
PlaybookNotFound, 1191
plug_network() (*in module ironic.drivers.modules.network.common*), 1202
plugin_data (*ironic.db.sqlalchemy.models.NodeInventory attribute*), 1144
plugin_data (*ironic.objects.node_inventory.NodeInventory property*), 1517
policy_deprecation_check() (*in module ironic.api.hooks*), 902
pop_node_nested_field() (*in module ironic.common.utils*), 1018
populate_node_uuid() (*in module ironic.api.controllers.v1.utils*), 885
populate_schema() (*ironic.objects.notification.NotificationPayloadBase method*), 1519
populate_storage_driver_internal_info() (*in module*

ironic.drivers.modules.deploy_utils), 1329
Port (*class in ironic.common.trait_based_networking.base*), 921
Port (*class in ironic.db.sqlalchemy.models*), 1145
Port (*class in ironic.objects.port*), 1519
PORT_ADDRESS (*ironic.common.trait_based_networking.base.Variables attribute*), 924
PORT_CATEGORY (*ironic.common.trait_based_networking.base.Variables attribute*), 924
port_changed() (*ironic.drivers.base.NetworkInterface method*), 1412
port_changed() (*ironic.drivers.modules.network.common.NeutronVIFPortIDMixin method*), 1239
port_changed() (*ironic.drivers.modules.network.ironic_networking.IronicNetworking method*), 1248
port_changed() (*ironic.drivers.modules.network.noop.NoopNetwork method*), 1254
PORT_IS_PORT (*ironic.common.trait_based_networking.base.Variables attribute*), 924
port_is_port() (*ironic.common.trait_based_networking.grammar.PortgroupFilterExpressionTransformer method*), 917
PORT_IS_PORTGROUP (*ironic.common.trait_based_networking.base.Variables attribute*), 924
port_is_portgroup() (*ironic.common.trait_based_networking.grammar.PortgroupFilterExpressionTransformer method*), 917
PORT_PHYSICAL_NETWORK (*ironic.common.trait_based_networking.base.Variables attribute*), 924
port_sanitize() (*in module ironic.api.controllers.v1.port*), 862
port_uuid (*ironic.common.trait_based_networking.base.AttachPortgroupAction attribute*), 918
port_uuids (*ironic.common.trait_based_networking.base.AttachPortgroupAction attribute*), 920
PORT_VENDOR (*ironic.common.trait_based_networking.base.Variables attribute*), 924
PortAlreadyExists, 954
PortCRUDNotification (*class in ironic.objects.port*), 1526
PortCRUDPayload (*class in ironic.objects.port*), 1526
PortDuplicateName, 955
Portgroup (*class in ironic.common.trait_based_networking.base*), 922
Portgroup (*class in ironic.db.sqlalchemy.models*), 1145
Portgroup (*class in ironic.objects.portgroup*), 1528
portgroup_changed() (*ironic.drivers.base.NetworkInterface method*), 1412
portgroup_changed() (*ironic.drivers.modules.network.common.NeutronVIFPortIDMixin method*), 1240
portgroup_changed() (*ironic.drivers.modules.network.ironic_networking.IronicNetworking method*), 1248
portgroup_changed() (*ironic.drivers.modules.network.noop.NoopNetwork method*), 1255
portgroup_id (*ironic.db.sqlalchemy.models.Port attribute*), 1145
portgroup_id (*ironic.objects.port.Port property*), 1525
portgroup_sanitize() (*in module ironic.api.controllers.v1.portgroup*), 864
PortgroupFilterExpressionTransformer (*ironic.common.trait_based_networking.base.AttachPortgroupAction attribute*), 918
portgroup_uuid (*ironic.objects.port.PortCRUDPayload property*), 1528
PortgroupAlreadyExists, 955
PortgroupCRUDNotification (*class in ironic.objects.portgroup*), 1533
PortgroupCRUDPayload (*class in ironic.objects.portgroup*), 1533
PortgroupDuplicateName, 955
PortgroupMACAlreadyExists, 955
PortgroupPortEmpty, 955
PortgroupNotFound, 955
PortgroupPhysnetInconsistent, 955
portgroups (*ironic.conductor.task_manager.TaskManager property*), 1070
PortgroupsController (*class in ironic.api.controllers.v1.portgroup*), 862
portlike_uuid() (*ironic.common.trait_based_networking.base.AttachAction method*), 918
portlike_uuid() (*ironic.common.trait_based_networking.base.AttachPort*

method), 918
portlike_uuid() (*ironic.common.trait_based_networking.base.AttachPortgroup method*), 918
PortNotFound, 955
PortRange (*class in ironic.conf.console*), 1085
ports (*ironic.conductor.task_manager.TaskManager property*), 1070
PortsController (*class in ironic.api.controllers.v1.port*), 859
PortsHook (*class in ironic.drivers.modules.inspector.hooks.ports*), 1230
post() (*ironic.api.controllers.v1.allocation.AllocationsController method*), 835
post() (*ironic.api.controllers.v1.chassis.ChassisController method*), 837
post() (*ironic.api.controllers.v1.deploy_template.DeployTemplatesController method*), 840
post() (*ironic.api.controllers.v1.event.EventsController method*), 843
post() (*ironic.api.controllers.v1.inspection_rule.InspectionRulesController method*), 845
post() (*ironic.api.controllers.v1.node.NodesController method*), 856
post() (*ironic.api.controllers.v1.node.NodeVIFController method*), 852
post() (*ironic.api.controllers.v1.node.NodeVmediaController method*), 853
post() (*ironic.api.controllers.v1.port.PortsController method*), 861
post() (*ironic.api.controllers.v1.portgroup.PortgroupsController method*), 863
post() (*ironic.api.controllers.v1.ramdisk.ContinueInspectionController method*), 864
post() (*ironic.api.controllers.v1.ramdisk.HeartbeatController method*), 864
post() (*ironic.api.controllers.v1.runbook.RunbooksController method*), 866
post() (*ironic.api.controllers.v1.volume_connector.VolumeConnectorsController method*), 889
post() (*ironic.api.controllers.v1.volume_target.VolumeTargetsController method*), 892
post_clean_step_hook() (*in module ironic.drivers.modules.agent_base*), 1309
post_configuration() (*ironic.drivers.modules.redfish.bios.RedfishBIOS method*), 1258
post_create_configuration() (*ironic.drivers.modules.redfish.raid.RedfishRAID method*), 1278
post_delete_configuration() (*ironic.drivers.modules.drac.raid.DracRedfishRAID method*), 1196
post_delete_configuration() (*ironic.drivers.modules.redfish.raid.RedfishRAID method*), 1279
post_deploy_step_hook() (*in module ironic.drivers.modules.agent_base*), 1309
POST_FINISHEDPOST_STATE (*in module ironic.drivers.modules.ilo.common*), 1205
POST_INPOST_STATE (*in module ironic.drivers.modules.ilo.common*), 1205
POST_INPOSTDISCOVERY_STATE (*in module ironic.drivers.modules.ilo.common*), 1205
POST_NULL_STATE (*in module ironic.drivers.modules.ilo.common*), 1206
POST_POWEROFF_STATE (*in module ironic.drivers.modules.ilo.common*), 1206
power_reset() (*ironic.drivers.modules.redfish.bios.RedfishBIOS method*), 1258
POST_RESET_STATE (*in module ironic.drivers.modules.ilo.common*), 1206
POST_UNKNOWN_STATE (*in module ironic.drivers.modules.ilo.common*), 1206
POWER (*in module ironic.common.components*), 939
power (*ironic.drivers.base.BareDriver attribute*), 1392
power() (*ironic.api.controllers.v1.node.NodeStatesController method*), 849
POWER_FAILURE (*in module ironic.common.faults*), 958
power_interface (*ironic.db.sqlalchemy.models.Node attribute*), 1140
power_interface (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
power_interface (*ironic.objects.node.Node property*), 1486
power_interface (*ironic.objects.node.NodeCorrectedPowerStatePayload*

property), 1498
power_interface (*ironic.objects.node.NodeCRUDPayload property*), 1493
power_interface (*ironic.objects.node.NodePayload property*), 1503
power_interface (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
power_interface (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
POWER_OFF (*in module ironic.common.states*), 1008
power_off() (*ironic.drivers.modules.agent_client.AgentClient method*), 1316
power_off() (*ironic.drivers.modules.snmp.SNMPDriverBase method*), 1382
power_off_and_on() (*in module ironic.drivers.utils*), 1433
POWER_ON (*in module ironic.common.states*), 1008
power_on() (*ironic.drivers.modules.snmp.SNMPDriverBase method*), 1382
power_on_node_if_needed() (*in module ironic.conductor.utils*), 1079
power_reset() (*ironic.drivers.modules.snmp.SNMPDriverBase method*), 1382
power_state (*ironic.db.sqlalchemy.models.Node attribute*), 1140
power_state (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
power_state (*ironic.objects.node.Node property*), 1486
power_state (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
power_state (*ironic.objects.node.NodeCRUDPayload property*), 1493
power_state (*ironic.objects.node.NodePayload property*), 1503
power_state (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
power_state (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
power_state() (*ironic.drivers.modules.snmp.SNMPDriverBase method*), 1382
power_state_error_handler() (*in module ironic.conductor.utils*), 1079
power_state_for_network_configuration() (*in module ironic.conductor.utils*), 1080
power_update() (*in module ironic.common.nova*), 992
PowerInterface (*class in ironic.drivers.base*), 1414
PowerStateFailure, 955
pre_create_configuration() (*ironic.drivers.modules.drac.raid.DracRedfishRAID method*), 1196
pre_create_configuration() (*ironic.drivers.modules.redfish.raid.RedfishRAID method*), 1279
pre_delete_configuration() (*ironic.drivers.modules.redfish.raid.RedfishRAID method*), 1279
PREFIX_MATCH (*ironic.common.trait_based_networking.base.Comp attribute*), 919
prefix_match() (*ironic.common.trait_based_networking.grammar.parser.I method*), 917
prepare() (*ironic.common.json_rpc.client.Client method*), 915
prepare() (*ironic.drivers.base.DeployInterface method*), 1398
prepare() (*ironic.drivers.modules.agent.CustomAgentDeploy method*), 1302
prepare() (*ironic.drivers.modules.ansible.deploy.AnsibleDeploy method*), 1190
prepare() (*ironic.drivers.modules.autodetect.AutodetectDeploy method*), 1320
prepare() (*ironic.drivers.modules.fake.FakeDeploy method*), 1337
prepare() (*ironic.drivers.modules.noop.NoDeploy method*), 1368
prepare() (*ironic.drivers.modules.pxe.PXEAnacondaDeploy method*), 1375
prepare() (*ironic.drivers.modules.ramdisk.RamdiskDeploy method*), 1379
prepare_agent_boot() (*in module ironic.drivers.modules.deploy_utils*), 1329
prepare_boot_iso() (*in module ironic.drivers.modules.image_utils*), 1353
prepare_cleaning() (*ironic.drivers.base.DeployInterface method*), 1398
prepare_cleaning() (*ironic.drivers.modules.agent_base.AgentBaseMixin method*), 1304
prepare_cleaning() (*ironic.drivers.modules.ansible.deploy.AnsibleDeploy method*), 1190

prepare_command() (*in module ironic.common.service*), 1005
prepare_config_drive() (*in module ironic.common.kickstart_utils*), 978
prepare_configdrive_image() (*in module ironic.drivers.modules.image_utils*), 1353
prepare_deploy_iso() (*in module ironic.drivers.modules.image_utils*), 1354
prepare_disk_image() (*in module ironic.drivers.modules.image_utils*), 1354
prepare_floppy_image() (*in module ironic.drivers.modules.image_utils*), 1355
prepare_host() (*ironic.conductor.base_manager.BaseConductorManager method*), 1024
prepare_host() (*ironic.networking.manager.NetworkingManager method*), 1438
prepare_host() (*ironic.pxe_filter.service.PXEFilterManager method*), 1554
prepare_inband_cleaning() (*in module ironic.drivers.modules.deploy_utils*), 1329
prepare_inband_service() (*in module ironic.drivers.modules.deploy_utils*), 1329
prepare_instance() (*ironic.drivers.base.BootInterface method*), 1396
prepare_instance() (*ironic.drivers.modules.fake.FakeBoot method*), 1335
prepare_instance() (*ironic.drivers.modules.ilo.boot.IloiPXEBoot method*), 1204
prepare_instance() (*ironic.drivers.modules.ilo.boot.IloPXEBoot method*), 1199
prepare_instance() (*ironic.drivers.modules.ilo.boot.IloUefiHttpsBoot method*), 1200
prepare_instance() (*ironic.drivers.modules.ilo.boot.IloVirtualMediaBoot method*), 1202
prepare_instance() (*ironic.drivers.modules.pxe_base.PXEBaseMixin method*), 1377
prepare_instance() (*ironic.drivers.modules.redfish.boot.RedfishHttpsBoot method*), 1259
prepare_instance() (*ironic.drivers.modules.redfish.boot.RedfishVirtualMediaB method*), 1262
prepare_instance_boot() (*ironic.drivers.modules.agent.AgentDeploy method*), 1296
prepare_instance_boot() (*ironic.drivers.modules.agent.CustomAgentDeploy method*), 1302
prepare_instance_kickstart_config() (*in module ironic.common.pxe_utils*), 1001
prepare_instance_pxe_config() (*in module ironic.common.pxe_utils*), 1001
prepare_instance_to_boot() (*ironic.drivers.modules.agent.AgentDeploy method*), 1296
prepare_managed_inspection() (*in module ironic.drivers.modules.inspector.interface*), 1235
prepare_node_for_deploy() (*in module ironic.drivers.modules.ilo.boot*), 1205
prepare_node_for_next_step() (*in module ironic.common.async_steps*), 930
prepare_ramdisk() (*ironic.drivers.base.BootInterface method*), 1396
prepare_ramdisk() (*ironic.drivers.modules.fake.FakeBoot method*), 1335
prepare_ramdisk() (*ironic.drivers.modules.ilo.boot.IloiPXEBoot method*), 1204
prepare_ramdisk() (*ironic.drivers.modules.ilo.boot.IloPXEBoot method*), 1199
prepare_ramdisk() (*ironic.drivers.modules.ilo.boot.IloUefiHttpsBoot method*), 1200
prepare_ramdisk() (*ironic.drivers.modules.ilo.boot.IloVirtualMediaBoot method*), 1203
prepare_ramdisk() (*ironic.drivers.modules.pxe_base.PXEBaseMixin method*), 1377
prepare_ramdisk() (*ironic.drivers.modules.redfish.boot.RedfishHttpsBoot method*), 1260

prepare_ramdisk() (*ironic.drivers.modules.redfish.boot.RedfishVirtualMediaBoot method*), 1262
prepare_remote_image() (*in module ironic.drivers.modules.image_utils*), 1355
prepare_service() (*in module ironic.common.service*), 1005
prepare_service() (*ironic.drivers.base.DeployInterface method*), 1399
prepare_service() (*ironic.drivers.modules.agent_base.AgentBaseMixin method*), 1305
preprocess() (*ironic.drivers.modules.inspector.hooks.base.InspectionHook method*), 1227
preprocess() (*ironic.drivers.modules.inspector.hooks.ramdisk_error.RamdiskErrorHook method*), 1232
preprocess() (*ironic.drivers.modules.inspector.hooks.validate_interfaces.ValidateInterfacesHook method*), 1233
preprocess_config() (*ironic.networking.switch_drivers.driver_adapter.NetworkingDriverAdapter method*), 1434
previous_provision_state (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
previous_target_provision_state (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
PrimordialPort (*class in ironic.common.trait_based_networking.base*), 922
priority (*ironic.db.sqlalchemy.models.DeployTemplateStep attribute*), 1137
priority (*ironic.db.sqlalchemy.models.InspectionRule attribute*), 1138
priority (*ironic.objects.inspection_rule.InspectionRule property*), 1478
priority (*ironic.objects.inspection_rule.InspectionRuleCRUDPayload property*), 1480
process_event() (*ironic.common.fsm.FSM method*), 959
process_event() (*ironic.conductor.task_manager.TaskManager method*), 1070
process_fw_on() (*ironic.drivers.modules.ilo.firmware_processor.FirmwareProcessor method*), 1213
process_launcher() (*in module ironic.common.service*), 1005
process_next_step() (*ironic.drivers.modules.agent_base.AgentBaseMixin method*), 1305
process_next_step() (*ironic.drivers.modules.agent_base.HeartbeatMixin method*), 1307
process_next_step() (*ironic.drivers.modules.ansible.deploy.AnsibleDeploy method*), 1190
properties (*ironic.db.sqlalchemy.models.Node attribute*), 1140
properties (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
properties (*ironic.db.sqlalchemy.models.Portgroup attribute*), 1146
properties (*ironic.db.sqlalchemy.models.VolumeTarget attribute*), 1147
properties (*ironic.objects.node.Node property*), 1486
properties (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
properties (*ironic.objects.node.NodeCRUDPayload property*), 1493
properties (*ironic.objects.node.NodePayload property*), 1503
properties (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
properties (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
properties (*ironic.objects.portgroup.Portgroup property*), 1532
properties (*ironic.objects.portgroup.PortgroupCRUDPayload property*), 1535
properties (*ironic.objects.volume_target.VolumeTarget property*), 1551
properties (*ironic.objects.volume_target.VolumeTargetCRUDPayload property*), 1553
properties() (*ironic.api.controllers.v1.driver.DriversController method*), 842
protected (*ironic.db.sqlalchemy.models.Node attribute*), 1140
protected (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1142
protected (*ironic.objects.node.Node property*), 1486
protected (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
protected (*ironic.objects.node.NodeCRUDPayload property*), 1493
protected (*ironic.objects.node.NodePayload property*), 1503
protected (*ironic.objects.node.NodeSetPowerStatePayload*

property), 1507
protected (ironic.objects.node.NodeSetProvisionStatePayload property), 1512
protected_reason (ironic.db.sqlalchemy.models.Node attribute), 1140
protected_reason (ironic.db.sqlalchemy.models.NodeBase attribute), 1142
protected_reason (ironic.objects.node.Node property), 1486
protected_reason (ironic.objects.node.NodeCorrectedPowerStatePayload property), 1498
protected_reason (ironic.objects.node.NodeCRUDPayload property), 1493
protected_reason (ironic.objects.node.NodePayload property), 1503
protected_reason (ironic.objects.node.NodeSetPowerStatePayload property), 1507
protected_reason (ironic.objects.node.NodeSetProvisionStatePayload property), 1512
provider (ironic.common.console_factory.ConsoleContainerFactory property), 939
provider (ironic.common.dhcp_factory.DHCPFactory property), 940
provision() (ironic.api.controllers.v1.node.NodeStatesController method), 849
provision_state (ironic.db.sqlalchemy.models.Node attribute), 1140
provision_state (ironic.db.sqlalchemy.models.NodeBase attribute), 1142
provision_state (ironic.objects.node.Node property), 1486
provision_state (ironic.objects.node.NodeCorrectedPowerStatePayload property), 1498
provision_state (ironic.objects.node.NodeCRUDPayload property), 1493
provision_state (ironic.objects.node.NodePayload property), 1503
provision_state (ironic.objects.node.NodeSetPowerStatePayload property), 1507
provision_state (ironic.objects.node.NodeSetProvisionStatePayload property), 1512
provision_updated_at (ironic.db.sqlalchemy.models.Node attribute), 1140
provision_updated_at (ironic.db.sqlalchemy.models.NodeBase attribute), 1143
provision_updated_at (ironic.objects.node.Node property), 1486
provision_updated_at (ironic.objects.node.NodeCorrectedPowerStatePayload property), 1498
provision_updated_at (ironic.objects.node.NodeCRUDPayload property), 1493
provision_updated_at (ironic.objects.node.NodePayload property), 1503
provision_updated_at (ironic.objects.node.NodeSetPowerStatePayload property), 1507
provision_updated_at (ironic.objects.node.NodeSetProvisionStatePayload property), 1512
PROVISIONING (ironic.drivers.modules.network.common.NetType attribute), 1239
provisioning_error_handler() (in module ironic.conductor.utils), 1080
public (ironic.db.sqlalchemy.models.Runbook attribute), 1146
public (ironic.objects.runbook.Runbook property), 1537
public (ironic.objects.runbook.RunbookCRUDPayload property), 1540
PublicUrlHook (class in ironic.api.hooks), 902
publish() (ironic.common.image_publisher.AbstractPublisher method), 960
publish() (ironic.common.image_publisher.CIFSPublisher method), 961
publish() (ironic.common.image_publisher.LocalPublisher method), 961
publish() (ironic.common.image_publisher.NFSPublisher method), 962
publish() (ironic.common.image_publisher.SwiftPublisher method), 962
publish_image() (ironic.drivers.modules.image_utils.ImageHandler

method), 1352
publisher (*ironic.objects.allocation.AllocationCRUDNotification property*), 1447
publisher (*ironic.objects.chassis.ChassisCRUDNotification property*), 1459
publisher (*ironic.objects.deploy_template.DeployTemplateCRUDNotification property*), 1466
publisher (*ironic.objects.inspection_rule.InspectionRuleCRUDNotification property*), 1479
publisher (*ironic.objects.node.NodeConsoleNotification property*), 1494
publisher (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1495
publisher (*ironic.objects.node.NodeCRUDNotification property*), 1489
publisher (*ironic.objects.node.NodeMaintenanceNotification property*), 1499
publisher (*ironic.objects.node.NodeSetPowerStateNotification property*), 1504
publisher (*ironic.objects.node.NodeSetProvisionStateNotification property*), 1508
publisher (*ironic.objects.notification.NotificationBase property*), 1518
publisher (*ironic.objects.port.PortCRUDNotification property*), 1526
publisher (*ironic.objects.portgroup.PortgroupCRUDNotification property*), 1533
publisher (*ironic.objects.runbook.RunbookCRUDNotification property*), 1539
publisher (*ironic.objects.volume_connector.VolumeConnectorCRUDNotification property*), 1546
publisher (*ironic.objects.volume_target.VolumeTargetCRUDNotification property*), 1552
put() (*ironic.api.controllers.v1.node.BootDeviceController method*), 845
put() (*ironic.api.controllers.v1.node.IndicatorController method*), 846
put() (*ironic.api.controllers.v1.node.InjectNmiController method*), 847
put() (*ironic.api.controllers.v1.node.NodeConsoleController method*), 847
put() (*ironic.api.controllers.v1.node.NodeMaintenanceController method*), 848
put() (*ironic.api.controllers.v1.node.NodeTraitsController method*), 852
PXE (*in module ironic.common.boot_devices*), 934
pxe_enabled (*ironic.db.sqlalchemy.models.Port attribute*), 1145
pxe_enabled (*ironic.objects.port.Port property*), 1525
pxe_enabled (*ironic.objects.port.PortCRUDPayload property*), 1528
PXEAnacondaDeploy (*class in ironic.drivers.modules.pxe*), 1375
PXEBaseMixin (*class in ironic.drivers.modules.pxe_base*), 1376
PXEBootCRUDNotification (*class in ironic.drivers.modules.pxe*), 1376
PXEFilterManager (*class in ironic.pxe_filter.service*), 1554

Q

query_node_history_records_for_purge() (*ironic.db.api.Connection method*), 1175
query_node_history_records_for_purge() (*ironic.db.sqlalchemy.api.Connection method*), 1124

R

RA2 (*ironic.console.rfb.auth.AuthType attribute*), 1093
RA2NE (*ironic.console.rfb.auth.AuthType attribute*), 1093
raid (*ironic.api.controllers.v1.driver.DriversController attribute*), 842
raid (*ironic.drivers.base.BareDriver attribute*), 1392
raid() (*ironic.api.controllers.v1.node.NodeStatesController method*), 851
RAID_APPLY_CONFIGURATION_ARGSINFO (*in module ironic.drivers.base*), 1418
raid_config (*ironic.db.sqlalchemy.models.Node attribute*), 1140
raid_config (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1143
raid_config (*ironic.objects.node.Node property*), 1486
raid_interface (*ironic.db.sqlalchemy.models.Node attribute*), 1140
raid_interface (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1143
raid_interface (*ironic.objects.node.Node property*), 1486
raid_interface (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
raid_interface (*ironic.objects.node.NodeCRUDPayload property*), 1493
raid_interface

(*ironic.objects.node.NodePayload property*), 1503
raid_interface (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
raid_interface (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
RaidDeviceHook (*class in ironic.drivers.modules.inspector.hooks.raid_device*), 1231
RAIDInterface (*class in ironic.drivers.base*), 1415
ramdisk_ref (*ironic.objects.deployment.Deployment property*), 1469
RamdiskDeploy (*class in ironic.drivers.modules.ramdisk*), 1378
RamdiskErrorHook (*class in ironic.drivers.modules.inspector.hooks.ramdisk_error*), 1232
read() (*ironic.common.trait_based_networking.config_fi.ConfigFile method*), 924
read_iso9600_config_drive() (*in module ironic.common.kickstart_utils*), 978
read_only (*ironic.db.sqlalchemy.models.BIOSSetting attribute*), 1135
read_only (*ironic.objects.bios.BIOSSetting property*), 1454
reason (*ironic.common.trait_based_networking.base.ValidationError attribute*), 921
reattach_networking() (*ironic.drivers.modules.network.neutron.NeutronNetwork method*), 1252
REBOOT (*in module ironic.common.states*), 1008
reboot (*ironic.drivers.modules.snmp.SNMPDriverServerTechSonicv4 attribute*), 1386
reboot() (*ironic.drivers.base.PowerInterface method*), 1415
reboot() (*ironic.drivers.modules.agent_client.AgentClient method*), 1317
reboot() (*ironic.drivers.modules.agent_power.AgentPower method*), 1318
reboot() (*ironic.drivers.modules.fake.FakePower method*), 1345
reboot() (*ironic.drivers.modules.ilo.power.IloPower method*), 1223
reboot() (*ironic.drivers.modules.ipmitool.IPMIPower method*), 1361
reboot() (*ironic.drivers.modules.redfish.power.RedfishPower method*), 1276
reboot() (*ironic.drivers.modules.snmp.SNMPPower method*), 1388
reboot_to_finish_step() (*in module ironic.drivers.modules.deploy_utils*), 1330
reboot_to_instance() (*ironic.drivers.modules.agent_base.HeartbeatMixin method*), 1308
reboot_to_instance() (*ironic.drivers.modules.pxe.PXEAnacondaDeploy method*), 1375
rebootOff (*ironic.drivers.modules.snmp.SNMPDriverVertivGeistI attribute*), 1387
rebootOn (*ironic.drivers.modules.snmp.SNMPDriverVertivGeistPI attribute*), 1387
REBUILD (*in module ironic.common.states*), 1008
recv() (*ironic.console.websocketproxy.TenantSock method*), 1096
RedfishBIOS (*class in ironic.drivers.modules.redfish.bios*), 1257
RedfishConnectionError, 955
RedfishError, 955
RedfishFirmware (*class in ironic.drivers.modules.redfish.firmware*), 1264
RedfishGraphicalConsole (*class in ironic.drivers.modules.redfish.graphical_console*), 1266
RedfishHardware (*class in ironic.drivers.redfish*), 1429
RedfishHttpsBoot (*class in ironic.drivers.modules.redfish.boot*), 1259
RedfishInspect (*class in ironic.drivers.modules.redfish.inspect*), 1267
RedfishManagement (*class in ironic.drivers.modules.redfish.management*), 1268
RedfishPower (*class in ironic.drivers.modules.redfish.power*), 1276
RedfishRAID (*class in ironic.drivers.modules.redfish.raid*), 1277
RedfishVendorPassthru (*class in ironic.drivers.modules.redfish.vendor*), 1284
RedfishVirtualMediaBoot (*class in ironic.drivers.modules.redfish.boot*), 1261

refresh() (*ironic.common.trait_based_networking.base.GenericConfigDriver method*), 925
refresh() (*ironic.objects.allocation.Allocation method*), 1446
refresh() (*ironic.objects.chassis.Chassis method*), 1458
refresh() (*ironic.objects.conductor.Conductor method*), 1461
refresh() (*ironic.objects.deploy_template.DeployTemplate method*), 1465
refresh() (*ironic.objects.deployment.Deployment method*), 1469
refresh() (*ironic.objects.inspection_rule.InspectionRule method*), 1478
refresh() (*ironic.objects.node.Node method*), 1486
refresh() (*ironic.objects.port.Port method*), 1525
refresh() (*ironic.objects.portgroup.Portgroup method*), 1532
refresh() (*ironic.objects.runbook.Runbook method*), 1538
refresh() (*ironic.objects.volume_connector.VolumeConnector method*), 1545
refresh() (*ironic.objects.volume_target.VolumeTarget method*), 1551
refresh_clean_steps() (*ironic.drivers.modules.agent_base.HeartbeatMixin method*), 1308
refresh_service_steps() (*ironic.drivers.modules.agent_base.HeartbeatMixin method*), 1308
refresh_steps() (*ironic.drivers.modules.agent_base.AgentBaseMixin method*), 1305
refresh_steps() (*ironic.drivers.modules.agent_base.HeartbeatMixin method*), 1308
regenerate_iso() (*in module ironic.conductor.configdrive_utils*), 1027
register() (*ironic.objects.conductor.Conductor class method*), 1461
register_all() (*in module ironic.objects*), 1553
register_auth_opts() (*in module ironic.common.keystone*), 978
register_auth_opts() (*in module ironic.conf.auth*), 1085
register_conductor() (*ironic.db.api.Connection method*), 1175
register_conductor() (*ironic.db.sqlalchemy.api.Connection method*), 1124
register_conductor_hardware_interfaces() (*ironic.db.api.Connection method*), 1175
register_conductor_hardware_interfaces() (*ironic.db.sqlalchemy.api.Connection method*), 1124
register_hardware_interfaces() (*ironic.objects.conductor.Conductor method*), 1462
register_opts() (*in module ironic.conf.agent*), 1084
register_opts() (*in module ironic.conf.agent_containers*), 1084
register_opts() (*in module ironic.conf.anaconda*), 1084
register_opts() (*in module ironic.conf.ansible*), 1084
register_opts() (*in module ironic.conf.api*), 1084
register_opts() (*in module ironic.conf.audit*), 1084
register_opts() (*in module ironic.conf.cifs*), 1085
register_opts() (*in module ironic.conf.cinder*), 1085
register_opts() (*in module ironic.conf.conductor*), 1085
register_opts() (*in module ironic.conf.console*), 1085
register_opts() (*in module ironic.conf.database*), 1085
register_opts() (*in module ironic.conf.default*), 1085
register_opts() (*in module ironic.conf.deploy*), 1085
register_opts() (*in module ironic.conf.dhcp*), 1086
register_opts() (*in module ironic.conf.disk_utils*), 1086
register_opts() (*in module ironic.conf.dnsmasq*), 1086
register_opts() (*in module ironic.conf.drac*), 1086
register_opts() (*in module ironic.conf.exception*), 1086
register_opts() (*in module ironic.conf.fake*), 1086
register_opts() (*in module ironic.conf.glance*), 1086

register_opts() (*in module ironic.conf.healthcheck*), 1086
register_opts() (*in module ironic.conf.ilo*), 1086
register_opts() (*in module ironic.conf.inspector*), 1086
register_opts() (*in module ironic.conf.inventory*), 1086
register_opts() (*in module ironic.conf.ipmi*), 1086
register_opts() (*in module ironic.conf.ironic_networking*), 1087
register_opts() (*in module ironic.conf.json_rpc*), 1087
register_opts() (*in module ironic.conf.mdns*), 1087
register_opts() (*in module ironic.conf.metrics*), 1087
register_opts() (*in module ironic.conf.molds*), 1087
register_opts() (*in module ironic.conf.neutron*), 1087
register_opts() (*in module ironic.conf.nfs*), 1088
register_opts() (*in module ironic.conf.nova*), 1088
register_opts() (*in module ironic.conf.oci*), 1088
register_opts() (*in module ironic.conf.pxe*), 1088
register_opts() (*in module ironic.conf.redfish*), 1088
register_opts() (*in module ironic.conf.sensor_data*), 1088
register_opts() (*in module ironic.conf.service_catalog*), 1088
register_opts() (*in module ironic.conf.snmp*), 1089
register_opts() (*in module ironic.conf.swift*), 1089
register_opts() (*in module ironic.conf.vnc*), 1089
register_service() (*ironic.common.mdns.Zeroconf method*), 979
register_translator() (*ironic.networking.switch_drivers.driver_adapter.NetworkDriverAdapter method*), 1434
registration_hook() (*ironic.objects.base.IronicObjectRegistry method*), 1451
registry_fields (*ironic.objects.bios.BIOSSetting attribute*), 1454
RegistrySessionHelper (*class in ironic.common.oci_registry*), 993
reject_fields_in_newer_versions() (*in module ironic.api.controllers.v1.node*), 858
reject_patch_in_newer_versions() (*in module ironic.api.controllers.v1.node*), 858
reject_when_reached() (*in module ironic.conductor.base_manager*), 1024
release() (*ironic.objects.node.Node class method*), 1487
release_node() (*ironic.db.api.Connection method*), 1176
release_node() (*ironic.db.sqlalchemy.api.Connection method*), 1124
release_port() (*in module ironic.drivers.modules.console_utils*), 1322
release_resources() (*ironic.conductor.task_manager.TaskManager method*), 1070
reload_configuration() (*ironic.networking.switch_drivers.driver_adapter.Network method*), 1434
remove() (*ironic.drivers.modules.ilo.firmware_processor.Firmwar method*), 1213
remove_agent_url() (*in module ironic.conductor.utils*), 1080
remove_cleaning_network() (*ironic.drivers.base.NetworkInterface method*), 1412
remove_cleaning_network() (*ironic.drivers.modules.network.flat.FlatNetwork method*), 1244
remove_cleaning_network() (*ironic.drivers.modules.network.ironic_networking.Ironicl method*), 1248
remove_cleaning_network() (*ironic.drivers.modules.network.neutron.NeutronNetwork method*), 1252
remove_cleaning_network() (*ironic.drivers.modules.network.noop.NoopNetwork method*), 1255
remove_http_instance_symlink() (*in module ironic.drivers.modules.deploy_utils*), 1330

remove_image_from_swift() (*in module ironic.drivers.modules.ilo.common*), 1209
remove_image_from_web_server() (*in module ironic.drivers.modules.ilo.common*), 1210
remove_inspection_network() (*ironic.drivers.base.NetworkInterface method*), 1412
remove_inspection_network() (*ironic.drivers.modules.network.flat.FlatNetwork method*), 1244
remove_inspection_network() (*ironic.drivers.modules.network.ironic_networking.IronicNetworking method*), 1248
remove_inspection_network() (*ironic.drivers.modules.network.neutron.NeutronNetwork method*), 1252
remove_large_keys() (*in module ironic.common.utils*), 1018
remove_neutron_ports() (*in module ironic.common.neutron*), 988
remove_node_capability() (*in module ironic.drivers.utils*), 1433
remove_node_flags() (*in module ironic.common.async_steps*), 930
remove_node_rescue_password() (*in module ironic.conductor.utils*), 1080
remove_node_traits() (*ironic.conductor.manager.ConductorManager method*), 1034
remove_node_traits() (*ironic.conductor.rpcapi.ConductorAPI method*), 1058
remove_ports_from_network() (*in module ironic.common.neutron*), 989
remove_provisioning_network() (*ironic.drivers.base.NetworkInterface method*), 1412
remove_provisioning_network() (*ironic.drivers.modules.network.flat.FlatNetwork method*), 1244
remove_provisioning_network() (*ironic.drivers.modules.network.ironic_networking.IronicNetworking method*), 1248
remove_provisioning_network() (*ironic.drivers.modules.network.neutron.NeutronNetwork method*), 1252
remove_provisioning_network() (*ironic.drivers.modules.network.noop.NoopNetwork method*), 1255
remove_rescuing_network() (*ironic.drivers.base.NetworkInterface method*), 1413
remove_rescuing_network() (*ironic.drivers.modules.network.flat.FlatNetwork method*), 1244
remove_rescuing_network() (*ironic.drivers.modules.network.ironic_networking.IronicNetworking method*), 1249
remove_rescuing_network() (*ironic.drivers.modules.network.neutron.NeutronNetwork method*), 1253
remove_servicing_network() (*ironic.drivers.base.NetworkInterface method*), 1413
remove_servicing_network() (*ironic.drivers.modules.network.flat.FlatNetwork method*), 1245
remove_servicing_network() (*ironic.drivers.modules.network.ironic_networking.IronicNetworking method*), 1249
remove_servicing_network() (*ironic.drivers.modules.network.neutron.NeutronNetwork method*), 1253
remove_single_or_list_of_files() (*in module ironic.drivers.modules.ilo.common*), 1210
remove_vifs_from_node() (*in module ironic.common.network*), 985
RemoveTraitAction (*class in ironic.common.inspection_rules.actions*), 909
render_template() (*in module ironic.common.utils*), 1018
RenderedAction (*class in ironic.common.trait_based_networking.base*), 922
ReOperator (*class in ironic.common.inspection_rules.operators*), 913
replace_node_id_with_uuid() (*in module ironic.api.controllers.v1.utils*), 885
replace_node_uuid_with_id() (*in module ironic.api.controllers.v1.utils*), 886
REQUEST (*in module ironic.common.lessee_sources*), 978
request_body_schema() (*in module ironic.api.validation*), 897
request_id (*ironic.common.json_rpc.server.EmptyContext attribute*), 916

request_parameter_schema() (*in module ironic.api.validation*), 897
request_query_schema() (*in module ironic.api.validation*), 897
RequestContext (*class in ironic.common.context*), 939
RequestContextSerializer (*class in ironic.common.rpc*), 1004
RequestLogMiddleware (*class in ironic.api.middleware.request_log*), 895
require_exclusive_lock() (*in module ironic.conductor.task_manager*), 1071
REQUIRES_PLUGIN_DATA (*ironic.common.inspection_rules.actions.ExtendPluginDataAction attribute*), 909
REQUIRES_PLUGIN_DATA (*ironic.common.inspection_rules.actions.SetPluginDataAction attribute*), 910
REQUIRES_PLUGIN_DATA (*ironic.common.inspection_rules.actions.UnsetPluginDataAction attribute*), 910
REQUIRES_PLUGIN_DATA (*ironic.common.inspection_rules.base.BaseAction attribute*), 910
RESCUE (*in module ironic.common.states*), 1008
rescue (*ironic.drivers.base.BareDriver attribute*), 1392
rescue() (*ironic.drivers.base.RescueInterface method*), 1418
rescue() (*ironic.drivers.modules.agent.AgentRescue method*), 1299
rescue() (*ironic.drivers.modules.fake.FakeRescue method*), 1347
rescue() (*ironic.drivers.modules.noop.NoRescue method*), 1371
RESCUE_ABORT_FAILURE (*in module ironic.common.faults*), 958
rescue_interface (*ironic.db.sqlalchemy.models.Node attribute*), 1140
rescue_interface (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1143
rescue_interface (*ironic.objects.node.Node property*), 1487
rescue_interface (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
rescue_interface (*ironic.objects.node.NodeCRUDPayload property*), 1493
rescue_interface (*ironic.objects.node.NodePayload property*), 1503
rescue_interface (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
rescue_interface (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
rescue_or_deploy_mode() (*in module ironic.drivers.modules.deploy_utils*), 1330
RESCUEFAIL (*in module ironic.common.states*), 1008
RescueInterface (*class in ironic.drivers.base*), 1418
RESCUEWAIT (*in module ironic.common.states*), 1008
RESCUING (*in module ironic.common.states*), 1008
RESCUING_interface (*ironic.drivers.modules.network.common.NetType attribute*), 1239
rescuing_error_handler() (*in module ironic.conductor.utils*), 1080
reservation (*ironic.db.sqlalchemy.models.Node attribute*), 1140
reservation (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1143
reservation (*ironic.objects.node.Node property*), 1487
reserve() (*ironic.objects.node.Node class method*), 1487
reserve_node() (*ironic.db.api.Connection method*), 1176
reserve_node() (*ironic.db.sqlalchemy.api.Connection method*), 1125
reset() (*ironic.common.hash_ring.HashRingManager class method*), 960
reset() (*ironic.common.wsgi_service.BaseWSGIService method*), 1021
reset_bios_to_default() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1219
reset_idrac() (*ironic.drivers.modules.drac.management.DracRe method*), 1194
reset_ilo() (*ironic.drivers.modules.ilo.management.IloManagen method*), 1219
reset_ilo_credential() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1219

reset_port() (*in module ironic.networking.api*), 1436
reset_port() (*ironic.drivers.modules.switch.base.NeutronSwitchDriverBase method*), 1290
reset_port() (*ironic.drivers.modules.switch.base.SwitchDriverBase method*), 1291
reset_port() (*ironic.drivers.modules.switch.generic_switch.GenericSwitchDriver method*), 1294
reset_port() (*ironic.networking.manager.NetworkingManager method*), 1438
reset_port() (*ironic.networking.rpcapi.NetworkingAPI method*), 1440
reset_required (*ironic.db.sqlalchemy.models.BIOSSetting attribute*), 1136
reset_required (*ironic.objects.bios.BIOSSetting property*), 1454
reset_secure_boot_keys_to_default() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1220
reset_secure_boot_keys_to_default() (*ironic.drivers.modules.redfish.management.RedfishManagement method*), 1273
resolve_type() (*ironic.api.functions.FunctionArgument method*), 899
resolve_types() (*ironic.api.functions.FunctionDefinition method*), 899
resource_class (*ironic.db.sqlalchemy.models.Allocation attribute*), 1135
resource_class (*ironic.db.sqlalchemy.models.Node attribute*), 1140
resource_class (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1143
resource_class (*ironic.objects.allocation.Allocation property*), 1446
resource_class (*ironic.objects.allocation.AllocationCRUDPayload property*), 1448
resource_class (*ironic.objects.node.Node property*), 1487
resource_class (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
resource_class (*ironic.objects.node.NodeCRUDPayload property*), 1493
resource_class (*ironic.objects.node.NodePayload property*), 1503
resource_class (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
resource_class (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
response_body_schema() (*in module ironic.api.validation*), 898
restore_boot_device() (*ironic.drivers.modules.redfish.management.RedfishManagement method*), 1273
restore_interface() (*ironic.drivers.base.DeployInterface method*), 1399
restore_interface() (*ironic.drivers.modules.agent_base.AgentBaseMixin method*), 1305
restore_power_state_if_needed() (*in module ironic.conductor.utils*), 1081
resume_cleaning() (*ironic.conductor.task_manager.TaskManager method*), 1070
retired (*ironic.db.sqlalchemy.models.Node attribute*), 1140
retired (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1143
retired (*ironic.objects.node.Node property*), 1487
retired (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
retired (*ironic.objects.node.NodeCRUDPayload property*), 1493
retired (*ironic.objects.node.NodePayload property*), 1503
retired (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
retired (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
retired_reason (*ironic.db.sqlalchemy.models.Node attribute*), 1140
retired_reason (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1143
retired_reason (*ironic.objects.node.Node property*), 1487

retired_reason (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
retired_reason (*ironic.objects.node.NodeCRUDPayload property*), 1493
retired_reason (*ironic.objects.node.NodePayload property*), 1503
retired_reason (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
retired_reason (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
retrieve_nic_components() (*ironic.drivers.modules.redfish.firmware.RedfishFirmware method*), 1264
retry_interval (*ironic.drivers.modules.snmp.SNMPDriverBase attribute*), 1382
retry_on_outdated_cache() (*in module ironic.drivers.modules.snmp*), 1389
return_type (*ironic.api.functions.FunctionDefinition attribute*), 900
revision
    ironic-dbsync command line option, 456
revision command line option
    --autogenerate, 458
    --help, 458
    --message, 458
    -h, 458
    -m, 458
revision() (*in module ironic.db.migration*), 1183
revision() (*in module ironic.db.sqlalchemy.migration*), 1134
revision() (*ironic.command.dbsync.DBCommand method*), 903
RFBAuthHandshakeFailed, 955
RFBAuthNoAvailableScheme, 955
RFBAuthScheme (*class in ironic.console.rfb.auth*), 1093
RFBAuthSchemeList (*class in ironic.console.rfb.auths*), 1094
RFBAuthSchemeNone (*class in ironic.console.rfb.authnone*), 1093
RFBSecurityProxy (*class in ironic.console.securityproxy.rfb*), 1095
ring (*ironic.common.hash_ring.HashRingManager property*), 960
ring_without_raise() (*in module ironic.common.utils*), 1018
rollback_ports() (*in module ironic.common.neutron*), 989
root() (*in module ironic.api.controllers.root*), 894
root_device (*ironic.objects.deployment.Deployment property*), 1470
root_gib (*ironic.objects.deployment.Deployment property*), 1470
RootController (*class in ironic.api.controllers.root*), 894
RootDeviceHook (*class in ironic.drivers.modules.inspector.hooks.root_device*), 1233
RPC_API_VERSION (*ironic.conductor.manager.ConductorManager attribute*), 1030
RPC_API_VERSION (*ironic.conductor.rpcapi.ConductorAPI attribute*), 1040
RPC_API_VERSION (*ironic.networking.manager.NetworkingManager attribute*), 1438
RPC_API_VERSION (*ironic.networking.rpcapi.NetworkingAPI attribute*), 1439
rpc_transport() (*in module ironic.networking.utils*), 1443
RPCHook (*class in ironic.api.hooks*), 902
RPCService (*class in ironic.command.pxe_filter*), 904
RPCService (*class in ironic.conductor.rpc_service*), 1038
RuleActionExecutionFailure, 955
RuleConditionCheckFailure, 955
rules_sanitize() (*in module ironic.api.controllers.v1.inspection_rule*), 845
run_inspection_hooks() (*in module ironic.drivers.modules.inspect_utils*), 1357
run_node_action() (*in module ironic.conductor.utils*), 1081
Runbook (*class in ironic.db.sqlalchemy.models*), 1146
Runbook (*class in ironic.objects.runbook*), 1535
RUNBOOK (*in module ironic.common.automated_clean_methods*), 933

runbook (*ironic.db.sqlalchemy.models.RunbookStep attribute*), 1147
runbook_id (*ironic.db.sqlalchemy.models.RunbookStep attribute*), 1147
runbook_sanitize() (*in module ironic.api.controllers.v1.runbook*), 867
RunbookAlreadyExists, 955
RunbookCRUDNotification (*class in ironic.objects.runbook*), 1538
RunbookCRUDPayload (*class in ironic.objects.runbook*), 1539
RunbookDuplicateName, 956
RunbookNotFound, 956
RunbooksController (*class in ironic.api.controllers.v1.runbook*), 866
RunbookStep (*class in ironic.db.sqlalchemy.models*), 1146

S

SAFE (*in module ironic.common.boot_devices*), 934
safe (*ironic.common.exception.IronicException attribute*), 952
safe_rstrip() (*in module ironic.common.utils*), 1018
safety_check_image() (*in module ironic.common.images*), 976
sanitize_dict() (*in module ironic.api.controllers.v1.utils*), 886
SAS (*in module ironic.common.raid*), 1002
SASL (*ironic.console.rfb.auth.AuthType attribute*), 1093
SATA (*in module ironic.common.raid*), 1002
save() (*ironic.objects.allocation.Allocation method*), 1446
save() (*ironic.objects.bios.BIOSSetting method*), 1454
save() (*ironic.objects.bios.BIOSSettingList class method*), 1455
save() (*ironic.objects.chassis.Chassis method*), 1458
save() (*ironic.objects.conductor.Conductor method*), 1462
save() (*ironic.objects.deploy_template.DeployTemplate method*), 1465
save() (*ironic.objects.firmware.FirmwareComponent method*), 1474
save() (*ironic.objects.inspection_rule.InspectionRule method*), 1479
save() (*ironic.objects.node.Node method*), 1487
save() (*ironic.objects.port.Port method*), 1525
save() (*ironic.objects.portgroup.Portgroup method*), 1532
save() (*ironic.objects.runbook.Runbook method*), 1538
save() (*ironic.objects.volume_connector.VolumeConnector method*), 1545
save() (*ironic.objects.volume_target.VolumeTarget method*), 1551
save_configuration() (*in module ironic.common.molds*), 984
SCHEMA (*ironic.objects.allocation.AllocationCRUDPayload attribute*), 1447
SCHEMA (*ironic.objects.chassis.ChassisCRUDPayload attribute*), 1459
SCHEMA (*ironic.objects.deploy_template.DeployTemplateCRUDPay attribute*), 1466
SCHEMA (*ironic.objects.inspection_rule.InspectionRuleCRUDPaylo attribute*), 1480
SCHEMA (*ironic.objects.node.NodeCRUDPayload attribute*), 1489
SCHEMA (*ironic.objects.node.NodePayload attribute*), 1499
SCHEMA (*ironic.objects.node.NodeSetProvisionStatePayload attribute*), 1508
SCHEMA (*ironic.objects.notification.NotificationPayloadBase attribute*), 1518
SCHEMA (*ironic.objects.port.PortCRUDPayload attribute*), 1526
SCHEMA (*ironic.objects.portgroup.PortgroupCRUDPayload attribute*), 1533
SCHEMA (*ironic.objects.runbook.RunbookCRUDPayload attribute*), 1539
SCHEMA (*ironic.objects.volume_connector.VolumeConnectorCRUD attribute*), 1547
SCHEMA (*ironic.objects.volume_target.VolumeTargetCRUDPayload attribute*), 1552
schema() (*in module ironic.common.args*), 929
Schemas (*class in ironic.api.validation*), 896
SchemaValidator (*class in ironic.api.validation.validators*), 896
scope (*ironic.db.sqlalchemy.models.InspectionRule attribute*), 1138
scope (*ironic.objects.inspection_rule.InspectionRule property*), 1479
scope (*ironic.objects.inspection_rule.InspectionRuleCRUDPayloa property*), 1480
SCSI (*in module ironic.common.raid*), 1002
second_method() (*ironic.drivers.modules.fake.FakeVendorB*

method), 1349
secure_boot (*ironic.db.sqlalchemy.models.Node attribute*), 1140
secure_boot (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1143
secure_boot (*ironic.objects.node.Node property*), 1488
secure_boot (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
secure_boot (*ironic.objects.node.NodeCRUDPayload property*), 1493
secure_boot (*ironic.objects.node.NodePayload property*), 1503
secure_boot (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
secure_boot (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
secure_boot() (*ironic.api.controllers.v1.node.NodesController method*), 852
security_handshake() (*ironic.console.rfb.auth.RFBAuthScheme method*), 1093
security_handshake() (*ironic.console.rfb.authnone.RFBAuthSchemeNone method*), 1093
security_parameters_update() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1220
security_type() (*ironic.console.rfb.auth.RFBAuthScheme method*), 1093
security_type() (*ironic.console.rfb.authnone.RFBAuthSchemeNone method*), 1093
SecurityProxy (*class in ironic.console.securityproxy.base*), 1094
SecurityProxyNegotiationFailed, 956
send_counter() (*ironic.common.metrics.MetricLogger method*), 981
send_gauge() (*ironic.common.metrics.MetricLogger method*), 981
send_head() (*ironic.console.websocketproxy.IronicWebSocketProxy method*), 1095
send_raw() (*in module ironic.drivers.modules.ipmitool*), 1365
send_raw() (*ironic.drivers.modules.ipmitool.VendorPassthru method*), 1363
send_timer() (*ironic.common.metrics.MetricLogger method*), 981
sendall() (*ironic.console.websocketproxy.TenantSock method*), 1096
sensitive (*ironic.db.sqlalchemy.models.InspectionRule attribute*), 1138
sensitive (*ironic.objects.inspection_rule.InspectionRule property*), 1479
sensitive (*ironic.objects.inspection_rule.InspectionRuleCRUDPayload property*), 1480
serialize_context() (*ironic.common.rpc.RequestContextSerializer method*), 1004
serialize_entity() (*ironic.common.rpc.RequestContextSerializer method*), 1004
serialize_entity() (*ironic.objects.base.IronicObjectSerializer method*), 1451
SERVICE (*in module ironic.common.states*), 1008
service (*ironic.objects.notification.NotificationPublisher property*), 1519
SERVICE_FAILURE (*in module ironic.common.faults*), 958
service_step (*ironic.db.sqlalchemy.models.Node attribute*), 1140
service_step (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1143
service_step (*ironic.objects.node.Node property*), 1488
service_step() (*in module ironic.drivers.base*), 1423
SERVICEFAIL (*in module ironic.common.states*), 1008
SERVICEHOLD (*in module ironic.common.states*), 1009
ServiceLookupFailure, 956
ServiceRegistrationFailure, 956
ServiceUnavailable, 956
SERVICEWAIT (*in module ironic.common.states*), 1009
SERVICING (*in module ironic.common.states*), 1009
SERVICING (*ironic.conductor.utils.StepFlow attribute*), 1071
SERVICING (*ironic.drivers.modules.network.common.NetType attribute*), 1239
servicing_error_handler() (*in module ironic.conductor.utils*), 1081
SERVICING_STATES (*in module ironic.common.states*), 1009
SessionCache (*class in ironic.drivers.modules.redfish.utils*),

1281
set() (*ironic.drivers.modules.snmp.SNMPClient method*), 1380
set_arg_types() (*ironic.api.functions.FunctionDefinition method*), 900
set_bmc_clock() (*ironic.drivers.modules.redfish.management.RedfishManagement method*), 1273
set_boot_device() (*ironic.conductor.manager.ConductorManager method*), 1034
set_boot_device() (*ironic.conductor.rpcapi.ConductorAPI method*), 1058
set_boot_device() (*ironic.drivers.base.ManagementInterface method*), 1408
set_boot_device() (*ironic.drivers.modules.fake.FakeManagement method*), 1344
set_boot_device() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1220
set_boot_device() (*ironic.drivers.modules.ipmitool.IPMIManagement method*), 1360
set_boot_device() (*ironic.drivers.modules.noop_mgmt.NoopManagement method*), 1374
set_boot_device() (*ironic.drivers.modules.redfish.management.RedfishManagement method*), 1273
set_boot_mode() (*in module ironic.drivers.modules.ilo.common*), 1210
set_boot_mode() (*ironic.drivers.base.ManagementInterface method*), 1408
set_boot_mode() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1220
set_boot_mode() (*ironic.drivers.modules.redfish.management.RedfishManagement method*), 1274
set_boot_to_disk() (*in module ironic.drivers.modules.agent*), 1303
set_boot_to_disk() (*ironic.drivers.modules.agent.BootcAgentDeploy method*), 1300
set_console_mode() (*ironic.conductor.manager.ConductorManager method*), 1034
set_console_mode() (*ironic.conductor.rpcapi.ConductorAPI method*), 1059
set_defaults() (*in module ironic.common.rpc*), 1005
set_driver_internal_info() (*ironic.objects.node.Node method*), 1488
set_failed_state() (*in module ironic.drivers.modules.deploy_utils*), 1330
set_global_manager() (*in module ironic.common.rpc*), 1005
set_image_auth() (*ironic.common.image_service.OciImageService method*), 969
set_indicator_state() (*ironic.conductor.manager.ConductorManager method*), 1034
set_indicator_state() (*ironic.conductor.rpcapi.ConductorAPI method*), 1059
set_indicator_state() (*ironic.drivers.base.ManagementInterface method*), 1409
set_indicator_state() (*ironic.drivers.modules.redfish.management.RedfishManagement method*), 1274
set_instance_info() (*ironic.objects.node.Node method*), 1488
set_iscsi_boot_target() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1221
set_local_link_connection() (*ironic.objects.port.Port method*), 1525
set_mask_enabled() (*ironic.common.inspection_rules.utils.ShallowMaskDict method*), 914
set_mask_enabled() (*ironic.common.inspection_rules.utils.ShallowMaskList method*), 914
set_node_cleaning_steps() (*in module ironic.conductor.steps*), 1066
set_node_deployment_steps() (*in module ironic.conductor.steps*), 1067
set_node_flags() (*in module ironic.common.async_steps*), 931
set_node_nested_field() (*in module*

ironic.common.utils), 1019
set_node_service_steps() (*in module ironic.conductor.steps*), 1067
set_node_tags() (*ironic.db.api.Connection method*), 1176
set_node_tags() (*ironic.db.sqlalchemy.api.Connection method*), 1125
set_node_traits() (*ironic.db.api.Connection method*), 1177
set_node_traits() (*ironic.db.sqlalchemy.api.Connection method*), 1125
set_options() (*ironic.api.functions.FunctionDefinition method*), 900
set_power_state() (*ironic.drivers.base.PowerInterface method*), 1415
set_power_state() (*ironic.drivers.modules.agent_power.AgentPower method*), 1318
set_power_state() (*ironic.drivers.modules.fake.FakePower method*), 1345
set_power_state() (*ironic.drivers.modules.ilo.power.IloPower method*), 1223
set_power_state() (*ironic.drivers.modules.ipmitool.IPMIPower method*), 1361
set_power_state() (*ironic.drivers.modules.redfish.power.RedfishPower method*), 1276
set_power_state() (*ironic.drivers.modules.snmp.SNMPPower method*), 1389
set_property() (*ironic.objects.node.Node method*), 1488
set_secure_boot_mode() (*in module ironic.drivers.modules.ilo.common*), 1210
set_secure_boot_state() (*ironic.drivers.base.ManagementInterface method*), 1409
set_secure_boot_state() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1221
set_secure_boot_state() (*ironic.drivers.modules.redfish.management.RedfishManagement method*), 1275
set_spawn_error_hook() (*ironic.conductor.task_manager.TaskManager method*), 1070
set_target_raid_config() (*ironic.conductor.manager.ConductorManager method*), 1034
set_target_raid_config() (*ironic.conductor.rpcapi.ConductorAPI method*), 1060
set_value() (*ironic.drivers.modules.inspector.lldp_parsers.LLDP method*), 1237
SetAttributeAction (*class in ironic.common.inspection_rules.actions*), 909
SetCapabilityAction (*class in ironic.common.inspection_rules.actions*), 909
SetPluginDataAction (*class in ironic.common.inspection_rules.actions*), 910
SetPortAttributeAction (*class in ironic.common.inspection_rules.actions*), 910
setup() (*in module ironic.common.profiler*), 996
setup_app() (*in module ironic.api.app*), 899
setup_uefi_https() (*in module ironic.drivers.modules.ilo.common*), 1210
setup_vmedia() (*in module ironic.drivers.modules.ilo.common*), 1211
setup_vmedia_for_boot() (*in module ironic.drivers.modules.ilo.common*), 1211
severity (*ironic.db.sqlalchemy.models.NodeHistory attribute*), 1143
severity (*ironic.objects.node_history.NodeHistory property*), 1516
ShallowMaskDict (*class in ironic.common.inspection_rules.utils*), 914
ShallowMaskList (*class in ironic.common.inspection_rules.utils*), 914
shard (*ironic.db.sqlalchemy.models.Node attribute*), 1141
shard (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1143
shard (*ironic.objects.node.Node property*), 1488
ShardController (*class in ironic.api.controllers.v1.shard*), 867
should_manage_boot()

(*ironic.drivers.modules.agent.CustomAgentDeploy method*), 1302
should_manage_boot() (*ironic.drivers.modules.agent_base.AgentBaseMixin method*), 1306
should_manage_boot() (*ironic.drivers.modules.pxe.PXEAnacondaDeploy method*), 1376
should_write_image() (*ironic.drivers.base.StorageInterface method*), 1419
should_write_image() (*ironic.drivers.modules.fake.FakeStorage method*), 1348
should_write_image() (*ironic.drivers.modules.storage.cinder.CinderStorage method*), 1286
should_write_image() (*ironic.drivers.modules.storage.external.ExternalStorage method*), 1287
should_write_image() (*ironic.drivers.modules.storage.noop.NoopStorage method*), 1288
show() (*ironic.common.glance_service.image_service.GlanceImageService method*), 906
show() (*ironic.common.image_service.BaseImageService method*), 963
show() (*ironic.common.image_service.CifsImageService method*), 964
show() (*ironic.common.image_service.FileImageService method*), 965
show() (*ironic.common.image_service.HttpImageService method*), 966
show() (*ironic.common.image_service.NfsImageService method*), 967
show() (*ironic.common.image_service.OciImageService method*), 969
shutdown (*ironic.drivers.modules.snmp.SNMPDriverFallback attribute*), 1386
sig (*in module ironic.api.functions*), 900
signature (*class in ironic.api.functions*), 900
simple_headers (*ironic.api.app.IronicCORS attribute*), 898
SimpleOperator (*class in ironic.common.inspection_rules.operators*), 913
single_expression() (*ironic.common.trait_based_networking.grammar.parameterizer.ExpressionTransformer method*), 917
SingleExpression (*class in ironic.common.trait_based_networking.base*), 922
skip_automated_cleaning() (*in module ironic.conductor.utils*), 1081
sleep() (*in module ironic.drivers.modules.fake*), 1350
SNMPClient (*class in ironic.drivers.modules.snmp*), 1380
SNMPDriverAPCMasterSwitch (*class in ironic.drivers.modules.snmp*), 1380
SNMPDriverAPCMasterSwitchPlus (*class in ironic.drivers.modules.snmp*), 1381
SNMPDriverAPCRackPDU (*class in ironic.drivers.modules.snmp*), 1381
SNMPDriverAten (*class in ironic.drivers.modules.snmp*), 1381
SNMPDriverAuto (*class in ironic.drivers.modules.snmp*), 1382
SNMPDriverBase (*class in ironic.drivers.modules.snmp*), 1382
SNMPDriverBaytechMRP27 (*class in ironic.drivers.modules.snmp*), 1382
SNMPDriverCyberPower (*class in ironic.drivers.modules.snmp*), 1383
SNMPDriverEatonPower (*class in ironic.drivers.modules.snmp*), 1383
SNMPDriverRaritanPDU2 (*class in ironic.drivers.modules.snmp*), 1384
SNMPDriverServerTechSentry3 (*class in ironic.drivers.modules.snmp*), 1385
SNMPDriverServerTechSentry4 (*class in ironic.drivers.modules.snmp*), 1385
SNMPDriverSimple (*class in ironic.drivers.modules.snmp*), 1386
SNMPDriverTeltronix (*class in ironic.drivers.modules.snmp*), 1387
SNMPDriverVertivGeistPDU (*class in ironic.drivers.modules.snmp*), 1387
SNMPFailure, 956
SNMPHardware (*class in ironic.drivers.snmp*), 1430
SNMPPower (*class in ironic.drivers.modules.snmp*), 1388
socket() (*ironic.console.websocketproxy.IronicProxyRequestHandler method*), 1096
SOFT_POWER_OFF (*in module ironic.common.states*), 1009
soft_power_off() (*in module ironic.drivers.modules.irmc.power*), 1203
SOFT_REBOOT (*in module ironic.common.states*), 1009
spawn_after() (*ironic.conductor.task_manager.TaskManager*

method), 1070
spawn_cleaning_error_handler() (*in module ironic.conductor.utils*), 1081
spawn_deploying_error_handler() (*in module ironic.conductor.utils*), 1082
spawn_rescue_error_handler() (*in module ironic.conductor.utils*), 1082
spawn_servicing_error_handler() (*in module ironic.conductor.utils*), 1082
STABLE_STATES (*in module ironic.common.states*), 1009
stage() (*in module ironic.drivers.modules.redfish.firmware_utils*), 1265
stamp
ironic-dbsync command line option, 456
stamp command line option
--help, 458
--revision, 458
-h, 458
stamp() (*in module ironic.db.migration*), 1183
stamp() (*in module ironic.db.sqlalchemy.migration*), 1134
stamp() (*ironic.command.dbsync.DBCommand method*), 904
standalone_ports_supported (*ironic.db.sqlalchemy.models.Portgroup attribute*), 1146
standalone_ports_supported (*ironic.objects.portgroup.Portgroup property*), 1533
standalone_ports_supported (*ironic.objects.portgroup.PortgroupCRUDPayload property*), 1535
standard_fields (*ironic.api.controllers.v1.node.NodeHistoryController attribute*), 848
standby (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1384
START (*ironic.objects.fields.NotificationStatus attribute*), 1472
start() (*ironic.common.rpc_service.BaseRPCService method*), 1005
start() (*ironic.common.wsgi_service.BaseWSGIService method*), 1021
start() (*ironic.console.novncproxy_service.NoVNCProxyService method*), 1095
start_console() (*ironic.drivers.base.ConsoleInterface method*), 1397
start_console() (*ironic.drivers.modules.fake.FakeConsole method*), 1336
start_console() (*ironic.drivers.modules.graphical_console.GraphicalCons method*), 1350
start_console() (*ironic.drivers.modules.ipmitool.IPMISocatConsole method*), 1362
start_console() (*ironic.drivers.modules.noop.NoConsole method*), 1367
start_container() (*ironic.console.container.base.BaseConsoleContainer method*), 1089
start_container() (*ironic.console.container.fake.FakeConsoleContainer method*), 1090
start_container() (*ironic.console.container.kubernetes.KubernetesConsoleC method*), 1091
start_container() (*ironic.console.container.systemd.SystemdConsoleContain method*), 1092
start_deploy() (*in module ironic.conductor.deployments*), 1029
start_socat_console() (*in module ironic.drivers.modules.console_utils*), 1322
state (*ironic.db.sqlalchemy.models.Allocation attribute*), 1135
state (*ironic.objects.allocation.Allocation property*), 1447
state (*ironic.objects.allocation.AllocationCRUDPayload property*), 1448
state (*ironic.objects.deployment.Deployment property*), 1470
states (*ironic.api.controllers.v1.node.NodesController attribute*), 857
StatsdMetricLogger (*class in ironic.common.metrics_statsd*), 983
status (*ironic.objects.notification.EventType property*), 1517
status_closed (*ironic.drivers.modules.snmp.SNMPDriverRarita attribute*), 1385
status_code (*ironic.api.controllers.v1.utils.PassthruResponse attribute*), 867
status_code (*ironic.api.functions.FunctionDefinition attribute*), 900
status_normal (*ironic.drivers.modules.snmp.SNMPDriverRarita attribute*), 1385

status_off (*ironic.drivers.modules.snmp.SNMPDriverEatonPower attribute*), 1383
status_off (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1385
status_off (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry3 attribute*), 1385
status_off (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry4 attribute*), 1386
status_off_wait (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry3 attribute*), 1385
status_on (*ironic.drivers.modules.snmp.SNMPDriverEatonPower attribute*), 1383
status_on (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1385
status_on (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry3 attribute*), 1385
status_on (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry4 attribute*), 1386
status_on_wait (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry3 attribute*), 1385
status_open (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1385
status_pending_off (*ironic.drivers.modules.snmp.SNMPDriverEatonPower attribute*), 1383
status_pending_on (*ironic.drivers.modules.snmp.SNMPDriverEatonPower attribute*), 1383
status_pendOff (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry4 attribute*), 1386
status_pendOn (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry4 attribute*), 1386
step (*ironic.db.sqlalchemy.models.DeployTemplateStep attribute*), 1137
step (*ironic.db.sqlalchemy.models.RunbookStep attribute*), 1147
step_error_handler() (*in module ironic.drivers.modules.deploy_utils*), 1330
step_id() (*in module ironic.conductor.steps*), 1067
step_sanitize() (*in module ironic.api.controllers.v1.deploy_template*), 840
step_sanitize() (*in module ironic.api.controllers.v1.runbook*), 867
StepFlow (*class in ironic.conductor.utils*), 1071
steps (*ironic.db.sqlalchemy.models.DeployTemplate attribute*), 1137
steps (*ironic.db.sqlalchemy.models.Runbook attribute*), 1146
steps (*ironic.objects.deploy_template.DeployTemplate property*), 1466
steps (*ironic.objects.deploy_template.DeployTemplateCRUDPayload property*), 1467
steps (*ironic.objects.runbook.Runbook property*), 1538
steps (*ironic.objects.runbook.RunbookCRUDPayload property*), 1540
Stop, 1037
stop() (*ironic.command.pxe_filter.RPCService method*), 904
stop() (*ironic.common.wsgi_service.BaseWSGIService method*), 1021
stop() (*ironic.conductor.rpc_service.RPCService method*), 1038
stop() (*ironic.networking.rpc_service.NetworkingRPCService method*), 1439
stop_after_retries() (*in module ironic.common.utils*), 1019
stop_all_containers() (*ironic.console.container.base.BaseConsoleContainer method*), 1090
stop_all_containers() (*ironic.console.container.fake.FakeConsoleContainer method*), 1090
stop_all_containers() (*ironic.console.container.kubernetes.KubernetesConsoleContainer method*), 1091
stop_all_containers() (*ironic.console.container.systemd.SystemdConsoleContainer method*), 1092
stop_console() (*ironic.drivers.base.ConsoleInterface method*), 1397
stop_console() (*ironic.drivers.modules.fake.FakeConsole method*), 1336
stop_console() (*ironic.drivers.modules.graphical_console.GraphicalConsole method*), 1350
stop_console() (*ironic.drivers.modules.ipmitool.IPMISocatConsole method*), 1362
stop_console() (*ironic.drivers.modules.noop.NoConsole method*), 1367
stop_container()

(*ironic.console.container.base.BaseConsoleContainer method*), 1090
stop_container() (*ironic.console.container.fake.FakeConsoleContainer method*), 1091
stop_container() (*ironic.console.container.kubernetes.KubernetesConsoleContainer method*), 1091
stop_container() (*ironic.console.container.systemd.SystemdConsoleContainer method*), 1092
stop_socat_console() (*in module ironic.drivers.modules.console_utils*), 1322
storage (*ironic.drivers.base.BareDriver attribute*), 1392
storage_interface (*ironic.db.sqlalchemy.models.Node attribute*), 1141
storage_interface (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1143
storage_interface (*ironic.objects.node.Node property*), 1488
storage_interface (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
storage_interface (*ironic.objects.node.NodeCRUDPayload property*), 1493
storage_interface (*ironic.objects.node.NodePayload property*), 1503
storage_interface (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
storage_interface (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
StorageError, 956
StorageInterface (*class in ironic.drivers.base*), 1419
StorageInterfaceFactory (*class in ironic.common.driver_factory*), 941
store_agent_certificate() (*in module ironic.conductor.utils*), 1082
store_inspection_data() (*in module ironic.drivers.modules.inspect_utils*), 1357
store_ramdisk_logs() (*in module ironic.drivers.utils*), 1433
String (*ironic.api.controllers.base.Version attribute*), 893
string() (*in module ironic.common.args*), 929
String_list() (*in module ironic.common.args*), 929
string_literal() (*ironic.common.trait_based_networking.grammar.parser.T method*), 917
StringAcceptsCallable (*class in ironic.objects.fields*), 1472
StringField (*class in ironic.objects.fields*), 1472
StringFieldThatAcceptsCallable (*class in ironic.objects.fields*), 1472
STUCK_STATES_TREATED_AS_FAIL (*in module ironic.common.states*), 1009
SUCCESS (*ironic.objects.fields.NotificationStatus attribute*), 1472
supported (*ironic.drivers.base.BaseInterface attribute*), 1395
supported (*ironic.drivers.hardware_type.AbstractHardwareType attribute*), 1427
supported (*ironic.drivers.modules.drac.bios.DracRedfishBIOS attribute*), 1191
supported (*ironic.drivers.modules.drac.power.DracRedfishPower attribute*), 1195
supported (*ironic.drivers.modules.drac.vendor_passthru.DracRed attribute*), 1197
SUPPORTED (*ironic.drivers.modules.switch.base.NoOpSwitchDriver attribute*), 1289
SUPPORTED (*ironic.drivers.modules.switch.base.SwitchDriverBase attribute*), 1290
supported (*ironic.drivers.modules.switch.base.SwitchDriverBase property*), 1292
SUPPORTED (*ironic.drivers.modules.switch.generic_switch.Generic attribute*), 1293
supported (*ironic.drivers.snmp.SNMPHardware attribute*), 1430
supported() (*ironic.api.controllers.v1.node.BootDeviceControlle method*), 846
supported_bios_interfaces (*ironic.drivers.drac.IDRACHardware property*), 1424
supported_bios_interfaces (*ironic.drivers.fake_hardware.FakeHardware property*), 1425
supported_bios_interfaces (*ironic.drivers.hardware_type.AbstractHardwareType property*), 1427
supported_bios_interfaces (*ironic.drivers.ilo.IloHardware property*), 1428

supported_bios_interfaces (*ironic.drivers.redfish.RedfishHardware property*), 1429
supported_boot_interfaces (*ironic.drivers.drac.IDRACHardware property*), 1424
supported_boot_interfaces (*ironic.drivers.fake_hardware.FakeHardware property*), 1425
supported_boot_interfaces (*ironic.drivers.generic.GenericHardware property*), 1426
supported_boot_interfaces (*ironic.drivers.hardware_type.AbstractHardwareType property*), 1427
supported_boot_interfaces (*ironic.drivers.ilo.Ilo5Hardware property*), 1428
supported_boot_interfaces (*ironic.drivers.ilo.IloHardware property*), 1428
supported_boot_interfaces (*ironic.drivers.redfish.RedfishHardware property*), 1429
SUPPORTED_BOOT_MODE_LEGACY_BIOS_AND_UEFI (*in module ironic.drivers.modules.ilo.common*), 1206
SUPPORTED_BOOT_MODE_LEGACY_BIOS_ONLY (*in module ironic.drivers.modules.ilo.common*), 1206
SUPPORTED_BOOT_MODE_UEFI_ONLY (*in module ironic.drivers.modules.ilo.common*), 1206
supported_console_interfaces (*ironic.drivers.fake_hardware.FakeHardware property*), 1425
supported_console_interfaces (*ironic.drivers.generic.GenericHardware property*), 1426
supported_console_interfaces (*ironic.drivers.hardware_type.AbstractHardwareType property*), 1427
supported_console_interfaces (*ironic.drivers.ilo.IloHardware property*), 1428
supported_console_interfaces (*ironic.drivers.ipmi.IPMIHardware property*), 1429
supported_console_interfaces (*ironic.drivers.redfish.RedfishHardware property*), 1430
supported_deploy_interfaces (*ironic.drivers.fake_hardware.FakeHardware property*), 1425
supported_deploy_interfaces (*ironic.drivers.generic.GenericHardware property*), 1426
supported_deploy_interfaces (*ironic.drivers.hardware_type.AbstractHardwareType property*), 1427
supported_firmware_interfaces (*ironic.drivers.fake_hardware.FakeHardware property*), 1425
supported_firmware_interfaces (*ironic.drivers.generic.GenericHardware property*), 1426
supported_firmware_interfaces (*ironic.drivers.hardware_type.AbstractHardwareType property*), 1427
supported_firmware_interfaces (*ironic.drivers.redfish.RedfishHardware property*), 1430
supported_inspect_interfaces (*ironic.drivers.drac.IDRACHardware property*), 1424
supported_inspect_interfaces (*ironic.drivers.fake_hardware.FakeHardware property*), 1425
supported_inspect_interfaces (*ironic.drivers.generic.GenericHardware property*), 1426
supported_inspect_interfaces (*ironic.drivers.hardware_type.AbstractHardwareType property*), 1427
supported_inspect_interfaces (*ironic.drivers.ilo.IloHardware property*), 1428
supported_inspect_interfaces (*ironic.drivers.redfish.RedfishHardware property*), 1430
supported_management_interfaces (*ironic.drivers.drac.IDRACHardware property*), 1425
supported_management_interfaces (*ironic.drivers.fake_hardware.FakeHardware property*), 1425
supported_management_interfaces (*ironic.drivers.generic.ManualManagementHardware property*), 1426
supported_management_interfaces

(*ironic.drivers.hardware_type.AbstractHardwareType property*), 1427
supported_management_interfaces (*ironic.drivers.ilo.Ilo5Hardware property*), 1428
supported_management_interfaces (*ironic.drivers.ilo.IloHardware property*), 1428
supported_management_interfaces (*ironic.drivers.intel_ipmi.IntelIPMIHardware property*), 1429
supported_management_interfaces (*ironic.drivers.ipmi.IPMIHardware property*), 1429
supported_management_interfaces (*ironic.drivers.redfish.RedfishHardware property*), 1430
supported_management_interfaces (*ironic.drivers.snmp.SNMPHardware property*), 1430
supported_network_interfaces (*ironic.drivers.generic.GenericHardware property*), 1426
supported_network_interfaces (*ironic.drivers.hardware_type.AbstractHardwareType property*), 1427
supported_power_interfaces (*ironic.drivers.drac.IDRACHardware property*), 1425
supported_power_interfaces (*ironic.drivers.fake_hardware.FakeHardware property*), 1425
supported_power_interfaces (*ironic.drivers.generic.ManualManagementHardware property*), 1427
supported_power_interfaces (*ironic.drivers.hardware_type.AbstractHardwareType property*), 1428
supported_power_interfaces (*ironic.drivers.ilo.IloHardware property*), 1429
supported_power_interfaces (*ironic.drivers.ipmi.IPMIHardware property*), 1429
supported_power_interfaces (*ironic.drivers.redfish.RedfishHardware property*), 1430
supported_power_interfaces (*ironic.drivers.snmp.SNMPHardware property*), 1430
supported_raid_interfaces (*ironic.drivers.drac.IDRACHardware property*), 1425
supported_raid_interfaces (*ironic.drivers.fake_hardware.FakeHardware property*), 1425
supported_raid_interfaces (*ironic.drivers.generic.GenericHardware property*), 1426
supported_raid_interfaces (*ironic.drivers.hardware_type.AbstractHardwareType property*), 1428
supported_raid_interfaces (*ironic.drivers.ilo.Ilo5Hardware property*), 1428
supported_raid_interfaces (*ironic.drivers.redfish.RedfishHardware property*), 1430
supported_rescue_interfaces (*ironic.drivers.fake_hardware.FakeHardware property*), 1425
supported_rescue_interfaces (*ironic.drivers.generic.GenericHardware property*), 1426
supported_rescue_interfaces (*ironic.drivers.hardware_type.AbstractHardwareType property*), 1428
supported_storage_interfaces (*ironic.drivers.fake_hardware.FakeHardware property*), 1426
supported_storage_interfaces (*ironic.drivers.generic.GenericHardware property*), 1426
supported_storage_interfaces (*ironic.drivers.hardware_type.AbstractHardwareType property*), 1428
supported_vendor_interfaces (*ironic.drivers.drac.IDRACHardware property*), 1425
supported_vendor_interfaces (*ironic.drivers.fake_hardware.FakeHardware property*), 1426
supported_vendor_interfaces (*ironic.drivers.generic.ManualManagementHardware property*), 1427
supported_vendor_interfaces (*ironic.drivers.hardware_type.AbstractHardwareType property*), 1428
supported_vendor_interfaces (*ironic.drivers.ilo.IloHardware property*), 1429
supported_vendor_interfaces

(*ironic.drivers.ipmi.IPMIHardware property*), 1429
supported_vendor_interfaces (*ironic.drivers.redfish.RedfishHardware property*), 1430
supports_deploy() (*ironic.drivers.base.DeployInterface method*), 1399
supports_deploy() (*ironic.drivers.modules.agent.BootcAgentDeploy method*), 1300
supports_deploy() (*ironic.drivers.modules.ramdisk.RamdiskDeploy method*), 1379
supports_ipxe_tag() (*ironic.dhcp.base.BaseDHCP method*), 1184
supports_ipxe_tag() (*ironic.dhcp.dnsmasq.DnsmasqDHCPApi method*), 1185
supports_ipxe_tag() (*ironic.dhcp.neutron.NeutronDHCPApi method*), 1186
supports_is_smartnic() (*ironic.objects.port.Port class method*), 1525
supports_physical_network() (*ironic.objects.port.Port class method*), 1526
supports_power_sync() (*ironic.drivers.base.PowerInterface method*), 1415
supports_power_sync() (*ironic.drivers.modules.agent_power.AgentPower method*), 1319
supports_version() (*ironic.objects.base.IronicObject class method*), 1450
swap_mib (*ironic.objects.deployment.Deployment property*), 1470
swift_temp_url() (*ironic.common.glance_service.image_service.GlanceImageService method*), 906
SwiftAPI (*class in ironic.common.swift*), 1010
SwiftObjectNotFoundError, 956
SwiftObjectStillExists, 956
SwiftOperationError, 956
SwiftPublisher (*class in ironic.common.image_publisher*), 962
switch_drivers() (*in module ironic.networking.switch_drivers.driver_factory*), 1435
switch_interface() (*ironic.drivers.base.DeployInterface method*), 1399
switch_interface() (*ironic.drivers.modules.autodetect.AutodetectDeploy method*), 1320
switch_pxe_config() (*in module ironic.drivers.modules.deploy_utils*), 1330
switch_to_tenant_network() (*ironic.drivers.modules.agent_base.AgentOobStepsMixin method*), 1307
SwitchDriverBase (*class in ironic.drivers.modules.switch.base*), 1290
SwitchDriverException, 1293
SwitchDriverFactory (*class in ironic.networking.switch_drivers.driver_factory*), 1435
SwitchMethodNotImplemented, 1293
SwitchNotFound, 956, 1293
SwitchPortConfig (*class in ironic.drivers.modules.network.switchport_config*), 1256
sync() (*in module ironic.pxe_filter.dnsmasq*), 1553
sync() (*ironic.drivers.modules.agent_client.AgentClient method*), 1317
sync_boot_mode() (*in module ironic.drivers.modules.boot_mode_utils*), 1321
sync_firmware_components() (*ironic.objects.firmware.FirmwareComponentList class method*), 1475
sync_node_setting() (*ironic.objects.bios.BIOSSettingList class method*), 1456
SYS_OBJ_OID (*ironic.drivers.modules.snmp.SNMPDriverAuto attribute*), 1382
SYSTEM (*in module ironic.common.components*), 970
system_id (*ironic.drivers.modules.snmp.SNMPDriverAPCMaster attribute*), 1381
system_id (*ironic.drivers.modules.snmp.SNMPDriverAPCMaster attribute*), 1381
system_id (*ironic.drivers.modules.snmp.SNMPDriverAPCRackPD attribute*), 1381
system_id (*ironic.drivers.modules.snmp.SNMPDriverAten attribute*), 1381
system_id (*ironic.drivers.modules.snmp.SNMPDriverCyberPower

attribute), 1383
system_id (*ironic.drivers.modules.snmp.SNMPDriverEatonPower attribute*), 1384
system_id (*ironic.drivers.modules.snmp.SNMPDriverTeltronix attribute*), 1387
SystemdConsoleContainer (*class in ironic.console.container.systemd*), 1092

T

table_args() (*in module ironic.db.sqlalchemy.models*), 1148
tag (*ironic.db.sqlalchemy.models.NodeTag attribute*), 1144
tags (*ironic.common.trait_based_networking.base.Network attribute*), 920
tags (*ironic.db.sqlalchemy.models.Node attribute*), 1141
take_over() (*ironic.drivers.base.DeployInterface method*), 1400
take_over() (*ironic.drivers.modules.agent_base.AgentBaseMixin method*), 1306
take_over() (*ironic.drivers.modules.ansible.deploy.AnsibleDeploy method*), 1190
take_over() (*ironic.drivers.modules.autodetect.AutodetectDeploy method*), 1320
take_over() (*ironic.drivers.modules.fake.FakeDeploy method*), 1338
take_over() (*ironic.drivers.modules.noop.NoDeploy method*), 1368
take_over_allocation() (*ironic.db.api.Connection method*), 1177
take_over_allocation() (*ironic.db.sqlalchemy.api.Connection method*), 1126
target (*ironic.conductor.manager.ConductorManager attribute*), 1034
target (*ironic.networking.manager.NetworkingManager attribute*), 1438
target_power_state (*ironic.db.sqlalchemy.models.Node attribute*), 1141
target_power_state (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1143
target_power_state (*ironic.objects.node.Node property*), 1488
target_power_state (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
target_power_state (*ironic.objects.node.NodeCRUDPayload property*), 1493
target_power_state (*ironic.objects.node.NodePayload property*), 1503
target_power_state (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
target_power_state (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1512
target_provision_state (*ironic.db.sqlalchemy.models.Node attribute*), 1141
target_provision_state (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1143
target_provision_state (*ironic.objects.node.Node property*), 1488
target_provision_state (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
target_provision_state (*ironic.objects.node.NodeCRUDPayload property*), 1493
target_provision_state (*ironic.objects.node.NodePayload property*), 1503
target_provision_state (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
target_provision_state (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1513
target_raid_config (*ironic.db.sqlalchemy.models.Node attribute*), 1141
target_raid_config (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1143
target_raid_config (*ironic.objects.node.Node property*), 1488
target_state (*ironic.common.fsm.FSM property*), 959
TaskManager (*class in ironic.conductor.task_manager*), 1069
tbn_config_file_traits() (*in module ironic.common.trait_based_networking.loader*), 925

tbn_traits (*ironic.conductor.task_manager.TaskManager property*), 1071
TBNAttributeRetrievalException, 956
TBNComparatorPrefixMatchTypeMismatch, 956
TBNNoPortMatchesFound, 956
TBNNoTraitsApplicableToNode, 957
tear_down() (*ironic.drivers.base.DeployInterface method*), 1400
tear_down() (*ironic.drivers.modules.agent_base.AgentBaseMixin method*), 1306
tear_down() (*ironic.drivers.modules.ansible.deploy.AnsibleDeploy method*), 1191
tear_down() (*ironic.drivers.modules.autodetect.AutodetectDeploy method*), 1320
tear_down() (*ironic.drivers.modules.fake.FakeDeploy method*), 1338
tear_down() (*ironic.drivers.modules.noop.NoDeploy method*), 1369
tear_down_agent() (*ironic.drivers.modules.agent.CustomAgentDeploy method*), 1302
tear_down_agent() (*ironic.drivers.modules.ansible.deploy.AnsibleDeploy method*), 1191
tear_down_cleaning() (*ironic.drivers.base.DeployInterface method*), 1400
tear_down_cleaning() (*ironic.drivers.modules.agent_base.AgentBaseMixin method*), 1306
tear_down_cleaning() (*ironic.drivers.modules.ansible.deploy.AnsibleDeploy method*), 1191
tear_down_inband_cleaning() (*in module ironic.drivers.modules.deploy_utils*), 1331
tear_down_inband_service() (*in module ironic.drivers.modules.deploy_utils*), 1331
tear_down_managed_boot() (*in module ironic.drivers.modules.inspector.interface*), 1235
tear_down_service() (*ironic.drivers.base.DeployInterface method*), 1400
tear_down_service() (*ironic.drivers.modules.agent_base.AgentBaseMixin method*), 1306
tear_down_storage_configuration() (*in module ironic.drivers.modules.deploy_utils*), 1332
tempdir() (*in module ironic.common.utils*), 1019
template_sanitize() (*in module ironic.api.controllers.v1.deploy_template*), 841
TemporaryFailure, 957
TempUrlCacheElement (*class in ironic.common.glance_service.image_service*), 906
TENANT (*ironic.drivers.modules.network.common.NetType attribute*), 1239
TenantSock (*class in ironic.console.websocketproxy*), 1096
terminate() (*ironic.console.websocketproxy.IronicWebSocketProxy method*), 1096
TFTPImageCache (*class in ironic.common.pxe_utils*), 996
third_method_sync() (*ironic.drivers.modules.fake.FakeVendorB method*), 1349
TIGHT (*ironic.console.rfb.auth.AuthType attribute*), 1093
Timer (*class in ironic.common.metrics*), 982
timer() (*ironic.common.metrics.MetricLogger method*), 982
TIMER_TYPE (*ironic.common.metrics_collector.DictCollectionMetr attribute*), 982
TIMER_TYPE (*ironic.common.metrics_statsd.StatsdMetricLogger attribute*), 983
timestamp_driver_internal_info() (*ironic.objects.node.Node method*), 1488
TLS (*ironic.console.rfb.auth.AuthType attribute*), 1093
to_dict() (*ironic.common.context.RequestContext method*), 939
to_dict() (*ironic.common.json_rpc.server.EmptyContext method*), 916
to_event_type_field() (*ironic.objects.notification.EventType method*), 1517
to_policy_values() (*ironic.common.context.RequestContext method*), 940
to_power (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
token_valid_until() (*in module ironic.common.vnc*), 1021
topic (*ironic.pxe_filter.service.PXEFilterManager attribute*), 1554

touch() (*ironic.objects.conductor.Conductor method*), 1462
touch_conductor() (*ironic.db.api.Connection method*), 1177
touch_conductor() (*ironic.db.sqlalchemy.api.Connection method*), 1126
touch_node_provisioning() (*ironic.db.api.Connection method*), 1177
touch_node_provisioning() (*ironic.db.sqlalchemy.api.Connection method*), 1126
touch_provisioning() (*ironic.objects.node.Node method*), 1488
trace_cls() (*in module ironic.common.profiler*), 996
Trait (*class in ironic.objects.trait*), 1540
trait (*ironic.db.sqlalchemy.models.NodeTrait attribute*), 1144
trait (*ironic.objects.trait.Trait property*), 1541
trait_action (*ironic.common.trait_based_networking.base.InterfaceTrait attribute*), 922
trait_name (*ironic.common.trait_based_networking.base.TraitAction attribute*), 923
TraitAction (*class in ironic.common.trait_based_networking.base*), 923
TraitBasedNetworkingException, 957
TraitList (*class in ironic.objects.trait*), 1541
traits (*ironic.common.trait_based_networking.loader.ConfigLoader property*), 925
traits (*ironic.db.sqlalchemy.models.Allocation attribute*), 1135
traits (*ironic.db.sqlalchemy.models.Node attribute*), 1141
traits (*ironic.objects.allocation.Allocation property*), 1447
traits (*ironic.objects.allocation.AllocationCRUDPayload property*), 1448
traits (*ironic.objects.node.Node property*), 1489
traits (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1498
traits (*ironic.objects.node.NodeCRUDPayload property*), 1493
traits (*ironic.objects.node.NodePayload property*), 1503
traits (*ironic.objects.node.NodeSetPowerStatePayload property*), 1507
traits (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1513
traits() (*ironic.common.trait_based_networking.config_file.ConfigFileLoader method*), 924
transfer_verified_checksum (*ironic.common.glance_service.image_service.GlanceImageService property*), 906
transfer_verified_checksum (*ironic.common.image_service.BaseImageService property*), 963
transfer_verified_checksum (*ironic.common.image_service.HttpImageService property*), 966
transfer_verified_checksum (*ironic.common.image_service.OciImageService property*), 970
TransferHelper (*class in ironic.common.checksum_utils*), 934
transform_header() (*ironic.api.middleware.json_ext.JsonExtensionMiddleware method*), 894
transform_header() (*ironic.api.middleware.JsonExtensionMiddleware method*), 895
translate_config() (*ironic.drivers.modules.switch.base.BaseTranslator method*), 1289
translate_configs() (*ironic.drivers.modules.switch.base.BaseTranslator method*), 1289
translate_from_glance() (*in module ironic.common.glance_service.service_utils*), 907
trigger_servicewait() (*ironic.drivers.modules.fake.FakeVendorB method*), 1349
try_execute() (*in module ironic.common.utils*), 1019
try_set_boot_device() (*in module ironic.drivers.modules.deploy_utils*), 1332
two (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1385
type (*ironic.db.sqlalchemy.models.VolumeConnector attribute*), 1147
type (*ironic.objects.volume_connector.VolumeConnector property*), 1546
type (*ironic.objects.volume_connector.VolumeConnectorCRUDPayload property*), 1547
types() (*in module ironic.common.args*), 929

U

UEFI (*in module ironic.common.boot_modes*), 934
UEFIHTTP (*in module ironic.common.boot_devices*), 934
ULTRA (*ironic.console.rfb.auth.AuthType attribute*), 1093
Unauthorized, 957
unauthorized() (*in module ironic.common.auth_basic*), 933
unavailable (*ironic.drivers.modules.snmp.SNMPDriverRaritanPX2 attribute*), 1385
unavailable (*ironic.drivers.modules.snmp.SNMPDriverVertivGeistPDU attribute*), 1388
unbind_neutron_port() (*in module ironic.common.neutron*), 989
unbind_neutron_port_if_bound() (*in module ironic.common.neutron*), 989
unconfigure_tenant_networks() (*ironic.drivers.base.NetworkInterface method*), 1413
unconfigure_tenant_networks() (*ironic.drivers.modules.network.flat.FlatNetwork method*), 1245
unconfigure_tenant_networks() (*ironic.drivers.modules.network.ironic_networking.IronicNetworking method*), 1249
unconfigure_tenant_networks() (*ironic.drivers.modules.network.neutron.NeutronNetwork method*), 1253
unconfigure_tenant_networks() (*ironic.drivers.modules.network.noop.NoopNetwork method*), 1255
UNDEPLOY (*in module ironic.common.states*), 1009
unique (*ironic.db.sqlalchemy.models.BIOSSetting attribute*), 1136
unique (*ironic.objects.bios.BIOSSetting property*), 1454
unit_dir (*ironic.console.container.systemd.SystemdConsoleContainer attribute*), 1092
unit_id (*ironic.drivers.modules.snmp.SNMPDriverBaytechMRP27 attribute*), 1383
universal_network() (*ironic.common.trait_based_networking.base.Network class method*), 920
universal_port() (*ironic.common.trait_based_networking.base.Port class method*), 922
unix_file_modification_datetime() (*in module ironic.common.utils*), 1019
UNKNOWN (*in module ironic.common.indicator_states*), 976
UnknownArgument, 957
UnknownAttribute, 957
unlink_without_raise() (*in module ironic.common.utils*), 1019
unpublish() (*ironic.common.image_publisher.AbstractPublisher method*), 961
unpublish() (*ironic.common.image_publisher.CIFSPublisher method*), 961
unpublish() (*ironic.common.image_publisher.LocalPublisher method*), 961
unpublish() (*ironic.common.image_publisher.NFSPublisher method*), 962
unpublish() (*ironic.common.image_publisher.SwiftPublisher method*), 962
unpublish_image() (*ironic.drivers.modules.image_utils.ImageHandler method*), 1352
unpublish_image_for_node() (*ironic.drivers.modules.image_utils.ImageHandler class method*), 1352
unregister() (*ironic.objects.conductor.Conductor method*), 1462
unregister_all_hardware_interfaces() (*ironic.objects.conductor.Conductor method*), 1462
unregister_conductor() (*ironic.db.api.Connection method*), 1178
unregister_conductor() (*ironic.db.sqlalchemy.api.Connection method*), 1126
unregister_conductor_hardware_interfaces() (*ironic.db.api.Connection method*), 1178
unregister_conductor_hardware_interfaces() (*ironic.db.sqlalchemy.api.Connection method*), 1127
unrescue() (*ironic.drivers.base.RescueInterface method*), 1418
unrescue() (*ironic.drivers.modules.agent.AgentRescue method*), 1299
unrescue() (*ironic.drivers.modules.fake.FakeRescue method*), 1347
unrescue() (*ironic.drivers.modules.noop.NoRescue method*), 1372
UNRESCUEFAIL (*in module ironic.common.states*), 1009
UNRESCUING (*in module ironic.common.states*), 1009
unset_node_tags() (*ironic.db.api.Connection method*), 1178
unset_node_tags()

(*ironic.db.sqlalchemy.api.Connection method*), 1127
unset_node_traits() (*ironic.db.api.Connection method*), 1178
unset_node_traits() (*ironic.db.sqlalchemy.api.Connection method*), 1127
UnsetCapabilityAction (*class in ironic.common.inspection_rules.actions*), 910
UnsetPluginDataAction (*class in ironic.common.inspection_rules.actions*), 910
UNSTABLE_STATES (*in module ironic.common.states*), 1009
UnsupportedDriverExtension, 957
UnsupportedHardwareFeature, 957
update() (*in module ironic.pxe_filter.dnsmasq*), 1553
update() (*ironic.drivers.base.FirmwareInterface method*), 1401
update() (*ironic.drivers.modules.fake.FakeFirmware method*), 1339
update() (*ironic.drivers.modules.noop.NoFirmware method*), 1370
update() (*ironic.drivers.modules.redfish.firmware.RedfishFirmware method*), 1264
update_allocation() (*ironic.db.api.Connection method*), 1178
update_allocation() (*ironic.db.sqlalchemy.api.Connection method*), 1127
UPDATE_ALLOWED_STATES (*in module ironic.common.states*), 1009
update_auth_failure_logging_threshold() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1221
update_bios_setting_list() (*ironic.db.api.Connection method*), 1179
update_bios_setting_list() (*ironic.db.sqlalchemy.api.Connection method*), 1127
update_boot_mode() (*in module ironic.drivers.modules.ilo.common*), 1212
update_chassis() (*ironic.db.api.Connection method*), 1179
update_chassis() (*ironic.db.sqlalchemy.api.Connection method*), 1128
update_cipher_suite_cmd() (*in module ironic.drivers.modules.ipmitool*), 1365
update_deploy_template() (*ironic.db.api.Connection method*), 1179
update_deploy_template() (*ironic.db.sqlalchemy.api.Connection method*), 1128
update_dhcp() (*ironic.common.dhcp_factory.DHCPFactory method*), 940
update_dhcp_opts() (*ironic.dhcp.base.BaseDHCP method*), 1184
update_dhcp_opts() (*ironic.dhcp.dnsmasq.DnsmasqDHCPApi method*), 1186
update_dhcp_opts() (*ironic.dhcp.neutron.NeutronDHCPApi method*), 1187
update_dhcp_opts() (*ironic.dhcp.none.NoneDHCPApi method*), 1188
update_driver_config() (*ironic.drivers.modules.image_utils.ImageHandler method*), 1353
update_firmware() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1221
update_firmware() (*ironic.drivers.modules.redfish.management.RedfishMana method*), 1275
update_firmware_component() (*ironic.db.api.Connection method*), 1180
update_firmware_component() (*ironic.db.sqlalchemy.api.Connection method*), 1129
update_firmware_sum() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1221
update_image_type() (*in module ironic.conductor.utils*), 1082
update_inspection_rule() (*ironic.db.api.Connection method*), 1180
update_inspection_rule() (*ironic.db.sqlalchemy.api.Connection method*), 1129
update_ipmi_properties() (*in module*

ironic.drivers.modules.ilo.common), 1212
update_lag() (*in module ironic.networking.api*), 1437
update_lag() (*ironic.drivers.modules.switch.base.NoOpSwitchDriver method*), 1290
update_lag() (*ironic.drivers.modules.switch.base.SwitchDriverBase method*), 1292
update_lag() (*ironic.drivers.modules.switch.generic_switch.GenericSwitchDriver method*), 1295
update_lag() (*ironic.networking.manager.NetworkingManager method*), 1439
update_lag() (*ironic.networking.rpcapi.NetworkingAPI method*), 1440
update_minimum_password_length() (*ironic.drivers.modules.ilo.management.IloManagement method*), 1222
update_nested_dict() (*in module ironic.common.inspection_rules.actions*), 910
update_neutron_port() (*in module ironic.common.neutron*), 990
update_next_step_index() (*in module ironic.conductor.utils*), 1082
update_node() (*ironic.conductor.manager.ConductorManager method*), 1034
update_node() (*ironic.conductor.rpcapi.ConductorAPI method*), 1060
update_node() (*ironic.db.api.Connection method*), 1180
update_node() (*ironic.db.sqlalchemy.api.Connection method*), 1129
update_opt_defaults() (*in module ironic.conf.opts*), 1088
update_physical_network() (*ironic.objects.portgroup.Portgroup method*), 1533
update_port() (*in module ironic.networking.api*), 1437
update_port() (*ironic.conductor.manager.ConductorManager method*), 1035
update_port() (*ironic.conductor.rpcapi.ConductorAPI method*), 1061
update_port() (*ironic.db.api.Connection method*), 1181
update_port() (*ironic.db.sqlalchemy.api.Connection method*), 1130
update_port() (*ironic.drivers.modules.switch.base.NoOpSwitchDriver method*), 1290
update_port() (*ironic.drivers.modules.switch.base.SwitchDriverBase method*), 1292
update_port() (*ironic.drivers.modules.switch.generic_switch.GenericSwitchDriver method*), 1295
update_port() (*ironic.networking.manager.NetworkingManager method*), 1439
update_port() (*ironic.networking.rpcapi.NetworkingAPI method*), 1441
update_port_address() (*in module ironic.common.neutron*), 990
update_port_dhcp_opts() (*ironic.dhcp.base.BaseDHCP method*), 1185
update_port_dhcp_opts() (*ironic.dhcp.dnsmasq.DnsmasqDHCPApi method*), 1186
update_port_dhcp_opts() (*ironic.dhcp.neutron.NeutronDHCPApi method*), 1187
update_port_dhcp_opts() (*ironic.dhcp.none.NoneDHCPApi method*), 1188
update_port_host_id() (*in module ironic.drivers.modules.network.common*), 1242
update_portgroup() (*ironic.conductor.manager.ConductorManager method*), 1035
update_portgroup() (*ironic.conductor.rpcapi.ConductorAPI method*), 1061
update_portgroup() (*ironic.db.api.Connection method*), 1181
update_portgroup() (*ironic.db.sqlalchemy.api.Connection method*), 1130
update_portgroup_physical_network() (*ironic.conductor.manager.ConductorManager method*), 1035
update_portgroup_physical_network() (*ironic.conductor.rpcapi.ConductorAPI method*), 1061
update_ports() (*in module ironic.drivers.modules.inspector.hooks.ports*), 1231
update_raid_config() (*in module ironic.drivers.modules.redfish.raid*), 1281
update_raid_info() (*in module ironic.drivers.modules.ilo.common.raid*), 1003
update_redfish_properties() (*in module ironic.drivers.modules.ilo.common*), 1212

update_runbook() (*ironic.db.api.Connection method*), 1181
update_runbook() (*ironic.db.sqlalchemy.api.Connection method*), 1130
update_state_in_older_versions() (*in module ironic.api.controllers.v1.node*), 858
update_to_latest_versions() (*ironic.db.api.Connection method*), 1182
update_to_latest_versions() (*ironic.db.sqlalchemy.api.Connection method*), 1131
update_volume_connector() (*ironic.conductor.manager.ConductorManager method*), 1035
update_volume_connector() (*ironic.conductor.rpcapi.ConductorAPI method*), 1061
update_volume_connector() (*ironic.db.api.Connection method*), 1182
update_volume_connector() (*ironic.db.sqlalchemy.api.Connection method*), 1131
update_volume_target() (*ironic.conductor.manager.ConductorManager method*), 1035
update_volume_target() (*ironic.conductor.rpcapi.ConductorAPI method*), 1062
update_volume_target() (*ironic.db.api.Connection method*), 1183
update_volume_target() (*ironic.db.sqlalchemy.api.Connection method*), 1131
updated_at (*ironic.db.sqlalchemy.models.Allocation attribute*), 1135
updated_at (*ironic.db.sqlalchemy.models.BIOSSetting attribute*), 1136
updated_at (*ironic.db.sqlalchemy.models.Chassis attribute*), 1136
updated_at (*ironic.db.sqlalchemy.models.Conductor attribute*), 1136
updated_at (*ironic.db.sqlalchemy.models.ConductorHardwareInterface attribute*), 1137
updated_at (*ironic.db.sqlalchemy.models.DeployTemplate attribute*), 1137
updated_at (*ironic.db.sqlalchemy.models.DeployTemplateStep attribute*), 1137
updated_at (*ironic.db.sqlalchemy.models.FirmwareComponent attribute*), 1138
updated_at (*ironic.db.sqlalchemy.models.InspectionRule attribute*), 1138
updated_at (*ironic.db.sqlalchemy.models.Node attribute*), 1141
updated_at (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1143
updated_at (*ironic.db.sqlalchemy.models.NodeHistory attribute*), 1143
updated_at (*ironic.db.sqlalchemy.models.NodeInventory attribute*), 1144
updated_at (*ironic.db.sqlalchemy.models.NodeTag attribute*), 1144
updated_at (*ironic.db.sqlalchemy.models.NodeTrait attribute*), 1144
updated_at (*ironic.db.sqlalchemy.models.Port attribute*), 1145
updated_at (*ironic.db.sqlalchemy.models.Portgroup attribute*), 1146
updated_at (*ironic.db.sqlalchemy.models.Runbook attribute*), 1146
updated_at (*ironic.db.sqlalchemy.models.RunbookStep attribute*), 1147
updated_at (*ironic.db.sqlalchemy.models.VolumeConnector attribute*), 1147
updated_at (*ironic.db.sqlalchemy.models.VolumeTarget attribute*), 1147
updated_at (*ironic.objects.allocation.Allocation property*), 1447
updated_at (*ironic.objects.allocation.AllocationCRUDNotificatio property*), 1447
updated_at (*ironic.objects.allocation.AllocationCRUDPayload property*), 1448
updated_at (*ironic.objects.bios.BIOSSetting property*), 1454
updated_at (*ironic.objects.bios.BIOSSettingList property*), 1456
updated_at (*ironic.objects.chassis.Chassis property*), 1458
updated_at (*ironic.objects.chassis.ChassisCRUDNotification property*), 1459
updated_at (*ironic.objects.chassis.ChassisCRUDPayload property*), 1459
updated_at (*ironic.objects.conductor.Conductor property*), 1462
updated_at (*ironic.objects.deploy_template.DeployTemplate property*), 1466
updated_at (*ironic.objects.deploy_template.DeployTemplateCRU property*), 1466

updated_at (*ironic.objects.deploy_template.DeployTemplateCRUDPayload property*), 1467
updated_at (*ironic.objects.deployment.Deployment property*), 1470
updated_at (*ironic.objects.firmware.FirmwareComponent property*), 1474
updated_at (*ironic.objects.firmware.FirmwareComponentList property*), 1475
updated_at (*ironic.objects.inspection_rule.InspectionRule property*), 1479
updated_at (*ironic.objects.inspection_rule.InspectionRuleCRUDNotification property*), 1479
updated_at (*ironic.objects.inspection_rule.InspectionRuleCRUDPayload property*), 1480
updated_at (*ironic.objects.node.Node property*), 1489
updated_at (*ironic.objects.node.NodeConsoleNotification property*), 1494
updated_at (*ironic.objects.node.NodeCorrectedPowerStateNotification property*), 1495
updated_at (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1499
updated_at (*ironic.objects.node.NodeCRUDNotification property*), 1489
updated_at (*ironic.objects.node.NodeCRUDPayload property*), 1494
updated_at (*ironic.objects.node.NodeMaintenanceNotification property*), 1499
updated_at (*ironic.objects.node.NodePayload property*), 1503
updated_at (*ironic.objects.node.NodeSetPowerStateNotification property*), 1504
updated_at (*ironic.objects.node.NodeSetPowerStatePayload property*), 1508
updated_at (*ironic.objects.node.NodeSetProvisionStateNotification property*), 1508
updated_at (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1513
updated_at (*ironic.objects.node_history.NodeHistory property*), 1516
updated_at (*ironic.objects.node_inventory.NodeInventory property*), 1517
updated_at (*ironic.objects.notification.EventType property*), 1518
updated_at (*ironic.objects.notification.NotificationBase property*), 1518
updated_at (*ironic.objects.notification.NotificationPayloadBase property*), 1519
updated_at (*ironic.objects.notification.NotificationPublisher property*), 1519
updated_at (*ironic.objects.port.Port property*), 1526
updated_at (*ironic.objects.port.PortCRUDNotification property*), 1526
updated_at (*ironic.objects.port.PortCRUDPayload property*), 1528
updated_at (*ironic.objects.portgroup.Portgroup property*), 1533
updated_at (*ironic.objects.portgroup.PortgroupCRUDNotification property*), 1533
updated_at (*ironic.objects.portgroup.PortgroupCRUDPayload property*), 1535
updated_at (*ironic.objects.runbook.Runbook property*), 1538
updated_at (*ironic.objects.runbook.RunbookCRUDNotification property*), 1539
updated_at (*ironic.objects.runbook.RunbookCRUDPayload property*), 1540
updated_at (*ironic.objects.trait.Trait property*), 1541
updated_at (*ironic.objects.trait.TraitList property*), 1542
updated_at (*ironic.objects.volume_connector.VolumeConnector property*), 1546
updated_at (*ironic.objects.volume_connector.VolumeConnectorCRUDNotification property*), 1546
updated_at (*ironic.objects.volume_connector.VolumeConnectorCRUDPayload property*), 1547
updated_at (*ironic.objects.volume_target.VolumeTarget property*), 1551
updated_at (*ironic.objects.volume_target.VolumeTargetCRUDNotification property*), 1552
updated_at (*ironic.objects.volume_target.VolumeTargetCRUDPayload property*), 1553
upgrade
    ironic-dbsync command line option, 456
upgrade command line option
    --help, 458
    --revision, 458
    -h, 458
upgrade() (*in module ironic.db.migration*), 1183
upgrade() (*in module ironic.db.sqlalchemy.migration*), 1134
upgrade() (*ironic.command.dbsync.DBCommand method*), 904
upgrade_lock() (*ironic.conductor.task_manager.TaskManager method*), 1071
upper_bound (*ironic.db.sqlalchemy.models.BIOSSetting attribute*), 1136
upper_bound (*ironic.objects.bios.BIOSSetting*

property), 1454
url (ironic.common.glance_service.image_service.TempUrlCacheElement attribute), 907
url_expires_at (ironic.common.glance_service.image_service.TempUrlCacheElement attribute), 907
use_reserved_step_handler() (in module ironic.conductor.steps), 1067
user (ironic.db.sqlalchemy.models.NodeHistory attribute), 1144
user (ironic.objects.node_history.NodeHistory property), 1516
uuid (ironic.common.trait_based_networking.base.PortInfo attribute), 922
uuid (ironic.db.sqlalchemy.models.Allocation attribute), 1135
uuid (ironic.db.sqlalchemy.models.Chassis attribute), 1136
uuid (ironic.db.sqlalchemy.models.DeployTemplate attribute), 1137
uuid (ironic.db.sqlalchemy.models.InspectionRule attribute), 1138
uuid (ironic.db.sqlalchemy.models.Node attribute), 1141
uuid (ironic.db.sqlalchemy.models.NodeBase attribute), 1143
uuid (ironic.db.sqlalchemy.models.NodeHistory attribute), 1144
uuid (ironic.db.sqlalchemy.models.Port attribute), 1145
uuid (ironic.db.sqlalchemy.models.Portgroup attribute), 1146
uuid (ironic.db.sqlalchemy.models.Runbook attribute), 1146
uuid (ironic.db.sqlalchemy.models.VolumeConnector attribute), 1147
uuid (ironic.db.sqlalchemy.models.VolumeTarget attribute), 1148
uuid (ironic.objects.allocation.Allocation property), 1447
uuid (ironic.objects.allocation.AllocationCRUDPayload property), 1448
uuid (ironic.objects.chassis.Chassis property), 1458
uuid (ironic.objects.chassis.ChassisCRUDPayload property), 1459
uuid (ironic.objects.deploy_template.DeployTemplate property), 1466
uuid (ironic.objects.deploy_template.DeployTemplateCRUDPayload property), 1467
uuid (ironic.objects.deployment.Deployment property), 1470
uuid (ironic.objects.inspection_rule.InspectionRule property), 1479
uuid (ironic.objects.inspection_rule.InspectionRuleCRUDPayload property), 1480
uuid (ironic.objects.node.Node property), 1489
uuid (ironic.objects.node.NodeCorrectedPowerStatePayload property), 1499
uuid (ironic.objects.node.NodeCRUDPayload property), 1494
uuid (ironic.objects.node.NodePayload property), 1504
uuid (ironic.objects.node.NodeSetPowerStatePayload property), 1508
uuid (ironic.objects.node.NodeSetProvisionStatePayload property), 1513
uuid (ironic.objects.node_history.NodeHistory property), 1516
uuid (ironic.objects.port.Port property), 1526
uuid (ironic.objects.port.PortCRUDPayload property), 1528
uuid (ironic.objects.portgroup.Portgroup property), 1533
uuid (ironic.objects.portgroup.PortgroupCRUDPayload property), 1535
uuid (ironic.objects.runbook.Runbook property), 1538
uuid (ironic.objects.runbook.RunbookCRUDPayload property), 1540
uuid (ironic.objects.volume_connector.VolumeConnector property), 1546
uuid (ironic.objects.volume_connector.VolumeConnectorCRUDPayload property), 1547
uuid (ironic.objects.volume_target.VolumeTarget property), 1551
uuid (ironic.objects.volume_target.VolumeTargetCRUDPayload property), 1553
uuid() (in module ironic.common.args), 929
uuid_or_name() (in module ironic.common.args), 930
UUIDField (class in ironic.objects.fields), 1472

V

valid (ironic.common.trait_based_networking.loader.ConfigLoader property), 925
VALID_LOG_LEVELS (ironic.common.inspection_rules.actions.LogAction attribute), 909
validate() (in module ironic.common.args), 930
validate() (ironic.api.controllers.v1.node.NodesController method), 857

validate() (*ironic.api.validation.validators.SchemaValidator method*), 896
validate() (*ironic.common.inspection_rules.base.Base method*), 911
validate() (*ironic.common.trait_based_networking.base.TraitAction method*), 923
validate() (*ironic.common.trait_based_networking.config_file.ConfigFile method*), 924
validate() (*ironic.drivers.base.BaseInterface method*), 1395
validate() (*ironic.drivers.base.NetworkInterface method*), 1413
validate() (*ironic.drivers.base.RAIDInterface method*), 1417
validate() (*ironic.drivers.base.VendorInterface method*), 1420
validate() (*ironic.drivers.modules.agent.AgentDeploy method*), 1297
validate() (*ironic.drivers.modules.agent.AgentRescue method*), 1299
validate() (*ironic.drivers.modules.agent.BootcAgentDeploy method*), 1300
validate() (*ironic.drivers.modules.agent.CustomAgentDeploy method*), 1302
validate() (*ironic.drivers.modules.agent_power.AgentPower method*), 1319
validate() (*ironic.drivers.modules.ansible.deploy.AnsibleDeploy method*), 1191
validate() (*ironic.drivers.modules.autodetect.AutodetectDeploy method*), 1320
validate() (*ironic.drivers.modules.fake.FakeBIOS method*), 1334
validate() (*ironic.drivers.modules.fake.FakeBoot method*), 1336
validate() (*ironic.drivers.modules.fake.FakeConsole method*), 1336
validate() (*ironic.drivers.modules.fake.FakeDeploy method*), 1338
validate() (*ironic.drivers.modules.fake.FakeFirmware method*), 1339
validate() (*ironic.drivers.modules.fake.FakeGraphicalConsole method*), 1340
validate() (*ironic.drivers.modules.fake.FakeInspect method*), 1341
validate() (*ironic.drivers.modules.fake.FakeManagement method*), 1344
validate() (*ironic.drivers.modules.fake.FakePower method*), 1345
validate() (*ironic.drivers.modules.fake.FakeRescue method*), 1347
validate() (*ironic.drivers.modules.fake.FakeStorage method*), 1348
validate() (*ironic.drivers.modules.fake.FakeVendorA method*), 1349
validate() (*ironic.drivers.modules.fake.FakeVendorB method*), 1349
validate() (*ironic.drivers.modules.ilo.bios.IloBIOS method*), 1198
validate() (*ironic.drivers.modules.ilo.boot.IloUefiHttpsBoot method*), 1201
validate() (*ironic.drivers.modules.ilo.boot.IloVirtualMediaBoot method*), 1203
validate() (*ironic.drivers.modules.ilo.inspect.IloInspect method*), 1215
validate() (*ironic.drivers.modules.ilo.management.IloManagem method*), 1222
validate() (*ironic.drivers.modules.ilo.power.IloPower method*), 1223
validate() (*ironic.drivers.modules.ilo.vendor.VendorPassthru method*), 1225
validate() (*ironic.drivers.modules.inspector.interface.Common method*), 1235
validate() (*ironic.drivers.modules.ipmitool.IPMIConsole method*), 1358
validate() (*ironic.drivers.modules.ipmitool.IPMIManagement method*), 1360
validate() (*ironic.drivers.modules.ipmitool.IPMIPower method*), 1362
validate() (*ironic.drivers.modules.ipmitool.VendorPassthru method*), 1363
validate() (*ironic.drivers.modules.network.flat.FlatNetwork method*), 1245
validate() (*ironic.drivers.modules.network.ironic_networking.Ir method*), 1249
validate() (*ironic.drivers.modules.network.neutron.NeutronNetw method*), 1253
validate() (*ironic.drivers.modules.noop.FailMixin method*), 1366
validate() (*ironic.drivers.modules.noop.NoDeploy method*), 1369
validate() (*ironic.drivers.modules.noop_mgmt.NoopManagemen method*), 1374
validate() (*ironic.drivers.modules.pxe.PXEAnacondaDeploy method*), 1376
validate() (*ironic.drivers.modules.pxe_base.PXEBaseMixin method*), 1377
validate() (*ironic.drivers.modules.ramdisk.RamdiskDeploy method*), 1379
validate() (*ironic.drivers.modules.redfish.bios.RedfishBIOS method*), 1258
validate() (*ironic.drivers.modules.redfish.boot.RedfishHttpsBoot method*), 1260

validate() (*ironic.drivers.modules.redfish.boot.RedfishVirtualMediaBoot method*), 1262
validate() (*ironic.drivers.modules.redfish.firmware.RedfishFirmware method*), 1265
validate() (*ironic.drivers.modules.redfish.graphical_console.RedfishGraphicalConsole method*), 1267
validate() (*ironic.drivers.modules.redfish.inspect.RedfishInspect method*), 1267
validate() (*ironic.drivers.modules.redfish.management.RedfishManagement method*), 1275
validate() (*ironic.drivers.modules.redfish.power.RedfishPower method*), 1277
validate() (*ironic.drivers.modules.redfish.vendor.RedfishVendorPassthru method*), 1285
validate() (*ironic.drivers.modules.snmp.SNMPPower method*), 1389
validate() (*ironic.drivers.modules.storage.cinder.CinderStorage method*), 1286
validate() (*ironic.drivers.modules.storage.external.ExternalStorage method*), 1287
validate() (*ironic.drivers.modules.storage.noop.NoopStorage method*), 1288
validate() (*ironic.drivers.utils.MixinVendorInterface method*), 1430
validate_and_normalize_datapath_id() (*in module ironic.common.utils*), 1019
validate_and_normalize_mac() (*in module ironic.common.utils*), 1019
validate_auth_file() (*in module ironic.common.auth_basic*), 933
validate_capabilities() (*in module ironic.drivers.modules.deploy_utils*), 1332
validate_cert_paths() (*in module ironic.common.wsgi_service*), 1022
validate_checksum() (*in module ironic.common.checksum_utils*), 935
validate_conductor_group() (*in module ironic.common.utils*), 1020
validate_configuration() (*in module ironic.common.raid*), 1003
validate_deploy_steps() (*in module ironic.conductor.deployments*), 1029
validate_driver_interfaces() (*ironic.conductor.manager.ConductorManager method*), 1035
validate_driver_interfaces() (*ironic.conductor.rpcapi.ConductorAPI method*), 1062
validate_firmware_interface_update_args() (*in module ironic.drivers.modules.redfish.firmware_utils*), 1266
validate_href() (*ironic.common.image_service.BaseImageService method*), 962
validate_href() (*ironic.common.image_service.CifsImageService method*), 964
validate_href() (*ironic.common.image_service.FileImageService method*), 965
validate_href() (*ironic.common.image_service.HttpImageService method*), 966
validate_href() (*ironic.common.image_service.NfsImageService method*), 967
validate_href() (*ironic.common.image_service.OciImageService method*), 970
validate_http_provisioning_configuration() (*in module ironic.drivers.modules.agent*), 1303
validate_image_properties() (*in module ironic.drivers.modules.deploy_utils*), 1332
validate_image_proxies() (*in module ironic.drivers.modules.agent*), 1303
validate_inspection() (*ironic.common.neutron.NeutronNetworkInterfaceMixin method*), 986
validate_inspection() (*ironic.drivers.base.BootInterface method*), 1396
validate_inspection() (*ironic.drivers.base.NetworkInterface method*), 1413
validate_inspection() (*ironic.drivers.modules.ilo.boot.IloUefiHttpsBoot method*), 1201
validate_inspection() (*ironic.drivers.modules.ilo.boot.IloVirtualMediaBoot method*), 1203
validate_inspection() (*ironic.drivers.modules.network.ironic_networking.IronicN method*), 1249
validate_inspection() (*ironic.drivers.modules.network.noop.NoopNetwork method*), 1255
validate_inspection() (*ironic.drivers.modules.pxe_base.PXEBaseMixin*

method), 1378
validate_inspection() (*ironic.drivers.modules.redfish.boot.RedfishHttpsBoot method*), 1260
validate_inspection() (*ironic.drivers.modules.redfish.boot.RedfishVirtualMediaBoot method*), 1263
validate_inspection_hooks() (*in module ironic.drivers.modules.inspect_utils*), 1357
validate_instance_info_traits() (*in module ironic.conductor.utils*), 1082
validate_interfaces() (*in module ironic.drivers.modules.inspector.hooks.validate_interfaces*), 1234
validate_kickstart_file() (*in module ironic.common.pxe_utils*), 1002
validate_kickstart_template() (*in module ironic.common.pxe_utils*), 1002
validate_limit() (*in module ironic.api.controllers.v1.utils*), 886
validate_network() (*in module ironic.common.neutron*), 990
validate_network_data() (*in module ironic.api.controllers.v1.node*), 858
validate_network_port() (*in module ironic.common.utils*), 1020
validate_node() (*in module ironic.conductor.deployments*), 1029
validate_port_info() (*in module ironic.common.neutron*), 990
validate_port_physnet() (*in module ironic.conductor.utils*), 1082
validate_raid_config() (*ironic.drivers.base.RAIDInterface method*), 1417
validate_raid_config() (*ironic.drivers.modules.noop.NoRAID method*), 1371
validate_raid_config() (*ironic.drivers.modules.redfish.raid.RedfishRAID method*), 1279
validate_regex() (*ironic.common.inspection_rules.operators.RuleOperator method*), 913
validate_rescue() (*ironic.drivers.base.BootInterface method*), 1396
validate_rescue() (*ironic.drivers.base.NetworkInterface method*), 1414
validate_rescue() (*ironic.drivers.modules.ilo.boot.IloUefiHttpsBoot method*), 1201
validate_rescue() (*ironic.drivers.modules.ilo.boot.IloVirtualMediaBoot method*), 1203
validate_rescue() (*ironic.drivers.modules.network.ironic_networking.IronicN method*), 1249
validate_rescue() (*ironic.drivers.modules.network.neutron.NeutronNetwork method*), 1253
validate_rescue() (*ironic.drivers.modules.pxe_base.PXEBaseMixin method*), 1378
validate_rescue() (*ironic.drivers.modules.redfish.boot.RedfishVirtualMediaB method*), 1263
validate_rule() (*in module ironic.common.inspection_rules.validation*), 915
validate_schemas() (*ironic.api.validation.Schemas method*), 896
validate_security_parameter_values() (*in module ironic.drivers.modules.ilo.common*), 1212
validate_servicing() (*ironic.drivers.modules.network.neutron.NeutronNetwork method*), 1254
validate_sort_dir() (*in module ironic.api.controllers.v1.utils*), 886
validate_text_checksum() (*in module ironic.common.checksum_utils*), 935
validate_update_firmware_args() (*in module ironic.drivers.modules.redfish.firmware_utils*), 1266
validate_user_deploy_steps_and_templates() (*in module ironic.conductor.steps*), 1067
validate_vlan_allowed() (*in module ironic.networking.switch_config*), 1442
validate_vlan_configuration() (*in module ironic.networking.manager*), 1439
validate_vlan_configuration() (*in module ironic.networking.switch_config*), 1442
ValidateInterfacesHook (*class in ironic.drivers.modules.inspector.hooks.validate_interfaces*), 1233
validator (*ironic.api.validation.validators.SchemaValidator*

attribute), 896
validator_org (*ironic.api.validation.validators.SchemaValidator attribute*), 896
value (*ironic.db.sqlalchemy.models.BIOSSetting attribute*), 1136
value (*ironic.objects.bios.BIOSSetting property*), 1454
value_power_off (*ironic.drivers.modules.snmp.SNMPDriverAPCMasterSwitch attribute*), 1381
value_power_off (*ironic.drivers.modules.snmp.SNMPDriverAPCMasterSwitchPlus attribute*), 1381
value_power_off (*ironic.drivers.modules.snmp.SNMPDriverAPCRackPDU attribute*), 1381
value_power_off (*ironic.drivers.modules.snmp.SNMPDriverAten attribute*), 1381
value_power_off (*ironic.drivers.modules.snmp.SNMPDriverBaytechMRP27 attribute*), 1383
value_power_off (*ironic.drivers.modules.snmp.SNMPDriverCyberPower attribute*), 1383
value_power_off (*ironic.drivers.modules.snmp.SNMPDriverEatonPower attribute*), 1384
value_power_off (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1385
value_power_off (*ironic.drivers.modules.snmp.SNMPDriverServerTech attribute*), 1385
value_power_off (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry4 attribute*), 1386
value_power_off (*ironic.drivers.modules.snmp.SNMPDriverSimple property*), 1387
value_power_off (*ironic.drivers.modules.snmp.SNMPDriverTeltronix attribute*), 1387
value_power_off (*ironic.drivers.modules.snmp.SNMPDriverVertivGeistPDU attribute*), 1388
value_power_on (*ironic.drivers.modules.snmp.SNMPDriverAPCMasterSwitch attribute*), 1381
value_power_on (*ironic.drivers.modules.snmp.SNMPDriverAPCMasterSwitchPlus attribute*), 1381
value_power_on (*ironic.drivers.modules.snmp.SNMPDriverAPCRackPDU attribute*), 1381
value_power_on (*ironic.drivers.modules.snmp.SNMPDriverAten attribute*), 1382
value_power_on (*ironic.drivers.modules.snmp.SNMPDriverBaytechMRP27 attribute*), 1383
value_power_on (*ironic.drivers.modules.snmp.SNMPDriverCyberPower attribute*), 1383
value_power_on (*ironic.drivers.modules.snmp.SNMPDriverEatonPower attribute*), 1384
value_power_on (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1385
value_power_on (*ironic.drivers.modules.snmp.SNMPDriverServerTech attribute*), 1385
value_power_on (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry4 attribute*), 1386
value_power_on (*ironic.drivers.modules.snmp.SNMPDriverSimple property*), 1387
value_power_on (*ironic.drivers.modules.snmp.SNMPDriverTeltronix attribute*), 1387
value_power_on (*ironic.drivers.modules.snmp.SNMPDriverVertivGeistPDU attribute*), 1388
value_within_timeout() (*in module ironic.conductor.utils*), 1083
variable_name() (*ironic.common.trait_based_networking.grammar.parser.I method*), 917
Variables (*class in ironic.common.trait_based_networking.base*), 923
VENCRYPT (*ironic.console.rfb.auth.AuthType attribute*), 1093
vendor (*ironic.common.trait_based_networking.base.PrimordialPo attribute*), 922
vendor (*ironic.db.sqlalchemy.models.Port attribute*), 1145
vendor (*ironic.drivers.base.BareDriver attribute*), 1392
vendor (*ironic.objects.port.Port property*), 1526

vendor (*ironic.objects.port.PortCRUDPayload property*), 1528
vendor_interface (*ironic.db.sqlalchemy.models.Node attribute*), 1141
vendor_interface (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1143
vendor_interface (*ironic.objects.node.Node property*), 1489
vendor_interface (*ironic.objects.node.NodeCorrectedPowerStatePayload property*), 1499
vendor_interface (*ironic.objects.node.NodeCRUDPayload property*), 1494
vendor_interface (*ironic.objects.node.NodePayload property*), 1504
vendor_interface (*ironic.objects.node.NodeSetPowerStatePayload property*), 1508
vendor_interface (*ironic.objects.node.NodeSetProvisionStatePayload property*), 1513
vendor_passthru (*ironic.api.controllers.v1.driver.DriversController attribute*), 842
vendor_passthru (*ironic.api.controllers.v1.node.NodesController attribute*), 857
vendor_passthru() (*in module ironic.api.controllers.v1.utils*), 886
vendor_passthru() (*ironic.conductor.manager.ConductorManager method*), 1035
vendor_passthru() (*ironic.conductor.rpcapi.ConductorAPI method*), 1063
VendorInterface (*class in ironic.drivers.base*), 1419
VendorMetadata (*class in ironic.drivers.base*), 1420
VendorPassthru (*class in ironic.drivers.modules.ilo.vendor*), 1225
VendorPassthru (*class in ironic.drivers.modules.ipmitool*), 1363
VendorPassthruException, 957
VERBS (*in module ironic.common.states*), 1010
verify_basic_auth_cred_format() (*ironic.common.image_service.HttpImageService static method*), 967
verify_bmc_clock() (*ironic.drivers.modules.redfish.management.RedfishManagement method*), 1275
verify_checksum() (*in module ironic.drivers.modules.redfish.firmware_utils*), 1266
verify_firmware_update_args() (*in module ironic.drivers.modules.ilo.firmware_processor*), 1214
verify_image_checksum() (*in module ironic.drivers.modules.ilo.common*), 1212
verify_node_for_deallocation() (*in module ironic.conductor.allocations*), 1022
verify_step() (*in module ironic.drivers.base*), 1424
VERIFYING (*in module ironic.common.states*), 1010
verifying_error_handler() (*in module ironic.conductor.utils*), 1083
version (*ironic-dbsync command line option*), 456
Version (*class in ironic.api.controllers.base*), 893
version (*ironic.db.sqlalchemy.models.Allocation attribute*), 1135
version (*ironic.db.sqlalchemy.models.BIOSSetting attribute*), 1136
version (*ironic.db.sqlalchemy.models.Chassis attribute*), 1136
version (*ironic.db.sqlalchemy.models.Conductor attribute*), 1136
version (*ironic.db.sqlalchemy.models.ConductorHardwareInterface attribute*), 1137
version (*ironic.db.sqlalchemy.models.DeployTemplate attribute*), 1137
version (*ironic.db.sqlalchemy.models.DeployTemplateStep attribute*), 1138
version (*ironic.db.sqlalchemy.models.FirmwareComponent attribute*), 1138
version (*ironic.db.sqlalchemy.models.InspectionRule attribute*), 1138
version (*ironic.db.sqlalchemy.models.IronicBase attribute*), 1139
version (*ironic.db.sqlalchemy.models.Node attribute*), 1141
version (*ironic.db.sqlalchemy.models.NodeBase attribute*), 1143
version (*ironic.db.sqlalchemy.models.NodeHistory*

attribute), 1144
version (*ironic.db.sqlalchemy.models.NodeInventory attribute*), 1144
version (*ironic.db.sqlalchemy.models.NodeTag attribute*), 1144
version (*ironic.db.sqlalchemy.models.NodeTrait attribute*), 1144
version (*ironic.db.sqlalchemy.models.Port attribute*), 1145
version (*ironic.db.sqlalchemy.models.Portgroup attribute*), 1146
version (*ironic.db.sqlalchemy.models.Runbook attribute*), 1146
version (*ironic.db.sqlalchemy.models.RunbookStep attribute*), 1147
version (*ironic.db.sqlalchemy.models.VolumeConnector attribute*), 1147
version (*ironic.db.sqlalchemy.models.VolumeTarget attribute*), 1148
VERSION (*ironic.objects.allocation.Allocation attribute*), 1443
VERSION (*ironic.objects.allocation.AllocationCRUDNotification attribute*), 1447
VERSION (*ironic.objects.allocation.AllocationCRUDPayload attribute*), 1447
VERSION (*ironic.objects.bios.BIOSSetting attribute*), 1451
VERSION (*ironic.objects.bios.BIOSSettingList attribute*), 1454
VERSION (*ironic.objects.chassis.Chassis attribute*), 1456
VERSION (*ironic.objects.chassis.ChassisCRUDNotification attribute*), 1458
VERSION (*ironic.objects.chassis.ChassisCRUDPayload attribute*), 1459
VERSION (*ironic.objects.conductor.Conductor attribute*), 1460
VERSION (*ironic.objects.deploy_template.DeployTemplate attribute*), 1462
VERSION (*ironic.objects.deploy_template.DeployTemplateCRUDNotification attribute*), 1466
VERSION (*ironic.objects.deploy_template.DeployTemplateCRUDPayload attribute*), 1466
VERSION (*ironic.objects.deployment.Deployment attribute*), 1467
VERSION (*ironic.objects.firmware.FirmwareComponent attribute*), 1473
VERSION (*ironic.objects.firmware.FirmwareComponentList attribute*), 1474
VERSION (*ironic.objects.inspection_rule.InspectionRule attribute*), 1476
VERSION (*ironic.objects.inspection_rule.InspectionRuleCRUDNotification attribute*), 1479
VERSION (*ironic.objects.inspection_rule.InspectionRuleCRUDPayload attribute*), 1480
VERSION (*ironic.objects.node.Node attribute*), 1481
VERSION (*ironic.objects.node.NodeConsoleNotification attribute*), 1494
VERSION (*ironic.objects.node.NodeCorrectedPowerStateNotification attribute*), 1494
VERSION (*ironic.objects.node.NodeCorrectedPowerStatePayload attribute*), 1495
VERSION (*ironic.objects.node.NodeCRUDNotification attribute*), 1489
VERSION (*ironic.objects.node.NodeCRUDPayload attribute*), 1490
VERSION (*ironic.objects.node.NodeMaintenanceNotification attribute*), 1499
VERSION (*ironic.objects.node.NodePayload attribute*), 1500
VERSION (*ironic.objects.node.NodeSetPowerStateNotification attribute*), 1504
VERSION (*ironic.objects.node.NodeSetPowerStatePayload attribute*), 1504
VERSION (*ironic.objects.node.NodeSetProvisionStateNotification attribute*), 1508
VERSION (*ironic.objects.node.NodeSetProvisionStatePayload attribute*), 1509
VERSION (*ironic.objects.node_history.NodeHistory attribute*), 1513
VERSION (*ironic.objects.node_inventory.NodeInventory attribute*), 1516
VERSION (*ironic.objects.notification.EventType attribute*), 1517
VERSION (*ironic.objects.notification.NotificationBase attribute*), 1518
VERSION (*ironic.objects.notification.NotificationPayloadBase attribute*), 1518
VERSION (*ironic.objects.notification.NotificationPublisher attribute*), 1519
VERSION (*ironic.objects.port.Port attribute*), 1519
VERSION (*ironic.objects.port.PortCRUDNotification attribute*), 1526
VERSION (*ironic.objects.port.PortCRUDPayload attribute*), 1527
VERSION (*ironic.objects.portgroup.Portgroup attribute*), 1528
VERSION (*ironic.objects.portgroup.PortgroupCRUDNotification attribute*), 1533
VERSION (*ironic.objects.portgroup.PortgroupCRUDPayload attribute*), 1534

VERSION (*ironic.objects.runbook.Runbook attribute*), 1535
VERSION (*ironic.objects.runbook.RunbookCRUDNotification attribute*), 1538
VERSION (*ironic.objects.runbook.RunbookCRUDPayload attribute*), 1539
VERSION (*ironic.objects.trait.Trait attribute*), 1540
VERSION (*ironic.objects.trait.TraitList attribute*), 1541
VERSION (*ironic.objects.volume_connector.VolumeConnector attribute*), 1542
VERSION (*ironic.objects.volume_connector.VolumeConnectorCRUDNotification attribute*), 1546
VERSION (*ironic.objects.volume_connector.VolumeConnectorCRUDPayload attribute*), 1547
VERSION (*ironic.objects.volume_target.VolumeTarget attribute*), 1547
VERSION (*ironic.objects.volume_target.VolumeTargetCRUDNotification attribute*), 1552
VERSION (*ironic.objects.volume_target.VolumeTargetCRUDPayload attribute*), 1552
version command line option
--help, 459
-h, 459
version() (*in module ironic.db.migration*), 1183
version() (*in module ironic.db.sqlalchemy.migration*), 1134
version() (*ironic.command.dbsync.DBCommand method*), 904
VersionSelectorApplication (*class in ironic.api.app*), 898
vif_attach() (*ironic.conductor.manager.ConductorManager method*), 1035
vif_attach() (*ironic.conductor.rpcapi.ConductorAPI method*), 1063
vif_attach() (*ironic.drivers.base.NetworkInterface method*), 1414
vif_attach() (*ironic.drivers.modules.network.common.NeutronVIFPortIDMixin method*), 1240
vif_attach() (*ironic.drivers.modules.network.ironic_networking.IronicNetworking method*), 1250
vif_attach() (*ironic.drivers.modules.network.noop.NoopNetwork method*), 1255
vif_detach() (*ironic.conductor.manager.ConductorManager method*), 1035
vif_detach() (*ironic.conductor.rpcapi.ConductorAPI method*), 1064
vif_detach() (*ironic.drivers.base.NetworkInterface method*), 1414
vif_detach() (*ironic.drivers.modules.network.common.NeutronVIFPortIDMixin method*), 1240
vif_detach() (*ironic.drivers.modules.network.ironic_networking.IronicNetworking method*), 1250
vif_detach() (*ironic.drivers.modules.network.noop.NoopNetwork method*), 1255
vif_key (*ironic.drivers.modules.network.common.NetType property*), 1239
vif_list() (*ironic.conductor.manager.ConductorManager method*), 1035
vif_list() (*ironic.conductor.rpcapi.ConductorAPI method*), 1064
vif_list() (*ironic.drivers.base.NetworkInterface method*), 1414
vif_list() (*ironic.drivers.modules.network.common.VIFPortIDMixin method*), 1241
vif_list() (*ironic.drivers.modules.network.ironic_networking.IronicNetworking method*), 1250
vif_list() (*ironic.drivers.modules.network.noop.NoopNetwork method*), 1256
VifAlreadyAttached, 957
VifInvalidForAttach, 958
VifNotAttached, 958
VIFPortIDMixin (*class in ironic.drivers.modules.network.common*), 1241
VIRTUAL_MEDIA_DEVICES (*ironic.drivers.modules.drac.boot.DracRedfishVirtualMediaBoot attribute*), 1192
VMEDIA_DEVICES (*in module ironic.common.boot_devices*), 934
VNC (*ironic.console.rfb.auth.AuthType attribute*), 1093
volume_connectors (*ironic.conductor.task_manager.TaskManager property*), 1071
volume_create_error_handler() (*ironic.drivers.modules.redfish.raid.RedfishRAID method*), 1279
volume_id (*ironic.db.sqlalchemy.models.VolumeTarget attribute*), 1148
volume_id (*ironic.objects.volume_target.VolumeTarget property*), 1551
volume_id (*ironic.objects.volume_target.VolumeTargetCRUDPayload property*), 1553
volume_targets (*ironic.conductor.task_manager.TaskManager property*), 1071
volume_type (*ironic.db.sqlalchemy.models.VolumeTarget attribute*), 1148
volume_type (*ironic.objects.volume_target.VolumeTarget property*), 1551
volume_type (*ironic.objects.volume_target.VolumeTargetCRUDPa*

property), 1553
VolumeConnector (*class in ironic.db.sqlalchemy.models*), 1147
VolumeConnector (*class in ironic.objects.volume_connector*), 1542
VolumeConnectorAlreadyExists, 958
VolumeConnectorCRUDNotification (*class in ironic.objects.volume_connector*), 1546
VolumeConnectorCRUDPayload (*class in ironic.objects.volume_connector*), 1547
VolumeConnectorNotFound, 958
VolumeConnectorsController (*class in ironic.api.controllers.v1.volume_connector*), 887
VolumeConnectorTypeAndIdAlreadyExists, 958
VolumeController (*class in ironic.api.controllers.v1.volume*), 887
VolumeTarget (*class in ironic.db.sqlalchemy.models*), 1147
VolumeTarget (*class in ironic.objects.volume_target*), 1547
VolumeTargetAlreadyExists, 958
VolumeTargetBootIndexAlreadyExists, 958
VolumeTargetCRUDNotification (*class in ironic.objects.volume_target*), 1551
VolumeTargetCRUDPayload (*class in ironic.objects.volume_target*), 1552
VolumeTargetNotFound, 958
VolumeTargetsController (*class in ironic.api.controllers.v1.volume_target*), 890

W

wait() (*ironic.common.wsgi_service.BaseWSGIService method*), 1021
wait_for_host_agent() (*in module ironic.common.neutron*), 991
wait_for_port_status() (*in module ironic.common.neutron*), 991
wait_until_get_system_ready() (*in module ironic.drivers.modules.redfish.utils*), 1284
wakeOff (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry4 attribute*), 1386
wakeOn (*ironic.drivers.modules.snmp.SNMPDriverServerTechSentry4 attribute*), 1386
WANBOOT (*in module ironic.common.boot_devices*), 934
warn_about_inconsistent_kernel_ramdisk() (*in module ironic.command.conductor*), 903
warn_about_max_wait_parameters() (*in module ironic.command.conductor*), 903
warn_about_sqlite() (*in module ironic.command.conductor*), 903
warn_about_unsafe_shred_parameters() (*in module ironic.command.conductor*), 903
WARNING (*ironic.common.health_states.HealthState attribute*), 960
WARNING (*ironic.objects.fields.NotificationLevel attribute*), 1471
wipe_cleaning_internal_info() (*in module ironic.conductor.utils*), 1083
wipe_deploy_internal_info() (*in module ironic.conductor.utils*), 1083
wipe_internal_info_on_power_off() (*in module ironic.conductor.utils*), 1083
wipe_service_internal_info() (*in module ironic.conductor.utils*), 1083
wipe_token_and_url() (*in module ironic.conductor.utils*), 1083
wrap_sqlite_retry() (*in module ironic.db.sqlalchemy.api*), 1134
wrapfunc() (*in module ironic.api.functions*), 900
write_image() (*ironic.drivers.modules.agent.AgentDeploy method*), 1297
write_image() (*ironic.drivers.modules.ansible.deploy.AnsibleDe method*), 1191
write_to_file() (*in module ironic.common.utils*), 1020
WSGIService (*class in ironic.common.json_rpc.server*), 916
WSGIService (*class in ironic.common.wsgi_service*), 1021

Y

yes (*ironic.drivers.modules.snmp.SNMPDriverRaritanPDU2 attribute*), 1385

Z

Zeroconf (*class in ironic.common.mdns*), 979